

Überarbeitung

Handbuch Base

25.12.17

Copyright

Dieses Dokument unterliegt dem Copyright © 2015. Die Beitragenden sind unten aufgeführt. Sie dürfen dieses Dokument unter den Bedingungen der GNU General Public License (<http://www.gnu.org/licenses/gpl.html>), Version 3 oder höher, oder der Creative Commons Attribution License (<http://creativecommons.org/licenses/by/3.0/>), Version 3.0 oder höher, verändern und/oder weitergeben.

Warennamen werden ohne Gewährleistung der freien Verwendbarkeit benutzt.

Fast alle Hardware- und Softwarebezeichnungen und weitere Stichworte und sonstige Angaben, die in diesem Buch verwendet werden, sind als eingetragene Marken geschützt.

Da es nicht möglich ist, in allen Fällen zeitnah zu ermitteln, ob ein Markenschutz besteht, wird das Symbol (R) in diesem Buch nicht verwendet.

Mitwirkende/Autoren

Titelblatt

Klaus-Jürgen Weghorn

Engl. Original:

Drew Jensen
Jean Hollis Weber

Christoph Noack
Klaus-Jürgen Weghorn

Vorwort

Klaus-Jürgen Weghorn

Engl. Original:

Jean Hollis Weber

Inhalt Base

Robert Großkopf
Jochen Schiffers

Jost Lange
Franklin Schifftan

Michael Niedermair
Jürgen Thomas

Glossar

Erhardt Balthasar
Michael Niedermair
Klaus-Jürgen Weghorn

Stefan Haas
Florian Reisinger

Christian Kühl
Jochen Schiffers

Rückmeldung (Feedback)

Kommentare oder Vorschläge zu diesem Dokument können Sie in deutscher Sprache an die Adresse discuss@de.libreoffice.org senden.

Vorsicht



Alles, was an eine Mailingliste geschickt wird, inklusive der E-Mail-Adresse und anderer persönlicher Daten, die die E-Mail enthält, wird öffentlich archiviert und kann nicht gelöscht werden. Also, schreiben Sie mit Bedacht!

Datum der Veröffentlichung und Softwareversion

Veröffentlicht am 15.2.2018. Basierend auf der LibreOffice Version 6.0.

Inhalt

Vorwort.....	11
Für wen ist dieses Buch?.....	12
Was finden Sie in diesem Buch?.....	12
Wo bekomme ich mehr Hilfe?.....	12
Hilfesystem.....	12
Freier Onlinesupport.....	12
Bezahlter Support und Schulungen.....	13
Sie sehen vielleicht etwas anderes.....	13
Anmerkung für Macintosh Nutzer.....	13
Verwenden von Tipps, Hinweisen und Warnungen.....	14
Wer hat dieses Buch geschrieben?.....	14
FAQs (häufig gestellte Fragen).....	15
Einführung in Base.....	16
Einführung.....	17
Base – ein Container für Datenbankinhalte.....	18
Formulare – Start für die Dateneingabe.....	20
Tabellen – Grundlagen für die Dateneingabe.....	21
Abfragen – Auswertungsmöglichkeiten für eingegebene Daten.....	23
Berichte – Präsentationen der Datenauswertung.....	23
Sicherer Umgang mit einer Base-Datei.....	26
Eine einfache Datenbank – Testbeispiel im Detail.....	26
Tabellenerstellung.....	27
Eingabeformular.....	35
Mit dem Tabulator zum Unterformular.....	43
Navigationsleiste des Hauptformulars auch im Unterformular aktivieren.....	44
Eingabe eines Kontrollfeldes einschränken.....	44
Abfrage.....	46
Ausdruck im Bericht.....	50
Abstände zwischen den Berichtsbereichen einstellen.....	55
Beeinflussung eines Textfeldinhaltes durch eine Formel.....	56
Umstellung der Formatierung eines Textfeldes.....	56
Verschieben von Feldern im Report-Builder.....	57
Erweiterungen der Beispieldatenbank.....	57
Weiteres Einführungs- und Beispielmaterial.....	58
Datenbank erstellen.....	59
Allgemeines bezüglich der Erstellung einer Datenbank.....	60
Neue Datenbank als interne Datenbank.....	60
Zugriff auf externe Datenbanken.....	62
MySQL/MariaDB-Datenbanken.....	63
Erstellen eines Nutzers und einer Datenbank.....	63
MySQL-Verbindung direkt mit der Extension.....	64
MySQL-Verbindung über JDBC.....	64
MySQL-Verbindung über ODBC.....	65
odbcinst.ini.....	65
odbc.ini.....	65
Verbindung zur MySQL-Datenbank über den Datenbank-Assistenten.....	65
Die direkte Verbindung.....	67
Die ODBC-Verbindung.....	71
Die JDBC-Verbindung.....	72
Zugriff auf gespeicherte Prozeduren in MySQL/MariaDB.....	73

PostgreSQL.....	73
Erstellen eines Nutzers und einer Datenbank.....	73
Direkte Verbindung zu Base.....	74
dBase-Datenbanken.....	76
Tabellendokumente.....	78
Thunderbird Adressbuch.....	79
Texttabellen.....	80
Texttabellen innerhalb einer internen HSQLDB-Datenbank.....	80
Texttabellen als Grundlage für eine eigenständige Datenbankdatei.....	83
Firebird.....	85
Erstellen eines Nutzers und einer Datenbank.....	85
Firebird-Verbindung über JDBC.....	86
Firebird-Verbindung über ODBC.....	87
odbcinst.ini.....	87
odbc.ini.....	87
SQLite.....	87
Erstellen einer Datenbank.....	87
SQLite-Verbindung über ODBC.....	88
odbcinst.ini.....	88
odbc.ini.....	88
Weitere SQLite-Verbindungen.....	88
Nachträgliche Bearbeitung der Verbindungseigenschaften.....	88
Tabellen.....	93
Allgemeines zu Tabellen.....	94
Beziehungen von Tabellen.....	94
Beziehungen zwischen Tabellen allgemein.....	94
Tabellen und Beziehungen der Beispieldatenbank.....	97
Tabellen Medienaufnahme.....	97
Tabellen Ausleihe.....	99
Tabellen Nutzerverwaltung.....	99
Erstellung von Tabellen.....	100
Erstellung mit der grafischen Benutzeroberfläche.....	101
Primärschlüssel.....	106
Formatierung von Feldern.....	106
Einstellung eines Indexes.....	109
Änderung bestehender Tabellen.....	111
Probleme bei der Änderung von Tabellen.....	112
Mängel der grafischen Tabellenerstellung.....	114
Direkte Eingabe von SQL-Befehlen.....	114
Tabellenerstellung.....	116
Tabellenänderung.....	119
Tabellen löschen.....	121
Verknüpfung von Tabellen.....	122
Eingabe von Daten in Tabellen.....	126
Eingabe über die grafische Benutzeroberfläche der Tabelle.....	126
Sortieren von Tabellen.....	129
Suchen in Tabellen.....	129
Filtern von Tabellen.....	131
Eingabemöglichkeiten über SQL direkt.....	133
Neue Datensätze einfügen.....	133
Bestehende Datensätze ändern.....	133
Bestehende Datensätze löschen.....	134
Import von Daten aus anderen Datenquellen.....	135
Importierte Daten an bestehende Daten einer Tabelle anfügen.....	136
Neue Tabelle beim Import erstellen.....	137

Daten Aufsplitten beim Import.....	139
Mängel dieser Eingabemöglichkeiten.....	140
Tabellen verstecken.....	141
Formulare.....	144
Formulare als Eingabeerleichterung.....	145
Erstellung von Formularen.....	145
Einfaches Formular.....	145
Symbolleisten des Formularentwurfs.....	147
Formulargründung über den Navigator.....	147
Formulargründung über ein Formularfeld.....	148
Externe Formulare.....	150
Formular-Eigenschaften.....	150
Eigenschaften der Kontrollfelder.....	153
Standardeinstellungen vieler Kontrollfelder.....	154
Textfeld.....	160
Numerisches Feld.....	161
Datumsfeld.....	162
Zeitfeld.....	162
Währungsfeld.....	163
Formatiertes Feld.....	164
Listefeld.....	165
Kombinationsfeld.....	170
Markierfeld.....	173
Optionsfeld.....	174
Grafisches Kontrollfeld.....	175
Maskiertes Feld.....	176
Tabellen-Kontrollfeld.....	177
Beschriftungsfeld.....	178
Gruppierungsrahmen.....	179
Schaltfläche.....	182
Grafische Schaltfläche.....	184
Navigationsleiste.....	184
Drehfeld und Bildlaufleiste.....	186
Verstecktes Steuerelement.....	187
Mehrfachselektion.....	188
Einfaches Formular komplett erstellt.....	190
Felder als Gruppe hinzufügen.....	190
Felder anpassen.....	191
Felder einzeln hinzufügen.....	195
Tabellenkontrollfeld.....	196
Hauptformular und Unterformular.....	201
Eine Ansicht – viele Formulare.....	215
Fehlermeldungen bei der Eingabe in Formulare.....	222
Suchen und Filtern in Formularen über die Navigationsleiste.....	223
Datensatzsuche mit Parametern.....	223
Filterung mit dem Autofilter.....	224
Filterung mit dem formularbasierten Filter.....	226
Filterung über den Standardfilter.....	228
Fazit.....	229
Dateneingabe und Navigation.....	229
Drucken aus Formularen.....	231
Darstellungsgröße von Formularen.....	231
Abfragen.....	232
Allgemeines zu Abfragen.....	233

Eingabemöglichkeiten für Abfragen.....	233
Abfrageerstellung mit der grafischen Benutzeroberfläche.....	233
Funktionen in der Abfrage.....	241
Beziehungsdefinition in der Abfrage.....	244
Abfrageeigenschaften definieren.....	248
Abfragen nachträglich ändern.....	250
Abfrageerweiterungen im SQL-Modus.....	250
Verwendung eines Alias in Abfragen.....	266
Abfragen für die Erstellung von Listenfeldern.....	267
Abfragen als Grundlage von Zusatzinformationen in Formularen.....	270
Eingabemöglichkeit in Abfragen.....	271
Verwendung von Parametern in Abfragen.....	276
Unterabfragen.....	277
Korrelierte Unterabfrage.....	278
Abfragen als Bezugstabellen von Abfragen.....	279
Zusammenfassung von Daten mit Abfragen.....	282
Schnellerer Zugriff auf Abfragen durch Tabellenansichten.....	283
Zeitdifferenzen berechnen.....	284
Berichte.....	286
Berichte mit dem Report-Designer.....	287
Die Benutzeroberfläche des Report-Designers.....	287
Allgemeine Eigenschaften von Feldern.....	295
Besondere Eigenschaften des grafischen Kontrollfeldes.....	298
Diagramme im Bericht einbinden.....	299
Dateneigenschaften von Feldern.....	304
Funktionen im Report-Designer.....	305
Formeleingaben.....	305
Benutzerdefinierte Funktionen.....	312
Formeleingabe für ein Feld.....	314
Bedingte Anzeige.....	314
Bedingte Formatierung.....	315
Beispiele für Berichte mit dem Report-Designer.....	315
Rechnungserstellung.....	316
Ausdruck von Berichten zum aktuellen Datensatz des Formulars.....	323
Aufbau der Filtertabelle.....	324
Aufbau des Makros zum Start des gefilterten Berichtes.....	324
Wechselnde Einfärbung von Zeilen.....	325
Zweispaltige Berichte.....	327
Fehlerquellen bei Berichten mit dem Report-Designer.....	331
Der Inhalt eines Feldes aus einer Abfrage erscheint nicht.....	331
Ein Bericht lässt sich nicht ausführen.....	331
Datums- und Zeitwerte werden in Diagrammen nicht angezeigt.....	332
Andere Formen der Berichtserstellung.....	333
Das alte Berichtsmodul.....	333
Die BaseReportExtension.....	333
Weitere Berichtsmöglichkeiten.....	333
Datenbank-Anbindung.....	334
Allgemeines zur Datenbank-Anbindung.....	335
Anmeldung der Datenbank.....	335
Datenquellenbrowser.....	335
Daten in Text.....	337
Daten in Felder.....	341

Seriendruck.....	341
Aktuelle Dokument-Datenquelle.....	342
Explorer ein/aus.....	342
Serienbriefferstellung.....	342
Erstellung von Etiketten.....	349
Serienbriefe und Etiketten direkt erstellen.....	353
Serienbrieffelder mit der Maus erstellen.....	353
Serienbrieffelder über Feldbefehle erstellen.....	354
Externe Formulare.....	356
Vorteil externer Formulare.....	357
Nachteil externer Formulare.....	357
Datenbanknutzung in Calc.....	357
Daten in Calc einfügen.....	358
Daten aus Calc in eine Datenbank exportieren.....	362
Daten von einer Datenbank zu einer anderen konvertieren.....	363
Daten über die Zwischenablage in eine Tabelle einfügen.....	363
Datenimport aus PDF-Formularen.....	363
Erstellen eines PDF-Formulars.....	364
Auslesen der Daten aus dem PDF-Formular.....	365
Datenbank-Aufgaben.....	371
Allgemeines zu Datenbankaufgaben.....	372
Datenfilterung.....	372
Datensuche.....	374
Suche mit LIKE.....	374
Suche mit LOCATE.....	376
Bilder und Dokumente mit Base verarbeiten.....	380
Bilder in die Datenbank einlesen.....	381
Bilder und Dokumente verknüpfen.....	381
Dokumente mit absoluter Pfadangabe verknüpfen.....	382
Dokumente mit relativer Pfadangabe verknüpfen.....	383
Verknüpfte Bilder und Dokumente anzeigen.....	385
Dokumente in die Datenbank einlesen.....	385
Bildnamen ermitteln.....	387
Bildnamen aus dem Speicher entfernen.....	388
Bilder und Dokumente auslesen und anzeigen.....	388
Diagramme in Formulare einbinden.....	389
Diagramme aus dem Writer importieren.....	390
Abfragen erstellen und als Ansichten speichern.....	390
Abfrage für ein Säulendiagramm.....	390
Abfrage für ein Kreisdiagramm.....	391
Abfrage für ein XY-Diagramm.....	392
Diagramme über Makros anpassen.....	393
Codeschnipsel.....	395
Aktuelles Alter ermitteln.....	396
Geburtstage in den nächsten Tagen anzeigen.....	397
Tage zu Datumswerten addieren.....	398
Zeiten zu Zeitstempeln addieren.....	400
Laufenden Kontostand nach Kategorien ermitteln.....	401
Zeilenummerierung.....	402
Zeilenumbruch durch eine Abfrage erreichen.....	405
Gruppieren und Zusammenfassen.....	406
Makros.....	408
Allgemeines zu Makros.....	409

Der Makro-Editor.....	411
Benennung von Modulen, Dialogen und Bibliotheken.....	412
Makros in Base.....	413
Makros benutzen.....	413
Makros zuweisen.....	413
Ereignisse eines Formulars beim Öffnen oder Schließen des Fensters.....	414
Ereignisse eines Formulars bei geöffnetem Fenster.....	414
Ereignisse innerhalb eines Formulars.....	415
Bestandteile von Makros.....	416
Der «Rahmen» eines Makros.....	417
Variablen definieren.....	417
Arrays definieren.....	417
Zugriff auf das Formular.....	418
Zugriff auf Elemente eines Formulars.....	419
Zugriff auf die Datenbank.....	420
Die Verbindung zur Datenbank.....	420
SQL-Befehle.....	420
Vorbereitete SQL-Befehle mit Parametern.....	421
Datensätze lesen und benutzen.....	422
Mithilfe des Formulars.....	422
Ergebnis einer Abfrage.....	423
Mithilfe eines Kontrollfelds.....	424
Datensätze wechseln und bestimmte Datensätze ansteuern.....	424
Datensätze bearbeiten – neu anlegen, ändern, löschen.....	424
Inhalt eines Kontrollfelds ändern.....	424
Zeile einer Datenmenge ändern.....	425
Zeilen anlegen, ändern, löschen.....	425
Kontrollfelder prüfen und ändern.....	426
Englische Bezeichner in Makros.....	426
Eigenschaften bei Formularen und Kontrollfeldern.....	427
Schrift.....	427
Formular.....	427
Einheitlich für alle Arten von Kontrollfeld.....	427
Einheitlich für viele Arten von Kontrollfeld.....	428
Textfeld – weitere Angaben.....	428
Numerisches Feld.....	428
Datumfeld.....	429
Zeitfeld.....	429
Währungsfeld.....	430
Formatiertes Feld.....	430
Listefeld.....	430
Kombinationsfeld.....	432
Markierfeld, Optionsfeld.....	432
Maskiertes Feld.....	432
Tabellenkontrollfeld.....	432
Beschriftungsfeld.....	433
Gruppierungsrahmen.....	433
Schaltfläche.....	433
Navigationsleiste.....	433
Methoden bei Formularen und Kontrollfeldern.....	433
In einer Datenmenge navigieren.....	434
Datenzeilen bearbeiten.....	434
Einzelne Werte bearbeiten.....	436
Parameter für vorbereitete SQL-Befehle.....	438
Bedienbarkeit verbessern.....	438
Automatisches Aktualisieren von Formularen.....	438
Filtern von Datensätzen.....	439

Daten über den Formularfilter filtern.....	442
Daten aus Textfeldern auf SQL-Tauglichkeit vorbereiten.....	443
Beliebige SQL-Kommandos speichern und bei Bedarf ausführen.....	443
Werte in einem Formular vorausberechnen.....	444
Die aktuelle Office-Version ermitteln.....	445
Wert von Listefeldern ermitteln.....	445
Listenfelder durch Eingabe von Anfangsbuchstaben einschränken.....	446
Datumswert aus einem Formularwert in eine Datumsvariable umwandeln.....	448
Suchen von Datensätzen.....	449
Suchen in Formularen und Ergebnisse farbig hervorheben.....	451
Rechtschreibkontrolle während der Eingabe.....	455
Kombinationsfelder als Listenfelder mit Eingabemöglichkeit.....	457
Textanzeige im Kombinationsfeld.....	458
Fremdschlüsselwert vom Kombinationsfeld zum numerischen Feld übertragen.....	460
Kontrollfunktion für die Zeichenlänge der Kombinationsfelder.....	466
Datensatzaktion erzeugen.....	467
Navigation von einem Formular zum anderen.....	467
Tabellen, Abfragen, Formulare und Berichte öffnen.....	468
Hierarchische Listenfelder.....	470
Zeiteingaben mit Millisekunden.....	473
Ein Ereignis – mehrere Implementationen.....	475
Abspeichern mit Nachfrage.....	475
Primärschlüssel aus Nummerierung und Jahreszahl.....	476
Datenbankaufgaben mit Makros erweitert.....	477
Verbindung mit Datenbanken erzeugen.....	477
Daten von einer Datenbank in eine andere kopieren.....	478
Direkter Import von Daten aus Calc.....	479
Zugriff auf Abfragen.....	482
Datenbanksicherungen erstellen.....	482
Datenbanken komprimieren.....	486
Tabellenindex heruntersetzen bei Autowert-Feldern.....	486
Drucken aus Base heraus.....	487
Druck von Berichten aus einem internen Formular heraus.....	487
Start, Formatierung, direkter Druck und Schließen des Berichts.....	488
Druck von Berichten aus einem externen Formular heraus.....	489
Serienbriefdruck aus Base heraus.....	490
Drucken über Textfelder.....	491
Aufruf von Anwendungen zum Öffnen von Dateien.....	492
Aufruf eines Mailprogramms mit Inhaltsvorgaben.....	493
Mauszeiger beim Überfahren eines Links ändern.....	494
Formulare ohne Symbolleisten präsentieren.....	494
Formulare ohne Symbolleisten in einem Fenster.....	495
Formulare im Vollbildmodus.....	496
Formular direkt beim Öffnen der Datenbankdatei starten.....	497
MySQL-Datenbank mit Makros ansprechen.....	497
MySQL-Code in Makros.....	497
Temporäre Tabelle als individueller Zwischenspeicher.....	497
Filterung über die Verbindungsnummer.....	498
Gespeicherte Prozeduren.....	498
Automatischer Aufruf einer Prozedur.....	499
Übertragung der Ausgabe einer Prozedur in eine temporäre Tabelle.....	499
Dialoge.....	499
Dialoge starten und beenden.....	500
Einfacher Dialog zur Eingabe neuer Datensätze.....	501
Dialog zum Bearbeiten von Daten in einer Tabelle.....	503
Fehleinträge von Tabellen mit Hilfe eines Dialogs bereinigen.....	508

Makrozugriff mit Access2Base.....	516
Wartung.....	517
Allgemeines zur Wartung von Datenbanken.....	518
Datenbank komprimieren.....	518
Autowerte neu einstellen.....	518
Datenbankeigenschaften abfragen.....	518
Daten exportieren.....	519
Tabellen auf unnötige Einträge überprüfen.....	520
Einträge durch Beziehungsdefinition kontrollieren.....	520
Einträge durch Formular und Unterformular bearbeiten.....	521
Verwaiste Einträge durch Abfrage ermitteln.....	523
Datenbankgeschwindigkeit.....	523
Einfluss von Abfragen.....	523
Einfluss von Listefeldern und Kombinationsfeldern.....	524
Einfluss des verwendeten Datenbanksystems.....	524
Anhang.....	525
Barcode.....	526
Datentypen des Tabelleneditors.....	526
Ganzzahlen.....	526
Fließkommazahlen.....	526
Text.....	527
Zeit.....	527
Sonstige.....	527
Datentypen in StarBasic.....	528
Zahlen.....	528
Sonstige.....	528
Eingebaute Funktionen und abgespeicherte Prozeduren.....	528
Numerisch.....	529
Text.....	531
Datum/Zeit.....	534
Datenbankverbindung.....	536
System.....	538
Steuerzeichen zur Nutzung in Abfragen.....	539
Einige uno-Befehle zur Nutzung mit einer Schaltfläche.....	540
Informationstabellen der HSQLDB.....	540
Informationstabellen der Firebird-Datenbank.....	542
Datenbankreparatur für *.odb-Dateien.....	543
Wiederherstellung der Datenbank-Archivdatei.....	543
Weitere Informationen zur Datenbank-Archivdatei.....	544
Behebung von Versionsproblemen.....	551
Weitere Tipps.....	552
Datenbankverbindung zu einer externen HSQLDB.....	552
Parallelinstallation von interner und externer HSQLDB.....	554
Änderung der Datenbankverbindung zur externen HSQLDB.....	555
Änderung der Datenbankverbindung für einen Mehrbenutzerbetrieb.....	556
Autoinkrementwerte mit der externen HSQLDB.....	558
Umgang mit der internen Firebird-Datenbank.....	559
Funktionserweiterungen und -änderungen in Base im Laufe der LO-Versionen.....	559
Glossar.....	561
Stichwortverzeichnis.....	575

Vorwort

Für wen ist dieses Buch?

Jeder, der sich in das Modul Base von LibreOffice einarbeiten und tiefer einsteigen will, findet hier die Möglichkeit. Sei es, dass Sie noch nie mit Datenbanken und damit in einem Datenbanksystem mit DBMS (**D**atabase **m**anagement **s**ystem) gearbeitet haben oder sei es, dass Sie ein anderes Datenbanksystem aus einer OfficeSuite oder ein eigenständiges Datenbanksystem wie beispielsweise MySQL gewohnt sind.

Was finden Sie in diesem Buch?

Dieses Buch führt Sie in die gebräuchlichsten Funktionen von LibreOffice-Base ein:

- Einführung
- Datenbank erstellen
- Ein- und Ausgabe der Datenbank: Tabellen, Formulare, Abfragen, Berichte
- Aufgaben einer Datenbank
- Makros
- Pflege und Wartung von Datenbanken
- und einiges mehr

Wo bekomme ich mehr Hilfe?

Dieses Buch, wie auch die anderen LibreOffice-Handbücher, das eingebaute Hilfesystem und die Benutzer-Supportsysteme setzen voraus, dass Sie mit Ihrem Computer und den Basisfunktionen wie das Starten eines Programms, Öffnen und Speichern von Dateien vertraut sind.

Hilfesystem

LibreOffice besitzt ein umfangreiches Hilfesystem.

Um zu dem Hilfesystem zu gelangen, drücken Sie **F1** oder wählen Sie **LibreOffice Hilfe** aus dem Hilfemenü. Zusätzlich können Sie wählen, ob Sie Tipps, Erweiterte Tipps und den Office-Assistenten einschalten (**Extras**→**Optionen**→**LibreOffice**→**Allgemein**).

Wenn die Tipps eingeschaltet sind, platzieren Sie den Mauszeiger über eines der Symbole um eine kleine Box («Tooltip») angezeigt zu bekommen. Darin befindet sich eine kurze Erklärung der Funktion des Symbols. Um noch mehr Erklärungen zu erhalten, wählen Sie **Hilfe** → **Direkthilfe** und halten den Mauszeiger über das Symbol.

Freier Onlinesupport

Die LibreOffice-Gemeinschaft, manchmal wegen ihrer Internationalität auch Community bezeichnet, entwickelt nicht nur die Software LibreOffice, sondern bietet auch kostenfreie, freiwilligen-basierende Unterstützung. Mehr dazu in *Tabelle 1: Kostenlose Unterstützung für Nutzer von LibreOffice* und auf dieser Webseite: <http://de.libreoffice.org/get-help/community-support/> .

Tabelle 1: Kostenlose Unterstützung für Nutzer von LibreOffice

Freier LibreOffice-Support	
• FAQ	• Antworten auf häufig gestellte Fragen (Frequently Asked Questions): https://wiki.documentfoundation.org/Faq/de http://de.libreoffice.org/discover/faq/
• Dokumentation	• Handbücher und andere Dokumentationen: http://de.libreoffice.org/get-help/documentation/ http://wiki.documentfoundation.org/Documentation/de
• Mailinglisten	• Freier Community-Support durch ein Netzwerk an unabhängigen, erfahrenen Benutzern: http://de.libreoffice.org/get-help/mailling-lists/ http://de.libreoffice.org/get-help/nabble/
• Forum	• Wir unterhalten ein eigenes Forum: http://ask.libreoffice.org/de/questions/
• Internationaler Support	• Die LibreOffice-Webseite in Ihrer Sprache http://de.libreoffice.org/ • Die internationalen Mailinglisten: http://wiki.documentfoundation.org/Local_Mailing_Lists

Benutzer können umfassenden Onlinesupport aus der Community über Mailinglisten und Foren bekommen. Andere Webseiten, die von Nutzern geführt werden, bieten auch kostenlose Tipps und Anleitungen.

Bezahlter Support und Schulungen

Auf LibreOffice spezialisierte Firmen bieten bezahlten Support und Service an. Eine Liste mit Firmen kann bei der Mailingliste angefragt werden.

Sie sehen vielleicht etwas anderes

LibreOffice läuft auf Windows, Linux, Mac OS X, FreeBSD und anderen Unix-Varianten, von denen jedes Betriebssystem unterschiedlichste Versionen hat, und kann von den Nutzern bezüglich Schriftarten, Farben und Themen angepasst werden.

Die Bilder in diesem Buch wurden von einer Vielzahl von Computern und Betriebssystemen übernommen. Einige Bilder werden deshalb eventuell nicht genauso aussehen, wie sie auf Ihrem Computer zu sehen sind.

Anmerkung für Macintosh Nutzer

Einige Tastenbelegungen (Tastenkürzel) und Menüeinträge unterscheiden sich zwischen der Macintosh Version und denen für Windows- und Linux-Rechnern. Die unten stehende Tabelle gibt Ihnen einige grundlegende Hinweise dazu. Eine ausführlichere Aufstellung dazu finden Sie in der Hilfedatei des jeweiligen Moduls.

Windows/Linux	entspricht am Mac	Effekt
Menü-Auswahl Extras → Optionen	LibreOffice → Einstellungen	Zugriff auf die Programmooptionen
Rechts-Klick	<i>Control</i> +Klick	Öffnen eines Kontextmenüs
<i>Ctrl</i> (Control) oder <i>Strg</i> (Steuerung)	⌘ (<i>Command</i>)	Tastenkürzel in Verbindung mit anderen Tasten
<i>F5</i>	<i>Shift</i> +⌘+ <i>F5</i>	Öffnen des Dokumentnavigator-Dialogs
<i>F11</i>	⌘+ <i>T</i>	Öffnen des Formatvorlagen-Dialogs

Verwenden von Tipps, Hinweisen und Warnungen

In den Handbüchern werden besondere Information zusätzlich gekennzeichnet: Tipps, Hinweise und Warnungen.

Tipp

Ein Tipp beschreibt eine praktische aber nicht wesentliche Information, die nicht in den Textfluss passt.

Hinweis

Ein Hinweis enthält Informationen mit Bezug auf den Text. Er erklärt, beschreibt oder kommentiert eine Aussage, um die Aufmerksamkeit des Lesers darauf zu richten.

Vorsicht



Warnungen (Vorsicht) zeigen Operationen, die zu Datenverlust führen können.

Wer hat dieses Buch geschrieben?

Dieses Buch wurde durch Freiwillige der LibreOffice-Gemeinschaft geschrieben. Gewinne aus dem Verkauf einer gedruckten Version werden für die Gemeinschaft bzw. die Stiftung „The Document Foundation“ verwendet.

FAQs (häufig gestellte Fragen)

Zu LibreOffice gibt es immer wieder häufig gestellte, allgemeine Fragen. Hier ein paar Antworten:

Wie ist LibreOffice lizenziert?

LibreOffice wird unter der von der Open Source Initiative (OSI) anerkannten Lesser General Public License (LGPL) vertrieben. Die LGPL-Lizenz ist auf der LibreOffice Website verfügbar:

<http://www.libreoffice.org/download/license/>

Darf ich LibreOffice an jeden vertreiben?

Ja.

Auf wie vielen Computern kann ich LibreOffice installieren?

Auf so vielen, wie Sie möchten.

Darf ich es verkaufen?

Ja.

Darf ich LibreOffice in meinem Unternehmen kostenlos verwenden?

Ja.

Ist LibreOffice in meiner Sprache verfügbar?

LibreOffice wurde in sehr vielen Sprachen übersetzt, immer wieder kommen Sprachen dazu, so dass die von Ihnen gewünschte Sprache sehr wahrscheinlich unterstützt wird. Darüber hinaus gibt es sehr viele Rechtschreib-, Silbentrenn- und Thesaurus-Wörterbücher für Sprachen und Dialekte, für die es keine lokalisierte Programmoberfläche gibt. Die Wörterbücher sind auf der LibreOffice Website erhältlich unter: <http://de.libreoffice.org/discover/templates-and-extensions/>.

Wie können Sie es kostenlos anbieten?

LibreOffice wird von Freiwilligen entwickelt und gepflegt und hat die Unterstützung von mehreren Unternehmen.

Ich schreibe eine Software-Anwendung, darf ich Programmcode von LibreOffice in meinem Programm einbauen?

Sie können dies im Rahmen der Parameter, die in der LGPL gesetzt sind, tun. Lesen Sie hierzu die Lizenzvereinbarung: <http://www.libreoffice.org/download/license/>

Wozu brauche ich Java, um LibreOffice laufen zu lassen? Ist es in Java geschrieben?

LibreOffice ist nicht in Java geschrieben, es wird in der Sprache C++ geschrieben. Java ist eine von mehreren Sprachen, die verwendet werden, um die Software zu erweitern. Das Java JDK / JRE ist nur für einige Funktionen erforderlich. Am wichtigsten davon ist die relationale Datenbank-Engine HSQLDB.

Hinweis: Java ist kostenlos erhältlich. Wenn Sie nicht möchten, dass Java verwendet wird, können Sie weiterhin nahezu alle Funktionen anderer Programmteile von LibreOffice nutzen. Base ist mit seiner Verbindung zu einer internen Java-Datenbank dabei allerdings ausgenommen.

Wie kann ich zu LibreOffice beitragen?

Sie können mit der Entwicklung und Pflege von LibreOffice in vielerlei Hinsicht helfen, und Sie brauchen kein Programmierer zu sein. Zum Einstieg finden Sie auf dieser Webseite weitere Informationen: <http://de.libreoffice.org/community/get-involved/>

Einführung in Base

Einführung

Im täglichen Büroinsatz werden häufig Tabellenkalkulationen dazu benutzt, Datensammlungen zu erstellen um anschließend damit eventuell noch kleine Berechnungen durchzuführen. Die Tabellensicht ist sofort da, der Inhalt einfach einzugeben – da fragen sich dann viele Nutzer, warum es denn eine Datenbank sein sollte. Dieses Handbuch versucht den Unterschied zwischen Tabellenkalkulation und Datenbank herauszuarbeiten. Zu Beginn wird erst einmal kurz dargestellt, was eine Datenbank denn überhaupt leisten kann.

Hinweis

In der Fachsprache wird statt von Datenbanken von einer Benutzeroberfläche und einem «Datenbanksystem» gesprochen. Dieser Begriff umfasst das «Datenbankmanagementsystem» (DBMS) und den eigentlichen Datenbestand, die «Datenbank».

Base hingegen bietet einen Zugriff auf verschiedene Datenbanksysteme über eine grafische Benutzeroberfläche. Base arbeitet standardmäßig mit dem eingebetteten Datenbanksystem «HSQLDB». Seit der Version LO 4.2 ist außerdem das Datenbanksystem «Firebird», zuerst als «experimentelle Funktion», hinzugekommen. Mit der Version LO 5.3 ist auf die Version 3.0 des Firebird-Datenbanksystems, ebenfalls zunächst noch als «experimentelle Funktion» umgestellt worden.

«(HSQLDB, FIREBIRD)» würde bedeuten: Funktioniert mit der HSQLDB, nicht aber mit Firebird. Unterschiede sind entsprechend gekennzeichnet.

Das gesamte Handbuch bezieht sich in der Hauptsache auf zwei Beispieldatenbanken. Die eine Datenbank hat die Bezeichnung «Medien_ohne_Makros.odt», die andere Datenbank ist entsprechend mit Makros erweitert worden und trägt die Bezeichnung «Medien_mit_Makros.odt». Beide Datenbanken sollen einen Bibliotheksbetrieb ermöglichen: Medienaufnahme, Nutzeraufnahme, Medienverleih und alles, was damit verbunden ist wie z. B. das Anmahnen säumiger EntleiherInnen.

Daneben sind viele Beispieldatenbanken¹ beigefügt, die nur auf spezielle Probleme zugeschnitten sind:

- Beispiel_Sport.odt (Kapitel «Einführung in Base»)
- Beispiel_CSV_Einbindung.odt (Kapitel «Datenbank erstellen»)
- Beispiel_Cursorsprung_Subform_Mainform.odt (Kapitel «Formulare»)
- Beispiel_Bericht_bedingte_Einblendung_von_Grafiken.odt (Kapitel «Berichte»)
- Beispiel_Bericht_Rechnung.odt (Kapitel «Berichte»)
- Beispiel_Bericht_Zeilen_Farbwechsel_Spalten.odt (Kapitel «Berichte»)
- Beispiel_PDFFormular_Import.odt (Kapitel «Datenbank-Anbindung»)
- Beispiel_Autotext_Suchmarkierung_Rechtschreibung.odt (Kapitel «Datenbank-Aufgaben»)
- Beispiel_Dokumente_einlesen_auslesen.odt (Kapitel «Datenbank-Aufgaben»)
- Beispiel_Baseformular_mit_Diagramm.odt (Kapitel «Datenbank-Aufgaben»)
- Beispiel_Suchen_und_Filtern.odt (Kapitel «Makros»)
- Beispiel_Direktberechnung_im_Formular.odt (Kapitel «Makros»)
- Beispiel_Combobox_Listfeld.odt (Kapitel «Makros»)
- Beispiel_Fortlaufende_Nummer_Jahr.odt (Kapitel «Makros»)
- Beispiel_Datenbank_Serienbrief_direkt.odt (Kapitel «Makros»)
- Beispiel_Mailstart_Dateiaufruf.odt (Kapitel «Makros»)
- Beispiel_Daten_Import.odt (Kapitel «Makros»)
- Beispiel_Dialoge.odt (Kapitel «Makros»)

¹ Alle Beispieldatenbanken können auf <http://de.libreoffice.org/get-help/documentation/> herunter geladen werden.

Hinweis

Wie jede Software läuft auch LO-Base nicht vollkommen fehlerfrei. Besonders ärgerlich sind hier die «Regressionen», also Rückschritte von einer vorhergehenden Version zur gerade aktuellen Version. Der folgende Link führt zu den momentan noch offenen Regressionen: https://bugs.documentfoundation.org/buglist.cgi?bug_status=UNCONFIRMED&bug_status=NEW&bug_status=REOPENED&bug_status=NEEDINFO&component=Base&keywords=regression&keywords_type=all-words&product=LibreOffice&query_format=advanced&resolution=---

Ein Blick auf die Bug-Liste kann also helfen, Unterschiede zwischen Dokumentation und eigener Programmversion zu verstehen.

Hinweis

In diesem Handbuch werden Anführungszeichen mit unterschiedlichen Intentionen verwendet:

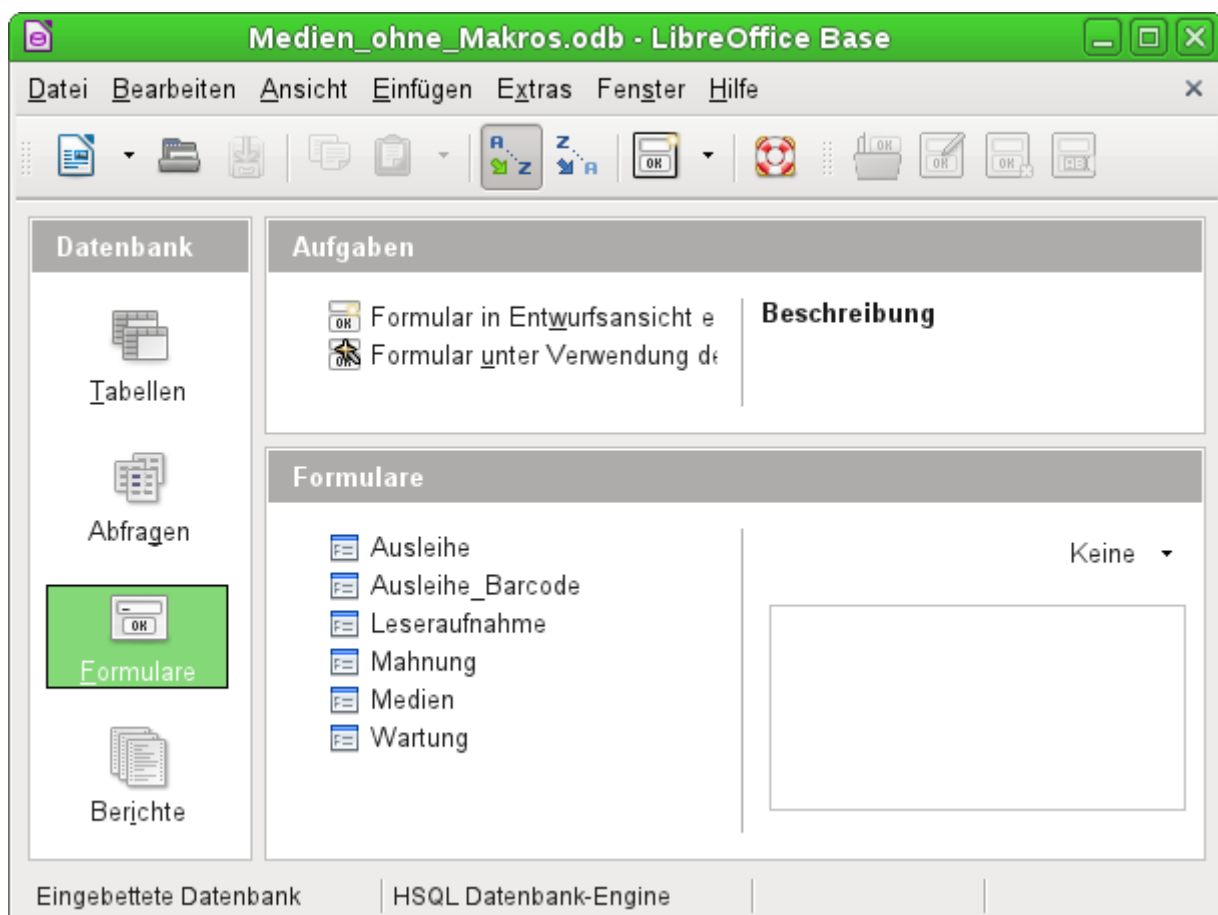
"Tabellenname"."Feldname" → Tabellen und Felder in Datenbanken werden in doppelten Anführungszeichen oben wieder gegeben.

'Wert' → Werte, die in die Datenbank eingegeben oder aus ihr gelesen werden, sollen in einfachen Anführungszeichen erscheinen.

«Hervorhebungen» → Alles, was sonst hervorgehoben werden soll, erscheint in der französischen Form der Anführungszeichen.

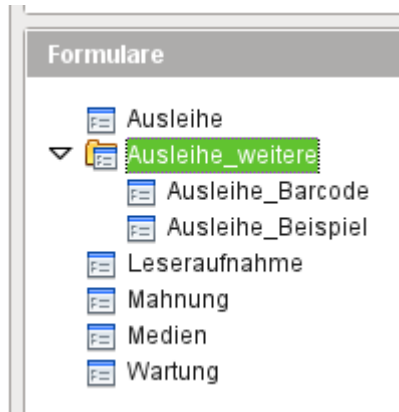
Base – ein Container für Datenbankinhalte

Eine Base-Datei ist eigentlich nur ein gepacktes Verzeichnis, in dem Informationen für die verschiedenen Arbeitsbereiche von Base stecken. In der täglichen Nutzung startet eine Base-Datei erst einmal mit der folgenden Ansicht:



Zur Arbeitsumgebung von Base gehören insgesamt vier Arbeitsbereiche: Tabellen, Abfragen, Formulare und Berichte. Je nach gewähltem Arbeitsbereich können bestimmte Aufgaben zur Neuerstellung der Elemente in Angriff genommen werden oder entsprechend fertiggestellte Elemente aufgerufen werden.

In den Arbeitsbereichen «Formulare» und «Berichte» können die jeweiligen Elemente auch innerhalb einer Verzeichnisstruktur angeordnet werden:



Dies geht entweder direkt beim Abspeichern über den Speicherdialog oder durch Neugründung von Verzeichnissen über **Einfügen → Ordner**.

Obwohl die Basis für eine Datenbank durch Tabellen gebildet wird, startet Base mit der Formularansicht, weil Formulare in der Regel die Elemente sind, mit denen die tägliche Datenbankarbeit vorstatten geht. Über die Formulare werden Einträge in die Tabellen vorgenommen und Inhalte aus den Tabellen ausgewertet.

Formulare – Start für die Dateneingabe

Medien_ohne_Makros.odb : Ausleihe - LibreOffice Base: Database Form

Datei Bearbeiten Ansicht Einfügen Format Tabelle Extras Fenster Hilfe

Ausleihe

Vorname	Nachname
Bert	Lederstrumpf
Heinrich	Müller
Lisa	Gerd
Monika	Mirinda
Hein	Keindurchblick

Filter (Nachname) OK

Datensatz 2 von 10

Datensatz 2 von 10

Ausleihe für Leser(in) Müller, Heinrich

Entleihdatum: 22.04.12 Aktualisieren

aktuelle Ausleihe

Medium	Leih-Datum
8 - im Augenblick - von van Veen, Herman	22.04.12

Datensatz 1 von 1

Rückgabe

Medium	Leih-Datum	Rück-Datum	Verlängerung	Leihzeit	Restzeit
2 - Eine kurze Geschichte der Zeit - von Hawking, Steven W	04.04.12		1	18 Tage	3

Datensatz 1 von 1

Seite 1 / 1 Standard STD 75%

Einfache Formulare bilden lediglich eine Tabelle wie im oben sichtbaren Tabellenkontrollfeld mit den Namen ab. Dieses Formular erfüllt durch seine Struktur einige zusätzliche Punkte.

- Die Auswahl der Personen kann mit Filtern eingeschränkt werden. Die einfache Eingabe des Buchstaben «G» erzeugt eine Auswahl aller Personen, deren Nachname mit «G» beginnt.
- Neue Mediennutzer können direkt in das Tabellenkontrollfeld eingegeben werden.
- Die Ausleihdaten des aktuell markierten Nutzers werden in den darunterliegenden Feldern angezeigt. Dabei wird auch noch einmal der Name des Nutzers deutlich hervorgehoben. Hat ein Nutzer ein Buch entliehen und müsste es bereits zurückgegeben werden, so steht das Tabellenkontrollfeld «aktuelle Ausleihe» erst einmal nicht zur Verfügung. Stattdessen wird angegeben, dass die Ausleihe zur Zeit gesperrt ist.
- Das Entleihdatum wird mit dem aktuellen Datum vorgegeben. In dem daneben stehenden Listenfeld werden die zu entleihenden Medien ausgewählt. Dabei können keine Medien ausgewählt werden, die zur Zeit noch entliehen sind.

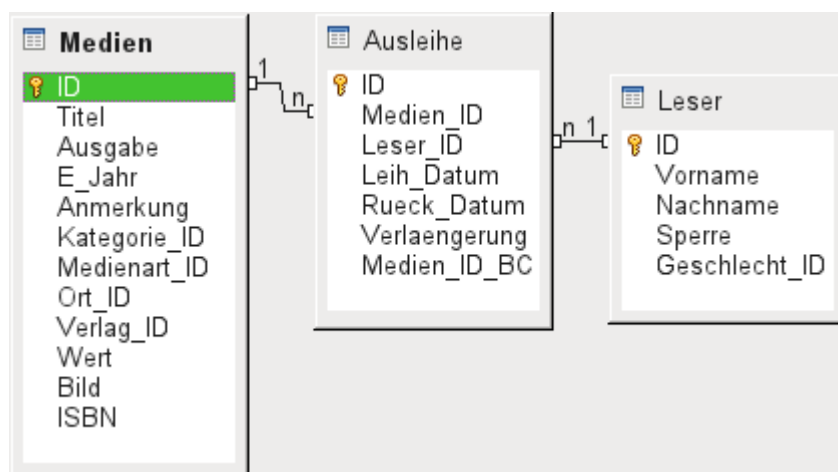
- Die ausgewählten Daten werden mit dem Button **Aktualisieren** in das Tabellenkontrollfeld für den aktuellen Ausleihvorgang übernommen.
- Im Tabellenkontrollfeld für die Rückgabe ist es nicht möglich, einen Datensatz einfach zu löschen. Nur die Felder «Rück-Datum» und «Verlängerung» können bearbeitet werden. War ein Mediennutzer vorübergehend gesperrt und hat seine entliehenen Medien zurückgegeben, so lässt sich die Ausleihe über den Button **Aktualisieren** wieder freischalten.

Alle diese Funktionen können ohne den Einsatz von zusätzlicher Programmierung mit Makros gewährleistet werden, wenn entsprechend an den Formularen gefeilt wird.

Tabellen – Grundlagen für die Dateneingabe

Die Tabellen in einer Datenbank hängen in einem großen Geflecht zusammen. Eine Tabelle bezieht Informationen aus einer anderen oder gibt Informationen an andere Tabellen weiter. Dies wird als «Relation» bezeichnet.

Die Tabelle "Ausleihe" steht in direkter Beziehung zu den Tabellen "Medien" und "Leser".



Statt in der Ausleihe den Titel eines Buches abzuspeichern, wird dort nur eine Zahl abgespeichert. Das eindeutige Kennzeichen eines Datensatzes der Tabelle "Medien" ist in dem Feld "ID" gespeichert. Das Feld ist das Schlüsselfeld der Tabelle "Medien", der Primärschlüssel.

In der Tabelle "Ausleihe" wird auch nicht jedes Mal der Lesername eingetragen. Hier gibt die Tabelle "Leser" Auskunft. Auch sie hat ein Primärschlüsselfeld. Der Wert dieses Feldes kann dann in die Tabelle "Ausleihe" eingetragen werden.

Die Beziehungen zwischen den Tabellen haben hier den Vorteil, dass die Schreibarbeit im Formular erheblich reduziert wird. Statt bei jeder Ausleihe ein Medium zumindest unverwechselbar und für jeden erkennbar aufzuschreiben und Leser mit Vor- und Nachnamen zu notieren, wird einfach im Inhalt der anderen Tabellen nachgesehen. Schließlich wird dasselbe Medium später noch öfter ausgeliehen und derselbe Leser kann schon beim ersten Entleihvorgang mehrere Medien entleihen.

Ausleihe - Medien_ohne_Makros - LibreOffice Base: Table Data V						
Datei Bearbeiten Ansicht Einfügen Extras Fenster Hilfe						
	ID	Medien_ID	Leser_ID	Leih_Datum	Rueck_Datum	Verlaengerung
▶	0	1	0	02.11.11	04.11.11	
	1	2	2	15.10.11	25.02.12	2
	2	0	3	02.11.11	04.04.12	1
	3	3	0	04.11.11	28.11.11	2
	9	5	0	28.11.11		
	10	4	0	28.11.11	04.04.12	
	11	4	0	09.11.11		
	12	3	0	09.12.11		
	13	7	0	09.12.11	04.04.12	
	15	0	0	24.02.12	25.02.12	
	16	7	0	25.02.12		
	17	6	2	25.02.12	25.02.12	
	18	1	2	25.02.12	04.04.12	
	19	2	2	25.02.12	04.04.12	
	21	0	9	04.04.12		
	22	2	1	04.04.12		1
	23	1	0	04.04.12		
	24	8	1	22.04.12		
☼	<Auto					
Datensatz 1 von 18						

Die Tabellengrundlage für das oben vorgestellte Formular sieht erst einmal recht nüchtern aus. In der oben abgebildeten Tabelle werden bis auf die Nutzerneueingabe alle Eingaben gemacht, die in dem Formular möglich sind. Dabei werden im Formular die Verbindungen dieser Tabelle zu den anderen Tabellen der Datenbank genutzt.

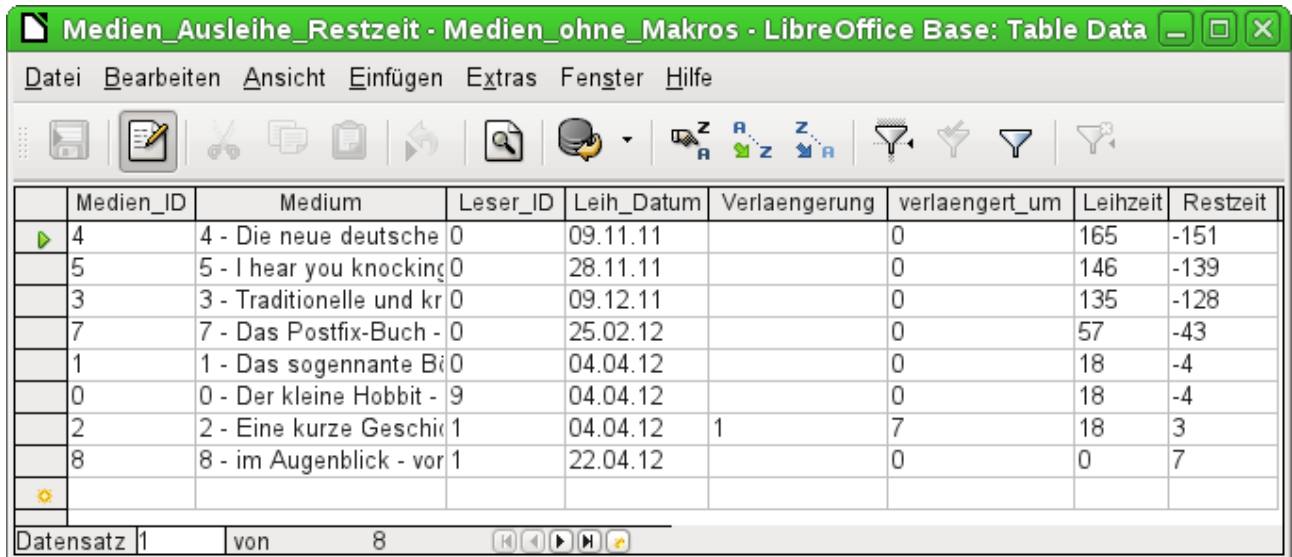
- Das erste Feld zeigt den für die meisten Datenbanken unabdingbaren, unverwechselbaren Inhalt, den Primärschlüssel ("ID"), der automatisch geschrieben wird. Mehr dazu im Kapitel «Beziehungen zwischen Tabellen allgemein».
- Das zweite Feld "Medien_ID" speichert den Primärschlüssel der Tabelle "Medien". Es verweist über die Nummer auf den entsprechenden Inhalt in dieser Tabelle. Solch ein Verweis auf einen Primärschlüssel wird als Fremdschlüssel bezeichnet. Im Formular werden statt des Fremdschlüssels über ein Listefeld der Titel und der Verfasser angezeigt. Das Listefeld gibt im Hintergrund den Fremdschlüssel an die Tabelle weiter.
- Das dritte Feld "Leser_ID" speichert den Primärschlüssel der Tabelle "Leser", also nur eine Nummer, die auf den Leser verweist. Im Formular wird aber der Nachname und der Vorname angegeben. Wie in der Tabelle zu sehen ist, hat der Leser mit der Primärschlüsselnummer '0' sehr viele Medien entliehen. Die Tabelle kann beliebig oft den einzigartigen Primärschlüssel der Tabelle "Leser" als Fremdschlüssel "Leser_ID" abspeichern. Auf keinen Fall darf aber ein Leser, der in dieser Tabelle über den Fremdschlüssel verzeichnet ist, gelöscht werden. Sonst wäre nicht mehr nachvollziehbar, wer denn nun das jeweilige Medium entliehen hat. Die Datenbank macht in den Standardeinstellungen so ein Löschen unmöglich. Der Fachbegriff dafür ist die Gewährung der «referentiellen Integrität».
- Im vierten Feld wird das Ausleihdatum abgespeichert. Ist dieses Datum abgespeichert und entspricht das Datum nicht dem aktuellen Datum, so erscheint der entsprechende Datensatz zu dem entsprechenden Leser im Formular im untersten Tabellenkontrollfeld zu Rückgabe der Medien.

- Im letzten Feld wird eine Verlängerung ermöglicht. Was hier eine 1, 2 usw. bedeutet wird an anderer Stelle festgelegt. Dafür enthält die Datenbank eine gesonderte Tabelle mit der Bezeichnung "Einstellungen".

Diese Eingaben reichen aus, um letztlich einen Bibliotheksbetrieb in Gang zu halten.

Abfragen – Auswertungsmöglichkeiten für eingegebene Daten

Abfragen zeigen eine Sicht auf die Tabellen. Sie bringen Inhalte mehrerer Tabellen zusammen in einer Übersicht. Abfragen werden nur in der Abfragesprache «SQL» gespeichert. Sie sind also keine neuen Tabellen, auch wenn sie von der Ansicht her in Base erst einmal gleich erscheinen.



Medien_ID	Medium	Leser_ID	Leih_Datum	Verlaengerung	verlaengert_um	Leihzeit	Restzeit
4	4 - Die neue deutsche	0	09.11.11		0	165	-151
5	5 - I hear you knocking	0	28.11.11		0	146	-139
3	3 - Traditionelle und kr	0	09.12.11		0	135	-128
7	7 - Das Postfix-Buch - 0	0	25.02.12		0	57	-43
1	1 - Das sogenannte Bi	0	04.04.12		0	18	-4
0	0 - Der kleine Hobbit - 9	0	04.04.12		0	18	-4
2	2 - Eine kurze Geschic	1	04.04.12	1	7	18	3
8	8 - im Augenblick - vor	1	22.04.12		0	0	7

Diese Abfrage listet alle Medien auf, die zur Zeit entliehen sind. Darüber hinaus berechnet sie, wie lange die Medien bereits entliehen sind und wie lange sie noch entliehen werden dürfen. Während das Feld "Medien_ID" das Fremdschlüsselfeld ausliest, wird in dem Feld "Medium" aus dem Primärschlüssel, dem Titel und dem Autor des Mediums ein zusammenhängender Text gebildet. Dieses Feld wird dann im Formular unter dem Untertitel «Rückgabe» benötigt. Einige Felder der Abfrage dienen dabei als Verbindungsfelder zu dem eigentlichen Ausleihformular mit der Tabelle "Ausleihe", nämlich die Felder "Medien_ID" und "Leser_ID".

- Alle Medien, bei denen das Rückgabedatum in der Tabelle "Ausleihe" nicht ausgefüllt ist, werden aufgelistet. Die Medien sind hier als zusätzliche Übersicht auch noch entsprechend benannt.
- Der Bezug zu den Lesern ist hergestellt über den Primärschlüssel der Tabelle "Leser"
- Aus dem Ausleihdatum "Leih_Datum" und dem aktuellen Datum wird die Zeitdifferenz in Tagen als "Leihzeit" angegeben.
- Von der Ausleihzeit, die je nach Medienart unterschiedlich sein kann, wird die Leihzeit abgezogen. Daraus wird die verbleibende Restzeit gebildet.
- Bei einer Verlängerung wurde in der Tabelle "Einstellungen" vermerkt, dass der Wert '1' für eine Verlängerung von 7 Tagen steht. Im vorletzten Datensatz mit der "Medien_ID" '2' ist so eine Verlängerung bei der Restzeit berücksichtigt worden.

Berichte – Präsentationen der Datenauswertung

Über Berichte werden die Daten so aufbereitet, dass sie sinnvoll ausgedruckt werden können. Formulare wie das folgende sind nicht dazu geeignet, z. B. einen sauber formatierten Brief auszugeben.

Medien_ohne_Makros.odb : Mahnung - LibreOffice Base: Database F

Datei Bearbeiten Ansicht Einfügen Format Tabelle Extras Fenster Hilfe

Vorname	Nachname
Bert	Lederstrumpf
Heinrich	Müller
Terence	Nobody

Datensatz 1 von 3

Datensatz 1 von 3

Mahnung für Leser(in) Lederstrumpf, Bert

Medium	Leih_Datum	Verlängerung	Leihzeit	Restzeit
4 - Die neue deutsche Rechtschreibung - von Hermann, Urs	09.11.11		65 Tage	151 Tage
5 - I hear you knocking - von Edmunds, Dave	28.11.11		46 Tage	139 Tage
3 - Traditionelle und kritische Theorie - von Horkheimer, Max	09.12.11		35 Tage	128 Tage
7 - Das Postfix-Buch - von ?	25.02.12		57 Tage	-43 Tage
1 - Das sogenannte Böse - von Lorenz, Konrad	04.04.12		18 Tage	-4 Tage

Datensatz 5 von 5 (1)

Medium: 1 - Das sogenannte Böse - von Lorenz, Konrad

Mahndatum	Mahnung Nr.
22.04.12	1

Datensatz 2 von 2

Seite 1 / 1 | Standard | STD | 7

Bevor ein aktueller Bericht in Form einer Mahnung ausgedruckt werden kann, müssen in diesem Formular erst einmal die Mahnungen bestätigt werden. Im oberen Tabellenkontrollfeld stehen dafür alle Namen der Leser, die ein Medium entliehen haben und für die eine negative Restzeit vermerkt ist.

Für jedes anzumahnende Buch wird im unteren Tabellenkontrollfeld ein Mahndatum festgelegt. Dies dürfte bei der Abarbeitung von Mahnungen das aktuelle Datum sein. Die Mahnungsnummer wird nicht extra geschrieben. Sie wird anhand der bisher ergangenen Mahnungen einfach durch Addition ermittelt.

Dieses Formular benötigt in der Fassung ohne Makros noch eine Eingabe durch den Nutzer. In der Fassung, die mit Makros arbeitet, wird einfach das Datum automatisch geschrieben und anschließend der Bericht zum Mahnungsdruck aufgerufen.

Mahnung1.odt (schreibgeschützt) - LibreOffice Writer

Datei Bearbeiten Ansicht Einfügen Format Tabelle Extras Fenster Hilfe

Libre Office Bibliothek

Herrn
Bert Lederstrumpf
Neuenkirchener Str. 72
D 45793 Pussemuckel

22.04.12

Mahnung

Sehr geehrter Herr Lederstrumpf,

leider haben Sie versäumt, die folgenden Medien rechtzeitig wieder zurück zu geben:

Mahndatum	Mahn-Nr	Medium	Leihdatum	Überzogen	Gebühr
22.04.12	2	3 - Traditionelle und kritische Theorie - von Horkheimer, Max	09.12.11	128Tage	4,50 €
22.04.12	2	5 - I hear you knocking - von Edmunds, Dave	28.11.11	139Tage	4,75 €
22.04.12	3	4 - Die neue deutsche Rechtschreibung - von Hermann, Ursula	09.11.11	151Tage	5,25 €
22.04.12	1	1 - Das sogenannte Böse - von Lorenz, Konrad	04.04.12	4Tage	0,00 €
22.04.12	1	7 - Das Postfix-Buch - von ?	25.02.12	43Tage	1,50 €
					16,00 €

Mit freundlichen Grüßen

(Bibliotheksverwaltung)

Seite 1 / 1 | Standard | STD | Gruppenkopf:A1

Mit Hilfe einer Abfrage lässt sich aus den getätigten Eingaben solch eine Mahnung zum Ausdruck fertig erstellen. Der Nutzer der Datenbank braucht dazu lediglich bei den Berichten den Bericht Mahnung auszuwählen und kann dann einen entsprechenden Mahnbrief an alle die Personen schicken, bei denen im vorher angegebenen Formular eine Mahnung bearbeitet wurde.

In so einem Bericht stehen also gegebenenfalls auf den folgenden Seiten jeweils auf einer Seite weitere Mahnungen für andere Personen. Sollte ein Leser so viele Medien entliehen haben, dass der Platz auf einer Seite nicht ausreicht, so wird die Tabelle auf der Folgeseite einfach fortgesetzt.

Ein so erstellter Bericht ist also umfangreicher als ein Serienbrief, der mit Writer erstellt wird: er stellt automatisch alle Datensätze zusammen, die gedruckt werden sollen und ordnet den zusätzlichen Text entsprechend an.

Ein ähnlicher Brief wie in der obigen Abbildung lässt sich sonst nur unter Zuhilfenahme von Makros realisieren, wie es in «Drucken über Textfelder» beschrieben wird.

Sicherer Umgang mit einer Base-Datei

Tabellen der internen Datenbank, Abfragen, Formulare und Berichte werden in einer Base-Datei gespeichert. Durch diese Vielzahl an Elementen, von denen z.B. die Datenbank auch noch während der Arbeit in den Arbeitsspeicher ausgelagert wird, ist solch eine Datei ein Element, mit dem sorgsam umzugehen ist. Manchmal sind Bugmeldungen zu lesen, die vor diesem Hintergrund deutlich machen, dass eine Datenbankdatei eben einen etwas sorgfältigeren Umgang benötigt als z.B. eine Textdatei, die mit dem Writer geschrieben wird.

Die folgenden Hinweise sollten deshalb beim Umgang mit einer Base-Datei berücksichtigt werden:

- Eine geöffnete Datenbankdatei sollte möglichst nicht über **Speichern unter** mit einem anderen Namen abgespeichert werden. Wenn es schon nicht anders geht, so sollten Tabellen, Abfragen, Formulare und Berichte vorher geschlossen werden. Besser ist es, die Datenbankdatei zu schließen und eine Kopie der Datei zu erstellen.
Der Report-Builder ist ein Addon. Er wird zwar inzwischen nicht mehr als gesonderte Erweiterung sichtbar, arbeitet aber weitgehend unabhängig von der Datenbankdatei. Das Umbenennen der Datenbankdatei über **Speichern unter** entzieht dem Reportbuilder seine Grundlage.
- Die Abspeicherung einer Tabelle, einer Abfrage, eines Formulars oder eines Berichtes hat nicht zur Folge, dass die komplette Datenbankdatei abgespeichert wird. Diese Abspeicherung muss separat erfolgen. Nach wichtigen Schritten bei der Erstellung von Elementen für die *.odb-Datei muss deshalb immer das Element und anschließend die Datenbankdatei gespeichert werden.
Besonders trifft dieses Speicherverhalten auf die Arbeit mit dem Report-Builder zu. Die Erstellung eines Berichtes ist noch die labilste Komponente innerhalb der Base-Datei. Deshalb nach jedem Schritt Bericht und *.odb-Datei sichern. Ist der Bericht erstellt, so funktionieren die Berichte selbst ohne besondere Probleme.
- Eine einmal fertig gestellte *.odb-Datei, die anschließend nur noch mit Daten gefüttert wird, wird nicht wieder über den Speicherbutton abgespeichert. Die Speicherung der Datensätze erfolgt letztlich beim Schließen der *.odb-Datei. Der Inhalt der Datenbank wird erst dann zurück in die Datei geschrieben. Ein Absturz an dieser Stelle kann zu Datenverlust führen. Deshalb sollte eine Strategie entwickelt werden, wie Sicherungskopien rechtzeitig erstellt werden. Im Kapitel «Makros» befindet sich ein Makro, mit dem solch eine Sicherungskopie beim Öffnen einer Datenbankdatei automatisch erstellt wird. Ebenso ist dort eine Möglichkeit aufgeführt, wie auch zwischendurch gesichert werden kann.
Einem deutlich höheren Sicherheitsstandard genügen schließlich externe Serverdatenbanken wie MySQL/MariaDB oder PostgreSQL. Für diese könnte dann Base als Frontend mit Abfragen, Formularen und Berichten dienen.

Eine einfache Datenbank – Testbeispiel im Detail

Die Erstellung einer Datenbank wird in «Neue Datenbank als interne Datenbank» abgehandelt. Dort werden neben der internen HSQLDB oder der internen Firebird-Datenbank verschiedene andere Datenbanksysteme eingebunden.

Das folgende Beispiel basiert auf der Standarddatenbank HSQLDB, die mit LibreOffice als interne Datenbank installiert wird. Es wird zuerst eine eingebettete Datenbank erstellt, die nicht angemeldet wird.

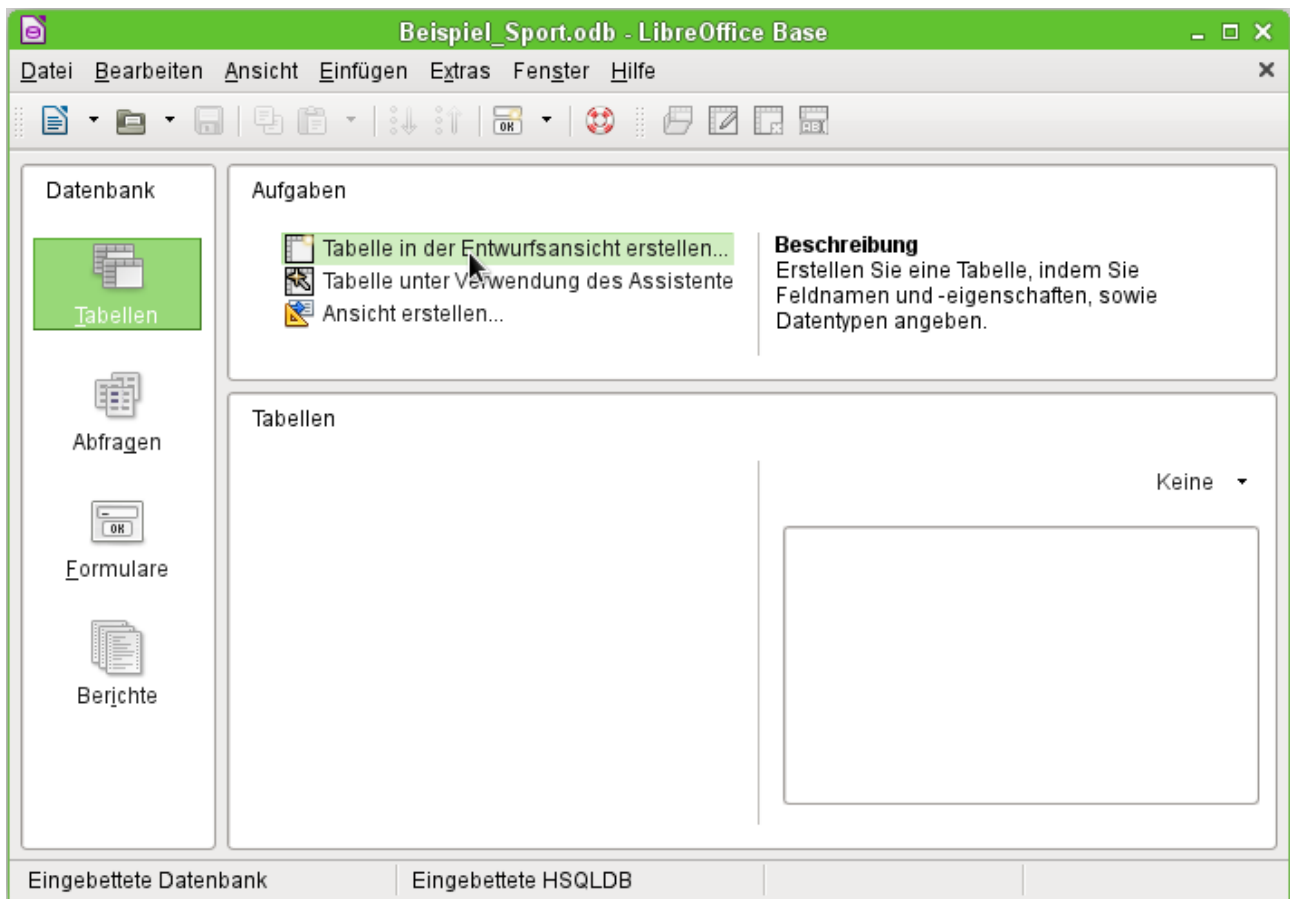
Erstes Kennzeichen bei der Erstellung einer Datenbank ist, dass zum Abschluss des Assistenten zuerst einmal ein Speicherort gesucht wird und die Datei abgespeichert wird.

Die Datenbank soll dazu dienen, einen Sportwettkampf in verschiedenen Disziplinen zu organisieren. Als Bezeichnung wurde deshalb «Beispiel_Sport»² gewählt.

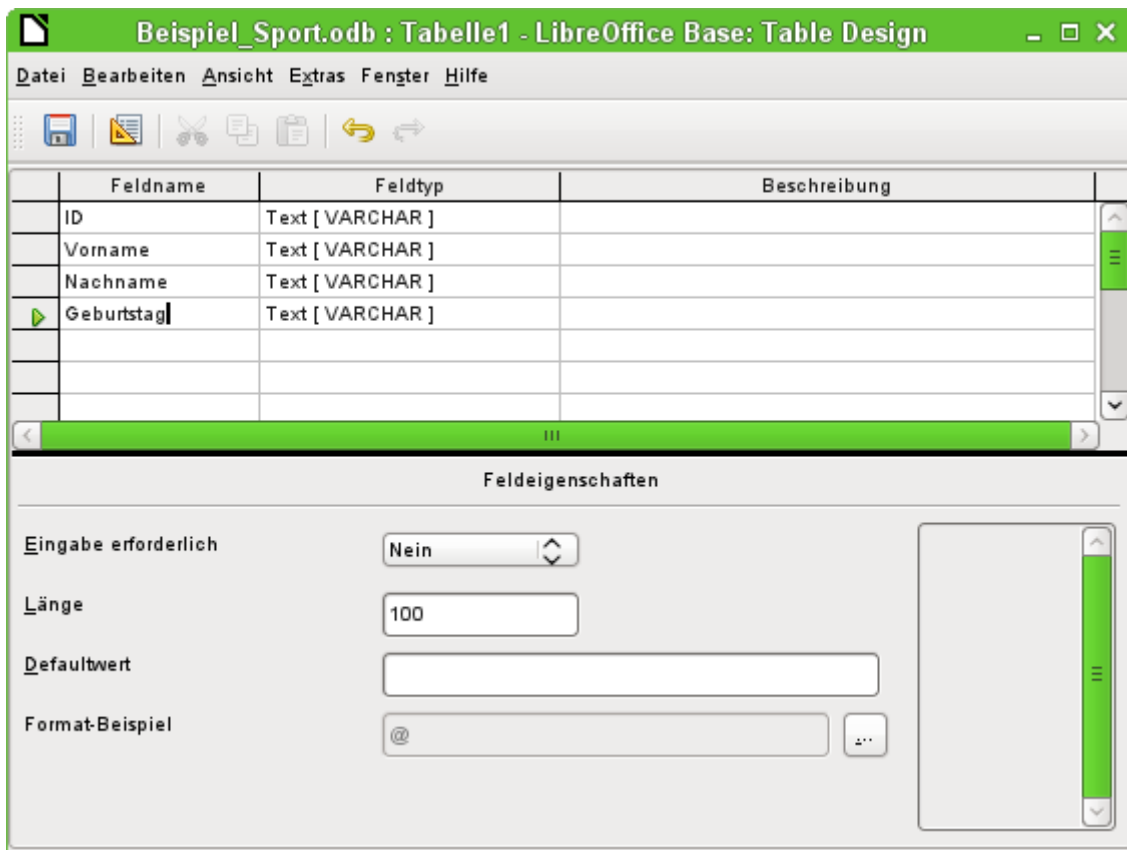
² Die Datenbank «Beispiel_Sport.odb» ist den Beispieldatenbanken für dieses Handbuch beigelegt.

Tabellenerstellung

Sobald die Abspeicherung erfolgt ist, erscheint der Startbildschirm der Benutzeroberfläche für die Datenbank. Standardmäßig sind zuerst auf der linken Seite im Bereich **Datenbank** → **Tabellen** ausgewählt. Die Tabellen sind das zentrale Speicherelement für die Daten. Ohne Tabellen keine Datenbank.



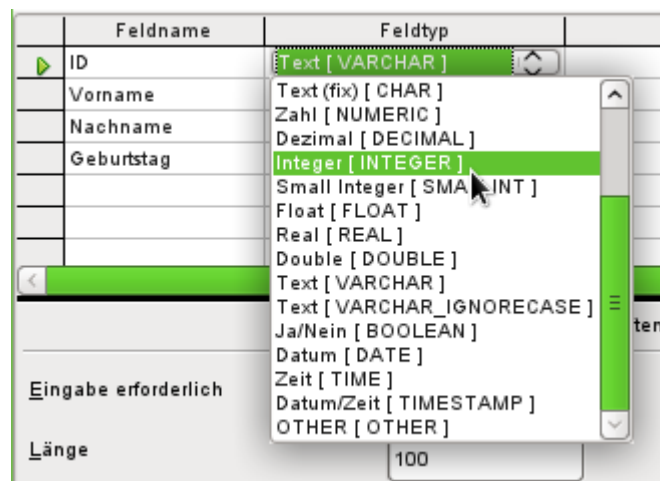
Tabellen → **Aufgaben** → **Tabelle in der Entwurfsansicht erstellen ...** wird angeklickt. Es öffnet sich der folgende Bildschirm:



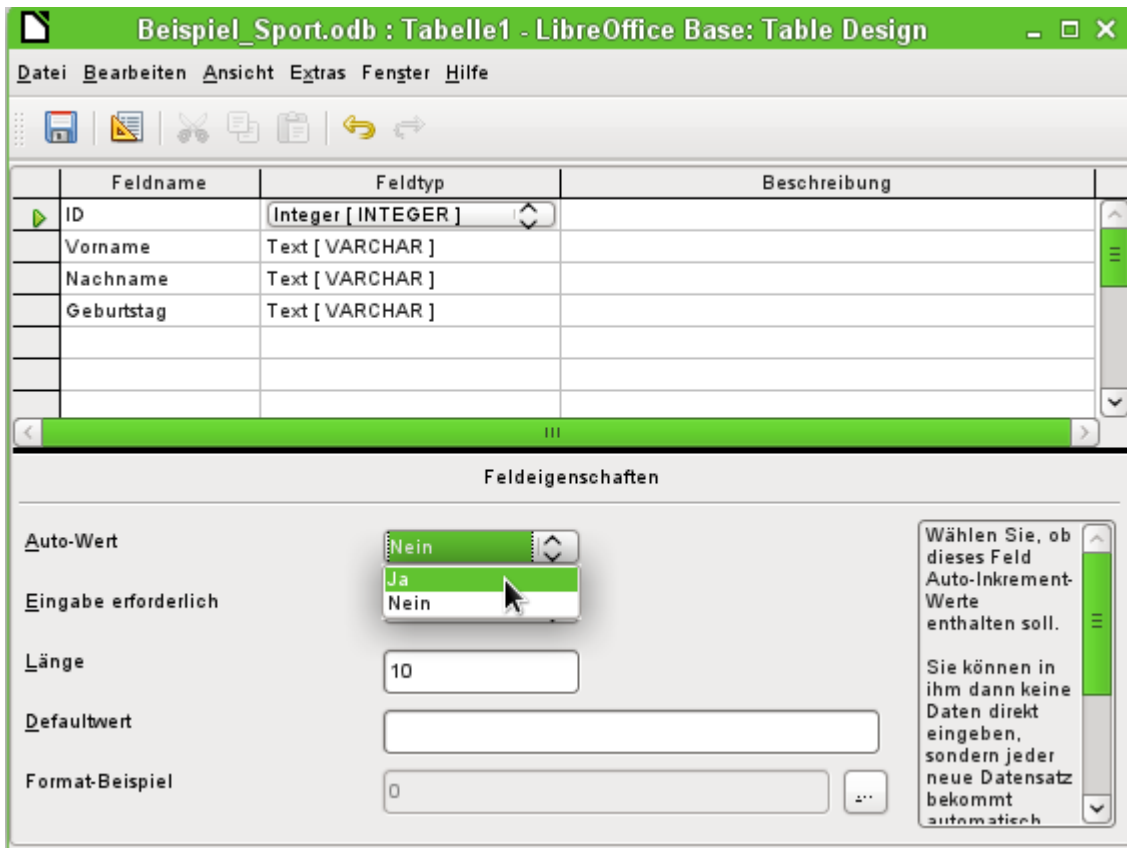
Hier werden zuerst einmal die verschiedenen Feldnamen für die erste Tabelle eingetragen. Die Tabelle soll die Teilnehmer und Teilnehmerinnen enthalten. Sie ist hier erst einmal auf die wesentlichen Bestandteile reduziert.

Die Feldnamen "Vorname", "Nachname" und "Geburtstag" dürften klar sein. Zusätzlich wurde ein Feld mit der Bezeichnung "ID" eingetragen. Dieses Feld soll später einen Wert aufnehmen, der die einzigartige Eigenschaft eines jeden Datensatzes ist. So ein einzigartiges Schlüsselfeld ist für die eingebettete Datenbank notwendig. Ansonsten lassen sich keine Datensätze in die Tabelle eingeben. Dieses Schlüsselfeld wird bei Datenbanken als «Primärschlüssel» bezeichnet.

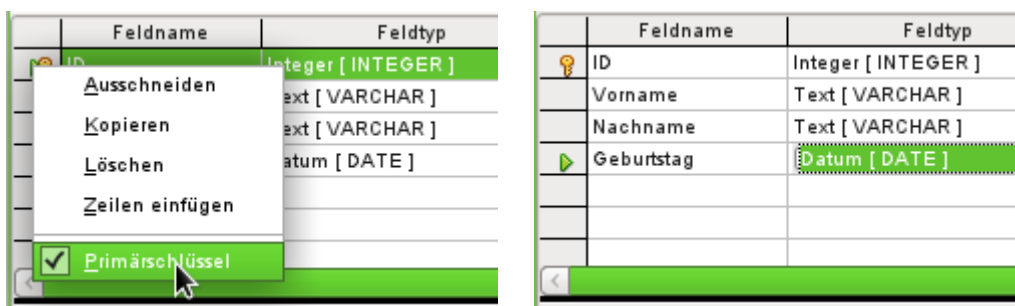
Es könnte natürlich auch ein anderes Feld für diese Eigenschaft genutzt werden. Wird aber z. B. der "Nachname" alleine dafür genutzt, so können nicht zwei Personen mit gleichem Nachnamen abgespeichert werden. In diesem Fall könnte es helfen, zwei Felder zusammen zu einem gemeinsamen Primärschlüssel zu erklären. Eine Garantie, dass das auf Dauer funktioniert, ist das aber nicht. Deshalb wird hier die einfachere Variante bevorzugt.



Im zweiten Arbeitsgang werden jetzt die Feldtypen für die bereits benannten Felder aus der Liste ausgewählt. Das Feld "ID" wird auf den **Feldtyp** «Integer» eingestellt. Es wird dann Ganzzahlen speichern. Dieser Feldtyp hat den Vorteil, dass er bei der eingebetteten HSQLDB automatisch mit der nächst höheren Ganzzahl versehen werden kann.



Die Feldeigenschaft des Feldes "ID" wird bearbeitet. Für dieses Feld wird die automatische Einsetzung von aufsteigenden Zahlenwerten aktiviert: **Feldeigenschaften** → **Auto-Wert** → **Ja**.



Durch die Einstellung des Auto-Wertes sollte beim Verlassen der Feldtypenauswahl auf dem Zeilenkopf ein Schlüsselssymbol zeigen. Das zeigt an, dass dieses Feld den Primärschlüssel der Tabelle bildet. Wird nicht der Auto-Wert gewählt, so kann der Primärschlüssel auch über das Kontextmenü der Maus (**rechte Maustaste** → **Primärschlüssel**) ausgewählt werden.

Für den Geburtstag ist noch der **Feldtyp** entsprechend auf «Datum» einzustellen. Dadurch wird gewährleistet, dass nur gültige Datumseingaben aufgenommen werden und außerdem mit Hilfe das Datums Sortierungen durchgeführt oder z. B. das Alter berechnet werden kann.

Die Tabelle wird nun unter dem Namen "Teilnehmer" abgespeichert. Anschließend können Daten eingegeben werden. Im Feld "ID" ist die Eingabe nicht nötig. Sie wird automatisch beim Abspeichern des Datensatzes erledigt.

Tipp

Bei der intensiveren Nutzung von Base vor allem mit Makros kann es zu Verwechslungen von Tabellenbezeichnungen, Abfragebezeichnungen, Formularbezeichnungen und Berichtsbezeichnungen führen. Um hier eine deutlichere Unterscheidung zu ermöglichen, werden häufig Kürzel vor die Namensbezeichnungen gesetzt:

tblName → Tabellenbezeichnung («tbl» steht für «table»)
qryName → Abfragebezeichnung («qry» steht für «query»)
frmName → Formularbezeichnung («frm» steht für «form»)
rptName → Berichtsbezeichnung («rpt» steht für «report»)

Die für dieses Handbuch erstellten Beispieldatenbanken folgen dieser Namensgebung nicht. Eine entsprechende Benennung wäre bei umfangreichen Datenbanken von Anfang an sinnvoll. Die umfangreichsten Datenbanken dieses Handbuches sind aber bereits entstanden, bevor dieser Vorschlag zur Benennung auftauchte.

Hinweis

Die Datenbankdatei ist ein gepacktes Verzeichnis von einzelnen Dateien. Das Speichern eines einzelnen Elementes wie der Tabelle wird deshalb nicht direkt in die Datenbankdatei selbst geschrieben. Deswegen muss nach der Erstellung von Tabellen, Abfragen, Formularen und Berichten anschließend auch der Speicherbutton für die Datenbankdatei selbst betätigt werden.

Einzig das Abspeichern von eingegebenen Daten funktioniert beim Verlassen der Datenzeile automatisch.

Die Teilnehmer können jetzt eingegeben werden. Folgende Informationen fehlen allerdings auf den ersten Blick:

- Eine Liste der Sportarten, die die Teilnehmer belegen wollen.
- Für Wettkämpfe die Unterscheidung von Teilnehmern und Teilnehmerinnen.

Feldname	Feldtyp
ID	Text [VARCHAR]
Sportart	Text [VARCHAR]

< |||

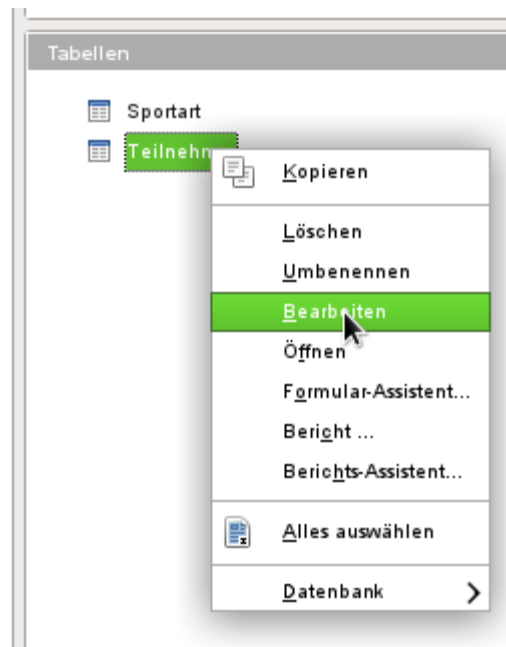
Feldeigenschaften

Länge

Defaultwert

Format-Beispiel

Es wird eine Tabelle "Sportarten" erstellt. Da nicht so sehr viele unterschiedliche Sportarten in Frage kommen, wird hier nicht der automatisch hoch zählende Primärschlüssel gewählt. Stattdessen wird der Feldtyp auf «Text» belassen, aber auf 5 Zeichen begrenzt. Die 5 Zeichen reichen aus, um damit auch passende Kürzel für die Sportarten zu finden.



Die Tabelle "Teilnehmer" soll noch einmal zum Bearbeiten, nicht zur Dateneingabe, geöffnet werden. Dies geschieht über das Kontextmenü der Tabelle.

	Feldname	Feldtyp	
🔑	ID	Integer [INTEGER]	
	Vorname	Text [VARCHAR]	
	Nachname	Text [VARCHAR]	
	Geburtstag	Datum [DATE]	
▶	Geschlecht	Text [VARCHAR]	⬆

Feldeigenschaften

Eingabe erforderlich Nein ⬆

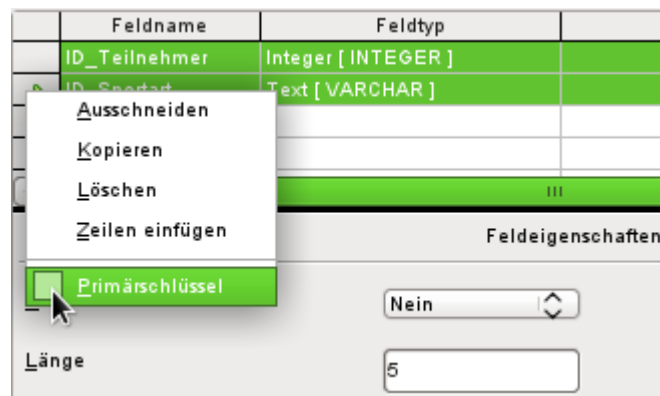
Länge

Defaultwert

Format-Beispiel

Der Tabelle wird das Feld "Geschlecht" hinzugefügt. Ein neues Feld kann mit der grafischen Benutzeroberfläche nur am Ende der Tabelle hinzugefügt werden. Über SQL ist auch die Möglichkeit gegeben, neue Felder an einer bestimmten Position einzufügen.

Die Länge des Textes in diesem Feld beschränkt sich auf einen Buchstaben. Es reicht also 'm' und 'w' als Eingabe.



Irgendwie müssen die beiden Tabellen verbunden werden, so dass jedem Teilnehmer mehrere Sportarten und jeder Sportart mehrere Teilnehmer zugeordnet werden können. Dies geschieht durch eine Tabelle, in der die Werte der beiden Primärschlüssel der Tabellen "Teilnehmer" und "Sportart" abgespeichert werden sollen. Da nicht mehr als die Kombination dieser Felder gespeichert werden soll, sind beide Felder zusammen der Primärschlüssel für diese Tabelle. Um beiden Feldern einen Primärschlüssel zuzuweisen, wird zunächst am Zeilenkopf eine Zeile markiert, danach die Großschreibtaste («Shift») betätigt und die zweite Zeile zusätzlich markiert. Über das Kontextmenü der Maus lässt sich jetzt der Primärschlüssel festlegen.

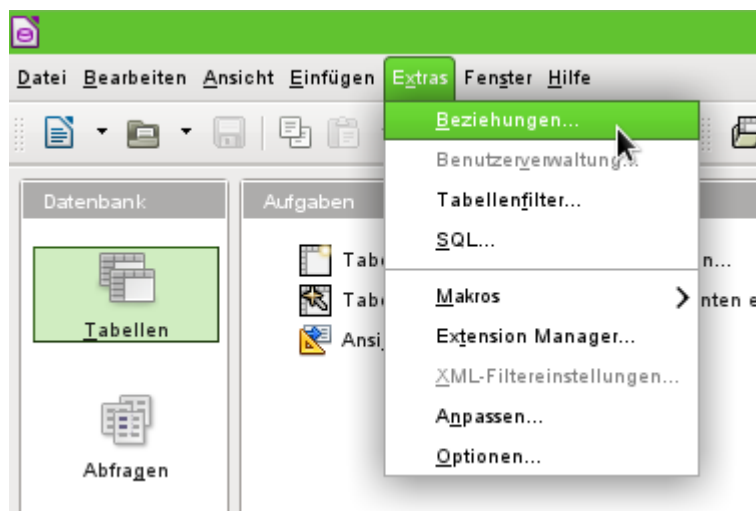
Damit wirklich die entsprechenden Werte aus "Teilnehmer" und "Sportart" aufgenommen werden können, müssen die Felder genau dem Feldtyp entsprechen, den sie speichern sollen. "ID_Teilnehmer" muss also den Feldtyp «Integer» haben. "ID_Sportart" muss den Feldtyp «Text» haben und außerdem, wie das Feld "ID" aus der Tabelle "Sportart", auf 5 Zeichen begrenzt werden.

Die Tabelle wird unter dem Namen "rel_Teilnehmer_Sportart" abgespeichert.

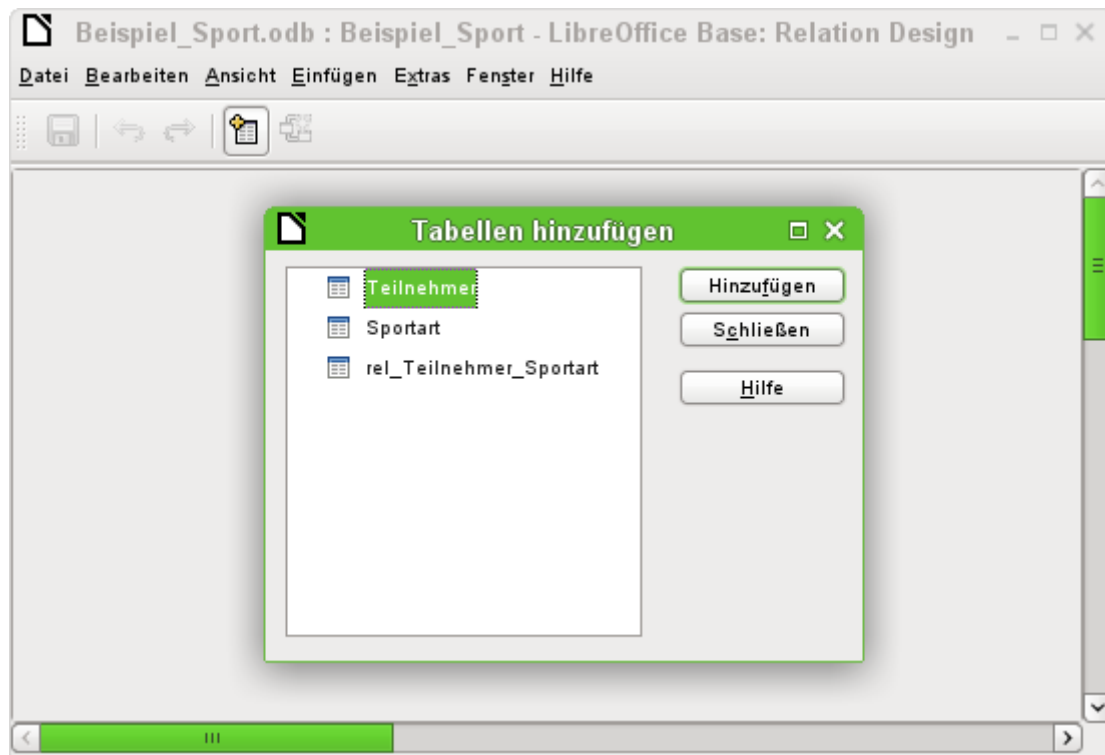
Tipp

In diese Verbindungstabelle würden auch z.B. die Ergebnisse eines Wettkampfes gehören. Sollen allerdings mehrere Wettkämpfe abgehalten werden, so ist zusätzlich ein Wettkampfdatum in den gemeinsamen Primärschlüssel mit aufzunehmen.

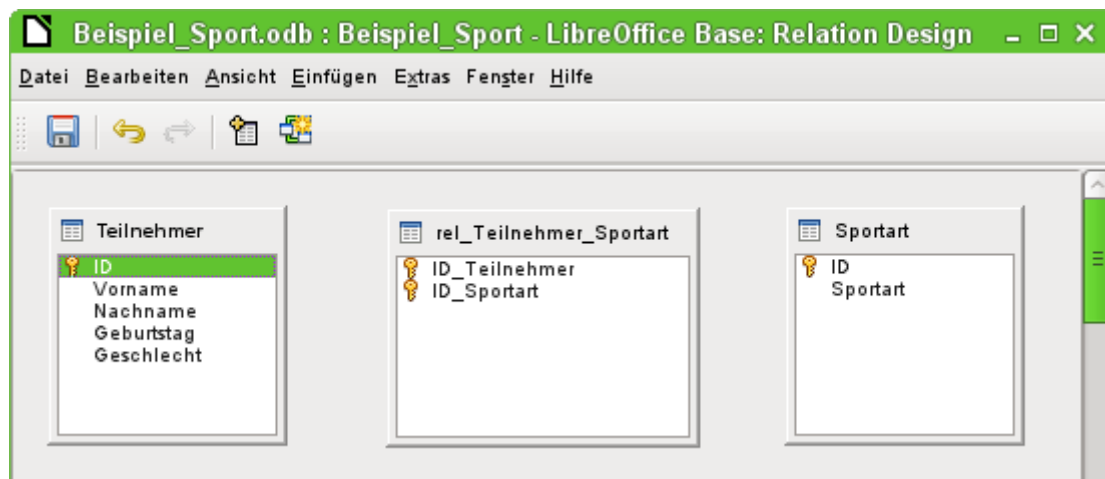
Sind die Tabellen fertiggestellt, so sollte eine entsprechende Verbindung der Tabellen fest definiert werden. So kann verhindert werden, dass in der Tabelle "rel_Teilnehmer_Sportart" z. B. eine Nummer für einen Teilnehmer auftaucht, die gar nicht in der Tabelle "Teilnehmer" verzeichnet ist.



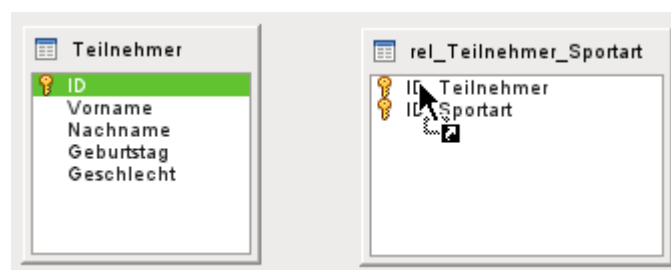
Über **Extras** → **Beziehungen** wird das Fenster für die Beziehungsdefinition geöffnet.



Für die Beziehungsdefinition werden alle bisher erstellten Tabellen notwendig. Die einzelnen Tabellen werden markiert und über *Hinzufügen* in den Beziehungsentwurf aufgenommen. Anschließend wird der Dialog «Tabellen hinzufügen» geschlossen.



Bei den hinzugefügten Tabellen sind alle Felder aufgeführt. Die Primärschlüsselfelder sind zusätzlich mit einem Schlüsselsymbol gekennzeichnet. Die vorgesehenen Rechtecke für die Tabellen können übrigens beliebig verschoben und in der Größe verändert werden.



Das Feld "Teilnehmer"."ID" wird mit der linken Maustaste markiert. Die Maustaste wird gehalten und die Maus auf das Feld "rel_Teilnehmer_Sportart"."ID_Teilnehmer" gezogen. Der Cursor zeigt eine Verknüpfung an. Die Maustaste wird über dem Feld "rel_Teilnehmer_Sportart"."ID_Teilnehmer" los gelassen. Es erscheint der folgende Dialog zur Definition der Beziehung («Relation»).

The screenshot shows the 'Relationen' dialog box. Under 'beteiligte Tabellen', 'rel_Teilnehmer_Sportart' and 'Teilnehmer' are listed. The 'beteiligte Felder' table shows 'ID_Teilnehmer' from 'rel_Teilnehmer_Sportart' linked to 'ID' from 'Teilnehmer'. The 'Update Optionen' section has 'Keine Aktion' selected. The 'Löschoptionen' section has 'Kask. Löschen' selected. Buttons for 'OK', 'Abbrechen', and 'Hilfe' are at the bottom.

rel_Teilnehmer_Sportart	Teilnehmer
ID_Teilnehmer	ID

Update Optionen: ☒ Keine Aktion, ☐ Kask. Update, ☐ Null setzen, ☐ Standard setzen

Löschoptionen: ☐ Keine Aktion, ☒ Kask. Löschen, ☐ Null setzen, ☐ Standard setzen

Buttons: OK, Abbrechen, Hilfe

Das Feld "rel_Teilnehmer_Sportart"."ID_Teilnehmer" soll nicht verändert werden, wenn "Teilnehmer"."ID" verändert würde. Das ist die Standardeinstellung. "Teilnehmer"."ID" wird sowieso nicht verändert, da es sich um ein automatisch hoch zählendes Feld handelt und dort kein Eingriff vorgenommen werden soll.

Der Datensatz der Tabelle "rel_Teilnehmer_Sportart" soll gelöscht werden, wenn "ID_Teilnehmer" gleich "Teilnehmer"."ID" ist und "Teilnehmer"."ID" gelöscht wird. Wenn also aus der Teilnehmer-Tabelle ein Teilnehmer entfernt wird, dann werden alle entsprechenden Datensätze aus der Tabelle "rel_Teilnehmer_Sportart" entfernt. Dieses Vorgehen wird als «kaskadierendes Löschen» bezeichnet.

The screenshot shows the 'Relationen' dialog box. Under 'beteiligte Tabellen', 'rel_Teilnehmer_Sportart' and 'Sportart' are listed. The 'beteiligte Felder' table shows 'ID_Sportart' from 'rel_Teilnehmer_Sportart' linked to 'ID' from 'Sportart'. The 'Update Optionen' section has 'Kask. Update' selected. The 'Löschoptionen' section has 'Kask. Löschen' selected. Buttons for 'OK', 'Abbrechen', and 'Hilfe' are at the bottom.

rel_Teilnehmer_Sportart	Sportart
ID_Sportart	ID

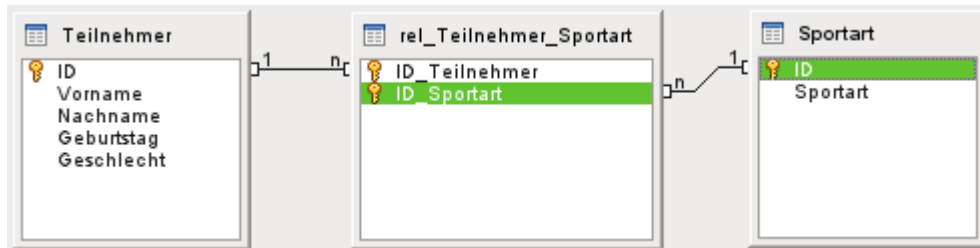
Update Optionen: ☐ Keine Aktion, ☒ Kask. Update, ☐ Null setzen, ☐ Standard setzen

Löschoptionen: ☐ Keine Aktion, ☒ Kask. Löschen, ☐ Null setzen, ☐ Standard setzen

Buttons: OK, Abbrechen, Hilfe

Im nächsten Schritt werden "rel_Teilnehmer_Sportart"."ID_Sportart" und "Sportart"."ID" durch Ziehen der Maus bei gedrückter linker Maustaste verbunden. Auch hier soll ein Datensatz gelöscht werden, wenn die entsprechende Sportart gelöscht wird.

Für die Datenänderung bei "Sportart"."ID" ist allerdings eine andere Variante gewählt worden. Wird "Sportart"."ID" geändert, so wird entsprechend "rel_Teilnehmer_Sportart"."ID_Sportart" angepasst. So kann gegebenenfalls das Kürzel von maximal 5 Zeichen für eine Sportart problemlos angepasst werden, auch wenn bereits viele Datensätze in der Tabelle "rel_Teilnehmer_Sportart" stehen. Dieser Vorgang wird als «kaskadierendes Update» bezeichnet.



Die Felder sind nun fertig verbunden. An den Enden der Verbindungen erscheint jeweils eine «1» und ein «n». Ein Teilnehmer kann mehrmals in der Tabelle "rel_Teilnehmer_Sportart" auftauchen. Eine Sportart kann ebenfalls mehrmals in der Tabelle "rel_Teilnehmer_Sportart" auftauchen. Nur die Kombination von Teilnehmer und Sportart gibt es in der Tabelle nur maximal 1 Mal. Aus zwei 1:n-Beziehungen wird so über die dazwischen gesetzte Tabelle "rel_Teilnehmer_Sportart" eine n:m-Beziehung.

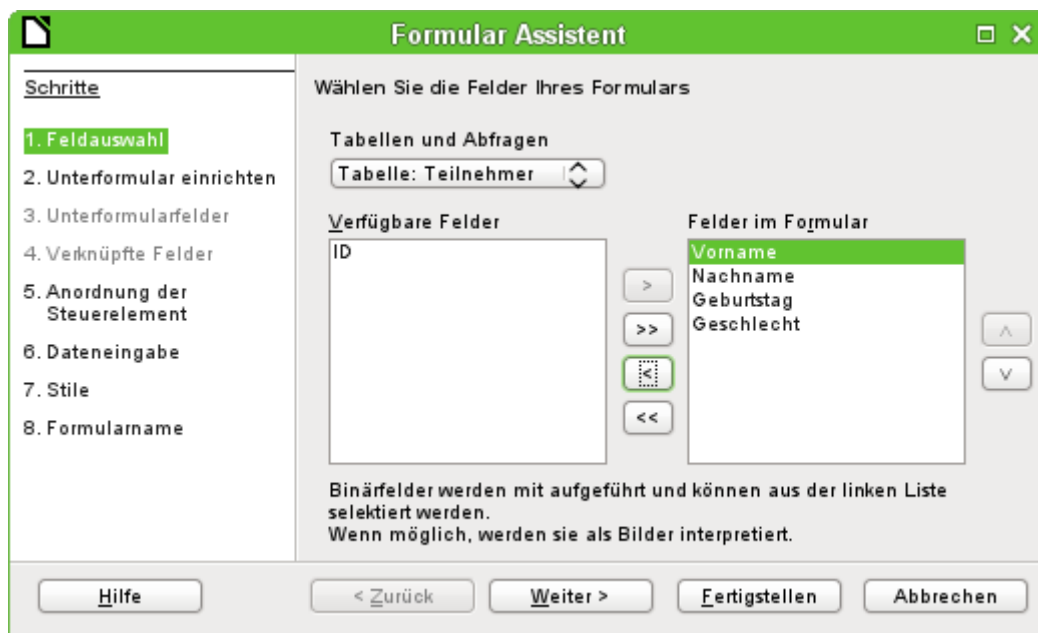
Solch eine Tabellenkonstruktion lässt sich nur schlecht über die Eingabe in Tabellen mit Inhalt füllen. Es müsste alle 3 Tabellen geöffnet sein, wenn einem Teilnehmer eine Sportart zugewiesen werden soll. In der Tabelle "Teilnehmer" muss "Teilnehmer"."ID" gesucht werden und nach "rel_Teilnehmer_Sportart" übertragen werden. In der Tabelle "Sportart" muss "Sportart"."ID" gesucht und entsprechend in "rel_Teilnehmer_Sportart" eingetragen werden. Zu umständlich: Ein Formular löst das eleganter.

Eingabeformular

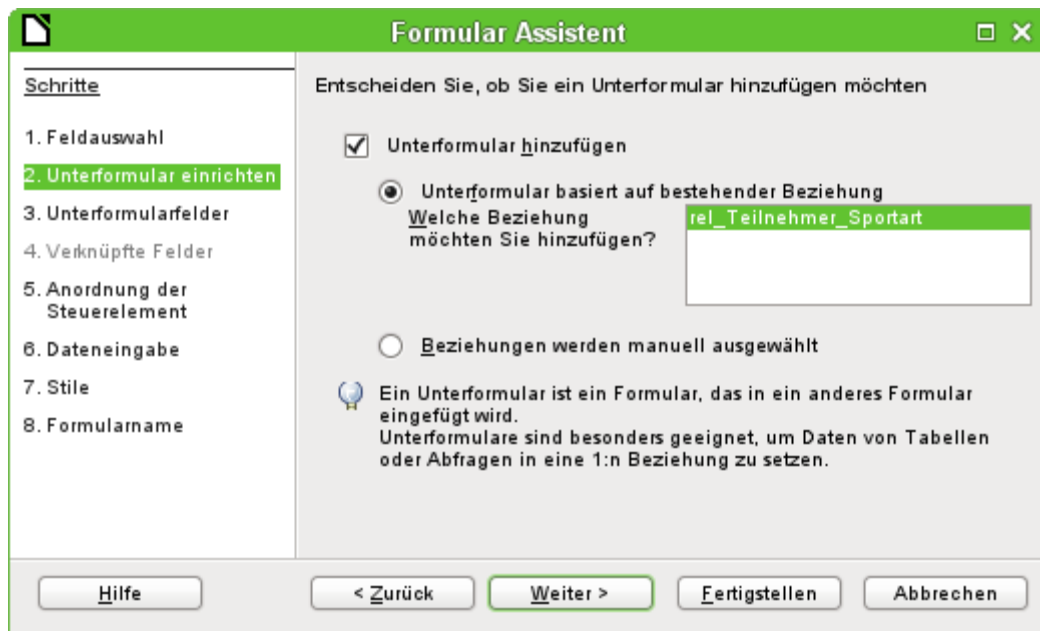
Formulare können direkt in der Entwurfsansicht oder mit Hilfe des Assistenten erstellt werden. Die Erstellung mit dem Assistenten greift dabei auch alten Hasen so gut unter die Arme, dass diese eben schnell den Assistenten durchlaufen lassen und anschließend ein erst einmal vom Grund her erstelltes Formular ihren Wünschen anpassen. Das ist häufig der deutlich Zeit sparendere Weg.



Zuerst wird unter «Datenbank» der Arbeitsbereich für «Formulare» gewählt. Dann unter «Aufgaben» «Formular unter Verwendung des Assistenten erstellen ...».



In der Hauptsache sollen in die Tabelle "Teilnehmer" Daten eingegeben werden. Die Tabelle "Sportarten" wird direkt mit den wenigen notwendigen Sportarten bestückt und nur selten ergänzt. Von der Tabelle "Teilnehmer" werden alle Felder bis auf das Primärschlüsselfeld "ID" benötigt. Das Primärschlüsselfeld wird ja automatisch mit einem entsprechend unverwechselbarem Wert gefüllt. Die Felder im Formular werden über die Pfeiltasten zwischen den beiden Auswahlkästchen bestückt.



Formular Assistent

Schritte

1. Felddauswahl
2. Unterformular einrichten
3. Unterformularfelder
4. Verknüpfte Felder
5. Anordnung der Steuerelement
6. Dateneingabe
7. Stile
8. Formularname

Entscheiden Sie, ob Sie ein Unterformular hinzufügen möchten

☒ Unterformular hinzufügen

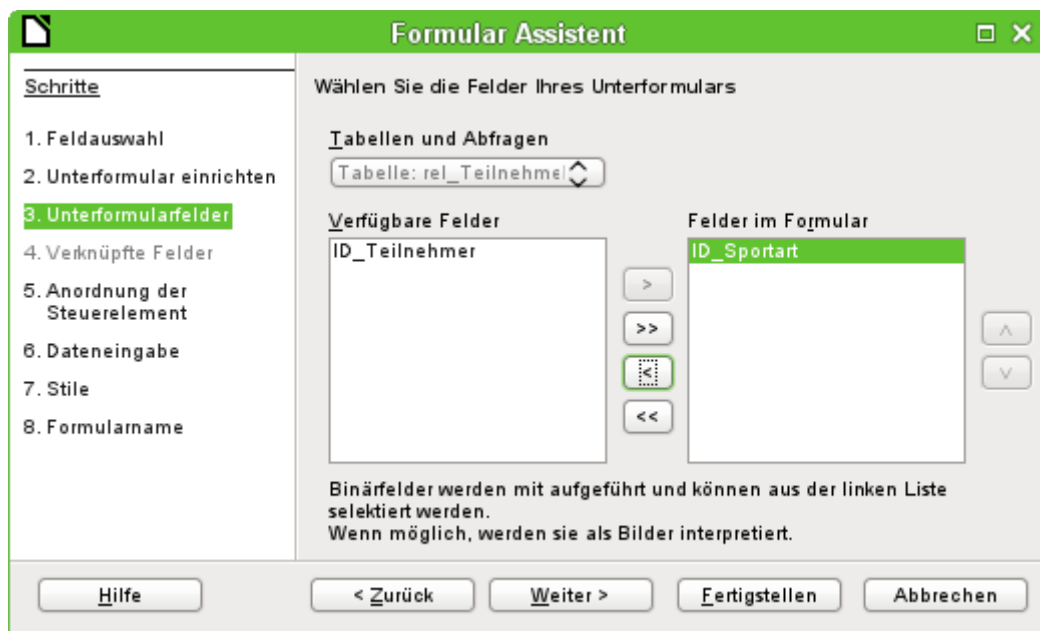
☒ Unterformular basiert auf bestehender Beziehung
 Welche Beziehung möchten Sie hinzufügen? rel_Teilnehmer_Sportart

☐ Beziehungen werden manuell ausgewählt

Ein Unterformular ist ein Formular, das in ein anderes Formular eingefügt wird. Unterformulare sind besonders geeignet, um Daten von Tabellen oder Abfragen in eine 1:n Beziehung zu setzen.

Hilfe < Zurück Weiter > Fertigstellen Abbrechen

Es soll ein Unterformular eingerichtet werden, in dem die Sportarten dem Teilnehmer zugewiesen werden können. Die Beziehung für das Unterformular wird aus der vorher definierten Beziehung ausgelesen. Allerdings muss sie hier noch durch einen Klick auf die Tabelle "rel_Teilnehmer_Sportart" bestätigt werden.



Formular Assistent

Schritte

1. Felddauswahl
2. Unterformular einrichten
3. Unterformularfelder
4. Verknüpfte Felder
5. Anordnung der Steuerelement
6. Dateneingabe
7. Stile
8. Formularname

Wählen Sie die Felder Ihres Unterformulars

Tabellen und Abfragen
 Tabelle: rel_Teilnehmer

Verfügbare Felder
 ID_Teilnehmer

Felder im Formular
 ID_Sportart

Binärfelder werden mit aufgeführt und können aus der linken Liste selektiert werden. Wenn möglich, werden sie als Bilder interpretiert.

Hilfe < Zurück Weiter > Fertigstellen Abbrechen

Aus der Tabelle "rel_Teilnehmer_Sportart" wird nur das Feld "ID_Sportart" benötigt. Das Feld "ID_Teilnehmer" wird über die Verbindung von Hauptformular zu Unterformular mit dem Wert versehen, der in der Teilnehmer-Tabelle für den Primärschlüssel des aktuellen Datensatzes steht.

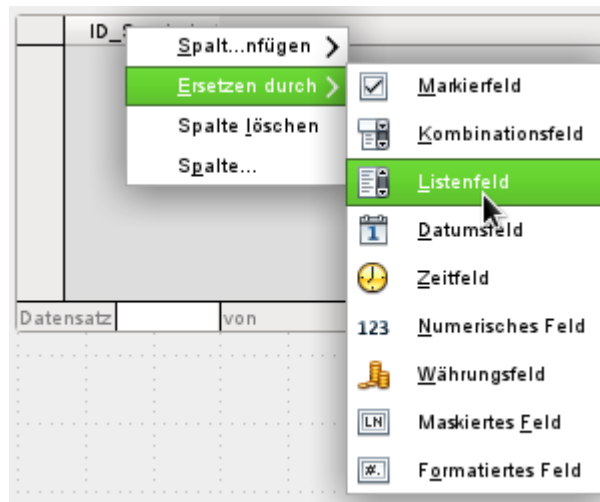
Dass diese Verknüpfung bereits geregelt ist, ist auch an dem 4. Schritt des Assistenten zu erkennen: Der Schritt «Verknüpfte Felder» ist inaktiv.

Wie die Elemente im Hauptformular und wie im Unterformular angeordnet sind, ist letztlich egal. Damit aber die Zuordnung von Daten auch für ungeübte einfach zu erkennen ist, sollte für das Hauptformular in unserem Beispiel nicht das Tabellenkontrollfeld gewählt werden. Im Unterformular soll das Feld später alle Sportarten des Teilnehmers anzeigen. Deswegen an dieser Stelle am besten das Tabellenkontrollfeld belassen.

Schritt 6 sollte so bleiben, wie er vorgesehen ist, also **Dateneingabe** → **Das Formular zeigt existierende Daten an**. Damit ist dann eine Neueingabe ebenso möglich wie eine Veränderung bereits bestehender Daten.

Schritt 7 ist reine Geschmackssache. Nur Vorsicht: manche Stile bergen unverhoffte kontrastarme Darstellungen vor allem in Tabellenkontrollfeldern. Hier muss dann gegebenenfalls die Schriftfarbe der Tabellenkontrollfelder nachjustiert werden.

Mit dem Formular soll noch nicht gearbeitet werden, weil das Unterformular noch die Eingabe des Kürzels für die Sportarten erfordert. Hier wäre eine Auswahl der Sportarten nach dem kompletten Namen wünschenswert. Deswegen: **Das Formular weiter verändern**.



Zuerst wird mit der rechten Maustaste auf den Tabellenkopf des Tabellenkontrollfeldes geklickt. Dort steht «ID_Sportart». Im Kontextmenü wird **Ersetzen durch → Listenfeld** ausgewählt.



Anschließend soll das Listenfeld bearbeitet werden, damit es auch die entsprechenden Daten anzeigen kann. Wieder über das Kontextmenü geht es zu **Spalte...**

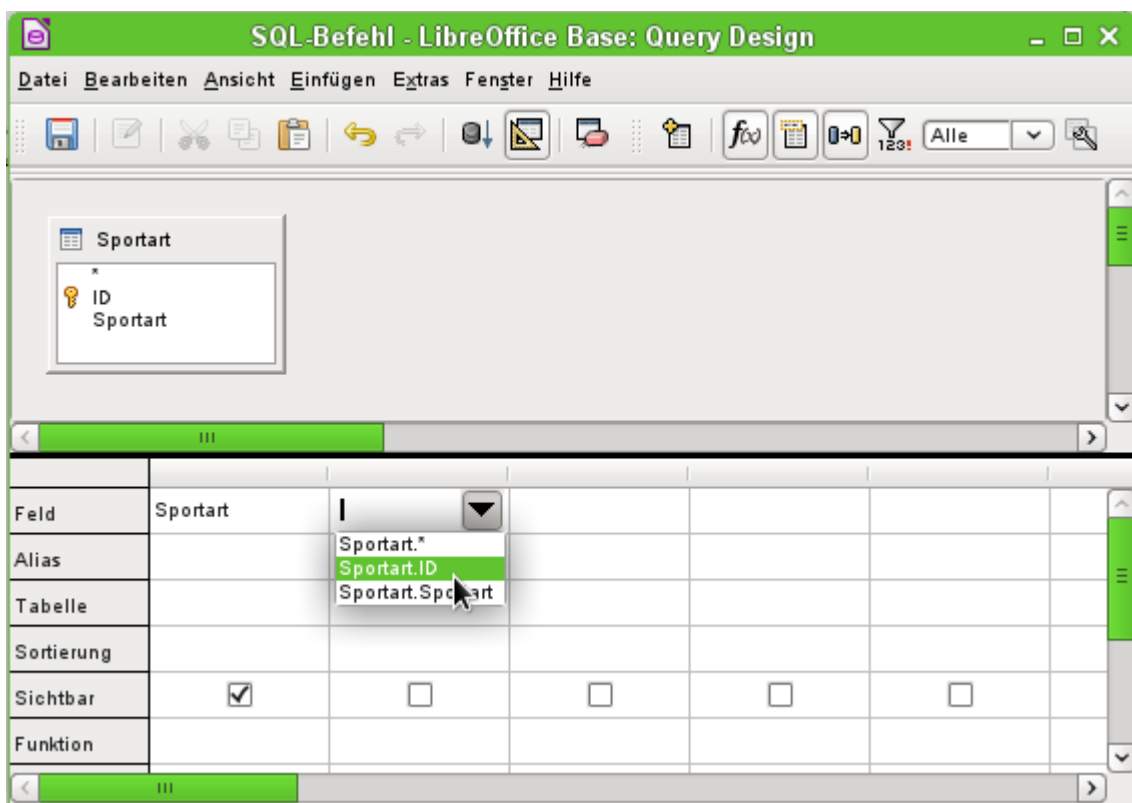


Es öffnen sich die Eigenschaften des ausgewählten Listenfeldes. **Daten → Art des Listeninhaltes → SQL** soll gewählt werden. Mit Hilfe von SQL (StructuredQueryLanguage – Standardabfragesprache für Datenbanken) soll das Feld seinen Inhalt aus der Tabelle "Sportart" erhalten.



Ist **SQL** ausgewählt, so erscheint in der Zeile **Listeninhalt** rechts ein Button mit 3 Punkten Wird dieser Button gedrückt, so öffnet sich der Editor für die Erstellung von Abfragen. Die im Folgenden zu erstellenden Abfrage wird hier nur zusammengestellt und schließlich in dem Listenfeld selbst gespeichert.

Im Dialog «Tabelle oder Abfrage hinzufügen» erfolgt ein Klick auf **Sportart** → **Hinzufügen** → **Schließen**.



Für die erste Spalte wird in dem Abfrage-Editor das Feld "Sportart" ausgewählt. Dieses Feld soll im Listenfeld sichtbar sein.

Die zweite Spalte soll mit dem Feld "ID" bestückt werden. Dieses Feld gibt nachher seinen Wert an die Tabelle weiter, auf der das Unterformular basiert. Es werden also die ausgeschriebenen Worte dargestellt und die passenden Kürzel abgespeichert.

Die Abfrage wird mit dem Speicherbutton in die Eigenschaften des Listenfeldes übertragen. Der Editor wird anschließend geschlossen.

Im Listeninhalt ist jetzt der folgende Text zu lesen:

```
SELECT "Sportart", "ID" FROM "Sportart"
```

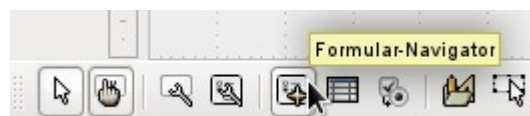
Das ist der SQL-Code, der aus der Betätigung des Editors erwachsen ist. Umgangssprachlich ausgedrückt heißt der Code: Wähle von der Tabelle "Sportart" als erstes die "Sportart" und als zweites den dazugehörigen Schlüsselwert aus.

Diese Abfrage stellt das Minimum dar, das ausgewählt werden sollte. Natürlich könnte auch eine Sortierung eingebaut werden. Mit geschickter Wahl der in "ID" abgespeicherten Kürzel ergibt sich aber bereits eine brauchbare Zusammenstellung der Sportarten. Wird nämlich keine Sortierung vorgegeben, so erfolgt die Sortierung immer nach dem Primärschlüsselfeld. Um später in dem Listenfeld Sportarten zu sehen, müssen diese natürlich zuerst einmal in die Tabelle "Sportart" entsprechend eingegeben werden.



Nun soll noch der Titel geändert werden, der für das Feld "ID_Sportart" eingetragen wurde. Es wird zwar weiterhin "ID_Sportart" abgespeichert. Sichtbar ist aber "Sportart". Deswegen: **Allgemein → Titel → Sportart**.

Soll z.B. das Design bei anderen Feldern angepasst werden, so lassen sich diese am besten über den Formularnavigator ansteuern. Werden Felder direkt angeklickt, so werden nicht nur die Eingabefelder, sondern auch die Beschriftungsfelder markiert. Durch den Assistenten wurden sie zu Gruppen zusammengefasst. Das erfordert dann weitere Aktionen im Kontextmenü der Auswahl.

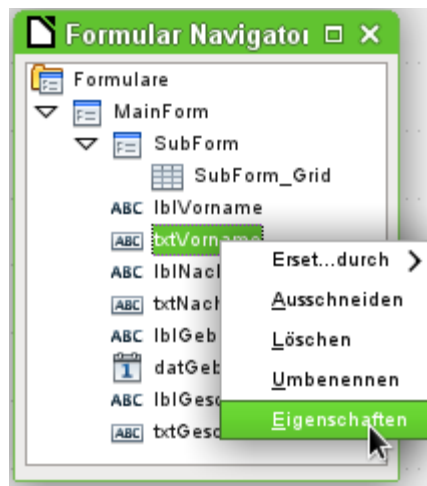


Der Formularnavigator wird über die Symbolleiste «Formular-Entwurf» am unteren Bildschirmrand gestartet.

Hinweis

Manchmal kann es vorkommen, dass die Symbolleisten für die Erstellung von Formularen beim Bearbeiten der Formulare nicht korrekt geöffnet werden. Der Formularnavigator ist dann nicht zu finden.

Die Symbolleisten lassen sich über **Ansicht → Symbolleisten → Formular-Entwurf** sowie **Formular-Steuerelemente** einblenden. Sollten sie danach auch bei der Dateneingabe sichtbar sein, so muss hier entsprechend wieder die Ansicht ausgeschaltet werden.



Über den Formelnavigator kann jedes Feld einzeln aufgesucht werden. Im Kontextmenü sind dann die Eigenschaften des Feldes aufrufbar. Die Eigenschaften werden nach dem Verlassen der Eigenschaften automatisch gespeichert. Es kann auch bei geöffnetem Eigenschaftsdialog von einem Feld zum anderen gesprungen werden. Hierbei wird auch der jeweilige Zwischenstand gespeichert.

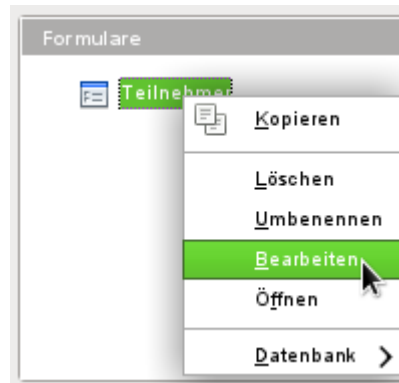
Wenn die Gestaltung nun beendet wurde, kann das Formular gespeichert und geschlossen werden. Auch eine Speicherung der Base-Datei ist jetzt wieder sinnvoll.

Wird jetzt das Formular zur Eingabe von Daten aufgerufen, dann könnte es ungefähr wie folgt aussehen. In das folgende Formular wurde zum Test ein Datensatz eingetragen. Nach der Eingabe des Geschlechtes wurde erst einmal der Button *Speichern* betätigt. Dann konnten auch Sportarten gewählt werden.

Bei der Nutzung des Formulars fallen einige Unannehmlichkeiten auf:

1. Das Unterformular mit den Sportarten ist nicht direkt erreichbar. Wird mit dem Tabulator navigiert, so springt der Tabulator nach der Eingabe des Geschlechtes direkt zum nächsten Teilnehmerdatensatz.
2. Die Navigationsleiste zeigt, wenn sie im Unterformular steht, die Datensatzzahl des Unterformulars an. Sie sollte besser nur durch das Hauptformular navigieren.
3. Das Geschlecht wird zur Zeit auswendig je nach Benutzervorliebe eingegeben. In der Tabelle ist lediglich die Länge des Feldes auf 1 Zeichen begrenzt worden. Hier muss also eine sicherere Eingabe gewährleistet werden.

Zur Lösung dieser Probleme muss das Formular nicht für die Dateneingabe, sondern zum Bearbeiten geöffnet werden:

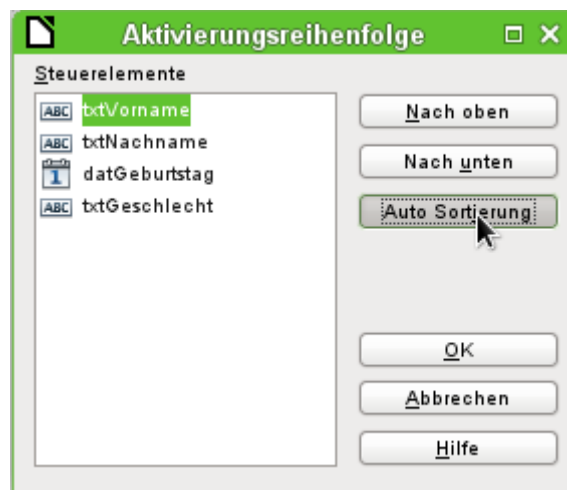


Mit dem Tabulator zum Unterformular

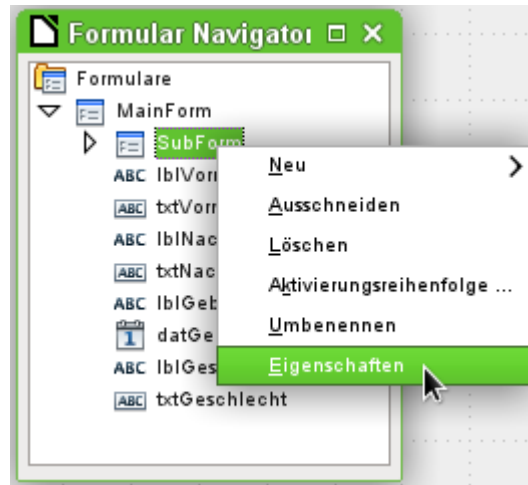
Um mit dem Tabulator nach der Eingabe in das Feld "Geschlecht" nicht gleich weiter zum nächsten Teilnehmer-Datensatz zu rutschen ist einmal die Bearbeitung der Aktivierungsreihenfolge, erreichbar über die Symbolleiste «Formular-Entwurf», notwendig.



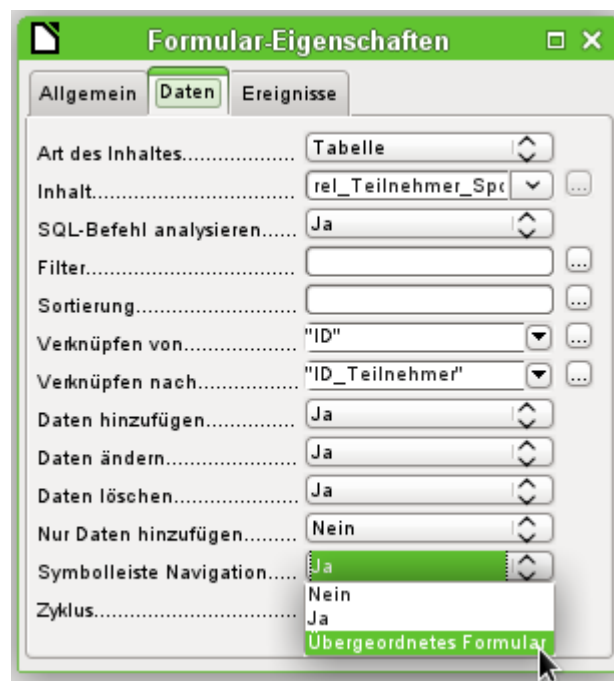
Mit der Auswahl *Auto Sortierung* erfolgt nicht nur die Sortierung der angezeigten Steuerelemente, sondern auch die automatische Weiterleitung in das Unterformular. Das ist zwar aus dem Dialog nicht ersichtlich, wird aber im Hintergrund so geregelt.



Navigationsleiste des Hauptformulars auch im Unterformular aktivieren



Der Formularnavigator wird geöffnet. Die Eigenschaften des Unterformulars werden aufgesucht.



Unter **Daten** → **Symbolleiste Navigation** wird der Wert von «Ja» auf «Übergeordnetes Formular» geändert. Anschließend zeigt die Symbolleiste Navigation immer die Nummer des Datensatzes aus der Tabelle "Teilnehmer" an.

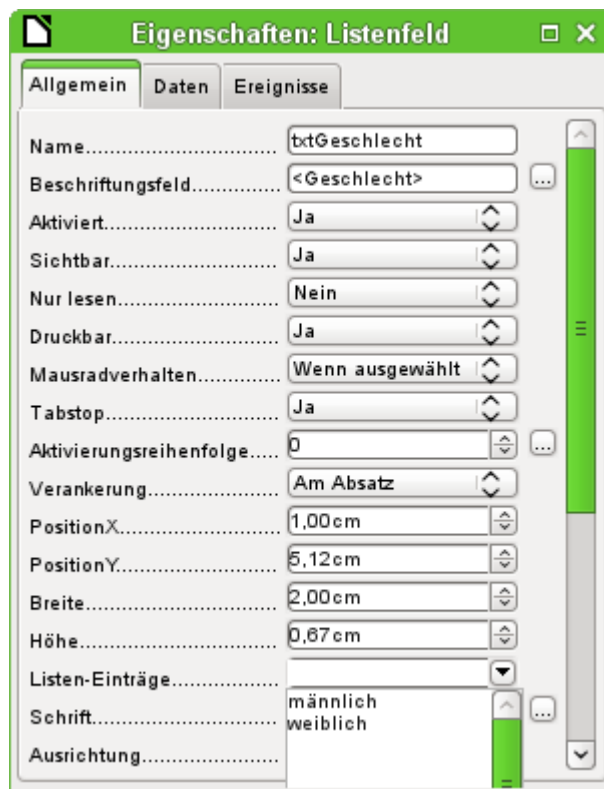
Eingabe eines Kontrollfeldes einschränken

Für die Einschränkung der Eingabe darf das Kontrollfeld kein einfaches Texteingabefeld sein. Im Hauptkapitel Formulare findet sich für die Eingabe des Geschlechtes die Lösung über einen «Gruppierungsrahmen». Hier soll deshalb eine andere Lösung, nämlich noch einmal über ein Listenfeld, vorgestellt werden.

Zuerst wird das Formularfeld für die Eingabe des Geschlechtes über den Navigator markiert. Über das mit der rechten Maustaste erreichbare Kontextmenü wird **Ersetzen durch** → **Listenfeld** ausgewählt. Anschließend wird über das Kontextmenü **Eigenschaften** ausgewählt.



Daten → Art des Listeninhalts bleibt auf dem Wert «Werteliste» stehen. In **Daten → Listeninhalt** werden die Kürzel 'm' und 'w' untereinander (erreichbar durch **Shift + Eingabetaste**) eingegeben. Diese Kürzel sind die Daten, die an die Tabelle "Teilnehmer" weiter gegeben werden.



In **Allgemein → Listen-Einträge** wird das angegeben, was durch das Listenfeld angezeigt werden soll. Das kann auch 'm' und 'w' sein. Es muss auf jeden Fall die gleiche Reihenfolge haben wie die Einträge unter **Daten**.

Jetzt wird noch unter **Allgemein → Aufklappbar** «Ja» gewählt. Hierzu muss in den allgemeinen Eigenschaften recht weit unten gesucht werden.

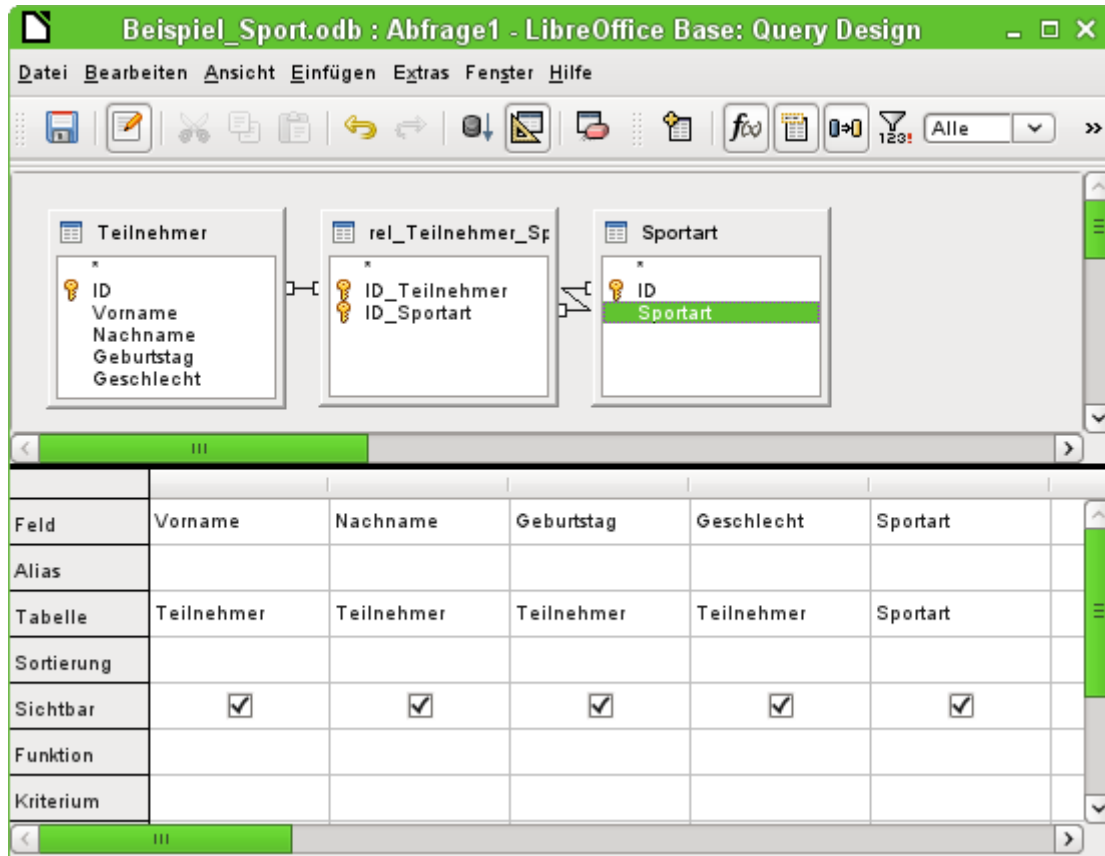
Damit ist das Eingabeformular so weit, dass die auffälligsten Unannehmlichkeiten beseitigt wurden. Das Formular und die Datenbankdatei müssen wieder abgespeichert werden. Die Eingabe von Teilnehmern und Teilnehmerinnen sowie die Zuordnung zu Sportarten kann beginnen.

Damit die folgenden Schritte sinnvoll sind, sollten erst einmal einige Datensätze eingegeben werden. Dabei sollte darauf geachtet werden, dass die Teilnehmer und Teilnehmerinnen vom Alter und von der Sportart auch einmal in Konkurrenz zueinander treten können. Sonst sehen die anschließenden Abfragen und Berichte doch recht nichtssagend aus.

Abfrage

In einer Abfrage können z. B. Inhalte verschiedener Tabellen zusammengefasst werden. Mit dem ersten Zugriff sollen erst einmal alle Teilnehmer mit den Sportarten dargestellt werden, die sie belegt haben.

Der Start dazu geht über **Abfragen → Abfrage in der Entwurfsansicht erstellen ...**. Es erscheint ein Dialog zur Tabellenauswahl, aus dem wir alle Tabellen für die Abfrage auswählen. Am übersichtlichsten ist es, wenn dabei die Tabelle "rel_Teilnehmer_Sportarten" als zweite Tabelle ausgewählt wird. Dann sind auch gleich die Verbindungen gut zu erkennen.



Die Felder, die in der Abfrage dargestellt werden sollen, können entweder durch einen Doppelklick auf die Feldnamen in den Tabellen oder durch Auswahl in der Zeile **Feld** eingestellt werden. Bei der Auswahl in der Zeile werden die Felder sämtlicher aufgeführter Tabellen aufgelistet. Um die Felder den Tabellen richtig zuordnen zu können, werden sie in Abfragen nach "Tabellenname"."Feldname" bezeichnet. Wird statt des Feldnamen ein «*» eingesetzt, so bedeutet dies, dass alle Felder der entsprechenden Tabelle angezeigt werden sollen.



Eine Abfrage sollte grundsätzlich vor dem Abspeichern auch einmal ausgeführt werden um zu sehen, ob sie denn wirklich das erhoffte Ergebnis liefert. Hierzu entweder direkt den oben gezeigten Button anklicken. Es kann auch **F5** gedrückt oder der Weg über **Bearbeiten → Abfrage ausführen** gewählt werden.

	Vorname	Nachname	Geburts-tag	Geschlecht	Sportart
▶	Karl	Müller	17.03.85	m	Kurzstecken
	Karl	Müller	17.03.85	m	Hochsprung
	Karl	Müller	17.03.85	m	Kugelstoßer
	Mia	Keller	07.04.78	w	Kurzstecken
	Mia	Keller	07.04.78	w	Weitsprung
	Mia	Keller	07.04.78	w	Kugelstoßer
	Dirk	Anders	28.09.87	m	Kurzstecken
	Dirk	Anders	28.09.87	m	Langstrecke
	Dirk	Anders	28.09.87	m	Weitsprung
Datensatz	1	von 26			

Die Abfrage zeigt jetzt alle Kombinationen der Teilnehmer und Sportarten an. Hat ein Teilnehmer mehrere Sportarten belegt, so werden mehrere Datensätze angezeigt. Teilnehmer ohne Sportarten erscheinen nicht.

Um bei einem Wettkampf unterschiedliche Altersgruppen zusammen zu stellen, ist der Geburts-jahrgang entscheidend. Es kommt darauf an, wie alt eine Person in dem jeweiligen Jahr wird.

Der Geburtsjahrgang kann mit unterschiedlichen Funktionen ermittelt werden. Die folgende Ein-gabe zeigt die Variante, die für Personen mit Englischkenntnissen vielleicht auch so erschließbar ist:

Feld	Vorname	Nachname	Geburts-tag	Geschlecht	Sportart	YEAR(NOW()) - YEAR("Geburts-tag")
Alias						
Tabelle	Teilnehmer	Teilnehmer	Teilnehmer	Teilnehmer	Sportart	

`YEAR( NOW( ) ) - YEAR( "Geburts-tag" )`

NOW() steht für «Jetzt», also den aktuellen Zeitpunkt. Mit **YEAR(NOW())** wird also das aktuelle Jahr ausgelesen. **YEAR("Geburts-tag")** liest das Jahr aus dem Geburts-tagsdatum aus. Aus bei-den wird hier eine Differenz gebildet, die angibt, wie alt diese Person in dem aktuellen Jahr wird oder bereits geworden ist. Für die **FIREBIRD**-Datenbank müsste eine andere Funktion gewählt wer-den. Über **EXTRACT(YEAR FROM CURRENT_DATE)** wird das aktuelle Jahr bestimmt. Entspre-chend wird auch über **EXTRACT(YEAR FROM "Geburts-tag")** das Jahr des Geburts-tags ermit-telt.

Diese und viele andere Funktionen, die mit der eingebauten **HSQldb** oder der **FIREBIRD**-Datenbank zusammen arbeiten, sind im Anhang des Handbuches beschrieben.

	Vorname	Nachname	Geburts-tag	Geschlecht	Sportart	YEAR(NOW()) - YEAR("Geburts-tag")
▶	Karl	Müller	17.03.85	m	Kurzstecken	29
	Karl	Müller	17.03.85	m	Hochsprung	29
	Karl	Müller	17.03.85	m	Kugelstoßer	29
	Mia	Keller	07.04.78	w	Kurzstecken	36
	Mia	Keller	07.04.78	w	Weitsprung	36
	Mia	Keller	07.04.78	w	Kugelstoßer	36
	Dirk	Anders	28.09.87	m	Kurzstecken	27
	Dirk	Anders	28.09.87	m	Langstrecke	27
	Dirk	Anders	28.09.87	m	Weitsprung	27
Datensatz	1	von 26				

Wird die Abfrage ausgeführt, so erscheint die entsprechende Berechnung für das Jahr 2014, da diese Abfrage eben in dem entsprechenden Jahr ausgeführt wurde.

Etwas störend dürfte dabei sein, dass der gesamte Code, den wir als Feld eingegeben haben, eben auch in der Spaltenüberschrift erscheint. Dem wird abgeholfen, indem für den Code ein alter-nativer Name eingegeben wird, unter dem das Ergebnis erscheinen soll.

Feld	YEAR(NOW()) - YEAR("Geburtstag")
Alias	Sportalter
Tabelle	

In der Zeile «Alias» wird die Bezeichnung «Sportalter» eingegeben. Hier sollte nicht «Alter» stehen, weil die Person vielleicht erst im Laufe des Jahres dieses Alter erreicht. Im Kapitel «Aktuelles Alter ermitteln» steht beschreiben, wie das tatsächliche Alter ermittelt werden kann.

Sportart	Sportalter
Kurzstecken	29
Hochsprung	29
Kugelstoßer	29
Kurzstecken	36
Weitsprung	36

Wird die Abfrage erneut ausgeführt, so erscheint in dem Spaltenkopf nicht mehr die Formel sondern die Bezeichnung «Sportalter».

Damit der Code für den Anfang nicht zu verwirrend wird, sollte die Abfrage unter dem Namen "Sportalter" abgespeichert werden. Diese Abfrage dient dann als Grundlage für die nächste Abfrage, die das "Sportalter" jetzt einer "Altersklasse" zuordnet.

	Vorname	Nachname	Geburtstag	Geschlecht	Sportart	Sportalter
▶	Karl	Müller	17.03.85	m	Kurzstecken	29
	Karl	Müller	17.03.85	m	Hochsprung	29
	Karl	Müller	17.03.85	m	Kugelstoßer	29
	Mia	Keller	07.04.78	w	Kurzstecken	36
	Mia	Keller	07.04.78	w	Weitsprung	36
	Mia	Keller	07.04.78	w	Kugelstoßer	36
	Dirk	Anders	28.09.87	m	Kurzstecken	27
Datensatz	1	von 26				
	Sportalter					
	* Vorname Nachname Geburtstag Geschlecht Sportart Sportalter					
Feld	Sportalter.*					
Alias						
Tabelle	Sportalter					
Sortierung						
Sichtbar	☒	☐	☐	☐	☐	☐
Funktion						

Die Abfrage "Sportalter" wurde als Grundlage für die 2. Abfrage ausgesucht. Vor der Bezeichnung «Sportalter» im Tabellenarbeitsbereich ist ein anderes Symbol zu sehen als bei den Tabellen der ersten Abfrage. Dieses Symbol zeigt an, dass die Grundlage dieser Abfrage eben eine Abfrage und keine Tabelle ist.

Durch Doppelklick auf «*» oder Auswahl von «Sportalter.*» werden alle Felder ausgewählt. Die gesamte Abfrage von vorher wird also übernommen. Nur die Formel ist nicht mehr zu sehen.

Auf das Feld "Sportalter" soll nun zugegriffen werden. Das Sportalter bestimmt ja, in welcher Altersklasse die jeweiligen Personen starten.

Damit die Ermittlung nicht allzu kompliziert wird soll hier folgende Zuordnung nachvollzogen werden: Unter 20 Jahren werden die Teilnehmer immer in zwei Jahrgängen, beginnend mit 0, zusammen gefasst. Ab 20 Jahren werden Gruppen für jeweils 10 Jahre gebildet, also von 20-30 Jahren usw.

	Vorname	Nachname	Geburtsdag	Geschlecht	Sportart	Sportalter	Altersklasse
	Karl	Müller	17.03.85	m	Kurzstecken	29	20
	Karl	Müller	17.03.85	m	Hochsprung	29	20
	Karl	Müller	17.03.85	m	Kugelstoßer	29	20
	Mia	Keller	07.04.78	w	Kurzstecken	36	30
	Mia	Keller	07.04.78	w	Weitsprung	36	30
	Mia	Keller	07.04.78	w	Kugelstoßer	36	30
	Diik	Anders	28.09.87	m	Kurzstecken	27	20

Datensatz	1	von 26					
-----------	---	--------	--	--	--	--	--

Sportalter * Vorname Nachname Geburtsdag Geschlecht Sportart Sportalter	
---	--

Feld	Sportalter.*	CASEWHEN("Sportalter" > 19, CEILING("Sportalter" / 10) * 10, "Sportalter" - MOD("Sportalter", 2))
Alias		Altersklasse
Tabelle	Sportalter	
Sortierung		
Sichtbar	☒	☒

Solche Formeln gehören eigentlich längst nicht mehr zum Einstiegs-Repertoire. Eine einfachere Zuteilung der Altersklassen wäre sogar mit Hilfe des folgenden Berichtes möglich. Für differenziertere Zuteilung sei erneut auf den Anhang des Handbuches verwiesen.

```
CASEWHEN( "Sportalter" > 19, CEILING( "Sportalter" / 10 ) * 10, "Sportalter" - MOD( "Sportalter", 2 ) )
```

CASEWHEN(Überprüfung, korrekt: diesen Wert nehmen, nicht korrekt: den anderen Wert nehmen).

In die deutsche Sprache übertragen bedeutet das so viel wie:

Für den Fall, dass das Sportalter über 19 Jahren ist, dividiere das Sportalter durch 10 und gib nur den Ganzzahlwert wieder. Diesen Ganzzahlwert multipliziere wieder mit 10.

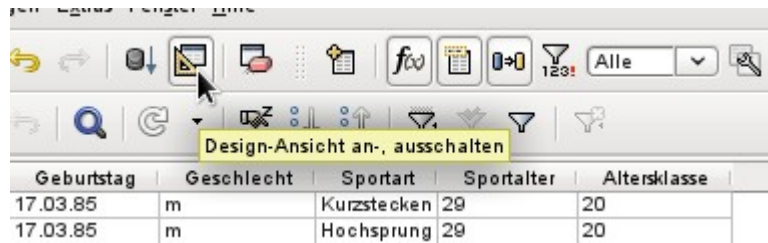
Ist das Sportalter bis einschließlich 19 Jahren, so nehme den Wert des Sportalters und subtrahiere davon den Rest, der sich ergibt, wenn das Sportalter durch 2 dividiert wird. Das ergibt dann z. B. für 19: $19/2 = 18$, Rest 1 $\rightarrow 19 - 1 = 18$.

Alle Teilnehmer über 19 Jahren werden damit einer Gruppe für das jeweilige Jahrzehnt zugewiesen. Alle Teilnehmer darunter werden einer Gruppe zugewiesen, die der jeweiligen niedrigeren geraden Zahl entspricht.

Dieser Formel wurde wieder ein Alias zugewiesen, in diesem Falle der Alias "Altersklasse".

Auch diese Formel müsste für FIREBIRD geändert werden. Statt **CASEWHEN** müsste hier **IIF** stehen.

Die Abfrage kann jetzt unter dem Namen "Meldung" gespeichert werden.



Das Ausschalten der Design-Ansicht ist hier eigentlich nicht notwendig, da alle Eingaben in der Design-Ansicht ohne größere Probleme möglich sind. Den Code von Abfragen einmal genauer anzusehen, kann allerdings nie schaden. Schließlich gibt es auch SQL-Formulierungen, die nur schlecht in die Design-Ansicht passen oder die dort gar nicht möglich sind. Dafür wird dann die direkte Eingabe im SQL-Code genutzt.

	Vorname	Nachname	Geburts-tag	Geschlecht	Sportart	Sportalter	Altersklasse
▶	Karl	Müller	17.03.85	m	Kurzstecken	29	20
	Karl	Müller	17.03.85	m	Hochsprung	29	20
	Karl	Müller	17.03.85	m	Kugelstoßer	29	20
	Mia	Keller	07.04.78	w	Kurzstecken	36	30
	Mia	Keller	07.04.78	w	Weitsprung	36	30
	Mia	Keller	07.04.78	w	Kugelstoßer	36	30
	Dirk	Anders	28.09.87	m	Kurzstecken	27	20
Datensatz	1	von 26					

```

SELECT "Sportalter".*, CASEWHEN( "Sportalter" > 19, CEILING( "Sportalter" / 10 ) * 10,
"Sportalter" - MOD( "Sportalter", 2 ) ) AS "Altersklasse" FROM "Sportalter"

```

Im Code sind Felder und Tabellen in doppelte Anführungszeichen gesetzt und in einem ockerfarbigen Farbton dargestellt. Begriffe des SQL-Codes sind in blau gehalten, Funktionen der Datenbank in grün. Die Bezeichnung **AS** erscheint in neueren Versionen von LO nicht mehr.

```

SELECT
  "Sportalter".*,
  CASEWHEN( "Sportalter" > 19, CEILING( "Sportalter" / 10 ) * 10,
    "Sportalter" - MOD( "Sportalter", 2 ) )
  AS "Altersklasse"
FROM "Sportalter"

```

Der Teil mit der Formel wurde ja schon erwähnt. Hier jetzt der Rahmen.

```
SELECT "Sportalter".*, ... AS "Altersklasse" FROM "Sportalter"
```

Wähle aus der Tabelle "Sportalter" alle Datensätze aus und zusätzlich das, was durch die Formel bestimmt wird. Das, was durch die Formel bestimmt wird, bezeichne als "Altersklasse".

Der Code unterscheidet nicht zwischen Tabellen und Abfragen als Grundlage der Daten. Er funktioniert deshalb nur in der grafischen Benutzeroberfläche von Base. Die Benennung von Abfragen muss dafür immer anders sein als die Benennung von Tabellen.

Sobald in einer Abfrage bei ausgeschalteter Design-Ansicht der Button **SQL** (für **SQL-Kommando direkt ausführen**) gedrückt wird, wird die Datenbank mit einer Fehlermeldung reagieren. Die eingebettete Datenbank kennt die Abfrage "Sportalter" nicht, auf die sich die aktuellen Abfrage "Meldung" bezieht.

Ausdruck im Bericht

Über **Berichte → Bericht in der Entwurfsansicht erstellen ...** wird ein Bericht erstellt, der eine übersichtliche Teilnehmerliste, sortiert nach Sportart, Geschlecht und Altersklasse darstellen soll.

Wird der Editor gestartet, so ist zuerst der Dialog «Feld hinzufügen» im Vordergrund. Für diesen Dialog wird die von der alphabetischen Sortierung her erste Tabelle als Basis genommen. Es muss für den Bericht aber die Abfrage als Datenquelle herausgesucht werden.

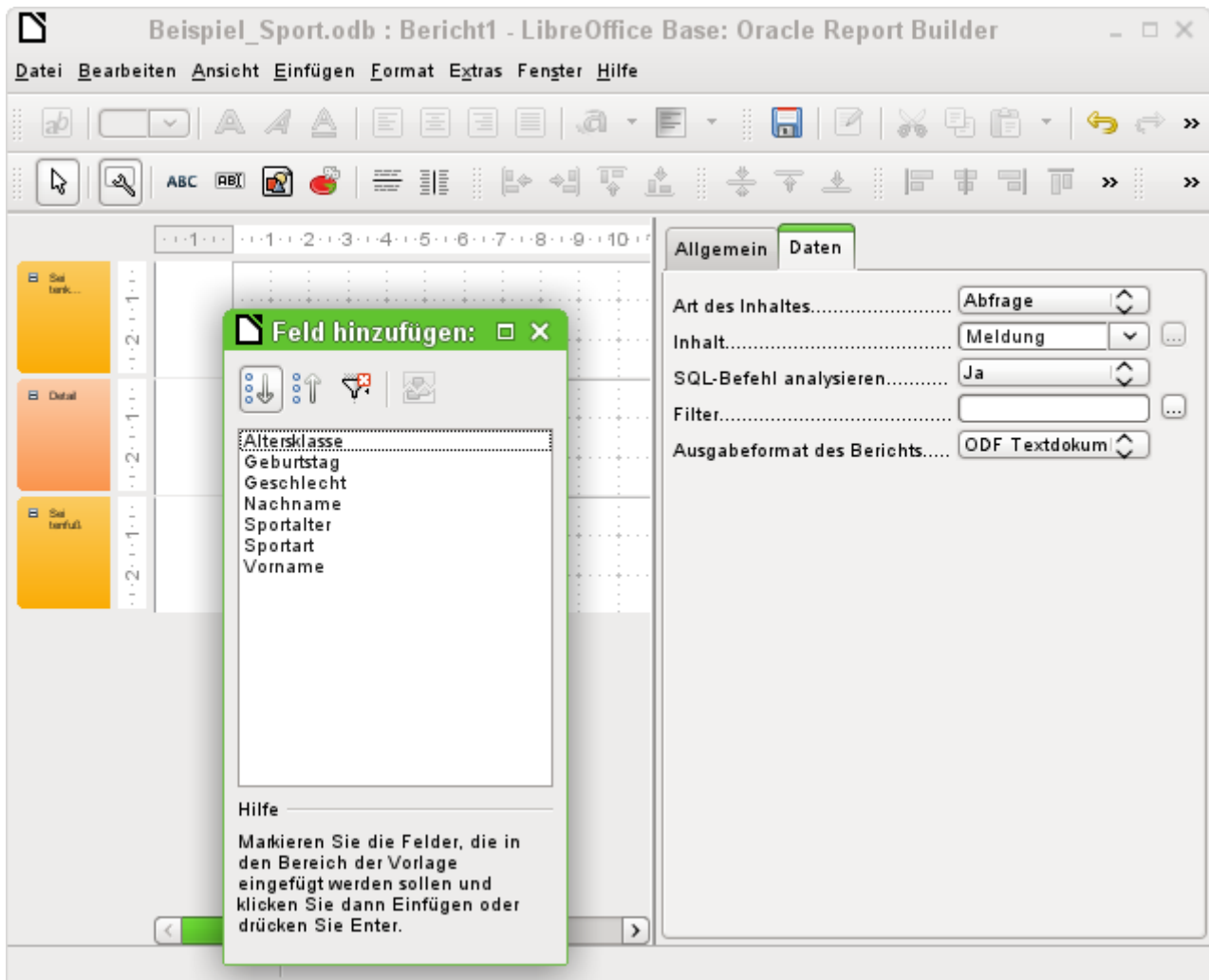
Im Berichtsfenster befindet sich auf der rechten Seite die Übersicht über die Eigenschaften des gerade aktiven Objektes. Sollte die Leiste nicht sichtbar sein, so muss sie über **Ansicht → Properties** ausgewählt werden.

Hinweis

Leider sind beim Report-Builder bereits auf der grafischen Benutzeroberfläche in den neueren Ausgaben von LibreOffice einige Elemente nicht mehr übersetzt. An dieser Stelle ist wohl zu viel an Code bereinigt worden.

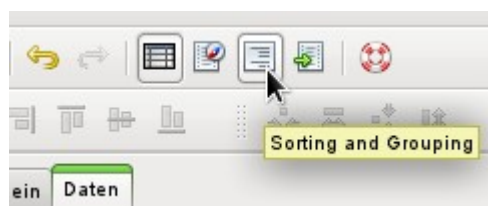
Formeln, wie weiter unten erwähnt, waren im Report-Builder allerdings nie übersetzt, sondern immer in der englischsprachigen Originalfassung zu erstellen.

Aus den Eigenschaften wird im Reiter **Daten** → **Art des Inhaltes** → **Abfrage** ausgewählt. Als **Inhalt** wird dann die Abfrage "Meldung" gewählt, die als letzte zusammenfassende Abfrage erstellt wurde.

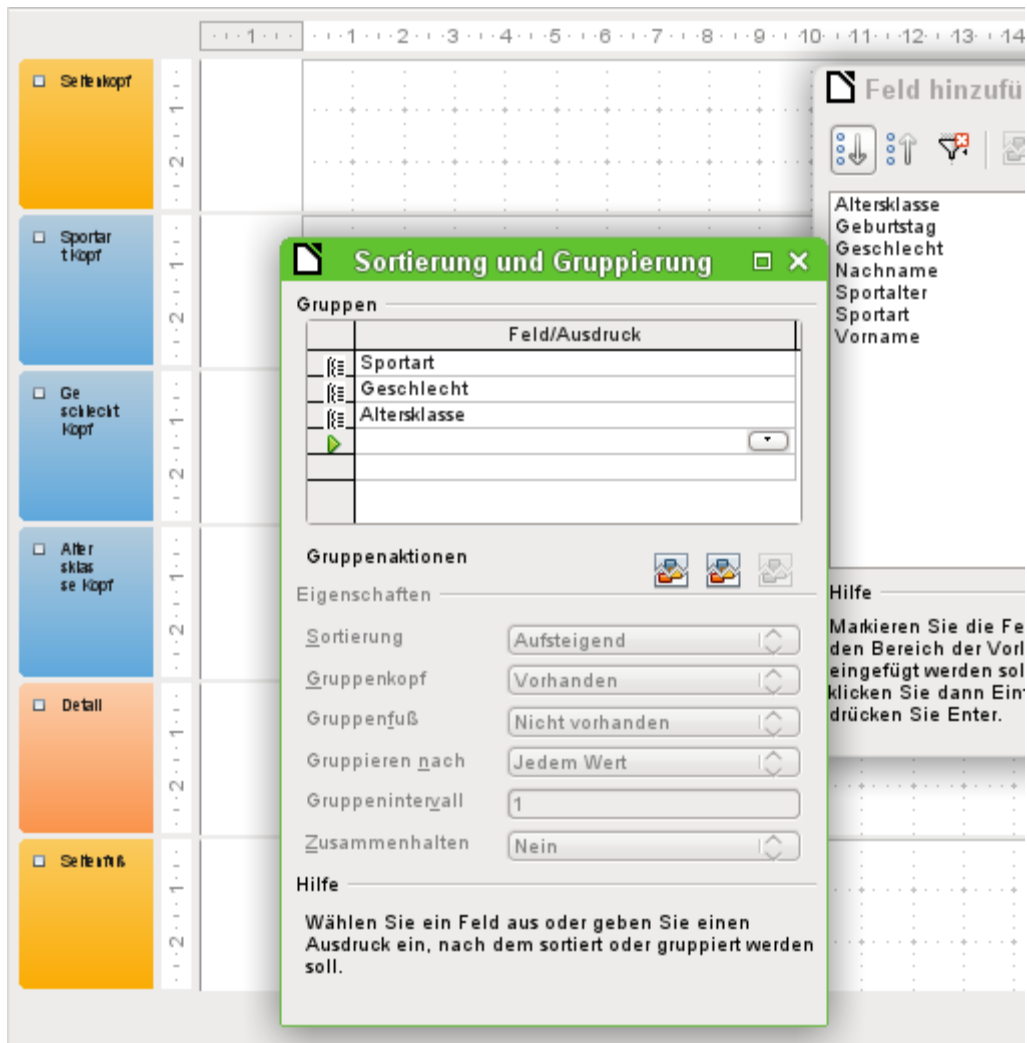


Der Report-Builder zeigt jetzt sämtliche Felder der ausgesuchten Abfrage an.

Im nächsten Schritt wird der Sortier-Dialog ausgewählt:



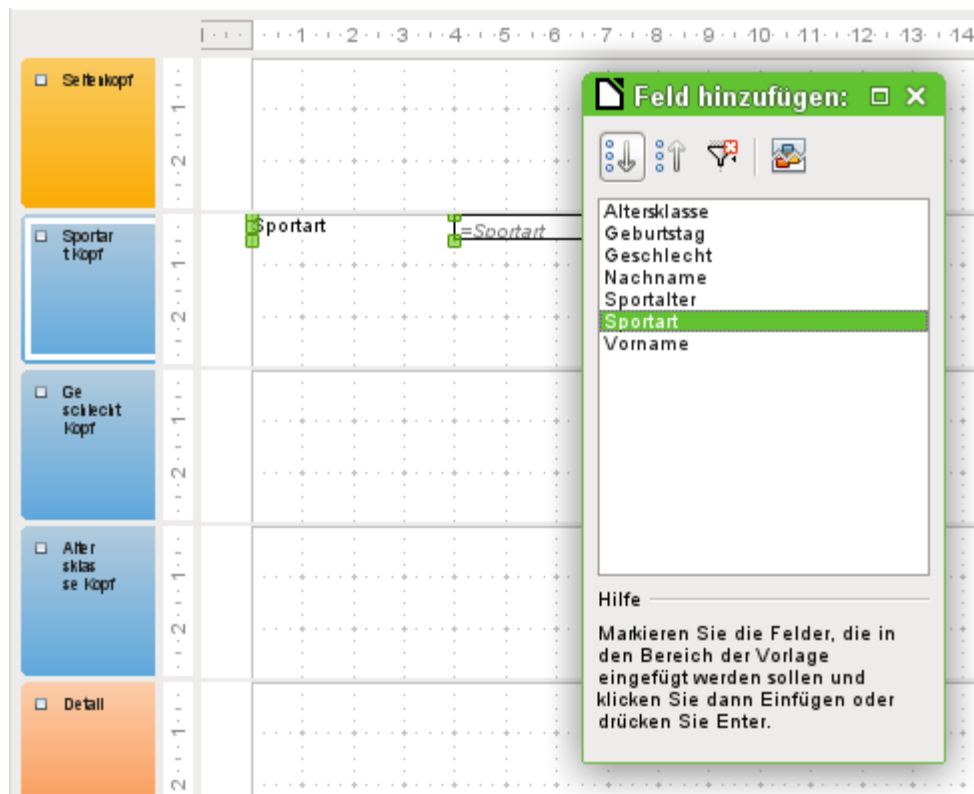
Die Sortierung kann auch über **Ansicht → Sorting and Grouping** angezeigt werden.



Im Sortierungs- und Gruppierungs-Dialog werden nacheinander die Felder "Sportart", "Geschlecht" und "Altersklasse" ausgewählt.

Auf der linken Seite des Berichtes erscheint bei jeder Sortierungsauswahl ein Gruppenkopf für die Sortierung. Die Eigenschaften der Sortierung und Gruppierung werden einfach erst einmal so gelassen wie im Standard vorgesehen.

Der Dialog «Sortierung und Gruppierung» kann nun geschlossen werden.



Der Gruppenkopf «Sportart Kopf» wird ausgewählt. Dies ist an der weißen Umrandung des Kopfes sichtbar. Anschließend wird im Dialog «Feld hinzufügen:» mit einem Mausklick auf das Feld «Sportart» ein Beschriftungsfeld und ein Textfeld eingefügt, das den Inhalt des Feldes «Sportart» aufnehmen soll.

Mit dem gleichen Verfahren werden die Felder «Geschlecht» und «Alter» den passenden Gruppenköpfen zugewiesen.

Sämtliche verbleibenden Felder werden in den Bereich «Detail» eingefügt.

Der Berichtsentwurf dürfte jetzt wie folgt aussehen:

		1	2	3	4	5	6	7	8	9
☐ Seitekopf										
☐ Sportartkopf	Sportart				=Sportart					
☐ Geschlechtkopf	Geschlecht				=Geschlecht					
☐ Altersklassekopf	Altersklasse				=Altersklasse					
☐ Detail	Nachname				=Nachname					
	Vorname				=Vorname					
	Geburtstag				=Geburtstag					
	Sportalter				=Sportalter					
☐ Seiteinh.										

Anschließend sollte der Bericht erst einmal abgespeichert werden. Als Name würde sich so etwas wie «Teilnehmerliste» anbieten.

Auch die Datenbankdatei selbst sollte abgespeichert werden, da sonst die Speicherung des Berichtes erst einmal nur temporär vorgenommen wird.

Hinweis

Gerade beim Gestalten eines Berichtes mit dem Report-Builder fallen oft Instabilitäten des Programms auf. Ist ein wichtiger Schritt gelungen, so sollte auf jeden Fall sowohl der Bericht als auch die Datenbankdatei gespeichert werden.

Die spätere Ausführung eines Berichtes ist von diesen Instabilitäten zum Glück nicht betroffen.

Wird dieser Bericht mit entsprechenden Daten ausgeführt, dann ergibt sich in etwa das folgende Bild:

Sportart	Hochsprung
Geschlecht	m
Altersklasse	20.0
Nachname	Tunichtgut
Vorname	Egon
Geburtsdag	05.07.89
Sportalter	25
Nachname	Müller
Vorname	Karl
Geburtsdag	17.03.85
Sportalter	29

Der Beginn des Berichtes zeigt hier zwei männliche Teilnehmer an, die Hochsprung machen wollen und der Altersklasse 20 angehören.

Bei der Ausführung des Berichtes fallen einige Designmängel auf:

1. Die Abstände zwischen den Gruppenköpfen und dem Inhalt des Detailbereichs ist viel zu groß.
2. Das Geschlecht wird mit «m» bzw. «w» angegeben. Besser wäre eine Bezeichnung wie «Herren» und «Damen».
3. Bei der Altersklasse steht, vermutlich wegen der Division in der Abfrage, eine Zahl mit einer Nachkommastelle, wobei als Dezimaltrenner ein Punkt gesetzt wird.
4. Die Bezeichner für die dargestellten Felder aus dem Bereich «Detail» («Nachname», «Vorname» usw.) sollten besser gemeinsam für alle Inhalte des Bereiches als Tabellenköpfe über den Inhalten stehen.

Abstände zwischen den Berichtsbereichen einstellen

Die Abstände zwischen den Bereichen können direkt an den Tabellenköpfen durch Schieben mit der Maus verringert werden. Es ist auch möglich, einen Bereich zu markieren und in den Eigenschaften des Gruppenkopfes die Höhe über eine Zahleneingabe zu regeln. Dafür darf kein Feld, sondern nur der Gruppenkopf markiert sein.

Es ist nicht möglich, einen Bereich kleiner zu wählen als die darin existierenden Beschriftungs- und Textfelder sind.

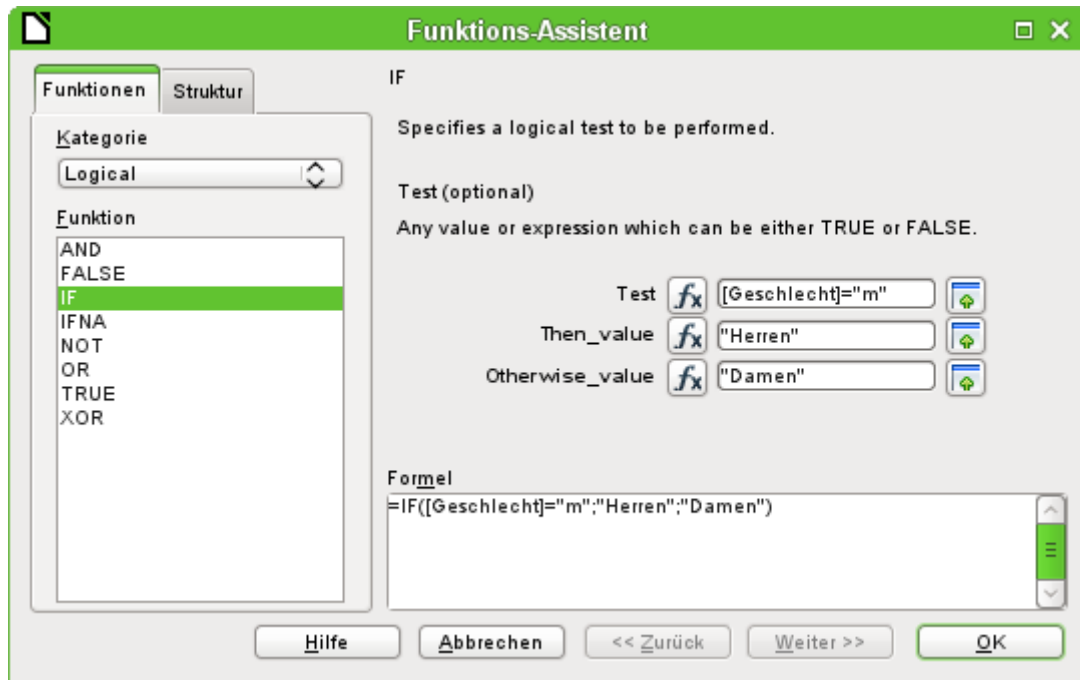
Der Bereich «Altersklasse» sollte dabei so groß gewählt werden, dass er noch die Beschriftungsfelder aus dem Bereich «Detail» aufnehmen kann.

Außerdem muss bei der Auswahl der Abstände berücksichtigt werden, dass eine folgende Gruppe nicht zu nah unter der vorhergehenden Gruppe erscheint. Die Beschriftungs- und Textfelder benötigen gegebenenfalls auch einen Abstand nach oben. Wird dieser Abstand nach oben nicht gewünscht, so kann damit nachgeholfen werden, dass zusätzlich zu den Gruppenköpfen auch Gruppenfüße angezeigt werden. Eine solche Einstellung ist über **Ansicht → Sorting and Grouping** für jede Gruppe möglich.

Beeinflussung eines Textfeldinhaltes durch eine Formel

Die Bezeichnung des Geschlechtes aus der Tabelle reicht für die Ansprüchen einer Teilnehmerliste nicht aus. Es könnte eine entsprechende Umbenennung bereits in der Abfrage erfolgen. Da die Abfrage aber bereits erstellt wurde, werden jetzt die Möglichkeiten des Berichtes genutzt.

Das Textfeld «=Geschlecht» wird markiert. In den Eigenschaften auf der rechten Seite des Report-Builders wird der Reiter **Daten** → **Datenfeld** aufgesucht. Dort wird der Button mit den 3 Punkten ... betätigt. Es erscheint der **Funktions-Assistent**. Alle Funktionen sind hier allerdings nicht übersetzt sondern in englischer Sprache erreichbar.



In der Kategorie «Logical» steckt die Funktion «IF». Die Vorgehensweise bei dieser Funktion ist gleich der Funktion «WENN» aus LO-Calc.

Ein Testwert wird eingegeben. Dabei ist das Feld der Abfrage, aus der die Daten ausgelesen werden, in eckige Klammern zu setzen. Texte müssen in doppelte Anführungszeichen gesetzt werden. Wenn im Feld "Geschlecht" der Wert «m» steht, dann soll in dem Feld des Berichtes «Herren» angezeigt werden. Steht dort kein «m», dann wird «Damen» angezeigt.

Die Eingabe wird mit **OK** bestätigt.

Damit ist die Formulierung des Feldes entsprechend umgestellt.

Umstellung der Formatierung eines Textfeldes

Das Feld, das den Inhalt der Datenbank aufnehmen soll, wird zwar in Berichten als «Textfeld» bezeichnet. Es kann aber genau so formatiert werden wie Felder in Tabellen im Writer oder in Calc.

Das Feld «=Altersklasse» wird markiert. Bei den Eigenschaften am rechten Bildschirmrand wird **Allgemein** → **Formatierung** aufgesucht. Die Formatierung steht dort auf «Text». Der Button mit den 3 Punkten (...) wird gedrückt. Es öffnet sich der Formatierungs-Dialog, der auch aus Calc, dem Writer oder bei Formularerstellungen bekannt ist. Hier wird **Kategorie** → **Zahl** ausgewählt und mit **OK** bestätigt.

Die Formatierung der Anzeige ist nun von Text auf Zahl umgestellt.

Verschieben von Feldern im Report-Builder

Felder können beim Report-Builder auch über die Grenzen eines Bereiches hinweg verschoben werden. Im Zielbereich muss allerdings genügend Platz für das Feld vorhanden sein. Es kann nämlich nicht ein Feld über einem anderen liegen.

Die Positionierung mit der Maus ist je nach System sogar so ungenau, dass am besten reichlich Platz vorhanden sein sollte. Sind erst einmal die Felder in einem Bereich wie hier dem Gruppenkopf «Altersklasse», so lassen sie sich auch mit den Pfeilfeldern und mit Tastatureingaben genau positionieren.

Zur Positionierung mit der Tastatur ist in den Eigenschaften **Allgemein** → **Position X** und **Allgemein** → **Position Y** auf zu suchen.

Teilnehmerliste			
Sportart	=Sportart		
Geschlecht	=F(I(Geschlecht))=m		
Altersklasse	=Altersklasse		
Nachname	Vorname	Geburtstag	Sportalter
=Nachname	=Vorname	=Geburtstag	=Sportalter

Hier wurde noch ein Beschriftungsfeld für die Überschrift hinzugefügt. Die Eingabe des Textes für das Beschriftungsfeld erfolgt in den Eigenschaften des Feldes.

Der Seitenfuß wurde in den Eigenschaften auf **Sichtbar** → **Nein** eingestellt. Er nimmt sonst am unteren Rand des Dokumentes zu viel Platz weg. Hierbei sollte bedacht werden, dass die verfügbare Höhe bereits durch die Größe der Seitenränder zusätzlich verringert wird. Standardmäßig sind alle Seitenränder bei einer DIN-A4-Seite auf 2 cm eingestellt.

Weitere Formatierungsmöglichkeiten für den Bericht werden in dem gesonderten Kapitel «Berichte mit dem Report-Designer» beschreiben.

Erweiterungen der Beispieldatenbank

Das vorgestellte Beispiel ist natürlich nur der erste Schritt für eine Datenbank im Sportbereich. Zuerst einmal sollte auf jeden Fall ein Feld ergänzt werden, das die jeweiligen Leistungen aufnimmt. Solch ein Feld wäre gut in der Tabelle "rel_Teilnehmer_Sportart" unter zu bringen.

Sollen die Eingaben allerdings für mehrere Wettkämpfe in gleichen Sportarten gelten, so ist außerdem noch ein Datumsfeld oder ein anderes Feld, das dem jeweiligen Wettkampf zugeordnet werden kann, in die Tabelle "rel_Teilnehmer_Sportart" aufzunehmen. Das Feld muss dann auch Teil des gemeinsamen Primärschlüssels sein.

Vielleicht käme noch die Vereinszugehörigkeit hinzu. Dazu würde ein Feld in der Tabelle "Teilnehmer" ausreichen. Bei vielen gleichen Vereinen bietet sich aber auch eine separate Tabelle "Vereine" sowie der entsprechende Fremdschlüssel in der Tabelle "Teilnehmer" an.

Dann muss natürlich, wie bei allen Wettbewerben, ermittelt werden, wer denn nun in der jeweiligen Altersklasse auf welcher Platzierung gelandet ist. Sortierung ist hier gefragt, die vielleicht wieder als Bericht in einer Ergebnisliste landet.

Schön wäre dann, wenn jeder Teilnehmer (wieder über einen Bericht) eine schön gestaltete Urkunde mit den persönlichen Leistungen und der Platzierung erhält.

Solche Erweiterungen sind ohne weiteres möglich. Das sollen die kommenden Kapitel des Handbuches zeigen.

Weiteres Einführungs- und Beispielmaterial

Neben den beiden Hauptdatenbanken für das Handbuch «Medien_ohne_Makros.odt» und «Medien_mit_Makros.odt» steht noch ein ganzes Bündel von Beispieldatenbanken mit einer ausführlichen Erklärung an gleicher Stelle wie das Handbuch zum Download bereit.

Ein mehrstufiges Videotutorial von F3K Total, das von den Grundlagen her den Einstieg ermöglicht ist gut von dieser Seite aus zu starten: <http://de.openoffice.info/viewtopic.php?f=27&t=54234#p237478>

Anlaufpunkte für Fragen zu Base sind im deutschsprachigen Raum vor allem die Foren <http://de.openoffice.info/viewforum.php?f=8> und <http://www.libreoffice-forum.de/viewforum.php?f=10>. Bei der Nutzung von Base sind zur Zeit nur geringe Unterschiede zwischen der Nutzung von Open Office, Apache Open Office und Libre Office festzustellen. Deshalb melden sich auch in beiden Foren Nutzer dieser verschiedenen Base-Varianten mit Fragen und Hilfestellungen.

Datenbank erstellen

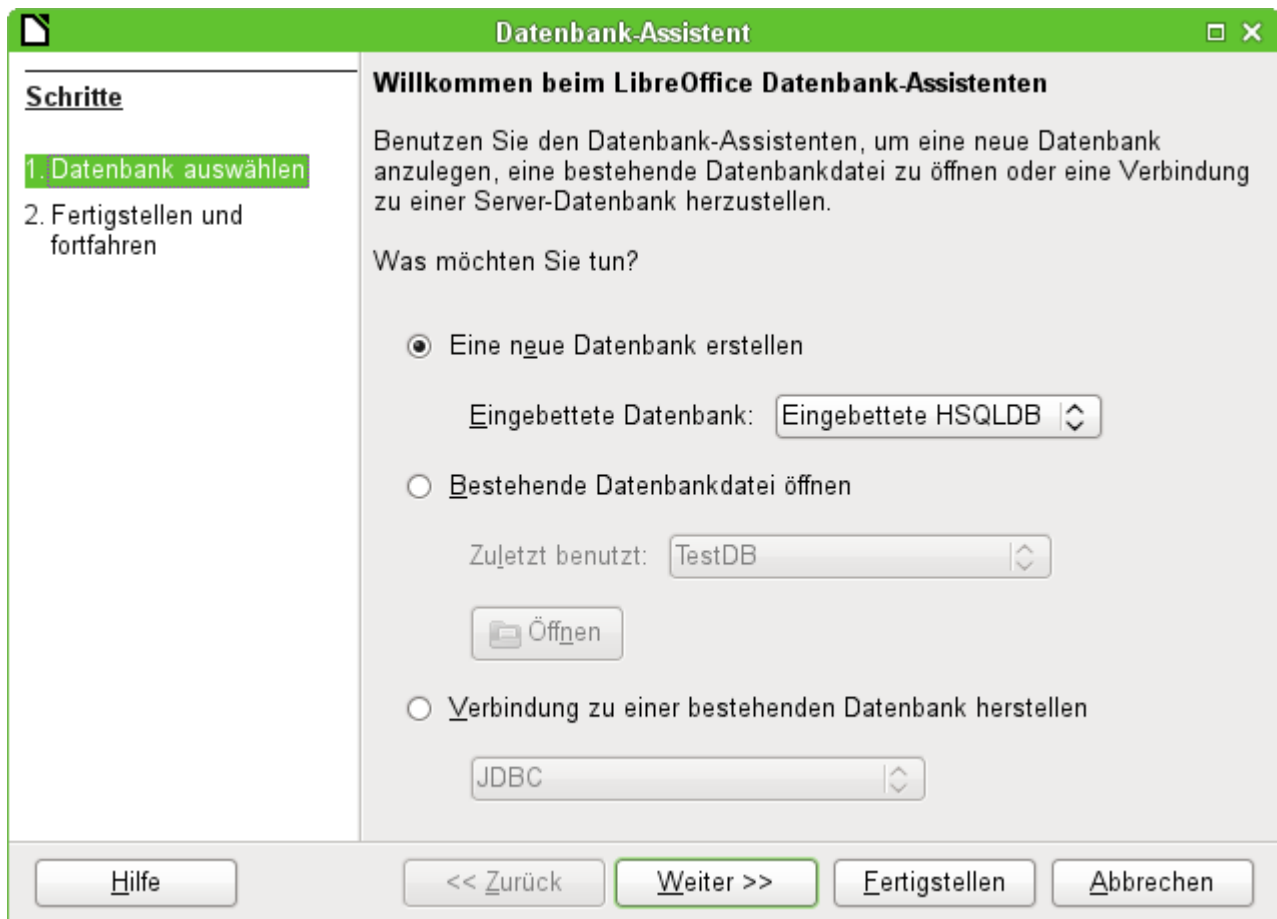
Allgemeines bezüglich der Erstellung einer Datenbank

In LibreOffice gibt es das Programmmodul «Base». Dies stellt eine grafische Benutzeroberfläche für Datenbanken zur Verfügung. Daneben existiert, in LibreOffice eingebunden, die interne Datenbank «HSQLDB». Diese interne Version der HSQLDB-Datenbank kann nur als Datenbank von einem Nutzer betrieben werden. Die gesamten Daten sind in der *.odb-Datei mit abgespeichert und diese Datei ist nur für einen Benutzer nicht schreibgeschützt zu öffnen.

Neue Datenbank als interne Datenbank

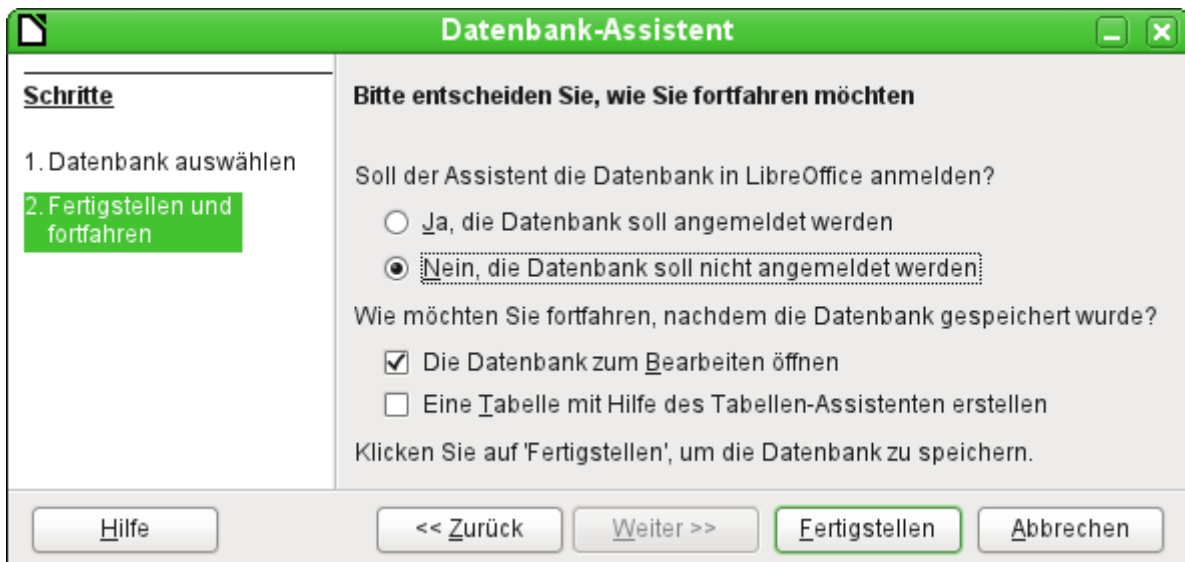
Sofern nicht von vornherein eine Multiuserdatenbank geplant ist oder erst einmal erste Erfahrungen mit einer Datenbank gesammelt werden sollen, ist die interne Datenbank gut nutzbar. Diese lässt sich später auch mit ein paar Kniffen noch zur externen Datenbank umwandeln, auf die dann auch mehrere User gleichzeitig zugreifen können, wenn der Datenbankserver läuft. Eine Beschreibung dazu in Bezug auf die HSQLDB erfolgt im Anhang dieses Handbuches im Kapitel «[Datenbankverbindung zu einer externen HSQLDB](#)».

Die Erstellung einer internen Datenbank erfolgt direkt über den Eingangsbildschirm von LO mit einem Klick auf den Button **Datenbank** oder über **Datei → Neu → Datenbank**. Der Datenbank-Assistent startet mit dem folgenden Bildschirm:



Das Optionsfeld «Neue Datenbank erstellen» ist für eine neue interne Datenbank angewählt. Standardmäßig ist dies die eingebettete HSQLDB. Ab LO 4.2 steht zusätzlich die interne Firebird-Datenbank zur Verfügung. Diese muss allerdings bisher extra aktiviert werden.

Die weiteren Optionen dienen dazu, eine bestehende Datei zu öffnen oder eine Verbindung zu einer externen Datenbank wie z.B. einem Adressbuch, einer MySQL-Datenbank o.ä. aufzubauen.



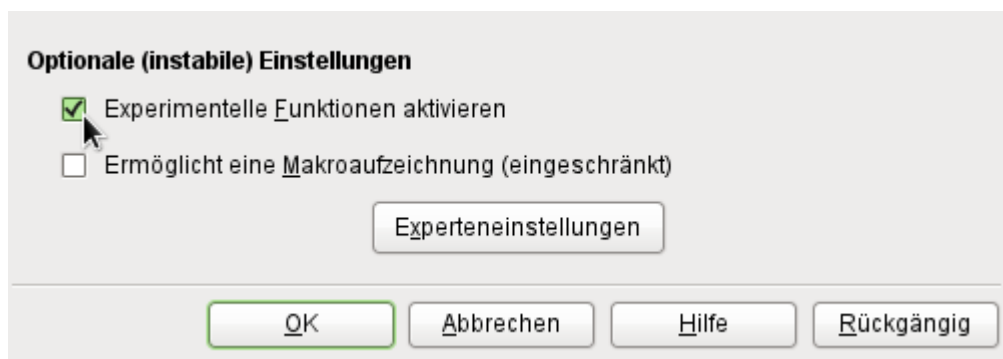
Eine in LibreOffice angemeldete Datenbank kann von den anderen Programmteilen von LibreOffice als Datenquelle genutzt werden (z. B. Serienbrief). Diese Anmeldung kann aber auch zu einem späteren Zeitraum erfolgen. Deshalb ist an dieser Stelle erst einmal das Optionsfeld «Nein, die Datenbank soll nicht angemeldet werden», ausgewählt.

Bei den Markierfeldern ist lediglich «Die Datenbank zum Bearbeiten öffnen» markiert. Assistenten zur Erstellung von Tabellen, Abfragen usw. werden in diesem Handbuch nicht genutzt. Ihre Funktion ist lediglich für den einfachen Einstieg in die Materie geeignet.

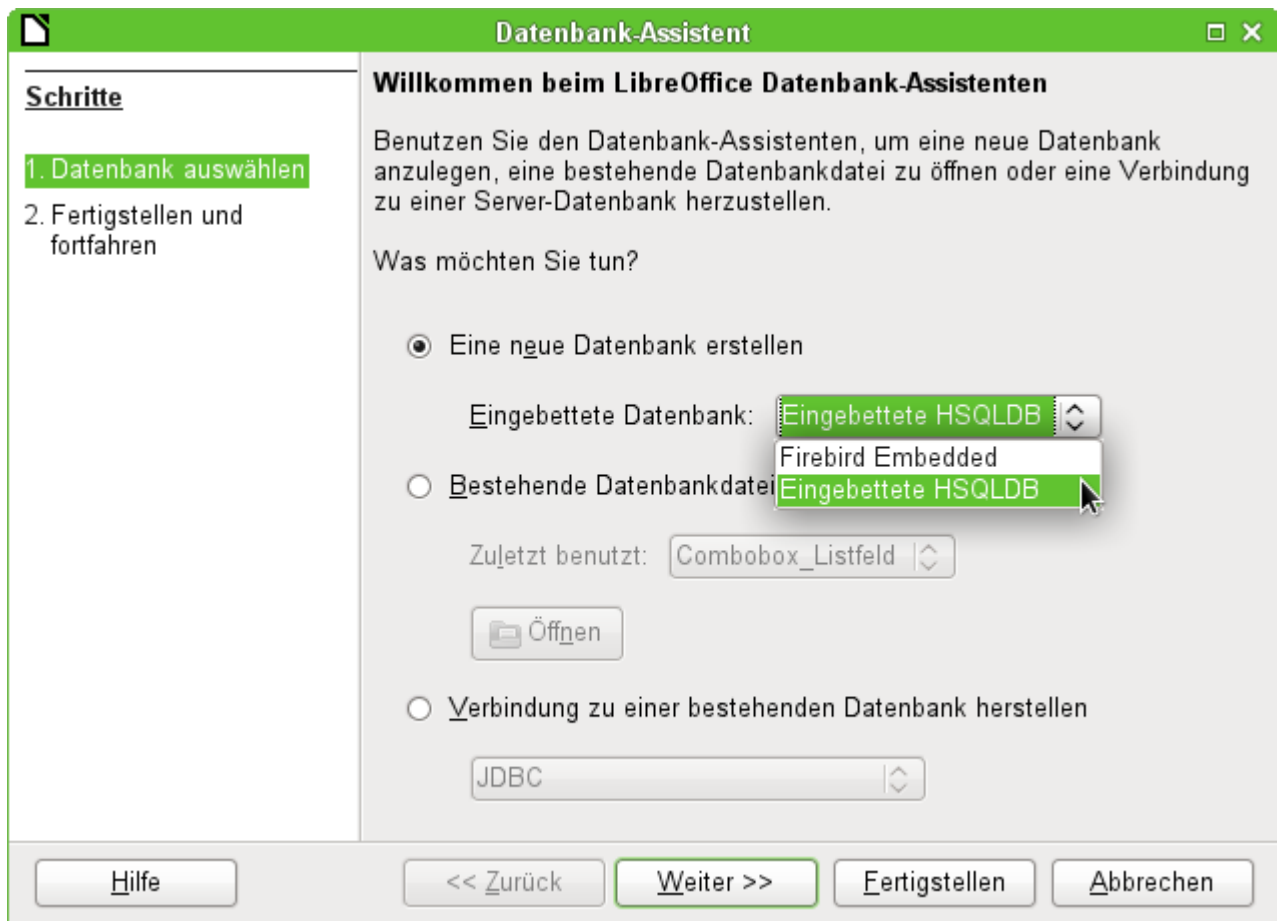
Die Datenbank wird fertiggestellt, indem direkt noch einem Namen und einem Speicherort gefragt wird. An der entsprechenden Stelle wird dann die *.odb-Datei erzeugt, die zur Aufnahme von Daten aus der internen Datenbank, zur Speicherung von Abfragen, Formularen und Berichten gedacht ist.

Im Gegensatz zu den anderen Programmteilen von LibreOffice wird also die Datei abgespeichert, bevor der Nutzer überhaupt irgendwelche für ihn sichtbare Eingaben getätigt hat.

Mit der Version LibreOffice 4.2 wurde zusätzlich zu der internen HSQLDB eine interne Firebird-Datenbank aufgenommen. Um diese bisher noch nicht einwandfrei funktionierende Datenbank zu nutzen, müssen über **Extras → Optionen → LibreOffice → Erweitert → Optionale (instabile) Einstellungen** die experimentellen Funktionen aktiviert werden:



Danach erscheint in dem Datenbankassistenten neben der internen HSQLDB auch die interne Firebird-Datenbank zur Auswahl:



Die eingebettete Firebird-Datenbank ist allerdings bisher noch weit entfernt von ihren tatsächlichen Möglichkeiten. SQL-Erweiterungen zu der HSQLDB sind erst teilweise implementiert.

Die HSQLDB ist auch in LibreOffice 5.3 die Standarddatenbank für jeden Nutzer; die Firebird-Datenbank hat mit LO 5.3 ein Update auf die aktuelle Firebird-Version 3.0 vollzogen, ist aber weiterhin nur experimentell zu empfehlen. Das Handbuch bezieht sich aus diesem Grunde in der Hauptsache auf die interne HSQLDB. Besonderheiten der Firebird-Datenbank werden gegebenenfalls zusätzlich aufgezeigt. Die Datenbank-Beispiele beziehen sich weiterhin auf die HSQLDB.

Zugriff auf externe Datenbanken

Alle externen Datenbanken müssen zuerst einmal existieren. Angenommen es soll der Zugriff auf die Datenbank einer eigenen Homepage erfolgen. Dort muss zuerst einmal die Datenbank selbst mit einem Namen und Zugangsdaten gegründet worden sein, bevor externe Programme darauf zugreifen können.

Existiert nun diese externe Datenbank, so kann sie, je nach vorhandener Treibersoftware, zur Erstellung von Tabellen und anschließender Dateneingabe und -abfrage genutzt werden.

Über **Datei → Neu → Datenbank** wird der Datenbankassistent geöffnet und die Verbindung zu einer bestehenden Datenbank hergestellt. Die Liste der hier aufgeführten Treiber, die mit Datenbanken verbinden, variiert etwas je nach Betriebssystem und Benutzeroberfläche. Immer dabei sind aber

- dBase
- JDBC
- MySQL
- ODBC

- Oracle JDBC
- PostgreSQL
- Tabellendokument
- Text
- sowie verschiedene Adressbücher

Je nach gewählter Datenbank variieren nun die Verbindungseinstellungen. Diese können auch gegebenenfalls später noch korrigiert werden, wenn erst einmal die *.odb-Datei erstellt worden ist.

Bei manchen Datenbanktypen können keine neuen Daten eingegeben werden. Sie sind deshalb nur zum Suchen von Daten geeignet (z. B. Tabellendokument, Adressbücher). Die Beschreibungen in den folgenden Kapiteln beziehen sich ausschließlich auf die Verwendung von LibreOffice Base mit der internen Datenbank HSQLDB. Die meisten Ausführungen lassen sich auf Datenbanken wie MySQL, PostgreSQL etc. übertragen und entsprechend anwenden.

Hier soll nur kurz an ein paar Beispielen vorgestellt werden, wie der Kontakt zu anderen externen Datenbanken hergestellt wird.

MySQL/MariaDB-Datenbanken

MySQL-Datenbanken oder auch MariaDB-Datenbanken können auf drei verschiedene Weisen mit Base verbunden werden. Die einfachste und schnellste Art ist die direkte Verbindung mit dem MySQL-Connector. Daneben steht noch die Verbindung über JDBC und ODBC zur Verfügung.

Hinweis

In MySQL und MariaDB ist es möglich, Daten in Tabellen ohne ein Primärschlüsselfeld einzugeben und zu ändern. Die GUI von Base zeigt diese Tabellen zwar an, bietet aber keine Eingabe- bzw. Änderungsmöglichkeit.

Wer Tabellen ohne Primärschlüssel verwenden möchte, kann stattdessen über **Extras → SQL**, oder innerhalb von Formularen über Makros, die Tabellen mit Daten versorgen.

Erstellen eines Nutzers und einer Datenbank

Nachdem MySQL bzw. MariaDB installiert ist, sollten nacheinander die folgenden Schritte vollzogen werden:

1. Der Administrationsuser in MySQL heißt «root». Für Linuxnutzer ist hier wichtig, dass es sich dabei nicht um den Administrator des Linuxsystems handelt. Diesem Nutzer wird direkt nach der Installation erst einmal ein Passwort zugewiesen. Dies kann je nach System auch schon vorher bei der Installation erfolgt sein.

mysql -u root -p

Am Anfang ist kein Passwort vorhanden, so dass bei der Eingabeaufforderung auch nur **Enter** gedrückt wird.

Es erscheint die Eingabeaufforderung

mysql>

Alle folgenden Eingaben werden jetzt auf dieser mysql-Konsole erstellt. Die Passwörter können sich unterscheiden, je nachdem, ob die Anmeldung von dem momentanen Rechner («localhost») oder von einem anderen Rechner zum MySQL-Serverrechner («host») erfolgt.

SET PASSWORD FOR root@localhost=PASSWORD('Passwort');

SET PASSWORD FOR root@host=PASSWORD('Passwort');

Windows-Nutzer geben statt der zweiten Zeile

SET PASSWORD FOR root@'%'=PASSWORD('Passwort');

ein.

2. Als Sicherheitsmaßnahme werden alle eventuell bestehenden anonymen Nutzer gelöscht.
DELETE FROM mysql.user WHERE User='';
DELETE FROM mysql.db WHERE User='';
FLUSH PRIVILEGES;
3. Eine Datenbank mit dem Namen «libretest» wird erstellt.
CREATE DATABASE libretest;
4. Alle Rechte an der Datenbank «libretest» werden dem Nutzer «lotest» übergeben, der sich durch das Passwort «libre» ausweisen soll.
GRANT ALL ON libretest.* TO lotest IDENTIFIED BY 'libre';

Damit ist die Datenbank vorhanden, mit der die folgenden Verbindungen aufgebaut werden.

MySQL-Verbindung direkt mit der Extension

Für die **MySQL-Datenbank** existieren Erweiterungen zur direkten Verbindung mit Base. Diese Erweiterungen tragen den Namen **MySQL-Connector**.

- Die mit der 3.3.4 funktionierende Erweiterung («Extension») ist unter http://extensions.services.openoffice.org/en/project/mysql_connector (Stand: 1/11) zu finden.
- Eine auch unter anderen Systemen lauffähige Erweiterung für Versionen nach 3.3.4 bis einschließlich 4.0.* steht unter <http://extensions.openoffice.org/en/project/aoo-my-sdbc> zum Download.
- Für Version LO 4.0 ist eine Extension für Windows über <https://dl.dropboxusercontent.com/u/193133/mysql-connector-ooo.oxt> downloadbar.
- Für Version LO 4.1 und 4.2 ist eine Extension für Windows über <http://extensions.libreoffice.org/extension-center/mysql-native-connector-for-libreoffice-4.x> downloadbar.
- Für die Versionen ab 4.1 ist eine Extension für Linux über <http://extensions.libreoffice.org/extension-center/mysql-native-connector> downloadbar.

Vorsicht



Ein Kontakt zu LO 4.1.* kommt zwar mit dem AOO-Connector zustande, aber da sich ab der Version LO 4.1 die Beschreibung der Sekundenbruchteile von LO und AOO unterscheiden, werden falsche Werte im Bereich der Zeiten und auch von kompletten Timestamp-Feldern angezeigt.

Verwenden sie also bitte immer die Version, die lt. Beschreibung auch zur LO-Version passt. Achten sie besonders beim Wechsel auf LO 4.1 darauf, ob überhaupt ein entsprechender Connector zur Verfügung steht.

MySQL-Verbindung über JDBC

Als allgemeiner Zugang zu MySQL ist der Zugang über JDBC oder ODBC zu wählen. Um den JDBC-Zugang nutzen zu können, wird der mysql-connector-java.jar in der jeweils zur Datenbank passenden Fassung benötigt. Dieses Java-Archiv wird am besten in das Verzeichnis kopiert, aus dem auch die aktuelle Java-Version von LibreOffice geladen wird. Als Speicherort bietet sich das Unterverzeichnis «...Javapfad.../lib/ext/» an. In Linux-Systemen wird dies durch die Softwareverwaltung oft automatisch erledigt.

Gegebenenfalls kann das Java-Archiv auch separat über **Extras → Optionen → LibreOffice → Erweitert → Java Optionen → Class-Pfad** in den Class-Path aus jeder beliebigen Stelle der Festplatte übernommen werden. Für diese Option sind dann auch keine Systemverwalterrechte notwendig.

MySQL-Verbindung über ODBC

Für eine Verbindung über ODBC muss natürlich erst einmal die entsprechende ODBC-Software installiert sein. Details dazu werden hier nicht weiter beschrieben.

Nach Installation der entsprechenden Software kann es passieren, dass LO den Dienst verweigert, weil es die libodbc.so.1 nicht findet. Hier existiert in den meisten Systemen inzwischen die libodbc.so.2. Auf diese Datei muss ein entsprechender Verweis mit dem Namen libodbc.so.1 in dem gleichen Verzeichnis abgespeichert werden.

In den für das System notwendigen Dateien odbcinst.ini und odbc.ini müssen Einträge ähnlich den folgenden existieren:

odbcinst.ini

```
[MySQL]
Description = ODBC Driver for MySQL
Driver = /usr/lib64/libmyodbc5.so
```

odbc.ini

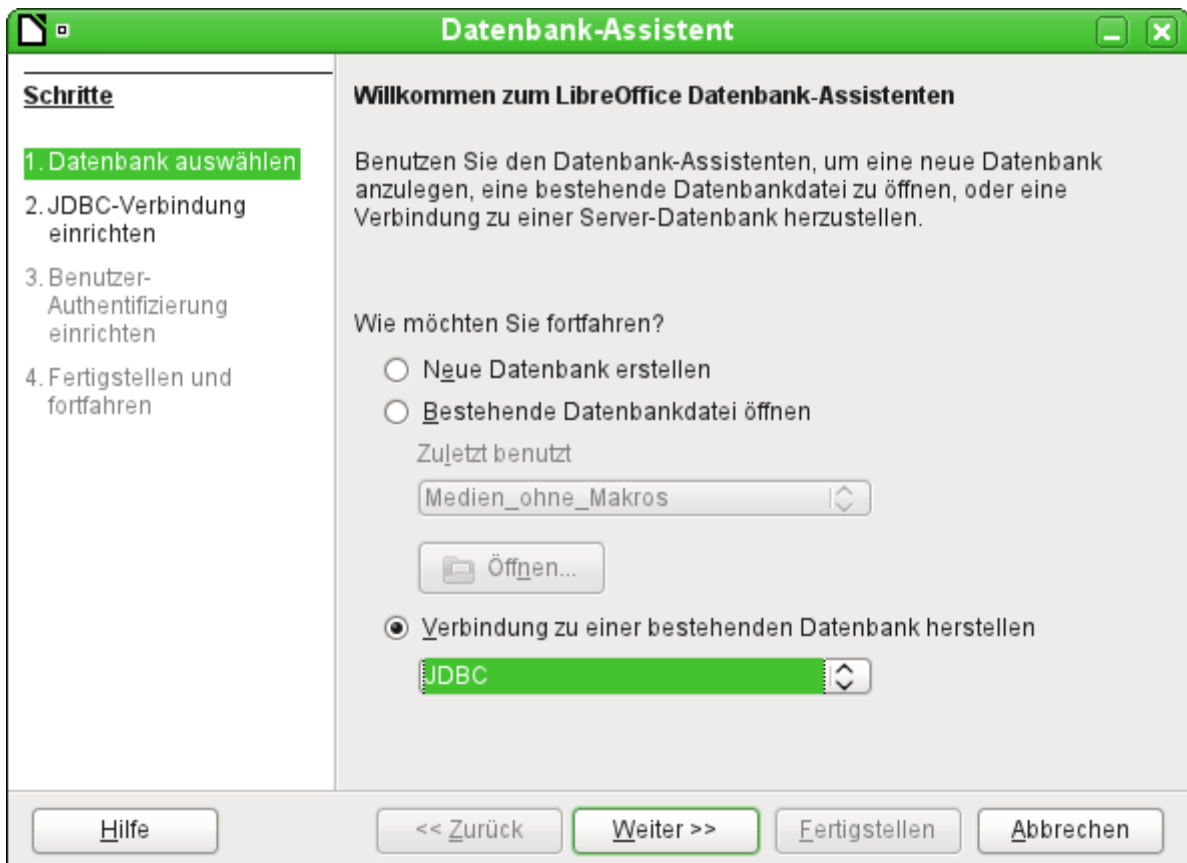
```
[MySQL-test]
Description = MySQL database test
Driver = MySQL
Server = localhost
Database = libretest
Port = 3306
Socket =
Option = 3
Charset = UTF8
```

Diese beiden Dateien liegen bei einem Linux-System im Verzeichnis /etc/unixODBC. Ohne den Eintrag zur Art des verwendeten Zeichensatzes kommt es bei Umlauten zu Problemen, auch wenn die Einstellungen in MySQL/MariaDB und Base übereinstimmen.

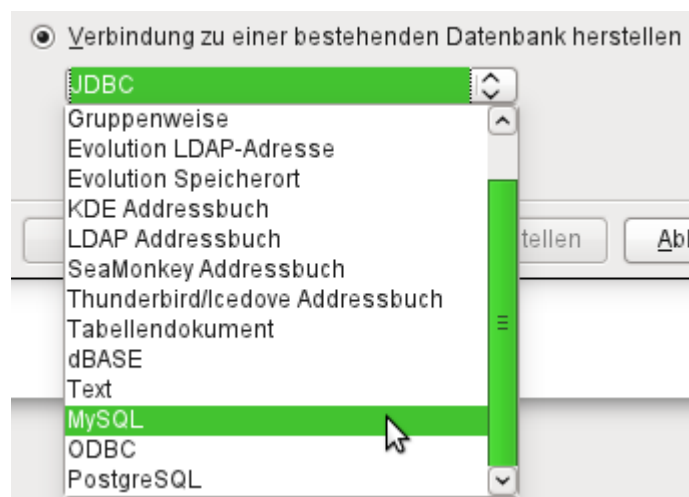
Details zu den Verbindungsparametern sind im [MySQL-Referenzhandbuch](#) zu finden.

Verbindung zur MySQL-Datenbank über den Datenbank-Assistenten

Der Zugriff auf eine existierende MySQL-Datenbank erfolgt mit der direkten Verbindung in den folgenden Schritten:



Über «*Neue Datenbank erstellen*» ist nur die Erstellung einer Datenbank im internen HSQLDB-Format möglich. Die Zusammenarbeit mit anderen Datenbanken kann nur erfolgen, wenn die Datenbank selbst bereits existiert. Dazu muss also **Verbindung zu einer bestehenden Datenbank herstellen** gewählt werden.



Hier wird aus den, teilweise betriebssystemspezifischen, Datenbanken die MySQL-Variante ausgewählt.

Die direkte Verbindung

Datenbank-Assistent

Schritte

1. Datenbank auswählen
2. MySQL Verbindung einrichten
3. MySQL Server-Daten einrichten
4. Benutzer-Authentifizierung einrichten
5. Fertigstellen und fortfahren

Verbindung zu einer MySQL Datenbank einrichten

Bitte wählen Sie, ob die Verbindung auf die MySQL Datenbank per ODBC oder per JDBC hergestellt werden soll. Fragen Sie Ihren Systemadministrator, wenn Sie sich bei den folgenden Einstellungen unsicher sind.

Wie möchten Sie sich mit Ihrer Datenbank verbinden?

☐ Verbindung über ODBC (Open Database Connectivity)

☐ Verbindung über JDBC (Java Database Connectivity)

☒ Direkte Verbindung

Hilfe << Zurück Weiter >> Fertigstellen Abbrechen

Der Kontakt zu MySQL kann über ODBC oder JDBC hergestellt werden, solange kein nativer³ MySQL-Connector als direkte Verbindung installiert ist oder unterstützt wird. Ist die Erweiterung für eine direkte Verbindung installiert, so wird «Direkte Verbindung» standardmäßig voreingestellt.

Die direkte Verbindung ist diejenige, die von der Geschwindigkeit her am besten gewählt werden sollte. Allerdings hat sie auch ein paar kleine Bugs, die gegebenenfalls besonders berücksichtigt werden müssen. So ist es z. B. dringend erforderlich, in Formularen bei Textfeldern die maximale Länge einzustellen, weil sonst die Eingabe einfach auf den bisher maximal eingegebenen Wert gekürzt wird.

Datenbank-Assistent

Schritte

1. Datenbank auswählen
2. MySQL Verbindung einrichten
3. MySQL Server-Daten einrichten
4. Benutzer-Authentifizierung einrichten
5. Fertigstellen und fortfahren

Verbindung zu einer MySQL Datenbank einrichten

Bitte geben Sie die benötigten Informationen ein, um eine Verbindung zu einer MySQL-Datenbank herzustellen.

Datenbankname libretest

☒ Server / Port

Server localhost

Port 3306 Default: 3306

☐ Socket

Hilfe << Zurück Weiter >> Fertigstellen Abbrechen

Der Datenbankname muss bekannt sein. Befindet sich der Server auf dem gleichen Rechner wie die Benutzeroberfläche, unter der die Datenbank erstellt wird, so kann als Server «localhost» gewählt werden. Ansonsten kann die IP-Adresse (z.B. 192.168.0.1) oder auch je nach Netzwerkstruktur der Rechnername oder gar eine Internetadresse angegeben werden. Es ist also ohne weiteres möglich, mit Base auf die Datenbank zuzugreifen, die vielleicht auf der eigenen Homepage bei irgendeinem Provider liegt.

³ «nativ» – unverändert, unmodifiziert

Bei der Arbeit mit Base über das Internet sollte sich der Nutzer allerdings darüber bewusst sein, wie seine Verbindung zu der Datenbank gestaltet ist. Gibt es eine verschlüsselte Verbindung? Wie erfolgt die Passwortübertragung?

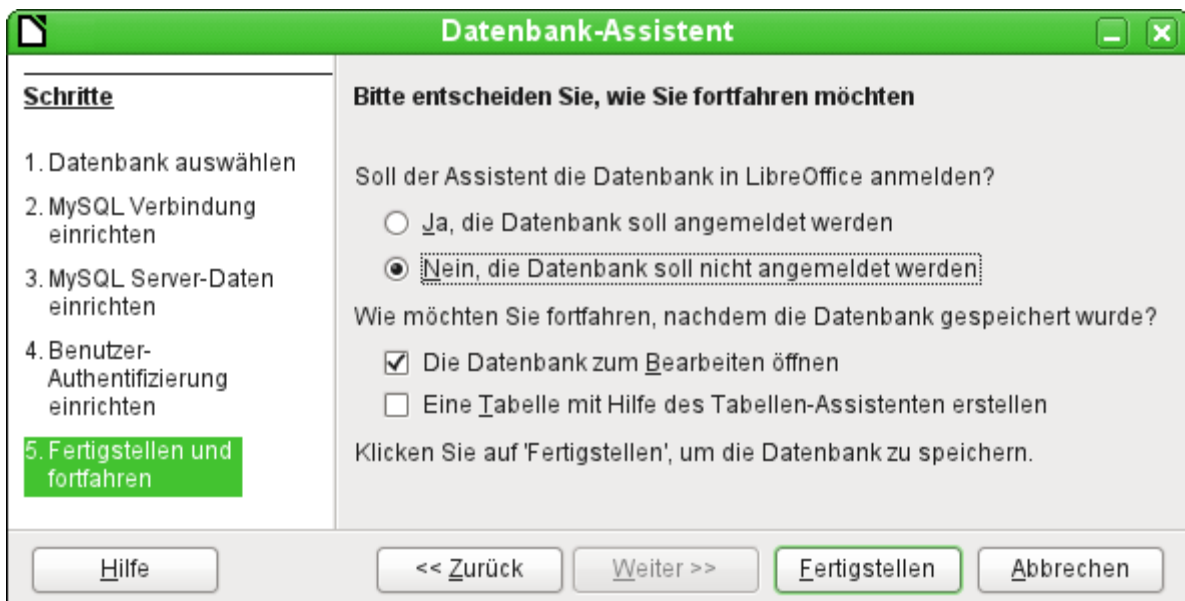
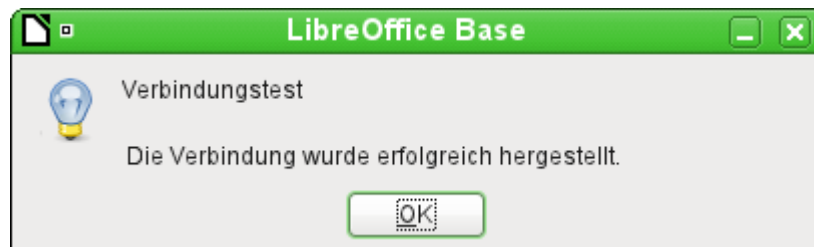
The screenshot shows the 'Datenbank-Assistent' window with a green title bar. On the left, a 'Schritte' (Steps) list contains five items: '1. Datenbank auswählen', '2. MySQL Verbindung einrichten', '3. MySQL Server-Daten einrichten', '4. Benutzer-Authentifizierung einrichten' (highlighted in green), and '5. Fertigstellen und fortfahren'. The main area is titled 'Benutzer-Authentifizierung einrichten' and contains the text: 'Geben Sie bitte jetzt einen Benutzernamen ein, falls die Datenbank zur Verbindungsherstellung einen bestimmten Benutzer erwartet.' Below this, there is a 'Benutzername' label and a text input field containing 'lotest'. A checkbox labeled 'Kennwort erforderlich' is checked. At the bottom right is a 'Verbindungstest' button. The bottom of the window has four buttons: 'Hilfe', '<< Zurück', 'Weiter >>' (highlighted in green), and 'Fertigstellen'. A disabled 'Abbrechen' button is also present.

Jede über das Netz erreichbare Datenbank sollte mit einem Benutzernamen und einem Kennwort geschützt sein. Hier wird direkt getestet, ob die Verbindung klappt. Wichtig ist natürlich, dass der entsprechende Benutzer in MySQL bzw. der MariaDB entsprechend für den benannten Server eingerichtet wurde.

The screenshot shows a dialog box titled 'Authentifizierung erforderlich' with a green title bar. It contains the text: 'Meldung vom Server: Die Verbindung zur Datenquelle erfordert die Eingabe eines Kennwortes.' Below this is a horizontal line, followed by the text: 'Geben Sie Benutzernamen und Kennwort ein für:'. Another horizontal line follows, then the 'Benutzername' label and a text input field containing 'lotest'. Below that is the 'Kennwort' label and a password input field showing six asterisks. At the bottom are three buttons: 'Hilfe', 'OK' (highlighted in green), and 'Abbrechen'.

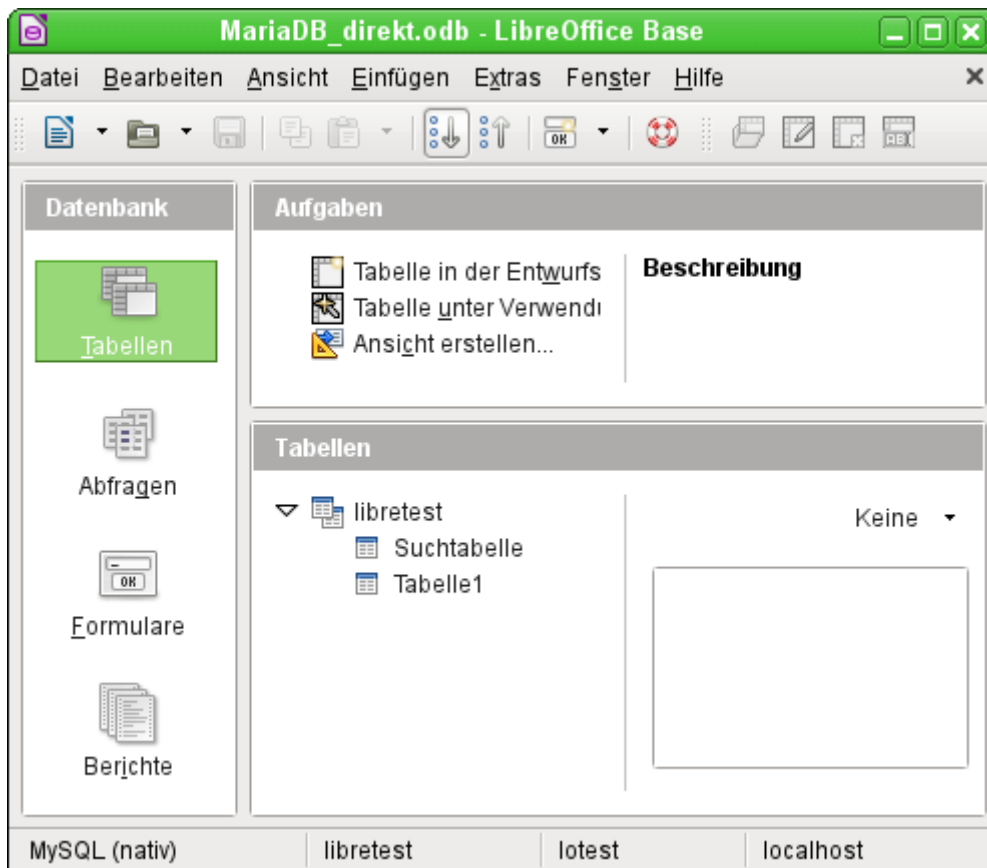
Dieses Fenster erscheint anschließend bei jedem Start der Datenbankdatei, wenn das erste Mal auf die MySQL-Datenbank zugegriffen wird.

Der Verbindungstest startet die Authentifizierung mit vorgegebenem Benutzernamen. Nach Passworteingabe erfolgt die Meldung, ob die Verbindung erfolgreich hergestellt werden kann. Läuft z.B. MySQL zur Zeit nicht, so kommt natürlich an dieser Stelle eine Fehlermeldung.



Auch hier wird die Datenbank nicht angemeldet, da sie nur zu ersten Tests aufgebaut wurde. Eine Anmeldung ist erst notwendig, wenn andere Programme wie z.B. der Writer für einen Serienbrief auf die Daten zugreifen sollen.

Der Assistent beendet die Verbindungserstellung mit dem Abspeichern der gewünschten Datenbankverbindung. In dieser *.odb-Datei liegen jetzt lediglich diese Verbindungsinformationen, die bei jedem Datenbankstart ausgelesen werden, so dass auf die Tabellen der MySQL-Datenbank zugegriffen werden kann.



Ansicht der geöffneten Datenbankdatei mit Tabellenübersicht und in der Fußzeile der Benennung des verwendeten Treibers «MySQL (nativ)», des Datenbanknamens «libretest», des Nutzers der Datenbank «lotest» und des Servers, auf dem die Datenbank läuft, nämlich «localhost».

Über «Fertigstellen» wird die Base-Datei erstellt und die Ansicht auf die Tabellen der MySQL-Datenbank geöffnet. Die Tabellen der Datenbank werden unter dem Namen der Datenbank selbst aufgeführt.

Beim manchen Treibern wird nur die Datenbank «libretest» angezeigt, für die auch die Verbindung bestimmt war. Andere Treiber von LO bieten auch andere MySQL- bzw. MariaDB-Datenbanken auf dem gleichen Server zur Auswahl.

Auch mit den Treibern für nur eine Datenbank ist aber ein Zugriff auf die anderen Tabellen z.B. für Abfragen möglich, sofern natürlich der angegebene Datenbanknutzer, im obigen Fall also «lotest», mit seinem Passwort auf die Daten zugreifen kann. Im Gegensatz zu den bisherigen nativen LO-Treibern ist hier allerdings kein schreibender Zugriff auf andere Datenbanken des gleichen MySQL-Datenbankservers möglich.

Im Unterschied zu der internen Datenbank von Base taucht bei Abfragen entsprechend in MySQL für die Definition der Tabelle immer auch der Datenbankname auf, hier z.B.

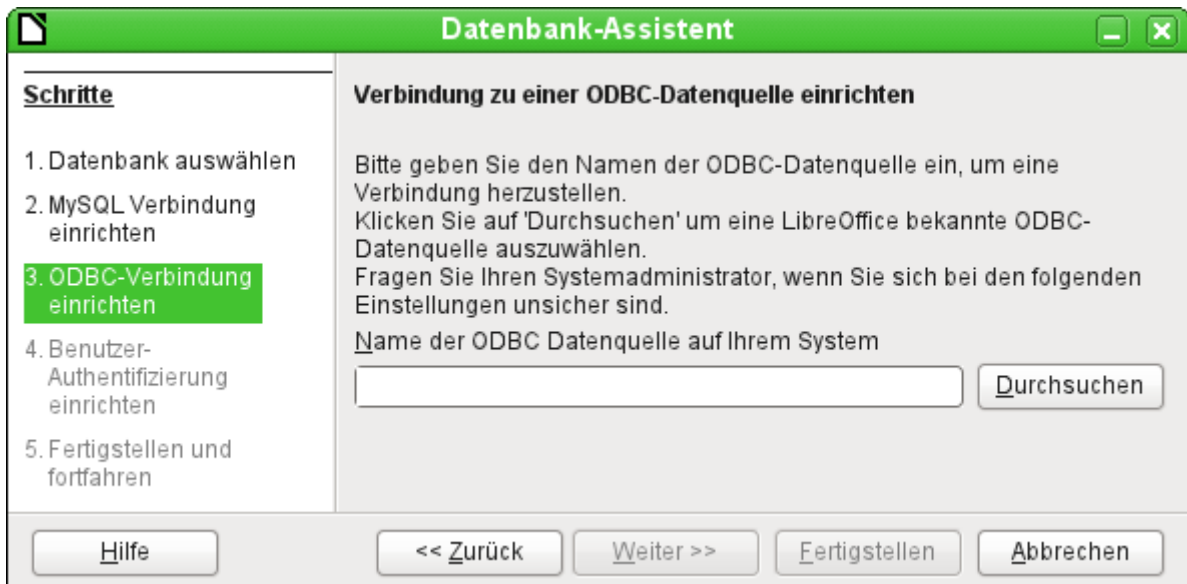
```
... FROM "test"."Klasse" AS "Klasse" ...
```

Hier ist es also auf jeden Fall notwendig, der Kombination aus Datenbankname und Tabellennamen mit dem Zusatz «AS» einen alternativen Alias-Namen zuzuweisen. Genauereres siehe dazu in dem Kapitel «[Verwendung eines Alias in Abfragen](#)».

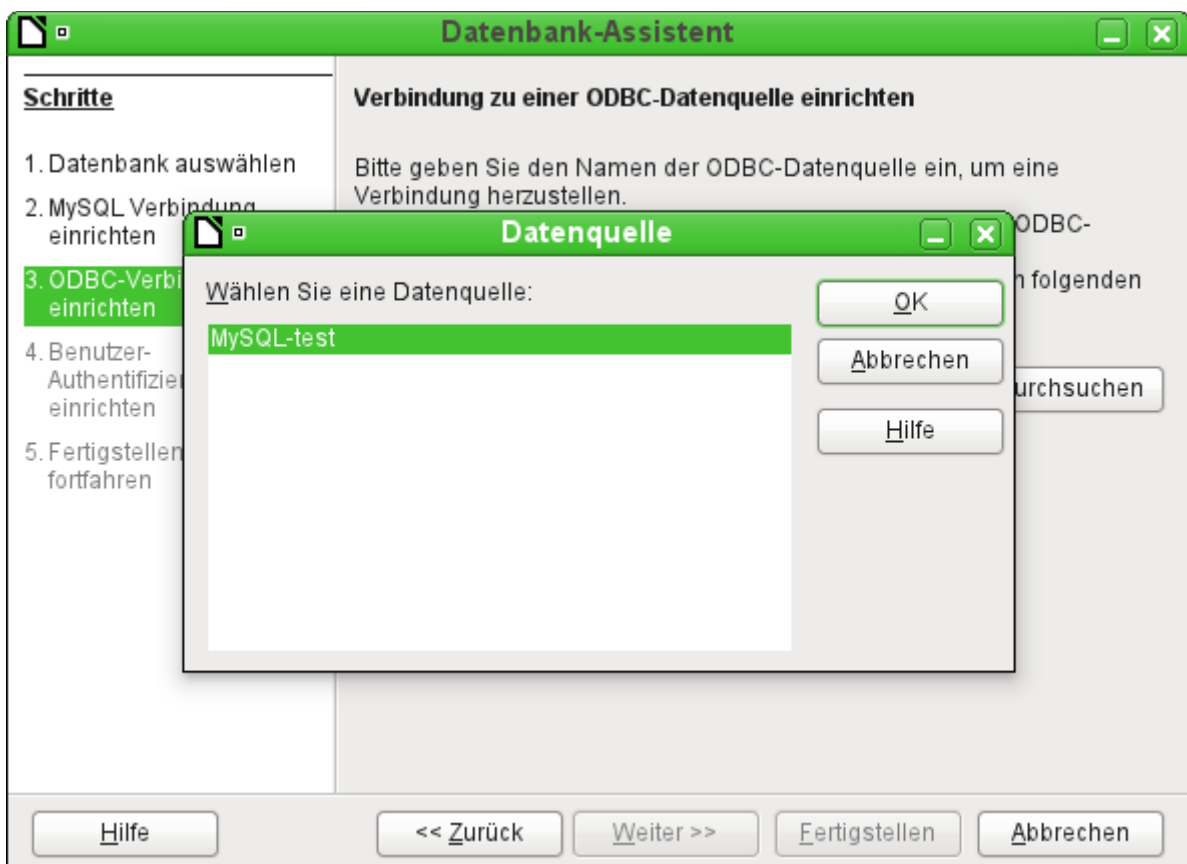
Tabellen können in der Datenbank erstellt und gelöscht werden. Automatisch hochzählende Zahlenfelder («Autowert») funktionieren und lassen sich auch bei der Tabellenerstellung auswählen. Sie starten bei MySQL mit dem Wert 1.

Die ODBC-Verbindung

Die ersten Schritte zur ODBC-Verbindung sind gleich denen zur direkten Verbindung. Wird beim zweiten Schritt dann die ODBC-Verbindung für MySQL gewählt, dann erscheint das folgende Fenster des Datenbank-Assistenten:



Die ODBC Datenquelle hat nicht unbedingt den gleichen Namen wie die Datenbank in MySQL selbst. Hier muss der Name eingetragen werden, der auch in der Datei «odbc.ini» steht. Die einfachste Möglichkeit besteht hier darin, über den Button **Durchsuchen** den Namen aus der «odbc.ini» direkt auszulesen.



Beim Test erscheint der Name aus der «odbc.ini». Auch wenn hier wieder nur mit einer Datenbank verknüpft wird, können andere Tabellen des MySQL-Servers sehr wohl gelesen werden.

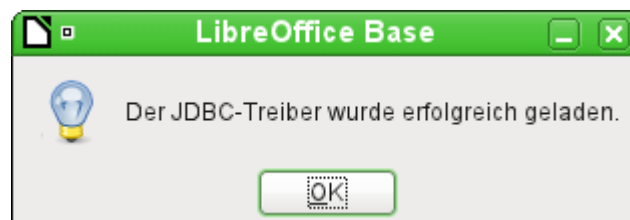
Schritt 4 und 5 laufen wieder identisch zur Direktverbindung ab.

Die JDBC-Verbindung

Auch bei der JDBC-Verbindung sind die ersten Schritte gleich, hier unterscheidet sich wieder nur Schritt 3 von den anderen Schritten des Assistenten:

Der Assistent fragt hier die gleichen Informationen ab wie bei der direkten Verbindung. Der Datenbankname ist der, der auch in MySQL selbst verwandt wird.

Über «Klasse testen» wird überprüft, ob das Archiv mysql-connector-java.jar über Java erreichbar ist. Entweder muss dieses Archiv im Pfad der ausgewählten Java-Version liegen oder direkt in LibreOffice eingebunden werden.



Alle weiteren Schritte sind wieder identisch zu denen der vorherigen Verbindungen. Die Verbindungen zu anderen Datenbanken des gleichen MySQL-Datenbankservers funktionieren ebenfalls nur lesend.

Zugriff auf gespeicherte Prozeduren in MySQL/MariaDB

Auf dem Datenbankserver können neben Tabellen und Ansichten auch Prozeduren gespeichert werden. Werden diese Prozeduren direkt auf der Konsole von MySQL aufgerufen, so können dort zum Teil Tabellenansichten ähnlich einer sonst in Base sichtbaren Ansicht («View») abgerufen werden. In Base sind diese Tabellenansichten nicht sichtbar. Ein Aufruf der Prozeduren unter **Extras → SQL** ist problemlos möglich, nur erfolgt leider keine Ausgabe der anzuzeigenden Inhalte. Um Inhalte der gespeicherten Prozeduren («Stored Procedures») zu sehen kann der folgende Weg beschritten werden:

- Statt einer Prozedur wie
**CREATE PROCEDURE AlleNamen()
BEGIN
 SELECT * FROM `Namen`;
END**
wird das Ergebnis der Prozedur in eine temporäre Tabelle geschrieben:
**CREATE PROCEDURE AlleNamen()
BEGIN
 DROP TEMPORARY TABLE IF EXISTS `TempNamen`;
 CREATE TEMPORARY TABLE `TempNamen` AS SELECT * FROM `Namen`;
END**
- Anschließend wird unter **Extras → SQL** die Prozedur aufgerufen:
CALL AlleNamen();
- Jetzt befinden sich die Ausgabedaten in einer temporären Tabelle, die nur für den aktuellen Nutzer von MySQL sichtbar ist. Die Ausgabedaten werden in einer Abfrage über
SELECT * FROM `TempNamen`
sichtbar.

Wie das Ganze etwas besser automatisiert ablaufen kann ist im Kapitel «Makros», Unterkapitel «MySQL-Datenbank mit Makros ansprechen» beschrieben.

Hinweis

Mit der direkten Verbindung lässt sich zur Zeit (LO 5.4.1.2) eine gespeicherte Prozedur wie die oben geschilderte nur einmal aufrufen. Danach kommt die Fehlermeldung

Commands out of sync; you can't run this command now.

Die JDBC-Verbindung zur MySQL- bzw. MariaDB ist davon nicht betroffen. Vor der Nutzung von gespeicherten Prozeduren sollte also wenigstens unter **Extras → SQL** zweimal hintereinander die Prozedur mit **CALL ...** testweise abgerufen werden.

PostgreSQL

Für die PostgreSQL-Datenbank gibt es einen direkten Treiber von LO, der von vornherein mit installiert wird. Damit der Kontakt auch sicher passt, zuerst aber eine kurze Anleitung zu den ersten Schritten nach der Installation von PostgreSQL.

Erstellen eines Nutzers und einer Datenbank

Die folgenden Schritte sind nach einer Installation über den Paketmanager in OpenSUSE erforderlich. Sie ähneln vermutlich denen unter anderen Betriebssystemen.

1. Dem Nutzer «postgres» muss zuerst ein Passwort zugewiesen werden. Das kann mit dem Administrationstool des Betriebssystems erfolgen.
2. Der Postgre-Server muss vom Administrator gestartet werden, z. B. **service postgresql start** oder **rcpostgresql start**.

3. Der Nutzer «postgres» muss sich mit **su postgres** auf der Konsole anmelden.
4. Ein einfacher Datenbanknutzer, hier «lotest», sollte mit Passwortzugang erstellt werden: **createuser -P lotest**
5. Damit der Datenbanknutzer anschließend auch auf die zu gründende Datenbank zugreifen kann, muss in der Datei /var/lib/pqsql/data/pg_hba.conf ein Eintrag geändert werden. In dieser Datei werden u.a. die Methoden zur Identifizierung der Nutzer auf verschiedenen Ebenen festgelegt. Die Methode, mit der LO-Base kommunizieren kann, ist die Methode «password», nicht, wie voreingestellt, die Methode «ident».
6. Der Systemnutzer «postgres» meldet sich über «psql» an:
psql -d template1 -U postgres
7. Der Systemnutzer «postgres» erstellt die Datenbank «libretest»:
CREATE DATABASE libretest;

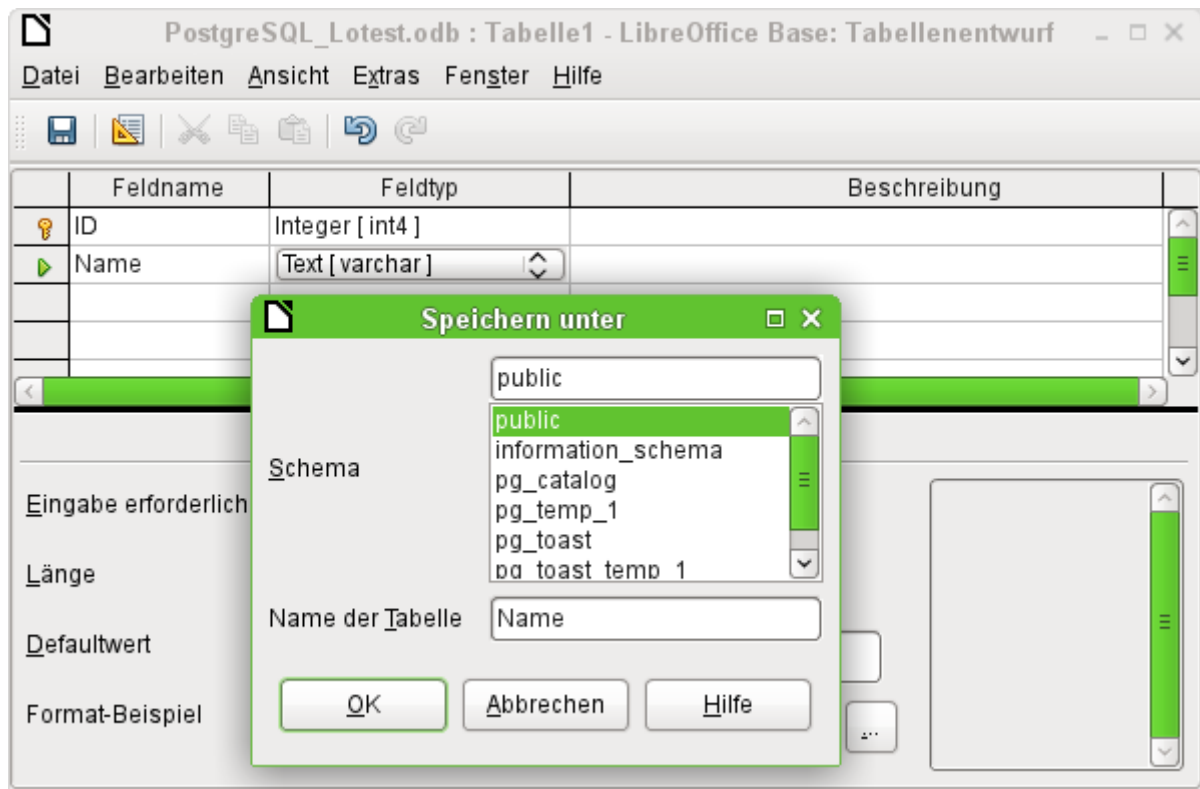
Direkte Verbindung zu Base

Im Assistenten wird der Eintrag «postgres» ausgewählt. Die Verbindungseinstellung geschieht über die Angabe des Datenbanknamens («dbname») und des Hostes. Unter bestimmten Umständen ist es erforderlich, statt der einfachen Angabe eines Hostes den kompletten Host einschließlich des vollständigen Domainnamens anzugeben.



Die Benutzer-Authentifizierung sieht genauso aus wie die unter MySQL.

Wird die Datenbank das erste Mal geöffnet, so erscheinen für den User erst einmal eine Unmenge an Tabellen aus dem Bereich «information_schema». Diese Tabellen sind ebenso wie die im Bereich «pg_catalog» für den normalen Nutzer nur lesbar, nicht schreibbar. Diese verschiedenen Bereiche bezeichnet PostgreSQL als «Schema». Der Nutzer legt neue Tabellen im Schema «public» an.

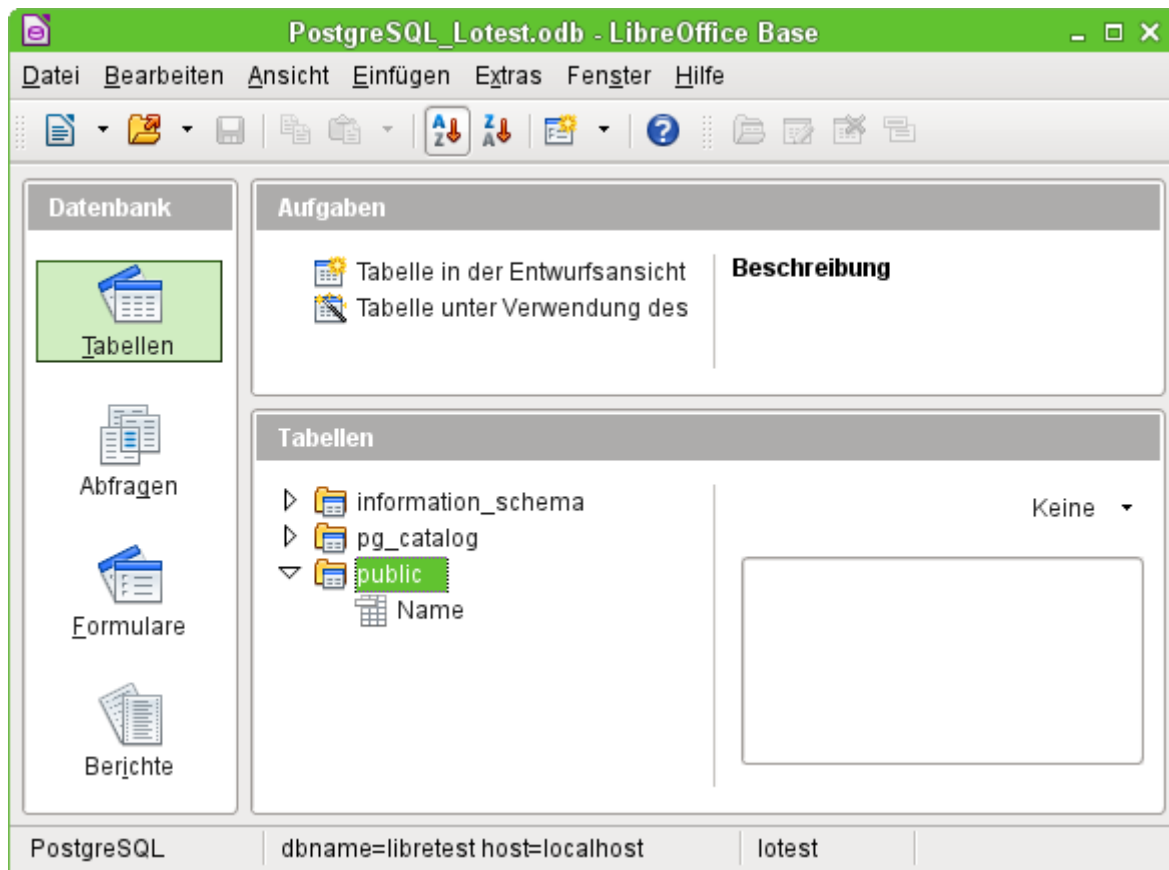


Der Dialog «Speichern unter» zeigt hier verschiedene Schema von PostgreSQL auf. Tatsächlich speicherbar ist allerdings nur in dem Schema «public», solange keine weitreichenderen Nutzerrechte für den entsprechenden Anmeldenamen vergeben wurden.

Hinweis

Werden Tabellen z.B. aus der internen HSQLDB nach PostgreSQL kopiert, so schlägt der Importassistent den einfachen Tabellennamen aus der HSQLDB, z.B. «Tabelle1», vor. Der Import mit diesem Namen schlägt allerdings fehl. Stattdessen muss vor den Namen die Schemabezeichnung ergänzt werden: aus «Tabelle1» wird «public.Tabelle1».

Bei der Erstellung von Tabellen kann es vorkommen, dass Base Datentypen vorschlägt, die die aktuelle PostgreSQL-Installation nicht verarbeiten kann. So werden standardmäßig die Felder mit dem Feldtyp «Text [character_data]» vorgeschlagen. Den kann PostgreSQL aber nicht verarbeiten. Eine Änderung auf den Typ «Text [varchar]» (siehe oben) schafft hier Abhilfe.



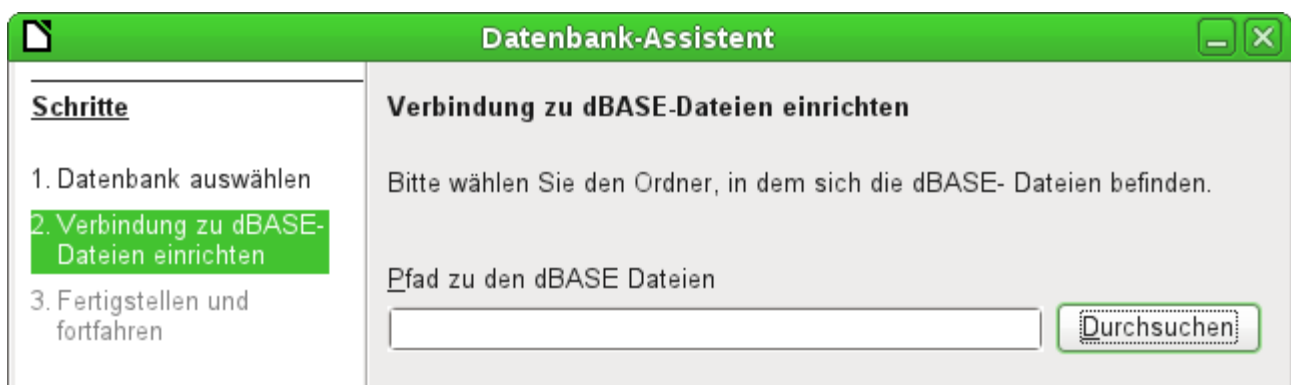
In der Tabellenübersicht von PostgreSQL erscheinen die unterschiedlichen Schemata. Im Schema «public» ist die abgespeicherte Tabelle «Name» zu sehen.

dBase-Datenbanken

Bei dBase handelt es sich um ein Datenbankformat, das alle Daten in einem beliebigen Verzeichnis in separaten Tabellen bereitstellt. Verknüpfungen zwischen den Tabellen müssen über die Programmstruktur erledigt werden. Dbase hat keine Möglichkeit, intern zu verhindern, dass z.B. in der Medien-Datenbank ein Medium gelöscht wird, der Verweis darauf aber weiterhin in der Tabelle zum Medienentleih erhalten bleibt.

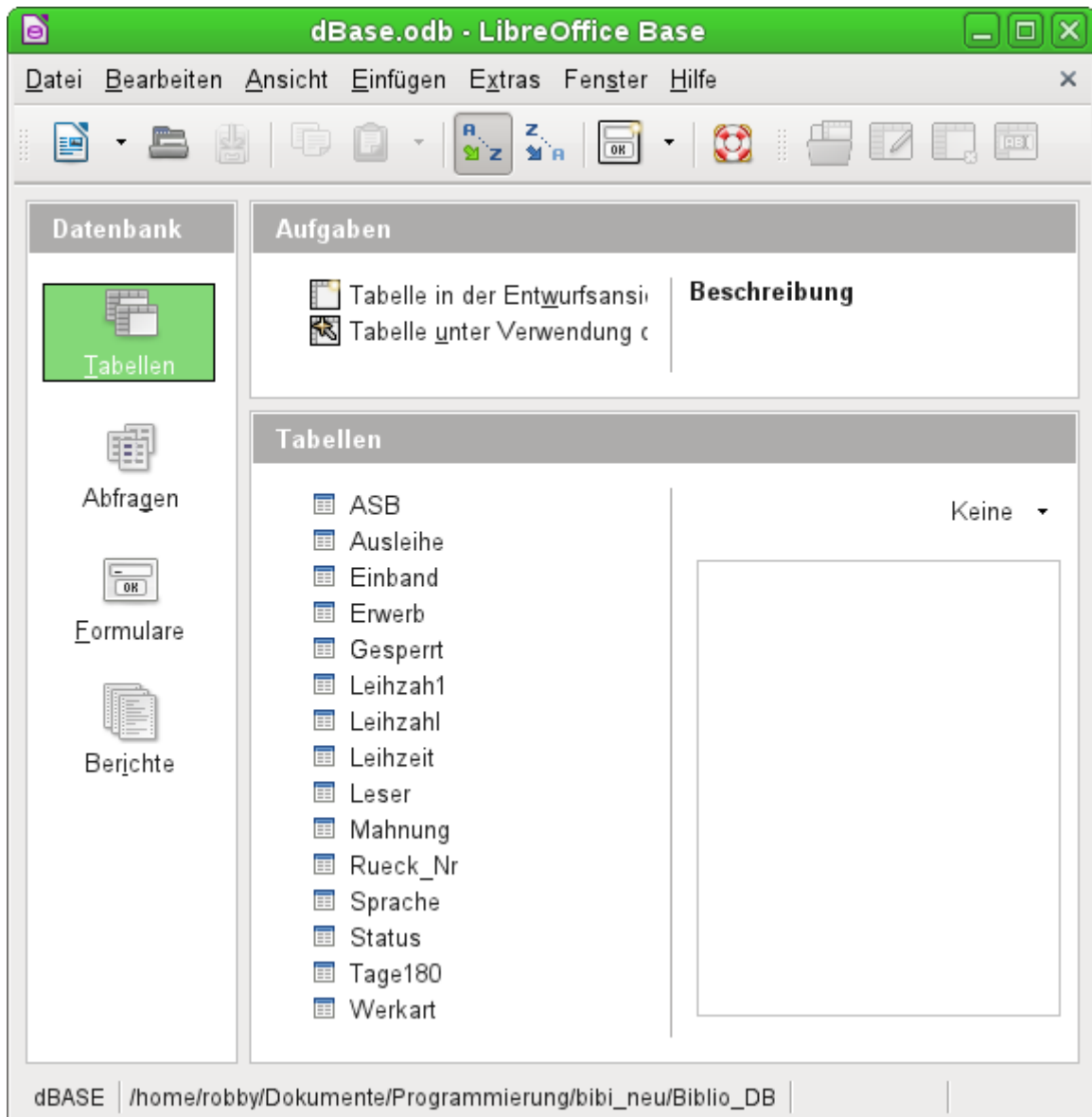
Hinweis

Zur Zeit werden nur dBase-Dateien berücksichtigt, die mit der in Kleinbuchstaben geschriebene Dateiendung *.dbf versehen sind. Andere Endungen wie z.B. *.DBF werden nicht ausgelesen. ([Bug 46180](#))



Die Verbindung wird zu einem Verzeichnis hergestellt. Alle *.dbf-Dateien, die sich in diesem Verzeichnis befinden, werden anschließend angezeigt und können über Formulare miteinander verbunden werden.

Tabellen in dBase haben kein unverwechselbares Feld für jeden Datensatz («Primärschlüssel»). Sie können also vom Prinzip her so beschrieben werden wie Tabellenblätter in Calc.



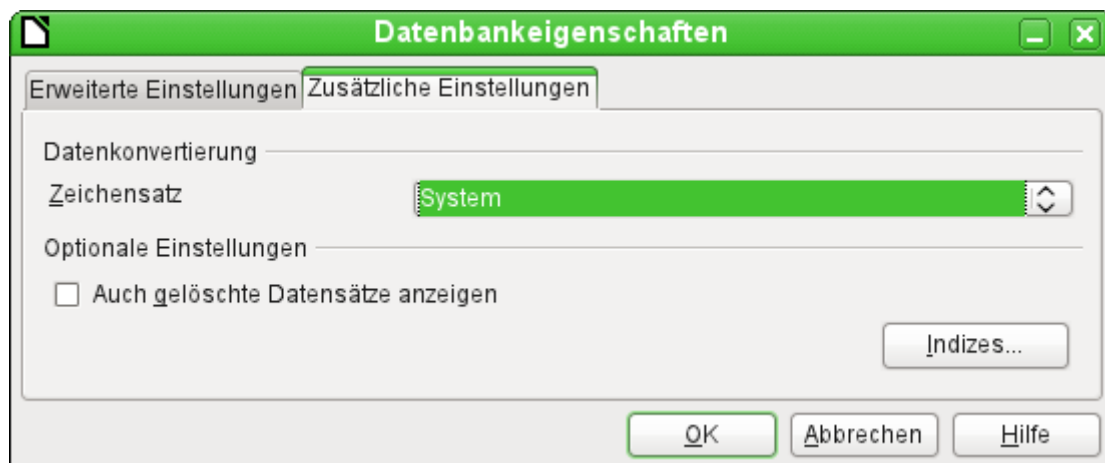
Tabellen können in Base neu erstellt werden. Sie werden dann einfach als neue Dateien in das angegebene Verzeichnis geschrieben.

Für neue Tabellen stehen deutlich weniger Feldtypen zur Verfügung als für die interne HSQLDB. Dabei sind noch einige Bezeichnungen der folgenden Abbildung als Verdoppelungen anzusehen.

	Feldname	Feldtyp	
	ID	Integer [INTEGER]	
	Vorname	Text [VARCHAR]	
	Nachname	Text [VARCHAR]	
		Ja/Nein [BOOLEAN]	
		Memo [LONGVARCHAR]	
		Dezimal [DECIMAL]	
		Dezimal [NUMERIC]	
		Integer [INTEGER]	
		Double [DOUBLE]	
		Double [DOUBLE]	
		Text [VARCHAR]	
		Datum [DATE]	
		Datum/Zeit [TIMESTAMP]	

Dbase bietet sich besonders an, wenn es um Weitergabe und vielfältige Verarbeitungsmöglichkeit von Daten geht. Schließlich können auch Tabellenkalkulationen dBase-Tabellen direkt einlesen.

Base übernimmt einfach die Codierung, die dem Betriebssystem entspricht. Alte dBase-Dateien weisen dadurch leicht Fehler beim Import von Sonderzeichen auf. Der Zeichensatz kann anschließend über **Bearbeiten → Datenbank → Eigenschaften → Zusätzliche Einstellungen** entsprechend korrigiert werden:

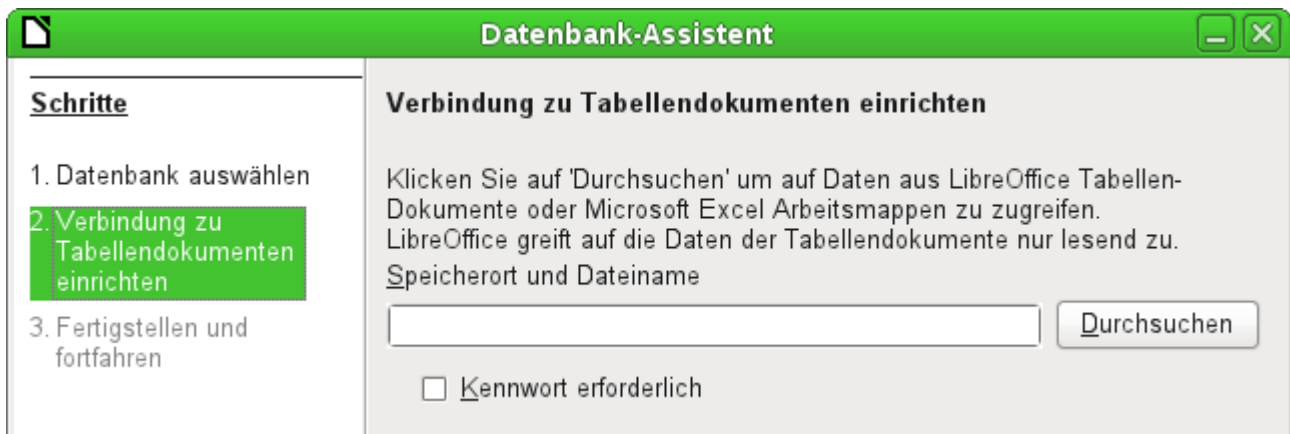


Hinweis

Der Importassistent für dBase hat Probleme, numerische Feldtypen und Ja/Nein-Felder automatisch richtig zu erkennen. ([Bug 53027](#)) Hier muss entsprechend nachgebessert werden.

Tabellendokumente

Tabellenquellen für Datenbanken können auch Tabellendokumente sein. Wird aber eine Calc-Tabelle als Grundlage genommen, so ist jedes Editieren der Tabelle ausgeschlossen. Selbst wenn das Calc-Dokument auch noch geöffnet wird, wird dort ein Schreibschutz gesetzt.

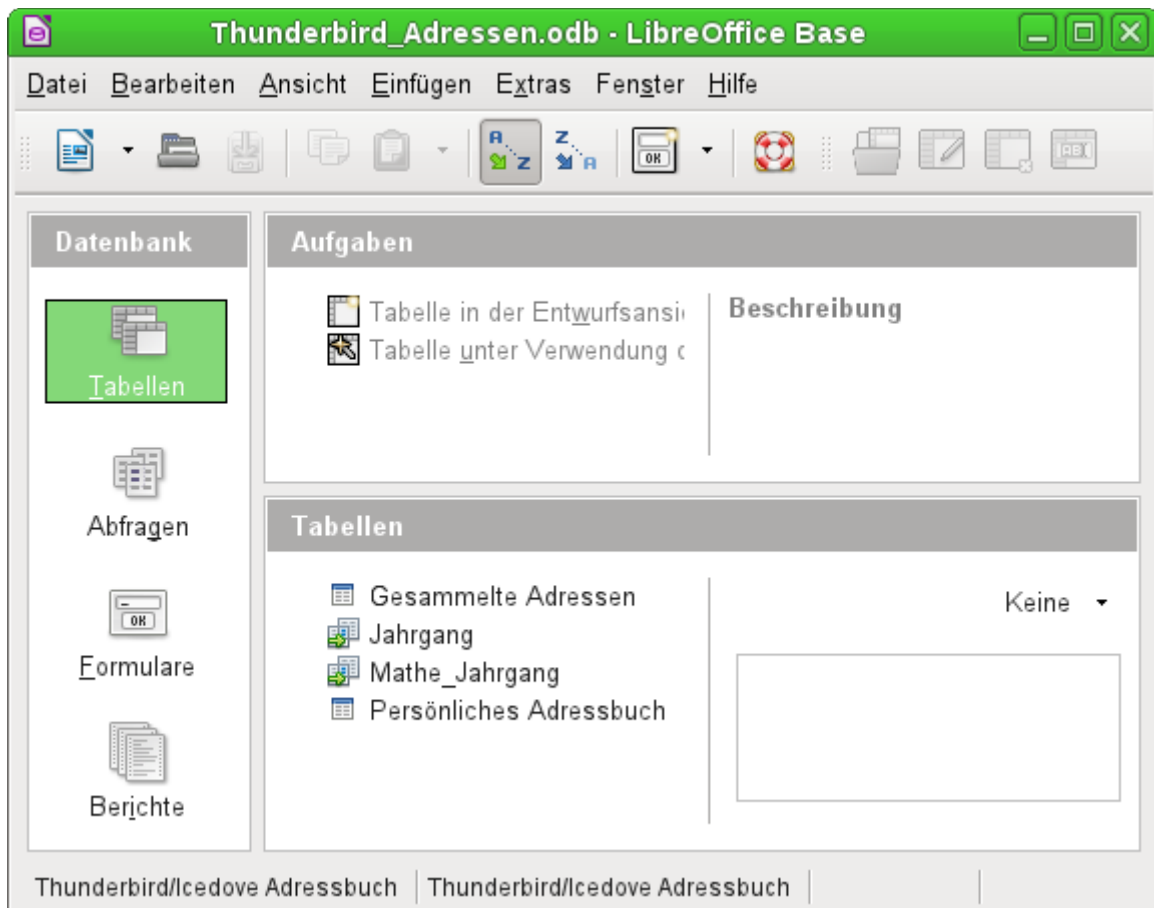


Die einzige Abfrage ist letztlich, an welcher Position das Tabellendokument liegt und ob die Tabelle passwortgeschützt ist. Base öffnet dann das Tabellendokument und übernimmt alle Tabellenblätter mit ihrem Inhalt. Aus der ersten Zeile bildet Base die Feldbezeichnungen, aus den Tabellenblattbezeichnungen die Tabellenbezeichnungen.

Beziehungen zwischen den Tabellen lassen sich in Base nicht erstellen, da Calc nicht Grundlage für eine relationale Datenbank ist. Auch lassen sich Abfragen nicht über eine Tabelle hinaus erstellen.

Thunderbird Adressbuch

Die Verbindung zu einem Adressbuch wie z.B. dem Thunderbird-Adressbuch sucht der Assistent automatisch. Er fragt lediglich nach einem Speicherort für die *.odb-Datei, die er erstellen soll.



Alle Tabellen werden dargestellt. Wie in Calc sind die Tabellen aber nicht editierbar. Base macht die Adressen damit nur für Abfragen und z. B. für die Verwendung in Serienbriefen zugänglich.

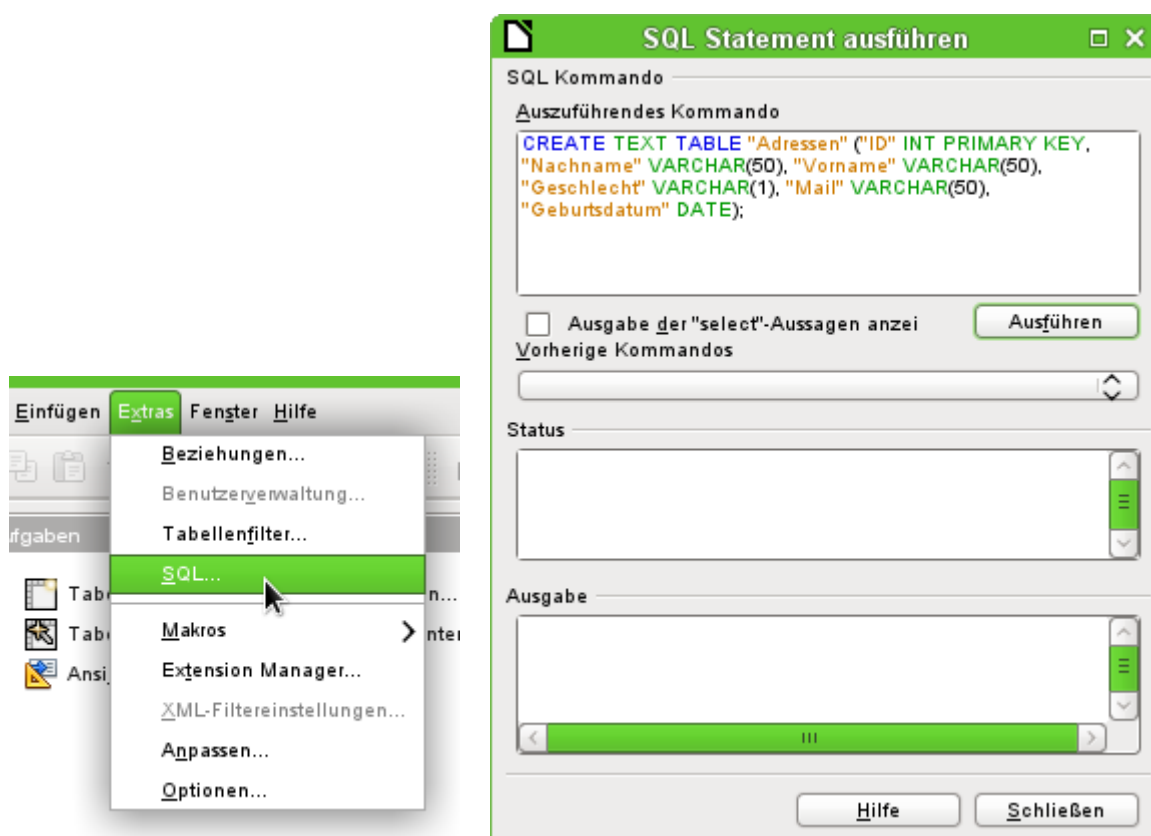
Texttabellen

In Base gibt es die Möglichkeit, eine komplette Datenbank mit Zugriff auf Texttabellen zu erstellen. Gleichzeitig können auch Texttabellen innerhalb einer internen Datenbank angesprochen werden.

Texttabellen innerhalb einer internen HSQLDB-Datenbank

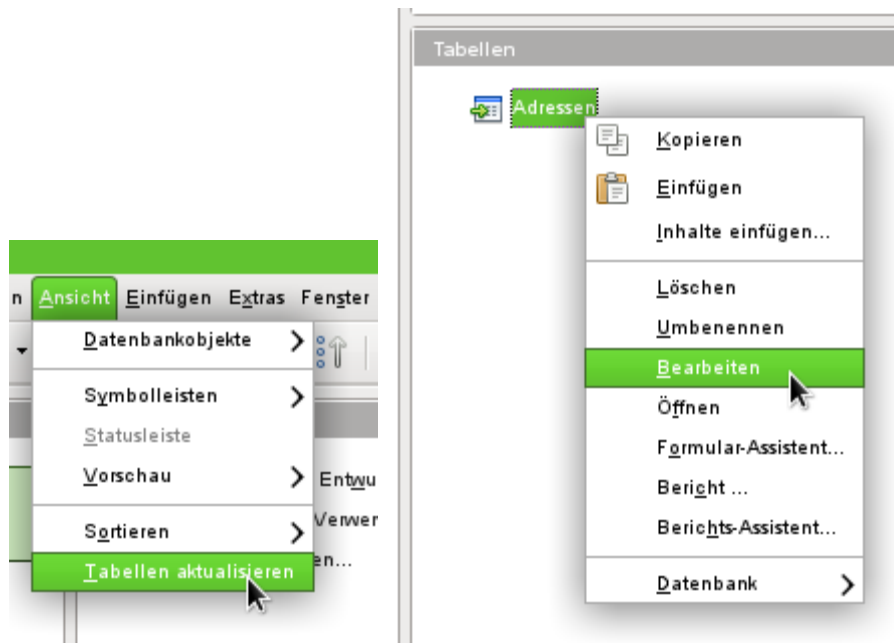
Ein gängiges Austauschformat zwischen Datenbanken ist das *.csv-Format. Daten werden hier in einer durch einfache Textbearbeitungsprogramme les- und schreibbaren Form gespeichert. Einzelne Felder werden durch Kommata getrennt. Enthält ein Feld Text mit einem Komma, so werden die Textfelder mit doppelten Anführungszeichen versehen. Ein neuer Datensatz beginnt nach einem Zeilenumbruch.

Soll zum Beispiel der Inhalt eines Adressbuches, das nicht direkt von den Treibern von Base unterstützt wird, eingelesen werden, so kann dies entweder über einen Import einer solchen *.csv-Datei erfolgen (hierzu ist gegebenenfalls ein Zwischenschritt über Calc notwendig) oder die Datei wird direkt als Texttabelle in die Datenbank eingefügt. Um dort bearbeitet werden zu können, benötigt die *.csv-Datei allerdings ein Feld, das als eindeutiges Feld («Primärschlüssel») erkennbar ist.

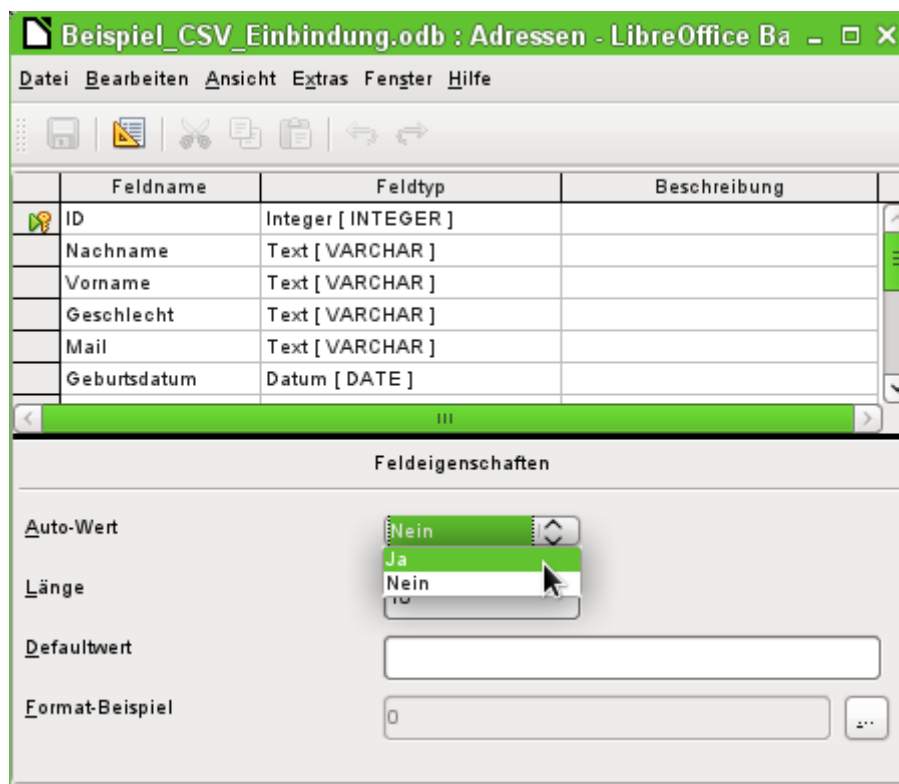


Der Zugriff zu einer Texttabelle kann nicht über die grafische Benutzeroberfläche erstellt werden.⁴ Stattdessen wird über **Extras → SQL** eine Texttabelle erstellt. Die Felder in der Texttabelle müssen dem entsprechen, was die Texttabelle in der entsprechenden Reihenfolge liefert. Das Feld "ID" muss z. B. positive Ganzzahlen enthalten, das Feld "Geburtsdatum" z. B. ein Datum in der Schreibweise Jahr – Monat – Tag.

⁴ Diesem Handbuch liegt die Datenbank «Beispiel_CSV_Einbindung.odt» bei.



Die Tabelle ist nicht direkt auf der Benutzeroberfläche sichtbar. Falls jetzt noch eine Ergänzung erfolgen soll, muss über **Ansicht → Tabellen aktualisieren** die Tabelle verfügbar gemacht werden. Das Symbol der Tabelle macht deutlich, dass es sich nicht um eine «normale» Tabelle der Datenbank handelt.



Die Tabelle wurde zum Bearbeiten geöffnet und das Primärschlüsselfeld auf einen automatisch hoch zählendes Feld umgestellt. Dies kann auch direkt mit der Eingabe des SQL-Befehls erfolgen:

```
CREATE TEXT TABLE "Adressen" ("ID" INT GENERATED BY DEFAULT AS IDENTITY, ...
```

Über **Extras → SQL** muss jetzt noch die Verbindung zur externen Texttabelle erstellt werden. Diese Texttabelle liegt im gleichen Verzeichnis wie die Datenbankdatei selbst.

```
SET TABLE "Adressen" SOURCE "Adressen.csv;encoding=UTF-8";5
```

ID	Nachname	Vorname	Geschlecht	Mail	Geburtsdatum
1	Löwe	Karl	m	loewe@l	13.02.87
2	Groß	Clärchen	w	gross@lil	17.10.79
3	Boss	Big	m	boss@lib	18.03.93
4	Bäcker	Käthe	w	kaethe@	01.07.01

Anschließend steht die Texttabelle ganz normal zur Eingabe zur Verfügung. Allerdings ist hierbei Vorsicht geboten:

- Die Texttabelle kann gleichzeitig auch von externen Textprogrammen oder Tabellenkalkulationen geöffnet und bearbeitet werden. Datenverlust ist dann nicht auszuschließen.
- Änderungen an bereits geschriebenen Datensätzen führen dazu, dass die entsprechende Zeile in der Originaldatei geleert wird und in der neuen Fassung an den Schluss der Tabelle angefügt wird. Die obige Ansicht präsentiert z.B. sauber 4 beschriftete Zeilen mit richtig sortierter ID. Die Originaldatei zeigt hingegen nach einer Änderung im 2. Datensatz die folgende Reihenfolge, geordnet nach der ID: 1, Leerzeile, 3, 4, 2

Für die Verbindung zur Textdatei stehen verschiedene Parameter zur Verfügung.

```
SET TABLE "Adressen" SOURCE "Adressen.csv;ignore_first=false;all_quoted=true;encoding=UTF-8";
```

Mit «ignore_first=true» wird die erste Zeile der Datei nicht eingelesen. Dies ist sinnvoll, wenn es sich dabei um die Feldbezeichnungen handelt. Der interne Standardwert der HSQLDB steht hier auf «false».

Standardmäßig werden von der HSQLDB Texte nur in doppelte Anführungszeichen gesetzt, wenn sie ein Komma enthalten, da das Komma die Standardtrennung der Felder ist. Soll jedes Feld in doppelte Anführungszeichen gefasst werden, so ist «all_quoted=true» zu setzen.

Zu weiteren Parametern siehe: http://hsqldb.org/doc/guide/ch09.html#set_table_source-section.

Mit

```
SET TABLE "Adressen" READONLY TRUE;
```

wird schließlich davor geschützt, dass in die Tabelle überhaupt etwas geschrieben wird. So steht dann die Tabelle als normale Adresstabelle wie eine Tabelle aus Adressbüchern z.B. von Mailprogrammen schreibgeschützt zur Verfügung. Die gesonderte Erstellung eines Schreibschutzes ist allerdings nur notwendig, wenn für die Tabelle erst einmal ein Primärschlüssel erstellt wurde.

Hinweis

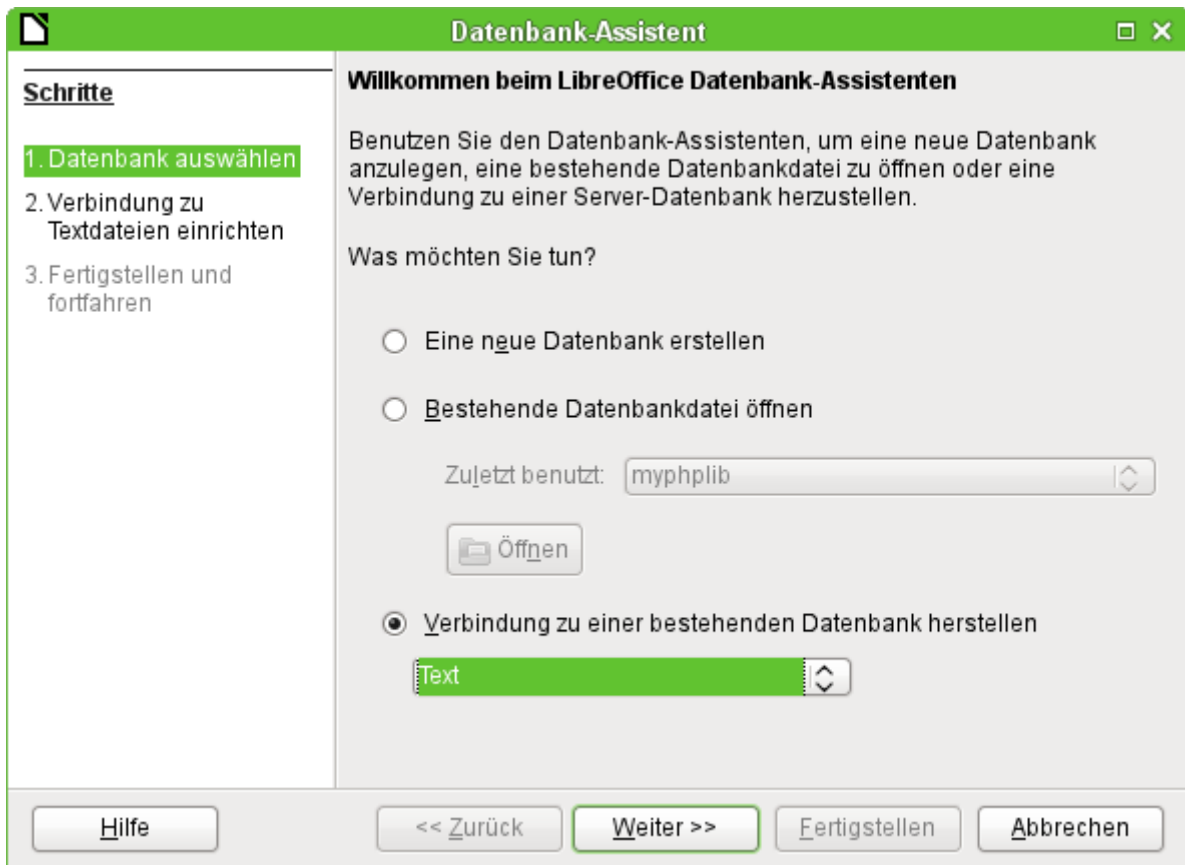
Texttabellen innerhalb der internen HSQLDB scheinen zur Zeit die einzige Möglichkeit zu sein, Zeilenumbrüche aus externen Tabellen wie z.B. einer Calc-Tabelle beim Import in eine HSQLDB zu erhalten:

Die Calc-Tabellen müssen jeweils als einzelne *.csv-Dateien exportiert werden. Zeilenköpfe sind zu entfernen. Für jede *.csv-Datei wird eine Texttabelle erstellt. Mehrzeilige Texte müssen in LONGVARCHAR-Spalten abgelegt sein. Anschließend werden diese Textdateien wieder kopiert und als normale Tabellen in die HSQLDB-Datenbank eingefügt.

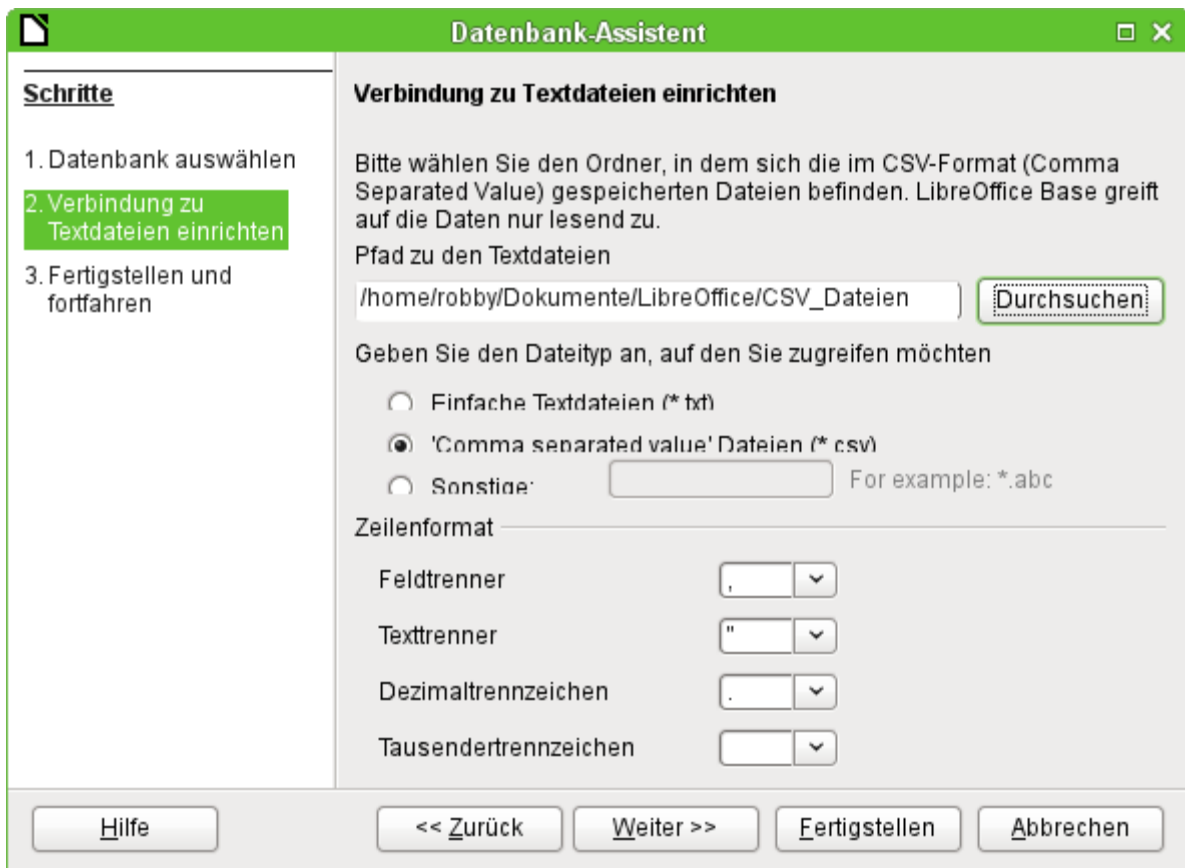
⁵ Die Angabe «encoding=UTF-8» funktioniert bei vielen Systemen. Eventuell muss auch statt «UTF-8» «ansi» gewählt werden.

Texttabellen als Grundlage für eine eigenständige Datenbankdatei

Wie im vorhergehenden Beispiel werden als Datengrundlage in dem folgenden Beispiel *.csv-Dateien genutzt. Über Base wird ein Verzeichnis, in dem die *.csv-Dateien liegen als Datenverzeichnis eingebunden.

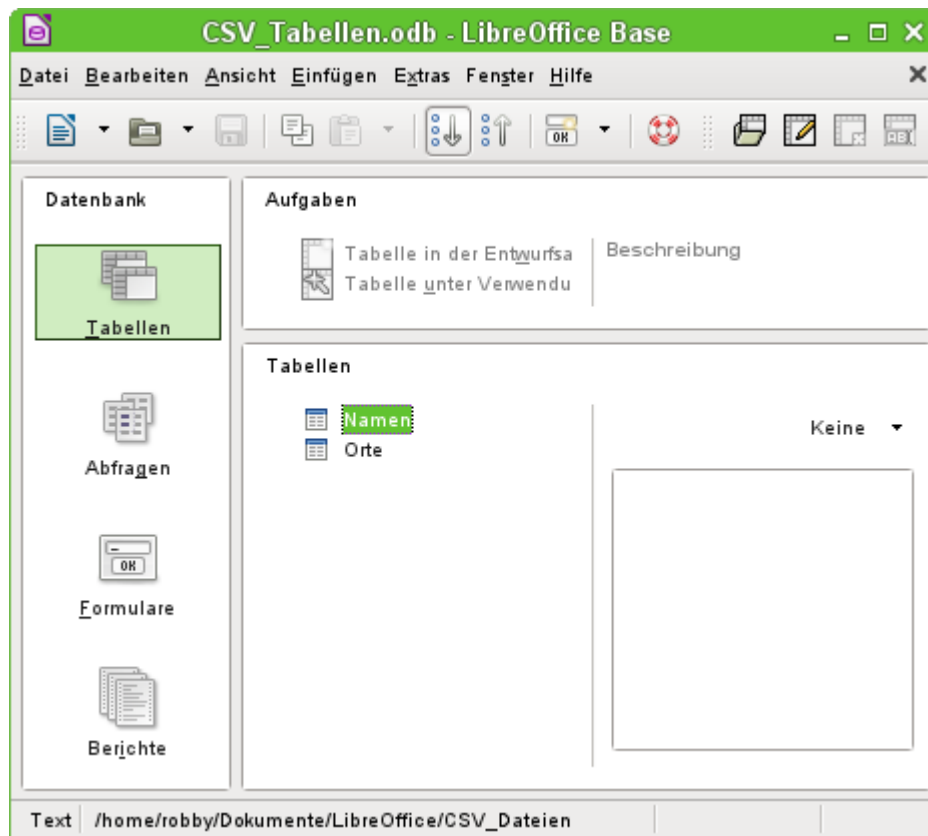


Startpunkt ist die Verbindung zu einer bestehenden Datenbank. Hier ist als Format «Text» gewählt.



Der Pfad zu den Textdateien wird ausgesucht. Innerhalb des Verzeichnisses werden später alle dem Dateityp entsprechenden Dateien aufgelistet. Für das *.csv-Format ist hier die zweite Option gewählt.

Bereits bei der Auswahl ist schon deutlich beschrieben: Auf die Tabellen kann nur lesend zugegriffen werden, nicht schreibend.



In der Tabellenansicht sind dann alle Tabellen mit dem Dateinamen aus dem aufgezeigten Verzeichnis ohne die Dateinamenserweiterung zu sehen. Die Aufgaben zum Erstellen von Tabellen stehen nicht zur Verfügung. Die Tabellen selbst sind nur lesbar, nicht schreibbar.

Auch der Zugriff auf die Tabellen mit Abfragen ist beschränkt auf eine Tabelle und keine Funktionen.

Wird eine solche Datenbank genutzt, um einmal kurz in einer *.csv-Datei nach entsprechenden Datensätzen zu suchen oder um eine *.csv-Datei in eine andere Datenbankdatei über die Kopierfunktion einzufügen, so erfüllt diese Text-Datenbank ihren Sinn. Die entsprechende *.csv-Datei wird nur in das definierte Verzeichnis geschoben und kann dann direkt durchsucht oder kopiert werden. Für weitergehende Datenbankaufgaben ist diese Text-Datenbank wohl nicht gedacht.

Firebird

Die etwas in die Jahre gekommene alte HSQLDB-Version der internen Datenbank soll durch eine interne Firebird-Datenbank ersetzt werden. Seit Version LO 4.2.* ist die interne Firebird-Datenbank erst einmal nur als «experimentelle Funktion» zusätzlich wählbar. Das liegt einfach daran, dass längst noch nicht alle Funktionen richtig integriert wurden. Um aber zu sehen, was Firebird leisten kann, hier jetzt also die Verbindung zu einer externen Firebird-Datenbank zum Testen.

Da die Dokumentation dazu nicht ganz so umfangreich ist wie z.B. zu MySQL oder PostgreSQL zuerst einmal die wichtigsten Schritte bei der Installation.

Erstellen eines Nutzers und einer Datenbank

Unter Linux stehen für Firebird entsprechende Pakete in der Paketauswahl bereit. Hier muss nach der Installation eigentlich nur noch der Server eingeschaltet werden. Die folgenden Schritte haben unter OpenSUSE 12.3 zu einer funktionierenden Datenbank mit Firebird geführt:

1. **sysdba** ist der Nutzernamen für den Admin-Account. Das Standardpasswort ist **masterkey**. Dies sollte in Produktivumgebungen geändert werden.

2. Eine Passwortänderung geht im Terminal über
gsec -user sysdba -pass masterkey -mo sysdba -pw neuespasswort
3. Notwendig für die Installationsschritte bis zur funktionierenden Datenbank sind Administratorrechte auf dem Rechner. Dem Systemuser «firebird» sollte ein Passwort zugeordnet werden.
4. Der Nutzer «firebird» muss sich mit
su firebird
 auf der Konsole anmelden.
5. Ein neuer Nutzer sollte gegründet werden - hier gezeigt mit altem Standardpasswort des sysdba:
gsec -user sysdba -pass masterkey -add lotest -pw libre
 Damit ist ein neuer Nutzer mit dem Namen **lotest** und dem Passwort **libre** erstellt.
6. Danach wird, immer noch als SuperUser, eine Datenbank für den neuen Nutzer erstellt. Hierzu wird das Hilfsprogramm **isql-fb** aufgerufen.
isql-fb
 Es erscheint die folgende Meldung:
Use CONNECT or CREATE DATABASE to specify a database
 direkt gefolgt von der Eingabeaufforderung **SQL>**. Hier entsprechend passende Einträge:
SQL> CREATE DATABASE 'libretest.fdb'
CON> user 'lotest' password 'libre';

Werden die Angaben als Systemadministratortor Root getätigt, so ist die Datenbank nicht dem richtigen Benutzer für den Netzbetrieb zugeordnet. Die Datenbank muss als Nutzer und als Gruppe «firebird» eingetragen bekommen. Sonst klappt der Kontakt später nicht.

Firebird-Verbindung über JDBC

Zuerst muss das Jar-Archiv in LO eingebunden werden. Irritierend ist, dass es kein Archiv mit einer Bezeichnung firebird-*.jar gibt. Der aktuelle JDBC-Treiber ist auf der Seite <http://www.firebirdsql.org/en/jdbc-driver/> zu finden. Der Treiber beginnt mit der Bezeichnung Jaybird...

Aus der *.zip-Datei muss nur das Archiv **jaybird-full-2.2.8.jar** (oder aktuellere Version) kopiert und entweder in den Java-Pfad der Installation gelegt oder als Archiv direkt in LO eingebunden werden. Siehe dazu auch die entsprechende Einbindung bei MySQL.

Für die Angaben bei der JDBC-Installation sind folgende Parameter wichtig:

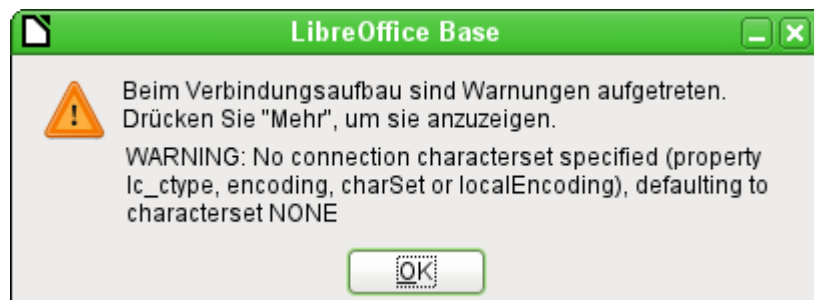
JDBC URL **jdbc:firebirdsql://host[:port]/<path_or_alias>**

Treiberbezeichnung: **org.firebirdsql.jdbc.FBDriver**

In obigen Beispiel wurde dies zu der URL

jdbc:firebirdsql://localhost/libretest.fdb?charSet=UTF-8

Ohne Zeichensatzangabe erscheint die folgende Fehlermeldung:



Bei der Erstellung von Tabellen muss darauf geachtet werden, dass die Formatierung der entsprechenden Felder («Feldeigenschaften») von Beginn an stimmt. Bei allen Zahlenwerten setzt sonst LO als Default-Format merkwürdigerweise ein Währungsfeld ein.

Nachträgliche Änderungen z.B. der Feldeigenschaften von Feldern einer Tabelle sind nicht möglich, wohl aber die Erweiterung der Tabelle oder das Löschen der Felder.

Firebird-Verbindung über ODBC

Aus dem Netz muss zuerst der passende ODBC-Treiber heruntergeladen werden: <http://www.firebirdsql.org/en/odbc-driver/>. Dieser Treiber besteht meist aus einer einfachen Datei, der libOdbcFb.so. Diese Datei wird meist mit Administratorrechten in einen vom System allgemein zugänglichen Pfad gelegt. Sie muss ausführbar sein.

In den für das System notwendigen Dateien odbcinst.ini und odbc.ini müssen Einträge ähnlich den folgenden existieren:

odbcinst.ini

```
[Firebird]
Description = Firebird ODBC driver
Driver64 = /usr/lib64/libOdbcFb.so
```

odbc.ini

```
[Firebird-libretest]
Description = Firebird database libreoffice test
Driver = Firebird
Dname = localhost:/srv/firebird/libretest.fdb
SensitiveIdentifier = Yes
```

Diese beiden Dateien liegen bei einem Linux-System im Verzeichnis /etc/unixODBC. Die Variable SensitiveIdentifier muss auf jeden Fall auf «Yes» stehen, damit die Eingabe in Tabellen funktioniert, deren Namen und Feldbezeichnungen nicht in Großbuchstaben angegeben sind.

Zur Erstellung der Verbindung von LO mit der entsprechenden Datenquelle siehe [Die ODBC-Verbindung](#).

Die ODBC-Verbindung ist leider zur Zeit nur sehr eingeschränkt funktionstüchtig. Beim Erstellen von Tabellen stürzt LO ab. Beim Eingeben von Werten in ein Integer-Feld wird bei Linux-64-bit-Versionen an die Datenbank ein leeres Feld weitergegeben (<http://tracker.firebirdsql.org/browse/ODBC-141>). Hier klappt die Zusammenarbeit mit dem ODBC-Treiber nicht. Da funktioniert der JDBC-Treiber eindeutig besser.

SQLite

SQLite-Datenbanken bestehen aus einer einzigen Datei. Diese Datei kann irgendwo im Dateisystem gespeichert werden und muss nur über einen entsprechenden Treiber mit Base verbunden werden.

Erstellen einer Datenbank

Das Erstellen einer Datenbank beschränkt sich darauf, eine leere Datei in einem Verzeichnis abulegen. Es ist nicht einmal erforderlich, der leeren Datei eine bestimmte Dateiendung zu geben. Unter Linux reicht es, eine einfache Textdatei zu erstellen. Die Datei wird dann mit dem Namen libretest.db versehen. Bei der Erstellung kann es passieren, dass bereits Absätze vorhanden sind. Diese Absätze sollten gelöscht werden, so dass der Inhalt der Datei 0 Byte groß ist.

Natürlich lässt sich eine leere Datei auch auf der Konsole erstellen, die bei der Installation des Treibers mitinstalliert wird:

```
sqlite3 libretest.db
```

erstellt die leere Datenbankdatei. Dabei wird dann die SQLite-Konsole geöffnet. Die Datei ist noch nicht in dem Verzeichnis sichtbar, in dem der Befehl ausgeführt wird. Auf der Konsole wird jetzt nur noch

```
.exit
```

einggegeben und die leere Datei wird geschrieben.

SQLite-Verbindung über ODBC

Unter Linux wird der ODBC-Treiber des Systems über die Paketverwaltung installiert. Die Einstellungen in der `odbcinst.ini` werden dabei automatisch vorgenommen.

`odbcinst.ini`

```
[SQLITE3]
Description=SQLite ODBC 3.X
Driver=/usr/lib64/libsqlite3odbc.so
Setup=/usr/lib64/libsqlite3odbc.so
Threading=2
FileUsage=1
UsageCount=1
```

In der `odbc.ini` wird dann die Verbindung zu der vorher erstellten Datei vorgenommen. Die Datei wird anschließend über die entsprechende Bezeichnung, hier `Sqlite3-libretest`, über ODBC mit Base verbunden.

`odbc.ini`

```
[Sqlite3-libretest]
Description = SQLite database libreoffice test
Driver= SQLITE3
Database= /home/<nutzernamen>/Dokumente/LibreOffice/libretest.db
```

Weitere SQLite-Verbindungen

Es existieren zwar JDBC-Treiber für SQLite. Der Zugriff auf eine SQLite-Datenbankdatei bricht unter LibreOffice-Base aber leider mit der Fehlermeldung ab:

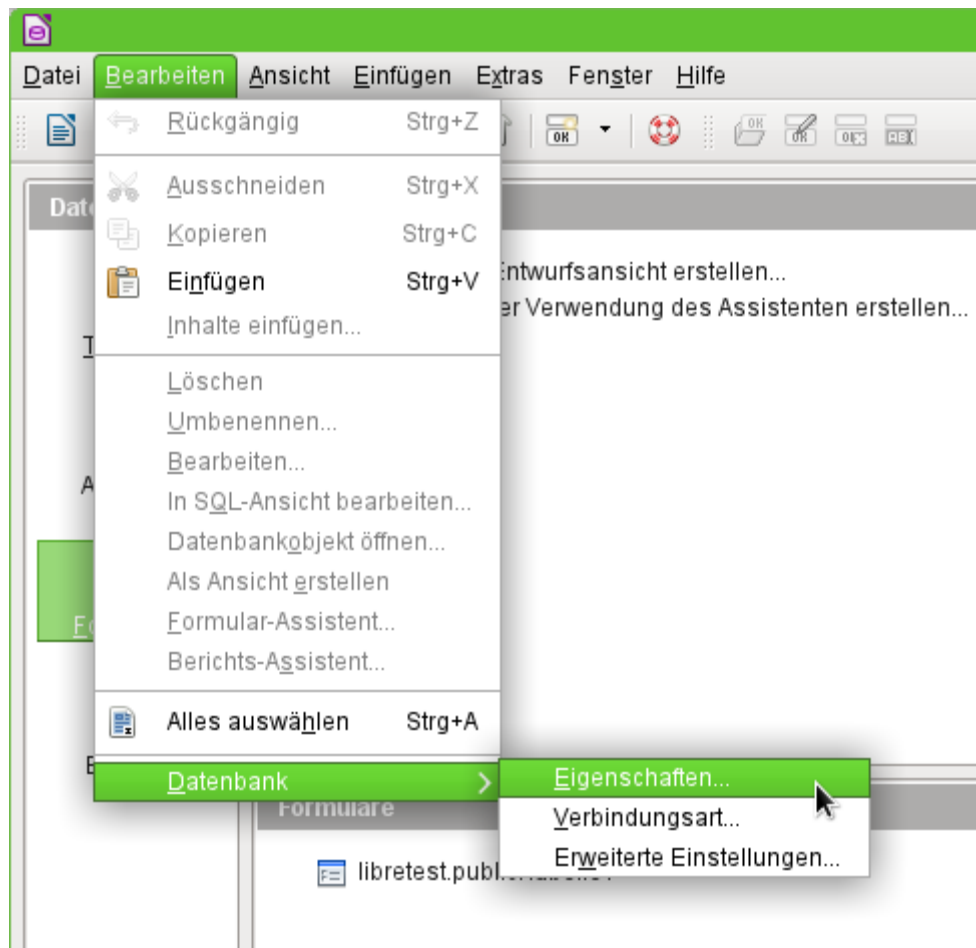
```
SQLite only supports TYPE_FORWARD_ONLY Cursors
```

Auch eine direkte Verbindung zu SQLite gibt es für LibreOffice bisher nicht.

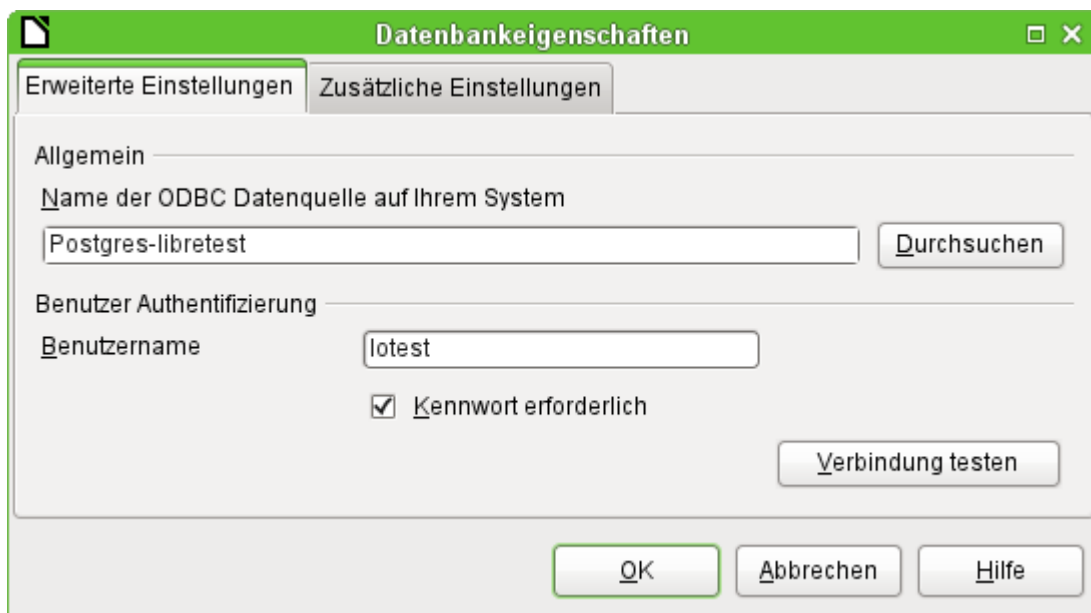
Nachträgliche Bearbeitung der Verbindungseigenschaften

Vor allem bei Verbindungen zu externen Datenbanken kann es hin und wieder vorkommen, dass eine Verbindung nicht wie gewünscht einwandfrei funktioniert. Mal ist der Zeichensatz nicht korrekt, ein anderes Mal funktionieren Unterformulare nicht einwandfrei oder es ist einfach etwas an den grundsätzlichen Verbindungsparametern zur Datenbank geändert worden.

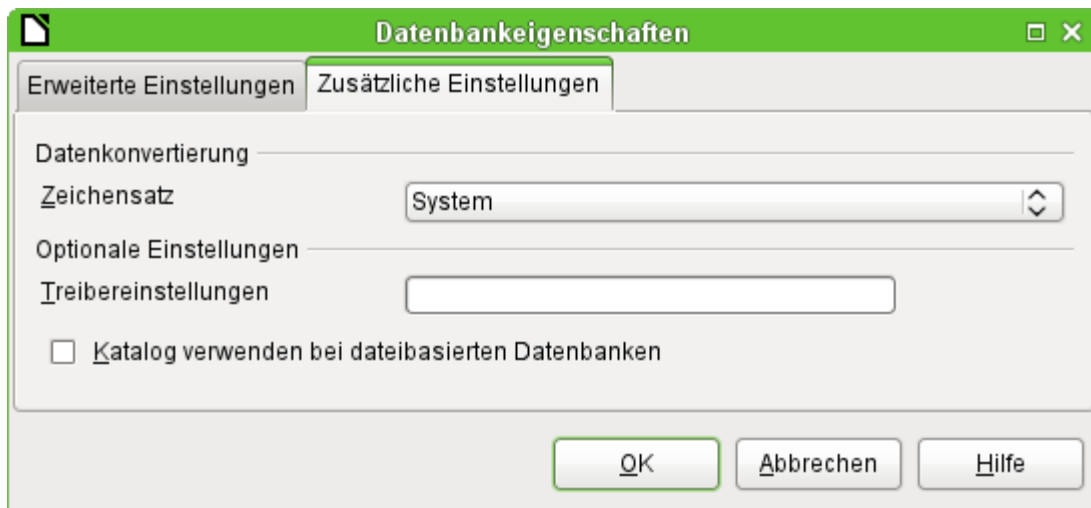
Die folgenden Screenshots zeigen die Möglichkeit auf, wie z.B. die Verbindungsparameter zu einer externen PostgreSQL-Datenbank im Nachhinein geändert werden können.



Über **Bearbeiten** → **Datenbank** werden die Einstellungsmöglichkeiten «Eigenschaften», «Verbindungsart» und «Erweiterte Einstellung» aufgerufen.

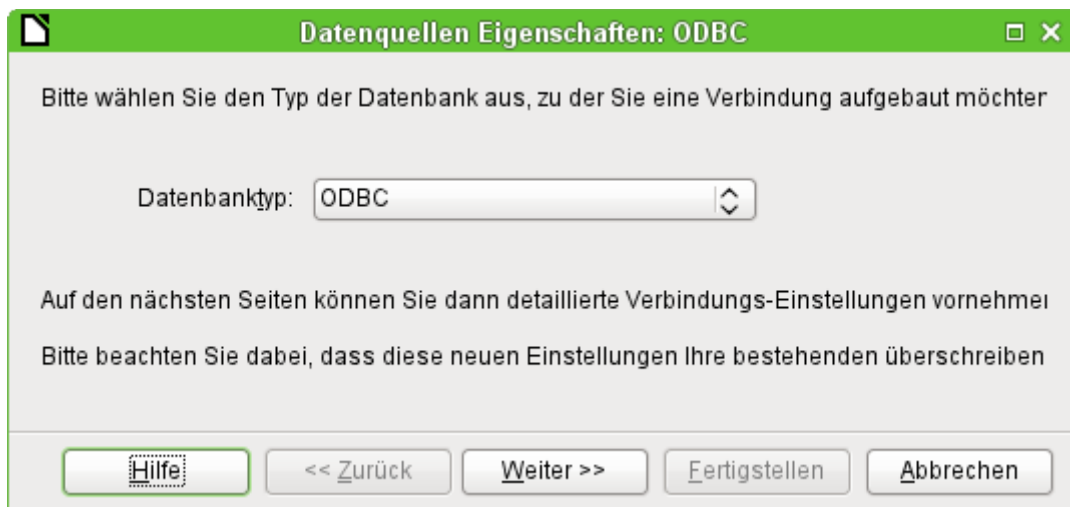


Falls sich der Name der Datenquelle geändert hat, kann dies hier geändert werden. Bei einer Verbindung über ODBC wird ja der Name, mit dem die Datenbank aufgerufen werden kann, in der `odbc.ini` festgelegt. Der Name ist in der Regel nicht gleich dem eigentlichen Datenbanknamen in PostgreSQL.

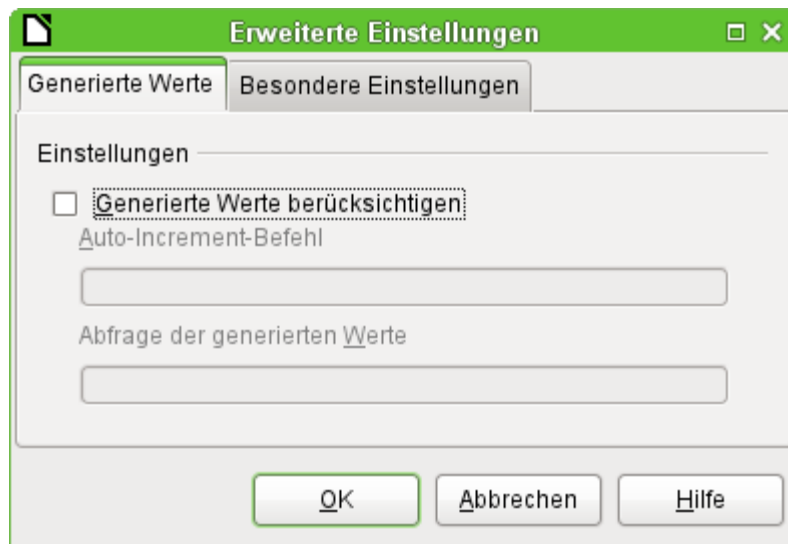


Gibt es Probleme mit dem Zeichensatz? Diese Probleme können eventuell in dem zweiten Reiter der Datenbankeigenschaften gelöst werden.

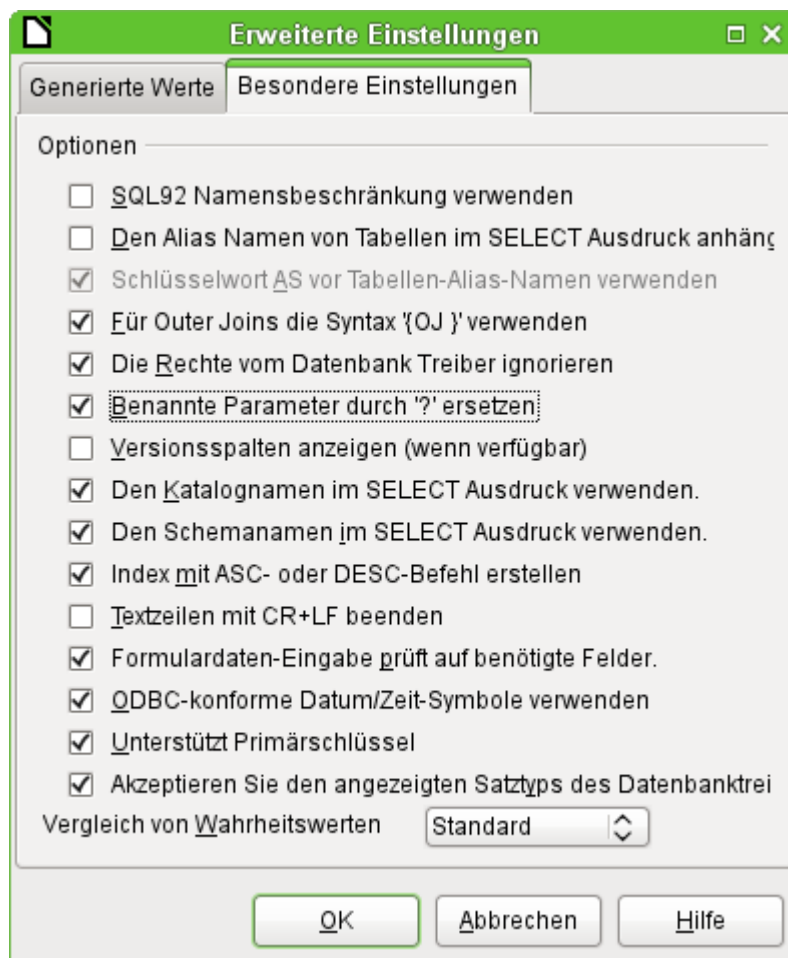
Auch eine zusätzliche spezielle Einstellung des Treibers ist möglich, wenn eventuell ein Parameter ausgeführt werden soll, der zur Zeit nicht bereits in der `odbc.ini` steht.



Wird die Verbindungsart ausgewählt, so kann der gesamte Kontakt zur Datenquelle geändert werden. Die darauf folgenden Schritte sind gleich denen des Datenbankassistenten ab Schritt 2. So kann z.B. von einer ODBC- auf eine JDBC- oder gar eine direkte Verbindung über die internen Treiber von LO gewechselt werden. Dies ist vor allem dann sinnvoll, wenn zuerst einmal ausgetestet werden soll, welche Verbindungsart zu dem eigenen Projekt am besten passt.



Je nach Datenbanksystem gibt es unterschiedliche Befehle um automatisch hoch zählende Werte zu erzeugen. Soll so etwas erreicht werden, was der Treiber zur Zeit nicht direkt ermöglicht, dann ist an dieser Stelle Handarbeit angesagt. Dabei ist sowohl ein Befehl für das Erstellen eines Auto-Wert-Feldes als auch ein Befehl zur Abfrage des zuletzt erstellten Wertes notwendig.



Die über **Bearbeiten → Datenbank → Erweitere Einstellungen** erreichbaren besonderen Einstellungen beeinflussen das Zusammenspiel von externer Datenbank und Base auf die unterschiedliche Art und Weise. Manche Einstellungen sind bereits ausgegraut, da sie für die zugrundeliegende Datenbank nicht änderbar sind. In dem obigen Beispiel wurde **Benannte Parameter durch '?' ersetzen** aus-

gewählt. Es hatte sich gezeigt, dass sonst bei PostgreSQL die Weitergabe von Werten von einem Hauptformular zum Unterformular nicht funktionierte. Erst mit dieser Einstellung funktionierten die weitergehenden Formularkonstruktionen aus dem Kapitel «Formulare» dieses Handbuches korrekt.

Tabellen

Allgemeines zu Tabellen

Daten werden in Datenbanken innerhalb von Tabellen gespeichert. Wesentlicher Unterschied zu Tabellen innerhalb einer einfachen Tabellenkalkulation ist, dass die Felder, in die geschrieben wird, klar vordefiniert werden müssen. Eine Datenbank erwartet innerhalb einer Textspalte keine Zahlen-eingaben, mit denen sie rechnen kann. Sie stellt die Zahlen dann zwar dar, geht aber von einem Wert «0» für diese Zahlen aus. Auch Bilder lassen sich nicht in jeder Feldform ablegen.

Welche Datentypen es im einzelnen gibt, kann bei der grafischen Benutzeroberfläche dem Tabelleneditor entnommen werden. Details dafür im Anhang dieses Handbuches.

Einfache Datenbanken beruhen lediglich auf einer Tabelle. Hier werden alle Daten unabhängig davon eingegeben, ob eventuell mehrfache Eingaben des gleichen Inhaltes gemacht werden müssen. Eine einfache Adressensammlung für den Privatgebrauch lässt sich so erstellen. Die Adressensammlung einer Schule oder eines Sportvereins würde aber so viele Wiederholungen bei Postleitzahl und Ort aufweisen, dass diese Tabellenfelder in eine oder sogar 2 separate Tabellen ausgelagert würden. Die Auslagerung von Informationen in andere Tabellen hilft:

- laufend wiederkehrende Eingaben gleichen Inhaltes zu reduzieren
- Schreibfehler bei diesen laufenden Eingaben zu vermeiden
- Daten besser nach den ausgelagerten Tabellen zu filtern

Bei der Erstellung der Tabellen sollte also immer wieder überlegt werden, ob eventuell viele Wiederholungen vor allem von Texten oder Bildern (hier stecken die Speicherfresser) in den Tabellen vorkommen. Dann empfiehlt sich eine Auslagerung der Tabelle. Wie dies prinzipiell geht ist in der Einführung im Kapitel «[Eine einfache Datenbank – Testbeispiel im Detail](#)» beschrieben.

Hinweis

In einer Datenbank, in der mehrere Tabellen in Beziehung zueinander stehen ("relationale Datenbank"), wird angestrebt, möglichst wenige Daten in einer Tabelle doppelt einzugeben. Es sollen «Redundanzen» vermieden werden.

Dies kann erreicht werden,

- indem Tabellenfelder nicht zu viel Inhalt auf einmal speichern (z.B. nicht eine komplette Adresse mit Straße, Hausnummer, Postleitzahl und Ort), sondern Straße, Hausnummer, Postleitzahl und Ort getrennt,
- doppelte Angaben in einem Feld vermieden werden (z.B. Postleitzahl und Ort aus einer Tabelle in eine andere auslagern)

Dieses Vorgehen wird als Normalisierung von Datenbanken bezeichnet.

Beziehungen von Tabellen

Anhand der Beispieldatenbanken «Medien_ohne_Makros» bzw. «Medien_mit_Makros» werden in diesem Handbuch viele Schritte möglichst detailliert erklärt. Bereits die Tabellenkonstruktion dieser Datenbank ist sehr umfangreich, da sie neben der Aufnahme von Medien in eine Mediothek auch die Ausleihe von Medien abdeckt.

Beziehungen zwischen Tabellen allgemein

Tabellen in der internen Datenbank haben immer ein unverwechselbares, einzigartiges Feld, den Primärschlüssel. Dieses Feld muss definiert sein, bevor überhaupt Daten in die Tabelle geschrieben werden können. Anhand dieses Feldes können bestimmte Datensätze einer Tabelle ermittelt werden.

Nur in Ausnahmefällen wird ein Primärschlüssel auch aus mehreren Feldern zusammen gebildet. Dann müssen diese Felder zusammen einzigartig sein.

Tabelle 2 kann ein Feld besitzen, das auf die Inhalte von Tabelle 1 hinweist. Hier wird der Primärschlüssel aus Tabelle 1 als Wert in das Feld der Tabelle 2 geschrieben. Tabelle 2 hat jetzt ein Feld, das auf ein fremdes Schlüsselfeld verweist, also einen Fremdschlüssel. Dieser Fremdschlüssel existiert in Tabelle 2 neben dem Primärschlüssel.

Je mehr Tabellen in Beziehung zueinander stehen, desto komplexer kann der Entwurf sein. Das folgende Bild zeigt die gesamte Tabellenstruktur der Beispieldatenbank in einer Übersicht, die von der Größe her die Seite dieses Dokuments sprengt:

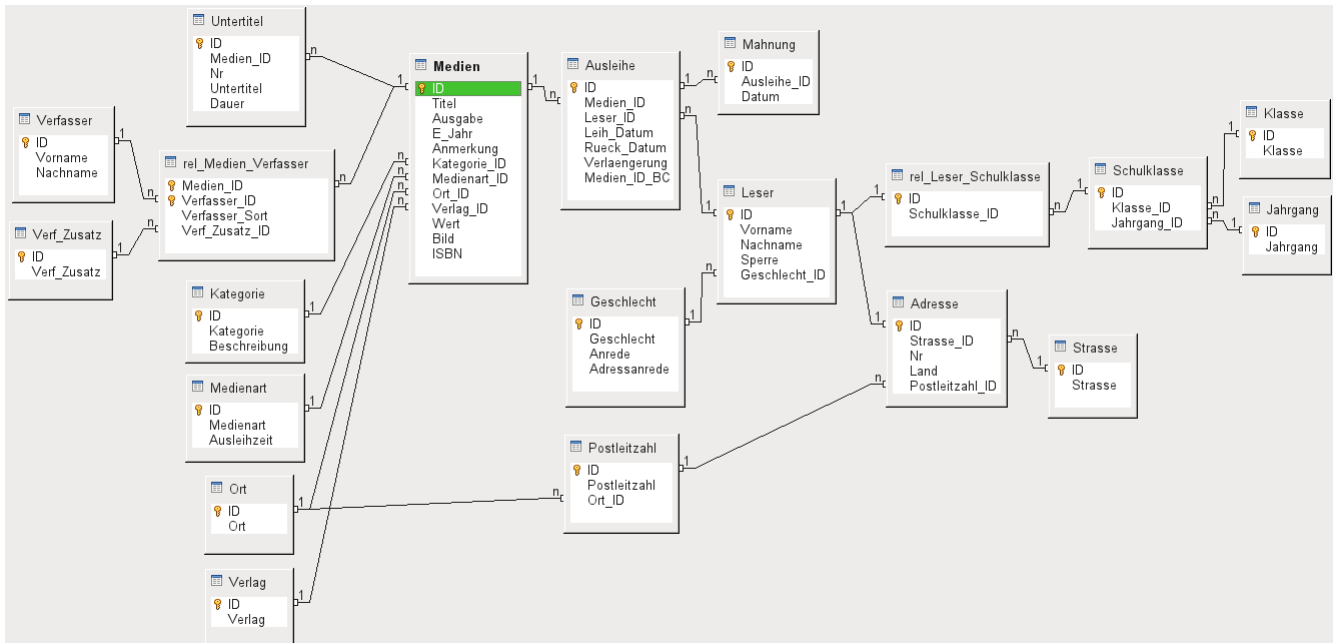


Abbildung 1: Beziehungsentwurf der Beispieldatenbank «Medien_ohne_Makros»

Eins-zu-Viele Beziehungen:

Eine Datenbank für Medien listet in einer Tabelle die Titel der Medien auf. Da es für jeden Titel unterschiedlich viele Untertitel gibt (manchmal auch gar keine) werden in einer gesonderten Tabelle diese Untertitel abgespeichert. Dies ist als eine Eins-zu-viele-Beziehung (1:n) bekannt. Einem Medium werden gegebenenfalls viele Untertitel zugeordnet, z.B. bei dem Medium Musik-CD die vielen Musiktitel auf dieser CD. Der Primärschlüssel der Tabelle "Medien" wird als Fremdschlüssel in der Tabelle "Untertitel" abgespeichert. Die meisten Beziehungen zwischen Tabellen in einer Datenbank sind Eins-zu-viele Beziehungen.

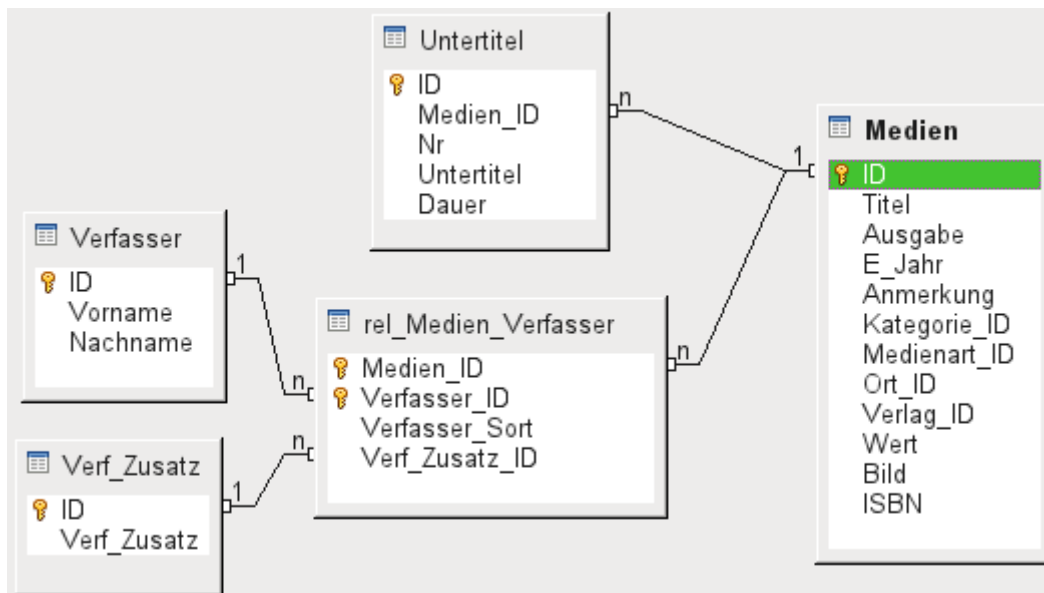


Abbildung 2: Beispiel 1:n-Beziehung; n:m-Beziehung

Viele-zu-Viele Beziehungen:

Eine Datenbank für eine Bibliothek wird eine Tabelle für den Namen der Verfasser und eine Tabelle für die Medien enthalten. Es gibt einen offensichtlichen Zusammenhang zwischen den Verfasser und z.B. Büchern, die sie geschrieben haben. Die Bibliothek kann mehr als ein Buch desselben Verfassers enthalten. Sie kann aber auch Bücher enthalten, die von mehreren Verfassern stammen. Dies ist als eine Viele-zu-viele-Beziehung (n:m) bekannt. Solche Beziehungen werden durch Tabellen gelöst, die als Mittler zwischen den beiden betroffenen Tabellen eingesetzt werden. Dies ist in der obigen Abbildung die Tabelle "rel_Medien_Verfasser".

Praktisch wird also die n:m-Beziehung über zwei 1:n-Beziehungen gelöst. In der Mittlertabelle kann die "Medien_ID" mehrmals erscheinen, ebenso die "Verfasser_ID". Dadurch, dass beide zusammen den Primärschlüssel ergeben ist nur ausgeschlossen, dass zu einem Medium wiederholt der gleiche Verfasser gewählt wird.

Eins-zu-Eins-Beziehung:

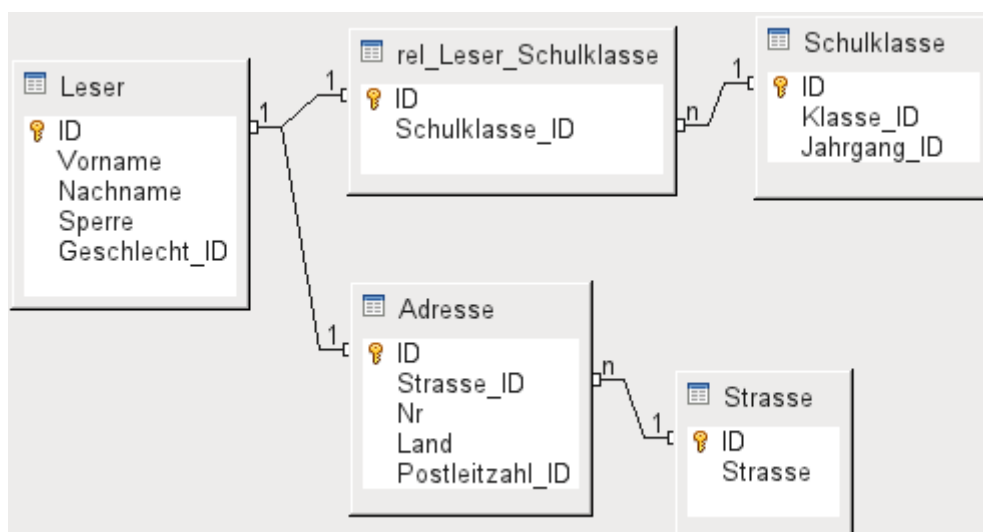


Abbildung 3: Beispiel 1:1-Beziehung

Die bereits angesprochene Bibliotheks-Datenbank enthält eine Tabelle für die Leser. In dieser Tabelle sind erst einmal nur die direkt notwendig erscheinenden Felder vorgesehen. Für eine

Datenbank im Bereich von Schulen würde noch die jeweilige Schulklasse benötigt. Über diese Klasse kann gegebenenfalls auch die Adresse erfahren werden. Eine Aufnahme der Adresse in die Datenbank ist also nicht notwendig. Die Klassenbeziehung des Schülers ist aus der Lesertabelle ausgegliedert, weil nicht in allen Bereichen mit der Zuordnung zu Klassen etwas angefangen werden kann. Dadurch entsteht eine 1:1-Beziehung zwischen Leser und seiner Klassenzuweisung über den entsprechenden Fremdschlüssel in "rel_Leser_Schulklasse".

Handelt es sich um eine Datenbank im öffentlichen Bereich, so wird wohl die Adresse der Leser benötigt. Einem Leser wird hier genau eine Adresse zugeordnet. Würde es mehrere Leser mit der gleichen Adresse geben, so müsste das bei dieser Konstruktion zu einer Neueingabe der Adresse führen, denn der Primärschlüssel aus der Tabelle "Leser" wird direkt als Primärschlüssel in die Tabelle "Adresse" eingetragen. Primärschlüssel und Fremdschlüssel sind in der Tabelle "Adresse" eins. Hier besteht also eine 1:1-Beziehung.

Eine 1:1-Beziehung bedeutet nicht, dass automatisch zu jedem Datensatz der einen Tabelle auch ein Datensatz der anderen Tabelle existiert. Es existiert allerdings **höchstens** ein Datensatz. Durch die 1:1-Beziehung werden also Felder ausgelagert, die vermutlich nur bei einem Teil der Datensätze mit Inhalt gefüllt sein werden.

Tabellen und Beziehungen der Beispieldatenbank

Die Beispieldatenbank muss drei verschiedene Aufgabenbereiche erfüllen. Zuerst einmal müssen Medien in die Datenbank aufgenommen werden. Dies soll so erfolgen, dass auch eine Bibliothek damit arbeiten kann.

Tabellen Medienaufnahme

Zentrale Tabelle der *Medienaufnahme* ist die Tabelle **Medien**. In dieser Tabelle werden alle Felder direkt verwaltet, die vermutlich nicht auch von anderen Medien mit dem gleichen Inhalt belegt werden. Doppelungen sollen also vermieden werden.

Aus diesem Grund sind in der Tabelle z.B. der Titel, die ISBN-Nummer, ein Bild des Umschlags oder das Erscheinungsjahr vorgesehen. Die Liste der Felder kann hier entsprechend erweitert werden. So sehen Bibliotheken z.B. Felder für den Umfang (Seitenanzahl), den Reihentitel und ähnlichem vor.

Die Tabelle **Untertitel** soll dazu dienen, z.B. den Inhalt von CDs im Einzelnen aufzunehmen. Da auf einer CD mehrere Musikstücke (*Untertitel*) vorhanden sind würde eine Aufnahme der Musikstücke in die Haupttabelle dazu führen, dass entweder viele zusätzliche Felder (Untertitel 1, Untertitel 2 usw.) erstellt werden müssten oder das gleiche Medium mehrmals hintereinander eingegeben werden müsste. Die Tabelle *Untertitel* steht also in einer *n:1-Beziehung* zu der Tabelle *Medien*.

Felder der Tabelle *Untertitel* sind neben dem Untertitel selbst die Nummerierung der Reihenfolge der Titel und die Dauer der Untertitel. Das Feld *Dauer* ist erst einmal als ein Zeitfeld vorgesehen. So kann gegebenenfalls die Gesamtdauer der CD in einer Übersicht berechnet und ausgegeben werden.

Die Verfasser haben zu den Medien eine n:m-Beziehung. Ein Medium kann mehrere Verfasser haben, ein Verfasser kann mehrere Medien herausgebracht haben. Dies wird mit der Tabelle **rel_Medien_Verfasser** geregelt. Primärschlüssel dieser Verbindungstabelle sind die Fremdschlüssel, die aus der Tabelle **Verfasser** und **Medien** ausgegeben werden. In der Tabelle *rel_Medien_Verfasser* wird zusätzlich noch eine Sortierung der Verfasser vorgenommen (z.B. nach der Reihenfolge, wie sie im Buch genannt werden). Außerdem wird gegebenenfalls ein Zusatz wie «Herausgeber», «Fotograf» o.ä. dem jeweiligen Verfasser beigelegt.

Kategorie, Medienart, Ort und Verlag haben jeweils eine 1:n-Beziehung.

In der **Kategorie** kann bei kleinen Bibliotheken so etwas stehen wie «Kunst», «Biologie» ... Bei größeren Bibliotheken gibt es gängige Systematiken wie z.B. die ASB (allgemeine Systematik für

Bibliotheken). Bei dieser Systematik gibt es Kürzel und ausführlichere Beschreibungen. Daher die beiden Felder für die Kategorie.

Die **Medienart** ist gekoppelt mit der *Ausleihzeit*. So kann es z.B. sein, dass Video-DVDs grundsätzlich nur eine Ausleihzeit von 1 Woche haben, Bücher aber eine von 3 Wochen. Wird die Ausleihzeit an ein anderes Kriterium gekoppelt, so muss entsprechend anders verfahren werden.

Die Tabelle **Ort** dient nicht nur dazu, die Ortsbenennungen aus den Medien aufzunehmen. In ihr werden gleichzeitig die Orte gespeichert, die für die Adressen der Nutzer Bedeutung haben.

Da der **Verlag** vermutlich auch häufiger vorkommt, ist für seine Eingabe ebenfalls eine gesonderte Tabelle vorgesehen.

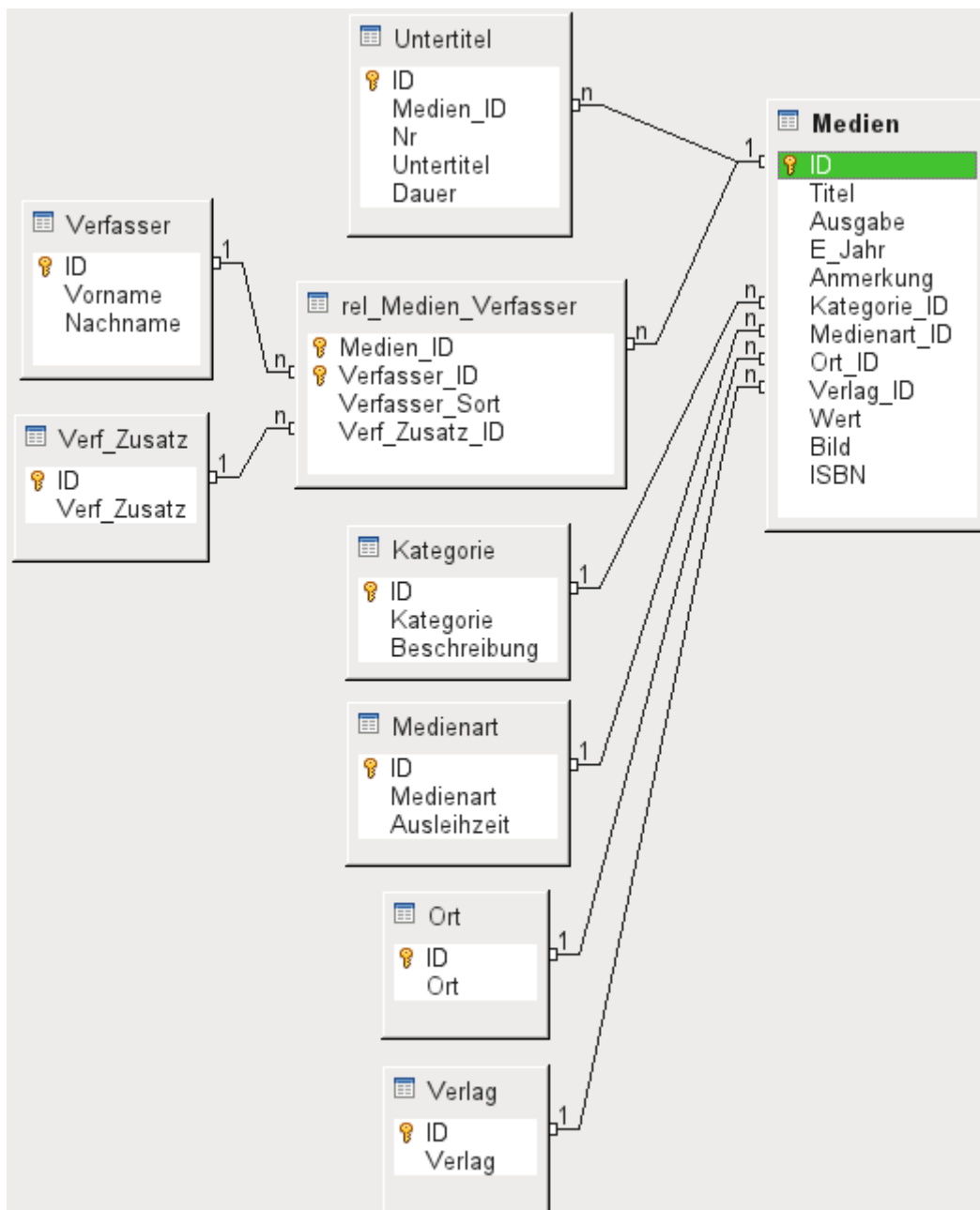


Abbildung 4: Medienaufnahme

Die Tabelle Medien hat so insgesamt vier Fremdschlüssel und einen Primärschlüssel, der für 2 Tabellen der Abbildung *Medienaufnahme* zum Fremdschlüssel wird.

Tabellen Ausleihe

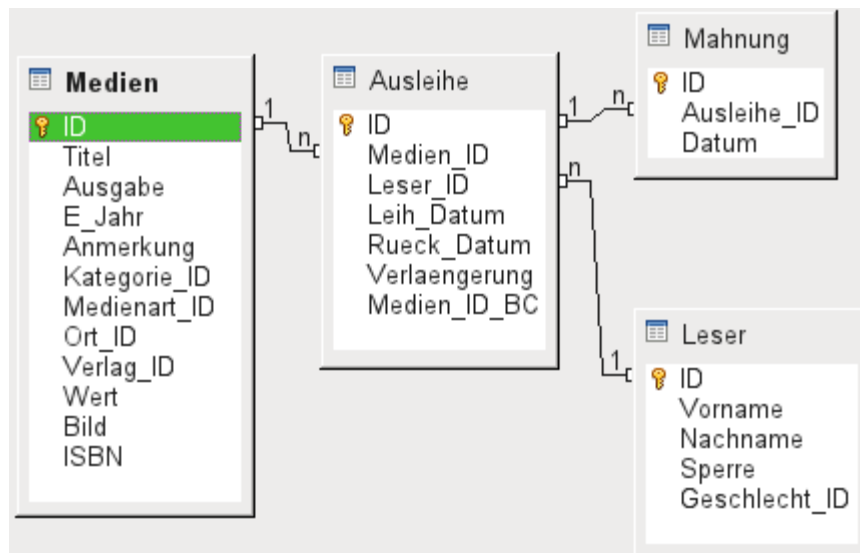


Abbildung 5: Ausleihe

Zentrale Tabelle ist die **Ausleihe**. In ihr werden die Tabellen Medien und Leser verknüpft. Da auch später noch nachvollzogen werden soll, wer ein Buch ausgeliehen hat (falls z.B. jemand beim Ausleihen bemerkt, dass das Buch beschädigt ist, oder falls eine Hitliste der Medien erstellt werden soll) wird der Datensatz in der Ausleihe bei der Rückgabe nicht einfach gelöscht. Vielmehr wird ein Rückgabedatum (*Rueck_Datum*) vermerkt.

Ebenso in die Ausleihe integriert ist das Mahnverfahren. Um die Anzahl der Mahnungen zu erfassen wird jede Mahnung separat in der Tabelle **Mahnung** eingetragen.

Neben der Verlängerung um einzelne Wochen steht noch ein gesondertes Feld in der Ausleihe, das es ermöglicht, Medien über einen Barcodescanner zu entleihen (*Medien_ID_BC*). Barcodes enthalten neben der eigentlichen "Medien_ID" auch eine Prüfziffer, mit der das Gerät feststellen kann, ob der eingelesene Wert korrekt ist. Dieses Barcodefeld ist hier nur testweise eingebaut. Besser wäre es, wenn der Primärschlüssel der Tabelle Medien direkt in Barcode-Form eingegeben würde oder per Makro aus der eingelesenen Barcodeziffer einfach die Prüfziffer vor dem Abspeichern entfernt wird.

Schließlich ist noch der **Leser** mit der Ausleihe in Verbindung zu bringen. In der eigentlichen Lesertabelle wird lediglich der Name, eine eventuelle Sperrung und ein Fremdschlüssel für eine Verbindung zur Tabelle Geschlecht vorgesehen.

Tabellen Nutzerverwaltung

In dieser Tabellenkonstruktion werden gleich zwei Szenarien bedient. Der obere Tabellenstrang ist dabei auf Schulen zugeschnitten. Hier werden keine Adressen benötigt, da die Schüler und Schülerinnen über die Schule selbst ansprechbar sind. Mahnungen müssen nicht postalisch zugestellt werden, sondern auf dem internen Wege weitergegeben werden.

Der Adressstrang ist dagegen bei öffentlichen Bibliotheken notwendig. Hier müssen sämtliche Daten erfasst werden, die zu Erstellung eines Mahnbriefes erforderlich sind.

Die Tabelle **Geschlecht** dient dazu, die richtige Anrede bei Mahnschreiben zu wählen. Die Mahnschreiben sollen schließlich möglichst automatisiert erfolgen. Außerdem gibt es Vornamen, die sowohl für männliche als auch für weibliche Leser stehen können. Deswegen ist die Abspeicherung des Geschlechts auch bei der Erstellung von handgeschriebenen Mahnungen sinnvoll.

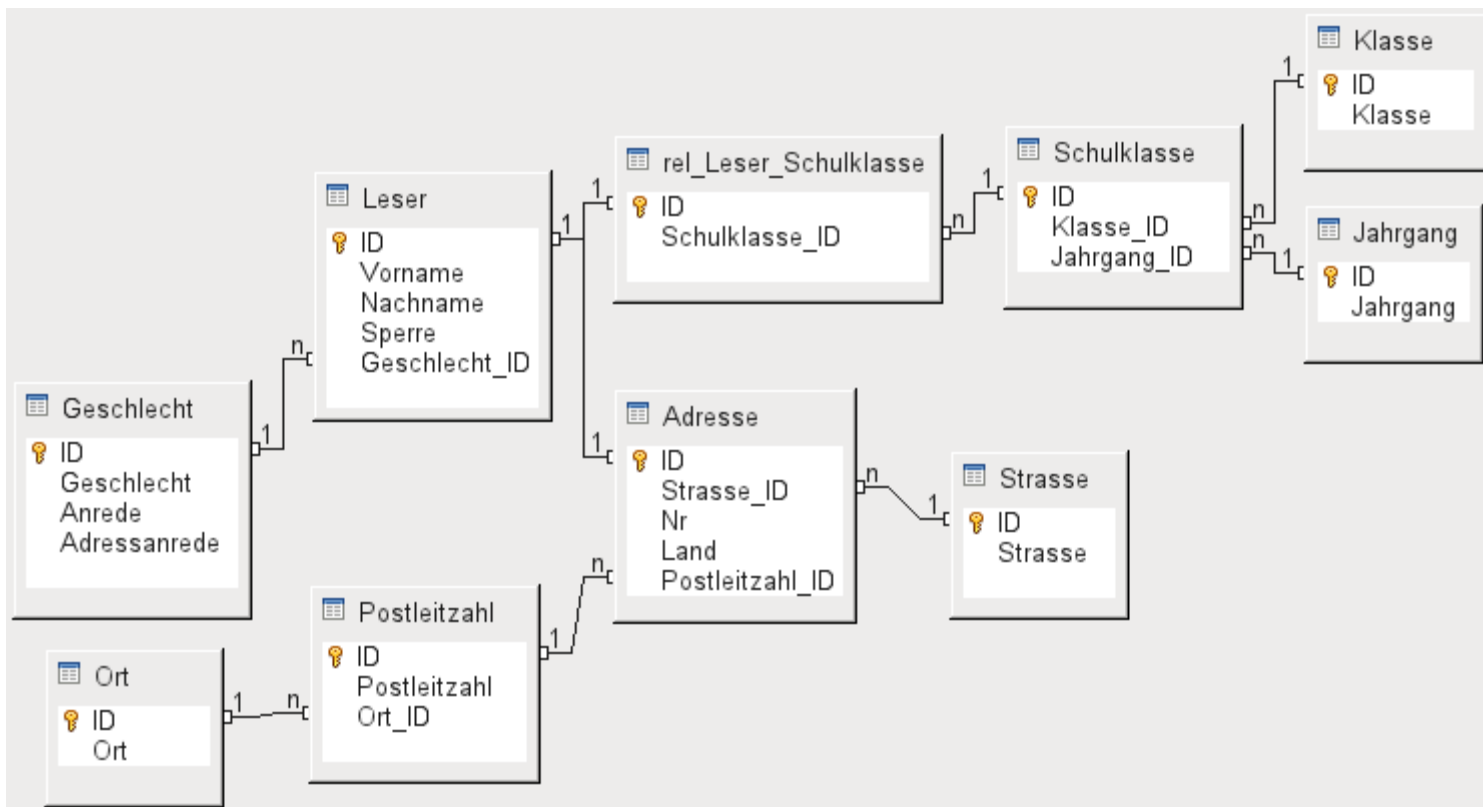


Abbildung 6: Leser - ein Schulklassenstrang und ein Adressenstrang

Die Tabelle **rel_Leser_Schulklasse** steht wie die Tabelle Adresse in einer 1:1-Beziehung zu der Tabelle "Leser". Dies ist gewählt worden, weil entweder die eine oder die andere Möglichkeit beschriftet werden soll. Sonst könnte die "Schulklasse_ID" direkt in der Tabelle Schüler existieren; gleiches gilt für den gesamten Inhalt der Tabelle Adresse.

Eine **Schulklasse** wird in der Regel durch eine Jahrgangsbezeichnung und einen Klassenzusatz gekennzeichnet. Bei einer 4-zügigen Schule kann dieser Zusatz z.B. von a bis d gehen. Der Zusatz wird in der Tabelle "Klasse" eingetragen. Der Jahrgang hat eine separate Tabelle. Sollen am Schluss eines Schuljahres die Leser aufgestuft werden, so wird einfach der Jahrgang für alle geändert.

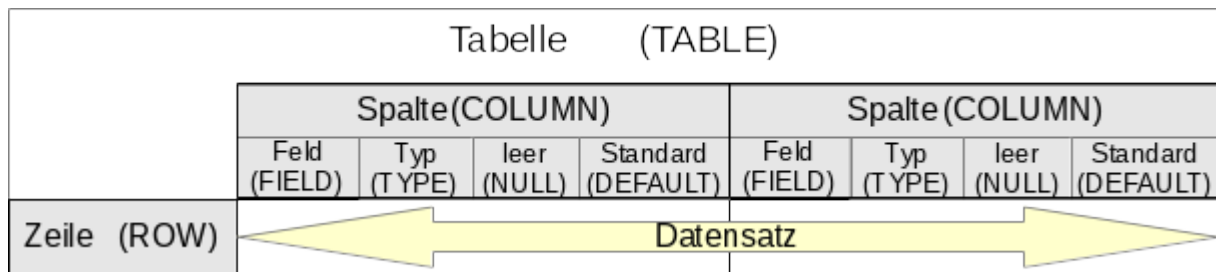
Die **Adresse** wird ebenfalls sehr differenziert dargestellt. Die Straße ist aus der Adresse ausgelagert, da Straßennamen innerhalb eines Ortes häufiger wiederholt werden. Postleitzahl und Ort sind voneinander getrennt, da oft mehrere Postleitzahlen für einen Ort gelten. Für die Post sind alle Ortsbezeichnungen, die auf die gleiche Postleitzahl passen, in einem Ort zusammengefasst. Es existieren postalisch also deutlich mehr Postleitzahlen als Orte. So werden von der Tabelle Adresse aus gesehen deutlich weniger Datensätze in der Tabelle **Postleitzahl** stehen und noch einmal deutlich weniger Datensätze in der Tabelle **Ort** existieren.

Wie eine derartige Tabellenkonstruktion später sinnvoll zu befüllen ist, wird weiter unten im Kapitel *Formulare* erläutert.

Erstellung von Tabellen

In der Regel wird sich der LibreOffice-User auf die Erstellung von Tabellen mit der grafischen Benutzeroberfläche beschränken. Die direkte Eingabe von SQL-Befehlen ist dann sinnvoll, wenn z.B. ein Tabellenfeld nachträglich an einer bestimmten Position eingefügt werden soll oder Standardwerte nach Abspeicherung der Tabelle noch gesetzt werden sollen.

Bezeichnungen bei Tabellen:



Die obige Skizze zeigt die allgemein übliche Aufteilung von Tabellen in Spalten und Zeilen. Die entsprechenden Datenbankbezeichnungen sind in Klammern hinzugefügt.

Datensätze werden in der Tabelle in einer Zeile gespeichert. Die einzelnen Spalten werden durch das Feld, den Typ und die Festlegung, ob das Feld leer sein darf, weitgehend beschrieben. Je nach Typ kann noch der Umfang an Zeichen festgelegt werden. Außerdem kann ein Standardwert eingegeben werden, der immer dann abgespeichert wird, wenn keine Eingabe erfolgt.

In der grafischen Benutzeroberfläche von Base sind die Begriffe einer Spalte etwas anders umschrieben:

Bezeichnungen in der Base-GUI			
Spalte(COLUMN)		Feldeigenschaften	
Feldname (FIELD)	Feldtyp (TYPE)	Eingabe erforderlich (NULL/NOT NULL)	Defaultwert (DEFAULT)

Feld wird zu Feldname, Typ wird zu Feldtyp. Feldname und Feldtyp werden im oberen Bereich des Tabelleneditors eingegeben. Im unteren Bereich gibt es dann die Möglichkeit, unter den Feldeigenschaften die anderen Spalteneigenschaften festzulegen, sofern dies durch die GUI festlegbar ist. Grenzen sind hier z.B., den Defaultwert eines Datumsfeldes mit dem bei der Eingabe aktuellen Datum festzulegen. Dies geht nur über eine entsprechende SQL-Eingabe, siehe dazu die [Felddefinition](#) im Kapitel [Direkte Eingabe von SQL-Befehlen](#).

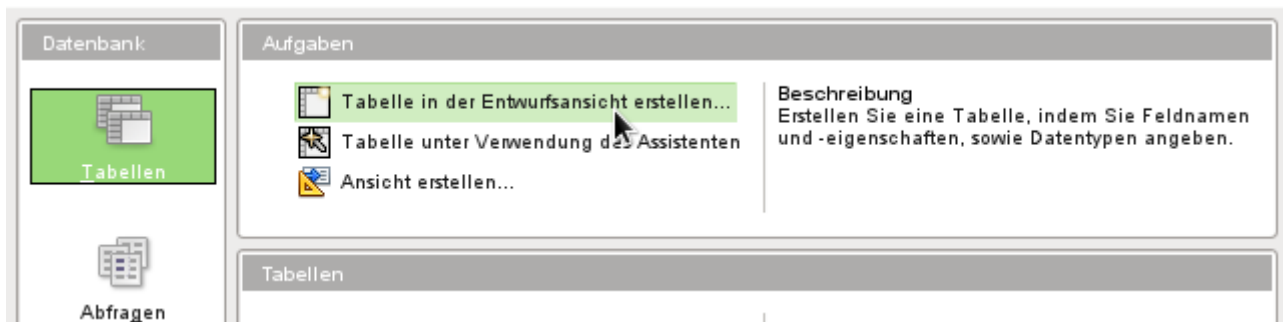
Hinweis

Defaultwert: Der Begriff «Defaultwert» in der GUI entspricht nicht dem, was Datenbanknutzer unter Defaultwert verstehen. Die GUI gibt hier einen bestimmten Wert sichtbar vor, der dann mit abgespeichert wird.

Der Defaultwert einer Datenbank wird in der Tabellendefinition gespeichert. Er wird dann in das Feld geschrieben, wenn es bei der neuen Erstellung eines Datensatzes nicht bearbeitet wurde und deshalb nicht im SQL-Code auftaucht. SQL-Defaultwerte erscheinen auch **nicht** bei der Bearbeitung der Eigenschaften einer Tabelle.

Erstellung mit der grafischen Benutzeroberfläche

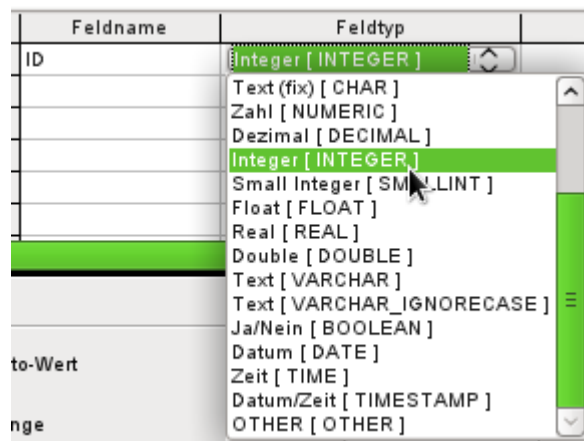
Die Erstellung innerhalb der grafischen Benutzeroberfläche wird beispielhaft für die Tabelle Medien Schritt für Schritt erklärt.



Durch einen Klick auf **Tabelle in der Entwurfsansicht erstellen** wird der Tabelleneditor gestartet.

1. ID-Feld:

- In der ersten Spalte wird «ID» als **Feldname** eingegeben. Danach wird die Tabulator-Taste gedrückt, um in die Spalte **Feldtyp** zu wechseln. Alternativ kann auch mit der Maus die Auswahlliste der nächsten Spalte angeklickt werden.



- **Integer [INTEGER]** als Feldtyp sollte aus der Auswahlliste gewählt werden. Die Standardeinstellung ist **Text [VARCHAR]**. Integer kann 10-stellige Zahlen speichern. Integer ist außerdem der Zahlentyp, der in der grafischen Benutzeroberfläche als einziger mit einem automatisch hochzählenden Wert verbunden werden kann.

Tipp

Die Erstellung einer Verknüpfung zur Auswahl aus der Auswahlliste **Feldtyp**: Drücken Sie die Taste für den ersten Buchstaben der Wahl. Sie können die Auswahl durch wiederholtes Drücken des Buchstabens wechseln. Zum Beispiel wechselt das Drücken der Taste **D** von «Datum» auf «Datum/Zeit» bzw. danach auf «Dezimal».

- «ID» wird als Primärschlüssel gesetzt, indem mit der rechten Maustaste auf das grüne Dreieck auf der linken Seite der Zeile «ID» geklickt und **Primärschlüssel** in dem Kontextmenü ausgesucht wird. Dies stellt ein Schlüssel-Symbol vor «ID».

Feldname	Feldtyp	Beschreibung
ID	Integer [INTEGER]	

☒ Primärschlüssel

Feldeigenschaften

Auto-Wert:

Länge:

Defaultwert:

Format-Beispiel:

Hinweis

Der Primärschlüssel dient nur einem Zweck – nämlich zur eindeutigen Identifizierung des Datensatzes. Daher kann ein beliebiger Name für dieses Feld verwendet werden. Im Beispiel wurde die allgemein übliche Bezeichnung ID (Identifikation) verwendet.

- Bei den Feldeigenschaften für das Feld «ID» wird die Eigenschaft **Auto-Wert** → **Ja** gesetzt. Damit wird der Primärschlüssel automatisch hoch gezählt. Die Zählung beginnt bei der internen Datenbank mit 0.
Der Auto-Wert lässt sich nur für **ein** Feld einer Tabelle einstellen. Die Auswahl **Auto-Wert** → **Ja** erzeugt automatisch den Primärschlüssel für das Feld, sofern der Primärschlüssel nicht schon vorher bestimmt wurde.

Feldname	Feldtyp	Beschreibung
ID	Integer [INTEGER]	

☒ Primärschlüssel

Feldeigenschaften

Auto-Wert:

Länge:

Defaultwert:

Format-Beispiel:

2. Das nächste Feld ist das Feld «Titel»

- Der Feldname «Titel» wird in der Spalte «Feldname» eingegeben.
- Der Feldtyp muss hier nicht geändert werden, da er bereits auf **Text** [VARCHAR] eingestellt ist.

	Feldname	Feldtyp	Beschreibung
🔑	ID	Integer [INTEGER]	
▶	Titel	Text [VARCHAR]	

Feldeigenschaften

Eingabe erforderlich

Länge

Defaultwert

Format-Beispiel

- In den Feldeigenschaften muss die Länge des Feldes angepasst werden. Die Standardlänge ist hier bei den neueren LO-Versionen 100, sollte für den Medientitel aber auf 250 Zeichen erweitert werden.
- In den Feldeigenschaften sollte wenigstens bei diesem Feld **Eingabe erforderlich** → **Ja** gewählt werden. Ein Medium ohne Titel wird keinen Sinn machen.

	Feldname	Feldtyp	Beschreibung
🔑	ID	Integer [INTEGER]	
	Titel	Text [VARCHAR]	
	Ausgabe	Text [VARCHAR]	Nr. der Auflage, Neuauflage usw.
▶	E_Jahr	Small Integer [SMALLINT]	Erscheinungsjahr

3. **Beschreibung** kann alles sein, Die Spalte kann auch leer gelassen werden. Die Beschreibung dient nur zur Erklärung des Feldinhaltes für Personen, die sich später einmal den Tabellenentwurf ansehen wollen.
4. Für das Feld «E_Jahr» wird der Zahlentyp **Small Integer [SMALLINT]** gewählt. Dabei handelt es sich um eine maximal fünfstellige ganze Zahl. Mit dem Erscheinungsjahr soll zwar nicht gerechnet werden, es sollte aber zumindest vermieden werden, dass dort Buchstaben auftauchen.

	Feldname	Feldtyp	Beschreibung
🔑	ID	Integer [INTEGER]	
	Titel	Text [VARCHAR]	
	Ausgabe	Text [VARCHAR]	Nr. der Auflage, Neuauflage usw.
	E_Jahr	Small Integer [SMALLINT]	Erscheinungsjahr
	Anmerkung	Text [VARCHAR]	
▶	Kategorie_ID	Integer [INTEGER]	Fremdschlüssel zu Kategorie

5. Für das Feld «Kategorie_ID» wird der Feldtyp **Integer [INTEGER]** gewählt. Da in der Tabelle «Kategorie» der Primärschlüssel diesen Feldtyp haben soll muss auch der hier eingetragene Fremdschlüssel den gleichen Feldtyp haben. Das gilt dann entsprechend auch für die nachfolgenden Fremdschlüssel «Medienart_ID», «Ort_ID» und «Verlag_ID».

Anmerkung	Text [VARCHAR]	
Kategorie_ID	Integer [INTEGER]	Fremdschlüssel zu Kategorie
Medienart_ID	Integer [INTEGER]	Fremdschlüssel zu Medienart
Ort_ID	Integer [INTEGER]	Fremdschlüssel zu Ort
Verlag_ID	Integer [INTEGER]	Fremdschlüssel zu Verlag
Wert	Zahl [NUMERIC]	Wertangabe in €

Feldeigenschaften	
Eingabe erforderlich	Nein
Länge	6
Nachkommastellen	2

6. Für die Wertangabe wird der Feldtyp **Zahl [NUMERIC]** oder auch **Dezimal [DECIMAL]** gewählt. Diese beiden Zahlenfelder können Zahlen mit Nachkommastellen speichern.
- Bei den **Feldeigenschaften** wird eine **Länge** von 6 Zeichen eingestellt. Das dürfte für die Wertangabe der Medien reichen.
 - Die Zahl der **Nachkommastellen** wird auf 2 Zeichen festgelegt. Damit ist jetzt maximal eine Wertangabe von 9999,99 möglich, da das Komma nicht als Stelle mitgerechnet wird.
 - Eine Formatierung der Ausgabe als € ist nicht erforderlich. Diese Einstellung erfolgt im Formular.

Verlag_ID	Integer [INTEGER]	Fremdschlüssel zu Verlag
Wert	Zahl [NUMERIC]	Wertangabe in €
Bild	Bild [LONGVARIABLE]	
ISBN	Zahl [NUMERIC]	max. 13-stellige ISBN-Nummer

Feldeigenschaften	
Eingabe erforderlich	Nein
Länge	13

7. Für das Feld «ISBN» wird der Feldtyp **Zahl [NUMERIC]** gesetzt. Dieser kann genau auf die Feldlänge einer ISBN-Nummer eingestellt werden. ISBN-Nummern sind 10 oder 13 Zeichen lang. Sie werden also später als Zahl ohne Trenner gespeichert. Die Länge wird entsprechend auf maximal 13 Zeichen eingestellt. Nachkommastellen bleiben dabei auf 0 gesetzt.
8. Die Tabelle wird mit dem Namen «Medien» abgespeichert.

Die zentrale Tabelle für die Beispieldatenbank wurde nun erstellt. Mit dem entsprechenden Verfahren können alle weiteren Tabelle ebenfalls erstellt werden. Achten Sie immer darauf, dass der Feldtyp zusammen mit den Feldeigenschaften vorbestimmt, was in dem Feld abgespeichert werden kann. Dies ist anders als in einer Tabellenkalkulation, die vollkommen durchmischte Eingaben in einer Spalte zulassen.

Hinweis

Die Reihenfolge der Felder in der Tabelle kann nur bis zum ersten Abspeichern über die grafische Benutzeroberfläche beeinflusst werden. Sie lässt sich später nur bei der Eingabe der Daten direkt in die Tabelle nicht mehr ändern.

In Abfragen, Formularen oder Berichten ist die Reihenfolge allerdings weiterhin frei zusammenstellbar.

Primärschlüssel

Wird beim Tabellenentwurf kein Primärschlüssel festgelegt, so erscheint beim Abspeichern eines Tabellenentwurfs die Nachfrage, ob ein Primärschlüssel erstellt werden soll. Dies deutet darauf hin, dass ein wesentliches Feld in der Tabelle fehlt. Ohne einen Primärschlüssel kann die interne Datenbank auf die Tabelle nicht zugreifen. In der Regel wird dieses Feld mit dem Kürzel "ID" bezeichnet, mit dem Zahlentyp INTEGER versehen und als «AutoWert» automatisch mit einer fortlaufenden Nummer versehen. Mit einem Rechtsklick auf das entsprechende Feld kann es zum Primärschlüsselfeld erklärt werden.

Es können ohne weiteres auch mehrere Felder zum gemeinsamen Primärschlüssel erstellt werden. Hierzu müssen nur die entsprechenden Felder gemeinsam markiert werden (Taste **Strg** oder **Shift** gedrückt halten). Dann kann über den Rechtsklick der Primärschlüssel allen markierten Feldern zugewiesen werden.

Sollen von einer anderen Tabelle in dieser Tabelle Informationen mitgeführt werden (Beispiel: Adressdatenbank, aber ausgelagert Postleitzahlen und Orte), so ist ein Feld mit dem gleichen Datentyp wie dem des Primärschlüssels der anderen Tabelle in die Tabelle aufzunehmen. Angenommen die Tabelle "PLZ_Ort" hat als Primärschlüssel das Feld "ID", als Datentyp 'Tiny Integer'. In der Tabelle Adressen erscheint jetzt ein Feld "ID_PLZ_Ort" mit dem Datentyp 'Tiny Integer'. Es wird also in der Tabelle Adressen immer nur die Zahl eingetragen, die als Primärschlüssel in der Tabelle "PLZ_Ort" steht. Für die Tabelle "Adresse" heißt das: Sie hat einen Fremdschlüssel zusätzlich zum eigenen Primärschlüssel bekommen.

Grundlage bei der Namenswahl von Feldern in der Tabelle: Keine 2 Felder dürfen gleich heißen. Deswegen darf auch nicht ein zweites Feld mit der Bezeichnung "ID" als Fremdschlüssel in der Tabelle "Adressen" auftauchen.

Die Feldtypen können nur begrenzt geändert werden. Eine Aufstufung (längeres Textfeld, größerer Zahlenumfang) ist ohne weiteres möglich, da alle eventuell schon eingegebenen Werte diesem Feldtyp entsprechen. Eine Abstufung wirft eher Probleme auf. Hier droht gegebenenfalls Datenverlust.

Zeitfelder in Tabellen können nicht als Felder mit Bruchteilen einer Sekunde dargestellt werden. Dies geht nur mit einem Timestamp-Feld. Mit der grafischen Benutzeroberfläche wird allerdings nur ein Timestamp-Feld erzeugt, das Datum, Stunde, Minute und Sekunde abspeichert. Dieses Feld muss über **Extras** → **SQL** noch entsprechend verändert werden.

```
ALTER TABLE "Tabellenname" ALTER COLUMN "Feldname" TIMESTAMP(6)
```

Mit dem Parameter «6» wird das Timestamp-Feld bei der internen HSQLDB auch für Bruchteile von Sekunden aufnahmefähig.

Die Firebird-Datenbank benötigt diese Parameter nicht, kann zur Zeit aber leider über die GUI noch keine Millisekunden in Zeitfeldern oder Timestampfeldern speichern.

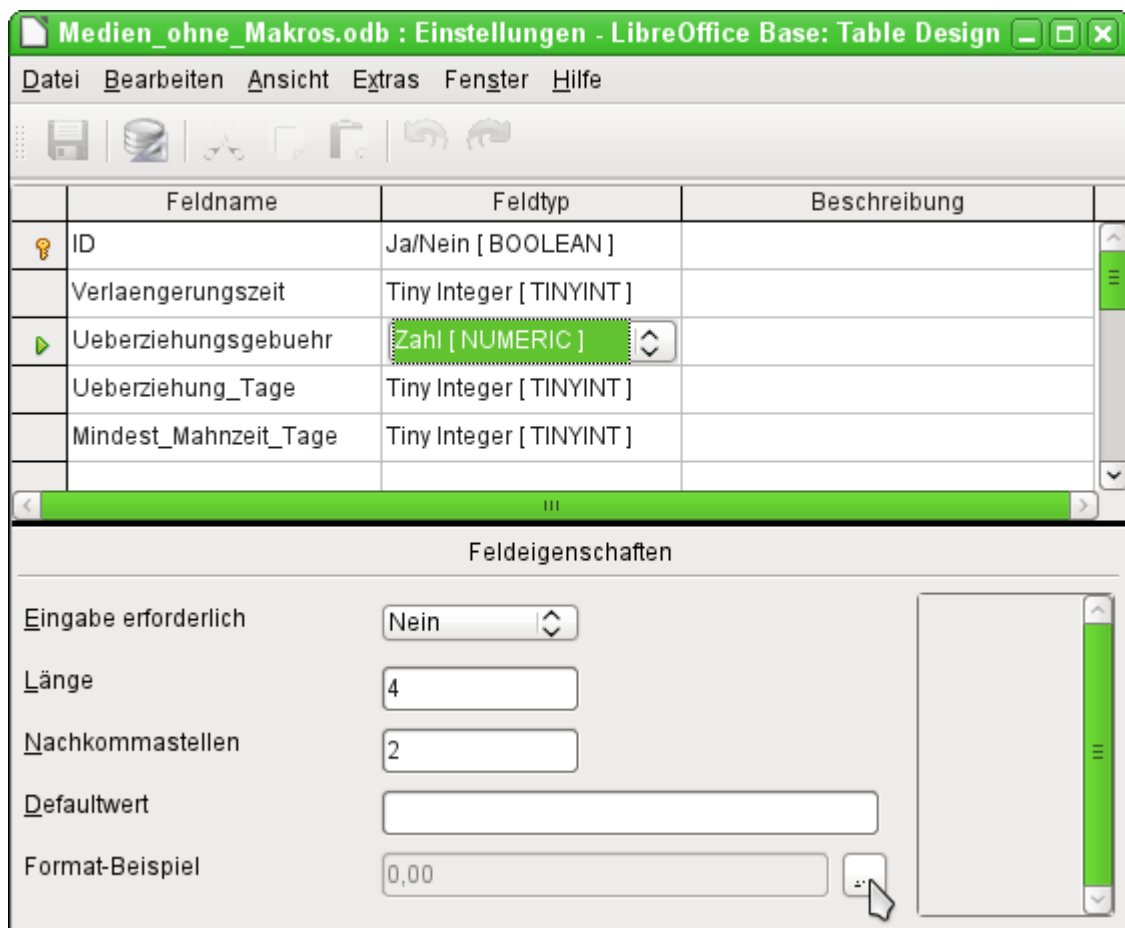
Formatierung von Feldern

Die Formatierung stellt die Werte der Datenbank für den Nutzer dar und erlaubt eine Eingabe von Werten in Abhängigkeit von landesüblichen Eingabeformen. Ohne Formatierung werden Dezimalstellen mit einem Punkt abgetrennt (**4.21** statt **4,21**), Datumswerte im Format **2014-12-22** dargestellt. Bei der Einstellung der Formatierung muss deshalb auf die Landeseinstellung geachtet werden.

Die Formatierung zeigt die Inhalte nur an. Eine Datumsdarstellung mit einer zweistelligen Jahreszahl wird dennoch als vierstellige Jahreszahl gespeichert. Wird ein Feld für eine Zahl mit zwei Nachkommastellen erstellt, wie es bei der Überziehungsgebühr in dem folgenden Beispiel der Fall ist, so wird auch eine Zahl mit zwei Nachkommastellen gespeichert, auch wenn irrtümlich die Formatierung vielleicht ohne Nachkommastellen erstellt wurde. Eine Zahl mit zwei Nachkommastellen lässt sich sogar in ein Feld eingeben, das ohne Nachkommastellen formatiert wurde. Die Nachkommastellen verschwinden scheinbar bei der Eingabe, werden aber wieder sichtbar, wenn die Formatierung angepasst wird.

Soll nur eine Zeit, aber kein Datum gespeichert werden, so sollte die Tabelle entsprechend z.B. so formatiert werden, dass nur Minuten, Sekunden und Zehntelsekunden abgefragt werden: MM:SS,00. Eine Formatierung mit Nachkommastellen ist später in Formularen nur über das formatierte Feld, nicht über das Zeitfeld möglich.

Die Formatierung von Feldern wird bei Erstellung der Tabelle oder auch anschließend in den Feldeigenschaften über einen gesonderten Dialog vorgenommen:



Über den Button in **Feldeigenschaften** → **Format-Beispiel** wird der Dialog zur Änderung des Formates gestartet.

Für die Erstellung von Feldern, die eine Währung aufnehmen sollen, ist darauf zu achten, dass die Zahlenfelder zwei Nachkommastellen haben. Die Formatierung kann in der Tabellenerstellung der grafischen Benutzeroberfläche in der gewünschten Währung für die Eingabe in die Tabelle vorgenommen werden. Dies hat allerdings nur Auswirkungen auf die Eingabe in der Tabelle und auf Abfragen, die den Wert ohne Umrechnungen auslesen. In Formularen muss die Währungsbezeichnung gesondert formatiert werden.

Hinweis

Base speichert zur Zeit nur die Formatierungen der Tabelle ab, die entweder beim Erstellen der Tabelle oder bei der Eingabe von Daten über die Spaltenköpfe erfolgt. Dies gilt auch für die Spaltenbreiten bei der Eingabe.

Gesonderte Formatierungen von Abfragen werden dagegen nicht gespeichert.

Bei Feldern, die einen Prozentsatz aufnehmen sollen, ist darauf zu achten, dass 1 % bereits als 0,01 gespeichert werden muss. Die Prozentschreibweise beansprucht also schon standardmäßig 2 Nachkommastellen. Sollen Prozentwerte wie 3,45 % abgespeichert werden, so sind also 4 Nachkommastellen bei dem numerischen Wert notwendig.

Hinweis

Hin und wieder tauchen in Formularen in einem Tabellenkontrollfeld Textfelder auf, die in einer Datenbank plötzlich einheitlich als '0' und nicht mit dem vorgesehenen Inhalt angezeigt werden. Hier hilft manchmal ein Blick in die Formatierung der Tabelle. Steht im Format-Beispiel eines Textfeldes statt einem «@» eine «0», so ist beim Format des Feldes etwas nicht in Ordnung. Lässt sich das Format des Feldes in dem Dialog nicht so ohne weiteres ändern, so kann es hilfreich sein, einen Blick auf die Sprache zu werfen. Hier kann zwischen verschiedenen Sprachwahlen eingestellt werden. Ein Umstellen der Sprache auf eine andere Sprache und anschließend zurück auf **Sprache → Standard (Deutschland)** führt dazu, dass anschließend wieder «@» statt «0» in der Formatierung gezeigt wird. Anschließend erscheint auch wieder der Text in den Tabellenkontrollfeldern des Formulars.

Dieser Bug ist ab der Version LibreOffice 4.3.2 behoben.

Einstellung eines Indexes

Manchmal erscheint es sinnvoll, neben dem Primärschlüssel auch andere Felder oder eine Kombination anderer Felder mit einem Index zu versehen. Ein Index dient dazu, Suchergebnisse schneller zu erhalten. Er kann außerdem dazu genutzt werden, Doppeleingaben zu vermeiden.

Jeder Index hat eine fest definierte Sortierreihenfolge. Wird eine Tabelle ohne Sortierung aufgerufen, so richtet sich die Sortierreihenfolge immer nach der Sortierreihenfolge der als Index definierten Felder. Bei einer Tabelle mit Primärschlüssel, wie dies in der internen HSQLDB üblich ist, ist allerdings der Index des Primärschlüssels als erster eindeutiger Index für die Sortierung bereits maßgebend.

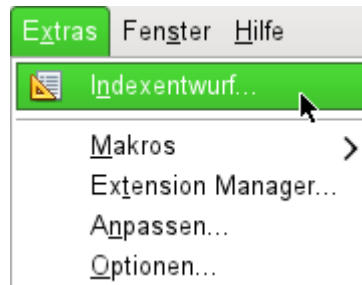


Abbildung 7: Zugriff auf den Indexentwurf

Zuerst muss die Tabelle mit einem rechten Mausklick über das Kontextmenü zum Bearbeiten geöffnet werden. Der Zugriff auf den Indexentwurf erfolgt dann über **Extras → Indexentwurf...** oder direkt über den entsprechenden Button in der Menüleiste des Tabellenentwurfes.

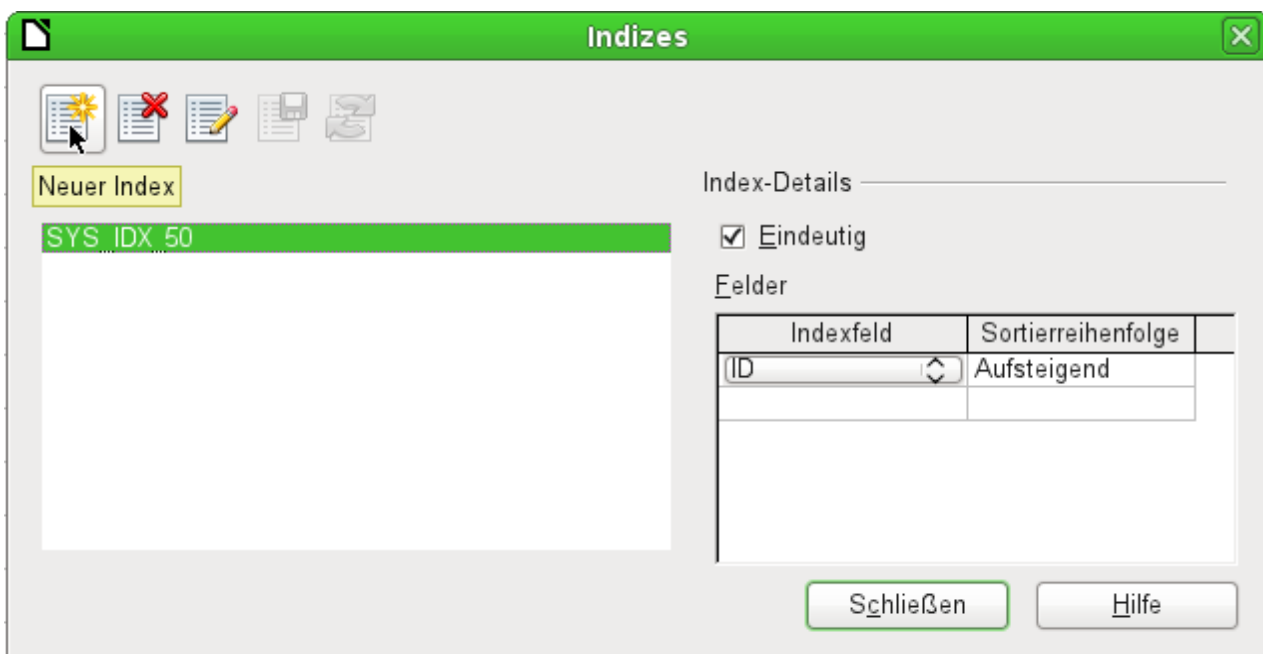


Abbildung 8: Erstellen eines neuen Indexes

Über «Neuer Index» wird ein Index neben dem des Primärschlüssels erstellt.

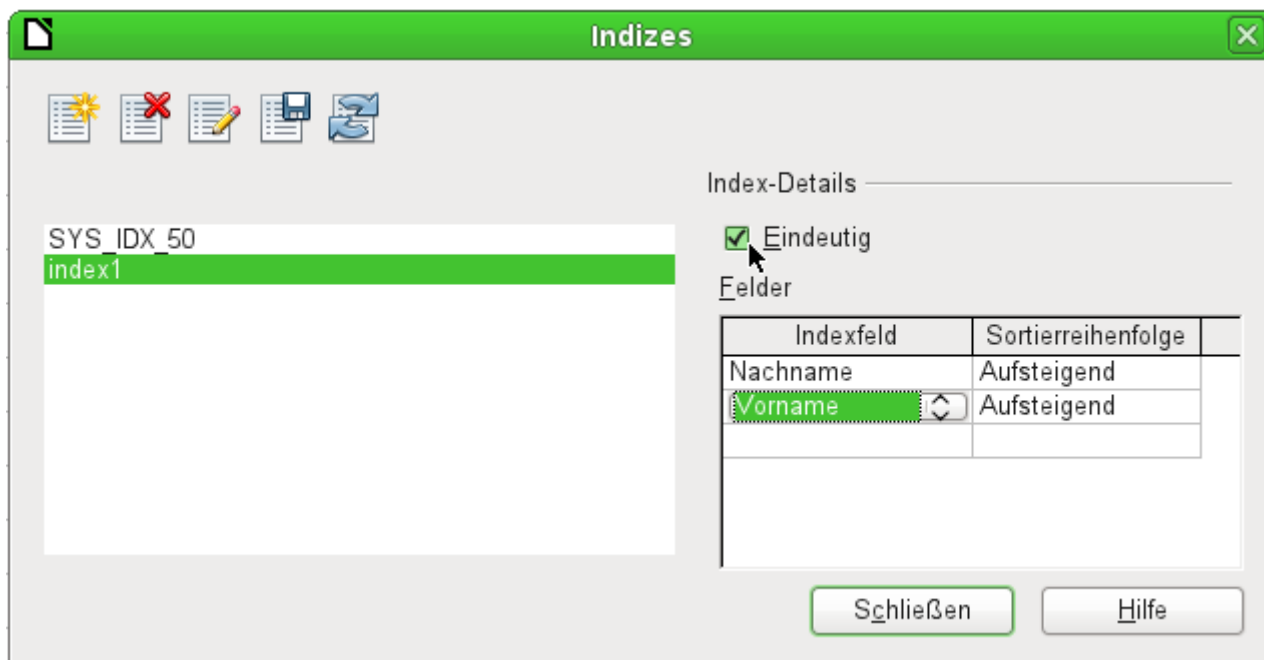


Abbildung 9: Der Index wird als «Eindeutig» definiert.

Dem neuen Index wird automatisch die Bezeichnung «index1» zugewiesen. Diese Bezeichnung kann geändert werden. Im Indexfeld wird ausgewählt, welches Feld bzw. welche Felder über den Index verwaltet werden sollen. Dabei wird gleichzeitig eine Sortierung eingestellt.

Ein Index kann prinzipiell auch über Tabellenfelder erstellt werden, die keine eindeutigen Werte haben. Im obigen Bild ist aber das Index-Detail «Eindeutig» gewählt, so dass in das Feld "Nachname" zusammen mit dem Feld "Vorname" nur Werte eingegeben werden können, die dort in der Kombination noch nicht stehen. So ist z.B. Robert Müller und Robert Maier möglich, ebenso Robert Müller und Eva Müller.

Wird ein Index über ein Feld erstellt, so wird die Eindeutigkeit auf ein Feld bezogen. Ein solcher Index ist in der Regel der Primärschlüssel. In diesem Feld darf jeder Wert nur einmal vorkommen. Beim Primärschlüssel darf allerdings zusätzlich das Feld auf keinen Fall NULL sein.

Eine Sonderstellung für einen eindeutigen Index nimmt in einem Feld das Fehlen eines Eintrages, also NULL, ein. Da NULL alle beliebigen Werte annehmen könnte ist es ohne weiteres erlaubt, bei einem Index über zwei Felder in einem Feld mehrmals hintereinander die gleiche Eingabe zu tätigen, solange in dem anderen Feld keine weitere Angabe gemacht wird.

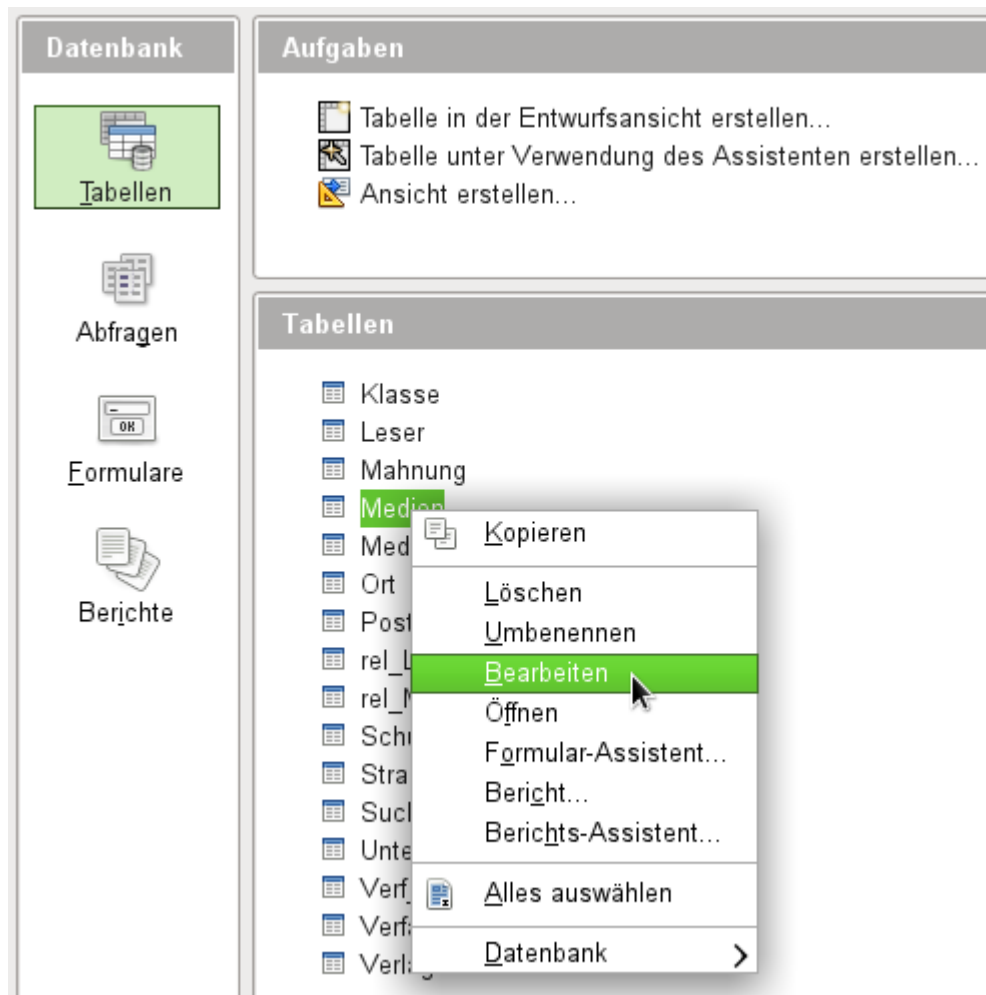
Hinweis

NULL ist für Datenbanken die Bezeichnung für eine leere Zelle, die nichts enthält. Mit einem Feld, das NULL ist kann also nicht gerechnet werden. Im Gegensatz dazu gehen Tabellenkalkulationen bei leeren Feldern automatisch davon aus, dass der Inhalt «0» ist.

Beispiel: In einer Mediendatenbank wird für die Ausleihe die Mediennummer und das Ausleihdatum eingegeben. Wird das Medium zurückgegeben, so wird dies durch ein Rückgabedatum vermerkt. Nun könnte ein Index über die Felder "Mediennummer" und "Rückgabedatum" doch leicht verhindern, dass das gleiche Medium mehrmals ausgeliehen wird, ohne dass die Rückgabe vermerkt wurde. Dies funktioniert aber leider nicht, da das Rückgabedatum ja noch nicht mit einem Wert versehen ist. Der Index verhindert stattdessen, dass ein Medium zweimal mit dem gleichen Datum zurückgegeben wird – sonst nichts.

Änderung bestehender Tabellen

Nicht alle Tabellen werden bereits beim Erstellen so weit durchgeplant sein, dass keine Änderungen mehr vorzunehmen sind. Mit einem rechten Mausklick auf eine Tabelle wird das folgende Kontextmenü sichtbar:



Mit einem Klick auf **Bearbeiten** öffnet sich der grafische Bearbeitungsmodus. Hier können Feldnamen, Feldtypen und Formate geändert werden. Natürlich können auch weitere Felder hinzugefügt werden.

Die Änderungen sind allerdings nicht völlig problemlos möglich. Soll ein Feldname geändert werden, so ist es angeraten, zuerst den neuen Namen zusätzlich einzutragen und dann die vorherige Benennung zu entfernen. Andernfalls verschwindet das ganze Feld.

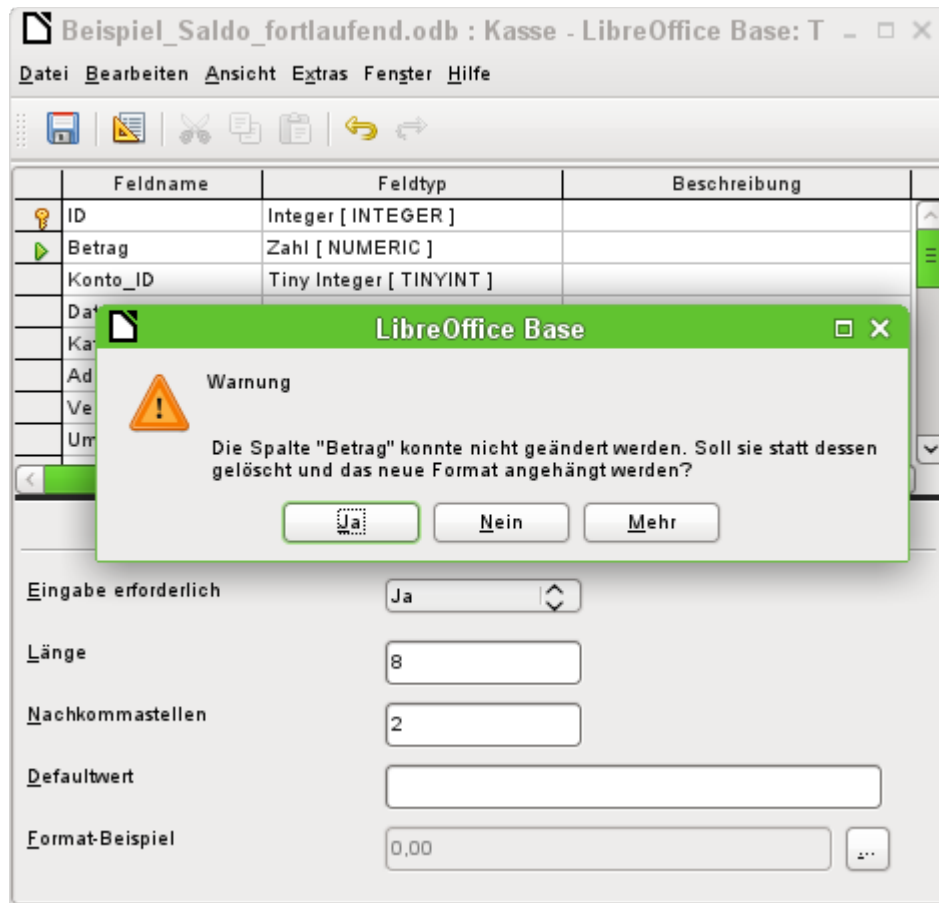
Neue Felder können nur am Ende der Liste hinzugefügt werden, auch wenn der Bearbeitungsmodus etwas anderes scheinbar ermöglicht. Das Einfügen eines Feldes mitten in eine Liste ist nicht möglich.

Auch Kommentare zu vorher erstellten Feldern werden bei einer anschließenden Tabellenänderung oft nicht mit abgespeichert.

Sind bereits erst einmal mehrere Tabellen erstellt worden, so können die Beziehungen zwischen den Tabellen eine Änderung von Feldeigenschaften blockieren. Als Beziehungen nimmt die Datenbank sowohl die unter **Extras → Beziehungen** erstellten Verknüpfungen als auch die in Tabellenansichten erstellten Beziehungen wahr.

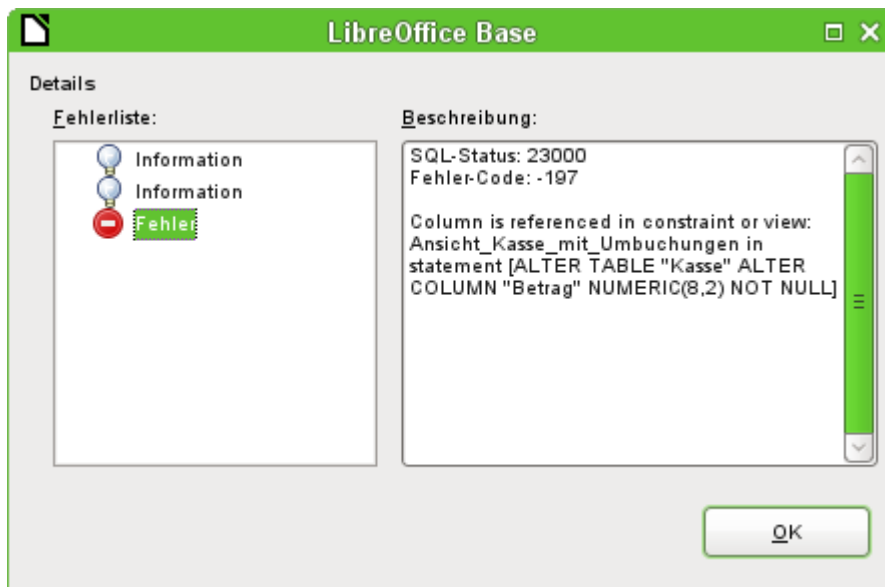
Probleme bei der Änderung von Tabellen

Tabellen sollten am besten direkt bei der Erstellung komplett mit allen nötigen Einstellungen versehen werden. Sollen hinterher die Eigenschaften von Feldern verändert werden (Feldname, erforderliche Eingabe usw.), so kann das häufig zu einer Fehlermeldung führen, die nicht Ursache der GUI ist, sondern in der darunterliegenden Datenbank begründet ist.

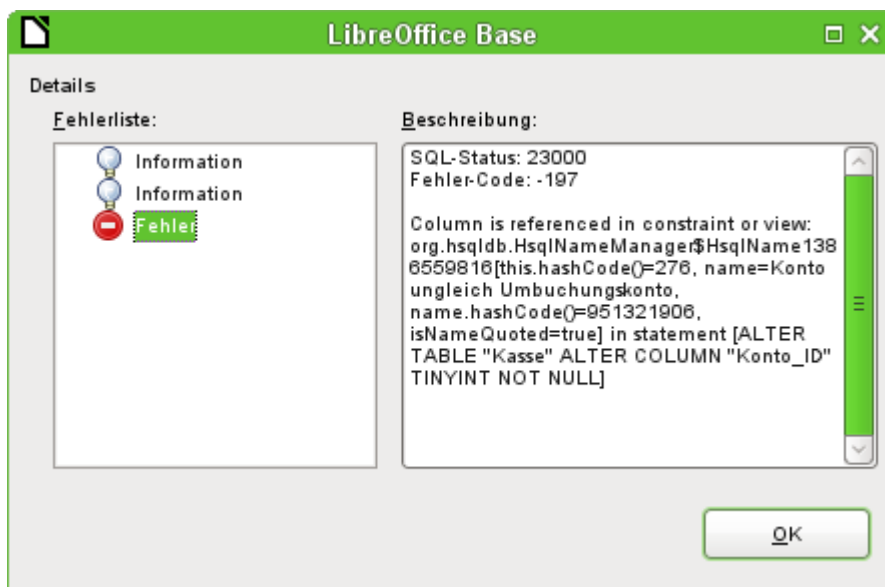


Hier sollte das Feld "Betrag" in der Eigenschaft «Eingabe erforderlich» auf «Ja» umgestellt werden. Das Warnsymbol macht bereits darauf aufmerksam: Die Änderung kann zu einem Verlust von Daten führen. Eine einfache Änderung ist nicht möglich. Bereits vorher ausgeschlossen wurde, dass in dem Feld "Betrag" eventuell ein Datensatz ohne Eingabe existiert.

Ein Klick auf **Ja** führt zu einer weiteren Fehlermeldung, da die Datenbank die Löschung nicht zulässt. Ein Klick auf **Nein** bricht den gesamten Vorgang ab. Häufig wird die Frage nach mehr Informationen gar nicht erst gestellt, weil die Informationen nur mit etwas gründlicherem Wissen auch sinnvoll zu einer anderen Handlungsweise und dann zum gewünschten Ziel führen können.



Die Fehlermeldung **Column is referenced in constraint or view** bedeutet: Auf die Spalte mit dem Feldnamen "Betrag" wird an anderer Stelle der Datenbank bereits Bezug genommen. Dies kann eine *Bedingungsdefinition* oder eine *Tabellenansicht* sein, die nach dem Erstellen der Tabelle von dem Nutzer erstellt wurde. In der obigen Abbildung wird noch darauf hingewiesen, wie der Name der Bedingungsdefinition oder Ansicht heißt: "Ansicht_Kasse_mit_Umbuchungen". Damit ist für den Nutzer klar, an welcher Stelle angesetzt werden muss. Zuerst sollte der SQL-Code der Ansicht z.B. in einer Abfrage gesichert werden, dann die Ansicht gelöscht werden und danach kann ein neuer Versuch starten.



Wieder die entsprechende Meldung, nur mit einer umfangreicheren Erklärung. Nur der sinnvollen Benennung der Bedingung «Konto ungleich Umbuchungskonto» ist es zu verdanken, dass die Bedingungsdefinition auch auffindbar ist. Hier ist der Spalte mit dem Feldnamen "Konto_ID" die Bedingung zugeordnet worden, dass eine weitere Spalte in der gleichen Tabelle im gleichen Datensatz nicht den gleichen Wert haben darf. Erst wenn diese Bedingung wieder entfernt wird ist es möglich, einen erneuten Versuch zu starten, die Spalte zu verändern.

Taucht jetzt noch wieder ein Fehler auf, so liegt dieser häufig darin begründet, dass das entsprechende Feld mit einem Feld einer anderen Tabelle in der Beziehungsdefinition verknüpft wurde. Hier muss dann zuerst die Beziehung unter **Extras → Beziehungen** gelöst werden, bevor die Änderung vorgenommen werden kann.

Mängel der grafischen Tabellenerstellung

Die Reihenfolge der Tabellenfelder kann im Anschluss an den Abspeichervorgang nicht mehr geändert werden. Für eine Darstellung in anderer Reihenfolge ist dann eine Abfrage notwendig. Dies gilt, obwohl die grafische Benutzeroberfläche etwas anderes vortäuscht. Hier kann bei der Tabellenerstellung und bei der Tabellenbearbeitung ein Kontextmenü aufgerufen werden, das z.B. anbietet, Felder auszuschneiden und an anderer Stelle einzufügen. Damit sind dann aber nur die Feldbezeichnungen und die Feldtypen gemeint, nicht aber die Inhalte der Tabelle. Die tauchen nach so einer Änderung mit anderer Feldbezeichnung und eventuell auch anderem Feldtyp wieder auf.⁶

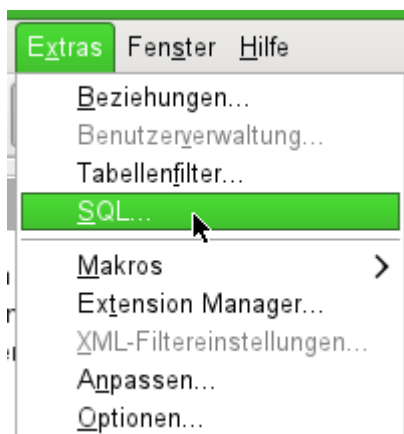
Nur über direkte SQL-Eingabe kann ein neues Feld an eine bestimmte Position innerhalb der Tabelle rutschen. Bereits erstellte Felder sind aber auch hier nicht beweglich.

Eigenschaften der Tabellen sollten sofort festgelegt werden. Welche Felder sollen nicht NULL sein, welche einen Standardwert (Default) erhalten. Diese Eigenschaft kann hinterher häufig nur unter Berücksichtigung der oben genannten Fehlermeldungen geändert werden.

Die dort abgelegten Default-Werte haben nichts mit den in der Datenbank selbst liegenden Default-Werten zu tun. So kann dort z.B. bei einem Datum nicht als Standard das aktuelle Datum vorgegeben werden. Dies ist der direkten Eingabe über SQL vorbehalten.

Direkte Eingabe von SQL-Befehlen

Die direkte Eingabe von SQL-Befehlen ist über das Menü **Extras** → **SQL** erreichbar.



Hier werden Befehle im oberen Fensterbereich eingegeben; im unteren Bereich wird der Erfolg oder gegebenenfalls die Gründe für den fehlenden Erfolg (auf Englisch) mitgeteilt. Abfragen können hier unter «Ausgabe» dargestellt werden, wenn das Ankreuzfeld markiert wird.

6 [Bug 51605](#)

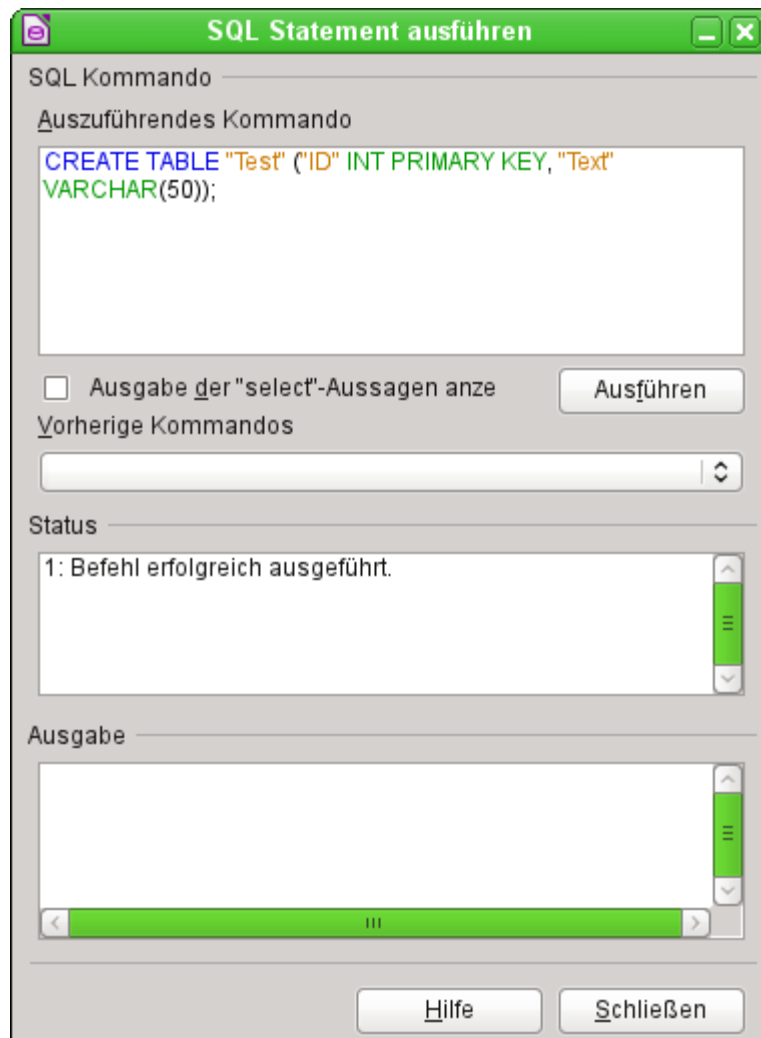


Abbildung 10: Fenster für direkte Eingabe von SQL-Befehlen

Eine Übersicht der für die eingebaute HSQLDB möglichen Eingaben ist unter <http://www.hsquidb.org/doc/1.8/guide/ch09.html> zu finden. Die dortigen Inhalte werden in den folgenden Abschnitten erklärt. Einige Befehle machen nur Sinn, wenn es sich dabei um eine externe HSQLDB handelt (Benutzerangaben usw.) Sie werden gegebenenfalls im Abschnitt «[Datenbankverbindung zu einer externen HSQLDB](#)» aufgeführt.

Hinweis

LibreOffice liegt die Version 1.8.0 der HSQLDB zugrunde. Die aktuell erhältliche Serverversion hat die Version 2.2. Die Funktionen der neuen Version sind umfangreicher. Sie sind direkt über <http://hsquidb.org/web/hsquidDocsFrame.html> zu erreichen. Die Beschreibung der Version 1.8 erfolgt jetzt unter <http://www.hsquidb.org/doc/1.8/guide/>. Außerdem ist sie in Installationspaketen zur HSQLDB enthalten, die von <http://sourceforge.net/projects/hsquidb/files/hsquidb/> heruntergeladen werden können.

Für die interne Firebird-Datenbank wurden die entsprechenden Informationen aus <http://www.firebirdsql.org/en/documentation/> herangezogen. Hier liegt zum Erscheinen der Version LO 5.3 allerdings nur eine ausführliche Dokumentation zu Firebird-Version 2.5, nicht zur in LO 5.3 verbauten Firebird-Version 3.0, vor. Auch hier gilt natürlich: Längst nicht alle Funktionen der externen Datenbank haben für die interne Datenbank eine Bedeutung. Manche der intern gut nutzbaren Funktionen sind leider auch noch nicht in LO 5.3 umgesetzt.

Viele Funktionen stehen für beide Datenbanken zur Verfügung. Unterschiede zwischen der HSQLDB und Firebird werden entsprechend gekennzeichnet. Mit **GRÜN** ist gekennzeichnet, wenn eine Funktion für eine Datenbank zur Verfügung steht. Mit **ROT-UND-DURCHGESTRICHEN** ist gekennzeichnet, wenn eine Funktion für die entsprechende Datenbank nicht verfügbar ist.

Tabellenerstellung

Ein einfacher Befehl um eine gebrauchstüchtige Tabelle zu erstellen, ist z. B.

```
CREATE TABLE "Test" ("ID" INT PRIMARY KEY, "Text" VARCHAR(50));
```

CREATE TABLE "Test": Erschaffe eine Tabelle mit dem Namen "Test".

(): mit den hierin enthaltenen Feldnamen, Feldtypen und Zusätzen.

"ID" INT PRIMARY KEY, "Text" VARCHAR(50): Feldname "ID" mit dem Zahlentyp «Integer» als Primärschlüssel, Feldname "Text" mit dem Texttyp «variable Textlänge» und der Textbegrenzung auf 50 Zeichen.

```
CREATE [MEMORY | CACHED | [GLOBAL] TEMPORARY | TEMP | TEXT] TABLE  
"Tabellennamen" ( <Felddefinition> [, ...] [,  
<Bedingungsdefinition>...] ) [ON COMMIT {DELETE | PRESERVE} ROWS];
```

[MEMORY | CACHED | [GLOBAL] TEMPORARY | TEMP | TEXT]: (**HSQLDB**, **FIREBIRD**)

Die Standardeinstellung ist hier **CACHED**. Die HSQLDB erstellt also grundsätzlich alle Tabellen in einem Zwischenspeicher auf dem Datenträger. Dies gilt auch für die Tabellen, die über LibreOffice Base in der internen Datenbank geschrieben werden, selbst wenn der Zusatz **CACHED** nicht angegeben wird. Eine andere Möglichkeit wäre, die Tabellen im Arbeitsspeicher erstellen zu lassen (**MEMORY**).

Hinweis

```
CREATE TEXT TABLE "Text" ("ID" INT PRIMARY KEY, "Text" VARCHAR(50));
```

Eine Texttabelle wird in der HSQLDB erstellt. Sie muss jetzt mit einer externen Textdatei (z. B. einer *.csv-Datei) verbunden werden:

```
SET TABLE "Text" SOURCE "Text.csv";
```

Die Datei "Text.csv" muss jetzt natürlich die entsprechenden Felder in der entsprechenden Reihenfolge enthalten. Es können bei der Erstellung der Verbindung zusätzliche Optionen gewählt werden. Details hierzu:

http://hsqldb.org/doc/guide/ch09.html#set_table_source-section

Texttabellen sind nicht gegenüber anderen Programmen schreibgeschützt. Es kann also passieren, dass ein anderes Programm/ein anderer Nutzer gerade die Tabelle ändert, während Base darauf zugreift. Texttabellen taugen in der Hauptsache zum Datenaustausch zwischen verschiedenen Programmen.

Auf **TEMPORARY** bzw. **TEMP** kann Base nicht zugreifen. Die SQL-Befehle werden hier wohl abgesetzt, die Tabellen aber nicht in der grafischen Benutzeroberfläche angezeigt (und damit auch nicht über die grafische Benutzeroberfläche löscherbar) und die Eingaben (über SQL) auch nicht anzeigbar, es sei denn die automatische Löschung des Inhaltes nach dem endgültigen Abspeichern ist ausgeschaltet. Eine Abfrage ergibt hier eine Tabelle ohne Inhalt.

GLOBAL TEMPORARY ist hier die einzige Zusatzoption, die auch **FIREBIRD** bietet. Die Tabelle wird in der grafischen Benutzeroberfläche zwar angezeigt, aber Eingaben auch direkt über SQL sind nicht abrufbar. Firebird lässt außerdem auch noch eine Definition als **EXTERNAL FILE** (direkt nach dem Tabellennamen) zu. Die Tabelle wird erstellt, ist aber für die Nutzung durch den Server gesperrt.

Tabellen, die mit SQL direkt gegründet wurden, werden nicht direkt angezeigt. Hier muss entweder über **Ansicht → Tabellen aktualisieren** eine Auffrischung erfolgen oder die Datenbank einfach geschlossen und erneut geöffnet werden.

<Felddefinition>:

```
"Feldname" Datentyp [(Zeichenanzahl[,Nachkommastellen])] [{DEFAULT  
'Standardwert' | GENERATED BY DEFAULT AS IDENTITY (START WITH <n>[,  
INCREMENT BY <m>)]}] | [[NOT] NULL] [IDENTITY] [PRIMARY KEY]
```

Erlaubte Standardwerte innerhalb der Felddefinition:

Für Textfelder kann ein Text in einfachen Anführungsstrichen oder **NULL** eingegeben werden. Die einzige SQL-Funktion, die erlaubt ist, ist **CURRENT_USER**. Dies ergibt allerdings nur dann einen Sinn, wenn die HSQLDB als externe Serverdatenbank mit mehreren Nutzern betrieben wird.

Für Datums- und Zeitfelder kann ein Datum, eine Zeit oder eine Kombination aus Datum und Zeit in einfachen Anführungsstrichen oder **NULL** eingegeben werden. Dabei ist zu beachten, dass das Datum amerikanischen Konventionen entspricht (yyyy-mm-dd), die Zeitangabe das Format hh:mm:ss hat und der Datums_Zeit_Wert das Format yyyy-mm-dd hh:mm:ss.

SQL-Funktionen, die erlaubt sind:

- für das aktuelle Datum - **CURRENT_DATE, TODAY** (HSQLDB),
DATE 'NOW' (FIREBIRD)
- für die aktuelle Zeit - **CURRENT_TIME, NOW** (HSQLDB),
TIME 'NOW' (FIREBIRD)
- für den aktuellen Datums-Zeit-Wert - **CURRENT_TIMESTAMP, NOW** (HSQLDB),
TIMESTAMP 'NOW' (FIREBIRD).

Zusätzlich bietet Firebird: **DATE 'TODAY', DATE 'YESTERDAY'** und **DATE 'TOMORROW'**.

Für boolesche Felder (Ja/Nein) können die Ausdrücke **FALSE, TRUE, NULL** gesetzt werden. Diese sind ohne einfache Anführungszeichen einzugeben.

Für numerische Felder ist jede in dem Bereich gültige Zahl sowie NULL möglich. Auch hier sind, zumindest bei **NULL**, keine einfachen Anführungszeichen einzugeben. Bei der Eingabe von Nachkommazahlen ist darauf zu achten, dass die Dezimalstellen durch einen Punkt und nicht durch ein Komma getrennt werden.

Für Binärfelder (Bilder etc.) ist jeder gültige Hexadezimalstring in einfachen Anführungsstrichen sowie **NULL** möglich. Beispiel für einen Hexadezimalstring: '0004ff' bedeutet 3 Bytes, zuerst 0, als zweites 4 und zum Schluss 255 (0xff). Da Binärfelder in der Praxis nur für Bilder eingesetzt werden müsste also der Binärcode des Bildes bekannt sein, das den Defaultwert bilden soll.

Hinweis

Hexadezimalsystem: Zahlen werden in einem Stellenwertsystem von 16 dargestellt. Die Ziffern 0 bis 9 und die Buchstaben a bis f ergeben pro Spalte 16 Ziffern im Mischsystem. Bei zwei Feldern kommen dann $16 \times 16 = 256$ mögliche Werte dabei zustande. Das entspricht schließlich 1 Byte.

NOT NULL → der Feldwert kann nicht NULL sein. Diese Bedingung kann lediglich in der Felddefinition mit angegeben werden.

Beispiel:

```
CREATE TABLE "Test" ("ID" INT GENERATED BY DEFAULT AS IDENTITY (START  
WITH 10) PRIMARY KEY, "Name" VARCHAR(50) NOT NULL, "Datum" DATE  
DEFAULT CURRENT_DATE);
```

Eine Tabelle "Test" wird erstellt. Das Schlüsselfeld "ID" wird als Autowert definiert. Der Autowert soll mit der Zahl 10 beginnen. Für Firebird ist zusätzlich notwendig, dass **PRIMARY KEY** zusätzlich zur der Definition des generierten Wertes erwähnt wird. Sonst fehlt hier der Primärschlüssel und eine Eingabe ist nur über SQL möglich.

Das Eingabefeld "Name" ist ein Textfeld für maximal 50 Zeichen. Es darf nicht leer sein.

Zuletzt kommt ein Datumsfeld "Datum", das als Standardwert das aktuelle Datum speichert,

wenn nicht ein anderes Datum eingegeben wurde. Dieser Standardwert wird aber nur bei der Erstellung eines neuen Datensatzes wirksam. Wird ein Datum gelöscht, so bleibt der Inhalt anschließend leer.

<Bedingungsdefinition>:

```
[CONSTRAINT "Name"]  
UNIQUE ( "Feldname 1" [, "Feldname 2"...] ) |  
PRIMARY KEY ( "Feldname 1" [, "Feldname 2"...] ) |  
FOREIGN KEY ( "Feldname 1" [, "Feldname 2"...] ) REFERENCES "anderer  
Tabellenname" ( "Feldname 1" [, "Feldname 2"...] ) [ON {DELETE | UPDATE}  
{CASCADE | SET DEFAULT | SET NULL}] |  
CHECK(<Suchbedingung>)
```

Bedingungsdefinitionen (Constraints) definieren Bedingungen, die beim Einfügen der Daten erfüllt sein müssen. Die Constraints können mit einem Namen versehen werden.

UNIQUE ("Feldname") → der Feldwert muss innerhalb des Feldes einzigartig sein

PRIMARY KEY ("Feldname") → der Feldwert muss einzigartig sein und kann nicht **NULL** sein (Primärschlüssel)

FOREIGN KEY ("Feldname") REFERENCES "anderer Tabellenname" ("Feldname") → Die aufgeführten Felder dieser Tabelle sind mit den Feldern einer anderen Tabelle sind verknüpft. Der Feldwert muss auf «Referentielle Integrität» geprüft werden (Fremdschlüssel), d.h. es muss ein entsprechender Primärschlüssel in der anderen Tabelle existieren, wenn hier ein Wert eingetragen wird.

[ON {DELETE | UPDATE} {CASCADE | SET DEFAULT | SET NULL}] → Wenn ein Fremdschlüssel besteht, so ist hier zu klären, wie damit verfahren werden soll, wenn z.B. der fremde Datensatz gelöscht wird. Z.B. macht es keinen Sinn, in einer Ausleihtabelle einer Bibliothek eine Nutzernummer weiter zu führen, wenn der Nutzer selbst gar nicht mehr existiert. Die entsprechenden Datensätze müssten behandelt werden, so dass die Beziehung zwischen den Tabellen stimmig bleibt. In der Regel würde der entsprechende Datensatz einfach gelöscht. Dies geschieht mit **ON DELETE CASCADE**.

CHECK(<Suchbedingung>) → Wird wie eine **WHERE**-Bedingung formuliert, bezieht sich aber nur auf den aktuellen Datensatz. Die Bedingungen sind unter *WHERE SQL-Expression* im Kapitel «Abfragen» aufgelistet.

```
CREATE TABLE "Zeitmessung" ("ID" INT PRIMARY KEY, "Startzeit" TIME,  
"Zielzeit" TIME, CHECK ("Startzeit" <= "Zielzeit"));
```

Mit der CHECK-Bedingung wird ausgeschlossen, dass eine "Zielzeit" eingegeben wird, die kleiner als die "Startzeit" ist. Wird dies versucht, so erscheint eine englischsprachige Fehlermeldung, die ungefähr folgendermaßen aussieht:

Check constraint violation SYS_CT_357 table: Zeitmessung ...

Der Suchbedingung wird hier gleich ein Name zugewiesen, der allerdings nicht sehr aussagekräftig ist. Stattdessen bietet es sich an, direkt bei der Tabellendefinition den Namen zu definieren:

```
CREATE TABLE "Zeitmessung" ("ID" INT PRIMARY KEY, "Startzeit" TIME,  
"Zielzeit" TIME, CONSTRAINT "Startzeit<=Zielzeit" CHECK ("Startzeit"  
<= "Zielzeit"));
```

Damit wird die Fehlermeldung etwas klarer. Der Name der Bedingung drückt dann wenigstens aus, worauf sie sich bezieht.

Mit Constraints wird vor allem gearbeitet, wenn die Beziehung zwischen Tabellen oder der Index für bestimmte Felder festgelegt werden soll. Die Constraints werden, bis auf die CHECK-Bedingung, in der GUI unter **Extras → Beziehungen** und als Indexentwurf in dem Tabellenentwurf unter **Extras → Indexentwurf** festgelegt.

[ON COMMIT {DELETE | PRESERVE} ROWS]: (HSQLDB)

Der Inhalt von Tabellen des Typs **TEMPORARY** oder **TEMP** wird nach Beendigung der Arbeit mit dem Datensatz standardmäßig gelöscht (**ON COMMIT DELETE ROWS**). Hier kann also nur ein flüchtiger Datensatz erstellt werden, der Informationen für andere Aktionen, die gleichzeitig laufen, vorhält.

Sollen diese Tabellentypen Daten für eine ganze Sitzung (Aufruf einer Datenbank und Schließen einer Datenbank) zur Verfügung stehen, so kann hier **ON COMMIT PRESERVE ROWS** gewählt werden.

Tabellenänderung

Manchmal wünscht sich der User, dass ein zusätzliches Feld an einer bestimmten Stelle in die Tabelle eingebaut wird. Angenommen es gibt die Tabelle "Adresse" mit den Feldern "ID", "Name", "Strasse" usw. Jetzt fällt dem Nutzer auf, dass vielleicht eine Unterscheidung in Name und Vorname sinnvoll wäre:

```
ALTER TABLE "Adresse" ADD "Vorname" VARCHAR(25) BEFORE "Name";
```

ALTER TABLE "Adresse": Ändere die Tabelle mit dem Namen "Adresse".

ADD "Vorname" VARCHAR(25): füge das Feld "Vorname" mit einer Länge von 25 Zeichen hinzu.

BEFORE "Name": und zwar vor dem Feld "Name". (HSQLDB)

In **FIREBIRD** muss der Schritt der Zuordnung nach dem Einfügen des Feldes erfolgen. Dafür ist die Position universell eben auch für alte Felder wählbar:

```
ALTER TABLE "Adresse" ADD "Vorname" VARCHAR(25);  
ALTER TABLE "Adresse" ALTER "Vorname" POSITION 2;
```

POSITION: Ändere die Position des Feldes zu der entsprechenden Stelle, hier als 2. Feld. Wird die Zahl kleiner 1 gewählt, so erscheint eine Errormeldung. Wird die Zahl größer als die Zahl der Felder gewählt, so wird das Feld als letztes Feld gesetzt.

Die Möglichkeit, die Position nach dem Erstellen einer Tabelle für zusätzliche Felder zu bestimmen, bietet die GUI nicht.

```
ALTER TABLE "Tabellenname" ADD <Felddefinition> [BEFORE "bereits_existierender_Feldname"];  
ALTER TABLE "Tabellenname" DROP "Feldname";
```

Das Feld "Feldname" wird aus der Tabelle "Tabellenname" gelöscht. Dies wird allerdings unterbunden, wenn das Feld in einer Ansicht (View) oder als Fremdschlüssel in einer anderen Tabelle Bedeutung hat.

```
ALTER TABLE "Tabellenname" ALTER COLUMN "Feldname" RENAME TO  
"neuer_Feldname";
```

Dies ändert den Namen eines Feldes. **RENAME TO** ist bei der **HSQLDB** erforderlich. Bei **FIREBIRD** funktioniert die Namensänderung nur mit **TO**.

```
ALTER TABLE "Tabellenname" ALTER COLUMN "Feldname" SET DEFAULT <Standardwert>;
```

Fügt dem Feld einen bestimmten Standardwert hinzu. **NULL** entfernt einen bestehenden Standardwert.

```
ALTER TABLE "Tabelle" ALTER COLUMN "Datum" SET DEFAULT CURRENT_DATE;
```

Dies ändert ein Datumsfeld so, dass das momentane Datum eingefügt wird, wenn das Feld "Datum" leer ist.

```
ALTER TABLE "Tabellenname" ALTER COLUMN "Feldname" SET [NOT] NULL
```

Setzt oder entfernt eine **NOT NULL** Bedingung für ein Feld.

```
ALTER TABLE "Tabellenname" ALTER COLUMN <Felddefinition>;
```

Die Felddefinition entspricht der aus der [Tabellenerstellung](#) mit den folgenden Einschränkungen:

- Das Feld muss bereits ein Primärschlüsselfeld sein um die Eigenschaft **IDENTITY** zu akzeptieren. **IDENTITY** bedeutet, dass das Feld die Eigenschaft «Autowert» erhält. Dies ist nur bei **INTEGER** oder **BIGINT** möglich. Zu den Feldtypenbezeichnungen siehe [Datentypen des Tabelleneditors](#) im Anhang dieses Handbuches.
- Wenn das Feld bereits die Eigenschaft **IDENTITY** hat und sie wird nicht in die Felddefinition erneut aufgenommen, so wird die bereits existierende Eigenschaft **IDENTITY** entfernt.
- Der Standardwert wird der der neuen Felddefinition sein. Wenn die Definition des Standardwertes leer gelassen wird, so wird ein bereits bestehender entfernt.
- Die Eigenschaft **NOT NULL** wird in die neue Definition übernommen, wenn nicht anders definiert. Dies entspricht dem Umgang mit dem Standardwert.
- Abhängig von der Art der Änderung muss eventuell die Tabelle leer sein, damit die Änderung durchgeführt werden kann. Auf jeden Fall wird die Änderung dann funktionieren, wenn die Änderung grundsätzlich möglich ist (z.B. Änderung von **NOT NULL** auf **NULL**) und die existierenden Werte alle umgewandelt werden können (z.B. von **TINYINT** zu **INTEGER**).

```
ALTER TABLE "Tabelle" ADD PRIMARY KEY ("Feldname1", "Feldname2" ...);
```

Dieser Befehl erstellt im Nachhinein einen Primärschlüssel, auch über mehrere Felder.

```
ALTER TABLE "Tabellenname"  
ALTER COLUMN "Feldname" RESTART WITH <neuer_Feldwert>;
```

Dieser Befehl wird ausschließlich für ein **IDENTITY** Feld genutzt. Damit wird der nächste Wert eines Feldes mit Autowert-Funktion festgelegt. Dies kann z.B. genutzt werden, wenn eine Datenbank erst einmal mit Testdaten versehen wurde, bevor sie mit den eigentlichen Daten bestückt wurde. Dann wird der Inhalt der Tabellen gelöscht und der neue Feldwert z.B. als 1 festgelegt. Der neue Feldwert ist dabei ohne einfache Anführungszeichen als Zahl einzugeben.

```
ALTER TABLE "Tabellenname"  
ADD [CONSTRAINT "Bedingungsname"] CHECK (<Suchbedingung>;
```

Dies fügt eine mit **CHECK** eingeleitete Suchbedingung hinzu. Solch eine Bedingung wird nicht auf bereits bestehende Datensätze angewandt, sondern bei allen zukünftigen Änderungen und neu erstellten Datensätzen berücksichtigt. Wird kein Bedingungsname definiert, so wird automatisch eine Bezeichnung zugewiesen. Beispiel:

```
ALTER TABLE "Ausleihe" ADD CHECK  
(COALESCE("Rueckdatum", "Leihdatum")>="Leihdatum")
```

Die Tabelle "**Ausleihe**" soll in Bezug auf Fehleingaben abgesichert werden. Es soll vermieden werden, dass ein Rückgabedatum angegeben wird, das vor dem Ausleihdatum liegt. Taucht jetzt bei der Eingabe des Rückgabedatums dieser Fehler auf, so erscheint die Fehlermeldung **check constraint violation ...**

```
ALTER TABLE "Tabellenname"  
ADD [CONSTRAINT "Bedingungsname"] UNIQUE ("Feldname1",  
"Feldname2" ...);
```

Hier wird hinzugefügt, dass die benannten Felder nur jeweils verschiedene Werte beinhalten dürfen. Werden mehrere Felder benannt, so gilt dies für die Kombination von Feldern. **NULL** wird hierbei nicht berücksichtigt. Ein Feld kann also ohne weiteres mehrmals die gleichen Werte haben, wenn das andere Feld bei den entsprechenden Datensätzen **NULL** ist.

Der Befehl wird nicht funktionieren, wenn bereits eine **UNIQUE** – Bedingung für die gleiche Felderkombination existiert.

```
ALTER TABLE "Tabellenname"
```

```
ADD [CONSTRAINT "Bedingungsname"] PRIMARY KEY ("Feldname1", "Feldname2" ...);
```

Fügt einen Primärschlüssel, gegebenenfalls mit einer Bedingungsdefinition, einer Tabelle hinzu. Die Syntax der Bedingungsdefinition entspricht der der Erstellung bei einer Tabelle.

```
ALTER TABLE "Tabellenname"  
ADD [CONSTRAINT "Bedingungsname"] FOREIGN KEY ("Feldname1",  
"Feldname2" ...) REFERENCES "Tabellenname_der_anderen_Tabelle"  
("Feldname1_andere_Tabelle", "Feldname2_andere_Tabelle" ...) [  
ON {DELETE | UPDATE} {CASCADE | SET DEFAULT | SET NULL}];
```

Hiermit wird eine Fremdschlüsselbedingung (**FOREIGN KEY**) zur Tabelle hinzugefügt. Die Syntax ist die gleiche wie bei der Erstellung einer Tabelle.

Das Verfahren wird mit einer Fehlermeldung beendet, wenn nicht für jeden Wert in der Tabelle ein entsprechender Wert aus der Tabelle mit dem entsprechenden Schlüsselfeld vorhanden ist.

Beispiel: Die Tabellen "Name" und "Adresse" sollen miteinander verbunden werden. In der Tabelle "Name" gibt es ein Feld mit der Bezeichnung "Adresse_ID". Dies soll mit seinen Werte mit dem Feld "ID" der Tabelle "Adresse" verbunden werden. Steht in "Adresse_ID" bereits der Wert 1, in dem "ID" der Tabelle "Adresse" aber nicht, so kann die Verbindung nicht funktionieren. Ebenfalls unmöglich ist es, wenn der Feldtyp in beiden Feldern nicht übereinstimmt.

```
ALTER TABLE "Tabellenname" DROP CONSTRAINT "Bedingungsname";
```

Der Befehl entfernt eine mit Namen versehene Bedingung (**UNIQUE**, **CHECK**, **FOREIGN KEY**) aus einer Tabelle.

```
ALTER TABLE "Tabellenname" RENAME TO "neuer_Tabellenname"; (HSQLDB)
```

Mit diesem Befehl schließlich wird einfach nur der Name einer Tabelle geändert.

Mit **FIREBIRD** ist es nicht möglich, einen Tabellennamen zu ändern. Stattdessen muss hier eine neue Tabelle mit neuem Namen und den alten Daten erstellt werden.

Hinweis

Bei der Änderung einer Tabelle über SQL wird die Änderung zwar in der Datenbank übernommen, nicht aber unbedingt sofort überall in der GUI sichtbar und verfügbar. Wird die Datenbank geschlossen und wieder geöffnet, so werden die Änderungen auch in der GUI angezeigt.

Die Änderungen werden auch dann angezeigt, wenn im Tabellencontainer **Ansicht → Tabellen aktualisieren** aufgerufen wird.

Tabellen löschen

```
DROP TABLE "Tabellenname" [IF EXISTS] [RESTRICT | CASCADE];
```

Löscht die Tabelle "Tabellenname".

Firebird kennt hier keine weiteren Optionen. Sobald über Verknüpfungen zu anderen Tabellen oder Ansichten Probleme auftauchen, wird das Löschen mit einer Fehlermeldung abgebrochen.

IF EXISTS schließt aus, dass eine Fehlermeldung erscheint, falls diese Tabelle nicht existiert. (HSQLDB)

RESTRICT ist die Standardeinstellung und muss nicht definitiv gewählt werden, d.h. ein Löschen wird dann nicht ausgeführt, wenn die Tabelle mit irgendeiner anderen Tabelle durch einen Fremdschlüssel verbunden wurde oder auf die Tabelle mit einer Ansicht (View) Bezug genommen wird. Abfragen sind davon nicht berührt, da die innerhalb der HSQLDB nicht gespeichert sind. (HSQLDB)

Wird statt **RESTRICT** **CASCADE** gewählt, so werden alle Beziehungen zu der Tabelle "Tabellenname" gelöscht. In den verknüpften Tabellen werden dann alle Fremdschlüsselfelder auf NULL

gesetzt. Alle Tabellenansichten (*Views*), in denen auf die entsprechende Tabelle Bezug genommen wird, werden komplett gelöscht. (HSQLDB)

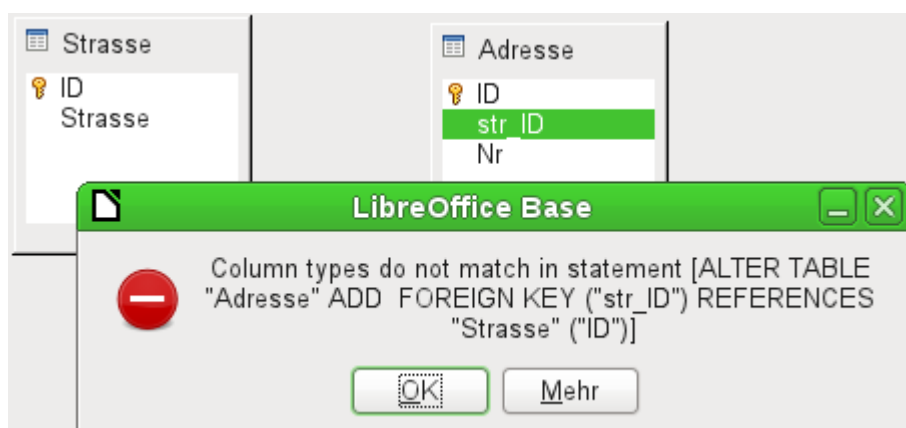
Verknüpfung von Tabellen

Prinzipiell kommt eine Datenbank auch ohne die Verknüpfung von Tabellen aus. Der Nutzer muss dann bei der Eingabe selbst darauf achten, dass die Beziehungen zwischen den Tabellen stimmig bleiben. In der Regel geschieht dies, indem er sich entsprechende Formulare erstellt, die dies bewerkstelligen sollen.

Das Löschen von Datensätzen bei verknüpften Tabellen ist nicht so einfach möglich. Angenommen es würde aus der Tabelle "Strasse" in [Abbildung 6](#) eine "Strasse" gelöscht, die aber durch die Verknüpfung mit der Tabelle "Adresse" in der Tabelle "Adresse" noch als Fremdschlüssel vertreten ist. Der Verweis in der Tabelle "Adresse" würde ins Leere gehen. Hiergegen sperrt sich die Datenbank, sobald der Relationenentwurf erstellt wurde. Um die "Strasse" löschen zu können, muss die Vorbedingung erfüllt sein, dass sie nicht mehr in "Adresse" benötigt wird.

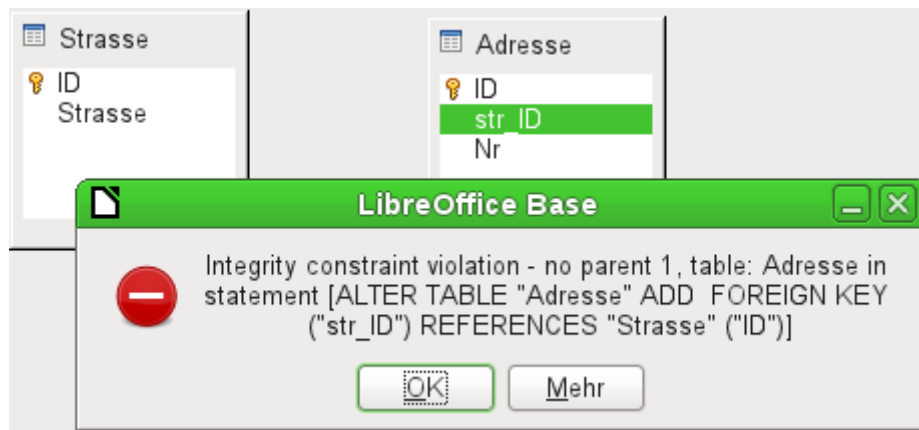
Die grundsätzlichen Verknüpfungen werden über **Extras → Beziehungen** festgelegt. Hierbei wird eine Verbindungslinie von dem Primärschlüssel einer Tabelle zum zu definierenden Sekundärschlüssel gezogen.

Die folgenden Fehlermeldungen können beim Ziehen so einer Verbindung auftreten:



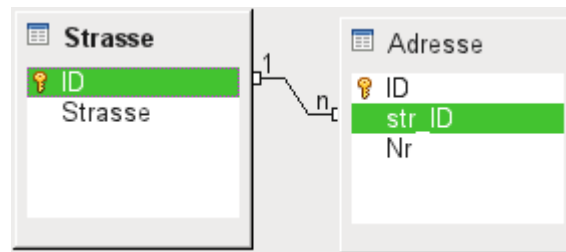
Die Meldung gibt einen englischen Text sowie das zu der Fehlermeldung führende SQL-Kommando wieder. Eine gute Möglichkeit also, auch an dieser Stelle etwas über die Sprache zu erfahren, mit der die Datenbank arbeitet.

«Column types do not match in statement» - die Spaltentypen stimmen in der SQL-Formulierung nicht überein. Da das SQL-Kommando gleich mitgeliefert wird, müssen das die Spalten *Adresse.str_ID* und *Strasse.ID* sein. Zu Testzwecken wurde hier das eine Feld als Integer, das andere als 'Tiny Integer' definiert. So eine Verbindung lässt sich nicht erstellen, da das eine Feld nicht die gleichen Werte annehmen kann wie das andere Feld.

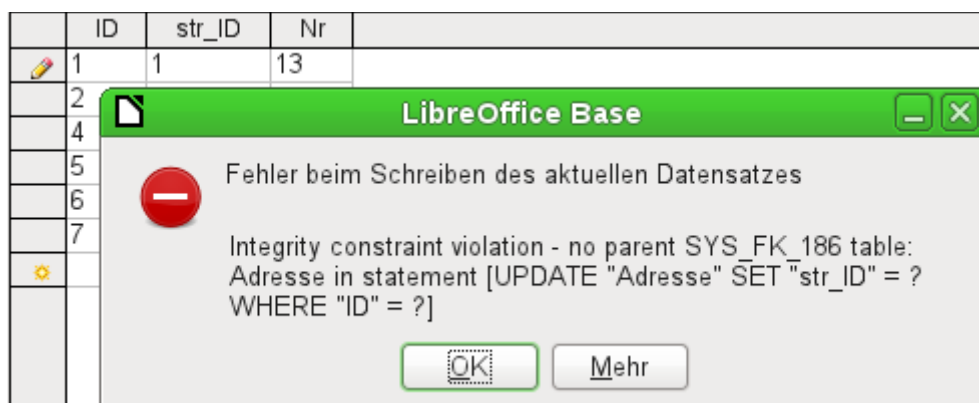


Jetzt stimmen die Spaltentypen überein. Die SQL-Formulierung (statement) ist die gleiche wie beim ersten Zugriff. Aber wieder taucht ein Fehler auf:

«Integrity constraint violation – no parent 1, table: Adresse ...» - die Integrität der Beziehung ist nicht gewährleistet. In dem Feld der Tabelle Adresse, also *Adresse.str_ID*, gibt es eine Ziffer 1, die es im Feld *Strasse.ID* nicht gibt. Parent ist hier die Tabelle *Strasse*, weil deren Primärschlüssel vorhanden sein muss. Dieser Fehler tritt häufig auf, wenn zwei Tabellen miteinander verbunden werden sollen, bei denen in den Feldern der Tabelle mit dem zukünftigen Fremdschlüssel schon Daten eingegeben wurden. Steht in dem Fremdschlüssel-Feld ein Eintrag, der in der Parent-Tabelle (Eltern-Tabelle, also der Tabelle, aus der der Primärschlüssel gestellt wird) nicht vorhanden ist, so würde ein ungültiger Eintrag erzeugt.



Ist die Verbindung erfolgreich erzeugt worden und wird später versucht einen entsprechend fehlerhaften Eintrag in die Tabelle einzugeben, so kommt die folgende Fehlermeldung:



Also wiederum die Integritätsverletzung. Base weigert sich, für das Feld *str_ID* nach der Verknüpfung den Wert 1 anzunehmen, weil die Tabelle *Strasse* so einen Wert im Feld *ID* nicht enthält.

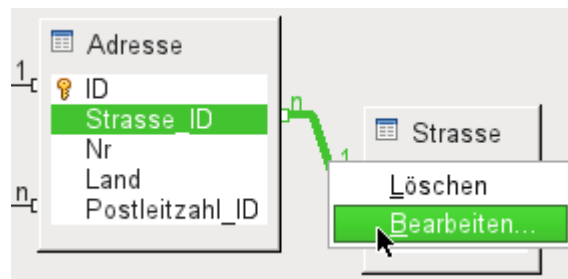


Abbildung 11: Durch Klick mit der rechten Maustaste können Verknüpfungen bearbeitet werden.

Die Eigenschaften der Verknüpfung können so bearbeitet werden, dass beim Löschen von Datensätzen aus der Tabelle *Strasse* gleichzeitig eventuell vorhandene Einträge in der Tabelle *Adresse* auf NULL gesetzt werden.

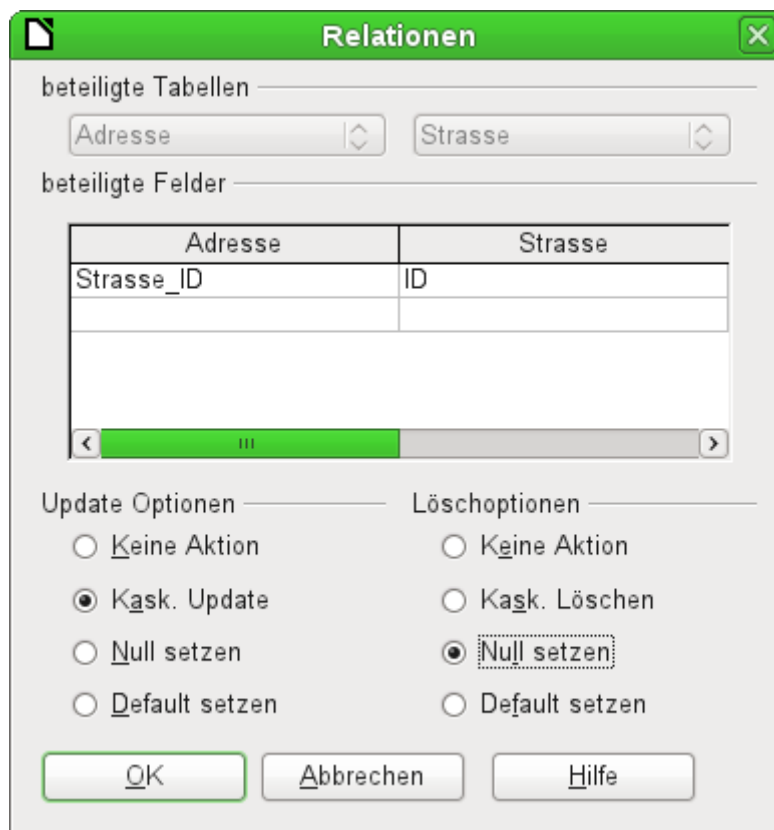


Abbildung 12: Dialog «Relationen»

Die oben abgebildeten Eigenschaften beziehen sich immer auf eine Aktion, die mit einer Änderung des Datensatzes aus der Tabelle zusammenhängt, zu der der betroffene Primärschlüssel gehört. In unserem Fall ist dies die Tabelle *Strasse*. Wird also dort **der Primärschlüssel eines Datensatzes "ID" geändert (Update)**, so können die folgenden Aktionen ausgeführt werden:

Keine Aktion:

Eine Änderung des Primärschlüssels *Strasse.ID* kann in diesem Fall nicht vorgenommen werden, da die Relation ansonsten zerstört wird. Da dies die Standardeinstellung der Relation ist gehen Nutzer häufig davon aus, dass eine Änderung des Primärschlüssels unmöglich ist. Die anderen Update-Optionen ermöglichen allerdings so eine Änderung.

Kask. Update:

Bei einer Änderung des Primärschlüsselwertes *Strasse.ID* allerdings wird der Fremdschlüsselwert automatisch auf den neuen Stand gebracht. Die Koppelung wird dadurch nicht beeinträchtigt. Wird z.B. der Wert von 3 auf 4 geändert, so wird bei allen Datensätzen aus *Adresse*, in denen der Fremdschlüssel *Adresse.Strasse_ID* 3 lautete, stattdessen eine 4 eingetragen.

Null setzen:

Alle Datensätze, die sich auf den Primärschlüssel bezogen haben, haben jetzt in dem Fremdschlüsselfeld *Adresse.Strasse_ID* stattdessen keinen Eintrag mehr stehen, sind also NULL.

Default setzen:

Wird der Primärschlüssel *Strasse.ID* geändert, so wird der damit ursprünglich verbundene Wert aus *Adresse.Strasse_ID* auf den dafür vorgesehenen Standardwert gesetzt. Hierfür ist allerdings eine eindeutige Definition eines Standardwertes erforderlich. Dies scheint die grafische Benutzeroberfläche bis zur Version LO 3.5 nicht bereitzustellen. Wird per SQL der Default-Wert durch

```
ALTER TABLE "Adresse" ALTER COLUMN "Strasse_ID" SET DEFAULT 1;
```

gesetzt, so übernimmt die Beziehungsdefinition auch die Zuweisung, dass bei einem Update auf diesen Wert zurückgegriffen werden kann. Wird also der Primärschlüssel aus der Tabelle *Strasse* geändert, so wird in der Tabelle *Adresse* das Fremdschlüsselfeld auf 1 gesetzt. Dies bietet sich z.B. an, wenn auf jeden Fall jeder Datensatz eine Straßenzuweisung erhalten soll, also nicht NULL sein soll. Aber Achtung: Ist 1 nicht belegt, so wird eine Verbindung zu einem nicht vorhandenen Wert erzeugt. Hier scheint die HSQLDB nicht mit letzter Konsequenz durchgedacht. Es ist also möglich, die Integrität der Relationen zu zerstören.

Vorsicht



Ist der Defaultwert im Fremdschlüsselfeld nicht durch einen Primärschlüssel der Ursprungstabelle belegt, so wird eine Verbindung zu einem nicht vorhandenen Wert erzeugt. Es ist also möglich, die referentielle Integrität der Datenbank zu zerstören.

Vor diesem Hintergrund scheint es sinnvoll, diese Einstellung nicht in Betracht zu ziehen.

Wird ein Datensatz aus der Tabelle *Strasse* **gelöscht**, so stehen die folgenden Optionen zur Verfügung:

Keine Aktion:

Es wird nichts unternommen. Ist ein Datensatz aus der Tabelle *Adresse* von der Löschung betroffen, so wird die Löschung abgelehnt. Es existiert schließlich weiterhin ein entsprechender Fremdschlüssel in der Tabelle *Adresse*.

Wie bei den Update-Optionen kann nur mit dieser Option eine Löschung des Datensatzes unterbunden werden. Die weiteren Optionen geben hingegen den Weg vor, den die Datenbank beschreiten soll, falls von dem Datensatz in der Tabelle *Strasse* ein Fremdschlüssel in der Tabelle *Adresse* betroffen ist.

Kaskadiert Löschen:

Wird ein Datensatz aus der Tabelle *Strasse* gelöscht und ist davon ein Datensatz aus der Tabelle *Adresse* betroffen, so wird auch dieser gelöscht.

Das mag in diesem Zusammenhang merkwürdig klingen, ergibt aber bei anderen Tabellenkonstruktionen sehr wohl einen Sinn. Angenommen es existiert eine Tabelle mit CDs und eine Tabelle, die die Titel, die auf diesen CDs enthalten sind, speichert. Wird nun ein Datensatz aus der CD-Tabelle gelöscht, so stehen lauter Titel in der anderen Tabelle, die gar keinen Bezug mehr haben, also gar nicht mehr vorhanden sind. Hier ist dann ein kaskadierendes Löschen sinnvoll. So muss der Nutzer nicht vor dem Entfernen der CD aus der Datenbank erst sämtliche Titel löschen.

Null setzen:

Dieses Verhalten entspricht der gleich lautenden Update-Option.

Default setzen:

Dieses Verhalten entspricht ebenfalls der gleich lautenden Update-Option. Die Bedenken bei dieser Option sind entsprechend die gleichen.

Tipp

Sollen möglichst Fehlermeldungen der Datenbank vermieden werden, da sie dem Datenbanknutzer vielleicht nicht deutbar sind, so sind auf jeden Fall die Einstellungen «Keine Aktion» zu vermeiden.

In **Extras → Beziehungen** können durch Ziehen mit der Maus nur Fremdschlüssel erstellt werden, die sich auf ein Feld einer Tabelle beziehen. Für die Verbindung mit einer Tabelle, die einen zusammengesetzten Primärschlüssel hat, muss in **Extras → Beziehungen** der Menüpunkt **Einfügen → Neue Relation** oder der entsprechende Button aufgesucht werden. Es erscheint dann der *Dialog «Relationen»* mit der freien Auswahl der beteiligten Tabellen.

Eingabe von Daten in Tabellen

Datenbanken, bestehend aus einer Tabelle, erfordern in der Regel keine Formulare zur Eingabe, es sei denn, sie enthalten ein Feld für Bildmaterial. Sobald allerdings eine Tabelle den Fremdschlüssel einer anderen Tabelle enthält, muss der Nutzer entweder auswendig wissen, welche Schlüsselnummern er eintragen muss, oder er muss die andere Tabelle zum Nachschauen gleichzeitig offen halten. Dafür wird dann spätestens ein Formular sinnvoll.

Eingabe über die grafische Benutzeroberfläche der Tabelle

Die Tabelle aus dem Tabellencontainer wird durch einen Doppelklick geöffnet. Wird der Primärschlüssel durch ein automatisch hochzählendes Feld erstellt, so enthält eins der jetzt zu sehenden Felder bereits den Text «AutoWert». Im «AutoWert»-Feld ist keine Eingabe möglich. Dieser Wert kann gegebenenfalls erst nach Abspeicherung des Datensatzes geändert werden.



Abbildung 13: Eingabe in Tabellen - Spalten ausblenden.



Abbildung 14: Eingabe in Tabellen - Spalten wieder einblenden.

Einzelne Spalten der Tabellenansicht können auch ausgeblendet werden. Wenn z.B. das Primärschlüsselfeld nicht unbedingt sichtbar sein soll, so lässt sich das in der Tabelleneingabe einstellen.

Diese Einstellung wird als Einstellung der GUI abgespeichert. Die Spalte existiert in der Tabelle weiter und kann gegebenenfalls auch wieder eingeblendet werden.

Die Eingabe in die Tabellenzellen erfolgt in der Regel von links nach rechts über Tastatur und Tabulator. Natürlich ist auch die Nutzung der Maus möglich.

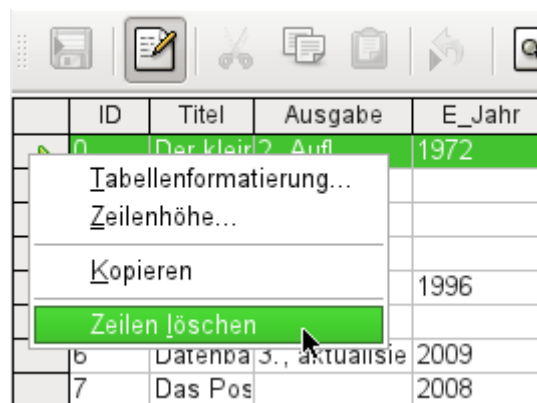
Nach Erreichen des letzten Feldes eines Datensatzes springt der Cursor automatisch in den nächsten Datensatz. Die vorhergehende Eingabe wurde dabei abgespeichert. Eine zusätzliche Abspeicherung unter **Datei → Speichern** ist bei der HSQLDB nicht nötig und nicht möglich. Die Daten sind bereits in der Datenbank gelandet, bei der HSQLDB also im Arbeitsspeicher. Sie werden (aus Sicht der Datensicherheit leider) erst auf der Festplatte abgespeichert, wenn Base geschlossen wird. Wenn Base also aus irgendwelchen Gründen nicht ordnungsgemäß beendet werden kann, kann dies immer noch zu Datenverlusten führen. Bei Firebird hingegen muss die Datenbank ausdrücklich vor dem Schließen abgespeichert werden.

Fehlt eine Eingabe, die vorher im Tabellenentwurf als zwingend notwendig deklariert wurde (**NOT NULL**), so wird eine entsprechende Fehlermeldung erzeugt:

Attempt to insert null into a non-nullable column ...

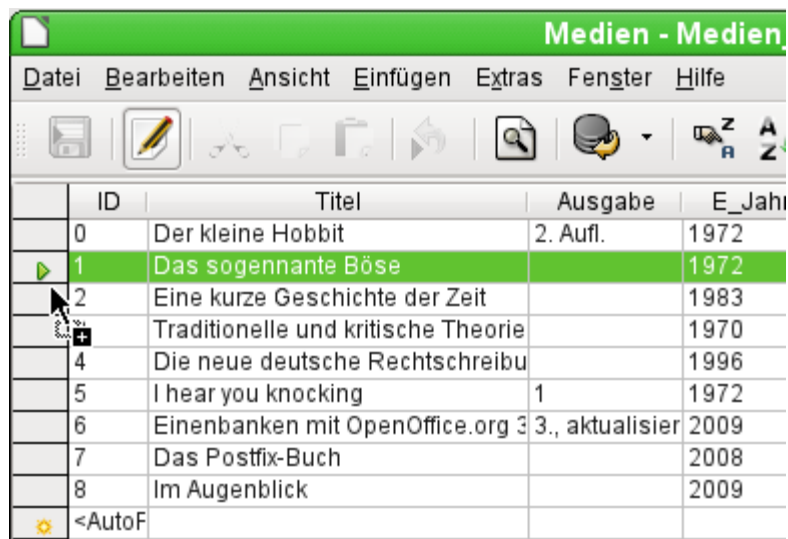
Die entsprechende Spalte, die Tabelle dazu und der von der GUI abgesetzte SQL-Befehl werden angezeigt.

Die Änderung eines Datensatzes ist entsprechend einfach möglich: Feld aufsuchen, geänderten Wert eingeben und Datenzeile wieder verlassen.



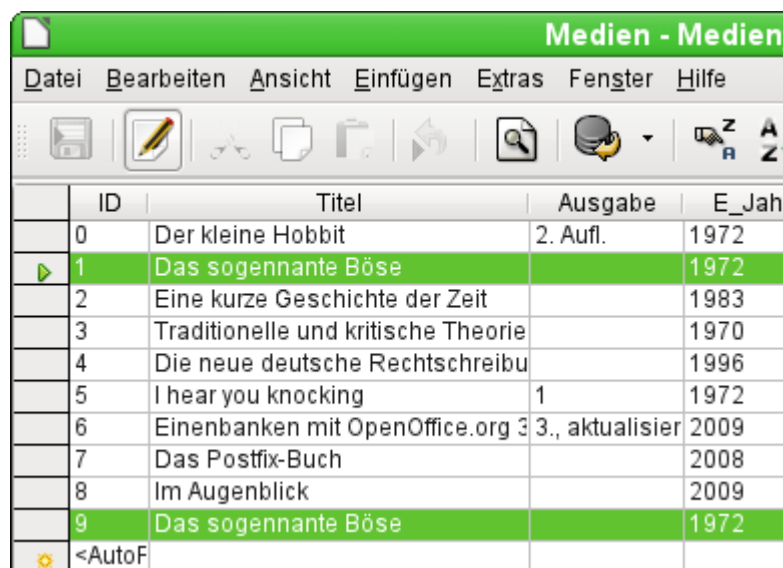
Zum Löschen eines Datensatzes wird der Datensatz auf dem Zeilenkopf markiert, dann die rechte Maustaste gedrückt und **Zeilen löschen** gewählt.

Recht gut versteckt gibt es auch die Möglichkeit, ganze Zeilen zu kopieren. Hierzu muss allerdings der Primärschlüssel der Tabelle als «AutoWert» definiert sein.



ID	Titel	Ausgabe	E_Jahr
0	Der kleine Hobbit	2. Aufl.	1972
1	Das sogenannte Böse		1972
2	Eine kurze Geschichte der Zeit		1983
3	Traditionelle und kritische Theorie		1970
4	Die neue deutsche Rechtschreibu		1996
5	I hear you knocking	1	1972
6	Einenbanken mit OpenOffice.org 3	3., aktualisier	2009
7	Das Postfix-Buch		2008
8	Im Augenblick		2009
<AutoF			

Der Zeilenkopf wird mit der linken Maustaste markiert. Die Maustaste wird gehalten. Das Cursor-Symbol verwandelt sich in ein Symbol mit einem Plus-Zeichen, das andeutet, dass etwas hinzugefügt wird. Sobald dieses Symbol erscheint, kann die Maustaste losgelassen werden.



ID	Titel	Ausgabe	E_Jahr
0	Der kleine Hobbit	2. Aufl.	1972
1	Das sogenannte Böse		1972
2	Eine kurze Geschichte der Zeit		1983
3	Traditionelle und kritische Theorie		1970
4	Die neue deutsche Rechtschreibu		1996
5	I hear you knocking	1	1972
6	Einenbanken mit OpenOffice.org 3	3., aktualisier	2009
7	Das Postfix-Buch		2008
8	Im Augenblick		2009
9	Das sogenannte Böse		1972
<AutoF			

Der Datensatz mit dem Primärschlüsselwert '1' wird als neuer Datensatz mit dem neuen Primärschlüsselwert '9' eingefügt.

Wird nicht nur ein Datensatz markiert, sondern unter Zuhilfenahme der **Shift**- oder der **Strg**-Taste gleich mehrere Datensätze, so können auch mehrere Datensätze direkt kopiert werden.

Tipp

Die Spaltenköpfe können auf die für die Eingabe erforderliche Breite gezogen werden. Wird dies in der Tabelle gemacht, so wird die Speicherfunktion von Base aktiviert. Base speichert diese Spaltenbreiten aus den Tabellen.

Die Spaltenbreiten der Tabellen beeinflussen die Spaltenbreiten in Abfragen. Sind Spalten in Abfragen zu klein, so hilft dort nur vorübergehend eine Breitenänderung. Die Einstellung der Abfrage wird nicht gespeichert. Hier hilft dann eine Breitenänderung in der Tabelle, sofern es sich in der Abfrage nicht um Felder handelt, die durch irgendwelche Funktionen beeinflusst wurden.

Hilfreich zum Aufsuchen des entsprechenden Datensatzes sind die Sortier-, Such- und Filterfunktionen.

Sortieren von Tabellen



Titel	Ausgabe	E Jahr	A
Traditionelle und kritische Theorie			
Im Augenblick		2009	
I hear you knocking	1		
Eine kurze Geschichte der Zeit			
Die neue deutsche Rechtschreibung		1996	
Der kleine Hobbit	2. Aufl.	1972	

Abbildung 15: Schnelle Sortierung

Die schnelle Sortiervariante verbirgt sich hinter den Buttons **A → Z** bzw. **Z → A**. Ein Feld innerhalb einer Spalte wird aufgesucht, ein Mausklick auf den Button und nach der Spalte wird sortiert. Hier wurde gerade die Spalte "Titel" absteigend sortiert.

Mit der schnellen Variante kann immer nur nach einer Spalte sortiert werden. Für eine Sortierung nach mehreren Spalten ist eine weitere Sortierfunktion vorgesehen:

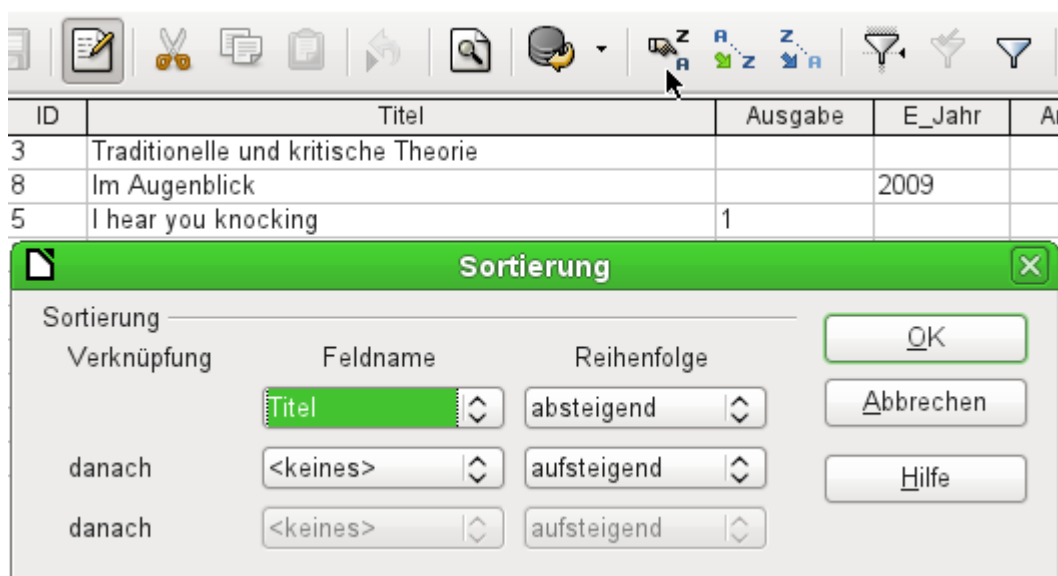
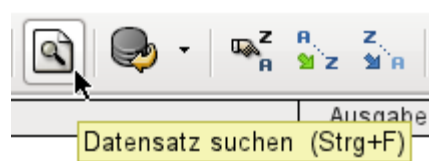


Abbildung 16: Sortierung nach mehreren Spalten

Der Feldname der Spalte sowie die jeweilige Sortierreihenfolge werden ausgesucht. Wurde vorher bereits eine schnelle Sortierung vorgenommen, so ist in der ersten Zeile der entsprechende Feldname und die Sortierreihenfolge bereits eingetragen.

Suchen in Tabellen



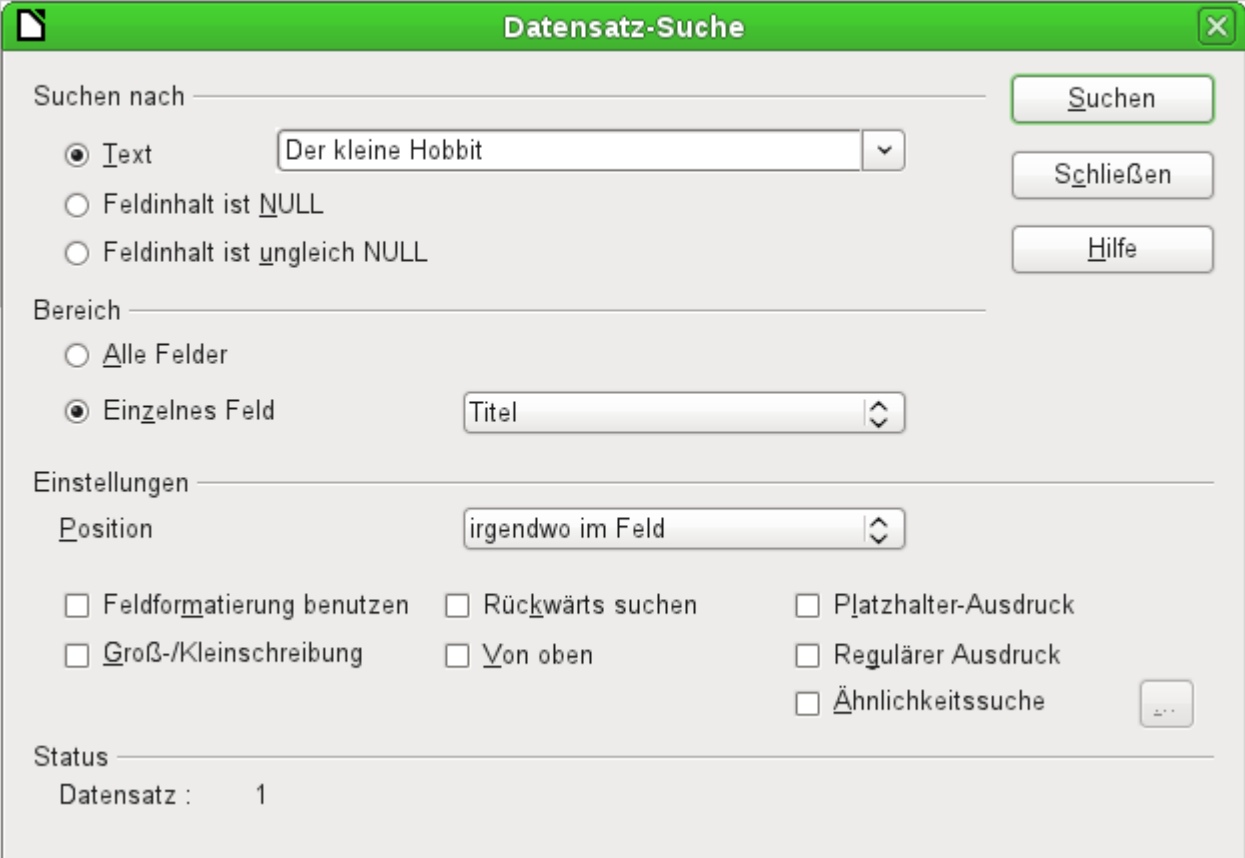
Die Funktion zum Suchen von Datensätzen ist recht umfangreich und für durch Suchmaschinen verwöhnte Nutzer nicht gerade die erste Wahl, um einen bestimmten Datensatz zu finden. Außerdem ist die Suchfunktion bei größeren Datenmengen äußerst langsam, da die Suche nicht mit SQL-Befehlen innerhalb der Datenbank erfolgt. Für eine schnellere Suche bietet sich stattdessen

an, statt der Tabelle eine entsprechende Abfrage zu nutzen. Diese Abfrage kann dann mit Hilfe von Parametern gestartet und durchsucht werden. Siehe dazu: [Verwendung von Parametern in Abfragen](#).

Tipp

Bevor die Suche aufgerufen wird, sollte auf jeden Fall darauf geachtet werden, dass die zu durchsuchenden Spalten von der Breite her weit genug eingestellt sind, um den gefundenen Datensatz auch anzuzeigen. Die Suchfunktion bleibt nämlich im Vordergrund und lässt keine Korrektur der Einstellungen der Spaltenweite in der darunterliegenden Tabelle zu. Um an die Tabelle zu gelangen, muss die Suche also abgebrochen werden.

Titel	Ausgabe	E_Jahr	Anmerkung	Kategorie
Der kleine Hobbit	2. Aufl.	1972		0



Das Bild zeigt eine Screenshot-Ansicht einer Software-Oberfläche. Oben ist eine Tabelle mit den Spalten 'Titel', 'Ausgabe', 'E_Jahr', 'Anmerkung' und 'Kategorie' zu sehen. Die erste Zeile enthält den Text 'Der kleine Hobbit', '2. Aufl.', '1972', eine leere Zelle und die Zahl '0'. Darunter befindet sich ein Dialogfenster mit der Überschrift 'Datensatz-Suche'. Innerhalb dieses Fensters gibt es verschiedene Einstellungsoptionen: 'Suchen nach' mit einem Textfeld, das 'Der kleine Hobbit' enthält; drei radio-kontrollierte Optionen ('Text', 'Feldinhalt ist NULL', 'Feldinhalt ist ungleich NULL'), wobei 'Text' ausgewählt ist; 'Bereich' mit 'Alle Felder' und 'Einzelnes Feld' (ausgewählt), letzteres mit einem Dropdown-Menü, das 'Titel' zeigt; 'Einstellungen' mit 'Position' (Dropdown, 'irgendwo im Feld') und sechs checkbox-Optionen ('Feldformatierung benutzen', 'Rückwärts suchen', 'Platzhalter-Ausdruck', 'Groß-/Kleinschreibung', 'Von oben', 'Regulärer Ausdruck', 'Ähnlichkeitssuche'). Am unteren Rand des Dialogs steht 'Status: Datensatz : 1'. Rechts im Dialog befinden sich drei Buttons: 'Suchen', 'Schließen' und 'Hilfe'.

Abbildung 17: Eingabemaske zur Datensatzsuche

Die Suche übernimmt beim Aufruf den Begriff des Feldes, von dem aus sie aufgerufen wurde.

Damit die Suche effektiv verläuft, sollte der Suchbereich möglichst weit eingegrenzt sein. So dürfte es sinnlos sein, den obigen Text aus dem Feld "Titel" in dem Feld "Autor" zu suchen. Stattdessen wird bereits als einzelnes Feld der Feldname "Titel" vorgeschlagen.

Die weiteren Einstellungen der Datensatzsuche können die Suche nach bestimmten Kombinationen vereinfachen. Als Platzhalter-Ausdruck sind hier die in SQL üblichen Platzhalterbezeichnungen («_» für ein variables Zeichen, «%» für eine beliebige Anzahl variabler Zeichen und «\» als Escape-Zeichen, um auch nach den variablen Zeichen selbst suchen zu können).

Reguläre Ausdrücke werden in der Hilfe von LibreOffice unter diesem Begriff ausführlich aufgeführt. Ansonsten gibt sich die Hilfestellung zu diesem Modul recht sparsam. Ein Klick auf den Button **Hilfe** landet bei LO 3.3.4 und LO 3.5.2 auf einer leeren Seite.

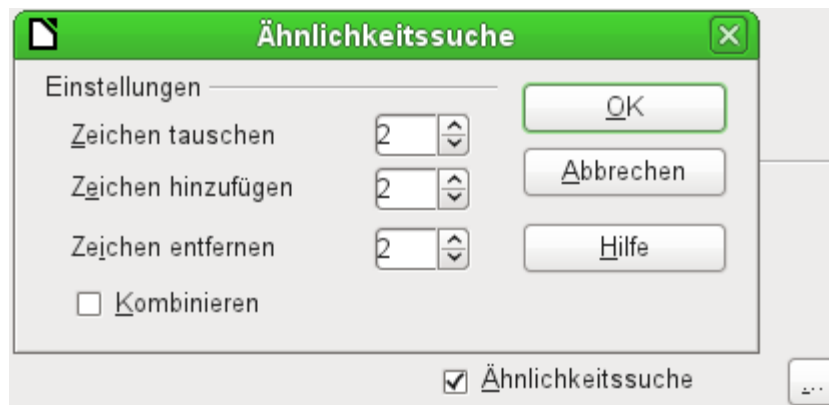


Abbildung 18: Eingrenzung der Ähnlichkeitssuche

Die Ähnlichkeitssuche lässt sich vor allem dann nutzen, wenn es darum geht, Schreibfehler auszu-schließen. Je höher die Werte in den Einstellungen gesetzt werden, desto mehr Datensätze wer-den schließlich in der Trefferliste verzeichnet sein.

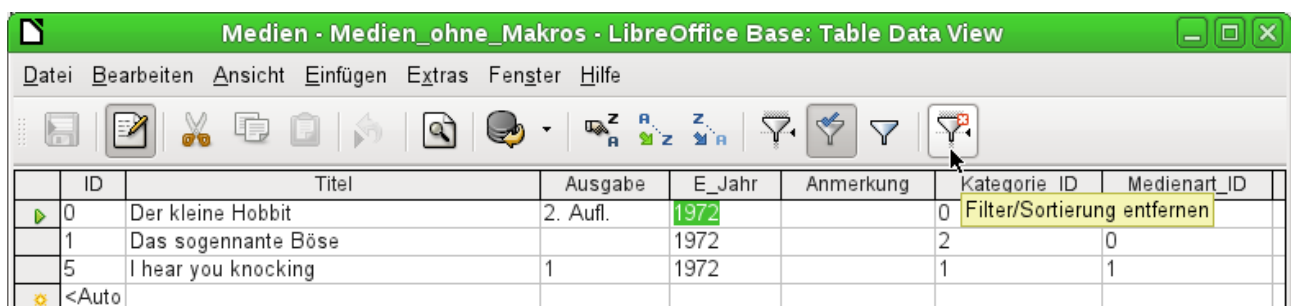
Insgesamt ist dieses Suchmodul eher etwas für Leute, die durch häufige Anwendung genau wis-sen, an welchen Stellen sie zum Erreichen eines Ziels drehen müssen. Für den Normaluser dürfte die Möglichkeit, Datensätze durch Filter zu finden, schneller zum Ziel führen.

Für Formulare ist in einem der folgenden Kapitel beschrieben, wie mittels SQL, und erweitert mit Makros, eine Stichwortsuche schneller zum Ziel führt.

Filtern von Tabellen



Die schnelle Filterung läuft über den AutoFilter. Der Cursor wird in ein Feld gesetzt, der Filter über-nimmt nach einem Klick auf den Button diesen Feldinhalt. Es werden nur noch die Datensätze angezeigt, die dem Inhalt des gewählten Feldes entsprechen. Die folgende Abbildung zeigt die Fil-terung nach einem Eintrag in der Spalte "E_Jahr".

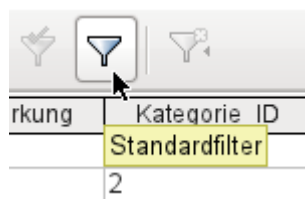


Der Filter ist aktiv. Dies ist an dem Filtersymbol mit einem grünen Haken zu erkennen. Das Filter-symbol erscheint gedrückt. Wird der Button erneut betätigt, so bleibt der Filter selbst erhalten, es werden aber wieder alle Datensätze angezeigt. So kann gegebenenfalls wieder in den Filterzu-stand zurückgeschaltet werden.

Durch Betätigung des ganz rechts stehenden Filtersymbols lassen sich alle bereits veranlassten Filterungen und Sortierungen wieder entfernen. Der Filter wird wieder inaktiv und kann nicht mehr mit seinem alten Wert aufgerufen werden.

Tipp

In eine Tabelle, die gefiltert oder durch Suche eingegrenzt wurde, können dennoch ganz normal Daten eingegeben werden. Sie bleiben so lange in der Tabellenansicht stehen, bis die Tabelle durch Betätigung des Buttons **Aktualisieren** aktualisiert wird.



Mit dem Standardfilter öffnet sich ein Fenster, in dem ähnlich der Sortierung eine Filterung über mehrere Zeilen ausgeführt werden kann. Ist bereits vorher ein AutoFilter eingestellt, so zeigt die erste Zeile des Standardfilters bereits diesen vorgefilterten Wert an.

Abbildung 19: Umfangreiche Datenfilterung im Standardfilter

Der Standardfilter bringt viele Funktionen einer SQL-Datenfilterung mit. Die folgenden SQL-Bedingungen stehen zur Verfügung:

Bedingung GUI	Beschreibung
=	Vollständige Gleichheit, entspricht dem Begriff wie, wenn keine zusätzlichen Platzhalterbezeichnungen verwendet werden.
<>	Ungleich
<	Kleiner als
<=	Kleiner als und gleich
>	Größer als
>=	Größer als und gleich
wie	Für Text, in Hochkommata geschrieben (' '); «_» für ein variables Zeichen, «%» für eine beliebige Anzahl variabler Zeichen (in der GUI auch «*»; wird intern umgewandelt). In SQL entspricht wie dem Begriff LIKE
nicht wie	Umkehrung von wie , in SQL NOT LIKE
leer	Kein Inhalt, auch nicht eine Leertaste. In SQL entspricht dies dem Begriff NULL
nicht leer	Umkehrung von leer , in SQL NOT NULL

Bevor die Verknüpfung eines Filterkriteriums mit dem nächsten Filterkriterium erfolgen kann, muss in der Folgezeile zumindest schon einmal ein Feldname ausgewählt worden sein. In der obigen

Abbildung steht dort statt eines Feldnamens «-keiner-», so dass die Verknüpfung inaktiv ist. Als Verknüpfung stehen hier **UND** und **ODER** zur Verfügung.

Als Feldname kann hier sowohl ein neuer Feldname als auch ein bereits ausgewählter Feldname erscheinen.

Selbst bei großen Datenbeständen dürfte sich bei geschickter Filterung die Anzahl der angezeigten Datensätze mit diesen 3 Bedingungsmöglichkeiten doch auf einen übersichtlichen Bestand eingrenzen lassen.

Auch für die Filterung werden für Formulare in einem der folgenden Kapitel einige weitere Möglichkeiten vorgestellt, die die GUI so nicht zur Verfügung stellt.

Eingabemöglichkeiten über SQL direkt

Die Eingabe direkt über SQL ist vor allem dann sinnvoll, wenn mehrere Datensätze mit einem Befehl eingefügt, geändert oder gelöscht werden sollen.

Neue Datensätze einfügen

```
INSERT INTO "Tabellenname" [( "Feldname" [, ...] )]  
{ VALUES("Feldwert" [, ...]) | <Select-Formulierung>};
```

Wird kein "Feldname" benannt, so müssen die Felder komplett und in der richtigen Reihenfolge der Tabelle als Werte übergeben werden. Dazu zählt auch das gegebenenfalls automatisch hochzählende Primärschlüsselfeld. Die Werte, die übergeben werden, können auch das Ergebnis einer Abfrage (<Select-Formulierung>) sein. Genauere Erläuterungen hierzu weiter unten.

```
INSERT INTO "Tabellenname" ("Feldname") VALUES ('Test');  
CALL IDENTITY(); (HSQLDB, FIREBIRD)
```

In die Tabelle wird in der Spalte "Name" der Wert 'Test' eingegeben. Das automatisch hochzählende Primärschlüsselfeld "ID" wird nicht angerührt. Der entsprechende Wert für die "ID" wird mit **CALL IDENTITY()** anschließend ausgelesen. Dies ist bei der Verwendung von Makros wichtig, damit entsprechend mit dem Wert dieses Schlüsselfeldes weiter gearbeitet werden kann. Bei Firebird sollte **RETURNING "ID"** direkt mit dem Befehl angegeben werden. Leider funktioniert aber unter LO 5.3 die Ausgabe der ID nicht, so dass die ID separat abgefragt werden muss.

```
INSERT INTO "Tabelle" ("Vorname", "Nachname") VALUES ('Eva', 'Müller');
```

Hiermit werden die Werte für 2 Felder auf einmal eingegeben. Die Reihenfolge der Felder und die Reihenfolge der entsprechenden Werte muss dabei gleich sein. **Text** ist bei den Werten in **einfache Anführungszeichen** zu setzen: 'Text'. Enthält der Text selbst einfache Anführungszeichen, so werden die einfachen Anführungszeichen wiederum durch einfache Anführungszeichen maskiert: **Robert's Datenbank**.

```
INSERT INTO "Tabelle" ("Ware", "Preis") VALUES ('Kaffee', 8.79);
```

Zahlenwerte werden **ohne einfache Anführungszeichen** eingegeben. Werte mit Nachkommastellen werden nicht mit einem Komma, sondern dem Dezimalpunkt eingegeben.

```
INSERT INTO "Tabellenname" ("Feldname") SELECT "anderer_Feldname" FROM  
"Name_anderer_Tabelle";
```

In die erste Tabelle werden jetzt so viele neue Datensätze in "Feldname" eingefügt, wie in der Spalte "anderer_Feldname" der zweiten Tabelle enthalten sind. Die SELECT-Formulierung kann hier natürlich einschränkend wirken.

Bestehende Datensätze ändern

```
UPDATE "Tabellenname" SET "Feldname" = <Expression> [, ...] [WHERE  
<Expression>];
```

Vor allem bei der Änderung vieler Datensätze bietet es sich an, doch einmal die SQL-Befehlseingabe aufzusuchen. Angenommen alle Schüler einer Klasse sollen zum neuen Schuljahr um eine Jahrgangsstufe heraufgesetzt werden:

```
UPDATE "Tabellenname" SET "Jahrgang" = "Jahrgang"+1
```

Schneller geht es nicht: Alle Datensätze werden mit einem Befehl geändert. Natürlich muss jetzt noch nachgesehen werden, welche Schüler denn davon nicht betroffen sein sollen. Einfacher wäre es, vorher in einem Ja/Nein-Feld die Wiederholungen anzukreuzen und dann nur diejenigen eine Stufe heraufzusetzen, die nicht angekreuzt wurden:

```
UPDATE "Tabellenname" SET "Jahrgang" = "Jahrgang"+1 WHERE "Wiederholung" = FALSE
```

Diese Bedingung funktioniert allerdings nur dann, wenn das Feld nur die Werte **FALSE** und **TRUE** annehmen kann, also nicht **NULL**. Sicherer wäre die Formulierung **WHERE "Wiederholung" <> TRUE**.

Soll nachträglich für ein Feld bei allen leeren Feldern ein Standardwert eingetragen werden, so geht dies mit

```
UPDATE "Tabelle" SET "Feld" = 1 WHERE "Feld" IS NULL
```

Die Änderung mehrerer Felder erfolgt mit der direkten Zuordnung von Feld zu Wert. Angenommen es wären in einer Tabelle mit Büchern die Autoren direkt mit eingetragen. Jetzt ist aufgefallen, dass statt «Erich Kästner» häufig «Eric Käschtner» eingetragen worden ist.

```
UPDATE "Bücher" SET "Autor_Vorname" = 'Erich', "Autor_Nachname" = 'Kästner' WHERE "Autor_Vorname" = 'Eric' AND "Autor_Nachname" = 'Käschtner'
```

Auch andere Rechenschritte sind beim Update möglich. Wenn z.B. Waren ab 150,- € zu einem Sonderangebot herausgegeben werden sollen und der Preis um 10 % herabgesetzt werden soll, geschieht das mit dem folgenden Befehl:

```
UPDATE "Tabellenname" SET "Preis" = "Preis"*0.9 WHERE "Preis" >= 150
```

Mit der Wahl des Datentyps CHAR wird eine fixe Breite festgelegt. Gegebenenfalls wird Text mit Leerzeichen aufgefüllt. Bei einer Umstellung auf VARCHAR bleiben diese Leerzeichen erhalten. Sollen die Leerzeichen entfernt werden, so gelingt dies mittels

```
UPDATE "Tabellenname" SET "Feldname" = RTRIM("Feldname")
```

Bestehende Datensätze löschen

```
DELETE FROM "Tabellenname" [WHERE <Expression>];
```

Ohne einen eingrenzenden Bedingungsausdruck wird durch

```
DELETE FROM "Tabellenname"
```

der gesamte Inhalt der Tabelle gelöscht.

Da ist es dann doch besser, wenn der Befehl etwas eingegrenzt ist. Wird z.B. der Wert des Primärschlüssels angegeben, so wird nur genau ein Datensatz gelöscht:

```
DELETE FROM "Tabellenname" WHERE "ID" = 5;
```

Sollen bei einer Medienausleihe die Datensätze von Medien, die zurückgegeben wurden, gelöscht werden, so geht dies mit

```
DELETE FROM "Tabellenname" WHERE NOT "RueckgabeDatum" IS NULL;
```

oder alternativ mit

```
DELETE FROM "Tabellenname" WHERE "RueckgabeDatum" IS NOT NULL;
```

Import von Daten aus anderen Datenquellen

Manchmal existieren schon komplette Datensammlungen in einem anderen Programm, die über die Zwischenablage in Base importiert werden sollen. Hier gibt es die Möglichkeit, beim Import eine neue Tabelle erstellen zu lassen oder auch Datensätze an eine bereits bestehende Tabelle anzufügen.

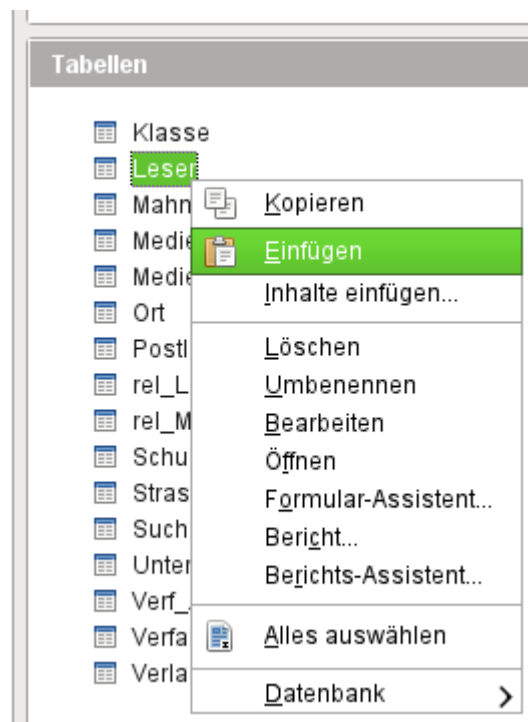
Hinweis

Um einen Import über die Zwischenablage zu gewährleisten, muss das Datenformat für Base lesbar sein. Dies ist bei allen Daten der Fall, die in LibreOffice geöffnet zur Verfügung stehen.

Sollen also z. B. Tabellen aus einer externen Datenbank in eine *.odb-Datei eingelesen werden, so muss zuerst zu der externen Datenbank über LibreOffice ein Kontakt hergestellt werden. Diese Datenbank muss außerdem geöffnet oder als Datenquelle in LibreOffice angemeldet sein. Siehe hierzu auch das Kapitel [Zugriff auf externe Datenbanken](#).

	A	B	C
1	ID	Vorname	Nachname
2	1	Robert	Großkopf
3	2	Maike	Langfuß
4	3	George	Orwell
5			

Eine kleine Beispieltabelle wird aus der Tabellenkalkulation «Calc» in die Zwischenablage kopiert. Anschließend wird nach Base in den Tabellencontainer gewechselt. Dies kann natürlich auch über Markierung mit der linken Maustaste und anschließendem Ziehen bei gleichzeitig gedrückter Maustaste geschehen.



In dem Tabellencontainer wird mit der rechten Maustaste das Kontextmenü über der Tabelle aufgerufen, an die die Daten angehängt werden sollen.

Importierte Daten an bestehende Daten einer Tabelle anfügen

The 'Tabelle kopieren' dialog box is shown with the following settings:

- Tabellenname:** Leser
- Optionen:**
 - ☐ Definition und Daten
 - ☐ Definition
 - ☐ Als Tabellensicht
 - ☒ Daten anhängen
- ☒ Erste Zeile als Spaltennamen verwenden
- ☐ Primärschlüssel erzeugen
- Name:** ID1

Buttons at the bottom: Hilfe, Abbrechen, < Zurück, Weiter >, Fertigstellen.

Der Tabellenname erscheint im Importassistenten. Gleichzeitig ist **Daten anhängen** vorgewählt. **Erste Zeile als Spaltennamen verwenden** muss je nach LO-Version noch markiert werden. Wenn die Daten angehängt werden sollen, dann wird natürlich keine Definition der Daten übernommen. Ein Primärschlüssel muss auch bereits vorhanden sein.

The 'Spalten zuordnen' dialog box shows the mapping between source and target tables:

- Quelltabelle:**
 - ☒ ID
 - ☒ Vorname
 - ☒ Nachname
- Zieltabelle:** Leser
 - ID
 - Vorname
 - Nachname
 - Sperre
 - Geschlecht_ID

Buttons on the left: Up arrow, Down arrow, alle, keine. Buttons on the right: Up arrow, Down arrow.

Buttons at the bottom: Hilfe, Abbrechen, < Zurück, Weiter >, Fertigstellen.

Die Spalten der Quelltable aus Calc und der Zieltabelle in Base müssen nicht von der Reihenfolge her, vom Namen her oder von der Anzahl her übereinstimmen. Es werden nur die auf der linken Seite ausgewählten Elemente übertragen. Die Zuordnung zwischen Quelltable und Zieltabelle muss mit den danebenliegenden Pfeiltasten vorgenommen werden.

Danach wird der Import vollzogen.

Der Import kann Probleme bereiten, wenn

- Felder der Zieltabelle eine Eingabe erfordern, in der Quelltable hierfür aber keine Daten vorliegen,
- Felddefinitionen der Zieltabelle mit den Daten der Quelltable nicht vereinbar sind (wenn z. B. der Vorname in ein Zahlenfeld geschrieben werden soll oder das Feld der Zieltabelle zu wenig Zeichen zulässt) oder
- die Quelltable Daten vorgibt, die mit der Zieltabelle nicht vereinbar sind, wie z. B. eindeutige Werte beim Primärschlüssel oder in anderen Feldern, die so definiert wurden.

Neue Tabelle beim Import erstellen

Zum Start des Importassistenten erscheint automatisch der vorher markierte Tabellenname. Dieser Tabellenname muss für die Gründung einer neuen Tabelle beim Import erst einmal geändert werden, da eine Tabelle mit gleicher Bezeichnung wie eine bereits bestehende Tabelle nicht existieren darf. Der Tabellenname ist "Namen". «*Definition und Daten*» sollen übernommen werden. Die erste Zeile enthält die Spaltennamen.

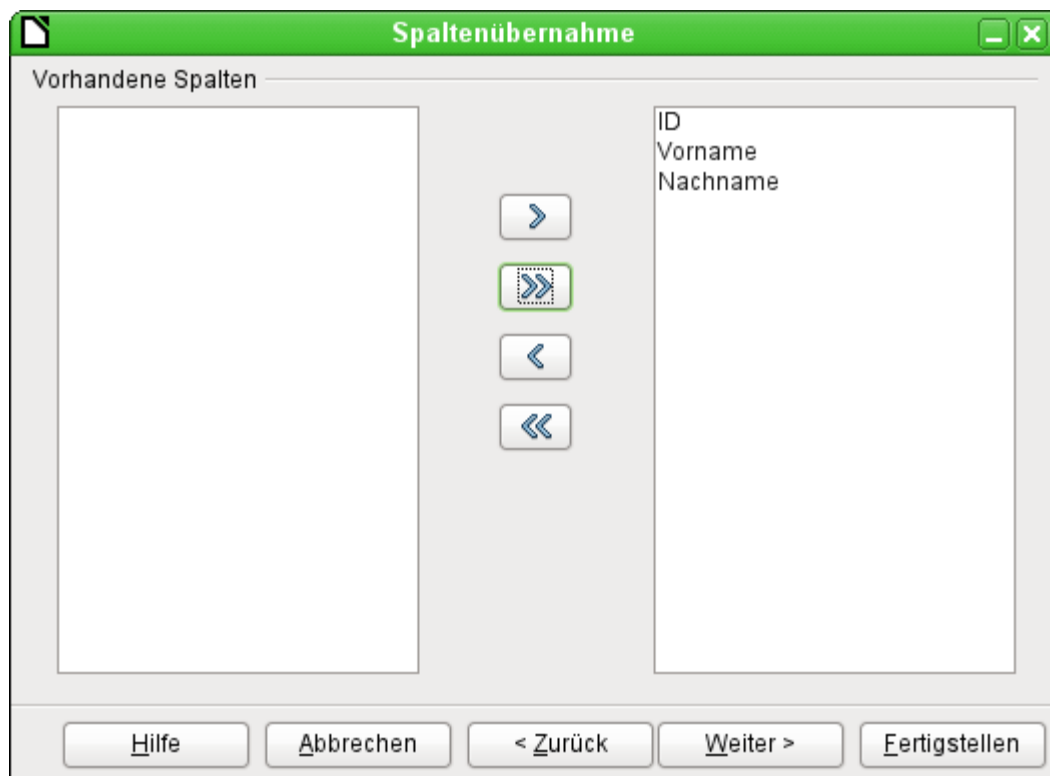
An dieser Stelle kann ein neues, zusätzliches Datenbankfeld für einen Primärschlüssel erzeugt werden. Der Name dieses Datenbankfeldes darf nicht schon als Spalte in der Calc-Tabelle existieren. Anderenfalls erscheint die Fehlermeldung:

'Es sind bereits folgende Felder als Primärschlüssel gesetzt : ID'.

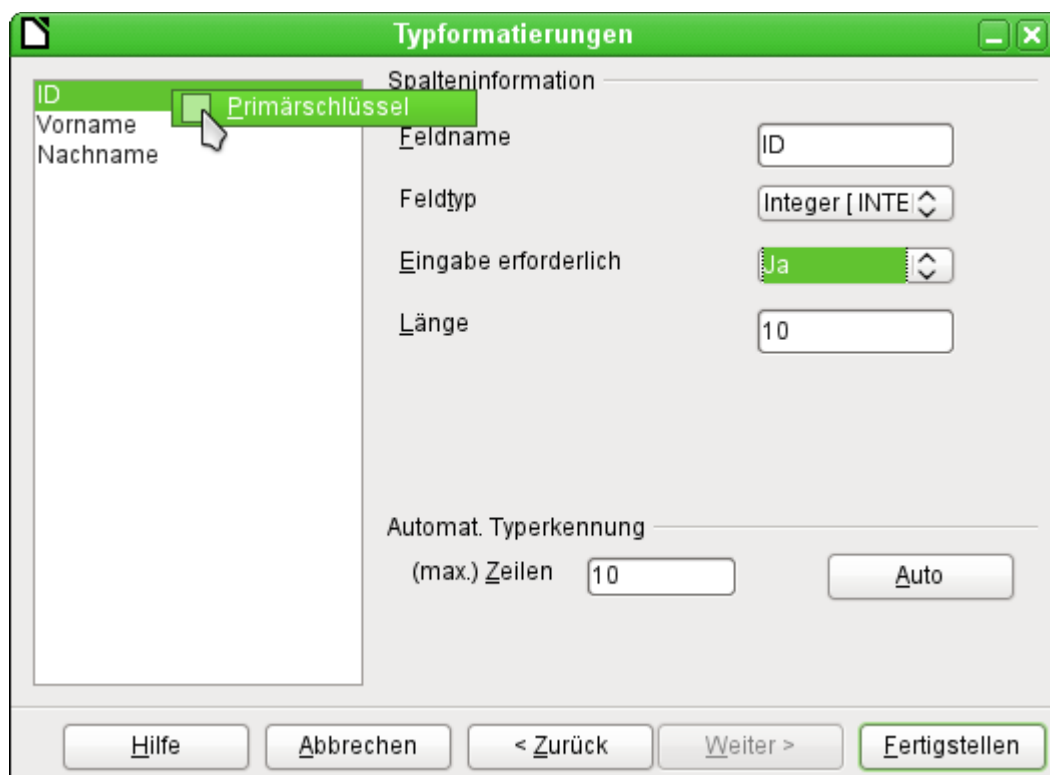
Diese Meldung gibt den Sachverhalt leider nicht ganz korrekt wieder.

Soll ein bereits vorhandenes Feld als Schlüsselfeld genutzt werden, lassen Sie «Primärschlüssel erzeugen» abgewählt. Das Festlegen als Primärschlüssel erfolgt in diesem Fall auf der dritten Dialogseite des Assistenten.

Beim Import soll die Definition der Tabelle erfolgen und die Daten übertragen werden.



Alle vorhandenen Spalten werden übernommen.



Bei der Formatierung des Tabellentyps muss meist nachgebessert werden. In der Regel werden die Felder als Textfelder mit sehr hoher Länge vordefiniert. Zahlen- und Datumsfelder sollten also auf jeden Fall unter **Typformatierungen** → **Spalteninformation** → **Feldtyp** entsprechend eingestellt werden. Bei Dezimalzahlen mit Nachkommastellen ist außerdem auf die Anzahl der Nachkommastellen zu achten.

Die Auswahl des Primärschlüssels liegt etwas versteckt im Kontextmenü des Feldes, das den Primärschlüssel erhalten soll. Hier wird gerade das Feld "ID" formatiert, das als Primärschlüssel vorgesehen ist. Der Primärschlüssel muss hier noch einmal über das Kontextmenü des Feldnamens gesondert ausgewählt werden, wenn er nicht durch den Assistenten im Fenster «*Tabelle kopieren*» als zusätzliches Feld erstellt wurde.

Nach der Betätigung des Buttons **Fertigstellen** wird die Tabelle erstellt und mit dem kopierten Inhalt gefüllt.

Der neue Primärschlüssel ist kein «Auto-Wert»-Schlüssel. Um einen entsprechenden «Auto-Wert»-Schlüssel zu erzeugen, muss die Tabelle zum Bearbeiten geöffnet werden. Dort können dann weitere Formatierungen der Tabelle vorgenommen werden.

Daten Aufsplitten beim Import

Manchmal liegen Daten in der Datenquelle nicht in der gewünschten normalisierten Form vor. Adressen, die z. B. in einer Tabellenkalkulation enthalten sind, enthalten häufig den kompletten Eintrag von Postleitzahl und Ort. Beim Import ist dann vielleicht gewünscht, diese Informationen in einer gesonderten Tabelle zu speichern und die Relationen zwischen den Tabellen zu gewährleisten.

Einen möglichen Weg, die Relation direkt zu erstellen, stellt die folgende Fassung dar:

1. Die komplette Tabelle mit allen Adressinformationen wird als Tabelle "Adresse" in Base importiert. Details dazu siehe in den vorherigen Kapiteln.
2. Die Felder PLZ und Ort werden daraus mit einer Abfrage ausgelesen, kopiert und als separate "PLZ_Ort"-Tabelle abgespeichert. Dabei wird für das Feld "ID" als Primärschlüssel der Autowert gewählt.

Abfrage hierfür:

```
SELECT DISTINCT "PLZ", "Ort" FROM "Adresse"
```

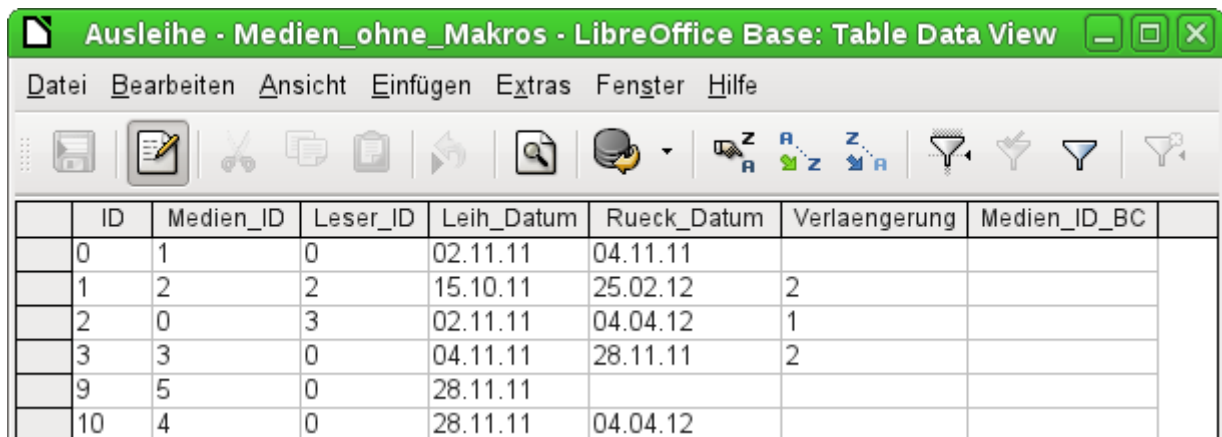
3. Der Tabelle "Adresse" wird ein Feld "PLZ_ID" hinzugefügt.
4. Über **Extras** → **SQL** wird ein Update für diese Tabelle ausgeführt:
UPDATE "Adresse" AS "a" SET "a"."PLZ_ID" = (SELECT "ID" FROM "PLZ_Ort" WHERE "PLZ" || "Ort" = "a"."PLZ" || "a"."Ort")
5. Die Tabelle "Adresse" wird zum Bearbeiten geöffnet und die Felder "PLZ" und "Ort" gelöscht. Die Änderung wird gespeichert und anschließend die Tabelle wieder geschlossen.

Damit sind die Tabellen so getrennt, dass eine 1:n-Beziehung zwischen der Tabelle "PLZ_Ort" und der Tabelle "Adresse" erstellt werden kann. Diese Beziehung wird anschließend über **Extras** → **Beziehungen** definiert.

Für Details zum SQL-Code siehe insbesondere [250](#) im Kapitel «Abfragen».

Mängel dieser Eingabemöglichkeiten

Eingaben mit einer Tabelle alleine berücksichtigen nicht die Verknüpfungen zu anderen Tabellen. Am Beispiel einer Medienausleihe sei das hier verdeutlicht:



	ID	Medien_ID	Leser_ID	Leih_Datum	Rueck_Datum	Verlaengerung	Medien_ID_BC
0	1	0	02.11.11	04.11.11			
1	2	2	15.10.11	25.02.12	2		
2	0	3	02.11.11	04.04.12	1		
3	3	0	04.11.11	28.11.11	2		
9	5	0	28.11.11				
10	4	0	28.11.11	04.04.12			

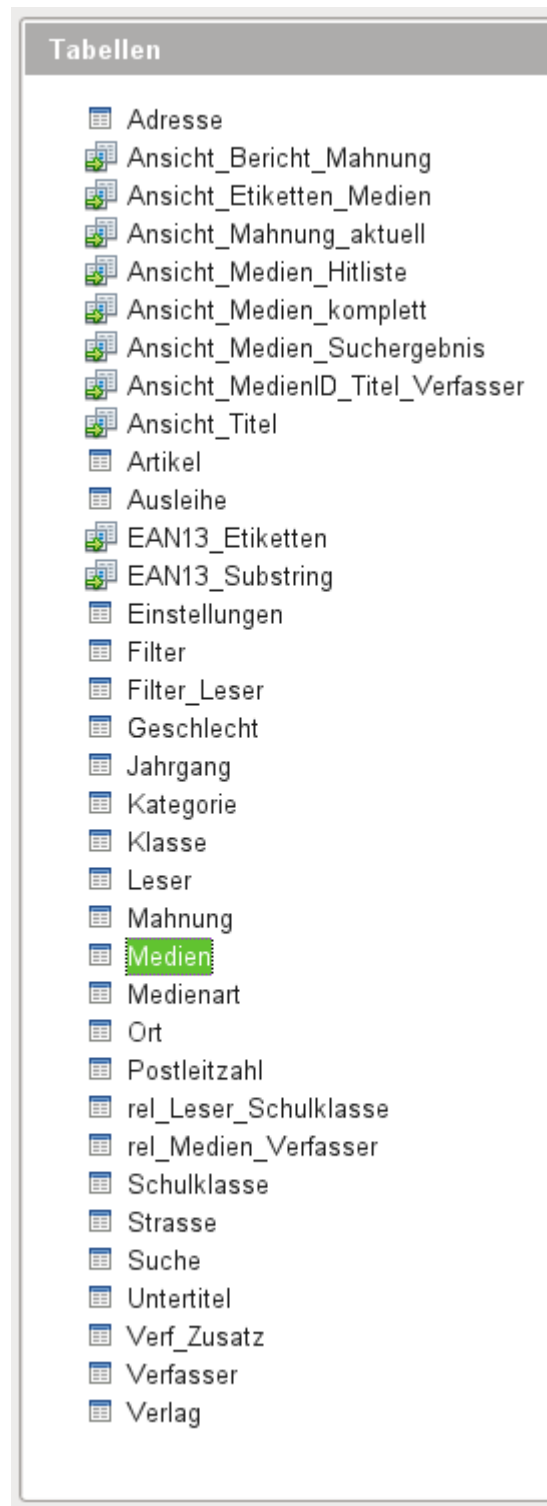
Die Ausleihtabelle besteht aus Fremdschlüsseln für das auszuleihende Medium *Medien_ID* und den entsprechenden Nutzer *Leser_ID* sowie einem Ausleihdatum *Leih_Datum*. In die Tabelle werden also bei der Ausleihe zwei Zahlenwerte (Mediennummer und Benutzernummer) und ein Datum eingetragen. Der Primärschlüssel wird im Feld *ID* automatisch erstellt. Ob der Benutzer zu der Nummer passt, bleibt unsichtbar, es sei denn, eine zweite Tabelle mit den Benutzern wird gleichzeitig offen gehalten. Ob das Medium mit der korrekten Nummer ausgeliehen wird, ist genauso wenig einsehbar. Hier muss sich die Ausleihe auf das Etikett auf dem Medium oder auf eine weitere geöffnete Tabelle verlassen.

All dies lässt sich mit Formularen wesentlich besser zusammenfassen. Hier können die Nutzer und die Medien durch Listenfelder nachgeschlagen werden. Im Formular stehen dann sichtbar die Nutzer und die Medien, nicht die versteckten Nummern. Auch kann das Formular so aufgebaut werden, dass zuerst ein Nutzer ausgewählt wird, dann das Ausleihdatum eingestellt wird und jede Menge Medien diesem einen Datum durch Nummer zugeordnet werden. An anderer Stelle werden dann diese Nummern wieder mit entsprechender genauer Medienbezeichnung sichtbar gemacht.

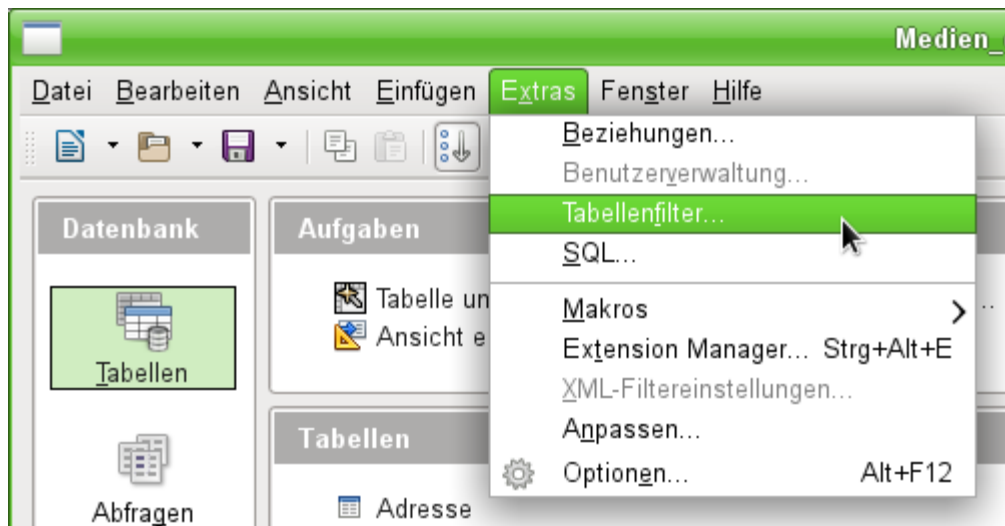
Die Eingabe in Tabellen ist in Datenbanken daher nur bei einfachen Tabellen sinnvoll. Sobald Tabellen in Relation zueinander gesetzt werden, bietet sich besser ein entsprechendes Formular an. In Formularen können diese Relationen durch Unterformulare oder Listenfelder gut bedienbar gemacht werden.

Tabellen verstecken

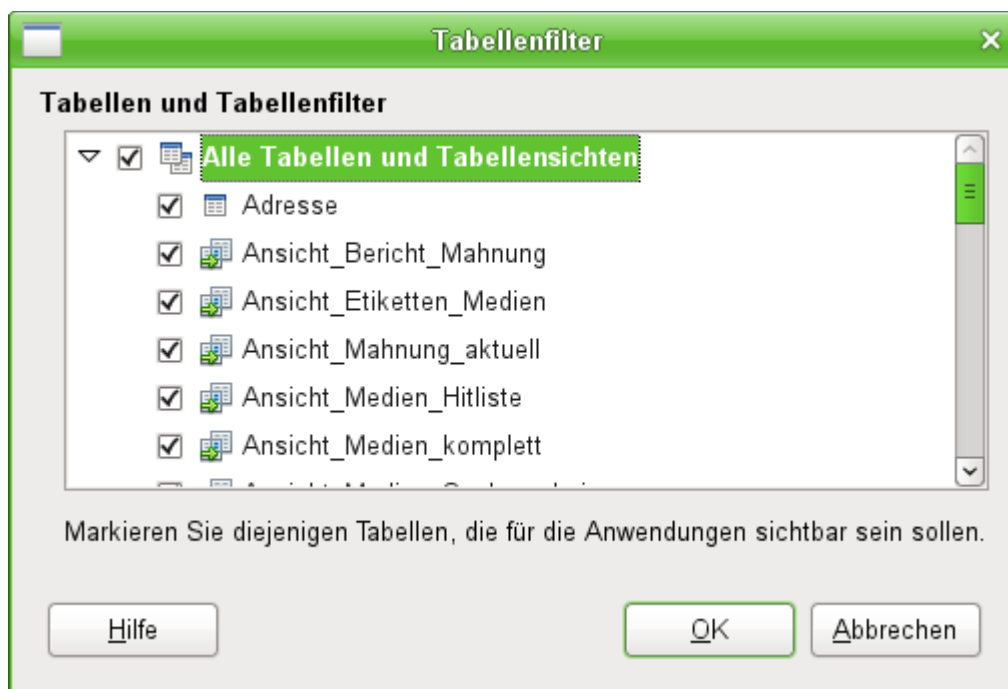
Bei einer entsprechend großen Datenbank erscheinen in der Tabellenübersicht sehr viele Tabellen und Tabellenansichten.



So eine Übersicht ist sicher beim Entwickeln der Datenbank sinnvoll. Mit Hilfe von Base ist es aber auch möglich, nur einen Teil dieser Tabellen überhaupt in der Ansicht aufzuführen. Sie erscheinen dann nicht in der Liste, sind aber sehr wohl für Eingaben und Abfragen über Formulare usw. verfügbar.



Über **Extras** → **Tabellenfilter** kann der Filter für Tabellen gestartet werden.



Würde hier der Haken bei **Alle Tabellen und Tabellenansichten** entfernt, so würde anschließend der Tabellencontainer komplett leer erscheinen. Die Tabellen ständen dann zur direkten Eingabe für den Normaluser nicht mehr zur Verfügung, solange er eben diesen Filter nicht entdeckt. Die Ausblendung schützt also nicht vor absichtlichem, vielleicht aber ein bisschen vor versehentlichem Missbrauch.



Ist eine entsprechende Auswahl getroffen worden, so sind erst einmal weiterhin alle Tabellen und Ansichten sichtbar. Hier muss über **Ansicht → Tabellen aktualisieren** die Ansicht neu eingelesen werden.



So könnte eine Filterung der Tabellen aussehen, die z.B. der Normaluser zu Gesicht bekommt. Alle Tabellen, die die Medien in der Datenbank betreffen, werden noch angezeigt. Alle weiteren Tabellen zur Ausleihe, zu den Lesern und auch alle Ansichten wurden einfach ausgeblendet.

Formulare

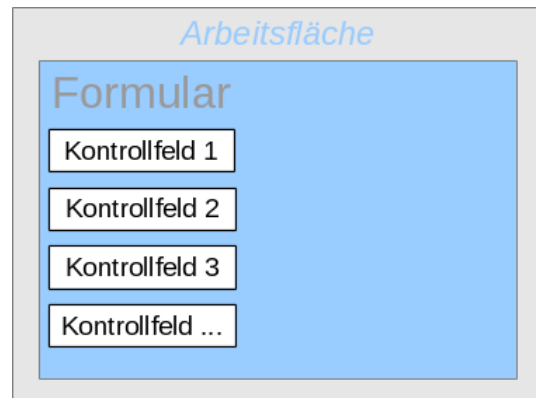
Formulare als Eingabeerleichterung

Formulare werden dann genutzt, wenn die Eingabe direkt über eine Tabelle zu unübersichtlich wird, eventuelle Fehleingaben rechtzeitig abgefangen werden sollen oder zu viele Tabellen eine direkte Verwaltung der Daten unmöglich machen.

Hinweis

Ein *Formular* ist in Base ein für den Nutzer *nicht sichtbares Konstrukt*. Es dient innerhalb von Base dazu, den Kontakt zur Datenbank zu ermöglichen.

Sichtbar sind für den Nutzer die *Kontrollfelder*, die dazu dienen, Text, Zahlen usw. einzugeben oder anzuzeigen. Diese Kontrollfelder werden über die GUI in verschiedene Feldarten unterteilt.



Der Begriff *Formular* hat eine doppelte Bedeutung.

Zum einen steht ein *Formular* für den gesamten Inhalt des Eingabefensters, in dem die Daten für eine oder mehrere Tabellen verwaltet werden. Hier handelt es sich genau genommen um ein Formulardokument, das in der Datenbankdatei auch als komplette Writer-Datei abgespeichert wird. Zum anderen enthält ein solches Formulardokument ein Fenster, das wiederum ein oder mehrere Hauptformulare enthalten kann; auch für diese Teilbereiche des Fensters wird der Begriff *Formular* verwendet. Jedes dieser Hauptformulare in dem Fenster des Formulardokumentes kann außerdem noch wieder Unterformulare enthalten.

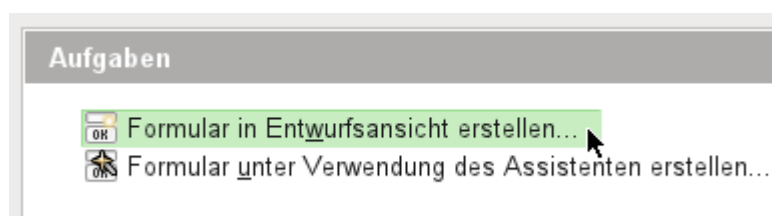
Aus dem Zusammenhang sollte immer klar werden, welche Art von Formular gemeint ist, sodass es hoffentlich niemals zu Missverständnissen kommt.

Erstellung von Formularen

Der einfachste Weg zur Erstellung von Formularen ist der über den Formular-Assistenten. Mit diesem Assistenten lassen sich auch Formulare mit Unterformularen erstellen. Dieser Weg wurde bereits im Kapitel «*Eingabeformular*» im Kapitel «Einführung in Base» beschrieben.

Einfaches Formular

Als Start dient uns aus dem Formularbereich die Aufgabe **Formular in Entwurfsansicht erstellen**.



Rufen wir damit den Formulareditor auf, so zeigt sich erst einmal das Fenster *Formular in der Entwurfsansicht*.

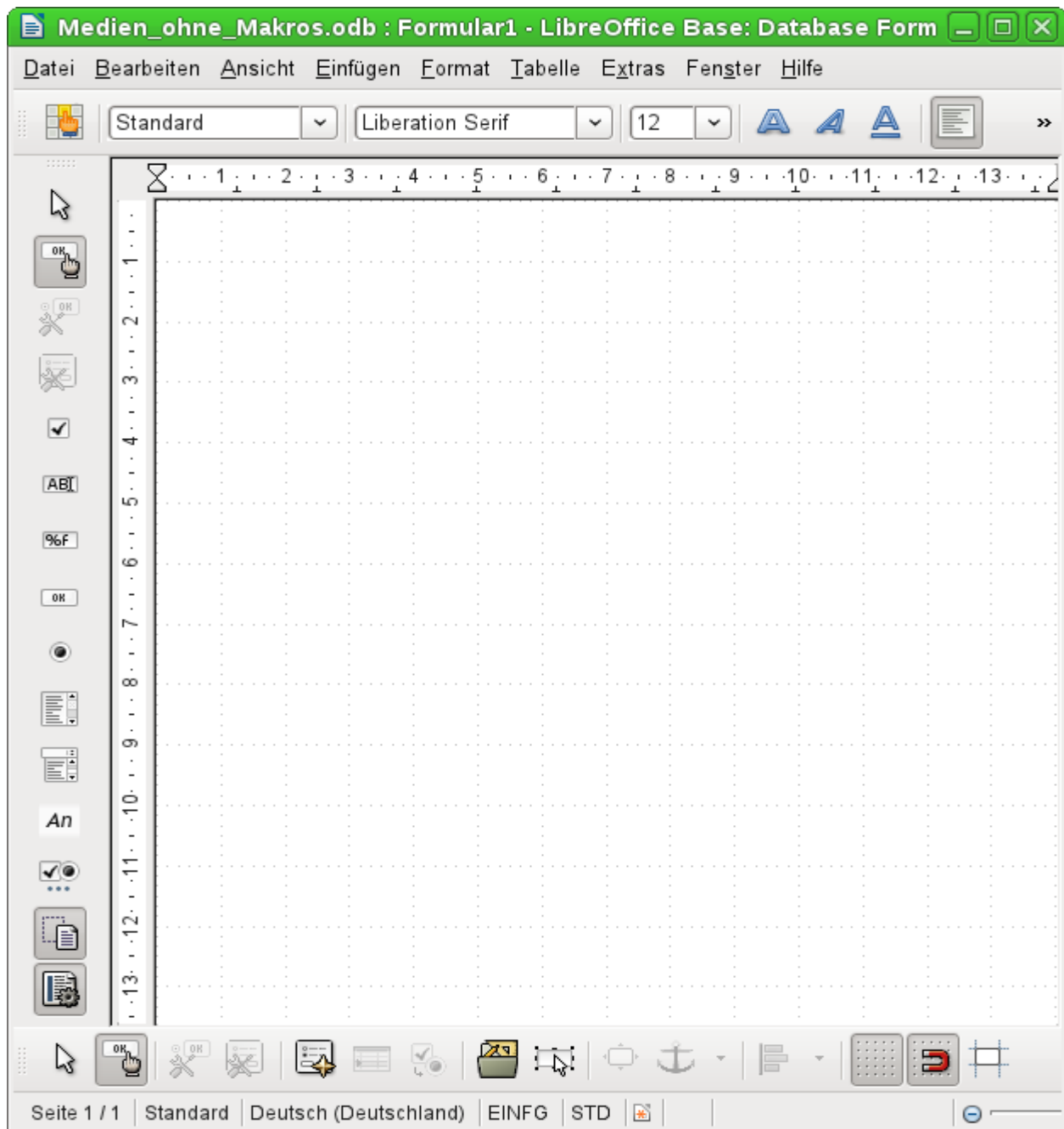


Abbildung 20: Formular in der Entwurfsansicht

Am linken Rand ist die Symbolleiste «Formular-Steuerelemente» eingeblendet. Am unteren Rand ist die Symbolleiste «Formular-Entwurf» angedockt. Sollten diese Symbolleisten nicht automatisch erscheinen, so können sie über **Ansicht → Symbolleisten** angewählt werden. Ohne diese Symbolleisten ist es nicht möglich, ein Formular zu erstellen.

Die weiße Fläche weist ein gepunktetes Raster auf. Dies dient dazu, die Elemente möglichst genau positionieren zu können – vor allem im Verhältnis zueinander. Dass das Raster sichtbar und eingeschaltet ist, ist an den Symbolen ganz rechts in der Leiste zum Formular-Entwurf zu erkennen.

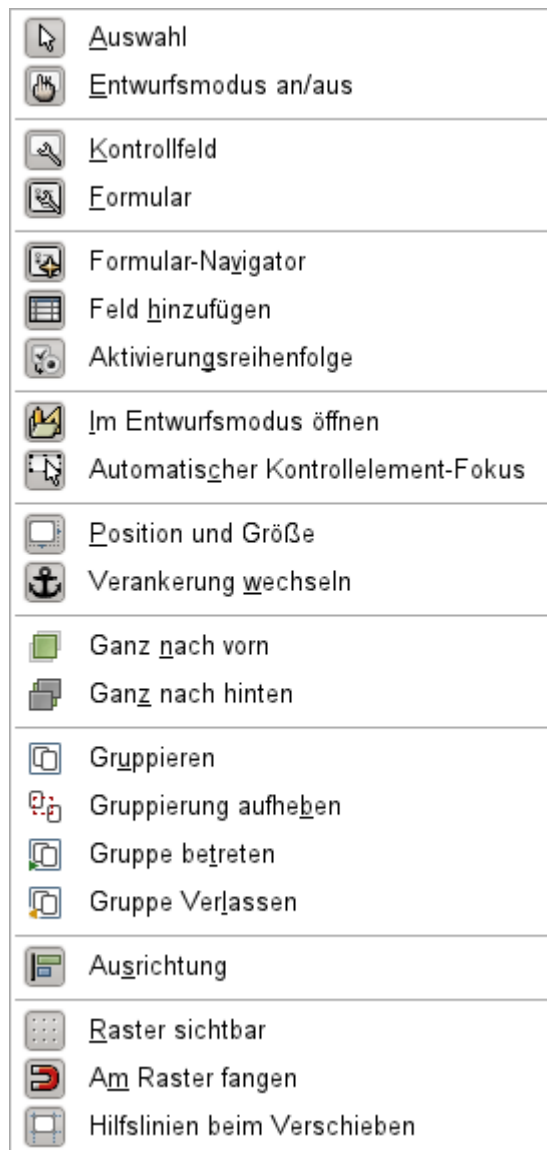


Abbildung 21: Alle verfügbaren Schaltflächen der Symbolleiste «Formular-Entwurf»

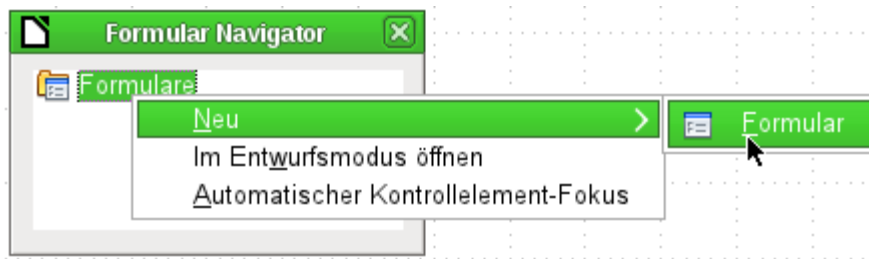
Symbolleisten des Formularentwurfs

Auf der leeren Fläche soll nun ein Formular entstehen. Dies kann auf verschiedene Arten geschehen:

- Aufruf des Formular-Navigators, von dort Gründung eines Formulars sowie
- Erstellung von Formularfeldern und Gründung des Formulars über das dortige Kontextmenü.

Formulargründung über den Navigator

Mit dem in der Abbildung «*Alle verfügbaren Schaltflächen der Symbolleiste «Formular-Entwurf»*» abgebildeten Button Formular-Navigator wird der Navigator gestartet. Es erscheint ein Fenster, das auf lediglich ein Verzeichnis hinweist. Dies ist die höchste Ebene der Fläche, die jetzt bearbeitet wird. Sie ist mit **Formulare** benannt. Dies weist darauf hin, dass nicht nur ein, sondern ohne weiteres mehrere Formulare auf der angezeigten Fläche untergebracht werden können.

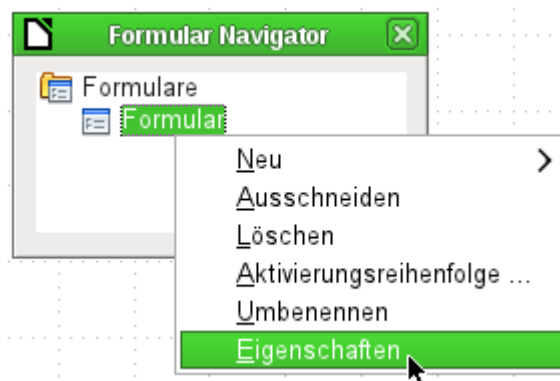


Mit einem Rechtsklick auf das Verzeichnis **Formulare** öffnet sich ein Kontextmenü, in dem über den Menüpunkt **Neu** ein neues **Formular** erstellt werden kann. Die weiteren Befehle des Kontextmenüs entsprechen denen der Buttons in der [Abbildung 21](#).

Hinweis

Soll ein Formular automatisch mit dem Cursor im ersten Formularelement starten, so kann dies durch die Auswahl **Automatischer Kontrollelement-Fokus** bewirkt werden. Das erste Formularelement wird über die [Aktivierungsreihenfolge](#) geregelt.

Leider hat diese Funktion zur Zeit einen Bug. Befindet sich in einem Formular ein Tabellenkontrollfeld, so wird der Fokus automatisch auf das erste Feld des Tabellenkontrollfeldes gesetzt. Dieser Bug ist merkwürdigerweise zu beheben, wenn nach der Wahl des automatischen Kontrollelement-Fokus die Sprache der Benutzeroberfläche umgestellt wird: [Bug 87290](#).



Das Formular erhält standardmäßig den Namen **Formular**. Diese Bezeichnung kann direkt oder später geändert werden. Sie hat allerdings für die spätere Funktion nur dann eine Bedeutung, wenn über Makros auf Teile des Formulars zugegriffen werden soll. Spätestens dann sollten nicht zwei Elemente mit gleicher Bezeichnung in der gleichen Ebene des Verzeichnisbaums auftauchen.

Über das Kontextmenü des Formulars geht schließlich der Weg zu den Formulareigenschaften.

Formulargründung über ein Formularfeld

Über die Symbolleiste für die Formularsteuerelemente ([Abbildung 22](#)) stehen direkt einige Formularfelder zur Verfügung. Während die ersten vier Elemente identisch zu denen des Formular-Entwurfs sind, folgen anschließend häufiger verwendete Formularfelder.

Über die Symbolleiste «Formular-Steuerelemente» kann die zusätzliche Symbolleiste «Weitere Steuerelemente» als frei schwebende Symbolleiste angezeigt werden.

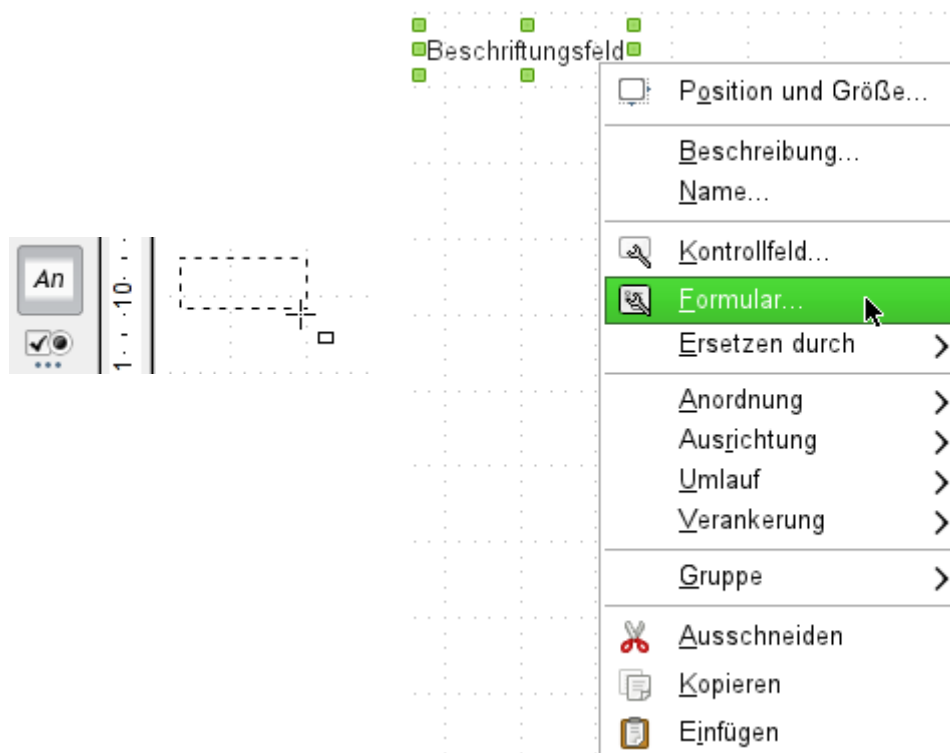


Abbildung 22: Alle verfügbaren Schaltflächen der Symbolleiste «Formular-Steuerelemente»



Abbildung 23: Alle verfügbaren Schaltflächen der Symbolleiste «Weitere Steuerelemente»

Über den Aufruf eines Formularfeldes wird automatisch ein Formular mit gegründet:



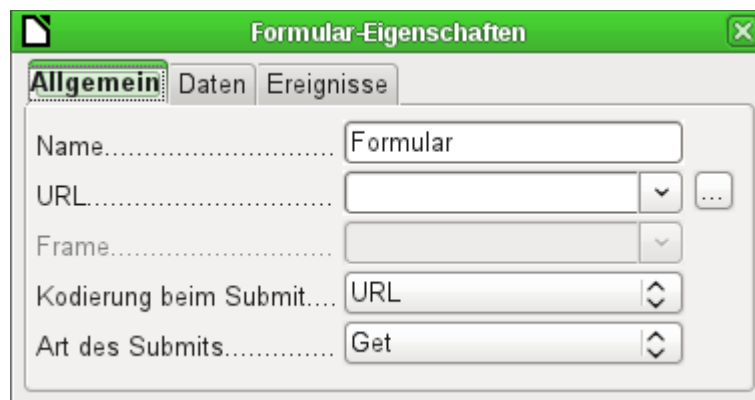
Dazu wird z. B. ein Beschriftungsfeld aufgerufen. Der Cursor verändert sein Erscheinungsbild. Es kann eine rechteckige Form auf die weiße Oberfläche des Formulars gezogen werden. Aus der gestrichelten Form entsteht anschließend ein Beschriftungsfeld. Jetzt wird zur Erstellung des Formulars das **Kontextmenü** des Kontrollfeldes aufgerufen.

Über den Menüpunkt **Formular** werden hier die Eigenschaften des nebenher gegründeten Formulars aufgerufen. Das Formular wurde mit dem Standardnamen «Formular» erstellt.

Externe Formulare

Neben den Formularen, die direkt in Base erstellt werden, gibt es auch die Möglichkeit, im Writer oder in Calc Formulare zu erstellen. Dies wird im Kapitel «Datenbank-Anbindung» beschrieben.

Formular-Eigenschaften



Werden die Formulareigenschaften über das Kontextmenü im Formular-Navigator oder das Kontextmenü eines Formularfeldes aufgerufen, so erscheint ein Fenster mit drei Registerreitern: **Allgemein**, **Daten** und **Ereignisse**

Allgemein

Hier kann der **Name** des Formulars geändert werden. Außerdem finden sich Einstellungsmöglichkeiten, die innerhalb von Base keine weitere Bedeutung haben. Sie zeigen lediglich die universelle Einsatzmöglichkeit des Formulareditors; bei Erstellung von Webformularen ist hier eine Einstellung notwendig. (URL: Ziel, an das die Daten gesandt werden sollen | Frame: Teil der Zielwebseite, der eventuell separat angesprochen werden muss | Kodierung beim Submit: Neben dem normalen Kodieren der Zeichen für eine Weitergabe über die URL sind hier Textkodierungen und Multipartkodierungen (z. B. zum Versenden von Dateien ...) möglich. | Art des Submits: Get (über die URL sichtbar an den Dateinamen angehängt) oder POST (nicht sichtbar, auch für größere Datenmengen geeignet).

The screenshot shows the 'Formular-Eigenschaften' dialog box with the 'Daten' tab selected. The settings are as follows:

Property	Value
Art des Inhaltes	Tabelle
Inhalt	
SQL-Befehl analysieren	Ja
Filter	
Sortierung	
Daten hinzufügen	Ja
Daten ändern	Ja
Daten löschen	Ja
Nur Daten hinzufügen	Nein
Navigationsleiste	Ja
Zyklus	Standard

Für die Erstellung interner Formulare von Base ist dies der wichtigste Registerreiter. Hier wird zuerst der Inhalt des Formulars festgelegt.

- **Art des Inhaltes:** Hier besteht die Wahl zwischen *Tabelle*, *Abfrage* und *SQL-Befehl*. Während Tabellen in der Regel immer für Eingaben in einem Formular genutzt werden können, so ist dies bei Abfragen eventuell nicht der Fall. Näheres dazu im Kapitel 'Abfragen'. Gleiches wie für Abfragen gilt auch für die direkte Eingabe eines SQL-Befehls. Hier handelt es sich dann lediglich um eine Abfrage, die nicht im Abfragecontainer von Base sichtbar ist, aber vom Prinzip her die gleiche Struktur aufweist.
- **Inhalt:** Wird unter Art des Inhaltes *Tabelle* oder *Abfrage* gewählt, so werden hier alle *verfügbaren Tabellen und Abfragen* gelistet. Soll ein SQL-Befehl erstellt werden, so besteht die Möglichkeit, den Abfrageeditor dazu über den Button mit den drei Punkten rechts von dem Inhaltsfeld aufzurufen.
- **SQL-Befehl analysieren:** Wird die Analyse des SQL-Befehls nicht zugelassen (weil z. B. mit Code gearbeitet wird, den die GUI eventuell nicht richtig deuten kann), so ist hier 'Nein' zu wählen. Allerdings schließt dies aus, dass das Formular weiterhin noch mit der Filterung oder mit der Sortierung auf die zugrundeliegenden Daten zugreifen kann.
- **Filter:** Hier kann ein Filter gesetzt werden. Hilfe dazu bietet ein Klick auf den Button rechts von dem Eingabefeld. Das Filtern entspricht dem [Filtern von Tabellen](#).
- **Sortierung:** Hier kann eine Sortierung der Daten festgelegt werden. Hilfe dazu bietet ein Klick auf den Button rechts von dem Eingabefeld. Das Sortieren entspricht dem [Sortieren von Tabellen](#).
- **Daten hinzufügen:** Sollen neue Daten erstellt werden können? Standardmäßig ist dies auf Ja eingestellt.
- **Daten ändern:** Sollen Daten geändert werden können? Ebenfalls Standard Ja.
- **Daten löschen:** Auch das Löschen von Daten wird standardmäßig ermöglicht.
- **Nur Daten hinzufügen:** Ist dies gewählt, so erscheint immer ein leeres Formular. Auf die alten Datensätze besteht kein Zugriff, sie können nicht bearbeitet oder auch nur angesehen werden.
- **Navigationsleiste:** Das Erscheinen der Navigationsleiste am unteren Bildschirmrand kann angeschaltet oder ausgeschaltet werden. Außerdem besteht die Möglichkeit, bei einem

Unterformular immer die Navigationsleiste des darüberliegenden Hauptformulars anzeigen zu lassen, so dass eine Betätigung der Navigationsleiste direkte Auswirkung auf das Hauptformular hat.

Diese Einstellung zur Navigationsleiste betrifft nicht die Leiste, die gegebenenfalls als Formularfeld eingefügt werden kann.

- *Zyklus*: Standard bedeutet hier für Base-Datenbanken, dass nach der Eingabe im letzten Feld innerhalb eines Formulars mit dem Tabulator zum ersten Feld des nächsten Datensatzes, also gegebenenfalls eines neuen Datensatzes, gesprungen wird. Dies ist für die Datenbanken gleichbedeutend mit «Alle Datensätze». Wird hingegen bei Datenbanken «Aktueller Datensatz» gewählt, so bewegt sich der Cursor nur innerhalb des einen Datensatzes, beim Erreichen des letzten Feldes also zum ersten Feld des gleichen Datensatzes. Aktuelle Seite bezieht sich wieder besonders auf HTML-Formulare. Hier springt dann der Cursor vom Ende eines Formulars gegebenenfalls zum nächsten Formular auf der Seite, das weiter unten liegt.

Ereignisse

Formular-Eigenschaften		
Allgemein Daten Ereignisse		
Vor dem Zurücksetzen.....		...
Nach dem Zurücksetzen.....		...
Vor dem Submit.....		...
Beim Laden.....		...
Vor dem erneuten Laden.....		...
Beim erneuten Laden.....		...
Vor dem Entladen.....		...
Beim Entladen.....		...
Löschen bestätigen.....		...
Vor der Datensatzaktion.....		...
Nach der Datensatzaktion.....		...
Vor dem Datensatzwechsel.....		...
Nach dem Datensatzwechsel.....		...
Parameter füllen.....		...
Fehler aufgetreten.....		...

Ereignisse können Makros auslösen. Durch einen Klick auf den rechts stehenden Button (...) können Makros mit dem Ereignis verbunden werden.

Zurücksetzen: Das Formular wird von allen neuen Einträgen geleert, die noch nicht abgespeichert sind.

Vor dem Submit: Bevor die Formulardaten gesendet werden. Dies hat nur bei Webformularen eine Bedeutung.

Beim Laden: Nur beim Öffnen des Formulars. Nicht beim Laden eines neuen Datensatzes in das Formular.

Erneutes Laden: Dies erfolgt, wenn der Inhalt des Formulars z. B. über einen Button in der Navigationsleiste aktualisiert wird.

Entladen: Nach einigen Tests scheint dies ohne Funktion zu sein. Erwartet würde der Ablauf eines Makros beim Schließen des Formulars.

Datensatzaktion: Dies ist z. B. das Abspeichern mittels Button. Im Test ergibt sich, dass die Aktion **Vor der Datensatzaktion** regelmäßig doppelt erscheint, d. h. Makros werden direkt nacheinander zweimal abgearbeitet. Dies liegt daran, dass hier unterschiedliche Funktionen (*Implementationen*) ausgeführt werden. Beide sind mit Namen versehen:

org.openoffice.comp.svx.FormController und

com.sun.star.comp.forms.ODatabaseForm. Wenn innerhalb des Makros mit **oForm.ImplementationName** der entsprechende Name abgefragt wird, so kann das Makro auf eine Ausführung begrenzt werden.

Datensatzwechsel: Bereits das Öffnen des Formulars stellt einen Datensatzwechsel dar. Beim Wechsel von einem Datensatz zum anderen innerhalb eines Formulars taucht diese Aktion ebenfalls zweimal auf. Makros werden also auch hier zweimal hintereinander ausgeführt. Auch hier kann aber zwischen den Ursachen des Ereignisses unterschieden werden.

Parameter füllen: Dieses Makro springt ein, wenn eine Parameterabfrage in einem Unterformular aufgerufen werden soll, aber aus irgendeinem Grund der Parameter vom Hauptformular nicht richtig weiter gegeben wird. Ohne das Ereignis abzufangen, erfolgt dann beim Laden des Formulars eine Parameterabfrage.

Fehler aufgetreten: Dieses Ereignis lässt sich nicht nachvollziehen.

Eigenschaften der Kontrollfelder

Ist ein Formular erstellt, so kann es mit den sichtbaren Kontrollfeldern bestückt werden. Die Kontrollfelder sind für verschiedene Aufgaben gedacht:

- Die meisten zeigen den Inhalt aus der Datenbank an oder nehmen die Daten entgegen, die in die Datenbank eingefügt werden.
- Andere Kontrollfelder dienen zur Navigation, zum Suchen und zur Ausführung von Befehlen (Interaktion).
- Weitere Kontrollfelder sorgen für eine zusätzliche grafische Aufarbeitung des Formulars.

Dateneingabe und Datenanzeige	
Kontrollfeld	Anwendungsgebiet
Textfeld	Texteingaben
Numerisches Feld	Zahleneingabe
Datumsfeld	Datumseingabe
Zeitfeld	Zeiteingabe
Währungsfeld	Zahleneingabe, vorformatiert für Währungen
Formatiertes Feld	Anzeige und Eingabe mit zusätzlicher Formatierung wie z. B. Maßeinheiten
Listenfeld	Auswahl zwischen vielen verschiedenen Möglichkeiten, Weitergabe eines anderen als des angezeigten Wertes an die Datenbank.
Kombinationsfeld	Wie Listenfeld, nur Weitergabe des angezeigten Wertes und dazu noch die Möglichkeit, auch neue Werte einzugeben.
Markierfeld	Ja/Nein-Felder
Optionsfeld	Auswahl zwischen verschiedenen, stark begrenzten Möglichkeiten.
Grafisches Kontrollfeld	Anzeige von Bildern und seit LO 5.0 auch *.pdf-Dateien (1. Seite) aus

	einer Datenbank und Neueingabe von Bildern und seit LO 5.0 auch anderen Dateien in eine Datenbank über eine Pfadangabe
Maskiertes Feld	Eingabe in eine vorgefertigte Maske; grenzt die Eingabemöglichkeiten auf bestimmte Zeichenkombinationen ein.
Tabellen-Kontrollfeld	Universelles Eingabemodul, das eine ganze Tabelle bedienen kann. Integriert in dieses Kontrollfeld sind wieder viele der obigen Kontrollfelder

Gestaltung	
Kontrollfeld	Anwendungsgebiet
Beschriftungsfeld	Überschrift über das Formular, Beschriftung anderer Kontrollfelder
Gruppierungsrahmen	Linienzug um z. B. verschiedene Optionsfelder

Interaktion	
Kontrollfeld	Anwendungsgebiet
Schaltfläche	Button mit Beschriftung
Grafische Schaltfläche	Wie der Button, nur mit einer zusätzlich auf dem Button erscheinenden Grafik
Navigationsleiste	Leiste mit geringen Abweichungen zu der, die am unteren Bildschirmrand angezeigt wird.
Dateiauswahl	Auswahl von Dateien z. B. zum Hochladen in HTML-Formularen. - nicht weiter beschrieben
Drehfeld	Nur über Makroprogrammierung verwendbar – nicht weiter beschrieben
Bildlaufleiste	Nur über Makroprogrammierung verwendbar – nicht weiter beschrieben
Verstecktes Steuerelement	Hier kann ein Wert über Makros eingespeichert und wieder ausgelesen werden.

Standardeinstellungen vieler Kontrollfelder

Die Eigenschaften werden wie beim Formular in drei Kategorien unterteilt: Allgemein, Daten und Ereignisse. Unter Allgemein wird all das eingestellt, was für den Nutzer sichtbar ist. In der Kategorie Daten wird die Verbindung zu einem Feld der Datenbank hergestellt. Die Kategorie Ereignisse schließlich regelt Auslösemomente, die mit irgendwelchen Makros verbunden werden können. Für eine Datenbank ohne Makros spielt diese Kategorie keine Rolle.

Eigenschaften: Textfeld

Allgemein Daten Ereignisse

Name.....

Beschriftungsfeld.....

Max. Textlänge.....

Aktiviert..... ☒

Sichtbar..... ☒

Nur lesen..... ☐

Druckbar..... ☒

Tabstop..... ☒

Aktivierungsreihenfolge...

Verankerung.....

PositionX.....

PositionY.....

Breite.....

Höhe.....

Standardtext.....

Schrift.....

Ausrichtung.....

Ausrichtung (vert.).....

Hintergrundfarbe.....

Rahmen.....

Umrandungsfarbe.....

Text-Typ.....

Textzeilen enden mit.....

Bildlaufleisten.....

Zeichen für Passwörter...

Auswahl verstecken..... ☒

Zusatzinformation.....

Hilfetext.....

Hilfe URL.....

Eigenschaften: Textfeld

Allgemein **Daten** Ereignisse

Datenfeld.....

Leere Zeichenfolge ist NULL... ☒

Eingabe erforderlich..... ☒

Filtervorschlag.....

Eigenschaften: Textfeld

Allgemein Daten **Ereignisse**

Modifiziert.....

Text modifiziert.....

Bei Fokuserhalt.....

Bei Fokusverlust.....

Taste gedrückt.....

Taste losgelassen.....

Maus innerhalb.....

Mausbewegung bei Tastendruck...

Mausbewegung.....

Maustaste gedrückt.....

Maustaste losgelassen.....

Maus außerhalb.....

Vor dem Zurücksetzen.....

Nach dem Zurücksetzen.....

Vor dem Aktualisieren.....

Nach dem Aktualisieren.....

Hinweis

In den folgenden Übersichten stehen die deutschen Begriffe vor allem der Eigenschaften – genau wie im Formular-Entwurf. In Makros werden zur Beeinflussung der Felder die englischen Begriffe benötigt; diese sind in [Klammern] angegeben.

Beispiel:

```
oDoc = thisComponent
oDrawpage = oDoc.drawpage
oForm = oDrawpage.forms.getByNamed("Formular")
oFeld = oForm.getByNamed("Formularfeld")
oFeld.Printable = True 'Das Feld wird auf "Druckbar"-"Ja" gesetzt.
```

Weitere Hinweise stehen im Kapitel «Makros».

Allgemein

Name.....

Die Bezeichnung für das Feld sollte innerhalb eines Formulars nur einmal vorkommen – Anwendung bei Zugriff über Makros.
[Name]

Beschriftungsfeld..... ...

Gehört zu dem Feld ein Beschriftungsfeld? Hiermit wird eine Gruppierung festgelegt. Über das Beschriftungsfeld kann dann das Formularfeld mit einer Tastenkombination direkt erreicht werden.
[LabelControl]

Aktiviert.....

Nicht aktivierte Felder sind nicht verwendbar und werden grau hinterlegt. Sinnvoll bei Steuerung über Makros (Entscheidung: Wenn in Feld 1 ein Wert eingegeben wird, darf in Feld 2 kein Wert eingegeben werden – Feld 2 wird deaktiviert)
[Enabled]

Sichtbar.....

In der Regel 'Ja'; nicht sichtbare Felder können Werte zwischenspeichern. Anwendung z. B. bei der Erstellung von Kombinationsfeldern mit Makros.
[EnableVisible]

Nur lesen.....

'Ja' würde eine Veränderung des Wertes ausschließen; z. B. für die Anzeige eines automatisch erstellten Primärschlüssels sinnvoll.
[ReadOnly]

Druckbar.....

Manchmal sind Seitenausdrucke aus einem Formular sinnvoller als ein separater Bericht. Hier sollen dann eventuell nicht alle Felder erscheinen.
[Printable]

Tabstop.....

Durch ein Formular wird in der Regel mit dem Tabulator navigiert. Ein Feld, das z. B. nur gelesen wird, braucht keinen Stop des Tabulators, würde also übersprungen.
[Tabstop]

Aktivierungsreihenfolge.... ...

Hat das Feld einen Tabstop? Hier wird die Reihenfolge innerhalb des Formulars eingestellt.
[TabIndex]

Verankerung.....	Am Absatz	Verankerung der Grafik, die das Textfeld darstellt.
PositionX.....	0,50cm	Position links oben vom linken Rand aus. [PosSize.X]
PositionY.....	2,83cm	Position von oben aus. [PosSize.Y]
Breite.....	4,00cm	Breite des Feldes [PosSize.Width]
Höhe.....	1,00cm	Höhe des Feldes [PosSize.Height]
Schrift.....	(Standard)	Schriftart, Schriftschnitt, Schriftgrad und Schrifteffekt sind hier einstellbar. [Fontxxx]
Ausrichtung.....	Links	Der Eintrag wird linksbündig dargestellt. [Align]
Ausrichtung (vert.).....	Standard	Standard Oben Mitte Unten [VerticalAlign]
Hintergrundfarbe.....	Standard	Hintergrundfarbe des angezeigten Textfeldes [BackgroundColor]
Rahmen.....	Flach	Rahmenform: Ohne Rahmen 3D-Look Flach [Border]
Umrandungsfarbe.....	 Hellgrau	Wenn ein Rahmen, dann kann hier die Umrandungsfarbe eingestellt werden. [BorderColor]
Auswahl verstecken.....	Ja	Markierter Text wird so nicht mehr als markiert angezeigt, wenn das Textfeld den Fokus verliert. [HideInactiveSelection]
Zusatzinformation.....		Gut nutzbar für Informationen, die mittels Makros ausgelesen werden sollen, siehe Zusatzinformation eines Feldes im Kapitel «Makros». [Tag]
Hilfetext.....		Erscheint als sogenannter Tooltip, wenn mit der Maus über das Textfeld gefahren wird. [HelpText]

Hilfe URL.....

Verweist auf eine Hilfedatei – eher für HTML-Zwecke geeignet. Durch F1 abrufbar, wenn der Fokus auf dem Feld liegt.
[HelpURL]

Zusätzlich sind bei Zahlenfeldern, Datumsfeldern u.ä. üblich:

Formatüberprüfung.....

Mit eingeschalteter Überprüfung ist nur die Eingabe von Ziffern und Komma möglich.
[EnforceFormat]

Mausradverhalten.....

Nie erlaubt keine Änderung mit dem Mausrad; *Wenn ausgewählt* lässt eine Änderung zu, wenn das Feld ausgewählt ist und die Maus sich über dem Feld befindet; *Immer* bedeutet, dass sich die Maus über dem Feld befinden muss.
[MouseWheelBehavior]

Drehfeld.....

Ein Drehsymbol wird an der rechten Seite des Feldes eingeblendet.
[Spin]

Wiederholung.....

Wenn ein Drehfeldpfeil länger gedrückt wird lässt sich hier einstellen, ob nicht nur zum nächsten Wert gedreht werden soll.
[Repeat]

Verzögerung.....

Stellt die Verzögerungszeit ein, nach der der Druck auf die Maustaste beim Drehfeld Wiederholung interpretiert wird.
[RepeatDelay]

Daten

The screenshot shows a dialog box titled 'Eigenschaften: Textfeld' with a green header bar. It has three tabs: 'Allgemein', 'Daten', and 'Ereignisse'. The 'Daten' tab is selected. Inside the 'Daten' tab, there are four properties, each with a label and a dropdown menu:

- Datenfeld.....**: The dropdown menu shows 'Nachname'.
- Leere Zeichenfolge ist NULL....**: The dropdown menu shows 'Ja'.
- Eingabe erforderlich.....**: The dropdown menu shows 'Ja'.
- Filtervorschlag.....**: The dropdown menu shows 'Nein'.

Datenfeld: Hier wird die Verbindung zur Tabelle hergestellt, die dem Formular zugrunde liegt.
[Model.DataField]

Leere Zeichenfolge ist NULL: Soll ein leeres Feld geleert werden (NULL) oder nur der Inhalt gelöscht werden?

Eingabe erforderlich: Dieser Eintrag sollte sich mit dem in der Tabelle decken. Dann fragt die GUI gegebenenfalls nach einer Eingabe, wenn vom Nutzer kein Wert eingegeben wurde.
[Model.InputRequired]

Filtervorschlag: Bei einer Filterung der Daten werden die Inhalte dieses Feldes als Vorschläge zwischengespeichert. Achtung – dies ist bei umfangreichen Inhalten recht speicherintensiv.
[Model.UserValueFilterProposal]

Ereignisse



Eigenschaften: Textfeld		
Allgemein	Daten	Ereignisse
Modifiziert.....		...
Text modifiziert.....		...
Bei Fokuserhalt.....		...
Bei Fokusverlust.....		...
Taste gedrückt.....		...
Taste losgelassen.....		...
Maus innerhalb.....		...
Mausbewegung bei Tastendruck....		...
Mausbewegung.....		...
Maustaste gedrückt.....		...
Maustaste losgelassen.....		...
Maus außerhalb.....		...
Vor dem Zurücksetzen.....		...
Nach dem Zurücksetzen.....		...
Vor dem Aktualisieren.....		...
Nach dem Aktualisieren.....		...

Modifiziert: Dieses Ereignis tritt ein, wenn das Kontrollfeld geändert wurde und anschließend den Fokus verloren hatte. Das Ereignis geht verloren, wenn direkt zum nächsten Datensatz gewechselt wird. Unter den Umständen ist also eine Änderung gespeichert, ohne vorher wahrgenommen zu werden. [com.sun.star.lang.EventObject]

Text modifiziert: Direkt auf den Inhalt bezogen; kann Text, Zahl o.a. sein, tritt also nach jeder Tastatureingabe auf. [com.sun.star.awt.TextEvent]

Fokuserhalt: Der Cursor kommt in das Feld hinein. Hier sollte auf keinen Fall über das Makro eine MessageBox auf dem Bildschirm erzeugt werden. Durch das Anklicken dieser Box verliert das Formularfeld den Fokus und erhält ihn direkt danach zurück – eine Schleife wird dadurch erzeugt. Sie kann nur durch Tastatureingaben unterbrochen werden.

Fokusverlust: Der Cursor bewegt sich aus dem Feld heraus. Auch dies kann zu einem Wechselspiel führen, wenn eine zu bestätigende Bildschirmausgabe erfolgt.

Taste: Bezieht sich auf die Tastatur. Die Taste muss ausgelöst werden, wenn der Cursor in dem Kontrollfeld steht. Auch der Tabulator zum Verlassen des Kontrollfeldes löst dieses Ereignis aus. Dem Ereignis wird über den **KeyCode** bzw. **KeyChar** die auslösende Taste (Buchstabe, Zahl, Spezialtaste) mitgegeben. [com.sun.star.awt.KeyEvent]

Maus: selbsterklärend; Ereignisse treten nur ein, wenn vorher die Maus innerhalb des Feldes ist oder war («außerhalb» entspricht javascript onMouseOut). [com.sun.star.awt.MouseEvent]

Zurücksetzen: Das Formular wird von allen Daten geleert (Anlegen eines neuen Datensatzes) oder auf den alten Datenstand zurück gesetzt (Änderung eines bestehenden Datensatzes). Bei einem Formularfeld wird das Ereignis nur ausgelöst, wenn über den Button in der Navigationsleiste die Dateneingabe rückgängig gemacht wird. [com.sun.star.lang.EventObject]

Wenn ein Formular aufgerufen wird, wird nacheinander das Ereignis *Vor dem Zurücksetzen* und *Nach dem Zurücksetzen* abgearbeitet, bevor das Formular für eine Eingabe verfügbar ist.

Aktualisieren: Ist dies Ereignis an ein Kontrollfeld des Formulars gebunden, so tritt die Aktualisierung bei Fokusverlust und Sprung zu einem anderen Formularfeld auf, wenn der Inhalt des Kontrollfeldes geändert wurde. Änderungen in dem Formular werden übernommen und angezeigt. Bei Schließen eines Formular werden nacheinander die Ereignisse *Vor dem Aktualisieren* und *Nach dem Aktualisieren* abgearbeitet. [com.sun.star.lang.EventObject]

Textfeld

Neben den bereits unter *Standardeinstellungen vieler Kontrollfelder* erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Max. Textlänge..... 0

Solange der Wert 0 ist wird die Einstellung nicht berücksichtigt. In der Praxis wird hier die Zeichenlänge des Feldes aus der Datenbank übernommen, auf das sich das Textfeld bezieht.

[MaxTextLen]

Standardtext.....

Soll in einem leeren Feld ein Standardtext erscheinen? Dieser Text muss gelöscht werden, wenn ein anderer Eintrag erfolgen soll.

[DefaultText]

Text-Typ..... Mehrzeilig

Mögliche Typen: Einzeilig | Mehrzeilig | Mehrzeilig mit Formatierungen (wobei sich die beiden letzten nur im Tabulator unterscheiden – und ein Feld mit Formatierungen nicht an eine Datenbank angebunden werden kann). Bei mehrzeiligen Feldern ist die vertikale Ausrichtung nicht aktiv.

[MultiLine]

Textzeilen enden mit..... LF (Unix) ▼

Unix oder Windows? Prinzipiell funktionieren beide Endungen. Intern müssen für Windowszeilenenden aber zwei Steuerzeichen verwendet werden (CR und LF).
[LineEndFormat]

Bildlaufleisten..... Keine ▼

Nur bei mehrzeiligen Feldern: Horizontal | Vertikal | Beide
[HScroll], [VScroll]

Zeichen für Passwörter....

Aktiv nur bei einzeiligen Feldern.
[EchoChar]

Daten

keine weiteren Besonderheiten

Ereignisse

keine weiteren Besonderheiten

Numerisches Feld

Neben den bereits unter *Standardeinstellungen vieler Kontrollfelder* erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Min. Wert..... -1000000,00 ▲▼

Mindestwert, den dieses Feld einnehmen kann. Sollte mit dem Mindestwert übereinstimmen, der in der Tabelle erwartet wird.
[ValueMin]

Max. Wert..... 1000000,00 ▲▼

Maximalwert
[ValueMax]

Intervall..... 1 ▲▼

Intervall-Wert für die Funktion als Scrollelement per Mausekranz bzw. Drehfeld
[ValueStep]

Standardwert.....

Wert, der beim Erstellen eines neuen Datensatzes angezeigt wird.
[DefaultValue]

Nachkommastellen..... 2 ▲▼

Anzahl Nachkommastellen, bei Integer-Feldern auf 0 zu stellen
[DecimalAccuracy]

Tausender-Trennzeichen..... Nein ▼

Trennzeichen für Tausenderstellen, in der Regel der Punkt
[ShowThousandsSeparator]

Daten

Es erfolgt keine Nachfrage, ob das Feld NULL sein soll. Eine fehlende Eingabe belässt dennoch das Feld auf NULL, nicht auf 0.

Ein Filtervorschlag wird ebenfalls nicht erstellt.

Ereignisse

Es fehlt das Feld «Modifiziert». Änderungen werden über «Text modifiziert» (hier wohl nicht wörtlich zu nehmen) angesprochen.

Datumfeld

Neben den bereits unter *Standardeinstellungen vieler Kontrollfelder* erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Min. Datum..... 

Mindestwert für das Feld, einstellbar über ein Aufklappfeld, das einen Kalender bereitstellt.

[DateMin]

Max. Datum..... 

Maximalwert.

[DateMax]

Datumsformat..... 

Kurzform wie 10.02.12 sowie unterschiedliche Formen auch mit ' / ' statt ' . ' oder ' - ' in amerikanischer Schreibweise.

[DateFormat]

Standarddatum..... 

Hier kann ein festes Datum vorgegeben werden. Das aktuelle Datum (Heute) beim Aufruf des Formulars muss leider (noch) durch ein Makro eingetragen werden.

[DefaultDate]

Aufklappbar..... 

Ein Monatskalender zur Auswahl des Tages kann eingeblendet werden.

[DropDown]

Daten

Es erfolgt keine Nachfrage, ob das Feld NULL sein soll. Eine fehlende Eingabe belässt dennoch das Feld auf NULL, nicht auf 0.

Ein Filtervorschlag wird ebenfalls nicht erstellt.

Ereignisse

Es fehlt das Feld «Modifiziert». Änderungen werden über «Text modifiziert» (hier wohl nicht wörtlich zu nehmen) angesprochen.

Zeitfeld

Neben den bereits unter *Standardeinstellungen vieler Kontrollfelder* erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Min. Zeit..... 00:00:00

Mindestwert für das Feld, standardmäßig auf 0 gesetzt.
[TimeMin]

Max. Zeit..... 23:59:59

Maximalwert, standardmäßig auf 1 Sekunde unter 24 Uhr gesetzt.
[TimeMax]

Zeitformat..... 13:45

Kurzform ohne Sekunden, Langform mit Sekunden sowie Benennungen mit PM (post meridiem / Nachmittag)
[TimeFormat]

Standardzeit.....

Eine feste Zeit ist voreinstellbar, die aktuelle Zeit beim Abspeichern des Formulars leider (bisher) nur mit Makro.
[DefaultTime]

Daten

Es erfolgt keine Nachfrage, ob das Feld NULL sein soll. Eine fehlende Eingabe belässt dennoch das Feld auf NULL, nicht auf 0.
Ein Filtervorschlag wird ebenfalls nicht erstellt.

Ereignisse

Es fehlt das Feld «Modifiziert». Änderungen werden über «Text modifiziert» (hier wohl nicht wörtlich zu nehmen) angesprochen.

Währungsfeld

Neben den bereits unter *Standarddeinstellungen vieler Kontrollfelder* erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Min. Wert, Max. Wert, Intervall, Standardwert, Nachkommastellen und Tausender-Trennzeichen entsprechen den allgemeinen Eigenschaften *Numerisches Feld*. Daneben gibt es lediglich:

Währungssymbol..... €

Das Symbol wird angezeigt, aber nicht in der dem Formular zugrundeliegenden Tabelle mit abgespeichert.
[CurrencySymbol]

Symbol voranstellen..... Nein

Soll das Symbol vor oder hinter der Zahl erscheinen?
[PrependCurrencySymbol]

Daten

Es erfolgt keine Nachfrage, ob das Feld NULL sein soll. Eine fehlende Eingabe belässt dennoch das Feld auf NULL, nicht auf 0.
Ein Filtervorschlag wird ebenfalls nicht erstellt.

Ereignisse

Es fehlt das Feld «Modifiziert». Änderungen werden über «Text modifiziert» (hier wohl nicht wörtlich zu nehmen) angesprochen.

Formatiertes Feld

Neben den bereits unter *Standardeinstellungen vieler Kontrollfelder* erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Min. Wert und Max. Wert und Standardwert hängen von der Formatierung ab. Hinter dem Button zur Formatierung versteckt sich ein «Allroundfeld», das Währungsfeld und Zahlenfeld meist überflüssig macht. Im Gegensatz zum einfachen Währungsfeld kann das formatierte Feld negative Beträge auch in roter Farbe darstellen.

Formatierung..... ...

Über den rechten Button mit den drei Punkten erscheint die Auswahl zum Zahlenformat, die auch in LibreOffice-Calc üblich ist. [FormatKey]

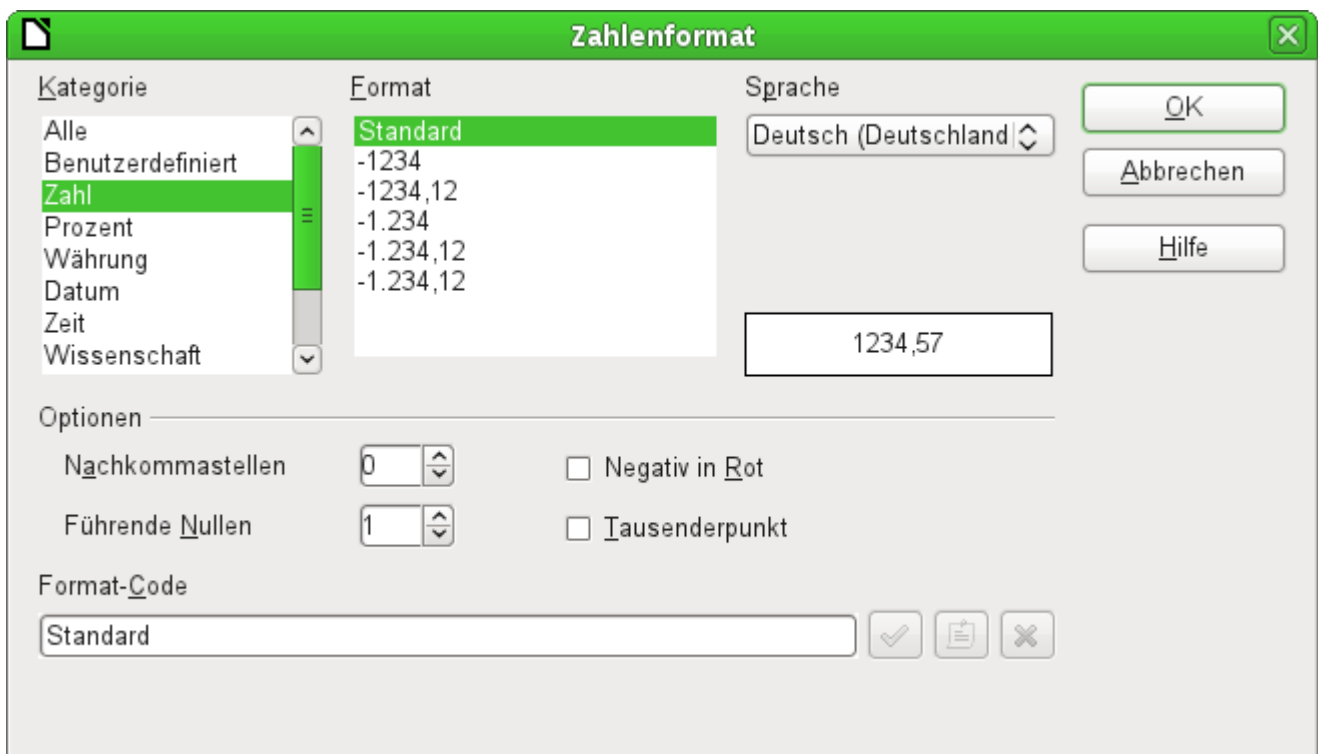


Abbildung 24: Formatiertes Feld mit allgemeinen Zahleneinstellungen

In dem Zahlenformat sind dann neben Datum, Zeit, Währung oder normalen Zahlenformaten auch Möglichkeiten gegeben, die Felder mit einer Maßeinheit wie z. B. **kg** zu bestücken. Siehe dazu die allgemeine Hilfe zu Zahlenformat-Codes.

Mit dem formatierten Feld ist es auch möglich, Timestamp-Felder aus Tabellen mit nur einem Feld darzustellen und zu beschreiben. Der Formularassistent nutzt dazu sonst ein Datumsfeld und ein Zeitfeld.

Nur mit Hilfe von Makros ist es bei Timestamp-Feldern möglich, Eingaben im Format Minuten:Sekunden,Hundertstelsekunden zu erreichen.

Daten

keine weiteren Besonderheiten.

Ereignisse

Es fehlt das Feld «Modifiziert». Änderungen werden über «Text modifiziert» (hier wohl nicht wörtlich zu nehmen) angesprochen.

Listenfeld

Sobald ein Listenfeld erstellt wird, erscheint standardmäßig der Listenfeldassistent. Diese Automatik lässt sich gegebenenfalls über Assistenten an/aus ([Abbildung 22](#)) abschalten.

Assistent

Listenfeld-Assistent - Tabellenauswahl

Formular

Art des Inhaltes	Tabelle
Inhalt	Ausleihe

Kontrollfeld

Auf der rechten Seite sehen Sie alle Tabellen aus der Datenquelle des Formulars.

Wählen Sie bitte die Tabelle, deren Daten die Grundlage für den Listeninhalt bilden sollen:

- Jahrgang
- Kategorie
- Klasse
- Leser**
- Mahnung
- Medien
- Medienart
- Ort
- Postleitzahl
- rel_Leser_Schulklasse
- rel_Medien_Verfasser
- Schulklasse
- Strasse
- Suche
- Untertitel

<< Zurück **Weiter >>** Fertigstellen Abbrechen

Das Formular ist bereits definiert. Es verbindet mit einer *Tabelle*, die den Namen "Ausleihe" trägt. Ein Listenfeld zeigt andere Daten für den Nutzer an als solche, die an die *Tabelle* weitergegeben werden. Diese Daten stammen bei Datenbanken in der Regel aus einer anderen Tabelle als der, mit der das Formular verbunden ist.

In der Tabelle "Ausleihe" soll verzeichnet werden, welcher "Leser" welche Medien entliehen hat. Allerdings wird in dieser Tabelle nicht der Name des Lesers gespeichert sondern der Primärschlüssel aus der Tabelle "Leser". Die Tabelle "Leser" bildet also die Grundlage für das Listenfeld.

Listenfeld-Assistent - Feldauswahl

Vorhandene Felder	Anzeigefeld
ID	Nachname
Vorname	
Nachname	
Sperre	
Geschlecht_ID	

Der Inhalt des ausgewählten Feldes wird in dem Listenfeld angezeigt, wenn die verknüpften Felder übereinstimmen

<< Zurück Weiter >> Fertigstellen Abbrechen

Das Feld "Nachname" aus der Tabelle "Leser" soll in dem Listenfeld sichtbar sein. Es ist also das *Anzeigefeld*.

Listenfeld-Assistent - Feldverknüpfung

Wählen Sie hier die Felder aus, deren Inhalt übereinstimmen muss, damit der Wert aus dem Anzeigefeld angezeigt wird.

Feld aus der Wertetabelle	Feld aus der Listentabelle
Leser_ID	ID
ID	ID
Medien_ID	Vorname
Leser_ID	Nachname
Leih_Datum	Sperre
Rueck_Datum	Geschlecht_ID
Verlaengerung	
Medien_ID_BC	

<< Zurück Weiter >> Fertigstellen Abbrechen

Das Feld "Leser_ID" befindet sich in der dem Formular zugrundeliegenden Tabelle "Ausleihe". Diese Tabelle wird hier als *Wertetabelle* bezeichnet. Der Primärschlüssel "ID" aus der Tabelle "Leser" soll mit diesem Feld verbunden werden. Die Tabelle "Leser" wird hier als *Listentabelle* bezeichnet.

Das Listenfeld ist jetzt komplett mit Daten und Standardeinstellungen erstellt und funktionsfähig.

Neben den bereits unter *Standardeinstellungen vieler Kontrollfelder* erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Listen-Einträge..... 

Die Listeneinträge wurden bereits durch den Assistenten erstellt. Hier könnten sonst auch Einträge stehen, die in keiner Tabelle der Datenbank verzeichnet sind. Mit Listeneinträgen sind hier die sichtbaren Einträge gemeint; nicht die Einträge, die über das Formular an die Tabelle weitergegeben werden.
[StringItemList]

Aufklappbar..... 

Wird das Feld als nicht aufklappbar festgelegt, so erscheinen bei Aufruf des Formulars am rechten Rand des Listenfeldes Scrollpfeile. Das Listenfeld wird dann automatisch zu einem mehrzeiligen Feld, bei dem die aktuelle Auswahl markiert ist.
[Dropdown]

Anzahl der Zeilen..... 

Falls aufklappbar wird hier die maximal sichtbare Anzahl der Zeilen eingegeben. Geht der Inhalt über mehr Zeilen, so erscheint beim Aufklappen ein Scrollbalken.
[LineCount]

Mehrfachselektion..... 

Sollen mehrere Werte selektiert werden können? Im obigen Beispiel ist das nicht möglich, da ein Fremdschlüssel abgespeichert wird. In der Regel wird diese Funktion in Verbindung mit Datenbanken nicht genutzt werden, da jedes Feld eigentlich nur einen Wert einnehmen sollte. Gegebenenfalls kann eine Auswertung des Eintrags in das Listenfeld über Makros hier Abhilfe schaffen.
[MultiSelection] [MultiSelectionSimpleMode]

Standardselektion.....  

Schon der deaktivierte Button macht deutlich, dass auch eine Standardselektion bei einer Verbindung mit einer Datenbanktabelle, wie über den Listenfeldassistenten erzeugt, wenig Sinn macht. Schließlich kann es passieren, dass der der Standardselektion entsprechende Datensatz in der Beispieltabelle "Leser" gar nicht mehr vorhanden ist.
[DefaultSelection]

Daten



The screenshot shows the 'Eigenschaften: Listenfeld' dialog box with the 'Daten' tab selected. The 'Datenfeld' is set to 'Leser_ID'. 'Eingabe erforderlich' is set to 'Ja'. 'Art des Listeninhalts' is set to 'SQL'. 'Listeninhalt' is set to 'SELECT "Nachname", "ID"'. 'Gebundenes Feld' is set to '1'.

Neben den üblichen Daten-Eigenschaften Datenfeld und Eingabe erforderlich sind hier Eigenschaften von Bedeutung, die die Verbindung von anzuzeigenden Daten und in die dem Formular zugrundeliegende Tabelle einzutragenden Daten herstellen.

Art des Listeninhaltes: Werteliste | Tabelle | Abfrage | SQL | SQL (Native) | Tabellenfelder [List-SourceType]

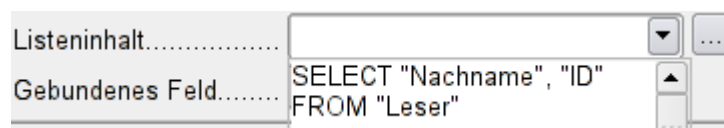
Listeninhalt Werteliste: Sind unter 'Allgemein' Listeneinträge gemacht worden, so werden hier die entsprechenden Werte eingegeben, die mit dem Formular abgespeichert werden sollen. Der Listeninhalt wird bestückt, indem bei der Eingabe die Inhalte über **Shift - Enter** aneinandergehängt werden. Es erscheint dann in dem Listeninhalt "**Wert1**"; "**Wert2**"; "**Wert3**" ... Die Eigenschaft 'Gebundenes Feld' ist *inaktiv*.

Listeninhalt Tabelle: Hier wird eine der Datenbanktabellen ausgesucht. Dabei muss aber berücksichtigt werden, dass dann immer das erste Feld der Tabelle in dem Listenfeld angezeigt wird.

Üblicherweise sind Tabellen so aufgebaut, dass zuerst der Primärschlüssel erscheint und anschließend die Inhalte. Solche Tabellen eignen sich nur dann für ein Listenfeld, wenn tatsächlich der Primärschlüssel angezeigt werden soll.

Listeninhalt Abfrage: Hier kann extern eine Abfrage vorformuliert werden, die dann auch als Abfrage sichtbar abgespeichert wird. Die Gestaltung solcher Abfragen wird im Kapitel 'Abfragen' erklärt. Durch die Abfrage ist es dann möglich, das Feld "ID" von der ersten Position in der ursprünglichen Tabelle an die zweite Position zu bewegen, was hier dem gebundenen Feld 1 entspricht.

Listeninhalt SQL: Hiermit bestückt der Listenfeldassistent das Listenfeld. Die von dem Assistenten konstruierte Abfrage sieht dann so aus:



The screenshot shows the 'Listeninhalt' and 'Gebundenes Feld' fields. 'Listeninhalt' is empty, and 'Gebundenes Feld' is set to '1'. The SQL query 'SELECT "Nachname", "ID" FROM "Leser"' is visible in the background.

Die Abfrage ist die einfachste Möglichkeit, die sich bietet. Das Feld "Nachname" erscheint an Position 0, das Feld "ID" an Position 1. Beide werden aus der Tabelle "Leser" ausgelesen. Da das gebundene Feld das Feld 1 ist reicht diese SQL-Formulierung aus. Schon vom Standard her sollte aber hier ergänzt werden **ORDER BY "Nachname" ASC**, denn ohne diese Formulierung werden die Nachnamen so wiedergegeben, dass ihre Reihenfolge der Eingabe in die Tabelle Leser entspricht. Zusätzliches Problem dürfte sein, dass "Nachname" bei einigen Lesern gleich ist. Hier muss dann "Vorname" hinzugezogen werden können und, als letztes Mittel, wohl auch der Primärschlüssel "ID". Wie so etwas genauer formuliert wird, kann im Kapitel 'Abfragen' genauer nachgelesen werden.

Listeninhalt SQL (Nativ): Die SQL-Formulierung wird direkt eingegeben, kein Assistent wird dazu aufgerufen. Base wertet die Abfrage nicht aus. Dies bietet sich an, wenn in der Abfrage Funktionen stecken, die eventuell von der GUI von Base nicht verstanden werden. Dann wird die Abfrage nicht weiter auf Fehler untersucht. Genauer zu dem direkten SQL-Modus im Kapitel 'Abfragen'.

Listeninhalt Tabellenfelder: Hier werden die *Feldnamen* einer Tabelle aufgelistet, nicht die Inhalte. Für die Tabelle "Leser" erscheint dort also die Liste "ID", "Vorname", "Nachname", "Sperr", "Geschlecht_ID".

Hinweis

Für Zeitfelder, die auch Zeiten im Millisekunden-Bereich darstellen sollen, sind, wie unter «[Zeitfelder in Tabellen](#)» beschrieben, Timestamp-Felder erforderlich. Die Darstellung der Millisekunden funktioniert bei der Zusammenfügung von Zeichen in einem Listenfeld nicht. Hier muss der Timestamp in Text umgewandelt werden:

```
SELECT REPLACE(LEFT(RIGHT(CAST("Zeit" AS  
VARCHAR(30)),15),8),'.','.') AS "Listinhalt", "ID"  
FROM "Leistung_erforderlich"
```

Dies erzeugt die Anzeige in Minuten:Sekunden,Hundertstelsekunden. (HSQLDB)

Gebundenes Feld: Im Listenfeld wird ein Inhalt angezeigt, der nicht mit dem Inhalt identisch sein muss, der auch im Formular abgespeichert werden soll. Üblicherweise wird z. B. Ein Name angezeigt und der Primärschlüssel zu diesem Namen als abzuspeicherndes Feld bestimmt.

```
SELECT "Name", "ID" FROM "Tabelle" ORDER BY "Name"
```

Das Feld "ID" wird als Fremdschlüssel in der Tabelle gespeichert, die dem Formular zugrunde liegt. Die Zählung der Felder in Datenbanken beginnt mit 0, das an das Formular gebundene Feld hat also die Position 1.

Wird stattdessen «0» gewählt, dann wird in dem Formular der Inhalt des Feldes "Name" abgespeichert. Das Feld "ID" könnte in der Abfrage entfallen.

Es ist sogar möglich, die Position «-1» zu wählen. Dann wird nicht der Inhalt der Abfrage sondern die Position des Eintrages in der Liste gespeichert. Der erste Datensatz hat dabei die Position 1.

Die oben aufgeführte Abfrage ergäbe z. B. die folgenden Daten:

Name	ID
Anneliese	2
Dorothea	3
Sieglinde	1

Im Listenfeld könnte nur der Name ausgewählt werden. Das Listenfeld wird auf den Namen «Dorothea» eingestellt. Folgende Abspeicherungen würden über das Listenfeld möglich:

Gebundenes Feld = 1 → «3» wird abgespeichert, da das Feld "ID" gespeichert wird.

Gebundenes Feld = 0 → «Dorothea» wird abgespeichert, da das Feld "Name" gespeichert wird.

Gebundenes Feld = -1 → «2» wird abgespeichert, da «Dorothea» als zweiter Datensatz in der Liste aufgeführt wird.

Hinweis

Die Änderung des gebundenen Feldes auf «0» oder «-1» ist erst mit der Version LO 4.1 eingeführt worden. Vorher war nur die Auswahl von Werten >= 1 möglich.

Hinweis

Ein Listenfeld zeigt in der Regel nicht nur die Werte an, die z.B. über SQL dargestellt werden. Als erstes wird ein leeres Feld (**NULL**) zur Auswahl angeboten. Dies ist notwendig, damit einmal eingegebene Daten über das Listenfeld auch wieder entfernt werden können.



Buchung von	Umbuchung auf
Bargeld	
Bargeld	Bargeld
Giro	Giro
Sparbuch	Sparbuch

Abbildung 25: Für das Buchungskonto ist in der Tabelle eine Eingabe erforderlich. Das Umbuchungskonto kann auch leer bleiben.

Soll von vornherein ausgeschlossen werden, dass das leere Feld angeboten wird, so ist die zugrundeliegende Tabelle für das Formular entsprechend einzustellen. Das Feld muss über **Feldeigenschaften** → **Eingabe erforderlich** → **Ja** so eingestellt sein, dass **NULL** nicht erlaubt ist.

Ereignisse

Neben den Standardereignissen werden folgende Ereignisse hinzugefügt:

Aktion ausführen: Wird durch Tastatureingabe ein Listenfeld dazu veranlasst, einen entsprechenden Inhalt anzuzeigen, so ist dies eine Aktion, die das Listenfeld ausführt. Auch die Auswahl des Wertes aus einem Listenfeld entspricht dieser Aktion.

Status geändert: Dies kann die Änderung des angezeigten Inhaltes eines Listenfeldes durch Betätigen des Aufklappbuttons sein. Es kann auch lediglich ein Klicken auf den Aufklappbutton des Feldes sein.

Fehler aufgetreten: Das Ereignis lässt sich leider beim Listenfeld nicht nachvollziehen.

Kombinationsfeld

Sobald ein Kombinationsfeld erstellt wird erscheint wie beim Listenfeld standardmäßig ein Assistent. Diese Automatik lässt sich gegebenenfalls über Assistenten an/aus ([Abbildung 22](#)) abschalten.

Kombinationsfelder schreiben direkt den ausgewählten Wert in die dem Formular zugrundeliegende Tabelle. Deshalb ist im folgenden Beispiel die dem Formular zugrundeliegende Tabelle die Tabelle "Leser" und die für das Kontrollfeld gewählte Tabelle ebenfalls die Tabelle "Leser".

Assistent

The dialog box is titled "Kombinationsfeld-Assistent - Tabellenauswahl". It has a green header bar with a close button (X) on the right. The main area is divided into two sections: "Formular" and "Kontrollfeld".

Formular:

Art des Inhaltes	Tabelle
Inhalt	Leser

Kontrollfeld:

Auf der rechten Seite sehen Sie alle Tabellen aus der Datenquelle des Formulars.

Wählen Sie bitte die Tabelle, deren Daten die Grundlage für den Listeninhalt bilden sollen:

- Adresse
- Artikel
- Ausleihe
- Einstellungen
- Filter
- Filter_Leser
- Geschlecht
- Jahrgang
- Kategorie
- Klasse
- Leser** (highlighted in green)
- Mahnung
- Medien
- Medienart
- Ort

At the bottom, there are four buttons: "<< Zurück", "Weiter >>" (highlighted in green), "Fertigstellen", and "Abbrechen".

Das Formular ist wieder vordefiniert, diesmal als Beispiel mit der Tabelle "Leser". Da die Daten, die in dem Listefeld angezeigt werden, auch in dieser Tabelle abgespeichert werden sollen, wird als Quelle für die Daten des Listefeldes ebenfalls die Tabelle "Leser" ausgewählt.

The dialog box is titled "Kombinationsfeld-Assistent - Feldauswahl". It has a green header bar with a close button (X) on the right. The main area is divided into two sections: "Vorhandene Felder" and "Anzeigefeld".

Vorhandene Felder:

- ID
- Vorname** (highlighted in green)
- Nachname
- Sperre
- Geschlecht_ID

Anzeigefeld:

Vorname

Der Inhalt des ausgewählten Feldes wird in der Liste des Kombinationsfeldes angezeigt.

At the bottom, there are four buttons: "<< Zurück", "Weiter >>" (highlighted in green), "Fertigstellen", and "Abbrechen".

In der Tabelle "Leser" befindet sich das Feld "Vorname". Es soll im Kombinationsfeld angezeigt werden.

Innerhalb einer Datenbank scheint es wenig Sinn zu machen, den Wert des Kombinationsfeldes nicht in einem Feld zu speichern. Aus der Tabelle "Leser" sollen die Vornamen ausgelesen werden und für neue Leser als Vornamen zur Verfügung stehen, damit bei gleichen Vornamen eben nicht immer wieder eine neue Eingabe erfolgen muss. Das Kombinationsfeld zeigt dann hier den Vornamen an und die Texteingabe braucht nicht weiter durchgeführt zu werden.

Soll ein neuer Wert eingegeben werden, so ist dies im Kombinationsfeld ohne weiteres möglich, denn das Kombinationsfeld zeigt genau das an, was es selbst an die zugrundeliegende Tabelle des Formulars weitergibt.

Neben den bereits unter *Standardeinstellungen vieler Kontrollfelder* und beim Listenfeld erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Automatisch füllen..... Ja ▼

Während der Eingabe von neuen Werten wird eine Liste der eventuell zutreffenden Werte angezeigt, die noch zur Auswahl stehen.
[AutoComplete]

Daten

Die Datenfeldern entsprechen den bereits vorgestellten Standardeinstellungen und den Einstellungen bei einem Listenfeld. Der SQL-Befehl weist hier allerdings eine Besonderheit auf:

`SELECT DISTINCT "Vorname" FROM "Leser"`

Durch den Zusatz des Begriffes **DISTINCT** werden gleiche Vornamen nur einmal angezeigt. Wieder fehlt allerdings bei der Erstellung durch den Assistenten eine Sortierung der Inhalte.

Ereignisse

Die Ereignisse entsprechen denen beim Listenfeld.

Markierfeld

Das Markierfeld erscheint direkt als eine Kombination von Markierbox und Beschriftung dieser Box.

Neben den bereits unter *Standardeinstellungen vieler Kontrollfelder* erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Titel..... Markierfeld ▼

Die Beschriftung dieser Box, erscheint standardmäßig rechts von der Box. Zusätzlich ist noch eine Verbindung zu einem Beschriftungsfeld möglich.
[Label]

Standardstatus..... Nicht ausgewählt ▼

Hier gibt es, abhängig von der Auswahl in 'Dreifacher Status', bis zu drei Möglichkeiten: *Nicht ausgewählt* | *Ausgewählt* | *Nicht definiert*. *Nicht definiert* entspricht als Eintrag in der dem Formular zugrundeliegenden Tabelle *NULL*.
[State]

Wortumbruch..... Nein ▼

Standardmäßig wird der Titel nicht umbrochen. Ein zu langer Titel wird abgeschnitten, wenn das Feld nicht groß genug ist.
[MultiLine]

Grafik..... ▼ ...

Hier kann eine Grafik statt bzw. zusätzlich zum Titel eingefügt werden. Dabei sollte beachtet werden, dass die Grafiken standardmäßig erst einmal nicht in das *.odb-Dokument eingebunden werden. Sinnvoll bei kleinen Grafiken ist es, die Grafiken einzubetten und nicht zu verknüpfen.
[Graphic]

Grafik-Ausrichtung..... Zentriert ▼

Ist eine Grafik ausgewählt, so kann sie im Verhältnis zum Titel ausgerichtet werden.
[ImagePosition] (0=links | 1=zentriert | 2=rechts)

Dreifacher Status.....

Standardmäßig gibt es für Markierfelder nur *Ausgewählt* (Wert: 1) und *Nicht ausgewählt* (Wert: 0). Beim dreifachen Status kommt die Definition als *leeres Feld* (NULL) hinzu. [TriState]

Daten

Dem Markierfeld kann ein Referenzwert [RefValue] zugewiesen werden. Allerdings wird an das darunterliegende Datenfeld nur der Wert 1 für (ein) und 0 für (aus) weitergegeben. Schließlich werden Markierfelder als Felder für die Auswahl von 'Ja' und 'Nein' genutzt.

Wird ein Markierfeld an ein Textfeld gebunden, so wird an das Textfeld ab der Version LO 4.2 standardmäßig «true» und «false» weiter gegeben. Allerdings liest das Markierfeld auch die Werte 1 und 0 korrekt aus. Eine klare Wahl des Referenzwertes als 1 und 0 dient dann dazu, auch 1 und 0 in die Felder zu schreiben.

Ereignisse

Es fehlen die Felder «Modifiziert», «Text modifiziert». «Vor der Aktualisieren» und «Nach dem Aktualisieren».

Zusätzlich erscheinen «Aktion ausführen» (siehe Listenfeld) und «Status geändert» (entspricht «Modifiziert»).

Optionsfeld

Das Optionsfeld entspricht bis auf eine Ausnahme in den allgemeinen Eigenschaften und die äußerlich andere Form (rund) dem eben beschriebenen Markierfeld.

Werden mehrere Optionsfelder mit gleichem Namen mit einem Tabellenfeld über das Formular verbunden, so kann von den Optionen immer nur eine Option gewählt werden.

Allgemein

Gruppenname.....

Das Optionsfeld wird bevorzugt für Gruppierungen verwandt. Zwischen mehreren Optionen kann dann ausgewählt werden. Deswegen erscheint hier auch der Gruppenname, unter dem die Option angesprochen werden kann. [GroupName]

Daten

Siehe Markierfeld, hier werden aber die Referenzwerte [RefValue], die eingetragen wurden, auch tatsächlich an das Datenfeld weitergegeben.

Ereignisse

Siehe Markierfeld

Grafisches Kontrollfeld

Ein grafisches Kontrollfeld übernimmt die Eingabe und Ausgabe von Bildmaterial in die Datenbank. Das darunterliegende Datenfeld muss ein Binärfeld sein, wenn es das Bild direkt speichern soll. Es kann auch ein Textfeld sein. Dann wird lediglich der relative Pfad zur Datei abgespeichert. Hierbei sollte dann aber darauf geachtet werden, dass der Pfad zu der Datei auch bei einer Weitergabe der Datenbank stimmig bleibt.

Seit LO 5.0 können über dieses Feld beliebige Dateien in Base eingebunden werden. Angezeigt werden in dem Fall aber nur Bilder und von *.pdf-Dateien die erste Seite.

Vorsicht



Bilder sollten auf jeden Fall verkleinert werden, wenn sie schon in der Datenbank gespeichert werden. Hat ein Kamerabild z.B. 3 MB, so passiert es sehr schnell bei Nutzung der internen HSQLDB, dass mit Angabe einer Java-Fehlermeldung (**NullPointerException**) die Abspeicherung der Daten nicht mehr möglich ist. Dies kann zum Verlust sämtlicher Daten führen, die seit dem Öffnen der Base-Datenbank in die Tabelle eingegeben wurden.

Die Eingabe in das grafische Kontrollfeld erfolgt entweder über einen Doppelklick mit der Maus; dann erscheint ein Dateiauswahlfenster. Oder sie erfolgt über einen Klick der rechten Maustaste; dann kann ausgewählt werden, ob die Grafik nur gelöscht oder eine andere geladen werden soll.

Ein Grafisches Kontrollfeld erhält standardmäßig erst einmal keinen Tabstop.

Neben den bereits unter [Standardeinstellungen vieler Kontrollfelder](#) erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Grafik..... ▼ ...

Die hier ausgesuchte Grafik wird nur im Kontrollfeld angezeigt, wenn das Formular bearbeitet wird. Für die spätere Eingabe ist sie ohne Bedeutung.
[Graphic]

Skalieren..... Seitenverhältnis beibehalten ▼

Nein: Das Bild wird nicht an das Kontrollfeld angepasst. Ist das Bild zu groß, so wird ein Ausschnitt des Bildes gezeigt. Das Bild wird nicht verzerrt.

Seitenverhältnis beibehalten: Das Bild wird in das Kontrollfeld angepasst, aber nicht verzerrt dargestellt.
Autom. Größe: Das Bild wird der Größe des Kontrollfeldes angepasst und gegebenenfalls verzerrt dargestellt.

[ScaleImage] [ScaleMode]

Daten

keine weiteren Besonderheiten

Ereignisse

Es fehlen die Ereignisse «Modifiziert», «Text modifiziert», «Vor dem Aktualisieren» und «Nach dem Aktualisieren».

Maskiertes Feld

Mittels einer Eingabemaske wird hier die Eingabe in das Feld gesteuert. Zeichen werden für eine bestimmte Position vorgegeben, einzugebende Zeichen in ihrer Eigenschaft festgelegt. Die vorgegebenen Zeichen werden dann mit abgespeichert.

Neben den bereits unter [Standardeinstellungen vieler Kontrollfelder](#) erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Eingabemaske.....

Hier wird festgelegt, welche Inhalte eingegeben werden können.
[EditMask]

Zeichenmaske.....

Diese Ansicht wird dem Formularnutzer präsentiert.
[LiteralMask]

Der folgende Inhalt entstammt der LibreOffice-Hilfe:

Die Länge der Eingabemaske bestimmt, wie viele Stellen die mögliche Eingabe haben darf. Sollten die vom Benutzer eingegebenen Zeichen der Eingabemaske nicht entsprechen, so wird die Eingabe beim Verlassen des Kontrollfelds verworfen. Zur Definition der Eingabemaske stehen die folgenden Zeichen zur Verfügung:

Zeichen	Bedeutung
L	Eine Textkonstante. Diese Stelle kann nicht editiert werden. Das Zeichen wird an der entsprechenden Position der Zeichenmaske angezeigt.
a	Es können die Buchstaben a-z und A-Z eingegeben werden. Großbuchstaben werden nicht in Kleinbuchstaben konvertiert.
A	Es können die Buchstaben A-Z eingegeben werden. Eingegebene Kleinbuchstaben werden automatisch in Großbuchstaben konvertiert.
c	Es können die Buchstaben a-z und A-Z sowie die Ziffern 0-9 eingegeben werden. Großbuchstaben werden nicht in Kleinbuchstaben konvertiert.
C	An dieser Stelle können die Zeichen A-Z und 0-9 angegeben werden. Wird ein Kleinbuchstabe angegeben, wird automatisch in Großschrift umgewandelt.
N	Es können nur die Zeichen 0-9 angegeben werden.
x	Es können alle druckbaren Zeichen angegeben werden.
X	Es können alle druckbaren Zeichen angegeben werden. Wird dabei ein Kleinbuchstabe verwendet, wird automatisch in Großschrift umgewandelt.

Definieren Sie z. B. für die Zeichenmaske «___.2000» die Eingabemaske «NNLNNLLLLL», um dem Benutzer nur noch die Eingabe von 4 Ziffern zur Datumsangabe zu ermöglichen.

Daten

keine weiteren Besonderheiten

Ereignisse

Es fehlt das Ereignis «Modifiziert»

Tabellen-Kontrollfeld

Dieses Kontrollfeld ist das umfassendste Feld. Es stellt eine Tabelle bereit, die wiederum durch Kontrollfelder für die einzelnen Spalten beschickt werden kann. Damit ist bei der Eingabe nicht nur der Blick auf die aktuell einzugebenden Daten, sondern auch auf vorher eingegebene Daten möglich, ohne mit der Navigationsleiste durch die Datensätze zu scrollen.

Nicht alle Felder, die auch im Formular möglich sind, können im Tabellen-Kontrollfeld wieder ausgewählt werden. So fehlen z. B. Schaltflächen, das grafische Kontrollfeld und das Optionsfeld.

Ein Assistent für das Tabellen-Kontrollfeld stellt in einem Fenster die Felder zusammen, die anschließend in der Tabelle zur Verfügung stehen sollen.

Tabellenelement-Assistent - Feldauswahl

Formular

Art des Inhaltes: Tabelle

Inhalt: Ausleihe

Tabellenelement

Vorhandene Felder: ID, Medien_ID_BC

Ausgewählte Felder: Medien_ID, Leser_ID, Leih_Datum, Rueck_Datum, Verlaengerung

Buttons: ->, ==>>, <-, <<=

Bottom Buttons: << Zurück, Weiter >>, Fertigstellen, Abbrechen

In dem Kontrollfeld soll die Tabelle "Ausleihe" zur Bearbeitung zur Verfügung stehen. Bis auf die Felder "ID" (Primärschlüssel) und das Feld "Medien_ID_BC" (Eingabe der Medien über einen Bar-codescanner) sollen alle Felder in dem Kontrollelement benutzt werden.

	Medien_ID	Leser_ID	Leih_Datum	Rueck_Datum	Verlaengerung	
▶	1,00	0,00	02.11.11	04.11.11		▲
	2,00	2,00	15.10.11	25.02.12	2,00	≡
	0,00	3,00	02.11.11	04.04.12	1,00	
	3,00	0,00	04.11.11	28.11.11	2,00	
	5,00	0,00	28.11.11			
	4,00	0,00	28.11.11	04.04.12		
	4,00	0,00	09.11.11			
	3,00	0,00	09.12.11			▼
Datensatz	1	von	18			

Abbildung 26: Ergebnis des Tabellen-Kontrollfeld-Assistenten

Das erst einmal erstellte Tabellen-Kontrollelement muss hier noch weiter bearbeitet werden, da ja nur die Eingaben der Tabelle "Ausleihe" ermöglicht werden. Für Felder wie "Leser_ID" oder "Medien_ID" wäre es aber sinnvoller, statt einer Nummer den Leser bzw. das Medium selbst auswählen zu können. Hierzu werden innerhalb des Tabellen-Kontrollelementes z. B. Listenfelder eingesetzt. Entsprechendes ist weiter unten beschrieben. Auch die Formatierung des Feldes "Verlängerung" mit zwei Nachkommastellen ist sicher nicht gewollt.

Neben den bereits unter [Standardeinstellungen vieler Kontrollfelder](#) erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Zeilenhöhe.....

In welcher Höhe sollen die einzelnen Zeilen angezeigt werden. Ohne einen Wert wird die Höhe automatisch der Schriftgröße angepasst. Mehrzeilige Textfelder würden so nur einzeilig zum Scrollen angezeigt.
[RowHeight]

Navigationsleiste.....

Wie bei Tabellen werden am unteren Rand des Kontrollfeldes die Datensatznummer und Navigationshilfen angezeigt.
[HasNavigationBar]

Datensatzmarkierer.....

Am linken Rand des Kontrollfeldes befindet sich standardmäßig der Datensatzmarkierer. Er zeigt den aktuellen Datensatz an. Über den Datensatzmarkierer wird die LösCHFunktion für den gesamten Datensatz zur Verfügung gestellt.
[HasRecordMarker]

Daten

Da es sich um ein Feld handelt, das selbst keine Daten anzeigt oder abspeichert, sondern stattdessen andere Felder verwaltet, die dies tun, gibt es keine Eigenschaft Daten.

Ereignisse

Es fehlt das Ereignis «Modifiziert» und «Text modifiziert». Das Ereignis «Fehler aufgetreten» kommt hinzu.

Beschriftungsfeld

Neben den bereits unter [Standardeinstellungen vieler Kontrollfelder](#) erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Titel..... Titel ▼

Der Titel dient als Beschriftung für ein anderes Kontrollfeld. Wird der Titel mit einem Kontrollfeld verbunden, dann kann er auch als Sprungziel für dieses Kontrollfeld festgelegt werden. Mit einem eingefügten «~» kann ein bestimmter Buchstabe des Titels als Sprungziel angegeben werden. «Na~me» definiert das «m» als Sprungziel. Befindet sich der Cursor in dem Formular, so wird die Schaltfläche über **Alt + m** erreicht.

Wortumbruch..... Nein ▼

Standardmäßig wird die Beschriftung nicht umbrochen. Ein zu langer Titel wird abgeschnitten, wenn das Feld nicht groß genug ist. Der Wortumbruch hat allerdings keine Trennfunktion, so dass bei zu kleinen Feldern innerhalb eines Wortes ein Umbruch erfolgt.
[MultiLine]

Daten

keine

Ereignisse

Selbst das Beschriftungsfeld reagiert noch auf Ereignisse, die mit der Maus einer Taste oder dem Fokuserhalt zusammenhängen.

Gruppierungsrahmen

Der Gruppierungsrahmen fasst lediglich grafisch mehrere Kontrollfelder zu einer Gruppe zusammen und versieht sie mit einem zusätzlich grafisch eingebundenen Titel.

Wird ein Gruppierungsrahmen mit eingeschaltetem Assistenten aufgezogen, so geht der Assistent davon aus, dass mehrere Optionsfelder zusammen in diesem Rahmen erscheinen sollen.

Gruppenelement-Assistent - Daten

Formular

Art des Inhaltes	Tabelle
Inhalt	Leser

Welche Bezeichnungen sollen die Optionsfelder erhalten?

weiblich >> männlich

<< Zurück Weiter >> Fertigstellen Abbrechen

Dem Formular liegt die Tabelle Leser zugrunde. Hier wird gerade die Auswahl des Geschlechtes zusammengestellt. Die Einträge sind anschließend die Beschriftungen der Optionsfelder.

Gruppenelement-Assistent - Standardfeldauswahl

Soll ein Optionsfeld standardmäßig ausgewählt sein?

☒ Ja, und zwar folgendes: weiblich

☐ Nein, kein Feld soll ausgewählt sein.

<< Zurück Weiter >> Fertigstellen Abbrechen

Standardmäßig wird hier vorgewählt, dass das Geschlecht «weiblich» angewählt wird. Wird kein Feld vorgewählt, so ist der Standardeintrag in die zugrundeliegende Tabelle *NULL*.

Gruppenelement-Assistent - Feldwerte

Wenn Sie eine Option auswählen, wird der Optionsgruppe ein bestimmter Wert zugewiesen.

Welchen Wert möchten Sie jeder Option zuweisen?

1

Optionsfelder

- männlich
- weiblich

<< Zurück Weiter >> Fertigstellen Abbrechen

Den Optionsfeldern wird bereits standardmäßig durch den Assistenten ein jeweils separater Wert zugeteilt. Da dieser Wert in unserem Fall nicht dem des Primärschlüssels entspricht, wird er geändert, und zwar für «männlich» auf den Wert 'm', für «weiblich» auf den Wert 'w'. Diese Werte entsprechen in dem Beispiel den Primärschlüsselfeldern der Tabelle "Geschlecht"

Der jeweils in den Optionsfeldern angeklickte Wert wird an das Feld "Geschlecht_ID" der dem Formular zugrundeliegenden Tabelle "Leser" übertragen. Die Tabelle "Leser" wird also über ein Optionsfeld mit den entsprechenden Fremdschlüsseln der Tabelle "Geschlecht" versorgt.

Die Gruppe der Optionsfelder erhält einen Rahmen, in den die Beschriftung 'Geschlecht' eingefügt wird.

Wird jetzt bei aktivem Formular 'männlich' ausgewählt, so wird gleichzeitig 'weiblich' abgewählt. Dies ist ein Kennzeichen der Optionsfelder, wenn sie mit gleichem Namen versehen und mit dem gleichen Feld der dem Formular zugrundeliegenden Tabelle verbunden sind. In dem obigen Beispiel ersetzen die Optionsfelder so ein Listefeld mit zwei Einträgen.

Neben den bereits unter [Standardeinstellungen vieler Kontrollfelder](#) erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Der Titel kann neben den Standardeinstellungen verändert werden. Eine Veränderung des Rahmens (Liniendicke, Linienfarbe) ist zur Zeit nicht vorgesehen, wohl aber eine Formatierung der Schrift.

Daten

Keine, fasst schließlich Eingabefelder nur optisch zusammen.

Ereignisse

Auch der Gruppierungsrahmen reagiert noch auf Ereignisse, die mit der Maus einer Taste oder dem Fokuserhalt zusammenhängen.

Schaltfläche

Neben den bereits unter *Standardeinstellungen vieler Kontrollfelder* erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Titel..... Schaltfläche ▼

Beschriftung auf der Schaltfläche.
Mit einem eingefügten «~» kann ein bestimmter Buchstabe des Titels als Sprungziel angegeben werden.
«Schalt~fläche» definiert das «f» als Sprungziel. Befindet sich der Cursor in dem Formular, so wird die Schaltfläche über **Alt + f** erreicht.
[Label]

Fokussieren bei Klick..... Ja ▼

Die Schaltfläche erhält den Fokus, wenn auf sie geklickt wird.
Dies kann manchmal nicht erwünscht sein, wenn z.B. über die Schaltfläche Inhalt in ein anderes Formularfeld eingefügt werden soll. Dann soll bei Betätigung der Schaltfläche der Cursor in dem anderen Feld stehen bleiben.
[FocusOnClick]

Umschalten..... Nein ▼

Wenn ja, so kann die Schaltfläche auch eingedrückt dargestellt werden. Der Schaltzustand wird wie bei einem Schalter angezeigt – im Gegensatz zu einem Taster wie sonst bei Buttons.
[Toggle]

Standardstatus..... Nicht ausgewählt ▼

Aktiv, wenn Umschalten auf Ja gesetzt ist. Ausgewählt ist dann der eingedrückte Zustand.
[DefaultState]

Wortumbruch..... Nein ▼

Wortumbruch, wenn die Schaltfläche zu schmal ist.
[MultiLine]

Aktion..... Keine ▼

Verschiedene Aktionen ähnlich denen der Navigationsleiste stehen zur Verfügung.
Es kann hier auch ein Dokument oder eine Webseite geöffnet werden. Hierzu ist die URL in dem folgenden Feld einzugeben.

URL..... ▼ ...

HTML: Datei, die hiermit aufgerufen werden soll. Hierzu muss unter 'Aktion' 'Dokument/Webseite öffnen' gewählt werden.
Neben HTML kann dies auch genutzt werden, um bestimmte Programmmodule von LO aufzurufen. So wird mit dem hier einzutragenden Befehl '**.uno:RecSearch**' z. B. die Suchfunktion aus dem Formularnavigator auf einen Button gelegt. Außerdem lassen sich auch z. B. Writerdateien öffnen, die anschließend per Druck einen mit der Datenbank zusammenhängenden Serienbrief ausgeben.
Welche Befehle hier zur Verfügung stehen, lässt sich über den Makrorecorder gegebenenfalls feststellen.
[TargetURL]

Frame..... ▼

Nur für HTML-Formulare: Der Ziel-Frame (Rahmeneinteilung für verschiedene HTML-Seiten), in dem die Datei geöffnet werden soll.
[TargetFrame]

Standardschaltfläche..... Nein ▼

Schaltfläche wird bei Ja zusätzlich umrandet. Bei mehreren alternativen Schaltflächen auf einem Formular soll dies die übliche Schaltfläche kennzeichnen. Sie wird durch Betätigung der Eingabetaste ausgelöst, wenn keine andere Aktion gerade auf die Eingabetaste reagieren muss. Nur eine Schaltfläche sollte im Formular die Standardschaltfläche sein.
[DefaultButton]

Grafik..... ▼ ...

Soll eine Grafik auf der Schaltfläche erscheinen?
[Graphic] [ImageURL]



Nur aktiv, wenn eine Grafik ausgewählt wurde. Die Grafik wird im Verhältnis zum Text angeordnet.
[ImagePosition] [ImageAlign]

Daten

Keine, es werden nur Aktionen ausgeführt.

Ereignisse

«Aktion bestätigen», «Aktion ausführen» und «Statusänderung»

Grafische Schaltfläche

Neben den bereits unter *Eigenschaften der Kontrollfelder* erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Entspricht der normalen Schaltfläche. Allerdings wird diese Schaltfläche ohne Text und ohne einen sichtbaren Button dargestellt. Lediglich eine Rahmen um die Grafik kann eingestellt werden.

Eine grafische Schaltfläche erhält standardmäßig erst einmal keinen Tabstop.

Achtung: zur Zeitpunkt der Handbucherstellung funktioniert nahezu keine Aktion mit dieser Schaltfläche. Sie ist fast ausschließlich mit Makros nutzbar.

Daten

Keine, es werden nur Aktionen ausgeführt.

Ereignisse

«Aktion bestätigen» sowie alle Ereignisse, die die Maus, eine Taste oder den Fokus betreffen.

Navigationsleiste



Abbildung 27: Formularkontrollelement Navigationsleiste

Die Standardnavigationsleiste wird beim Formular am unteren Bildrand eingeblendet. Durch das Einblenden dieser Leiste kann es beim Bildschirmaufbau zu einem kurzen Zurechtschieben des Formulars kommen. Dies ist vor allem dann störend, wenn die Navigationsleiste eventuell in Teilen des sichtbaren Formulars ausgeschaltet ist.

Eine separat zu den jeweiligen Elementen eines Formulars angeordnete Navigationsleiste macht dagegen klar, durch welche Elemente mit der Leiste navigiert wird. Diese Navigationsleiste bietet die folgenden Elemente:



Die Such- und Filterfunktionen werden zum Schluss des Kapitels erklärt.

Die Navigation durch die Datensatznummern dürfte von der Bedienung von Multimedia-Geräten bekannt sein. Wird bei der Datensatznummer eine konkrete Nummer eingegeben, so bedeutet dies einen Sprung zum entsprechenden Datensatz.

Beim Aktualisieren gibt es auch den Button «Steuerelement aktualisieren». Werden Inhalte von Listefeldern durch ein anderes Formular beeinflusst, so muss der Cursor in das Listefeld gesetzt werden und der Inhalt kann über diesen Button neu eingelesen werden.

Neben den bereits unter *Standardeinstellungen vieler Kontrollfelder* erklärten Eigenschaften gibt es die folgenden Besonderheiten:

Allgemein

Symbolgröße.....	Klein	▼
Positionierung.....	Anzeigen	▼
Navigation.....	Anzeigen	▼
Datensatzaktionen.....	Anzeigen	▼
Filter / Sortierung.....	Anzeigen	▼

Die Größe der Symbole kann eingestellt werden. Außerdem ist auswählbar, welche Gruppen angezeigt werden. Dies sind in [Abbildung 27](#) von links nach rechts, bei den Symbolen durch eine senkrechte Linie getrennt, die Positionierung, die Navigation, die Datensatzaktionen sowie die Gruppe zur Filterung und Sortierung.

[IconSize]

[ShowPosition]

[ShowNavigation]

[ShowRecordActions]

[ShowFilterSort]

Daten

Keine, es werden nur Aktionen ausgeführt.

Ereignisse

Alle Ereignisse, die die Maus, eine Taste oder den Fokus betreffen.

Unabhängig von diesem Formularkontrollelement existiert natürlich auch die unten **einblendbare Navigationsleiste** mit den Elementen der obigen Abbildung.

Diese einblendbare Navigationsleiste bietet neben den Elementen des entsprechenden Formularkontrollelementes noch die *allgemeine Datensatzsuche*, den *formularbasierten Filter* und die Darstellung der dem Formular zugrundeliegenden *Datenquelle in der Tabellenansicht* oberhalb des Formulars.

Wird nicht nur mit einem Formular, sondern mit Unterformularen und Nebenformularen gearbeitet, so ist darauf zu achten, dass diese einblendbare Navigationsleiste nicht beim Formularwechsel je nach Einstellung verschwindet. Das erzeugt nur Unruhe auf dem Bildschirm.

Drehfeld und Bildlaufleiste

Beide Felder haben keine direkte Verbindung zu Daten des Formulars. Sie lassen sich nur über Makros entsprechend nutzen, wobei das Drehfeld von der Anwendung her bereits z.B. in das Zahlenfeld integriert ist.

Die Bildlaufleiste unterscheidet sich von dem Drehfeld dadurch, dass sie neben den Buttons für steigende und fallende Werte innerhalb einer vorher festgelegten Grenze auch einen Schiebeknopf haben. Der Name «Bildlaufleiste» ist dabei etwas irritierend, da über dieses Schieberegler nicht irgendwelche Bilder verschoben werden, sondern lediglich Werte in einem Feld verändert werden können.

Beim Start eines Formulars muss der Bildlaufleiste zuerst mitgeteilt werden, welchen Wert das Feld hat, das mit ihr verbunden sein soll. Dieses Makro ist an die **Formular-Eigenschaften → Ereignisse → Nach dem Datensatzwechsel** gebunden.

```
SUB Bildlaufleiste_Formular(oEvent AS OBJECT)
    DIM oForm AS OBJECT
    DIM oFeld AS OBJECT
    oForm = oEvent.Source
    oFeld = oForm.getByName("Bildlaufleiste")
    oFeld.ScrollValue = oForm.getInt(oForm.findColumn("Zahlenwert"))
END SUB
```

Zuerst werden die Variablen deklariert. Das auslösende Ereignis weist dabei auf das Formular hin, da das Makro schließlich an das Formular gebunden wurde. Die Bildlaufleiste wird nach dem entsprechenden Namen in dem Formular gefunden. Der Wert der Bildlaufleiste wird auf den Wert eingestellt, der in dem Feld "Zahlenwert" der dem Formular zugrundeliegenden Tabelle liegt.

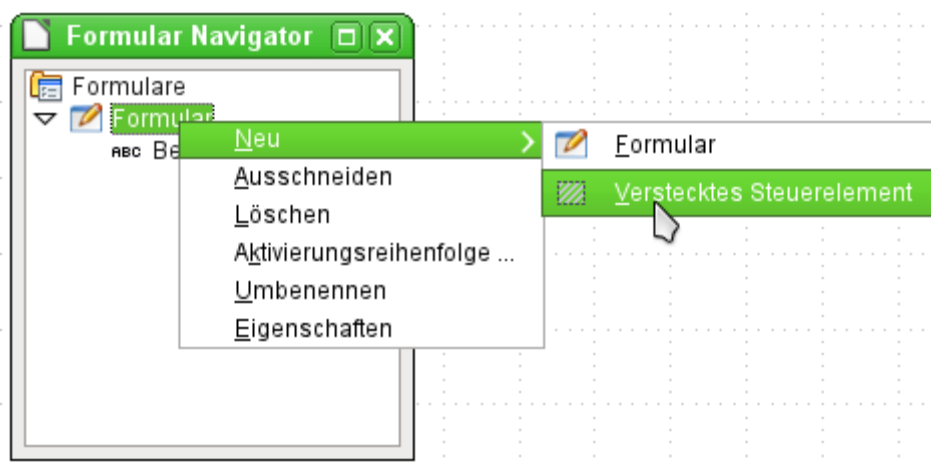
Wird der Wert der Bildlaufleiste geändert, so muss entsprechend der Wert in dem verbundenen Feld geändert werden. Dieses Makro wird an **Eigenschaften: Bildlaufleiste → Ereignisse → Beim Justieren** gebunden

```
SUB Bildlaufleiste_Aenderung(oEvent AS OBJECT)
    DIM oForm AS OBJECT
    DIM oFeld AS OBJECT
    DIM inValue AS INTEGER
    oFeld = oEvent.Source.Model
    inValue = oEvent.Value
    oForm = oFeld.Parent
    oForm.updateInt(oForm.findColumn("Zahlenwert"), inValue)
END SUB
```

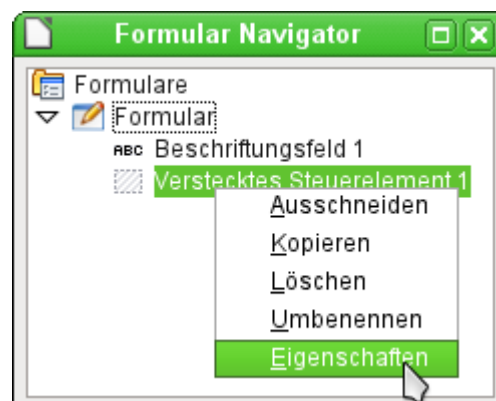
Zuerst werden die Variablen deklariert. Aus dem Feld wird der gewählte Wert als Ganzzahlenwert ausgelesen. Das dem Formular zugrundeliegende Tabellenfeld "Zahlenwert" wird auf den entsprechenden Wert eingestellt.

Details zu einem Code wie diesem sind in dem Kapitel «Makros» zu finden.

Verstecktes Steuerelement



Ein Kontrollfeld, das nicht über die Symbolleisten einfügbar ist, ist das versteckte Steuerelement. Es wird über den Formelnavigator im Kontextmenü eines Formulars neu erstellt.



Das versteckte Steuerelement erscheint wie jedes andere Kontrollfeld als Teil des Formulars. Es ist auf der Benutzeroberfläche nicht sichtbar.

Die kompletten Eigenschaften des versteckten Steuerelementes sind recht spärlich. Der Name und die Zusatzinformationen sind von den anderen Steuerelemente bekannt. Zusätzlich kann hier ein Wert angegeben werden. [HiddenValue]

Für die Nutzung ohne Makros ist dies sicher nicht weiter von Bedeutung. Bei der Arbeit mit Makros kann es aber sinnvoll sein, an einer Stelle des Formular Werte zwischenspeichern zu können, um auf sie bei einer weiteren Aktion wieder zugreifen zu können. Ein entsprechendes Anwendungsbeispiel findet sich bei den Makros im Kapitel «[Hierarchische Listenfelder](#)».

Mehrfachselektion

Wird mit dem Auswahlpfeil (siehe [Abbildung 22](#)) ein größerer Bereich oder sogar sämtliche Elemente eines Formulars ausgewählt, so sind über diese Mehrfachselektion die Änderungen nach [Abbildung 28](#) möglich.

Eigenschaften: Mehrfachselektion

Allgemein

Name.....

Aktiviert.....

Sichtbar.....

Druckbar.....

Verankerung.....

PositionX.....

PositionY.....

Breite.....

Höhe.....

Schrift..... ...

Ausrichtung.....

Ausrichtung (vert.)....

Hintergrundfarbe..... ...

Rahmen.....

Umrandungsfarbe..... ...

Zusatzinformation.....

Hilfetext.....

Hilfe URL.....

Abbildung 28: Allgemeine Eigenschaften von Formularfeldern in der Mehrfachselektion

Den Namen sollte hier möglichst nicht geändert werden. Schließlich lauten so plötzlich alle Elemente gleich. Das Auffinden eines einzelnen Elementes über den Formularnavigator wird erschwert, die Verarbeitung des Formulars nach Namensbezeichnung der Kontrollfelder in Makros unmöglich.

Eher bietet sich eine Mehrfachselektion an, wenn insgesamt die Schriftart, die Höhe oder die Hintergrundfarbe gewechselt werden soll. Allerdings wirkt sich natürlich die Hintergrundfarbe auch auf die Beschriftungsfelder aus.

Sollen z. B. nur die Beschriftungsfelder geändert werden, so bietet es sich an, bei gedrückter Strg-Taste mit der linken Maustaste diese Felder direkt oder im Navigator anzuklicken, mit der rechten Maustaste über eins der Felder zu gehen und die Kontrollfeldeigenschaften aufzurufen. Jetzt wird die Auswahl der änderbaren Eigenschaften größer, da ja nur gleiche Felder ausgewählt wurden. Entsprechend kann hier auch all das geändert werden, was sonst in einem Beschriftungsfeld zur Verfügung steht.

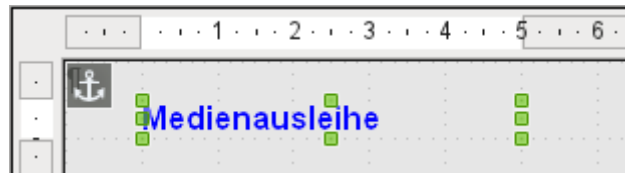
Die Möglichkeiten der Mehrfachselektion richten sich also nach der Auswahl der Felder. Kontrollfelder der gleichen Art können in allen allgemeinen Eigenschaften, die das einzelne Kontrollfeld bietet, auf diese Art gemeinsam angepasst werden.

Einfaches Formular komplett erstellt

Ein einfaches Formular stellt Formularkontrollfelder für das Schreiben oder Lesen von Datensätzen aus einer einzigen Tabelle oder Abfrage zur Verfügung. Seiner Konstruktion entspricht dem folgenden Schaubild:

Das Diagramm zeigt eine graue Rahmenfläche mit der Aufschrift 'Arbeitsfläche' in blauer Kursivschrift. Innerhalb dieser Fläche befindet sich ein hellblaues Rechteck mit der Aufschrift 'Formular' in brauner Schrift. Darunter sind vier übereinander gestapelte weiße Rechtecke mit schwarzen Rahmen, die jeweils den Text 'Kontrollfeld 1', 'Kontrollfeld 2', 'Kontrollfeld 3' und 'Kontrollfeld ...' enthalten.

Am Beispiel eines einfachen Formulars für die Bibliotheksausleihe sollen hier verschiedenen Varianten eines Formulars vorgestellt werden.



Die Überschrift für das Formular wurde über ein Beschriftungsfeld erzeugt. Die Schrift wurde geändert. Das Beschriftungsfeld ist am Absatz verankert, der sich in dem Dokument an der linken oberen Ecke befindet. Über das Kontextmenü des Beschriftungsfeldes wurde ein Formular erzeugt, das mit der Tabelle Ausleihe verbunden wurde (siehe: [Formulargründung über ein Formularfeld](#)). Die Seite wurde außerdem mit einem einfarbigen Hintergrund versehen.

Felder als Gruppe hinzufügen

Eine schnelle Variante, direkt Felder mit Beschriftungen einzufügen, bietet die Funktion *Feld hinzufügen*.



Über diese auf der Symbolleiste Formular-Entwurf (siehe [Abbildung 21](#)) erreichbaren Funktion lassen sich alle Felder der dem Formular zugrundeliegenden Tabelle auswählen



Über einen Doppelklick auf die Felder werden sie als Gruppierung zusammen mit einer Beschriftung (leider alle an die gleiche Stelle) in das Formular eingegliedert. Die Gruppen müssen also auf jeden Fall noch verschoben werden, damit anschließend das Formular im Einsatz wie das folgende Bild aussehen kann. Für die bessere Übersicht wurden alle unnötigen Leisten des Fensters entfernt und das Fenster entsprechend schmal zusammengeschoben, so dass nicht mehr alle Elemente der Navigationsleiste sichtbar sind.

Es wurden alle Felder bis auf "Medien_ID_BC" ausgewählt, da dies speziell für die Bedienung mit einem Barcodescanner gedacht ist.

Abbildung 29: Einfaches Formular über «Feld hinzufügen»

Für alle Tabellenfelder wurden die korrekten Formulkontrollfelder ausgesucht. Zahlen werden in numerische Felder gesetzt und gleich als Ganzzahlen ohne Nachkommastellen erkannt. Datumsfelder werden ebenfalls korrekt als Datumsfelder wiedergegeben. Alle Felder werden in gleicher Breite dargestellt. Würde ein Grafisches Kontrollfeld eingesetzt, so würde hier ein quadratisches Feld erzeugt.

Felder anpassen

Gestalterisch kann jetzt einiges getan werden, indem die Felder von der Länge her angepasst werden und die Datumsfelder aufklappbar gemacht werden. Wichtiger ist aber, dass die Felder für die "Medien_ID" und die "Leser_ID" für den Nutzer lösbar werden – es sei denn, jeder Bibliotheksbe-

sucher muss einen Ausweis mit der "ID" mitbringen und jedes Medium wird bei der Aufnahme mit der "ID" versehen. Hiervon wurde im Folgenden nicht ausgegangen.

Um einzelne Felder anzupassen, muss zuerst einmal die Gruppe betreten werden. Dies kann über einen rechten Mausklick auf die Gruppe und das entsprechende Kontextmenü erfolgen. Übersichtlicher für die späteren Verfahren ist allerdings die Arbeit mit dem Formularnavigator.

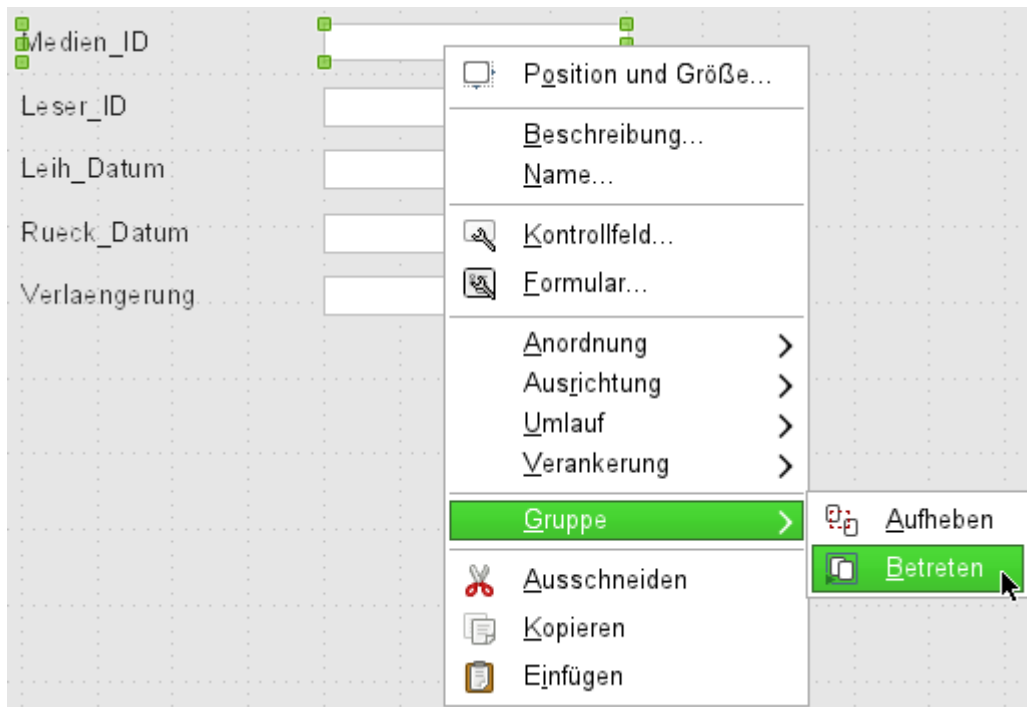


Abbildung 30: Formularkontrollelemente: Gruppe betreten



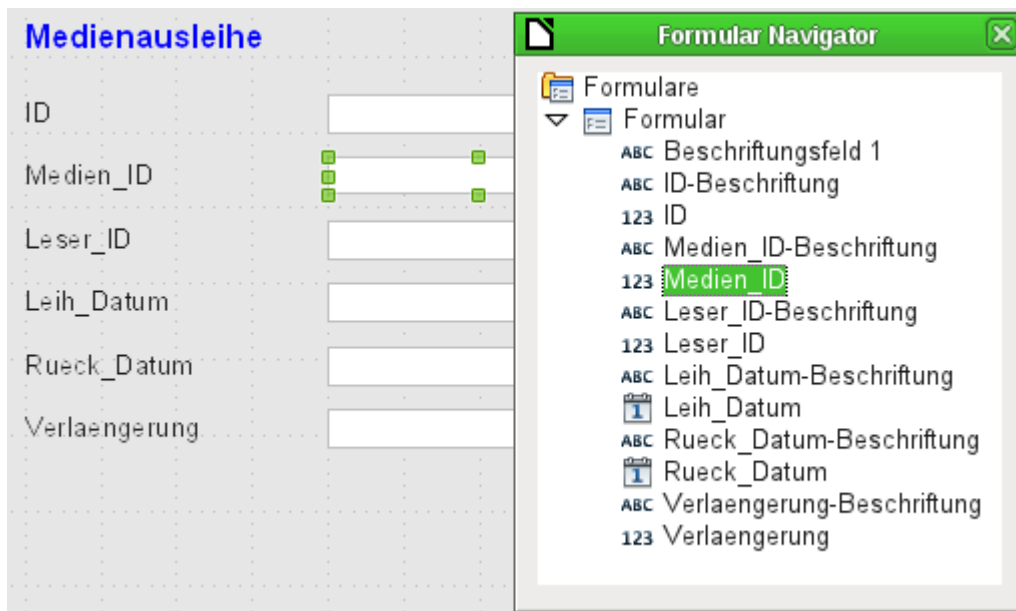


Abbildung 31: Formularelemente über den Formelnavigator direkt anwählen

Der Formelnavigator stellt alle Elemente des Formulars mit ihren Bezeichnungen dar. Für die Kontrollfelder wurden als Bezeichnungen direkt die Namen der Felder aus der dem Formular zugrundeliegenden Tabelle genommen. Die Beschriftungselemente haben den Zusatz «Beschriftung».

Durch einen Klick auf "Medien_ID" ist dieses Feld ausgewählt. Mit einem Rechtsklick wird es über das Kontextmenü möglich, das ausgewählte Feld durch eine andere Feldart zu ersetzen:

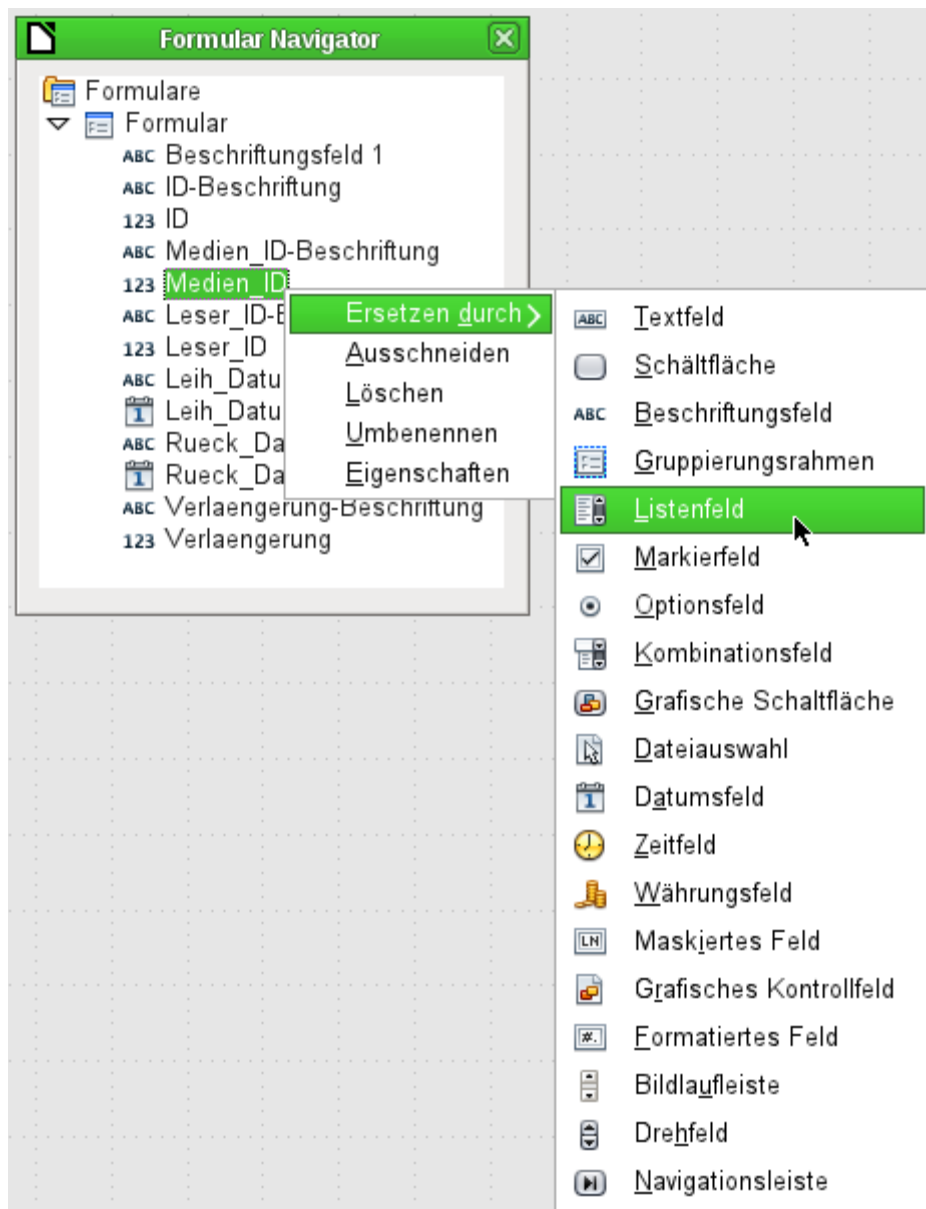


Abbildung 32: Kontrollelemente über den Formularnavigator durch andere Kontrollelemente ersetzen

Diese Ersetzung wird in dem Formular für "Medien_ID" und "Leser_ID" vorgenommen.

ABC Medien_ID-Beschriftung
 [List Field Icon] Medien_ID
 ABC Leser_ID-Beschriftung
 [List Field Icon] Leser_ID

Die Änderung ist im Formularnavigator über die Symbole sichtbar.



Die SQL-Abfragen für die Listenfelder können jetzt über den rechten Button mit der grafischen Benutzeroberfläche erstellt werden. Der Listenfeldassistent steht hier nicht zur Verfügung. Er springt nur automatisch ein, wenn ein Listenfeld direkt neu gegründet, nicht aber durch Umwandlung von einem anderen Feld aus erstellt wird. Zum SQL-Befehl siehe das Kapitel «[Abfragen für die Erstellung von Listenfeldern](#)».

Nachdem die Listenfelder als aufklappbar angepasst wurden, können noch die folgenden Mängel behoben werden:

- Die Beschriftung der Listenfelder sollte auf «Medien» statt «Medien_ID» und «Leser» statt «Leser-ID» geändert werden.
- Das Kontrollfeld «ID» sollte als nicht beschreibbar erklärt werden.
- Alle Felder, die nicht beim Aufruf der Ausleihe auf jeden Fall durchlaufen werden müssen, wenn ein neues Medium ausgeliehen wird, benötigen keinen Tabstop. Ohne den Tabstop geht es schneller durch das Formular. Eventuell muss der Tabstop auch über die Aktivierungsreihenfolge (siehe [Abbildung 21](#)) nachjustiert werden. Lediglich die Felder «Medien», «Leser» und «Leihdatum» müssen für eine Ausleihe auf jeden Fall über den Tabulator erreichbar sein.
- Soll über das Formular die Ausleihe erfolgen, so ist es eigentlich unnötig und auch unübersichtlich, bereits zurückgegebene Medien angezeigt zu bekommen. Medien mit einem Rückgabedatum sollten ausgefiltert werden. Außerdem könnte die Reihenfolge der Anzeige nach dem Leser sortiert werden, damit Medien, die von der gleichen Person entliehen wurden, nacheinander angezeigt werden. Siehe dazu die Hinweise unter [Formular-Eigenschaften](#). Speziell die Lesersortierung hat hier allerdings den Haken, dass lediglich nach der ID, nicht aber nach dem Alphabet sortiert werden kann, da die Tabelle für das Formular ja nur die ID enthält.

Felder einzeln hinzufügen

Das Hinzufügen einzelner Felder gestaltet sich zuerst einmal etwas aufwändiger. Die Felder müssen ausgewählt, auf der Formularfläche aufgezogen und dem Feld der dem Formular zugrundeliegenden Tabelle zugewiesen werden. Außerdem ist noch die Art des Feldes richtig einzustellen, da z. B. numerische Felder standardmäßig erst einmal 2 Nachkommastellen haben.

Lediglich beim Aufziehen der Listenfelder kommt der Assistent zum Einsatz, der die Schritte zur Erstellung eines korrekten Feldes für Ungeübte vereinfacht. Nach kurzer Anwendungsphase wird der Assistent allerdings den Ansprüchen nicht mehr gerecht, da er

- Die Einträge nicht automatisch sortiert.
- Eine Zusammenfassung von mehreren Feldern im aufzulistenden Inhalt nicht ermöglicht.

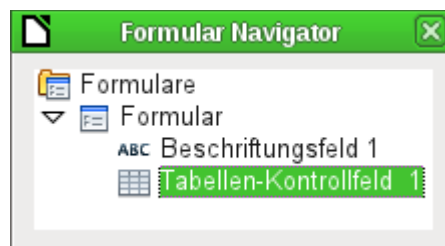
Hier muss dann immer wieder nachgebessert werden, so dass ganz schnell der SQL-Code direkt über den eingeblendeten Abfrageeditor erstellt wird.

Beim Hinzufügen einzelner Felder müsste eine Gruppierung von Feld und Beschriftung gesondert vorgenommen werden (siehe [Standardeinstellungen vieler Kontrollfelder](#)). In der Praxis kann sich aber die fehlende Verknüpfung auch positiv bemerkbar machen, da der Zugang zu den Eigenschaften eines Kontrollfeldes über das Kontextmenü sofort möglich ist und nicht erst ein Betreten der Gruppe erfordert.

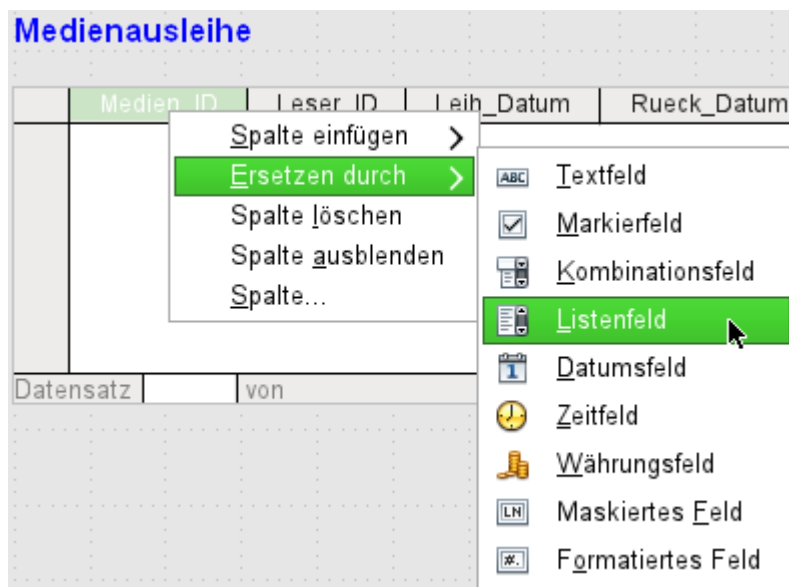
Tabellenkontrollfeld

Unter der Beschreibung zum [Tabellen-Kontrollfeld](#) wurde bereits über den Tabellenassistenten das entsprechende Tabellenkontrollfeld erstellt. Es hat allerdings noch einige Nachteile, die zu verbessern sind:

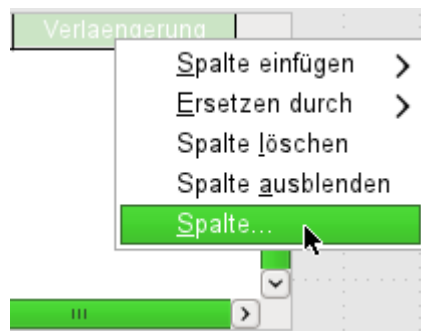
- Aus den Feldern «Medien_ID» und «Leser_ID» muss ein Listenfeld erstellt werden.
- Numerische Felder müssen gegebenenfalls auf Felder ohne Nachkommastellen umgestellt werden, da der Assistent hier grundsätzlich 2 Nachkommastellen belässt.



Die Änderung von Feldern innerhalb des Tabellen-Kontrollfeldes ist allerdings nicht auf die gleiche Art möglich, wie es oben für andere Kontrollfelder beschrieben wurde. Im Navigator endet die Beschreibung der Felder beim Tabellen-Kontrollfeld. Von den im Tabellen-Kontrollfeld liegenden Kontrollfeldern für die dem Formular zugrundeliegenden Tabelle weiß der Navigator nichts. Dies gilt in gleichem Maße übrigens später auch, wenn mittels Makros auf die Felder zugegriffen werden soll. Sie sind mit Namen nicht ansprechbar.



Die Kontrollfelder innerhalb des Tabellenkontrollfeldes werden als Spalten bezeichnet. Über das Kontextmenü ist es jetzt möglich, Felder durch andere Felder zu ersetzen. Allerdings steht nicht die ganze Palette an Feldern zur Verfügung. So fehlen Buttons, Optionsfelder oder das grafische Kontrollfeld.



Die Eigenschaften der Felder sind über das Kontextmenü unter dem Begriff Spalte verborgen. Hier kann dann z. B. das numerische Feld *Verlängerung* geändert werden, so dass keine Nachkommastellen mehr angezeigt werden. Auch der dort standardmäßig eingetragene minimale Wert von -1.000.000,00 macht wohl für Verlängerungen wenig Sinn. Die Anzahl dürfte wohl im positiven einstelligen Bereich bleiben.

Sobald die Eigenschaften einer Spalte aufgerufen sind, lässt sich ohne das Eigenschaftsfeld zu schließen auch eine andere Spalte aufrufen. Ohne eine separate Speicherung ist es so möglich, alle Felder nacheinander abzuarbeiten.

Die Speicherung erfolgt schließlich im gesamten Formular und letztlich mit dem Datenbankdokument selbst.

Die Eigenschaften dieser in das Tabellenkontrollfeld eingebauten Felder sind übrigens nicht ganz so umfangreich wie die der Felder außerhalb. Die Schrift z. B. lässt sich nur für das ganze Tabellenkontrollfeld einstellen. Außerdem gibt es hier nicht die Möglichkeit, einzelne Spalten ohne Tabulatorstop zu überspringen.

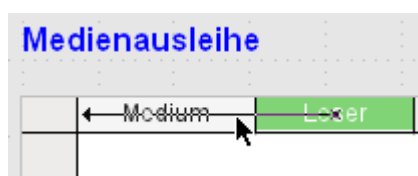
Tipp

Durch ein Formular erfolgt die Bewegung entweder mit der Maus oder mit dem Tabulator.

Gerät ein Tabulator einmal in ein Tabellenkontrollfeld, so bewegt sich der Cursor dort mit jeder Tabulatorbewegung ein Feld weiter nach rechts und bei Zeilenende schließlich zum ersten Feld des nächsten Datensatzes im Tabellenkontrollfeld.

Aus dem Tabellenkontrollfeld heraus geht es mittels **Strg + Tab**.

Die Anordnung der Spalten lässt sich über das Verschieben der Spaltenköpfe ändern:



Wird diese Verschiebung im Formularentwurf gemacht, so ist sie dauerhaft wirksam. Vorübergehend lässt sich so eine Verschiebung auch im Formular erledigen, wenn es zur Eingabe von Daten geöffnet ist.

Hinweis

Leider enthält diese Funktion zur Zeit einen Bug. Verschiebungen sind seit der Version LO 3.6 nicht mehr möglich. ([Bug 54021](#))

Zur Zeit können also nur Felder neu eingefügt und auf diese Weise auch positioniert werden.

Sollen nur bestimmte Felder zum Bearbeiten offen stehen, so bietet es sich an, das Formular gleich mit mehreren Tabellenkontrollfeldern zu bestücken, denn der Tabulator wird standardmäßig von einem Tabellenkontrollfeld gefangen.

In dem in [Abbildung 33](#) aufgezeigten Formular wird oben die Ausgabe von Medien geregelt. Hier sind nur die direkt notwendigen Felder.

Im unteren Tabellen-Kontrollfeld erscheinen noch einmal alle Felder, damit auch ersichtlich ist, für welche Person und welches Medium denn die Rückgabe erfolgt.

Medienausleihe

	Leser	Medium	Ausleihdatum
▶	(Lederstrumpf, Bert - Nr. 0	Das sogenannte Böse	02.11.11
	Gerd, Lisa - Nr. 2	Eine kurze Geschichte der Zeit	15.10.11
	Mirinda, Monika - Nr. 3	Der kleine Hobbit	02.11.11
	Lederstrumpf, Bert - Nr. 0	Traditionelle und kritische Theorie	04.11.11
	Lederstrumpf, Bert - Nr. 0	I hear you knocking	28.11.11
	Lederstrumpf, Bert - Nr. 0	Die neue deutsche Rechtschreibu	28.11.11
	Lederstrumpf, Bert - Nr. 0	Die neue deutsche Rechtschreibu	09.11.11
	Lederstrumpf, Bert - Nr. 0	Die neue deutsche Rechtschreibu	09.11.11

Datensatz 1 von 18

	Leser	Medium	Ausleihdatum	Verlängerung	Rückgabedatum
▶	(Lederstrumpf, Bert - Nr. 0	Das sogenannt	02.11.11		04.11.11
	Gerd, Lisa - Nr. 2	Eine kurze G	15.10.11	2	25.02.12
	Mirinda, Monika - Nr. 3	Der kleine H	02.11.11	1	04.04.12
	Lederstrumpf, Bert - Nr. 0	Traditionelle	04.11.11	2	28.11.11
	Lederstrumpf, Bert - Nr. 0	I hear you ki	28.11.11		
	Lederstrumpf, Bert - Nr. 0	Die neue de	28.11.11		04.04.12
	Lederstrumpf, Bert - Nr. 0	Die neue de	09.11.11		
	Lederstrumpf, Bert - Nr. 0	Die neue de	09.11.11		

Datensatz 1 von 18

Abbildung 33: Ein Formular - mehrere Tabellen-Kontrollfelder

Diese Abbildung zeigt allerdings noch einen Schönheitsfehler, der dringend behoben werden sollte. In dem oberen Tabellenkontrollfeld fällt auf, dass das gleiche Medium mehrmals auftaucht. Dies liegt daran, dass auch die Medien angezeigt werden, die eigentlich zurückgegeben wurden. Die Daten müssen für eine saubere Ausleihe also gefiltert werden. Daten mit «Rückgabedatum» brauchen gar nicht erst zu erscheinen.

Diese Filterung ist entweder über eine Abfrage oder direkt in den Formulareigenschaften möglich. Wird in den Formulareigenschaften eine Filterung erstellt, so kann diese auch bei der Eingabe in das Formular vorübergehend ausgeschaltet werden. Die Filterung mittels Abfrage wird in dem entsprechenden Kapitel «Abfragen» aufgegriffen. Hier wird jetzt der Weg über die Formulareigenschaften gezeigt:

The 'Formular-Eigenschaften' dialog box has three tabs: 'Allgemein', 'Daten', and 'Ereignisse'. The 'Daten' tab is active. It contains the following properties:

- Art des Inhaltes: Tabelle
- Inhalt: Ausleihe
- SQL-Befehl analysieren: Ja
- Filter: (empty text field) [three dots button]
- Sortierung: (empty text field) [three dots button]
- Daten hinzufügen: Ja
- Daten ändern: Ja
- Daten löschen: Ja
- Nur Daten hinzufügen: Nein
- Navigationsleiste: Ja
- Zyklus: Standard

Die Filterung wird über den Button mit den drei Punkten gestartet. Sie könnte allerdings auch direkt in das Textfeld eingegeben werden, wenn die SQL-Formulierung bekannt ist.

The 'Filter' dialog box has a table with the following columns: Verknüpfung, Feldname, Bedingung, and Wert. The first row is pre-filled with 'Rueck_Datum' in the 'Feldname' column and 'leer' in the 'Bedingung' column. There are three rows in total, each with a 'UND' connector in the 'Verknüpfung' column. Buttons for 'OK', 'Abbrechen', and 'Hilfe' are on the right.

Verknüpfung	Feldname	Bedingung	Wert
	Rueck_Datum	leer	
UND	- keiner -		
UND	- keiner -		

In der grafischen Benutzeroberfläche lässt sich jetzt das Feld mit dem Namen "Rueck_Datum" auswählen. Angezeigt werden sollen nur die Datensätze, bei denen diese Feld «leer» ist, wobei «leer» für die SQL-Bezeichnung **NULL** steht.

In dem **Formular-Eigenschaften** → **Daten** → **Filter** steht dann ("**Rueck_Datum**" **IS NULL**). Mit Hilfe des Filters können alle Formulierungen gewählt werden, die auch in Abfragen erstellt würden. Es muss lediglich darauf geachtet werden, dass der Filter in einfachen Klammern eingetragen wird. Siehe dazu auch [Filtern von Tabellen](#) und die Erweiterung mit der [WHERE SQL-Expression](#) aus dem Kapitel «Abfragen».

Das auf diese Art bereinigte Formular sieht dann schon etwas übersichtlicher aus:

Medienausleihe

	Leser	Medium	Ausleihdatum
▶	Lederstrumpf, Bert - Nr. 0	I hear you knocking	28.11.11
	Lederstrumpf, Bert - Nr. 0	Die neue deutsche Rechtschreibu	09.11.11
	Lederstrumpf, Bert - Nr. 0	Traditionelle und kritische Theorie	09.12.11
	Lederstrumpf, Bert - Nr. 0	Das Postfix-Buch	25.02.12
	Nobody, Terence - Nr. 9	Der kleine Hobbit	04.04.12
	Müller, Heinrich - Nr. 1	Eine kurze Geschichte der Zeit	04.04.12
	Lederstrumpf, Bert - Nr. 0	Das sogenannte Böse	04.04.12

Datensatz 1 von 8

	Leser	Medium	Ausleihdatum	Verlängerung	Rückgabedatum
▶	Lederstru	I hear you ki	28.11.11		
	Lederstru	Die neue de	09.11.11		
	Lederstru	Traditionelle	09.12.11		
	Lederstru	Das Postfix-	25.02.12		
	Nobody, T	Der kleine H	04.04.12		
	Müller, H	Eine kurze C	04.04.12	1	
	Lederstru	Das sogen	04.04.12		

Datensatz 1 von 8

Sicher ist dies noch verbesserbar, bietet aber neben der Funktionalität der anderen Formulare den unbestreitbaren Vorteil, dass alle Medien auf einen Blick sichtbar sind.

Die Bearbeitung von Daten mit Tabellenkontrollfeldern ist ähnlich der von Tabellen. Mit einem Rechtsklick auf den Datensatzmarkierer wird bei existierenden Datensätzen die Löschung des Datensatzes angeboten, bei Neueingaben kann die Dateneingabe rückgängig gemacht oder abgespeichert werden.

Wird eine Zeile verlassen, so wird der Datensatz automatisch abgespeichert.

Noch ist das Formular zur Medienausleihe in vielen Bereichen zu verbessern.

- Es wäre wünschenswert, wenn an einer Stelle der Leser ausgewählt würde und an anderer Stelle zu diesem Leser die entliehenen Medien erscheinen.
- In der oberen Tabelle sind lauter Datensätze zu sehen, die dort eigentlich gar nicht notwendig sind. Die Medien sind ja schon entliehen. Das obere Tabellenblatt ist aber erstellt worden, um die Ausleihe zu ermöglichen. Besser wäre es, wenn hier nur ein leeres Blatt erscheint, das dann mit den neuen Ausleihen ausgefüllt wird.

Solche Lösungen sind mit Hilfe von weiteren Formularen möglich, die hierarchisch aufeinander aufgebaut sind und eine getrennte Sicht auf die Daten ermöglichen.

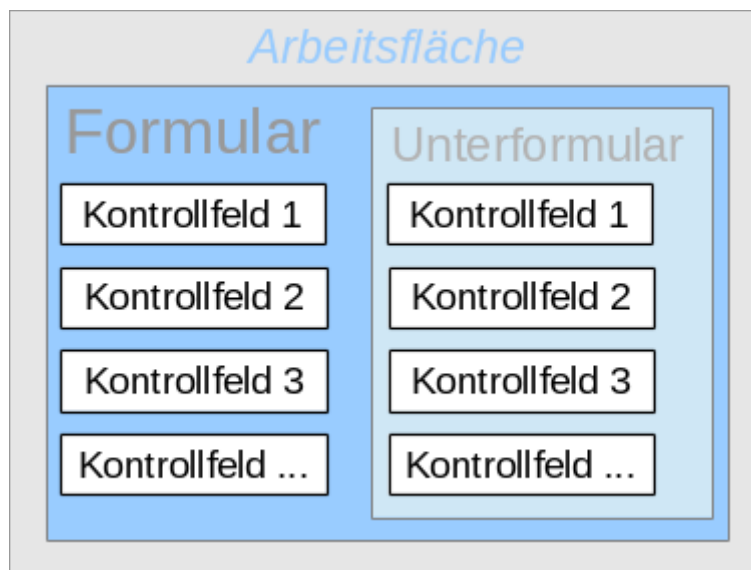
Tipp

Das Tabellenkontrollfeld kann mit anderen Feldern im Formular kombiniert werden. In dem Tabellenkontrollfeld werden z. B. nur die Medientitel angezeigt, in den darunter stehenden Formularfeldern dann an diesem Datensatz Änderungen vorgenommen.

Bei der Kombination von Tabellenkontrollfeld und weiteren Formularfeldern gibt es einen kleinen, leicht behebbaren Bug. Wenn beide Feldarten zusammen in einem Formular liegen, so läuft der Cursor von den anderen Formularfeldern automatisch in das Tabellenkontrollfeld, obwohl dieses Feld standardmäßig auf **Tabstop → Nein** eingestellt ist. Das kann behoben werden, indem der Tabstop einmal auf «Ja» und anschließend wieder auf «Nein» eingestellt wird. Dann erst wird «Nein» wirklich übernommen.

Hauptformular und Unterformular

Ein Unterformular liegt wie ein Formularkontrollfeld innerhalb eines Formulars. Wie ein Formularkontrollfeld wird es mit den Daten des (Haupt)-Formulars verbunden. Allerdings kann es als Datenquelle eine andere Tabelle oder eine Abfrage (bzw. einen SQL-Befehl) beinhalten. Für ein Unterformular ist nur wichtig, dass seine Datenquelle irgendwie mit der Datenquelle des Hauptformulars verbunden werden kann.



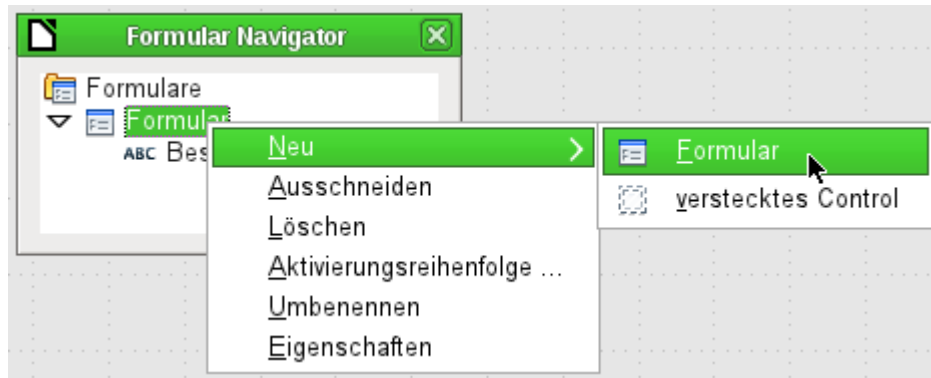
Typische Tabellenkonstruktionen, die sich für Unterformulare eignen, sind Tabellen mit einer *Eins-zu-Viele Beziehung*. Hauptformulare zeigen eine Tabelle an, zu deren Datensatz dann in den Unterformularen viele abhängige Datensätze aufgezeigt werden.

Wir nutzen jetzt erst einmal die Beziehung der Tabelle "Leser" zur Tabelle "Ausleihe" (siehe «*Tabellen Ausleihe*»). Die Tabelle "Leser" wird Grundlage für das Hauptformular, die Tabelle "Ausleihe" wird in dem Unterformular wiedergegeben.

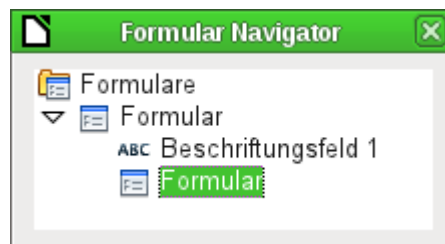
Das Bild zeigt das Dialogfeld 'Formular-Eigenschaften' mit der 'Daten'-Karte ausgewählt. Es enthält folgende Einstellungen:

Eigenschaft	Wert
Art des Inhaltes	Tabelle
Inhalt	Leser
SQL-Befehl analysieren	Ja
Filter	
Sortierung	"Nachname" ASC, "Vorname"
Daten hinzufügen	Ja
Daten ändern	Ja
Daten löschen	Ja
Nur Daten hinzufügen	Nein
Navigationsleiste	Nein
Zyklus	Standard

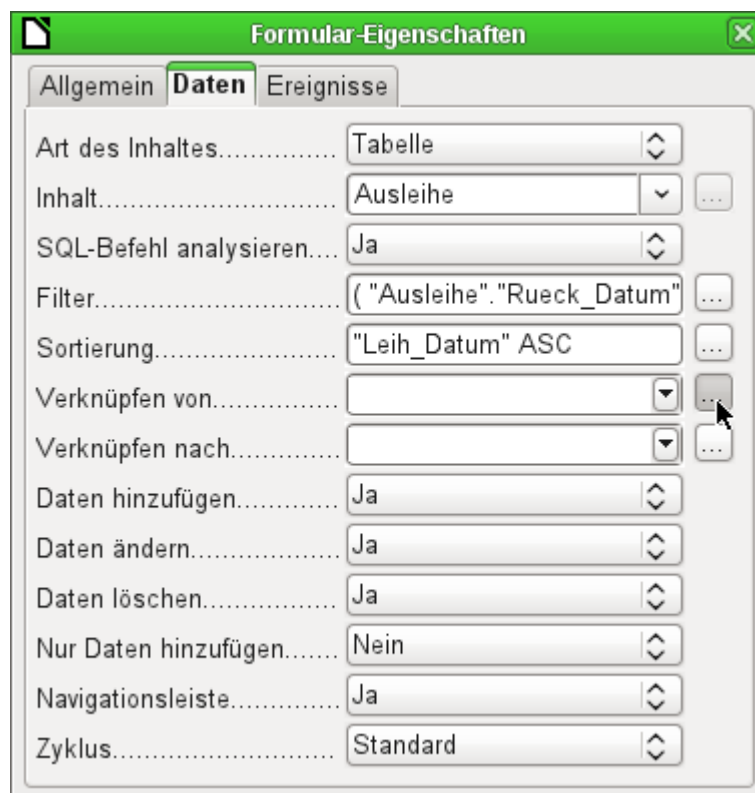
Das Hauptformular wird auf die Tabelle "Leser" eingestellt. Damit die Leser auch schnell genug gefunden werden, wird die Tabelle sortiert wiedergegeben. Auf eine Navigationsleiste wird verzichtet, weil zwischen Hauptformular und Navigationsleiste der Inhalt des Unterformulars angezeigt wird. Stattdessen soll das Formularkontrollelement *Navigationsleiste* eingebaut werden.



Durch Rechtsklick auf das Hauptformular im Formular-Navigator wird über das Kontextmenü ein neues Formular gegründet. Das Formular hat standardmäßig wieder den Namen Formular, ist jetzt aber ein Element im Unterverzeichnis des Hauptformulars.



Die Eigenschaften des Unterformulars müssen jetzt entsprechend eingestellt werden, damit es die richtige Datenquelle und die zu dem entsprechenden Leser gehörigen Daten wiedergibt.



Für das Unterformular wird die Tabelle Ausleihe gewählt. Beim Filter wird eingestellt, dass das Rückgabedatum leer sein soll ("**Rueck_Datum**" **IS NULL**). Dadurch werden keine bereits zurückgegebenen Medien angezeigt. Die Datensätze sollen nach dem Entleihdatum vorsortiert werden. Diese Sortierung zeigt das am längsten entliehene Medium am weitesten oben an.

Über **Verknüpfen von** und **Verknüpfen nach** wird eine Verbindung zum Hauptformular hergestellt, in dem das Unterformular liegt. Der Button mit den drei Punkten zeigt wieder an, dass es hier ein helfendes Fenster für die Auswahl der Einstellungen gibt.

Unter «Ausleihe» werden die Felder der Tabelle "Ausleihe" angezeigt, unter «Leser» die der Tabelle "Leser". Die "Leser_ID" der "Ausleihe" soll gleichbedeutend sein mit der "ID" der Tabelle "Leser".

Obwohl diese Verknüpfung bereits in der Datenbank unter **Extras → Beziehungen** erstellt wurde (siehe [Verknüpfung von Tabellen](#)) greift die hinter dem Button Vorschlagen liegende Funktion hierauf nicht zurück und möchte stattdessen den ersten Fremdschlüssel aus der Tabelle "Ausleihe", nämlich "Medien_ID", mit "ID" aus der Tabelle "Leser" verbinden. Dies gelingt dem Assistenten zur Erstellung von Formularen besser.

Die ausgewählte Verknüpfung von der Tabelle des Unterformulars nach der Tabelle des Hauptformulars wird jetzt mit den entsprechenden Feldern der Tabellen angegeben.

Tipp

Der Dialog **Felder verknüpfen** zeigt die Möglichkeit an, 4 Felder zu verknüpfen. Es ist aber auch problemlos möglich, mehr Felder in die Verknüpfung mit einzubeziehen:

Am einfachsten ist es, die Bezeichnungen in ein aufgeklapptes Listenfeld zu schreiben. Der Zeilenumbruch wird dabei durch **Shift + Enter** erreicht.

Um jetzt ein Tabellenkontrollfeld für das Hauptformular zu erstellen, muss das Hauptformular im Formular-Navigator markiert sein. Dann zeigt bei eingeschaltetem Tabellenkontrollfeld-Assistenten der Assistent die Felder an, die im Hauptformular zur Verfügung stehen. Entsprechend wird mit dem Unterformular verfahren.

Nachdem so die Tabellenkontrollfelder aufgezogen wurden, werden die entsprechenden Änderungen durchgeführt, die schon beim einfachen Formular erklärt wurden:

- Ersetzen des numerischen Feldes «Medien_ID» im Unterformular durch ein Listenfeld.
- Umbenennung des Feldes «Medien_ID» in Medien.
- Anpassung der numerischen Felder an ein Format ohne Nachkommastellen.
- Eingrenzung der minimalen und maximalen Werte.
- Umbenennung anderer Felder, um Platz zu sparen oder Umlaute darzustellen, die bei der Feldbenennung in den Datenbanktabellen vermieden wurden.

Sortier- und Filterfunktion werden für das Hauptformular ergänzt, indem eine Navigationsleiste hinzugefügt wird. Die anderen Felder der Navigationsleiste werden nicht benötigt, da sie vom Tabellenkontrollfeld weitgehend zur Verfügung gestellt werden. (Datensatzanzeige, Datensatznavigation) bzw. durch die Bewegung im Tabellenkontrollfeld erledigt werden (Speicherung von Daten).

Das erstellte Formular könnte schließlich wie in der folgenden Abbildung aussehen:

Medienausleihe

ID	Nachname	Vorname	Sperre	Geschlecht
4	Keindurchblick	Hein	☐	männlich
0	Lederstrumpf	Bert	☐	männlich
3	Mirinda	Monika	☐	weiblich
1	Müller	Heinrich	☐	männlich
7	Müßiggang	Kerstin	☐	weiblich

Datensatz 7 von 10 (1)

Ausgeliene Medien des ausgewählten Lesers

Medien	Ausleihdatum	Rückgabedatum	Verlängerung
Eine kurze Geschichte der Zeit - Nr. 8	04.04.12		1
Im Augenblick - Nr. 8	22.04.12		

Datensatz 1 von 2

Abbildung 34: Formular, bestehend aus Hauptformular (oben) und Unterformular (unten).

Wird jetzt im Hauptformular ein Leser ausgewählt, so werden im Unterformular nur die Medien angezeigt, die der Leser zur Zeit entliehen hat. Wird ein Medium zurückgegeben, so erscheint dies noch so lange im Formular, bis das Formular selbst aktualisiert wird. Dies geschieht automatisch, wenn im Hauptformular ein anderer Datensatz gewählt wurde. Bei der erneuten Anwahl des ursprünglichen Lesers sind also die zurückgegebenen Medien nicht mehr in der Anzeige.

Diese verzögerte Aktualisierung ist in diesem Fall wohl auch erwünscht, da so direkt eingesehen werden kann, welche Medien denn jetzt gerade auf der Theke der Mediothek liegen und ob diese schon registriert wurden.

Diese Formularkonstruktion bietet schon deutlich mehr Komfort als die vorherige mit nur einem einzigen Formular. Allerdings gibt es noch Details, die verbesserungswürdig erscheinen:

- Medien und Ausleihdaten können geändert werden, wenn die Medien schon länger entliehen sind.
Eine Änderung der Medien-Daten führt dazu, dass nicht mehr nachvollzogen werden kann, welches Medium denn nun noch in der Mediothek vorhanden ist und welches entliehen wurde.
Eine Änderung des Ausleihdatums kann zu fehlerhaften bzw. nicht beweisbaren Annahmen führen.
- Wird ein Leser nicht durch Klick auf den Datensatzmarkierer markiert, so zeigt nur der kleine grüne Pfeil auf dem Markierer an, welcher Datensatz gerade aktiv ist. Es ist auch möglich, den aktiven Datensatz komplett aus dem Tabellenkontrollfenster zu scrollen. Statt des Textes «Ausgeliehene Medien des ausgewählten Lesers» würde hier besser auch der Name erwähnt.
- Es ist möglich, mehrmals das gleiche Medium auszuleihen, ohne dass es zurückgegeben wurde.
- Es ist möglich, die Datensätze für ausgeliehene Medien einfach zu löschen.
- Auch im Hauptformular sind Änderung und Löschung von Daten möglich. Dies kann bei kleinen Mediotheken mit wenig Publikumsbetrieb sinnvoll sein. Sobald aber am Ausgabeschalter größere Hektik entsteht, ist die Bearbeitung von Nutzerdaten nicht an der gleichen Stelle vorzunehmen wie die Ausleihe.
Eine Vereinfachung wäre schon, wenn eine Neuaufnahme ermöglicht würde, alte Daten aber nicht angerührt werden dürfen. Denn ob nun Löschung oder komplette Veränderung des Namens – das Ergebnis bleibt für die Mediothek das Gleiche.

Zuerst wird einmal die Auswahl der Leser verbessert. Dies soll vor Änderungen in der Ausleihe schützen. Eine einfache Lösung wäre, keine Änderungen zuzulassen, aber neue Datensätze eingeben zu können. Dazu wird immer noch eine Suchfunktion benötigt, wenn ein Leser ein Medium entleihen will. Besser wäre, in einem Listenfeld die Leser auszusuchen und in voneinander getrennten Tabellenkontrollfeldern die Ausgabe und die Rückgabe zu erledigen.

Für das Hauptformular benötigen wir eine Tabelle, in die das Listenfeld seinen mit dieser Tabelle verbundenen Wert schreiben kann. Die Tabelle muss also ein Integer-Feld und einen Primärschlüssel haben. Sie wird beständig nur einen Datensatz enthalten; daher kann das Feld ID als Primärschlüssel ruhig 'Tiny Integer' sein. Die folgende Tabelle mit der Bezeichnung "Filter" soll also gebildet werden:

Tabellenname: Filter	
Feldname	Feldtyp
ID	Tiny Integer, Primärschlüssel
Integer	Integer

Die Tabelle wird mit einem Primärschlüsselwert gefüllt, und zwar dem Wert 0. Diesen Datensatz wird das Hauptformular beständig lesen und neu schreiben.

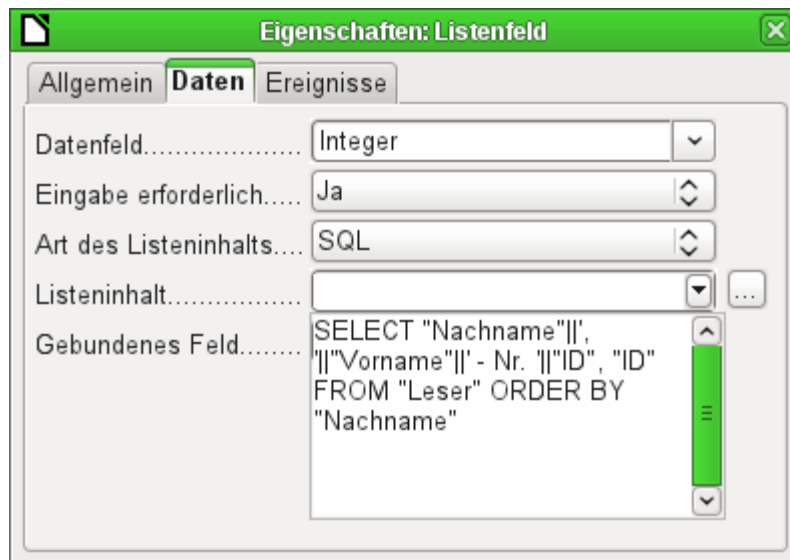
Formular-Eigenschaften	
Allgemein Daten Ereignisse	
Art des Inhaltes.....	Tabelle
Inhalt.....	Filter
SQL-Befehl analysieren....	Ja
Filter.....	("Filter"."ID" = 0)
Sortierung.....	
Daten hinzufügen.....	Nein
Daten ändern.....	Ja
Daten löschen.....	Nein
Nur Daten hinzufügen.....	Nein
Navigationsleiste.....	Nein
Zyklus.....	Standard

Das Hauptformular beruht auf der Tabelle "Filter". Es wird nur der Wert aus der Tabelle gelesen, bei dem der Primärschlüssel "ID" '0' ist. Es sollen keine Daten hinzugefügt werden, sondern nur der aktuelle Datensatz beständig neu geschrieben werden. Daher ist hier nur das Ändern erlaubt, eine Navigationsleiste sowieso überflüssig.

Verknüpfen von.....	Integer
Verknüpfen nach.....	Leser_ID

Das Hauptformular wird mit dem Unterformular so verknüpft, dass der Wert aus dem Feld "Integer" der Tabelle "Filter" gleich dem Wert aus dem Feld "Leser_ID" aus der Tabelle "Ausleihe" ist. Das Unterformular bleibt in seinen Eigenschaften ansonsten gegenüber der Vorversion unberührt.

Bevor jetzt im Hauptformular ein Listenfeld aufgezo-gen wird, wird erst einmal der Assistent ausgeschaltet. Mit dem Assistenten könnte nur ein Feld erzeugt werden, das lediglich einen Feldinhalt anzeigt; es wäre unmöglich, Nachname und Vorname und zusätzlich noch eine Nummer in dem Anzeigefeld des Listenfeldes zu positionieren. Wie bereits bei dem einfachen Formular wird jetzt das Listenfeld mit *Nachname, Vorname – Nr. ID* bestückt. Das Listenfeld gibt außerdem die ID an die darunterliegende Tabelle weiter.



Neben dem Listenfeld wird ein Button erstellt. Dieser Button ist allerdings Bestandteil des Unterformulars. Er soll gleichzeitig zwei Funktionen übernehmen: Abspeicherung des Datensatzes aus dem Hauptformular und Aktualisierung der Tabelle im Unterformular. Dazu reicht es, dem Button im Unterformular die Aktualisierung zuzuweisen. Der Speichervorgang für das veränderte Hauptformular wird dadurch automatisch ausgelöst.

Der Button kann einfach mit 'OK' als Aufschrift versehen werden. Als Aktion wird **Formular aktualisieren** zugewiesen.

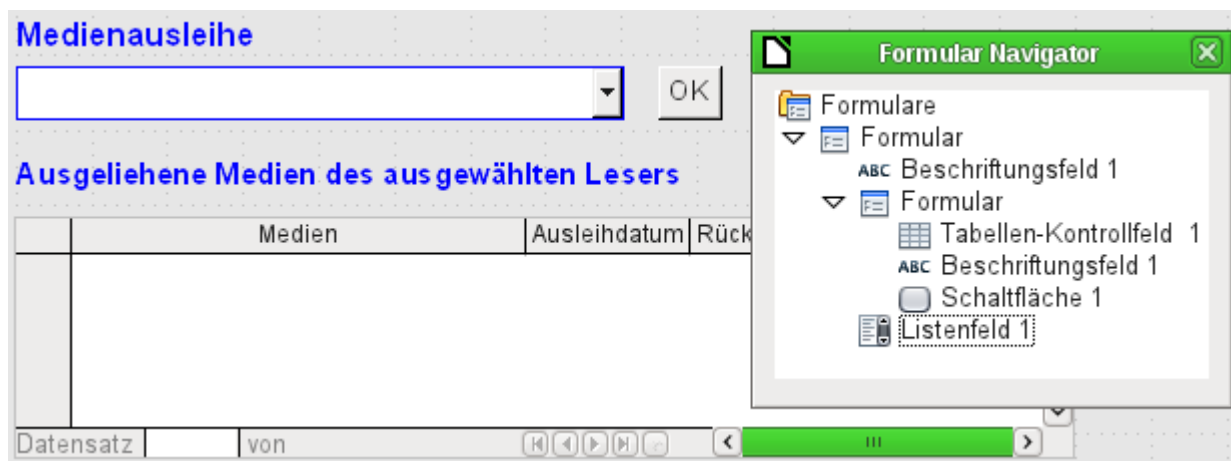


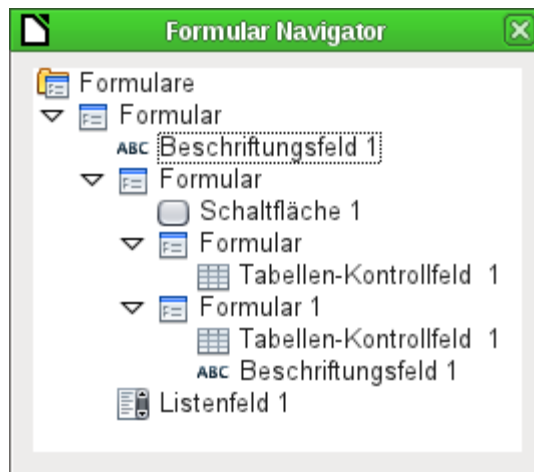
Abbildung 35: Hauptformular als Filter für ein Unterformular

Das Hauptformular besteht nur aus der Überschrift und dem Listenfeld, das Unterformular aus einer weiteren Überschrift, dem Tabellen-Kontrollfeld der vorherigen Version und dem Button.

Das Formular funktioniert jetzt schon insofern besser, als

- keine Leser mehr bearbeitet, vor allem verändert und gelöscht werden können und
- Leser schneller über das Eintippen in das Kontrollfeld gefunden werden als über Filter.

Für eine weitreichendere Funktionalität (Rückgabe ohne Änderung der vorherigen Daten) muss ein zweites Unterformular gegründet werden, das sich auf die gleiche Tabelle "Ausleihe" bezieht. Damit dennoch die Funktionalität des Listenfeldes aus [Abbildung 35](#) gewährleistet wird, müssen beide Unterformulare allerdings noch eine Ebene tiefer gelegt werden, also Unterformulare eines Unterformulars werden. Eine Aktualisierung von Daten verläuft in der Hierarchie nur vom Hauptformular zum Unterformular abwärts. Der Button im letzten vorgestellten Formular würde nur ein Unterformular aktualisieren, nicht aber das zweite, daneben liegende Unterformular.



Der Formularnavigator zeigt hier die verschiedenen Ebenen an. Im Hauptformular befindet sich das Beschriftungsfeld für die Formularüberschrift und das Listenfeld, in dem die Leser ausgesucht werden. Das Listenfeld steht in der Ansicht ganz unten, da es nach dem Unterformular gegründet wurde. Diese Reihenfolge der Anzeige lässt sich leider nicht beeinflussen. Das Unterformular hat lediglich eine Schaltfläche, mit der sein Inhalt aktualisiert und der des Hauptformulars gleichzeitig abgespeichert wird. Noch eine Ebene tiefer liegen dann zwei Unter-Unterformulare. Diese werden bei der Gründung bereits unterschiedlich benannt, so dass in keiner Ebene von der Benennung her Verwechslungen auftreten können.

Hinweis

Grundsätzlich sind die Benennungen der Formulare und Kontrollfelder erst einmal ohne Bedeutung. Wenn sie aber über den Namen durch Makros angesprochen werden sollen, müssen sie unterscheidbar sein. Gleiche Namen in der gleichen Formularebene erlauben keine Unterscheidung.

Natürlich ist es sinnvoll, bei größeren Formularkonstruktionen aussagekräftigere Namen für die Formulare und ihre Kontrollfelder zu nutzen. Ansonsten dürfte ein Auffinden des richtigen Feldes schnell zum Problem werden.

Formular-Eigenschaften	
Allgemein Daten Ereignisse	
Art des Inhaltes.....	Tabelle
Inhalt.....	Filter
SQL-Befehl analysieren....	Ja
Filter.....	
Sortierung.....	
Verknüpfen von.....	"Integer"
Verknüpfen nach.....	"Integer"
Daten hinzufügen.....	Nein
Daten ändern.....	Nein
Daten löschen.....	Nein
Nur Daten hinzufügen.....	Nein
Navigationsleiste.....	Nein
Zyklus.....	Standard

Das Hauptformular und das Unterformular nutzen einfach die gleiche Tabelle. Im Unterformular werden keine Daten eingegeben. Deshalb stehen alle diesbezüglichen Felder auf *Nein*. Verknüpft werden Hauptformular und Unterformular durch das Feld, dessen Wert auch an die Unter-Unterformulare weitergegeben werden soll: das Feld "Integer" der Tabelle "Filter".

Filter..... ("Ausleihe"."Leih_Datum" IS NULL)

Im ersten Unter-Unterformular werden keine alten Daten angezeigt, sondern nur neue Daten verarbeitet. Hierzu reicht der Filter, der gesetzt wurde. Es werden nur Daten angezeigt, die zu der "Leser_ID" passen und deren Leihdatum leer ist ("**Leih_Datum**" **IS NULL**). Das bedeutet beim Aufruf ein leeres Tabellen-Kontrollfeld. Da das Tabellen-Kontrollfeld zwischendurch nicht laufend aktualisiert wird, bleiben die gerade neu ausgeliehenen Medien so lange in dem Tabellen-Kontrollfeld stehen, bis über den Aktualisierungsbutton 'OK' entweder ein neuer Name ausgewählt oder auch nur die Übernahme der Daten in das zweite Unter-Unterformular veranlasst wird.

Formular-Eigenschaften	
Allgemein Daten Ereignisse	
Art des Inhaltes.....	Tabelle
Inhalt.....	Ausleihe
SQL-Befehl analysieren....	Ja
Filter.....	("Ausleihe"."Rueck_Datum"
Sortierung.....	"Leih_Datum" ASC
Verknüpfen von.....	"Integer"
Verknüpfen nach.....	"Leser_ID"
Daten hinzufügen.....	Nein
Daten ändern.....	Ja
Daten löschen.....	Nein
Nur Daten hinzufügen.....	Nein
Navigationsleiste.....	Nein
Zyklus.....	Standard

Das zweite Unter-Unterformular erfordert mehr Einstellungen. Auch dieses Formular enthält die Tabelle "Ausleihe". Hier werden aber die Daten gefiltert, bei denen das Rückgabedatum leer ist ("**Rueck_Dat**" **IS NULL**). Die Daten wird wie im vorhergehenden Formular so sortiert, dass die am längsten entliehenen Medien direkt sichtbar sind.

Wichtig sind jetzt auch die weiter unten stehenden Einträge. Alte Datensätze können geändert werden, aber es können keine neuen Datensätze hinzugefügt werden. Ein Löschen ist nicht möglich. Damit ist der erste Schritt gemacht, der notwendig ist, um später nicht Entleihdaten einfach zu löschen. Noch wäre es aber möglich, z. B. das Medium und das Entleihdatum zu ändern. Hier muss in den Eigenschaften der Spalten weiter justiert werden. Schließlich soll das Medium und das Entleihdatum nur angezeigt, aber von der Änderung ausgeschlossen werden.

Das Tabellen-Kontrollfeld wird nach der Erstellung der Formulare einfach verdoppelt. Dazu wird es markiert, anschließend kopiert, danach wird die Markierung aufgehoben und aus der Zwischenablage wieder eingeführt. Das Doppel befindet sich an der gleichen Position wie das Original, muss also noch verschoben werden. Danach können beide Tabellenkontrollfelder entsprechend bearbeitet werden. Das Tabellenkontrollfeld zur Medienrückgabe bleibt nahezu unverändert. Lediglich die Schreibrechte für die Spalten «Medien» und «Ausleihdatum» müssen geändert werden.

Ausgeliehene Medien des ausgewählten Lesers

Medien	Ausleihdatum	Rückgabedatum	Verlängerung

Datensatz von

Eigenschaften: Listenfeld

Allgemein
Daten
Ereignisse

Name..... NumericField1
Titel..... Medien
Aktiviert..... **Nein**
Nur lesen..... Ja
Mausradverhalten..... Nie
Breite..... 6,01cm
Listen-Einträge.....
Ausrichtung..... Standard
Anzahl der Zeilen..... 5
Standardselektion.....
Zusatzinformation....
Hilfetext.....
Hilfe URL.....

Während bei «Ausleihdatum» lediglich **Nur lesen** gewählt werden muss, ist dies bei Listenfeldern nicht ausreichend. Diese Einstellung verhindert nicht, dass das Listenfeld weiterhin betätigt werden kann. Wird aber **Aktiviert** → **Nein** gestellt, so kann dort eine Auswahl nicht mehr stattfinden. Im Tabellen-Kontrollfeld wird ein enthaltenes Listenfeld dann wie ein nicht veränderbares Textfeld angezeigt.

Im oberen Tabellen-Kontrollfeld werden alle Felder entfernt, die nichts mit der Ausleihe zu tun haben. Es bleibt lediglich das Medium als Auswahlfeld sowie das Ausleihdatum "Leih_Dat" stehen.

Wird schließlich noch die Abfrage für das Listenfeld im oberen Tabellen-Kontrollfeld entsprechend gewählt, so werden dort nur Medien angezeigt, die noch entliehen werden können. Mehr dazu im Kapitel «Abfragen».

Medienausleihe

Lederstrumpf, Bert - Nr. 0 OK

Medien	Ausleihdatum
4 - Die neue deutsche Rechtschreibung - von Hermann, Ursula	
5 - I hear you knocking - von Edmunds, Dave	
6 - Datenbanken mit OpenOffice.org 3 - von ?	

Datensatz 1 von 1 ⏮ ⏪ ⏩ ⏭

Ausgeliehene Medien des ausgewählten Lesers

Medien	Ausleihdatum	Rückgabedatum	Verlängerung
Traditionelle und kritische Theorie - Nr.	09.12.11		
Das Postfix-Buch - Nr. 7	25.02.12		
Das sogenannte Böse - Nr. 1	04.04.12		

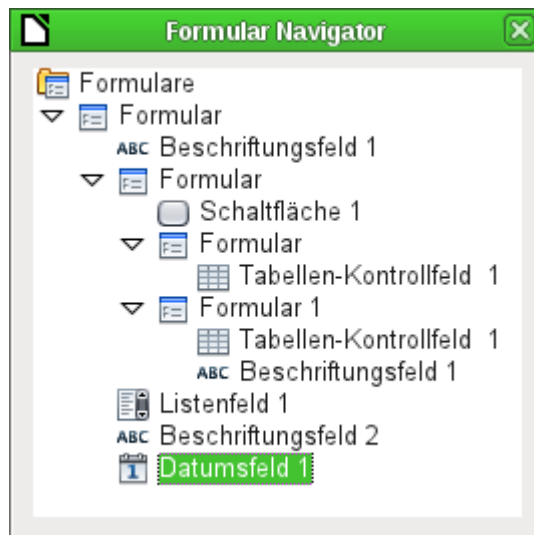
Datensatz 1 von 3 ⏮ ⏪ ⏩ ⏭

Abbildung 36: Das Auswahlfeld im oberen Unterformular zeigt nur Medien an, die nicht ausgeliehen sind.

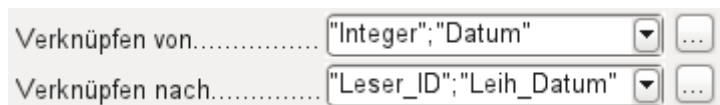
Das Formular zur Medienausleihe ist jetzt schon wesentlich besser zu bedienen. Kommt ein Leser an die Ausleihtheke, so wird der Name herausgesucht. Die zu entleihenden Medien können aus dem Listefeld gewählt und das Entleihdatum eingestellt werden. Mit dem Tabulator geht es dann zum nächsten Datensatz.

Eine letzte Verbesserung wäre noch wünschenswert: Das Entleihdatum muss jedes Mal gewählt werden. Stellen wir uns einen Tag in der Mediothek mit vielleicht 200 Entleihvorgängen vor, vielleicht auch nur eine Person, die gleich 10 Medien auf einmal entleiht. Das wären mehrmals hintereinander die gleichen Eingabevorgänge für ein Feld. Hier muss eine Einsparmöglichkeit her.

Unser Hauptformular beruht auf einer Tabelle "Filter". Das Hauptformular arbeitet dabei immer nur mit dem Datensatz, der als Primärschlüssel die "ID" '0' hat. In die Tabelle "Filter" können ohne weiteres noch mehr Felder eingebaut werden. Da noch kein Feld enthalten ist, das ein Datum speichern kann, gründen wir einfach ein neues Feld mit dem **Feldname** → **Datum** und dem **Feldtyp** → **Datum**. In der Tabelle "Filter" wird jetzt nicht nur die "Leser_ID" ("Filter"."Integer") sondern auch das "Leih_Datum" ("Filter"."Datum") gespeichert.



Im Hauptformular erscheint jetzt zusätzlich ein Datumsfeld, außerdem noch ein Beschriftungsfeld, das auf den Inhalt des Datumsfeldes hinweist. Der Wert aus dem Datumsfeld wird in der Tabelle "Filter" gespeichert und über die Verbindung vom Unterformular zum Unter-Unterformular weitergegeben.



Die Verknüpfung zwischen beiden Formularen weist jetzt zwei Felder auf. Das Feld "Integer" wird mit dem Feld "Leser_ID" des Unter-Unterformulars verbunden. Das Feld "Datum" mit dem Feld "Leih_Datum". Damit wird das "Leih_Datum" automatisch aus der Tabelle "Filter" bei der Ausleihe in die Tabelle "Ausleihe" übertragen.

Abbildung 37: Das Datum der Ausleihe wird einmal eingestellt. Bei einem Wechsel des Lesers muss es nicht neu eingegeben werden.

Aus dem Tabellenkontrollfeld wurde jetzt auch noch das Datumsfeld entfernt. Das Tabellenkontrollfeld besteht jetzt lediglich noch aus einem Auswahlfeld. Dies wäre die ideale Voraussetzung um bei einer Mediothek noch an der Beschleunigungsschraube zu drehen. Denn eigentlich haben die Medien ja eine Nummer. Wozu müssen sie also ausgesucht werden. Da könnte doch gleich die Nummer eingetragen werden. Oder, noch besser, die Medien könnten mit Barcode-Etiketten versorgt werden. Ein Scanner dafür ist mit ca. 50.- € mittlerweile recht preisgünstig zu haben. Dann würden die Medien schneller ausgeliehen als der Entleiher sie in die Tasche packen kann.

In der Beispieldatenbank ist dies entsprechend aufgezeigt. Zur Vorstellung des ersten Formularentwurfs sollte das obige Beispiel aber erst einmal ausreichend sein.

Da allerdings das letztlich in der Beispieldatenbank «Medien_ohne_Makros.odt» vorgesehene Formular noch weiter entwickelt wurde, sollen die Erweiterungen hier noch kurz vorgestellt werden.

Medien_ohne_Makros.odb : Ausleihe - LibreOffice Base: Database Form

Datei Bearbeiten Ansicht Einfügen Format Tabelle Extras Fenster Hilfe

Ausleihe

Vorname	Nachname
Bert	Lederstrumpf
Heinrich	Müller
Lisa	Gerd
Monika	Mirinda
Hein	Keindurchblick

Filter (Nachname)

Datensatz 2 von 10 (1)

Datensatz 2 von 10

Ausleihe für Leser(in) Müller, Heinrich

Entleihdatum: 22.04.12

aktuelle Ausleihe

Medium	Leih-Datum
8 - im Augenblick - von van Veen, Herman	22.04.12

Datensatz 1 von 1

Rückgabe

Medium	Leih-Datum	Rück-Datum	Verlängerung	Leihzeit	Restzeit
2 - Eine kurze Geschichte der Zeit - von Hawking, Steven W	04.04.12		1	18 Tage	3

Datensatz 1 von 1

Seite 1 / 1 | Standard | STD 75%

In die Ausleihe wurden die folgenden Eigenschaften aufgenommen:

- In einem Tabellenkontrollfeld werden die Leser und Leserinnen angezeigt. Hier können auch neue Leser und Leserinnen eingegeben werden.
- Über einen Filter, der mit der Tabelle "Filter" zusammenarbeitet, kann nach den Anfangsbuchstaben des Namens gefiltert werden. So werden bei einem «A» nur die Personen angezeigt, deren Nachname mit «A» beginnt. Die Filterung ist dabei unabhängig von der Eingabe von Groß- und Kleinschreibung.
- Im Untertitel wird noch einmal der Name der Person aufgezeigt, für die die Ausleihe erfolgen soll. Ist die Ausleihe für die Person gesperrt, so wird dies angezeigt.
- Das Entleihdatum ist auf das aktuelle Datum eingestellt. Dazu wurde die Filtertabelle über SQL so eingestellt, dass bei einem nicht eingegebenen Datum der Default-Wert das aktuelle Datum abspeichert.
- Die noch entlehbaren Medien werden in einem Listefeld ausgewählt. Über den Button 'Aktualisieren' wird die Ausleihe in das darunter stehende Tabellenkontrollfeld übertragen.

- Das mittlere Tabellenkontrollfeld dient lediglich der Anzeige der zu dem angegebenen aktuellen Datum ausgeliehenen Medien. Hier kann auch eine irrtümliche Ausleihe durch Löschen der Zeile rückgängig gemacht werden.
- Im unteren Tabellenkontrollfeld ist wie im vorher gezeigten Beispiel die Änderung von Medien aus Ausleihdatum nicht möglich. Auch eine Löschung ist nicht möglich.
- Neben der Eingabe des Rückgabedatums oder gegebenenfalls einer Verlängerung wird angezeigt, für wie viele Tage das Medium entliehen werden darf und wie viele Tage die restliche Entleihzeit beträgt.
- Geht die Restzeit in den negativen Bereich, so muss das Medium sofort zurückgegeben werden. Die Ausgabe ist deswegen gesperrt. Sie wird dadurch wieder ermöglicht, dass die Medien zurückgegeben werden. Nach der Medienrückgabe muss lediglich einmal auf 'Aktualisieren' gedrückt werden.

Dieses Formular ist mit Hilfe von Abfragen wesentlich komplexer strukturiert als die vorher vorgestellte Variante. Mehr zu den Grundlagen ist deshalb im Kapitel 'Abfragen' zu erfahren.

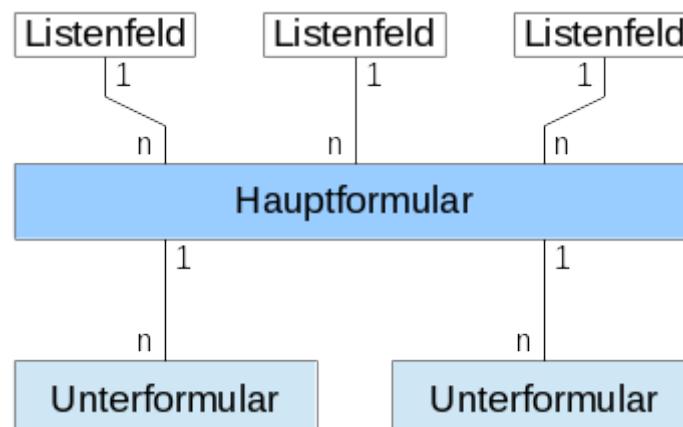
Eine Ansicht – viele Formulare

Während das Beispiel für die Ausleihe nur Eingaben in eine Tabelle erforderte (Tabelle "Ausleihe") und zusätzlich die Eingabe in der einfacheren Form auch für neue Leser ermöglichte, ist die Eingabe für die Medien wesentlich umfassender. Schließlich spielen rund um die Medien-Tabelle insgesamt 8 zusätzliche Tabellen mit (siehe «[Tabellen Medienaufnahme](#)»).

Durch die Zuordnung im Verhältnis n:1 bieten sich die Tabellen "Untertitel" und "rel_Medien_Verfasser" als Unterformulare zum Formular 'Medien' an. Tabellen, die hingegen ein Verhältnis von 1:n zur Tabelle "Medien" haben, müssten eigentlich ein Formular bilden, das über dem der Tabelle Medien liegt. Da es sich aber um mehrere entsprechende Tabellen handelt, werden deren Werte über Listenfelder in das Hauptformular eingetragen.

Die Tabelle eines Hauptformulars steht zur Tabelle eines Unterformulars grundsätzlich im Verhältnis 1:n, in seltenen Ausnahmen im Verhältnis 1:1. Das Hauptformular beherbergt in der Regel nach längerem Gebrauch der Datenbank also eine Tabelle, die deutlich weniger Datensätze hat als die Tabelle des Unterformulars.

Mehrere Hauptformulare können nicht auf ein Unterformular zugreifen. Es ist also nicht möglich, viele 1:n-Beziehungen gleichzeitig über die Formularanordnung zu lösen, bei denen das Unterformular den gleichen Inhalt hat. Gibt es eine 1:n-Beziehung für die Tabelle eines Formulars, so lässt sich dies über ein Listenfeld regeln. Hier stehen wenige Begriffe aus einer anderen Tabelle zur Auswahl, deren Fremdschlüssel auf diese Art in die Tabelle des Hauptformulars eingetragen werden.



Über Listenfelder werden dem Hauptformular, das auf der Tabelle "Medien" basiert, z. B. die Inhalte der Tabellen "Kategorie", "Ort" oder "Verlag" zugewiesen. Über Unterformulare sind die Tabelle "rel_Medien_Verfasser" und "Untertitel" mit dem Hauptformular und damit mit der Tabelle "Medien" verbunden.

Das Unterformular für die Tabelle "rel_Medien_Verfasser" besteht außerdem wieder aus zwei Listenfeldern, damit nicht die Fremdschlüssel der Tabelle "Verfasser" und "Verf_Zusatz" (Zusätze wie Hrsg., Gesang usw.) direkt als Ziffern eingegeben werden müssen.

Bei dem Formular für die Medieneingabe müssen die Listenfelder meist während der Eingabe nach und nach aufgefüllt werden. Hierzu werden neben dem Hauptformular weitere Formulare eingebaut. Sie existieren unabhängig vom Hauptformular:

Das Diagramm zeigt eine graue Rahmenbox mit der Aufschrift "Arbeitsfläche" in blauer Kursivschrift. Innerhalb dieser Box befinden sich zwei übereinander gestapelte, hellblaue Rechtecke, die als "Formular 1" und "Formular 2" beschriftet sind. Jedes Formular enthält eine vertikale Liste von drei weißen Eingabefeldern mit schwarzen Rahmen. Die ersten beiden Felder sind jeweils mit "Kontrollfeld 1" und "Kontrollfeld 2" beschriftet, das dritte Feld trägt die Beschriftung "Kontrollfeld ...".

Das gesamte Formular zur Medieneingabe sieht so aus:

Medieneingabe und Mediensuche

Suchbegriff

ID Titel

Kategorie Medienart

Ort Verlag E_Jahr Ausgabe Wert Bild 

Verfasser_Reihenfolge	Verfasser	Zusatz	ISBN/ISSN
1	van Veen, Herman		
Datensatz 1 von 1			

Nr	Untertitel	Dater
1	Amsterdam	
2	Hier unten am Deich	
3	Köln-Ehrenfeld	
4	Bei Mir	
5	Gott sei Dank	
Datensatz 1 von 5		

Anmerkung

Listfeldinhalte bearbeiten

Kategorie	Beschreibung
Fantasy	
Liedermacher	
Rock	
Datensatz 1 von 3	

Medienart	Anskizzezeit
Buch	14 Tage
CD	7 Tage
DVD	7 Tage
Datensatz 1 von 3	

ort	
Dresden	
Hamburg	
Nürnberg	
Pussemuckel	
Datensatz 1 von 4	

Verlag	
Datensatz 1 von 1	

Vorname	Nachname
Dave	Edmunds
Dr. Lutz	Götze
Steven W.	Hawking
Dr. Klaus	Heller
Datensatz 1 von 10	

Verf_Zusatz	
Geleitwort	
Hrsg.	
Illustration	
Überarbeit	
Datensatz 1 von 4	

Auf der linken Seite befindet sich das Hauptformular mit Blick auf die Suche und Eingabe von neuen Medien. Auf der rechten Seite des Formulars ist durch einen Gruppierungsrahmen mit der Bezeichnung «Listfeldinhalte bearbeiten» eine Bereich abgegrenzt, der zum Auffüllen der Listenfelder (hier verkürzt: «Listfeld») im Hauptformular gedacht ist. Existiert die Datenbank erst kurz, so wird hier wohl häufig eine Eingabe erledigt werden müssen. Je mehr Eingaben allerdings in den Listenfeldern des Hauptformulars zur Verfügung stehen, desto seltener ist ein Zugriff auf die Tabellenkontrollfelder aus dem Gruppierungsrahmen notwendig.

Die folgenden Tabellenkontrollfelder sind alle in einzelnen Nebenformularen zum Hautformular, dem Eingabeformular, untergebracht:

Listfeldinhalte bearbeiten

	Kategorie	Beschreibung	
▶	Fantasy		
	Liedermacher		
	Rock		
☼			

Datensatz 1 von 3

	Medienart	Ausleihzeit	
▶	Buch	14 Tage	
	CD	7 Tage	
	DVD	7 Tage	
☼			

Datensatz 1 von 3

	Ort	
▶	Dresden	
	Hamburg	
	Nürnberg	
	Pusemuckel	

Datensatz 1 von 4

	Verlag	
▶		

Datensatz 1 von 1

	Vorname	Nachname	
▶	Dave	Edmunds	
	Dr. Lutz	Götze	
	Steven W.	Hawking	
	Dr. Klaus	Heller	

Datensatz 1 von 10

	Verf_Zusatz	
▶	Geleitwort	
	Hrsg.	
	Illustration	
	überarbeitet	

Datensatz 1 von 4

Hier werden jeweils die kompletten Daten für eine Tabelle eingegeben. Am Anfang ist es häufig erforderlich, auf diese Nebenformulare auszuweichen, da z. B. nicht viele Verfasser in der entsprechenden Tabelle bereits abgespeichert wurden.

Wurde in einem der Tabellenkontrollfelder ein neuer Datensatz abgespeichert, so ist in dem Hauptformular das entsprechende Listfeld aufzusuchen und über **Kontrollfeld aktualisieren** (siehe «Navigationsleiste») neu einzulesen.

Der Formularnavigator zeigt entsprechend viele Formulare an:



Die Formulare sind einzeln benannt, so dass sie erkennbar bleiben. Lediglich das Hauptformular hat vom Formularassistenten noch die Bezeichnung «MainForm» behalten. Insgesamt existieren also 8 Formulare parallel. Das Formular «Filter» beherbergt eine Suchfunktion, das Formular «MainForm» die Haupteingabefläche. Alle anderen Formulare stehen für je eins der oben abgebildeten Tabellenkontrollfelder.

Ohne die Tabellenkontrollfelder erscheint das Hauptformular schon etwas übersichtlicher:

Medieneingabe und Mediensuche

Suchbegriff

ID Titel

Kategorie Medienart

Ort Verlag E_Jahr Ausgabe Wert

	Verfasser_Reihenfolge	Verfasser	Zusatz
☒	1	van Veen, Herman	
☐			
☐			

Datensatz 1 von 1

	Nr	Untertitel	Dater
☒	1	Amsterdam	
☐	2	Hier unten am Deich	
☐	3	Köln-Ehrenfeld	
☐	4	Bei Mir	
☐	5	Gott sei Dank	

Datensatz 1 von 5

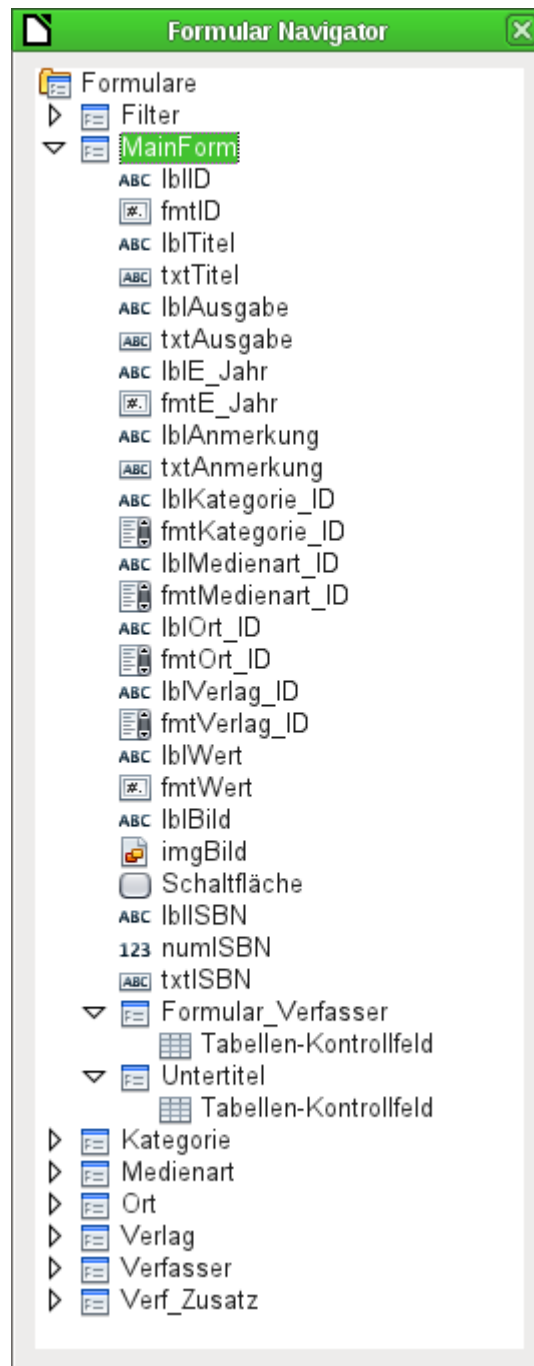
ISBN/ISSN

Bild

Anmerkung

Das Feld für den Suchbegriff liegt in dem *Nebenformular* «Filter», die beiden *Tabellenkontrollfelder* (für die Verfasser und für die Untertitel) liegen in *Unterformularen* zum Hauptformular der Medieneingabe.

Im Formularnavigator sieht das Formular selbst dann schon wesentlich unübersichtlicher aus, da natürlich alle Kontrollfelder, auch die Beschriftungen, dort auftauchen. In den vorhergehenden Formularen waren ja die meisten Felder als Spalten von Tabellen-Kontrollfeldern im Formularnavigator nicht zu sehen.



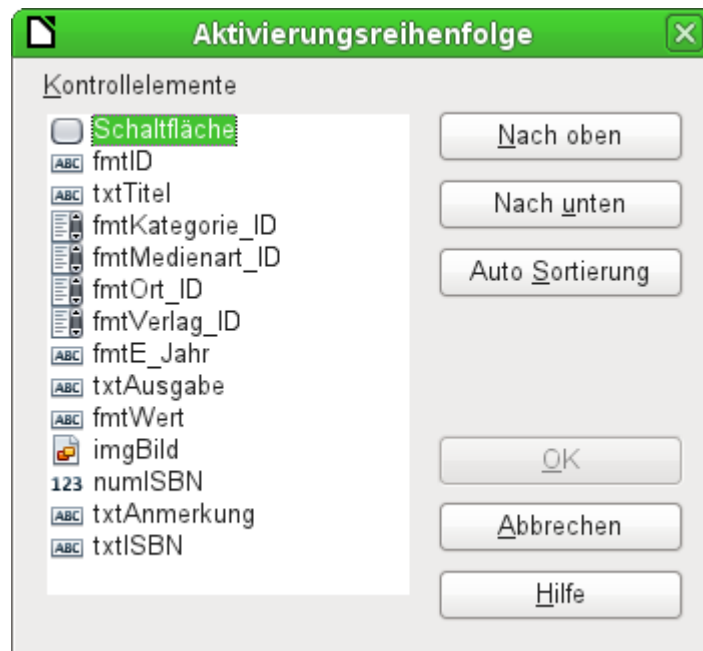
Die Reihenfolge innerhalb des Formularnavigators lässt sich leider nicht so einfach verändern. So erscheint es z. B. sinnvoll, die Unterformulare «Untertitel» und «Formular_Verfasser» als Verzweigung direkt zum Beginn des Formulars «MainForm» auftauchen zu lassen. Innerhalb des Formularnavigators werden die einzelnen Kontrollfelder und Unterformulare einfach in der Reihenfolge aufgelistet, in der sie gegründet wurden.

Durch den Formularassistenten werden die Elemente mit bestimmten Kürzeln versehen, die neben den Symbolen andeuten, um welche Art Feld es sich handelt. Mit 'lbl' beginnen Beschriftungsfelder ('label'), mit 'txt' Textfelder usw. Sämtliche Beschriftungsfelder interessieren für die Eingabe von Daten eigentlich nur als zusätzliche Informationen. Sie können auch direkt über Textrahmen erstellt werden und würden dann im Formularnavigator nicht erscheinen.

Die Reihenfolge, in der die Elemente im Navigator auftauchen, hat aber nichts damit zu tun, in welcher Reihenfolge die Elemente durch einen Tabulatorsprung erreicht werden. Dies wird durch die *Aktivierungsreihenfolge* bestimmt.



Die Aktivierungsreihenfolge für ein bestimmtes Formular wird aufgerufen, indem ein Element dieses Formulars markiert wird und dann erst der Button **Aktivierungsreihenfolge** betätigt wird. Bei einem einzigen Formular ist diese Reihenfolge natürlich nicht notwendig. Die Funktion muss aber bei vielen parallel liegenden Formularen erst einmal wissen, welches Formular denn nun ausgewählt werden soll. Standardmäßig ist es sonst das erste Formular im Formularnavigator – und das enthält im obigen Beispiel nur ein Textfeld.



Über die Aktivierungsreihenfolge werden alle Elemente, die Daten an die dem Formular zugrundeliegende Tabelle weitergeben oder Aktionen hervorrufen können, in ihrer Reihenfolge zueinander festgelegt. Dies entspricht der Einstellung in den Eigenschaften in [Standardeinstellungen vieler Kontrollfelder](#) zur Aktivierungsreihenfolge.

In der Aktivierungsreihenfolge tauchen allerdings auch die Elemente auf, bei denen der Tabstop eigentlich abgeschaltet ist. Sie werden zwar in die Nummerierung aufgenommen, aber tatsächlich bei der Arbeit mit dem Formular über die Tastatur nicht angesprungen.

Automatisch werden die Felder nach der Lage auf dem Formularhintergrund sortiert. Je weiter oben ein Feld liegt, desto eher wird es über die Auto Sortierung angesprungen. Je weiter links ein Feld bei gleicher Höhe liegt, desto eher wird es angesprungen. Diese Sortierung funktioniert dann tadellos, wenn die Elemente wirklich genau ausgerichtet sind (Raster bei der Formularerstellung). Ansonsten muss nachgebessert werden. Hierzu wird ein Element einfach markiert und z. B. über **Nach unten** weiter nach unten in der Reihenfolge verlegt.

Existiert ein Unterformular, so wird bei der **Auto Sortierung** das Ganze so eingestellt, dass nach dem Hauptformular direkt in das Unterformular gesprungen wird. Bei einem Tabellenkontrollfeld führt dies über die Tastatureingabe dann dazu, dass der Cursor in diesem Unterformular gefangen ist und nur mit der Maus oder über **Strg + Tab** aus dem Subformular wieder herausbewegt werden kann.

Die **Auto Sortierung** funktioniert allerdings, bezogen auf das Tabellenkontrollfeld, nur einmal. Ein weiteres Unterformular mit Tabellenkontrollfeld wird nicht mit einbezogen. Parallel liegende Formulare werden also nicht berücksichtigt. Eine Auto-Sortierung kann in Bezug auf ein Unterfor-

mular mit Tabellenkontrollfeld auch nicht rückgängig gemacht werden. Hierzu müsste das Unterformular komplett entfernt (oder vorübergehend in ein anderes Formular verschoben) werden.

The image shows a snippet of a web form. It has two rows. The first row is labeled 'Art des Inhaltes.....' and has a dropdown menu with 'Abfrage' selected. The second row is labeled 'Inhalt.....' and has a dropdown menu with 'Medien_Eingabe_Suchergebnis' selected. To the right of the second dropdown is a small button with three dots '...'.

Die Datengrundlage für das Formular zur Medieneingabe ist in diesem Fall nicht eine Tabelle, sondern eine Abfrage. Dies ist notwendig, da das Formular nicht nur zur Eingabe sondern auch zur Suche benutzt werden soll. Auch wird in dem Formular noch in einem Textfeld nach dem Abspeichern darüber aufgeklärt, ob die eingegebene ISBN-Nummer korrekt ist. Auch dies ist nur über eine umfassende Abfrage möglich. Um diese Hintergründe zu verstehen, ist es also notwendig, sich erst einmal mit den Grundlagen von Abfragen auseinander zu setzen.

Fehlermeldungen bei der Eingabe in Formulare

Einige Fehlermeldungen treten gerade bei den ersten Formularerstellungen gehäuft auf, so dass sie hier kurz erklärt werden sollen:

Attempt to insert null into a non-nullable column: column: ID table: Tabellenname in statement ...

Es gibt Felder, die nicht leer sein dürfen. Wird dies nur in der Tabelle definiert (**NOT NULL**), so erscheint eine englischsprachige Fehlermeldung. Erfolgt die Definition auch in den Kontrollfeldern des Formulars, dann erscheint eine deutschsprachige Meldung mit dem genauen Hinweis, welches Feld denn nun ausgefüllt werden muss.

Die obige Meldung passiert besonders häufig, wenn das Primärschlüsselfeld, hier '**ID**', nicht in das Formular mit aufgenommen wurde, da es ja als automatisch hoch zählendes AutoWert-Feld gedacht war. Leider wurde aber die entsprechende Definition als AutoWert-Feld schlicht vergessen. Also muss dies entweder nachgeholt werden oder das Feld muss im Formular mit erscheinen, damit ein Wert eingegeben werden kann.

Die nachträgliche Definition eines Feldes als AutoWert-Feld ist manchmal etwas schwierig, wenn die Tabelle mit anderen in der Beziehungsdefinition bereits verknüpft wurde oder wenn bereits mit einer Ansicht auf die Tabelle zugegriffen wird. Hier müssen alle Verbindungen gelöst werden, damit die Tabelle gerade im Bereich des Primärschlüssels wieder editierbar wird. Der Inhalt des SQL-Befehls, mit dem z. B. eine Ansicht erstellt wurde, kann ja gegebenenfalls als Abfrage zwischendurch abgespeichert werden.

Integrity constraint violation - no parent SYS_FK_95 table: Tabellenname in statement ...

Hier besteht eine Verknüpfung der dem Formular zugrundeliegenden Tabelle mit einer anderen Tabelle. Diese Tabelle soll ein Fremdschlüsselfeld mit seinem Primärschlüssel belegen. Dies geschieht in der Regel durch Listenfelder, die den Inhalt der Tabelle richtig abfragen. Ist aber gar kein Listinfeld vorhanden oder die Konstruktion des Listenfeldes falsch, so kann in das Fremdschlüsselfeld auch irrtümlich ein Wert eingegeben werden, der in der zweiten Tabelle gar nicht als Primärschlüssel vorhanden ist. Dies wird als '**Integritäts-Verletzung**' bezeichnet. '**no parent SYS_FK_95**' deutet darauf hin, dass in der zweiten Tabelle, der abgebenden '**Elterntabelle**', der entsprechende Indexwert mit dem Namen '**SYS_FK_95**' nicht vorhanden ist.

Fehler beim Einfügen des neuen Datensatzes. Fehler in der Funktionsfolge

Einem Unterformular werden Daten von den darüber liegenden Formularen weitergegeben. Dies sieht Base nicht als eine Änderung eines Feldes an. Die grafische Benutzeroberfläche bietet zwar das Speichern an – nur scheint der Datensatz dann leer zu sein. Hier hilft es, ein einfaches Feld wie z. B. ein Ja/Nein-Feld in die dem Formular zugrundeliegende Tabelle einzubauen. Jetzt wird

vor dem Abspeichern dieses Feld abgehakt und der Datensatz kann problemlos gespeichert werden.

Die über die anderen Formulare weitergegebenen Werte werden anscheinend erst dann in die entsprechenden Felder eingefügt, wenn wenigstens eine zusätzliche Aktion im Formular stattfindet.

Suchen und Filtern in Formularen über die Navigationsleiste

Die Navigationsleiste der Formulare bietet verschiedene Such- und Filtermöglichkeiten an. Die einzelnen Elemente der Navigationsleiste wurden bereits weiter oben aufgezählt.

Datensatzsuche mit Parametern

The screenshot shows a 'Medieneingabe' (Media Entry) form in the background and a 'Datensatz-Suche' (Record Search) dialog box in the foreground.

Medieneingabe Form:

- ID: 4
- Titel: Die neue deutsche Rechtschreibung
- Kategorie: (empty dropdown)
- Medienart: Buch
- Ort: (empty dropdown)
- Verlag: (empty dropdown)
- E_Jahr: 1996
- Ausgabe: (empty dropdown)
- Wert: 15,80 €
- Bild: (empty image field)
- Verfasser_Reihenfolge table:

Nr	Verfasser	Zusatz
1	Hermann, Ursula	
2	Götze, Dr. Lutz	überarbeitet
3	Heller, Dr. Klaus	Gelöscht
- ISBN/ISSN: 1500000003

Datensatz-Suche Dialog:

- Suchen nach:
 - ☒ Initialtext: 5
 - ☐ Feldinhalt ist NULL
 - ☐ Feldinhalt ist ungleich NULL
- Bereich:
 - Formular: Formular_Verfasser (MainForm)
 - ☒ Alle Felder
 - ☐ Einzelnes Feld: Verfasser_Reihenfolge
- Einstellungen:
 - Position: irgendwo im Feld
 - ☐ Feldformatierung benutzen
 - ☐ Rückwärts suchen
 - ☐ Platzhalter-Ausdruck
 - ☐ Groß-/Kleinschreibung
 - ☐ Von oben
 - ☐ Regulärer Ausdruck
 - ☐ Ähnlichkeitssuche
- Status: Datensatz: 2
- Buttons: Hilfe, Suchen, Schließen

Die einfache Datensatzsuche mit Parametern wurde bereits im Kapitel «Tabellen» vorgestellt. Sie bietet sehr umfangreiche Einstellmöglichkeiten. Charakteristik der Suche ist, dass grundsätzlich alle Datensätze durchsucht werden und nicht die Datensätze so eingeschränkt werden, dass nur Datensätze mit dem Suchwert zu sehen sind.

Die Datensatzsuche im Formular bietet die Möglichkeit, das entsprechende Formular sowie die Unterformulare zu durchsuchen. Es ist allerdings nicht möglich, alle Formulare gleichzeitig zu durchsuchen.

Das Bild zeigt den Zugriff auf das Unterformular «Formular_Verfasser» des Hauptformulars «MainForm». Dabei wird nach einem Listenfeld, nämlich dem Feld für den Verfasser, gesucht. Allerdings wird nicht der Text des Listenfeldes benötigt, sondern der Wert, der als Fremdschlüsselwert an dieser Stelle eingegeben wird.

Leider kennzeichnen diese Suche aber die folgenden Nachteile:

- Die Parametersuche ist für den Normalgebrauch viel zu umständlich zu handhaben.
- Die Suchfunktion selbst ist äußerst langsam, da nicht die Abfragefunktionen der Datenbank genutzt werden.
- Die Suche funktioniert nur innerhalb eines Formulars, dort dann aber auch gegebenenfalls für alle Felder. Unterformulare müssen separat angewählt werden.
- Es wird nur in dem Unterformular gesucht, das zu dem aktuellen Hauptformular passt. Einträge anderer Unterformulare lassen sich nicht ermitteln. So ist es z.B. nicht möglich, einen Untertitel eines anderen Mediums als des gerade angezeigten Mediums zu finden.
- Die Suche funktioniert bei Listenfeldern auf der Basis des Schlüsselfeldes (Fremdschlüssels), das in der Tabelle abgespeichert ist. Es lassen sich nicht die angezeigten Begriffe verwenden.

Filterung mit dem Autofilter

Der Autofilter kann direkt in der Navigationsleiste angewählt werden. Wird auf ein Feld im Hauptformular geklickt, so wird treffsicher nach diesem Feld gefiltert. Der Autofilter funktioniert auch sicher mit Listenfeldern, hat hier also einen klaren Vorteil gegenüber der händischen Eingabe von Fremdschlüsselwerten zu verbuchen.

Medieneingabe

ID: 8 Titel: Im Augenblick

Kategorie: Liedermacher Medienart: CD

Ort: Verlag: E_Jahr: 2009 Ausgabe: Wert: 20,00 € Bild: 

Verfasser_Reihenfolge	Verfasser	Zusatz
1	van Veen, Herman	

Datensatz: 1 von 1

Nr	Untertitel	Dauer
1	Amsterdam	
2	Hier unten am Deich	
3	Köln-Ehrenfeld	
4	Bei Mir	
5	Gott sei Dank	

Datensatz: 1 von 5

Anmerkung:

Datensatz: 3 von 3

In dem obigen Beispiel wurde einfach ein Datensatz gesucht, bei dem im Feld «Medienart» 'CD' steht. Anschließend wurde der Autofilter betätigt. Von den ursprünglich 9 Datensätzen der Medien-Tabelle werden jetzt noch 3 Datensätze in der Datensatzzählung angezeigt.

Anmerkung:

Nr	Untertitel	Dauer
3	Köln-Ehrenfeld	

Datensatz: 1 von 1

Datensatz: 9 von 9

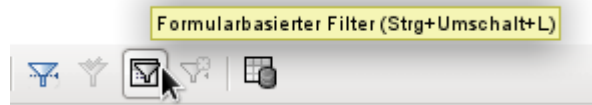
Obwohl nur in zwei Datensätzen im Unterformular eine «Nr.» '3' wie oben existiert, werden alle anderen Datensätze des Hauptformulars weiter angezeigt. Im Unterformular erscheinen hingegen nur die Felder mit «Nr.» '3'.

Der Autofilter kann die folgenden Probleme allerdings nicht lösen:

- Die Suche in Unterformularen führt dazu, dass zwar in den Unterformularen nur der entsprechende Wert angezeigt wird, die Formulare selbst aber weiter komplett angezeigt werden, auch wenn sie den entsprechenden Wert im Unterformular gar nicht verzeichnen können. Wird z.B. nach einem Medium mit einem zweiten Verfasser gesucht, so werden trotzdem auch alle Medien angezeigt, für die nur ein Verfasser verzeichnet ist. Hier bleibt das Feld für die Verfasser schlicht leer.
- Die Wahl des Filterinhaltes muss eindeutig sein. Es gibt nicht die Möglichkeit, über Ähnlichkeitssuche zu einem entsprechenden Wert zu kommen, da eben nur der Originalwert aus dem Formular verwandt wird.

Filterung mit dem formularbasierten Filter

Der formularbasierte Filter bietet statt eines Filtersymbols das Formular zur Eingabe von entsprechenden Filterwerten an. Hier kann gleichzeitig nach mehreren Werten gefiltert werden. Diese Werte können entweder zusammen als gemeinsame Bedingung formuliert werden (UND) oder als unterschiedliche Bedingungen gesetzt werden (ODER).



Start formularbasierter Filter

Die Eingabe der Filterwerte muss nicht eindeutig sein. Hier hilft dann die Formulierung von Bedingungen, wie sie bei der Filterung von Tabellen geschildert wurde. So führt ein «WIE '%Auge%'», eingetragen in das Eingabefeld für den Titel des Mediums, direkt zur Auffindung aller Titel, in denen 'Auge' enthalten ist – in der Beispieldatenbank «Medien ohne Makros» z.B. zur Anzeige des Titels «Im Augenblick».

Filter für die Listenfelder werden direkt über die Listenfelder ausgewählt. Eine Eingabe von Fremdschlüsselwerten ist also nicht nötig.

The screenshot shows a software interface with a form titled 'Medieneingabe' and a separate 'Filter Navigator' dialog box. The form has fields for ID, Titel (containing 'WIE "%Auge%"'), Kategorie, Medienart, Ort, Verlag, E_Jahr, and Ausgabe. Below these are two list boxes: 'Verfasser_Reihenfolge' (showing 'Edmunds, Dave') and 'Untertitel'. At the bottom is a 'Schließen' button. The 'Filter Navigator' dialog shows a tree structure under 'MainForm'. It lists several filters: 'Medienart: 1', 'Titel: WIE "%Auge%"', 'E_Jahr: < 1980', and 'Verfasser_ID: 8'. The 'Formular_Verfasser' filter is highlighted in green.

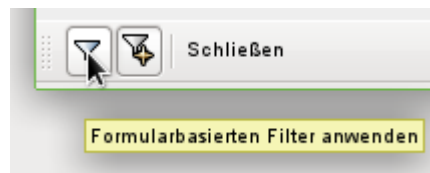
Hier werden alle Datensätze gesucht, die die Medienart 'CD' haben (entspricht dem Fremdschlüsselwert '1') und gleichzeitig im Titel irgendwo die Buchstabenfolge 'Auge' haben. Bei der Eingabe «WIE '%Auge%'» ersetzt die grafische Benutzeroberfläche das in SQL übliche Allroundzeichen für mehrere Werte «%» durch das Zeichen «*», das für viele Nutzer eher gebräuchliche Allroundzeichen.

Neben der Bedingung dürfen auch alle Datensätze des Hauptformulars angezeigt werden, bei denen die Jahresangabe «< 1980» ist.

Aus dem Unterformular wird noch der Verfasser 'Edmunds, Dave' aus dem Listefeld ausgewählt. Der entsprechende Fremdschlüsselwert ist hier die '8'.

Der formularbasierte Filter zeigt bei der Eingabe nur die Felder an, die vor dem ODER mit einem Haken versehen sind. So lassen sich auch mehrere Bedingungen für ein Feld angeben. Leider unterschlägt der Filter dabei die Anzeige des Listenfeldes «Medienart» mit dem entsprechenden Wert im Hauptformular, während dies im Tabellenkontrollfeld des Unterformulars gelingt.

Statt der Navigationsleiste werden während des Erstellens des Filters unter dem Formular lediglich drei Auswahlmöglichkeiten für den formularbasierten Filter dargestellt: Filtern nach den Vorgaben, Filterwerte hinzufügen und die Anzeige des formularbasierten Filters einfach schließen.



Der Filter wird angewandt und das Formular nach den vorgegebenen Werten gefiltert.

Medieneingabe

ID: Titel:

Kategorie: Medienart:

Ort: Verlag: E_Jahr: Ausgabe: Wert: Bild:

Verfasser_Reihen...	Verfasser	Zusatz

Datensatz 1 von 1

Nr	Untertitel	Dauer

Datensatz 1 von 1

ISBN/ISSN:

Anmerkung:

Datensatz 1 von 5

Ist die Filterung aktiv, so wird die Zahl der Datensätze auf die Datensätze mit einem Treffer eingeschränkt. Hier gibt es insgesamt 5 Datensätze mit Treffern. In dem obigen Bild stimmt zwar nicht der Titel und die Medienart, dafür aber die Bedingung, dass das Erscheinungsjahr vor 1980 liegen soll.

Eine Merkwürdigkeit zeigt sich im Unterformular. Da auch hier eine Filterung vorgenommen wurde, wird der Verfasser des Buches 'Der kleine Hobbit' nicht angezeigt. Der Filterwert war mit dem Verfasser 'Dave Edmunds' verbunden. Dies zeigt wieder die Logik, die hinter der Filterung in allen Fällen steckt: Es werden die Filtereigenschaften des jeweiligen Formulars genutzt. Unterformulare werden separat gefiltert und eine Filterung der Unterformulare wirkt sich nicht auf das Hauptformular aus – und umgekehrt.

Diese schon sehr gelungene Filterfunktion leidet allerdings an den folgenden Nachteilen:

- Die Filterung funktioniert wieder nur im Hauptformular. Im Unterformular werden wie bei Autofilter zwar die fehlenden Werte unterdrückt. Dies wirkt sich aber nicht auf die Anzeige

im Hauptformular aus. Es werden also im Hauptformular auch Inhalte angezeigt, die den gefilterten Inhalt aus dem Unterformular nicht vorweisen können.

- Es sind umfangreiche Kenntnisse der Bedingungsformulierung notwendig, wenn mehr als nur die kompletten Begriffe gesucht werden sollen. Hier müssen Kenntnisse über Abfrage-techniken vorhanden sein, die aber den meisten suchmaschinenverwöhnten Nutzern fehlen.

Filterung über den Standardfilter

Die Filterung über den Standardfilter kann durch die Anzeige der Datenquelle als Tabelle erreicht werden. Dort gelten dann die gleichen Regeln wie bei der Filterung mit dem Standardfilter in Tabellen.



ID	Titel	Ausgabe	E_Jahr	Anmerkung	Kategorie	Medienart	Ort	Verlag	Wert (€)	ISBN
0	Der kleine Hobbit	2. Aufl.	1972		Fantasy	Buch			7,20 €	
1	Das sogenannte		1972		Liedermacher	Buch			5,80 €	
2	Eine kurze		1983			Buch			25,00 €	
3	Tradition		1970			CD			12,50 €	
4	Die neue		1996			Buch			15,80 €	-214748
5	Über...		1972		Rock	CD				214748

Datensatz 1 von 9

Medieneingabe

ID: 0 Titel: Der kleine Hobbit

Kategorie: Fantasy Medienart: Buch

Ort: Verlag: E_Jahr: 1972 Ausgabe: 2. Aufl. Wert (€): 7,20 € Bild:

Verfasser_Reihenfolge: Verfassernamen: Zusatz: ISBN/ISSN:

Datensatz 1 von 1

Nr.: Untertitel: Dauer:

Anmerkung:

Datensatz 1 von 9

Rechts unten im Formularnavigator wird die Datenquelle als Tabelle eingeblendet. In der Tabellensicht steht dann auch der Standardfilter zur Verfügung. Im Gegensatz zur Ansicht der eigentlichen

Tabelle "Medien" werden hier die Fremdschlüsselfelder nicht mit ihren Schlüsselwerten, sondern mit den Inhalten dargestellt. Ein Klick auf das Feld «Medienart» zeigt, dass hier ein Listefeld nach den Vorgaben des Formulars erstellt wurde.

Hinweis

Die obige Ansicht zeigt dann leider auch gleich einen Bug, der seit den Anfängen von Base wohl schon existiert: Die 13-stellige ISBN-Nummer wird nicht korrekt dargestellt. Das entsprechende Feld wird stattdessen mit der niedrigst möglichen Integer-Zahl versehen. Hat eine Zahl mehr als 9 Stellen, so ist hier also Vorsicht geboten: Datenänderung kann zu Datenverlusten führen. ([Bug 82411](#))

Dieser Bug tritt auch in Tabellenkontrollfeldern auf. Die Darstellung ist korrekt, wenn statt eines Zahlenfeldes im Formular ein formatierbares Feld genommen wird.

Die Tabellenansicht des Formulars hat die folgenden Nachteile für diese Recherchemethode:

- Die Suche funktioniert nur innerhalb der einem Formular zugrundeliegenden Tabelle oder Abfrage, nicht in einem dazugehörigen Unterformular.
- In der Datenansicht werden Listensfelder angezeigt, nach denen aber nicht gefiltert werden kann. Wieder müssen die Fremdschlüssel für die Felder bekannt sein, damit eine Filterung Erfolg hat. Hat z.B. die "Medienart" «Buch» den Primärschlüsselwert «1», so wird in der Tabelle "Medien" der Fremdschlüssel «1» gespeichert. Nach diesem Schlüsselwert ist zu suchen – obwohl die eingeblendete Ansicht «Buch» in einem Listensfeld zeigt.

Fazit

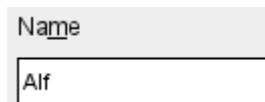
Sämtliche Such- und Filterfunktionen eignen sich für Formulare nur eingeschränkt. Die eingebaute Suche ist viel zu langsam und schränkt die Zahl der Datensätze nicht auf die mit entsprechenden Treffern ein. Die Filter funktionieren grundsätzlich nur innerhalb eines Formulars. Soll nach Werten im Unterformular gefiltert werden, so ist nur das aktuelle Unterformular betroffen. Es werden einfach nicht alle Datensätze korrekt ermittelt.

Aus diesem Grund wurde weiter oben in die Formulare der Beispieldatenbank eine entsprechend einfacher zu bedienende Suchfunktion integriert, die allerdings etwas Handarbeit und SQL-Kenntnis verlangt.

Dateneingabe und Navigation

Wird ein Formular so eingestellt, dass beim Öffnen gleich der Kontrollelemente-Fokus aktiviert wird, so steht der Cursor direkt im ersten Eingabefeld. Statt mit der Maus von einem Feld zum anderen zu springen, ist bei Formularen die Bewegung mit der Tabulatortaste von einem Feld zum nächsten üblich.

Manchmal erfordern Formulare auch Sprünge zu einem anderen Formularfeld. Diese können natürlich mit der Maus geregelt werden, sind aber auch über Kurzbefehle anspringbar.



A screenshot of a text input field. Above the field is a label 'Name'. Inside the field, the text 'Alf' is entered.

Dem Textfeld ist über **Eigenschaften: Textfeld → Beschriftungsfeld** ein Beschriftungsfeld zugewiesen. Die Beschriftung des Beschriftungsfeldes wurde von «Name» auf «Na~me» geändert. Dadurch erscheint im Formular jetzt ein Unterstrich unter dem «m». Das Textfeld kann jetzt bei der Eingabe mit dem Kürzel **Alt + m** angesprungen werden. Dies funktioniert allerdings nur, wenn der Cursor bereits in einem Formularfeld steht.

Prinzipiell können alle Buchstaben als Sprungziel genommen werden, da der Sprung innerhalb des Formulars erfolgt. Steht der Cursor aber nicht in einem Formularfeld, so werden mit den Kürzeln

eventuell Sprungziele aus der LibreOffice-Oberfläche angesteuert. Mit dem Sprungziel «a» z. B. wäre dies dann statt des Feldes «Name» das Öffnen des Menüs «Ansicht».

Leider funktioniert das Anspringen von Feldern nur innerhalb eines Formulars, nicht aber bei einer Konstruktion mit Unterformularen. Ein Sprung aus einem Unterformular zu einem Hauptformular ist so also zur Zeit nicht möglich.

Aus einem Tabellenkontrollfeld sollte der Cursor mit **Strg + Tab** herausspringen. Manchmal kann es passieren, dass die Tastenkombination nicht den gewünschten Erfolg erzielt. Daher hier eine völlig andere Möglichkeit:⁷

In dem Unterformular wird eine Schaltfläche angelegt. Der Titel der Schaltfläche heißt z.B. «~neu». Das «~» sorgt, wie oben erwähnt, dafür, dass der Button mit dem Kürzel **Alt + N** ausgelöst werden kann. In den Zusatzinformationen des Buttons wird außerdem noch der Name des Feldes im Hauptformular benannt, zu dem der Cursor springen soll. Die Schaltfläche wird über **Eigenschaften → Ereignisse → Aktion ausführen** mit dem folgenden Makro verbunden:

```
SUB JumpToMainform(oEvent AS OBJECT)
    DIM oField AS OBJECT
    DIM oForm AS OBJECT
    DIM oDoc AS OBJECT
    DIM oController AS OBJECT
    DIM oView AS OBJECT
    DIM stShortcut AS STRING
    oField = oEvent.Source.Model
    stShortcut = Mid(oField.Label, InStr(oField.Label, "~") + 1, 1)
    oForm = oField.Parent.Parent
    SELECT CASE stShortcut
        CASE "n"
            oForm.MoveToInsertRow()
        CASE "ä"
            IF oForm.IsLast() THEN
                oForm.MoveToInsertRow()
            ELSE
                oForm.Next()
            END IF
    END SELECT
    oDoc = thisComponent
    oController = oDoc.GetCurrentController()
    oView = oController.getControl(oForm.getByName(oField.Tag))
    oView.setFocus
END SUB
```

Das Makro kann dazu genutzt werden, nur ins Hauptformular zurück zu springen. Es kann bei der obigen Einstellung auch dazu dienen, direkt einen neuen Datensatz anzulegen oder auch über einen weiteren Button mit dem Kürzel **Alt + ä** (für «nächster Datensatz») zum nächsten Datensatz zu springen. Befindet sich der Cursor des Hauptformulars allerdings schon auf dem letzten Datensatz, so muss der Sprung zu einem neuen Datensatz erfolgen. Weiteres zu Makros siehe im entsprechenden Kapitel.

Der Sprung auf die Schaltfläche gelingt nur mit einer sichtbaren Schaltfläche. Die Schaltfläche kann aber ruhig eine ganz geringe Ausdehnung oder eine Breite von 0 cm und eine Höhe von 0 cm haben. Bei der Definition einer nicht vorhandenen Breite und einer nicht vorhandenen Höhe kann es allerdings vorkommen, dass der Button als Strich im Formular über die gesamte Bildschirmbreite sichtbar wird.

⁷ Siehe hierzu auch die Beispieldatenbank «Beispiel_Cursorsprung_Subform_Mainform.odt»

Hinweis

Bei Tabellenkontrollfeldern funktioniert das Setzen des Focus nicht wie bei einfachen Formularfeldern. Hier muss das Tabellenkontrollfeld angesteuert werden und dann über

```
oView.CurrentColumnPosition = 2
```

z. B. das Feld in der 2. Spalte angesteuert werden,

Drucken aus Formularen

Formulare können so aufgebaut werden, dass ein direkter Druck der Ansicht möglich ist. Um dabei zu brauchbaren Ergebnissen zu kommen, muss allerdings darauf geachtet werden, Elemente nicht außerhalb des druckbaren Bereiches zu verschieben. Dazu wird beim Bearbeiten des Formulars **Ansicht → Drucklayout** gewählt. Die Seiteneigenschaften können eingestellt werden. Ein farbiger Hintergrund ist hier sicher nicht von Vorteil.

Jedes einzelne Formularelement kann in seinen **Eigenschaften → Allgemein → Druckbar** von einem Druck ausgeschlossen werden. Bilder, die aus einer Tabelle über grafische Steuerelemente ausgelesen werden, können leider aufgrund des Bugs [90388](#) in den Versionen LO 3.4 bis LO 4.4 nicht gedruckt werden.

Ist die Datenbank in LibreOffice registriert worden (über **Extras → Optionen → LibreOffice Base → Datenbanken** oder direkt beim Erstellen der Datenbank), so kann ein Formular auch zum Seriendruck genutzt werden. Das Formular wird zum Bearbeiten geöffnet. Über **Ansicht → Datenquellen** oder mit F4 werden die Datenquellen zugänglich. Felder aus der Datenbank können jetzt über die Tabellenköpfe in das Formular gezogen werden. Das Formular wird anschließend abgespeichert. Wird das Formular anschließend wie zur Eingabe geöffnet, so erkennt LibreOffice, dass Serienbrieffelder enthalten sind und fragt nach, ob ein Serienbrief gedruckt werden soll.

Details zu der Erstellung von Serienbriefen sind in dem Kapitel «Datenbank-Anbindung» enthalten.

Darstellungsgröße von Formularen

Beim Öffnen von Formularen kann der Nutzer manchmal von einer unüblichen Zoom-Einstellung des Formulars überrascht werden. Die Darstellungsgröße des Formulars richtet sich leider nicht nach der Zoom-Einstellung des Formulars beim Speichern. Vielmehr übernimmt Base die jeweils letzte Einstellung des Zooms von einem vorhergehenden Writer-Dokument.

Der Zoom kann über die Zoomeinstellung oder den Maßstab am unteren rechten Fensterrand eingestellt werden. Eine andere Einstellungsmöglichkeit ist über das Drücken von **Strg** und das gleichzeitige Betätigen des Scrollrades der Maus gegeben.

Wer automatisch bei Start eines Formulars immer die gleiche Größe erreichen möchte, kann dies über das folgende Makro erreichen:

```
SUB Formularzoom
  ThisComponent.CurrentController.ViewSettings.ZoomValue=100
END SUB
```

Dieses Makro soll beim Öffnen des Formulars ausgelöst werden. Das Formular muss zum Bearbeiten geöffnet werden. Unter **Extras → Anpassen → Ereignisse → Dokument öffnen** wird das Makro eingebunden.

Abfragen

Allgemeines zu Abfragen

Abfragen an eine Datenbank sind das mächtigste Werkzeug, was uns zur Verfügung steht, um Datenbanken sinnvoll zu nutzen. Sie fassen Daten aus unterschiedlichen Tabellen zusammen, berechnen gegebenenfalls irgendwelche Ergebnisse, filtern einen ganz bestimmten Datensatz aus einer Unmenge an Daten mit hoher Geschwindigkeit heraus. Die großen Internetdatenbanken, die viele täglich nutzen, haben ihren Hauptsinn darin, dass aus der Unmenge an Informationen durch geschickte Wahl der Schlüsselwörter schnell ein brauchbares Ergebnis für den Nutzer geliefert wird – einschließlich natürlich der zu den Suchbegriffen gehörenden Anzeigen, die zum Kauf animieren sollen.

Eingabemöglichkeiten für Abfragen

Die Eingabe von Abfragen kann sowohl in der GUI als auch direkt per SQL erfolgen. In beiden Fällen öffnet sich ein Fenster, das es ermöglicht, die Abfragen auszuführen und gegebenenfalls zu korrigieren.

Abfrageerstellung mit der grafischen Benutzeroberfläche

Die Erstellung von Abfragen mit dem Assistenten wird hier nicht dargestellt, da der Assistent zwar bei wenig Tabellen schnell zu einem Ergebnis führt, für eine größere Datenbank aber wenig zielführend ist. Hier wird stattdessen die direkte Erstellung über **Abfrage in der Entwurfsansicht erstellen** erklärt.

Nach einem Aufruf der Funktion erscheinen zwei Fenster. Ein Fenster stellt die Grundlagen für den Design-Entwurf der Abfrage zur Verfügung, das andere dient dazu, Tabellen, Ansichten oder Abfragen der Abfrage hinzuzufügen.

Da unser einfaches Formular auf der Tabelle "Ausleihe" beruhte, wird zuerst einmal an dieser Tabelle die Grundkonstruktion von Abfragen mit dem Abfrageeditor erklärt.



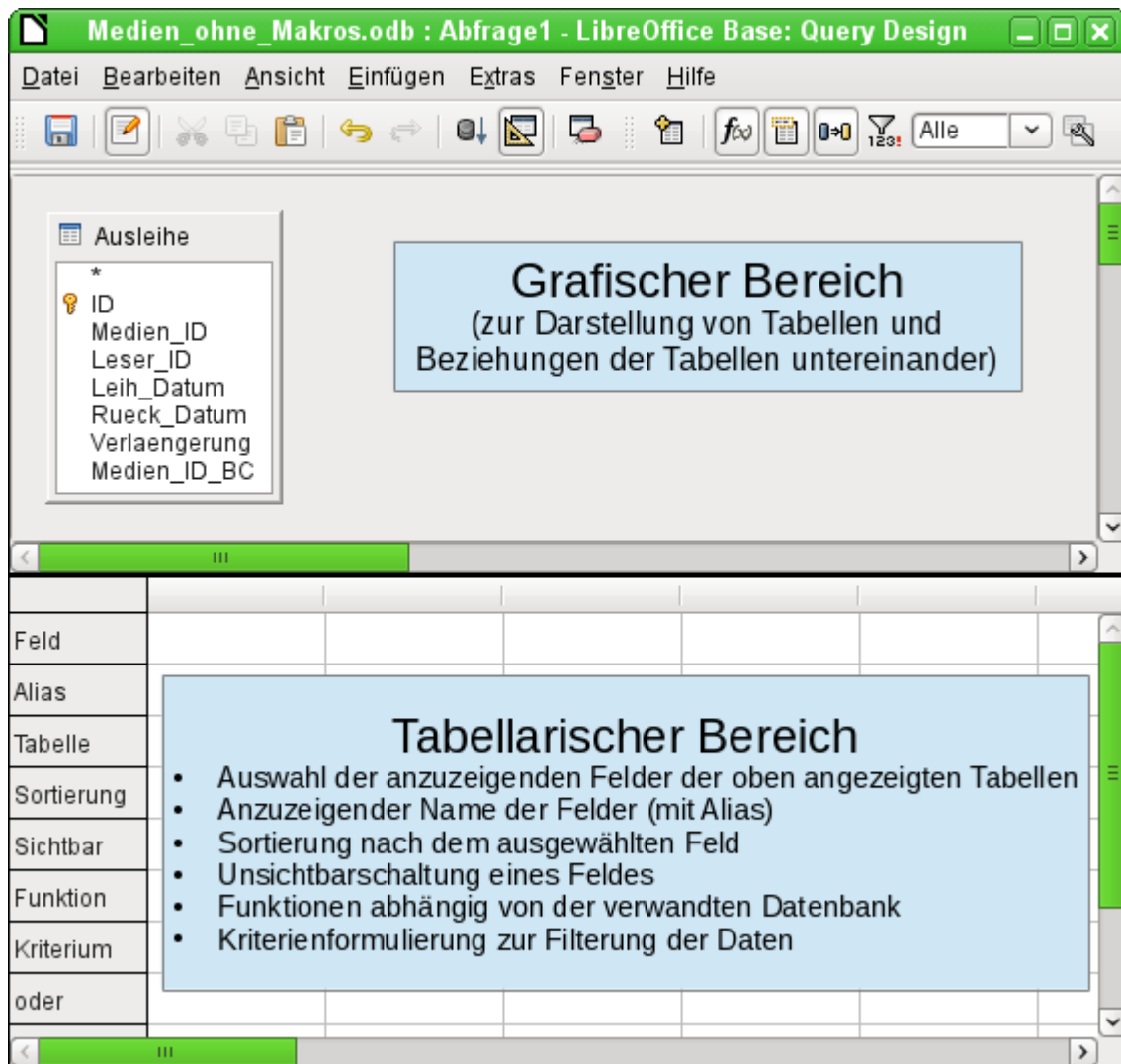


Abbildung 38: Bereiche des Abfrageentwurfs

Aus den zur Verfügung stehenden Tabellen wird die Tabelle Ausleihe ausgewählt. Dieses Fenster bietet die Möglichkeit, gleich mehrere Tabellen (und unter diesen auch Ansichten) sowie Abfragen miteinander zu kombinieren. Jede gewünschte Tabelle wird markiert (linke Maustaste) und dann dem grafischen Bereich des Abfrageeditors hinzugefügt.

Sind alle erforderlichen Tabellen ausgewählt, so wird dieses Fenster geschlossen. Gegebenenfalls können später noch mehr Tabellen und Abfragen hinzugefügt werden. Ohne eine einzige Tabelle lässt sich aber keine Abfrage erstellen, so dass eine Auswahl zu Beginn schon sein muss.

Abbildung 38 zeigt die grundsätzliche Aufteilung des grafischen Abfrageeditors: Im grafischen Bereich werden die Tabellen angezeigt, die mit der Abfrage zusammen hängen sollen. Hier kann auch ihre Beziehung zueinander in Bezug auf die Abfrage angegeben werden. Im tabellarischen Bereich erfolgt die Auswahl der Felder, die angezeigt werden sollen, sowie Bedingungen, die mit diesen Feldern verbunden sind.

Ein Klick mit der Maustaste auf das Feld der ersten Spalte im tabellarischen Bereich öffnet die Feldauswahl:

Feld		▼
Alias	Ausleihe.*	
Tabelle	Ausleihe.ID	
Sortierung	Ausleihe.Medien_ID	
Sichtbar	Ausleihe.Leser_ID	
Funktion	Ausleihe.Leih_Datum	
	Ausleihe.Rueck_Datum	
	Ausleihe.Verlaengerung	
	Ausleihe.Medien_ID_BC	

Hier stehen jetzt alle Felder der Tabelle "Ausleihe" zur Verfügung. Die Schreibweise: **Tabellenname.Feldname** – deshalb beginnen alle Feldbezeichnungen hier mit dem Begriff "Ausleihe.".

Eine besondere Bedeutung hat die markierte Feldbezeichnung **Ausleihe.***. Hier wird mit einem Klick jeder Feldname der zugrundeliegenden Tabelle in der Abfrage wiedergegeben. Wenn allein diese Feldbezeichnung mit dem Jokerzeichen «*» für alle Felder gewählt wird, unterscheidet sich die Abfrage nicht von der Tabelle.

Tipp

Sollen schnell alle möglichen Felder aus Tabellen in die Abfrage übertragen werden, so kann auch direkt im grafischen Bereich auf die Tabellenübersicht geklickt werden. Mit einem Doppelklick auf ein Feld wird das Feld im tabellarischen Bereich an der nächsten freien Position eingefügt.

Medien_ohne_Makros.odb : Abfrage1 - LibreOffice Base: Query Design

Datei Bearbeiten Ansicht Einfügen Extras Fenster Hilfe

Abfrage ausführen (F5)

	ID	Medien_ID	Leser_ID	Ausleihdatum	Rückgabedatum
▶	12	3	0	09.12.11	
	16	7	0	25.02.12	
	23	1	0	04.04.12	
	22	2	1	04.04.12	
	24	8	1	22.04.12	
	21	0	9	04.04.12	
⚙	<Auto				

Datensatz 1 von 6

Ausleihe

- ID
- Medien_ID
- Leser_ID
- Leih_Datum
- Rueck_Datum
- Verlaengerung
- Medien_ID_BC

Feld	ID	Medien_ID	Leser_ID	Leih_Datum	Rueck_Datum
Alias				Ausleihdatum	Rückgabedatum
Tabelle	Ausleihe	Ausleihe	Ausleihe	Ausleihe	Ausleihe
Sortierung			aufsteigend	aufsteigend	
Sichtbar	☒	☒	☒	☒	☒
Funktion					
Kriterium					IST LEER
oder					

Es werden die ersten fünf Felder der Tabelle Ausleihe ausgewählt. Abfragen können auch im Design-Modus immer wieder testweise ausgeführt werden. Dann erscheint oberhalb der grafischen Ansicht der Tabelle eine tabellarische Übersicht über die Daten. Die testweise Ausführung von Abfragen ist vor dem Abspeichern immer sinnvoll, damit für den Nutzer klar ist, ob die Abfrage auch wirklich das erreicht, was sie erreichen soll. Manchmal wird durch einen Denkfehler ausgeschlossen, dass eine Abfrage überhaupt je Daten ausgeben kann. Ein anderes Mal kann es passieren, dass plötzlich genau die Datensätze angezeigt werden, die ausgeschlossen werden sollten.

Grundsätzlich lässt sich eine Abfrage, die eine Fehlerrückmeldung bei der zugrundeliegenden Datenbank produziert, gar nicht erst abspeichern.

	ID	Medien_ID
▶	12	3
	16	7
	23	1
	22	2
	24	8
	21	0
☀	<Auto	

Abbildung 39: Abfrage bearbeitbar

	Medien_ID	Leser
▶	3	0
	7	0
	1	0
	2	1
	8	1
	0	9

Abbildung 40: Abfrage nicht bearbeitbar

Besonderes Augenmerk sollte in dem obigen Test einmal auf die erste Spalte des dargestellten Abfrageergebnisses geworfen werden. Auf der linken Seite der Tabelle erscheint immer der Datensatzmarkierer, der hier auf den ersten Datensatz als aktivem Datensatz hinweist. Während in [Abbildung 39](#) aber das erste Feld des ersten Datensatzes grün markiert ist, zeigt das erste Feld in [Abbildung 40](#) nur eine gestrichelte Umrandung an. Die grüne Markierung deutet bereits an, dass hier im Feld selbst etwas geändert werden kann. Die Datensätze sind also änderbar. In [Abbildung 39](#) ist außerdem eine zusätzliche Zeile zur Eingabe neuer Daten vorhanden, in der für das Feld "ID" schon <AutoWert> vorgemerkt ist. Auch hier also sichtbar, dass Neueingaben möglich sind.

Grundsätzlich sind dann keine Neueingaben möglich, wenn der Primärschlüssel der abgefragten Tabelle nicht in der Abfrage enthalten ist.

Feld	ID	Medien_ID	Leser_ID	Leih_Datum	Rueck_Datum
Alias				Ausleihdatum	Rückgabedatum

Den Feldern "Leih_Datum" und "Rueck_Datum" wurde ein Aliasname zugewiesen. Sie wurden damit nicht umbenannt, sondern unter diesem Namen für den Nutzer der Abfrage sichtbar gemacht.

	ID	Medien_ID	Leser_ID	Ausleihdatum	Rückgabedatum
▶	12	3	0	09.12.11	

Entsprechend ist in der Tabellenansicht der Alias statt der eigentlichen Feldbezeichnung zu sehen.

Rueck_Datum
Rückgabedatum
Ausleihe
☒
IST LEER

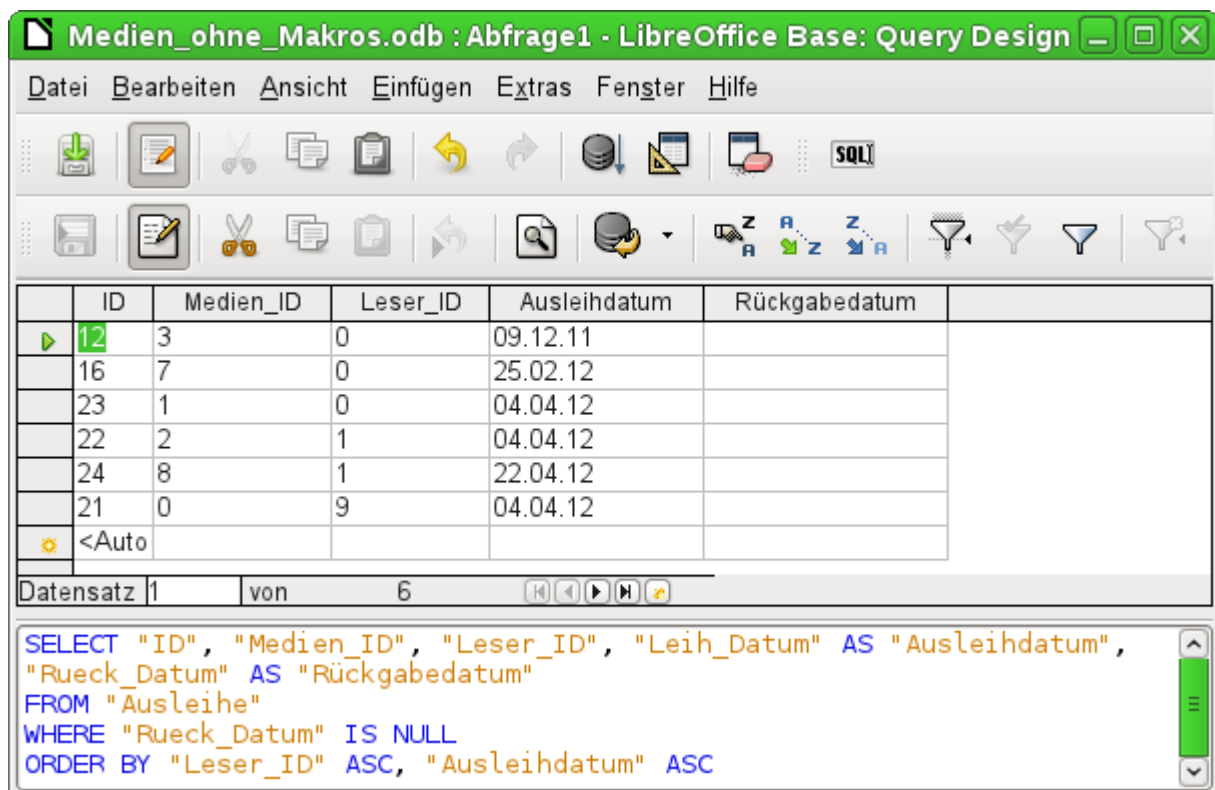
Dem Feld "Rueck_Datum" wurde nicht nur ein Alias, sondern auch ein Kriterium zugewiesen, nach dem nur die Datensätze angezeigt werden sollen, bei denen das Feld "Rueck_Datum" leer ist. Die

Angabe erfolgt hier in deutscher Sprache, wird dann aber in der eigentlichen Abfrage in SQL übersetzt.

Durch dieses Ausschlusskriterium werden nur die Datensätze von Medien angezeigt, die ein Medium enthalten, das noch nicht zurückgegeben wurde.



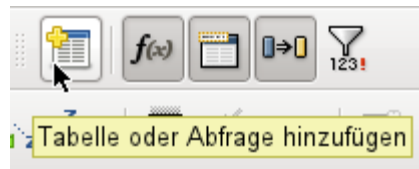
Um die SQL-Sprache besser kennen zu lernen, empfiehlt es sich immer wieder einmal vom Design-Modus aus in den SQL-Darstellungsmodus zu wechseln.



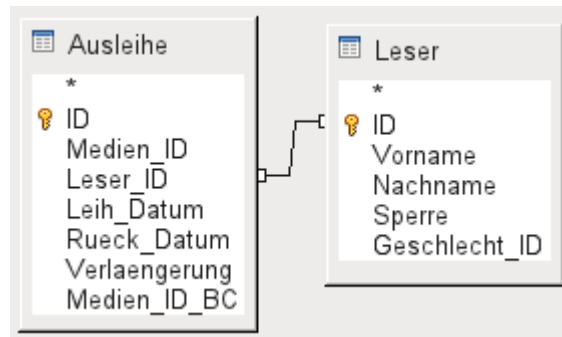
Hier wurde die durch die Auswahlen erstellte SQL-Formulierung sichtbar gemacht. Für die bessere Übersicht ist die Ansicht mit Zeilenumbrüchen versehen worden. Leider speichert der Editor diese Zeilenumbrüche nicht mit ab, so dass beim nächsten Aufruf die Abfrage wieder komplett als eine durchgängige Zeile mit Umbruch am Fensterrand wiedergegeben wird.

Über **SELECT** wird die Auswahl gestartet. Mit **AS** können die Aliasbezeichnungen eingeführt werden. **FROM** zeigt auf die Tabellenquelle der Abfrage. **WHERE** gibt die Bedingung für die Abfrage wieder, hier also, dass der Inhalt des Feldes "Rueck_Datum" leer ist (**IS NULL**). Mit **ORDER BY** wird die Sortierung definiert, und zwar als aufsteigend (**ASC** – ascending) für die beiden Felder "Leser_ID" und "Ausleihdatum". Diese Sortierung zeigt auch, dass die Zuweisung eines Alias das Feld "Leih_Datum" auch in der Abfrage selbst mit dem Alias ansprechbar macht.

Bisher sind die Felder "Medien_ID" und "Leser_ID" nur als Zahlenfelder sichtbar. Welchen Namen der Leser hat, bleibt unklar. Um dies in einer Abfrage anzuzeigen, muss die Tabelle Leser eingebunden werden. Um die folgende Ansicht zu erhalten, muss in den Design-Modus zurückgeschaltet werden. Danach kann dann eine neue Tabelle in der Design-Ansicht hinzugefügt werden.



Hier können im Nachhinein weitere Tabellen oder Abfragen in der grafischen Benutzeroberfläche sichtbar gemacht werden. Sind bei der Erstellung der Tabellen Beziehungen geklärt worden (siehe Kapitel «*Beziehungen zwischen Tabellen allgemein*»), dann werden die Tabellen entsprechend direkt miteinander verbunden angezeigt:



Fehlt die Verbindung, so kann hier durch ein Ziehen mit der Maus von "Ausleihe"."Leser_ID" zu "Leser"."ID" eine direkte Verknüpfung erstellt werden.

Hinweis

Eine Verbindung von Tabellen geht zur Zeit nur, wenn es sich um die interne Datenbank oder ein relationales Datenbanksystem handelt. Tabellen einer Tabellenkalkulation z. B. lassen sich so nicht miteinander verbinden. Sie müssen dazu in eine interne Datenbank importiert werden.

Um die Verbindung der Tabellen herzustellen, reicht ein Import ohne zusätzliche Erstellung eines Primärschlüssels aus.

Jetzt können im tabellarischen Bereich auch die Felder der Tabelle "Leser" ausgewählt werden. Die Felder werden dabei erst einmal am Schluss der Abfrage angeordnet.

Leser_ID	Leih_Datum	Rueck_Datum	Vorname	Nachname
	Ausleihdatum	Rückgabedatum		
Ausleihe	Ausleihe	Ausleihe	Leser	Leser

Mit der Maus kann in dem tabellarischen Bereich des Editors die Lage der Felder korrigiert werden. Hier wird z. B. gerade das Feld "Vorname" direkt vor das Feld "Leih_Datum" verlegt.

	ID	Medien_ID	Leser_ID	Vorname	Nachname	Ausleihdatum	Rückgabedatum
▶	12	3	0	Bert	Lederstrumpf	09.12.11	
	16	7	0	Bert	Lederstrumpf	25.02.12	
	23	1	0	Bert	Lederstrumpf	04.04.12	
	22	2	1	Heinrich	Müller	04.04.12	
	24	8	1	Heinrich	Müller	22.04.12	
	21	0	9	Terence	Nobody	04.04.12	
Datensatz	1	von	6				

Die Namen wurden jetzt sichtbar gemacht. Die "Leser_ID" ist eigentlich überflüssig. Auch ist die Sortierung nach "Nachname" und "Vorname" eigentlich sinnvoller als nach der "Leser_ID".

Diese Abfrage eignet sich nicht mehr für Base als Abfrage mit Eingabemöglichkeit, da zu der neu hinzugekommenen Tabelle Leser der Primärschlüssel fehlt. Erst wenn auch dieser Primärschlüssel eingebaut wird, ist die Abfrage wieder editierbar – allerdings komplett editierbar, so dass auch die Namen der Leser geändert werden können. Die Möglichkeit der Editierbarkeit ist also sehr vorsichtig zu nutzen, gegebenenfalls über ein Formular einzuschränken.

Selbst wenn die Abfrage weiter editierbar ist, lässt sie sich nicht so komfortabel nutzen wie ein Formular mit Listenfeldern, die zwar die Lesernamen anzeigen, aber die "Leser_ID" an die Tabelle weitergeben. Listenfelder lassen sich in eine Abfrage nicht einfügen. Sie sind den Formularen vorbehalten.

```
SELECT "Ausleihe"."ID", "Ausleihe"."Medien_ID", "Ausleihe"."Leser_ID",  
"Leser"."Vorname", "Leser"."Nachname", "Ausleihe"."Leih_Datum" AS  
"Ausleihdatum", "Ausleihe"."Rueck_Datum" AS "Rückgabedatum"  
FROM "Ausleihe", "Leser"  
WHERE "Ausleihe"."Leser_ID" = "Leser"."ID" AND  
"Ausleihe"."Rueck_Datum" IS NULL  
ORDER BY "Ausleihe"."Leser_ID" ASC, "Ausleihdatum" ASC
```

Wird jetzt auf die SQL-Ansicht umgeschaltet, so zeigt sich, dass alle Felder mit einer Doppelbezeichnung gekennzeichnet sind: "**Tabellenname**". "**Feldname**". Dies ist notwendig, damit der Datenbank klar wird, aus welcher Tabelle die jeweiligen Feldinhalte stammen. Schließlich können Felder in unterschiedlichen Tabellen ohne weiteres den gleichen Feldnamen tragen. Bei den bisherigen Tabellenkonstruktionen trifft dies z. B. immer auf das Feld "ID" zu.

Hinweis

Folgende Abfrage funktioniert auch ohne Tabellennamen vor den Feldnamen:

```
SELECT  
    "Ware"."ID",  
    "Abgang"."Anzahl",  
    "Ware"."Preis"  
FROM "Ware",  
    "Abgang"  
WHERE "Abgang"."Ware_ID" = "Ware"."ID"
```

Hierbei wird "ID" aus der Tabelle gezogen, die als erste in der **FROM**-Definition steht. Auch die Tabellendefinition in der **WHERE**-Formulierung wäre überflüssig, da "Ware_ID" nur einmal in der Tabelle "Abgang" vorkommt und die ID aus der Tabelle "Ware" genommen wird (Position der Tabelle). Die folgende Abfrage liefert also das gleiche Ergebnis:

```
SELECT  
    "ID",  
    "Anzahl",  
    "Preis"  
FROM "Ware",  
    "Abgang"  
WHERE "Ware_ID" = "ID"
```

Wird einem Feld in der Abfrage ein Alias zugewiesen, so kann es z. B. in der Sortierung mit diesem Alias ohne einen Tabellennamen angesprochen werden. Sortierungen werden in der grafischen Benutzeroberfläche nach der Reihenfolge der Felder in der Tabellenansicht vorgenommen. Sollte stattdessen zuerst nach "Ausleihdatum" und dann nach "Ausleihe"."Leser_ID" sortiert werden, so kann dies erzeugt werden, indem

- die Reihenfolge der Felder im tabellarischen Bereich der grafischen Benutzeroberfläche geändert wird,
- ein zweites Feld eingefügt wird, das auf unsichtbar geschaltet ist und nur die Sortierung gewährleisten soll (wird allerdings beim Editor nur vorübergehend angenommen, wenn kein Alias definiert wurde) oder
- der Text für die **ORDER BY** – Anweisung im SQL-Editor entsprechend umgestellt wird.

Die Beeinflussung der Sortierreihenfolge arbeitet je nach Version *nicht ganz fehlerfrei*. Wird in LO 3.3.4 die Reihenfolge anders gewählt als durch die GUI vorgegeben, so funktioniert die Abfrage korrekt. Ein erneutes Aufrufen der Abfrage zur Bearbeitung zeigt aber die Einstellung der Sortierung nach Reihenfolge der Felder in der GUI. Die Kontrolle ergibt dann auch eine entsprechend geänderte Sortierreihenfolge wie angezeigt wieder. Sobald also die Abfrage nach einer zusätzlichen Änderung an anderer Stelle gespeichert wird, ist die Sortierung leider unbeabsichtigt mit geändert worden. In LO 3.5.3 RC2 wird die Sortierung aus der SQL-Ansicht korrekt übernommen und entsprechend mit nicht sichtbaren Feldern in der grafischen Benutzeroberfläche angezeigt.

Funktionen in der Abfrage

Mittels Funktionen lässt sich aus Abfragen auch mehr ersehen als nur ein gefilterter Blick auf die Daten einer oder mehrerer Tabellen. In der folgenden Abfrage wird, abhängig von der "Leser_ID", gezählt, wie viele Medien ausgeliehen wurden.

Feld	ID	Leser_ID	Rueck_Datum
Alias	Anzahl		
Tabelle	Ausleihe	Ausleihe	Ausleihe
Sortierung			
Sichtbar	☒	☒	☐
Funktion	Anzahl	Gruppierung	
Kriterium			IST LEER

Für die "ID" der Tabelle "Ausleihe" wird die Funktion **Anzahl** ausgewählt. Prinzipiell ist hier egal, welches Feld einer Tabelle gewählt wurde. Die einzige Bedingung: *Das Feld darf nicht in irgendwelchen Datensätzen leer sein*. Aus diesem Grunde ist der Primärschlüssel, der ja nie leer ist, immer eine geeignete Wahl. Gezählt werden die Felder, die einen Inhalt enthalten, der von **NULL** verschieden ist.

Hinweis

Sobald für ein Feld eine andere Funktion als die Gruppierung gewählt wird, sollte das Feld einen **Aliasnamen** erhalten. Ansonsten kann es bei der Weiterverarbeitung z.B. in Berichten, verschachtelten Abfragen oder auch Ansichten zu Problemen kommen.

Für die "Leser_ID", die ja Rückschlüsse auf den Leser zulässt, wird als Funktion die **Gruppierung** gewählt. Dadurch werden die Datensätze nach der "Leser_ID" zusammengefasst. So zählt denn die Anweisung die Datensätze, die zu jeder "Leser_ID" passen.

Als Kriterium ist wie in den vorhergehenden Beispielen das "Rueck_Datum" auf **IST LEER** gesetzt.

	Anzahl	Leser_ID	
	3	0	
	1	9	
▶	2	1	
Datensatz 3	von	3	⏮ ⏪ ⏩ ⏭
<pre>SELECT COUNT("ID") AS "Anzahl", "Leser_ID" FROM "Ausleihe" WHERE "Rueck_Datum" IS NULL GROUP BY "Leser_ID"</pre>			

Die Abfrage zeigt im Ergebnis, dass z. B. "Leser_ID" '0' insgesamt noch 3 Medien entliehen hat. Wäre die Funktion **Anzahl** statt der "ID" dem "Rueck_Datum" zugewiesen worden, so würden für alle "Leser_ID" jeweils '0' entlehene Medien dargestellt, da ja "Rueck_Datum" als **LEER** vordefiniert ist.

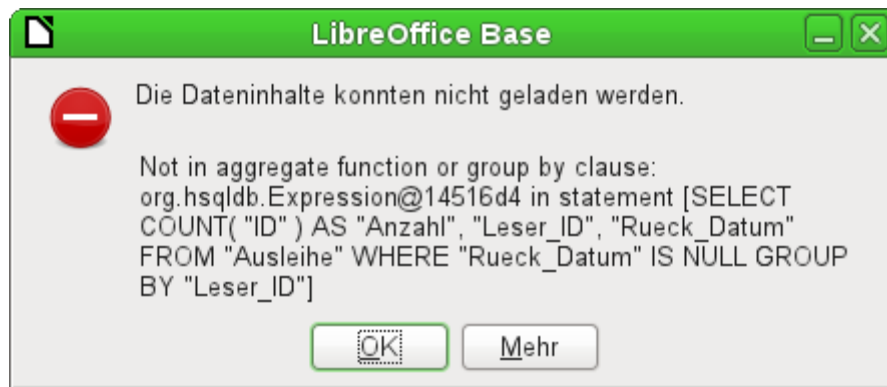
Die entsprechende Formulierung für den SQL-Code ist oben wieder abgebildet. Aus dem Begriff **Anzahl** der deutschen GUI wird **COUNT()**. Aus dem Begriff **Gruppierung** wird der Zusatz **GROUP BY**.

Insgesamt stehen über die grafische Benutzeroberfläche folgende Funktionen zur Verfügung, die ihre Entsprechung zu Funktionen in der zugrundeliegenden HSQLDB haben:



Eine Erläuterung zu den Funktionen ist in dem folgenden Kapitel [Abfrageerweiterungen im SQL-Modus](#) nachzulesen. Die Funktionen «Sammeln», «Vereinigung» und «Durchschnitt» funktionieren mit keiner der eingebauten Datenbanken. Sie sind Standardfunktionen von Oracle.

Wird einem Feld in einer Abfrage eine Sammelfunktion hinzugefügt, so müssen alle anderen Felder der Abfrage auch mit Funktionen versehen sein, sofern die Felder sichtbar sein sollen. Dies liegt daran, dass in einem Datensatz nicht plötzlich zwischendurch Felder mehrere Datensätze abbilden können. Wird dies nicht beachtet, so erscheint die folgende Fehlermeldung:



Etwas frei übersetzt: Der folgenden Ausdruck enthält ein Feld ohne eine der Sammelfunktionen oder eine Gruppierung.

Danach wird die gesamte Abfrage aufgelistet, leider ohne das Feld konkret zu benennen. Hier wurde einfach das Feld "Rueck_Datum" als sichtbar hinzugefügt. Dieses Feld hat keine Funktion zugewiesen bekommen und ist auch nicht in der Gruppierung enthalten.

Die über den Button **Mehr** erreichbaren Informationen sind für den Normalnutzer einer Datenbank nicht aufhellender. Hier wird lediglich zusätzlich noch der SQL-Fehlercode aufgeführt.

Innerhalb der GUI können auch die Grundrechenarten sowie weitere Funktionen angewandt werden.

Feld	ID	Medien_ID	Leser_ID	Datum	Anzahl("Mahnung"."Datum") * 2	Rueck_Datum
Alias				Mahnzahl	Mahnbetrag	
Tabelle	Ausleihe	Ausleihe	Ausleihe	Mahnung		Ausleihe
Sortierung						
Sichtbar	☒	☒	☒	☒	☒	☐
Funktion	Gruppierung	Gruppierung	Gruppierung	Anzahl		
Kriterium						IST LEER

Hier wurden die Tabelle "Ausleihe" und die Tabelle "Mahnung" zusammen abgefragt. Aus der Zahl der Datumseinträge in der Tabelle "Mahnung" wird auf die Anzahl der Mahnungen geschlossen. Als Mahnbetrag wird in der Abfrage 2,- € festgelegt. Statt der Feldauswahl wird in das Feld einfach geschrieben: **Anzahl(Mahnung.Datum)*2**. Die grafische Benutzeroberfläche setzt anschließend die Anführungsstriche und wandelt den Begriff Anzahl in den entsprechenden SQL-Begriff um.

Vorsicht



Werden in der grafischen Benutzeroberfläche Zahlen mit Nachkommastellen eingegeben, so ist auf jeden Fall darauf zu achten, dass statt eines Kommas ein Punkt der Trenner für Dezimalzahlen in SQL ist. Kommata sind hingegen die Trenner der Felder. Ein Komma in der GUI-Eingabe bringt LO 3.3.4 direkt zum Totalabsturz.

Seit der Version 3.5.3 ist dies nicht mehr der Fall. Stattdessen werden neue Abfragefelder gegründet, die die Nachkommastellen ausgeben.

Eine Eingabe mit Komma in der SQL-Ansicht führt hingegen dazu, dass ein weiteres Feld allein mit dem Zahlenwert der Nachkommastelle angezeigt wird. Dies entspricht dem Verhalten der grafischen Benutzeroberfläche in LO 3.5.3.

	ID	Medien_ID	Leser_ID	Mahnzahl	Mahnbetrag	
▶	12	3	0	2	4	
	16	7	0	1	2	
	23	1	0	1	2	
Datensatz 1 von 3						
<pre> SELECT "Ausleihe"."ID", "Ausleihe"."Medien_ID", "Ausleihe"."Leser_ID", COUNT("Mahnung"."Datum") AS "Mahnzahl", COUNT("Mahnung"."Datum") * 2 AS "Mahnbetrag" FROM "Mahnung", "Ausleihe" WHERE "Mahnung"."Ausleihe_ID" = "Ausleihe"."ID" AND "Ausleihe"."Rueck_Datum" IS NULL GROUP BY "Ausleihe"."ID", "Ausleihe"."Medien_ID", "Ausleihe"."Leser_ID" </pre>						

Die Abfrage ermittelt jetzt für jedes noch entlehene Medium anhand der herausgegebenen Mahnungen und der zusätzlich eingefügten Multiplikation die Mahngebühren. Die folgende Abfragekonstruktion hilft weiter, wenn die Gebühren für jeden Leser berechnet werden sollen:

Feld	Leser_ID	Datum	Anzahl("Mahnung"."Datum") * 2	Rueck_Datum
Alias		Mahnzahl	Mahnbetrag	
Tabelle	Ausleihe	Mahnung		Ausleihe
Sortierung				
Sichtbar	☒	☒	☒	☐
Funktion	Gruppierung	Anzahl		
Kriterium				IST LEER

Die Felder "Ausleihe"."ID" und "Ausleihe"."Medien_ID" wurden entfernt. Sie erzeugten in der vorherigen Abfrage über die Gruppierung für jedes Medium einen separaten Datensatz. Jetzt wird nur noch nach den Lesern gruppiert. Das Abfrageergebnis sieht dann so aus:

	Leser_ID	Mahnzahl	Mahnbetrag
▶	0	4	8

Statt die Medien für "Leser_ID" '0' separat aufzulisten werden alle Felder aus "Mahnung"."Datum" zusammengezählt und die Summe von 8,- € als Mahngebühr ermittelt.

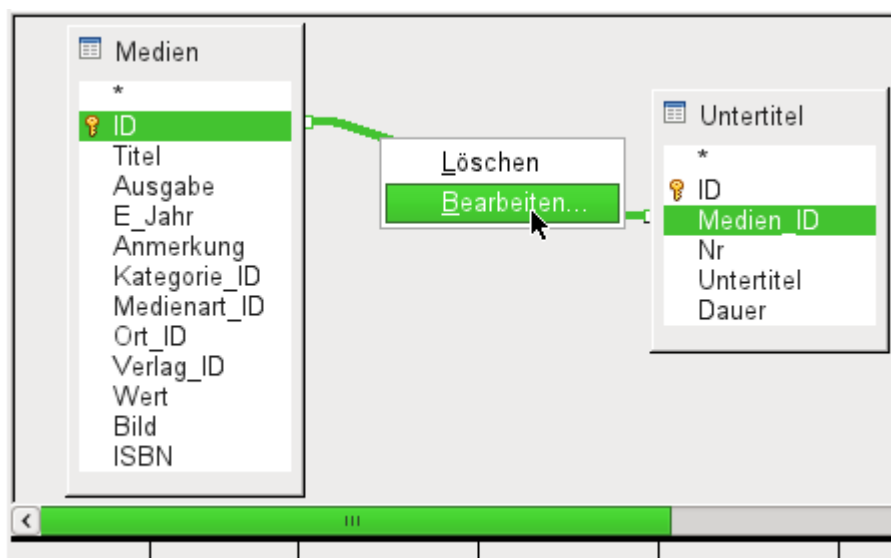
Beziehungsdefinition in der Abfrage

Werden Daten in Tabellen oder einem Formular gesucht, so beschränkt sich die Suche in der Regel auf eine Tabelle bzw. auf ein Formular. Selbst der Weg von einem Hauptformular zu einem Unterformular ist für die eingebauten Suchfunktionen nicht gangbar. Da bietet es sich dann an, zu durchsuchende Daten mit einer Abfrage zusammenzufassen.

	Titel		
▶	Der kleine Hobbit		
	Das sogenannte Böse		
	Eine kurze Geschichte der Zeit		
	Traditionelle und kritische Theorie		
	Die neue deutsche Rechtschreibung		
	I hear you knocking		
	Datenbanken mit OpenOffice.org 3		
	Das Postfix-Buch		
	Im Augenblick		
Datensatz 1 von 9			
<			
Feld	Titel		
Alias			
Tabelle	Medien		
Sortierung			
Sichtbar	☒	☐	

	Titel	Untertitel	
▶	I hear you knocking	Youn can't catch me	
	I hear you knocking	The stumble	
	I hear you knocking	Sabre dance (Single version)	
	Im Augenblick	Amsterdam	
	Im Augenblick	Hier unten am Deich	
	Im Augenblick	Köln-Ehrenfeld	
	Im Augenblick	Bei Mir	
	Im Augenblick	Gott sei Dank	
Datensatz 1 von 8			
<			
Feld	Titel	Untertitel	
Alias			
Tabelle	Medien	Untertitel	
Sortierung			
Sichtbar	☒	☒	☐

Die einfache Abfrage an die "Titel" aus der Tabelle "Medien" zeigt den eingegebenen Testbestand dieser Tabelle mit 9 Datensätzen an. Wird jedoch die Tabelle "Untertitel" mit in die Abfrage aufgenommen, so reduziert sich der Datenbestand aus der Tabelle "Medien" auf lediglich 2 "Titel". Nur für diese beiden "Titel" gibt es auch "Untertitel" in der Tabelle "Untertitel". Für alle anderen "Titel" existieren keine "Untertitel". Dies entspricht der Verknüpfungsbedingung, dass nur die Datensätze angezeigt werden sollen, bei denen in der Tabelle "Untertitel" das Feld "Medien_ID" gleich dem Feld "ID" aus der Tabelle "Medien" ist. Alle anderen Datensätze werden ausgeschlossen.



Die Verknüpfungsbedingung muss zum Bearbeiten geöffnet werden, damit alle gewünschten Datensätze angezeigt werden. Es handelt sich hier **nicht** um die Verknüpfung von Tabellen im *Relationenentwurf*, **sondern** um die Verknüpfung in einer *Abfrage*.

Verbundeigenschaften

beteiligte Tabellen

Untertitel Medien

Optionen

Typ Innerer Verbund

☐ Natürlich

beteiligte Felder

Untertitel	Medien
Medien_ID	ID

Beinhaltet nur die Datensätze, bei denen die Inhalte der verknüpften Felder beider Tabellen gleich sind.

OK Abbrechen Hilfe

Standardmäßig steht die Verknüpfung als *Innerer Verbund* zur Verfügung. Das Fenster gibt darüber Aufschluss, wie diese Form der Verknüpfung sich auswirkt.

Als beteiligte Tabellen werden die beiden vorher ausgewählten Tabellen gelistet. Sie sind hier nicht wählbar. Die beteiligten Felder der beiden Tabellen werden aus der Tabellendefinition ausgelesen. Ist eine Beziehung in der Tabellendefinition nicht vorgegeben, so kann sie hier für die Abfrage erstellt werden. Eine saubere Datenbankplanung mit der HSQLDB sieht aber so aus, dass auch an diesen Feldern nichts zu verstellen ist.

Wichtigste Einstellung ist die Option des *Verbundes*. Hier können Verknüpfungen so gewählt werden, dass alle Datensätze von der Tabelle "Untertitel" und nur die Datensätze aus "Medien" gewählt werden, die in der Tabelle "Untertitel" auch "Untertitel" verzeichnet haben.

Umgekehrt kann gewählt werden, dass auf jeden Fall alle Datensätze aus der Tabelle "Medien" angezeigt werden – unabhängig davon, ob für sie auch "Untertitel existieren.

Die Option **Natürlich** setzt voraus, dass die zu verknüpfenden Felder in den Tabellen gleich lauten. Auch von dieser Einstellung ist Abstand zu nehmen, wenn bereits zu Beginn bei der Datenbankplanung die Beziehungen definiert wurden.

Verbundeigenschaften

beteiligte Tabellen

Untertitel Medien

Optionen

Typ: **Rechter Verbund** ☐ Natürlich

beteiligte Felder

Untertitel	Medien
Medien_ID	ID

Beinhaltet ALLE Datensätze aus 'Medien' und nur die Datensätze aus 'Untertitel', bei denen die Inhalte der verknüpften Felder beider Tabellen gleich sind.
Bitte beachten Sie, dass einige Datenbanken eventuell diese Art der Verknüpfung nicht unterstützen.

OK Abbrechen Hilfe

Für den **Typ → Rechter Verbund** zeigt die Beschreibung an, dass aus der Tabelle "Medien" auf jeden Fall alle Datensätze angezeigt werden. Da es keine "Untertitel" gibt, die nicht in "Medien" mit einem "Titel" verzeichnet sind, sehr wohl aber "Titel" in "Medien", die nicht mit einem "Untertitel" versehen sind, ist dies also die richtige Wahl.

	Titel	Untertitel
▶	Der kleine Hobbit	
	Das sogenannte Böse	
	Eine kurze Geschichte der Zeit	
	Traditionelle und kritische Theorie	
	Die neue deutsche Rechtschreibung	
	I hear you knocking	Youn can't catch me
	I hear you knocking	The stumble
	I hear you knocking	Sabre dance (Single version)
	Datenbanken mit OpenOffice.org 3	
	Das Postfix-Buch	
	Im Augenblick	Amsterdam
	Im Augenblick	Hier unten am Deich
	Im Augenblick	Köln-Ehrenfeld
	Im Augenblick	Bei Mir
	Im Augenblick	Gott sei Dank
Datensatz	1	von 15

Nach Bestätigung des *rechten Verbundes* sieht das Abfrageergebnis aus wie gewünscht. "Titel" und "Untertitel" werden komplett zusammen in einer Abfrage angezeigt. Natürlich kommen jetzt "Titel" wie in der vorhergehenden Verknüpfung mehrmals vor. Solange allerdings Suchtreffer nicht gezählt werden, könnte diese Abfrage im weiteren Verlauf als Grundlage für eine Suchfunktion die-

nen. Siehe hierzu die Codeschnipsel in diesem Kapitel, im Kapitel «Makros» (S. 449 ff.)und im Kapitel «DB-Aufgaben komplett» (S. 374 ff.).

Vorsicht



Werden mehrere Tabellen über einen rechten oder linken Verbund bearbeitet, so verarbeitet die grafische Benutzeroberfläche unter LO 3.3.4 den Verbundbefehl nicht korrekt. Dies führt dazu, dass die Abfrage mit einem SQL-Fehler abgebrochen wird. Seit der Version 3.5.3 ist dies nicht mehr der Fall! Die Eingabe im SQL-Modus ist hiervon nicht berührt.

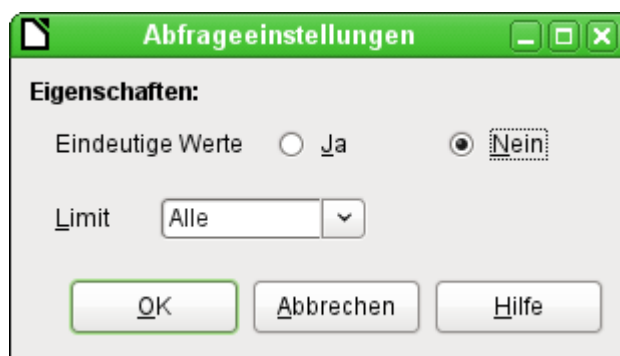
Abfrageeigenschaften definieren

Mit der Version 4.1 von LibreOffice ist es möglich, in dem Abfrageeditor zusätzliche Abfrageeigenschaften zu definieren.



Abbildung 41: Aufruf der Abfrageeigenschaften im Abfrageeditor (ab LO 4.1)

Neben dem Button zum Aufruf der Abfrageeigenschaften befindet sich noch ein Kombinationsfeld, mit dem die Anzahl der anzuzeigenden Datensätze reguliert werden kann, sowie ein Button «Eindeutige Werte». Diese Funktionen sind zusätzlich noch einmal in dem folgenden Dialog untergebracht:



Mit der Einstellung «**Eindeutige Werte**» wird beeinflusst, ob gleichlautende Datensätze in den Abfragen unterdrückt werden sollen.

	Vorname	Nachname	Rueck_Datum
▶	Lisa	Gerd	
	Lisa	Gerd	
	Lisa	Gerd	
	Bert	Lederstrumpf	
	Bert	Lederstrumpf	
	Bert	Lederstrumpf	
	Bert	Lederstrumpf	
	Bert	Lederstrumpf	
	Heinrich	Müller	
	Heinrich	Müller	
	Terence	Nobody	

In einer Abfrage soll ermittelt werden, welche Leser und Leserinnen noch Medien entliehen haben. Die Namen werden angezeigt, wenn das Rückgabedatum leer ist. Allerdings werden die Namen mehrmals angezeigt, wenn ein Leser oder eine Leserin noch mehrere Medien entliehen hat.

	Vorname	Nachname	Rueck_Datum
▶	Lisa	Gerd	
	Bert	Lederstrumpf	
	Heinrich	Müller	
	Terence	Nobody	

Wird «Eindeutige Werte» ausgewählt, so verschwinden die Datensätze mit gleichem Inhalt.

Die Abfrage sind dann so aus:

```
SELECT DISTINCT
"Leser"."Vorname", "Leser"."Nachname", "Ausleihe"."Rueck_Datum"
FROM "Ausleihe", "Leser"
WHERE "Ausleihe"."Leser_ID" = "Leser"."ID" AND "Ausleihe"."Rueck_Datum" IS NULL
ORDER BY "Leser"."Nachname" ASC
```

Der ursprünglichen Abfrage

```
SELECT "Leser"."Vorname", "Leser"."Nachname" ...
```

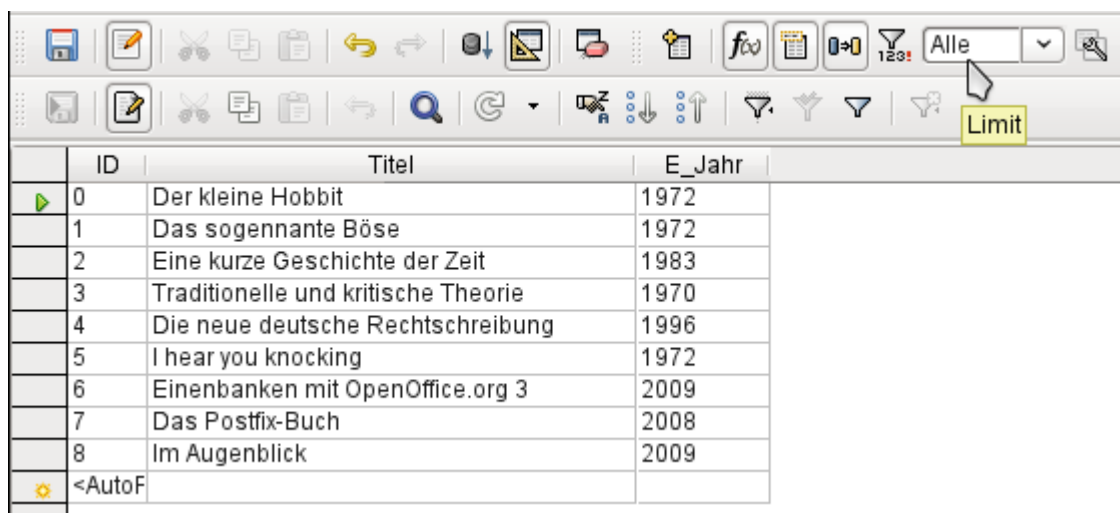
wird ein «DISTINCT» hinzugefügt:

```
SELECT DISTINCT "Leser"."Vorname", "Leser"."Nachname" ...
```

Damit werden alle gleichlautenden Zeilen unterdrückt.

Die Auswahl eindeutiger Werte war auch in den Vorversionen möglich. Allerdings musste hier von der Design-Ansicht in die SQL-Ansicht umgeschaltet werden, um den Begriff «DISTINCT» einzufügen. Diese Eigenschaft ist also ohne Probleme abwärtskompatibel zu den Vorversionen von LO.

Mit der Einstellung «Limit» (HSQLDB, FIREBIRD) wird beeinflusst, wie viele Datensätze in der Abfrage angezeigt werden sollen. Es wird also nur eine begrenzte Zahl an Datensätzen wieder gegeben.

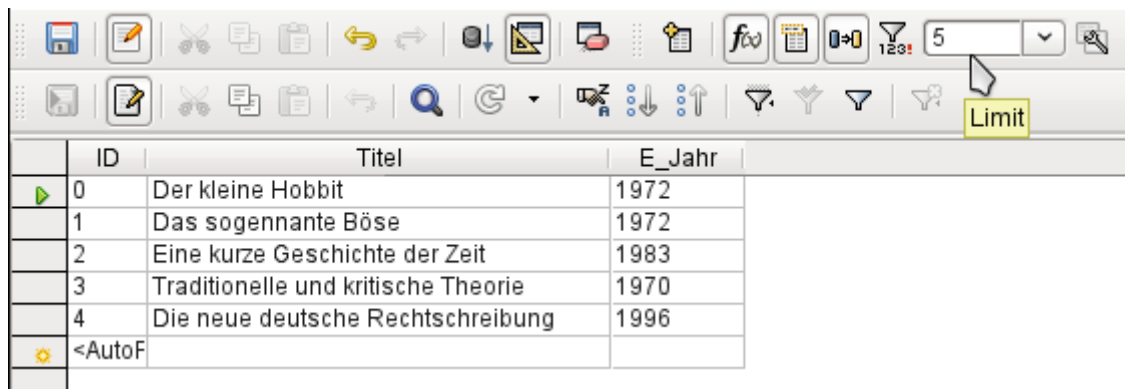


The screenshot shows a database application window. At the top is a toolbar with various icons for file operations, editing, and viewing. Below the toolbar is a table with the following data:

ID	Titel	E_Jahr
0	Der kleine Hobbit	1972
1	Das sogenannte Böse	1972
2	Eine kurze Geschichte der Zeit	1983
3	Traditionelle und kritische Theorie	1970
4	Die neue deutsche Rechtschreibung	1996
5	I hear you knocking	1972
6	Einenbanken mit OpenOffice.org 3	2009
7	Das Postfix-Buch	2008
8	Im Augenblick	2009
<AutoF		

The 'Limit' button in the toolbar is highlighted with a yellow box.

Alle Datensätze der Tabelle "Medien" werden angezeigt. Die Abfrage ist editierbar, da auch der Primärschlüssel enthalten ist.



	ID	Titel	E_Jahr
▶	0	Der kleine Hobbit	1972
	1	Das sogenannte Böse	1972
	2	Eine kurze Geschichte der Zeit	1983
	3	Traditionelle und kritische Theorie	1970
	4	Die neue deutsche Rechtschreibung	1996
⚙	<AutoF		

Nur die ersten fünf Datensätze werden angezeigt (ID 0 bis 4). Eine Sortierung wurde nicht vorgewählt. Die Standardsortierung ist hier die nach dem Primärschlüssel, sofern nichts anderes festgelegt wurde. Die Abfrage ist trotz der Begrenzung weiterhin editierbar. Dies unterscheidet die Eingabe im grafischen Modus von der, die in früheren Versionen nur mit dem direkten SQL-Modus erreichbar ist.

```
SELECT "ID", "Titel", "E_Jahr" FROM "Medien" LIMIT 5
```

Der ursprünglichen Abfrage wurde lediglich «LIMIT 5» hinzugefügt. Die entsprechende Größe des Limits kann beliebig festgelegt werden.

Vorsicht



Die Einstellung des Limits durch die grafische Benutzeroberfläche ist nicht abwärtskompatibel. In allen LO-Versionen vor der Version 4.1 konnte ein Limit nur im direkten SQL-Modus eingegeben werden. Dort erforderte das Limit eine Sortierung (**ORDER BY ...**) oder eine Bedingung (**WHERE ...**).

Abfragen nachträglich ändern

Soll eine einmal erstellte Abfrage weiter bearbeitet werden, so wird mit einem rechten Mausklick über der Abfrage das **Kontextmenü** geöffnet und **Bearbeiten...** ausgewählt. Hier kann es manchmal dazu kommen, dass die GUI die Abfrage nicht richtig deuten kann. Manchmal passiert das schon beim Öffnen, so dass die Abfrage im SQL-Modus geöffnet wird. Es kann auch sein, dass die GUI die Abfrage öffnet, aber beim Umschalten in den SQL-Modus einen «Fehler in der SQL-Syntax» findet, der gar nicht da ist. Schließlich hat die Abfrage vorher funktioniert. Hier hilft es dann, direkt aus dem **Kontextmenü** über der Abfrage **In SQL-Ansicht bearbeiten...** zu wählen.

Abfrageerweiterungen im SQL-Modus

Wird von der grafischen Eingabe über **Ansicht → Design-Ansicht an-, ausschalten** die Design-Ansicht ausgeschaltet, so erscheint der SQL-Befehl, der bisher in der Design-Ansicht erstellt wurde. Für Neueinsteiger ist dies der beste Weg, die Standardabfragesprache für Datenbanken kennen zu lernen. Manchmal ist es auch der einzige Weg, eine Abfrage an die Datenbank abzusetzen, da die GUI die Abfrage nicht in den für die Datenbank notwendigen SQL-Befehl umwandeln kann.

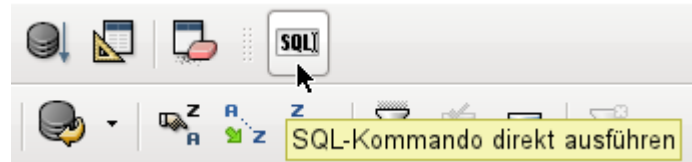
```
SELECT
*
FROM "Tabellenname"
```

Dies zeigt wirklich alles an, was in der Tabelle "Tabellenname" steht. Das «*» berücksichtigt sämtliche Felder der Tabelle.

```
SELECT
*
FROM "Tabellenname"
WHERE "Feldname" = 'Karl'
```

Eine deutliche Einschränkung wurde gemacht. Jetzt werden nur noch die Datensätze angezeigt, die in dem Feld "Feldname" den Begriff 'Karl' stehen haben – aber wirklich nur den Begriff, nicht z. B. 'Karl Egon'.

Manchmal sind Abfragen in Base nicht über die GUI ausführbar, da bestimmte Kommandos nicht bekannt sind. Hier hilft es dann die Design-Ansicht zu verlassen und über **Bearbeiten → SQL-Kommando direkt ausführen** den direkten Weg zur Datenbank zu wählen. Diese Methode hat allerdings den Nachteil, dass in dem angezeigten Abfrageergebnis keine Eingaben mehr möglich sind. Siehe hierzu [Eingabemöglichkeit in Abfragen](#).



Die direkte Ausführung ist auch über die grafische Benutzeroberfläche erreichbar. Wie in der Abbildung zu sehen muss aber auch hier die Designansicht ausgeschaltet sein. Entsprechende Abfrageanweisungen sind teilweise mit **SQL** gekennzeichnet.

Hinweis

Wird eine SQL-Anweisung als direkte Anweisung geschrieben und ist **SQL-Kommando direkt ausführen** gewählt, so bleiben sämtliche Formatierungen des SQL-Kommandos erhalten. Hier können dann auch Kommentare in den SQL-Code eingefügt werden. Zeichen hierfür sind «-- Kommentar...» für eine Kommentarzeile und «/* Kommentar ...*/» für mehrere Kommentarzeilen direkt hintereinander.

Hier jetzt also die recht umfangreichen Möglichkeiten, an die Datenbank Fragen zu stellen und auf entsprechendes Ergebnis zu hoffen:

```
SELECT [{LIMIT <offset> <limit> | TOP <limit>}][ALL | DISTINCT]
{ <Select-Formulierung> | "Tabellenname".* | * } [, ...]
[INTO [CACHE | TEMP | TEXT] "neueTabelle"]
FROM "Tabellenliste"
[WHERE SQL-Expression]
[GROUP BY SQL-Expression [, ...]]
[HAVING SQL-Expression]
[{ UNION [ALL | DISTINCT] | {MINUS [DISTINCT] | EXCEPT [DISTINCT] } |
INTERSECT [DISTINCT] } Abfrageaussage]
[ORDER BY Ordnungs-Expression [, ...]]
[LIMIT <limit> [OFFSET <offset>]];
```

[{LIMIT <offset> <limit> | TOP <limit>}]: (HSQLDB)

[{FIRST <limit> | SKIP <limit>}]: (FIREBIRD)

Hiermit wird die Menge der anzuzeigenden Datensätze begrenzt.

HSQLDB: Mit **LIMIT 10 20** werden ab dem 11. Datensatz die folgenden 20 Datensätze angezeigt. Mit **TOP 10 (HSQLDB)** werden immer die ersten 10 angezeigt. Dies ist gleichbedeutend mit **LIMIT 0 10**. **LIMIT 10 0** lässt die ersten 10 Datensätze aus und zeigt alle Datensätze ab dem 11. Datensatz an.

FIREBIRD: Mit **FIRST 10** werden die ersten 10 Datensätze angezeigt. Mit **FIRST 10 SKIP 5** die Datensätze von 6 bis 15.

Den gleichen Sinn erfüllt die zum Schluss der SELECT-Bedingung erscheinende Formulierung **[LIMIT <limit> [OFFSET <offset>]] (HSQLDB, FIREBIRD)**. **LIMIT 10** zeigt lediglich 10 Datensätze an. Wird **OFFSET 20** hinzugefügt, so beginnt die Anzeige ab dem 21. Datensatz. Für die zum Schluss stehende Begrenzung ist eine Anweisung zur Sortierung (ORDER BY ...) oder eine Bedingung (WHERE ...) Voraussetzung.

FIREBIRD: Firebird kennt **LIMIT** nicht. Hier gibt es **[ROWS <limit> TO <rownr>]**. Mit

ROWS 10 werden die ersten 10 Datensätze angezeigt. Mit **ROWS 10 TO 20** werden die Datensätze 10 bis 20 angezeigt. **rownr** muss also immer größer oder mindestens gleich **limit** sein.

Sämtliche Begrenzungen des anzuzeigenden Abfrageergebnisses sind bis einschließlich der Version LO 4.0 nur über die direkte Ausführung des SQL-Kommandos verfügbar. Erst ab LO 4.1 ist eine Limitierung ohne Sortierung oder Bedingung in der grafischen Benutzeroberfläche möglich. Dies wird sogar dann noch aufrecht erhalten, wenn in den direkten SQL-Modus umgeschaltet wurde. Die Limitierung kann in LO 4.1 in der SQL-Ansicht um den «OFFSET» ergänzt werden. Die folgende Abfrage ist also ab LO 4.1 editierbar:

```
SELECT * FROM "Tabelle" LIMIT 20 OFFSET 10      (HSQLDB)
SELECT * FROM "Tabelle" ROWS 10 TO 20          (FIREBIRD)
```

Alle Eingaben in der Limitierung können nur direkt als Ganzzahl erfolgen. Es ist nicht möglich, die Eingaben durch eine Unterabfrage zu ersetzen, so dass z. B. fortlaufend die 5 letzten Datensätze einer Datenreihe angezeigt werden können.

[ALL | DISTINCT]

SELECT ALL ist die Standardeinstellung. Es werden alle Ergebnisse angezeigt, auf die die Bedingungen zutreffen. Beispiel:

SELECT ALL "Name" FROM "Tabellenname" gibt alle Namen an; kommt «Peter» dreifach und «Egon» vierfach in der Tabelle vor, so werden die Datensätze eben dreifach oder vierfach angezeigt. **SELECT DISTINCT "Name" FROM "Tabellenname"** sorgt hingegen dafür, dass alle Abfrageergebnisse mit gleichem Inhalt unterdrückt werden. Hier würden also «Peter» und «Egon» nur einmal erscheinen. **DISTINCT** bezieht sich dabei auf den ganzen Datensatz, der in der Abfrage erfasst wird. Wird z. B. auch der Nachname erfasst, so unterscheiden sich die Datensätze mit «Peter Müller» und «Peter Maier». Sie werden also auch bei der Bedingung **DISTINCT** auf jeden Fall angezeigt.

<Select-Formulierung>

```
{ Expression | COUNT(*) |
{ COUNT | MIN | MAX | SUM | AVG | SOME | EVERY | VAR_POP | VAR_SAMP |
STDDEV_POP | STDDEV_SAMP }
([ALL | DISTINCT]] Expression) } [[AS] "anzuzeigende Bezeichnung"]
```

Feldnamen, Berechnungen, Zählen der gesamten Datensätze – alles mögliche Eingaben.

Hinweis

Berechnungen in einer Datenbank können manchmal zu erstaunlichen Ergebnissen führen. Angenommen in der Datenbank würden Zensuren einer Klassenarbeit verwaltet und mit der Berechnung sollte die Durchschnittszensur ermittelt werden.

Zuerst werden die Zensurenwerte addiert. Die Summe ergibt z. B. 80. Jetzt wird durch die Zahl der Klassenarbeiten (30) dividiert. Die Abfrage ergibt 2.

Dies liegt daran, dass es sich bei der Berechnung um eine Rechnung mit Feldern des Typs INTEGER handelt. Auch die Berechnung gibt dann nur Ergebnisse des Zahlentyps INTEGER aus. In der Berechnung muss mindestens ein Feld enthalten sein, das vom Zahlentyp DEZIMAL ist. Dies kann entweder durch Umwandlung mittels einer Funktion der HSQLDB erfolgen oder, einfacher, indem z. B. durch 30.0 dividiert wird. Dann erscheint eine Nachkommastelle, bei 30.00 zwei Nachkommastellen usw. Zu beachten ist hier, dass bei Berechnungen Dezimalzahlen mit dem in der englischen Schreibweise üblichen Dezimalpunkt dargestellt werden. Ein Komma ist bei Abfragen für die Trennung von Feldern reserviert. (FIREBIRD-Bug)

Außerdem stehen in der Felddarstellung auch verschiedene Funktionen zur Verfügung. Mit Ausnahme von **COUNT (*)** (zählt alle Datensätze) berücksichtigen die verschiedenen Funktionen keine Felder, die **NULL** sind.

COUNT | MIN | MAX | SUM | AVG | SOME | EVERY | VAR_POP | VAR_SAMP | STDDEV_POP
| STDDEV_SAMP

COUNT("Name") zählt alle Felder, die einen Namen enthalten.

FIREBIRD zusätzlich: **COUNT(DISTINCT "Name")** zählt alle unterschiedlichen Namen.

MIN("Name") zeigt den ersten Namen im Alphabet. Das Ergebnis dieser Funktion ist so formatiert, wie es dem Feldinhalt entspricht. Text wird im Ergebnis Text, Ganzzahl zu Ganzzahl, Dezimalzahl zu Dezimalzahl usw.

MAX("Name") zeigt entsprechend den letzten Namen im Alphabet.

SUM("Zahl") kann nur Werte aus Zahlenfeldern addieren. Die Funktion versagt bei Datumsfeldern.

Tipp

Auch bei Zeitfeldern versagt die Funktion. Hier kann folgendes hilfreich sein:

```
SELECT (SUM( HOUR("Zeit") ) * 3600 +  
SUM( MINUTE("Zeit") ) * 60 + SUM( SECOND("Zeit") ) ) AS  
"Sekunden" FROM "Tabelle"
```

Die Summe wird von den Stunden, Minuten und gegebenenfalls Sekunden gebildet. Anschließend wird so erweitert, dass alles zusammen in einer Maßeinheit, hier in Sekunden, wiedergegeben wird. Es werden also weiter nur Zahlen mit der Summenformel addiert. Mit

```
SELECT ((SUM( HOUR("Zeit") ) * 3600 +  
SUM( MINUTE("Zeit") ) * 60 + SUM( SECOND("Zeit") ) ) /  
3600.0000 AS "Stunden" FROM "Tabelle"
```

wird aus der Addition wieder eine Zeit in Stunden mit den Minuten und Sekunden als Nachkommastellen. Daraus kann dann über entsprechende Formatierung auch wieder eine Zeitdarstellung in einer Abfrage oder einem Formular erreicht werden.

FIREBIRD: Stunde, Minute und Sekunde müssen über die Funktion **EXTRACT()** ermittelt werden. Allerdings kennt Firebird die Funktion **DATEADD**, so dass sich hier eine Möglichkeit ergibt, direkt über die Summe von Zeiten wieder eine Zeit zu erhalten:

```
SELECT DATEADD( SUM( DATEDIFF( SECOND FROM TIME '00:00' TO  
"Zeit")) SECOND TO TIME '00:00') FROM "Tabelle"
```

Die Zeitdifferenz zur Zeit 00:00 wird mittels **DATEDIFF** in Sekunden berechnet. Die Sekunden werden mit **SUM** addiert. Zu der Zeit 00:00 wird jetzt mittels **DATEADD** die Anzahl der Sekunden addiert und wieder als Zeit formatiert.

AVG("Zahl") zeigt den Mittelwert der Inhalte einer Spalte. Auch diese Funktion beschränkt sich auf Zahlenfelder. Hier sollte darauf geachtet werden, dass bei Ganzzahlen auch nur ein Ergebnis in Ganzzahlen ermittelt wird. **AVG("Ganzzahl" * 1.00)** zeigt dann in der **HSQldb** einen Mittelwert mit maximal 2 Nachkommastellen. **FIREBIRD** hat hier leider einen Bug mit Dezimalzahlen.

FIREBIRD zusätzlich: **AVG(DISTINCT "Zahl")** erstellt den Durchschnitt aller unterschiedlichen Zahlen.

SOME("Ja_Nein"), EVERY("Ja_Nein"): **SOME** zeigt bei Ja/Nein Feldern (boolschen Feldern) die Version an, die nur einige Felder erfüllen. Da ein boolsches Feld die Werte 0 und 1 wiedergibt, erfüllen nur einige (**SOME**) die Bedingung 1, aber jeder (**EVERY**) mindestens die Bedingung 0. Über eine gesamte Tabelle abgefragt wird bei **SOME** also immer «Ja» erscheinen, wenn mindestens 1 Datensatz mit «Ja» angekreuzt ist. **EVERY** wird so lange «Nein» ergeben, bis alle Datensätze mit «Ja» angekreuzt sind. Beispiel:

```
SELECT  
    "Klasse",  
    EVERY("Schwimmer")  
FROM "Tabelle1"  
GROUP BY "Klasse";
```

Die Klassen werden alle angezeigt. Erscheint irgendwo kein Kreuz für «Ja», so muss auf jeden Fall eine Betreuung für das Nichtschwimmerbecken im Schwimmunterricht dabei sein, denn es gibt mindestens eine Person in der Klasse, die nicht schwimmen kann. (HSQLDB, FIREBIRD)

VAR_POP | **VAR_SAMP** | **STDDEV_POP** | **STDDEV_SAMP** sind statistische Funktionen und greifen nur bei Ganzzahl- und Dezimalzahlfeldern.

Alle Funktionen ergeben 0, wenn die Werte einer Gruppe alle gleich sind.

Die statistischen Funktionen erlauben nicht die Einschränkung von **DISTINCT**. Sie rechnen also grundsätzlich über alle Werte, die die Abfrage beinhaltet. **DISTINCT** hingegen würde Datensätze mit gleichen Werten von der Anzeige ausschließen.

VAR_POP: $(\text{SUM}(\text{expr}^2) - \text{SUM}(\text{expr})^2 / \text{COUNT}(\text{expr})) / \text{COUNT}(\text{expr})$. Dies gibt die Populationsabweichung eines Satzes von Zahlen zurück, nachdem die Nullwerte in diesem Satz verworfen wurden

VAR_SAMP: $(\text{SUM}(\text{expr}^2) - \text{SUM}(\text{expr})^2 / \text{COUNT}(\text{expr})) / (\text{COUNT}(\text{expr}) - 1)$. Dies gibt die Stichprobenabweichung eines Satzes von Zahlen zurück, nachdem die Nullwerte in diesem Satz verworfen wurden.

STDDEV_POP: **SQRT(VAR_POP)**. Berechnet die Populationsstandardabweichung und gibt die Quadratwurzel der Populationsabweichung zurück.

STDDEV_SAMP: **SQRT(VAR_SAMP)**. Berechnet die kumulative Stichproben-Standardabweichung und gibt die Quadratwurzel der Probenvarianz zurück.

[AS] "anzuzeigende Bezeichnung": Den Feldern kann in der Abfrage eine andere Bezeichnung (Alias) gegeben werden. **AS** ist zur Einführung eines Alias nicht erforderlich.

"Tabellenname".* | * [, ...]

Jedes anzuzeigende Feld kann mit seinem Feldnamen, getrennt durch Komma, angegeben werden. Werden Felder aus mehreren Tabellen in der Abfrage aufgeführt, so ist zusätzlich eine Kombination mit dem Tabellennamen notwendig: **"Tabellenname". "Feldname"**.

Statt einer ausführlichen Formulierung kann auch der gesamte Inhalt einer Tabelle angezeigt werden. Hierfür steht das Symbol «*».

[INTO [CACHED | TEMP | TEXT] "neueTabelle"] (HSQLDB)

Das Ergebnis dieser Abfrage soll direkt in eine **neue** Tabelle geschrieben werden. Die neue Tabelle wird hier benannt. Die Definition der Feldeigenschaften der neuen Tabelle wird dabei aus der Definition der Felder, die in der Abfrage enthalten sind, erstellt.

Das Schreiben in eine Tabelle funktioniert nicht vom Abfrageeditor aus, da dieser nur anzeigbare Ergebnisse liefert. Hier muss die Eingabe über **Extras → SQL** erfolgen. Die Tabelle, die entsteht, ist anschließend erst einmal nicht editierbar, da ein Primärschlüsselfeld fehlt.

Standardmäßig wird der Inhalt in eine Tabelle des Typs **CACHED** geschrieben, der dem Typ der anderen Tabellen entspricht. Der Typ **TEMP** ist, wie bei den Tabellen beschrieben, nur begrenzt in Base nutzbar. Der Typ **TEXT** hingegen exportiert den Inhalt der Abfrage in eine kommaseparierte Textdatei, die auch von Tabellenkalkulationsprogrammen eingelesen werden kann. Die Datei liegt anschließend in dem Verzeichnis, in dem auch die *.odb-Datei der Datenbank liegt. Außerdem wird die Datei anschließend als Ansicht in dem Tabellenordner der Datenbank angezeigt. Hierbei ist allerdings zu beachten, dass ein solcher Export standardmäßig im ASCII-Zeichensatz erfolgt, Sonderzeichen also nicht berücksichtigt werden.

Um Sonderzeichen anzeigen zu können, muss der Zeichensatz geändert werden. Dies kann entweder durch einen SQL-Befehl geschehen, der den Zeichensatz für die aktuelle Sitzung einstellt, oder aber durch eine dauerhafte Einstellung in der Datenbankdatei.

Die vorübergehende Änderung ist durch die Eingabe von

SET PROPERTY "textdb.encoding" 'UTF-8';

unter **Extras → SQL** möglich. Hier kann natürlich auch entsprechend 'ansi' gewählt werden.

Zur dauerhaften Änderung des Zeichensatzes muss die Datenbankdatei entpackt werden. Das in dieser Datei liegende Verzeichnis **database** enthält eine Datei **properties**. Diese muss um einen Eintrag erweitert werden: **textdb.encoding=UTF-8** für Linux-Systeme oder **textdb.encoding=ansi** für Windows-Systeme. Wie der Zugriff auf diese Einstellungen möglich ist, wird im Anhang im Kapitel zur *Datenbankreparatur* beschrieben.

FROM <Tabellenliste>

```
"Tabellenname 1" [{CROSS | INNER | LEFT OUTER | RIGHT OUTER} JOIN  
"Tabellenname 2" ON Expression] [, ...]
```

Die Tabellen, aus denen die Daten zusammengesucht werden sollen, werden in der Regel durch Komma getrennt aufgeführt. Die Beziehung der Tabellen zueinander wird anschließend mit dem Schlüsselwort **WHERE** definiert.

Werden die Tabellen durch einen **JOIN** statt durch ein Komma miteinander verbunden, so wird die Beziehung der Tabellen zueinander direkt nach der jeweils folgenden Tabelle mit dem Begriff **ON** beginnend definiert.

Ein einfacher **JOIN** bewirkt, dass nur die Datensätze angezeigt werden, auf die die Bedingung in beiden Tabellen zutrifft. Beispiel:

```
SELECT  
    "Tabelle1"."Name",  
    "Tabelle2"."Klasse"  
FROM "Tabelle1",  
    "Tabelle2"  
WHERE "Tabelle1"."KlasseID" = "Tabelle2"."ID"
```

entspricht von der Wirkung her

```
SELECT  
    "Tabelle1"."Name",  
    "Tabelle2"."Klasse"  
FROM "Tabelle1"  
    JOIN "Tabelle2"  
ON "Tabelle1"."KlasseID" = "Tabelle2"."ID"
```

Es werden hier die Namen und die dazugehörigen Klassen aufgelistet. Fehlt zu einem Namen eine Klasse, so wird der Name nicht aufgelistet. Fehlen zu einer Klasse Namen, so werden diese ebenfalls nicht aufgelistet. Der Zusatz **INNER** bewirkt hierbei keine Änderung.

```
SELECT  
    "Tabelle1"."Name",  
    "Tabelle2"."Klasse"  
FROM "Tabelle1"  
    LEFT JOIN "Tabelle2"  
ON "Tabelle1"."KlasseID" = "Tabelle2"."ID"
```

Bei dem Zusatz **LEFT** würden auf jeden Fall alle Inhalte von "Name" aus "Tabelle1" angezeigt – auch die, zu denen keine "Klasse" existiert. Beim Zusatz **RIGHT** hingegen würden alle Klassen angezeigt – auch die, zu denen kein Name existiert. Der Zusatz **OUTER** muss hier nicht unbedingt mit angegeben werden.

Zusätzliche Filter einzelner Tabellen sollten als **Unterabfrage** eingefügt werden:

```
SELECT  
    "Tabelle1".*,  
    "Tabelle2".*,  
    "Tabelle3".*,  
FROM "Tabelle1"  
    LEFT JOIN
```

```

    (SELECT * FROM "Tabelle2" WHERE "Name" = 'BigBoss') AS "Tabelle2"
ON "Tabelle1"."Tab2ID" = "Tabelle2"."ID"
    LEFT JOIN
    (SELECT * FROM "Tabelle3" WHERE "Ort" = 'Hintertupfingen') AS
    "Tabelle3"
ON "Tabelle2"."Tab3ID" = "Tabelle3"."ID"

```

Von "Tabelle1" werden **alle** Datensätze angezeigt. Aus "Tabelle2" werden nur die Datensätze angezeigt, die im Feld "Name" 'BigBoss' stehen haben. "Tabelle3" wird an "Tabelle2" angehängt. Es werden also in "Tabelle3" nur die Datensätze angezeigt, die mit Datensätzen von "Tabelle2" zu verbinden sind, in denen im Feld "Name" 'BigBoss' steht. Zusätzlich wird die Anzeige bei "Tabelle3" auf die Datensätze begrenzt, die den "Ort" 'Hintertupfingen' enthalten.

Wird statt einer Unterabfrage die Bedingung direkt an die Beziehungsdefinition (mit **WHERE** oder **AND**) angehängt, so werden dadurch alle Datensätze gefiltert und gegebenenfalls nicht alle Datensätze von "Tabelle1" angezeigt.

```

SELECT
    "Tabelle1"."Spieler1",
    "Tabelle2"."Spieler2"
FROM "Tabelle1" AS "Tabelle1"
    CROSS JOIN "Tabelle2" AS "Tabelle2"
WHERE "Tabelle1"."Spieler1" <> "Tabelle2"."Spieler2"

```

Beim **CROSS JOIN** müssen auf jeden Fall die Tabellen mit einem Aliasnamen versehen werden, wobei das Hinzufügen des Begriffes **AS** nicht unbedingt notwendig ist. Es werden einfach alle Datensätze aus der ersten Tabelle mit allen Datensätzen der zweiten Tabelle gekoppelt. So ergibt die obige Abfrage alle möglichen Paarungen aus der ersten Tabelle mit denen aus der zweiten Tabelle mit Ausnahme der Paarungen, bei denen es sich um gleiche Spieler handelt. Die Bedingung darf beim **CROSS JOIN** allerdings **keine Verknüpfung der Tabellen mit ON** enthalten. Stattdessen können unter **WHERE** Bedingungen eingegeben werden. Würde hier die Bedingung genauso formuliert wie beim einfachen JOIN, so wäre das Ergebnis gleich:

```

SELECT
    "Tabelle1"."Name",
    "Tabelle2"."Klasse"
FROM "Tabelle1"
    JOIN "Tabelle2"
ON "Tabelle1"."KlasseID" = "Tabelle2"."ID"

```

liefert das gleiche Ergebnis wie

```

SELECT
    "Tabelle1"."Name",
    "Tabelle2"."Klasse"
FROM "Tabelle1" AS "Tabelle1"
    CROSS JOIN "Tabelle2" AS "Tabelle2"
WHERE "Tabelle1"."KlasseID" = "Tabelle2"."ID"

```

[WHERE SQL-Expression]

Die Standardeinleitung, um Bedingungen für eine genauere Filterung der Daten zu formulieren. Hier werden in der Regel auch die Beziehungen der Tabellen zueinander definiert, sofern die Tabellen nicht mit JOIN verbunden sind.

[GROUP BY SQL-Expression [, ...]]

Wenn Felder mit einer bestimmten Funktion bearbeitet werden (z. B. **COUNT**, **SUM** ...), so sind alle Felder, die nicht mit einer Funktion bearbeitet werden, aber angezeigt werden sollen, mit **GROUP BY** zu einer Gruppe zusammen zu fassen.

Beispiel:

```
SELECT
    "Name",
    SUM("Einnahme"-"Ausgabe") AS "Saldo"
FROM "Tabelle1"
GROUP BY "Name";
```

Datensätze mit gleichen Namen werden jetzt aufsummiert. Im Ergebnis wird jeweils **Einnahme** – **Ausgabe** ermittelt und darüber die Summe, die jede Person erzielt hat, aufgelistet. Das Feld wird unter dem Namen **Saldo** dargestellt.

[HAVING SQL-Expression]

Die **HAVING**-Formulierung ähnelt sehr der **WHERE**-Formulierung. Sie kann für Bedingungen eingesetzt werden, die mit Hilfe von Funktionen wie MIN, MAX formuliert werden. **Nur HAVING ist dafür geeignet, in den Bedingungen auch Berechnungen durchzuführen.** HAVING erscheint dabei nach einer eventuell vorhandenen GROUP BY – Formulierung.

Beispiel:

```
SELECT
    "Name",
    "Laufzeit"
FROM "Tabelle1"
GROUP BY "Name",
    "Laufzeit"
HAVING MIN("Laufzeit") < '00:40:00';
```

Es werden alle Namen und Laufzeiten aufgelistet, bei denen die Laufzeit weniger als 40 Minuten beträgt.

[SQL Expression]

SQL-Ausdrücke werden nach dem folgenden Schema miteinander verbunden:

[NOT] Bedingung [{ OR | AND } Bedingung]

Beispiel:

```
SELECT
    *
FROM "Tabellenname"
WHERE
    NOT "Rückgabedatum" IS NULL
    AND "LeserID" = 2;
```

Aus der Tabelle werden die Datensätze ausgelesen, bei denen ein "Rückgabedatum" eingetragen wurde und die "LeserID" 2 lautet. Das würde in der Praxis bedeuten, dass alle Medien, die eine bestimmte Person ausgeliehen und wieder zurückgegeben hat, damit ermittelt werden könnten. Die Bedingungen sind nur durch **AND** miteinander verbunden. Das **NOT** bezieht sich rein auf die erste Bedingung.

```
SELECT
    *
FROM "Tabellenname"
WHERE NOT ("Rückgabedatum" IS NULL AND "LeserID" = 2);
```

Wird eine Klammer um die Bedingung gesetzt und **NOT** steht außerhalb der Klammer, so werden genau die Datensätze angezeigt, die die in den Klammern stehenden Bedingungen zusammen komplett nicht erfüllen. Das wären alle Datensätze mit Ausnahme derer, die "LeserID" mit der Nummer 2 noch nicht zurückgegeben hat.

[SQL Expression]: Bedingungen

{ Wert [|] Wert }

Ein Wert kann einzeln oder mit mehreren Werten zusammen über zwei senkrechte Striche || kombiniert werden. Dies gilt dann natürlich auch für Feldinhalte.

```
SELECT
    "Nachname" || ', ' || "Vorname" AS "Name"
FROM "Tabellenname"
```

Die Inhalte aus den Feldern "Nachname" und "Vorname" werden in einem Feld "Name" gemeinsam angezeigt. Dabei wird ein Komma und eine Leertaste zwischen "Nachname" und "Vorname" eingefügt.

| Wert { = | < | <= | > | >= | <> | != } Wert

Die Zeichen entsprechend den aus der Mathematik bekannten Operatoren:

{ Gleich | kleiner als | kleiner oder gleich | größer als | größer oder gleich | nicht gleich | nicht gleich }

FIREBIRD kennt hier noch die folgenden Alternativen, die allerdings nur mit direkten SQL funktionieren:

{ ~= | ^= | !> | ~> | ^> | !< | ~< | ^< }

{ Nicht gleich | nicht gleich | nicht größer als | nicht größer als | nicht größer als | nicht kleiner als | nicht kleiner als | nicht kleiner als }

| Wert IS [NOT] NULL

Das entsprechende Feld hat keinen Inhalt, ist auch nicht beschrieben worden. Dies kann in der GUI nicht unbedingt beurteilt werden, denn ein leeres Textfeld bedeutet noch nicht, dass das Feld völlig ohne Inhalt ist. Die Standardeinstellung von Base ist aber so, dass leere Felder in der Datenbank auf **NULL** gesetzt werden.

| EXISTS(Abfrageaussage)

Beispiel:

```
SELECT
    "Name"
FROM "Tabelle1"
WHERE EXISTS
    (SELECT
        "Vorname"
        FROM "Tabelle2"
        WHERE "Tabelle2"."Vorname" = "Tabelle1"."Name")
```

Es werden die Namen aus Tabelle1 aufgeführt, die als Vornamen in Tabelle2 verzeichnet sind.

| SINGULAR(Abfrageaussage) (**HSQLDB**, **FIREBIRD**)

Beispiel:

```
SELECT
    "Name"
FROM "Tabelle1"
WHERE SINGULAR
    (SELECT
        "Vorname"
        FROM "Tabelle2"
        WHERE "Tabelle2"."Vorname" = "Tabelle1"."Name")
```

Es werden die Namen aus Tabelle1 aufgeführt, die als Vornamen in Tabelle2 nur genau einmal verzeichnet sind.

| Wert BETWEEN Wert AND Wert

BETWEEN Wert1 AND Wert2 gibt alle Werte ab Wert1 bis einschließlich Wert2 wieder. Werden hier Buchstaben als Werte eingesetzt, so wird die alphabetische Sortierung angenommen, wobei Kleinbuchstaben und Großbuchstaben die gleichen Werte haben.

```
SELECT
    "Name"
FROM "Tabellenname"
WHERE "Name" BETWEEN 'A' AND 'E';
```

Diese Abfrage gibt alle Namen wieder, die mit A, B, C und D beginnen (ggf. auch mit entsprechendem Kleinbuchstaben). Da als unterer Begrenzung E gesetzt wurde, sind alle Namen mit E nicht mehr in der Auswahl enthalten. Der Buchstabe E würde in einer Sortierung ganz am Anfang der Namen mit E stehen.

| Wert [NOT] IN ({Wert [, ...] | Abfrageaussage })

Hier wird entweder eine Liste von Werten oder eine Abfrage eingesetzt. Die Bedingung ist erfüllt, wenn der Wert in der Werteliste bzw. im Abfrageergebnis enthalten ist.

| Wert [NOT] LIKE Wert [ESCAPE] Wert }

Der **LIKE**-Operator ist derjenige, der in vielen einfachen Suchfunktionen benötigt wird. Die Angabe der Werte erfolgt hier nach folgendem Muster:

'%' steht für beliebig viele, ggf. auch 0 Zeichen,

'_' ersetzt genau ein Zeichen.

Um nach '%' oder '_' selbst zu suchen müssen die Zeichen direkt nach einem zweiten Zeichen auftauchen, das nach **ESCAPE** definiert wird.

```
SELECT
    "Name"
FROM "Tabellenname"
WHERE "Name" LIKE '\_%' ESCAPE '\'
```

Diese Abfrage zeigt alle Namen auf, die mit einem Unterstrich beginnen. Als **ESCAPE**-Zeichen ist hier '\' definiert worden.

| Wert [NOT] STARTING WITH Wert } (**HSQLDB**, **FIREBIRD**)

Hier wird nur nach dem Beginn des Strings in einem Feld gesucht.

STARTING WITH 'Li' ergibt das Gleiche wie **LIKE 'Li%'**.

| Wert [NOT] CONTAINING Wert } (**HSQLDB**, **FIREBIRD**)

Ein Feld wird mit einem Wert (String) verglichen. Enthält das Feld den String oder die Zahlenkombination, so wird das Feld wiedergegeben.

CONTAINING 'Li' ergibt das Gleiche wie **LIKE '%Li%'**.

| Wert IS [NOT] DISTINCT FROM Wert } (**HSQLDB**, **FIREBIRD**)

Ein Wert ist dann **NOT DISTINCT** von einem anderen Wert, wenn er gleich ist oder wenn beide Inhalte **NULL** sind.

"Feld1" NOT DISTINCT "Feld2" ergibt das Gleiche wie **"Feld1" = "Feld2" OR ("Feld1" IS NULL AND "Feld2" IS NULL)**

| string-expression [NOT] SIMILAR TO <pattern> [ESCAPE <escape-char>]
(**HSQLDB**, **FIREBIRD**)

<pattern> ::= ein regulärer SQL-Ausdruck

<escape-char> ::= ein einzelnes Zeichen

SIMILAR TO⁸ vergleicht eine Zeichenkette, also auch den Inhalt eines Tabellenfeldes, mit einem in SQL definierten regulären Ausdruck. Anders als in einigen anderen Sprachen muss das Muster mit der gesamten Zeichenfolge übereinstimmen, um erfolgreich zu sein - die Über-

8 Übersetzung aus der Firebird 2.5 Language Reference siehe: <http://www.firebirdsql.org/en/documentation/>

einstimmung eines Teilstrings reicht nicht aus. Wenn ein Operand **NULL** ist, ist das Ergebnis **NULL**. Andernfalls ist das Ergebnis **TRUE** oder **FALSE**.

Die folgende Syntax definiert das Format des regulären SQL-Ausdrucks. Es handelt sich um eine vollständige Top-Down-Definition. Die Definition ist sehr formell, ziemlich lang und wahrscheinlich perfekt geeignet, um alle zu entmutigen, die nicht bereits einige Erfahrungen mit regulären Ausdrücken (oder mit sehr formalen, ziemlich langen Top-down-Definitionen) haben. Deshalb wird in einem weiteren Abschnitt die Erstellung regulärer Ausdrücke an Beispielen erklärt.

Syntax des regulären SQL-Ausdrucks

```
<regular expression>      ::= <regular term> ['|' <regular term> ...]
<regular term>             ::= <regular factor> ...
<regular factor>          ::= <regular primary> [<quantifier>]
<quantifier>              ::= ?
                           | *
                           | +
                           | '{' <m> [, <n>] '}'
<m>, <n>                  ::= Integer ohne Vorzeichen, mit <m> <= <n>, wenn beide
                           angegeben werden
<regular primary>         ::= <character>
                           | <character class>
                           | %
                           | (<regular expression>)
<character>              ::= <escaped character>
                           | <non-escaped character>
<escaped character>       ::= <escape-char> <special character>
                           | <escape-char> <escape-char>
<special character>      ::= eines der Zeichen [>()|^+*%_?{
<non-escaped character>  ::= Eines der Zeichen, das nicht ein <special character>
                           und nicht gleich <escape-char> ist, falls definiert
<character class>        ::= '['
                           | '[' <member> ... ']'
                           | '[' ^ <non-member> ... ']'
                           | '[' <member> ... '^' <non-member> ... ']'
<member>, <non-member>  ::= <character>
                           | <range>
                           | <predefined class>
<range>                  ::= <character>-<character>
<predefined class>       ::= ':' <predefined class name> ':'
<predefined class name>  ::= ALPHA | UPPER | LOWER | DIGIT | ALNUM | SPACE |
                           WHITESPACE
```

Erstellen regulärer Ausdrücke

Zeichen

Innerhalb regulärer Ausdrücke repräsentieren sich die meisten Zeichen selbst. Die einzigen Ausnahmen sind die folgenden Sonderzeichen:

[] () | ^ - + * % _ ? {

...und das Escape-Zeichen, wenn es definiert ist.

Ein regulärer Ausdruck, der keine Sonder- oder Escape-Zeichen enthält, stimmt nur mit identischen Strings überein (abhängig von dem verwendeten Zeichensatz). Das heißt, es funktioniert genau wie der "=" - Operator:

```
'Apple' similar to 'Apple'      -- true
'Apples' similar to 'Apple'    -- false
'Apple' similar to 'Apples'    -- false
'APPLE' similar to 'Apple'     -- abhängig vom verwendeten Zeichensatz
```

Wildcards

Die bekannten SQL-Wildcards entsprechen einem einzelnen Zeichen (`_`) und einem String jeder beliebigen Länge (`%`):

```
'Birne' similar to 'B_rne'      -- true
'Birne' similar to 'B_ne'      -- false
'Birne' similar to 'B%ne'      -- true
'Birne' similar to 'Bir%ne%'   -- true
'Birne' similar to 'Birr%ne'   -- false
```

`%` kann auch für einen leeren String stehen.

Zeichenklassen

Ein Sammlung von Zeichen, die in Klammern eingeschlossen sind, definiert eine Zeichenklasse. Ein Zeichen in dem String entspricht einer Klasse im Muster, wenn das Zeichen in der Klasse enthalten ist:

```
'Citroen' similar to 'Cit[arju]oen' -- true
'Citroen' similar to 'Ci[tr]oen'    -- false
'Citroen' similar to 'Ci[tr][tr]oen' -- true
```

Wie aus der zweiten Zeile ersichtlich ist, entspricht die Klasse nur einem einzelnen Zeichen, nicht mehreren Zeichen hintereinander.

Innerhalb einer Klassendefinition definieren zwei Zeichen, die durch einen Bindestrich verbunden sind, einen Bereich. Ein Bereich umfasst die beiden Endpunkte und alle Zeichen, die zwischen ihnen in der aktiven Sortierung liegen. Bereiche können an beliebiger Stelle in der Klassendefinition platziert werden, ohne dass spezielle Trennzeichen vorhanden sind, um sie von den anderen Elementen getrennt zu halten.

```
'Datte' similar to 'Dat[q-u]e'      -- true
'Datte' similar to 'Dat[abq-uy]e'    -- true
'Datte' similar to 'Dat[bcg-km-pwz]e' -- false
```

Die folgenden vordefinierten Zeichenklassen können auch in einer Klassendefinition verwendet werden:

[:ALPHA:]

Buchstaben a..z und A..Z. Abhängig vom Zeichensatz der Datenbank enthält dies auch entsprechende Sonderzeichen.

[:DIGIT:]

Dezimalziffern 0..9.

[:ALNUM:]

Zusammenschluss von **[:ALPHA:]** und **[:DIGIT:]**.

[:UPPER:]

Großgeschriebene Buchstaben A..Z. Abhängig vom Zeichensatz auch Kleinbuchstaben wie z.B. 'ß'.

[:LOWER:]

Kleingeschriebene Buchstaben a..z. Abhängig vom Zeichensatz gegebenenfalls auch Großbuchstaben.

[:SPACE:]

Leerzeichen (ASCII 32).

[:WHITESPACE:]

Vertikaler Tabulator (ASCII 9), Zeilenvorschub (ASCII 10), horizontaler Tabulator (ASCII 11), Seitenvorschub (ASCII 12), Wagenrücklauf (ASCII 13) und Leerzeichen (ASCII 32).

Der Einschluss einer vordefinierten Klasse hat die gleiche Wirkung wie die Aufzählung aller ihrer Mitglieder. Vordefinierte Klassen sind nur innerhalb von Klassendefinitionen zulässig. Wenn nur mit einer vordefinierten Klasse verglichen werden soll, so muss ein zusätzliches Paar von Klammern um die vordefinierte Klasse gelegt werden:

```
'Erdbeere' similar to 'Erd[[ :ALNUM: ]]eere'      -- true
'Erdbeere' similar to 'Erd[[ :DIGIT: ]]eere'      -- false
'Erdbeere' similar to 'Erd[a[:SPACE:]]beere'      -- true
'Erdbeere' similar to [[ :ALPHA: ]]              -- false
'E' similar to [[ :ALPHA: ]]                      -- true
```

Wenn eine Klassendefinition mit einem Caret '^' beginnt, darf alles, was folgt, nicht im String an der Stelle existieren:

```
'Framboise' similar to 'Fra[^ck-p]boise'      -- false
'Framboise' similar to 'Fr[^a][^a]boise'      -- false
'Framboise' similar to 'Fra[^[:DIGIT:]]boise'  -- true
```

Wenn das Caret nicht am Anfang der Sequenz platziert wird, enthält die Klasse alles vor dem Caret, mit Ausnahme der Elemente, die auch nach dem Caret auftreten:

```
'Grapefruit' similar to 'Grap[a-m^f-i]fruit'  -- true
'Grapefruit' similar to 'Grap[abc^xyz]fruit'  -- false
'Grapefruit' similar to 'Grap[abc^de]fruit'   -- false
'Grapefruit' similar to 'Grap[abe^de]fruit'   -- false
'3' similar to '[[[:DIGIT:]]^4-8]'            -- true
'6' similar to '[[[:DIGIT:]]^4-8]'            -- false
```

Schließlich ist die bereits erwähnte Wildcard "_" eine eigenständige Zeichenklasse, die mit jedem einzelnen Zeichen übereinstimmt.

Quantoren

Ein Fragezeichen unmittelbar nach einem Zeichen oder einer Klasse weist darauf hin, dass das vorhergehende Element 0 oder 1 Mal auftreten kann:

```
'Hallon' similar to 'Hal?on'                  -- false
'Hallon' similar to 'Hal?lon'                 -- true
'Hallon' similar to 'Hall?lon'                -- true
'Hallon' similar to 'Hall?llon'               -- false
'Hallon' similar to 'Halx?lon'                -- true
'Hallon' similar to 'H[a-c]?llon[x-z]?'      -- true
```

Ein Stern "*", der unmittelbar einem Zeichen oder einer Klasse folgt, zeigt an, dass das vorhergehende Element 0 oder mehrere Male auftreten kann:

```
'Icaque' similar to 'Ica*que'                 -- true
'Icaque' similar to 'Icar*que'                -- true
'Icaque' similar to 'I[a-c]*que'              -- true
'Icaque' similar to 'I_*'                    -- true
'Icaque' similar to 'I[[:ALPHA:]]*'           -- true
'Icaque' similar to 'Ica[xyz]*e'              -- false
```

Ein Pluszeichen unmittelbar nach einem Zeichen oder einer Klasse weist darauf hin, dass das vorhergehende Element 1 oder mehrere Male auftreten muss:

```
'Jujube' similar to 'Ju_+'                   -- true
'Jujube' similar to 'Ju+jube'                 -- true
'Jujube' similar to 'Jujuber+'                -- false
'Jujube' similar to 'J[jux]+be'               -- true
'Jujube' similar to 'J[[[:DIGIT:]]+ujube'     -- false
```

Wenn einem Zeichen oder einer Klasse eine in geschweiften Klammern eingeschlossene Zahl folgt, muss die Überprüfung genau so oft wiederholt werden:

```
'Kiwi' similar to 'Ki{2}wi'                  -- false
'Kiwi' similar to 'K[ipw]{2}i'               -- true
'Kiwi' similar to 'K[ipw]{2}'                -- false
'Kiwi' similar to 'K[ipw]{3}'                -- true
```

Wenn der Zahl ein Komma folgt, muss das Element mindestens so oft wiederholt werden:

```
'Limon' similar to 'Li{2,}mon'               -- false
'Limon' similar to 'Li{1,}mon'               -- true
'Limon' similar to 'Li[nezm]{2,}'            -- true
```

Wenn die geschweiften Klammern zwei Zahlen enthalten, die durch ein Komma getrennt sind, und die zweite Zahl nicht kleiner als die erste ist, dann muss das Element mindestens so oft wie die erste Zahl und höchstens so oft die zweite Zahl wiederholt werden:

```
'Mandarijn' similar to 'M[a-p]{2,5}rijn'    -- true
'Mandarijn' similar to 'M[a-p]{2,3}rijn'    -- false
'Mandarijn' similar to 'M[a-p]{2,3}arijn'    -- true
```

Die Quantoren ?, * und + entsprechen jeweils Kurzformen von {0,1}, {0,} und {1,}.

ODER-Ausdrücke

Reguläre Ausdrücke können mit dem '|' -Operator erzeugt werden. Eine Übereinstimmung ist dann gegeben, wenn der Argumentstring mit mindestens einem der Begriffe übereinstimmt:

```
'Nektarin' similar to 'Nek|tarin'          -- false
'Nektarin' similar to 'Nektarin|Persika'    -- true
'Nektarin' similar to 'M_+|N_+|P_+'        -- true
```

Unterausdrücke

Ein oder mehrere Teile des regulären Ausdrucks können in Unterausdrücken (auch als Unter-muster bezeichnet) gruppiert werden, indem sie zwischen Klammern platziert werden. Ein Unterausdruck ist ein regulärer Ausdruck in seinem eigenen Bereich. Er kann alle Elemente enthalten, die in einem regulären Ausdruck erlaubt sind. Es können auch Quantifizierer hinzugefügt werden:

```
'Orange' similar to 'O(ra|ri|ro)nge'      -- true
'Orange' similar to 'O(r[a-e])+nge'        -- true
'Orange' similar to 'O(ra){2,4}nge'        -- false
'Orange' similar to 'O(r(an|in)g|rong)?e'  -- true
```

Maskieren von Sonderzeichen

Um einem Zeichen zu entsprechen, das in regulären Ausdrücken enthalten ist, muss dieses Zeichen maskiert werden. Es gibt kein Standard-Zeichen zur Maskierung (Escape-Zeichen). Das Escape-Zeichen legt der Benutzer bei Bedarf fest:

```
'Peer (Poire)' similar to 'P[^ ]+ \ (P[^ ]+ \)' escape '\ ' -- true
'Pera [Pear]' similar to 'P[^ ]+ # [P[^ ]+ #]' escape '#'   -- true
'Päron-Äppledryck' similar to 'P%$-Ä%' escape '$'          -- true
'Pärondryck' similar to 'P%-Ä%' escape '-'                  -- false
```

Die letzte Zeile zeigt, dass das Escape-Zeichen bei Bedarf auch zur Maskierung des eigenen Zeichens genutzt werden kann.

Hinweis

Datumswerte können bei der internen Datenbank im Format 'YYYY-MM-DD' angegeben werden. Bei externen Datenbanken kann es aber passieren, dass das Datum unter diesen Umständen nicht korrekt gelesen werden kann. Hier kann die ältere Version {D 'YYYY-MM-DD'} oder die neuere Version {d 'YYYY-MM-DD'} zum Erfolg führen.

Entsprechende Formate existieren auch für Zeitfelder und für Datums-Zeit-Felder: {t 'HH:MI:SS[.SS]'} bzw. {ts 'YYYY-MM-DD HH:MI:SS[.SS]'}

Entsprechend sind die Buchstaben «D» und «T» sowie die Kombination «TS» reservierte Abkürzungen, die nicht an anderer Stelle alleinstehend benutzt werden können. Eine Parameterabfrage mit dem Parameter «:D» wird z.B. nicht funktionieren.

Siehe hierzu auch die Hilfe von LO zum Stichwort «Abfrageentwurf».

[SQL Expression]: Werte

[+ | -] { Ausdruck [{ + | - | * | / | || } Ausdruck]

Vorzeichen vor den Werten sind möglich. Die Addition, Subtraktion, Multiplikation, Division und Verkettung von Ausdrücken ist erlaubt. Beispiel für eine Verkettung:

```
SELECT
  "Nachname" || ', ' || "Vorname"
FROM "Tabelle"
```

Auf diese Art und Weise werden Datensätze in der Abfrage als ein Feld ausgegeben, in der "Nachname, Vorname" steht. Die Verkettung erlaubt also jeden der weiter unten genannten Ausdrücke.

| (Bedingung)

Siehe hierzu den vorhergehenden Abschnitt

| Funktion ([Parameter] [,...])

Siehe hierzu im Anhang das Kapitel «*Eingebaute Funktionen und abgespeicherte Prozeduren*».

Die folgenden Abfragen werden auch als Unterabfragen (Subselects) bezeichnet.

| Abfrageaussage, die nur genau einen Wert ergibt

Da ein Datensatz für jedes Feld nur einen Wert darstellen kann, kann auch nur die Abfrage komplett angezeigt werden, die genau einen Wert ergibt.

| {ANY|ALL} (Abfrageaussage, die den Inhalt einer ganzen Spalte wiedergibt)

Manchmal gibt es Bedingungen, bei denen ein Ausdruck mit einer ganzen Gruppe von Werten verglichen wird. Zusammen mit **ANY** bedeutet das, dass der Ausdruck mindestens einmal in der Gruppe vorkommen muss. Dies ließe sich auch mit der **IN**-Bedingung beschreiben. **= ANY** ergibt das gleiche Ergebnis wie **IN**., funktioniert aber nur mit einer Unterabfrage.

Statt **ANY** kann in Firebird alternativ auch **SOME** genutzt werden.

Zusammen mit **ALL** bedeutet dies, dass alle Werte der Gruppe dem einen Ausdruck entsprechen müssen.

[SQL Expression]: Ausdrücke

```
{ 'Text' | Ganzzahl | Fließkommazahl  
| ["Tabelle"."Feld" | TRUE | FALSE | NULL ] }
```

Als Grundlage dienen Werte, die, abhängig vom Quellformat, mit unterschiedlichen Ausdrücken angegeben werden. Wird nach Textinhalten gesucht, so ist der Inhalt in Hochkommata zu setzen. Ganzzahlen werden ohne Hochkommata geschrieben, ebenso Fließkommazahlen (statt Komma ist in SQL direkt der Dezimalpunkt zu setzen).

Felder stehen für die Werte, die sich in den Feldern einer Tabelle befinden. Meist werden entweder Felder miteinander verglichen oder Felder mit Werten. In SQL werden die Feldbezeichnungen besser in doppelte Anführungsstriche gesetzt, da sonst eventuell Feldbezeichnungen nicht richtig erkannt werden. Üblicherweise geht SQL ohne doppelte Anführungsstriche davon aus, dass alles ohne Sonderzeichen, in einem Wort und mit Großbuchstaben geschrieben ist. Sind mehrere Tabellen in der Abfrage enthalten, so ist neben dem Feld auch die Tabelle, vom Feld getrennt durch einen Punkt, aufzuführen.

TRUE und **FALSE** stammen üblicherweise von Ja/Nein-Feldern.

NULL bedeutet, dass nichts angegeben wurde. Es ist nicht gleichbedeutend mit 0, sondern eher mit Leer.

UNION [ALL | DISTINCT] Abfrageaussage SQL GUI

Hiermit werden Abfragen so verknüpft, dass der Inhalt der 2. Abfrage unter die erste Abfrage geschrieben wird. Dazu müssen alle Felder der beiden Abfragen vom Typ her übereinstimmen. Diese Verknüpfung von mehreren Abfragen funktioniert nur über die direkte Ausführung des SQL-Kommandos.

```
SELECT  
    "Vorname"  
FROM "Tabelle1"  
    UNION DISTINCT  
    SELECT  
        "Vorname"  
    FROM "Tabelle2";
```

Diese Abfrage liefert alle Vornamen aus Tabelle1 und Tabelle2; der Zusatz **DISTINCT** zeigt an, dass keine doppelten Vornamen ausgegeben werden. **DISTINCT** ist in diesem Zusammenhang die Standardeinstellung. Die Vornamen sind dabei standardmäßig nach dem Alphabet

aufsteigend sortiert. Mit **ALL** werden einfach alle Vornamen der Tabelle1 angezeigt gefolgt von den Vornamen Tabelle2. Sie sind jetzt standardmäßig nach dem Primärschlüssel sortiert.

Mit Hilfe dieser Abfragetechnik ist es auch möglich, Werte einer Datenzeile z.B. für eine Liste untereinander in einer Spalte anzuordnen. Angenommen es existiert eine Tabelle "Ware", in der der "Verkaufspreis" sowie ein "Rabattpreis_1" und ein "Rabattpreis_2" enthalten sind. Daraus soll der Inhalt für ein Kombinationsfeld erstellt werden, das genau diese Preise untereinander auflistet:

```
SELECT
    "Verkaufspreis"
FROM "Ware" WHERE "Ware_ID" = 1
UNION
SELECT
    "Rabattpreis_1"
FROM "Ware" WHERE "Ware_ID" = 1
UNION
SELECT
    "Rabattpreis_2"
FROM "Ware" WHERE "Ware_ID" = 1;
```

Der Primärschlüsselwert für die Ware müsste hier natürlich entsprechend über ein Makro gesetzt werden, das dem Kombinationsfeld je nach Ware einen entsprechenden Inhalt zuweist.

MINUS [DISTINCT] | EXCEPT [DISTINCT] Abfrageaussage SQL GUI (HSQLDB, FIREBIRD)

```
SELECT
    "Vorname"
FROM "Tabelle1"
EXCEPT
SELECT
    "Vorname"
FROM "Tabelle2";
```

Zeigt alle Vornamen aus Tabelle1 mit Ausnahme der Vornamen an, die in Tabelle 2 enthalten sind. **MINUS** und **EXCEPT** führen zum gleichen Ergebnis. Sortierung ist alphabetisch. Dies funktioniert zur Zeit nur, wenn das SQL-Kommando direkt ausgeführt wird.

INTERSECT [DISTINCT] Abfrageaussage SQL GUI (HSQLDB, FIREBIRD)

```
SELECT
    "Vorname"
FROM "Tabelle1"
INTERSECT
SELECT
    "Vorname"
FROM "Tabelle2";
```

Hier werden nur die Vornamen angezeigt, die in beiden Tabellen vorhanden sind. Die Sortierung ist wieder alphabetisch. Dies funktioniert zur Zeit nur, wenn das SQL-Kommando direkt ausgeführt wird.

[ORDER BY Ordnungs-Expression [, ...]]

Hier können Feldnamen, die Nummer der Spalte (beginnend mit 1 von links), ein Alias (formuliert z. B. mit **AS**) oder eine Wertzusammenführung (siehe [SQL Expression]: Werte) angegeben werden. Die Sortierung erfolgt in der Regel aufsteigend (**ASC**). Nur wenn die Sortierung absteigend erfolgen soll, muss **DESC** angegeben werden.

```
SELECT
    "Vorname",
    "Nachname" AS "Name"
```

```
FROM "Tabelle1"  
ORDER BY "Nachname";
```

ist identisch mit

```
SELECT  
    "Vorname",  
    "Nachname" AS "Name"  
FROM "Tabelle1"  
ORDER BY "Nachname" ASC;
```

ist identisch mit

```
SELECT  
    "Vorname",  
    "Nachname" AS "Name"  
FROM "Tabelle1"  
ORDER BY "Name";
```

Verwendung eines Alias in Abfragen

Durch Abfragen können auch Felder in einer anderen Bezeichnung wiedergegeben werden.

```
SELECT  
    "Vorname",  
    "Nachname" AS "Name"  
FROM "Tabelle1"
```

Dem Feld *Nachname* wird in der Anzeige die Bezeichnung *Name* zugeordnet.

Wird eine Abfrage aus zwei Tabellen erstellt, so steht vor den jeweiligen Feldbezeichnungen der Name der Tabelle:

```
SELECT  
    "Tabelle1"."Vorname",  
    "Tabelle1"."Nachname" AS "Name",  
    "Tabelle2"."Klasse"  
FROM "Tabelle1",  
    "Tabelle2"  
WHERE "Tabelle1"."Klasse_ID" = "Tabelle2"."ID"
```

Auch den Tabellennamen kann ein Aliasname zugeordnet werden, der allerdings in der Tabellenansicht nicht weiter erscheint. Wird so ein Alias zugeordnet, so müssen sämtliche Tabellenbezeichnungen in der Abfrage entsprechend ausgetauscht werden:

```
SELECT  
    "a"."Vorname",  
    "a"."Nachname" AS "Name",  
    "b"."Klasse"  
FROM "Tabelle1" AS "a",  
    "Tabelle2" AS "b"  
WHERE "a"."Klasse_ID" = "b"."ID"
```

Die Zuweisung eines Aliasnamens kann auch verkürzt ohne den Zuweisungsbegriff **AS** erfolgen:

```
SELECT  
    "a"."Vorname",  
    "a"."Nachname" "Name",  
    "b"."Klasse"  
FROM "Tabelle1" "a",  
    "Tabelle2" "b"  
WHERE "a"."Klasse_ID" = "b"."ID"
```

Dies erschwert allerdings die eindeutige Lesbarkeit des Codes. Daher sollte nur in Ausnahmefällen auf die Verkürzung zurückgegriffen werden.

Hinweis

Leider ist in der grafischen Benutzeroberfläche des Abfrageeditors seit einiger Zeit der Code so geändert worden, dass Alias-Bezeichnungen grundsätzlich ohne den Zusatz **AS** erstellt werden. Das liegt daran, dass bei der Verwendung von externen Datenbanken, bei denen diese Art der Alias-Zuweisung nicht vorgesehen war, der Zusatz zu Fehlermeldungen führte.

Wenn der Code, der zum Bearbeiten geöffnet wird, nicht direkt in der SQL-Ansicht, sondern über die GUI geöffnet wird, verlieren bereits erstellte Alias-Bezeichnungen hierbei die Zuweisung **AS**. Wer Wert auf diese Bezeichnung legt, sollte also Abfragen immer über **In SQL-Ansicht bearbeiten** öffnen.

Über den Aliasnamen kann auch eine Tabelle mit entsprechenden Filterungen mehrmals innerhalb einer Abfrage genutzt werden:

```
SELECT "Kasse"."Betrag", "Kasse"."Datum",
       "a"."Betrag" AS "Haben",
       "b"."Betrag" AS "Soll"
FROM "Kasse"
     LEFT JOIN "Kasse" AS "a"
       ON "Kasse"."ID" = "a"."ID" AND "a"."Betrag" >= 0
     LEFT JOIN "Kasse" AS "b"
       ON "Kasse"."ID" = "b"."ID" AND "b"."Betrag" < 0
```

Abfragen für die Erstellung von Listenfeldern

In Listenfeldern wird ein Wert angezeigt, der nicht an die den Formularen zugrundeliegenden Tabellen weitergegeben wird. Sie werden schließlich eingesetzt, um statt eines Fremdschlüssels zu sehen, welchen Wert der Nutzer damit verbindet. Der Wert, der schließlich in Formularen abgespeichert wird, darf bei den Listenfeldern nicht an der ersten Position stehen.

```
SELECT
    "Vorname",
    "ID"
FROM "Tabelle1";
```

Diese Abfrage würde alle Vornamen anzeigen und den Primärschlüssel "ID" an die dem Formular zugrundeliegende Tabelle weitergeben. So ist das natürlich noch nicht optimal. Die Vornamen erscheinen unsortiert; bei gleichen Vornamen ist außerdem unbekannt, um welche Person es sich denn handelt.

```
SELECT
    "Vorname" || ' ' || "Nachname",
    "ID"
FROM "Tabelle1"
ORDER BY "Vorname" || ' ' || "Nachname";
```

Jetzt erscheint der Vorname, getrennt von dem Nachnamen, durch ein Leerzeichen. Die Namen werden unterscheidbarer und sind sortiert. Die Sortierung folgt der Logik, dass natürlich nach den vorne stehenden Buchstaben zuerst sortiert wird, also Vornamen zuerst, dann Nachnamen. Andere Sortierreihenfolgen als nach der Anzeige des Feldes würden erst einmal verwirren.

```
SELECT
    "Nachname" || ', ' || "Vorname",
    "ID"
FROM "Tabelle1"
ORDER BY "Nachname" || ', ' || "Vorname";
```

Dies würde jetzt zu der entsprechenden Sortierung führen, die eher unserer Gewohnheit entspricht. Familienmitglieder erscheinen zusammen untereinander, bei gleichen Nachnamen und unterschiedlichen Familien wird allerdings noch durcheinander gewürfelt. Um dies zu unterscheiden müsste eine Gruppenzuordnung in der Tabelle gemacht werden.

Letztes Problem ist noch, dass eventuell auftauchende Personen mit gleichem Nachnamen und gleichem Vornamen nicht unterscheidbar sind. Variante 1 wäre, einfach einen Namenszusatz zu entwerfen. Aber wie sieht das denn aus, wenn ein Anschreiben mit Herr «Müller II» erstellt wird?

```
SELECT
    "Nachname" || ', ' || "Vorname" || ' - ID: ' || "ID",
    "ID"
FROM "Tabelle1"
ORDER BY "Nachname" || ', ' || "Vorname" || "ID";
```

Hier wird auf jeden Fall jeder Datensatz unterscheidbar. Angezeigt wird «Nachname, Vorname – ID:IDWert».

Falls es einmal passieren sollte, dass Felder für die Vornamen leer (**NULL**) sind, so wird so ein Datensatz in den Listefeldern natürlich erst einmal nicht angezeigt. Leere Felder, verbunden mit anderen Feldern, sind ebenfalls **NULL**. Hier hilft dann eine entsprechend eingefügte Bedingung:

```
SELECT
    "Nachname" || COALESCE(', ' || "Vorname", ''),
    "ID"
FROM "Tabelle1"
ORDER BY "Nachname" || COALESCE(', ' || "Vorname", '');
```

Damit wird das Komma und der anschließende Vorname durch einen leeren Text ersetzt, wenn das Feld "Vorname" **NULL** ist.

Im Formular Ausleihe wurde ein Listefeld erstellt, das lediglich die Medien darstellen sollte, die noch nicht entliehen wurden. Dies wird über die folgende SQL-Formulierung möglich:

```
SELECT
    "Titel" || ' - Nr. ' || "ID",
    "ID"
FROM "Medien"
WHERE "ID"
    NOT IN
        (SELECT
            "Medien_ID"
            FROM "Ausleihe"
            WHERE "Rueck_Datum" IS NULL)
ORDER BY "Titel" || ' - Nr. ' || "ID" ASC
```

Dieses Listefeld muss allerdings immer dann aktualisiert werden, wenn eine Ausleihe eines darin verzeichneten Mediums erfolgt ist.

Das folgende Listefeld stellt die Inhalte mehrerer Felder in Tabellenform dar, so dass zusammengehörige Elemente direkt untereinander erscheinen:

Anzahl	Ware	
2	Papier, 500 Blatt	- 5.65 €
10	Bleistift HB	- 0.25 €
5	Schnellhefter, Pappe	- 0.46 €
1	Hefter, Tischgerät	- 11.25 €
1	Locher, Registratur	- 15.48 €
	Bleistift HB	- 0.25 €
	Hefter, Tischgerät	- 11.25 €
	Locher, Registratur	- 15.48 €
	Papier, 500 Blatt	- 5.65 €
	Schnellhefter, Pappe	- 0.46 €

Datensatz 5 von 5

Damit so eine Darstellung gelingt, muss zuerst einmal eine entsprechende nicht proportionale Schrift ausgewählt werden. Courier oder alle Mono-Schriftarten wie z.B. Liberation Mono können hier genutzt werden. Die Darstellung in tabellarischer Form gelingt über den SQL-Code ([HSQLDB](#))

```
SELECT
    LEFT("Ware" || SPACE(25), 25) || ' - ' ||
    RIGHT(SPACE(8) || "Preis", 8) || ' €',
    "ID"
FROM "Waren"
ORDER BY ("Ware" || ' - ' || "Preis" || ' €') ASC
```

FIREBIRD kennt die Funktion **SPACE** nicht, dafür aber Funktionen wie **LPAD** und **RPAD**:

LEFT("Ware" || SPACE(25), 25) wird ersetzt durch **RPAD("Ware", 25)**
RIGHT(SPACE(8) || "Preis", 8) wird ersetzt durch **LPAD("Preis", 8)**

An den Inhalt des Feldes "Ware" werden entsprechend viele Leerzeichen angehängt, so dass "Ware" zusammen mit den Leerzeichen eine Mindestlänge von 25 Zeichen hat. Anschließend werden die ersten 25 Buchstaben dargestellt, die überflüssigen Leerzeichen also abgeschnitten.

Komplizierter wird es, wenn in dem Inhalt des Listenfeldes auch nicht druckbare Zeichen wie z.B. Zeilenumbrüche enthalten sind. Dann muss der Code entsprechend angepasst werden:

```
SELECT
    LEFT(REPLACE("Ware", CHAR(10), ' ') || SPACE(25), 25) || ' - ' || ...
```

Damit wird ein Zeilenvorschub in Linux durch ein Leerzeichen ersetzt. In Windows muss zusätzlich noch der Wagenrücklauf (**CHAR(13)**) entfernt werden.

FIREBIRD: Die Funktion **CHAR()** heißt dort **ASCII_Char()**.

Die Anzahl der notwendigen Leerzeichen kann übrigens auch per Abfrage ermittelt werden. So wird vermieden, dass doch einmal ein Wert aus "Ware" in seiner Länge beschnitten wird.

```
SELECT
    LEFT("Ware" || SPACE((SELECT MAX(LENGTH("Ware")) FROM "Waren")),
    (SELECT MAX(LENGTH("Ware")) FROM "Waren")) || ' - ' ||
    RIGHT(' ' || "Preis", 8) || ' €',
    "ID"
FROM "Waren"
ORDER BY ("Ware" || ' - ' || "Preis" || ' €') ASC
```

Da der Preis rechtsbündig dargestellt werden soll, wird hier vor dem Preis mit Leerzeichen aufgefüllt und entsprechend maximal 8 Zeichen von rechts aus dargestellt. Die angestrebte Darstellung reicht also für alle Preise bis 99999,99 € aus.

Soll auch noch der Punkt aus der SQL-Darstellung durch ein Komma ersetzt werden, so ist der SQL-Code entsprechend zu ergänzen. Dies wäre über

```
REPLACE(RIGHT(' ' || "Preis", 8), '.', ',')
```

möglich.

Abfragen als Grundlage von Zusatzinformationen in Formularen

Will man zusätzliche Informationen im Formular im Auge haben, die so gar nicht sichtbar wären, so bieten sich verschiedene Abfragemöglichkeiten an. Die einfachste ist, mit gesonderten Abfragen diese Informationen zu ermitteln und in andere Formulare einzustellen. Der Nachteil dieser Variante kann sein, dass sich Datenänderungen auf das Abfrageergebnis auswirken würden, aber leider die Änderungen nicht automatisch angezeigt werden.

Hier ein Beispiel aus dem Bereich der Warenwirtschaft für eine einfache Kasse:

Die Tabelle für eine Kasse enthält Anzahl und Fremdschlüssel von Waren, außerdem eine Rechnungsnummer. Besonders wenig Informationen erhält der Nutzer an der Supermarktkasse, wenn keine zusätzlichen Abfrageergebnisse auf den Kassenzettel gedruckt werden. Schließlich werden die Waren nur über den Barcode eingelesen. Ohne Abfrage ist im Formular nur zusehen:

Anzahl	Barcode
3	17
2	24

usw.

Was sich hinter den Nummern versteckt, kann nicht mit einem Listenfeld sichtbar gemacht werden, da ja der Fremdschlüssel über den Barcode direkt eingegeben wird. So lässt sich auch nicht über das Listenfeld neben der Ware zumindest der Einzelpreis ersehen.

Hier hilft eine Abfrage:

```
SELECT
    "Kasse"."RechnungID",
    "Kasse"."Anzahl",
    "Ware"."Ware",
    "Ware"."Einzelpreis",
    "Kasse"."Anzahl"*"Ware"."Einzelpreis" AS "Gesamtpreis"
FROM "Kasse", "Ware"
WHERE "Ware"."ID" = "Kasse"."WareID";
```

Jetzt ist zumindest schon einmal nach der Eingabe die Information da, wie viel denn für 3 * Ware'17' zu bezahlen ist. Außerdem werden nur die Informationen durch das Formular zu filtern sein, die mit der entsprechenden "RechnungID" zusammenhängen. Was auf jeden Fall noch fehlt, ist das, was denn der Kunde nun bezahlen muss:

```
SELECT
    "Kasse"."RechnungID",
    SUM("Kasse"."Anzahl"*"Ware"."Einzelpreis") AS "Summe"
FROM "Kasse", "Ware"
WHERE "Ware"."ID" = "Kasse"."WareID"
GROUP BY "Kasse"."RechnungID";
```

Für das Formular liefert diese Abfrage nur eine Zahl, da über das Formular natürlich wieder die "RechnungID" gefiltert wird, so dass ein Kunde nicht gleichzeitig sieht, was andere vor ihm eventuell gezahlt haben.

Tipp

Sollen in einem Formular Datumswerte dargestellt werden, die in Abhängigkeit von einem anderen Datum definiert sind (z.B.: Eine Entleihzeit für ein Medium beträgt 21 Tage – wann muss das Medium zurückgegeben werden?), dann ist dies mit Standardfunktionen der **HSQLDB** nicht zu machen. Es fehlt eine Funktion wie **DATEADD**.

Die Abfrage

```
SELECT "Datum", DATEDIFF('dd', '1899-12-30', "Datum")+21 AS  
"RueckDatum" FROM "Tabelle"
```

würde in einem Formular als Datum formatiert das korrekte anvisierte Rückgabedatum ergeben. Mit dieser Abfrage werden die Tage ab dem 30.12.1899 gezählt. Der 30.12.1899 ist das Standarddatum, das z.B. auch Calc als 0-Wert nutzt.

Allerdings handelt es sich bei dem ermittelten Wert um eine Zahl, nicht um ein Datum, das anschließend in z.B. einer Abfrage weiter genutzt werden kann.

In einer Abfrage lässt sich die ermittelte Zahl schlecht nutzen, da die Formatierung von Abfragen nicht gespeichert wird. Dafür müsste eine Ansicht erstellt werden.

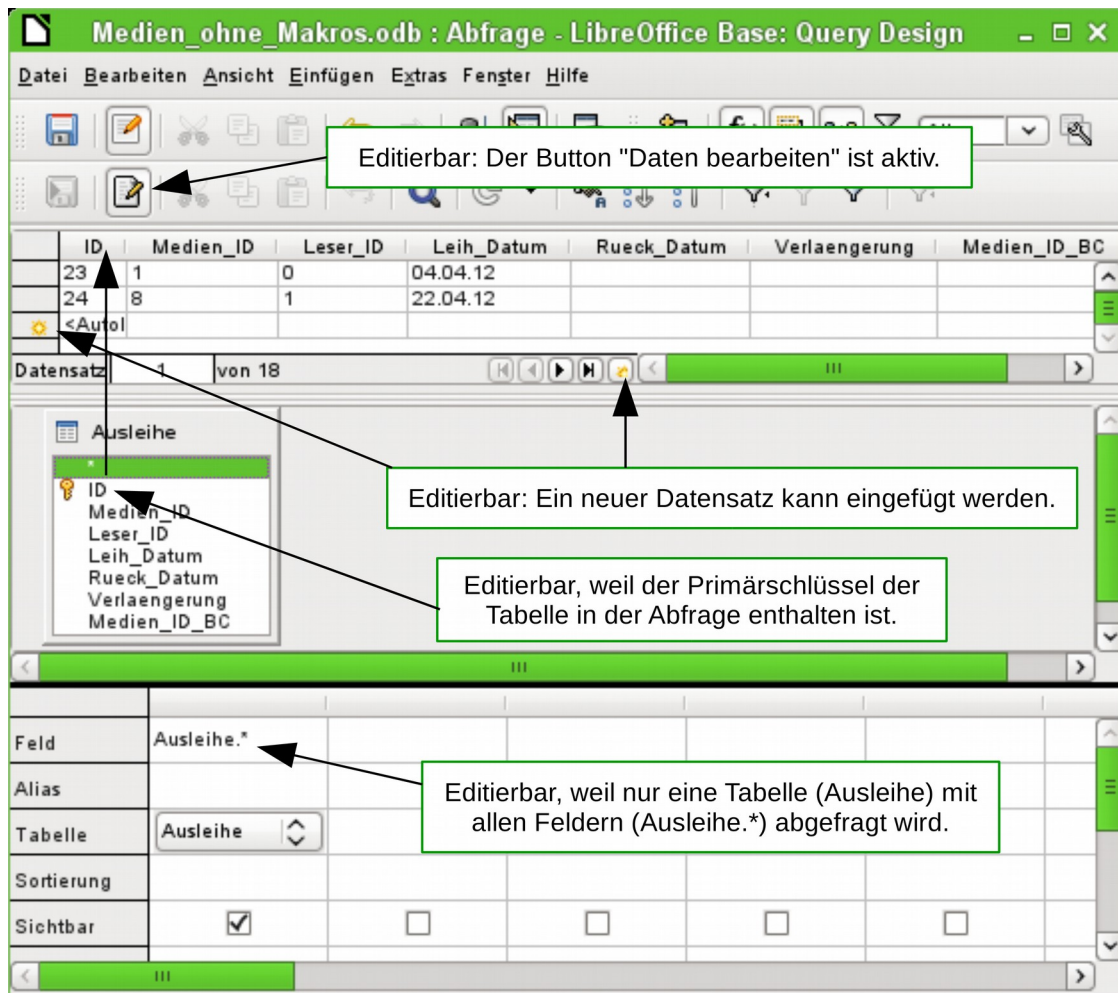
Achtung! Base selbst zeigt zumindest bis LO 5.0 Datumswerte, die auf Integer-Zahlen beruhen, unterschiedlich an. In Tabellen, Abfragen und formatierbaren Feldern des Formulars wird der 30.12.1899 als 0-Wert genommen. Datumsfelder hingegen berechnen aus 0 das Datum 1.1.1900.

FIREBIRD kennt die Funktion **DATEADD** und kann sogar direkt zu einem Datumswert eine Ganzzahl addieren, die dann als Tag gedeutet wird. Für **FIREBIRD** sähe die obige Abfrage entsprechend deutlich einfacher aus:

```
SELECT "Datum", "Datum"+21 AS "RueckDatum" FROM "Tabelle"
```

Eingabemöglichkeit in Abfragen

Um Eingaben in Abfragen zu tätigen, muss der Primärschlüssel für die jeweils zugrundeliegende Tabelle in der Abfrage enthalten sein.



Bei der Ausleihe von Medien ist es z. B. nicht sinnvoll, für einen Leser auch die Medien noch anzeigen zu lassen, die längst zurückgegeben wurden.

```
SELECT
    "ID",
    "LeserID",
    "MedienID",
    "Ausleihdatum"
FROM "Ausleihe"
WHERE "Rückgabedatum" IS NULL;
```

So lässt sich im Formular innerhalb eines Tabellenkontrollfeldes all das anzeigen, was ein bestimmter Leser zur Zeit entliehen hat. Auch hier ist die Abfrage über entsprechende Formular-konstruktion (Leser im Hauptformular, Abfrage im Unterformular) zu filtern, so dass nur die tatsäch-lich entliehenen Medien angezeigt werden. Die Abfrage ist zur Eingabe geeignet, da **der Primär-schlüssel in der Abfrage enthalten** ist.

Die Abfrage wird dann nicht mehr editierbar, wenn sie aus mehreren Tabellen besteht und die Tabellen über einen Alias-Namen angesprochen werden. Dabei ist es unerheblich, ob die Primär-schlüssel in der Abfrage enthalten sind.

Medien_ohne_Makros.odt : Abfrage - LibreOffice Base: Query Design

Datei Bearbeiten Ansicht Einfügen Extras Fenster Hilfe

ID Titel Kategorie_ID katID Kategorie

0	Der kleine Hobbit	0	0	Fantasy
5	I hear you knocking	1	1	Rock
8	Im Augenblick	2	2	Liedermacher
<AutoI			<AutoFel	

Datensatz 1 von 3

Medien

- ID
- Titel
- Ausgabe
- E_Jahr
- Anmerkung
- Kategorie_ID
- Medienart_ID

Kategorie

- ID
- Kategorie
- Beschreibung

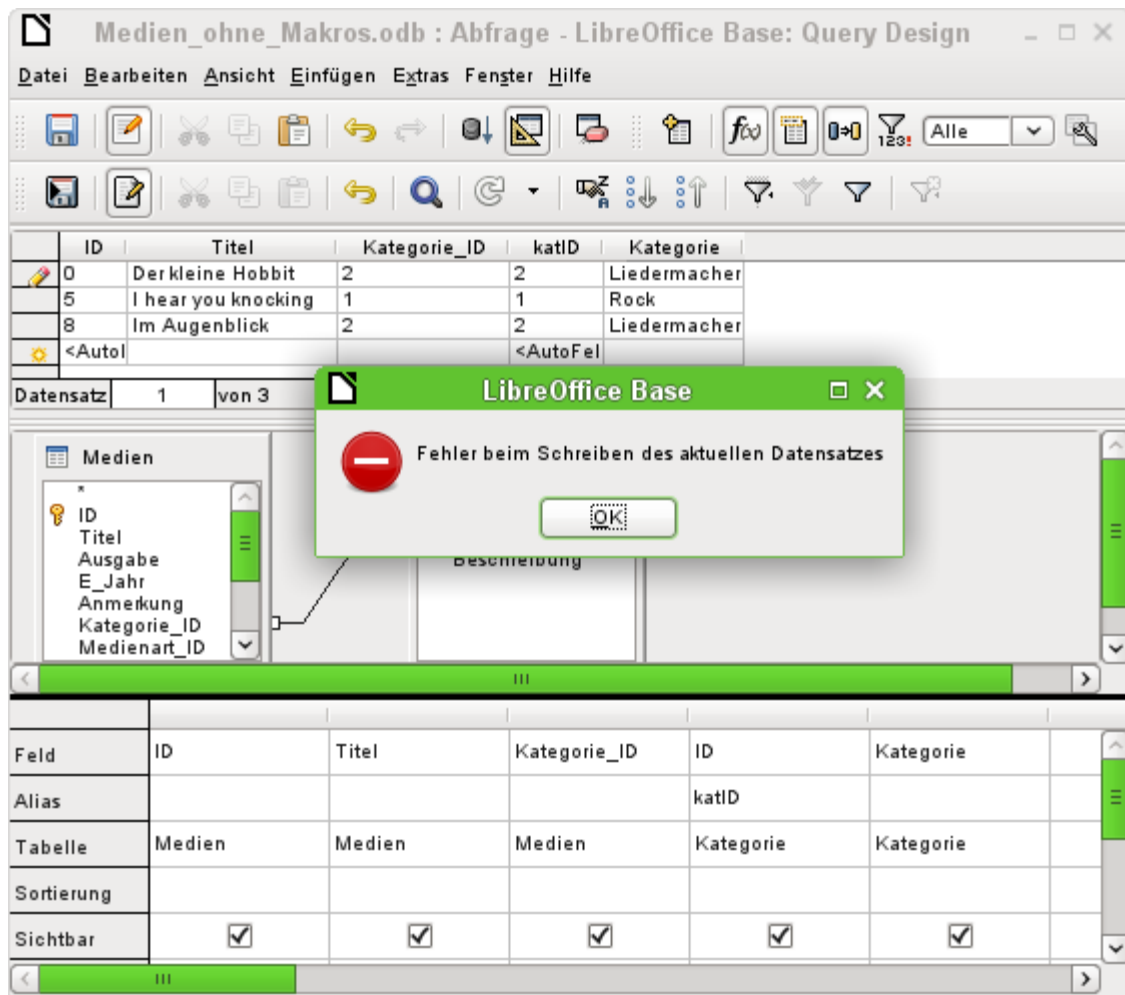
Feld	ID	Titel	Kategorie_ID	ID	Kategorie
Alias				katID	
Tabelle	Medien	Medien	Medien	Kategorie	Kategorie
Sortierung					
Sichtbar	☒	☒	☒	☒	☒

```

SELECT
    "Medien"."ID",
    "Medien"."Titel",
    "Medien"."Kategorie_ID",
    "Kategorie"."ID" AS "katID",
    "Kategorie"."Kategorie",
FROM "Medien",
    "Kategorie"
WHERE "Medien"."Kategorie_ID" = "Kategorie"."ID";

```

Diese Abfrage bleibt editierbar, da **beide Primärschlüssel enthalten** sind und auf die Tabellen nicht mit einem Alias zugegriffen wird. Auch bei einer Abfrage, die mehrere Tabellen miteinander verknüpft, müssen alle Primärschlüssel vorhanden sein. Außerdem muss eine Verbindung zwischen einem Feld der einen Tabelle und dem Primärschlüssel der anderen Tabelle definiert werden. Dies ist unabhängig davon, ob unter **Extras → Beziehungen** solch eine Verbindung (als Fremdschlüssel zu Primärschlüssel) bereits existiert.



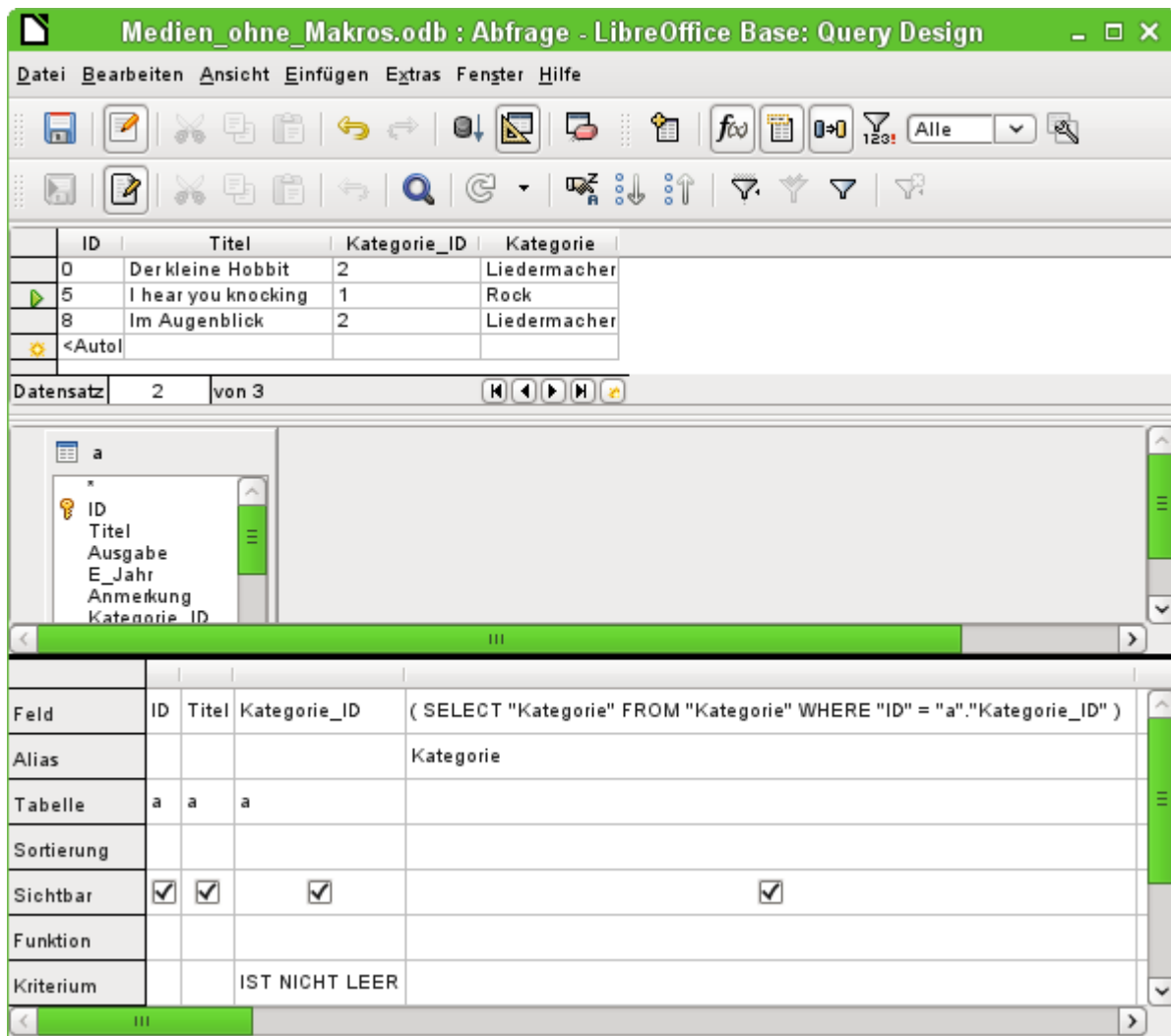
Es ist allerdings nicht möglich, in einer Abfrage, die auf mehreren Tabellen beruht, das Fremdschlüsselfeld der einen Tabelle, das sich auf die andere Tabelle bezieht, zu ändern. Im angezeigten Datensatz wurde versucht, die Kategorie für den Titel «Der kleine Hobbit» umzustellen. Das Feld "Kategorie_ID" wurde von '0' auf '2' geändert. Die Änderung schien vollzogen, die neue Kategorie erschien auch. Das Speichern war dann aber nicht möglich.

Allerdings ist es möglich, den Inhalt von "Kategorie" selbst zu bearbeiten, also z.B. 'Fantasy' zum Begriff 'Fantasie' zu ändern. Das ändert dann allerdings diesen Begriff für die Kategorie, betrifft also alle damit verbundenen Datensätze.

```
SELECT
    "a"."ID",
    "a"."Titel",
    "Kategorie"."Kategorie",
    "Kategorie"."ID" AS "katID"
FROM "Medien" AS "a",
    "Kategorie"
WHERE "a"."Kategorie_ID" = "Kategorie"."ID";
```

In dieser Abfrage wird auf die Tabelle "Medien" mit einem **Alias** zugegriffen. Sie wird jetzt **nicht editierbar** sein.

In dem obigen Beispiel ist das leicht vermeidbar. Wenn aber eine *Korrelierte Unterabfrage* gestellt wird, so muss mit einem Tabellenalias gearbeitet werden. So eine **Abfrage mit Alias** kann nur **dann editierbar** bleiben, wenn sie **lediglich eine Tabelle in der Hauptabfrage** enthält.



In der Design-Ansicht erscheint hier lediglich eine Tabelle. Die Tabelle "Medien" ist mit einem Alias versehen, damit auf die Inhalte des Feldes "Kategorie_ID" mit der korrelierenden Unterabfrage zugegriffen werden kann.

In so einer Abfrage ist es jetzt möglich, das Fremdschlüsselfeld "Kategorie_ID" auf eine andere Kategorie umzustellen. In dem obigen Beispiel wurde das Feld "Kategorie_ID" von '0' auf '2' geändert. Dem "Titel" 'Der kleine Hobbit' wurde so die "Kategorie" 'Liedermacher' zugeordnet.



Allerdings ist es jetzt nicht mehr möglich, einen Wert in dem Feld zu ändern, das seinen Inhalt über die korrelierende Unterabfrage erhält. Die Änderung der "Kategorie" von 'Fantasie' in 'Fantasie' wird erst einmal angezeigt. Die Änderung wird allerdings nicht registriert und ist auch nicht abspeicherbar. In der Tabelle, die die Design-Ansicht anzeigt, ist das Feld "Kategorie" schließlich nicht enthalten.

Verwendung von Parametern in Abfragen

Wird viel mit gleichen Abfragen, aber eventuell unterschiedlichen Werten als Voraussetzung zu diesen Abfragen gearbeitet, so sind Parameterabfragen möglich. Vom Prinzip her funktionieren Parameterabfragen erst einmal genauso wie Abfragen, die in Unterformularen abgelegt werden:

```
SELECT
    "ID",
    "Leser_ID",
    "Medien_ID",
    "Ausleihdatum"
FROM "Ausleihe"
WHERE "Rückgabedatum" IS NULL
    AND "Leser_ID"=2;
```

Diese Abfrage zeigt nur die entliehenen Medien des Lesers mit der Nummer 2 an.

```
SELECT
    "ID",
    "Leser_ID",
    "Medien_ID",
    "Ausleihdatum"
FROM "Ausleihe"
WHERE "Rückgabedatum" IS NULL
    AND "LeserID" = :Lesernummer;
```

Jetzt erscheint beim Aufrufen der Abfrage ein Eingabefeld. Es fordert zur Eingabe einer Lesernummer auf. Wird hier ein Wert eingegeben, so werden die zur Zeit entliehenen Medien dieses Lesers angezeigt.

```
SELECT
    "Ausleihe"."ID",
    "Leser"."Nachname" || ', ' || "Leser"."Vorname",
    "Ausleihe"."Medien_ID",
    "Ausleihe"."Ausleihdatum"
FROM "Ausleihe", "Leser"
WHERE "Ausleihe"."Rückgabedatum" IS NULL
    AND "Leser"."ID" = "Ausleihe"."Leser_ID"
    AND "Leser"."Nachname" LIKE '%' || :Lesername || '%'
ORDER BY "Leser"."Nachname" || ', ' || "Leser"."Vorname" ASC;
```

Diese Abfrage ist noch deutlich komfortabler als die vorhergehende. Jetzt muss nicht eine Nummer des Lesers bekannt sein. Es reicht die Eingabe eines Teils des Nachnamen des Lesers und alle Medien von Lesern, auf die diese Beschreibung zutrifft, werden angezeigt.

Wird

```
"Leser"."Nachname" LIKE '%' || :Lesername || '%'
```

durch

```
LOWER("Leser"."Nachname") LIKE '%' || LOWER( :Lesername ) || '%'
```

ersetzt, so ist es auch noch egal, ob die Namen groß oder klein geschrieben sind.

Wird der **Parameter** in der obigen Abfrage **leer** gelassen, so werden bis LO 4.4 **alle Leser** angezeigt, da ein leeres Parameterfeld erst ab der Version LO 4.4 auch tatsächlich nicht als leerer Text, sondern als **NULL** weiter gegeben wird. Soll dies vermieden werden, so muss etwas in die Trickkiste gegriffen werden:

```
LOWER ("Leser"."Nachname") LIKE '%' || IFNULL( NULLIF ( LOWER (:Leser-
name), '' ), '§§' ) || '%'
```


Das leere Parameterfeld gibt an die Abfrage einen leeren String, aber nicht NULL weiter. Deshalb muss erst einmal dem leeren Parameterfeld die Eigenschaft NULL mit **NULLIF** zugewiesen werden. Anschließend wird für den Fall, dass eben die Parametereingabe jetzt NULL ergibt, dieser Eingabe ein Wert zugewiesen, der in der Regel in keinem der Datensätze vorkommt. In dem obigen Beispiel ist das '\$\$'. Dieser Wert wird mit der Abfrage entsprechend auch nicht gefunden.

Ab der Version **LO 4.4** muss diese Abfragetechnik etwas angepasst werden:

```
LOWER ("Leser"."Nachname") LIKE '%' || LOWER (:Lesername) || '%'
```

ergibt zwangsläufig bei einer fehlenden Eingabe für die gesamte Kombination '%' || **LOWER (:Lesername)** || '%' den Wert **NULL**.

Dagegen hilft das Hinzufügen einer weiteren Bedingung, dass bei einem leeren Feld tatsächlich alle Werte aufgezeigt werden:

```
(LOWER ("Leser"."Nachname") LIKE '%' || LOWER (:Lesername) || '%'  
OR :Lesername IS NULL)
```

Dies sollte in Klammern gesetzt werden. So wird entweder ein Name ausgesucht oder, bei leerem Feld, also **NULL** ab LO 4.4, die zweite Bedingung erfüllt.

Ein Parameter kann bei Formularen auch von einem Hauptformular an ein Unterformular weitergegeben werden. Es kann allerdings passieren, dass Parameterabfragen in Unterformularen nicht aktualisiert werden, wenn Daten geändert oder neu eingegeben werden.

Manchmal wäre es wünschenswert, Listenfelder in Abhängigkeit von Einstellungen des Hauptformulars zu ändern. So könnte beispielsweise ausgeschlossen werden, dass in einer Bibliothek Medien an Personen entliehen werden, die zur Zeit keine Medien ausleihen dürfen. Leider ist aber eine derartige Listenfelderstellung in Abhängigkeit von der Person über Parameter nicht möglich.

Tipp

Parameterabfragen sind auch eine Möglichkeit, für Berichte Daten vor dem Start des Berichtes zu filtern. Daneben ermöglichen solche Abfragen auch, einem Bericht bestimmte Textbausteine mitzugeben, die sonst nicht in der Abfrage enthalten sind.

Mit

```
SELECT "Tabelle".*, :Ueberschrift FROM "Tabelle"
```

wird die Variable «Ueberschrift» abgefragt und kann dann in einem Textfeld des Berichtes über **=Ueberschrift** oder **=:Ueberschrift** ausgelesen werden.

Unterabfragen

Unterabfragen, die in Felder eingebaut werden, dürfen immer nur genau einen Datensatz wiedergeben. Das Feld kann schließlich auch nur einen Wert wiedergeben.

```
SELECT  
  "ID",  
  "Einnahme",  
  "Ausgabe",  
  (  
    SELECT  
      SUM( "Einnahme" ) - SUM( "Ausgabe" )  
    FROM "Kasse"  
    AS "Saldo"  
  )  
FROM "Kasse";
```

Diese Abfrage ist eingabefähig (Primärschlüssel vorhanden). Die Unterabfrage liefert nur genau einen Wert, nämlich die Gesamtsumme. Damit lässt sich nach jeder Eingabe der Kassenstand ablesen. Dies ist noch nicht vergleichbar mit der Supermarktkasse aus [Abfragen als Grundlage von Zusatzinformationen in Formularen](#). Es fehlt natürlich die Einzelberechnung aus Anzahl * Ein-

zelpreis, aber auch die Berücksichtigung von Rechnungsnummer. Es wird immer die Gesamtsumme ausgegeben. Zumindest die Rechnungsnummer lässt sich über eine Parameterabfrage einbauen:

```
SELECT
    "ID",
    "Einnahme",
    "Ausgabe",
    ( SELECT
        SUM( "Einnahme" ) - SUM( "Ausgabe" )
      FROM "Kasse"
      WHERE "RechnungID" = :Rechnungsnummer )
    AS "Saldo"
FROM "Kasse"
WHERE "RechnungID" = :Rechnungsnummer;
```

Bei einer Parameterabfrage muss der Parameter in beiden Abfrageanweisungen gleich sein, wenn er als ein Parameter verstanden werden soll.

Für Unterformulare können diese Parameter mitgegeben werden. Unterformulare erhalten dann statt der Feldbezeichnung die darauf bezogene Parameterbezeichnung. Die Eingabe dieser Verknüpfung ist nur in den Eigenschaften der Unterformulare, nicht über den Assistenten möglich.

Hinweis

Unterformulare, die auf Abfragen beruhen, werden nicht in Bezug auf die Parameter automatisch aktualisiert. Es bietet sich also eher an, den Parameter direkt aus dem darüber liegenden Formular weiter zu geben.

Korrelierte Unterabfrage

Mittels einer noch verfeinerten Abfrage lässt sich innerhalb einer bearbeitbaren Abfrage sogar der laufende Kontostand mitführen:

```
SELECT
    "ID",
    "Einnahme",
    "Ausgabe",
    ( SELECT
        SUM( "Einnahme" ) - SUM( "Ausgabe" )
      FROM "Kasse"
      WHERE "ID" <= "a"."ID" )
    AS "Saldo"
FROM "Kasse" AS "a"
ORDER BY "ID" ASC
```

Die Tabelle "Kasse" ist gleichbedeutend mit der Tabelle "a". "a" stellt aber nur den Bezug zu den in diesem Datensatz aktuellen Werten her. Damit ist der aktuelle Wert von "ID" aus der äußeren Abfrage innerhalb der Unterabfrage auswertbar. Auf diese Art und Weise wird in Abhängigkeit von der "ID" der jeweilige Saldo zu dem entsprechenden Zeitpunkt ermittelt, wenn einfach davon ausgegangen wird, dass die "ID" über den Autowert selbständig hoch geschrieben wird.

Hinweis

Soll nach den Inhalten der Unterabfrage mit Hilfe der Filterfunktionen des Abfrageeditors gefiltert werden, so funktioniert dies zur Zeit nur, wenn statt der einfachen Klammern zu Beginn und Ende der Unterabfrage die Klammern doppelt gesetzt werden: « **((SELECT ...)) AS "Saldo"** »

Abfragen als Bezugstabellen von Abfragen

In einer Abfrage soll für alle Leser, bei denen eine 3. Mahnung zu einem Medium vorliegt, ein Sperrvermerk ausgegeben werden.

```
SELECT
    "Ausleihe"."Leser_ID",
    '3 Mahnungen - der Leser ist gesperrt' AS "Sperrre"
FROM
    (SELECT COUNT( "Datum" ) AS "Anzahl",
        "Ausleihe_ID"
    FROM "Mahnung"
    GROUP BY "Ausleihe_ID")
    AS "a",
    "Ausleihe"
WHERE "a"."Ausleihe_ID" = "Ausleihe"."ID"
AND "a"."Anzahl" > 2
```

Zuerst wird die **innere Abfrage** konstruiert, auf die sich die äußere Abfrage bezieht. In dieser Abfrage wird die Anzahl der Datumeinträge, gruppiert nach dem Fremdschlüssel "Ausleihe_ID", ermittelt. Dies muss unabhängig von der "Leser_ID" geschehen, da sonst nicht nur 3 Mahnungen bei einem Medium, sondern auch drei Medien mit einer ersten Mahnung zusammengezählt würden. Die innere Abfrage wird mit einem Alias versehen, damit sie mit der "Leser_ID" der äußeren Abfrage in Verbindung gesetzt werden kann.

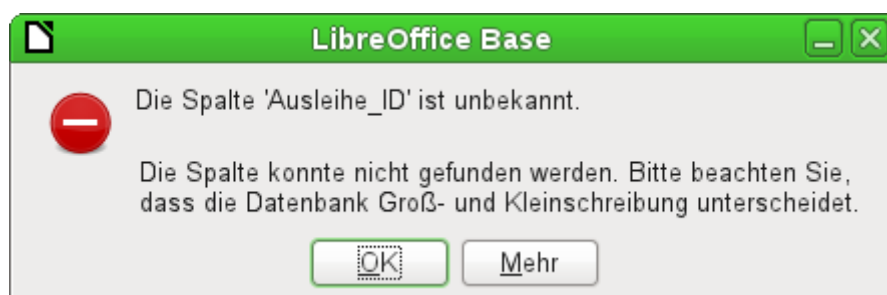
Die **äußere Abfrage** bezieht sich in diesem Fall nur in der Bedingungsformulierung auf die innere Abfrage. Sie zeigt nur dann eine "Leser_ID" und den Text zur "Sperrre" an, wenn "Ausleihe"."ID" und "a"."Ausleihe_ID" gleich sind sowie "a"."Anzahl" > 2 ist.

Prinzipiell stehen der äußeren Abfrage alle Felder der inneren Abfrage zur Verfügung. So ließe sich beispielsweise die Anzahl mit "a"."Anzahl" in der äußeren Abfrage einblenden, damit auch die tatsächliche Mahnzahl erscheint.

Es kann allerdings sein, dass im Abfrageeditor die grafische Benutzeroberfläche nach so einer Konstruktion nicht mehr verfügbar ist. Soll die Abfrage anschließend wieder zum Bearbeiten geöffnet werden, so erscheint die folgende Meldung:



Ist die Abfrage dann in der SQL-Ansicht zum Bearbeiten geöffnet und wird versucht von dort in die grafische Ansicht zu wechseln, so erscheint die Fehlermeldung



Die grafische Benutzeroberfläche findet also nicht das in der inneren Abfrage enthaltene Feld "Ausleihe_ID", mit dem die Beziehung von innerer und äußerer Abfrage geregelt wird.

Wird die Abfrage allerdings aufgerufen, so wird der entsprechende Inhalt anstandslos wiedergegeben. Es muss also *nicht der direkte SQL-Mode* bemüht werden. Damit stehen auch die Sortier- und Filterfunktionen der grafischen Benutzeroberfläche weiter zur Verfügung.

Die folgenden Screenshots zeigen, wie der unterschiedliche Weg zu einem Abfrageergebnis mit Unterabfragen auch verlaufen kann. Hier soll in der Abfrage einer Rechnungsdatenbank ermittelt werden, was der Kunde an der Kasse letztlich zahlen muss. Die Preise werden multipliziert mit der Anzahl der entsprechenden Ware zum "Teilbetrag". Außerdem soll auch noch die Summe der Teilbeträge ausgegeben werden. Und all das soll editierbar bleiben, damit die Abfrage als Grundlage für ein Formular dienen kann.

	ID	Anzahl	warID	WarenID	Preis	Teilbetrag
▶	40	5	17	17	0,75 €	3,75
	41	1	0	0	4,23 €	4,23
	43	2	31	31	0,23 €	0,46
	44	3	24	24	1,27 €	3,81
	45	7	0	0	4,23 €	29,61
	46	4	24	24	1,27 €	5,08
⚙	<AutoF					

Datensatz 1 von 6

```
SELECT
    "Abgang"."ID",
    "Abgang"."Anzahl",
    "Abgang"."warID",
    "Ware"."ID" AS "WarenID",
    "Ware"."Preis",
    "Anzahl" * "Preis" AS "Teilbetrag"
FROM "Abgang",
     "Ware"
WHERE "Abgang"."warID" = "Ware"."ID"
```

Abbildung 42: Abfrage über zwei Tabellen. Damit die Abfrage editierbar bleibt, muss aus beiden Tabellen der Primärschlüssel in der Abfrage enthalten sein.

Hinweis

Aufgrund des [Bugs 61871](#) aktualisiert Base leider den Teilbetrag nicht automatisch.

Datensatz 1 von 6 

43: Die Tabelle "Ware" wurde in eine Unterabfrage verschoben. Die Abfrage wird im Tabellenbereich (nach dem Begriff «FROM») mit einem Alias versehen. Jetzt ist der Primärschlüssel aus der Tabelle "Ware" in der Abfrage nicht mehr zwingend erforderlich. Die Abfrage bleibt auch so editierbar.

Datensatz	1	von	2	⏮ ⏪ ⏩ ⏭
-----------	---	-----	---	---------

Abbildung 44: Jetzt soll die Rechnungssumme noch in der Abfrage erscheinen. Bereits die einfache Abfrage der Rechnungssumme ist nicht editierbar, da hier gruppiert und summiert wird.

	ID	Anzahl	warID	Preis	Teilbetrag	Summe	
▶	40	5	17	0,75	3,75	12,25	
	41	1	0	4,23	4,23	12,25	
	43	2	31	0,23	0,46	12,25	
	44	3	24	1,27	3,81	12,25	
	45	7	0	4,23	29,61	34,69	
	46	4	24	1,27	5,08	34,69	
☀	<AutoF						
Datensatz 1		von 6					
<pre>SELECT "Abgang"."ID", "Abgang"."Anzahl", "Abgang"."warID", "Ware"."Preis", "Anzahl" * "Preis" AS "Teilbetrag", "Rechnungsbetrag"."Summe" FROM "Abgang", (SELECT "ID", "Preis" FROM "Ware") AS "Ware", (SELECT "Abgang"."recID", SUM("Anzahl" * "Preis") AS "Summe" FROM "Abgang", "Ware" WHERE "Abgang"."warID" = "Ware"."ID" GROUP BY "Abgang"."recID") AS "Rechnungsbetrag" WHERE "Abgang"."warID" = "Ware"."ID" AND "Abgang"."recID" = "Rechnungsbetrag"."recID" ORDER BY "Abgang"."recID", "Abgang"."ID"</pre>							

Abbildung 45: Mit der zweiten Unterabfrage wird das scheinbar Unmögliche möglich. Die vorhergehenden Abfrage wird als Unterabfrage in der Tabellendefinition dieser Abfrage (nach «FROM») eingefügt. Im Ergebnis bleibt die gesamte Abfrage so editierbar. Eingaben sind in diesem Fall nur in den Spalten "Anzahl" und "warID" möglich. Dies wird im Formular anschließend berücksichtigt.

Zusammenfassung von Daten mit Abfragen

Sollen Daten in der gesamten Datenbank gesucht werden, so ist dies über die einfachen Formularfunktionen meist nur mit Problemen zu bewältigen. Ein Formular greift schließlich nur auf eine Tabelle zu, und die Suchfunktion bewegt sich nur durch die Datensätze dieses Formulars.

Einfacher wird der Zugriff auf alle Daten mittels Abfragen, die wirklich alle Datensätze abbilden. Im Kapitel *Beziehungsdefinition in der Abfrage* wurde so eine Abfragekonstruktion bereits angedeutet. Diese wird im Folgenden entsprechend der Beispieldatenbank ausgebaut.

```

SELECT
    "Medien"."Titel",
    "Untertitel"."Untertitel",
    "Verfasser"."Verfasser"
FROM "Medien"
    LEFT JOIN "Untertitel"

```

```

ON "Medien"."ID" = "Untertitel"."Medien_ID"
LEFT JOIN "rel_Medien_Verfasser"
ON "Medien"."ID" = "rel_Medien_Verfasser"."Medien_ID"
LEFT JOIN "Verfasser"
ON "rel_Medien_Verfasser"."Verfasser_ID" = "Verfasser"."ID"

```

Hier werden alle "Titel", "Untertitel" und "Verfasser" zusammen angezeigt.

Die Tabelle "Medien" hat insgesamt 9 "Titel". Zu zwei Titeln existieren insgesamt 8 "Untertitel". Beide Tabellen zusammen angezeigt ergäben ohne einen **LEFT JOIN** lediglich 8 Datensätze. Zu jedem "Untertitel" würde der entsprechende "Titel" gesucht und damit wäre die Abfrage zu Ende. "Titel" *ohne* "Untertitel" würden nicht angezeigt.

Jetzt sollen aber alle "Medien" angezeigt werden, auch die *ohne* "Untertitel". "Medien" steht auf der linken Seite der Zuweisung, "Untertitel" auf der rechten Seite. Mit einem **LEFT JOIN** werden alle "Titel" aus "Medien" angezeigt, aber nur die "Untertitel", zu denen auch ein "Titel" existiert. "Medien" wird dadurch zu der Tabelle, die entscheidend für alle anzuzeigenden Datensätze wird. Dies ist bereits aufgrund der Tabellenkonstruktion so vorgesehen (siehe dazu das Kapitel «[Tabellen Medienaufnahme](#)»). Da zu 2 der 9 "Titel" "Untertitel" existieren, erscheinen in der Abfrage jetzt $9 + 8 - 2 = 15$ Datensätze.

Hinweis

Die normale Verbindung von Tabellen erfolgt, nachdem alle Tabellen durch Komma voneinander getrennt aufgezählt wurden, nach dem Schlüsselwort **WHERE**.

Wird mit einem **LEFT JOIN** oder **RIGHT JOIN** gearbeitet, so wird die Zuweisung direkt nach den beiden Tabellennamen mit **ON** definiert. Die Reihenfolge ist also immer

```

Tabelle1 LEFT JOIN Tabelle2 ON Tabelle1.Feld1 =
Tabelle2.Feld1 LEFT JOIN Tabelle3 ON Tabelle2.Feld1 =
Tabelle3.Feld1 ...

```

Zu 2 Titeln der Tabelle "Medien" existiert noch keine Verfassereingabe und kein "Untertitel". Gleichzeitig existieren bei einem "Titel" insgesamt 3 "Verfasser". Würde jetzt die Tabelle Verfasser einfach ohne **LEFT JOIN** verbunden, so würden die beiden "Medien" *ohne* "Verfasser" nicht angezeigt. Da aber ein Medium statt einem Verfasser drei Verfasser hat, würde die angezeigte Zahl an Datensätzen bei 15 Datensätzen bleiben.

Erst über die Kette von **LEFT JOIN** wird die Abfrage angewiesen, weiterhin die Tabelle "Medien" als den Maßstab für alles anzuzeigende zu nehmen. Jetzt erscheinen auch wieder die Datensätze, zu denen weder "Untertitel" noch "Verfasser" existieren, also insgesamt 17 Datensätze.

Durch entsprechende Joins wird also der angezeigte Datenbestand in der Regel größer. Durch diesen großen Datenbestand kann schließlich gesucht werden und neben den Titeln werden auch Verfasser und Untertitel erfasst. In der Beispieldatenbank können so alle von den Medien abhängigen Tabellen erfasst werden.

Schnellerer Zugriff auf Abfragen durch Tabellenansichten

Ansichten, in der SQL-Sprache «View», sind, besonders bei externen Datenbanken, schneller als Abfragen, da sie direkt in der Datenbank verankert sind und vom Server nur das Ergebnis präsentiert wird. Abfragen werden hingegen erst einmal zum Server geschickt und dort dann verarbeitet.

Bezieht sich eine neue Abfrage auf eine andere Abfrage, so sieht das in Base in der SQL-Ansicht so aus, als ob die andere Abfrage eine Tabelle wäre. Wird daraus ein 'View' erstellt, so zeigt sich, dass eigentlich mit Unterabfragen (Subselects) gearbeitet wird. Eine Abfrage 2, die sich auf eine andere Abfrage 1 bezieht, kann daher nicht über **SQL-Kommando direkt ausführen** ausgeführt werden, da nur die grafische Benutzeroberfläche, nicht aber die Datenbank selbst die Abfrage 1 kennt.

Auf Abfragen kann von der Datenbank her nicht direkt zugegriffen werden. Dies gilt auch für den Zugriff über Makros. Views hingegen können von Makros wie Tabellen angesprochen werden. Allerdings können in Views keine Datensätze geändert werden. Diesen Komfort bietet nur die Abfrage unter bestimmten Abfragebedingungen.

Tipp

Eine Abfrage, die über **SQL-Kommando direkt ausführen** gestartet wird, hat den Nachteil, dass sie für die GUI nicht mehr sortiert und gefiltert werden kann. Sie kann also nur begrenzt genutzt werden.

Eine Tabellenansicht (View) hingegen ist für Base handhabbar wie eine normale Tabelle – mit der Ausnahme, dass keine Änderung der Daten möglich ist. Hier stehen also trotz direktem SQL-Befehl alle Möglichkeiten zur Sortierung und zur Filterung zur Verfügung. Auch bleibt die Formatierung von Spalten in der Ansicht im Gegensatz zu Spalten in der Abfrage beim Schließen der Datenbank erhalten.

Ansichten sind bei manchen Abfragekonstruktionen eine Lösung, um überhaupt ein Ergebnis zu erzielen. Wenn z. B. auf das Ergebnis eines Subselects zugegriffen werden soll, so lässt sich dies nur über den Umweg eines Views erledigen. Entsprechende Beispiele im Kapitel «Datenbank-Aufgaben komplett»: «[Zeilennummerierung](#)» und «[Gruppieren und Zusammenfassen](#)».

Zeitdifferenzen berechnen

Eine Tabelle hat zwei Zeitangaben: Startzeit und Zielzeit. Die Zeitdifferenz soll berechnet werden.

```
SELECT "Startzeit", "Zielzeit",  
CAST('00:00:' || DATEDIFF('ss', "Startzeit", "Zielzeit") AS TIME)  
AS "Zeitdifferenz"  
FROM "Zeitmessung"
```

Hier wird mit dem Trick gearbeitet, dass die **HSQLDB** auch Zeitangaben umwandeln kann, die von der Textdarstellung her keine Zeitangaben mehr wären. So ergibt die erste Zeitdifferenz erst einmal über die Verbindung von

```
'00:00:' || DATEDIFF('ss', "Startzeit", "Zielzeit")
```

den Text

```
00:00:3397
```

der anschließend anstandslos in eine korrekte Zeit von 56 Minuten und 31 Sekunden umgewandelt wird.

Startzeit	Zielzeit	Zeitdifferenz
12:13:34	13:10:11	00:56:37
12:01:23	14:08:13	02:06:50

Leider funktioniert die Darstellung der Zeit in einer Abfrage nur dann korrekt, wenn die Spalte nach Durchführung der Abfrage auf die entsprechende Formatierung umgewandelt wird. Dieses Problem stellt sich in Formularen und Berichten nicht, da dort die Formatierung dauerhaft gespeichert wird. Soll anschließend nicht mehr mit der Zeitdifferenz gerechnet werden, so kann die Zeitdifferenz in einen Text mit der korrekten Schreibweise umgewandelt werden:

```
SELECT "Startzeit", "Zielzeit",  
TO_CHAR(  
    CAST('00:00:' || DATEDIFF('ss', "Startzeit", "Zielzeit") AS TIME),  
    'HH:MI:SS') AS "Zeitdifferenz"  
FROM "Zeitmessung"
```


Mit der FIREBIRD-Datenbank lässt sich dieses Problem wesentlich eleganter lösen:

```
SELECT "Startzeit", "Zielzeit", "Zielzeit" - "Startzeit" AS "Zeitdif-  
ferenz" FROM "Zeitmessung"
```

Firebird hat mit der Funktion DATEADD den Vorteil, dass es viele verschiedene Additions und Subtraktionsverfahren für Datums- und Zeitwerte kennt.

Berichte

Berichte mit dem Report-Designer

Mit Hilfe von Berichten werden Daten so dargestellt, dass sie auch für Personen ohne Datenbankkenntnisse gut lesbar sind. Berichte können

- Daten tabellarisch gut lesbar darstellen,
- zu Daten Diagramme erstellen,
- mit Hilfe von Daten Etikettendruck ermöglichen,
- Serienbriefe wie z. B. Rechnungen, Mahnungen oder auch nur Bestätigungen über einen Vereinsbeitritt oder -austritt erstellen

Um einen Bericht zu erstellen, muss die Datenbankgrundlage des Berichtes gut vorbereitet sein. Ein Bericht kann nicht, wie ein Formular, Unterberichte und damit zusätzliche Datenquellen aufnehmen. Ein Bericht kann auch nicht, wie im Formular, über Listenfelder andere Daten darstellen als in der zugrundeliegenden Datenquelle vorhanden sind.

Am besten werden Berichte mit Abfragen vorbereitet. Dort sollten alle variablen Inhalte festgeschrieben werden. Es sollte aber, wenn in dem Bericht noch sortiert werden soll, auf jeden Fall eine Abfrage erstellt werden, die das Sortieren zulässt. Dies bedeutet, dass Abfragen im direkten SQL-Modus unter diesen Bedingungen vermieden werden müssen. Muss der Datenbestand über so eine Abfrage zur Verfügung gestellt werden, so lässt sich eine Sortierung erreichen, indem aus der Abfrage eine *Tabellenansicht* erstellt wird. Diese Ansicht ist in der grafischen Benutzeroberfläche von Base immer sortierbar und filterbar.

Vorsicht



Der Report-Designer ist beim Editieren eines Berichtes mit laufendem Abspeichern zu begleiten. Dazu zählt nicht nur das Abspeichern im Report-Designer selbst nach jedem wichtigen Arbeitsschritt, sondern auch das Abspeichern der gesamten Datenbank.

Je nach Version von LibreOffice kann es beim Editieren auch zu plötzlichen Abstürzen des Report-Designers kommen.

Die Funktionsweise fertiger Berichte ist davon nicht betroffen – auch wenn diese Berichte unter einer anderen Version erstellt wurden, wo eben z. B. das oben genannte Verhalten nicht auftritt.

Hinweis

Der Report-Designer ist seit LO 4.1 bereits so in LO integriert, dass er nicht mehr sichtbar in den Erweiterungen («Extensions») erscheint. Es sollte auf jeden Fall vermieden werden, neben dem integrierten Report-Designer noch eine nicht zu der LO-Version passende Report-Designer-Erweiterung zu installieren.

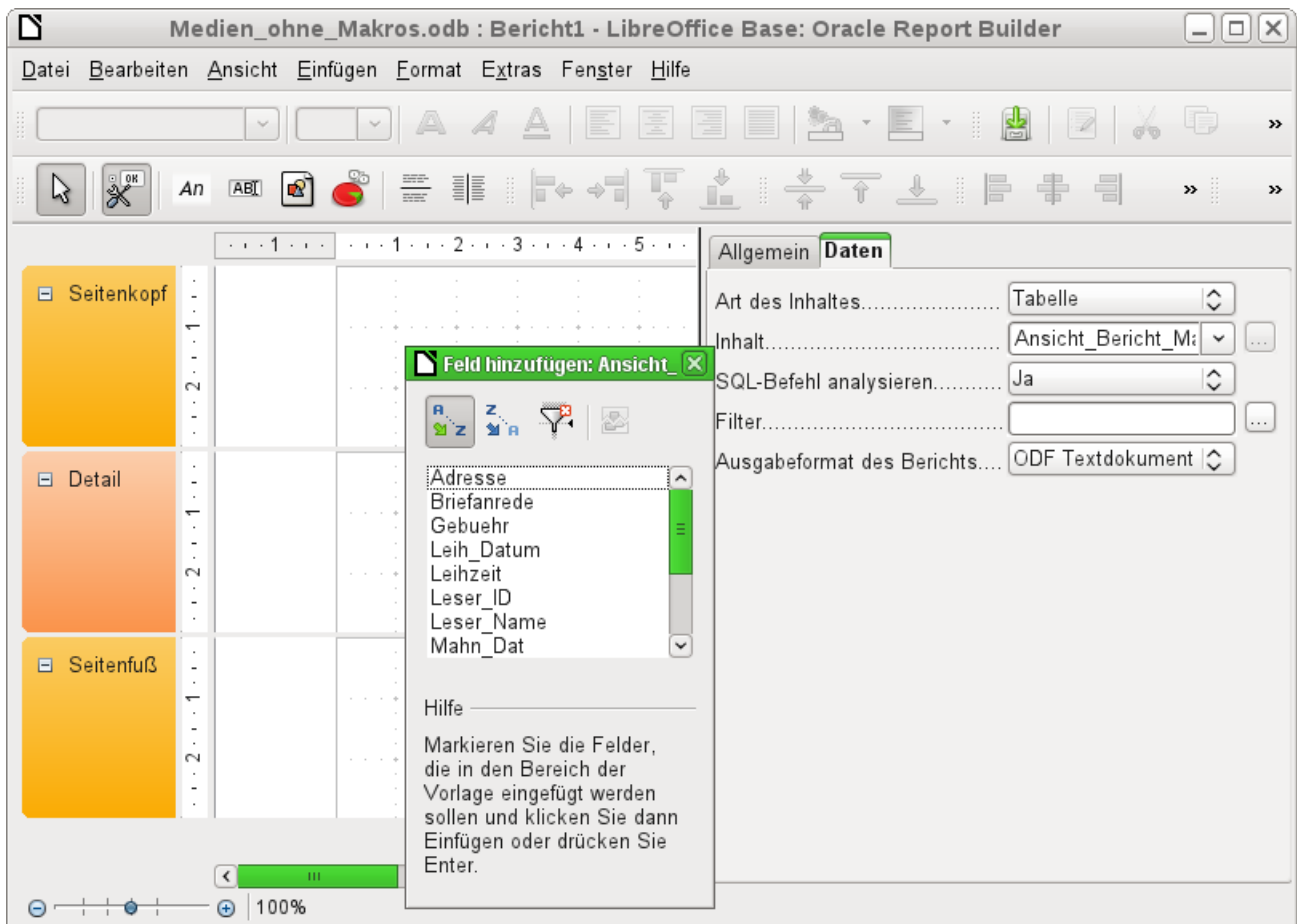
Neben dem Report-Designer existiert auch in LibreOffice Base weiterhin ein Assistent zur Berichtserstellung mittels eines Serienbriefverfahrens. Dieser Assistent ist allerdings nicht zugänglich, sobald der Report-Designer installiert wurde.

Der Assistent ist weitgehend selbsterklärend. Er produziert schnell übersichtliche Berichte in vorgefertigten Designs. Seine Berichte sind rein tabellarisch angelegt.

Da mit dem Report-Designer Berichte weiter ausgestaltet werden können, behandelt dieses Kapitel nur diese Berichtsvariante.

Die Benutzeroberfläche des Report-Designers

Über **Berichte** → **Bericht in der Entwurfsansicht erstellen ...** wird der Report-Designer aufgerufen.



Der Report-Designer startet in einer dreiteiligen Ansicht. Links ist die vorläufige Einteilung des Berichts in Seitenkopf, Detail und Seitenfuß zu sehen, in der Mitte befinden sich die entsprechenden Bereiche, die Inhalte des Berichtes aufnehmen, und rechts werden die Eigenschaften des Berichtes angezeigt. Die oberen und unteren Seitenränder sind nicht sichtbar.

Gleichzeitig wird bereits der Dialog «Feld hinzufügen» angezeigt. Dieser Dialog entspricht dem Dialog aus der Formularerstellung. Er erzeugt Felder mit der entsprechenden dazugehörigen Feldbezeichnung.

Ohne einen Inhalt aus der Datenbank lässt sich ein Bericht nicht sinnvoll nutzen. Deshalb wird zu Beginn direkt der Reiter «Daten» angezeigt. Hier kann der Inhalt des Berichtes eingestellt werden, im obigen Beispiel also die 'Tabelle' «Ansicht_Bericht_Mahnung». Solange **SQL-Befehl analysieren** auf «Ja» steht, kann der Bericht auch Sortierungen, Gruppierungen und Filterungen vornehmen. Da bereits als Grundlage eine Ansicht (View) gewählt wurde, kommt eine Filterung nicht zum Einsatz. Sie wurde bereits in der der Ansicht zugrundeliegenden Abfrage vorgenommen.

Hinweis

Als **Art des Inhaltes** kann eine Tabelle, eine Ansicht (View), eine Abfrage oder direkt SQL-Code angegeben werden. Am besten zu handhaben ist der Report-Designer, wenn er die Daten so weit wie möglich aufbereitet erhält. So können z.B. in Abfragen Berechnungen vorher durchgeführt werden und auch der Bereich der Daten, die im Bericht erscheinen sollen, begrenzt werden.

Vor allem bei früheren LO-Versionen (ebenso bei AOO oder OpenOffice) kommt es manchmal zu Problemen, wenn Abfragen aus mehreren Tabellen später gruppiert werden sollen. Solche Probleme können vermieden werden, indem auf eine Ansicht zugegriffen wird. Eine Ansicht erscheint für Programmteile wie den Report-Designer wie eine Tabelle der Datenbank. Feldnamen sind dann unverrückbar vorgegeben. Der anschließende Zugriff mit Sortier- und Gruppierbefehlen ist fehlerfrei möglich.

Zwei Ausgabeformate für Berichte sind wählbar: «ODF Textdokument», also ein Writer-Dokument oder «ODF Tabellendokument», also ein Calc-Dokument. Soll einfach nur eine tabellarische Übersicht ausgegeben werden, so ist das Calc-Dokument für die Berichtserstellung eindeutig vorzuziehen. Es ist wesentlich schneller erstellt und kann anschließend auch besser nachformatiert werden, da weniger Formatvorgaben berücksichtigt werden und Spalten einfach nach der erforderlichen Breite anschließend entsprechend gezogen werden können.

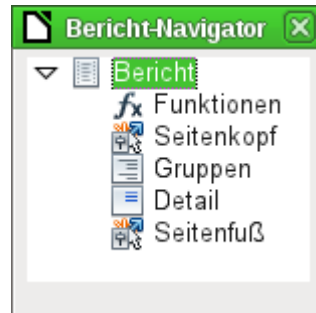
Standardmäßig sucht der Report-Designer als Datenquelle die erste Tabelle der Datenbank aus. So ist auf jeden Fall gewährleistet, dass zumindest ein Ausprobieren der Funktionen möglich ist. Erst nach Auswahl der Datenquelle kann der Bericht mit Feldern beschickt werden.

Schaltflächen inhaltliche Bearbeitung	Schaltflächen Elementausrichtung
 <u>S</u> peichern  <u>D</u> atei bearbeiten  <u>A</u> usschneiden  <u>K</u> opieren  <u>E</u> infügen  Eigenschaft 'C <u>o</u> mmand' ändern  <u>W</u> iederherstellen  <u>F</u> eld hinzufügen  <u>B</u> ericht-Navigator  Sortierung <u>u</u> nd Gruppierung  B <u>e</u> richt ausführen  <u>L</u> ibreOffice Hilfe ☐ D <u>i</u> rekthilfe	 <u>L</u> inks im Bereich ausrichten  <u>R</u> echts im Bereich ausrichten  <u>O</u> ben im Bereich ausrichten  <u>U</u> nten im Bereich ausrichten  <u>V</u> erkleinern  Von <u>o</u> ben verkleinern  Von <u>u</u> nten verkleinern  A <u>n</u> Element mit geringster Breite anpassen  A <u>n</u> b <u>r</u> eitesten Element anpassen  A <u>n</u> E <u>l</u> ement mit geringster Höhe anpassen  A <u>n</u> h <u>ö</u> chstes Element anpassen
 <u>A</u> uswahl  <u>E</u> igenschaften  <u>B</u> eschriftungsfeld  <u>T</u> extfeld  B <u>i</u> ld  D <u>i</u> agramm  <u>H</u> orizontale Linie  <u>V</u> ertikale Linie  <u>R</u> aster sichtbar  A <u>m</u> Raster f <u>a</u> ngen  H <u>i</u> lfslinien b <u>e</u> im V <u>e</u> rchieben	 <u>L</u> inks  <u>Z</u> entriert  <u>R</u> echts  <u>O</u> ben  <u>M</u> itte  <u>U</u> nten

Der Report-Designer stellt einige zusätzliche Schaltflächen zur Verfügung, so dass in der vorstehenden Tabelle noch einmal die Schaltflächen mit einer entsprechenden Beschriftung abgebildet sind. Die Schaltflächen zur Elementausrichtung werden in diesem Kapitel nicht weiter beschrieben.

Sie sind hilfreich beim schnellen Anpassen von Feldern in einem Bereich des Report-Designers. Prinzipiell geht dies alles aber auch über die direkte Bearbeitung der Eigenschaften des jeweiligen Feldes.

Wie schon bei den Formularen ist es hilfreich, den entsprechenden Navigator bei Problemen aufzurufen. So kann es zum Beispiel sein, dass durch einen unvorsichtigen Klick beim Start des Report-Designers die Eigenschaften zu den Daten des Berichts verzweifelt gesucht werden. Diese Daten können nur über den Bericht-Navigator erreicht werden:



Ein Klick mit der linken Maustaste auf «Bericht» und die Eigenschaften des Berichtes sind wieder erreichbar.

Der Navigator zeigt zu Beginn neben den sichtbaren Unterteilungen des Dokumentes (Seitenkopf, Gruppen, Detail und Seitenfuß) noch die möglichen Inhalte von Funktionen an. Gruppen ordnen z. B. alle anzumahnenden Medien einer Person zu, so dass nicht viele Einzelmahnungen erstellt werden müssen. Detailbereiche zeigen die zu den Gruppen passenden Datensätze an. Funktionen dienen z. B. zur Berechnung einer Summe einer Rechnung.

Um bei dem oben geöffneten Beispiel sinnvolle Ausgaben zu erhalten, muss der Inhalt der Ansicht gruppiert wiedergegeben werden. Ein Leser soll gebündelt die Anmahnungen für alle seine entliehenen und überzogenen Medien erhalten.

Über **Ansicht → Sortierung und Gruppierung** bzw. den entsprechenden Button startet die Gruppierungsfunktion. Hier können neben Gruppen, die angezeigt werden sollen, auch einfach Sortierungen eingestellt werden. Soll nur sortiert werden, so wird zwar eine Gruppierung ausgewählt, der Gruppenkopf und der Gruppenfuß aber auf «Nicht vorhanden» eingestellt.

Zuerst erfolgt die Sortierung nach dem obersten eingetragenen Feld. Gegebenenfalls müssen also die Einträge entsprechend verschoben werden, wenn die gewünschte Sortierung nicht eintritt.

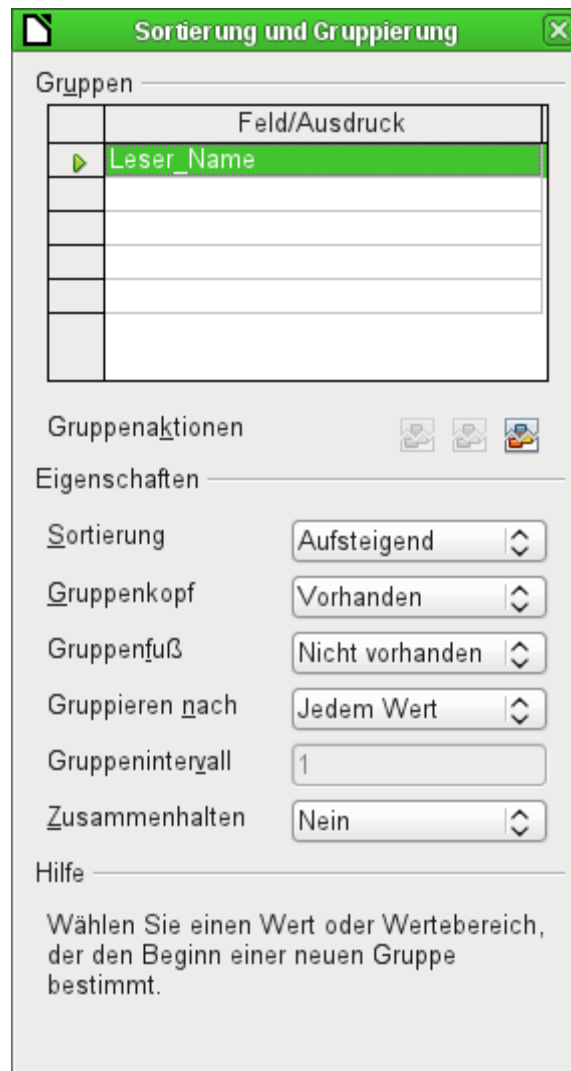


Abbildung 46: Sortierung und Gruppierung

Hier wurde nach dem Feld "Leser_Name" gruppiert und sortiert. In die obere Tabelle können untereinander mehrere Felder eingetragen werden. Soll z. B. zusätzlich nach dem "Leih_Datum" gruppiert und sortiert werden, so ist dies als zweite Zeile anzuwählen.

Direkt unter der Tabelle erscheinen verschiedene Gruppenaktionen zur Auswahl: Eine Verschiebung der Gruppe nach oben, eine Verschiebung nach unten oder die komplette Löschung der Gruppe. Da für den geplanten Bericht nur eine Gruppierung notwendig ist, steht in [Abbildung 46](#) nur mit dem Symbol ganz rechts die Gruppenaktion «Löschung» zur Verfügung.

Die Eigenschaft «Sortierung» ist selbsterklärend. Bei der Erstellung des Eintrags hat sich im Report-Designer auf der linken Seite sofort eine neue Einteilung gebildet. Neben der Feldbezeichnung "Leser_Name" steht dort noch «Kopf». Diese Einteilung nimmt also die Kopfzeile des Berichtes auf. In der Kopfzeile steht nachher z. B. der Name der Person, die eine Mahnung erhalten soll. Ein Gruppenfuß ist hingegen bisher nicht vorhanden. Er könnte z. B. den zu zahlenden Betrag oder den Ort und das aktuelle Datum sowie einen Bereich für die Unterschrift der anmahrenden Person enthalten.

Standardmäßig wird nach jedem Wert gruppiert. Ändert sich also "Leser_Name", so entsteht eine neue Gruppe. Alternativ kann hier nach dem Anfangsbuchstaben gruppiert werden. Das würde aber bei einem Mahnverfahren alle Leser-Nachnamen mit gleichem Anfangsbuchstaben zusammenfassen in einer Gruppe. 'Schmidt', 'Schulze' und 'Schulte' erhielten so eine gemeinschaftliche Mahnung. Eine wohl recht sinnlose Aktion an dieser Stelle für dieses Beispiel.

Nur wenn nach Anfangsbuchstaben gruppiert wird, kann noch zusätzlich eingegeben werden, nach wie vielen Werten die nächste Gruppe beginnen soll. Denkbar wäre hier z. B. eine Gruppierung für ein kleines Telefonbüchlein. Je nach Bekanntenkreis reicht da vielleicht eine Gruppierung nach jedem 2. Wert, also A und B in einer Gruppe, dann C und D usw.

Je nach Einstellung kann eine Gruppe entweder mit dem ersten Detail zusammen gehalten werden, oder, sofern möglich, als ganze Gruppe. Standardmäßig ist das Zusammenhalten auf «Nein» eingestellt. Für ein Mahnverfahren wird vermutlich sowieso die Gruppe so angeordnet, dass für jede Person, die eine Mahnung erhalten soll, eine Seite ausgedruckt wird. Daher ist stattdessen an anderer Stelle zu wählen, dass nach der Gruppe jeweils ein Seitenumbruch erfolgt, bevor der nächste Wert abzuarbeiten ist.

Sind Gruppenkopf und gegebenenfalls Gruppenfuß ausgewählt, so erscheinen diese Elemente als Teile des Berichtsnavigators unter dem entsprechenden Feldnamen "Leser_Name". Zusätzlich wird auch da wieder die Möglichkeit für Funktionen geboten, die sich nur auf diese Gruppe beschränken.

The screenshot displays the Libre Office Bibliothek report designer interface. The layout is organized into sections: 'Seitenkopf' (Page Header), 'Detail', and 'Seitenfuß' (Page Footer). The 'Detail' section contains a table with columns for 'Mahn-Nr', 'Medium', 'Leihdatu...', 'Überzog...', and 'Gebühr'. The 'Seitenkopf' section includes a title 'Libre Office Bibliothek' and a section for 'Mahnung' (Reminder). The 'Seitenfuß' section includes a closing 'Mit freundlichen Grüßen' and a signature '(Bibliotheksverwaltung)'. The interface shows various fields and formulas like '=Adresse', '=Briefanrede', and '=Mahnung'.

Das Hinzufügen der Felder läuft über die Funktion «Feld hinzufügen» wie im Formular. Allerdings sind hier die Beschreibungen und die Inhaltsfelder nicht miteinander gruppiert. Beide können also unabhängig voneinander verschoben, in der Größe beeinflusst und auf unterschiedliche Einteilungsebenen gezogen werden.

Das obige Bild zeigt den Berichtsentwurf für die Mahnung an. Im Seitenkopf ist fest die Überschrift Libre-Office Bibliothek als Beschriftungsfeld eingesetzt. Hier könnte auch ein Briefkopf mit Logo stehen, da auch die Einbindung von Grafiken möglich ist. Wenn die Ebene «Seitenkopf» heißt, so

bedeutet das nicht, dass darüber kein Rand existiert. Dieser wurde in den Seiteneinstellungen bereits festgelegt und liegt oberhalb des Seitenkopfes.

«*Leser_Name Kopf*» ist die Gruppierung und Sortierung, nach der die Daten zusammengefasst werden sollen. In den Feldern, die später Daten aufnehmen, steht hellgrau die Bezeichnung der Datenfelder, die hier ausgelesen werden. So hat die dem Bericht zugrundeliegende Ansicht z. B. ein Feld mit der Bezeichnung Adresse, in dem die komplette Adresse mit Straße und Ort für die anzumahnende Person steht. Um dies in ein Feld zu setzen, sind Absatzumbrüche in der Abfrage notwendig. Mittels **CHAR(13) || CHAR(10)** (**FIREBIRD**: Statt **CHAR() ASCII_CHAR()** verwenden) wird in einer Abfrage ein Absatz erzeugt. Beispiel:

```
SELECT "Anrede" || CHAR(13) || CHAR(10) || "Vorname" || ' ' || "Nachname" ||  
CHAR(13) || CHAR(10) || "Strasse" || ' ' || "Nr" || CHAR(13) || CHAR(10) || "Post-  
leitzahl" || ' ' || "Ort" AS "Adresse" FROM "Leser"
```

Bei dem Feld «=*TODAY()*» handelt es sich um eine eingebaute Funktion, die das aktuelle Datum an dieser Stelle einliest. Diese Funktion ist über **Einfügen → Datum und Uhrzeit** direkt zu erreichen.

In «*Leser_Name Kopf*» sind außer der Anrede und weiteren Informationen auch die Spaltenköpfe für die anschließende Tabellenansicht untergebracht. Sie sollen ja nur einmal auftauchen, auch wenn mehrere Medien aufgelistet werden.

In den Hintergrund dieser Spaltenköpfe wurde je ein graues Rechteck gelegt. Dieses Rechteck sorgt gleichzeitig für eine entsprechende Umrandung.

Der Detailbereich wird so oft wiederholt, wie unterschiedliche Datensätze mit den gleichen Daten existieren, die in "*Leser_Name*" stehen. Hier werden also alle Medien aufgelistet, die nicht rechtzeitig zurückgegeben wurden. Auch hier liegt im Hintergrund ein Rechteck, um die Inhalte zu umranden. Das Rechteck selbst hat die Füllfarbe «weiß».

Hinweis

Grundsätzlich gibt es in LO auch die Möglichkeit, horizontale und vertikale Linien hinzu zu fügen. Im Design-Modus werden diese auch angezeigt. Beim Ausführen des Berichtes erscheinen sie erst ab der Version LO 4.0.5 bzw. LO 4.1.1.

Diese Linien haben den Nachteil, dass sie nur als Haarlinien ausgelegt sind. Sie lassen sich besser nachbilden, indem Rechtecke benutzt werden. Der Hintergrund der Rechtecke wird auf die Farbe Schwarz eingestellt, die Größe wird z. B. mit einer Breite von 17 cm und einer Höhe von 0,03 cm festgelegt. Dann erscheint eine horizontale Linie mit einer Dicke von 0,03 cm mit einer Länge von 17 cm.

Leider hat auch diese Variante einen Nachteil: grafische Elemente lassen sich nicht richtig positionieren, wenn der Bereich über eine Seite hinweg geht.

Der «*Leser_Name Fuß*» schließt schließlich das Briefformat mit einer Grußformel und einem Unterschriftsbereich ab. Der Fuß ist so definiert, dass anschließend ein Seitenumbruch nach dem Bereich erfolgt. Außerdem ist er gegenüber den Standardeinstellungen verändert worden und so eingestellt, dass der Bereich auf jeden Fall zusammen gehalten werden soll. Schließlich würde es reichlich merkwürdig aussehen, wenn bei vielen Mahnungen allein z. B. das Unterschriftsfeld auf die nächste Seite verschoben würde.

Zusammenhalten bezieht sich hier immer auf den Seitenumbruch. Soll der Inhalt eines Datensatzes unabhängig vom Seitenumbruch zusammengehalten werden, so ist dies zur Zeit nur möglich, indem der Datensatz nicht als «Detail» eingelesen wird, sondern als Grundlage für eine Gruppierung genommen wird. Der Bereich «Detail» wird leider auch dann aufgetrennt, wenn «Zusammenhalten – Ja» ausgewählt wird.

Libre Office Bibliothek

Herrn
Bert Lederstrumpf
Neuenkirchener Str. 72
D 45793 Pussemuckel

22.04.12

Mahnung

Sehr geehrter Herr Lederstrumpf,

leider haben Sie versäumt, die folgenden Medien rechtzeitig wieder zurück zu geben:

Mahndatum	Mahn-Nr	Medium	Leihdatum	Überzogen	Gebühr
22.04.12	2	3 - Traditionelle und kritische Theorie - von Horkheimer, Max	09.12.11	128Tage	4,50 €
22.04.12	2	5 - I hear you knocking - von Edmunds, Dave	28.11.11	139Tage	4,75 €
22.04.12	3	4 - Die neue deutsche Rechtschreibung - von Hermann, Ursula	09.11.11	151Tage	5,25 €
22.04.12	1	1 - Das sogenannte Böse - von Lorenz, Konrad	04.04.12	4Tage	0,00 €
22.04.12	1	7 - Das Postfix-Buch - von ?	25.02.12	43Tage	1,50 €
					16,00 €

Mit freundlichen Grüßen

(Bibliotheksverwaltung)

Seite 1 / 1 | Standard | STD | Gruppenkopf:A1

Für die Aufsummierung der Gebühren wurde eine interne Funktion benutzt. Hierfür wurde zuerst ein Textfeld ohne Inhalt aufgezogen. Anschließend wurde in den **Eigenschaften** → **Daten** → **Datenfeld-Typ** → **Funktion** gewählt. Anschließend wurde **Datenfeld** → **Gebuehr** eingestellt. Unter **Funktion** stehen Summe, Minimum und Maximum zur Verfügung. Folgerichtig wurde hier die **Summe** ausge-

wählt. Schließlich ist im **Geltungsbereich** noch wählbar, ob die Funktion für eine bestimmte Gruppe oder für den ganzen Bericht gelten soll. Die Summe soll bezogen auf den Leser erstellt werden. Also kam hier nur die **Gruppe: Leser_Name** in Frage. Intern weist der Report-Builder dieser Funktion den Namen *SummeGebuehrLeser_Name* zu. Aus dem Namen ist ersichtlich, dass es sich um eine Summierung der «Gebuehr» handelt. Diese Summierung wird für jede Gruppe «Leser_Name» durchgeführt.

So könnte dann eine entsprechende Mahnung aussehen. Im Detailbereich sind hier 5 Medien angegeben, die der Leser entliehen hat. Im Gruppenfuß wird die Summe für die Anmahnung ausgegeben.

Hinweis

Berichte können auch für einzelne Datensätze über mehrere Seiten gehen. Die Bereichsgröße sagt nichts über die Seitengröße aus. Allerdings kann die Ausdehnung des Bereiches Detail über mehr als eine Seite dazu führen, dass die Umbrüche nicht einwandfrei sind. Hier ist der Report-Designer in der Abstandsberechnung noch fehlerhaft. Kommen Gruppierungsbereiche und grafische Elemente hinzu, so entstehen teilweise nicht mehr durchschaubare Bereichsgrößen.

Tipp

Soll sich ein Bereich über mehrere Seiten erstrecken, so muss der Beginn des Bereiches klar auf der ersten Seite definiert sein. Um vertikale Abstände zwischen Textfeldern des Berichtes zu konstruieren, setzt der Report-Designer Tabellenzeilen ein. Passt eine Tabellenzeile nicht mehr auf eine Seite, so wird sie komplett auf die nächste Seite verschoben – mit dem gesamten folgenden Inhalt. Hier hilft es, ein kleines leeres Beschriftungsfeld genau dort zu positionieren, wo der Beginn der neuen Seite sein soll. Der Report-Designer erstellt dort dann die erste Tabellenzeile der neuen Seite. So ist die Positionierung von Textfeldern und Beschriftungsfeldern auch bei größeren Bereichen möglich.

Allgemeine Eigenschaften von Feldern

Zur Darstellung von Daten gibt es lediglich drei unterschiedliche Felder. Neben dem Textfeld, das im Gegensatz zu seiner Namensgebung auch Zahlen und Formatierungen beherbergen kann, gibt es noch ein Feld, das Bilder aus der Datenbank aufnehmen kann. Das Diagrammfeld stellt eine Zusammenfassung von Daten dar.

Allgemein	
Name.....	Gebuehr
Position X.....	15,55cm
Sichtbar.....	Ja
Position Y.....	0,00cm
Breite.....	1,85cm
Höhe.....	0,50cm
Wiederholende Werte anzeigen....	Ja
Ausdruck für bedingte Anzeige.....	
Mauseverhalten.....	Wenn ausgewählt
Bei Gruppenwechsel anzeigen.....	Nein
Hintergrund transparent.....	Ja
Hintergrundfarbe.....	
Schrift.....	Liberation Sans, Sta
Horizontale Ausrichtung.....	Links
Vertikale Ausrichtung.....	Oben
Formatierung.....	1.234,57 €

Felder werden wie bei den Formularen mit Namen bezeichnet. Standardmäßig ist hier der Name gleich der Feldbezeichnung der zugrundeliegenden Datenbank.

Ein Feld kann unsichtbar geschaltet werden. Bei Feldern macht dies vielleicht wenig Sinn, bei Gruppenkopf oder Gruppenfuß hingegen schon eher, da hier auch andere Funktionen der Gruppierung erfüllt sein sollen, der Gruppenkopf oder Gruppenfuß aber nicht unbedingt mit Inhalt versehen ist.

Wenn «*Wiederholende Werte anzeigen*» deaktiviert wird, so wird die Anzeige ausgesetzt, wenn direkt davor das Feld mit einem gleichen Dateninhalt bestückt wurde. Dies funktioniert einwandfrei nur bei Datenfeldern, die einen Text beinhalten. Zahlenfelder oder Datumsfelder ignorieren die Deaktivierung, Beschriftungsfelder werden bei einer Deaktivierung komplett ausgeblendet, auch wenn sie nur einmal vorkommen.

In dem Report-Designer kann die Ansicht bestimmter Inhalte durch einen «*Ausdruck für bedingte Anzeige*» unterdrückt werden oder der Wert des Feldes als Grundlage für eine Formatierung von Schrift und Hintergrund genommen werden. Mehr zu diesen Ausdrücken unter «*Bedingte Anzeige*».

Die Einstellung zum Mauseverhalten spielt keine Rolle, da ja die Felder in dem Bericht nicht editierbar sind. Sie scheint ein Überbleibsel aus dem Bereich der Formulare zu sein.

Die Funktion «*Bei Gruppenwechsel anzeigen*» konnte im Bericht ebenfalls nicht nachvollzogen werden.

Ist der Hintergrund nicht als transparent definiert, so kann für das jeweilige Feld eine Hintergrundfarbe definiert werden.

Die weiteren Einträge beziehen sich auf den Inhalt innerhalb der gezogenen Felder. Dies sind im Einzelnen die Schriftart (mit Schriftfarbe, Schriftdicke etc., siehe [Abbildung 47](#)), die Ausrichtung des Schriftzugs in dem Feld und die Formatierung mit dem entsprechenden Dialog «*Zahlenformat*» (siehe [Abbildung 48](#)).

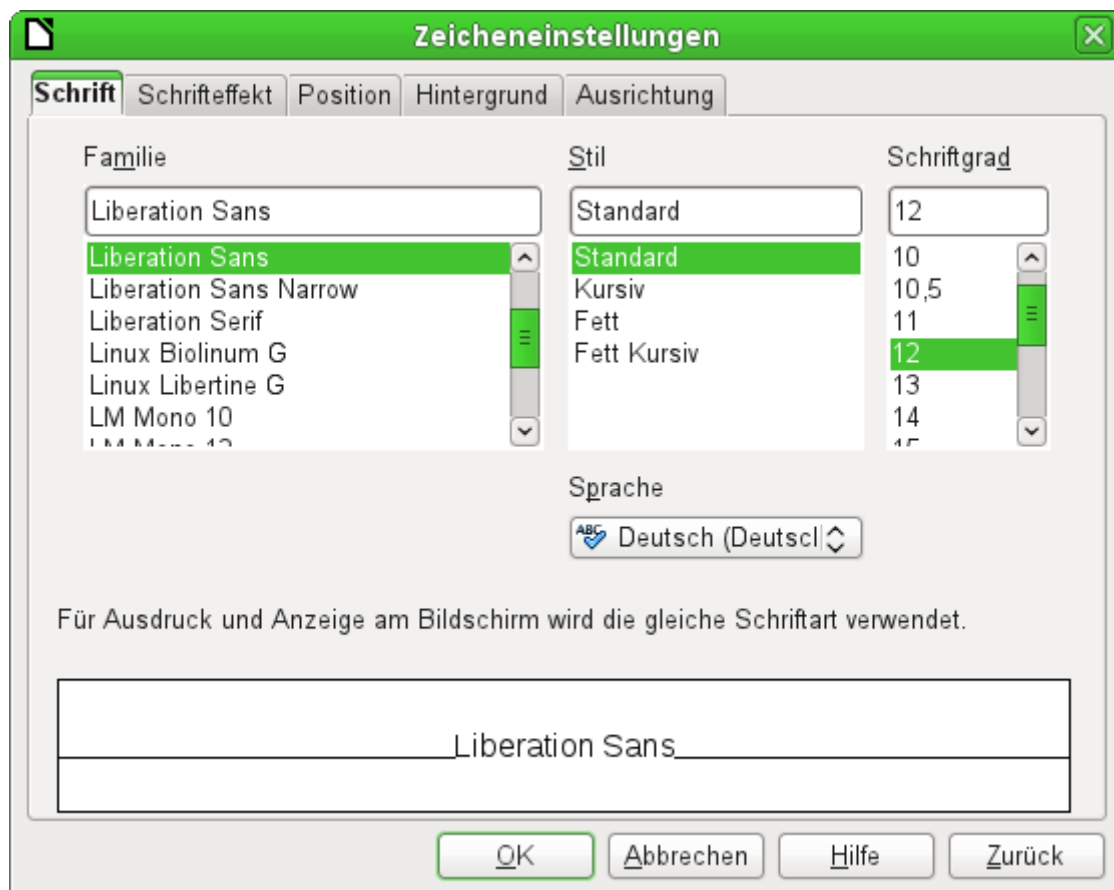


Abbildung 47: Schrift - Zeicheneinstellungen

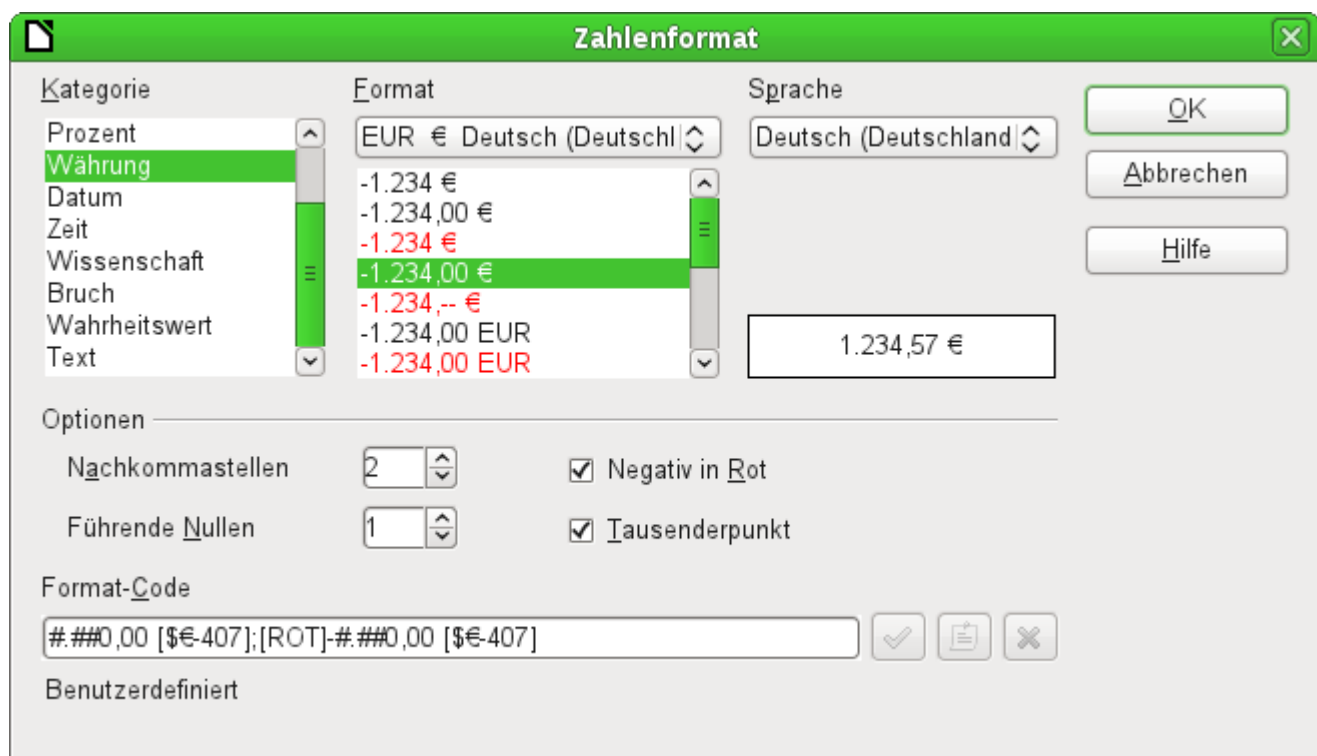


Abbildung 48: Formatierung von Zellen

Besondere Eigenschaften des grafischen Kontrollfeldes

Das grafische Kontrollfeld kann sowohl Grafiken von außerhalb der Datenbank darstellen als auch Grafiken aus der Datenbank auslesen. Leider ist es zur Zeit nicht möglich, eine Grafik dauerhaft in Base zu speichern, um z. B. ein Brieflogo einzulesen. Hierzu muss zwingend die Grafik in dem gesuchten Pfad vorhanden sein, auch wenn die Auswahl anbietet, Bilder ohne Verknüpfung aufzunehmen und das erste Feld mit «*Als Verknüpfung vorbereiten*» auf eine entsprechende geplante Funktionalität schließen lässt.

Alternativ dazu kann natürlich eine Grafik innerhalb der Datenbank selbst gespeichert werden und so auch intern verfügbar bleiben. Sie muss dann aber über die dem Bericht zugrundeliegende Abfrage in einem der Felder zugänglich sein.

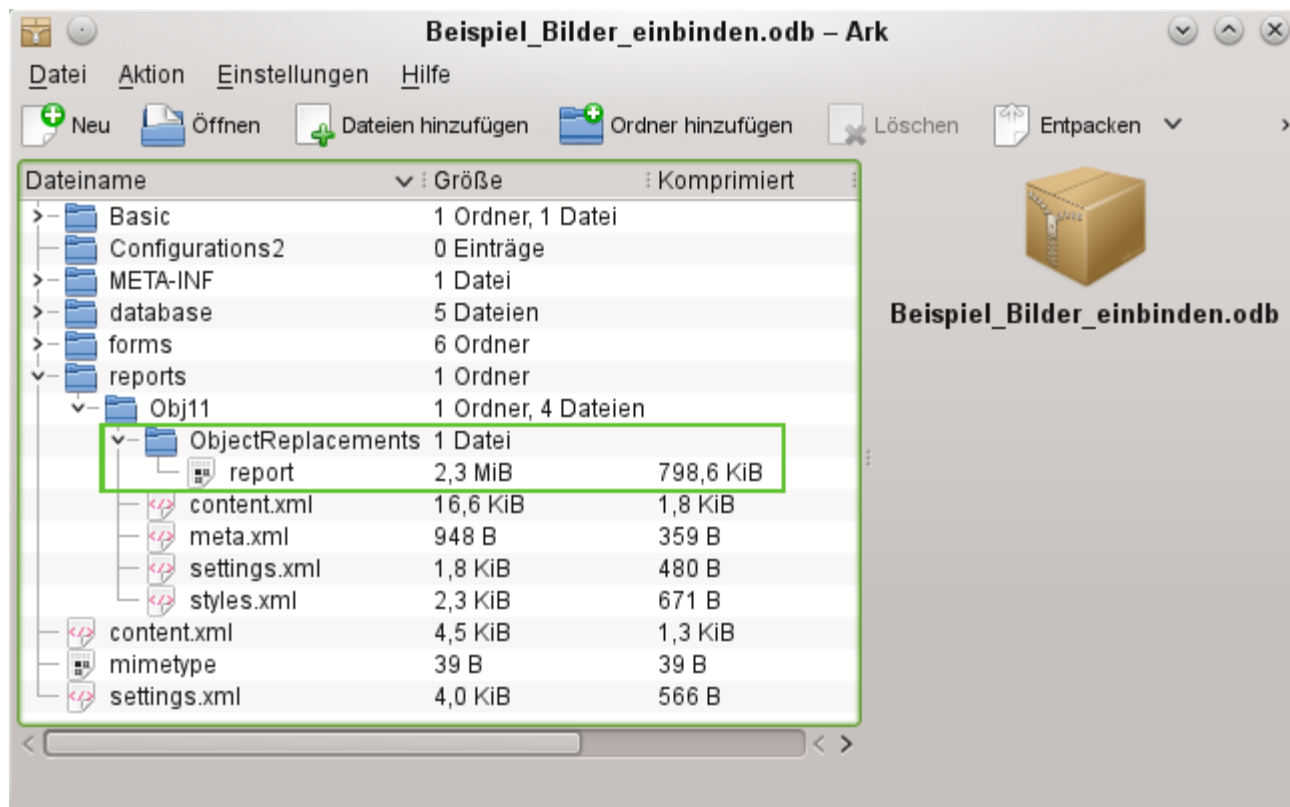
Um eine «*externe Grafik*» aufzunehmen, ist über den Auswahlbutton neben dem Feld «*Grafik*» die Grafik zu laden. Um ein Datenbankfeld auszulesen, muss im Register «*Daten*» das Feld angegeben werden. Dieses Feld kann auch den Pfad zu einer externen Grafik beinhalten. Ist also in der Tabelle ein relativer Pfad zu einer Grafik angegeben, so wird diese Grafik entsprechend auch dargestellt.

Die Einstellung der vertikalen Ausrichtung scheint im Entwurf nichts zu bewirken. Wird allerdings der Bericht aufgerufen, so erscheint die Grafik entsprechend positioniert.

Beim Skalieren kann «*Nein*», «*Seitenverhältnis beibehalten*» und «*Autom. Größe*» gewählt werden. Dies entspricht den Einstellungen im Formular:

- '*Nein*': Das Bild wird nicht an das Kontrollfeld angepasst. Ist das Bild zu groß, so wird ein Ausschnitt des Bildes gezeigt. Das Bild wird nicht verzerrt.
- '*Seitenverhältnis beibehalten*': Das Bild wird in das Kontrollfeld eingepasst, aber nicht verzerrt dargestellt.
- '*Automatische Größe*': Das Bild wird der Größe des Kontrollfeldes angepasst und gegebenenfalls verzerrt dargestellt.

Nach der Bearbeitung von Berichten mit Bildern fällt auf, dass die Datenbankdatei deutlich größer wird. Innerhalb der *.odb-Datei legt Base aus nicht nachvollziehbaren Gründen im Berichtsverzeichnis einen Ordner «ObjectReplacements» an. Dieser Ordner enthält dann eine Datei «report», die für eine entsprechende Vergrößerung der *.odb-Datei sorgt.



Wenn die Datenbankdatei in einem Packprogramm geöffnet wird, ist dieser Ordner mit Inhalt im Verzeichnis «reports» im Unterverzeichnis zu dem jeweiligen Bericht sichtbar. Dieses Verzeichnis kann über das Packprogramm gefahrlos gelöscht werden.

Hinweis

Wenn Berichte nicht wiederholt editiert werden, reicht ein einmaliges Löschen des Verzeichnisses «ObjectReplacements» aus. Der Umfang des Verzeichnisses kann sehr rasch anschwellen. Dies hängt von der Menge und Größe der Dateien ab. Allein eine 2,8MB-jpg-Datei vergrößerte so in einem Test eine *.odb-Datei um 11MB! Der Bug ist hier gemeldet: https://bugs.freedesktop.org/show_bug.cgi?id=80320

Diagramme im Bericht einbinden

Über das entsprechende Kontrollfeld oder **Einfügen** → **Bericht-Steuerelemente** → **Diagramm** lässt sich ein Diagramm dem Bericht hinzufügen. Ein Diagramm wäre hier die einzige Möglichkeit, in einem Bericht Daten wiederzugeben, die nicht aus der Quelle stammen, die im Bericht als Datenquelle angegeben ist. Ein Diagramm ist insofern ein Unterbericht des Berichtes, kann aber auch als eigenständiger Berichtsteil gesehen werden.

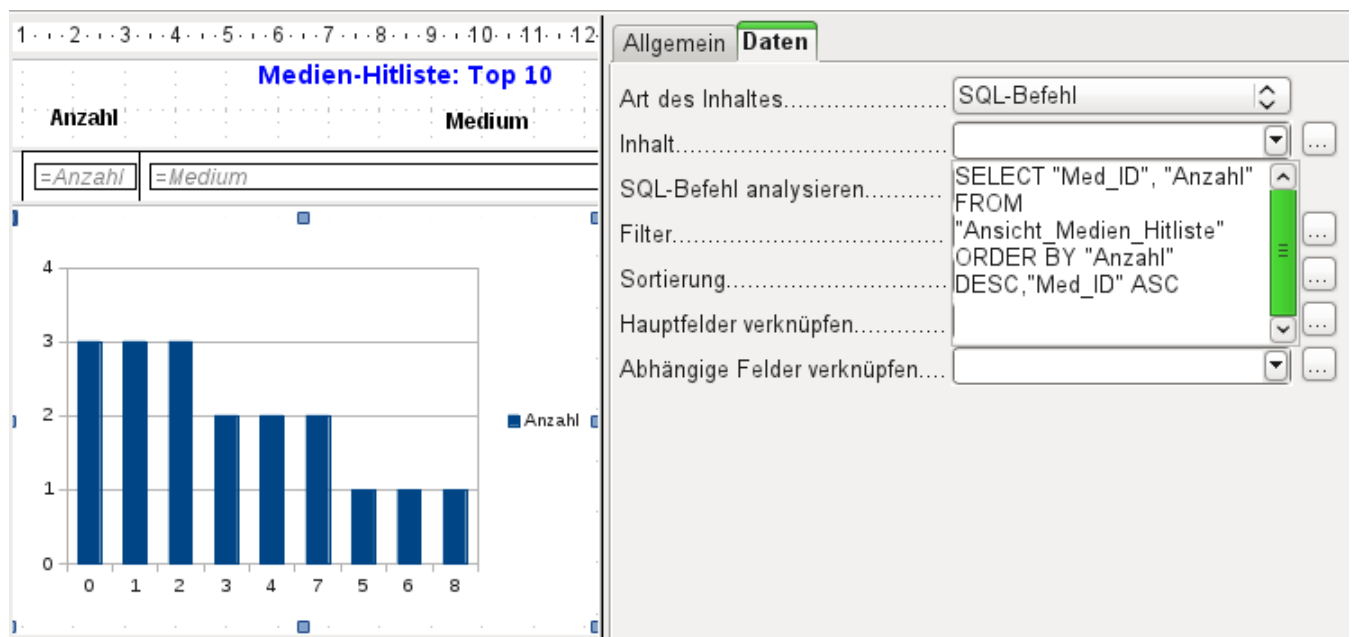
Diagrammtyp..... ...

Vorschau für Zeile(n).....

Das Diagramm wird aufgezogen. Bei den allgemeinen Eigenschaften zeigt sich neben bekannten Feldern die Möglichkeit, einen *Diagrammtypen* auszuwählen (siehe entsprechende Typen in Calc). Außerdem wird eine maximal für die Vorschau benutzte Anzahl an Datensätzen (*Vorschau für*

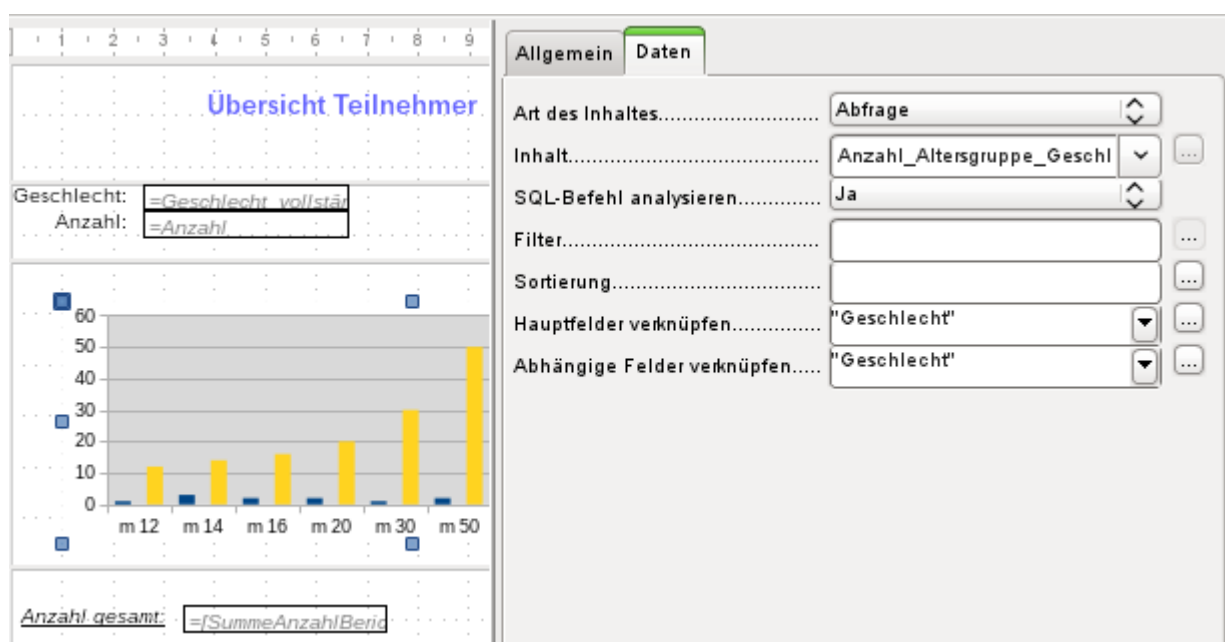
Zeile(n)) eingestellt, die schon einmal eine Vorstellung davon geben kann, wie das Diagramm letztlich aussieht.

Diagramme können, ebenfalls wie in Calc, entsprechend formatiert werden (Doppelklick auf das Diagramm). Hierzu siehe die Beschreibungen im Handbuch Calc.



Das Diagramm wird im Bereich «Daten» mit den erforderlichen Datenfeldern verbunden. Hier, im Beispielbericht «Medien-Hitliste», soll das Diagramm die Häufigkeit deutlich machen, mit der bestimmte Medien entliehen wurden. Dazu wurde ein SQL-Befehl über den Abfrageeditor wie bei Listefeldern erstellt und eingebunden. Die erste Spalte wird als die angesehen, mit der die Säulen beschriftet werden sollen, die zweite Spalte liefert dann die Anzahl der Entleihvorgänge, die sich in den Höhen der Säulen widerspiegelt.

In dem obigen Beispiel zeigt das Diagramm erst einmal nur sehr wenig an, da die Test-Entleihvorgänge sich zum Zeitpunkt der SQL-Eingabe in Grenzen hielten.



In den Dateneigenschaften des Diagramms ist hier eine **Abfrage** eingebunden. Dieses Diagramm aus der Datenbank «Beispiel_Sport.odt»⁹ zeigt neben den Grundlagen für die Erstellung von Diagrammen in Berichten eine Besonderheit auf: Die Vorschau des Diagramms zeigt mehr Säulen auf, als an sich für das Diagramm vorgesehen sind. Dies liegt an der Zusammensetzung der Abfrage, die gleichzeitig mehrere Spalten darstellt, die eben nicht alle im Diagramm selbst dargestellt werden sollen.

Eine Filterung und Sortierung mit den internen Werkzeugen des Report-Designers ist nicht notwendig, weil dies eben schon so weit wie möglich über die Abfrage erledigt wurde.

Tipp

Grundsätzlich sollte die Berichtserstellung von so viel Aufgaben wie möglich entlastet werden. Was vorher durch Abfragen bewerkstelligt werden kann, das muss nachher nicht mehr die im Verhältnis zur Datenbank doch recht träge Ausführung des Berichtes belasten.

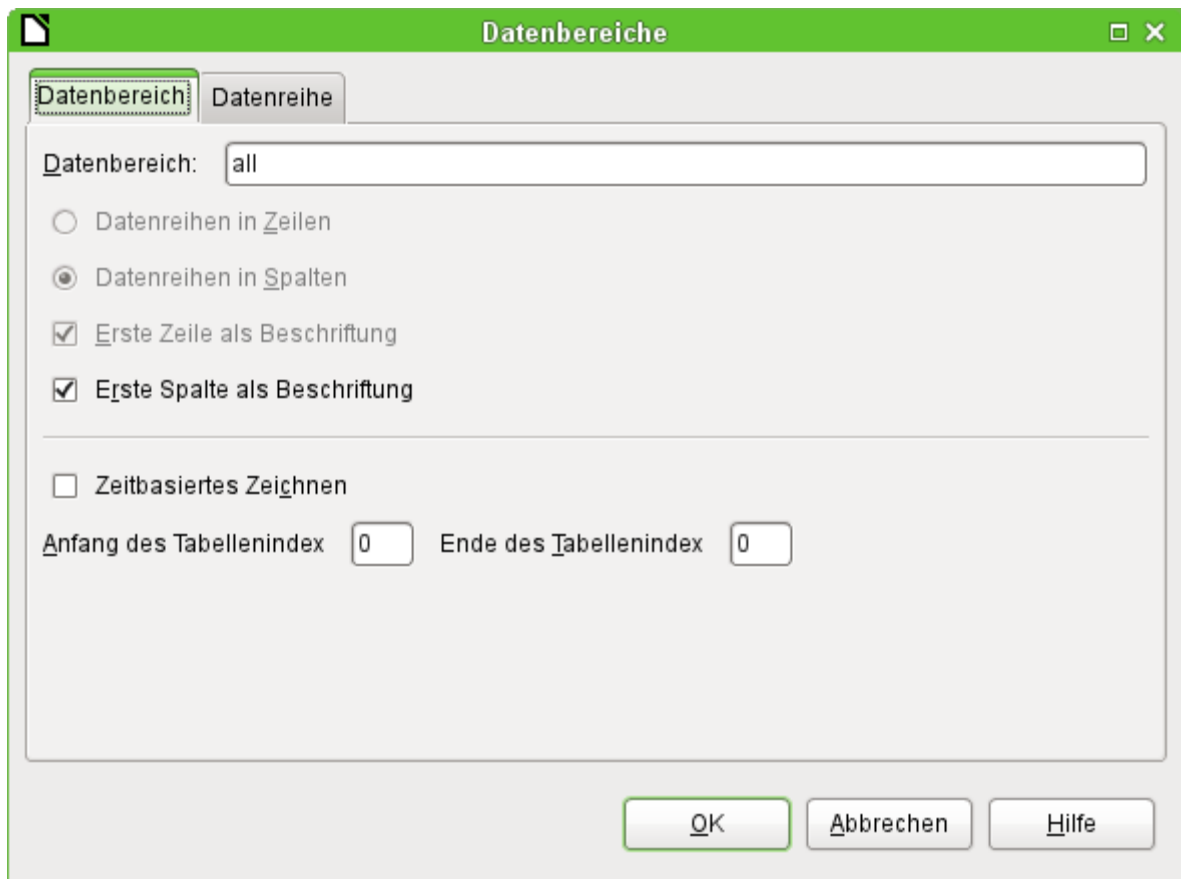
Wie bei Hauptformularen und Unterformularen werden jetzt Felder verknüpft. Im eigentlichen Bericht werden tabellarisch die Altersgruppen für männliche und weibliche Teilnehmer des Sportwettkampfs gelistet. Dabei ist nach den Geschlechtern gruppiert worden. In jeder Gruppe erscheint jetzt ein gesondertes Diagramm. Damit das Diagramm nur die zu dem Geschlecht gehörigen Daten übermittelt, sind die Felder «*Geschlecht*» aus dem Bericht und «*Geschlecht*» aus dem Diagramm miteinander verbunden.

Hinweis

Haben Diagramme die gleiche Datenquelle (Tabelle, Abfrage) wie der Bericht selbst, so funktioniert die Verknüpfung von Hauptfeldern und abhängigen Feldern nicht. Hier muss für eine funktionierende Verknüpfung eine zweite Abfrage gleichen Inhaltes erstellt oder der SQL-Code direkt als Datenquelle für das Diagramm eingegeben werden.

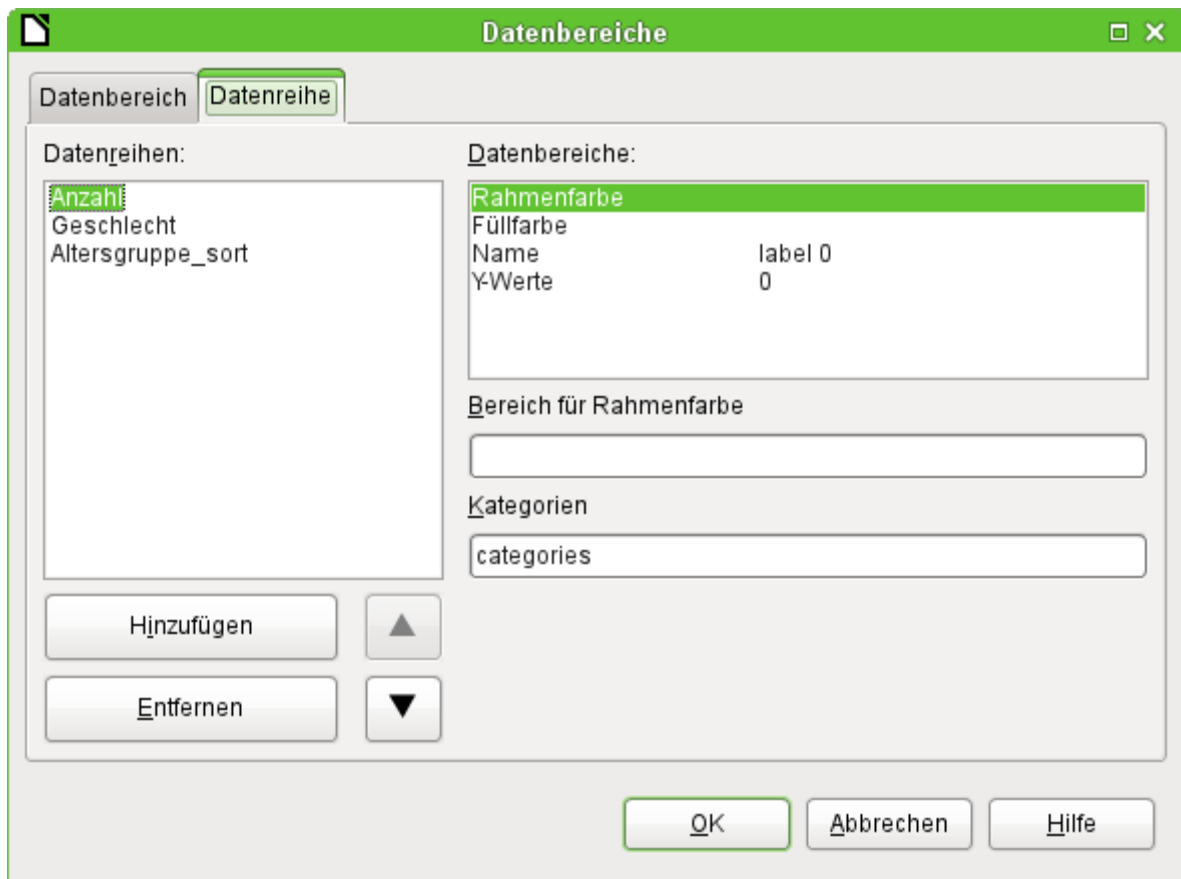
Die x-Achse des Diagramms wird automatisch erst einmal mit der ersten Tabellenspalte der Datenquelle verbunden. Bei mehr als zwei Tabellenspalten stellt die Automatik gleich mehrere Säulen in dem Diagramm dar. Weitere Einstellungen zu dem Diagramm sind zu erreichen, wenn das gesamte Diagramm mit einem Doppelklick markiert wird. Mit einem Klick der rechten Maustaste öffnet sich über dem Diagramm, je nach markiertem Element, ein Kontextmenü. Hierin enthalten ist immer die Einstellmöglichkeit für die Datenbereiche:

⁹ Die Datenbank «Beispiel_Sport.odt» ist den Beispieldatenbanken für dieses Handbuch beigelegt.



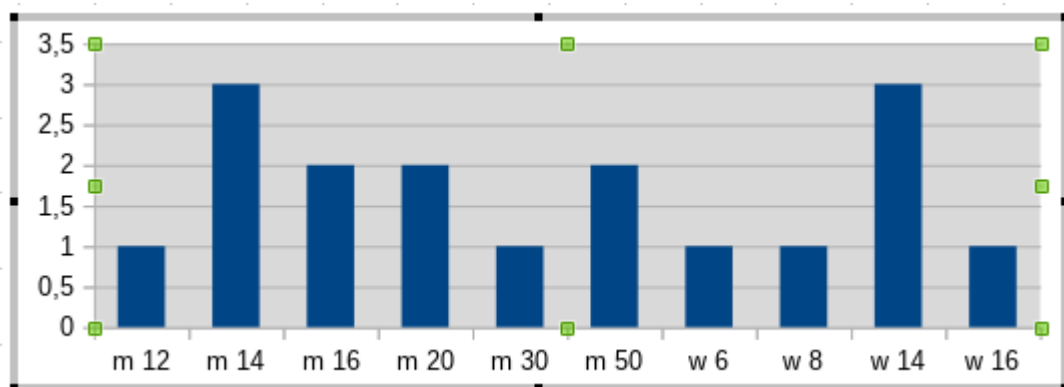
"Datenreihen in Spalten" ist ausgegraut, also nicht änderbar. Auch eine Änderung des Markierfeldes «Erste Spalte als Beschriftung» ist nicht möglich. An den weiteren Einstellungen des Datenbereichs sollten tunlichst keine Änderungen vorgenommen werden, da dieser Dialog mehr Möglichkeiten offeriert, als tatsächlich für den Report-Builder sinnvoll machbar sind.

Der Reiter «Datenreihen» hingegen verbirgt ein paar Einstellungsmöglichkeiten, die die Standarddarstellung deutlich verändern können. Es werden dort alle Datenreihen angeboten, die neben der ersten Spalte der Abfrage noch verfügbar sind. Datenreihen, die nicht angezeigt werden sollen, können hier entfernt werden.



In dem obigen Beispiel waren zu viele Säulen in dem Diagramm sichtbar. Hier muss also nachgebessert werden. Weder die Datenreihe «Geschlecht» noch die Datenreihe «Altersgruppe_sort», deren Bezeichnungen jeweils mit denen in der zugrundeliegenden Abfrage überein stimmt, ergeben hier einen Sinn. Die Datenreihe «Geschlecht» dient schließlich zur Verbindung des Diagramms mit der Datengrundlage des Reports und lässt sich numerisch überhaupt nicht darstellen. Die Datenreihe «Altersgruppe_sort» dient nur zur Sortierung der Werte der Abfrage, da sonst Bezeichnungen wie «m 8» nicht am Anfang des Diagramms sondern direkt vor «m 80» einsortiert würden – die Sortierung eines Textes führt eben manchmal zu unerwünschten Ergebnissen.

Nachdem bis auf die Datenreihe «Anzahl» alle Datenreihen entfernt wurden, sieht die Vorschau des Diagramms so aus:



Die Vorschau zeigt hier 10 Säulen auf – die ersten 10 Säulen aus der Abfrage. In der Ausführung werden allerdings anschließend nur die Säulen dargestellt, die zu der entsprechenden Gruppe passen: «m» für «männlich» und «w» für «weiblich».

Die y-Achse weist noch eine ungünstige Achsenteilung auf. Schließlich kann es nicht halbe Personen geben. Hier könnte zwar nachgebessert werden. Allerdings wird bei der automatischen Fassung dieser Einstellungen schnell eine Anpassung zu Ganzzahlen erfolgen, wenn die Werte eben nicht, wie in obigen Beispiel, bereits bei einer Anzahl von '3' enden. Dann wäre erneut ein Nachbessern notwendig, wenn vorher die Automatik ausgeschaltet wurde.

Hinweis

Aufgrund verschiedener Bugs hat die Anzeige eines Diagramms Probleme mit Datumsfeldern und Zeitfeldern. Hier empfiehlt es sich, die Datums- bzw. Zeitangaben in einer Abfrage in Zahlen umzuwandeln. Eine mögliche Abfrage für ein Datumsfeld in der **HSQLDB** wäre:

```
SELECT DATEDIFF( 'dd', '1899-12-30', "Datum" ) AS "DatumInteger", "Betrag"
FROM "Verkauf"
```

Anschließend muss allerdings die entsprechende Achse des Diagramms bearbeitet werden. Die automatische Erkennung aus der Datenquelle erstellt hieraus eine Zahl. Diese automatische Erkennung muss abgewählt und stattdessen das Datum ausgewählt werden.

Alle weiteren Einstellungen sind ähnlich den Möglichkeiten, die LO-Calc für die Einstellung von Diagrammen bietet.

Hinweis

Die Darstellung von Diagrammen gelingt in LibreOffice zur Zeit nur den Versionen 3.3.*, 3.4.* sowie 4.*. Die Versionen 3.5.* und 3.6.* von LO sind nicht in der Lage, diese Berichte zu öffnen. Bei der Vorversion LO 5.0.0.3 tritt leider erneut ein Fehlverhalten auf, das mit einer langen Fehlerliste das Öffnen des Berichtes aufgibt.

Nicht nur LibreOffice hat hiermit Probleme. Auch in den Versionen ab 3.3 von OpenOffice klappt die Darstellung nicht. OpenOffice öffnet zwar die Berichte – aber eben einfach ohne die Diagramme.

Sollen also Diagramme in einem Bericht auf jeden Fall genutzt werden, so empfiehlt sich zur Zeit immer eine Version installiert zu lassen – zur Not auch als portable Version – die sich beim Öffnen der Diagramme bewährt hat.

Dateneigenschaften von Feldern

The image shows a dialog box with two tabs: 'Allgemein' and 'Daten'. The 'Daten' tab is active. It contains four rows of settings:

- Datenfeld-Typ.....**: A dropdown menu showing 'Feld oder Formel'.
- Datenfeld.....**: A text box containing 'Gebuehr' and a small '...' button to its right.
- Funktion.....**: A dropdown menu.
- Geltungsbereich....**: A dropdown menu.

In den Eigenschaften zeigt sich hinter dem Reiter Daten standardmäßig nur das Feld der Datenbank, aus dem die Daten für die Felder des Berichtes ausgelesen werden. Allerdings stehen neben dem Datenfeld-Typ «Feld» oder «Formel» noch die Typen «Funktion», «Zähler» und «Benutzerdefinierte Funktion» zur Verfügung.

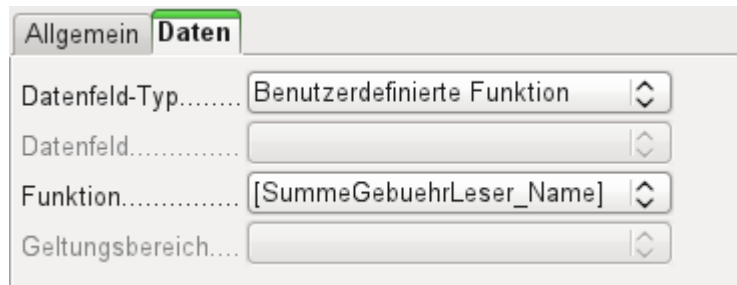
Vorwählbar sind die Funktionen «Summe», «Minimum» und «Maximum». Ihr Geltungsbereich bezieht sich entweder auf die aktuelle Gruppe oder auf den gesamten Bericht. Bereits diese Funktionen können zu Problemen führen, wenn ein Feld leer, also NULL, ist. In solchen Feldern, sofern sie als Zahl formatiert sind, erscheint «N,aN», was wohl so viel wie «kein Zahlenwert» bedeuten soll. Mit leeren Feldern wird keine Rechnung durchgeführt; das Ergebnis ist 0.

Solche Felder können zwar in der Ansicht durch die folgende Formel im Bereich «Daten» zur Anzeige von 0-Werten umformatiert werden.

`IF([Zahlenfeld];[Zahlenfeld];0)`

Die Funktion rechnet aber mit dem tatsächlichen Wert, also mit dem Feld, das eben keinen Wert besitzt. Da erscheint es dann einfacher, wenn gewünscht, die dem Bericht zugrundeliegende Abfrage so zu formulieren, dass statt NULL in Zahlenfeldern '0' wiedergegeben wird.

Der «Zähler» zählt einfach nur die Datensätze, die entweder in einer Gruppe oder im ganzen Bericht enthalten sind. Wird der Zähler in den Bereich Detail eingesetzt, so wird durch ihn jeder Datensatz mit einer fortlaufenden Nummer versehen. Dabei kann die Nummerierung auch hier nur die Gruppe oder sämtliche Datensätze des Berichts fortlaufend nummerieren.



The screenshot shows a configuration window with two tabs: 'Allgemein' and 'Daten'. The 'Daten' tab is active. It contains four fields: 'Datenfeld-Typ' (set to 'Benutzerdefinierte Funktion'), 'Datenfeld' (empty), 'Funktion' (set to '[SummeGebuehrLeser_Name]'), and 'Geltungsbereich' (empty). Each field has a dropdown arrow on its right side.

Schließlich stehen noch die detaillierten *Benutzerdefinierte Funktionen* zur Verfügung. Es kann passieren, dass der Report-Designer diese Variante selbst wählt, wenn er zwar den Rechenschritt vorgegeben bekommt, aber die Datenquelle aus irgendeinem Grunde nicht richtig interpretieren kann. Dann können allerdings dort auch weitere Einstellungen vorgenommen werden.

Alle Funktionen sind anschließend auch über den *Bericht-Navigator* zur weiteren Bearbeitung verfügbar. Dort kann dann z.B. auch eine Vorausberechnung von Werten eingestellt werden. Über den Bericht-Navigator können auch eigene benutzerdefinierte Funktionen für jede Gruppe oder den gesamten Bericht erstellt werden.

Funktionen im Report-Designer

Der Report-Designer bietet sowohl bei den Daten als auch bei den Bedingungen für eine Anzeige die Nutzung verschiedenen Funktionen an. Reichen die Funktionen nicht, so lassen sich mit einfachen Rechenschritten auch benutzerdefinierte Funktionen erstellen, die vor allem in Gruppenfüßen als Zusammenfassungen Sinn machen.

Formeleingaben

Dem Report-Designer liegt der «Pentaho Report Designer» zugrunde. Ein kleiner Teil der Dokumentation ist unter

<http://wiki.pentaho.com/display/Reporting/9.+Report+Designer+Formula+Expressions> zu finden.

Eine weitere Quelle sind die Spezifikationen nach «OpenFormular-Standard»:

<http://www.oasis-open.org/committees/download.php/16826/openformula-spec-20060221.html>

Teilweise ist auch ein einfacher Blick in Richtung Calc sinnvoll, da dort eine deutsche Beschreibung zumindest einiger Formeln aus dem Report-Designer enthalten ist.

Grundlegend gilt:

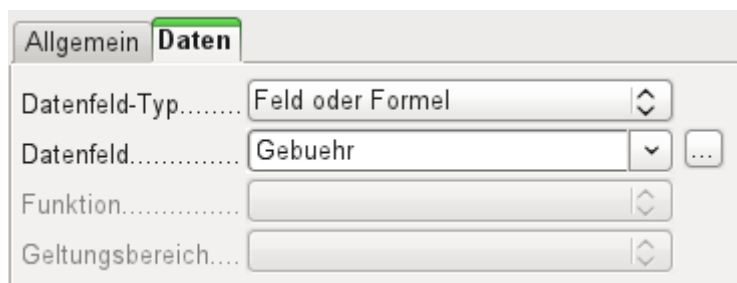
Formeleingaben starten mit einem Gleichheitszeichen	=
Bezüge zu Datenfeldern werden in eckige Klammern gesetzt	[Feldbezeichnung]
Enthalten die Datenfelder Sonderzeichen, zu denen auch Leertasten gehören, so ist die Feldbezeichnung außerdem noch in Anführungsstriche zu setzen	["Diese Feldbezeichnung wäre in Anführungsstriche zu setzen"]
Texteingaben haben immer in doppelten Anführungszeichen zu stehen	"Texteingabe"
Folgende Operatoren sind möglich	+, -, * (Multiplikation), / (Division), % (dividiert vorherstehende Zahl durch 100), ^ (Potenzierung mit der nachfolgenden Zahl), & (verbindet Texte miteinander),
Folgende Beziehungsdefinitionen sind möglich	= , <> , < , <= , > , >=
Runde Klammerungen sind ebenfalls möglich	()
Standardfehlermeldung	NA (vermutlich not available, in Calc in NV übersetzt – nicht verfügbar)
Fehlermeldung beim Bericht, wenn ein leeres Feld auftaucht und als Zahl definiert ist.	N, aN (erscheint, wenn kein Wert enthalten ist – vielleicht not a number?)

Sämtliche Formeleingaben wirken sich nur auf den aktuell eingelesenen Datensatz aus. Beziehungen zu vorhergehenden oder nachfolgenden Datensätzen sind also nicht möglich, wohl aber z. B. die Berechnung einer Summe zwischen Feldern eines Datensatzes. Enthält z.B. ein Datensatz die Felder "Nettobetrag" und "MWSt", so ergibt die folgende Formel den Bruttobetrag:

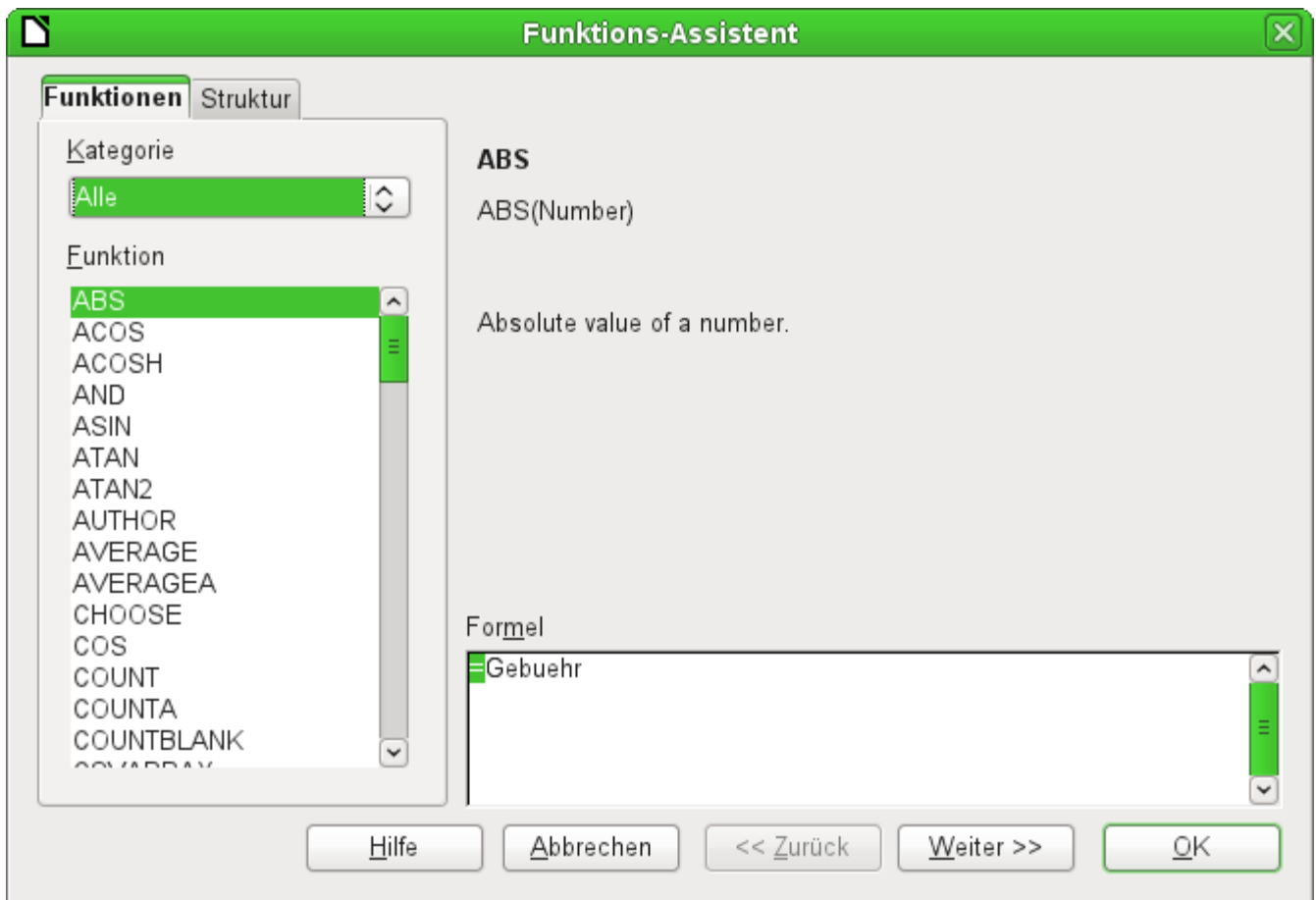
=[Nettobetrag]+[MWSt]

Möglich ist aber auch:

=SUM([Nettobetrag];[MWSt])



Neben dem Datenfeld erscheint ein Button mit 3 Punkten, sofern eine Formeleingabe möglich ist. Dieser Button startet den Funktionsassistenten.



Im Gegensatz zu den Funktionen in Calc sind hier die Funktionen nicht ins Deutsche übertragen. Die Funktionen sind allerdings längst nicht so umfangreich wie die, die in Calc üblich sind. Viele Funktionen haben allerdings ihre Entsprechung in Calc. Dort gibt der Assistent dann auch direkt das Ergebnis der Funktion wieder.

Der Funktions-Assistent funktioniert nicht immer einwandfrei. So werden z. B. Texteingaben nicht mit doppelten Anführungsstrichen übernommen. Nur Eingaben mit doppelten Anführungsstrichen werden allerdings verarbeitet.

Die folgenden Funktionen stehen zur Verfügung: [**Funktion im Report-Designer I** (Funktion in Calc)]

Funktion	Beschreibung
Datums – und Zeitfunktionen	
DATE (DATUM)	Erzeugt aus einer Zahlenangabe für das Jahr, den Monat und den Tag ein gültiges Datum.
DATEDIF (TAGE MONATE JAHRE)	Gibt die Anzahl an Jahren, Monaten oder Tagen zwischen zwei Datumswerten wieder.
DATEVALUE (DATWERT)	Wandelt eine amerikanische Datumseingabe in Textform (Anführungsstriche) in eine Datumsbeschreibung um. Die erzeugte amerikanische Variante kann durch Formatierungseinstellung umgewandelt werden.
DAY (TAG)	Gibt den Tag des Monats eines angegebenen Datums wieder. DAY([Datumsfeld])

DAYS (TAGE)	Gibt die Zahl der Tage zwischen zwei Datumseingaben wieder.
HOURL (STUNDE)	Gibt die Stunde einer angegebenen Zeit im Rahmen von 0 bis 23 wieder. HOUR([DatumZeitFeld]) gibt die Stunde des Feldes wieder.
MINUTE (MINUTE)	Gibt die Minuten einer internen Zahl wieder. MINUTE([Zeitfeld]) gibt den Minutenanteil der Zeit wieder.
MONTH (MONAT)	Gibt den Monat einer Datumseingabe als Zahl wieder. MONTH([Datumsfeld])
NOW (JETZT)	Gibt das aktuelle Datum und die aktuelle Zeit wieder.
SECOND (SEKUNDE)	Gibt die Sekunden einer internen Zahl wieder. SECOND(NOW()) gibt den Sekundenanteil der Zeit beim Ausführen des Berichts wieder.
TIME (ZEIT)	Zeigt die aktuelle Zeit an.
TIMEVALUE (ZEITWERT)	Wandelt die Texteingabe einer Zeit in eine Zeit für Berechnungen um
TODAY (HEUTE)	Gibt das aktuelle Datum wieder
WEEKDAY (WOCHENTAG)	Gibt den Wochentag einer Datumseingabe als Zahl wieder. Der Tag mit der Nummer 1 ist der Sonntag.
YEAR (JAHR)	Gibt das Jahr einer Datumseingabe wieder.
Logische Funktionen	
AND (UND)	Gibt TRUE wieder, wenn alle Argumente TRUE sind
FALSE (FALSCH)	Definiert den logischen Wert als FALSE
IF (WENN)	Wenn eine Bedingung TRUE, dann diesen Wert, ansonsten einen anderen Wert.
IFNA	Wenn der Wert nicht verfügbar ist, dann setze diesen Wert. Ist ähnlich der Anweisung IFNULL in SQL. Vor allem bei Funktionen sinnvoll, die Feldwerte addieren sollen, bei denen aber nicht klar ist, ob jedes Feld einen Wert hat. IFNA([Feldwert];0) rechnet dann trotzdem z.B. die Summe aus. (ab LO 3.5)
NOT (NICHT)	Kehrt den logischen Wert eines Argumentes um
OR (ODER)	Gibt TRUE zurück, wenn eine der Bedingungen TRUE ist.
TRUE (WAHR)	Definiert den logischen Wert als TRUE
XOR	Nur wenn ein einziger Wert der Verknüpfung TRUE ist, ist auch der logische Wert TRUE.

Rundungsfunktionen	
INT	Rundet zum nächsten Ganzzahlwert ab
Mathematische Funktionen	
ABS (ABS)	Gibt den absoluten, nicht negativen Wert einer Zahl wieder.
ACOS (ARCCOS)	Berechnet den Arcuscosinus einer Zahl. - Werteingabe zwischen -1 und 1 (ab LO 3.5)
ACOSH (ARCCOSHYP)	Berechnet den Areacosinus (inverser hyperbolischer Cosinus) – Werteingabe ≥ 1 (ab LO 3.5)
ASIN (ARCSIN)	Berechnet den Arcussinus einer Zahl. - Werteingabe zwischen -1 und 1 (ab LO 3.5)
ATAN (ARCTAN)	Berechnet den Arcustangens einer Zahl. (ab LO 3.5)
ATAN2 (ARCTAN2)	Berechnet den Arcustangens einer x-Koordinate und einer y-Koordinate. (ab LO 3.5)
AVERAGE (MITTELWERT)	Ermittelt den Mittelwert der angegebenen Werte (taucht im Formularassistenten LO 3.3.4 doppelt auf).
AVERAGEA (MITTELWERTA)	Ermittelt den Mittelwert der angegebenen Werte. Text wird als 0 gewertet. (ab LO 3.5)
COS (COS)	Winkel im Bogenmaß, dessen Cosinus berechnet werden soll. (ab LO 3.5)
EVEN (GERADE)	Rundet eine positive Zahl zum nächsten geraden Integer auf, eine negative Zahl zum nächsten geraden Integer ab.
EXP (EXP)	Berechnet die Exponentialfunktion zur Basis 'e' (ab LO 3.5)
LN (LN)	Berechnet den natürlichen Logarithmus einer Zahl. (ab LO 3.5)
LOG10 (LOG10)	Berechnet den Logarithmus einer Zahl zur Basis '10'. (ab LO 3.5)
MAX (MAX)	Gibt den Maximalwert aus einer Reihe von Werten wieder.
MAXA (MAXA)	Gibt den Maximalwert aus einer Reihe wieder. Eventuell vorhandener Text wird als '0' gesetzt.
MIN (MIN)	Gibt den Minimalwert aus einer Reihe von Werten wieder.
MINA (MINA)	Gibt den Minimalwert aus einer Reihe wieder. Eventuell vorhandener Text wird als '0' gesetzt.

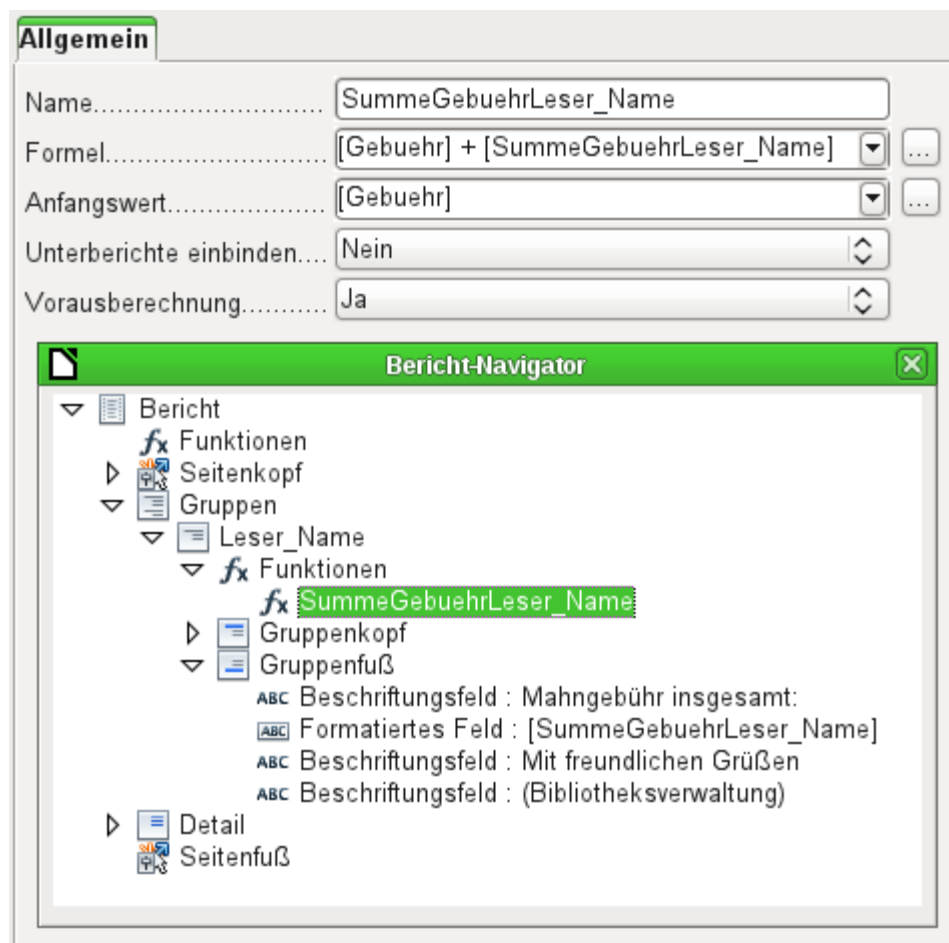
MOD (REST)	Gibt den Rest einer Division nach Eingabe von Dividend und Divisor wieder
ODD (UNGERADE)	Rundet eine positive Zahl zum nächsten ungeraden Integer auf, eine negative Zahl zum nächsten ungeraden Integer ab.
PI (PI)	Liefert den Wert der Zahl 'π' (ab LO 3.5)
POWER (POTENZ)	Liefert aus Basis und Exponent die Potenz. (ab LO 3.5)
SIN (SIN)	Berechnet den Sinus einer Zahl. (ab LO 3.5)
SQRT (WURZEL)	Berechnet die Quadratwurzel einer Zahl. (ab LO 3.5)
SUM (SUMME)	Summiert eine Liste von Zahlenwerten
SUMA	Summiert eine Liste von Zahlenwerten. Text und Ja/Nein-Felder sind erlaubt. Leider endet diese Funktion (noch) mit Fehlermeldung. (ab LO 3.5)
VAR (VARIANZ)	Berechnet die Varianz, ausgehend von einer Stichprobe. (ab LO 3.5)
Textfunktionen	
EXACT (IDENTISCH)	Zeigt an, ob zwei Texte völlig gleich sind.
FIND (FINDEN)	Gibt die Position eines zu suchenden Textes in einem anderen Text an.
LEFT (LINKS)	Der Text wird von links aus in der angegebenen Zeichenzahl wiedergegeben.
LEN (LÄNGE)	Gibt die Anzahl an Zeichen wieder, die ein Text hat.
LOWER (KLEIN)	Gibt den Text in Kleinbuchstaben wieder.
MESSAGE	Formatiert die Werte in dem angegebenen Ausgabeformat. (ab LO 3.5)
MID (TEIL)	Gibt Text ab einer Anfangsposition mit einer gegebenen Zeichenzahl wieder.
REPLACE (ERSETZEN)	In dem Text wird ein Teil durch einen anderen Text ersetzt. Die Startposition und die Länge des zu ersetzenden Textes wird eingegeben.
REPT (WIEDERHOLEN)	Wiederholt einen Text die angegebenen Male.
RIGHT (RECHTS)	Der Text wird von rechts aus in der angegebenen Zeichenzahl wiedergegeben.
SUBSTITUTE (WECHSELN)	Ersetzt in einem vorgegebenen Text bestimmte Textteile durch andere Textteile. Zusätzlich kann angegeben werden, der wievielte Textteil ersetzt werden soll.

T (<i>T</i>)	Gibt den Text wieder oder einen leeren Textwert, wenn es sich z. B. um eine Zahl handelt.
TEXT (<i>TEXT</i>)	Konvertierung von Zahlen oder Uhrzeiten in Text.
TRIM (<i>GLÄTTEN</i>)	Entfernt führende Leerzeichen, Leerzeichen am Textende und reduziert mehrere Leerzeichen hintereinander im Text auf ein Leerzeichen.
UNICHAR (<i>UNIZEICHEN</i>)	Wandelt eine Unicode-Nummer in Dezimalschreibweise in ein Unicode-Zeichen um 196 wird zu 'Ä' ('Ä' hat den Hexadezimalwert 00C4, daraus wird bei der Dezimalschreibweise 196 ohne führende Nullen)
UNICODE (<i>UNICODE</i>)	Wandelt eine Unicode-Zeichen in eine Unicode-Nummer in Dezimalschreibweise um 'Ä' wird zu 196
UPPER (<i>GROSS</i>)	Gibt den Text in Großbuchstaben wieder.
URLENCODE	Wandelt einen vorgegebenen Text in einen Text um, der URL-Konform ist. Wenn keine besonderer Code vorgegeben wurde, passiert dies nach ISO-8859-1.
Informationsfunktionen	
CHOOSE (<i>Wahl</i>)	Zuerst wird ein Index angegeben, danach eine Liste von Werten, aus der über den Index der entsprechende Wert ausgesucht wird. CHOOSE(2;"Apfel";"Birne";"Banane") gibt Birne wieder. CHOOSE([Altersstufenfeld];"Milch";"Cola";"Bier") gibt je nach «Altersstufenfeld» wieder, welches Getränk möglich ist.
COUNT (<i>Anzahl</i>)	Nur die Felder werden gezählt, die eine Zahl oder einen Datums- bzw. Zeitwert enthalten. COUNT([Zeit];[Nummer]) gibt 2 aus, wenn beide Felder einen Wert enthalten, also nicht NULL sind, ansonsten 1 oder schließlich 0.
COUNTA (<i>Anzahl2</i>)	Berücksichtigt auch die Felder, die einen Text beinhalten. Auch NULL wird gezählt, ebenso boolsche Felder.
COUNTBLANK (<i>ANZAHLLEEREZELLEN</i>)	Zählt die leeren Felder in einem Bereich.
HASCHANGED	Überprüft, ob mit ihrem Namen angegebene Spalte sich geändert hat. Allerdings werden keine Spaltenangaben übernommen.
INDEX (<i>INDEX</i>)	Arbeitet mit Bereichen (ab LO 3.5)
ISBLANK (<i>ISTLEER</i>)	Prüft, ob das Feld NULL (leer) ist.
ISERR (<i>ISTFEHL</i>)	Gibt TRUE zurück, wenn die Eingabe fehlerhaft ist, aber nicht vom Typ NA ISERR(1/0) ergibt TRUE
ISERROR (<i>ISTFEHLER</i>)	Wie ISERR, nur dass auch NA als Meldung TRUE ergibt.

ISEVEN (ISTGERADE)	Prüft, ob es sich um einen gerade Zahl handelt.
ISLOGICAL (ISTLOG)	Prüft, ob es sich um einen Ja/Nein-Wert handelt. ISLOGICAL (TRUE ()) oder ISLOGICAL (FALSE ()) ergeben TRUE , Textwerte wie ISLOGICAL ("TRUE") ergeben FALSE .
ISNA (ISTNV)	Prüft, ob der Ausdruck vom Fehlertyp NA ist.
ISNONTEXT (ISTKTEXT)	Prüft, ob es sich bei dem Wert nicht um einen Text handelt.
ISNUMBER (ISTZAHL)	Prüft, ob es sich um eine Zahl handelt. ISNUMBER(1) ergibt TRUE , ISNUMBER("1") ergibt FALSE
ISODD (ISTUNGERADE)	Prüft, ob es sich um eine ungerade Zahl handelt.
ISREF (ISTBEZUG)	Prüft, ob es sich um einen Bezug handelt. ISREF([Feldname]) ergibt TRUE , ISREF(1) ergibt FALSE .
ISTEXT (ISTTEXT)	Prüft, ob es sich bei dem Wert um einen Text handelt.
NA (NV)	Gibt den Fehlercode NA wieder.
VALUE	(ab LO 3.5)
Benutzerdefiniert	
CSVARRAY	Wandelt einen CSV-Text in ein Array um. (ab LO 3.5)
CSVTEXT	Wandelt ein Array in einen CSV-Text um. (ab LO 3.5)
NORMALIZEARRAY	(ab LO 3.5)
NULL	Gibt NULL wieder
PARSEDATE	Wandelt Text in ein Datum um. Nutzt das SimpleDateFormat. Benötigt ein Datum als Text sowie die Beschreibung des Datumformates. Beispiel: PARSEDATE("9.10.2012";"dd.MM.yyyy") ergibt die intern verwertbare Zahl für das Datum. (ab LO 3.5)
Dokumentsinformationen	
AUTHOR	Autor, wird ausgelesen aus Extras → Optionen → LibreOffice → Benutzerdaten . Der eigentliche Autor wird hier also nicht wiedergegeben, sondern der aktuelle Nutzer der Datenbank.
TITLE	Gibt den Titel des Berichts wieder.

Benutzerdefinierte Funktionen

Benutzerdefinierte Funktionen können z. B. dazu dienen, bestimmte Zwischenergebnisse nach einer Gruppe von Datensätzen wieder zu geben. Im obigen Beispiel ist so etwas über die Vordefinierte Funktion der Summe bei der Berechnung der Gebühr im Bereich «Leser_Name_Fuß» automatisch erstellt worden.



Im Bericht-Navigator ist für die Gruppe «*Leser_Name*» eine Funktion verzeichnet. Durch Rechtsklick auf Funktionen können zusätzliche Funktionen hier mit Namen definiert werden.

Die Funktion «*SummeGebuehrLeser_Name*» wird in den Eigenschaften angezeigt. Die Formel führt eine Addition des Feldes «*Gebuehr*» mit dem in der Funktion selbst bereits gespeicherten Wert durch. Der Anfangswert ergibt sich aus dem Wert des Feldes «*Gebuehr*» beim ersten Durchgang durch die Gruppe. Dieser Wert wird in der Funktion unter dem Funktionsnamen zwischengespeichert und in der Formel wieder benutzt, bis die Schleife beendet ist und der Gruppenfuß geschrieben wird.

«Unterberichte einbinden» scheint momentan keine Funktion zu haben, es sei denn, dass Diagramme als Unterberichte verstanden werden.

Ist für die Funktion «*Vorausberechnung*» aktiviert, so kann das Ergebnis auch im Gruppenkopf stehen. Ohne die Aktivierung steht im Gruppenkopf nur der entsprechende Wert des abgefragten ersten Feldes der Gruppe.

Selbstdefinierte Funktionen können auch auf selbstdefinierte Funktionen zurückgreifen. Dabei ist dann aber zu beachten, dass die genutzten Funktionen vorher ausgeführt sein müssen. Wird die Vorausberechnung in dieser Funktion, die auf andere Funktionen zurückgreift, ausgeschaltet, so lässt sich mit der folgenden Kombination im Detail-Bereich der Zensurenschnitt nach jedem Datensatz berechnen.

[SummeZensurKlasse] / [ZählerKlasse]

Wird die Kombination in den Gruppenfuß der Gruppe «*Klasse*» gesetzt, so erscheint dort der Durchschnitt für die gesamte Klasse. Im Gruppenkopf erscheint hingegen als Durchschnitt nur die

Zensur des ersten Schülers, sofern die Vorausberechnung für **SummeZensurKlasse** ausgeschaltet wurde. Mit eingeschalteter Vorausberechnung ist auch im Gruppenkopf der korrekte Wert zu erreichen.

Funktionen wie «ZählerKlasse» zeigen ihren Inhalt nur in der Gruppe an. Werden sie im Seitenkopf oder Seitenfuß genutzt, so bleibt der Inhalt leer.

Formeleingabe für ein Feld

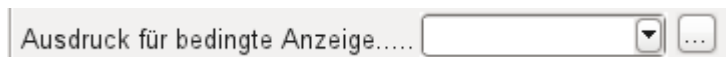
Über den Weg **Daten → Datenfeld** können Formeln eingegeben werden, die nur ein einziges Feld im Bereich «Detail» betreffen.

```
IF([boolschesFeld];"ja";"nein")
```

schreibt, dort eingegeben, statt WAHR und FALSCH einfach "ja" und "nein".

Es kann passieren, dass in einem Feld mit einer Formeleingabe grundsätzlich eine Zahl erscheint. Bei Text ist das dann eine «0». Hier muss nachgebessert werden, indem für das Textfeld vom Standardformat «Zahl» zum Format «Text» gewechselt wird.

Bedingte Anzeige



Gruppenköpfe, Gruppenfüße, Felder – in sämtlichen Untergliederungen befindet sich unter den allgemeinen Eigenschaften das Feld «*Ausdruck für bedingte Anzeige*». Formeln, die in dieses Feld geschrieben werden, beeinflussen den Inhalt eines Feldes oder gleich die Anzeige eines ganzen Bereiches. Auch hier steht der Funktions-Assistent zur Verfügung.

```
[Feldbezeichnung]="true"
```

sorgt dafür, dass der Inhalt nur dann angezeigt wird, wenn der Inhalt des Feldes «Feldbezeichnung» wahr ist.¹⁰ Dies funktioniert auch bei grafischen Steuerelementen.

Grafische Formen reagieren zur Zeit nicht auf Einträge in der bedingten Anzeige. Soll z. B. eine farbige Trennlinie nach dem 10. Platz einer Wettkampfliste eingezogen werden, so geht dies nicht, indem der grafischen Form über die bedingte Anzeige mitgegeben wird

```
[Platz]=10
```

Dieser Befehl wirkt nicht auf die Grafik. Sie erscheint in dem Abschnitt *Detail* dann weiter nach jedem Datensatz.

Dieser Bug ist als [Bug 73707](#) gemeldet.

Soll lediglich eine rechteckige Form bedingt an dieser Stelle eingeblendet werden, so geht dies aber über ein grafische Steuerelement, das mit einer entsprechenden (eventuell einfarbigen) Grafikdatei angesprochen wird. Bei den allgemeinen Eigenschaften wird **Skalieren → Autom. Größe** gewählt. Dann passt sich die Grafik der Form an und die Bedingung greift.

Sicherer ist es, die bedingte Anzeige an einen *Gruppenfuß* statt an die Grafik zu binden, sofern dieser nicht anderweitig benötigt wird. Die Linie wird im *Gruppenfuß* positioniert. Der *Gruppenfuß* wird mit der Bedingung versehen. Dann erscheint die Linie auch tatsächlich nach dem 10. Platz, wenn sie wie oben formuliert wird. Dadurch, dass der Gruppenfuß mit der Bedingung versehen wird, nimmt die Linie auch nur dann den entsprechenden Platz ein, wenn sie wirklich benötigt wird. Sonst erfolgt durch die bedingte Anzeige die Ausgabe eines Leerraumes statt der Linie.

Hier lassen sich dann auch neben den Formen aus **Einfügen → Formen → Standardformen** z. B. die horizontalen Linien mit dem Gruppenfuß ausblenden.

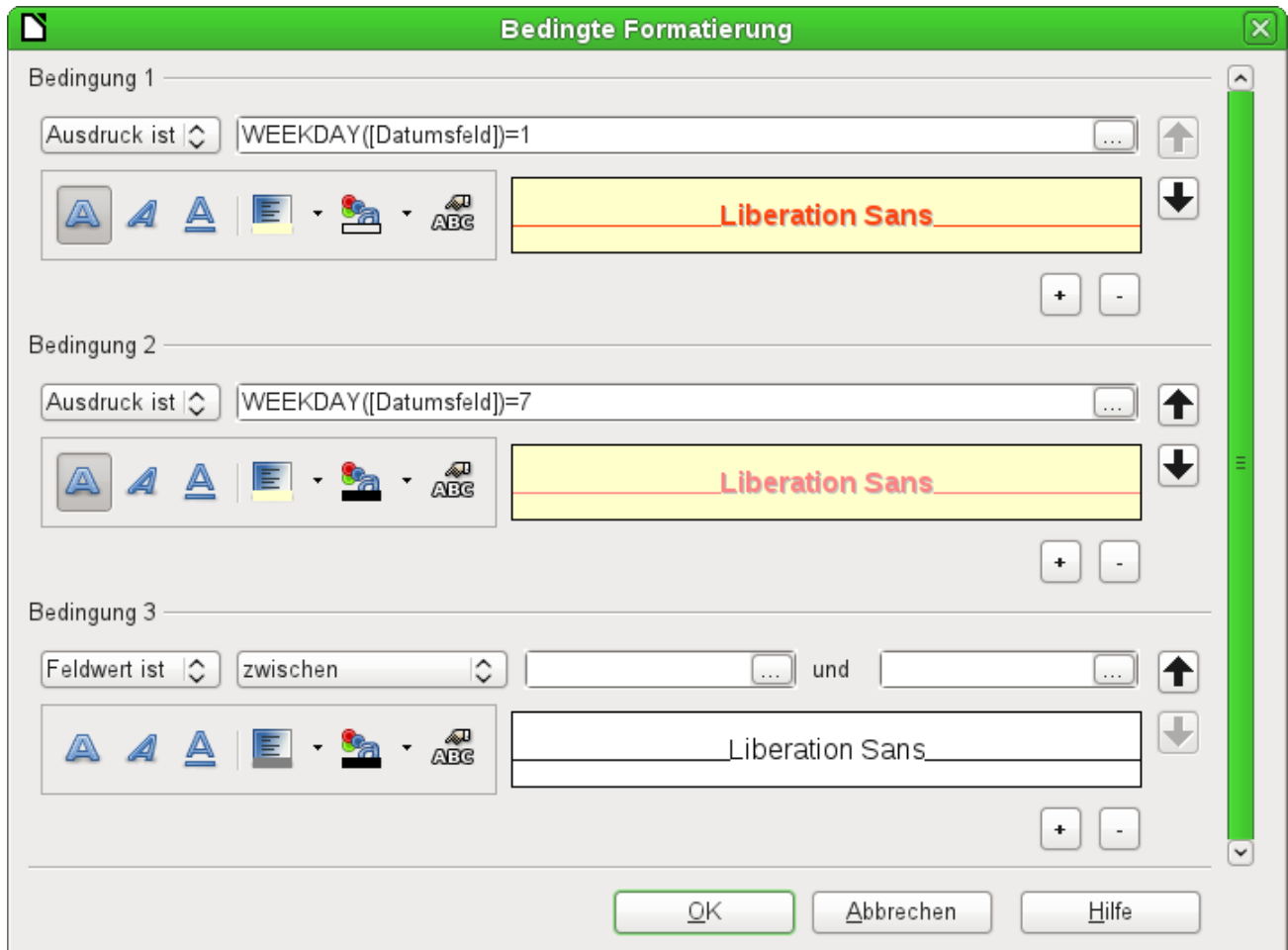
¹⁰ Siehe hierzu die Datenbank «Beispiel_Bericht_bedingte_Einblendung_von_Grafiken.odt», die diesem Handbuch beiliegt.

Bedingte Formatierung

Bei der bedingten Formatierung kann z. B. ein Kalender so formatiert werden, dass die Wochenenden besonders gekennzeichnet werden. Unter **Format** → **bedingte Formatierung** ist dann einzutragen

`WEEKDAY([Datumsfeld])=1`

sowie die entsprechende Formatierung für den Sonntag.



Wird in der bedingten Formatierung «Ausdruck ist» gewählt, so kann eine Formel eingegeben werden. Wie auch in Calc üblich können mehrere Bedingungen formuliert werden, die nacheinander abgearbeitet werden. Im obigen Beispiel wird so zuerst der Sonntag abgefragt und dann der Samstag. Zum Schluss könnte dann noch eine Abfrage nach dem Inhalt des Feldes kommen. So könnte z. B. der Inhalt 'Urlaub' mit einer entsprechend anderen Formatierung angezeigt werden.

Hinweis

Der Report-Designer ist ein Addon. Tauchen anscheinend nicht behebbare Fehler auf (Formel wird nicht umgesetzt, zu langer Text wird als ein leeres Feld angezeigt ...), so empfiehlt es sich manchmal, Teile des Berichts zu löschen oder einfach den Bericht neu zu erstellen.

Beispiele für Berichte mit dem Report-Designer

Der Report-Designer birgt einige Tücken in der Anwendung, da bestimmte Funktionen zwar grundsätzlich vorgesehen sind, aber zur Zeit nicht richtig funktionieren. Auch gibt es eine nur sehr geringe Hilfestellung innerhalb der Hilfefunktion von LibreOffice. Deshalb hier einige recht kom-

plexe Beispiele, wie der Report-Designer für verschiedene Berichtstypen genutzt werden kann. Zum Einstieg in den Report-Builder sei hier noch auf das Einführungskapitel dieses Handbuches verwiesen. Dort wird auf mehreren Seiten Schritt für Schritt ein Bericht erstellt.

Rechnungserstellung

Folgende Vorgaben sollen für die Erstellung der Rechnung¹¹ erfüllt werden:

- Die einzelnen Rechnungspositionen sollen durchnummeriert werden.
- Rechnungen, die mehr als eine Seite umfassen, sollen mit einer Seitennummerierung versehen werden.
- Rechnungen, die mehr als eine Seite umfassen, sollen auf jeder Seite eine Zwischensumme bilden und auf der darauffolgenden Seite diese Zwischensumme als Übertrag darstellen.

Mehrere aktuelle existierende Bugs scheinen das Verfahren unmöglich zu machen:

- **Bug 51452:** Wird eine Gruppierung auf «Bereich wiederholen» eingestellt, so wird automatisch vor und hinter der Gruppierung ein Seitenumbruch eingefügt.
- **Bug 51453:** Gruppierungen mit neu startender Seitenzählung sind zwar eigentlich vorgesehen, funktionieren aber nicht.
- **Bug 51959:** Ein Gruppenfuß lässt sich nicht wiederholen. Er kann nur am Ende einer Gruppe auftauchen, nicht z. B. am Ende einer jeden Seite. Wird er auf «Bereich wiederholen» gesetzt, so verschwindet er komplett.

Außerdem gibt es noch Schwierigkeiten, Linien in den Bericht einzufügen. Die vorgesehenen horizontalen und vertikalen Linien werden erst ab den Versionen LO 4.0.5 bzw. 4.1.1 angezeigt. Als Ersatz dafür können Rechtecke genutzt werden. Die lassen sich aber nicht richtig positionieren, wenn in einem Bereich ein Seitenumbruch stattfindet.

Der Bericht in einer einfachen Form sollte also so aussehen:

¹¹ Die Datenbank «Beispiel_Bericht_Rechnung.odt» ist den Beispieldatenbanken für dieses Handbuch beigefügt. Eine komplette Beschreibung der Datenbank ist in «Base_Beispieler.pdf» verfügbar.

Rechnungsnummer: 2013-0
Datum: 11.03.2013

Anzahl	Ware	Preis	Anzahl*Preis
2	Papier, 500 Blatt	4,23 €	8,46 €
1	Radlergummi	0,75 €	0,75 €
4	Anspritze	1,27 €	5,08 €
2	Bleistift HB	0,23 €	0,46 €
1	Collegoblock	0,98 €	0,98 €
1	Briefumschläge, 25 Stck.	1,25 €	1,25 €
4	CD-Hüllen, 100 Stck.	1,89 €	7,56 €
1	Wachsmalkreiden, 6 Stck.	3,85 €	3,85 €
1	Ordner, 5cm Rückenbreite	1,89 €	1,89 €
2	Klemmbrett	4,45 €	8,90 €
1	Ringbuch A4	5,76 €	5,76 €
1	CD-Beschriftungsstifte, 4 Stck.	4,56 €	4,56 €
2	CD-Rohlinge, 50 Stck.	12,34 €	24,68 €
1	Papier, Recycling, 500 Blatt	3,76 €	3,76 €
4	Ordnermappe mit Register	6,87 €	27,48 €
2	Outlooksbuch, 50 Blatt	1,35 €	2,70 €
10	Rechnungsbuch, 50 Blatt	3,15 €	31,50 €
1	Datumsstempel, einfach	18,50 €	18,50 €
1	Geldkassette, klein	12,00 €	12,00 €
12	Hängemappen, 25 Stck.	14,75 €	177,00 €
2	Universalketten 70x32, 2700 Stck.	20,50 €	41,00 €
1	Universalketten Kleinpaket 12x30, 700 Stck.	4,15 €	4,15 €
2	Druckerköpfe, schwarz	24,00 €	48,00 €
1	Druckerköpfe, color	26,30 €	26,30 €
3	Druckertintenpatronen, schwarz, 32ml	5,40 €	16,20 €
5	Druckertintenpatronen, color, 17ml	5,20 €	26,00 €
Übertrag:			508,77 €

Seite 1

Kontoverbindung: Stadtparkasse Phantastica - BLZ 123 456 01 - Konto 1029384756

Seite 2

Anzahl	Ware	Preis	Anzahl*Preis
2	Toner Laserdrucker schwarz	65,89 €	131,78 €
1	Toner, Laserdrucker, color	78,89 €	78,89 €
2	Register für Ordner, A-Z	1,25 €	2,50 €
1	Heftstreifen, 25 Stck.	0,85 €	0,85 €
2	Schnellhefter Recyclingkarton	0,65 €	1,30 €
1	Papier, 500 Blatt, Recycling	4,15 €	4,15 €
1	Bleistifte, 10 Stck., versch. Stärken	4,85 €	4,85 €
2	Kugelschreiber	1,35 €	2,70 €
1	Wasserfarbkasten, 12 Farben	8,75 €	8,75 €
1	Aquarellkasten, 24 Farben	17,15 €	17,15 €
2	Aquarellpinsel, 3 Stck., versch. Stärken	8,34 €	16,68 €
1	Zeichenblock, A3, 20 Blatt	3,85 €	3,85 €
4	Schreibunterlage 50*70 cm	15,67 €	62,68 €
2	Tonpapier, div. Farben, 50*70 cm	0,45 €	0,90 €
10	Tonkarton, div. Farben, 50*70 cm	0,89 €	8,90 €
1	Datumsstempel, einfach	18,50 €	18,50 €
1	Geldkassette, klein	12,00 €	12,00 €
12	Hängemappen, 25 Stck.	14,75 €	177,00 €
2	Universalketten 70x32, 2700 Stck.	20,50 €	41,00 €
1	Universalketten Kleinpaket 12x30, 700 Stck.	4,15 €	4,15 €
2	Druckerköpfe, schwarz	24,00 €	48,00 €
1	Druckerköpfe, color	26,30 €	26,30 €
3	Druckertintenpatronen, schwarz, 32ml	5,40 €	16,20 €
5	Druckertintenpatronen, color, 17ml	5,20 €	26,00 €
2	Toner Laserdrucker schwarz	65,89 €	131,78 €
1	Toner, Laserdrucker, color	78,89 €	78,89 €
Übertrag:			1.434,52 €

Seite 2

Kontoverbindung: Stadtparkasse Phantastica - BLZ 123 456 01 - Konto 1029384756

Um trotz der oben genannten Einschränkungen eine den Anforderungen entsprechende Rechnungserstellung durchführen zu können, muss genau mit den Seitenmaßen des auszudruckenden Dokumentes gearbeitet werden. In diesem Beispiel wird von einem DIN-A4-Ausgabeformat ausgegangen. Die Gesamthöhe eines Blattes beträgt also 29,7 cm.

Der Bericht muss dazu in mehrere Gruppen unterteilt werden. Zwei Gruppen beziehen sich auf das gleiche Tabellenfeld und enthalten jeweils nur einen entsprechenden Tabellenwert.

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

☐ Seitenrand...

Büroshop
Norderstr. 17, 43219 Phantastica

☐ Rechnungskopf

Rechnungsnummer:
Datum:

Anzahl	Ware	Preis	Anzahl*Preis
=Anzahl	=Ware	=Preis	=Anzahl*Preis

☐ ID Kopf

☐ ID Kopf

Übertrag:

☐ Detail

☐ Rechnung...

☐ Seitenrand...

Kontoverbindung: Stadtparkasse Phantastica - BLZ 123 456 01 - Konto 1029384756

Die folgende Tabelle zeigt die Einteilung der Seite in unterschiedliche Bereiche des Berichtes:

A	Seitenrand oben (nicht im Screenshot zu sehen, über Format → Seite einstellbar)	2,00 cm
B	Seitenkopf (erscheint auf jeder Seite, enthält keine Eingaben der Datenbank, sondern z. B. nur das Firmenlogo sowie die Adresse des Absenders)	3,00 cm
C	Gruppenkopf für die Rechnungsnummer (Nur die Beträge, die zu einer Rechnungsnummer gehören, sollen später auch addiert werden. Der Gruppenkopf erscheint nur zum Beginn der Rechnung.)	2,50 cm
D	Gruppenkopf für die Rechnungsposten (Der Bereich «Detail» wird für andere Inhalte benötigt. Deshalb erfolgt hier eine Gruppierung, die gleichzeitig die Sortierung der Rechnungsinhalte z. B. nach der Eingabe vornimmt. Diese Gruppe besteht immer nur aus einem Wert.)	0,70 cm
E	Gruppenkopf, ebenfalls an Rechnungsposten gebunden (Dieser Bereich wird nur dann angezeigt, wenn so viele Rechnungsposten vorkommen, dass auf jeden Fall eine weitere Rechnungsseite erforderlich ist. Er enthält die Zwischensumme und die Seitennummerierung unten auf der Seite. Nach diesem Bereich erfolgt ein Seitenumbruch.)	2,00 cm
F	Detail-Bereich (Dieser Bereich wird nur dann angezeigt, wenn so viele Rechnungsposten vorkommen, dass auf jeden Fall eine weitere Rechnungsseite erforderlich ist. Er enthält den Übertrag, die Seitennummerierung oben auf der Seite)	2,50 cm
G	Gruppenfuß für die Rechnungsnummer (Hier erfolgt die Ausgabe des Rechnungsbetrages, ggf. auch mit Angabe der Mehrwertsteuer. Der Gruppenfuß erscheint nur zum Schluss der Rechnung.)	1,60 cm
H	Seitenfuß (z. B. Angabe der Kontoverbindung)	1,00 cm
I	Seitenrand (wie «Seitenrand oben» nicht im Screenshot zu sehen, über Format → Seite einstellbar)	1,00 cm

Ein Seitenumbruch soll nur dann erfolgen, wenn zu viele Rechnungsposten vorhanden sind. Für die Rechnungsposten ergibt sich der folgende freie Platz:

	29,70 cm	(DIN A 4)
-	2,00 cm	(Pos. A)
-	3,00 cm	(Pos. B)
-	2,50 cm	(Pos. C)
-	1,60 cm	(Pos. G)
-	1,00 cm	(Pos. H)
-	1,00 cm	(Pos. I)
=	18,60 cm	

Der verbleibende freie Platz kann also maximal $18,60 \text{ cm} / 0,70 \text{ cm} = 26,57\dots$, abgerundet also 26 Rechnungspositionen erfassen.

Sobald die 27. Rechnungsposition vorkommt, muss auf jeden Fall ein Seitenumbruch erfolgen. Das bedeutet, dass dann auf jeden Fall Gruppenkopf E und der Detail-Bereich angezeigt werden müssen. Es muss also ein Zähler für die Rechnungsnummer (Bereich C) eingerichtet werden. Erreicht dieser Zähler die Nummer 27, so wird der Detail-Bereich (F) angezeigt.

Der Zähler der Rechnungsnummer wird wie folgt definiert:



Über den Berichtsnavigator wird die Gruppe «Rechnung_ID» aufgesucht. Die neu zu gründende Funktion heißt einfach «ZählerRechnungsnummer». Die Formel ist «[ZählerRechnungsnummer] + 1». Der Startwert ist 1. Es werden keine Unterberichte eingebunden (die Funktion gibt es gar nicht ...). Es wird auch nicht vorausberechnet. Für die Vorausberechnung wird ein gesonderter Zähler «ZählerKomplett» erstellt.

Der Gruppenkopf E und der Detailbereich F werden angezeigt, wenn insgesamt mehr als 26 Positionen in der Rechnung stehen und die momentane Rechnungsposition 26 erreicht hat. Der Ausdruck für die bedingte Anzeige ist also in beiden Bereichen

`AND([ZählerRechnungsnummer]=26;[ZählerKomplett]>26)`

Der Inhalt dieser Bereiche erscheint also nur, wenn mindestens ein 27. Rechnungsposten zu erwarten ist. Gruppenkopf E erscheint auf der ersten Seite. Er ist mit einem Seitenumbruch nach dem Bereich versehen. Der Inhalt des Bereiches «Detail» wird auf der folgenden Seite angezeigt.

Jetzt ist zu berechnen, wie groß der Anteil des Bereiches ist, der noch auf der ersten Seite angezeigt wird:

	29,70 cm	(DIN A 4)
-	2,00 cm	(Pos. A)
-	3,00 cm	(Pos. B)
-	2,50 cm	(Pos. C)
-	18,20 cm	(Pos. D * 26)
-	1,00 cm	(Pos. H)
-	1,00 cm	(Pos. I)
=	2,00 cm	

Der Gruppenfuß fällt aus der ersten Seite raus, insgesamt 26 Rechnungsposten sind enthalten. Der Gruppenkopf E kann also maximal 2 cm auf der ersten Seite anzeigen. In diesem 2 cm muss auf jeden Fall die Zwischensumme und die Seitenzahl untergebracht werden. Um auf jeden Fall einen korrekten Seitenumbruch zu erzeugen, sollte der Bereich also etwas kleiner sein. In dem Beispiel wurde er auf 1,90 cm eingestellt.

Der Bereich «Detail» wird auf der Folgeseite oben angezeigt. Da auf der Folgeseite der Gruppenkopf der Rechnungsnummer (C) nicht mehr erscheint, kann hier der Bereich «Detail» so viel Platz einnehmen wie der Gruppenkopf, nämlich 2,50 cm. Dann starten die darauffolgenden Rechnungsposten wieder mit der gleichen Einstellung wie auf der Vorseite.

Der Übertrag wird jeweils als einfache Summierung der vorhergehenden Posten erreicht.

Über den Berichtsnavigator wird die Gruppe «Rechnungsnummer» aufgesucht. Die neu zu gründende Funktion heißt einfach «SummePreis». Die Formel ist «[Preis] + [SummePreis]». Der Startwert ist [Preis]. Es werden keine Unterberichte eingebunden. Es wird auch nicht vorausberechnet.

Der Übertrag wird in dem Gruppenkopf E und der Detailbereich F angezeigt. Im Gruppenkopf E ist der Übertrag ganz oben positioniert. Er erscheint auf der ersten Seite unten. Im Detailbereich F ist der Übertrag ganz unten positioniert. Er erscheint auf der zweiten Seite direkt unterhalb der Tabellenköpfe.

Die Seitennummerierung wird ähnlich abgefragt wie die Anzeige des Gruppenkopfes E des Bereiches «Detail».

```
IF([ZählerRechnungsnummer]=26;"Seite 1";"
```

Hiermit wird die Seitennummer auf der ersten Seite erstellt. Weitere IF-Abfragen können dann für die weiteren Seitennummern eingebunden werden.

Die Seitennummer für die Folgeseite wird bei der gleichen Bedingung einfach auf "Seite 2" gesetzt.

Werden die Formeln entsprechend weitgehender formuliert, so können beliebig viele Seiten des Berichtes abgedeckt werden.

Der Ausdruck für die **bedingte Anzeige** wechselt also von

```
AND([ZählerRechnungsnummer]=26;[ZählerKomplett]>26)
```

zu

```
AND(MOD([ZählerRechnungsnummer];26)=0;[ZählerKomplett]>[ZählerRechnungsnummer])
```

Der Gruppenkopf E und der Detailbereich F erscheinen also nur, wenn sich aus der Division des Zählers der Rechnungsnummer mit 26 kein Rest ergibt und die Komplettzahl der Rechnungspos-ten größer ist als der Zähler der Rechnungsnummer.

Der Ausdruck für die **Seitenzahl** wechselt von

`IF([ZählerRechnungsnummer]=26;"Seite 1";"")`

zu

`"Seite "&[ZählerRechnungsnummer]/26`

für die aktuelle Seite bzw.

`"Seite "&([ZählerRechnungsnummer]/26)+1`

als Anzeige für die Folgeseite.

Der folgende Berichtsausdruck ist mit diesen Einstellungen noch nicht zu bewerkstelligen:

Büroshop

Norderstr. 17, 43219 Phantastica

Büroshop - Nordenstr. 17 - 43219 Phantastica

Herrn
Marko Mustermann
Schloßallee 42
06741 Hinterrnberg

Rechnungsnummer: 2013-0
Datum: 11.03.2013

Anzahl	Ware	Preis	Anzahl	Preis
2	Papier, 500 Blatt	4,23 €		8,46 €
1	Radlergummi	0,75 €		0,75 €
4	Ansclizer	1,27 €		5,08 €
2	Bleistift HB	0,23 €		0,46 €
1	Collegoblock	0,98 €		0,98 €
1	Briefumschläge, 25 Stck.	1,25 €		1,25 €
4	CD-Hüllen, 100 Stck.	1,89 €		7,56 €
1	Wachsmalkreiden, 6 Stck.	3,85 €		3,85 €
1	Ordner, 5cm Rückenbreite	1,89 €		1,89 €
2	Kleimbrett	4,45 €		8,90 €
1	Ringbuch A4	5,76 €		5,76 €
1	CD-Beschriftungsstifte, 4 Stck.	4,56 €		4,56 €
2	CD-Rohlinge, 50 Stck.	12,34 €		24,68 €
1	Papier, Recycling, 500 Blatt	3,76 €		3,76 €
4	Ordnungsmappe mit Register	6,87 €		27,48 €
2	Outtingsblock, 50 Blatt	1,35 €		2,70 €
10	Rechnungsbuch, 50 Blatt	3,15 €		31,50 €
1	Datumsstempel, einfach	18,50 €		18,50 €
1	Geldkassette, klein	12,00 €		12,00 €
12	Hängemappen, 25 Stck.	14,75 €		177,00 €
	Übertrag:			347,12 €

Seite 1

Kontoverbindung: Stadtparkasse Phantastica - BLZ 123 456 01 - Konto 1023084796

Büroshop

Norderstr. 17, 43219 Phantastica

Seite 2

Rechnungsnummer: 2013-0
Datum: 11.03.2013

Anzahl	Ware	Preis	Anzahl	Preis
		Übertrag:		347,12 €
2	Universaletiketten 70x32, 2700 Stck.	20,50 €		41,00 €
1	Universaletiketten Kleinpack 12x30, 700 Stck.	4,15 €		4,15 €
2	Druckerköpfe, schwarz	24,00 €		48,00 €
1	Druckerköpfe, color	26,30 €		26,30 €
3	Druckertintenpatronen, schwarz, 32ml	5,40 €		16,20 €
5	Druckertintenpatronen, color, 17ml	5,20 €		26,00 €
2	Toner Laserdrucker schwarz	65,89 €		131,78 €
1	Toner, Laserdrucker, color	78,89 €		78,89 €
2	Register für Ordner, A-Z	1,25 €		2,50 €
1	Heftstreifen, 25 Stck.	0,85 €		0,85 €
2	Schnellhefter Recyclingkarton	0,65 €		1,30 €
1	Papier, 500 Blatt, Recycling	4,15 €		4,15 €
1	Bleistifte, 10 Stck., versch. Stärken	4,85 €		4,85 €
2	Kugelschreiber	1,35 €		2,70 €
1	Wasserfarbkasten, 12 Farben	8,75 €		8,75 €
1	Aquarellkasten, 24 Farben	17,15 €		17,15 €
2	Aquarellpinsel, 3 Stck., versch. Stärken	8,34 €		16,68 €
1	Zeichenblock, A3, 20 Blatt	3,85 €		3,85 €
4	Schreibunterlage 50*70 cm	15,67 €		62,68 €
2	Tonpapier, div. Farben, 50*70 cm	0,45 €		0,90 €
10	Tonkarton, div. Farben, 50*70 cm	0,89 €		8,90 €
1	Datumsstempel, einfach	18,50 €		18,50 €
1	Geldkassette, klein	12,00 €		12,00 €
12	Hängemappen, 25 Stck.	14,75 €		177,00 €
	Übertrag:			1.062,20 €

Seite 2

Kontoverbindung: Stadtparkasse Phantastica - BLZ 123 456 01 - Konto 1023084796

Durch das Adressfeld sind auf der ersten Seite der Rechnung weniger Rechnungsposten enthalten als auf der zweiten Seite. Der Bereich «Detail», der auf der zweiten Seite ganz oben abgebildet ist, ist also deutlich kleiner als der Gruppenkopf für die Rechnungsnummer (C).

Um unterschiedliche Mengen an Rechnungsposten auf der ersten Seite und den Folgeseiten zuzu-lassen, müssen also die Formeln noch einmal angepasst werden.

Die folgende Berechnung sichert ab, dass die entsprechenden Bereiche korrekt angezeigt werden.

`AND(MOD([ZählerRechnungsnummer]-20;24)=0;
[ZählerRechnungKomplett]>[ZählerRechnungsnummer])`

Vom Zähler für die Rechnungsnummer wird die Anzahl der Rechnungsposten auf der ersten Seite subtrahiert. Diese Differenz wird durch die mögliche Anzahl der Rechnungsposten auf der zweiten Seite dividiert. Wenn die Division glatt aufgeht (kein Rest, $\text{MOD} = 0$), dann ist die erste Bedingung für die Anzeige von Gruppenkopf E und der Detailbereich F gegeben. Außerdem muss, wie bereits vorher definiert, der Zähler für die Rechnungsnummer kleiner sein als die insgesamt zu erwartenden Rechnungsposten. Sonst würde ja die Rechnungssumme noch genug Platz auf der entsprechenden Seite haben.

Die mögliche Anzahl der Rechnungsposten auf der zweiten Seite wurde dadurch geringer, dass jetzt auf der zweiten Seite zusätzlich die Rechnungsnummer und das Datum enthalten sind.

Die Seitenzahl wird jetzt etwas einfacher berechnet:

"Seite "&INT([ZählerRechnungsnummer]/24)+1

INT rundet auf die nächste Ganzzahl ab. Die erste Seite enthält maximal 20 Rechnungsposten. Die Division ergibt für die erste Seite also ein Ergebnis < 1 . Abgerundet wird das zu 0. Also muss zu der ermittelten Seitenzahl 1 addiert werden, damit auf der ersten Seite auch 1 erscheint. Entsprechend muss für die zweite Seite 2 addiert werden.

Der Bericht weist in der obigen Form noch einen Schönheitsfehler auf. Ein genauer Blick auf die Rechnungsposten zeigt, dass sich die unteren drei Rechnungsposten gleichen. Dies wurde hier durch einfaches Kopieren der Datensätze erzeugt. Es handelt sich also nicht um die gleichen Datensätze, sondern unterschiedliche Rechnungspositionen, die nacheinander über die Software abgearbeitet wurden. Besser wäre hier, die Waren entsprechend gruppiert anzuzeigen, damit die gleiche Ware nicht mehrmals, sondern nur einmal mit entsprechender Anzahl aufgelistet wird.

Grundsätzlich sollten möglichst viele Berechnungen, Gruppierungen usw. aus dem Report-Designer ausgelagert werden. Deshalb wird für eine entsprechende Gruppierung nicht mit den Gruppen des Report-Designers gearbeitet, sondern mit den Gruppierungsfunktionen des Abfrageeditors. Damit der Report-Designer die Abfrage einwandfrei verarbeiten kann, wird schließlich daraus eine Ansicht gemacht. Der Report-Designer versucht ansonsten, die Abfrage weiter durch seine Gruppierungsvorgaben und Sortierungsvorgaben zu ergänzen, was sehr schnell zu unbrauchbarem Code führt.

Dann ergibt sich die folgende Rechnungsübersicht:

Büroshop Norderstr. 17, 43219 Phantastica			
<u>Büroshop - Norderstr. 17 - 43219 Phantastica</u> Herrn Marko Mustermann Schloßallee 42 05741 Hinfornberg			
Rechnungsnummer: 2013-0 Datum: 11.03.2013			
Anzahl	Ware	Preis	Anzahl*Preis
2	Papier, 500 Blatt	4,23 €	8,46 €
1	Radiergummi	0,75 €	0,75 €
4	Anspritze	1,27 €	5,08 €
2	Bleistift HB	0,23 €	0,46 €
1	Collegblock	0,98 €	0,98 €
1	Brotumschläge, 25 Stck.	1,25 €	1,25 €
4	CD-Hüllen, 100 Stck.	1,89 €	7,56 €
1	Wachsmalkreiden, 6 Stck.	3,85 €	3,85 €
1	Ordner, 5cm Rückenbreite	1,89 €	1,89 €
2	Klemmbrett	4,45 €	8,90 €
1	Ringbuch A4	5,76 €	5,76 €
1	CD-Beschriftungsstifte, 4 Stck.	4,56 €	4,56 €
2	CD-Rohlinge, 50 Stck.	12,34 €	24,68 €
1	Papier, Recycling, 500 Blatt	3,76 €	3,76 €
4	Ordnungsmappe mit Register	6,87 €	27,48 €
2	Outliningsblock, 50 Blatt	1,35 €	2,70 €
10	Rechnungsblock, 50 Blatt	3,15 €	31,50 €
2	Datumsstempel, einfach	18,50 €	37,00 €
2	Geldkassette, klein	12,00 €	24,00 €
24	Hängemappen, 25 Stck.	14,75 €	354,00 €
		Übertrag:	554,62 €
Seite 1			
Kontoverbindung: Stadtparkasse Phantastica - BLZ 123 456 01 - Konto 1020304756			

Büroshop Norderstr. 17, 43219 Phantastica			
Seite 2 Rechnungsnummer: 2013-0 Datum: 11.03.2013			
Anzahl	Ware	Preis	Anzahl*Preis
		Übertrag:	554,62 €
4	Universaletiketten 70x32, 2700 Stck.	20,50 €	82,00 €
2	Universaletiketten Kleinpäck 12x30, 700 Stck.	4,15 €	8,30 €
4	Druckerköpfe, schwarz	24,00 €	96,00 €
2	Druckerköpfe, color	26,30 €	52,60 €
6	Druckertintenpatronen, schwarz, 32ml	5,40 €	32,40 €
10	Druckertintenpatronen, color, 17ml	5,20 €	52,00 €
4	Toner Laserdrucker schwarz	65,89 €	263,56 €
2	Toner, Laserdrucker, color	78,89 €	157,78 €
4	Register für Ordner, A-Z	1,25 €	5,00 €
2	Heftstreifen, 25 Stck.	0,85 €	1,70 €
4	Schnellhefter Recyclingkarton	0,65 €	2,60 €
1	Papier, 500 Blatt, Recycling	4,15 €	4,15 €
1	Bleistifte, 10 Stck., versch. Stärken	4,85 €	4,85 €
2	Kugelschreiber	1,35 €	2,70 €
1	Wasserfarbkasten, 12 Farben	8,75 €	8,75 €
1	Aquarellkasten, 24 Farben	17,15 €	17,15 €
2	Aquarellpinsel, 3 Stck., versch. Stärken	8,34 €	16,68 €
1	Zeichenblock, A3, 20 Blatt	3,85 €	3,85 €
4	Schreibunterlage 50*70 cm	15,67 €	62,68 €
2	Toppapier, div. Farben, 50*70 cm	0,45 €	0,90 €
10	Tonkarton, div. Farben, 50*70 cm	0,89 €	8,90 €
		Summe:	1.439,17 €
Kontoverbindung: Stadtparkasse Phantastica - BLZ 123 456 01 - Konto 1020304756			

Alle Rechnungsposten sind nur einmal enthalten. Aus den ursprünglich auf Seite 1 unten stehenden 12 Hängemappen wurden in der Rechnung jetzt 24 Hängemappen. "Anzahl*Preis" wurde entsprechend angepasst.

Ausdruck von Berichten zum aktuellen Datensatz des Formulars

Gerade bei der Rechnungserstellung wie in dem vorhergehenden Beispiel kann es sinnvoll sein, nach jeder Eingabe der Rechnungsposten einen entsprechenden Ausdruck anzufertigen. In einem Formular soll der Inhalt für die Rechnung bestimmt werden und schließlich der Ausdruck des Einzeldokumentes erfolgen.

Berichte lassen sich nicht über Makros mit einem Filter starten. Allerdings kann die Abfrage, die als Grundlage für den Bericht genutzt wird, vorher gefiltert werden. Dies geht entweder über die Form einer Parameterabfrage

```
SELECT * FROM "Rechnung" WHERE "ID" = :ID
```

oder über eine Abfrage, die über eine einzeilige Filtertabelle mit Daten versorgt wird:

```
SELECT * FROM "Rechnung" WHERE "ID" = (SELECT "Integer" FROM "Filter"
WHERE "ID" = TRUE)
```

Bei der Parameterabfrage muss der Inhalt in ein entsprechendes Dialogfeld nach dem Start des Berichtes eingegeben werden.

Bei der Steuerung über eine Filtertabelle wird der Inhalt der Filtertabelle per Makro geschrieben. Eine separate Eingabe ist also nicht mehr nötig. Diese Möglichkeit ist also für den Nutzer einfacher zu handhaben und soll deshalb im Folgenden beschrieben werden.

Aufbau der Filtertabelle

Die Filtertabelle soll lediglich einen Datensatz enthalten. Deshalb kann das Primärschlüsselfeld ein «Ja/Nein»-Feld sein. Andere Felder der Tabelle werden von der Benennung so gewählt, dass bereits eindeutig ist, was für einen Inhalt sie speichern können. In diesem Beispiel heißt das Feld, das den Primärschlüssel der Tabelle "Rechnung" filtern soll, "Integer", da der Primärschlüssel der Tabelle "Rechnung" eben vom Feldtyp «Integer» ist. Für andere Filterungen können andere Felder zusätzlich eingebaut werden. Der Filter "Integer" kann auch für mehrere Tabellen genutzt werden, da ja vor dem Druck der alte Wert einfach mit dem aktuellen Wert überschrieben wird. Diese Doppelnutzung funktioniert so allerdings nur in einer Einzeldatenbank (Base zusammen mit der internen HSQLDB). In einer Mehrbenutzerdatenbank könnte in dem Moment, in dem die Abfrage des Filters erfolgt, von einem anderen Nutzer der Filterwert in einer normalen Tabelle schon wieder geändert werden.

Feldname	Feldtyp
ID	Ja/Nein [BOOLEAN]
Integer	Integer [INTEGER]

Diese Tabelle wird zum Start mit einem Datensatz gefüllt. Hierzu muss lediglich das Feld "ID" einmal markiert werden, so dass es den Wert «Ja» (oder in SQL «TRUE») hat.

Aufbau des Makros zum Start des gefilterten Berichtes

Das Formular muss für das Aufrufen eines einzelnen Berichtes an irgendeiner Stelle den Primärschlüssel der Tabelle "Rechnung" enthalten. Dieser Primärschlüsselwert wird ausgelesen und über ein Makro in die Tabelle "Filter" übertragen. Anschließend wird in dem Makro noch der gewünschte Bericht gestartet.

```
SUB Filtern_und_Drucken
  DIM oDoc AS OBJECT
  DIM oDrawpage AS OBJECT
  DIM oForm AS OBJECT
  DIM oFeld AS OBJECT
  DIM oDatenquelle AS OBJECT
  DIM oVerbindung AS OBJECT
  DIM oSQL_Anweisung AS OBJECT
  DIM stSQL AS STRING
  oDoc = thisComponent
  oDrawpage = oDoc.Drawpage
  oForm = oDrawpage.Forms.getByName("MainForm")
  oFeld = oForm.getByName("fmtID")
  oDatenquelle = ThisComponent.Parent.CurrentController
  If NOT (oDatenquelle.isConnected()) THEN
    oDatenquelle.connect()
  END IF
  oVerbindung = oDatenquelle.ActiveConnection()
  oSQL_Anweisung = oVerbindung.createStatement()
  stSql = "UPDATE ""Filter"" SET ""Integer"" = '" + oFeld.GetCurrentValue() + "' WHERE ""ID"" = TRUE"
  oSQL_Anweisung.executeUpdate(stSql)
  ThisDatabaseDocument.ReportDocuments.getByName("Rechnung").open
END SUB
```

Das Formular hat in diesem Beispiel den Namen "MainForm". Das Primärschlüsselfeld heißt "fmtID". Dieses Schlüsselfeld muss nicht sichtbar sein, um darauf mit dem Makro zugreifen zu können. Der Wert dieses Feldes wird ausgelesen und mit dem UPDATE-Befehl in die Tabelle "Filter" geschrieben. Anschließend wird der Bericht gestartet. Die Ansicht, auf der der Bericht beruht, wurde entsprechend um eine Bedingung erweitert:


```
... WHERE "Rechnung_ID" = COALESCE((SELECT "Integer" FROM "Filter" WHERE
"ID" = TRUE), "Rechnung_ID") ...
```

Es wird das Feld "Integer" ausgelesen. Falls dieses Feld keinen Wert enthält, wird stattdessen "Rechnung_ID" = "Rechnung_ID" gesetzt. Das bedeutet, dass alle Datensätze angezeigt werden – nicht nur der Filterdatensatz. So können also gegebenenfalls mit der gleichen Ansicht auch alle gespeicherten Rechnungen ausgedruckt werden.

Wechselnde Einfärbung von Zeilen

Tabellenzeilen in einem Bericht können manchmal beim Lesen dazu führen, dass der Blick um eine Zeile nach unten oder oben rutscht. Dagegen kann eine Einfärbung von mindestens einer Zeile helfen. Im folgenden Beispiel¹² werden die Zeilen einfach wechselseitig mit unterschiedlichen Farben eingefärbt. Der Bericht sieht dann wie folgt aus:

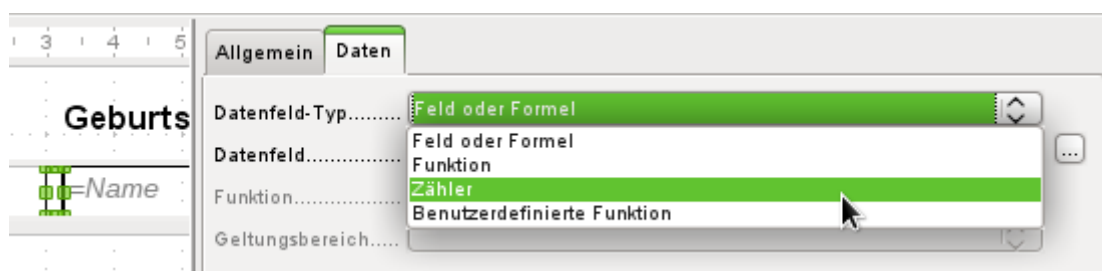
Geburtsstagsliste

Kathrin	17.01.84
Sally	15.02.91
Mick	03.03.53
Hanne	13.04.70
Meike	13.04.71
Lara	23.04.85

Grundlage des Berichtes ist eine Abfrage mit Namen und Datumseinträgen. Es wurde aus der ursprünglichen Tabelle eine Abfrage erstellt, damit die Daten nach Monaten und Tagen sortiert sind, also die Geburtsstagsreihenfolge im Jahr anzeigen. Dies geht über

```
... ORDER BY MONTH("Geburtsstag") ASC, DAY("Geburtsstag") ASC    (HSQLDB)
... ORDER BY EXTRACT(MONTH FROM "Geburtsstag") ASC,
    EXTRACT(DAY FROM "Geburtsstag") ASC    (HSQLDB, FIREBIRD)
```

Um eine wechselseitige Einfärbung zu erreichen muss irgendeine Funktion erstellt werden, aus der heraus über einen Wert später die Bedingung für die Einfärbung abgeleitet werden kann. Hier wird ein Textfeld im Bericht aufgezogen und über die **Eigenschaften → Daten → Datenfeld-Typ** ein Zähler definiert.



Der Name der Funktion wird für die bedingte Formatierung benötigt. Der Zähler selbst braucht beim Ausdruck nicht zu erscheinen. Der Name kann direkt aus dem aufgezogenen Feld abgelesen werden. Ist das Feld wieder gelöscht, so ist der Funktionsname weiterhin über **Ansicht → Berichts-Navigator** nachschlagbar.

¹² Die Datenbank «Beispiel_Bericht_Zeilen_Farbwechsel_Spalten.odt» zu diesem Bericht ist den Beispieldatenbanken für dieses Handbuch beigelegt.

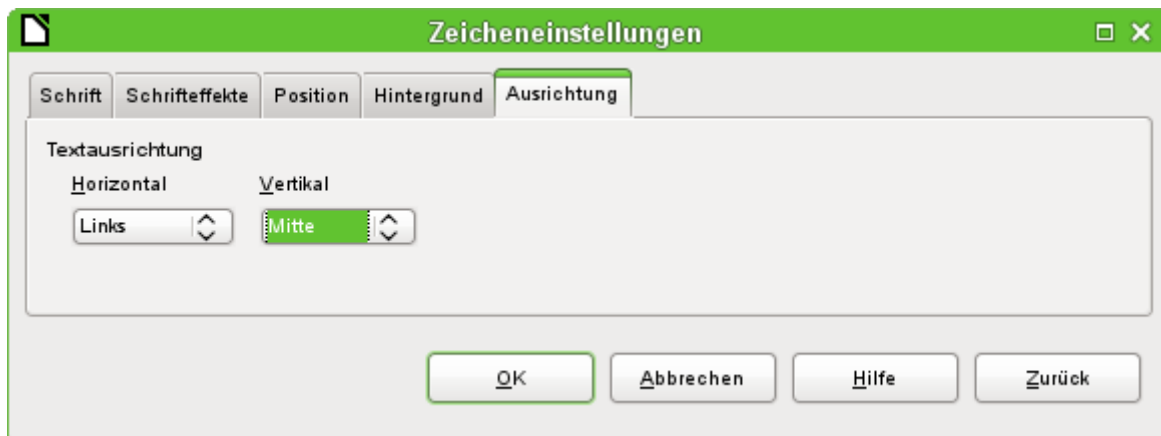


Jetzt muss jedem der Textfelder über den Zähler ein entsprechendes Format zugewiesen werden. Die Bedingung ist jedes Mal ein Ausdruck, der nicht direkt mit dem Feld zusammen hängt. Deshalb wird als **Bedingung 1** → **Ausdruck ist** → **MOD([ZählerBericht];2)>0** eingestellt. MOD berechnet den Rest einer Division. Bei allen ungeraden Zahlen ist der Rest größer als 0, bei allen Geraden Zahlen ist der Rest 0. Zeile 1, 3, 5 usw. wird also mit dem entsprechenden Format versehen.



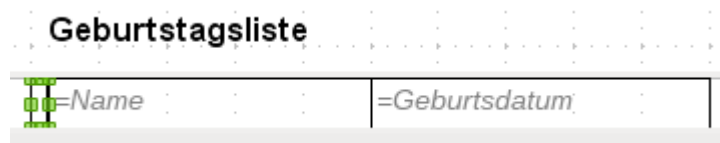
Als zweite Bedingung wird jetzt das Gegenteil von der ersten Bedingung formuliert und eine entsprechende Formatierung zugewiesen. Diese zweite Bedingung könnte auch ausgelassen werden und stattdessen in den Eigenschaften eines jeden Feldes ein entsprechendes Format eingestellt werden. Die bedingte Formatierung der ersten Bedingung würde dann nur die Standardformatierung ersetzen, wenn die erste Bedingung zutrifft.

Da die bedingte Formatierung alle Formate der Standardformatierung überschreibt, muss z. B. eine Ausrichtung der Schrift in den Zeicheneinstellungen der bedingten Formatierung erfolgen:



Hier wird die Schrift vertikal mittig in dem eingefärbten Textfeld dargestellt. Horizontal lassen sich die üblichen Ausrichtungen einstellen, nicht aber eine Einrückung, so dass die Buchstaben nicht am linken Rand des Textfeldes stehen bleiben.

Versuche mit der Verbindung von Leerzeichen mit dem Inhalt in einer Abfrage oder einer Formel führen hier nicht dazu, dass das Textfeld tatsächlich den Text eingerückt darstellt. Die Leerzeichen werden vermutlich einfach abgeschnitten.



Zielführender ist es dagegen, einfach ein Textfeld vor dem eigentlichen Text zu positionieren, das aber nicht mit irgendeinem Datenfeld-Typ verbunden wird. Das Textfeld wird genau wie die anderen enthaltenen Felder mit der entsprechenden bedingten Formatierung versehen, so dass eine einheitlich breite (scheinbare) Einrückung beim Ausdruck erfolgt.

Zweispaltige Berichte

Mit geschickter Abfragetechnik ist es möglich, einen Bericht mit mehreren Spalten zu erstellen, bei dem die folgenden Spalten auch die folgenden Datensätze darstellen:

Kathrin	17.01.84	Sally	15.02.91
Mick	03.03.53	Hanne	13.04.70
Meike	13.04.71	Lara	23.04.85
Monika	05.06.86	Karl	01.07.67
Paul	11.07.89	Egon	23.07.67
Susanne	02.08.65	Georg	28.08.95
Ysabelle	17.09.89	Maik	28.10.93
John	19.11.97	Erkan	17.12.75
Johann	23.12.91		

Der erste Datensatz wird in der linken Spalte aufgeführt, der zweite Datensatz in der rechten Spalte. Die Daten sind nach der Lage des Geburtstages im Jahr sortiert.

Gerade die Sortierung nach Geburtstagen führt dazu, dass die Abfrage für diesen Bericht recht lang ausfällt. Könnte stattdessen eine Sortierung nur nach dem Primärschlüssel der zugrundeliegenden Tabelle erfolgen, so würde der Inhalt der Abfrage stark abnehmen. Bei dem Sortierkriterium handelt es sich um einen immer wiederkehrenden Textblock, der weiter unten erklärt wird.

Grundlage für diesen Bericht ist die folgende Abfrage: (HSQLDB, FIREBIRD (siehe unten))

```

SELECT "T1"."Name" AS "Name1", "T1"."Geburtsdatum" AS "Geburtsdatum1",
"T2"."Name" AS "Name2", "T2"."Geburtsdatum" AS "Geburtsdatum2"
FROM
  (SELECT "Name", "Geburtsdatum", "ZeilenNr" AS "Zeile" FROM
    (SELECT "a".*,
      ( SELECT COUNT( "ID" ) FROM "Geburtstage" WHERE
        RIGHT( '0' || MONTH( "Geburtsdatum" ), 2 ) ||
        RIGHT( '0' || DAY( "Geburtsdatum" ), 2 ) || "ID"
        <= RIGHT( '0' || MONTH( "a"."Geburtsdatum" ), 2 ) ||
        RIGHT( '0' || DAY( "a"."Geburtsdatum" ), 2 ) || "a"."ID" )
      AS "ZeilenNr"
    FROM "Geburtstage" AS "a")
    WHERE MOD( "ZeilenNr", 2 ) > 0 )
  AS "T1"
LEFT JOIN
  (SELECT "Name", "Geburtsdatum", "ZeilenNr"-1 AS "Zeile" FROM
    (SELECT "a".*,
      ( SELECT COUNT( "ID" ) FROM "Geburtstage" WHERE
        RIGHT( '0' || MONTH( "Geburtsdatum" ), 2 ) ||
        RIGHT( '0' || DAY( "Geburtsdatum" ), 2 ) || "ID"
        <= RIGHT( '0' || MONTH( "a"."Geburtsdatum" ), 2 ) ||
        RIGHT( '0' || DAY( "a"."Geburtsdatum" ), 2 ) || "a"."ID" )
      AS "ZeilenNr"
    FROM "Geburtstage" AS "a")
    WHERE MOD( "ZeilenNr", 2 ) = 0 )
  AS "T2"
ON "T1"."Zeile" = "T2"."Zeile"
ORDER BY "T1"."Zeile"

```

Es werden zwei identische Unterabfragen in der Abfrage dargestellt. Die ersten beiden Spalten beziehen sich auf die Unterabfrage mit dem Alias "T1", die letzten beiden Spalten auf die Unterabfrage mit dem Alias "T2".

Die Unterabfragen stellen neben den Feldern der Tabelle "Geburtstage" noch Felder zur Verfügung, die eine Unterscheidung in Zeilen und damit eine Sortierung ermöglichen. Dazu wird im Kern die Abfragemöglichkeit zur [Zeilennummerierung](#) genutzt.

```

RIGHT( '0' || MONTH( "Geburtsdatum" ), 2 ) || RIGHT( '0' ||
DAY( "Geburtsdatum" ), 2 ) || "ID"

```

Diese Formulierung dient dazu, eine eindeutige Reihenfolge der Datensätze zu gewährleisten. Da die Beispieldatensätze nach dem Datum sortiert werden sollen, ließe sich ja leicht sagen, dass lediglich das Datum zum Vergleich herangezogen werden sollte. Etwas schwieriger ist es allerdings schon dadurch, dass nicht das Geburtsdatum, sondern die Lage des Datums im Jahr maßgebend sein soll. Zusätzliche Probleme bereiten schließlich gleiche Datumswerte, die eine eindeutige Reihenfolge verhindern. Deshalb wird zur Sortierung neben dem Monat und dem Tag auch noch der Primärschlüssel der Tabelle herangezogen, der schließlich eindeutig ist. Damit nicht der Monat '10' vor den Monat '2' gesetzt wird, wird bei der Zusammenführung des Sortierkriteriums in einen Text über || vor jede Monatszahl eine führende '0' positioniert, die anschließend bei zweistelligen Monatszahlen über **RIGHT(... , 2)** wieder entfernt wird.

Mit **SELECT COUNT("ID")** wird die Anzahl der Datensätze ermittelt, deren Kombination aus Monat, Tag und Primärschlüssel kleiner oder gleich der entsprechenden Kombination des aktuellen Datensatzes der Tabelle "Geburtstage" ist. Hier handelt es sich um eine [Korrelierte Unterabfrage](#).

Über **MOD("ZeilenNr", 2)** wird aus den so ermittelten Zeilennummerierungen ermittelt, ob es sich um eine gerade oder ungerade Zahl handelt. **MOD** ermittelt den Rest der Division, in dem

genannten Beispiel der Division durch 2. Dadurch gibt die "**ZeilenNr**" wechselweise die Nummern '1' und '0' aus. Hierdurch werden die Abfragen für "T1" und "T2" unterschieden.

In der nächsthöheren Abfrageebene von "T2" wird als "Zeile" "ZeilenNr"-1 definiert. Dadurch werden "T1" und "T2" direkt vergleichbar.

"T1" wird mit "T2" über **LEFT JOIN** verbunden, damit auch bei ungerader Datensatzzahl der Tabelle "Geburtstage" alle Daten dargestellt werden. In der Zusammenführung von "T1" und "T2" kann jetzt auf die Spalten direkt Bezug genommen werden: "**T1"."Zeile"** = "**T2"."Zeile"**".

Zum Schluss wird der ganze Inhalt noch nach dem Wert aus "Zeile" sortiert, der für "T1" und "T2" gleich ist. Dies könnte auch der Bericht über die Gruppierung direkt übernehmen.

Geburtstagsliste

Januar		Februar	
Kathrin	17.01.84	Sally	15.02.91
März		April	
Mick	03.03.53	Hanne	13.04.70
		Meike	13.04.71
		Lara	23.04.85
Mai		Juni	
		Monika	05.06.86
Juli		August	
Karl	01.07.67	Susanne	02.08.65
Paul	11.07.89	Georg	28.08.95
Egon	23.07.67		
September		Oktober	
Ysabelle	17.09.89	Maik	28.10.93
November		Dezember	
John	19.11.97	Erkan	17.12.75
		Johann	23.12.91

Wesentlich komplizierter sind von der Zusammensetzung Abfragetechniken, die neben der zweispaltigen Darstellung auch noch zusätzliche Unterteilungen ermöglichen. Hier fallen dann nämlich Leerzeilen mitten im Bericht an, die bei der vorhergehenden zweispaltigen Darstellung höchstens am Schluss vorkommen. Mit einer so erstellten Abfrage kommt der Report-Builder erst einmal nicht zurecht. Deswegen wird stattdessen auf zwei miteinander verbundene Ansichten zurückgegriffen.

Die folgende Ansicht "MonatZ" wird zuerst erstellt:

```
SELECT "a"."ID", "a"."Name", "a"."Geburtsdatum",
      MONTH( "a"."Geburtsdatum" ) AS "MonatZahl",
      ( SELECT COUNT( "ID" ) FROM "Geburtstage"
        WHERE MONTH( "Geburtsdatum" ) = MONTH( "a"."Geburtsdatum" )
          AND RIGHT( '0' || DAY( "Geburtsdatum" ), 2 ) || "ID" <=
            RIGHT( '0' || DAY( "a"."Geburtsdatum" ), 2 ) || "a"."ID" )
      AS "MonatZaehler"
FROM "Geburtstage" AS "a"
```

Alle Felder der Tabelle "Geburtstage" werden übernommen. Zusätzlich wird der Monat als Zahl dargestellt. Der Tabelle "Geburtstage" wird ein Alias "a" zugewiesen, damit auf die Tabelle mit einer korrelierenden Unterabfrage zugegriffen werden kann.

Die Unterabfrage zählt innerhalb eines Monats alle Datensätze, deren Datumswert einen kleiner oder gleichen Tag vorweist. Bei gleichen Tagen unterscheidet schließlich der Primärschlüssel, welcher Datensatz die niedrigere Nummer erhält. Die Technik ist hier gleich wie bei dem vorhergehenden Beispiel.

Auf die Ansicht "MonatZ" greift die Ansicht "Bericht_Monat_zweispaltig" zu. Diese Ansicht wird hier nur in Ausschnitten wieder gegeben:

```

SELECT
  "Tab1"."Name" AS "Name1",
  "Tab1"."Geburtsdatum" AS "Geburtsdatum1",
  1 AS "MonatZahl1",
  IFNULL("Tab1"."MonatZaehler",999) AS "MonatZaehler1",
  'Januar' AS "Monat1",
  "Tab2"."Name" AS "Name2",
  "Tab2"."Geburtsdatum" AS "Geburtsdatum2",
  2 AS "MonatZahl2",
  IFNULL("Tab2"."MonatZaehler",999) AS "MonatZaehler2",
  'Februar' AS "Monat2"
FROM
  (SELECT * FROM "MonatZ" WHERE "MonatZahl" = 1) AS "Tab1"
  RIGHT JOIN (SELECT * FROM "MonatZ" WHERE "MonatZahl" = 2) AS "Tab2"
  ON "Tab1"."MonatZaehler" = "Tab2"."MonatZaehler"
UNION
SELECT
  "Tab1"."Name" AS "Name1",
  "Tab1"."Geburtsdatum" AS "Geburtsdatum1",
  1 AS "MonatZahl1",
  IFNULL("Tab1"."MonatZaehler",999) AS "MonatZaehler1",
  'Januar' AS "Monat1",
  "Tab2"."Name" AS "Name2",
  "Tab2"."Geburtsdatum" AS "Geburtsdatum2",
  2 AS "MonatZahl2",
  IFNULL("Tab2"."MonatZaehler",999) AS "MonatZaehler2",
  'Februar' AS "Monat2"
FROM
  (SELECT * FROM "MonatZ" WHERE "MonatZahl" = 1) AS "Tab1"
  LEFT JOIN (SELECT * FROM "MonatZ" WHERE "MonatZahl" = 2) AS "Tab2"
  ON "Tab1"."MonatZaehler" = "Tab2"."MonatZaehler"
UNION
...
ORDER BY "MonatZahl1", "MonatZahl2", "MonatZaehler1", "MonatZaehler2"

```

Zuerst werden in der Unterabfrage aus "MonatZ" alle Daten ausgelesen, bei denen die "Monat-Zahl" 1 ist. Diese Auswahl wird mit dem Alias "Tab1" versehen. Gleichzeitig werden mit dem Alias "Tab2" alle Daten ausgelesen, bei denen die "MonatZahl" 2 ist. Beide Tabellen werden durch einen **RIGHT JOIN** verbunden, so dass alle Datensätze aus "Tab2" und nur die Datensätze aus "Tab1" angezeigt werden, bei denen der "MonatZaehler" mit dem von "Tab2" übereinstimmt.

Die Spalten der Ansicht müssen unterschiedliche Bezeichnungen ausweisen, so dass jede Spalte mit einem Alias versehen ist. Außerdem wird direkt als Spaltenwert für "Tab1" eine 1 als "Monat-Zahl" sowie 'Januar' als "Monat1" eingegeben. Diese Einträge erscheinen auch, wenn es keinen Datensatz aus "Tab1", wohl aber noch Datensätze aus "Tab2" gibt. Ist kein "MonatZaehler" vorhanden, so soll dort der Wert 999 eingetragen werden. Da "Tab1" mit "Tab2" über einen **RIGHT JOIN** verbunden ist, kann es ja vorkommen, dass bei weniger Datensätzen in "Tab1" stattdessen leere Felder angezeigt werden. Leere Felder würden aber bei einer späteren Sortierung vor allen Feldern mit Inhalt sortiert, so dass stattdessen ein besonders hoher Wert gewählt wurde.

Bei der Darstellung der Spalten für "Tab2" wird entsprechend vorgegangen. Hier könnte allerdings **IFNULL("Tab2"."MonatZaehler",999)** entfallen, da bei einem **RIGHT JOIN** zugunsten von "Tab2" zwar alle Zeilen aus "Tab2", nicht aber mehr Zeilen aus "Tab1" als aus "Tab2" dargestellt werden.

Genau dieses Problem wird mit der Verbindung zweier Abfragen gelöst. Mit **UNION** werden alle Datensätze aus der ersten Abfrage und alle Datensätze aus der zweiten Abfrage dargestellt. Die Datensätze aus der zweiten Abfrage erscheinen aber nur dann, wenn sie nicht identisch mit einem vorhergehenden Datensatz sind. **UNION** wirkt also wie **DISTINCT**.

Über **UNION** wird die gleiche Abfrage noch einmal gestellt, nur sind "Tab1" und "Tab2" jetzt mit einem **LEFT JOIN** verbunden. Damit erscheinen auf jeden Fall alle Datensätze aus "Tab1" auch wenn in "Tab2" weniger Datensätze vorhanden sind als in "Tab1".

Für die Monate 3 und 4, 5 und 6 usw. werden entsprechend angepasste Abfragen verwendet und wiederum mit **UNION** an die vorhergehenden Abfragen angehängt.

Das Ergebnis der Ansicht wird schließlich nach "MonatZahl1", "MonatZaehler1" und "MonatZaehler2" sortiert. Nach "MonatZahl2" muss nicht sortiert werden, da "MonatZahl1" ja bereits die entsprechende Reihenfolge wiedergibt.

Sollen die Abfragen zur Geburtstagsliste an **FIREBIRD** angepasst werden, so sind Tage und Monate durch **EXTRACT(DAY FROM "Datum")** bzw. **EXTRACT(MONTH FROM "Datum")** zu ermitteln. Außerdem ist statt **IFNULL()** die Funktion **COALESCE()** erforderlich. Hier braucht allerdings nur der Name ausgetauscht zu werden.

Fehlerquellen bei Berichten mit dem Report-Designer

Der Report-Builder birgt manchmal Fehler, die nicht so ohne weiteres nachvollzogen werden können. Hier einige Fehlerquellen und eventuell nützliche Gegenmaßnahmen.

Der Inhalt eines Feldes aus einer Abfrage erscheint nicht

Eine Datenbank soll den Verkauf von Waren simulieren. In einer Abfrage wird aus der Anzahl einer gekauften Ware und dem Einzelpreis der Gesamtpreis ermittelt.

```
SELECT "Verkauf"."Anzahl", "Ware"."Ware", "Ware"."Preis",  
       "Verkauf"."Anzahl"*"Ware"."Preis" FROM "Verkauf", "Ware"  
WHERE "Verkauf"."Ware_ID" = "Ware"."ID"
```

Diese Abfrage dient als Grundlage des Berichtes. Wird aber das Feld **"Verkauf"."Anzahl"*"Ware"."Preis"** in dem Bericht aufgerufen, so bleibt es ohne Inhalt. Wird dem Feld in der Abfrage hingegen ein Alias zugewiesen, so kann der Report-Builder darauf einwandfrei zugreifen:

```
SELECT "Verkauf"."Anzahl", "Ware"."Ware", "Ware"."Preis",  
       "Verkauf"."Anzahl"*"Ware"."Preis" AS "GPreis"  
FROM "Verkauf", "Ware" WHERE "Verkauf"."Ware_ID" = "Ware"."ID"
```

Das Feld greift jetzt auf **"Gpreis"** zu und stellt den entsprechenden Wert dar.

Ein Bericht lässt sich nicht ausführen

Eine Zeit lang konnte der Report-Builder Berichte nicht öffnen, die ein Diagramm enthielten. Es kam dann die Meldung «Failed to parse the report». Hier war als Abhilfe nur möglich, das Diagramm zu entfernen. Dieser Bug ist allerdings schon länger behoben.

Manchmal kommt es aber vor, dass sich ein Bericht zwar erstellen, aber anschließend nicht ausführen oder nicht einmal speichern lässt. Es erscheint eine Fehlermeldung, die erst einmal wenig aussagekräftig ist:

«Bericht konnte nicht ausgeführt werden. Eine Ausnahme vom Typ `com.sun.star.lang.WrappedTargetException` wurde entdeckt.»

Hier kann es schon einmal hilfreich sein, sich die entsprechend angebotenen zusätzlichen Informationen anzeigen zu lassen. Taucht dort irgendwie ein Bezug zu «SQL» auf, so ist vermutlich der Report-Builder nicht in der Lage, den SQL-Code der Datenquelle korrekt zu interpretieren.

Hier hilft es vielleicht, über den Berichtsnavigator die Eigenschaften des Berichtes aufzusuchen: **Daten → SQL-Befehl analysieren → Nein**. Leider hat diese Problemlösung zur Folge, dass eine einmal eingestellte Gruppierung nicht mehr funktioniert.

Ein eventuell besser geeigneter Weg ist, statt einer Abfrage als Grundlage für den Bericht eine Ansicht zu nutzen. Die wird von der Datenbank so erstellt, dass sie für den Report-Builder wie eine Tabelle erscheint und ohne Probleme zu verarbeiten sein sollte. Hier funktioniert sogar die Sortiervorgabe der Ansicht reibungslos.

Datums- und Zeitwerte werden in Diagrammen nicht angezeigt

Leider kam es wiederholt vor, dass Berichte mit Diagrammen im Report-Builder überhaupt nicht angezeigt wurden. Lässt sich ein Bericht mit Diagrammen anzeigen, so kann es vorkommen, dass Achsen mit einer Datums- oder Zeitangabe die Anzeige der Werte für das Diagramm unmöglich machen. Hier hilft die Umwandlung der Datums- und Zeitwerte in einer Abfrage. Aus Datumsangaben werden dadurch Integer-Zahlen, aus Zeitwerten werden dadurch Dezimalzahlen, die die Zeit als Bruchteil eines Tages darstellen.

HSQLDB:

```
SELECT
    DATEDIFF( 'dd', '1899-12-30', "Datum" ) AS "DatumInteger",
    "Temperatur"
FROM "Tabelle"
```

FIREBIRD:

```
SELECT
    "Datum" - DATE '1899-12-30' AS "DatumInteger",
    "Temperatur"
FROM "Tabelle"
```

rechnet den Zeitunterschied in Tagen zu dem internen 0-Wert um. Die Datumszählung beginnt für LO beim 30.12.1899.

HSQLDB:

```
SELECT
    HOUR( "Zeit" ) / 24.000000 + MINUTE( "Zeit" ) / 1440.000000
    AS "ZeitDezimal",
    "Temperatur"
FROM "Tabelle"
```

FIREBIRD:

```
SELECT
    ("Zeit" - TIME '00:00')/86400.000000 AS "ZeitDezimal",
    "Temperatur"
FROM "Tabelle"
```

wandelt die Zeitangabe in Bruchteile eines Tages um. Dabei muss darauf geachtet werden, dass die Division durch Dezimalzahlen mit Nachkommastellen erfolgt. Sonst werden nur ganze Tage wiedergegeben.

Die Integerangaben für das Datum bzw. die Dezimalzahlen für die Zeit können für die Achsen des Diagramms als Datum bzw. Zeit formatiert werden. Die dargestellten Werte sind korrekt. Das Diagramm wird auch gezeichnet.

Andere Formen der Berichterstellung

Der Report-Builder ist zur Zeit die einzige Möglichkeit, über die grafische Benutzeroberfläche direkt Berichte zu gestalten. Nur wenn der Report-Builder nicht installiert ist, kommt eine ältere Möglichkeit der Berichterstellung zutage.

Das alte Berichtsmodul

Bevor der Report-Designer entwickelt wurde, ist ein anderes Modul für die Erstellung von Berichten entwickelt worden. Dieses Modul kann Berichte gruppiert darstellen, ist aber nur zu tabellarischen Berichten in der Lage. Ist der Report-Designer nicht mit LO installiert worden, so steht dieses Berichtsmodul weiterhin für die Erstellung von Berichten zur Verfügung. Ein Aufruf der so erstellten Berichte ist auch mit installiertem Report-Designer weiterhin möglich.

Die BaseReportExtension

Unter <https://extensions.libreoffice.org/extensions/basereportextension> stellt Georg Mößlacher eine Berichtsmöglichkeit zur Verfügung, die er seit Jahren selbst nutzt. Diese Berichtsmöglichkeit benötigt eine Writer-Vorlagendatei (Endung: .ott). Diese Vorlagendatei wird mit mindestens einer zweizeiligen Tabelle an beliebiger Stelle versehen, die über den Tabellennamen aus der Extension heraus gefunden und mit Inhalten gefüllt wird. Die erste Zeile der Tabelle enthält die Spaltenbeschriftungen, in den nachfolgenden Zeilen wird der Inhalt eingefügt. Fehlende Tabellenzeilen werden über die Extension hinzugefügt.

Mit der BaseReportExtension kann auf den Inhalt geöffneter Formulare Bezug genommen werden. Auch lässt sich der Inhalt der Ausgabe über eine Input-Box filtern. Die Extension ist damit vor allem für so etwas wie die Erstellung von Rechnungen gut geeignet. Die Extension ist allerdings nicht in der Lage, Berichte wie im Report-Builder zu gruppieren.

Weitere Berichtsmöglichkeiten

Weitere Möglichkeiten, einen Bericht zu erstellen, sind im Kapitel «Makros» > «Datenbankaufgaben mit Makros erweitert» > «Drucken aus Base heraus» beschrieben. Auch die diesem Handbuch beigelegten Base-Beispiele enthalten Möglichkeiten des Seriendrucks und des Rechnungsdrucks in Tabellenform mit dem Writer incl. der Erstellung eines Feldes, das beim Seitenwechsel einen Übertrag ermittelt.

Datenbank-Anbindung

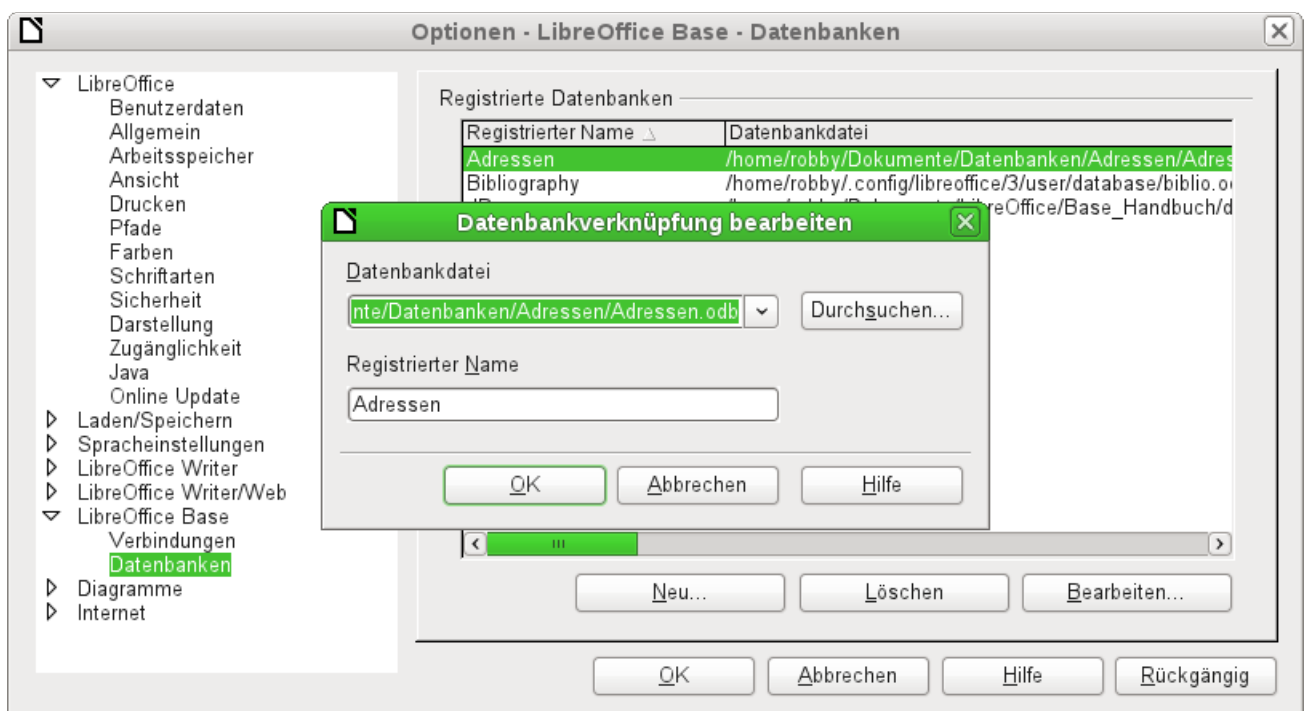
Allgemeines zur Datenbank-Anbindung

Base lässt sich in LibreOffice Writer und LibreOffice Calc auf verschiedene Weise als Datenquelle nutzen. Die Nutzung von Base ist dabei nicht unbedingt an eine Anmeldung der Datenbank in den Einstellungen von LibreOffice gebunden. So interagieren externe Formulare auch direkt mit Base, sofern dort als Datenquelle der Weg zur Datenbank angegeben wird.

Anmeldung der Datenbank

Für viele Funktionen wie z.B. die Nutzung beim Etikettendruck, die Nutzung von Daten für Serienbriefe usw. ist die Anmeldung einer Datenbank in den Einstellungen von LibreOffice notwendig.

Über **Extras → Optionen → LibreOffice Base → Datenbanken → Neu** lässt sich eine Datenbank für den weitergehenden Gebrauch in anderen LibreOffice-Komponenten anmelden.



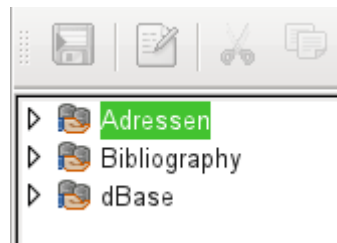
Die Datenbank wird über den Dateibrowser gesucht und auf ähnliche Weise mit LibreOffice verknüpft wie sonst bei einem einfachen Formular. Der Datenbank selbst wird allerdings ein eingängigerer Name gegeben, der ohne weiteres anders lauten kann als der Name der eigentlichen Datenbankdatei. Die Namensbezeichnung ist so etwas wie ein Alias-Name, was ja auch bei Abfragen in Datenbanken vergeben werden kann.

Datenquellenbrowser

Über den Datenquellenbrowser erhalten sie Zugriff auf Tabellen und Abfragen der registrierten Datenbanken unter dem dort angegebenen registrierten Namen. Der Browser öffnet sich über **Ansicht → Datenquellen** oder nach dem Drücken der Taste F4 oder über das entsprechende Symbol in der Standard-Symboleiste.



Auf der linken Seite im Datenquellen-Browser sind die angemeldeten Datenquellen zu sehen. Die Datenquelle "Bibliography" ist standardmäßig bei LibreOffice mit dabei. Die weiteren Datenquellen unterscheiden sich je nach angemeldetem Namen.



Durch einen Klick auf das + - Zeichen vor den Datenbankbezeichnungen öffnet sich die Datenbank und zeigt an, dass sich darin ein Unterordner für Abfragen und ein Unterordner für Tabellen befindet. Die weiteren Unterordner der Datenbank werden nicht zur Verfügung gestellt. Interne Formulare sowie Berichte sind also nur über die Datenbank selbst verfügbar.

Erst beim Klick auf den Ordner Tabellen erfolgt der tatsächliche Datenbankzugriff. Bei passwortgeschützten Datenbanken erfolgt zu diesem Zeitpunkt die Passwortabfrage.

Rechts von dem Verzeichnisbaum zeigt sich die angewählte Tabelle. Sie kann wie in Base selbst bearbeitet werden. Bei stärker ausgeprägten relationalen Datenbanken ist allerdings die Eingabe in die Tabelle mit Vorsicht zu bewerkstelligen, da die Tabellen ja über Fremdschlüssel miteinander verknüpft sind. Allein die unten abgebildete Datenbank zeigt eine separate Tabelle für die Straßennamen, eine für Postleitzahlen und noch eine für den Ort.

Für den richtigen Blick auf die Daten ohne entsprechende Editiermöglichkeit eignen sich daher Abfragen oder entsprechende Ansichten (Views) besser.

	ID	Vorname	Nachname	GebDat	Hausnummer	strID	plzID	Telefon	Geschlecht
	0	Rob	van Delft	27.07.77	137	2	4	09871/1	m
1	1	Mirina	Milinda	08.07.09	27 b	1	5	487531	w
2	2	Heinz	Tunichtgut	24.12.95	159 b	0	2	0375/12	m
3	3	Moni	Hastnicht	03.07.91	37	3	4		w
4	4	Kerstin	Springinsfeld	27.02.68	45	5	5	0587643	w
5	5	Tunicht	Gut	31.12.04	71	0	5		m
6	6	Karl	Springinsfeld	05.01.76	12	0	5		m
8	8	Anna	Ahaus	17.08.61	1	1	2		w
9	9	Berta	Blocker	09.10.02	2	2	3		w
10	10	Cecilia	Cologne	03.09.94	3	3	4		w
11	11	Dorothea	Düse	23.11.57	4	1	5		w
12	12	Elfriede	Erkelenz	15.07.46	5	2	4		w
	<Auto								

Datensatz 1 von 12

Von den Zeichen in der Symbolleiste sind viele bereits aus der Eingabe von Daten in Tabellen bekannt. Neu ist insbesondere der letzte Abschnitt: «Daten in Text, Daten in Felder, Seriendruck, Aktuelle Dokument-Datenquelle, Explorer ein/aus».



Abbildung 49: Symbolleiste Datenquellenbrowser

Daten in Text

Die Funktion «Daten in Text» steht zur Verfügung, sobald ein Datensatz markiert wurde.

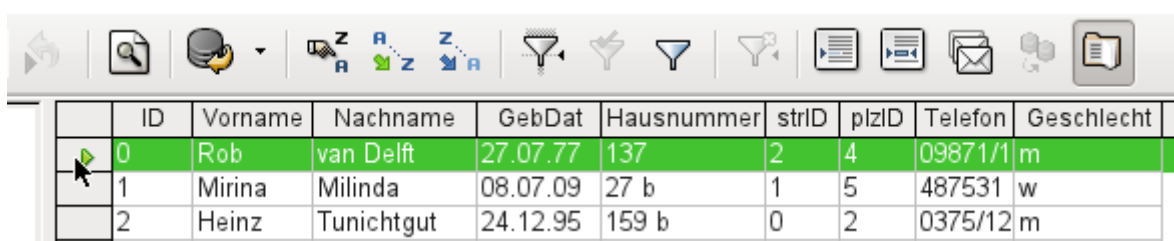


Abbildung 50: Markierung eines Datensatzes

Wird jetzt «Daten in Text» angewählt, so erscheint ein Assistent, der die erforderlichen Formatierungen vornimmt:

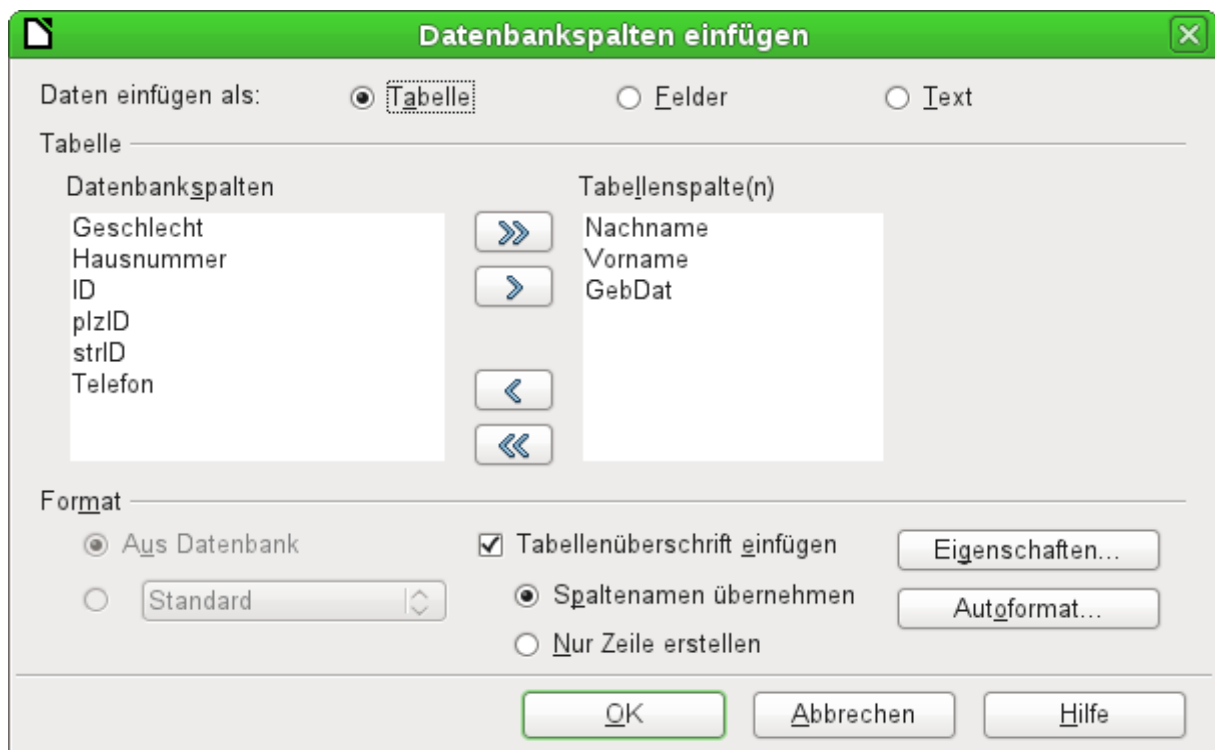


Abbildung 51: Daten Einfügen als Tabelle

Für das Einfügen von Daten in den Text stehen die Möglichkeiten der Darstellung in Tabellenform, als einzelne Felder oder als Gesamttext zur Verfügung.

Die obige Abbildung zeigt den Aufbau «**Daten einfügen als Tabelle**». Bei Zahlenfeldern und Datumsfeldern kann das Format der Datenbank durch ein eigenes gewähltes Format geändert werden. Ansonsten erfolgt die Formatierung mit der Auswahl der Tabellenfelder. Die Reihenfolge der Felder wird über die Pfeiltasten festgelegt.

Sobald Tabellenspalten gewählt sind, wird der Button **Eigenschaften** für die Tabellen aktiviert. Hier können die üblichen Tabelleneigenschaften des Writer eingestellt werden (Breite der Tabelle, Spaltenbreite ...).

Das Markierfeld gibt darüber Auskunft, ob eine Tabellenüberschrift gewünscht ist. Ist dieses Markierfeld nicht angewählt, so wird für die Überschrift keine separate Zeile eingefügt.

Die so eventuell gewählte Zeile für die Tabellenüberschrift kann entweder die Spaltennamen übernehmen oder nur die Zeile erstellen und den Platz für die Überschrift zum späteren Editieren schaffen.

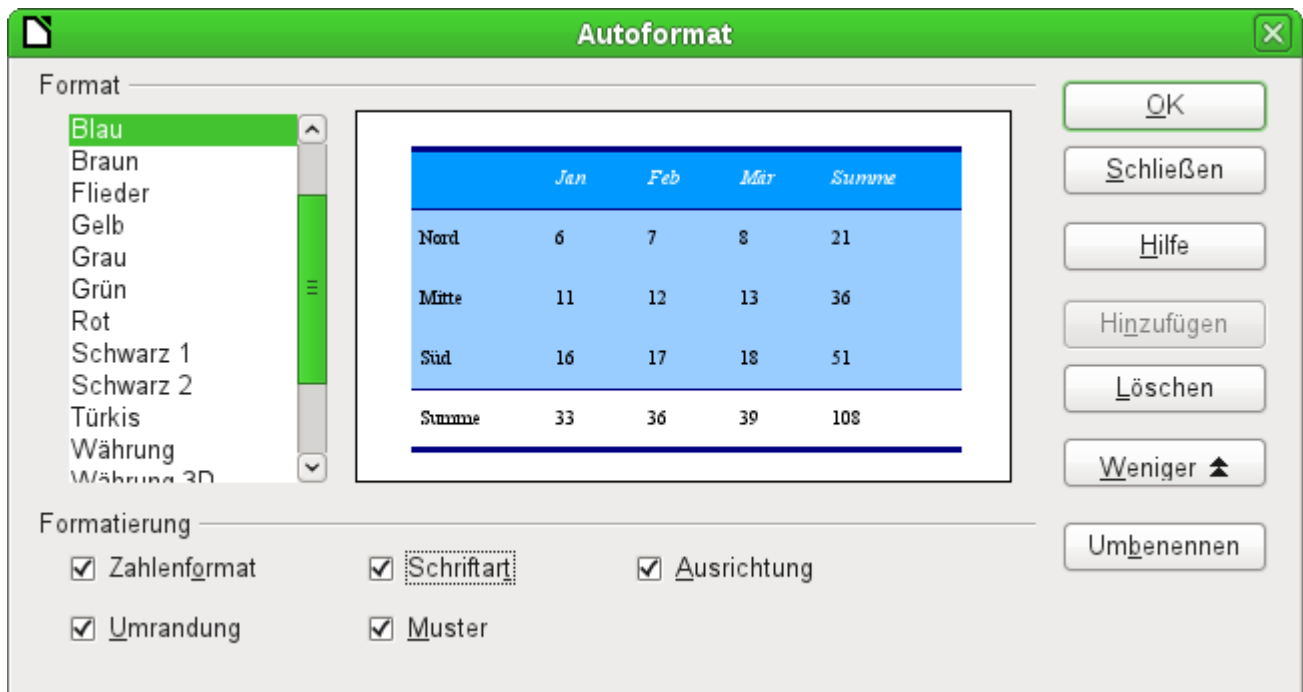


Abbildung 52: Über «Autoformat» stehen Tabellenformate zur Auswahl

Über den Button **Autoformat** werden einige vorformatierte Tabellenansichten angeboten. Bis auf das Vorschlagsformat «Standard» können alle hier enthaltenen Formatierungen umbenannt werden.

Um dem Autoformat-Vorschlag für Tabellen einer Tabelle hinzuzufügen, muss eine Tabelle erstellt worden sein. Diese wird dann markiert und kann über den Button **Hinzufügen** in die Tabellenliste aufgenommen werden.

Die Tabelle wird schließlich mit der Anzahl der markierten Zeilen erstellt.

Über «**Daten einfügen als Felder**» wird die Möglichkeit geboten, in einem kleinen Editor die verschiedenen Tabellenfelder hintereinander in einem Text zu positionieren. Dem erstellten Text kann außerdem eine Absatzvorlage zugewiesen werden. Auch hier ist die Formatierung von Datums- und Zahlenwerten separat wählbar, kann aber auch direkt aus den Tabelleneinstellungen der Datenbank gelesen werden.

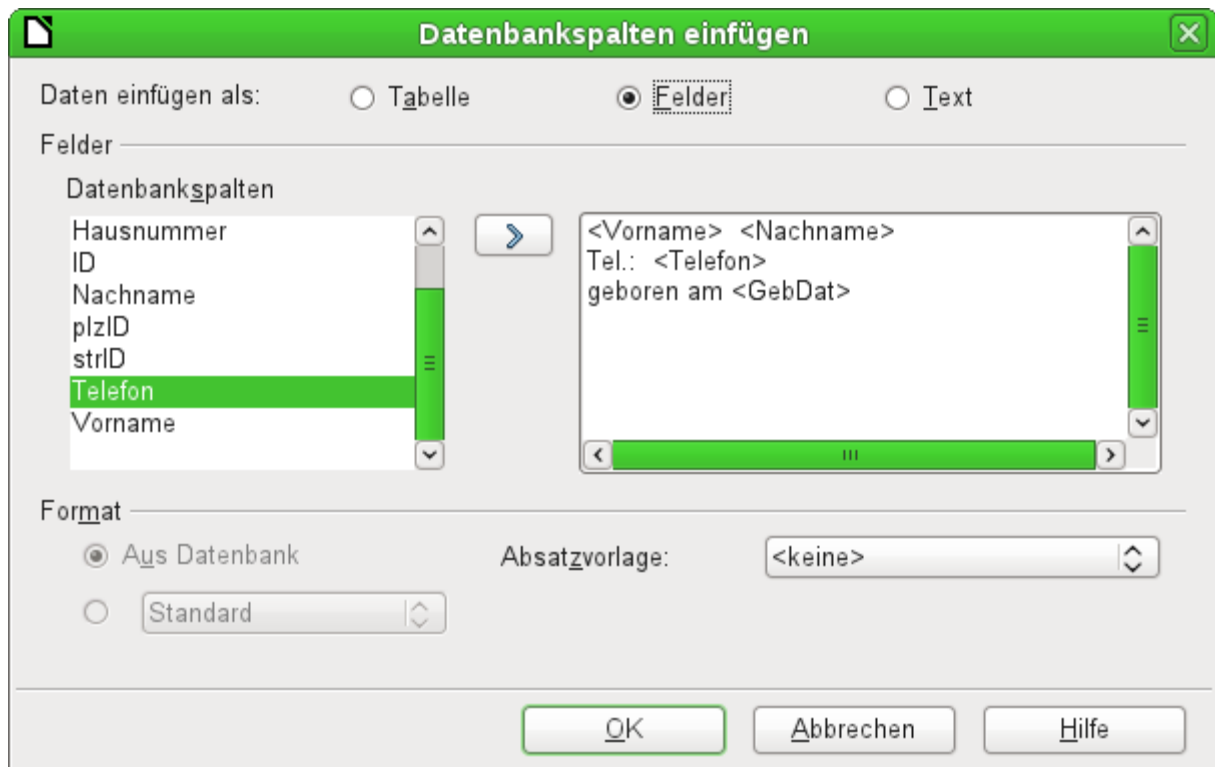


Abbildung 53: «Daten einfügen als Felder» – entspricht auch dem Fenster für «Daten einfügen als Text»

Die auf diese Weise in den Text eingefügten Felder können nachher auch einzeln gelöscht oder als Serienbrieffelder weiter genutzt werden.

Wird **Daten einfügen als Text** gewählt, so unterscheidet sich das gegenüber den Feldern nur in sofern, als bei den Feldern weiterhin die Datenbankverknüpfung bestehen bleibt. Bei der Einfügung als Text wird lediglich der direkte Inhalt der jeweiligen Felder übernommen, nicht aber die Verknüpfung zur Datenbank selbst. Daher unterscheidet sich auch das Fenster für diese Funktion nicht von dem Fenster der vorhergehenden.

Das Ergebnis der beiden Funktionen sieht im Vergleich so aus:

Daten einfügen als Felder	Daten einfügen als Text
Rob van Delft	Rob van Delft
Tel.: 09871/1946703	Tel.: 09871/1946703
geboren am 27.07.17	geboren am 27.07.77
Adressen.Person.GebDat	

Abbildung 54: Vergleich: «Daten als Felder» - Daten als «Text»

Die Felder sind grau hinterlegt. Wird mit einer Maus über die Felder gefahren, so zeigt sich, dass die Felder weiterhin eine Verbindung zur Datenbank "Adressen", zur dortigen Tabelle "Person" und zum, in dieser Tabelle befindlichen, Feld "GebDat" haben.

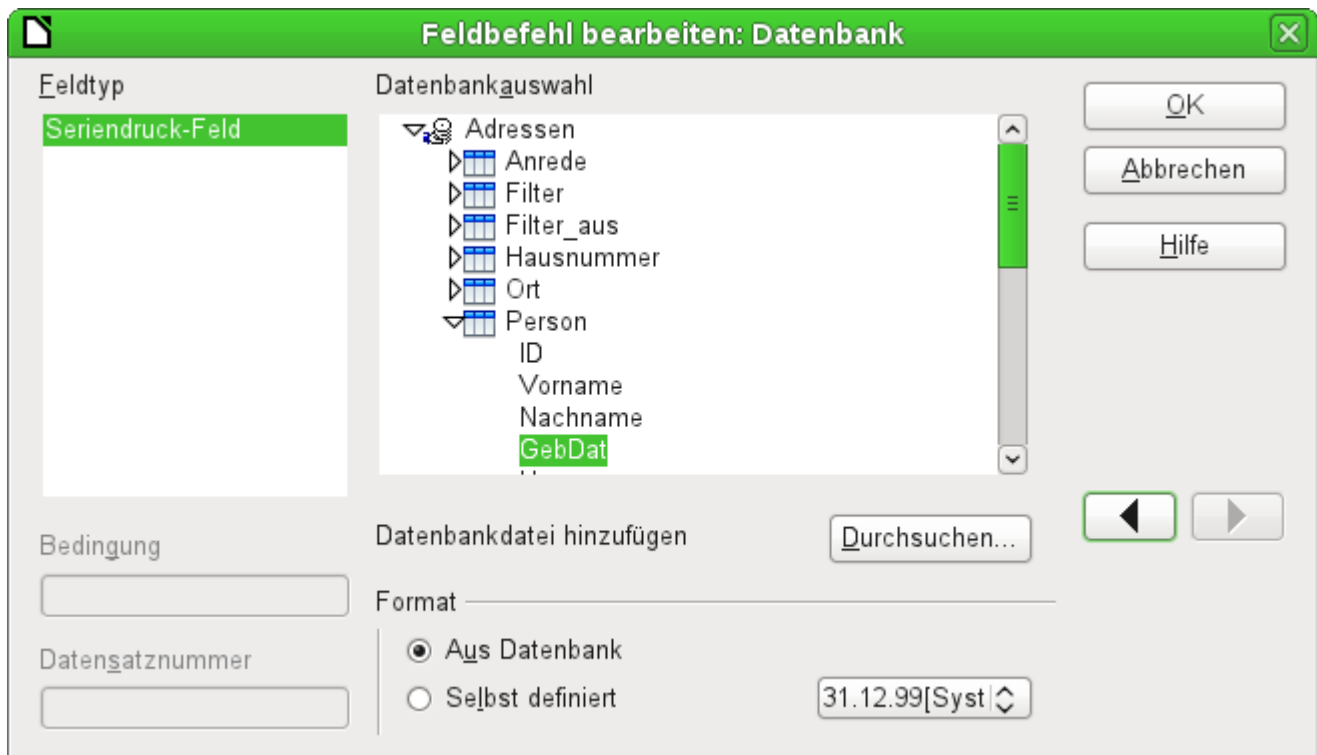


Abbildung 55: Doppelklick auf ein eingefügtes Feld zeigt die Eigenschaften des Seriendruckfeldes

Ein Doppelklick z.B. auf das Feld "GebDat" öffnet schließlich folgende Übersicht. Hier wird dann klar, welches Feld letztlich über die Funktion «Daten einfügen als Felder» gegründet wurde. Es ist der gleiche Feldtyp, der auch über **Einfügen → Feldbefehl → Andere → Datenbank** erzeugt wird.

Einfacher lässt sich so ein Feld übrigens erzeugen, wenn im Datenquellenbrowser der Spaltenkopf der Tabelle markiert und mit der Maus in das Dokument gezogen wird. So kann direkt ein Serienbrief erstellt werden.

Daten in Felder

Über «Daten einfügen als Felder» im vorherigen Abschnitt wurden in einem Writer-Dokument Seriendruckfelder erzeugt. Wird jetzt im Datenquellen-Browser ein anderer Datensatz markiert und dann «Daten in Felder» ausgewählt, so werden die vorherigen dargestellten Daten durch die neuen Daten ersetzt:

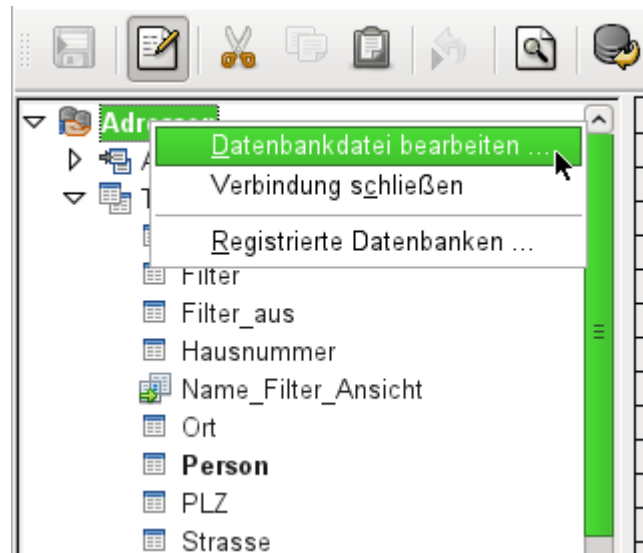
Daten einfügen als Felder	Daten einfügen als Text
Mirina Milinda	Rob van Delft
Tel.: 487531	Tel.: 09871/1946703
geboren am 08.07.09	geboren am 27.07.77

Hier wurde ein anderer Datensatz markiert. Während bei Betätigung von «Daten in Felder» die vorher eingefügten Felder auf den neuen Datensatz umgeschrieben werden, bleibt der über «Daten einfügen als Text» erstellte Text bestehen.

Seriendruck

Mit diesem Button wird der Seriendruck-Assistent gestartet. Da für einen Serienbrief im obigen Beispiel die Daten aus verschiedenen Tabelle zusammengefasst werden müssen, muss zuerst einmal

die Datenbank gestartet werden. In der Datenbank ist dann eine neue Abfrage zu erstellen, die die Daten entsprechend zur Verfügung stellt.



Erfolgt der Start der Datenbank durch einen rechten Mausklick auf die Datenbank oder eine der Tabellen oder Abfragen der Datenbank, so wird die Ansicht im Datenquellenbrowser anschließend sofort aktualisiert. Anschließend kann dann der Serienbrief-Assistent über den entsprechenden Button aufgerufen werden.

Aktuelle Dokument-Datenquelle

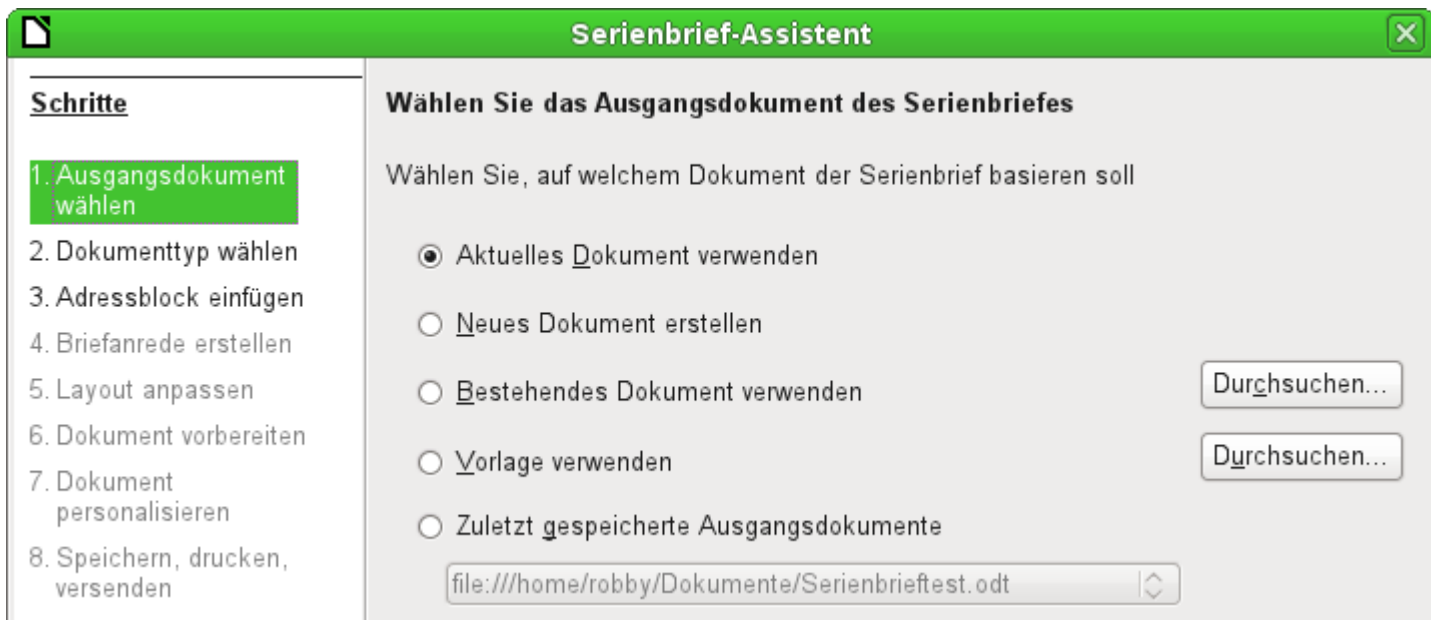
Ein Klick auf diesen Button öffnet direkt die Ansicht auf die Tabelle, die die Grundlage für die Daten bildet, die in das Dokument eingefügt wurden. Im obigen Beispiel zeigt sich also sofort die Tabelle "Person" aus der Datenbank "Adressen".

Explorer ein/aus

Mit Betätigung dieses Buttons wird die Ansicht auf den Verzeichnisbaum links von der Tabellenansicht ein- und ausgeschaltet. Dies dient dazu, gegebenenfalls mehr Platz für eine Sicht auf die Daten zu erhalten. Um auf eine andere Tabelle zugreifen zu können, muss der Explorer eingeschaltet werden.

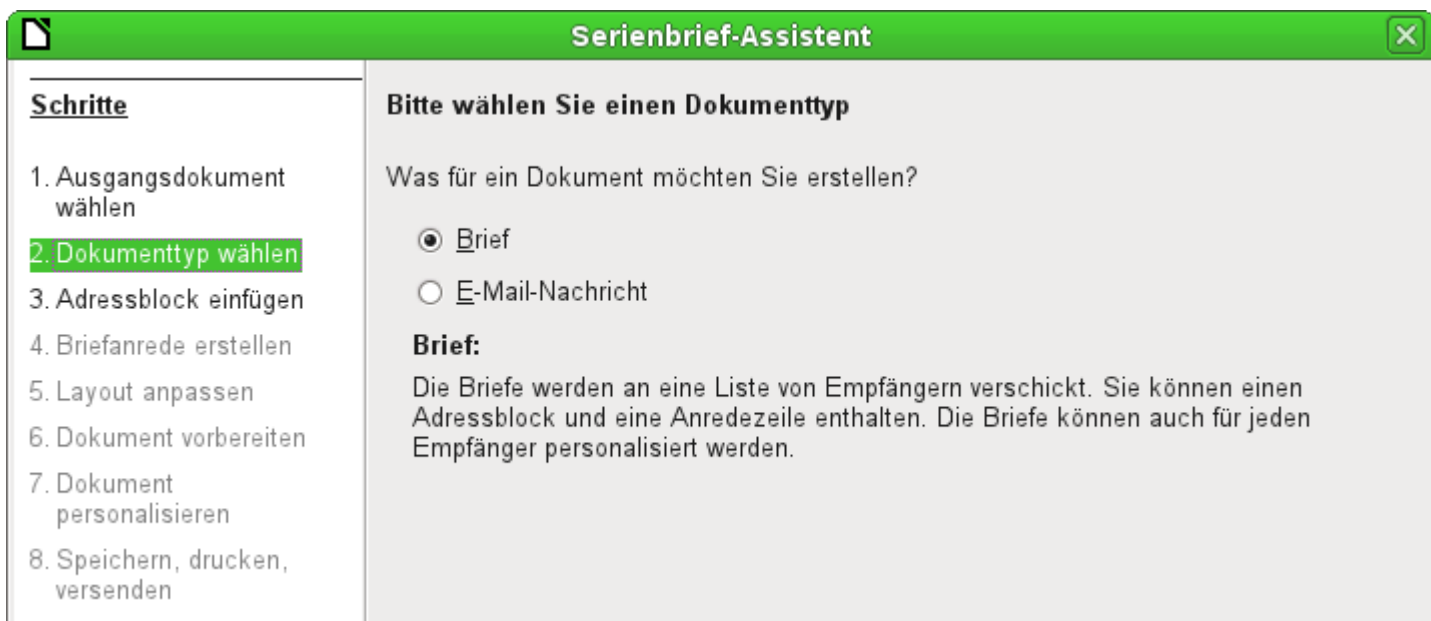
Serienbriefferstellung

Über den Datenbankbrowser ist auch der Serienbriefassistent zugänglich. Mit diesem Assistenten wird kleinschrittig das Adressfeld und die Anrede anhand einer Datenquelle nachvollzogen. Prinzipiell ist die Erstellung von solchen Feldern natürlich auch ohne den Assistenten möglich. Hier wird einmal beispielhaft der gesamte Assistent durchgearbeitet.

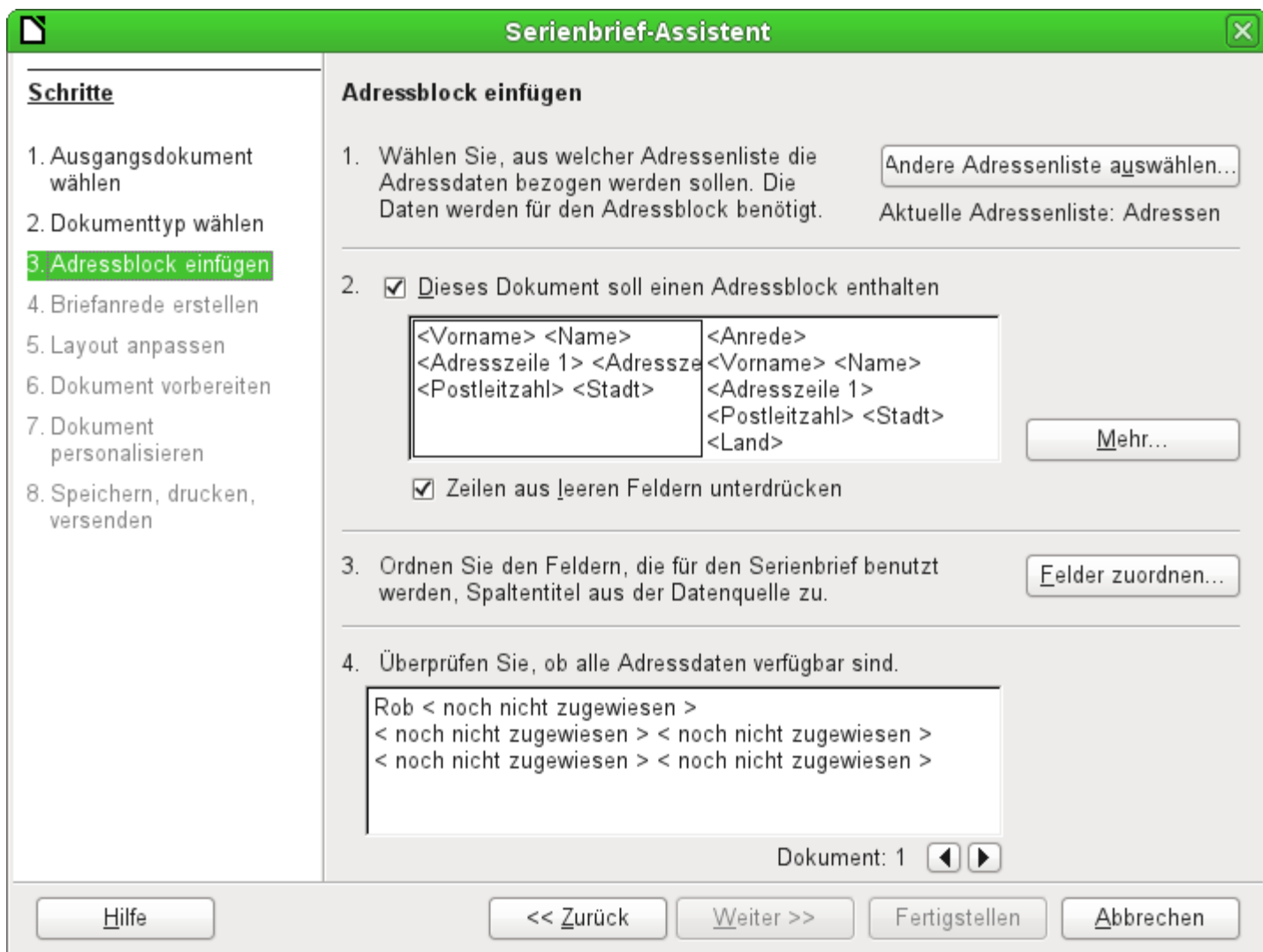


Das **Ausgangsdokument** des Serienbriefes ist das Dokument, mit dem die **Datenbankfelder verknüpft** werden.

Das **Serienbriefdokument** hingegen ist das, in dem nachher die Daten der verschiedenen Personen stehen, die den Serienbrief erhalten haben. Im Serienbriefdokument ist **keine Verknüpfung** zur Datenquelle mehr vorhanden. Dies entspricht der Einstellung aus «Daten einfügen als Text».



Mit dem Serienbrief-Assistenten können entweder Briefe oder E-Mails entsprechend mit Daten aus der Datenbank versorgt werden.



Das Einfügen des Adressblocks erfordert die umfangreichsten Einstellungen. Als Adressenliste wird erst einmal die aktuell gewählte Abfrage oder Tabelle der aktuell gewählten Datenbank vorgeschlagen.

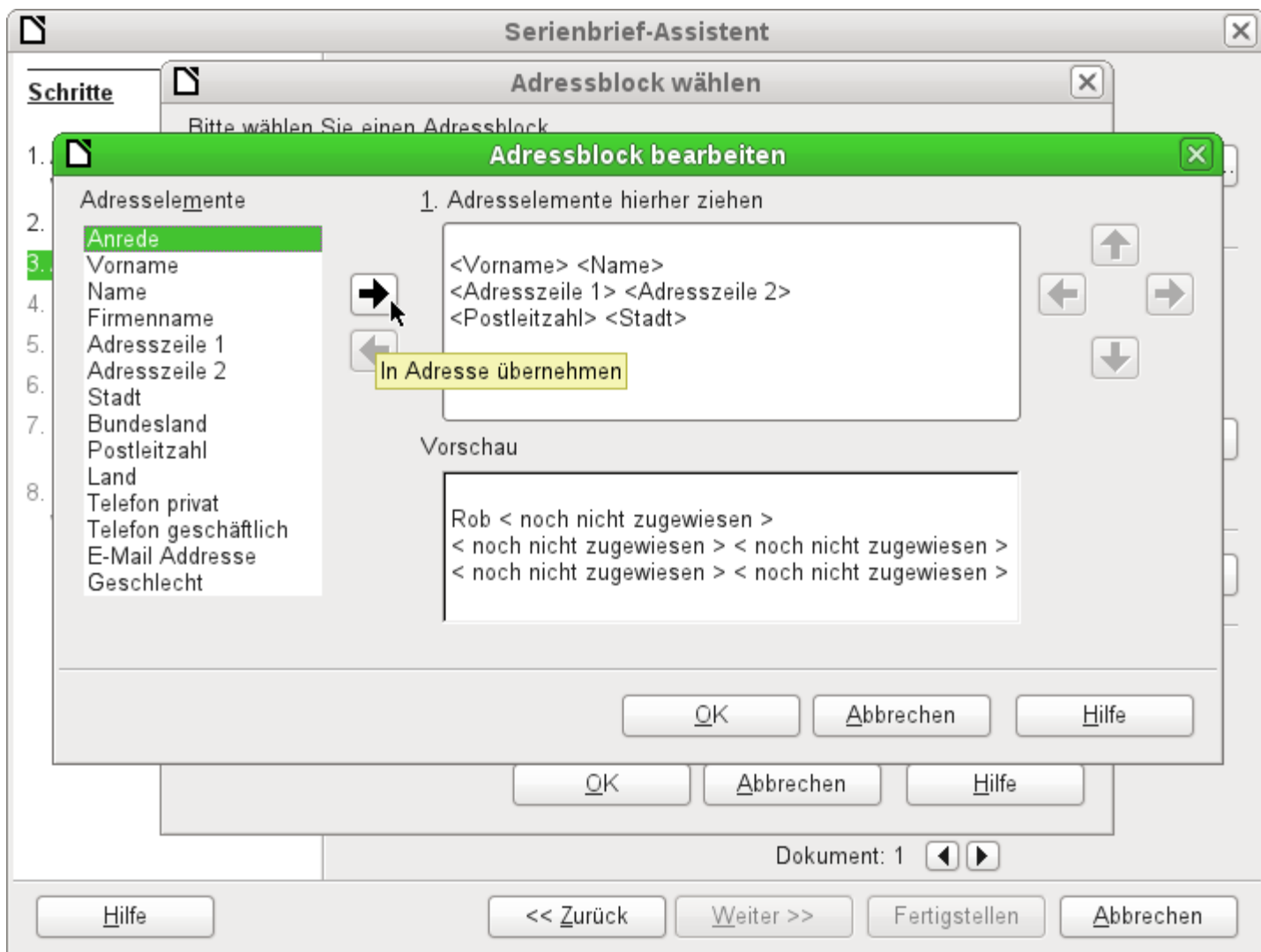
Unter Punkt 2 wird das prinzipielle Aussehen des Adressblocks festgelegt. Dieser Adressblock kann entsprechend über den Button **Mehr** angepasst werden. Siehe hierzu die nächste Abbildung.

Punkt 3 dient dazu, den entsprechend angedeuteten Feldern aus dem Adressblock die richtigen Felder aus der Datenbank zuzuweisen. Der Assistent erkennt zunächst einmal nur die Felder aus der Datenbank, die genau so geschrieben wurden wie die Felder des Assistenten.

Unter Punkt 4 werden die Adressen angezeigt. Mit den Pfeiltasten können verschiedene Adressen aus der Datenbank abgerufen werden. Bei der abgebildeten Adresse fallen 2 Elemente auf, die einer Nachbearbeitung bedürfen:

- Eine Anrede fehlt
- Bis auf den Vornamen sind alle anderen Felder *< noch nicht zugewiesen >*, da die Benennung in der Datenbank anders ist als die Feldbenennungen, mit denen der Assistent zuerst einmal arbeitet.

Um diese Fehler zu bearbeiten, muss zuerst der Adressblock aus Punkt 2 nachbearbeitet werden.



Im Hintergrund ist zu erkennen, dass bei der Nachbearbeitung zuerst einmal noch wieder eine erweiterbare Auswahl von Adressblöcken präsentiert wird. Der passendste Adressblock wurde hier ausgewählt und entsprechend bearbeitet.

Mit den Pfeiltasten werden Felder in den Adressblock übernommen bzw. aus dem Adressblock entfernt. Der Adressblock kann nicht direkt editiert werden. Stattdessen wird die Anordnung der Felder über die auf der rechten Seite sichtbaren Pfeiltasten hergestellt.

Für die Adressanrede wurde einfach das Element «Anrede» eingefügt. Bis auf den Vornamen müssen jetzt noch alle anderen Felder entsprechend zugewiesen werden.

In unserem Fall muss auch das Element «Anrede» anders zugewiesen werden, da ein entsprechendes Feld mit etwas anderem Inhalt in der Abfrage existiert und hier aufgrund der Namensgleichheit als "Adressanrede" benutzt wird. Die "Adressanrede" für Rob lautet in der Abfrage der Datenbank «Herrn», die "Anrede" «Herr» für eine gegebenenfalls gesondert zu erstellende Anredezeile zum Beginn des Briefinhaltes.

Felder zuordnen [X]

Weisen Sie den Adresselementen Felder der Datenquelle zu.

Adresselement	Zuordnung zu Feld:	Vorschau
<Anrede>	Adressanrede	Herrn
<Vorname>	Vorname	Rob
<Name>	Nachname	van Delft
<Firmenname>	<keine>	
<Adresszeile 1>	Strasse	Großer Markt
<Adresszeile 2>	Hausnummer	137

Adressblock Vorschau

Herrn
Rob van Delft
Großer Markt 137
48931 Bucksberghausen

OK Abbrechen Hilfe

Hier wurden die Adresselemente aus dem Serienbriefassistenten erfolgreich entsprechenden Elementen aus der Abfrage der Datenbank zugeordnet. Als Vorschau dient weiterhin der erste Datensatz der Abfrage.

Serienbrief-Assistent

Schritte

1. Ausgangsdokument wählen
2. Dokumenttyp wählen
3. Adressblock einfügen
- 4. Briefanrede erstellen**
5. Layout anpassen
6. Dokument vorbereiten
7. Dokument personalisieren
8. Speichern, drucken, versenden

Briefanrede erstellen

☒ Eine Briefanrede in das Dokument einfügen

☒ Personalisierte Briefanrede einfügen

Weiblich

Sehr geehrte Frau <Name>,

Neu...

Männlich

Sehr geehrter Herr <Name>,

Neu...

Adresslistenwert für einen weiblichen Empfänger

Spaltentitel

Geschlecht

Feldinhalt

w

Allgemeine Briefanrede

Sehr geehrte Damen und Herren,

Vorschau

Sehr geehrter Herr van Delft,

Felder wählen...

Dokument: 1

◀ ▶

Hilfe

<< Zurück

Weiter >>

Fertigstellen

Abbrechen

Die wesentlichen Datenbankeneinstellungen werden mit dem 4. Schritt eigentlich schon beendet. Hier geht es lediglich darum, aus welchem Feld das Geschlecht des Adressaten abgelesen werden soll. Dies war bereits so benannt, dass nur noch der Feldinhalt für den weiblichen Empfänger gekennzeichnet werden musste.

Drei verschiedene Briefanreden werden so erzeugt. Alle mit einem 'w' versehenen Datensätze starten mit 'Sehr geehrte Frau ...', alle mit 'm' versehenen Datensätze mit 'Sehr geehrter Herr ...'. Ist kein Geschlecht angegeben, so wird schließlich 'Sehr geehrte Damen und Herren' gewählt.

Serienbrief-Assistent

Schritte

1. Ausgangsdokument wählen

2. Dokumenttyp wählen

3. Adressblock einfügen

4. Briefanrede erstellen

5. Layout anpassen

6. Dokument vorbereiten

7. Dokument personalisieren

8. Speichern, drucken, versenden

Layout des Adressblocks und der Briefanrede anpassen

Adressblock-Position

☒ Am Text ausrichten

Von links

2,50cm

Von oben

5,49cm

Briefanrede-Position

Nach

Oben

Nach

Unten

Ansicht

Ganze Seite

Serienbrief-Assistent

Schritte

1. Ausgangsdokument wählen

2. Dokumenttyp wählen

3. Adressblock einfügen

4. Briefanrede erstellen

5. Layout anpassen

6. Dokument vorbereiten

7. Dokument personalisieren

8. Speichern, drucken, versenden

Kontrollieren Sie das Dokument und ändern Sie es falls notwendig

Sie sehen jetzt eine Vorschau auf ein Serienbriefdokument. Um eine Vorschau auf weitere Dokumente zu sehen, klicken Sie auf die Pfeile.

Empfänger

|<

<

1

>

>|

☐ Diesen Empfänger ausschließen

Dokument bearbeiten

Sie können jetzt das Dokument erstellen oder bearbeiten. Die Veränderungen wirken sich auf alle Serienbriefdokumente aus.

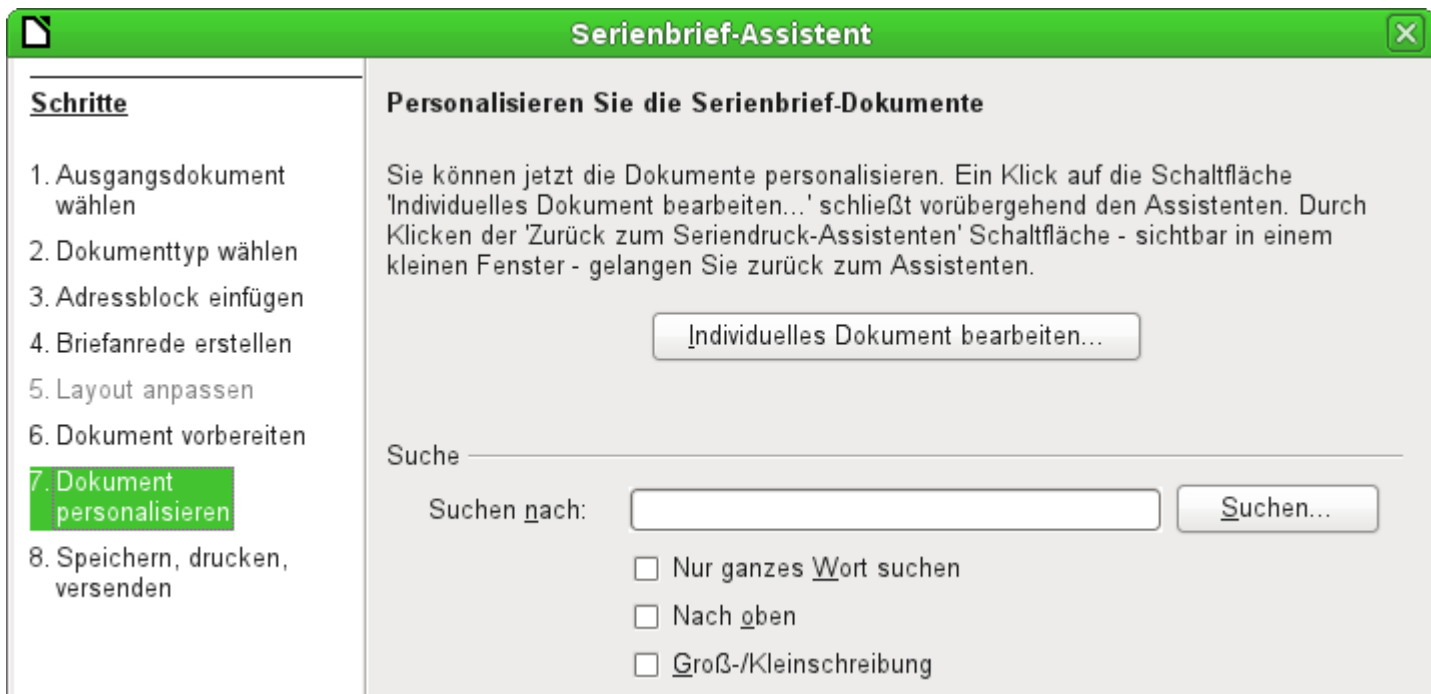
Ein Klick auf die Schaltfläche 'Dokument bearbeiten...' schließt vorübergehend den Assistenten. Durch Klicken der 'Zurück zum Seriendruck-Assistenten'-Schaltfläche - sichtbar in einem kleinen Fenster - gelangen Sie zurück zum Assistenten.

Dokument bearbeiten...

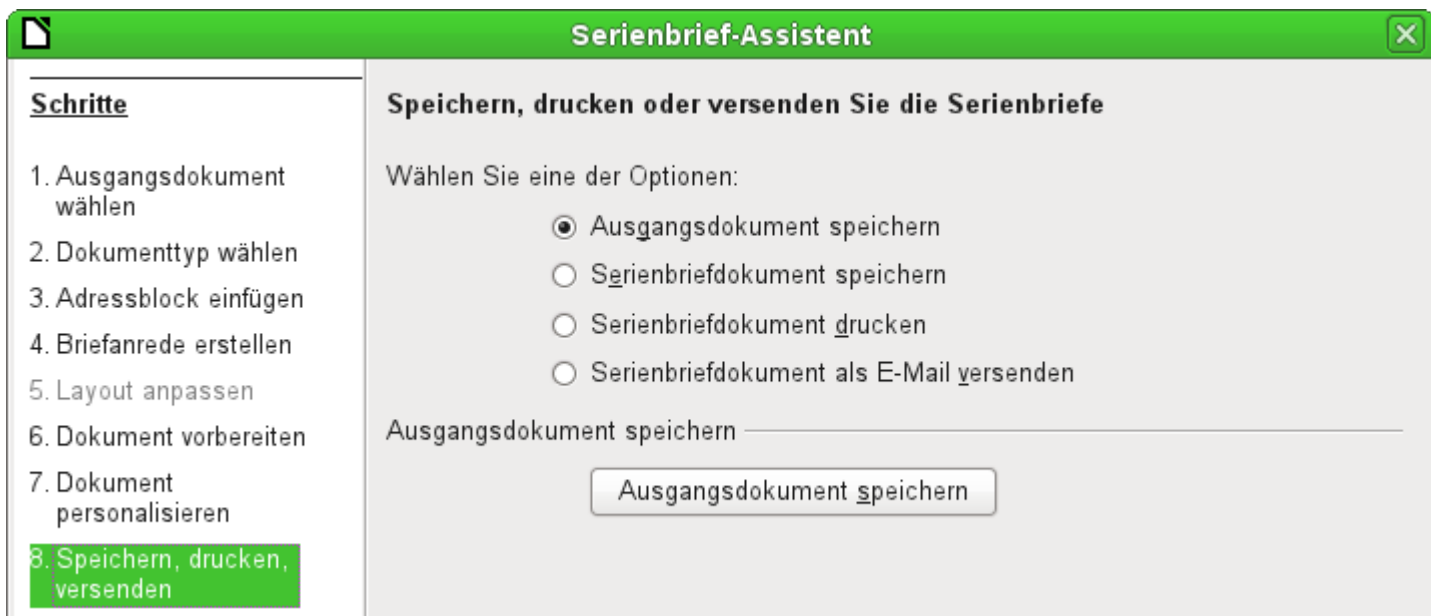
In der Regel ist das Dokument erst einmal eine Rohform, so dass es für den Gebrauch im Writer noch weiter bearbeitet werden kann. Dies kann im 6. Schritt erfolgen.

Datenbank-Anbindung: Serienbriefferstellung

348



Bisher sind alle Dokumente gleichlautend – bis auf die unterschiedlichen Inhalte, die aus der Datenbank gelesen werden. Dies kann in Schritt 7 geändert werden.



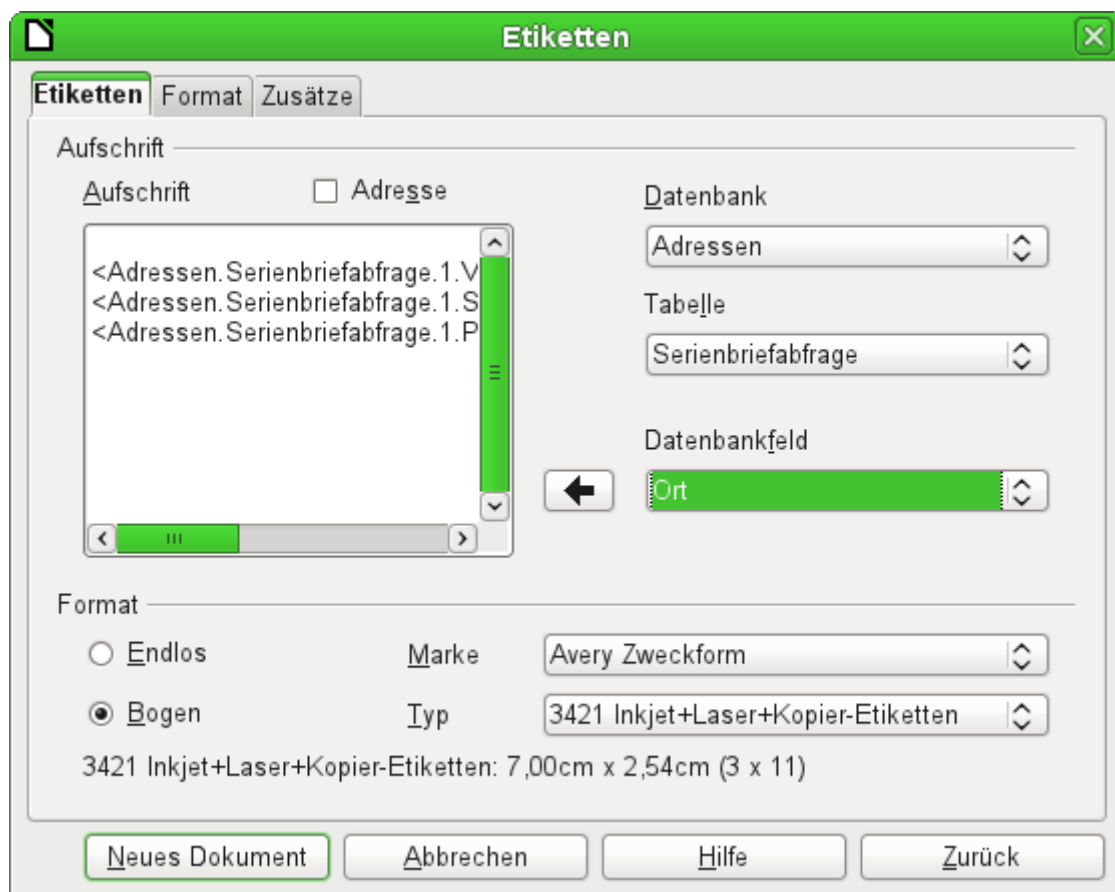
Das **Ausgangsdokument** ist das Dokument, in dem die **Feldeigenschaften** und die **Verbindung zur Datenbank** gespeichert werden. Im Hintergrund ist währenddessen das Originaldokument des ersten Datensatzes sichtbar, das in der Form verschickt würde. Dies wird als Serienbriefdokument bezeichnet.

Erst wenn eine der Optionen durchgeführt wurde, in dem obigen Fall also das Ausgangsdokument gespeichert wurde, kann der Serienbriefassistent beendet werden.

Erstellung von Etiketten

Über **Dateien → Neu → Etiketten** wird der Assistent zur Erstellung von Etiketten gestartet. Es öffnet sich ein Fenster, das alle Formatierungs- und Inhaltsfragen zu den Etiketten abfragt, bevor die Eti-

ketten selbst erstellt werden. Die Einstellungen dieses Fensters werden in den persönlichen Einstellungen des Nutzers gespeichert.



Die Grundeinstellungen zum Inhalt werden im Reiter «*Etiketten*» erstellt. Wird hier statt *Aufschrift* über das Markierfeld *Adresse* gewählt, so wird jedes Etikett mit dem gleichen Inhalt ausgefüllt, der aus den Einstellungen von LibreOffice zu dem Nutzer des Programms gelesen wird.

Als Beispieldatenbank dient hier wieder die Datenbank "*Adressen*". Auch wenn über dem nächsten Selektionsfeld der Begriff «*Tabelle*» steht, werden hier **Tabellen und Abfragen** gelistet, wie sie im Datenquellenbrowser verfügbar sind.

Über die Pfeiltasten werden die einzelnen Datenbankfelder in den Editor eingefügt. Die Bezeichnung für das Datenbankfeld "*Nachname*" wird hier zu *<Adressen.Serienbriefabfrage.1.Nachname>*. Die Reihenfolge ist also *<Datenbank.Tabelle.1.Datenbankfeld>*.

In dem Editor kann mit der Tastatur gearbeitet werden. So ist es z. B. möglich, zu Beginn einen Zeilenumbruch einzufügen, damit die Etiketten nicht direkt an der oberen Kante beschriftet werden, sondern der Inhalt möglichst komplett und gut sichtbar gedruckt werden kann.

Das Format kann im Reiter «*Etiketten*» vorgewählt werden. Hier sind viele Etikettenmarken eingearbeitet, so dass meist keine weiteren Einstellungen im Reiter «*Format*» notwendig sind.

Hinweis

In den Versionen 3.4.x bis 3.5.2 kann aufgrund einer Änderung der Grundeinstellungen im Etikettenassistenten ein Darstellungsfehler vorliegen, wenn die Etiketten zusammen genau die gleiche Breite haben wie die Seitenbreite. Unter diesen Bedingungen rutscht das letzte Etikett einfach eine Zeile tiefer.

In der Version 3.3.x wurde hier standardmäßig die Seitenbreite für DIN-A4-Etikettenbögen geringfügig von 21,00 cm auf 21,04 cm erweitert. Ein nachträgliches Erweitern der Seitenbreite bringt bei den aktuellen Etikettenbögen auch hier die Lösung.

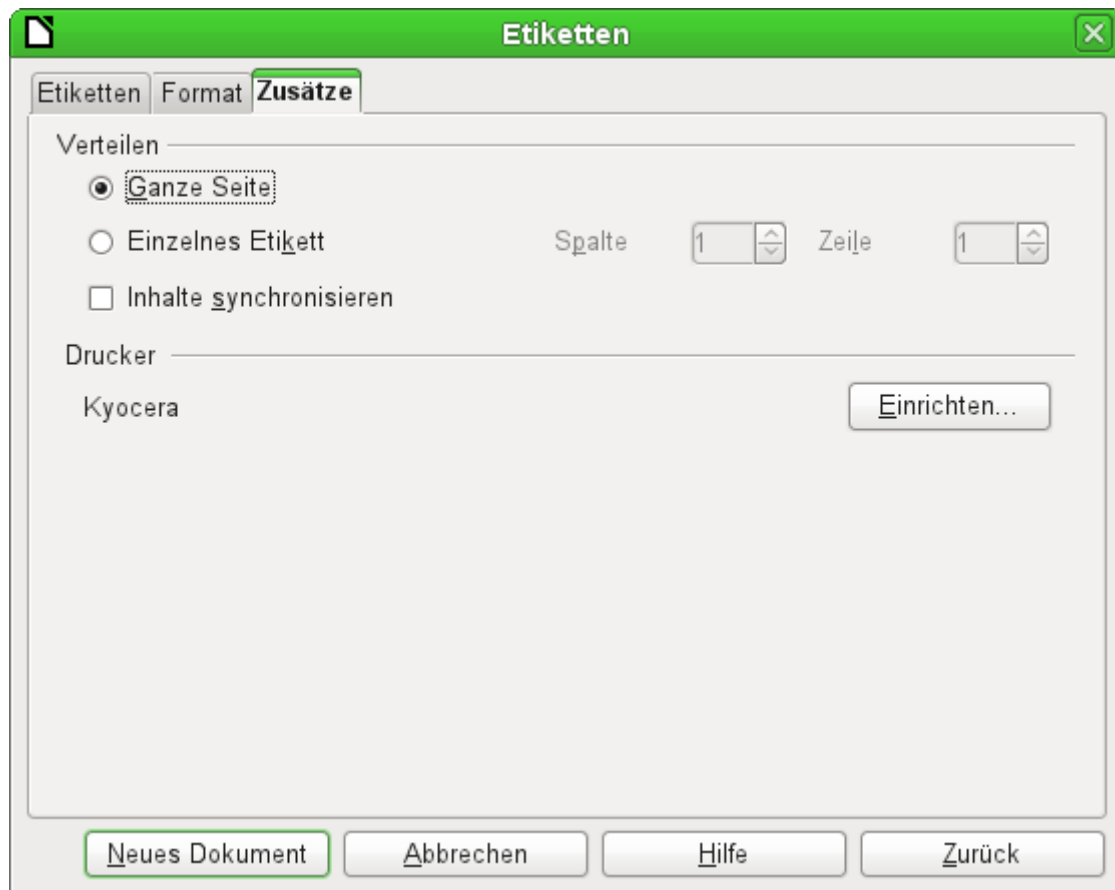
In der Version 3.5.3 wurden deshalb im Reiter «Format» Seiteneinstellungen hinzugefügt.

The screenshot shows the 'Etiketten' (Labels) dialog box with the 'Format' tab selected. The settings are as follows:

Parameter	Value
Horiz. Abstand	7,00cm
Vert. Abstand	2,54cm
Breite	7,00cm
Höhe	2,54cm
Rand links	0,00cm
Rand oben	0,87cm
Spalten	3
Zeilen	11
Page Width	21,01cm
Page Height	29,70cm

The diagram on the right shows a grid of labels. The dimensions are labeled: 'H. Abstand' (horizontal distance), 'V. Abstand' (vertical distance), 'Breite' (width), 'Höhe' (height), 'Spalten' (columns), and 'Zeilen' (rows). The 'Speichern...' button is at the bottom left of the dialog.

Unter dem Reiter «Format» lässt sich eine genaue Einstellung der Etikettengröße vornehmen. Die Einstellungen haben nur eine Bedeutung, wenn die Marke und der Typ nicht bekannt sind. Wichtig ist hier bei Etiketten der Breite 7,00 cm, die Seitenbreite (hier: «Page Width») geringfügig größer als $3 \times 7,00 \text{ cm} = 21,00 \text{ cm}$ zu stellen. Erst dann werden 3 Etiketten nebeneinander auf einer Seite ausgegeben.

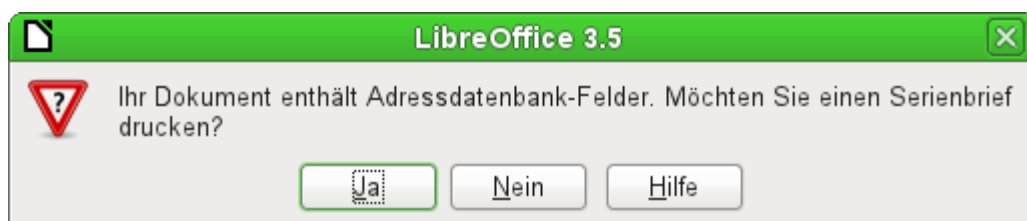


Unter dem Reiter «Zusätze» kann schließlich noch eingestellt werden, ob nur ein einzelnes Etikett oder eine ganze fortlaufende Seite erstellt werden soll. Die fortlaufende Seite wird dann, beginnend mit dem ersten Datensatz, mit Daten aus der Datenbank bestückt. Sind mehr Datensätze vorhanden als auf eine Seite passen, so wird automatisch das nächste Blatt mit den fortlaufenden Daten versehen.

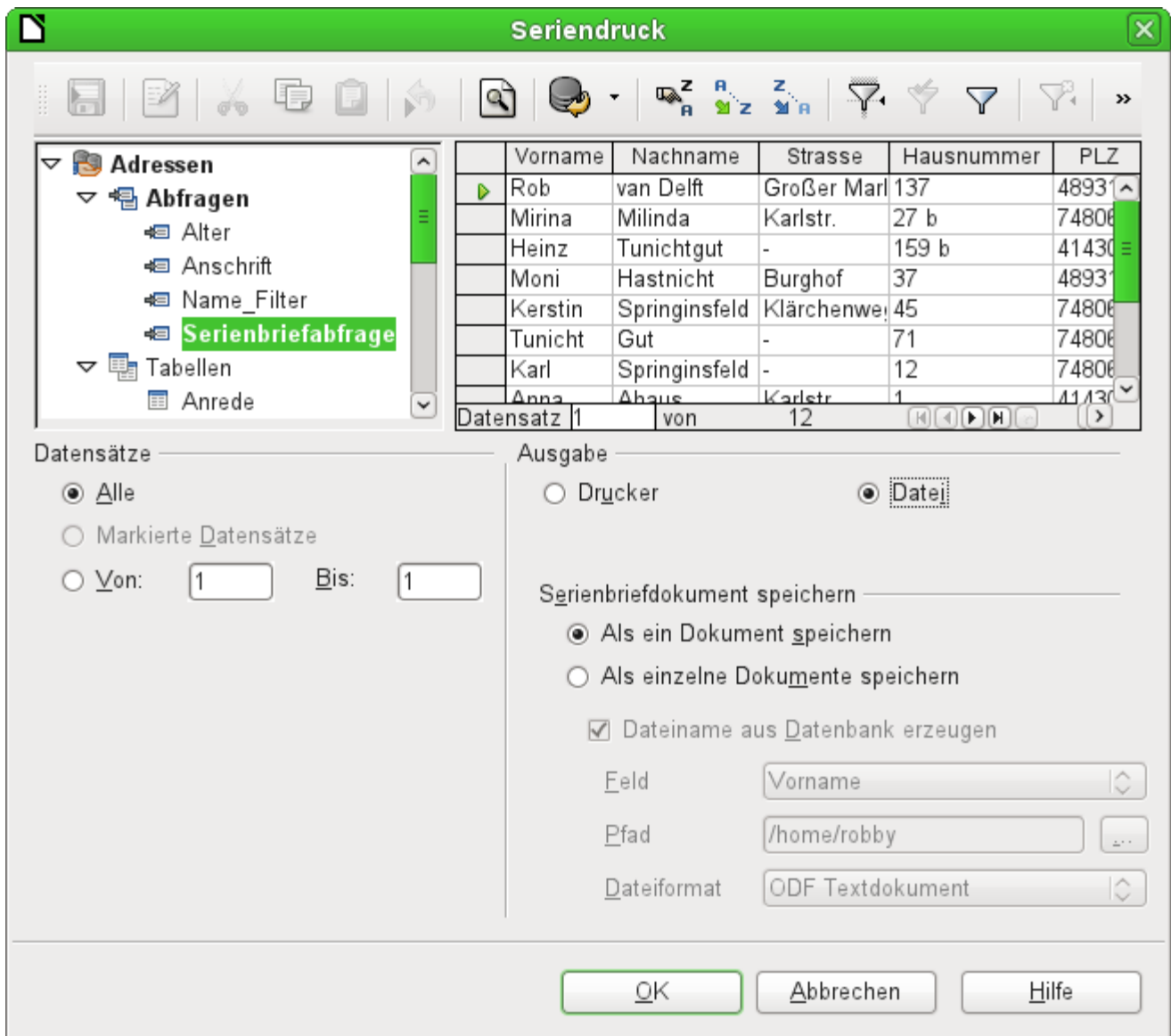
Mit dem Markierfeld «Inhalte synchronisieren» werden alle Etiketten miteinander verbunden, so dass die anschließenden Änderungen im Layout einer Etikette auf die anderen Etiketten übernommen werden. Um den editierten Inhalt zu übertragen, muss lediglich die eingeblendete Schaltfläche mit dem Aufdruck **Synchronisieren** betätigt werden.

Über den Button **Neues Dokument** wird schließlich ein Dokument mit Serienbrieffeldern erzeugt.

Mit dem Aufruf des Druckvorganges erscheint die folgende Nachfrage:



Erst wenn diese Nachfrage mit «Ja» beantwortet wird, werden die Adressdatenbank-Felder auch mit einem entsprechenden Inhalt gefüllt.



Die Vorlage für den Druck wird nicht automatisch gefunden. Es wird lediglich die Datenbank vorge-
wählt. Die entsprechende Abfrage muss vermutlich selbst gesucht werden, weil es sich in diesem
Fall eben nicht um eine Tabelle handelt.

Ist die Abfrage ausgesucht und sind die entsprechenden Datensätze ausgewählt (hier: «Alle»), so
kann mit dem Druck begonnen werden. Ratsam vor allem bei ersten Tests ist die «Ausgabe» in
eine «Datei», die als ein Dokument gespeichert wird. Die Speicherung in mehrere Dokumente
dient hier nicht dem Etikettendruck, sondern dem Druck von Briefen an unterschiedliche Personen,
die so separat nachbearbeitet werden können.

Serienbriefe und Etiketten direkt erstellen

Neben den Assistenten steht natürlich der Weg offen, durch möglichst direkte Interaktion Serien-
briefe und Etiketten selbst zu erstellen.

Serienbrieffelder mit der Maus erstellen

Serienbrief-Felder können mit der Maus aus dem Datenbankbrowser heraus erstellt werden:

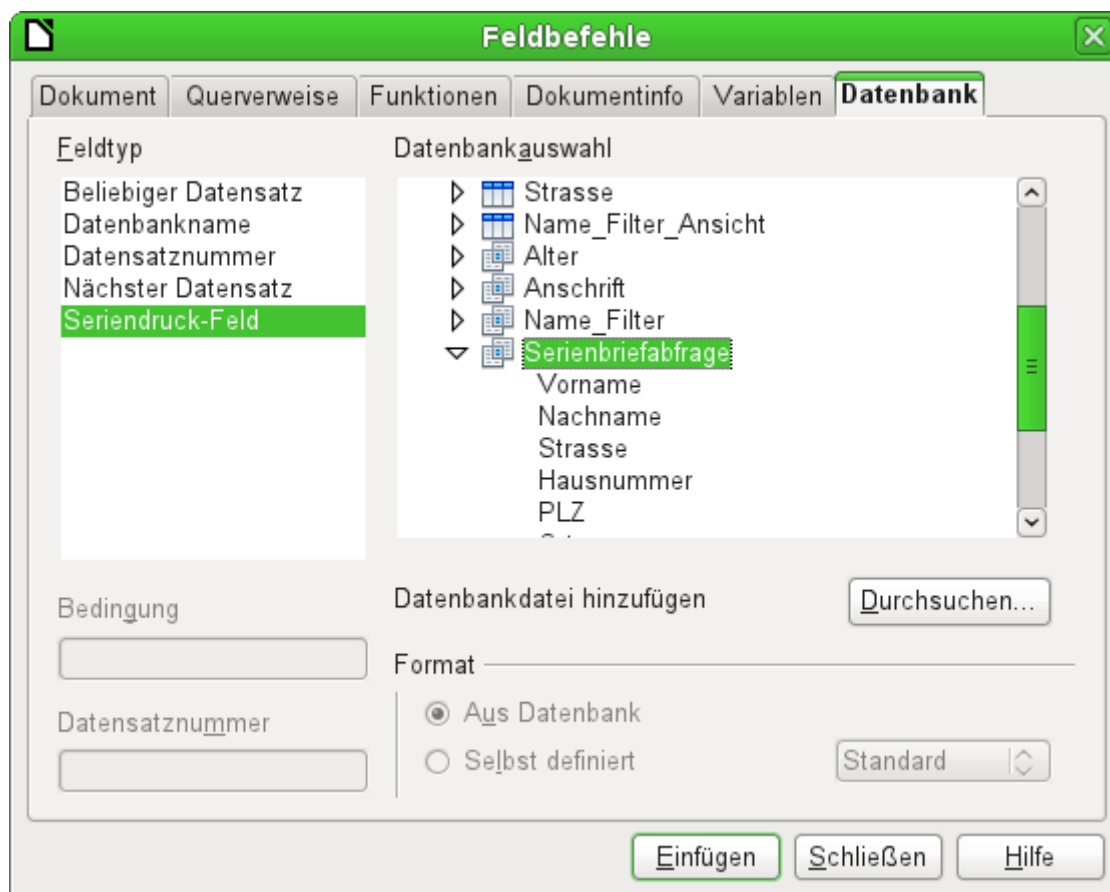


Der Tabellenkopf wird mit der linken Maustaste markiert. Die Maustaste wird gedrückt gehalten und der Cursor in das Textdokument gezogen. Der Cursor verändert sein Symbol zu einem Einfügesymbol. In dem Textdokument wird schließlich das Serienbrieffeld eingefügt, das hier in der kompletten Beschreibung über **Ansicht → Feldname** sichtbar gemacht wurde.

Auch aus einer nicht angemeldeten Datenbank heraus können Serienbrieffelder mit der Maus direkt erstellt werden. Hierzu muss die Datenbank neben dem Writer-Dokument geöffnet sein. Ist eine Tabelle oder Abfrage ausgewählt, so wird mit der linken Maustaste der jeweilige Tabellenkopf markiert und in das Writer-Dokument hinüber gezogen. Der Ablauf entspricht also dem im vorherigen Abschnitt geschilderten Vorgehen.

Serienbrieffelder über Feldbefehle erstellen

Serienbrief-Felder können über **Einfügen → Feldbefehl → Andere → Datenbank** eingefügt werden.



Hier stehen in der gewählten Datenbank alle Tabellen und Abfragen zur Verfügung. Über den Button **Einfügen** können nacheinander die verschiedenen Felder direkt in den Text an der momentanen Cursorposition eingebunden werden.

Soll, wie im Serienbrief, eine Anrede erstellt werden, so kann dies mit Hilfe eines versteckten Absatzes oder versteckten Textes funktionieren: **Einfügen → Feldbefehl → Andere → Funktionen →**

Versteckter Absatz. Bei beiden Varianten ist zu beachten, dass die Bedingung, die formuliert wird, **nicht** erfüllt sein darf, damit der Absatz erscheint.

Damit die Formulierung '*Sehr geehrte Frau <Nachname>*,' nur dann erscheint, wenn die Person weiblich ist, reicht als Bedingung

[Adressen.Serienbriefabfrage.Geschlecht] != "w".

Nur kann es jetzt noch passieren, dass kein Nachname existiert. Unter diesen Umständen sollte dort stattdessen '*Sehr geehrte Damen und Herren*,' erscheinen. Also ist diese Bedingung hinzuzufügen. Insgesamt ergibt das die Bedingung:

[Adressen.Serienbriefabfrage.Geschlecht] != "w" OR NOT [Adressen.Serienbriefabfrage.Nachname]

Damit ist schließlich ausgeschlossen, dass der Absatz erscheint, wenn die Person nicht weiblich ist oder kein Nachname eingetragen wurde.

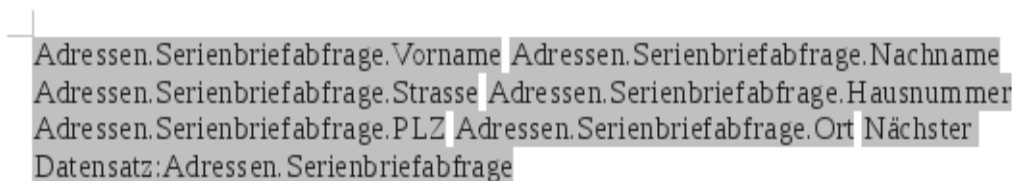
Entsprechend kann mit den Eintragungen für das männliche Geschlecht und für fehlende Eintragungen für die beiden verbleibenden Anreden verfahren werden.

Auf die gleiche Art und Weise kann natürlich auch eine Anrede im Adressfeld erstellt werden, soweit eben das Geschlecht angegeben wurde.

Weitere Informationen hierzu in der Hilfestellung unter dem Titel «*Versteckter Text*» bzw. «*Bedingter Text*».

Noch einfacher wäre es natürlich für Personen mit Datenbankkenntnissen, die gesamte Anrede bereits in der Abfrage zu hinterlegen. Dies ist über eine *Korrelierte Unterabfrage* (Kapitel Abfragen) möglich.

Für Etiketten besonders interessant ist der Feldtyp «Nächster Datensatz». Wird dieser Feldtyp am Schluss einer Etikette gewählt, so wird die nächste Etikette mit dem darauffolgenden Datensatz gefüllt. Typische Etiketten für den fortlaufenden Etikettendruck sehen also wie im folgenden Bild aus, wenn über **Ansicht → Feldnamen** die entsprechenden Bezeichnungen sichtbar gemacht werden:



Adressen.Serienbriefabfrage.Vorname Adressen.Serienbriefabfrage.Nachname
Adressen.Serienbriefabfrage.Strasse Adressen.Serienbriefabfrage.Hausnummer
Adressen.Serienbriefabfrage.PLZ Adressen.Serienbriefabfrage.Ort Nächster
Datensatz: Adressen.Serienbriefabfrage

Abbildung 56: Feldebefehle für Etiketten mit fortlaufendem Inhalt

Bei der letzten Etikette auf der Seite ist allerdings darauf zu achten, dass hier der nächste Datensatz schon automatisch mit dem Seitenumbruch aufgerufen wird. Dort darf also der Feldtyp «Nächster Datensatz» nicht erscheinen. Sonst fehlt ein Datensatz, weil ein doppelter Datensatzsprung enthalten ist.

Tip

Das Erstellen von Serienbriefen ist auch direkt im Formular einer Datenbank möglich. Voraussetzung ist lediglich, dass die Datenbank in LibreOffice registriert wird.

Beim Erstellen der Serienbriefe sollte allerdings darauf geachtet werden, dass **Ansicht** → **Drucklayout** gewählt wird. Dadurch ist sichergestellt, dass die Elemente anschließend auch an der korrekten Position auf dem Blatt erscheinen. Wird ein so erstelltes Formular gedruckt, so erscheint die übliche Serienbriefabfrage.

Serienbriefe dieser Art haben den Vorteil, dass neben der *.odb-Datei keine weiteren Dateien für den Druck existieren müssen.

Externe Formulare

Sollen einfache Formulareigenschaften unter LibreOffice in anderen Programmteilen wie Writer und Calc genutzt werden, so reicht es aus, über **Ansicht** → **Symbolleiste** → **Formularentwurf** die Formularentwurfsleiste anzeigen zu lassen, den «*Formularnavigator*» zu öffnen und ein Formular zu gründen oder, wie im Kapitel «Formulare» beschrieben, eine *Formulargründung über ein Formularfeld*. Es erscheint bei den Eigenschaften unter dem Reiter «Daten» ein etwas anderer Aufbau als bei Formularen direkt in der Datenbankdatei *.odb:

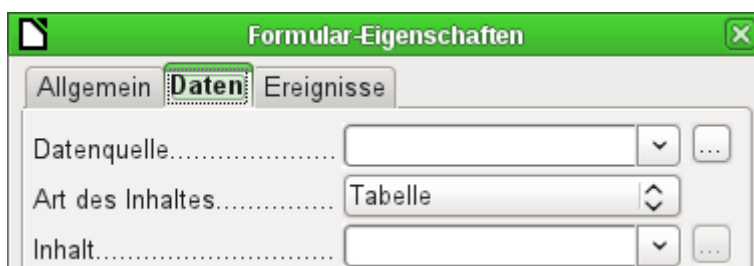


Abbildung 57: Formular mit einer externen ...

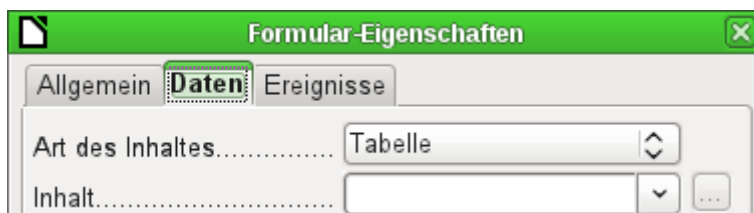


Abbildung 58: ... und einer internen Datenquelle.

Lediglich die «Datenquelle» muss bei externen Formularen zusätzlich ausgewählt werden. Geschieht dies mit dem Button «...» rechts von dem Listefeld zur Datenquelle, so wird der Dateibrowser geöffnet. Eine beliebige *.odb-Datei kann ausgewählt werden. Anschließend steht in dem Feld zur Datenquelle ein Link, der mit der Bezeichnung «file:///» beginnt.

Wird stattdessen nur im Listefeld gesucht, so stehen dort die bereits in LibreOffice registrierten Datenbanken unter dem registrierten Namen zur Verfügung.

Die Formulare werden im Weiteren dann genau so erstellt wie unter Base selbst. Zur Formularerstellung gilt daher das Gleiche wie unter Base.

Die so erstellten Formulare werden standardmäßig bei jedem neuen Öffnen der Datei im Bearbeitungsmodus und nicht, wie in Base, schreibgeschützt geöffnet. Um eine versehentliche Änderung des Formulars zu vermeiden kann über **Datei** → **Eigenschaften** → **Sicherheit** die Datei schreibgeschützt geöffnet werden. Das Formular kann hier sogar mit einem Passwort gegen Veränderungen geschützt werden. Auf Betriebssystemebene lässt sich letztlich auch die ganze Datei als schreibgeschützt deklarieren. Dann sind immer noch die Eingaben in die Formularfelder möglich, aber nicht mehr das Verschieben der Felder oder eine Texteingabe zwischen den Feldern.

Tipp

Das Erstellen von Formularen kann auch im Schnellverfahren durch Ziehen und Ablegen erfolgen. Dafür ist die Datenbank zu öffnen, die Tabelle bzw. Abfrage aufzusuchen und der jeweilige Tabellenkopf zu markieren.

Für den Writer gilt: Mit der linken Maustaste markieren, **Shift**- und **Strg**-Taste gedrückt halten, so dass ein Verknüpfungssymbol als Mauscursor erscheint, in das Writerdokument ziehen.

Bei Calc-Dateien muss ohne Zuhilfenahme zusätzlicher Tasten in das Calc-Dokument gezogen werden. Dort erscheint das Kopiersymbol als Mauscursor.

In beiden Fällen wird ein Eingabefeld sowie ein dazugehöriges Beschriftungsfeld mit der Bezeichnung des Feldnamens erzeugt. Beim ersten Einfügen wird gleichzeitig die Verbindung zur Datenquelle erstellt. So kann direkt nach dem Ziehen und Ablegen mit der Eingabe von Daten in diesem Formular begonnen werden.

Vorteil externer Formulare

Base muss nicht erst geöffnet werden, um mit der Datenbank zu arbeiten. Im Hintergrund ist also nicht immer ein weiteres Fenster geöffnet.

Bei einer fertigen Datenbank können anderen Nutzern der Datenbank anschließend verbesserte Formulare problemlos zugesandt werden. So können sie während der Entwicklung weiterer Formulare die Datenbank weiter nutzen und müssen nicht, für Außenstehende kompliziert, Formulare aus einer Datenbank in eine andere kopieren.

Formulare zu einer Datenbank können je nach Nutzer der Datenbank unterschiedlich sein. Nutzer, die keine Datenkorrekturen und Neueingaben tätigen, können von anderen Nutzern den jeweils aktuellen Datenbestand zugesandt bekommen und einfach die *.odb-Datei austauschen, um einen aktuellen Bestand zu haben. Dies könnte z.B. bei einer Datenbank für Vereine sinnvoll sein, wo alle Vorstandsmitglieder die Datenbank erhalten, aber nur eine Person Daten letztlich bearbeitet, die anderen aber einen Blick auf die Adressen ihrer jeweiligen Abteilung haben.

Über externe Formulare ist der Zugriff und die Bearbeitung von Inhalten mehrerer Datenbanken möglich. Die unterschiedlichen Datenbanken brauchen nur im jeweiligen Neben- oder Unterformular selbst angegeben zu werden. Auch der Zugriff auf mehrere externe Datenbanken wie z.B. eine Datenbanksammlung unter MySQL/MariaDB ist damit möglich.

Nachteil externer Formulare

Andere Nutzer müssen immer Formulare und Base in der gleichen Verzeichnisstruktur installieren. Nur so kann der einwandfreie Kontakt zur Datenbank hergestellt werden. Sind die Datenbanken angemeldet, so muss allerdings lediglich der Anmeldename übereinstimmen.

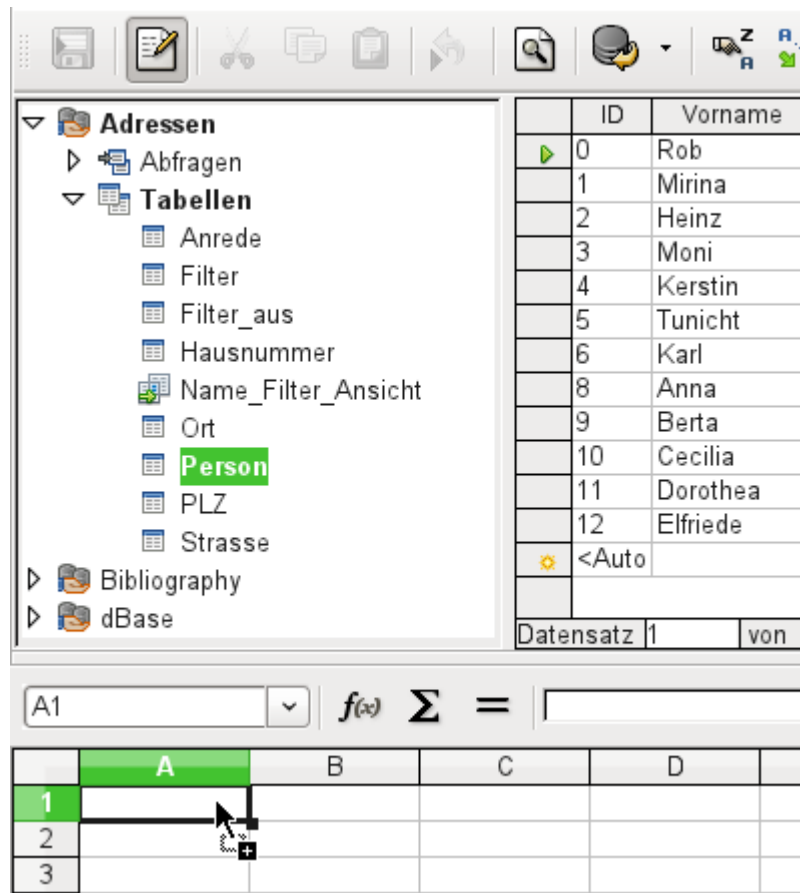
Nur die Formulare sind extern erstellbar, nicht aber Abfragen und Berichte. Ein einfacher Blick auf eine Abfrage muss also über ein Formular erfolgen. Als Darstellung eignet sich da sicher das Tabellenkontrollfeld ganz gut. Ein Bericht hingegen erfordert die Öffnung der Datenbank. Dies kann auch über Makros mit einem [Druck von Berichten aus einem externen Formular heraus](#) erfolgen. Alternativ dazu kann er zumindest teilweise mit Hilfe von Serienbriefefeldern nachgestellt werden. Zu weiteren Druckmöglichkeiten siehe auch das entsprechende Beispieldatenbankpaket «Serienbrief».

Datenbanknutzung in Calc

Daten können in Calc zu Berechnungszwecken genutzt werden. Dazu ist es notwendig, die Daten zuerst in einem Tabellenblatt von Calc verfügbar zu machen.

Daten in Calc einfügen

Daten lassen sich aus der Datenbank auf verschiedene Weisen in Calc einfügen:



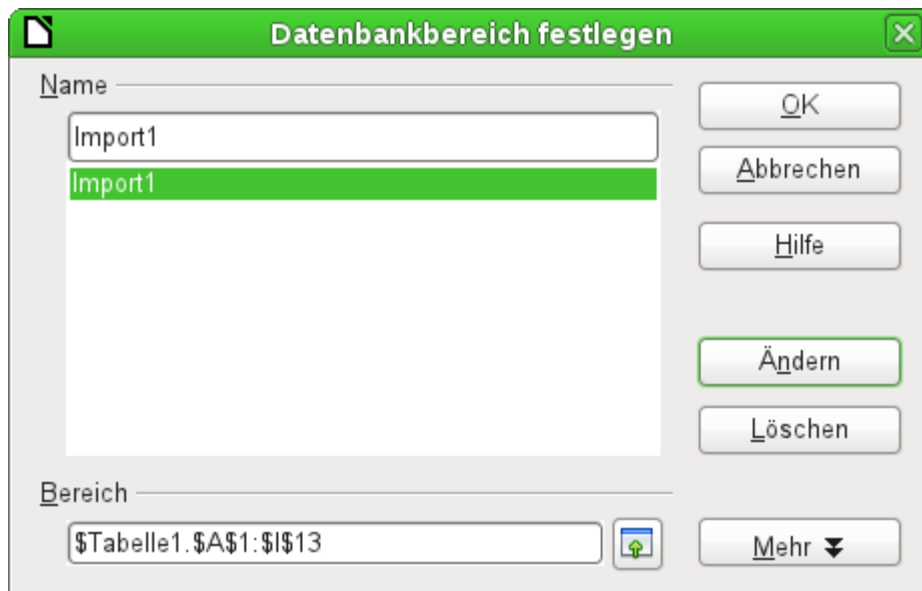
Die Tabelle wird ausgewählt, mit der linken Maustaste markiert und in ein Tabellenblatt von Calc hereingezogen. Mit dem Cursor wird die linke obere Ecke der Tabelle festgelegt. Die Tabelle wird mit Feldbezeichnungen erstellt. Der Datenquellen-Browser bietet hier nicht erst «Daten in Text» oder «Daten in Felder» an.

Die so nach Calc hereingezogenen Daten weisen die folgenden Eigenschaften auf:

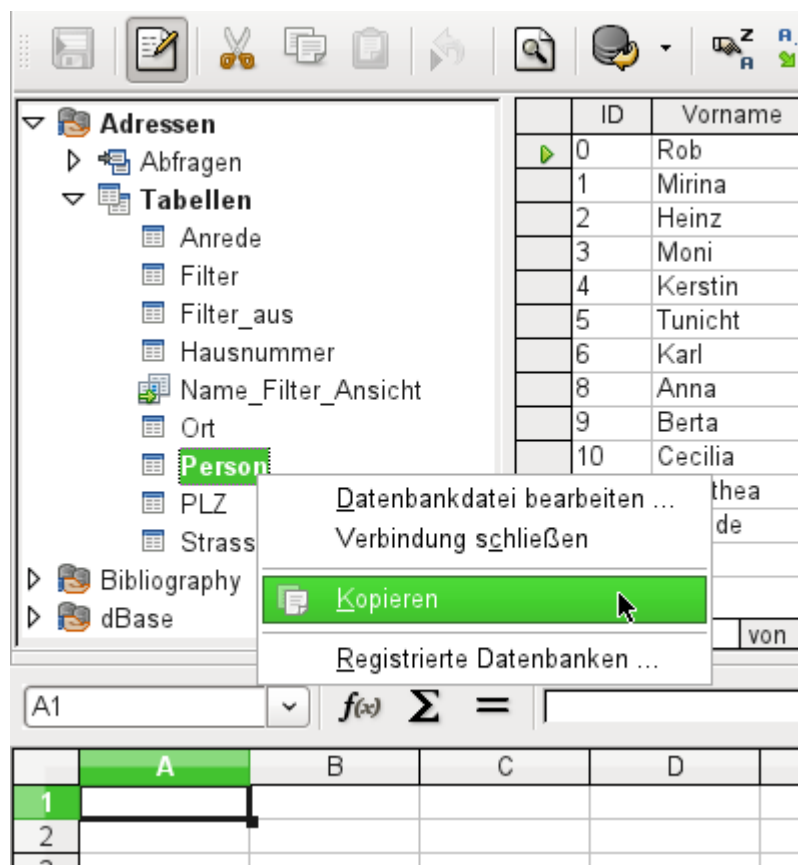
Nicht nur die Daten werden importiert, sondern auch die Eigenschaften der Felder werden ausgelesen und beim Import beachtet. Felder wie z.B. die Hausnummern, die als Textfelder deklariert wurden, werden auch als Text formatiert in Calc eingefügt.

Der Import wird direkt als Bereich eingefügt. Ihm wird standardmäßig der Name Import1 zugewiesen. Über diesen Bereich können die Daten später angesprochen werden. Über **Daten → Bereich aktualisieren** wird der Bereich gegebenenfalls einem neuen Datenstand aus der Datenbank heraus angepasst.

Werden einzelne Datenzeilen oder die komplette Tabelle in der Datenansicht markiert, so kann der Bereich auch über einen Klick auf «Daten in Text» aktualisiert werden. Hierbei werden dann nur die markierten Zeilen der Tabelle neu geschrieben. Alle anderen Zeilen werden aus der Tabelle in Calc entfernt.



Die hereingezogenen Daten sind nicht weiter formatiert als es den Eigenschaften der Datenbankfelder entspricht.



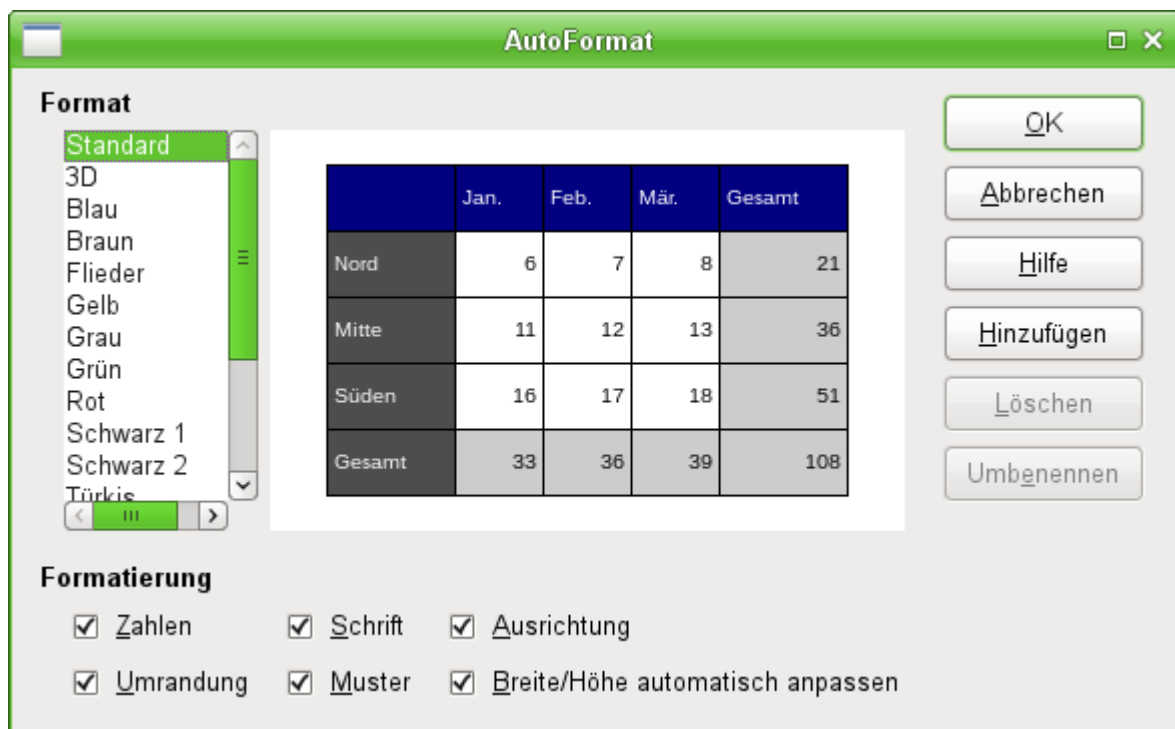
Auch über das Kontextmenü der Tabelle kann eine Kopie der Daten erfolgen. Hier wird allerdings kein Importbereich erzeugt, sondern eine Kopie. Die Eigenschaften der Datenfelder werden nicht ausgelesen, sondern von Calc analysiert. Außerdem werden die Feldbezeichnungen als Tabellenköpfe vorformatiert.

B3		$f(x)$	Σ	=	'137
	A	B	C	D	E
1	Import			Kopie	
2	Vorname	Hausnummer		Vorname	Hausnummer
3	Rob	137		Rob	137
4	Mirina	27 b		Mirina	27 b
5	Heinz	159 b		Heinz	159 b
6	Moni	37		Moni	37
7	Kerstin	45		Kerstin	45

Die Unterschiede zeigen sich besonders bei Feldern, die in der Datenbank als Text formatiert sind. Beim Import macht Calc daraus Textfelder und setzt Zahlen, die es sonst auch als Zahlen interpretieren würde, ein Hochkomma ('137) voran. Mit diesen Zahlen kann also nicht direkt weiter gerechnet werden.

Bei einem eventuellen Export wird das Hochkomma allerdings wieder entfernt, so dass die Daten letztlich in der gleichen Art bestehen bleiben.

Importierte Daten können natürlich im Nachhinein formatiert werden. Hierzu wird der importierte Bereich komplett markiert und dann über **Format → AutoFormat Vorlagen** weiter formatiert:



Diese Formatierung bleibt erhalten, wenn unter **Daten → Bereich festlegen** die folgenden Angaben gemacht werden:

Datenbankbereich festlegen

Name

Import1

Import1

Umfang

\$Tabelle1.\$A\$1:\$I\$16

Ändern Löschen

Einstellungen

- ☒ Enthält Spaltenbeschriftungen
- ☐ Enthält Ergebniszeile
- ☐ Zellen einfügen/löschen
- ☒ Formatierung beibehalten
- ☒ Impортиerte Daten nicht speichern

Quelle: Adressen/Serienbriefabfrage
Operationen: Kein

Hilfe OK Abbrechen

Die **Einstellungen** müssen aufgeklappt werden. Dann sollte **Formatierung beibehalten** ausgewählt werden. Zusätzlich ist noch sinnvoll, **Importierte Daten nicht speichern** zu wählen, wenn eine automatische Abfrage der Daten bei jedem Programmstart erfolgen soll. Dabei werden im übrigen auch Daten aufgenommen, die neu hinzugefügt werden. In sofern ist der Eintrag in **Umfang** nur der Startbereich. Es erfolgt automatisch eine Erweiterung des Bereiches nach unten.

Tipp

Der Import von Daten nach Calc überschreibt den vorherigen Inhalt – und auch die eventuell bereits erledigten Formatierungen, die anschließend direkt für einzelne Felder erfolgt sind. Werden beständig in die gleiche Tabelle Daten exportiert, so empfiehlt es sich, ein separates Tabellenblatt für den Datenimport zu nutzen. Die Daten werden dann über den Bezug Tabellenname.Feldbezeichnung in einem anderen Tabellenblatt ausgelesen. Die Felder in diesem Tabellenblatt können beliebig vorformatiert werden, ohne dass das Format überschrieben wird.

Tipp

Daten können auch direkt aus der Datenbank über die Zwischenablage oder über Ziehen und Ablegen mit der Maus kopiert werden. Wird eine Tabelle oder Abfrage in ein Calc-Blatt gezogen, so wird der gesamte Inhalt eingefügt. Wird die Tabelle oder Abfrage geöffnet und ein oder mehrere Datensätze markiert, so werden nur diese Datensätze zusammen mit den Feldnamen beim Ziehen kopiert.

Vorsicht



Kopien über die Zwischenablage werden in Calc so übernommen, wie sie in der Datenquelle sichtbar sind. Tabellenköpfe werden als Feldbezeichner hervorgehoben, Zahlen mit der Anzahl an Nachkommastellen übernommen, die in Base **angezeigt** werden.

Kopien über Drag-and-Drop transportieren den tatsächlichen Inhalt der Datenquelle, also auch z.B. Nachkommastellen von Zahlen, die zur Zeit in der Datenquelle nicht zu sehen, wohl aber vorhanden sind. ([Bug 112023](#))

Daten aus Calc in eine Datenbank exportieren

Die Daten werden in dem Tabellenblatt von Calc markiert. Die linke Maustaste wird gedrückt und auf den Tabellenbereich der gewünschten Datenbank im Datenbankbrowser gezogen.

The screenshot shows the 'Adressen' database browser on the left, with the 'Person' table selected. Below it, the spreadsheet shows the data from the 'Person' table, with columns A, B, and C. The data is as follows:

	A	B	C
1	ID	Vorname	Nachname
2	0	Rob	van Delft
3	1	Mirina	Milinda
4	2	Heinz	Tunichtgut
5	3	Moni	Hastnicht
6	4	Kerstin	Springinsfeld
7	5	Tunicht	Gut
8	6	Karl	Springinsfeld
9	8	Anna	Ahaus
10	9	Berta	Blocker
11	10	Cecilia	Cologne
12	11	Dorothea	Düse
13	12	Elfriede	Erkelenz

Der Cursor verändert sein Symbol und deutet an, dass etwas hinzugefügt werden kann.

Es öffnet sich das erste Fenster des Importassistenten. Die weiteren Schritte mit dem Assistenten sind im Kapitel «Tabellen» im Abschnitt «*Import von Daten aus anderen Datenquellen*» beschrieben.

Daten von einer Datenbank zu einer anderen konvertieren

Im Explorer des Datenquellenbrowsers können Tabellen von einer Datenbank zur anderen kopiert werden, indem die Quelltablette mit der linken Maustaste markiert wird, die Taste dann gedrückt gehalten wird und über dem Tabellencontainer der Zieldatenbank losgelassen wird. Es erscheint dann der Dialog zum Kopieren von Tabellen.

Auf diese Weise können beispielsweise Datenbanken, die sonst nur gelesen werden können (Datenquelle z.B. ein Adressbuch aus einem Mailprogramm oder eine Tabellenkalkulationstabelle), als Grundlage für eine Datenbank genutzt werden, die anschließend auch schreibend auf die Daten zugreift. Auch können beim Wechsel eines Datenbankprogramms (z.B. von PostgreSQL zu MySQL) die Daten direkt kopiert werden.

Wird erwünscht, dass die neue Datenbank andere Relationen aufweisen soll als die alte Datenbank, so kann dies durch entsprechende Abfragen in der Datenbank realisiert werden. Wer darin nicht so firm ist, kann stattdessen Calc nutzen. Hier werden die Daten in ein Tabellenblatt gezogen und anschließend mit Hilfe von Calc für den Import in die Zieldatenbank vorbereitet.

Für einen möglich sauberen Import in eine neue Datenbank sollten die Tabellen der neuen Datenbank vorher erstellt werden. So können Formatierungsprobleme und Probleme bei der Erstellung von Primärschlüsseln rechtzeitig erkannt werden.

Daten über die Zwischenablage in eine Tabelle einfügen

Sind Daten in Tabellenform vorhanden, so können diese über die Zwischenablage und den Assistenten in Base eingefügt werden.

Wird in Base mit einem rechten Mausklick auf die Zieltabelle der Einfügevorgang begonnen, so erscheinen in dem Kontextmenü der Maus unter «Kopieren» die Befehle «Einfügen» und «Inhalte einfügen ...». Wird hier «Einfügen» gewählt, so ist beim Importassistenten die entsprechende Tabelle sowie das Anhängen von Daten bereits vorgewählt. «Inhalte einfügen ...» schaltet hier lediglich eine Abfrage für einen Importfilter vor. Hier steht «HTML» und «RTF» zur Verfügung.

Wird stattdessen nur in den Tabellencontainer mit der rechten Maustaste geklickt, so erscheint der Importassistent mit der entsprechenden Wahl einer neuen Tabelle.

Datenimport aus PDF-Formularen

Wer Daten aus verschiedenen Quellen von außen her importieren will, sollte am besten eine Formularstruktur wählen, die bei der Eingabe der Daten nicht weiter verändert werden kann. Mit Hilfe des Writers lassen sich solche PDF-Formulare erstellen, im Internet verteilen und entsprechend z.B. als E-Mail-Anhang ausgefüllt zurückschicken. Fehlt schließlich nur noch der möglichst einfache Import in die Base-Datenbank. Das Beispiel¹³ soll so eine Importmöglichkeit aufzeigen.

¹³ Die Datenbank «Beispiel_PDFFormular_Import.odt» zu diesem Bericht ist den Beispieldatenbanken für dieses Handbuch beigelegt.

Erstellen eines PDF-Formulars

Ein PDF-Formular wird als externes Formular ohne Datenbankanbindung erstellt. Über **Ansicht → Symbolleisten → Formular-Steuerelemente** werden die notwendigen Elemente für das Formular angezeigt und können entsprechend eingefügt werden.

Leider werden im PDF-Formular keine Unterschiede zwischen Zahlenfeldern, Datumsfeldern und Textfeldern gemacht. Für das beigefügte Beispiel reicht es also völlig aus, bei jedem Eingabefeld ein Textfeld aufzuziehen. Weitere Feldformatierungen des Writer-Formulars gehen bei dem Export in ein PDF-Formular zwangsläufig verloren.

Grundsätzlich ermöglichen PDF-Formulare nur die folgenden Formularfelder:

- Schaltfläche
- Textfeld
- Checkbox
- Combobox und
- Listbox

Das Testformular enthält insgesamt 4 Textfelder. In **Eigenschaften: Textfeld → Allgemein → Name** soll für den folgenden Importmodus die jeweilige Bezeichnung gewählt werden, die auch in der Tabelle der Datenbank als Feldname vorgesehen ist. Dadurch kann einwandfrei Feldname und Feldinhalt zugeordnet werden.

Hilfetexte werden zwar beim Auslesen der Daten mit angezeigt, erscheinen allerdings wohl nicht in jedem *pdf-Viewer.

Damit das Formular die Daten anschließend auch enthält, ist nach der Dateneingabe über **Datei → Speichern unter** im PDF-Viewer eine Speicherung der Datei vorzunehmen. Dieser Befehl kann von Viewer zu Viewer unterschiedlich sein. Ohne diese Maßnahme zeigt der Viewer die Daten nach dem Öffnen des Formular auf dem eigenen Rechner zwar an, liest sie aber wohl aus den temporä-

ren Dateien des Viewers, nicht aus der *.pdf-Datei direkt. Das Formular bleibt dann beim Transport von einem Rechner auf einen anderen leer.

Auslesen der Daten aus dem PDF-Formular

Das Formular der Base-Datenbank sieht recht einfach aus. Es ist mit der Tabelle verbunden und zeigt die gerade eingelesenen Daten an. Dabei sind die jüngsten Einträge in dem Tabellenkontrollfeld oben zu sehen.

	ID	Vorname	Nachname	Geburtsdatum	Punktzahl
▶	17	Karl	Käfer	01.03.12	3,71
	18	Annabelle		05.07.31	123,47
☼ Feld>					

PDF-Formular Importieren

Datensatz 1 von 2

Das Makro zum Einlesen der Daten wird unter **Eigenschaften : Schaltfläche → Ereignisse → Aktion ausführen** eingetragen.

Zum Auslesen der Daten wird hier auf das OpenSource-Programm «pdftk» zurückgegriffen. Das Programm ist auf jeden Fall frei für Linux und Windows erhältlich. Linux-Distributionen haben meist bereits ein Paket in den Repositories. Windows-User finden das Programm hier: <https://www.pdflabs.com/tools/pdftk-the-pdf-toolkit/>.

Die mit «pdftk» ausgelesenen Daten werden in eine Textdatei geschrieben, die wie folgt aussieht:

```
1 ---
2 FieldType: Text
3 FieldName: Vorname
4 FieldFlags: 0
5 FieldValue: Karl
6 FieldJustification: Left
7 ---
8 FieldType: Text
9 FieldName: Nachname
10 FieldFlags: 0
11 FieldValue: Käfer
12 FieldJustification: Left
13 ---
14 FieldType: Text
15 FieldName: Geburtsdatum
16 FieldNameAlt: Datum mit mindestens zweistelliger Jahreszahl
17 FieldFlags: 0
18 FieldValue: 1.3.12
19 FieldJustification: Left
20 ---
21 FieldType: Text
22 FieldName: Punktzahl
23 FieldNameAlt: Dezimalzahl, 2 Nachkommastellen
24 FieldFlags: 0
25 FieldValue: 3,71
26 FieldJustification: Left
27
```

Für jedes Feld sind in der Textdatei 5 Zeilen enthalten. Für die Auswertung im Makro sind die Zeilen «FieldName» (Feldname auch in der Zieltabelle), «FieldValue» (Inhalt des Feldes nach Abspeicherung der *.pdf-Datei) sowie «FieldJustification» (letzte Zeile, die einen Feldeintrag beendet) von Bedeutung.

Der gesamte Import wird über Makros geregelt. Hier muss das PDF-Formular im gleichen Pfad wie die Datenbank abgelegt werden. Die Daten werden in die Textdatei ausgelesen und aus dieser Textdatei wieder eingelesen. Dies geschieht so oft, wie *.pdf-Dateien mit Formulardaten in dem

Verzeichnis liegen. Alte Dateien sollten also tunlichst aus dem Verzeichnis entfernt werden, denn die Funktion überprüft nicht auf Duplikate.

```
SUB PDF_Form_Import(oEvent AS OBJECT)
    DIM inNumber AS INTEGER
    DIM stRow AS STRING
    DIM i AS INTEGER
    DIM k AS INTEGER
    DIM oDatasource AS OBJECT
    DIM oConnection AS OBJECT
    DIM oSQL_Command AS OBJECT
    DIM oResult AS OBJECT
    DIM stSql AS STRING
    DIM oDB AS OBJECT
    DIM oFileAccess AS OBJECT
    DIM inFields AS INTEGER
    DIM stFieldName AS STRING
    DIM stFieldValue AS STRING
    DIM stFieldType AS STRING
    DIM stDir AS STRING
    DIM stDir2 AS STRING
    DIM stPDFForm AS STRING
    DIM stFile AS STRING
    DIM stTable AS STRING
    DIM inNull AS INTEGER
    DIM aFiles()
    DIM aNull()
    DIM stCommand AS STRING
    DIM stParameter AS STRING
    DIM oShell AS OBJECT
```

Nach der Deklaration der Variablen wird die Anzahl der Felder angegeben, die das PDF-Formular enthält. Die Zählung beginnt hier mit 0. Beim Wert '3' sind also insgesamt 4 Felder in dem Formular vorhanden. Mit Hilfe dieser Zählung wird ermittelt, wann alle Daten für einen Datensatz ausgelesen wurden, so dass die Daten in die Tabelle der Datenbank übergeben werden können.

```
inFields = 3
stTable = "Name"
oDatasource = ThisComponent.Parent.CurrentController
If NOT (oDatasource.isConnected()) THEN
    oDatasource.connect()
END IF
oConnection = oDatasource.ActiveConnection()
oSQL_Command = oConnection.createStatement()
```

Die Datenbankverbindung wurde ermittelt. Anschließend wird der Pfad zu der bestehenden Datenbankdatei im Dateisystem ausgelesen. Mit Hilfe dieses Pfades wird in dem Array **aFiles** der Inhalt des Verzeichnisses ermittelt. Für jede der Dateien des Verzeichnisses wird in einer Schleife nachgesehen, ob die Dateiendung «.pdf» lautet. Groß- und Kleinschreibung sind hier egal, da das Ergebnis der Suche über **LCase** direkt in Kleinschreibung umgesetzt wird.

```
oDB = ThisComponent.Parent
stDir = Left(oDB.Location, Len(oDB.Location) - Len(oDB.Title))
oFileAccess = createUnoService("com.sun.star.ucb.SimpleFileAccess")
aFiles = oFileAccess.getFolderContents(stDir, False)
FOR k = 0 TO uBound(aFiles())
    IF LCase(Right(aFiles(k), 4)) = ".pdf" THEN
        stDir2 = ConvertFromUrl(stDir)
        stPDFForm = ConvertFromUrl(aFiles(k))
```

Um das Kommando für das Auslesen betriebssystemspezifisch adressieren zu können, muss der ursprünglich mit **file://** beginnende Pfadname dem jeweiligen System angepasst werden. Das Startkommando für das Programm «pdftk» ist abhängig vom Betriebssystem entweder mit dem Zusatz «.exe», vielleicht sogar mit dem gesamten Pfad zum Programm wie z.B. «C:\Program Files (x86)\pdftk\pdftk.exe», oder eben ohne Zusatz zu verstehen. Über **GetGuiType** wird der Systemtyp ermittelt, wobei '1' für Windows, '3' für MAC und '4' für Unix/Linux steht. Im folgenden wird nur zwischen Windows und allen anderen Systemen unterschieden.

Im Anschluss daran gibt der Befehl **Shell()** die Startinformationen für **pdftk** an die Konsole weiter. Über die Angabe von **True** wird dabei geregelt, dass LibreOffice wartet, bis der Shell-Prozess beendet wird.

```
IF GetGuiType = 1 THEN '()
    stCommand = "pdftk.exe"
ELSE
    stCommand = "pdftk"
END IF
stParameter = stPDFForm & " dump_data_fields_utf8 output "
    & stDir2 & "PDF_Form_Data.txt"
Shell(stCommand,0,stParameter,True)
stFile = stDir & "PDF_Form_Data.txt"
i = -1
inNumber = FreeFile
```

Mit der Funktion **FreeFile** wird ermittelt, welchen freien Datenkanal das Betriebssystem als nächstes zur Verfügung stellen wird. Dieser Datenkanal wird als eine Integerziffer ausgelesen und direkt zur Öffnung der gerade erstellten PDF-Datendatei genutzt. Die Datei wird über INPUT ausgelesen. Dies ist aus Sicht von LibreOffice gesehen. Daten von außerhalb werden in das Programm LibreOffice eingelesen.

```
OPEN stFile FOR INPUT AS inNumber
DO WHILE NOT Eof(inNumber)
    LINE INPUT #inNumber, stRow
```

Zeile für Zeile wird die PDF-Datendatei jetzt ausgelesen. Taucht der Begriff «FieldName: » auf, so entspricht der folgende Inhalt der Zeile der Bezeichnung des Feldes im PDF-Formular und durch die Definition des Formulars auch der Bezeichnung des Feldes in der Datenbanktabelle, in die der Inhalt geschrieben werden soll.

Alle Feldnamen werden direkt für den späteren SQL-Befehl zusammengefasst. Das bedeutet, dass die Feldnamen mit doppelten Anführungszeichen versehen und durch Kommas voneinander getrennt werden.

Zu jedem Feldnamen wird außerdem durch eine Abfrage ermittelt, um welchen Feldtyp der Tabelle es sich hierbei handelt (**HSQLDB**, SQL-Code für Firebird siehe Anhang). Datumswerte und Dezimalzahlen müssen anders weiter gegeben werden als Texte.

```
IF instr(stRow, "FieldName: ") THEN
    IF stFieldName = "" THEN
        stFieldName = """" + mid(stRow,12) + """"
    ELSE
        stFieldName = stFieldName & ", "" + mid(stRow,12) + """"
    END IF
    stSql = "SELECT TYPE_NAME FROM INFORMATION_SCHEMA.SYSTEM_COLUMNS
        WHERE TABLE_NAME = '" + stTable + "' AND
        COLUMN_NAME = '" + mid(stRow,12) + "'"
    oResult = oSQL_Command.executeQuery(stSql)
    WHILE oResult.next
        stFieldType = oResult.getString(1)
    WEND
END IF
```

Wie bei den Feldnamen wird auch bei den Werten verfahren. Sie dürfen allerdings nicht in doppelten Anführungszeichen weitergegeben werden, sondern müssen für den SQL-Code entsprechend vorbereitet werden. So muss Schrift in einfache Anführungszeichen gesetzt werden, Datumsangaben SQL-konform umgewandelt werden usw. Dies geschieht durch die extra ausgelagerte Funktion **SQL_Value**.

```
IF instr(stRow, "FieldValue: ") THEN
    IF stFieldValue = "" THEN
        stFieldValue = SQL_Value(mid(stRow,13), stFieldType)
    ELSE
        stFieldValue = stFieldValue & ", " &
            SQL_Value(mid(stRow,13), stFieldType)
    END IF
```

```
END IF
```

Taucht der Begriff «FieldJustification:» auf, so ist das Ende einer Kombination von Feldbezeichnung und Feldwert erreicht. Der Zähler **i**, der anschließend mit der vorher angegebenen Anzahl der Felder **inFields** verglichen werden soll, wird um '1' heraufgesetzt.

Wenn schließlich **i** und **inFields** gleich ist wird der SQL-Befehl zusammengestellt. Allerdings soll vermieden werden, dass leere Datensätze bei leeren Formularen entstehen. Deshalb wird vorher nachgesehen, ob die Werte aller Felder **NULL** sind. Ist dies der Fall, so wird kein SQL-Befehl ausgelöst. Ansonsten wird der Datensatz in die Tabelle "Name" eingefügt. Anschließend werden die Variablen wieder auf ihre Standardwerte zurückgesetzt und das nächste PDF-Formular kann ausgelesen werden.

```
IF instr(stRow, "FieldJustification:") THEN
    i = i + 1
END IF
IF i = inFields THEN
    aNull = Split(stFieldValue, ",")
    FOR n = 0 TO Ubound(aNull())
        IF aNull(n) = "NULL" THEN inNull = inNull + 1
    NEXT
    IF inNull < inFields THEN
        stSql = "INSERT INTO "" + stTable + "" (" + stFieldName + ")"
        stSql = stSql + "VALUES (" + stFieldValue + ")"
        oSQL_Command.executeUpdate(stSql)
    END IF
    stFieldName = ""
    stFieldValue = ""
    stFieldType = ""
    i = -1
    inNull = 0
END IF
LOOP
CLOSE inNumber
```

Zum Schluss der Prozedur bleibt schließlich eine Datei «PDF_Form_Data.txt» übrig. Diese Datei wird gelöscht. Anschließend wird das Formular neu geladen, damit die eingelesenen Daten direkt angezeigt werden können.

```
Kill(stFile)
END IF
NEXT
oEvent.Source.Model.Parent.reload()
END SUB
```

Enthält ein Text ein einfaches Anführungszeichen «'», so wird es beim Einfügen in SQL als Ende des Textes angesehen. Der SQL-Code für den Insert-Befehl scheitert, wenn anschließend noch Text folgt, der nicht in einfachen Anführungszeichen steht. Um dies zu vermeiden muss jedes einfache Anführungszeichen innerhalb des Textes mit einem weiteren einfachen Anführungszeichen maskiert werden. Das erledigt die Funktion **String_to_SQL**.

```
FUNCTION String_to_SQL(st AS STRING) AS STRING
    IF Instr(st, "'") THEN
        st = Join(Split(st, "'"), "''")
    END IF
    String_to_SQL = st
END FUNCTION
```

Die Datumsangabe in einem PDF-Formular wird als Text ausgelesen. Sie kann nicht vorher auf korrekte Eingabe kontrolliert werden.

In der deutschen Schreibweise werden Tag, Monat und Jahr voneinander häufig durch einen Punkt getrennt. Dabei kann die Tagesangabe und die Monatsangabe einstellig oder zweistellig, die Jahresangabe zweistellig oder vierstellig sein.

Für den SQL-Code muss die Datumsangabe beginnend mit der vierstelligen Jahreszahl in dem folgenden international üblichen erweiterten ISO-Format Format geschrieben werden: YYYY-MM-DD. Es muss also gegebenenfalls eine Umwandlung des eingegebenen Datums erfolgen.

Die Datumsangabe wird in den Tagesanteil, den Monatsanteil und den Jahresanteil aufgesplittet. Die Tagesangabe und die Monatsangabe wird mit einer führenden «0» versehen und anschließend von rechts aus auf zwei Zeichen begrenzt. Damit ist die Angabe auf jeden Fall zweistellig.

Ist die Jahresangabe bereits vierstellig (größer als 1000), so wird der Wert nicht geändert. Ansonsten wird bei einer Jahresangabe größer als 30 davon ausgegangen, dass es sich um eine Angabe handelt, die im letzten Jahrhundert liegt und um 1900 erhöht werden muss. Alle anderen Jahresangaben werden als Angaben für das aktuelle Jahrhundert angesehen.

```
FUNCTION Date_to_SQLDate(st AS STRING) AS STRING
    DIM stDay AS STRING
    DIM stMonth AS STRING
    DIM stDate AS STRING
    DIM inYear AS INTEGER
    stDay = Right("0" & Day(CDate(st)), 2)
    stMonth = Right("0" & Month(CDate(st)), 2)
    inYear = Year(CDate(st))
    IF inYear = 0 THEN
        inYear = Year(Now())
    END IF
    IF inYear > 1000 THEN
    ELSEIF inYear > 30 THEN
        inYear = 1900 + inYear
    ELSE
        inYear = 2000 + inYear
    END IF
    stDate = inYear & "-" & stMonth & "-" & stDay
    Date_to_SQLDate = stDate
END FUNCTION
```

Die Funktion **SQL_Value** fasst die vorhergehenden Funktionen sowie die **NULL**-Werte zusammen und gibt entsprechend vformatierte Werte für die Eingabe in die Datenbank an die aufrufende Prozedur zurück.

Für leere Felder wird NULL zurück gegeben. Damit bleibt das Feld auch in der Tabelle anschließend leer.

```
FUNCTION SQL_Value(st AS STRING, stType AS STRING) AS STRING
    DIM stValue AS STRING
    IF st = "" THEN
        SQL_Value = "NULL"
```

Handelt es sich bei dem Feldtyp um ein Datumsfeld und ist der Inhalt als Datum zu erkennen, so soll der Inhalt in ein SQL-Datumsformat umgewandelt werden. Ist der Inhalt nicht als Datum erkennbar, so soll das Feld leer bleiben.

```
    ELSEIF stType = "DATE" THEN
        IF isDate(st) THEN
            SQL_Value = "'" & Date_to_SQLDate(st) & "'"
        ELSE
            SQL_Value = "NULL"
        END IF
```

Handelt es sich bei dem Feldtyp um ein Dezimalfeld, so kann es Nachkommastellen geben. Der Dezimaltrenner in Basic und auch in SQL ist allerdings ein Punkt. Entsprechend müssen Zahlen umgewandelt werden, sofern sie ein Komma enthalten. Das Feld kann nur Zahlen aufnehmen, so dass andere Zeichen wie z.B. Maßeinheiten entfernt werden müssen. Dies geschieht mit der Funktion **Val()**. Leider gibt diese Funktion aber den Wert mit dem Dezimaltrenner der eingestellten Sprache zurück, so dass hier anschließend noch das Komma erneut durch einen Punkt ersetzt werden muss.

```
    ELSEIF stType = "DECIMAL" THEN
        stValue = Str(Val(Join(Split(st, ","), ".")))
```

```
SQL_Value = Join(Split(stValue, ","), ".")
```

Alle anderen Inhalte werden als Text behandelt. Einfache Anführungszeichen werden mit einem weiteren einfachen Anführungszeichen maskiert und der Gesamtbegriff wiederum in einfachen Anführungszeichen gefasst weitergegeben.

```
ELSE
    SQL_Value = "'" & String_to_SQL(st) & "'"
END IF
END FUNCTION
```

Zu weiteren Details beim Aufbau von Makros sollte das separate Kapitel dieses Handbuches zu Rate gezogen werden. Dieses Beispiel sollte lediglich aufzeigen, dass es auch möglich ist, Daten aus PDF-Formularen nach Base zu übertragen, ohne die Werte über die Zwischenablage aus jedem Feld in die Datenbank kopieren zu müssen. Der Aufbau der obigen Prozedur ist dabei recht allgemein gehalten und müsste sicher den jeweiligen Bedürfnissen angepasst werden.

Datenbank-Aufgaben

Allgemeines zu Datenbankaufgaben

Hier werden einige Lösungen für Problemstellungen vorgestellt, die im Laufe der Zeit viele Datenbankuser beschäftigen werden. Anfragen dazu kamen vor allem aus den Mailinglisten, insbesondere users@de.libreoffice.org, sowie aus den Foren <http://de.openoffice.info/viewforum.php?f=8> und <http://www.libreoffice-forum.de/viewforum.php?f=10>.

Datenfilterung

Die Datenfilterung mittels der GUI ist bereits bei der Dateneingabe in Tabellen beschrieben. Hier soll eine Lösung aufgezeigt werden, die bei vielen Nutzern gefragt ist: Mittels Listenfeldern werden Inhalte von Tabellenfeldern ausgesucht, die dann im darunterliegenden Formularteil herausgefiltert erscheinen und bearbeitet werden können.

Grundlage für diese Filterung ist neben einer bearbeitbaren Abfrage (siehe das Kapitel «[Eingabemöglichkeit in Abfragen](#)») eine weitere Tabelle, in der die zu filternden Daten abgespeichert werden. Die Abfrage zeigt aus der ihr zugrundeliegenden Tabelle nur die Datensätze an, die dem eingegebenen Filterwert entsprechen. Ist kein Filterwert angegeben, so zeigt die Abfrage alle Datensätze an.

Für das folgenden Beispiel wird von einer Tabelle "**Medien**" ausgegangen, die unter anderem die folgenden Felder beinhaltet: "**ID**" (Primärschlüssel), "**Titel**", "**Kategorie**".

Zuerst wird eine Tabelle "**Filter**" benötigt. Diese Tabelle erhält einen Primärschlüssel und 2 Filterfelder (das kann natürlich beliebig erweitert werden): "**ID**" (Primärschlüssel), "**Filter_1**", "**Filter_2**". Da die Felder der Tabelle "**Medien**", die gefiltert werden sollen, vom Typ **VARCHAR** sind, haben auch die Felder "**Filter_1**" und "**Filter_2**" diesen Typ. "**ID**" kann ein **Ja/Nein**-Feld sein. Die Tabelle "**Filter**" wird sowieso nur einen Datensatz abspeichern.

Feldname	Feldtyp
ID	Ja/Nein [BOOLEAN]
Filter_1	Text [VARCHAR]
Filter_2	Text [VARCHAR]

Tipp

Wird keine Einbenutzer-Datenbank wie Base mit der internen HSQLDB genutzt, so würde so eine Filtertabelle erst einmal die Filterung auch bei anderen Nutzern erzeugen. Hier könnte gegebenenfalls mit temporären Tabellen gearbeitet werden, die nur für den Nutzer sichtbar sind, der sie erstellt hat. Es könnte aber auch einfach der Nutzernamen direkt als Primärschlüssel der Filtertabelle genutzt werden. Dann wäre das Feld ID kein Ja/Nein-Feld, sondern ein VARCHAR-Feld.

Über

```
SELECT CURRENT_USER From "Medien"
```

würde in diesem Falle der aktuelle Nutzer abgefragt. Bei der internen Datenbank ist das immer 'SA'. Entsprechend darf bei der Filterung dann allerdings nur der Datensatz ausgelesen werden, bei dem

```
"ID" = CURRENT_USER
```

ist.

Natürlich kann auch nach Feldern gefiltert werden, die in der Tabelle "Medien" nur über einen Fremdschlüssel vertreten sind. Dann müssen die entsprechenden Felder in der Tabelle "Filter" natürlich dem Typ des Fremdschlüssels entsprechen, in der Regel also «Integer» sein.

Folgende Abfrageform bleibt sicher editierbar:

```
SELECT * FROM "Medien"
```

Alle Datensätze der Tabelle "**Medien**" werden angezeigt, auch der Primärschlüssel.

```
SELECT * FROM "Medien"
WHERE "Titel" = COALESCE( ( SELECT "Filter_1" FROM "Filter" ),
"Titel" )
```

Ist das Feld "**Filter_1**" nicht **NULL**, so werden die Datensätze angezeigt, bei denen der "**Titel**" gleich dem "**Filter_1**" ist. Wenn das Feld "**Filter_1**" **NULL** ist wird stattdessen der Wert des Feldes "**Titel**" genommen. Da "**Titel**" gleich "**Titel**" ist, werden so alle Datensätze angezeigt – sollte angenommen werden, trifft aber nicht zu, wenn im Feld "**Titel**" irgendwo ein leeres Feld '**NULL**' enthalten ist. Das bedeutet, dass die Datensätze nie angezeigt werden, die keinen Titeleintrag haben. Hier muss in der Abfrage nachgebessert werden.

```
SELECT * , COALESCE( "Titel", '' ) AS "T"
FROM "Medien"
WHERE "T" = COALESCE( ( SELECT "Filter_1" FROM "Filter" ), "T" )
```

Diese Variante würde zum Ziel führen. Statt "**Titel**" direkt zu filtern, wird ein Feld gefiltert, das den Alias-Namen "**T**" erhält. Dieses Feld ist zwar weiter ohne Inhalt, aber eben nicht **NULL**. In der Bedingung wird nur auf dieses Feld "**T**" Bezug genommen. Alle Datensätze werden angezeigt, auch wenn "**Titel**" **NULL** sein sollte.

Leider spielt hier die GUI nicht mit. Der Befehl ist nur direkt über SQL absetzbar. Um ihn mit der GUI editierbar zu machen, ist weitere Handarbeit erforderlich:

```
SELECT "Medien".* , COALESCE( "Medien"."Titel", '' ) AS "T"
FROM "Medien"
WHERE "T" = COALESCE( ( SELECT "Filter_1" FROM "Filter" ), "T" )
```

Wenn jetzt der Tabellenbezug zu den Feldern hergestellt ist, ist die Abfrage auch in der GUI editierbar.

Zum Testen kann jetzt einfach ein Titel in "**Filter**". "**Filter_1**" eingegeben werden. Als "**Filter**". "**ID**" wird der Wert '**0**' gesetzt. Der Datensatz wird abgespeichert und die Filterung kann nachvollzogen werden. Wird "**Filter**". "**Filter_1**" wieder geleert, so macht die GUI daraus **NULL**. Ein erneuter Test ergibt, dass jetzt wieder alle Medien angezeigt werden. Bevor ein Formular erstellt und getestet wird, sollte auf jeden Fall ein Datensatz, aber wirklich nur einer, mit einem Primärschlüssel in der Tabelle "**Filter**" stehen. Nur ein Datensatz darf es sein, da Unterabfragen wie oben gezeigt nur einen Wert wiedergeben dürfen.

Die Abfrage wird jetzt erweitert, um auch ein 2. Feld zu filtern:

```
SELECT "Medien".* , COALESCE( "Medien"."Titel", '' ) AS "T", COALESCE(
"Medien"."Kategorie", '' ) AS "K"
FROM "Medien"
WHERE "T" = COALESCE( ( SELECT "Filter_1" FROM "Filter" ), "T" ) AND
"K" = COALESCE( ( SELECT "Filter_2" FROM "Filter" ), "K" )
```

Damit ist die Erstellung der editierbaren Abfrage abgeschlossen. Jetzt wird noch die Grundlage für die beiden Listenfelder als Abfrage zusammengestellt:

```
SELECT DISTINCT "Titel", "Titel"
FROM "Medien" ORDER BY "Titel" ASC
```

Das Listenfeld soll sowohl die "**Titel**" anzeigen als auch die "**Titel**" an die dem Formular zugrundeliegende Tabelle "**Filter**" in das Feld "**Filter_1**" weitergeben. Dabei sollen keine

doppelten Werte angezeigt werden (Anordnung «**DISTINCT**») . Und das Ganze soll natürlich richtig sortiert erscheinen. Dabei ist die Abfrage an die Standardeinstellung der Listenfelder angepasst, die dem gebundenen Feld eine '1' zuweist. Wird stattdessen unter **Eigenschaften Listefeld → Daten → Gebundenes Feld** eine '0' zugewiesen, so braucht nur einmal das Feld "Titel" abgefragt werden.

Eine entsprechende Abfrage wird dann auch für das Feld "**Kategorie**" erstellt, die ihre Daten in der Tabelle "**Filter**" in das Feld "**Filter_2**" schreiben soll.

Handelt es sich bei einem der Felder um ein Fremdschlüsselfeld, so ist die Abfrage entsprechend so anzupassen, dass der Fremdschlüssel an die zugrundeliegende Tabelle "Filter" weitergegeben wird.

Das Formular besteht aus zwei Teilformularen. Formular 1 ist das Formular, dem die Tabelle "**Filter**" zugrunde liegt. Formular 2 ist das Formular, dem die Abfrage zugrunde liegt. Formular 1 hat **keine Navigationsleiste** und den Zyklus «**Aktueller Datensatz**». Die Eigenschaft «**Daten hinzufügen**» ist außerdem auf «**Nein**» gestellt. Der erste und einzige Datensatz existiert ja bereits.

Formular 1 enthält 2 Listenfelder mit entsprechenden Überschriften. Listefeld 1 soll Werte für "**Filter_1**" liefern und wird mit der Abfrage für das Feld "**Titel**" versorgt. Listefeld 2 soll Werte für "**Filter_2**" weitergeben und beruht auf der Abfrage für das Feld "**Kategorie**".

Formular 2 enthält ein Tabellenkontrollfeld, in dem alle Felder aus der Abfrage aufgelistet sein können – mit Ausnahme der Felder "**T**" und "**K**". Mit den Feldern wäre der Betrieb auch möglich – sie würden aber wegen der doppelten Feldinhalte nur verwirren. Außerdem enthält das Formular 2 noch einen Button, der die Eigenschaft «**Formular aktualisieren**» hat. Zusätzlich kann noch eine Navigationsleiste eingebaut werden, damit nicht bei jedem Formularwechsel der Bildschirm aufflackert, weil die Navigationsleiste in einem Formular ein-, in dem anderen ausgestellt ist.

Wenn das Formular fertiggestellt ist, geht es zur Testphase. Wird ein Listefeld geändert, so reicht die Betätigung des Buttons aus dem Formular 2 aus, um zuerst diesen Wert zu speichern und dann das Formular 2 zu aktualisieren. Das Formular 2 bezieht sich jetzt auf den Wert, den das Listefeld angibt. Die Filterung kann über die Wahl des im Listefeld enthaltenen leeren Feldes rückgängig gemacht werden.

Datensuche

Der Hauptunterschied zwischen der Suche von Daten und der Filterung von Daten liegt in der Abfragetechnik. Schließlich soll zu frei eingegebenen Begriffen ein Ergebnis geliefert werden, das diese Begriffe auch nur teilweise beinhaltet.

Suche mit LIKE

Die Tabelle für die Suchinhalte kann die gleiche sein, in die bereits die Filterwerte eingetragen werden. Die Tabelle "**Filter**" wird einfach ergänzt um ein Feld mit der Bezeichnung "**Suchbegriff**". So kann gegebenenfalls auf die gleiche Tabelle zugegriffen werden und in Formularen gleichzeitig gefiltert und gesucht werden. "**Suchbegriff**" hat die Feldeigenschaft «**VARCHAR**».

Das Formular wird wie bei der Filterung aufgebaut. Statt eines Listefeldes muss für den Suchbegriff ein Texteingabefeld erstellt werden, zusätzlich vielleicht auch ein Labelfeld mit dem Titel «Suche». Das Feld für den Suchbegriff kann alleine in dem Formular stehen oder zusammen mit den Feldern für die Filterung, wenn eben beide Funktionen gewünscht sind.

Der Unterschied zwischen Filterung und Suche liegt in der Abfragetechnik. Während die Filterung bereits von einem Begriff ausgeht, den es in der zugrundeliegenden Tabelle gibt (schließlich baut das Listefeld auf den Tabelleninhalten auf) geht die Suche von einer beliebigen Eingabe aus.

```
SELECT * FROM "Medien"
WHERE "Titel" = ( SELECT "Suchbegriff" FROM "Filter" )
```

Diese Abfrage würde in der Regel ins Leere führen. Das hat mehrere Gründe:

- Selten weiß jemand bei der Eingabe des Suchbegriffs den kompletten Titel fehlerfrei auswendig. Damit würde der Titel nicht angezeigt. Um das Buch «Per Anhalter durch die Galaxis» zu finden müsste es ausreichen, in das Suchfeld 'Anhalter' einzugeben, vielleicht auch nur 'Anh'.
- Ist das Feld "Suchbegriff" leer, so würde überhaupt kein Datensatz angezeigt. Die Abfrage gäbe **NULL** zurück und **NULL** kann in einer Bedingung nur mittels **IS NULL** erscheinen.
- Selbst wenn dies ignoriert würde, so würde die Abfrage dazu führen, dass alle die Datensätze angezeigt würden, die keine Eingabe im Feld "**Titel**" haben.

Die letzten beiden Bedingungen könnten erfüllt werden, indem wie bei der Filterung vorgegangen würde:

```
SELECT * FROM "Medien"
WHERE "Titel" = COALESCE( ( SELECT "Suchbegriff" FROM "Filter" ),
"Titel" )
```

Mit den entsprechenden Verfeinerungen aus der Filterung (was ist mit Titeln, die **NULL** sind?) würde das zum entsprechenden Ergebnis führen. Nur würde die erste Bedingung nicht erfüllt. Die Suche lebt ja schließlich davon, dass nur Bruchstücke geliefert werden. Die Abfragetechnik der Wahl müsste daher über den Begriff «**LIKE**» gehen:

```
SELECT * FROM "Medien"
WHERE "Titel" LIKE ( SELECT '%' || "Suchbegriff" || '%' FROM "Filter" )
```

oder besser:

```
SELECT * FROM "Medien"
WHERE "Titel" LIKE COALESCE( ( SELECT '%' || "Suchbegriff" || '%' FROM
"Filter" ), "Titel" )
```

LIKE, gekoppelt mit '%', bedeutet ja, dass alle Datensätze gezeigt werden, die an irgendeiner Stelle den gesuchten Begriff stehen haben. '%' steht als Joker für beliebig viele Zeichen vor und hinter dem Suchbegriff. Verschiedene Baustellen bleiben nach dieser Abfrageversion:

- Besonders beliebt ist ja, in Suchformularen alles klein zu schreiben. Wie bekomme ich mit 'anhalter' statt 'Anhalter' auch noch ein Ergebnis?
- Welche anderen Schreibgewohnheiten gibt es noch, die vielleicht zu berücksichtigen wären?
- Wie sieht es mit Feldern aus, die nicht als Textfelder formatiert sind? Lassen sich auch Datumsanzeigen oder Zahlen mit dem gleichen Feld suchen?
- Und was ist, wenn, wie bei dem Filter, ausgeschlossen werden muss, dass **NULL**-Werte in dem Feld verhindern, dass alle Datensätze angezeigt werden?

Die folgende Variante deckt ein paar mehr an Möglichkeiten ab:

```
SELECT * FROM "Medien"
WHERE LOWER("Titel") LIKE COALESCE( ( SELECT '%' || LOWER("Suchbe-
griff") || '%' FROM "Filter" ), LOWER("Titel") )
```

Die Bedingung ändert den Suchbegriff und den Feldinhalt auf Kleinschreibweise. Damit werden auch ganze Sätze vergleichbar.

```
SELECT * FROM "Medien" WHERE
LOWER("Titel") LIKE COALESCE( ( SELECT '%' || LOWER("Suchbegriff")
|| '%' FROM "Filter" ), LOWER("Titel") ) OR
LOWER("Kategorie") LIKE ( SELECT '%' || LOWER("Suchbegriff") || '%'
FROM "Filter" )
```

Die **COALESCE**-Funktion muss nur einmal vorkommen, da bei dem "**Suchbegriff**" **NULL** ja dann **LOWER("Titel") LIKE LOWER("Titel")** abgefragt wird. Und da der Titel ein Feld sein

soll, das nicht **NULL** sein darf, werden so auf jeden Fall alle Datensätze angezeigt. Für entsprechend viele Felder wird dieser Code natürlich entsprechend lang. Schöner geht so etwas mittels Makro, das dann den Code in einer Schleife über alle Felder erstellt.

Aber funktioniert der Code auch mit Feldern, die keine Textfelder sind? Obwohl die Bedingung **LIKE** ja eigentlich auf Texte zugeschnitten ist, brauchen Zahlen, Datums- oder Zeitangaben keine Umwandlung, um damit zusammen zu arbeiten. Allerdings können hierbei die Textumwandlungen unterbleiben. Nur wird natürlich ein Zeitfeld auf eine Mischung aus Text und Zahlen nicht mit einer Fundstelle reagieren können – es sei denn die Abfrage wird ausgeweitet, so dass der eine Suchbegriff an jeder Leerstelle unterteilt wird. Dies bläht allerdings die Abfrage noch wieder deutlich auf.

Tipp

Die Abfragen, die zur Filterung und zum Durchsuchen von Daten genutzt werden, lassen sich auch direkt in ein Formular einbauen.

Die gesamten obigen Bedingungen sind bei den Formular-Eigenschaften in der Zeile Filter eintragbar. Aus

```
SELECT * FROM "Medien" WHERE "Titel" = COALESCE( ( SELECT  
"Suchbegriff" FROM "Filter" ), "Titel" )
```

wird dann ein Formular, das als Inhalt die Tabelle "Medien" nutzt.

Unter «Filter» steht dann

```
("Medien"."Titel" = COALESCE( ( SELECT "Suchbegriff" FROM  
"Filter" ), "Medien"."Titel" ))
```

In den Filtereingaben ist darauf zu achten, dass die Bedingung in Klammern gesetzt wird und jeweils mit der Angabe "Tabelle"."Feld" arbeitet.

Vorteil dieser Variante ist, dass der Filter bei geöffnetem Formular ein- und wieder ausgeschaltet werden kann.

Suche mit LOCATE

Die Suche mit **LIKE** ist in der Regel völlig ausreichend für Datenbanken mit Feldern, die Text in überschaubarem Maße enthalten. Was aber, wenn der Inhalt über Memo-Felder eingegeben wird, also ohne weiteres auch einmal mehrere Seiten Text enthalten kann? Dann geht die Suche erst einmal los, wo denn nun der Text zu finden ist.

Um Text genau zu finden, gibt es in der **HSQldb** die Funktion **LOCATE**. **LOCATE** erwartet einen Suchbegriff sowie den Text, der durchsucht werden soll, als Parameter. Zusätzlich *kann* noch angegeben werden, ab welcher Position gesucht werden soll. Kurz also: **LOCATE(Suchbegriff, Textfeld aus der Datenbank, Startposition der Suche)**.

In **FIREBIRD** gibt es die Funktion **LOCATE** nicht. Hier muss auf **POSITION** zurückgegriffen werden: **POSITION(Suchbegriff, Textfeld aus der Datenbank, Startposition der Suche)**. Die Startposition kann hier eingegeben werden, muss es aber nicht.

Für die folgende Erklärung wird eine Tabelle genutzt, die den Namen "Tabelle" hat. Der Primärschlüssel heißt "ID" und muss lediglich einzigartig sein. Zusätzlich gibt es noch ein Feld "Memo", das als Feld des Typs **Memo (LONGVARCHAR)** erstellt wurde. In dem Feld "Memo" sind ein paar Absätze dieses Handbuchs gespeichert.¹⁴

14 Die Screenshots zu diesem Kapitel entstammen der Datenbank
«Beispiel_Autotext_Suchmarkierung_Rechtschreibung.odt», die dem Handbuch beiliegt.

Die Beispielabfragen sind als Parameterabfragen angelegt. Der einzugebende Suchtext ist jeweils 'office'.

ID	Memo
3	Jeder, der sich in das Modul Base von LibreOffice einarbeiten und tiefer einsteigen will, findet hier
4	Dieses Buch, wie auch die anderen LibreOffice-Handbücher, das eingebaute Hilfesystem und die
5	LibreOffice besitzt ein umfangreiches Hilfesystem.

Datensatz 1 von 3

```
SELECT "ID", "Memo" FROM "Tabelle"
WHERE LOWER ( "Memo" ) LIKE '%' || LOWER ( :Suchtext ) || '%'
```

Zuerst ein Zugriff über **LIKE**. **LIKE** kann nur in der Bedingung stehen. Wird der Suchtext irgendwo gefunden, dann wird der entsprechende Datensatz angezeigt. Durch den Vergleich von der Kleinschreibung des Feldinhaltes über **LOWER("Memo")** mit der Kleinschreibung des Suchtextes über **LOWER(:Suchtext)** werden die Inhalte unabhängig von der Schreibweise gefunden. Je länger der Text in dem Memo-Feld ist, desto schwerer wird es, den Begriff dann tatsächlich zu sehen.

ID	Memo	Position
1	Dieses Dokument unterliegt dem Copyright © 2014. Die Beitragenden sind unten aufgeführt. Sie	0
2	Alles, was an eine Mailingliste geschickt wird, inklusive der E-Mail-Adresse und	0
3	Jeder, der sich in das Modul Base von LibreOffice einarbeiten und tiefer einsteigen will, findet hier	44
4	Dieses Buch, wie auch die anderen LibreOffice-Handbücher, das eingebaute Hilfesystem und die	40
5	LibreOffice besitzt ein umfangreiches Hilfesystem.	6

Datensatz 1 von 5

```
SELECT "ID", "Memo", LOCATE( LOWER ( :Suchtext ), LOWER ( "Memo" ) ) AS "Position"
FROM "Tabelle"
```

LOCATE gibt genauer wieder, an welcher Stelle sich der Suchbegriff befindet. In Datensatz 1 und 2 ist der Suchbegriff nicht vorhanden. **LOCATE** gibt als Position hier '0' aus. Leicht nachzählen lässt sich das Ergebnis am Datensatz 5: Mit dem 6. Buchstaben beginnt hier die Textfolge 'Office'.

Natürlich wäre es auch möglich, das entsprechende Ergebnis wie bei **LIKE** auch über **LOCATE** zu erhalten:

```
SELECT "ID", "Memo"
FROM "Tabelle"
WHERE LOCATE(LOWER(:Suchtext), LOWER("Memo")) > 0
```

Das Auffinden der Position allein ist im obigen Beispiel auch mit einem Blick auf das Feld "Memo" schon recht einfach. Komplizierter wird es aber, wenn der Inhalt eben nicht gerade, wie hier zur Demonstration, in den ersten 70 Zeichen enthalten ist. Dann wird es sinnvoll, Textstücke mit dem Inhalt direkt zu finden.

	ID	Memo	Position	Treffer
▶	1	Dieses Dokument	0	**keine Fundstelle**
	2	Alles, was an eine	0	**keine Fundstelle**
	3	Jeder, der sich in das	44	von LibreOffice einarbeiten und tiefer einsteigen will, findet hier die
	4	Dieses Buch, wie auch	40	LibreOffice-Handbücher, das eingebaute Hilfesystem und die Benutzer-
	5	LibreOffice besitzt ein	6	LibreOffice besitzt ein umfangreiches Hilfesystem.

Datensatz	1	von 5	⏮ ⏪ ⏩ ⏭
-----------	---	-------	---------

```

SELECT "ID", "Memo", "Position",
CASE
WHEN "Position" = 0 THEN '**keine Fundstelle**'
WHEN "Position" < 10 THEN SUBSTRING ( "Memo", 1 )
ELSE SUBSTRING ( "Memo", LOCATE( ' ', "Memo", "Position" - 10 ) + 1 )
END AS "Treffer"
FROM
( SELECT "ID", "Memo", LOCATE( LOWER ( :Suchtext ), LOWER ( "Memo" ) ) AS "Position"
FROM "Tabelle" )

```

In der Spalte «Treffer» wird das Suchergebnis genauer dargestellt. Die vorherige Abfrage ist einfach als Basis für diese Abfrage genommen worden. Dies bewirkt, dass in der äußeren Abfrage nicht jedes Mal **LOCATE(LOWER(:Suchtext), LOWER("Memo"))** sondern einfach "Position" eingegeben werden kann. Vom Prinzip her ist dieses Verfahren nicht anders, als wenn die vorhergehende Abfrage gespeichert würde und diese Abfrage als Quellenangabe auf die vorherige Abfrage zugreift.

"Position" = 0 bedeutet, dass kein Suchergebnis vorhanden ist. In dem Fall also die Ausgabe ****keine Fundstelle****.

"Position" < 10 bedeutet, dass sich der Suchbegriff direkt am Anfang des Textes befindet. 10 Zeichen können leicht überblickt werden. Es wird also der gesamte Text wiedergegeben. Hier könnte also auch statt **SUBSTRING("Memo", 1)** direkt **"Memo"** stehen.

Für alle anderen Treffer wird ab 10 Zeichen vor der Position des Treffers nach einer Leerstelle ' ' gesucht. Der Text soll nicht mitten in einem Wort starten, sondern nach so einer Leerstelle beginnen. Über **SUBSTRING("Memo", LOCATE(' ', "Memo", "Position"-10)+1)** wird erreicht, dass der Text mit dem Beginn eines Wortes startet, das maximal 10 Zeichen vor dem Begriff 'office' erscheint.

In der Praxis dürften hier mehr Zeichen erforderlich sein, da doch sehr viele Worte die Anzahl von 10 Zeichen übersteigen und selbst der Suchbegriff ja in einem Wort liegen kann, das noch 10 Zeichen vor dem eigentlich Begriff hat. 'LibreOffice' wird bei Suchbegriff 'office' so noch dargestellt, da das 'O' an der 6. Stelle steht. Stellen wir uns aber z.B. den Begriff 'hand' vor, so würde im 4. Datensatz bereits das Aus für die Darstellung stehen. 'LibreOffice-Handbücher' hat, von 'hand' aus nach links gezählt, 12 Zeichen. Wird aber höchsten 10 Zeichen nach links gesucht, so wird als erstes Leerzeichen das Zeichen hinter dem Komma gefunden. Die Darstellung in «Treffer» würde mit 'das eingebaute Hilfesystem ...' beginnen.

	ID	Memo	Position	Treffer
▶	1	Dieses Dokument unterliegt dem	0	**keine Fundstelle**
	2	Alles, was an eine Mailingliste	0	**keine Fundstelle**
	3	Jeder, der sich in das Modul Base von	44	von LibreOffice einarbeit
	4	Dieses Buch, wie auch die anderen	40	LibreOffice-Handbücher, d
	5	LibreOffice besitzt ein umfangreiches	6	LibreOffice besitzt ein u

Datensatz	1	von 5	⏮ ⏪ ⏩ ⏭
-----------	---	-------	---------

```

SELECT "ID", "Memo", "Position",
CASE
WHEN "Position" = 0 THEN '**keine Fundstelle**'
WHEN "Position" < 10 THEN SUBSTRING ( "Memo", 1 , 25 )
ELSE SUBSTRING ( "Memo", LOCATE( ' ', "Memo", "Position" - 10 ) + 1 , 25 )
END AS "Treffer"
FROM
( SELECT "ID", "Memo", LOCATE( LOWER ( :Suchtext ), LOWER ( "Memo" ) ) AS "Position"
FROM "Tabelle" )

```

Die Abfragetechnik ist gegenüber der vorhergehenden Abfrage gleich geblieben. Lediglich die Länge des auszugebenden Treffers ist beschränkt worden. In diesem Falle erfolgte die Beschränkung hart auf 25 Zeichen. Die Funktion **SUBSTRING** erfordert als erstes die Angabe des zu durchsuchenden Textes, als zweites dann die Startposition der Ausgabe und als drittes optional die Länge des auszugebenden Textes. Natürlich hier auch reichlich kurz gehalten, aber eben nur zu Demonstrationszwecken. Vorteil der Verkürzung ist natürlich ein deutlich geringerer Speicherverbrauch bei entsprechend großen Datenmengen und ein direkter Blick auf die Fundstelle. Sichtbarer Nachteil dieser Form der Verkürzung ist aber, dass der Schnitt rigoros nach dem 25. Zeichen gemacht wird – ohne Rücksicht auf einen Wortbeginn.

	ID	Memo	Position	Treffer
▶	1	Dieses Dokument unterliegt dem	0	**keine Fundstelle**
	2	Alles, was an eine Mailingliste	0	**keine Fundstelle**
	3	Jeder, der sich in das Modul Base von	44	von LibreOffice einarbeiten und
	4	Dieses Buch, wie auch die anderen	40	LibreOffice-Handbücher, das
	5	LibreOffice besitzt ein umfangreiches	6	LibreOffice besitzt ein umfangreiches

Datensatz	1	von 5	◀◀▶▶▶
-----------	---	-------	-------


```

SELECT "ID", "Memo", "Position",
CASE
WHEN "Position" = 0 THEN '**keine Fundstelle**'
WHEN "Position" < 10 THEN SUBSTRING ( "Memo", 1, LOCATE( ' ', "Memo", 25 ) )
ELSE SUBSTRING ( "Memo", LOCATE( ' ', "Memo", "Position" - 10 ) + 1,
( LOCATE( ' ', "Memo", "Position" + 20 ) -
( LOCATE( ' ', "Memo", "Position" - 10 ) + 1 ) )
)
END AS "Treffer"
FROM
( SELECT "ID", "Memo", LOCATE( LOWER ( :Suchtext ), LOWER ( "Memo" ) ) AS "Position"
FROM "Tabelle" )

```

Hier wird ab dem 25. Zeichen in dem darzustellenden «Treffer» nach dem nächsten Leerzeichen gesucht. Der auszugebende Inhalt wird dann durch die gefundene Position begrenzt.

Recht einfach gestaltet sich dies noch, wenn der Treffer am Anfang liegt. Hier gibt **LOCATE(' ', "Memo", 25)** genau die Position vom Anfang des gesamten Textes an wieder. Sie entspricht, da der Text von Anfang an ausgegeben werden soll, auch genau der Länge des auszugebenden Begriffes.

Die Suche der dem Suchbegriff folgenden Leerzeichen ist auch bei einem weiter hinten liegenden Treffer nicht weiter kompliziert. Die Suche beginnt einfach an der Position des Treffers. Hinzu-gezählt werden noch 20 Zeichen, die auf jeden Fall folgen sollen. Danach wird das nächste Leerzeichen ausgemacht: **LOCATE(' ', "Memo", "Position"+20)**. Hiermit ist aber nur die Position im Gesamtfeld ausgemacht. Die ermittelte Position gibt also auf keinen Fall die Länge des auszugebenden Textes wieder. Von dem ermittelten Positionswert muss hingegen der Positionswert abgezogen werden, bei dem die Ausgabe des Treffers starten soll. Dies wurde durch **LOCATE(' ', "Memo", "Position"-10)+1** vorher bereits einmal abgefragt. Erst so kann dann die korrekte Länge des Textes dargestellt werden.

	ID	Memo	Position01	Treffer01	Position02	Treffer02	Position03
1	Dieses	0		**keine Fundstelle**	0	**keine Fundstelle**	0
2	Alles, was	0		**keine Fundstelle**	0	**keine Fundstelle**	0
3	Jeder, der	44	von LibreOffice		312	einer OfficeSuite oder ein	0
4	Dieses	40	LibreOffice-Handbücher,		0	**keine Fundstelle**	0
5	LibreOffice	8	LibreOffice besitzt ein		123	Sie LibreOffice Hilfe aus dem	223

Datensatz 1 von 5

```

SELECT "ID", "Memo", "Position01", "Treffer01", "Position02",
CASE
  WHEN "Position02" = 0 THEN '**keine Fundstelle**'
  WHEN "Position02" < 10 THEN SUBSTRING ( "Memo", 1, LOCATE( ' ', "Memo", 25 ) )
  ELSE SUBSTRING ( "Memo", LOCATE( ' ', "Memo", "Position02" - 10 ) + 1,
    ( LOCATE( ' ', "Memo", "Position02" + 20 ) -
      ( LOCATE( ' ', "Memo", "Position02" - 10 ) + 1 ) )
  )
END AS "Treffer02",
CASE
  WHEN "Position02" = 0 THEN 0
  ELSE LOCATE( LOWER ( :Suchtext ), LOWER ( "Memo" ), "Position02" + 1 )
END AS "Position03"
FROM
( SELECT "ID", "Memo", "Position01",
CASE
  WHEN "Position01" = 0 THEN '**keine Fundstelle**'
  WHEN "Position01" < 10 THEN SUBSTRING ( "Memo", 1, LOCATE( ' ', "Memo", 25 ) )
  ELSE SUBSTRING ( "Memo", LOCATE( ' ', "Memo", "Position01" - 10 ) + 1,
    ( LOCATE( ' ', "Memo", "Position01" + 20 ) -
      ( LOCATE( ' ', "Memo", "Position01" - 10 ) + 1 ) )
  )
END AS "Treffer01",
CASE
  WHEN "Position01" = 0 THEN 0
  ELSE LOCATE( LOWER ( :Suchtext ), LOWER ( "Memo" ), "Position01" + 1 )
END AS "Position02"
FROM
( SELECT "ID", "Memo", LOCATE( LOWER ( :Suchtext ), LOWER ( "Memo" ) ) "Position01"
FROM "Tabelle" )
)

```

Mit der gleichen Technik können mehrere Abfragen hintereinander geschachtelt erfolgen. Die vorhergehende Abfrage ist jetzt die Datenquelle dieser Abfrage. Sie ist komplett unterhalb des Begriffes **FROM** in Klammern eingefügt worden. Lediglich die Felder wurden etwas umbenannt, da ja jetzt mehrere Positionen und Treffer angegeben werden. Außerdem wurde die nächste Position einer Fundstelle über **LOCATE(LOWER(:Suchtext), LOWER("Memo"), "Position01"+1)** ermittelt. Es wird also mit dem Suchen der nächsten Stelle einfach eine Stelle hinter dem vorhergehenden Treffer gestartet.

Die äußerste Abfrage stellt die entsprechenden Felder der anderen beiden Abfragen dar und fügt zusätzlich «Treffer02» auf die gleiche Weise hinzu, mit der vorher «Treffer01» ermittelt wurde. Außerdem wird in der äußeren Abfrage schon ermittelt, ob es vielleicht noch weitere Treffer gibt. Die entsprechende Position wird in «Position03» ausgegeben. Lediglich Datensatz 5 hat noch weitere Trefferpositionen vorzuweisen und könnte also in einer weiteren Nachfrage noch weitere Treffer ermöglichen.

Die Staffelung der Abfragen ist hier beliebig weit möglich. Allerdings werden sie mit jeder weiteren äußeren Abfrage natürlich für das System immer belastender. Hier sind entsprechende Tests notwendig, was denn nun sinnvoll und was realistisch machbar ist. Wie mit Hilfe von Makros über ein Formular sogar eine Abfragetechnik machbar ist, die gleichzeitig alle Fundstellen im Text markiert, ist im Kapitel «Makros» erklärt.

Bilder und Dokumente mit Base verarbeiten

Base-Formulare bieten für die Verarbeitung von Bildern grafische Kontrollfelder an. Nur über diese grafischen Kontrollfelder ist ohne Einsatz von Makros bei der internen HSQLDB möglich, Bilder in

die Datenbank einzulesen. Das grafische Kontrollfeld kann aber auch dazu genutzt werden, nur die Verknüpfung zu Bildern außerhalb der Datenbankdatei zu ermöglichen.¹⁵

Bilder in die Datenbank einlesen

Die Datenbank benötigt eine Tabelle, die mindestens die folgenden Voraussetzungen erfüllt:

Feldname	Feldtyp	Beschreibung
ID	Integer	Die ID ist Primärschlüssel dieser Tabelle.
Bild	Bild	Nimmt das Bild als Binärdatenstrom auf.

Beim Primärschlüssel ist natürlich nicht unbedingt der Integer-Feldtyp bestimmend. Ein Primärschlüssel ist aber unabdingbar. Andere Felder, die zumindest Informationen zu dem Bild enthalten, sollten noch hinzugefügt werden.

Daten, die irgendwann in das Bild-Feld eingetragen werden, sind in den Tabellen nicht lesbar. Dort erscheint als Anzeige nur **<OBJECT>**. Entsprechend sind Bilder auch nicht direkt in die Tabelle einfügbar. Sie müssen über ein Formular und dort mit Hilfe des grafischen Kontrollfeldes eingegeben werden. Das grafische Kontrollfeld öffnet beim Mausklick auf das Feld eine Dateiauswahl. Es zeigt hinterher das Bild an, das über die Dateiauswahl in die Datenbank eingelesen wurde.

Bilder, die direkt in die Datenbank eingefügt werden, sollten möglichst klein sein. Da Base ohne den Einsatz von Makros auch keine Möglichkeit bietet, die Bilder in Originalgröße wieder aus der Datenbank heraus zu befördern, macht es aus dieser Warte erst einmal Sinn, als Maßstab für die Größe z.B. einen möglichen Ausdruck im Bericht anzusehen. Originalbilder im Megapixelbereich sind hier völlig unnötig und blähen die Datenbank stark auf. Bereits nach wenigen Bildern meldet bei der internen HSQLDB Base eine **Java.NullPointerException** und kann den Datensatz nicht mehr speichern. Auch wenn die Bilder nicht ganz so groß sind, kann es irgendwann passieren, dass die Datenbankdatei unbedienbar wird.

Bilder sollten außerdem möglichst nicht in Tabellen integriert werden, die als Suchgrundlage gedacht werden. Wird z.B. in einer Datenbank zur Personenverwaltung auch das Passbild mit abgespeichert, so ist es besser in einer separaten Tabelle über einen Fremdschlüssel mit der Haupttabelle verbunden. Die Suche in der Haupttabelle kann dann deutlich schneller erfolgen, da die Tabelle selbst nicht so viel Speicher beansprucht.

Bilder und Dokumente verknüpfen

Mit einer entsprechend durchdachten Ordnerstruktur ist es günstiger, direkt auf die Dateien von außerhalb zuzugreifen. Dateien außerhalb der Datenbank können beliebig groß sein, ohne dass die Funktionen der Datenbank selbst beeinflusst werden. Leider bedeutet dies aber auch, dass eine Umbenennung von Ordnern auf dem eigenen Rechner oder im Internet dazu führt, dass der Zugriff auf die entsprechenden Dateien verloren geht.

Um Bilder nicht in eine Datenbankdatei einzulesen, sondern nur zu verknüpfen, bedarf es nur einer kleinen Änderung gegenüber der vorhergehenden Tabelle:

Feldname	Feldtyp	Beschreibung
ID	Integer	Die ID ist Primärschlüssel dieser Tabelle.
Bild	Text	Nimmt den Pfad zu dem Bild auf.

Wird einfach statt des Feldtyps **Bild** der Feldtyp **Text** gewählt, so wird über das grafische Kontrollfeld der Pfad zu dem Bild eingetragen. Das Bild kann über das Kontrollfeld genauso betrachtet werden wie ein Bild, das in die Datenbank eingefügt wurde. Leider lässt sich aber nicht auf die glei-

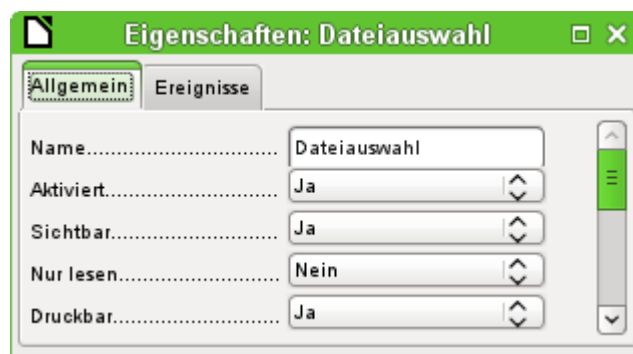
¹⁵ Das Beispiel «Externe_Bilder» ist als gepacktes Verzeichnis diesem Handbuch beigelegt.

che Art und Weise auch die Verbindung zu einem Dokument herstellen. Schon das Einlesen des Pfades ist nicht möglich, da das grafische Kontrollfeld von der Konstruktion her erst einmal nur für Bilder gedacht ist und auch entsprechend nur Bildtypen zur Auswahl anbietet, die es selbst auch anzeigen kann.

Bei einem Bild kann über den Pfad wenigstens noch der Inhalt auf dem grafischen Kontrollfeld gelesen werden. Bei einem Dokument kann aber keine Anzeige erfolgen, selbst wenn der Pfad in der Tabelle verzeichnet ist. Zuerst ist deshalb die Tabelle etwas zu erweitern, damit wenigstens ein geringfügiges Maß an Information zu dem Dokument sichtbar wird.

Feldname	Feldtyp	Beschreibung
ID	Integer	Die ID ist Primärschlüssel dieser Tabelle.
Beschreibung	Text	Beschreibung des Dokumentes, Suchbegriffe ...
Datei	Text	Nimmt den Pfad zu dem Bild auf.

Damit der Pfad zur Datei sichtbar wird, muss in dem Formular ein Dateiauswahlfeld mit eingebaut werden.



Ein Dateiauswahlfeld hat in seinen Eigenschaften keinen Reiter für Daten, ist also auch nicht mit irgendeinem Feld der dem Formular zugrundeliegenden Tabelle verbunden.

Dokumente mit absoluter Pfadangabe verknüpfen

Über das Dateiauswahlfeld kann zwar der Pfad angezeigt, aber nicht gespeichert werden. Hierfür ist eine gesonderte Prozedur erforderlich, die über **Ereignisse → Text modifiziert** ausgelöst wird:

```
SUB Pfad_Einlesen(oEvent AS OBJECT)
    DIM oForm AS OBJECT
    DIM oFeld AS OBJECT
    DIM oFeld2 AS OBJECT
    DIM stUrl AS STRING
    oFeld = oEvent.Source.Model
    oForm = oFeld.Parent
    oFeld2 = oForm.getByName("GraphischesFeld")
    IF oFeld.Text <> "" THEN
        stUrl = ConvertToUrl(oFeld.Text)
        oFeld2.BindField.updateString(stUrl)
    END IF
END SUB
```

Das auslösende Ereignis wird beim Aufruf der Prozedur mitgeliefert und hilft dabei, das Formular und auch das Feld zu finden, in dem der Pfad gespeichert werden soll. Mit **oEvent AS OBJECT** ist der Zugriff vor allem dann einfacher, wenn ein anderer User ein gleichlautendes Makro in irgendeinem Unterformular verwenden will. Das Dateiauswahlfeld ist von dort aus über **oEvent.Source.Model** zu erreichen. Das Formular ist **Parent** (Elternteil) zu dem Dateiauswahlfeld. Der Name des Formulars spielt also keine Rolle. Vom Formular aus geht dann der Zugriff auf das Feld mit dem Namen «GraphischesFeld». Über dieses Feld werden die Dateipfade der Bil-

der abgespeichert. Jetzt wird in das Feld die URL der ausgesuchten Datei geschrieben. Damit die URL betriebssystemunabhängig verwendet werden kann, wird der in dem Dateiauswahlfeld stehende Text vorher über **ConvertToUrl** in eine allgemeingültige URL-Schreibweise überführt.

In der Tabelle der Datenbank steht jetzt ein Pfad mit absoluter Schreibweise: **file:///...**

Werden allerdings über ein graphisches Kontrollfeld Pfadeingaben eingelesen, so erfolgt die Aufnahme in relativer Pfadangabe. Um dies zu bewerkstelligen, müsste nachgebessert werden. Die Prozedur für diesen Zweck sieht wesentlich umfangreicher aus, da ein Vergleich der Pfadeingabe mit der aktuellen Speicherposition der Datenbankdatei nötig ist.

Dokumente mit relativer Pfadangabe verknüpfen

Das folgende Makro ist an die Eigenschaft «Text modifiziert» des Dateiauswahlfeldes gebunden.

```
SUB PfadAuslesen
  DIM oDoc AS OBJECT
  DIM oDrawpage AS OBJECT
  DIM oForm AS OBJECT
  DIM oFeld AS OBJECT
  DIM oFeld2 AS OBJECT
  DIM arUrl_Start()
  DIM ar()
  DIM ar1()
  DIM ar2()
  DIM stText AS STRING
  DIM stUrl_gesamt AS STRING
  DIM stUrl_Text AS STRING
  DIM stUrl AS STRING
  DIM stUrl_weg AS STRING
  DIM ink AS INTEGER
  DIM i AS INTEGER
  oDoc = thisComponent
  oDrawpage = oDoc.Drawpage
  oForm = oDrawpage.Forms.getByName("Formular")
  oFeld = oForm.getByName("GraphischesFeld")
  oFeld2 = oForm.getByName("Dateiauswahl")
```

Zuerst werden, wie bei allen Prozeduren, die Variablen deklariert. Anschließend werden die entsprechenden Felder aufgesucht, die für die Aufnahme des Pfades wichtig sind. Der gesamte anschließende Code wird nur dann ausgeführt, wenn in der Dateiauswahl auch Inhalt steht, das Feld also z.B. nach einem Datensatzwechsel nicht einfach geleert wird.

```
IF oFeld2.Text <> "" THEN
  arUrl_Start = split(oDoc.Parent.Url, right(convertToUrl(oDoc.Parent.Title),
    len(convertToUrl(oDoc.Parent.Title))-8))
  ar = split(ConvertToUrl(oFeld2.Text), "/")
  stText = ""
```

Der Pfad zur Datenbankdatei wird ausgelesen. Dies erfolgt, wie vorher schon erklärt, indem zuerst die gesamte Url gelesen wird und anschließend der Dateiname, ebenfalls in Url-Schreibweise geändert, als Trenner eines Arrays dafür sorgt, dass im ersten Teil des Arrays nur der direkte Pfad steht.

Anschließend werden alle Elemente des Pfades, der über die Dateiauswahl ausgesucht wurde, in das Array **ar** eingelesen. Als Trenner dient /. Das gelingt unter Linux direkt. Unter Windows muss der Inhalt von oFeld2.Text auf jeden Fall noch in eine URL umgewandelt werden, die mit dem Slash und nicht mit dem Backslash \ arbeitet.

Die Trennung dient dazu, für die Ermittlung des Pfades zu der Datei einfach nur den Dateinamen hinten abzuschneiden. Daher wird im nächsten Schritt der Pfad zu der aufzurufenden Datei in der Variablen **stText** wieder zusammengesetzt. Die Schleife endet allerdings nicht mit dem höchsten Wert des vorher erstellten Arrays **ar**, sondern schon beim Erreichen der vorhergehenden Wertes.

```
FOR i = LBound(ar()) TO UBound(ar()) - 1
```

```

    stText = stText & ar(i) & "/"
NEXT
stText = Left(stText, Len(stText)-1)
arUrl_Start(0) = Left(arUrl_Start(0), Len(arUrl_Start(0))-1)

```

Das zuletzt angehängte / wird wieder entfernt, da sonst im folgenden Array zum Schluss ein leerer Arraywert erzeugt wird, der den Pfadvergleich stören würde. Für einen richtigen Vergleich ist jetzt der Text noch zu einer tatsächlichen Url, beginnend mit **file:///**, umzuwandeln. Schließlich soll mit dem Pfad der Datenbankdatei verglichen werden, der genau auf dieser Basis zusammengesetzt ist.

```

stUrl_Text = ConvertToUrl(stText)
ar1 = split(stUrl_Text, "/")
ar2 = split(arUrl_Start(0), "/")
stUrl = ""
ink = 0
stUrl_weg = ""

```

In **ar1** werden alle Elemente gespeichert, die den Pfad zu der aufzurufenden Datei wiedergeben. In **ar2** sind alle Elemente gespeichert, die den Pfad zur Datenbankdatei wiedergeben. Anschließend wird das gesamte Array **ar2** schrittweise für einen Vergleich in einer Schleife durchlaufen.

```

FOR i = LBound(ar2()) TO UBound(ar2())
    IF i <= UBound(ar1()) THEN

```

Nur wenn die Zahl **i** nicht größer ist als die Anzahl der in **ar1** enthaltenen Elemente wird der folgende Code ausgeführt. Ist der Wert aus **ar2** gleich dem entsprechenden Wert aus **ar1** und ist bisher noch kein unterschiedlicher Wert aufgetaucht, dann wird der gemeinsame Inhalt in einer Variablen gespeichert, die zum Schluss von der Pfadangabe abgeschnitten werden kann.

```

        IF ar2(i) = ar1(i) AND ink = 0 THEN
            stUrl_weg = stUrl_weg & ar1(i) & "/"
        ELSE

```

Ist irgendwann ein Unterschied zwischen den beiden Arrays aufgetaucht, so wird für jeden unterschiedlichen Wert die Bezeichnung für «setze ein Verzeichnis höher an» in der Variablen **stUrl** gespeichert.

```

            stUrl = stUrl & "../"
            ink = 1
        END IF

```

Sobald der Index, der durch die Variable **i** dargestellt wird, größer wird als die Anzahl der Elemente von **ar1**, wird für jeden weiteren Wert von **ar2** wieder ein weiteres **../** in der Variablen **stUrl** gespeichert.

```

        ELSE
            stUrl = stUrl & "../"
        END IF
    NEXT
    stUrl_gesamt = ConvertToUrl(oFeld2.Text)
    oFeld.boundField.UpdateString(stUrl & Right(stUrl_gesamt, Len(stUrl_gesamt)-
        Len(stUrl_weg)))
END IF
END SUB

```

Ist die Schleife durch **ar2** komplett durchlaufen, so steht fest, ob und wie viele Verzeichnisse höher von der Datenbankdatei aus gesehen das Verzeichnis für die aufzurufende Datei liegt. Jetzt wird einmal die **stUrl_gesamt** aus dem Text im Dateiauswahlfeld erstellt. Diese enthält auch den Dateinamen. Anschließend wird in das graphische Feld der Wert für die Url übertragen. Der Wert für die Url beginnt mit **stUrl**, in der **../** in der erforderlichen Anzahl stehen. Dann wird von **stUrl_gesamt** der vordere Teil abgeschnitten, der bei den Pfaden zur Datenbankdatei und zur externen Datei gleich war. Der weg zu schneidende Inhalt wurde in **stUrl_weg** gespeichert.

Verknüpfte Bilder und Dokumente anzeigen

Verknüpfte Bilder können direkt in dem kleinen graphischen Kontrollfeld angezeigt werden. Besser wäre aber eine größere Anzeige, um auch Details erkennen zu können.

Dokumente lassen sich standardmäßig in Base überhaupt nicht anzeigen.

Um dennoch eine Anzeige zu ermöglichen, bedarf es wieder einer Makrolösung. Dieses Makro wird über einen Button in dem Formular gestartet, in dem das graphische Kontrollfeld liegt.

```
SUB Betrachten(oEvent AS OBJECT)
    DIM oDoc AS OBJECT
    DIM oForm AS OBJECT
    DIM oFeld AS OBJECT
    DIM oShell AS OBJECT
    DIM stUrl AS STRING
    DIM stFeld AS STRING
    DIM arUrl_Start()
    oDoc = thisComponent
    oForm = oEvent.Source.Model.Parent
    oFeld = oForm.getByName("GraphischesFeld")
    stUrl = oFeld.BoundField.getString
```

Das graphische Kontrollfeld im Formular wird aufgesucht. Da in der Tabelle nicht das Bild selbst, sondern nur der Pfad als Text gespeichert wird, wird hier über **getString** dieser Text ausgelesen.

Anschließend wird der Pfad zu der Datenbankdatei ermittelt. Mit **oDoc.Parent** wird die *.odb-Datei erreicht. Sie ist der Container für die Formulare. Über **oDoc.Parent.Url** wird schließlich die gesamte URL incl. Dateinamen ausgelesen. Der Dateiname ist auch zu sehen in **oDoc.Parent.Title**. Der Text wird jetzt mit der Funktion **split** aufgetrennt, wobei als Trenner der Dateiname, umgewandelt in Url-Schreibweise, benutzt wird. Die Auftrennung gibt so nur als erstes und einziges Element des Arrays den Pfad zur *.odb-Datei wieder.

```
    arUrl_Start = split(oDoc.Parent.Url, right(convertToUrl(oDoc.Parent.Title),
        len(convertToUrl(oDoc.Parent.Title))-8))
    oShell = createUnoService("com.sun.star.system.SystemShellExecute")
    stFeld = convertToUrl(arUrl_Start(0) + stUrl)
    oShell.execute(stFeld,,0)
END SUB
```

Externe Programme können über das Struct **com.sun.star.system.SystemShellExecute** gestartet werden. Dem externen Programm wird hier nur der Pfad zur Datei mitgegeben, der aus dem Pfad zur Datenbankdatei und dem intern gespeicherten relativen Pfad von der Datenbankdatei aus zusammengesetzt wurde. Die grafische Benutzeroberfläche des Betriebssystems entscheidet jetzt darüber, mit welchem Programm die entsprechende Datei zu öffnen ist.

Mit dem Kommando **oShell.execute** werden 3 Parameter übergeben. Als erstes wird eine ausführbare Datei oder der Pfad zu einer Datei aufgeführt, die im System mit einem Programm verbunden sind. Als zweites werden Parameter aufgeführt, mit denen das Programm gestartet werden soll. Als drittes wird über eine Ziffer mitgeteilt, wie mit Fehlermeldungen des Systems bei missglückter Ausführung umzugehen ist. Hier stehen 0 (Standard), 1 (keine Meldung anzeigen) und 2 (nur das Öffnen von absoluten URLs erlauben) zur Verfügung.

Dokumente in die Datenbank einlesen

Beim Einlesen der Dokumente sollten folgende Bedingungen immer im Auge behalten werden:¹⁶

- Je größer die Dokumente sind, desto schwerfälliger wird die Datenbank. Bei entsprechend großen Dokumenten, vor allem Bildern, ist deshalb eine externe Datenbank der internen Datenbank vorzuziehen.

¹⁶ Die Datenbank «Beispiel_Dokumente_einlesen_auslesen.odb» liegt diesem Handbuch bei.

- In Dokumenten kann ebenso wenig wie in Bildern recherchiert werden. Sie werden als Binärdaten gespeichert und können in einem Feld gespeichert werden, das einem Bildfeld entspricht..
- Dokumente, die in die HSQLDB-Datenbank eingelesen werden, können nur über den Makroweg auch wieder ausgelesen werden. Ein SQL-Weg ist für die interne Datenbank nicht erreichbar.

Die folgenden Makros zum Ein-und Auslesen bauen dabei auf eine Tabelle auf, die neben der Datei im Binärformat eine Beschreibung der Datei und den ursprünglichen Dateinamen enthalten soll. Schließlich wird der Dateiname ja nicht mit abgespeichert und sollte beim Auslesen der Datei aber darauf Schlüsse zulassen, um welchen Dateityp es sich denn handelt. Nur dann kann die Datei auch zum Lesen durch andere Programme einwandfrei erkannt werden.

Die Tabelle enthält die folgenden Felder:

Feldname	Feldtyp	Beschreibung
ID	Integer	Die ID ist Primärschlüssel dieser Tabelle.
Beschreibung	Text	Beschreibung des Dokumentes, Suchbegriffe ...
Datei	Bild	Nimmt den binären Inhalt des Bildes oder der Datei auf.
Dateiname	Text	Soll die Bezeichnung der Datei mit Dateiendung abspeichern. Wichtig für das spätere Auslesen.

Das Formular zum Einlesen und wieder Ausgeben der Dateien sieht so aus:

ID

Beschreibung

Bild oder Datei 

Dateiname

Datensatz von 4

Navigation icons: back, forward, search, etc.

Solange sich Bilddateien in der Datenbank befinden, können diese Dateien auch in dem graphischen Kontrollfeld angesehen werden. Alle anderen Dateien werden in dem Kontrollfeld nicht sichtbar.

Das folgende Makro für das Einlesen der Dateien wird über **Eigenschaften: Dateiauswahl → Ereignisse → Text modifiziert** ausgelöst.

```
SUB DateiEinlesen_mitName(oEvent AS OBJECT)
    DIM oForm AS OBJECT
    DIM oFeld AS OBJECT
    DIM oFeld2 AS OBJECT
    DIM oFeld3 AS OBJECT
    DIM oStream AS OBJECT
    DIM oSimpleFileAccess AS OBJECT
    DIM stUrl AS STRING
```



```

DIM stName AS STRING
oFeld = oEvent.Source.Model
oForm = oFeld.Parent
oFeld2 = oForm.getByName("DateiName")
oFeld3 = oForm.getByName("GraphischesFeld")
IF oFeld.Text <> "" THEN
    stUrl = ConvertToUrl(oFeld.Text)
    ar = split(stUrl, "/")
    stName = ar(UBound(ar))
    oFeld2.BoundField.updateString(stName)
    oSimpleFileAccess = createUnoService("com.sun.star.ucb.SimpleFileAccess")
    oStream = oSimpleFileAccess.openFileRead(stUrl)
    oFeld3.BoundField.updateBinaryStream(oStream, oStream.getLength())
END IF
END SUB

```

Da das Makro über das auslösende Ereignis die Position der anderen Formularfelder ermittelt, muss nicht besonders überprüft werden, ob die Felder nun in einem Formular oder Unterformular liegen. Alle Felder müssen lediglich im gleichen Formular positioniert sein.

Das Feld «DateiName» speichert den Namen der Datei ab, die ausgesucht wird. Bei Bildern muss dieser Name ohne ein zusätzliches Makro händisch eingegeben werden. Hier wird stattdessen der Dateiname aus der URL ermittelt und automatisch beim Einlesen der Datei mit eingefügt.

Das Feld «GraphischesFeld» speichert die Daten in dem gemeinsamen Feld für Bilder und Dateien ab.

Aus dem Dateiauswahlfeld wird über **oFeld.Text** der Pfad komplett mit Dateinamen ausgelesen. Damit die URL-Schreibweise nicht an systemspezifischen Bedingungen scheitert, wird der ausgelesene Text mit **ConvertToUrl** in eine allgemeingültige URL umgewandelt. Die so erstellte allgemeingültige URL wird in ein Array aufgeteilt. Der Trenner ist das /. Letztes Element dieser Pfadangabe ist der Dateiname. **Ubound(ar)** gibt die Nummer für das letzte Element an. Daher kann der Dateiname über **ar(Ubound(ar))** direkt ausgelesen und anschließend als String an das Feld übergeben werden.

Um die Datei selbst einzulesen, muss der **UnoService com.sun.star.ucb.SimpleFileAccess** bemüht werden. Über diesen Service kann der Inhalt der Datei als Datenstrom ausgelesen werden. Anschließend wird der so in dem Objekt **oStream** zwischengespeicherte Inhalt wiederum als Datenstrom in das Feld eingefügt, das mit dem Feld "Datei" der Tabelle verbunden ist. Dabei muss neben dem Objekt **oStream** auch die Länge des Datenstromes als Parameter angegeben werden.

Die Daten sind jetzt wie eine normale Eingabe in die Formularfelder eingefügt. Wird das Formular einfach geschlossen, so sind die Daten noch nicht abgespeichert. Die Speicherung erfolgt erst bei Betätigung des Speicherbuttons in der Navigationsleiste oder automatisch bei der Navigation zum nächsten Datensatz.

Bildnamen ermitteln

Bei dem obigen Verfahren wurde kurz erwähnt, dass der Name der Datei bei der Eingabe über das graphische Kontrollfeld so nicht ermittelt werden kann. Hier jetzt kurz ein Makro zur Ermittlung des Dateinamens, das zum obigen Formular passt. Der Dateiname lässt sich nicht sicher durch ein Ereignis ermitteln, das mit dem grafischen Kontrollfeld direkt verbunden ist. Deswegen wird das Makro über **Formular-Eigenschaften → Ereignisse → Vor der Datensatzaktion** gestartet.

```

SUB BildnamenAuslesen(oEvent AS OBJECT)
oForm = oEvent.Source
IF InStr(oForm.ImplementationName, "ODatabaseForm") THEN
    oFeld = oForm.getByName("GraphischesFeld")
    oFeld2 = oForm.getByName("DateiName")
    IF oFeld.ImageUrl <> "" THEN
        stUrl = ConvertToUrl(oFeld.ImageUrl)
        ar = split(stUrl, "/")
    END IF
END IF
END SUB

```

```

        stName = ar(UBound(ar))
        oFeld2.BoundField.updateString(stName)
    END IF
END IF
END SUB

```

Vor der Datensatzaktion werden zwei Implementationen mit unterschiedlichem Implementationsnamen ausgeführt. Das Formular ist am einfachsten über die Implementation erreichbar, das in seinem Namen als **ODatabaseForm** bezeichnet wird.

In dem graphischen Kontrollfeld ist die URL der Datenquelle über die **ImageUrl** erreichbar. Diese URL wird ausgelesen, der Dateinamen wie in der vorhergehenden Prozedur «DateiEinlesen_mit-Name» ausgelesen und in das Feld «DateiName» übertragen.

Bildnamen aus dem Speicher entfernen

Wird nach dem Ablauf des obigen Makros zum nächsten Datensatz gewechselt, so ist der Pfad zu dem ursprünglichen Bild weiter vorhanden. Würde jetzt eine allgemeine Datei über das Dateiauswahlfeld eingelesen, so würde der Name der Datei ohne das folgende Makro einfach durch den Namen der zuletzt eingelesenen Bilddatei überschrieben.

Der Pfad kann leider nicht mit dem vorhergehenden Makro entfernt werden, da das Einlesen der Bilddatei erst beim Abspeicherungsvorgang erfolgt. Eine Entfernung des Pfades vor der Abspeicherung löscht das Bild.

Das Makro wird über **Formular-Eigenschaften → Ereignisse → Nach der Datensatzaktion** gestartet.

```

SUB BildnamenZuruecksetzen(oEvent AS OBJECT)
    oForm = oEvent.Source
    IF InStr(oForm.ImplementationName, "ODatabaseForm") THEN
        oFeld = oForm.getByName("GraphischesFeld")
        IF oFeld.ImageUrl <> "" THEN
            oFeld.ImageUrl = ""
        END IF
    END IF
END SUB

```

Es wird, wie in der Prozedur «BildnamenAuslesen», auf das graphische Kontrollfeld zugegriffen. Existiert dort noch ein Eintrag in der **ImageUrl**, dann wird dieser geleert.

Bilder und Dokumente auslesen und anzeigen

Für Dateien wie für die Bilder in Originalgröße gilt, dass der Button **Datei mit externem Programm betrachten** ausgelöst werden muss. Dann werden die Dateien in das temporäre Verzeichnis ausgelesen und anschließend über das mit der Endung verknüpfte Programm des Betriebssystems angezeigt.

Das Makro wird über **Eigenschaften: Schaltfläche → Ereignisse → Aktion ausführen** gestartet.

```

SUB DateiAuslesen_mitName(oEvent AS OBJECT)
    DIM oDoc AS OBJECT
    DIM oDrawpage AS OBJECT
    DIM oForm AS OBJECT
    DIM oFeld AS OBJECT
    DIM oFeld2 AS OBJECT
    DIM oStream AS OBJECT
    DIM oShell AS OBJECT
    DIM oPath AS OBJECT
    DIM oSimpleFileAccess AS OBJECT
    DIM stName AS STRING
    DIM stPfad AS STRING
    DIM stFeld AS STRING
    oForm = oEvent.Source.Model.Parent
    oFeld = oForm.getByName("GraphischesFeld")
    oFeld2 = oForm.getByName("DateiName")

```



```

stName = oFeld2.Text
IF stName = "" THEN
    stName = "Datei"
END IF
oStream = oFeld.BindField.getBinaryStream
oPath = createUnoService("com.sun.star.util.PathSettings")
stPfad = oPath.Temp & "/" & stName
oSimpleFileAccess = createUnoService("com.sun.star.ucb.SimpleFileAccess")
oSimpleFileAccess.writeFile(stPfad, oStream)
oShell = createUnoService("com.sun.star.system.SystemShellExecute")
stFeld = convertToUrl(stPfad)
oShell.execute(stFeld,,0)
END SUB

```

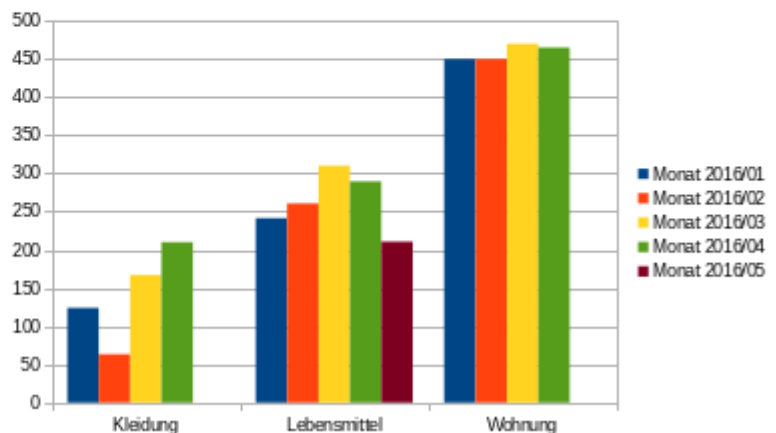
Die Lage der anderen betroffenen Felder im Formular wird abhängig von dem auslösenden Button ermittelt. Fehlt ein Dateiname, so wird der ausgelesenen Datei einfach der Name «Datei» zugeschrieben.

Der Inhalt des Formularfeldes «GraphischesFeld» entspricht dem Inhalt, der in dem Feld "Datei" der Tabelle liegt. Er wird als Datenstrom ausgelesen. Als Pfad für die ausgelesene Datei wird der Pfad zum temporären Verzeichnis genutzt, der in LibreOffice über **Extras → Optionen → LibreOffice → Pfade** eingestellt werden kann. Soll also die Datei anschließend noch anderweitig verwendet und nicht nur angezeigt werden, so ist sie aus diesem Pfad heraus auch kopierbar. Innerhalb des Makros wird die Datei direkt nach dem erfolgreichen Auslesen mit dem Programm geöffnet, das in der grafischen Benutzeroberfläche des Betriebssystems mit dem Dateityp verbunden ist.

Diagramme in Formulare einbinden

Das Einbinden von Diagrammen in Formulare ist von der GUI her nicht vorgesehen. Mit einem Umweg über Writer, etwas Makrohilfe und allgemein verarbeitbaren Ansichten lässt sich so ein Diagramm dennoch in ein Formular integrieren und darüber hinaus beständig an die momentane Datenlage anpassen.

ID	Kategorie	Datum	Betrag
1	Lebensmittel	15.01.16	241,51 €
2	Lebensmittel	15.02.16	260,75 €
3	Kleidung	15.01.16	124,00 €
4	Wohnung	15.01.16	450,00 €
5	Kleidung	15.02.16	63,20 €
6	Wohnung	15.02.16	450,00 €
7	Lebensmittel	15.03.16	310,23 €
8	Kleidung	15.03.16	167,00 €
9	Wohnung	15.03.16	470,00 €
10	Lebensmittel	15.04.16	289,50 €
11	Kleidung	15.04.16	210,00 €
12	Wohnung	15.04.16	465,00 €
13	Lebensmittel	15.05.16	210,76 €



Als Datenbank für die Diagrammdarstellung wurden beispielhaft Daten aus einer Haushaltsführung und Daten aus einer Temperaturmessung über einen kürzeren Zeitraum verwandt.¹⁷ Während sich die Haushaltsführung für verschiedene Diagrammtypen eignet, ist die Messung eines Temperaturverlaufes auf ein XY-Diagramm zugeschnitten.

Diagramme aus dem Writer importieren

Zuerst wird ein neues Writer-Dokument erstellt: **Datei → Neu → Textdokument**. Anschließend wird über die Symbolleiste oder über **Einfügen → Objekt → Diagramm** ein Diagramm erstellt. Standardmäßig wird hier zuerst einmal ein Säulendiagramm in das Dokument eingefügt.

Das Diagramm wird kopiert und in ein Formular der Datenbank eingefügt. Alle weiteren Einstellungen des Diagramms können auch im Base-Formular über das Kontextmenü vorgenommen werden.

Daten für das Diagramm werden später automatisch eingelesen. Es ist also nicht notwendig, hier irgendeine Voreinstellung vorzunehmen.

Abfragen erstellen und als Ansichten speichern

Für die Darstellung der verschiedenen Diagrammtypen werden Zeilenbeschreibungen (**RowDescriptions**), Spaltenbeschreibungen (**ColumnDescriptions**) und Daten (**Data**) benötigt. Um Probleme bei der Darstellung von XY-Diagrammen zu umgehen, wird zusätzlich eine Information zum Diagrammtyp in den Abfragen dargestellt.

Abfrage für ein Säulendiagramm

Würde von vornherein sichergestellt, dass nur ein Eintrag pro Monat erstellt würde und das Diagramm nur nach allen Einträgen für den Monat aktualisiert werden soll, so könnte für ein **Säulendiagramm** einfach die Tabelle direkt abgefragt werden:

```
SELECT "Kategorie", "Datum", "Betrag", 'BC' AS "Type" FROM "Kategorien"
```

Die Kategorien stellen die Zeilenbeschriftungen dar. Die Zeilenbeschriftungen erscheinen unter den Säulen des Diagramms. Für jeden Monat wird eine neue Säule dargestellt. Die Monatsbezeichnungen erscheinen beim Säulendiagramm als Spaltenbeschriftungen rechts vom Diagramm in der Legende. Der Betrag gibt als Dezimalzahl die Höhe der jeweiligen Säule an. Die Information zum Diagrammtypen wird später nur für das XY-Diagramm separat ausgewertet. 'BC' steht hier schlicht für «bar chart» (Säulendiagramm).

Damit auch mehrere Einträge pro Monat angefertigt werden können, sind die Einträge für alle gleichen Kategorien innerhalb eines Monats zusammen zu fassen. Die Darstellung des Monats soll in dem Diagramm z.B. als «Monat 2016/02» erfolgen. Außerdem sollen die Einträge in dem Diagramm auch dann aktualisiert werden, wenn für eine Kategorie eventuell noch gar kein Monatseintrag existiert.

Um all dies zu bewerkstelligen, wird zuerst einmal eine Kombination aller Kategorien mit allen in der Tabelle "Kategorien" vorhandenen Monaten erstellt:

```
SELECT
    "a"."Kategorie",
    "b"."Monat"
FROM "Kategorien" AS "a",
    ( SELECT 'Monat ' || YEAR( "Datum" ) || '/' || RIGHT( '0'
        || MONTH( "Datum" ), 2 ) AS "Monat" FROM "Kategorien" ) AS "b"
ORDER BY "Kategorie", "Monat"
```

¹⁷ Die Datenbank «Beispiel_Baseformular_mit_Diagramm» ist den Beispieldatenbanken für dieses Handbuch beigelegt.

Diese Konstruktion kann allerdings den Datenbestand deutlich aufblähen. Hat die Tabelle "Kategorien" 5 Einträge, so werden bei dieser Abfrage 5 Einträge zu "Kategorie" mit 5 Einträgen zu der Monatsdarstellung kombiniert. Es entstehen zwangsläufig 25 Einträge, von denen dann gewiss viele Einträge mehrfach vorkommen.

Hinweis

Für **FIREBIRD** muss statt **MONTH("Datum")** **EXTRACT(MONTH FROM "Datum")** usw. eingefügt werden. Firebird kennt die hier verwendeten Kurzformen nicht.

Mit Hilfe des Zusatzes **DISTINCT** kann die Ausgabe der Einträge eingeschränkt werden. Dies sollte möglichst schon bei den Datenquellen für die Abfrage ansetzen, damit nicht unnötig erst Daten geladen und wieder aus dem Speicher entfernt werden. Deshalb wird auf die Tabelle "Kategorien" zweimal mit einer Abfrage und dem Zusatz **DISTINCT** zugegriffen:

```
SELECT
    "a"."Kategorie",
    "b"."Monat"
FROM ( SELECT DISTINCT "Kategorie" FROM "Kategorien" ) AS "a",
    ( SELECT DISTINCT 'Monat ' || YEAR( "Datum" ) || '/' || RIGHT( '0'
        || MONTH( "Datum" ), 2 ) AS "Monat" FROM "Kategorien" ) AS "b"
ORDER BY "Kategorie", "Monat"
```

Mit Hilfe einer korrelierenden Unterabfrage wird jetzt zu jeder Kombination von "Monat" und "Kategorie" der entsprechende "Betrag" aufsummiert. So werden auch mehrere Einträge im Monat bei der Darstellung im Diagramm berücksichtigt:

```
SELECT
    "a"."Kategorie" AS "RowDescription",
    "b"."Monat" AS "ColumnDescription",
    ( SELECT SUM( "Betrag" ) FROM "Kategorien" WHERE 'Monat ' ||
        YEAR( "Datum" ) || '/' || RIGHT( '0' || MONTH( "Datum" ), 2 ) =
        "b"."Monat" AND "Kategorie" = "a"."Kategorie" ) AS "Data",
    'BC' AS "Type"
FROM ( SELECT DISTINCT "Kategorie" FROM "Kategorien" ) AS "a",
    ( SELECT DISTINCT 'Monat ' || YEAR( "Datum" ) || '/' || RIGHT( '0'
        || MONTH( "Datum" ), 2 ) AS "Monat" FROM "Kategorien" ) AS "b"
ORDER BY "Kategorie", "Monat"
```

Diese Abfrage wird abgespeichert, mit der rechten Maustaste angeklickt und über **Abfrage → Als Ansicht erstellen** unter der Bezeichnung «Chart_Kategorie» als Ansicht gespeichert.

Abfrage für ein Kreisdiagramm

Für die Darstellung eines **Kreisdiagramms** ist es erforderlich, dass statt der einzelnen Monatsdarstellungen die Summe über alle Monate erstellt wird. Hier wird der Einfachheit halber direkt auf die eben erstellte Ansicht zugegriffen:

```
SELECT
    "RowDescription",
    '' AS "ColumnDescription",
    SUM( "Data" ) AS "Data",
    'CC' AS "Type"
FROM "Chart_Kategorie"
GROUP BY "RowDescription"
```

Nach der "Kategorie", hier unter dem Alias "RowDescription", wird die Abfrage gruppiert. Für das Feld "Data" wird die Summe des Feldes "Data" aus der Ansicht "Chart_Kategorie", bezogen auf diese Gruppierung, erstellt.

Eine Spaltenbeschreibung ist nicht erforderlich und wird mit einem leeren Text versehen. Als Typ des Diagramms wird 'CC' für «circle chart» (Kreisdiaagramm) angegeben.

Die Abfrage wird abgespeichert, mit der rechten Maustaste angeklickt und über **Abfrage → Als Ansicht erstellen** unter der Bezeichnung «Chart_Kategorie_Circle» als Ansicht gespeichert.

Abfrage für ein XY-Diagramm

Daten für ein **XY-Diagramm** liegen in der Regel in anderer Weise vor als für Säulendiagramme oder Kreisdiaagramme. Statt einen Wert in einer Zeile zu speichern, werden hier Wertepaare gespeichert. Damit dennoch auf die Daten mit der gleichen Makrokonstruktion zugegriffen werden kann, ist eine besondere Abfragekonstruktion vorgesehen, die wieder in der ersten Spalte die Zeilenbeschreibung, in der zweiten Spalte die Spaltenbeschreibung, in der dritten Spalte die Daten und in der vierten Spalte den Diagrammtyp wiedergeben kann.

```
SELECT
    "Zeit" AS "RowDescription",
    '' AS "ColumnDescription",
    "Zeit" AS "Data",
    'XY' AS "Type"
FROM "Temperaturverlauf"
UNION
SELECT
    "Zeit" AS "RowDescription",
    'Temperatur [°C]' AS "ColumnDescription",
    "Temperatur" AS "Data",
    'XY' AS "Type"
FROM "Temperaturverlauf"
ORDER BY "RowDescription" ASC, "ColumnDescription" ASC
```

Zuerst werden alle Zeiten ausgelesen. Das Ergebnis dieser Abfrage wird mit der Folgeabfrage über **UNION** kombiniert, die jetzt alle Temperaturdaten ausliest. Leider funktioniert diese Kombination nicht wie gewünscht, da die Zeit einen anderen Datentypen hat als die Temperatur. Es kommt noch hinzu, dass mit dem Datentypen für die Zeit leider auch keine kontinuierliche Darstellung in einem XY-Diagramm möglich ist. Aus der Zeit muss durch Umformung ein Dezimalwert erstellt werden. Dies erfolgt, indem der Tag als Grundmaß angesehen wird. Stunden, Minuten und Sekunden werden als Bruchteile des Tages errechnet und addiert.

```
SELECT
    HOUR( "Zeit" ) / 24.00000 + MINUTE( "Zeit" ) / 1440.00000 +
    SECOND( "Zeit" ) / 86400.00000 AS "RowDescription",
    '' AS "ColumnDescription",
    HOUR( "Zeit" ) / 24.00000 + MINUTE( "Zeit" ) / 1440.00000 +
    SECOND( "Zeit" ) / 86400.00000 AS "Data", 'XY' AS "Type"
FROM "Temperaturverlauf"
UNION
SELECT
    HOUR( "Zeit" ) / 24.00000 + MINUTE( "Zeit" ) / 1440.00000 +
    SECOND( "Zeit" ) / 86400.00000 AS "RowDescription",
    'Temperatur [°C]' AS "ColumnDescription",
    "Temperatur" AS "Data",
    'XY' AS "Type" FROM "Temperaturverlauf"
ORDER BY "RowDescription" ASC, "ColumnDescription" ASC
```

Hinweis

Für **FIREBIRD** muss statt **HOUR("Zeit") / 24.00000 + MINUTE("Zeit") / 1440.00000 + SECOND("Zeit") / 86400.00000** einfach **("Zeit" - TIME '00:00') / 86400.00000** eingefügt werden. Firebird kennt die hier verwendeten Kurzformen nicht, kann aber Datumswerte und Zeitwerte voneinander subtrahieren.

Die Zeilenbeschriftung "RowDescription" wird für das XY-Diagramm gar nicht benötigt. Hier stehen schließlich die Daten der X-Achse. Diese Spalte wird über die zu einer Dezimalzahl umformatierten Inhalte dazu genutzt, die Zeilen in der korrekten Reihenfolge zu sortieren.

Das Diagramm stellt lediglich die Werte der Y-Achse als 'Temperatur [°C]' dar. Deswegen wird für die Zeilen, die jetzt die Zeitangaben als Daten wiedergeben, eine leere Spaltenbeschriftung "ColumnDescription" ausgegeben.

Der gesamte Inhalt wird nach den Zeiten in "RowDescription" und den Einträgen in "ColumnDescription" sortiert, so dass die Werte für X- und Y-Achse direkt aufeinander folgen und durch gleiche "RowDescription"-Werte einander zugeordnet werden können.

Diagramme über Makros anpassen

Um den Makros mitteilen zu können, welche Ansicht in dem jeweiligen Diagramm dargestellt werden soll, wird in den Formularen ein «verstecktes Steuerelement» eingebaut. Über den Formularnavigator wird das Formular, das als Auslöser des Diagramms genutzt werden soll, mit der rechten Maustaste angeklickt. Im Kontextmenü wird **Neu → Verstecktes Steuerelement** aufgerufen.

Das Element wird über das Makro mit dem Namen 'Chart' gesucht. Entsprechend wird das versteckte Steuerelement erst einmal umbenannt: **rechte Maustaste → Kontextmenü → Umbenennen**. Anschließend wird in dem Steuerelement über **rechte Maustaste → Kontextmenü → Eigenschaften → Allgemein → Zusatzinformation** der Name für die Ansicht vermerkt.

Jetzt ist nur noch notwendig, die folgenden beiden Makros mit den Ereignissen des Formulars «Beim Laden» (Prozedur: **ActivateDiagram**) bzw. «Nach der Datensatzaktion» (Prozedur: **ChangeData**) zu verknüpfen.

Damit ein Diagramm im Formular kontinuierlich geändert werden kann, muss es zuerst einmal aktiviert werden. Das Diagramm selbst ist dem Formular sonst völlig unbekannt:

```
SUB ActivateDiagram(oEvent AS OBJECT)
    DIM oDiag AS OBJECT
    DIM oXC0E0 AS OBJECT
    oDiag = thisComponent.embeddedObjects.getByIndex(0)
    oXC0E0 = oDiag.ExtendedControlOverEmbeddedObject
    oXC0E0.changeState(4)
    oXC0E0.changeState(0)
    ChangeData(oEvent)
END SUB
```

Es wird hier angenommen, dass lediglich das Diagramm in die Oberfläche des Dokumentes eingefügt wurde. Das Diagramm hat daher die Indexnummer '0', auf die direkt Bezug genommen wird. Anschließend wird der Controller für das eingebettete Diagramm angesprochen. Tests haben hier ergeben, dass zuerst mit '4' die UI aktiviert werden und anschließend mit '0' das Objekt geladen werden muss. Die möglichen Parameter sind unter **com.sun.star.embed.EmbedStates**: LOADED = 0, RUNNING = 1, ACTIVE = 2, INPLACE_ACTIVE = 3, UI_ACTIVE = 4.

Anschließend wird die Prozedur gestartet, die dazu dienen soll, die Daten aus dem entsprechenden **View** zu laden. Damit auf das im Formular befindliche versteckte Steuerelement zugegriffen werden kann, wird das auslösende Ereignis über die Prozedur **ActivateDiagram** an die Prozedur **ChangeData** weitergeleitet.

```
SUB ChangeData(oEvent AS OBJECT)
    DIM oDiag AS OBJECT
```

```

DIM oDatasource AS OBJECT
DIM oConnection AS OBJECT
DIM oSQL_Command AS OBJECT
DIM oResult AS OBJECT
DIM oForm AS OBJECT
DIM oHiddenControl AS OBJECT
DIM stSql AS STRING
DIM stView AS STRING
DIM stRow AS STRING
DIM stType AS STRING
DIM i AS INTEGER
DIM k AS INTEGER
DIM x AS INTEGER
DIM aNewData(0)
DIM aNewRowDescription(0)
DIM aTmp() AS DOUBLE
DIM aNewColumnDescription()
DIM aType()
oForm = oEvent.Source
oHiddenControl = oForm.getByName("Chart")
stView = oHiddenControl.Tag
stSql = "SELECT * FROM ""+stView+""
oDatasource = ThisComponent.Parent.CurrentController
IF NOT (oDatasource.isConnected()) THEN
    oDatasource.connect()
END IF
oConnection = oDatasource.ActiveConnection()
oSQL_Command = oConnection.createStatement()
oResult = oSQL_Command.executeQuery(stSql)
i = 0
k = 0
x = 0
WHILE oResult.next
    stRow = oResult.getString(1)
    stType = oResult.getString(4)
    IF aNewRowDescription(i) = stRow THEN
        ReDim Preserve aNewColumnDescription(k)
        ReDim Preserve aTmp(k)
    ELSE
        IF x > 0 THEN
            i = i + 1
        ELSE
            x = 1
        END IF
        ReDim Preserve aNewRowDescription(i)
        ReDim Preserve aNewData(i)
        k = 0
        ReDim Preserve aNewColumnDescription(k)
        ReDim aTmp(k)
    END IF
    aNewRowDescription(i) = stRow
    aNewColumnDescription(k) = oResult.getString(2)
    aTmp(k) = oResult.getDouble(3)
    aNewData(i) = aTmp()
    k = k + 1
WEND
oDiag = thisComponent.EmbeddedObjects.getByIndex(0)
IF stType <> "XY" THEN
    oDiag.model.Data.setData(aNewData)
END IF
oDiag.model.DataProvider.setData(aNewData)
oDiag.model.DataProvider.setRowDescriptions(aNewRowDescription)
oDiag.model.DataProvider.setColumnDescriptions(aNewColumnDescription)
oDiag.Component.setmodified(true)
END SUB

```

Nach der Deklaration der Variablen wird zuerst aus den Zusatzinformationen (**Tag**) des versteckten Kontrollfeldes der Name für die Ansicht ausgelesen. Der erforderliche SQL-Code zum Ausle-

sen der gesamten Ansicht wird formuliert, die Verbindung zur Datenbank, falls erforderlich, hergestellt und der SQL-Code an die Datenbank weitergeleitet. In **oResult** wird das Ergebnis der Abfrage gespeichert.

Vor dem Start der Schleife zur Ermittlung des Inhaltes aus **oResult** werden noch einige Zahlenvariablen auf '0' gesetzt, die im Laufe der Schleife verändert werden sollen.

Innerhalb der Schleife werden die Inhalte der verschiedenen Spalten der Ansicht ausgelesen. Die Inhalte werden in unterschiedliche Arrays zur Weitergabe an den DataProvider abgespeichert. Hier werden wieder Daten (**Data**), Zeilenbeschriftungen (**RowDescriptions**) und Spaltenbeschriftungen (**ColumnDescriptions**) unterschieden.

Mit **oResult.getString(1)** wird das Ergebnis zum jeweiligen Datensatz aus der ersten Spalte als Text ausgelesen. Hier handelt es sich um Zeilenbeschriftungen, die eben einfach Text sind. Solange die Abfrage nacheinander gleiche Zeilenbeschriftungen liefert, werden alle weiteren Inhalte dem gleichen Datenbestand zugeordnet. Um dies zu gewährleisten, erfolgt zuerst eine Abfrage, ob der entsprechende Eintrag von **stRow** bereits als letzter Eintrag von **aNewRowDescriptions()** vorhanden ist.

Ist dies nicht der Fall, wie z.B. direkt beim Einlesen des ersten Datensatzes, dann erfolgt das Vorgehen, das unter **ELSE** beschrieben ist. Nur wenn der Zähler **x** größer als '0' ist, wird der Zähler für **i** heraufgesetzt. Dies soll vermeiden, dass der erste Eintrag des zu erzeugenden Arrays später leer ist. Für alle späteren Eintritte in diese Schleife wird allerdings dann **x** auf '1' gesetzt, so dass **i** heraufgesetzt wird und die Arrays mit einem weiteren Datensatz beschrieben werden können.

Redim Preserve sichert den bisherigen Inhalt eines Arrays und eröffnet gleichzeitig die Möglichkeit, einen zusätzlichen Eintrag ans Ende des Arrays anzufügen.

aNewData und **aNewRowDescriptions** werden immer zusammen abgespeichert. Deswegen haben die Arrays den gemeinsamen Zähler **i**. **aNewColumnDescriptions** sowie die in einem temporären Array **aTmp** gespeicherten Dateninhalte werden bei jedem neuen Durchgang durch die **WHILE-NEXT**-Schleife mit einem zusätzlichen Datensatz versehen. Auch sie haben deshalb einen gemeinsamen Zähler, hier **k**. Dieser Zähler wird bei jeder Änderung von **aNewRowDescriptions** neu mit '0' gestartet. Dabei wird das temporäre Array nicht gesichert, da es zwischenzeitig in dem Array **aNewData** abgespeichert wurde.

Die Inhalte für das Array **aTmp** werden immer als Dezimalzahlen aus der Abfrage ausgelesen. Daher der Eintrag **oResult.getDouble(3)**.

Nachdem alle Daten ausgelesen wurden, werden diese in den **DataProvider** übertragen. Mit **oDiag.model.Data.setData(aNewData)** werden nicht nur die Daten neu geschrieben, sondern z.B. auch die Anzahl der Säulen in einem Spaltendiagramm aktualisiert. Dieser Eintrag führt allerdings bei einem XY-Diagramm dazu, dass hier die Grundstruktur des Diagramms zerstört und der erste Dateneintrag nicht als X-Achsenwert gesehen wird. Deshalb ist dieser Eintrag für XY-Diagramme ausgeschlossen.

Die gesamte Ansprache des Diagramms wurde hier nur nach dem Prinzip «Versuch und Irrtum» ermittelt. Die Aktualisierung funktioniert, kann aber sicher noch verbessert werden.

Codeschnipsel

Die Codeschnipsel erwachsen aus Anfragen innerhalb von Mailinglisten. Bestimmte Problemstellungen tauchen dort auf, die vielleicht gut als Lösungen innerhalb der eigenen Datenbankentwürfe genutzt werden können.

Aktuelles Alter ermitteln

Aus einem Datum soll mittels Abfrage das aktuelle Alter ermittelt werden. Siehe hierzu die Funktionen im Anhang zu diesem Base-Handbuch.

```
SELECT DATEDIFF('yy','Geburtsdatum',CURDATE()) AS "Alter" FROM "Person"
```

Hinweis

Für **FIREBIRD** muss der Code entsprechend angepasst werden. **CURDATE()** als Kurzform ist hier unbekannt. Es muss schon **CURRENT_DATE** sein. Und statt **'yy'** muss an dieser Stelle ein String ohne Hochkommata angegeben werden: **year**. Auch die Kurzform **DAYOFYEAR** kennt Firebird nicht. Hier muss wieder **EXTRACT(YEARDAY FROM "Geburtsdatum")** usw. geschrieben werden.

Die Abfrage gibt das Alter als Jahresdifferenz aus. Das Alter eines Kindes, das am 31.12.2011 geboren ist, wird am 1.1.2012 mit 1 Jahr angegeben. Es muss also die Lage des Tages im Jahr berücksichtigt werden. Dies ist mit der Funktion **'DAYOFYEAR()'** ermittelbar. Mittels einer Funktion wird der Vergleich durchgeführt.

```
SELECT CASEWHEN
( DAYOFYEAR("Geburtsdatum") > DAYOFYEAR(CURDATE()) ,
DATEDIFF ('yy','Geburtsdatum',CURDATE())-1,
DATEDIFF ('yy','Geburtsdatum',CURDATE()))
AS "Alter" FROM "Person"
```

Jetzt wird das aktuelle Alter in Jahren ausgegeben.

Über **'CASEWHEN'** könnte dann auch in einem weiteren Feld der Text **'Heute Geburtstag'** ausgegeben werden, wenn **DAYOFYEAR("Geburtsdatum") = DAYOFYEAR(CURDATE())**.

Spitzfindig könnte jetzt der Einwand kommen: «Wie steht es mit Schaltjahren?». Für Personen, die nach dem 28. Februar geboren wurden, kann es zu Abweichungen um einen Tag kommen. Für den Hausgebrauch nicht weiter schlimm, aber wo bleibt der Ehrgeiz, es möglichst doch genau zu machen?

Mit

```
CASEWHEN (
(MONTH("Geburtsdatum") > MONTH(CURDATE())) OR
((MONTH("Geburtsdatum") = MONTH(CURDATE())) AND (DAY("Geburtsdatum") >
DAY(CURDATE())))) ,
DATEDIFF('yy','Geburtsdatum',CURDATE())-1,
DATEDIFF('yy','Geburtsdatum',CURDATE()))
```

wird das Ziel erreicht. Solange der Monat des Geburtsdatum größer ist als der aktuelle Monat wird auf jeden Fall von der Jahresdifferenz 1 Jahr abgezogen. Ebenfalls 1 Jahr abgezogen wird, wenn zwar der Monat gleich ist, der Tag im Monat des Geburtsdatums aber größer ist als der Tag im aktuellen Monat. Leider ist diese Eingabe für die GUI nicht verständlich. Erst **'SQL-Kommando direkt ausführen'** lässt die Abfrage erfolgreich absetzen. So ist also unsere Abfrage nicht mehr editierbar. Die Abfrage soll aber weiter editierbar sein; also gilt es die GUI zu überlisten:

```
CASE
WHEN MONTH("Geburtsdatum") > MONTH(CURDATE())
THEN DATEDIFF('yy','Geburtsdatum',CURDATE())-1
WHEN (MONTH("Geburtsdatum") = MONTH(CURDATE()) AND DAY("Geburtsdatum")
> DAY(CURDATE()))
THEN DATEDIFF('yy','Geburtsdatum',CURDATE())-1
ELSE DATEDIFF('yy','Geburtsdatum',CURDATE())
END
```


Auf diese Formulierung reagiert die GUI nicht mit einer Fehlermeldung. Das Alter wird jetzt auch in Schaltjahren genau ausgegeben und die Abfrage bleibt editierbar.

Geburtstage in den nächsten Tagen anzeigen

Mit Hilfe von einigen kleinen Berechnungskniffen lässt sich auch aus einer Tabelle z.B. auslesen, wer in den nächsten 8 Tagen Geburtstag hat.

```
SELECT *
FROM "Tabelle"
WHERE
    DAYOFYEAR("Datum") BETWEEN DAYOFYEAR(CURDATE()) AND
        DAYOFYEAR(CURDATE()) + 7
    OR DAYOFYEAR("Datum") < 7 -
        DAYOFYEAR(CAST(YEAR(CURDATE()) || '-12-31' AS DATE)) +
        DAYOFYEAR(CURDATE())
```

Die Abfrage zeigt alle Datensätze an, deren Datumseintrag zwischen dem momentanen Jahrestag und den folgenden 7 Tagen liegt.

Damit auch am Jahresende entsprechend 8 Tage angezeigt werden müssen die Anfangstage des Jahres grundsätzlich auch überprüft werden. Diese Überprüfung findet aber nur für die Jahrestage statt, die höchstens den Wert 7 abzüglich des Jahrestages des letzten Datumswertes des aktuellen Jahres (meist 365) zuzüglich des Jahrestages des aktuellen Datumswertes. Liegt das aktuelle Datum mehr als 7 Tage vom Jahresende entfernt, so ist der Gesamtwert also < 1. Kein Eintrag in der Tabelle hat so ein Datum. Diese Teilbedingung wird dann nicht erfüllt.

In der oben stehenden Formulierung werden bei Schaltjahren noch Unstimmigkeiten hervortreten, da sich dann ab dem 29.2. die Jahrestagzählung verschiebt. Der Code, um dieses zu berücksichtigen, gestaltet sich um einiges umfangreicher:

```
SELECT *
FROM "Tabelle"
WHERE
    CASE
        WHEN
            DAYOFYEAR(CAST(YEAR("Datum") || '-12-31' AS DATE)) = 366
            AND DAYOFYEAR("Datum") > 60 THEN DAYOFYEAR("Datum") - 1
        ELSE
            DAYOFYEAR("Datum")
    END
    BETWEEN
        CASE
            WHEN
                DAYOFYEAR(CAST(YEAR(CURDATE()) || '-12-31' AS DATE)) = 366
                AND DAYOFYEAR(CURDATE()) > 60 THEN DAYOFYEAR(CURDATE()) - 1
            ELSE
                DAYOFYEAR(CURDATE())
        END
    AND
        CASE
            WHEN
                DAYOFYEAR(CAST(YEAR(CURDATE()) || '-12-31' AS DATE)) = 366
                AND DAYOFYEAR(CURDATE()) > 60 THEN DAYOFYEAR(CURDATE()) + 6
            ELSE
                DAYOFYEAR(CURDATE()) + 7
        END
    OR DAYOFYEAR("Datum") < 7 -
```

```
DAYOFYEAR(CAST(YEAR(CURDATE()) || '-12-31' AS DATE)) +  
DAYOFYEAR(CURDATE())
```

Schaltjahre sind daran zu erkennen, dass das gesamte Jahr 366 und nicht 365 Tage hat. Dies wird für die entsprechenden Unterscheidungen genutzt.

Zum einen muss jeder Datumswert darauf überprüft werden, ob er in einem Schaltjahr liegt und außerdem für die korrekte Zählung der 60. Tag im Jahr ist (31 Tage im Januar und 29 Tage im Februar). Für diesen Fall werden alle darauffolgende DAYOFYEAR – Werte für das Datum um 1 herabgestuft. Dann trifft ein 1.3. eines Schaltjahres wieder exakt auf einen 1.3. eines normalen Jahres.

Zum anderen muss auch beim aktuellen Jahr (CURDATE()) überprüft werden, ob es sich um ein Schaltjahr handelt. Auch hier wird entsprechend die Tageszahl um 1 herabgestuft.

Auch eine Anzeige von Terminen für die nächsten 8 Tage ist so noch nicht einwandfrei möglich, da das Jahr in der Abfrage noch ausgeklammert ist. Dies wäre aber über eine einfache zusätzliche Bedingung **YEAR("Datum") = YEAR(CURDATE())** für das aktuelle bzw. **YEAR("Datum") = YEAR(CURDATE()) + 1** für das zukünftige Jahr zu erfüllen.

Tage zu Datumswerten addieren

Bei der Ausleihe von Medien möchte vielleicht der Entleiher genau wissen, an welchem Tag denn die Rückgabe des Mediums erfolgen soll. Leider bietet die interne HSQLDB dazu nicht die Funktion **DATEADD()** an, wie dies in vielen der externen Datenbanken und auch bei Firebird der Fall ist.

HSQLDB - Firebird

Für die interne Firebirddatenbank ist diese Vorgehensweise zum Addieren von Tagen zu Datumswerten überflüssig. Mit

```
SELECT "Leih_Datum", DATEADD(DAY, 14, "LeihDatum") AS "Rueckgabeda-  
tum" FROM "Ausleihe"
```

werden zum Ausleihdatum 14 Tage addiert. Der Befehl funktioniert allerdings nur bei direkter Ausführung des SQL-Kommandos.

Für die interne HSQLDB lässt sich für einen begrenzten Zeitraum der folgende Umweg nutzen:

Zuerst wird eine Tabelle mit dem Datumsverlauf über den gewünschten Zeitraum erstellt. Hierzu wird einfach Calc geöffnet, in das Feld A1 die Bezeichnung «ID» und in das Feld B1 die Bezeichnung «Datum» geschrieben. In Feld A2 wird eine 1 eingetragen, in Feld B2 das gewünschte Startdatum, z.B. 1.1.15. A2 und B2 werden markiert und weiter nach unten gezogen, so dass daraus eine fortlaufende Nummer und ein fortlaufendes Datum entstehen.

Anschließend wird diese Tabelle mit den Spaltenüberschriften zusammen markiert und in Base eingefügt: **rechte Maustaste → Einfügen → Tabellenname → Datum**. Bei den Optionen wird **Definition und Daten** sowie **Erste Zeile als Spaltennamen verwenden** angeklickt. Alle Spalten werden übernommen. Anschließend ist nur noch darauf zu achten, dass bei den Typformatierungen das Feld «ID» dem Typ **Integer** [**INTEGER**] zugeordnet werden soll und dem Datumsfeld auch der Typ **Datum** [**DATE**] zugeordnet wird. Ein Primärschlüssel ist nicht erforderlich, da die Daten später nicht geändert werden sollen. Dadurch, dass eben der Primärschlüssel nicht definiert wird, wird die Tabelle gleichzeitig schreibgeschützt.

Tip

Auch über Abfragetechniken ist die Erstellung so einer Übersicht möglich. Diese Übersicht kann, wenn eine Filtertabelle genutzt wird, in Startdatum und ihrem Umfang an Datumswerten sogar gesteuert werden:

```
SELECT DISTINCT CAST
  ("Y"."Nr" + (SELECT "Jahr" FROM "Filter" WHERE "ID" = True) - 1 || '-' ||
  CASEWHEN( "M"."Nr" < 10, '0' || "M"."Nr", " || "M"."Nr" ) || '-' ||
  CASEWHEN( "D"."Nr" < 10, '0' || "D"."Nr", " || "D"."Nr" )
  AS DATE ) AS "Datum"
FROM "NrBis31" AS "D", "NrBis31" AS "M", "NrBis31" AS "Y"
WHERE "Y"."Nr" <= (SELECT "Jahre" FROM "Filter" WHERE "ID" = True)
AND "M"."Nr" <= 12 AND "D"."Nr" <= 31
```

Diese Ansicht greift auf eine Tabelle zu, die lediglich die Nummern von 1 bis 31 erfasst und schreibgeschützt ist. In einer weiteren Filtertabelle wird das Startjahr und der Umfang in Jahren festgelegt, den die Ansicht umfassen soll. Das Datum wird dadurch zusammengestellt, dass ein Datumsausdruck aus Jahr, Monat und Tag als Text erstellt und anschließend in ein Datum umgewandelt wird. Die HSQLDB akzeptiert alle Tage bis zum 31. eines Monats, auch z.B. den 31.02.2015. Aus dem 31.02.2015 wird allerdings entsprechend der 3.03.2015 wiedergegeben. Deshalb muss bei der Erstellung der Ansicht über DISTINCT ausgeschlossen werden, dass Datumswerte doppelt vorkommen.

Hierauf greift die folgende Ansicht zu:

```
SELECT "a"."Datum",
  (SELECT COUNT(*) FROM "Ansicht_Datum" WHERE "Datum" <=
  "a"."Datum")
  AS "lfdNr"
FROM "Ansicht_Datum" AS "a"
```

Über eine [Zeilennummerierung](#) wird den Datumswerten eine Nummer hinzugefügt. Da aus Ansichten sowieso keine Daten gelöscht werden können, muss hier natürlich nicht extra ein Schreibschutz bemüht werden.

Mit einer Abfrage kann jetzt zu einem bestimmten Datum ermittelt werden, wie z.B. das Datum in 14 Tagen lauten wird:

```
SELECT "a"."Leih_Datum",
  (SELECT "Datum" FROM "Datum" WHERE "ID" =
  (SELECT "ID" FROM "Datum" WHERE "Datum" = "a"."Leih_Datum")+14)
  AS "Rueckgabedatum"
FROM "Ausleihe" AS "a"
```

In der ersten Spalte wird das Ausleihdatum ermittelt. Auf diese Spalte greift eine korrelierende Unterabfrage zu, die wieder in zwei Abfragen geschachtelt ist. Es wird über **SELECT "ID" FROM "Datum"** der Wert des Feldes "ID" ermittelt, der dem Ausleihdatum entspricht. Zu diesem Wert wird 14 addiert. Dieser Wert wird dem Feld "ID" bei der äußeren Unterabfrage zugewiesen. Zu dieser neuen "ID" wird nachgesehen, welches Datum im Datumsfeld des Datensatzes steht.

Leider wird bei der Anzeige in der Abfrage der Datumstyp nicht automatisch erkannt, so dass hier entsprechend formatiert werden muss. In einem Formular ist die entsprechende Anzeige aber dauerhaft speicherbar, so dass auch bei jeder Abfrage entsprechend ein Datumswert ausgegeben wird.

Eine Direktvariante zur Ermittlung des Datumswertes ist sogar auf kürzerem Wege möglich. Hierbei wird der Startwert genutzt, ab dem die interne Tageszählung von Base beginnt:

```
SELECT "Leih_Datum",
  DATEDIFF( 'dd', '1899-12-30', "Leih_Datum" ) + 14
  AS "Rueckgabedatum"
FROM "Tabelle"
```

Der ermittelte Zahlenwert kann im Formular als Datum über ein formatiertes Feld dargestellt werden. Es ist allerdings nur mit großem Aufwand möglich, ihn einfach für weiteren Abfragecode in ein SQL-Datum zu übertragen.

Zeiten zu Zeitstempeln addieren

In MySQL gibt es die Funktion **TIMESTAMPADD()**. Eine vergleichbare Funktion existiert in der HSQLDB nicht. Aber auch hier kann über den internen Zahlenwert, den so ein Zeitstempel einnimmt, mittels eines formatierten Feldes im Formular auch die Addition oder Subtraktion einer Zeit dargestellt werden.

HSQLDB - Firebird

Für die interne Firebirddatenbank ist diese Vorgehensweise zum Addieren von Zeiten zu Zeitstempeln überflüssig. Mit

```
SELECT "Zeitstempel", DATEADD( MINUTE, 14, "Zeitstempel" ) FROM  
"Tabelle"
```

werden zum Zeitstempel 14 Minuten addiert. Der Befehl funktioniert allerdings nur bei direkter Ausführung des SQL-Kommandos.

Im Gegensatz zur Addition von Tagen zu einem Datum tritt bei den Zeiten allerdings ein Problem auf, das anfangs vielleicht gar nicht auffällt:

```
SELECT "DatumZeit"  
    DATEDIFF( 'ss', '1899-12-30', "DatumZeit" ) / 86400.0000000000 +  
    36/24 AS "DatumZeit+36Stunden"  
FROM "Tabelle"
```

Für die neue Zeitberechnung wird der Unterschied zur Startzeit '0' des Systems genommen. Das ist, wie bei der Datumsberechnung bereits angewandt, der Datumsstempel vom 30.12.1899.

Hinweis

Das Nulldatum am 30.12.1899 ist vermutlich deshalb entstanden, weil das Jahr 1900 im Gegensatz zur üblichen 4-Jahres-Rechnung kein Schaltjahr war. So ist der Tag '1' der internen Rechnung auf den 31.12.1899 vorverlegt worden und eben nicht der 1.1.1900.

Der Unterschied wird hier allerdings in Sekunden ausgedrückt. Die interne Zahl rechnet allerdings die Tage als Stellen vor dem Komma, die Stunden, Minuten und Sekunden als Stellen nach dem Komma. Da in einen Tag $60 \cdot 60 \cdot 24$ Sekunden passen, muss die entsprechend ermittelte Sekundenzahl durch 86400 geteilt werden, um schließlich vor dem Komma mit den entsprechenden Tagen und hinter dem Komma mit den Bruchteilen rechnen zu können. Damit die interne HSQLDB überhaupt Nachkommastellen bei dieser Berechnung ausgibt, müssen diese auch in der Rechnung vorkommen. Statt durch 86400 zu teilen, wurde deshalb durch 86400,0000000000 geteilt. Dezimalstellen erscheinen in der Abfrage durch einen Punkt getrennt. Das Ergebnis hat letztlich also 10 Dezimalstellen hinter dem Komma.

Zu diesem Ergebnis wird die Stundenzahl als Bruchteil eines Tages hinzugezählt. Die berechnete Ziffer lässt sich bei entsprechender Formatierung in der Anfrage darstellen. Dort wird die Formatierung aber leider nicht gespeichert. Innerhalb eines Formulars mit formatierbarem Feld oder innerhalb eines Berichts kann sie aber dauerhaft mit entsprechender Formatierung übernommen werden.

Sollen Minuten oder Sekunden addiert werden, so ist hier auch immer darauf zu achten, dass die Angaben der Minuten und Sekunden als Tagesbruchteile angegeben werden.

Liegt das Datum in den Monaten November, Dezember, Januar usw., dann fällt bei der Rechnung erst einmal nichts auf. Die Darstellung ist stimmig, zum Zeitstempel von 20.01.2015 13:00:00 eine Zeit von 36 Stunden addiert ergibt den neuen (nur dargestellten) Stempel 22.01.2015 01:00:00. Anders verhält es sich bei 20.04.2015 13:00:00. Da wird anschließend der 22.04.2015 00:00:00

ausgegeben. Das liegt an der Sommerzeit, die der Berechnung hier in die Quere kommt. Die gerade bei der Zeitumstellung «verlorene» oder «gewonnene» Stunde lässt sich bei einem Übergang nicht berücksichtigen. Innerhalb einer Zeitzone kann allerdings auf verschiedene Weise eine entsprechende Berechnung mit «korrektem» Ergebnis durchgeführt werden. Hier die dafür einfachere Variante:

```
SELECT "DatumZeit"
      DATEDIFF( 'dd', '1899-12-30', "DatumZeit" ) +
      HOUR( "DatumZeit" ) / 24.0000000000 +
      MINUTE( "DatumZeit" ) / 1440.0000000000 +
      SECOND( "DatumZeit" ) / 86400.0000000000 +
      36/24
      AS "DatumZeit+36Stunden"
FROM "Tabelle"
```

Statt die Stunden, Minuten und Sekunden aus dem Ursprungsdatum zu beziehen, werden sie hier im Verhältnis zum aktuellen Datum dargestellt. Am 20.05.2015 steht die Uhr auf 13:00 Uhr, würde aber ohne die Sommerzeit 12:00 Uhr anzeigen. Die Funktion **HOUR** berücksichtigt die Sommerzeit und gibt 13 Stunden als Stundenanteil aus. Damit kann dann der Stundenanteil korrekt zum Tagesanteil addiert werden. Auf gleiche Art und Weise geschieht dies mit Hilfe des Minutenanteils und des Sekundenanteils. Anschließend werden die zu addierenden Stunden wieder als Tagesbruchteil addiert und das Ganze mit Hilfe der Zellenformatierung als berechneter Zeitstempel ausgegeben.

Zwei Dinge sollten aber bei diesen Berechnungen nie aus den Augen verloren werden:

1. Bei Übergängen von Winterzeit zu Sommerzeit lassen sich die Stundenwerte nicht korrekt berechnen. Hier könnte mit Hilfe einer Zusatztabelle nachgeholfen werden, die Beginn und Ende der Sommerzeiten aufnimmt und dann mit einer Stundenkorrektur einspringt. Ein etwas aufwändiges Verfahren.
2. Die Ausgabe der Zeitangabe ist nur über formatierbare Felder zu erreichen. Das Ergebnis ist ein Dezimalwert, kein Zeitstempelwert, der z.B. wiederum in einer Datenbank direkt gespeichert werden könnte. Hier müsste entweder innerhalb des Formular kopiert werden oder über aufwändige Abfragen aus dem Dezimalwert zu einem Zeitstempelwert portiert werden. Der Knackpunkt bei dieser Portierung ist der Datumswert, da es eben Schaltjahre und Monate mit unterschiedlicher Tagesanzahl gibt.

Laufenden Kontostand nach Kategorien ermitteln

Statt eines Haushaltsbuches wird eine Datenbank im PC die leidigen Aufsummierungen von Ausgaben für Lebensmittel, Kleidung, Mobilität usw. erleichtern. Ein Großteil dieser Angaben sollte natürlich auf Anhieb in der Datenbank sichtbar sein. Dabei wird in dem Beispiel davon ausgegangen, dass Einnahmen und Ausgaben in einem Feld "Betrag" mit Vorzeichen abgespeichert werden. Prinzipiell lässt sich das Ganze natürlich auf getrennte Felder und eine Summierung hierüber erweitern.

```
SELECT "ID", "Betrag",
      ( SELECT SUM( "Betrag" ) FROM "Kasse" WHERE "ID" <= "a"."ID" )
      AS "Saldo"
FROM "Kasse" AS "a" ORDER BY "ID" ASC
```

Mit dieser Abfrage wird bei jedem neuen Datensatz direkt ausgerechnet, welcher Kontostand jetzt erreicht wurde. Dabei bleibt die Abfrage editierbar, da das Feld "Saldo" durch eine korrelierende Unterabfrage erstellt wurde. Die Abfrage gibt den Kontostand in Abhängigkeit von dem automatisch erzeugten Primärschlüssel "ID" an. Kontostände werden aber eigentlich täglich ermittelt. Es muss also eine Datumsabfrage her.

```
SELECT "ID", "Datum", "Betrag",
      ( SELECT SUM( "Betrag" ) FROM "Kasse" WHERE "Datum" <= "a"."Datum"
```

```

    ) AS "Saldo"
FROM "Kasse" AS "a" ORDER BY "Datum", "ID" ASC

```

Die Ausgabe erfolgt jetzt nach dem Datum sortiert und nach dem Datum summiert. Bleibt noch die Kategorie, nach der entsprechende Salden für die einzelnen Kategorien dargestellt werden sollen.

```

SELECT "ID", "Datum", "Betrag", "Konto_ID",
    ( SELECT "Konto" FROM "Konto" WHERE "ID" = "a"."Konto_ID" )
    AS "Kontoname",
    ( SELECT SUM( "Betrag" ) FROM "Kasse" WHERE "Datum" <= "a"."Datum"
      AND "Konto_ID" = "a"."Konto_ID" ) AS "Saldo",
    ( SELECT SUM( "Betrag" ) FROM "Kasse" WHERE "Datum" <= "a"."Datum"
      ) AS "Saldo_gesamt"
FROM "Kasse" AS "a" ORDER BY "Datum", "ID" ASC

```

Hiermit wird eine editierbare Abfrage erzeugt, in der neben den Eingabefeldern (Datum, Betrag, Konto_ID) der Kontoname, der jeweilige Kontostand und der Saldo insgesamt erscheinen. Da sich die korrelierenden Unterabfragen teilweise auch auf vorhergehende Eingaben stützen ("Datum" <= "a"."Datum") werden nur Neueingaben reibungslos dargestellt. Änderungen eines vorhergehenden Datensatzes machen sich zuerst einmal nur in diesem Datensatz bemerkbar. Die Abfrage muss aktualisiert werden, damit auch die davon abhängigen späteren Berechnungen neu durchgeführt werden.

Zeilennummerierung

Automatisch hochzählende Felder sind etwas feines. Nur sagen sie nicht unbedingt etwas darüber aus, wie viele Datensätze denn nun in der Datenbank oder dem Abfrageergebnis wirklich vorhanden sind. Häufig werden Datensätze gelöscht und manch ein User versucht verzweifelt dahinter zu kommen, welche Nummer denn nun nicht mehr vorhanden ist, damit die laufenden Nummern wieder stimmen.

```

SELECT "ID", ( SELECT COUNT( "ID" ) FROM "Tabelle" WHERE "ID" <=
    "a"."ID" ) AS "lfdNr."
FROM "Tabelle" AS "a"

```

Das Feld "ID" wird ausgelesen, im zweiten Feld wird durch eine korrelierende Unterabfrage festgestellt, wie viele Feldinhalte von "ID" kleiner oder gleich dem aktuellen Feldinhalt sind. Daraus wird dann die laufende Zeilennummer erstellt.

Wird einer Abfrage eine Bedingung hinzugefügt oder beruht eine Abfrage aus einer Verbindung von mehreren Tabellen, so ist diese Bedingung bzw. die Verbindung von mehreren Tabellen auch in der korrelierenden Unterabfrage hinzuzufügen. Andernfalls stimmt die Zählung nicht:

```

SELECT "Name", ( SELECT COUNT( "ID" ) FROM "Name" WHERE "GebDat" >
    '1995-12-31' AND "ID" <= "a"."ID" ) AS "lfdNr."
FROM "Name" AS "a" WHERE "GebDat" > '1995-12-31'

```

Die gesamte Bedingung der äußeren Abfrage muss in der korrelierenden Unterabfrage wiederholt werden. Hinzu kommt in der korrelierenden Unterabfrage noch die Bedingung, mit der sich die korrelierende Unterabfrage auf den aktuellen Datensatz bezieht.

Auch eine Nummerierung für entsprechende Gruppierungen ist möglich:

```

SELECT "ID", ( SELECT COUNT( "ID" ) FROM "Tabelle" WHERE "ID" <=
    "a"."ID" AND "RechnungsID" = "a"."RechnungsID" ) AS "lfdNr."
FROM "Tabelle" AS "a"

```

Hier gibt es in einer Tabelle verschiedene Rechnungsnummern ("RechnungsID"). Für jede Rechnungsnummer wird separat die Anzahl der Felder "ID" aufsteigend nach der Sortierung des Feldes "ID" wiedergegeben. Das erzeugt für jede Rechnung die Nummerierung von 1 an aufwärts.

Soll die aktuelle Sortierung der Abfrage mit der Zeilennummer übereinstimmen, so ist die Art der Sortierung entsprechend abzubilden. Dabei muss die Sortierung allerdings einen eindeutigen Wert ergeben. Sonst liegen 2 Nummern auf dem gleichen Wert.

```
SELECT "Name", ( SELECT COUNT( "ID" ) FROM "Tabelle"
  WHERE "Name" <= "a"."Name" ) AS "lfdNr."
FROM "Tabelle" AS "a"
ORDER BY "Name"
```

Nur wenn ein eindeutigen Index für das Feld "Name" definiert wurde, ist hier eine eindeutige Sortierung und auch eine klare Nummerierung zu erwarten. Tauchen aber zwei gleiche Namen auf, so erhalten beide die gleiche Nummer.

```
SELECT "Name", ( SELECT COUNT( "ID" ) FROM "Tabelle"
  WHERE "Name" || "ID" <= "a"."Name" || "a"."ID" ) AS "lfdNr."
FROM "Tabelle" AS "a"
ORDER BY "Name" || "ID"
```

Hier wird der Inhalt des Feldes "Name" mit dem Inhalt des Feldes "ID" zusammengefügt. Die Sortierung ist jetzt eindeutig, die laufende Nummer entsprechend auch. Bei genauerer Betrachtung fällt allerdings auf, dass die Sortierung der "ID" natürlich jetzt nicht der Zahlensortierung folgt. Stattdessen wird z.B. die Zahl '9' nach der Zahl '10' einsortiert, da 9 größer als 1 ist. Wer dem noch abhelfen will muss entsprechend mit Leerstellen oder Nullen auffüllen:

```
SELECT "Name", ( SELECT COUNT( "ID" ) FROM "Tabelle"
  WHERE "Name" || RIGHT( '0000000000' || "ID", 10 ) <= "a"."Name" ||
    RIGHT( '0000000000' || "a"."ID", 10 ) ) AS "lfdNr."
FROM "Tabelle" AS "a"
ORDER BY "Name" || RIGHT( '0000000000' || "ID", 10 )
```

Das Feld "ID" ist in diesem Beispiel ein INTEGER-Feld. INTEGER-Zahlen haben maximal 10 Stellen. Der Wert des Feldes wird mit 9 führenden Nullen aufgefüllt. Anschließend werden die 10 rechts stehenden Zeichen für die weitere Verarbeitung genutzt. '0000000009' ist nun kleiner als '0000000010'. Die Sortierung ist eindeutig und erfolgt bei gleichem Namen schließlich in gewohnter Reihenfolge der Schlüsselnummerierung.

Sollen Zeilen für eine Abfrage nummeriert werden, die sich nicht nur auf eine Tabelle bezieht oder mit einer zusätzlichen Bedingung versehen ist, so muss in der korrelierenden Unterabfrage die komplette Bedingung mit allen betroffenen Tabellen enthalten sein. Nur dann kann die Unterabfrage die gleichen Daten verarbeiten wie die äußere Abfrage.

Gleiche Werte können natürlich genutzt werden, wenn z.B. die Platzierung bei einem Wettkampf wiedergegeben werden soll, da hier gleiche Wettkampfergebnisse auch zu einer gleichen Platzierung führen. Damit die Platzierung allerdings so wiedergegeben wird, dass bei einer gleichen Platzierung der nachfolgende Wert ausgelassen wird, ist die Abfrage etwas anders zu konstruieren:

```
SELECT "ID",
  ( SELECT COUNT( "ID" ) + 1
    FROM "Tabelle" WHERE "Zeit" < "a"."Zeit" ) AS "Platz"
FROM "Tabelle" AS "a"
```

Es werden alle Einträge ausgewertet, die in dem Feld "Zeit" einen kleineren Eintrag haben. Damit werden alle Sportler erfasst, die vor dem aktuellen Sportler ins Ziel gekommen sind. Zu diesem Wert wird der Wert 1 addiert. Damit ist der Platz des aktuellen Sportlers bestimmt. Ist dieser zeitgleich mit einem anderen Sportler, so ist auch der Platz gleich. Damit sind Platzierungen wie 1. Platz, 2. Platz, 2. Platz, 4. Platz usw. möglich.

Schwieriger wird es, wenn neben der Platzierung auch eine Zeilennummerierung erfolgen soll. Dies kann z.B. sinnvoll sein, um mehrere Datensätze in einer Zeile zusammen zu fassen.

```
SELECT "ID", ( SELECT COUNT( "ID" ) + 1 FROM "Tabelle" WHERE "Zeit" <
  "a"."Zeit" ) AS "Platz",
```



```

CASE WHEN
  ( SELECT COUNT( "ID" ) + 1 FROM "Tabelle"
    WHERE "Zeit" = "a"."Zeit" ) = 1
THEN
  ( SELECT COUNT( "ID" ) + 1 FROM "Tabelle"
    WHERE "Zeit" < "a"."Zeit" )
ELSE (SELECT (
  SELECT COUNT( "ID" ) + 1 FROM "Tabelle" WHERE "Zeit" < "a"."Zeit" )
  + COUNT( "ID" ) FROM "Tabelle"
  WHERE "Zeit" = "a"."Zeit" "ID" < "a"."ID"
)
END
AS "Zeilennummer" FROM "Tabelle" AS "a"

```

Die zweite Spalte gibt weiterhin die Platzierung wieder. In der 3. Spalte wird zuerst nachgefragt, ob auch wirklich nur eine Person mit der gleichen Zeit durchs Ziel gekommen ist. Wenn dies erfüllt ist, wird die Platzierung auf jeden Fall direkt als Zeilennummer übernommen. Wenn dies nicht erfüllt ist, wird zu der Platzierung ein weiterer Wert addiert. Bei gleicher Zeit ("**Zeit**" = "**a**".**"Zeit"**) wird dann mindestens 1 addiert, wenn es eine weitere Person mit dem Primärschlüssel ID gibt, deren Primärschlüssel kleiner ist als der aktuelle Primärschlüssel des aktuellen Datensatzes ("**ID**" < "**a**".**"ID"**). Diese Abfrage gibt also solange identische Werte zur Platzierung heraus, wie keine zweite Person mit der gleichen Zeit existiert. Existiert eine zweite Person mit der gleichen Zeit, so wird nach der ID entschieden, welche Person die geringere Zeilennummer enthält.

Diese Zeilensortierung entspricht übrigens der, die die Datenbanken anwenden. Wird z.B. eine Reihe Datensätze nach dem Namen sortiert, so erfolgt die Sortierung bei gleichen Datensätzen nicht nach dem Zufallsprinzip, sondern aufsteigend nach dem Primärschlüssel, der ja schließlich eindeutig ist. Es lässt sich auf diese Weise also über die Nummerierung eine Sortierung der Datensätze abbilden.

Die Zeilennummerierung ist auch eine gute Voraussetzung, um einzelne Datensätze als einen Datensatz zusammen zu fassen. Wird eine Abfrage zur Zeilennummerierung als Ansicht erstellt, so kann darauf mit einer weiteren Abfrage problemlos zugegriffen werden. Als einfaches Beispiel hier noch einmal die erste Abfrage zur Nummerierung, nur um ein Feld ergänzt:

```

SELECT "ID", "Name",
  ( SELECT COUNT( "ID" ) FROM "Tabelle" WHERE "ID" <= "a"."ID" )
  AS "lfdNr."
FROM "Tabelle" AS "a"

```

Aus dieser Abfrage wird jetzt die Ansicht '*Ansicht1*' erstellt. Die Abfrage, mit der z.B. die ersten 3 Namen zusammen in einer Zeile erscheinen können, lautet:

```

SELECT "Name" AS "Name_1",
  ( SELECT "Name" FROM "Ansicht1" WHERE "lfdNr." = 2 ) AS "Name_2",
  ( SELECT "Name" FROM "Ansicht1" WHERE "lfdNr." = 3 ) AS "Name_3"
FROM "Ansicht1" WHERE "lfdNr." = 1

```

Auf diese Art und Weise können mehrere Datensätze nebeneinander als Felder dargestellt werden. Allerdings läuft diese Nummerierung einfach vom ersten bis zum letzten Datensatz durch.

Sollen alle Personen zu einem Nachnamen zugeordnet werden, so ließe sich das folgendermaßen realisieren:

```

SELECT "ID", "Name", "Nachname",
  ( SELECT COUNT( "ID" ) FROM "Tabelle" WHERE "ID" <= "a"."ID"
    AND "Nachname" = "a"."Nachname" ) AS "lfdNr."
FROM "Tabelle" AS "a"

```

Jetzt kann über die erstellte Ansicht eine entsprechende Familienzusammenstellung erfolgen:

```

SELECT "Nachname", "Name" AS "Name_1",
  ( SELECT "Name" FROM "Ansicht1" WHERE "lfdNr." = 2

```



```

        AND "Nachname" = "a"."Nachname") AS "Name_2",
    ( SELECT "Name" FROM "Ansicht1" WHERE "lfdNr." = 3
      AND "Nachname" = "a"."Nachname") AS "Name_3"
FROM "Ansicht1" AS "a"
WHERE "lfdNr." = 1

```

In einem Adressbuch ließen sich so alle Personen einer Familie ("Nachnamen") zusammenfassen, damit jede Adresse nur einmal für ein Anschreiben berücksichtigt würde, aber alle Personen, an die das Anschreiben gehen soll, aufgeführt würden.

Da es sich um keine fortwährende Schleifenfunktion handelt, ist hier allerdings Vorsicht geboten. Schließlich wird die Grenze der parallel als Felder angezeigten Datensätze durch die Abfrage im obigen Beispiel z.B. auf 3 begrenzt. Diese Grenze wurde willkürlich gesetzt. Weitere Namen tauchen nicht auf, auch wenn die Nummerierung der "lfdNr." größer als 3 ist.

In seltenen Fällen ist so eine Grenze aber auch klar nachvollziehbar. Soll z.B. ein Kalender erstellt werden, so können mit diesem Verfahren die Zeilen die Wochen des Jahres darstellen, die Spalten die Wochentage. Da im ursprünglichen Kalender nur das Datum über den Inhalt entscheidet, werden durch die Zeilennummerierung immer die Tage einer Woche durchnummeriert und nach Wochen im Jahr als Datensatz später ausgegeben. Spalte 1 gibt dann Montag wieder, Spalte 2 Dienstag usw. Die Unterabfrage endet also jeweils bei der "lfdNr." = 7. Damit lassen sich dann im Bericht alle sieben Wochentage nebeneinander anzeigen und eine entsprechende Kalenderübersicht erstellen.

Zeilenumbruch durch eine Abfrage erreichen

Manchmal ist es sinnvoll, durch eine Abfrage verschiedene Felder zusammenzufassen und mit einem Zeilenumbruch zu trennen. So ist es z.B. einfacher eine Adresse in einen Bericht komplett einzulesen.

Der Zeilenumbruch innerhalb einer Abfrage erfolgt durch **'Char(13)'**. Beispiel:

```

SELECT "Vorname" || ' ' || "Nachname" || Char(13) || "Straße" || Char(13) || "Ort"
FROM "Tabelle"

```

Dies erzeugt nachher:

```

Vorname Nachname
Straße
Ort

```

Hinweis

Für **FIREBIRD** muss statt **CHAR(13)** **ASCII_CHAR(13)** eingefügt werden. Firebird kennt die hier verwendete Kurzform nicht.

Mit so einer Abfrage, zusammen mit einer Nummerierung jeweils bis zur Nummer 3, lassen sich auch dreispaltige Etikettendrucke von Adressetiketten über Berichte realisieren. Eine Nummerierung ist in diesem Zusammenhang nötig, damit drei Adressen nebeneinander in einem Datensatz erscheinen. Nur so sind sie auch nebeneinander im Bericht einlesbar.

Je nach Betriebssystem kann es auch notwendig sein, zusätzlich zu Char(13) auch Char(10) in den Code mit aufzunehmen:

```

SELECT "Vorname" || ' ' || "Nachname" || Char(13) || Char(10) ||
      "Straße" || Char(13) || Char(10) || "Ort"
FROM "Tabelle"

```

Solche Zeilenumbrüche werden allerdings nicht in der Abfrage angezeigt. Die Steuerzeichen werden in einem VARCHAR-Feld nicht umgesetzt. Entsprechend rückt der Text dort ohne Leerzeichen aneinander. Soll der Zeilenumbruch in der Abfrage angezeigt werden, so muss der zusammengefügte Text von VARCHAR nach LONGVARCHAR umgewandelt werden:

```
SELECT CAST("Vorname" || ' ' || "Nachname" || Char(13) || Char(10) ||
"Straße" || Char(13) || Char(10) || "Ort" AS LONGVARCHAR) AS "Adresse"
FROM "Tabelle"
```

Damit wird dann die Adresse auch in einer Abfrage mehrzeilig unter dem Alias "Adresse" dargestellt.

Gruppieren und Zusammenfassen

Für andere Datenbanken, auch neuere Versionen der HSQLDB, ist der Befehl **'Group_Concat()'** verfügbar. Mit ihm können einzelne Felder einer Datensatzgruppe zusammengefasst werden. So ist es z.B. möglich, in einer Tabelle Vornamen und Nachnamen zu speichern und anschließend die Daten so darzustellen, dass in einem Feld die Nachnamen als Familiennamen erscheinen und in dem 2. Feld alle Vornamen hintereinander, durch z.B. Komma getrennt, aufgeführt werden.

HSQLDB - Firebird

Für die interne Firebird-Datenbank ist diese Vorgehensweise zum Gruppieren überflüssig. Mit

```
SELECT "Nachname", CAST( LIST( "Vorname", ' ' ) AS VARCHAR ( 32000 ) )
AS "Vornamen" FROM "Tabelle" GROUP BY "Nachname"
```

werden die Vornamen, gruppiert nach den Nachnamen, zusammengefasst. Bei Einführung von LO 5.3 ist es noch notwendig, die über **LIST** zusammengestellte Gruppe mit **CAST** in einen **VARCHAR**-Text umzuwandeln. Mit der Version LO 5.3.1 ist dieser Bug behoben.

Dieses Beispiel entspricht in vielen Teilen dem der Zeilennummerierung. Die Gruppierung zu einem gemeinsamen Feld stellt hier eine Ergänzung dar.

Nachname	Vorname
Müller	Karin
Schneider	Gerd
Müller	Egon
Schneider	Volker
Müller	Monika
Müller	Rita

Wird nach der Abfrage zu:

Nachname	Vornamen
Müller	Karin, Egon, Monika, Rita
Schneider	Gerd, Volker

Dieses Verfahren kann in Grenzen auch in der HSQLDB nachgestellt werden. Das folgende Beispiel bezieht sich auf eine Tabelle "Name" mit den Feldern "ID", "Vorname" und "Nachname". Folgende Abfrage wird zuerst an die Tabelle gestellt und als Ansicht "Ansicht_Gruppe" gespeichert:

```
SELECT "Nachname", "Vorname", ( SELECT COUNT( "ID" ) FROM "Name" WHERE
"ID" <= "a"."ID" AND "Nachname" = "a"."Nachname" ) AS "GruppenNr" FROM
"Name" AS "a"
```

Im Kapitel «Abfragen» ist nachzulesen, wie diese Abfrage über die [Korrelierte Unterabfrage](#) auf Feldinhalte in der gleichen Abfragezeile zugreift. Dadurch wird eine aufsteigende Nummerierung,

gruppiert nach den "Nachnamen", erzeugt. Diese Nummerierung wird in der folgenden Abfrage benötigt, so dass in dem Beispiel maximal 5 Vornamen aufgeführt werden.

```
SELECT DISTINCT "Nachname",
( SELECT "Vorname" FROM "Ansicht_Gruppe" WHERE "Nachname" =
"a"."Nachname" AND "GruppenNr" = 1 ) ||
COALESCE( ( SELECT ', ' || "Vorname" FROM "Ansicht_Gruppe" WHERE
"Nachname" = "a"."Nachname" AND "GruppenNr" = 2 ), '' ) ||
COALESCE( ( SELECT ', ' || "Vorname" FROM "Ansicht_Gruppe" WHERE
"Nachname" = "a"."Nachname" AND "GruppenNr" = 3 ), '' ) ||
COALESCE( ( SELECT ', ' || "Vorname" FROM "Ansicht_Gruppe" WHERE
"Nachname" = "a"."Nachname" AND "GruppenNr" = 4 ), '' ) ||
COALESCE( ( SELECT ', ' || "Vorname" FROM "Ansicht_Gruppe" WHERE
"Nachname" = "a"."Nachname" AND "GruppenNr" = 5 ), '' )
AS "Vornamen"
FROM "Ansicht_Gruppe" AS "a"
```

Durch Unterabfragen werden nacheinander die Vornamen zu Gruppenmitglied 1, 2 usw. abgefragt und zusammengefasst. Ab der 2. Unterabfrage muss abgesichert werden, dass 'NULL'-Werte nicht die Zusammenfassung auf 'NULL' setzen. Deshalb wird bei einem Ergebnis von 'NULL' stattdessen ' ' angezeigt.

Makros

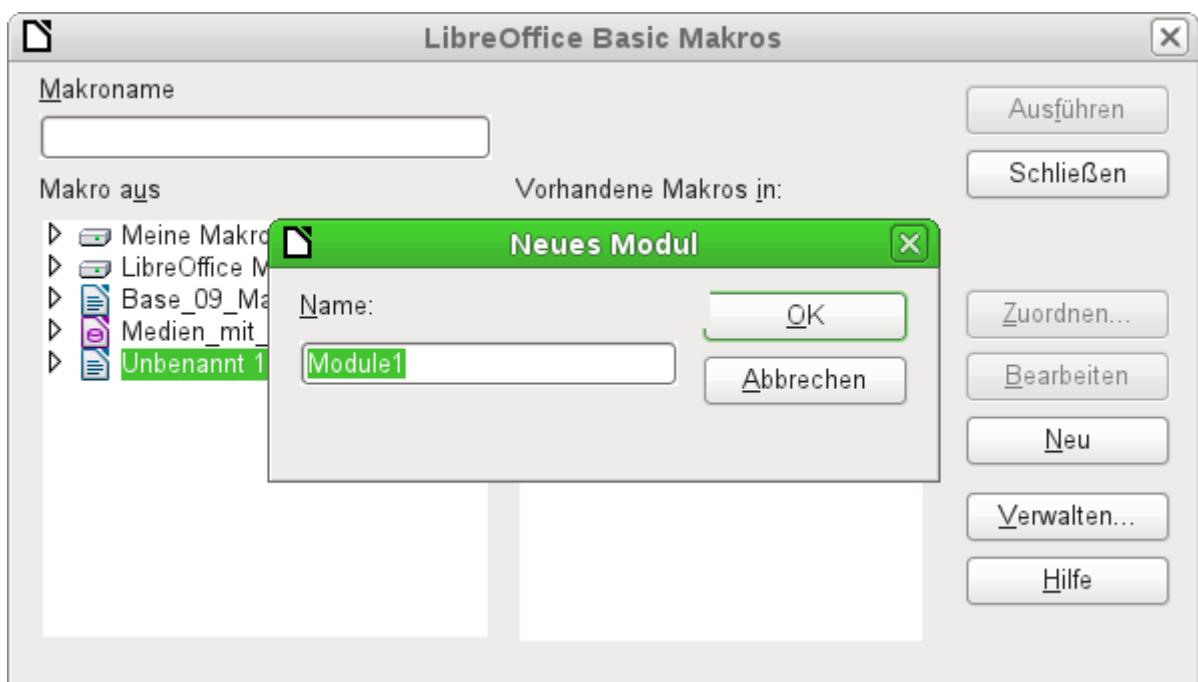
Allgemeines zu Makros

Prinzipiell kommt eine Datenbank unter Base ohne Makros aus. Irgendwann kann aber das Bedürfnis kommen,

- bestimmte Handlungsschritte zu vereinfachen (Wechsel von einem Formular zum anderen, Aktualisierung von Daten nach Eingabe in einem Formular ...),
- Fehleingaben besser abzusichern,
- häufigere Aufgaben zu automatisieren oder auch
- bestimmte SQL-Anweisungen einfacher aufzurufen als mit dem separaten SQL-Editor.

Es ist natürlich jedem selbst überlassen, wie intensiv er/sie Makros in Base nutzen will. Makros können zwar die Bedienbarkeit verbessern, sind aber auch immer mit geringen, bei ungünstiger Programmierung auch stärkeren Geschwindigkeitseinbußen des Programms verbunden. Es ist immer besser, zuerst einmal die Möglichkeiten der Datenbank und die vorgesehenen Einstellmöglichkeiten in Formularen auszureizen, bevor mit Makros zusätzliche Funktionen bereitgestellt werden. Makros sollten deshalb auch immer wieder mit größeren Datenbanken getestet werden, um ihren Einfluss auf die Verarbeitungsgeschwindigkeit abschätzen zu können.

Makros werden über den Weg **Extras → Makros → Makros verwalten → LibreOffice Basic...** erstellt. Es erscheint ein Fenster, das den Zugriff auf alle Makros ermöglicht. Makros für Base werden meistens in dem Bereich gespeichert, der dem Dateinamen der Base-Datei entspricht.



Über den Button **Neu** im Fenster «LibreOffice Basic Makros» wird ein zweites Fenster geöffnet. Hier wird lediglich nach der Bezeichnung für das Modul (Ordner, in dem das Makro abgelegt wird) gefragt. Der Name kann gegebenenfalls auch noch später geändert werden.

Sobald dies bestätigt wird, erscheint der Makro-Editor und auf seiner Eingabefläche wird bereits der Start und das Ende für eine Prozedur angegeben:

```
REM      *****  BASIC      *****
```

```
Sub Main
```

```
End Sub
```

Um Makros, die dort eingegeben wurden, nutzen zu können, sind folgende Schritte notwendig:

- Unter **Extras** → **Optionen** → **Sicherheit** → **Makrosicherheit** ist die Sicherheitsstufe auf «Mittel» herunter zu stellen. Gegebenenfalls kann auch zusätzlich unter «Vertrauenswürdige Quellen» der Pfad angegeben werden, in dem eigene Dateien mit Makros liegen, um spätere Nachfragen nach der Aktivierung von Makros zu vermeiden.
- Die Datenbankdatei muss nach der Erstellung des ersten Makro-Moduls einmal geschlossen und anschließend wieder geöffnet werden.

Einige Grundprinzipien zur Nutzung des Basic-Codes in LibreOffice:

- Zeilen haben keine Zeilenendzeichen. Zeilen enden mit einem festen Zeilenumbruch.
- Zwischen Groß- und Kleinschreibung wird bei Funktionen, reservierten Ausdrücken usw. nicht unterschieden. So ist z.B. die Bezeichnung «String» gleichbedeutend mit «STRING» oder auch «string» oder eben allen anderen entsprechenden Schreibweisen. Groß- und Kleinschreibung dienen nur der besseren Lesbarkeit.
- Eigentlich wird zwischen Prozeduren (beginnend mit **SUB**) und Funktionen (beginnend mit **FUNCTION**) unterschieden. Prozeduren sind ursprünglich Programmabschnitte ohne Rückgabewert, Funktionen können Werte zurückgeben, die anschließend weiter ausgewertet werden können. Inzwischen ist diese Unterscheidung weitgehend irrelevant; man spricht allgemein von Methoden oder Routinen – mit oder ohne Rückgabewert. Auch eine Prozedur kann einen Rückgabewert (außer «Variant») erhalten; der wird einfach in der Definition zusätzlich festgelegt:

```
SUB myProcedure AS INTEGER
END SUB
```

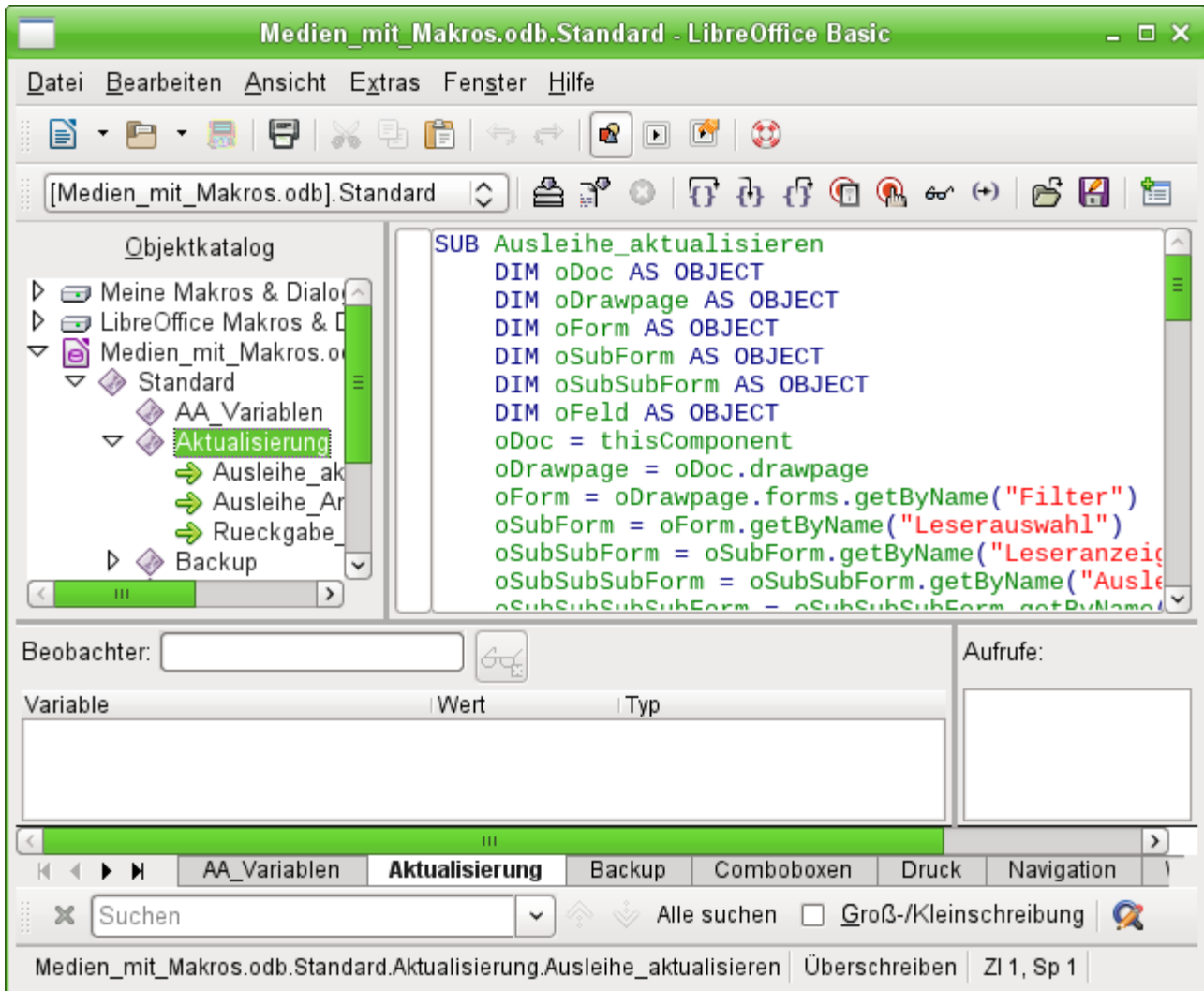
Zu weiteren Details siehe auch das Handbuch 'Erste Schritte Makros mit LibreOffice'.

Hinweis

Makros in diesem Kapitel sind entsprechend den Vorgaben aus dem Makro-Editor von LibreOffice eingefärbt:

Makro-Bezeichner
 Makro-Kommentar
 Makro-Operator
 Makro-Reservierter-Ausdruck
 Makro-Zahl
 Makro-Zeichenkette

Der Makro-Editor



Der Objektkatalog auf der linken Seite zeigt alle zur Zeit verfügbaren Bibliotheken und darin Module an, die über ein Ereignis aufgerufen werden können. «Meine Makros & Dialoge» ist für alle Dokumente eines Benutzers verfügbar. «LibreOffice Makros & Dialoge» sind für alle Benutzer des Rechners und auch anderer Rechner nutzbar, da sie standardmäßig mit LibreOffice installiert werden. Hinzu kommen noch die Bibliotheken, die in dem jeweiligen Dokument, hier «Medien_mit_Makros.odt», abgespeichert sind.

Prinzipiell ist es zwar möglich, aus allen verfügbaren Bibliotheken die Module und die darin liegenden Makros zu nutzen. Für eine sinnvolle Nutzung empfiehlt es sich aber nicht, Makros aus anderen Dokumenten zu nutzen, da diese eben nur bei Öffnung des entsprechenden Dokumentes verfügbar sind. Ebenso ist es nicht empfehlenswert, Bibliotheken aus «Meine Makros & Dialoge» einzubinden, wenn die Datenbankdatei auch an andere Nutzer weitergegeben werden soll. Ausnahmen können hier Erweiterungen («Extensions») sein, die dann mit der Datenbankdatei weiter gegeben werden.

In dem Eingabebereich wird aus dem Modul «Aktualisierung» die Prozedur «Ausleihe_aktualisieren» angezeigt. Eingegebene Zeilen enden mit einem Return. Groß- und Kleinschreibung sowie Einrückung des Codes sind in Basic beliebig. Lediglich der Verweis auf Zeichenketten, z.B. "Filter", muss genau der Schreibweise in dem Formular entsprechen.

Makros können schrittweise für Testzwecke durchlaufen werden. Entsprechende Veränderungen der Variablen werden im Beobachter angezeigt.

Benennung von Modulen, Dialogen und Bibliotheken

Die Benennung von Modulen, Dialogen und Bibliotheken sollte erfolgen, bevor irgendein Makro in die Datenbank eingebunden wird. Sie definieren schließlich den Pfad, in dem das auslösende Ereignis nach dem Makro sucht.

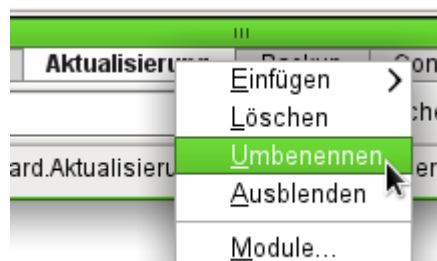
Innerhalb einer Bibliothek kann auf alle Makros der verschiedenen Module zugegriffen werden. Sollen Makros anderer Bibliotheken genutzt werden, so müssen diese extra geladen werden:

```
GlobalScope.BasicLibraries.LoadLibrary("Tools")
```

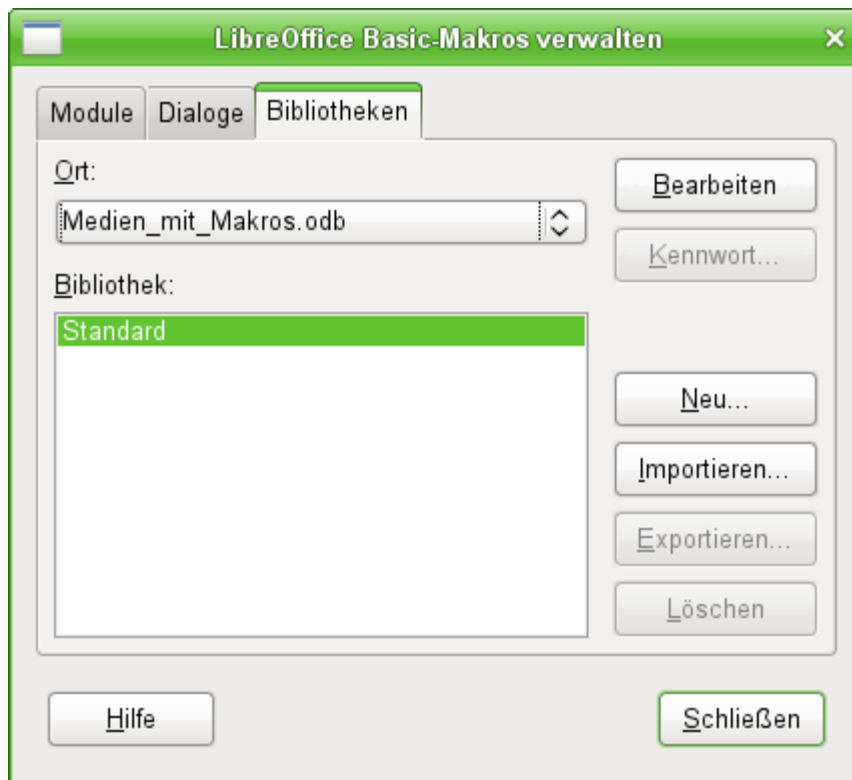
lädt die Bibliothek «Tools», die eine Bibliothek von LibreOffice Makros ist.



Über **Extras** → **Makros verwalten** → **LibreOffice Basic** → **Verwalten** kann der obige Dialog aufgerufen werden. Hier können neue Module und Dialoge erstellt und mit einem Namen versehen werden. Die Namen können allerdings nicht hier, sondern nur in dem Makroeditor selbst verändert werden.



Im Makroeditor wird mit einem rechten Mausklick auf die Reiter mit der Modulbezeichnung direkt oberhalb der Suchleiste ein Kontextmenü geöffnet, das u.a. die Änderung des Modulnamens ermöglicht.



Neue Bibliotheken können innerhalb der Base-Datei angelegt werden. Die Bezeichnung «Standard» der ersten erstellten Bibliothek lässt sich nicht ändern. Die Namen der weiteren Bibliotheken sind frei wählbar, anschließend aber auch nicht änderbar. In eine Bibliothek können Makros aus anderen Bibliotheken importiert werden. Sollte also der dringende Wunsch bestehen, eine andere Bibliotheksbezeichnung zu erreichen, so müsste eine neue Bibliothek mit diesem Namen erstellt werden und sämtlicher Inhalt der alten Bibliothek in die neue Bibliothek exportiert werden. Dann kann anschließend die alte Bibliothek gelöscht werden.

Makros in Base

Makros benutzen

Der «direkte Weg» über **Extras → Makros → Makros ausführen** ist zwar auch möglich, aber bei Base-Makros nicht üblich. Ein Makro wird in der Regel einem Ereignis zugeordnet und durch dieses Ereignis gestartet.

- Ereignisse eines Formular
- Bearbeitung einer Datenquelle innerhalb des Formulars
- Wechsel zwischen verschiedenen Kontrollfeldern
- Reaktionen auf Maßnahmen innerhalb eines Kontrollfelds

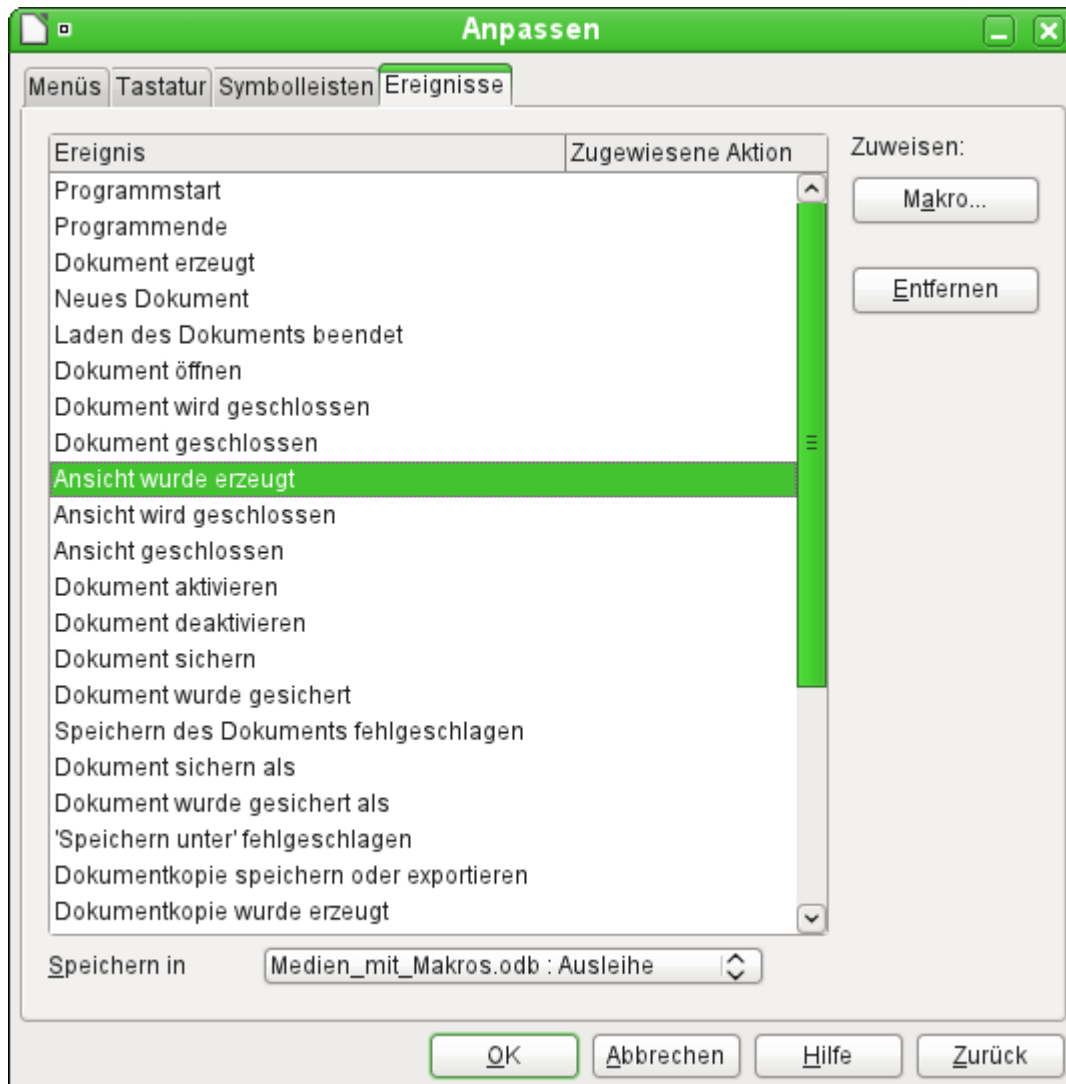
Der «direkte Weg» ist vor allem dann nicht möglich – auch nicht zu Testzwecken –, wenn eines der Objekte **thisComponent** (siehe den Abschnitt «[Zugriff auf das Formular](#)») oder **oEvent** (siehe den Abschnitt «[Zugriff auf Elemente eines Formulars](#)») benutzt wird.

Makros zuweisen

Damit ein Makro durch ein Ereignis gestartet werden kann, muss es zunächst definiert werden (siehe den einleitenden Abschnitt «[Allgemeines zu Makros](#)»). Dann kann es einem Ereignis zugewiesen werden. Dafür gibt es vor allem zwei Stellen.

Ereignisse eines Formulars beim Öffnen oder Schließen des Fensters

Maßnahmen, die beim Öffnen oder Schließen eines Formulars erledigt werden sollen, werden so registriert:

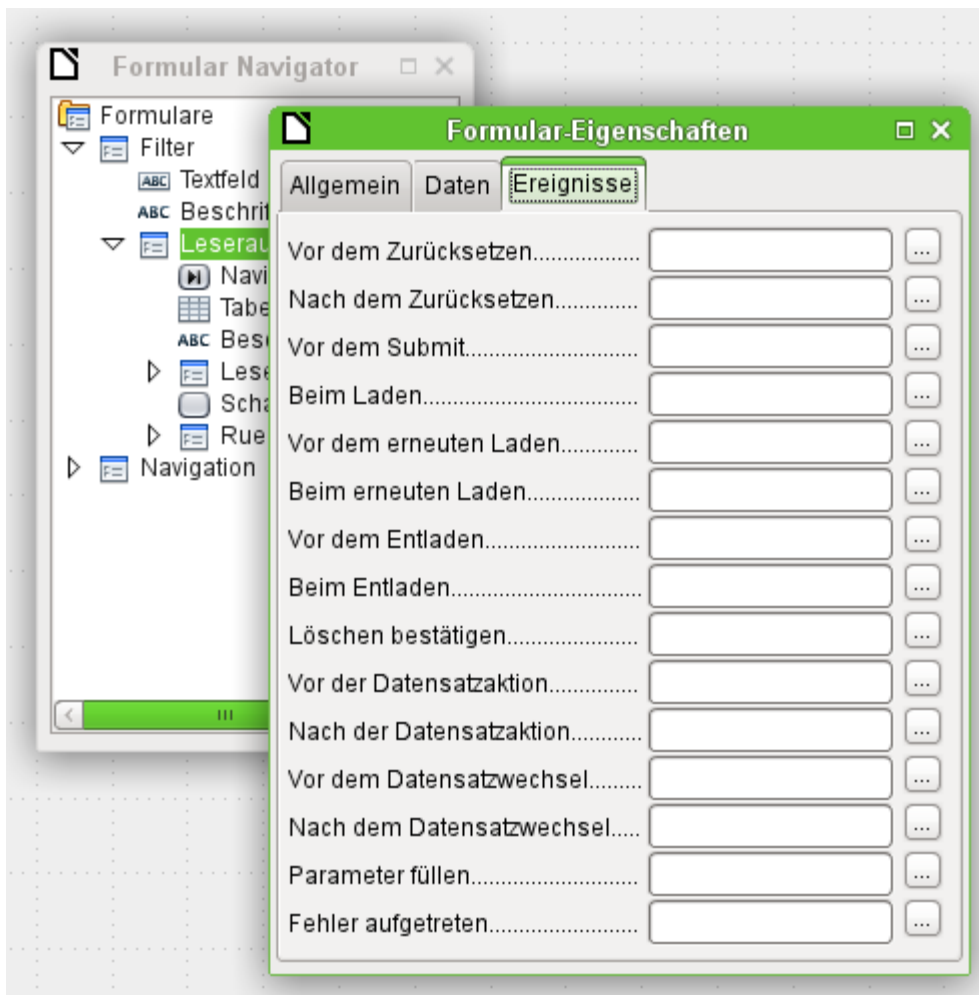


- Rufen Sie im Formularentwurf über **Extras → Anpassen** das Register **Ereignisse** auf.
- Wählen Sie das passende Ereignis aus. Bestimmte Makros lassen sich nur starten, wenn **Ansicht wurde erzeugt** gewählt ist. Andere Makros wie z.B. das Erzeugen eines Vollbild-Formulars kann über **Dokument öffnen** gestartet werden.
- Suchen Sie über die Schaltfläche **Makro** das dafür definierte Makro und bestätigen Sie diese Auswahl.
- Unter **Speichern in** ist das Formular anzugeben.

Dann kann diese Zuweisung mit **OK** bestätigt werden.

Ereignisse eines Formulars bei geöffnetem Fenster

Nachdem das Fenster für die gesamten Inhalte des Formulars geöffnet wurde, kann auf die einzelnen Elemente des Formulars zugegriffen werden. Hierzu gehören auch die dem Formular zugeordneten Formularelemente.



Die Formularelemente können, wie in obigem Bild, über den Formularnavigator angesteuert werden. Sie sind genauso gut über jedes einzelne Kontrollfeld der Formularoberfläche über das Kontextmenü des Kontrollfeldes zu erreichen.

Die unter **Formular-Eigenschaften** → **Ereignisse** aufgezeigten Ereignisse finden statt während das Formularfenster geöffnet ist. Sie können für jedes Formular oder Unterformular des Formularfensters separat ausgewählt werden.

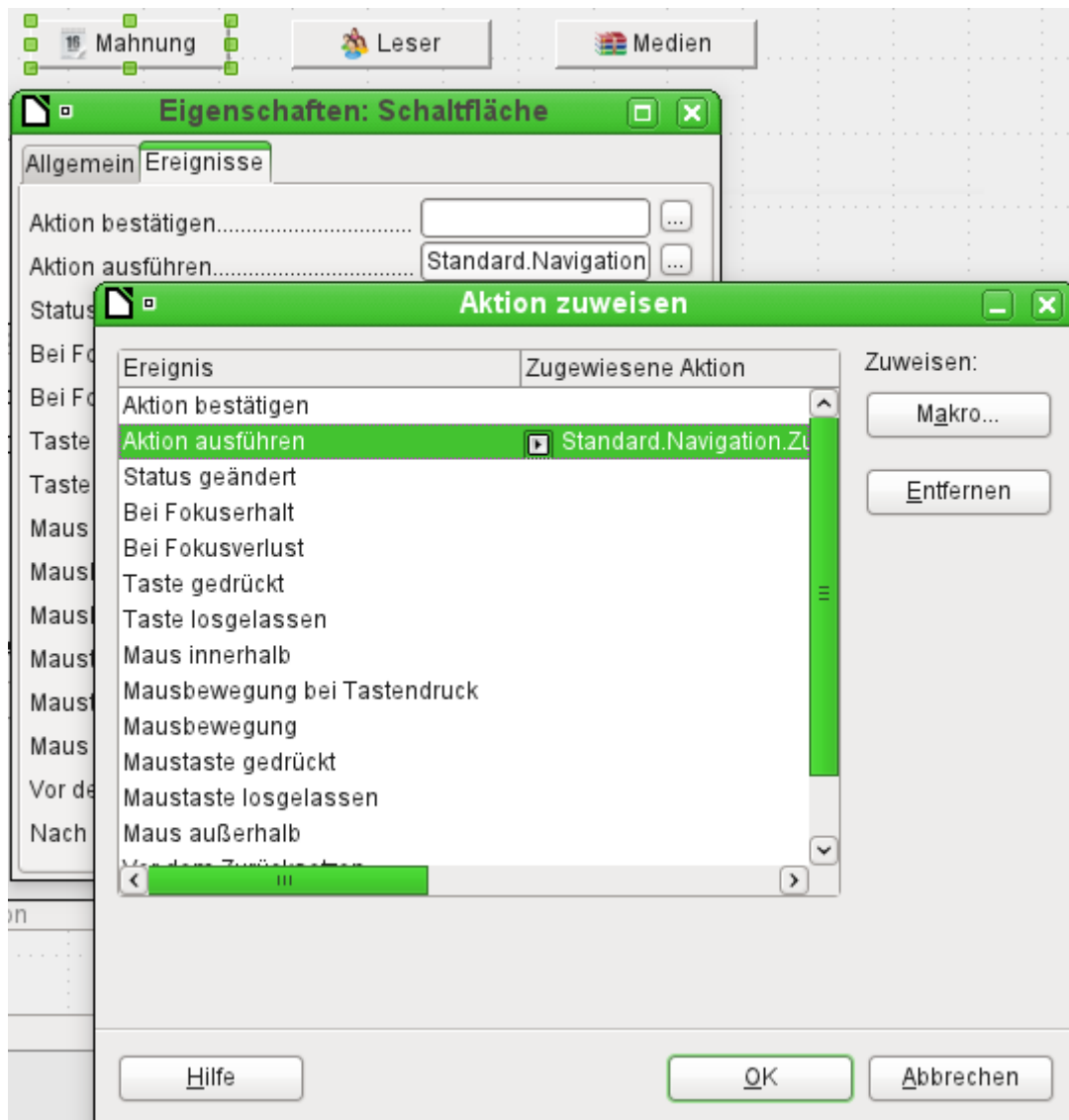
Hinweis

Der Gebrauch des Begriffes «Formular» ist bei Base leider nicht eindeutig. Der Begriff wird zum einen für das Fenster benutzt, das zur Eingabe von Daten geöffnet wird. Zum anderen wird der Begriff für das Element genutzt, das mit diesem Fenster eine bestimmte Datenquelle (Tabelle oder Abfrage) verbindet.

Es können auf einem Formularfenster sehr wohl mehrere Formulare mit unterschiedlichen Datenquelle untergebracht sein. Im Formularnavigator steht zuerst immer der Begriff «Formulare», dem dann in einem einfachen Formular lediglich ein Formular untergeordnet wird.

Ereignisse innerhalb eines Formulars

Alle anderen Makros werden bei den Eigenschaften von Teilformularen und Kontrollfeldern über das Register *Ereignisse* registriert.



- Öffnen Sie (sofern noch nicht geschehen) das Fenster mit den Eigenschaften des Kontrollfelds.
- Wählen Sie im Register **Ereignisse** das passende Ereignis aus.
 - Um die Datenquelle zu bearbeiten, gibt es vor allem die Ereignisse, die sich auf *Datensatz* oder *Aktualisieren* oder *Zurücksetzen* beziehen.
 - Zu Schaltflächen oder einer Auswahl bei Listen- oder Optionsfeldern gehört in erster Linie das Ereignis *Aktion ausführen*.
 - Alle anderen Ereignisse hängen vom Kontrollfeld und der gewünschten Maßnahme ab.
- Durch einen Klick auf den rechts stehenden Button ... wird das Fenster «Aktion zuweisen» geöffnet.
- Über die Schaltfläche **Makro** wird das dafür definierte Makro ausgewählt.

Über mehrfaches **OK** wird diese Zuweisung bestätigt.

Bestandteile von Makros

In diesem Abschnitt sollen einige Teile der Makro-Sprache erläutert werden, die in Base – vor allem bei Formularen – immer wieder benutzt werden. (Soweit möglich und sinnvoll, werden dabei die Beispiele der folgenden Abschnitte benutzt.)

Der «Rahmen» eines Makros

Die Definition eines Makros beginnt mit dem Typ des Makros – **SUB** oder **FUNCTION** – und endet mit **END SUB** bzw. **END FUNCTION**. Einem Makro, das einem Ereignis zugewiesen wird, können Argumente (Werte) übergeben werden; sinnvoll ist aber nur das Argument **oEvent**. Alle anderen Routinen, die von einem solchen Makro aufgerufen werden, können abhängig vom Zweck mit oder ohne Rückgabewert definiert werden und beliebig mit Argumenten versehen werden.

```
SUB Ausleihe_aktualisieren
END SUB

SUB Zu_Formular_von_Formular(oEvent AS OBJECT)
END SUB

FUNCTION Loeschen_bestatigen(oEvent AS OBJECT) AS BOOLEAN
    Loeschen_bestatigen = FALSE
END FUNCTION
```

Es ist hilfreich, diesen Rahmen sofort zu schreiben und den Inhalt anschließend einzufügen. Bitte vergessen Sie nicht, Kommentare zur Bedeutung des Makros nach dem Grundsatz «so viel wie nötig, so wenig wie möglich» einzufügen. Außerdem unterscheidet Basic nicht zwischen Groß- und Kleinschreibung. In der Praxis werden feststehende Begriffe wie **SUB** vorzugsweise groß geschrieben, während andere Bezeichner Groß- und Kleinbuchstaben mischen.

Variablen definieren

Im nächsten Schritt werden – am Anfang der Routine – mit der **DIM**-Anweisung die Variablen, die innerhalb der Routine vorkommen, mit dem jeweiligen Datentyp definiert. Basic selbst verlangt das nicht, sondern akzeptiert, dass während des Programmlaufs neue Variablen auftreten. Der Programmcode ist aber «sicherer», wenn die Variablen und vor allem die Datentypen festgelegt sind. Viele Programmierer verpflichten sich selbst dazu, indem sie Basic über **Option Explicit** gleich zu Beginn eines Moduls mitteilen: Erzeuge nicht automatisch irgendwelche Variablen, sondern nutze nur die, die ich auch vorher definiert habe.

```
DIM oDoc AS OBJECT
DIM oDrawpage AS OBJECT
DIM oForm AS OBJECT
DIM sName AS STRING
DIM bOKEnabled AS BOOLEAN
DIM iCounter AS INTEGER
DIM dBirthday AS DATE
```

Für die Namensvergabe stehen nur Buchstaben (A–Z oder a–z), Ziffern und der Unterstrich '_' zur Verfügung, aber keine Umlaute oder Sonderzeichen. (Unter Umständen ist das Leerzeichen zulässig. Sie sollten aber besser darauf verzichten.) Das erste Zeichen muss ein Buchstabe sein.

Üblich ist es, durch den ersten Buchstaben den Datentyp deutlich zu machen.¹⁸ Dann erkennt man auch mitten im Code den Typ der Variablen. Außerdem sind «sprechende Bezeichner» zu empfehlen, sodass die Bedeutung der Variablen schon durch den Namen erkannt werden kann.

Die Liste der möglichen Datentypen in Star-Basic steht im Anhang des Handbuches. An verschiedenen Stellen sind Unterschiede zwischen der Datenbank, von Basic und der LibreOffice-API zu beachten. Darauf wird bei den Beispielen hingewiesen.

Arrays definieren

Gerade für Datenbanken ist die Sammlung von mehreren Variablen in einem Datensatz von Bedeutung. Werden mehrere Variablen zusammen in einer gemeinsamen Variablen gespeichert, so wird dies als ein Array bezeichnet. Ein Array muss definiert werden, bevor Daten in das Array geschrieben werden können.

¹⁸ Die Kennzeichnung sollte eventuell noch verfeinert werden, da mit nur einem Buchstaben zwischen dem Datentyp «Double» und dem Datentyp «Date» bzw. «Single» und «String» nicht unterschieden werden kann.

```
DIM arDaten()
```

erzeugt eine leeres Array.

```
arDaten = array("Lisa", "Schmidt")
```

So wird ein Array auf eine bestimmte Größe von 2 Elementen festgelegt und gleichzeitig mit Daten versehen.

Über

```
print arDaten(0), arDaten(1)
```

werden die beiden definierten Inhalte auf dem Bildschirm ausgegeben. Die Zählung für die Felder beginnt hier mit 0.

```
DIM arDaten(2)
arDaten(0) = "Lisa"
arDaten(1) = "Schmidt"
arDaten(2) = "Köln"
```

Dies erstellt ein Array, in dem 3 Elemente beliebigen Typs gespeichert werden können, also z.B. ein Datensatz mit den Variablen «Lisa» «Schmidt» «Köln». Mehr passt leider in dieses Array nicht hinein. Sollen mehr Elemente gespeichert werden, so muss das Array vergrößert werden. Wird während der Laufzeit eines Makros allerdings die Größe eines Arrays einfach nur neu definiert, so ist das Array anschließend leer wie eben ein neues Array.

```
ReDIM Preserve arDaten(3)
arDaten(3) = "18.07.2003"
```

Durch den Zusatz **Preserve** werden die vorherigen Daten beibehalten, das Array also tatsächlich zusätzlich um die Datumseingabe, hier in Form eines Textes, erweitert.

Das oben aufgeführte Array kann leider nur einen einzelnen Satz an Daten speichern. Sollen stattdessen, wie in einer Tabelle einer Datenbank, mehrere Datensätze gespeichert werden, so muss dem Array zu Beginn eine zusätzliche Dimension hinzugefügt werden.

```
DIM arDaten(2,1)
arDaten(0,0) = "Lisa"
arDaten(1,0) = "Schmidt"
arDaten(2,0) = "Köln"
arDaten(0,1) = "Egon"
arDaten(1,1) = "Müller"
arDaten(2,1) = "Hamburg"
```

Auch hier gilt bei einer Erweiterung über das vorher definierte Maß hinaus, dass der Zusatz **Preserve** die vorher eingegebenen Daten mit übernimmt.

Zugriff auf das Formular

Das Formular liegt in dem momentan aktiven Dokument. Der Bereich, der dargestellt wird, wird als **drawpage** bezeichnet. Der Behälter, in dem alle Formulare aufbewahrt werden, heißt **forms** – im Formularnavigator ist dies sozusagen der oberste Begriff, an den dann sämtliche Formulare angehängt werden. Die o.g. Variablen erhalten auf diesem Weg ihre Werte:

```
oDoc = thisComponent
oDrawpage = oDoc.drawpage
oForm = oDrawpage.forms.getByNamed("Filter")
```

Das Formular, auf das zugegriffen werden soll, ist hier mit dem Namen "Filter" versehen. Dies ist der Name, der auch im Formularnavigator in der obersten Ebene sichtbar ist. (Standardmäßig erhält das erste Formular den Namen "MainForm".) Unterformulare liegen – hierarchisch angeordnet – innerhalb eines Formulars und können Schritt für Schritt erreicht werden:

```
DIM oSubForm AS OBJECT
DIM oSubSubForm AS OBJECT
oSubForm = oForm.getByNamed("Leserauswahl")
oSubSubForm = oSubForm.getByNamed("Leseranzeige")
```

Anstelle der Variablen in den «Zwischenstufen» kann man auch direkt zu einem bestimmten Formular gelangen. Ein Objekt der Zwischenstufen, das mehr als einmal verwendet wird, sollte selbstständig deklariert und zugewiesen werden. (Im folgenden Beispiel wird **oSubForm** nicht mehr benutzt.)

```
oForm = thisComponent.drawpage.forms.getByName("Filter")
oSubSubForm = oForm.getByName("Leserauswahl").getByName("Leseranzeige")
```

Hinweis

Sofern ein Name ausschließlich aus Buchstaben und Ziffern besteht (keine Umlaute, keine Leer- oder Sonderzeichen), kann der Name in der Zuweisung auch direkt verwendet werden:

```
oForm = thisComponent.drawpage.forms.Filter
oSubSubForm = oForm.Leserauswahl.Leseranzeige
```

Anders als bei Basic sonst üblich, ist bei solchen Namen auf Groß- und Kleinschreibung genau zu achten.

Einen anderen Zugang zum Formular ermöglicht das auslösende Ereignis für das Makro.

Startet ein Makro über ein Ereignis des Formulars, wie z. B. **Formular-Eigenschaften → Vor der Datensatzaktion**, so wird das Formular selbst folgendermaßen erreicht:

```
SUB MakrobeispielBerechne(oEvent AS OBJECT)
  oForm = oEvent.Source
  ...
END SUB
```

Startet ein Makro über ein Ereignis eines Formularfeldes, wie z. B. Eigenschaften: **Textfeld → Bei Fokusverlust**, so kann sowohl das Formularfeld als auch das Formular ermittelt werden:

```
SUB MakrobeispielBerechne(oEvent AS OBJECT)
  oFeld = oEvent.Source.Model
  oForm = oFeld.Parent
  ...
END SUB
```

Die Zugriffe über das Ereignis haben den Vorteil, dass kein Gedanke darüber verschwendet werden muss, ob es sich bei dem Formular um ein Hauptformular oder Unterformular handelt. Auch interessiert der Name des Formulars für die Funktionsweise des Makros nicht.

Zugriff auf Elemente eines Formulars

In gleicher Weise kann man auf die Elemente eines Formulars zugreifen: Deklarieren Sie eine entsprechende Variable als **object** und suchen Sie das betreffende Kontrollfeld innerhalb des Formulars:

```
DIM btnOK AS OBJECT ' Button »OK«
btnOK = oSubSubForm.getByName("Schaltfläche 1") ' aus dem Formular Leseranzeige
```

Dieser Weg funktioniert immer dann, wenn bekannt ist, mit welchem Element das Makro arbeiten soll. Wenn aber im ersten Schritt zu prüfen ist, welches Ereignis das Makro gestartet hat, ist der o.g. Weg über **oEvent** sinnvoll. Dann wird die Variable innerhalb des Makro-"Rahmens" deklariert und beim Start des Makros zugewiesen. Die Eigenschaft **Source** liefert immer dasjenige Element, das das Makro gestartet hat; die Eigenschaft **Model** beschreibt das Kontrollfeld im Einzelnen:

```
SUB Auswahl_bestatigen(oEvent AS OBJECT)
  DIM btnOK AS OBJECT
  btnOK = oEvent.Source.Model
END
```

Mit dem Objekt, das man auf diesem Weg erhält, werden die weiteren angestrebten Maßnahmen ausgeführt.

Bitte beachten Sie, dass auch Unterformulare als Bestandteile eines Formulars gelten.

Zugriff auf die Datenbank

Normalerweise wird der Zugriff auf die Datenbank über Formulare, Abfragen, Berichte oder die Serienbrief-Funktion geregelt, wie es in allen vorhergehenden Kapiteln beschrieben wurde. Wenn diese Möglichkeiten nicht genügen, kann ein Makro auch gezielt die Datenbank ansprechen, wofür es mehrere Wege gibt.

Die Verbindung zur Datenbank

Das einfachste Verfahren benutzt dieselbe Verbindung wie das Formular, wobei **oForm** wie oben bestimmt wird:

```
DIM oConnection AS OBJECT
oConnection = oForm.activeConnection()
```

Oder man holt die Datenquelle, also die Datenbank, durch das Dokument und benutzt die vorhandene Verbindung auch für das Makro:

```
DIM oDatasource AS OBJECT
DIM oConnection AS OBJECT
oDatasource = thisComponent.Parent.dataSource
oConnection = oDatasource.getConnection("", "")
```

Ein weiterer Weg stellt sicher, dass bei Bedarf die Verbindung zur Datenbank hergestellt wird:

```
DIM oDatasource AS OBJECT
DIM oConnection AS OBJECT
oDatasource = thisComponent.Parent.CurrentController
IF NOT (oDatasource.isConnected()) THEN oDatasource.connect()
oConnection = oDatasource.ActiveConnection()
```

Die **IF**-Bedingung bezieht sich hier nur auf eine Zeile. Deshalb ist **END IF** nicht erforderlich.

Wenn das Makro durch die Benutzeroberfläche – nicht aus einem Formlardokument heraus – gestartet werden soll, ist folgende Variante geeignet:

```
DIM oDatasource AS OBJECT
DIM oConnection AS OBJECT
oDatasource = thisDatabaseDocument.CurrentController
IF NOT (oDatasource.isConnected()) THEN oDatasource.connect()
oConnection = oDatasource.ActiveConnection()
```

Der Zugriff auf Datenbanken außerhalb der aktuellen Datenbank ist folgendermaßen möglich:

```
DIM oDatabaseContext AS OBJECT
DIM oDatasource AS OBJECT
DIM oConnection AS OBJECT
oDatabaseContext = createUnoService("com.sun.star.sdb.DatabaseContext")
oDatasource = oDatabaseContext.getByName("angemeldeter Name der Datenbank in LO")
oConnection = oDatasource.getConnection("", "")
```

Auch die Verbindung zu nicht in LO angemeldete Datenbanken ist möglich. Hier muss dann lediglich statt des angemeldeten Namens der Pfad zur Datenbank mit «file:///.../Datenbank.odt» angegeben werden.

Ergänzende Hinweise zur Datenbankverbindung stehen im Abschnitt «[Verbindung mit Datenbanken erzeugen](#)».

SQL-Befehle

Die Arbeit mit der Datenbank erfolgt über SQL-Befehle. Ein solcher muss also erstellt und an die Datenbank geschickt werden; je nach Art des Befehls wird das Ergebnis ausgewertet und weiter verarbeitet. Mit der Anweisung **createStatement** wird das Objekt dafür erzeugt:

```
DIM oSQL_Statement AS OBJECT ' das Objekt, das den SQL-Befehl ausführt
DIM stSql AS STRING          ' Text des eigentlichen SQL-Befehls
DIM oResult AS OBJECT        ' Ergebnis für executeQuery
DIM iResult AS INTEGER        ' Ergebnis für executeUpdate
oSQL_Statement = oConnection.createStatement()
```


Um *Daten abzufragen*, wird mit dem Befehl die Methode **executeQuery** aufgerufen und ausgeführt; das Ergebnis wird anschließend ausgewertet. Tabellennamen und Feldnamen werden üblicherweise in doppelte Anführungszeichen gesetzt. Diese müssen im Makro durch weitere doppelte Anführungszeichen maskiert werden, damit sie im Befehl erscheinen.

```
stSql = "SELECT * FROM ""Tabelle1"""  
oResult = oSQL_Statement.executeQuery(stSql)
```

Um *Daten zu ändern* – also für **INSERT**, **UPDATE** oder **DELETE** – oder um die *Struktur der Datenbank* zu beeinflussen, wird mit dem Befehl die Methode **executeUpdate** aufgerufen und ausgeführt. Je nach Art des Befehls und der Datenbank erhält man kein nutzbares Ergebnis (ausgedrückt durch die Zahl 0) oder die Anzahl der bearbeiteten Datensätze.

```
stSql = "DROP TABLE ""Suchtmp"" IF EXISTS"  
iResult = oSQL_Statement.executeUpdate(stSql)
```

Der Vollständigkeit halber sei noch ein Spezialfall erwähnt: Wenn **oSQL_Statement** unterschiedlich für **SELECT** oder für andere Zwecke benutzt wird, steht die Methode **execute** zur Verfügung. Diese benutzen wir nicht; wir verweisen dazu auf die API-Referenz.

Vorbereitete SQL-Befehle mit Parametern

In allen Fällen, in denen manuelle Eingaben der Benutzer in einen SQL-Befehl übernommen werden, ist es einfacher und sicherer, den Befehl nicht als lange Zeichenkette zu erstellen, sondern ihn vorzubereiten und mit Parametern zu benutzen. Das vereinfacht die Formatierung von Zahlen, Datumsangaben und auch Zeichenketten (die ständigen doppelten Anführungszeichen entfallen) und verhindert Datenverlust durch böswillige Eingaben.

Bei diesem Verfahren wird zunächst das Objekt für einen bestimmten SQL-Befehl erstellt und vorbereitet:

```
DIM oSQL_Statement AS OBJECT ' das Objekt, das den SQL-Befehl ausführt  
DIM stSql AS STRING ' Text des eigentlichen SQL-Befehls  
stSql = "UPDATE Verfasser " _  
    & "SET Nachname = ?, Vorname = ?" _  
    & "WHERE ID = ?"  
oSQL_Statement = oConnection.prepareStatement(stSql)
```

Das Objekt wird mit **prepareStatement** erzeugt, wobei der SQL-Befehl bereits bekannt sein muss. Jedes Fragezeichen markiert eine Stelle, an der später – vor der Ausführung des Befehls – ein konkreter Wert eingetragen wird. Durch das «Vorbereiten» des Befehls stellt sich die Datenbank darauf ein, welche Art von Angaben – in diesem Fall zwei Zeichenketten und eine Zahl – vorgesehen ist. Die verschiedenen Stellen werden durch die Position (ab 1 gezählt) unterschieden.

Anschließend werden mit passenden Anweisungen die Werte übergeben und danach der SQL-Befehl ausgeführt. Die Werte werden hier aus Kontrollfeldern des Formulars übernommen, können aber auch aus anderen Makro-Elementen stammen oder im Klartext angegeben werden:

```
oSQL_Statement.setString(1, oTextfeld1.Text) ' Text für den Nachnamen  
oSQL_Statement.setString(2, oTextfeld2.Text) ' Text für den Vornamen  
oSQL_Statement.setLong(3, oZahlenfeld1.Value) ' Wert für die betreffende ID  
iResult = oSQL_Statement.executeUpdate
```

Die vollständige Liste der Zuweisungen findet sich im Abschnitt «*Parameter für vorbereitete SQL-Befehle*».

Wer sich weiter über die Vorteile dieses Verfahrens informieren möchte, findet hier Erläuterungen:

- *SQL-Injection (Wikipedia)* (<http://de.wikipedia.org/wiki/SQL-Injection>)
- *Why use PreparedStatement (Java JDBC)* (<http://javarevisited.blogspot.de/2012/03/why-use-preparedstatement-in-java-jdbc.html>)
- *SQL-Befehle (Einführung in SQL)* (http://de.wikibooks.org/wiki/Einführung_in_SQL:_SQL-Befehle#Hinweis_f.C3.BCr_Programmierer:_Parameter_benutzen.21)

Datensätze lesen und benutzen

Es gibt – abhängig vom Zweck – mehrere Wege, um Informationen aus einer Datenbank in ein Makro zu übernehmen und weiter zu verarbeiten.

Bitte beachten Sie: Wenn hier von einem «Formular» gesprochen wird, kann es sich auch um ein Unterformular handeln. Es geht dann immer über dasjenige (Teil-) Formular, das mit einer bestimmten Datenmenge verbunden ist.

Mithilfe des Formulars

Der aktuelle Datensatz und seine Daten stehen immer über das Formular zur Verfügung, das die betreffende Datenmenge (Tabelle, Abfrage, SELECT) anzeigt. Dafür gibt es mehrere Methoden, die mit **get** und dem Datentyp bezeichnet sind, beispielsweise diese:

```
DIM ID AS LONG
DIM sName AS STRING
DIM dValue AS CURRENCY
DIM dEintritt AS NEW com.sun.star.util.Date
ID = oForm.getLong(1)
sName = oForm.getString(2)
dValue = oForm.getDouble(4)
dEintritt = oForm.getDate(7)
```

Bei allen diesen Methoden ist jeweils die Nummer der Spalte in der Datenmenge anzugeben – gezählt ab 1.

Hinweis

Bei allen Methoden, die mit Datenbanken arbeiten, wird ab 1 gezählt. Das gilt sowohl für Spalten als auch für Zeilen.

Möchte man anstelle der Spaltennummern mit den Spaltennamen der zugrundeliegenden Datenmenge (Tabelle, Abfrage, Ansicht (View)) arbeiten, kann man die Spaltennummer über die Methode **findColumn** ermitteln – hier ein Beispiel zum Auffinden der Spalte "Name":

```
DIM sName AS STRING
nName = oForm.findColumn("Name")
sName = oForm.getString(nName)
```

Man erhält immer einen Wert des Typs der Methode, wobei die folgenden Sonderfälle zu beachten sind.

- Es gibt keine Methode für Daten des Typs **Decimal**, **Currency** o.ä., also für kaufmännisch exakte Berechnungen. Da Basic automatisch die passende Konvertierung vornimmt, kann ersatzweise **getDouble** verwendet werden.
- Bei **getBoolean** ist zu beachten, wie in der Datenbank «Wahr» und «Falsch» definiert sind. Die «üblichen» Definitionen (logische Werte, 1 als «Wahr») werden richtig verarbeitet.
- Datumsangaben können nicht nur mit dem Datentyp **DATE** definiert werden, sondern auch (wie oben) als **util.Date**. Das erleichtert u.a. Lesen und Ändern von Jahr, Monat, Tag.
- Bei ganzen Zahlen sind Unterschiede der Datentypen zu beachten. Im obigen Beispiel wird **getLong** verwendet; auch die Basic-Variable ID muss den Datentyp **Long** erhalten, da dieser vom Umfang her mit **Integer** aus der Datenbank übereinstimmt.

Die vollständige Liste dieser Methoden findet sich im Abschnitt «[Datenzeilen bearbeiten](#)».

Tip

Sollen Werte aus einem Formular für direkte Weiterverarbeitung in SQL genutzt werden (z.B. für die Eingabe der Daten in eine andere Tabelle), so ist es wesentlich einfacher, nicht nach dem Typ der Felder zu fragen.

Das folgende Makro, an **Eigenschaften: Schaltfläche → Ereignisse → Aktion ausführen** gekoppelt, liest das erste Feld des Formulars aus – unabhängig von dem für die Weiterverarbeitung in Basic erforderlichen Typ.

```
SUB WerteAuslesen(oEvent AS OBJECT)
    DIM oForm AS OBJECT
    DIM stFeld1 AS STRING
    oForm = oEvent.Source.Model.Parent
    stFeld1 = oForm.getString(1)
END SUB
```

Werden die Felder über **getString()** ausgelesen, so werden die Formatierungen beibehalten, die für eine Weiterverarbeitung in SQL notwendig sind. Ein Datum, das in einem deutschsprachigen Formular als '08.03.15' dargestellt wird, wird so im Format '2015-03-08' ausgelesen und kann direkt in SQL weiter verarbeitet werden.

Die Auslesung in dem dem Typ entsprechenden Format ist nur erforderlich, wenn im Makro Werte weiter verarbeitet, z.B. mit ihnen gerechnet werden soll.

Ergebnis einer Abfrage

In gleicher Weise kann die Ergebnismenge einer Abfrage benutzt werden. Im Abschnitt «**SQL-Befehle**» steht die Variable **oResult** für diese Ergebnismenge, die üblicherweise so oder ähnlich ausgelesen wird:

```
WHILE oResult.next          ' einen Datensatz nach dem anderen verarbeiten
    rem übernimm die benötigten Werte in Variablen
    stVar = oResult.getString(1)
    inVar = oResult.getLong(2)
    boVar = oResult.getBoolean(3)
    rem mach etwas mit diesen Werten
WEND
```

Je nach Art des SQL-Befehls, dem erwarteten Ergebnis und dem Zweck kann vor allem die **WHILE**-Schleife verkürzt werden oder sogar entfallen. Aber grundsätzlich wird eine Ergebnismenge immer nach diesem Schema ausgewertet.

Soll nur der erste Datensatz ausgewertet werden, so wird mit

```
oResult.next
```

zuerst die Zeile auf diesen Datensatz bewegt und dann mit

```
stVar = oResult.getString(1)
```

z.B. der Inhalt des ersten Datenfeldes gelesen. Die Schleife entfällt hier.

Die Abfrage zu dem obigen Beispiel hat in der ersten Spalte einen Text, in der zweiten Spalte einen Integer-Zahlenwert (**Integer** aus der Datenbank entspricht **Long** in Basic) und in der dritten Spalte ein Ja/Nein-Feld. Die Felder werden durch den entsprechenden Indexwert angesprochen. Der Index für die Felder beginnt hier, im Gegensatz zu der sonstigen Zählung bei Arrays, mit dem Wert '1'.

In dem so erstellten Ergebnis ist allerdings keine Navigation möglich. Nur einzelne Schritte zum nächsten Datensatz sind erlaubt. Um innerhalb der Datensätze navigieren zu können, muss der **ResultSetType** bei der Erstellung der Abfrage bekannt sein. Hierauf wird über

```
oSQL_Anweisung.ResultSetType = 1004
```

oder

```
oSQL_Anweisung.ResultSetType = 1005
```

zugegriffen. Der Typ **1004 - SCROLL_INTENSIVE** erlaubt eine beliebige Navigation. Allerdings bleibt eine Änderung an den Originaldaten während des Auslesens unbemerkt. Der Typ **1005 -**

SCROLL_SENSITIVE berücksichtigt zusätzlich gegebenenfalls Änderungen an den Originaldaten, die das Abfrageergebnis beeinflussen könnten.

Die Anzahl der Zeilen, die die Ergebnismenge enthält, kann nur nach Wahl der entsprechenden Typen so bestimmt werden:

```
DIM iResult AS LONG
IF oResult.last          ' gehe zum letzten Datensatz, sofern möglich
    iResult = oResult.getRow ' die laufende Nummer ist die Anzahl
ELSE
    iResult = 0
END IF
```

Mithilfe eines Kontrollfelds

Wenn ein Kontrollfeld mit einer Datenmenge verbunden ist, kann der Wert auch direkt ausgelesen werden, wie es im nächsten Abschnitt beschrieben wird. Das ist aber teilweise mit Problemen verbunden. Sicherer ist – neben dem Verfahren *«Mithilfe des Formulars»* – der folgende Weg, der für verschiedene Kontrollfelder gezeigt wird:

```
sValue = oTextField.BoundField.Text          ' Beispiel für ein Textfeld
nValue = oNumericField.BoundField.Value      ' Beispiel für ein numerisches Feld
dValue = oDateField.BoundField.Date          ' Beispiel für ein Datumsfeld
```

BoundField stellt dabei die Verbindung her zwischen dem (sichtbaren) Kontrollfeld und dem eigentlichen Inhalt der Datenmenge.

Datensätze wechseln und bestimmte Datensätze ansteuern

Im vorletzten Beispiel wurde mit der Methode **Next** von einer Zeile der Ergebnismenge zur nächsten gegangen. In gleicher Weise gibt es weitere Maßnahmen und Prüfungen, und zwar sowohl für die Daten eines Formulars – angedeutet durch die Variable **oForm** – als auch für eine Ergebnismenge. Beispielsweise kann man beim Verfahren *«Automatisches Aktualisieren von Formularen»* den vorher aktuellen Datensatz wieder markieren:

```
DIM loRow AS LONG
loRow = oForm.getRow() ' notiere die aktuelle Zeilennummer
oForm.reload()         ' lade die Datenmenge neu
oForm.absolute(loRow)  ' gehe wieder zu der notierten Zeilennummer
```

Im Abschnitt *«In einer Datenmenge navigieren»* stehen alle dazu passenden Methoden.

Datensätze bearbeiten – neu anlegen, ändern, löschen

Um Datensätze zu bearbeiten, müssen mehrere Teile zusammenpassen: Eine Information muss vom Anwender in das Kontrollfeld gebracht werden; das geschieht durch die Tastatureingabe. Anschließend muss die Datenmenge «dahinter» diese Änderung zur Kenntnis nehmen; das geschieht durch das Verlassen eines Feldes und den Wechsel zum nächsten Feld. Und schließlich muss die Datenbank selbst die Änderung erfahren; das erfolgt durch den Wechsel von einem Datensatz zu einem anderen.

Bei der Arbeit mit einem Makro müssen ebenfalls diese Teilschritte beachtet werden. Wenn einer fehlt oder falsch ausgeführt wird, gehen Änderungen verloren und «landen» nicht in der Datenbank. In erster Linie muss die Änderung nicht in der Anzeige des Kontrollfelds erscheinen, sondern in der Datenmenge. Es ist deshalb sinnlos, die Eigenschaft **Text** des Kontrollfelds zu ändern.

Bitte beachten Sie, dass nur Datenmengen vom Typ «Tabelle» problemlos geändert werden können. Bei anderen Datenmengen ist dies nur unter besonderen Bedingungen möglich.

Inhalt eines Kontrollfelds ändern

Wenn es um die Änderung eines einzelnen Wertes geht, wird das über die Eigenschaft **BoundField** des Kontrollfelds mit einer passenden Methode erledigt. Anschließend muss nur noch die Änderung an die Datenbank weitergegeben werden. Beispiel für ein Datumsfeld, in das das aktuelle Datum eingetragen werden soll:

```

DIM unoDate AS NEW com.sun.star.util.Date
unoDate.Year = Year(Date)
unoDate.Month = Month(Date)
unoDate.Day = Day(Date)
oDateField.BoundField.updateDate( unoDate )
oForm.updateRow() ' Weitergabe der Änderung an die Datenbank

```

Für **BoundField** wird diejenige der **updateXxx**-Methoden aufgerufen, die zum Datentyp des Feldes passt – hier geht es um einen **Date**-Wert. Als Argument wird der gewünschte Wert übergeben – hier das aktuelle Datum, konvertiert in die vom Makro benötigte Schreibweise. Die entsprechende Erstellung des Datums kann auch durch die Formel **CDateToUnoDate** erreicht werden:

```

oDateField.BoundField.updateDate( CDateToUnoDate(NOW()) )
oForm.updateRow() ' Weitergabe der Änderung an die Datenbank

```

Zeile einer Datenmenge ändern

Wenn mehrere Werte in einer Zeile geändert werden sollen, ist der vorstehende Weg ungeeignet. Zum einen müsste für jeden Wert ein Kontrollfeld existieren, was oft nicht gewünscht oder sinnvoll ist. Zum anderen muss man sich für jedes dieser Felder ein Objekt «holen». Der einfache und direkte Weg geht über das Formular, beispielsweise so:

```

DIM unoDate AS NEW com.sun.star.util.Date
unoDate.Year = Year(Date)
unoDate.Month = Month(Date)
unoDate.Day = Day(Date)
oForm.updateDate(3, unoDate )
oForm.updateString(4, "ein Text")
oForm.updateDouble(6, 3.14)
oForm.updateInt(7, 16)
oForm.updateRow()

```

Für jede Spalte der Datenmenge wird die zum Datentyp passende **updateXxx**-Methode aufgerufen. Als Argumente werden die Nummer der Spalte (ab 1 gezählt) und der jeweils gewünschte Wert übergeben. Anschließend muss nur noch die Änderung an die Datenbank weitergegeben werden.

Zeilen anlegen, ändern, löschen

Die genannten **Änderungen** beziehen sich immer auf die aktuelle Zeile der Datenmenge des Formulars. Unter Umständen muss vorher eine der Methoden aus «*In einer Datenmenge navigieren*» aufgerufen werden. Es werden also folgende Maßnahmen benötigt:

1. Wähle den aktuellen Datensatz.
2. Ändere die gewünschten Werte, wie im vorigen Abschnitt beschrieben.
3. Bestätige die Änderungen mit folgendem Befehl:

```
oForm.updateRow()
```
4. Als Sonderfall ist es auch möglich, die Änderungen zu verwerfen und den vorherigen Zustand wiederherzustellen:

```
oForm.cancelRowUpdates()
```

Für einen **neuen Datensatz** gibt es eine spezielle Methode (vergleichbar mit dem Wechsel in eine neue Zeile im Tabellenkontrollfeld). Es werden also folgende Maßnahmen benötigt:

1. Berechne einen neuen Datensatz vor:

```
oForm.moveToInsertRow()
```
2. Trage alle vorgesehenen und benötigten Werte ein. Dies geht ebenfalls mit den **updateXxx**-Methoden, wie im vorigen Abschnitt beschrieben.
3. Bestätige die Neuaufnahme mit folgendem Befehl:

```
oForm.insertRow()
```
4. Die Neuaufnahme kann nicht einfach rückgängig gemacht werden. Stattdessen ist die soeben neu angelegte Zeile wieder zu löschen.

Für das **Löschen** eines Datensatzes gibt es einen einfachen Befehl; es sind also folgende Maßnahmen nötig:

1. Wähle – wie für eine Änderung – den gewünschten Datensatz und mache ihn zum aktuellen.
2. Bestätige die Löschung mit folgendem Befehl:

```
oForm.deleteRow()
```

Tip

Damit eine Änderung in die Datenbank übernommen wird, ist sie durch **updateRow** bzw. **insertRow** ausdrücklich zu bestätigen. Während beim Betätigen des Speicher-Buttons die passende Funktion automatisch ermittelt wird, muss vor dem Abspeichern ermittelt werden, ob der Datensatz neu ist (**Insert**) oder ein bestehender Datensatz bearbeitet wurde (**Update**).

```
IF oForm.isNew THEN
    oForm.insertRow()
ELSE
    oForm.updateRow()
END IF
```

Kontrollfelder prüfen und ändern

Neben dem Inhalt, der aus der Datenmenge kommt, können viele weitere Informationen zu einem Kontrollfeld gelesen, verarbeitet und geändert werden. Das betrifft vor allem die Eigenschaften, die im Kapitel «Formulare» aufgeführt werden. Eine Übersicht steht im Abschnitt «*Eigenschaften bei Formularen und Kontrollfeldern*».

In mehreren Beispielen des Abschnitts «*Bedienbarkeit verbessern*» wird die Zusatzinformation eines Feldes benutzt:

```
DIM stTag AS STRING
stTag = oEvent.Source.Model.Tag
```

Die Eigenschaft **Text** kann – wie im vorigen Abschnitt erläutert – nur dann sinnvoll geändert werden, wenn das Feld nicht mit einer Datenmenge verbunden ist. Aber andere Eigenschaften, die «eigentlich» bei der Formulardefinition festgelegt werden, können zur Laufzeit angepasst werden. Beispielsweise kann in einem Beschriftungsfeld die Textfarbe gewechselt werden, wenn statt einer Meldung ein Hinweis oder eine Warnung angezeigt werden soll:

```
SUB showWarning(oField AS OBJECT, iType AS INTEGER)
    SELECT CASE iType
        CASE 1
            oField.TextColor = RGB(0,0,255) ' 1 = blau
        CASE 2
            oField.TextColor = RGB(255,0,0) ' 2 = rot
        CASE ELSE
            oField.TextColor = RGB(0,255,0) ' 0 = grün (weder 1 noch 2)
    END SELECT
END SUB
```

Englische Bezeichner in Makros

Während der Formular-Designer in der deutschen Version auch deutsche Bezeichnungen für die Eigenschaften und den Datenzugriff verwendet, müssen in Basic englische Begriffe verwendet werden. Diese sind in den folgenden Übersichten aufgeführt.

Eigenschaften, die üblicherweise nur in der Formular-Definition festgelegt werden, stehen nicht in den Übersichten. Gleiches gilt für Methoden (Funktionen und Prozeduren), die nur selten verwendet werden oder für die kompliziertere Erklärungen nötig wären.

Die Übersichten nennen folgende Angaben:

- *Name* Bezeichnung der Eigenschaft oder Methode im Makro-Code

- **Datentyp** Einer der Datentypen von Basic
Bei Funktionen ist der Typ des Rückgabewerts angegeben; bei Prozeduren entfällt diese Angabe.
- **L/S** Hinweis darauf, wie der Wert der Eigenschaft verwendet wird:
 - L** nur Lesen
 - S** nur Schreiben (Ändern)
 - (L)** Lesen möglich, aber für weitere Verarbeitung ungeeignet
 - (S)** Schreiben möglich, aber nicht sinnvoll
 - L+S** geeignet für Lesen und Schreiben

Weitere Informationen finden sich vor allem in der [API-Referenz](#) mit Suche nach der englischen Bezeichnung des Kontrollfelds. Gut geeignet, um herauszufinden, welche Eigenschaften und Methoden denn eigentlich bei einem Element zur Verfügung stehen, ist auch das Tool [Xray](#).

```
SUB Main(oEvent)
  Xray(oEvent)
END SUB
```

Hiermit wird die Erweiterung Xray aus dem Aufruf heraus gestartet.

Eigenschaften bei Formularen und Kontrollfeldern

Das «Modell» eines Kontrollfelds beschreibt seine Eigenschaften. Je nach Situation kann der Wert einer Eigenschaft nur gelesen und nur geändert werden. Die Reihenfolge orientiert sich an den Aufstellungen «Eigenschaften der Kontrollfelder» im Kapitel «[Formular](#)».

Schrift

In jedem Kontrollfeld, das Text anzeigt, können die Eigenschaften der Schrift angepasst werden.

Name	Datentyp	L/S	Eigenschaft
FontName	string	L+S	Schriftart.
FontHeight	single	L+S	Schriftgröße.
FontWeight	single	L+S	Schriftstärke.
FontSlant	integer	L+S	Art der Schrägstellung.
FontUnderline	integer	L+S	Art der Unterstreichung.
FontStrikeout	integer	L+S	Art des Durchstreichens.

Formular

Englische Bezeichnung: *Form*

Name	Datentyp	L/S	Eigenschaft
ApplyFilter	boolean	L+S	Filter aktiviert.
Filter	string	L+S	Aktueller Filter für die Datensätze.
FetchSize	long	L+S	Anzahl der Datensätze, die «am Stück» geladen werden.
Row	long	L	Nummer der aktuellen Zeile.
RowCount	long	L	Anzahl der Datensätze.

Einheitlich für alle Arten von Kontrollfeld

Englische Bezeichnung: *Control* – siehe auch *FormComponent*

Name	Datentyp	L/S	Eigenschaft
Name	string	L+(S)	Bezeichnung für das Feld.

Name	Datentyp	L/S	Eigenschaft
Enabled	boolean	L+S	Aktiviert: Feld kann ausgewählt werden.
EnableVisible	boolean	L+S	Sichtbar: Feld wird dargestellt.
ReadOnly	boolean	L+S	Nur lesen: Inhalt kann nicht geändert werden.
TabStop	boolean	L+S	Feld ist in der Tabulator-Reihenfolge erreichbar.
Align	integer	L+S	Horizontale Ausrichtung: 0 = links, 1 = zentriert, 2 = rechts
BackgroundColor	long	L+S	Hintergrundfarbe.
Tag	string	L+S	Zusatzinformation.
HelpText	string	L+S	Hilfetext als «Tooltip».

Einheitlich für viele Arten von Kontrollfeld

Name	Datentyp	L/S	Eigenschaft
Text	string	(L+S)	Inhalt des Feldes aus der Anzeige. Bei Textfeldern nach dem Lesen auch zur weiteren Verarbeitung geeignet, andernfalls nur in Ausnahmefällen.
Spin	boolean	L+S	Drehfeld eingeblendet (bei formatierten Feldern).
TextColor	long	L+S	Textfarbe.
DataField	string	L	Name des Feldes aus der Datenmenge
BoundField	object	L	Objekt, das die Verbindung zur Datenmenge herstellt und vor allem dem Zugriff auf den Feldinhalt dient.

Textfeld – weitere Angaben

Englische Bezeichnung: *TextField*

Name	Datentyp	L/S	Eigenschaft
String	string	L+S	Inhalt des Feldes aus der Anzeige.
MaxTextLen	integer	L+S	Maximale Textlänge.
DefaultText	string	L+S	Standardtext.
MultiLine	boolean	L+S	Mehrzeilig oder einzeilig.
EchoChar	(integer)	L+S	Zeichen für Kennwörter (Passwort-Eingabe verstecken).

Numerisches Feld

Englische Bezeichnung: *NumericField*

Name	Datentyp	L/S	Eigenschaft
ValueMin	double	L+S	Minimalwert zur Eingabe.
ValueMax	double	L+S	Maximalwert zur Eingabe.
Value	double	L+(S)	Aktueller Wert nicht für Werte aus der Datenmenge verwenden.

Name	Datentyp	L/S	Eigenschaft
ValueStep	double	L+S	Intervall bei Verwendung mit Mausrad oder Drehfeld.
DefaultValue	double	L+S	Standardwert.
DecimalAccuracy	integer	L+S	Nachkommastellen.
ShowThousandsSeparator	boolean	L+S	Tausender-Trennzeichen anzeigen.

Datumsfeld

Englische Bezeichnung: *DateField*

Datumswerte werden als Datentyp **long** definiert und im ISO-Format YYYYMMDD angezeigt, also 20120304 für den 04.03.2012. Zur Verwendung dieses Typs zusammen mit **getDate** und **updateDate** sowie dem Typ **com.sun.star.util.Date** verweisen wir auf die Beispiele.

Name	Datentyp	Datentyp ab LO 4.1.1	L/S	Eigenschaft
DateMin	long	com.sun.star.util.Date	L+S	Minimalwert zur Eingabe.
DateMax	long	com.sun.star.util.Date	L+S	Maximalwert zur Eingabe.
Date	long	com.sun.star.util.Date	L+(S)	Aktueller Wert nicht für Werte aus der Datenmenge verwenden.
DateFormat	integer		L+S	Datumsformat nach Festlegung des Betriebssystems: 0 = kurze Datumsangabe (einfach) 1 = kurze Datumsangabe tt.mm.jj (Jahr zweistellig) 2 = kurze Datumsangabe tt.mm.jjjj (Jahr vierstellig) 3 = lange Datumsangabe (mit Wochentag und Monatsnamen) Weitere Möglichkeiten sind der Formulardefinition oder der API-Referenz zu entnehmen.
DefaultDate	long	com.sun.star.util.Date	L+S	Standardwert.
DropDown	boolean		L+S	Aufklappbaren Monatskalender anzeigen.

Zeitfeld

Englische Bezeichnung: *TimeField*

Auch Zeitwerte werden als Datentyp **long** definiert.

Name	Datentyp	Datentyp ab LO 4.1.1	L/S	Eigenschaft
TimeMin	long	com.sun.star.util.Time	L+S	Minimalwert zur Eingabe.
TimeMax	long	com.sun.star.util.Time	L+S	Maximalwert zur Eingabe.

Name	Datentyp	Datentyp ab LO 4.1.1	L/S	Eigenschaft
Time	long	com.sun.star.util.Time	L+(S)	Aktueller Wert nicht für Werte aus der Datenmenge verwenden.
TimeFormat	integer		L+S	Zeitformat: 0 = kurz als hh:mm (Stunde, Minute, 24 Stunden) 1 = lang als hh:mm:ss (dazu Sekunden, 24 Stunden) 2 = kurz als hh:mm (12 Stunden AM/PM) 3 = lang als hh:mm:ss (12 Stunden AM/PM) 4 = als kurze Angabe einer Dauer 5 = als lange Angabe einer Dauer
DefaultTime	long	com.sun.star.util.Time	L+S	Standardwert.

Währungsfeld

Englische Bezeichnung: *CurrencyField*

Ein Währungsfeld ist ein numerisches Feld mit den folgenden zusätzlichen Möglichkeiten.

Name	Datentyp	L/S	Eigenschaft
CurrencySymbol	string	L+S	Währungssymbol (nur zur Anzeige).
PrependCurrencySymbol	boolean	L+S	Anzeige des Symbols vor der Zahl.

Formatiertes Feld

Englische Bezeichnung: *FormattedControl*

Ein formatiertes Feld wird wahlweise für Zahlen, Währungen oder Datum/Zeit verwendet. Sehr viele der bisher genannten Eigenschaften gibt es auch hier, aber mit anderer Bezeichnung.

Name	Datentyp	L/S	Eigenschaft
CurrentValue	variant	L	Aktueller Wert des Inhalts; der konkrete Datentyp hängt vom Inhalt des Feldes und dem Format ab.
EffectiveValue		L+(S)	
EffectiveMin	double	L+S	Minimalwert zur Eingabe.
EffectiveMax	double	L+S	Maximalwert zur Eingabe.
EffectiveDefault	variant	L+S	Standardwert.
FormatKey	long	L+(S)	Format für Anzeige und Eingabe. Es gibt kein einfaches Verfahren, das Format durch ein Makro zu ändern.
EnforceFormat	boolean	L+S	Formatüberprüfung: Bereits während der Eingabe sind nur zulässige Zeichen und Kombinationen möglich.

Listenfeld

Englische Bezeichnung: *ListBox*

Der Lese- und Schreibzugriff auf den Wert, der hinter der ausgewählten Zeile steht, ist etwas umständlich, aber möglich.

Name	Datentyp	L/S	Eigenschaft
ListSource	array of string	L+S	Datenquelle: Herkunft der Listeneinträge oder Name der Datenmenge, die die Einträge liefert.
ListSourceType	integer	L+S	Art der Datenquelle: 0 = Werteliste 1 = Tabelle 2 = Abfrage 3 = Ergebnismenge eines SQL-Befehls 4 = Ergebnis eines Datenbank-Befehls 5 = Feldnamen einer Datenbank-Tabelle
StringItemList	array of string	L	Listeneinträge, die zur Auswahl zur Verfügung stehen.
ItemCount	integer	L	Anzahl der vorhandenen Listeneinträge.
ValueItemList	array of string	L	Liste der Werte, die über das Formular an die Tabelle weitergegeben werden.
DropDown	boolean	L+S	Aufklappbar.
LineCount	integer	L+S	Anzahl der angezeigten Zeilen im aufgeklappten Zustand.
MultiSelection	boolean	L+S	Mehrfachselektion vorgesehen.
SelectedItems	array of integer	L+S	Liste der ausgewählten Einträge, und zwar als Liste der Positionen in der Liste aller Einträge.

Das (erste) ausgewählte Element aus dem Listenfeld erhält man auf diesem Weg:

```
oControl = oForm.GetByName("Name des Listenfelds")
sEintrag = oControl.ValueItemList( oControl.SelectedItems(0) )
```

Hinweis

Seit LO 4.1 wird direkt der Wert ermittelt, der bei einem Listenfeld an die Datenbank weitergegeben wird.

```
oControl = oForm.GetByName("Name des Listenfelds")
iD = oControl.GetCurrentValue()
```

Mit `GetCurrentValue()` wird also immer der Wert ausgegeben, der auch tatsächlich in der Tabelle der Datenbank abgespeichert wird. Dies ist beim Listenfeld von dem hiermit verknüpften gebundenen Feld (`BoundField`) abhängig.

Bis einschließlich LO 4.0 wurde hier immer der angezeigte Inhalt, nicht aber der an die darunterliegende Tabelle weitergegebene Wert wiedergegeben.

Soll für die Einschränkung einer Auswahlmöglichkeit die Abfrage für ein Listenfeld ausgetauscht werden, so ist dabei zu beachten, dass es sich bei dem Eintrag um ein «array of string» handelt:

```
SUB Listenfeldfilter
  DIM stSql(0) AS STRING
  DIM oDoc AS OBJECT
  DIM oDrawpage AS OBJECT
  DIM oForm AS OBJECT
  DIM oFeld AS OBJECT
  oDoc = thisComponent
  oDrawpage = oDoc.drawpage
  oForm = oDrawpage.forms.GetByName("MainForm")
  oFeld = oForm.GetByName("Listenfeld")
  stSql(0) = "SELECT ""Name"", ""ID"" FROM ""Filter_Name"" ORDER BY ""Name""
  oFeld.ListSource = stSql
  oFeld.refresh
END SUB
```

Kombinationsfeld

Englische Bezeichnung: *ComboBox*

Trotz ähnlicher Funktionalität wie beim Listenfeld weichen die Eigenschaften teilweise ab.

Hier verweisen wir ergänzend auf das Beispiel «*Kombinationsfelder als Listenfelder mit Eingabemöglichkeit*».

Name	Datentyp	L/S	Eigenschaft
Autocomplete	boolean	L+S	Automatisch füllen.
StringItemList	array of string	L+S	Listeneinträge, die zur Auswahl zur Verfügung stehen.
ItemCount	integer	L	Anzahl der vorhandenen Listeneinträge.
DropDown	boolean	L+S	Aufklappbar.
LineCount	integer	L+S	Anzahl der angezeigten Zeilen im aufgeklappten Zustand.
Text	string	L+S	Aktuell angezeigter Text.
DefaultText	string	L+S	Standardeintrag.
ListSource	string	L+S	Name der Datenquelle, die die Listeneinträge liefert.
ListSourceType	integer	L+S	Art der Datenquelle; gleiche Möglichkeiten wie beim Listenfeld (nur die Auswahl «Werteliste» wird ignoriert).

Markierfeld, Optionsfeld

Englische Bezeichnungen: *CheckBox* (Markierfeld) bzw. *RadioButton* (Optionsfeld; auch «Option Button» möglich)

Name	Datentyp	L/S	Eigenschaft
Label	string	L+S	Titel (Beschriftung)
State	short	L+S	Status 0 = nicht ausgewählt 1 = ausgewählt 2 = unbestimmt
MultiLine	boolean	L+S	Wortumbruch (bei zu langem Text).
RefValue	string	L+S	Referenzwert

Maskiertes Feld

Englische Bezeichnung: *PatternField*

Neben den Eigenschaften für «einfache» Textfelder sind folgende interessant.

Name	Datentyp	L/S	Eigenschaft
EditMask	string	L+S	Eingabemaske.
LiteralMask	string	L+S	Zeichenmaske.
StrictFormat	boolean	L+S	Formatüberprüfung bereits während der Eingabe.

Tabellenkontrollfeld

Englische Bezeichnung: *GridControl*

Name	Datentyp	L/S	Eigenschaft
Count	long	L	Anzahl der Spalten.
ElementNames	array of string	L	Liste der Spaltennamen.
HasNavigationBar	boolean	L+S	Navigationsleiste vorhanden.
RowHeight	long	L+S	Zeilenhöhe.

Beschriftungsfeld

Englische Bezeichnung: *FixedText* – auch *Label* ist üblich

Name	Datentyp	L/S	Eigenschaft
Label	string	L+S	Der angezeigte Text.
MultiLine	boolean	L+S	Wortumbruch (bei zu langem Text).

Gruppierungsrahmen

Englische Bezeichnung: *GroupBox*

Keine Eigenschaft dieses Kontrollfelds wird üblicherweise durch Makros bearbeitet. Wichtig ist der Status der einzelnen Optionsfelder.

Schaltfläche

Englische Bezeichnungen: *CommandButton* – für die grafische Schaltfläche *ImageButton*

Name	Datentyp	L/S	Eigenschaft
Label	string	L+S	Titel – Text der Beschriftung.
State	short	L+S	Standardstatus «ausgewählt» bei «Umschalten».
MultiLine	boolean	L+S	Wortumbruch (bei zu langem Text).
DefaultButton	boolean	L+S	Standardschaltfläche

Navigationsleiste

Englische Bezeichnung: *NavigationBar*

Weitere Eigenschaften und Methoden, die mit der Navigation zusammenhängen – z.B. Filter und das Ändern des Datensatzzeigers –, werden über das Formular geregelt.

Name	Datentyp	L/S	Eigenschaft
IconSize	short	L+S	Symbolgröße.
ShowPosition	boolean	L+S	Positionierung anzeigen und eingeben.
ShowNavigation	boolean	L+S	Navigation ermöglichen.
ShowRecordActions	boolean	L+S	Datensatzaktionen ermöglichen.
ShowFilterSort	boolean	L+S	Filter und Sortierung ermöglichen.

Methoden bei Formularen und Kontrollfeldern

Die Datentypen der Parameter werden durch Kürzel angedeutet:

- c Nummer der Spalte des gewünschten Feldes in der Datenmenge – ab 1 gezählt
- n numerischer Wert – je nach Situation als ganze Zahl oder als Dezimalzahl
- s Zeichenkette (String); die maximale Länge ergibt sich aus der Tabellendefinition
- b *boolean* (Wahrheitswert) – *true* (wahr) oder *false* (falsch)

- d Datumswert

In einer Datenmenge navigieren

Diese Methoden gelten sowohl für ein Formular als auch für die Ergebnismenge einer Abfrage.

Mit «Cursor» ist in den Beschreibungen der Datensatzzeiger gemeint.

Name	Datentyp	Beschreibung
Prüfungen für die Position des Cursors		
isBeforeFirst	boolean	Der Cursor steht vor der ersten Zeile, wenn der Cursor nach dem Einlesen noch nicht gesetzt wurde.
isFirst	boolean	Gibt an, ob der Cursor auf der ersten Zeile steht.
isLast	boolean	Gibt an, ob der Cursor auf der letzten Zeile steht.
isAfterLast	boolean	Der Cursor steht hinter der letzten Zeile, wenn er von der letzten Zeile aus mit <i>next</i> weiter gesetzt wurde.
getRow	long	Nummer der aktuellen Zeile
Setzen des Cursors Beim Datentyp boolean steht das Ergebnis «true» dafür, dass das Navigieren erfolgreich war.		
beforeFirst	–	Wechselt vor die erste Zeile.
first	boolean	Wechselt zur ersten Zeile.
previous	boolean	Geht um eine Zeile zurück.
next	boolean	Geht um eine Zeile vorwärts.
last	boolean	Wechselt zur letzten Zeile.
afterLast	–	Wechselt hinter die letzte Zeile.
absolute(n)	boolean	Geht zu der Zeile mit der angegebenen Nummer.
relative(n)	boolean	Geht um eine bestimmte Anzahl von Zeilen weiter: bei positivem Wert von n vorwärts, andernfalls zurück.
Maßnahmen zum Status der aktuellen Zeile		
refreshRow	–	Liest die ursprünglichen Werte der aktuellen Zeile neu ein.
rowInserted	boolean	Gibt an, ob es sich um eine neue Zeile handelt.
rowUpdated	boolean	Gibt an, ob die aktuelle Zeile geändert wurde.
rowDeleted	boolean	Gibt an, ob die aktuelle Zeile gelöscht wurde.

Datenzeilen bearbeiten

Die Methoden zum Lesen stehen bei jedem Formular und bei einer Ergebnismenge zur Verfügung. Die Methoden zum Ändern und Speichern gibt es nur bei einer Datenmenge, die geändert werden kann (in der Regel also nur bei Tabellen, nicht bei Abfragen).

Name	Datentyp	Beschreibung
Maßnahmen für die ganze Zeile		
insertRow	–	Speichert eine neue Zeile.
updateRow	–	Bestätigt Änderungen der aktuellen Zeile.
deleteRow	–	Löscht die aktuelle Zeile.

Name	Datentyp	Beschreibung
cancelRowUpdates	–	Macht Änderungen der aktuellen Zeile rückgängig.
moveToInsertRow	–	Wechselt den Cursor in die Zeile für einen neuen Datensatz.
moveToCurrentRow	–	Kehrt nach der Eingabe eines neuen Datensatzes zurück zur vorherigen Zeile.
Werte lesen		
getString(c)	string	Liefert den Inhalt der Spalte als Zeichenkette.
getBoolean(c)	boolean	Liefert den Inhalt der Spalte als Wahrheitswert.
getByte(c)	byte	Liefert den Inhalt der Spalte als einzelnes Byte.
getShort(c)	short	Liefert den Inhalt der Spalte als ganze Zahl.
getInt(c)	integer	
getLong(c)	long	
getFloat(c)	float	Liefert den Inhalt der Spalte als Dezimalzahl von einfacher Genauigkeit.
getDouble(c)	double	Liefert den Inhalt der Spalte als Dezimalzahl von doppelter Genauigkeit. – Wegen der automatischen Konvertierung durch Basic ist dies auch für decimal- und currency-Werte geeignet.
getBytes(c)	array of bytes	Liefert den Inhalt der Spalte als Folge einzelner Bytes.
getDate(c)	Date	Liefert den Inhalt der Spalte als Datumswert.
getTime(c)	Time	Liefert den Inhalt der Spalte als Zeitwert.
getTimestamp(c)	DateTime	Liefert den Inhalt der Spalte als Zeitstempel (Datum und Zeit).
In Basic selbst werden Datums- und Zeitwerte einheitlich mit dem Datentyp DATE verarbeitet. Für den Zugriff auf die Datenmenge gibt es verschiedene Datentypen: com.sun.star.util.Date für ein Datum, com.sun.star.util.Time für eine Zeit, com.sun.star.util.DateTime für einen Zeitstempel.		
wasNull	boolean	Gibt an, ob der Wert der zuletzt gelesenen Spalte NULL war.
Werte speichern		
updateNull(c)	–	Setzt den Inhalt der Spalte c auf NULL.
updateBoolean(c,b)	–	Setzt den Inhalt der Spalte c auf den Wahrheitswert b.
updateByte(c,x)	–	Speichert in Spalte c das angegebene Byte x.
updateShort(c,n)	–	Speichert in Spalte c die angegebene ganze Zahl n.
updateInt(c,n)	–	
updateLong(c,n)	–	
updateFloat(c,n)	–	Speichert in Spalte c die angegebene Dezimalzahl n.
updateDouble(c,n)	–	
updateString(c,s)	–	Speichert in Spalte die angegebene Zeichenkette s.

<i>Name</i>	<i>Datentyp</i>	<i>Beschreibung</i>
updateBytes(c,x)	–	Speichert in Spalte das angegebene Byte-Array x.
updateDate(c,d)	–	Speichert in Spalte das angegebene Datum d.
updateTime(c,d)	–	Speichert in Spalte den angegebenen Zeitwert d.
updateTimestamp(c,d)	–	Speichert in Spalte den angegebenen Zeitstempel d.

Einzelne Werte bearbeiten

Mit diesen Methoden wird über **BoundField** aus einem Kontrollfeld der Inhalt der betreffenden Spalte gelesen oder geändert. Diese Methoden entsprechen fast vollständig denen im vorigen Abschnitt; die Angabe der Spalte entfällt.

Name	Datentyp	Beschreibung
Werte lesen		
getString	string	Liefert den Inhalt der Spalte als Zeichenkette.
getBoolean	boolean	Liefert den Inhalt der Spalte als Wahrheitswert.
getByte	byte	Liefert den Inhalt der Spalte als einzelnes Byte.
getShort	short	Liefert den Inhalt der Spalte als ganze Zahl.
getInt	integer	
getLong	long	
getFloat	float	Liefert den Inhalt der Spalte als Dezimalzahl von einfacher Genauigkeit.
getDouble	double	Liefert den Inhalt der Spalte als Dezimalzahl von doppelter Genauigkeit. – Wegen der automatischen Konvertierung durch Basic ist dies auch für decimal- und currency-Werte geeignet.
getBytes	array of bytes	Liefert den Inhalt der Spalte als Folge einzelner Bytes.
getDate	Date	Liefert den Inhalt der Spalte als Datumswert.
getTime	Time	Liefert den Inhalt der Spalte als Zeitwert.
getTimestamp	DateTime	Liefert den Inhalt der Spalte als Zeitstempel (Datum und Zeit).
In Basic selbst werden Datums- und Zeitwerte einheitlich mit dem Datentyp DATE verarbeitet. Für den Zugriff auf die Datenmenge gibt es verschiedene Datentypen: com.sun.star.util.Date für ein Datum, com.sun.star.util.Time für eine Zeit, com.sun.star.util.DateTime für einen Zeitstempel.		
wasNull	boolean	Gibt an, ob der Wert der zuletzt gelesenen Spalte NULL war.
Werte speichern		
updateNull	–	Setzt den Inhalt der Spalte auf NULL.
updateBoolean(b)	–	Setzt den Inhalt der Spalte auf den Wahrheitswert b.
updateByte(x)	–	Speichert in der Spalte das angegebene Byte x.
updateShort(n)	–	Speichert in der Spalte die angegebene ganze Zahl n.
updateInt(n)	–	
updateLong(n)	–	
updateFloat(n)	–	Speichert in der Spalte die angegebene Dezimalzahl n.
updateDouble(n)	–	
updateString(s)	–	Speichert in der Spalte die angegebene Zeichenkette s.
updateBytes(x)	–	Speichert in der Spalte das angegebene Byte-Array x.
updateDate(d)	–	Speichert in der Spalte das angegebene Datum d.
updateTime(d)	–	Speichert in der Spalte den angegebenen Zeitwert d.
updateTimestamp(d)	–	Speichert in der Spalte den angegebenen Zeitstempel d.

Parameter für vorbereitete SQL-Befehle

Die Methoden, mit denen die Werte einem vorbereiteten SQL-Befehl – siehe «[Vorbereitete SQL-Befehle mit Parametern](#)» – übergeben werden, sind ähnlich denen der vorigen Abschnitte. Der erste Parameter – mit *i* bezeichnet – nennt seine Nummer (Position) innerhalb des SQL-Befehls.

Name	Datentyp	Beschreibung
setNull(i, n)	–	Setzt den Inhalt der Spalte auf NULL n bezeichnet den SQL-Datentyp gemäß API-Referenz .
setBoolean(i, b)	–	Fügt den angegebenen Wahrheitswert b in den SQL-Befehl ein.
setByte(i, x)	–	Fügt das angegebene Byte x in den SQL-Befehl ein.
setShort(i, n)	–	Fügt die angegebene ganze Zahl n in den SQL-Befehl ein.
setInt(i, n)		
setLong(i, n)		
setFloat(i, n)	–	Fügt die angegebene Dezimalzahl n in den SQL-Befehl ein.
setDouble(i, n)		
setString(i, s)	–	Fügt die angegebene Zeichenkette s in den SQL-Befehl ein.
setBytes(i, x)	–	Fügt das angegebene Byte-Array x in den SQL-Befehl ein.
setDate(i, d)	–	Fügt das angegebene Datum d in den SQL-Befehl ein.
setTime(i, d)	–	Fügt den angegebenen Zeitwert d in den SQL-Befehl ein.
setTimestamp(i, d)	–	Fügt den angegebenen Zeitstempel d in den SQL-Befehl ein.
clearParameters	–	Entfernt die bisherigen Werte aller Parameter eines SQL-Befehls.

Bedienbarkeit verbessern

Als erste Kategorie werden verschiedene Möglichkeiten vorgestellt, die zur Verbesserung der Bedienbarkeit von Base-Formularen dienen. Sofern nicht anders erwähnt, sind diese Makros Bestandteil der **Beispieldatenbank** «Medien_mit_Makros.odt».

Automatisches Aktualisieren von Formularen

Oft wird in einem Formular etwas geändert und in einem zweiten, auf der gleichen Seite liegenden Formular, soll die Änderung anschließend erscheinen. Hier hilft bereits ein kleiner Codeschnipsel, um das betreffende Anzeigeformular zu aktualisieren.

SUB Aktualisieren

Zuerst wird einmal das Makro benannt. Die Standardbezeichnung für ein Makro ist **SUB**. Dies kann groß oder klein geschrieben sein, Mit **SUB** wird eine Prozedur ablaufen gelassen, die nach außen keinen Wert weitergibt. Weiter unten wird im Gegensatz dazu einmal eine Funktion beschrieben, die im Unterschied dazu Rückgabewerte erzeugt.

Das Makro hat jetzt den Namen «*Aktualisieren*». Um sicher zu gehen, dass keine Variablen von außen eingeschleust werden gehen viele Programmierer so weit, dass sie Basic über **Option Explicit** gleich zu Beginn mitteilen: Erzeuge nicht automatisch irgendwelche Variablen, sondern nutze nur die, die ich auch vorher definiert habe.

Deshalb werden jetzt standardgemäß erst einmal die Variablen deklariert. Bei allen hier deklarierten Variablen handelt es sich um Objekte (nicht z.B. Zahlen oder Texte), so dass der Zusatz **AS OBJECT** hinter der Deklaration steht. Um später noch zu erkennen, welchen Typ eine Variable hat, ist vor die Variablenbezeichnung ein «o» gesetzt worden. Prinzipiell ist aber die Variablenbezeichnung nahezu völlig frei wählbar.

```
DIM oDoc AS OBJECT
DIM oDrawpage AS OBJECT
DIM oForm AS OBJECT
```

Das Formular liegt in dem momentan aktiven Dokument. Der Behälter, in dem alle Formulare aufbewahrt werden, wird als **Drawpage** bezeichnet. Im Formularnavigator ist dies sozusagen der oberste Begriff, an den dann sämtliche Formulare angehängt werden.

Das Formular, auf das zugegriffen werden soll, ist hier mit den Namen "Anzeige" versehen. Dies ist der Name, der auch im Formularnavigator sichtbar ist. So hat z.B. das erste Formular standardmäßig erst einmal den Namen "MainForm".

```
oDoc = thisComponent
oDrawpage = oDoc.Drawpage
oForm = oDrawpage.forms.getByName("Anzeige")
```

Nachdem das Formular jetzt ansprechbar gemacht wurde und der Punkt, an dem es angesprochen wurde, in der Variablen **oForm** gespeichert wurde, wird es jetzt mit dem Befehl **reload()** neu geladen.

```
oForm.reload()
END SUB
```

Die Prozedur hat mit **SUB** begonnen. Sie wird mit **END SUB** beendet.

Dieses Makro kann jetzt z.B. ausgelöst werden, wenn die Abspeicherung in einem anderen Formular erfolgt. Wird z.B. in einem Kassenformular an einer Stelle die Anzahl der Gegenstände und (über Barcodescanner) die Nummer eingegeben, so kann in einem anderen Formular im gleichen geöffneten Fenster hierdurch der Kassenstand, die Bezeichnung der Ware usw. nach dem Abspeichern sichtbar gemacht werden.

Filtern von Datensätzen

Der Filter selbst funktioniert ja schon ganz ordentlich in einer weiter oben beschriebenen Variante im Kapitel «Datenfilterung». Die untenstehende Variante ersetzt den Abspeicherungsbutton und liest die Listfelder neu ein, so dass ein gewählter Filter aus einem Listfeld die Auswahl in dem anderen Listfeld einschränken kann.¹⁹

```
SUB Filter
  DIM oDoc AS OBJECT
  DIM oDrawpage AS OBJECT
  DIM oForm1 AS OBJECT
  DIM oForm2 AS OBJECT
  DIM oFeldList1 AS OBJECT
  DIM oFeldList2 AS OBJECT
  oDoc = thisComponent
  oDrawpage = oDoc.drawpage
```

Zuerst werden die Variablen definiert und auf das Gesamtformular zugegriffen. Das Gesamtformular besteht aus den Formularen "Filter" und "Anzeige". Die Listfelder befinden sich in dem Formular "Filter" und sind mit dem Namen "Liste_1" und "Liste_2" versehen.

```
oForm1 = oDrawpage.forms.getByName("Filter")
oForm2 = oDrawpage.forms.getByName("Anzeige")
oFeldList1 = oForm1.getByName("Liste_1")
oFeldList2 = oForm1.getByName("Liste_2")
```

¹⁹ Siehe zu diesem Abschnitt auch die Datenbank «Beispiel_Suchen_und_Filtern.odt», die diesem Handbuch beiliegt.

Zuerst wird der Inhalt der Listenfelder an das darunterliegende Formular mit **commit()** weitergegeben. Die Weitergabe ist notwendig, da ansonsten die Änderung eines Listenfeldes bei der Speicherung nicht berücksichtigt wird. Genau genommen müsste der **commit()** nur auf dem Listenfeld ausgeführt werden, das gerade betätigt wurde. Danach wird der Datensatz mit **updateRow()** abgespeichert. Es existiert ja in unserer Filtertabelle prinzipiell nur ein Datensatz, und der wird zu Beginn einmal geschrieben. Dieser Datensatz wird also laufend durch ein Update-Kommando überschrieben.

```
oFeldList1.commit()
oFeldList2.commit()
oForm1.updateRow()
```

Die Listenfelder sollen einander beeinflussen. Wird in einem Listenfeld z.B. eingegrenzt, dass an Medien nur CDs angezeigt werden sollen, so muss das andere Listenfeld bei den Autoren nicht noch sämtliche Buchautoren auflisten. Eine Auswahl im 2. Listenfeld hätte dann allzu häufig ein leeres Filterergebnis zur Folge. Daher müssen die Listenfelder jetzt neu eingelesen werden. Genau genommen müsste der **refresh()** nur auf dem Listenfeld ausgeführt werden, das gerade nicht betätigt wurde.

Anschließend wird das Formular2, das den gefilterten Inhalt anzeigen soll, neu geladen.

```
oFeldList1.refresh()
oFeldList2.refresh()
oForm2.reload()
END SUB
```

Soll mit diesem Verfahren ein Listenfeld von der Anzeige her beeinflusst werden, so kann das Listenfeld mit Hilfe verschiedener Abfragen bestückt werden.

Die einfachste Variante ist, dass sich die Listenfelder mit ihrem Inhalt aus dem Filterergebnis versorgen. Dann bestimmt der eine Filter, aus welchen Datenbestand anschließend weiter gefiltert werden kann.

```
SELECT
    "Feld_1" || ' - ' || "Anzahl" AS "Anzeige",
    "Feld_1"
FROM
    ( SELECT COUNT( "ID" ) AS "Anzahl", "Feld_1" FROM "Suchtabelle"
      GROUP BY "Feld_1" )
ORDER BY "Feld_1"
```

Es wird der Feldinhalt und die Trefferzahl angezeigt. Um die Trefferzahl zu errechnen, wird eine Unterabfrage gestellt. Dies ist notwendig, da sonst nur die Trefferzahl ohne weitere Information aus dem Feld in der Listbox angezeigt würde.

Das Makro erzeugt durch dieses Vorgehen ganz schnell Listboxen, die nur noch mit einem Wert gefüllt sind. Steht eine Listbox nicht auf NULL, so wird sie schließlich bei der Filterung bereits berücksichtigt. Nach Betätigung der 2. Listbox stehen also bei beiden Listboxen nur noch die leeren Felder und jeweils 1 angezeigter Wert zur Verfügung. Dies mag für eine eingrenzende Suche erst einmal praktisch erscheinen. Was aber, wenn z.B. in einer Bibliothek die Zuordnung zur Systematik klar war, aber nicht eindeutig, ob es sich um ein Buch, eine CD oder eine DVD handelt? Wurde einmal die Systematik ausgewählt und dann die 2. Listbox auf CD gestellt so muss, um auch die Bücher zu sehen, die 2. Listbox erst einmal wieder auf NULL gestellt werden, um dann auch die Bücher anwählen zu können. Praktischer wäre, wenn die 2. Listbox direkt die verschiedenen Medienarten anzeigen würde, die zu der Systematik zur Verfügung stehen – natürlich mit den entsprechenden Trefferquoten.

Um dies zu erreichen, wurde die folgende Abfrage konstruiert, die jetzt nicht mehr direkt aus dem Filterergebnis gespeist wird. Die Zahlen für die Treffer müssen anders ermittelt werden.

```
SELECT
    COALESCE( "Feld_1" || ' - ' || "Anzahl", 'leer - ' || "Anzahl" ) AS
    "Anzeige",
```

```

    "Feld_1"
FROM
    ( SELECT COUNT( "ID" ) AS "Anzahl", "Feld_1" FROM "Tabelle"
      WHERE "ID" IN
        ( SELECT "Tabelle"."ID" FROM "Filter", "Tabelle"
          WHERE "Tabelle"."Feld_2" = COALESCE( "Filter"."Filter_2",
            "Tabelle"."Feld_2" ) )
        GROUP BY "Feld_1"
      )
ORDER BY "Feld_1"

```

Diese doch sehr verschachtelte Abfrage kann auch unterteilt werden. In der Praxis bietet es sich häufig an, die Unterabfrage in einer Tabellenansicht ('**VIEW**') zu erstellen. Das Listenfeld bekommt seinen Inhalt dann über eine Abfrage, die sich auf diesen '**VIEW**' bezieht.

Die Abfrage im Einzelnen:

Die Abfrage stellt 2 Spalten dar. Die erste Spalte enthält die Ansicht, die die Person sieht, die das Formular vor sich hat. In der Ansicht werden die Inhalte des Feldes und, mit einem Bindestrich abgesetzt, die Treffer zu diesem Feldinhalt gezeigt. Die zweite Spalte gibt ihren Inhalt an die zugrundeliegende Tabelle des Formulars weiter. Hier steht nur der Inhalt des Feldes. Die Listenfelder beziehen ihre Inhalte dabei aus der Abfrage, die als Filterergebnis im Formular dargestellt wird. Nur diese Felder stehen schließlich zur weiteren Filterung zur Verfügung.

Als Tabelle, aus der diese Informationen gezogen werden, liegt eine Abfrage vor. In dieser Abfrage werden die Primärschlüsselfelder gezählt (**SELECT COUNT("ID") AS "Anzahl"**). Dies geschieht gruppiert nach der Bezeichnung, die in dem Feld steht (**GROUP BY "Feld_1"**). Als zweite Spalte stellt diese Abfrage das Feld selbst als Begriff zur Verfügung. Diese Abfrage wiederum basiert auf einer weiteren Unterabfrage:

```

SELECT "Tabelle"."ID"
FROM "Filter", "Tabelle"
WHERE "Tabelle"."Feld_2" = COALESCE( "Filter"."Filter_2",
  "Tabelle"."Feld_2" )

```

Diese Unterabfrage bezieht sich jetzt auf das andere zu filternde Feld. Prinzipiell muss das andere zu filternde Feld auch zu den Primärschlüsselnummern passen. Sollten noch mehrere weitere Filter existieren, so ist diese Unterabfrage zu erweitern:

```

SELECT "Tabelle"."ID"
FROM "Filter", "Tabelle"
WHERE "Tabelle"."Feld_2" = COALESCE( "Filter"."Filter_2",
  "Tabelle"."Feld_2" )
AND
  "Tabelle"."Feld_3" = COALESCE( "Filter"."Filter_3",
    "Tabelle"."Feld_3" )

```

Alle weiteren zu filternden Felder beeinflussen, was letztlich in dem Listenfeld des ersten Feldes, "Feld_1", angezeigt wird.

Zum Schluss wird die gesamte Abfrage nur noch nach dem zugrundeliegenden Feld sortiert.

Wie letztlich die Abfrage aussieht, die dem anzuzeigenden Formular zugrunde liegt, ist im Kapitel «Datenfilterung» nachzulesen.

Mit dem folgenden Makro kann über das Listenfeld gesteuert werden, welches Listenfeld abgespeichert werden muss und welches neu eingelesen werden muss.

Die Variablen für das Array werden in den Eigenschaften des Listenfeldes unter Zusatzinformationen abgelegt. Die erste Variable enthält dort immer den Namen des Listenfeldes selbst, die weiteren Variablen die Namen aller anderen Listenfelder, getrennt durch Kommata.

```

SUB Filter_Zusatzinfo(oEvent AS OBJECT)

```

```

DIM oDoc AS OBJECT
DIM oDrawpage AS OBJECT
DIM oForm1 AS OBJECT
DIM oForm2 AS OBJECT
DIM stTag AS String
stTag = oEvent.Source.Model.Tag

```

Ein Array (Ansammlung von Daten, die hier über Zahlenverbindungen abgerufen werden können) wird gegründet und mit den Feldnamen der Listenfelder gefüllt. Der erste Name ist der Name des Listenfelds, das mit der Aktion (Event) verbunden ist.

```

aList() = Split(stTag, ",")
oDoc = thisComponent
oDrawpage = oDoc.drawpage
oForm1 = oDrawpage.forms.getByName("Filter")
oForm2 = oDrawpage.forms.getByName("Anzeige")

```

Das Array wird von seiner Untergrenze (**LBound()**) bis zu seiner Obergrenze (**UBound()**) in einer Schleife durchlaufen. Alle Werte, die in den Zusatzinformationen durch Komma getrennt erschienen, werden jetzt nacheinander weitergegeben.

```

FOR i = LBound(aList()) TO UBound(aList())
    IF i = 0 THEN

```

Das auslösende Listenfeld muss abgespeichert werden. Es hat die Variable **aList(0)**. Zuerst wird die Information des Listenfeldes auf die zugrundeliegende Tabelle übertragen, dann wird der Datensatz gespeichert.

```

        oForm1.getByName(aList(i)).commit()
        oForm1.updateRow()
    ELSE

```

Die anderen Listenfelder müssen neu eingelesen werden, da sie ja in Abhängigkeit vom ersten Listenfeld jetzt andere Werte abbilden.

```

        oForm1.getByName(aList(i)).refresh()
    END IF
NEXT
oForm2.reload()
END SUB

```

Die Abfragen für dieses besser nutzbare Makro sind natürlich die gleichen wie in diesem Abschnitt zuvor bereits vorgestellt.

Daten über den Formularfilter filtern

Alternativ zu dieser Vorgehensweise ist es auch möglich, die Filterfunktion des Formulars direkt zu bearbeiten.

```

SUB FilterSetzen
    DIM oDoc AS OBJECT
    DIM oForm AS OBJECT
    DIM oFeld AS OBJECT
    DIM stFilter As String
    oForm = thisComponent.Drawpage.Forms.getByName("MainForm")
    oFeld = oForm.getByName("Filter")
    stFilter = oFeld.Text
    oForm.filter = " UCASE(" & Name & ") LIKE '%" & stFilter & "%' + UCASE(stFilter) + '%" & stFilter & "%'"
    oForm.ApplyFilter = TRUE
    oForm.reload()
END SUB

```

Das Feld wird im Formular aufgesucht, der Inhalt ausgelesen. Der Filter wird entsprechend gesetzt. Die Filterung wird angeschaltet und das Formular neu geladen.

```

SUB FilterEntfernen
    DIM oForm AS OBJECT
    oForm = thisComponent.Drawpage.Forms.getByName("MainForm")
    oForm.ApplyFilter = False
    oForm.reload()

```

END SUB

Die Beendigung des Filters kann natürlich auch über die Navigationsleiste erfolgen. In diesem Fall wird einfach ein weiteres Makro genutzt.

Über diese Filterfunktion kann ein Formular auch direkt mit einem Filter z. B. für nur einen Datensatz gestartet werden. Aus dem startenden Formular wird ein Wert (z.B. ein Primärschlüssel für den aktuellen Datensatz) ausgelesen und an das Zielformular als Filterwert weiter gegeben.

Daten aus Textfeldern auf SQL-Tauglichkeit vorbereiten

Beim Speichern von Daten über einen SQL-Befehl können vor allem Hochkommata (') Probleme bereiten, wie sie z.B. in Namensbezeichnungen wie O'Connor vorkommen können. Dies liegt daran, dass Texteingaben in Daten in ' ' eingeschlossen sind. Hier muss eine Funktion eingreifen und die Daten entsprechend vorbereiten.

```
FUNCTION String_to_SQL(st AS STRING)
    IF InStr(st, "'") THEN
        st = Join(Split(st, "'"), "''")
    END IF
    String_to_SQL = st
END FUNCTION
```

Es handelt sich hier um eine Funktion. Eine Funktion nimmt einen Wert auf und liefert anschließend auch einen Gegenwert zurück.

Der übergebende Text wird zuerst einmal daraufhin untersucht, ob er ein Hochkomma enthält. Ist dies der Fall, so wird der Text an der Stelle aufgetrennt - der Trenner dafür ist das Hochkomma – und anschließend stattdessen mit zwei Hochkommata wieder zusammengefügt. Der SQL-Code wird so maskiert.

Die Funktion übergibt ihr Ergebnis durch den folgenden Aufruf:

```
stTextneu = String_to_SQL(stTextalt)
```

Es wird also einfach nur die Variable stTextalt überarbeitet und der entsprechende Wert wieder in der Variablen stTextneu gespeichert. Dabei müssen die Variablen gar nicht unterschiedlichen heißen. Der Aufruf geht praktischer direkt mit:

```
stText = String_to_SQL(stText)
```

Diese Funktion wird in den nachfolgenden Makros immer wieder benötigt, damit Hochkommata auch über SQL abgespeichert werden können.

Beliebige SQL-Kommandos speichern und bei Bedarf ausführen

Über das Abfragemodul können nur Abfragen gestartet werden. SQL-Code, der auch verändernd auf Daten wirkt, ist dort nicht ausführbar, sofern nicht der Datenbanktreiber das, wie bei dem direkten Treiber für MySQL, zulässt. Der über **Extras → SQL** zur Verfügung stehende Editor lässt zwar die Ausführung von Code zu, bietet aber keine Speichermöglichkeit. Laufend wiederkehrende Befehle müssen so umständlich irgendwo separat abgespeichert werden und können dann in den Editor über die Zwischenablage kopiert werden. Das folgende Makro schafft hier Abhilfe.²⁰

Der für eine Operation wie **UPDATE**, **DELETE** oder **INSERT** gedachte Code wird in einer Tabelle abgelegt, deren erstes Feld den Primärschlüssel **"ID"** und deren zweites Feld den SQL-Code enthält. Der jeweils aktuelle Datensatz wird in einem Formular ausgewählt und dann über einen Button ausgeführt.

```
SUB ChangeData(oEvent AS OBJECT)
    DIM oConnection AS OBJECT
    DIM oForm AS OBJECT
    DIM stSql AS STRING
```

20 Die Beispieldatenbank «Beispiel_InsertUpdateDelete_SQL.odb» ist den Beispieldatenbanken für dieses Handbuch beigelegt.


```

DIM oSql_Statement AS OBJECT
DIM inValue AS INTEGER
oForm = oEvent.Source.Model.Parent
oConnection = oForm.activeConnection()
stSQL = oForm.getString(2)
inValue = MsgBox("Soll der SQL-Code" & CHR(13) & stSQL & CHR(13) & "ausgeführt
werden?", 20, "SQL-Code ausführen")
IF inValue = 6 THEN
    oSQL_Statement = oConnection.createStatement()
    oSQL_Statement.executeUpdate(stSQL)
END IF
END SUB

```

Nach der Definition der Variablen wird das aktuelle Formular über das auslösende Ereignis des Buttons ermittelt. Die Verbindung zur Datenbank wird aus dem Formular ausgelesen. Als zweites Feld steht in der Tabelle der gewünschte SQL-Code.

Zur Sicherheit wird dieser SQL-Code zuerst in einer Messagebox noch einmal dargestellt. Wird hier nicht mit **Ja** bestätigt, so wird der Code nicht ausgeführt. Diese Bestätigung wird über den Rückgabewert der Messagebox ermittelt (**6** entspricht **Ja**). Anschließend wird zuerst die Verbindung für das Statement hergestellt und dann das Statement ausgeführt.

Werte in einem Formular vorausberechnen

Werte, die über die Datenbankfunktionen berechnet werden können, werden in der Datenbank nicht extra gespeichert. Die Berechnung erfolgt allerdings nicht während der Eingabe im Formular, sondern erst nachdem der Datensatz abgespeichert ist. Solange das Formular aus einem Tabellenkontrollfeld besteht, mag das nicht so viel ausmachen. Schließlich kann direkt nach der Eingabe ein berechneter Wert ausgelesen werden. Bei Formularen mit einzelnen Feldern bleibt der vorherige Datensatz aber nicht unbedingt sichtbar. Hier bietet es sich an, die Werte, die sonst in der Datenbank berechnet werden, direkt in entsprechenden Feldern anzuzeigen.²¹

Die folgenden drei Makros zeigen, wie so etwas vom Prinzip her ablaufen kann. Beide Makros sind mit dem Verlassen bestimmter Felder gekoppelt. Dabei ist auch berücksichtigt, dass hinterher eventuell Werte in einem bereits bestehenden Feld geändert werden.

```

SUB Berechnung_ohne_MWSt(oEvent AS OBJECT)
    DIM oForm AS OBJECT
    DIM oFeld AS OBJECT
    DIM oFeld2 AS OBJECT
    oFeld = oEvent.Source.Model
    oForm = oFeld.Parent
    oFeld2 = oForm.getByName("Preis_ohne_MWSt")
    oFeld2.BoundField.UpdateDouble(oFeld.getCurrentValue / 1.19)
    IF NOT IsEmpty(oForm.getByName("Anzahl").getCurrentValue()) THEN
        Berechnung_gesamt2(oForm.getByName("Anzahl"))
    END IF
END SUB

```

Ist in einem Feld «Preis» ein Wert eingegeben, so wird beim Verlassen des Feldes das Makro ausgelöst. Im gleichen Formular wie das Feld «Preis» liegt das Feld «Preis_ohne_MWSt». Für dieses Feld wird mit **BoundField.UpdateDouble** der berechnete Preis ohne Mehrwertsteuer festgelegt. Das Datenfeld dazu entstammt einer Abfrage, bei der vom Prinzip her die gleiche Berechnung, allerdings bei bereits gespeicherten Daten, durchgeführt wird. Auf diese Art und Weise wird der berechnete Wert sowohl während der Eingabe als auch später während der Navigation durch die Datensätze sichtbar, ohne abgespeichert zu werden.

Ist bereits im Feld «Anzahl» ein Wert enthalten, so wird eine Folgerechnung auch für die damit verbundenen Felder durchgeführt.

```

SUB Berechnung_gesamt(oEvent AS OBJECT)
    oFeld = oEvent.Source.Model
    Berechnung_gesamt2(oFeld)

```

²¹ Siehe hierzu die Beispieldatenbank «Beispiel_Direktberechnung_im Formular.odt»

END SUB

Diese kurze Prozedur dient nur dazu, die Auslösung der Folgeprozedur vom Verlassen des Formularfeldes «Anzahl» weiter zu geben. Die Angabe könnte genauso gut mit Hilfe der Bestimmung des Feldes über die Drawpage in der Folgeprozedur integriert werden.

```
SUB Berechnung_gesamt2(oFeld AS OBJECT)
    DIM oForm AS OBJECT
    DIM oFeld2 AS OBJECT
    DIM oFeld3 AS OBJECT
    DIM oFeld4 AS OBJECT
    oForm = oFeld.Parent
    oFeld2 = oForm.getByName("Preis")
    oFeld3 = oForm.getByName("Preis_gesamt_mit_MWSt")
    oFeld4 = oForm.getByName("MWSt_gesamt")
    oFeld3.BoundField.UpdateDouble(oFeld.getCurrentValue * oFeld2.getCurrentValue)
    oFeld4.BoundField.UpdateDouble(oFeld.getCurrentValue * oFeld2.getCurrentValue -
        oFeld.getCurrentValue * oFeld2.getCurrentValue / 1.19)
END SUB
```

Diese Prozedur ist lediglich eine Prozedur, bei der mehrere Felder berücksichtigt werden sollen. Die Prozedur wird aus einem Feld «Anzahl» gestartet, das die Anzahl bestimmter gekaufter Waren vorgeben soll. Mit Hilfe dieses Feldes und des Feldes «Preis» wird jetzt der «Preis_gesamt_mit_MWSt» und die «MWSt_gesamt» berechnet und in die entsprechenden Felder übertragen.

Nachteil in den Prozeduren und auch bei Abfragen: Der Steuersatz wird hier fest einprogrammiert. Besser wäre eine entsprechende Angabe dazu in Verbindung mit dem Preis, da ja Steuersätze unterschiedlich sein können und auch nicht immer konstant sind. In dem Fall müsste eben der Mehrwertsteuersatz aus einem Feld des Formulars ausgelesen werden.

Die aktuelle Office-Version ermitteln

Mit der Version 4.1 sind Änderungen bei Listenfeldern und Datumswerten vorgenommen worden, die es erforderlich machen, vor der Ausführung eines Makros für diesen Bereich zu erkunden, welche Office-Version denn nun verwendet wird. Dazu dient der folgende Code:

```
FUNCTION OfficeVersion()
    DIM aSettings, aConfigProvider
    DIM aParams2(0) AS NEW com.sun.star.beans.PropertyValue
    DIM sProvider$, sAccess$
    sProvider = "com.sun.star.configuration.ConfigurationProvider"
    sAccess = "com.sun.star.configuration.ConfigurationAccess"
    aConfigProvider = createUnoService(sProvider)
    aParams2(0).Name = "nodepath"
    aParams2(0).Value = "/org.openoffice.Setup/Product"
    aSettings = aConfigProvider.createInstanceWithArguments(sAccess, aParams2())
    OfficeVersion() = array(aSettings.oName, aSettings.oSetupVersionAboutBox)
END FUNCTION
```

Diese Funktion gibt eine Array wieder, das als ersten Wert z.B. "LibreOffice" und als zweiten Wert die detaillierte Version, z.B. "4.1.5.2" ausgibt.

Wert von Listenfeldern ermitteln

Mit LibreOffice 4.1 wird der Wert, den Listenfelder an die Datenbank weitergeben, über «CurrentValue» ermittelt. In Vorversionen, auch OpenOffice oder AOO, ist dies nicht der Fall. Die folgende Funktion soll dem Rechnung tragen. Die ermittelte LO-Version muss daraufhin untersucht werden, ob sie nach der Version 4.0 entstanden ist.

```
FUNCTION ID_Ermittlung(oFeld AS OBJECT) AS INTEGER
    a() = OfficeVersion()
    IF a(0) = "LibreOffice" AND ((LEFT(a(1),1) = 4 AND RIGHT(LEFT(a(1),3),1) > 0) OR
        LEFT(a(1),1) > 4) THEN
        stInhalt = oFeld.currentValue
```

ELSE

Vor LO 4.1 wird der Wert, der weiter gegeben wird, aus der Werteliste des Listenfeldes ausgelesen. Der sichtbar ausgewählte Datensatz ist SelectedItems(0). '0', weil auch mehrere Werte in einem Listenfeld ausgewählt werden könnten.

```
stInhalt = oFeld.ValueItemList(oFeld.SelectedItems(0))
END IF
IF IsEmpty(stInhalt) THEN
```

Mit -1 wird ein Zahlenwert weiter gegeben, der nicht als AutoWert verwendet wird, also in vielen Tabellen nicht als Fremdschlüssel existiert.

```
ID_Ermittlung = -1
ELSE
ID_Ermittlung = Cint(stInhalt)
```

Umwandlung von Text in Integer

```
END IF
END FUNCTION
```

Die Funktion gibt den Wert als Integer wieder. Meist werden für Primärschlüssel ja automatisch hoch zählende Integer-Werte verwendet. Für eine Verwendung von Fremdschlüsseln, die diesem Kriterium nicht entsprechen, muss die Ausgabe der Variablen entsprechend angepasst werden.

Der angezeigte Wert eines Listenfeldes lässt sich weiterhin über die Ansicht des Feldes ermitteln:

```
SUB Listenfeldanzeige
DIM oDoc AS OBJECT
DIM oForm AS OBJECT
DIM oListbox AS OBJECT
DIM oController AS OBJECT
DIM oView AS OBJECT
oDoc = thisComponent
oForm = oDoc.Drawpage.Forms(0)
oListbox = oForm.getByName("Listenfeld")
oController = oDoc.getCurrentController()
oView = oController.getControl(oListbox)
print "Angezeigter Inhalt: " & oView.SelectedItem
END SUB
```

Es wird über den Controller auf die Ansicht des Formulars zugegriffen. Damit wird ermittelt, was auf der sichtbaren Oberfläche tatsächlich erscheint. Der ausgewählte Wert ist der **SelectedItem**.

Listenfelder durch Eingabe von Anfangsbuchstaben einschränken

Manchmal kann es vorkommen, dass der Inhalt für Listenfelder unübersichtlich groß wird. Damit eine Suche schneller zum Erfolg führt, wäre es sinnvoll, hier den Inhalt des Listenfeldes nach Eingabe eines oder mehrerer Buchstaben einzugrenzen. Das Listenfeld selbst wird erst einmal mit einem SQL-Befehl versehen, der nur als Platzhalter dient. Hier könnte z.B. stehen:

```
SELECT "Name", "ID" FROM "Tabelle" ORDER BY "Name" LIMIT 5 (HSQLDB)
SELECT "Name", "ID" FROM "Tabelle" ORDER BY "Name" ROWS 5 (FIREBIRD)
```

So wird beim Öffnen des Formulars vermieden, dass Base erst einmal die umfangreiche Liste einlesen muss.

Das folgende Makro ist dafür an **Eigenschaften: Listenfeld → Ereignisse → Taste losgelassen** gekoppelt.

```
GLOBAL stListStart AS STRING
GLOBAL lZeit AS LONG
```

Zuerst werden globale Variablen erstellt. Diese Variablen sind notwendig, damit nicht nur nach einem Buchstaben, sondern nach dem Betätigen weiterer Tasten schließlich auch nach einer Buchstabenkombination gesucht werden kann.

In der globalen Variablen **stListStart** werden die Buchstaben in der eingegebenen Reihenfolge gespeichert.

Die globale Variable **lZeit** wird mit der aktuellen Zeit in Sekunden versorgt. Bei einer längeren Pause zwischen den Tastatureingaben soll die Variable **stListStart** wieder zurückgesetzt werden können. Deswegen wird jeweils der Zeitunterschied zur vorhergehenden Eingabe abgefragt.

```
SUB ListFilter(oEvent AS OBJECT)
    oFeld = oEvent.Source.Model1
    IF oEvent.KeyCode < 538 OR oEvent.KeyCode = 1283 OR oEvent.KeyCode = 1284 THEN
```

Das Makro wird durch einen Tastendruck ausgelöst. Eine Taste hat innerhalb der API einen bestimmten Zahlencode, der unter [com>sun>star>awt>Key](#) nachgeschlagen werden kann. Sonderzeichen wie das «ä», «ö» und «ü» haben den **KeyCode** 0, alle anderen Schriftzeichen und Zahlen haben einen **KeyCode** kleiner als 538. Den **KeyCode** 1283 belegt die Backspace-Taste. Wird dieser Code mit ausgelesen, so können auch Korrekturen durchgeführt werden. Mit dem **KeyCode** 1284 wird auch die Leertaste in die möglichen Zeichen aufgenommen.

Die Abfrage des **KeyCode** ist hier wichtig, da auch der Schritt mit der Tabulatortaste auf das Auswahlfeld natürlich das Makro auslöst. Der **KeyCode** für die Tabulatortaste liegt allerdings bei 1282, so dass der weitere Code der Prozedur hier nicht ausgeführt wird.

```
DIM stSql(0) AS STRING
```

Der SQL-Code für das Listenfeld wird in einem Array gespeichert. Das Array hat im Falle des SQL-Codes aber nur ein Datenfeld. Deshalb ist das Array direkt auf **stSql(0)** begrenzt.

Entsprechend muss auch beim Auslesen des SQL-Codes aus dem Listenfeld darauf geachtet werden, dass der SQL-Code nicht direkt als Text zugänglich ist. Stattdessen ist der Code in einem Array als einziger Eintrag vorhanden: **oFeld.ListSource(0)**.

Der SQL-Code wird nach der Deklaration der Variablen für die weitere Verwendung aufgesplittet. Für das Feld, das gefiltert werden soll, wird nach dem ersten Komma der Code abgetrennt. Das Feld muss also an der ersten Position stehen. Anschließend wird der verbleibende Teil an dem ersten erscheinenden Anführungsstrich «"» aufgetrennt. Damit beginnt die Feldbezeichnung. Diese Aufteilungen erfolgen hier mit einfachen Arrays. Der Variablen **stFeld** wird schließlich wieder das doppelte Anführungszeichen am Beginn hinzugefügt. Außerdem wird über **Rtrim** vermieden, dass eine eventuell noch vorhandene Leertaste am Schluss des Ausdrucks bestehen bleibt.

```
DIM stText AS STRING
DIM stFeld AS STRING
DIM stQuery AS STRING
DIM ar0()
DIM ar1()
ar0() = Split(oFeld.ListSource(0), ",", 2)
ar1() = Split(ar0(0), "\"", 2)
stFeld = "\"" & Rtrim(ar1(1))
```

In dem SQL-Code wird eine Sortieranweisung erwartet. Allerdings kann die Anweisung in SQL in Großbuchstaben, Kleinbuchstaben oder beliebig gemischt erfolgen. Deshalb wird hier nicht mit **Split**, sondern mit Hilfe der Funktion **inStr** nach der Zeichenkette «ORDER» gesucht. Der abschließende Parameter in dieser Funktion sagt mit der «1» aus, dass nicht nach Groß- und Kleinschreibung unterschieden werden soll. Alles, was links von der Zeichenkette «ORDER» steht, soll für die Konstruktion des neuen SQL-Codes weiter genutzt werden. Damit ist gewährleistet, dass auch Listenfelder bestückt werden können, die aus unterschiedlichen Tabellen oder über weitere Bedingungen im SQL-Code definiert worden sind.

```
stQuery = Left(oFeld.ListSource(0), inStr(1,oFeld.ListSource(0), "ORDER",1)-1)
IF inStr(stQuery, "LOWER") > 0 THEN
    stQuery = Left(stQuery, inStr(stQuery, "LOWER")-1)
ELSEIF inStr(1,stQuery, "WHERE",1) > 0 THEN
    stQuery = stQuery & " AND "
ELSE
    stQuery = stQuery & " WHERE "
END IF
```

Enthält die ermittelte Abfrage den Begriff «LOWER», so wird davon ausgegangen, dass die Abfrage bereits über die Prozedur **ListFilter** erstellt wurde. Deswegen wird die neu zu konstruierende Abfrage nur bis zur Position dieses Begriffes übernommen.

Ist dies nicht der Fall und es existiert in der Abfrage bereits der Begriff «WHERE» in beliebiger Schreibweise, so müssen weitere Bedingungen an die Abfrage mit **AND** angehängt werden.

Sind beide Bedingungen nicht erfüllt, so wird ein **WHERE** an den bestehenden Code angehängt.

```
IF lZeit > 0 AND Timer() - lZeit < 5 THEN
    stListStart = stListStart & oEvent.KeyChar
ELSE
    stListStart = oEvent.KeyChar
END IF
lZeit = Timer()
```

Ist bereits einmal eine Zeit in der globalen Variablen abgespeichert worden und beträgt die Distanz zu dieser Zeit zum Zeitpunkt der Eingabe weniger als 5 Sekunden, so wird der eingegebene Buchstabe an die vorher eingegebenen Buchstaben angehängt. Anderenfalls wird der eingegebene Buchstabe als einzige (neue) Eingabe verstanden. Das Listenfeld wird dann einfach neu nach dem entsprechenden Buchstaben gefiltert. Anschließend wird die aktuelle Zeit wieder in der globalen Variablen **lZeit** gespeichert.

```
stText = LCase( stListStart & "%" )
stSql(0) = stQuery + "LOWER("+stFeld+") LIKE '"+stText+"'" ORDER BY "+stFeld+"
oFeld.ListSource = stSql
oFeld.refresh
END IF
END SUB
```

Der SQL-Code wird schließlich zusammengefügt. Die Kleinschreibweise des Feldinhaltes wird mit der Kleinschreibweise des eingegebenen Buchstabens verglichen. Der Code wird dem Listenfeld hinzugefügt und das Listenfeld aufgefrischt, so dass nur noch der gefilterte Inhalt nachgeschlagen werden kann.

Datumswert aus einem Formularwert in eine Datumsvariable umwandeln

```
FUNCTION Datumswert(oFeld AS OBJECT) AS DATE
    a() = OfficeVersion()
    IF a(0) = "LibreOffice" AND (LEFT(a(1),1) = 4 AND RIGHT(LEFT(a(1),3),1) > 0)
        OR LEFT(a(1),1) > 4 THEN
```

Hier werden alle Versionen ab 4.1 durch die oben vorgestellte Funktion «OfficeVersion()» abgefangen. Dazu wird die Version in ihre Bestandteile aufgesplittet. Die Hauptversion und die erste Unterversion werden abgefragt. Das funktioniert vorerst bis zur LO-Version 9 einwandfrei.

```
        DIM stMonat AS STRING
        DIM stTag AS STRING
        stMonat = Right(Str(0) & Str(oFeld.CurrentValue.Month),2)
        stTag = Right(Str(0) & Str(oFeld.CurrentValue.Day),2)
        Datumswert = CDateFromIso(oFeld.CurrentValue.Year & stMonat & stTag)
    ELSE
        Datumswert = CDateFromIso(oFeld.CurrentValue)
    END IF
END FUNCTION
```

Das Datum wird seit LO 4.1.2 als Array im Formularfeld gespeichert. Mit dem aktuellen Wert kann also nicht auf das Datum selbst zugegriffen werden. Entsprechend ist es neu aus den Werten für Tag, Monat und Jahr zusammen zu setzen, damit anschließend in Makros damit weiter gearbeitet werden kann.

Suchen von Datensätzen

Ohne Makro funktioniert das Suchen von Datensätzen auch. Hier ist aber die entsprechende Abfrage äußerst unübersichtlich zu erstellen. Da könnte eine Schleife mittels Makro Abhilfe schaffen.

Die folgende Variante liest die Felder einer Tabelle aus, gründet dann intern eine Abfrage und schreibt daraus schließlich eine Liste der Primärschlüsselnummern der durchsuchten Tabelle auf, auf die der Suchbegriff zutrifft. Für die folgende Beschreibung existiert eine Tabelle "Suchtmp", die aus einem per Autowert erstellten Primärschlüsselfeld "ID" und einem Feld "Nr." besteht, in das die aus der zu durchsuchenden Tabelle gefundenen Primärschlüssel eingetragen werden. Der Tabellenname wird dabei der Prozedur am Anfang als Variable mitgegeben.

Um ein entsprechendes Ergebnis zu bekommen, muss die Tabelle natürlich nicht die Fremdschlüssel, sondern entsprechende Feldinhalte in Textform enthalten. Dafür ist gegebenenfalls eine Tabellenansicht (View) zu erstellen, auf den das Makro auch zugreifen kann.²²

```
SUB Suche(stTabelle AS STRING)
  DIM oDatenquelle AS OBJECT
  DIM oVerbindung AS OBJECT
  DIM oSQL_Anweisung AS OBJECT
  DIM stSql AS STRING
  DIM oAbfrageergebnis AS OBJECT
  DIM oDoc AS OBJECT
  DIM oDrawpage AS OBJECT
  DIM oForm AS OBJECT
  DIM oForm2 AS OBJECT
  DIM oFeld AS OBJECT
  DIM stInhalt AS STRING
  DIM arInhalt() AS STRING
  DIM inI AS INTEGER
  DIM inK AS INTEGER
  oDoc = thisComponent
  oDrawpage = oDoc.drawpage
  oForm = oDrawpage.forms.getByName("Suchform")
  oFeld = oForm.getByName("Suchtext")
  stInhalt = oFeld.getCurrentValue()
  stInhalt = LCase(stInhalt)
```

Der Inhalt des Suchtext-Feldes wird hier von vornherein in Kleinbuchstaben umgewandelt, damit die anschließende Suchfunktion nur die Kleinschreibweisen miteinander vergleicht.

```
oDatenquelle = ThisComponent.Parent.DataSource
oVerbindung = oDatenquelle.GetConnection("", "")
oSQL_Anweisung = oVerbindung.createStatement()
```

Zuerst wird einmal geklärt, ob überhaupt ein Suchbegriff eingegeben wurde. Ist das Feld leer, so wird davon ausgegangen, dass keine Suche vorgenommen wird. Alle Datensätze sollen angezeigt werden; eine weitere Abfrage erübrigt sich.

Ist ein Suchbegriff eingegeben worden, so werden die Spaltennamen der zu durchsuchenden Tabelle ausgelesen, um auf die Felder mit einer Abfrage zugreifen zu können.

```
IF stInhalt <> "" THEN
  stInhalt = String_to_SQL(stInhalt)
  stSql = "SELECT ""COLUMN_NAME"" FROM ""INFORMATION_SCHEMA"". ""SYSTEM_COLUMNS""
    WHERE ""TABLE_NAME"" = '" + stTabelle + "' ORDER BY ""ORDINAL_POSITION"""
  oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
```

²² Siehe zu diesem Abschnitt auch die Datenbank «Beispiel_Suchen_und_Filtern.odt», die diesem Handbuch beiliegt.

Hinweis

Der SQL-Code müsste für **FIREBIRD** hier angepasst werden:

```
stSql = "SELECT RDB$FIELD_NAME FROM RDB$RELATION_FIELDS WHERE  
RDB$RELATION_NAME = '" + stTabelle + "' ORDER BY RDB$Field_POSITION"
```

Auf die Doppelung der doppelten Anführungszeichen kann hier verzichtet werden, da die Bezeichnungen sowieso keine Sonderzeichen enthalten und nur aus Großbuchstaben zusammengesetzt sind.

Leider ist der folgende SQL-Code dieses Makros für Firebird so nicht geeignet, da Firebird nicht in der Lage ist, aus selektierten Daten eine neue Tabelle zu erstellen. (**FIREBIRD**)

Hinweis

SQL-Formulierungen müssen in Makros wie normale Zeichenketten zuerst einmal in doppelten Anführungsstrichen gesetzt werden. Feldbezeichnungen und Tabellenbezeichnungen stehen innerhalb der SQL-Formulierungen in der Regel bereits in doppelten Anführungsstrichen. Damit letztlich ein Code entsteht, der auch diese Anführungsstriche weitergibt, müssen für Feldbezeichnungen und Tabellenbezeichnungen diese Anführungsstriche verdoppelt werden.

Aus `stSql = "SELECT ""Name"" FROM ""Tabelle"";"` wird, wenn es mit dem Befehl `msgbox stSql` auf dem Bildschirm angezeigt wird, **SELECT "Name" FROM "Tabelle";**

Der Zähler des Arrays, in das die Feldnamen geschrieben werden, wird zuerst auf 0 gesetzt. Dann wird begonnen die Abfrage auszulesen. Da die Größe des Arrays unbekannt ist, muss immer wieder nachjustiert werden. Deshalb beginnt die Schleife damit, über **ReDim Preserve arInhalt(inI)** die Größe des Arrays festzulegen und den vorherigen Inhalt dabei zu sichern. Anschließend werden die Felder ausgelesen und der Zähler des Arrays um 1 heraufgesetzt. Damit kann dann das Array neu dimensioniert werden und wieder ein weiterer Wert abgespeichert werden.

```
inI = 0  
WHILE oAbfrageergebnis.next  
    ReDim Preserve arInhalt(inI)  
    arInhalt(inI) = oAbfrageergebnis.getString(1)  
    inI = inI + 1  
WEND  
stSql = "DROP TABLE ""Suchtmp"" IF EXISTS"  
oSQL_Anweisung.executeUpdate (stSql)
```

Jetzt wird die Abfrage in einer Schleife zusammengestellt, die anschließend an die zu Beginn angegebene Tabelle gestellt wird. Dabei werden alle Schreibweisen untersucht, da auch der Inhalt des Feldes in der Abfrage auf Kleinbuchstaben umgewandelt wird.

Die Abfrage wird direkt so gestellt, dass die Ergebniswerte in der Tabelle "Suchtmp" landen. Dabei wird davon ausgegangen, dass der Primärschlüssel an der ersten Position der Tabelle steht (**arInhalt(0)**).

```
stSql = "SELECT """+arInhalt(0)+"""INTO ""Suchtmp"" FROM """+stTabelle+"""  
WHERE "  
FOR inK = 0 TO (inI - 1)  
    stSql = stSql+"LCase("""+arInhalt(inK)+""") LIKE '%" + stInhalt + "%'"  
    IF inK < (inI - 1) THEN  
        stSql = stSql+" OR "  
    END IF  
NEXT  
oSQL_Anweisung.executeQuery(stSql)  
ELSE  
    stSql = "DELETE FROM ""Suchtmp""  
    oSQL_Anweisung.executeUpdate (stSql)  
END IF
```


Das Anzeigeformular muss neu geladen werden. Es hat als Datenquelle eine Abfrage, in diesem Beispiel "Suchabfrage"

```
oForm2 = oDrawpage.forms.getByName("Anzeige")
oForm2.reload()
End Sub
```

Damit wurde eine Tabelle erstellt, die nun in einer Abfrage ausgewertet werden soll. Die Abfrage ist dabei möglichst so zu fassen, dass sie anschließend noch editiert werden kann. Im Folgenden also ein Abfragecode:

```
SELECT * FROM "Suchtabelle"
WHERE "Nr." IN ( SELECT "Nr." FROM "Suchtmp" )
OR "Nr." =
CASE WHEN ( SELECT COUNT( "Nr." ) FROM "Suchtmp" ) > 0
THEN '0' ELSE "Nr." END
```

Alle Elemente der **"Suchtabelle"** werden dargestellt. Auch der Primärschlüssel. Keine andere Tabelle taucht in der direkten Abfrage auf; somit ist auch kein Primärschlüssel einer anderen Tabelle nötig, damit das Abfrageergebnis weiterhin editiert werden kann.

Der Primärschlüssel ist in dieser Beispieltabelle unter dem Titel **"Nr."** abgespeichert. Durch das Makro wird genau dieses Feld ausgelesen. Es wird jetzt also zuerst nachgesehen, ob der Inhalt des Feldes **"Nr."** in der Tabelle **"Suchtmp"** vorkommt. Bei der Verknüpfung mit **'IN'** werden ohne weiteres auch mehrere Werte erwartet. Die Unterabfrage darf also auch mehrere Datensätze liefern.

Bei größeren Datenmengen wird der Abgleich von Werten über die Verknüpfung **IN** aber zusehends langsamer. Es bietet sich also nicht an, für eine leere Eingabe in das Suchfeld einfach alle Primärschlüsselfelder der **"Suchtabelle"** in die Tabelle **"Suchtmp"** zu übertragen und dann auf die gleiche Art die Daten anzusehen. Stattdessen erfolgt bei einer leeren Eingabe eine Leerung der Tabelle **"Suchtmp"**, so dass gar keine Datensätze mehr vorhanden sind. Hierauf zielt der zweite Bedingungsteil:

```
OR "Nr." = CASE WHEN ( SELECT COUNT( "Nr." ) FROM "Suchtmp" ) > 0
THEN '-1' ELSE "Nr." END
```

Wenn in der Tabelle **"Suchtmp"** ein Datensatz gefunden wird, so ist das Ergebnis der ersten Abfrage größer als 0. Für diesen Fall gilt: **"Nr." = '-1'** (hier steht am Besten ein Zahlenwert, der als Primärschlüssel nicht vorkommt, also z.B. **'-1'**). Ergibt die Abfrage genau 0 (Dies ist der Fall wenn keine Datensätze da sind), dann gilt **"Nr." = "Nr."**. Es wird also jeder Datensatz dargestellt, der eine **"Nr."** hat. Da **"Nr."** der Primärschlüssel ist, gilt dies also für alle Datensätze.

Suchen in Formularen und Ergebnisse farbig hervorheben

Bei größeren Inhalten eines Textfeldes ist oft unklar, an welcher Stelle denn nun die Suche den Treffer zu verzeichnen hat. Da wäre es doch gut, wenn das Formular den entsprechenden Treffer auch markieren könnte. So sollte das dann im Formular aussehen:

Suchbegriff: office		Anzeigen
ID	5	
Memo	LibreOffice besitzt ein umfangreiches Hilfesystem. Um zu dem Hilfesystem zu gelangen, drücken Sie F1 oder wählen Sie LibreOffice Hilfe aus dem Hilfemenü. Zusätzlich können Sie wählen, ob Sie Tipps, Erweiterte Tipps und den Office-Assistenten einschalten (Extras→ Optionen→ LibreOffice→ Allgemein). Wenn die Tipps eingeschaltet sind, platzieren Sie den Mauszeiger über eines der Symbole um eine kleine Box («Tooltip») angezeigt zu bekommen. Darin befindet sich eine kurze Erklärung der Funktion des Symbols. Um noch mehr Erklärungen zu erhalten, wählen Sie Hilfe → Direkthilfe und halten den Mauszeiger über das Symbol.	

Um so ein Formular zum Laufen zu bekommen, bedarf es ein paar zusätzlicher Eingriffe in die Trickkiste.²³

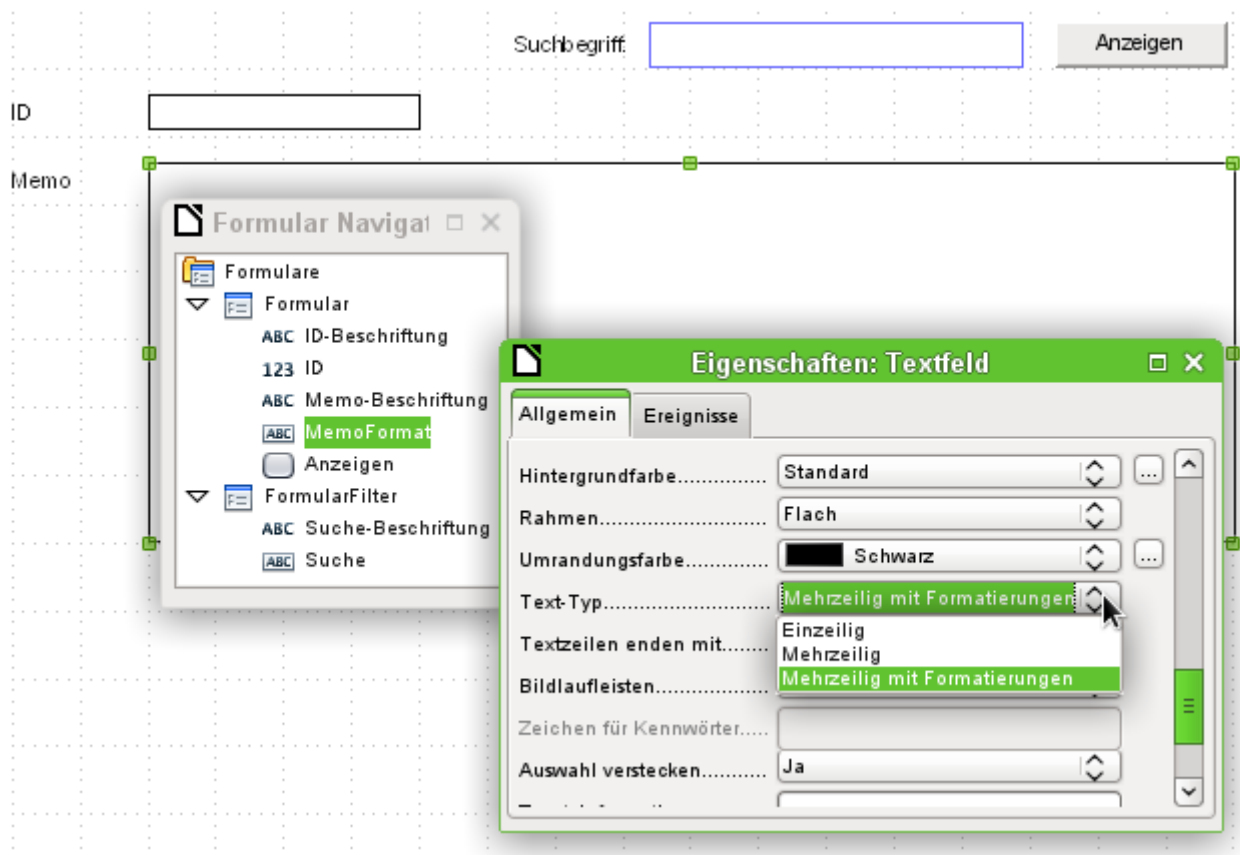
Die Funktionsweise so eines Suchfeldes wurde bereits bei den Abfragetechniken erklärt: Es wird eine Filtertabelle erstellt. Über ein Formular wird nur der aktuelle Wert des einzigen Datensatzes in dieser Tabelle neu geschrieben. Das Hauptformular wird über eine Abfrage mit dem entsprechenden Inhalt versorgt. Die Abfrage sieht im obigen Fall so aus:

```
SELECT "ID", "Memo"
FROM "Tabelle"
WHERE LOWER ( "Memo" ) LIKE '%' || LOWER (
    ( SELECT "Suchtext" FROM "Filter" WHERE "ID" = TRUE ) ) || '%'
```

Wird ein Suchtext eingetragen, so werden nur die Datensätze der Tabelle "Tabelle" angezeigt, bei denen der Text im Feld "Memo" vorkommt. Dabei ist Groß- und Kleinschreibung egal.

Wird kein Suchtext eingetragen, so werden alle Datensätze der Tabelle "Tabelle" angezeigt. Da der Primärschlüssel dieser Tabelle auch in der Abfrage enthalten ist, ist die Abfrage außerdem editierbar.

²³ Siehe zu diesem Abschnitt auch die Datenbank «Beispiel_Autotext_Suchmarkierung_Rechtschreibung.odt», die dem Handbuch beiliegt.



In dem Formular ist neben dem Feld «ID» für den Primärschlüsseintrag nur ein Feld «MemoFormat», das über **Eigenschaften → Allgemein → Text-Typ → Mehrzeilig mit Formatierungen** so eingestellt ist, dass es überhaupt Farben innerhalb von schwarzem Text darstellen kann. Die genaue Betrachtung der Eigenschaften des Textfeldes zeigt, dass der Reiter **Daten** fehlt. Daten lassen sich über ein Feld, das zusätzlich formatierbar ist, nicht eingeben. Das ist wohl dadurch begründet, dass die Datenbank selbst auch solche Formatierungen nicht speichert. Und trotzdem ist es durch den entsprechenden Makroeinsatz möglich, Text in dieses Feld hinein zu bekommen, ihn zu markieren und bei Änderungen auch wieder aus dem Feld hinaus in die Datenbank zu befördern.

Die Prozedur «InhaltUebertragen» dient dazu, den Inhalt aus dem Datenbankfeld "Memo" in das formatierbare Textfeld «MemoFormat» zu übertragen und so zu formatieren, dass bei einem entsprechenden Eintrag im Suchfeld der dazugehörige Begriff hervorgehoben wird.

Die Prozedur ist an das folgende Ereignis gebunden: **Formular → Ereignisse → Nach dem Datensatzwechsel**

```
Sub InhaltUebertragen(oEvent AS OBJECT)
    DIM inMemo AS INTEGER
    DIM oFeld AS OBJECT
    DIM stSuchtext AS STRING
    DIM oCursor AS OBJECT
    DIM inSuch AS INTEGER
    DIM inSuchAlt AS INTEGER
    DIM inLen AS INTEGER
    oForm = oEvent.Source
    inMemo = oForm.findColumn("Memo")
    oFeld = oForm.getByName("MemoFormat")
    oFeld.Text = oForm.getString(inMemo)
```

Zuerst werden die Variablen definiert. Anschließend wird über das Formular das Tabellenfeld "Memo" gesucht und aus diesem Feld über **getString()** der entsprechende Text des Feldes "Memo" der Tabelle "Tabelle" ausgelesen. Der entsprechende Feldinhalt wird in das Feld übertragen, das sich formatieren lässt, aber keine Verbindung zur Datenbank hat: «MemoFormat».

Bei Tests ist es zuerst vorgekommen, dass sich das Formular zwar öffnete, aber leider die Formularleiste am unteren Rand des Formulars nicht mehr aufgebaut wurde. Deswegen erfolgt hier ein sehr kurzer Wartebefehl von 5/1000 Sekunden. Danach wird aus dem parallel zum «Formular» liegenden «FormularFilter» der angezeigte Inhalt als Suchtext ausgelesen.

```
Wait 5
stSuchtext = oForm.Parent.getByName("FormularFilter").getByName("Suche").Text
```

Um Textteile formatieren zu können muss ein (nicht sichtbarer) **TextCursor** in dem Feld erstellt werden, das den Text enthält. Die Darstellung des Textes in der Standardversion hat eine serifenbetonte Schriftart in 12-Punkt-Größe, die in anderen Formularteilen nicht unbedingt vorkommt und über das Formularfeld auch nicht direkt abwählbar ist. In dieser Prozedur wird direkt zu Beginn der Text einmal auf die gewünschte Darstellungsart eingestellt. Erfolgt dies nicht schon zu Beginn, so wird wegen der unterschiedlichen Formatierungen der oberer Textrand in dem Feld erst einmal angeschnitten. Die erste Zeile war in Versuchen nur zu 2/3 lesbar.

Damit der Cursor (wieder nicht sichtbar) den Text markiert, wird er zuerst an den Anfang gesetzt und mit dem Zusatz **true** weiterbewegt zum Endpunkt, der ebenfalls den Zusatz **true** hat. Dann erfolgt die Zuweisung der notwendigen Eigenschaften wie Schriftgröße, Schriftstil, Schriftfarbe oder auch Schriftdicke. Anschließend wird der Cursor wieder zur Startposition gesetzt.

```
oCursor = oFeld.createTextCursor()
oCursor.gotoStart(true)
oCursor.gotoEnd(true)
oCursor.CharHeight = 10
oCursor.CharFontName = "Arial, Helvetica, Tahoma"
oCursor.CharColor = RGB(0,0,0)
oCursor.CharWeight = 100.000000 'com::sun::star::awt::FontWeight
oCursor.gotoStart(false)
```

Enthält das Feld Text und ist ein Eintrag zum Suchen vorhanden, so wird jetzt der Text nach dem Suchbegriff durchsucht. Die äußere Schleife fragt erst einmal nur nach der Bedingung, die nächste Schleife klärt noch einmal, ob der Suchtext denn tatsächlich in dem Text enthalten ist, der in «MemoFormat» steht. Diese Einstellung könnte auch unterlassen werden, da die Abfrage, auf der das Formular basiert, nur solchen Text anzeigt, auf den diese Bedingung zutrifft.

```
IF oFeld.Text <> "" AND stSuchtext <> "" THEN
  IF inStr(oFeld.Text, stSuchtext) THEN
    inSuch = 1
    inSuchAlt = 0
    inLen = Len(stSuchtext)
```

Der Text wird nach dem Suchtext durchsucht. Dies erfolgt in einer Schleife, die dann endet, wenn keine weitere Trefferposition mehr angezeigt wird. **InStr()** liefert dabei die Fundstelle des ersten Zeichens des Suchtextes, in der aufgezeigten Fassung unabhängig von Groß- und Kleinschreibung. Die Schleife wird dadurch gesteuert, dass der Suchbeginn **inSuch** bei jedem Schleifenende in der Summe um 1 erhöht wird (erste Schleifenzeile -1, letzte Schleifenzeile +2). Bei jedem Durchgang wird der Cursor mit **oCursor.goRight(Position, false)** zuerst ohne zu markieren an die Startstelle gesetzt, dann um die Länge des Suchtextes weiter mit der Markierungsaufforderung nach rechts gesetzt. Dann wird die gewünschte Formatierung (blau, etwas dicker) vorgenommen und der Cursor wieder für den nächsten Start an den Startpunkt der Markierung zurückgesetzt.

```
DO WHILE inStr(inSuch, oFeld.Text, stSuchtext) > 0
  inSuch = inStr(inSuch, oFeld.Text, stSuchtext) - 1
  oCursor.goRight(inSuch-inSuchAlt, false)
  oCursor.goRight(inLen, true)
  oCursor.CharColor = RGB(102,102,255)
  oCursor.CharWeight = 110.000000
  oCursor.goLeft(inLen, false)
  inSuchAlt = inSuch
  inSuch = inSuch + 2
LOOP
END IF
END IF
```

End Sub

Die Prozedur «InhaltSchreiben» dient dazu, den Inhalt aus dem formatierbaren Textfeld «Memo-Format» in die Datenbank zu übertragen. Dies erfolgt in dieser Fassung unabhängig davon, ob eine Änderung vorgenommen wurde.

Die Prozedur ist an das folgende Ereignis gebunden: **Formular → Ereignisse → Vor dem Datensatzwechsel**

```
Sub InhaltSchreiben(oEvent AS OBJECT)
    DIM oForm AS OBJECT
    DIM inMemo AS INTEGER
    DIM loID AS LONG
    DIM oFeld AS OBJECT
    DIM stMemo AS STRING
    oForm = oEvent.Source
    IF InStr(oForm.ImplementationName, "ODatabaseForm") THEN
```

Das auslösende Ereignis ist doppelt belegt. Nur der Implementationsname, der auf **ODatabaseForm** endet, gibt den richtigen Zugriff auf den Datensatz.

```
    IF NOT oForm.isBeforeFirst() AND NOT oForm.isAfterLast() THEN
```

Beim Einlesen, auch beim Reload des Formulars, steht der Cursor vor dem ersten Datensatz. Würde jetzt ein Schreibversuch unternommen, dann erscheint die Meldung «ungültiger Cursorstatus».

```
        inMemo = oForm.findColumn("Memo")
        loID = oForm.findColumn("ID")
        oFeld = oForm.getByName("MemoFormat")
        stMemo = oFeld.Text
        IF stMemo <> "" THEN
            oForm.updateString(inMemo, stMemo)
        END IF
        IF stMemo <> "" AND oForm.getString(loID) <> "" THEN
            oForm.UpdateRow()
        END IF
    END IF
END IF
End Sub
```

Das Tabellenfeld "Memo" wird aus der Datenquelle des Formulars herausgesucht. Ebenso das Feld "ID". Befindet sich im Feld «MemoFormat» Text, so wird er mit **oForm.updateString()** in das Feld "Memo" der Datenquelle übertragen. Nur wenn bereits ein Eintrag im Feld "ID" existiert, also der Primärschlüssel belegt ist, erfolgt ein Update. Ansonsten wird ja sowieso ein neuer Datensatz über die Formularfunktionen eingefügt, da das Formular die Änderung entsprechend bemerkt und eine Abspeicherung selbständig vornimmt.

Rechtschreibkontrolle während der Eingabe

Auf **mehrzeilige Textfelder mit Formatierungen** greift auch dieses Makro zu. Entsprechend muss auch, wie bei dem vorherigen Kapitel, der Inhalt bei jedem Datensatzwechsel zuerst geschrieben und danach der Inhalt des neuen Datensatzes in das Formularfeld geladen werden. Die Prozeduren «InhaltUebertragen» und «InhaltSchreiben» unterscheiden sich höchstens in dem Punkt, dass die Suchfunktion ausgeklammert werden kann.²⁴

24 Siehe auch hierzu: «Beispiel_Autotext_Suchmarkierung_Rechtschreibung.odt»

ID

1

Memo

Dieses Dokument unterliegt dem Copyright © 2014. Die Beitragenden sind unten aufgeführt. Sie dürfen dieses Dokument unter den Bedingungen der GNU General Public License (<http://www.gnu.org/licenses/gpl.html>), Version 3 oder höher, oder der Creative Commons Attribution License (<http://creativecommons.org/licenses/by/3.0/>), Version 3.0 oder höher, verändern und/oder weiter geben.

Warennamen werden ohne Gewährleistung der freien Verwendbarkeit benutzt.

Fast alle Hardware- und Softwarebezeichnungen und weitere Stichworte und sonstige Angaben, die in diesem Buch verwendet werden, sind als eingetragene Marken geschützt.

Da es nicht möglich ist, in allen Fällen zeitnah zu ermitteln, ob ein Markenschutz besteht, wird das Symbol (R) in diesem Buch nicht verwendet.

Die Rechtschreibkontrolle wird in dem obigen Formular dadurch ausgelöst, dass in dem Formularfeld entweder eine Leertaste oder ein Return betätigt wird. Sie läuft also nach der Beendigung eines Wortes jedes Mal ab und könnte gegebenenfalls auch noch mit dem Fokusverlust des Formularfeldes gekoppelt werden, damit auch das letzte Wort sicher überprüft wird.

Die Prozedur ist an das folgende Ereignis gebunden: **Formular → Ereignisse → Taste losgelassen**

```
SUB MarkierungFehlerDirekt(oEvent AS OBJECT)
    GlobalScope.BasicLibraries.LoadLibrary("Tools")
```

Es wird die Funktion **RTrimStr** zum Entfernen von Satzzeichen am Ende vor Worten benötigt. Sonst werden alle Worte, denen ein Komma, Punkt oder irgendein anderes Satzzeichen folgt, als falsch angesehen. Mit **LTrimChar** müssen außerdem Klammern zum Beginn des Wortes entfernt werden.

```
DIM aProp() AS NEW com.sun.star.beans.PropertyValue
DIM oLinuSvcMgr AS OBJECT
DIM oSpellChk AS OBJECT
DIM oFeld AS OBJECT
DIM arText()
DIM stWort AS STRING
DIM inlenWort AS INTEGER
DIM ink AS INTEGER
DIM i AS INTEGER
DIM oCursor AS OBJECT
DIM stText AS OBJECT
oLinguSvcMgr = createUnoService("com.sun.star.linguistic2.LinguServiceManager")
IF NOT IsNull(oLinguSvcMgr) THEN
    oSpellChk = oLinguSvcMgr.getSpellChecker()
END IF
```

Zuerst werden alle Variablen deklariert. Danach wird auf das Rechtschreibüberprüfungsmodul **SpellChecker** zugegriffen. Mit diesem Modul werden anschließend die einzelnen Worte auf ihre Richtigkeit hin überprüft.

```
oFeld = oEvent.Source.Model
ink = 0
IF oEvent.KeyCode = 1280 OR oEvent.KeyCode = 1284 THEN
```

Das Ereignis, das das Makro auslöst, ist ein Tastendruck. Zu dem Ereignis wird ein Code für jede Taste mitgeliefert, der **KeyCode**. Der **KeyCode** für die Returnntaste ist 1280, der für die Leertaste ist 1284. Wie viele andere Informationen sind diese Informationen einfach durch das Tool «Xray» gewonnen worden. Wird also eine Leertaste oder die Returnntaste betätigt, so wird die Rechtschreibung überprüft. Sie startet also zu jedem Wortende. Lediglich die Überprüfung für das letzte Wort ist so nicht automatisch möglich.

Bei jedem Durchlauf werden alle Worte des Textes überprüft. Die Überprüfung einzelner Worte könnte eventuell auch möglich sein, bedeutet aber erheblich mehr Aufwand.

Der Text wird also in Worte aufgesplittet. Trenner ist hier das Leerzeichen. Vorher müssen allerdings noch Trennungen an Zeilenumbrüchen erzeugt werden, die sonst später als ein Wort wahrgenommen werden.

```
stText = Join(Split(oFeld.Text, CHR(10)), " ")
stText = Join(Split(stText, CHR(13)), " ")
arText = Split(RTrim(stText), " ")
FOR i = LBound(arText) TO Ubound(arText)
    stWort = arText(i)
    inlenWort = len(stWort)
    stWort = Trim( RtrimStr( RtrimStr( RtrimStr( RtrimStr( RtrimStr(
        RtrimStr(stWort, ","), "."), "?"), "!", "."), ",")
    stWort = LTrimChar(stWort, "(")
```

Das einzelne Wort wird ausgelesen, seine ungekürzte Länge ist notwendig für die folgenden Bearbeitungsschritte. Nur so kann die Position des Wortes innerhalb des gesamten Textes bestimmt werden, die auch für die gezielte Markierung von Schreibfehlern gebraucht wird.

Mit **Trim** werden Leerzeichen entfernt, mit der Funktion **RTrimStr** Kommas und Satzzeichen am Ende des Textes, mit der Funktion **LTrimChar** Zeichen am Anfang des Textes.

```
IF stWort <> "" THEN
    oCursor = oFeld.createTextCursor()
    oCursor.gotoStart(false)
    oCursor.goRight(ink, false)
    oCursor.goRight(inLenWort, true)
    If Not oSpellChk.isValid(stWort, "de", aProp()) Then
        oCursor.CharUnderline = 9
        oCursor.CharUnderlineHasColor = True
        oCursor.CharUnderlineColor = RGB(255, 51, 51)
    ELSE
        oCursor.CharUnderline = 0
    END IF
END IF
ink = ink + inLenWort + 1
NEXT
END IF
END SUB
```

Hat das Wort einen Inhalt, so wird zuerst einmal ein Textcursor erstellt. Der Textcursor wird ohne Markierung an den Start des Textes in dem Eingabefeld gesetzt. Dann geht es, immer noch ohne Markierung, um den Betrag nach rechts im Text vorwärts, der in der Variablen **ink** gespeichert ist. Diese Variable ist am Anfang 0, nach Durchlaufen der ersten Schleife dann so groß wie das vorhergehende Wort lang war +1 für das angehängte Leerzeichen. Dann wird der Cursor mit Markierung um die Länge des aktuellen Wortes weiter gesetzt. Erfolgt jetzt eine Änderung der Buchstabeigenschaften, so betrifft diese nur den markierten Bereich.

Der **Spellchecker** startet. Als Variablen müssen das Wort und der Landescode übergeben werden. Ohne Landescode ist alles richtig. Das Array bleibt in der Regel leer.

Ist das Wort nicht in den Lexika eingetragen, so wird es mit einer roten Wellenlinie versehen. Die Wellenlinie entspricht hier der '9'. Ist das Wort eingetragen, so wird statt einer Wellenlinie keine Linie ('0') gezeichnet. Dieser Schritt ist notwendig, weil sonst ein einmal als falsch erkannt Wort bei einer Korrektur auch weiterhin mit der roten Wellenlinie gekennzeichnet würde. Eine rote Wellenlinie würde nie aufgehoben, da es keine entgegengesetzte Formatierung gibt.

Kombinationsfelder als Listenfelder mit Eingabemöglichkeit

Aus Kombinationsfeldern und Tabellenfeldern aus dem Formular kann direkt eine Tabelle mit einem Datensatz versehen und der entsprechende Primärschlüssel in eine andere Tabelle eingetragen werden.²⁵

²⁵ Die Beispieldatenbank «Beispiel_Combobox_Listfeld.odb» zum Einsatz von Kombinationsfeldern statt Listenfeldern ist den Beispieldatenbanken für dieses Handbuch beigelegt.

Das Modul «Comboboxen» macht aus den Formularfeldern zur Eingabe und Auswahl von Werten (Kombinationsfelder) Listenfelder mit Eingabemöglichkeiten. Dazu werden neben den Kombinationsfeldern im Formular die jeweils an die zugrundeliegende Tabelle zu übergebenden Schlüssel-feldwerte in den Tabellenspalten abgelegt, die dem Formular zugrundeliegen. Die Schlüssel aus den Tabellenspalten werden beim Start des Formulars ausgelesen und das Kombinationsfeld auf den entsprechenden Inhalt eingestellt. Wird der Inhalt des Kombinationsfeldes geändert, so wird er neu abgespeichert und der neue Primärschlüssel zum Abspeichern in der Haupttabelle in das entsprechende numerische Fremdschlüsselfeld übertragen.

Werden statt der Tabellen entsprechend konstruierte eingabefähige Abfragen erstellt, so kann der Text, den die Kombinationsfelder darstellen sollen, direkt aus den Abfragen ermittelt werden. Ein Makro ist dann für diesen Arbeitsschritt nicht notwendig.

Voraussetzung für die Funktionsweise des Makros ist, dass alle Primärschlüssel der Tabellen, die in den Kombinationsfeldern als Datenquellen auftauchen, mit einem automatisch hochzählenden Autowert versehen sind. Außerdem ist als Bezeichnung hier vorausgesetzt, dass die Primärschlüssel den Namen "ID" tragen.

Textanzeige im Kombinationsfeld

Diese Prozedur soll Text in den Kombinationsfeldern nach den Werten der (unsichtbaren) Fremdschlüssel-Felder aus dem Hauptformular einstellen. Dabei werden gegebenenfalls auch Listenfelder berücksichtigt, die sich auf 2 unterschiedliche Tabellen beziehen. Dies kann z.B. dann sein, wenn bei einer Ortsangabe die Postleitzahl vom Ort abgetrennt wurde. Dann wird die Postleitzahl aus einer Tabelle ausgelesen, in der auch ein Fremdschlüssel für den Ort liegt. Im Listenfeld werden Postleitzahl und Ort zusammen angezeigt.

```
SUB TextAnzeigen(oEvent AS OBJECT)
```

Dieses Makro sollte an das folgende Ereignis des Formulars gebunden werden: 'Nach dem Datensatzwechsel'

Das Makro wird direkt aus dem Formular angesprochen. Über das auslösende Ereignis werden die gesamten notwendigen Variablen für das Makro ermittelt.

Die Variablen werden deklariert. Einige Variablen sind in einem separaten Modul bereits global deklariert und werden hier nicht noch einmal erwähnt.

```
DIM oForm AS OBJECT
DIM oFeld AS OBJECT
DIM oFeldList AS OBJECT
DIM stAbfrage AS STRING
DIM stFeldWert AS STRING
DIM stFeldID AS STRING
DIM inCom AS INTEGER
oForm = oEvent.Source
```

Das Formular startet das Ereignis. Es ist die Quelle für das das Makro auslösende Ereignis.

In dem Formular befindet sich ein verstecktes Kontrollelement, aus dem hervorgeht, wie die verschiedenen Kombinationsfelder in diesem Formular heißen. Nacheinander werden dann in dem Makro die Kombinationsfelder abgearbeitet.

```
aComboboxen() = Split(oForm.getByName("Comboboxen").Tag, ",")
FOR inCom = LBound(aComboboxen) TO Ubound(aComboboxen)
    ...
NEXT inCom
```

Aus den Zusatzinformationen («Tag») des versteckten Kontrollelementes wird die Bezeichnung der Kombinationsfelder ermittelt. Sie sind dort durch Kommas voneinander getrennt aufgeschrieben. Die Namen der Felder werden in ein Array geschrieben und nacheinander in einer Schleife abgearbeitet. Die Schleife endet mit der Bezeichnung **NEXT ...**.

Das Kombinationsfeld, das jetzt statt eines Listenfeldes existiert, wird anschließend als **oFeldList** bezeichnet. Der Fremdschlüssel wird über die Bezeichnung des Tabellenfeldes, die in

den Zusatzinformationen des Kombinationsfeldes steht, aus der Tabellenspalte des Formulars ermittelt.

```
oFeldList = oForm.getByName(Trim(aComboboxen(inCom)))
stFeldID = oForm.getString(oForm.findColumn(oFeldList.Tag))
oFeldList.Refresh()
```

Das Kombinationsfeld wird mit **Refresh()** neu eingelesen. Es kann ja sein, dass sich der Inhalt des Feldes durch Neueingaben geändert hat. Diese müssen schließlich verfügbar gemacht werden.

Die Abfrage, die zur Ermittlung des anzuzeigenden Inhaltes des Kombinationsfeldes notwendig ist, wird aus der Abfrage des Kombinationsfeldes und dem ermittelten Wert des Fremdschlüssels erstellt. Damit der SQL-Code brauchbar wird, wird zuerst eine eventuelle Sortierungsanweisung entfernt. Anschließend wird nachgesehen, ob bereits eine Beziehungsdefinition (beginnend mit **WHERE**) existiert. Da die **InStr()**-Funktion standardmäßig keinen Unterschied zwischen Groß- und Kleinschreibung macht, werden hier gleich alle Schreibweisen abgedeckt. Existiert eine Beziehungsdefinition, so enthält die Abfrage Felder aus zwei unterschiedlichen Tabellen. Es muss jetzt die Tabelle herausgesucht werden, aus der der Fremdschlüssel für die Verbindung zur Verfügung gestellt wird. Das Makro funktioniert hier nur mit Hilfe der Information, dass der Primärschlüssel einer jeden Tabelle "ID" heißt.

Existiert keine Beziehungsdefinition, so beruht die Abfrage nur auf einer Tabelle. Die Tabelleninformation kann entfallen, die Bedingung direkt mit dem Fremdschlüsselwert zusammen formuliert werden.

```
IF stFeldID <> "" THEN
    stAbfrage = oFeldList.ListSource
    IF InStr(stAbfrage, "order by") > 0 THEN
        stSql = Left(stAbfrage, InStr(stAbfrage, "order by")-1)
    ELSE
        stSql = stAbfrage
    END IF
    IF InStr(stSql, "where") THEN
        st = Right(stSql, Len(stSql)-InStr(stSql, "where")-4)
        IF InStr(Left(st, InStr(st, "=")), ".ID") THEN
            a() = Split(Right(st, Len(st)-InStr(st, "=")-1), ".")
        ELSE
            a() = Split(Left(st, InStr(st, "=")-1), ".")
        END IF
        stSql = stSql + "AND " + a(0) + ".ID" = " + stFeldID
    ELSE
        stSql = stSql + "WHERE ID" = " + stFeldID
    END IF
END IF
```

Jedes Feld und jeder Tabellename muss bereits in der SQL-Eingabe mit doppelten Anführungsstrichen oben versehen werden. Da bereits Anführungsstriche einfacher Art in Basic als die Einführung zu Text interpretiert werden, sind diese bei der Weitergabe des Codes nicht mehr sichtbar. Erst bei einer Doppelung der Anführungsstriche wird ein Element mit einfachen Anführungsstrichen weitergegeben. **"ID"** bedeutet also, dass in der Abfrage auf das Feld **ID** (mit einfachen Anführungsstrichen für die SQL-Verbindung) zugegriffen wird.

Die in der Variablen **stSql** abgespeicherte Abfrage wird jetzt ausgeführt und das Ergebnis dieser Abfrage in der Variablen **oAbfrageergebnis** gespeichert.

```
oDatenquelle = ThisComponent.Parent.CurrentController
IF NOT (oDatenquelle.isConnected()) Then
    oDatenquelle.connect()
End IF
oVerbindung = oDatenquelle.ActiveConnection()
oSQL_Anweisung = oVerbindung.createStatement()
oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
```

Das Abfrageergebnis wird über eine Schleife ausgelesen. Hier könnten, wie in einer Abfrage aus der GUI, mehrere Felder und Datensätze dargestellt werden. Von der Konstruktion der Abfrage her

wird aber nur ein Ergebnis erwartet. Dieses Ergebnis wird in der ersten Spalte (**1**) der Abfrage zu finden sein. Es ist der Datensatz, der den anzuzeigenden Inhalt des Kombinationsfeldes wiedergibt. Der Inhalt ist ein Textinhalt (**getString()**), deshalb hier **oAbfrageergebnis.getString(1)**.

```
WHILE oAbfrageergebnis.next
    stFeldWert = oAbfrageergebnis.getString(1)
WEND
```

Das Kombinationsfeld muss jetzt auf den aus der Abfrage sich ergebenden Textwert eingestellt werden.

```
oFeldList.Text = stFeldWert
ELSE
```

Falls überhaupt kein Wert in dem Feld für den Fremdschlüssel **oFeld** vorhanden ist, ist auch die ganze Abfrage nicht gelaufen. Das Kombinationsfeld wird jetzt auf eine leere Anzeige eingestellt.

```
oFeldList.Text = ""
END IF
NEXT inCom
END SUB
```

Diese Prozedur erledigt jetzt also den Kontakt von dem in der Datenquelle des Formulars abgelegten Fremdschlüssel zu dem Kombinationsfeld. Für die Anzeige der richtigen Werte im Kombinationsfeld würde das ausreichen. Eine Abspeicherung von neuen Werten hingegen benötigt eine weitere Prozedur.

Fremdschlüsselwert vom Kombinationsfeld zum numerischen Feld übertragen

Wird nun ein neuer Wert ausgewählt oder neu in das Kombinationsfeld eingegeben (nur wegen dieser Eigenschaft wurde ja das Makro konstruiert), so muss der entsprechende Primärschlüssel als Fremdschlüssel in die dem Formular zugrundeliegende Tabelle eingetragen werden.

```
SUB TextAuswahlWertSpeichern(oEvent AS OBJECT)
```

Dieses Makro sollte an das folgende Ereignis des Formulars gebunden werden: 'Vor der Datensatzaktion'.

Nach Deklaration der Variablen (hier nicht weiter aufgeführt) wird zuerst differenziert, bei welchem Ereignis genau das Makro überhaupt ablaufen soll. Vor der Datensatzaktion werden zwei Implementationen nacheinander aufgerufen. Für das Makro selbst ist es wichtig, das Formularobjekt zu erhalten. Das geht prinzipiell über beide Implementationen, aber eben auf unterschiedliche Weise. Es wird hier die Implementation mit dem Namen "**ODatabaseForm**" herausgefiltert.

```
IF InStr(oEvent.Source.ImplementationName, "ODatabaseForm") THEN
    ...
END IF
END SUB
```

In diese Schleife wird der gleiche Start wie bei der Prozedur **TextAnzeigen** eingebaut:

```
oForm = oEvent.Source
aComboboxen() = Split(oForm.getByName("Comboboxen").Tag, ",")
FOR inCom = LBound(aComboboxen) TO Ubound(aComboboxen)
    ...
NEXT inCom
```

Das Feld **oFeldList** zeigt den Text an. Es kann in einem Tabellenkontrollfeld liegen. Dann kann nicht direkt vom Formular auf das Feld zugegriffen werden. In den Zusatzinformationen des versteckten Kontrollfeldes «Comboboxen» ist für diesen Fall der Pfad zum Kombinationsfeld über «Tabellenkontrollfeld>Kombinationsfeld» eingetragen. Durch Aufsplittung dieses Eintrages wird ermittelt, wie das Kombinationsfeld anzusprechen ist.

```
a() = Split(Trim(aComboboxen(inCom)), ">")
IF Ubound(a) > 0 THEN
    oFeldList = oForm.getByName(a(0)).getByName(a(1))
ELSE
```



```

oFeldList = oForm.getByName(a(0))
END IF

```

Anschließend wird die Abfrage aus dem Kombinationsfeld ausgelesen und in ihre Einzelteile zerlegt. Bei einfachen Kombinationsfeldern wären die notwendigen Informationen lediglich der Feldname und der Tabellename:

```
SELECT "Feld" FROM "Tabelle"
```

Dies könnte gegebenenfalls noch durch eine Sortierung erweitert sein. Sobald zwei Felder in dem Kombinationsfeld zusammen dargestellt werden, muss aber bereits bei den Feldern zur Trennung entsprechend mehr Aufwand getrieben werden:

```
SELECT "Feld1"||' '||"Feld2" FROM "Tabelle"
```

Diese Abfrage fasst zwei Felder zusammen und setzt dazwischen eine Leertaste ein. Da der Trenner eine Leertaste ist, wird in dem Makro nach so einem Trenner gesucht und danach der Text in zwei Teile gesplittet. Das funktioniert natürlich nur dann einwandfrei, wenn "Feld1" nicht bereits Text enthalten soll, der eine Leertaste erlaubt. Sonst wird z.B. aus dem Vornamen «Anne Marie» und dem Nachnamen «Müller» durch das Makro der Vorname «Anne» und der Nachname «Marie Müller». Für solch einen Fall sollte ein passender Trenner eingesetzt werden, der dann auch vom Makro gefunden werden kann. Bei Namen ist dies z. B. ein Komma: «Nachname, Vorname».

Noch komplizierter wird es, wenn die beiden enthaltenen Felder aus zwei verschiedenen Tabellen stammen:

```

SELECT "Tabelle1"."Feld1"||' ' > '||"Tabelle2"."Feld2"
FROM "Tabelle1", "Tabelle2"
WHERE "Tabelle1"."ID" = "Tabelle2"."FremdID"
ORDER BY "Tabelle1"."Feld1"||' ' > '||"Tabelle2"."Feld2" ASC

```

Hier müssen die Felder voneinander getrennt, die Tabellenzuordnungen zu den Feldern erfasst und die Fremdschlüsselzuweisung ermittelt werden.

```

stAbfrage = oFeldList.ListSource
aFelder() = Split(stAbfrage, " ")
stInhalt = ""
FOR i=LBound(aFelder)+1 TO UBound(aFelder)

```

Der Inhalt der Abfrage wird von Ballast befreit. Die Teile werden anschließend über eine nicht übliche Zeichenkombination zu einem Array wieder zusammengefügt. «FROM» trennt die sichtbare Feldanzeige von der Tabellenbezeichnung. «WHERE» trennt die Beziehungsdefinition von der Tabellenbezeichnung. Joins werden nicht unterstützt.

```

IF Trim(UCASE(aFelder(i))) = "ORDER BY" THEN
EXIT FOR
ELSEIF Trim(UCASE(aFelder(i))) = "FROM" THEN
stInhalt = stInhalt+" §§ "
ELSEIF Trim(UCASE(aFelder(i))) = "WHERE" THEN
stInhalt = stInhalt+" §§ "
ELSE
stInhalt = stInhalt+Trim(aFelder(i))
END IF
NEXT i
aInhalt() = Split(stInhalt, " §§ ")

```

Die sichtbare Feldanzeige wird gegebenenfalls in Inhalte aus verschiedenen Feldern aufgeteilt:

```

aErster() = Split(aInhalt(0), "||")
IF UBound(aErster) > 0 THEN
IF UBound(aInhalt) > 1 THEN

```

Der erste Teil enthält mindestens 2 Felder. Die Felder haben zu Beginn eine Tabellenbezeichnung. Der zweite Teil enthält zwei Tabellenbezeichnungen, die aber schon aus dem ersten Teil ermittelt werden können. Der dritte Teil enthält eine Beziehung über einen Fremdschlüssel mit «=>» getrennt:

```

aTest() = Split(aErster(0), ".")
NameTabelle1 = aTest(0)

```

```

NameTabellenFeld1 = aTest(1)
Erase aTest
Feldtrenner = Join(Split(aErster(1), ""), "")
aTest() = Split(aErster(2), ".")
NameTabelle2 = aTest(0)
NameTabellenFeld2 = aTest(1)
Erase aTest
aTest() = Split(aInhalt(2), "=")
aTest1() = Split(aTest(0), ".")
IF aTest1(1) <> "ID" THEN
    NameTab12ID = aTest1(1)
    IF aTest1(0) = NameTabelle1 THEN
        Position = 2
    ELSE
        Position = 1
    END IF
ELSE
    Erase aTest1
    aTest1() = Split(aTest(1), ".")
    NameTab12ID = aTest1(1)
    IF aTest1(0) = NameTabelle1 THEN
        Position = 2
    ELSE
        Position = 1
    END IF
END IF
END IF
ELSE

```

Der erste Teil enthält zwei Feldbezeichnungen ohne Tabellenbezeichnung mit Trenner, der zweite Teil enthält die Tabellenbezeichnung. Ein dritter Teil ist nicht vorhanden:

```

NameTabellenFeld1 = aErster(0)
Feldtrenner = Join(Split(aErster(1), ""), "")
NameTabellenFeld2 = aErster(2)
NameTabelle1 = aInhalt(1)
END IF
ELSE

```

Es existiert nur ein Feld, das aus einer Tabelle stammt:

```

NameTabellenFeld1 = aErster(0)
NameTabelle1 = aInhalt(1)
END IF

```

Die maximale Zeichenlänge, die eine Eingabe haben darf, wird im Folgenden mit der Funktion **Spaltengroesse** ermittelt. Das Kombinationsfeld kann hier mit seiner Beschränkung nicht sicher weiterhelfen, da ja ermöglicht werden soll, gleichzeitig 2 Felder in einem Kombinationsfeld einzutragen.

```

LaengeFeld1 = Spaltengroesse(NameTabelle1, NameTabellenFeld1)
IF NameTabellenFeld2 <> "" THEN
    IF NameTabelle2 <> "" THEN
        LaengeFeld2 = Spaltengroesse(NameTabelle2, NameTabellenFeld2)
    ELSE
        LaengeFeld2 = Spaltengroesse(NameTabelle1, NameTabellenFeld2)
    END IF
ELSE
    LaengeFeld2 = 0
END IF

```

Der Inhalt des Kombinationsfeldes wird ausgelesen:

```

stInhalt = oFeldList.GetCurrentValue()

```

Der angezeigte Inhalt des Kombinationsfeldes wird ausgelesen. Leertasten und nicht druckbare Zeichen am Anfang und Ende der Eingabe werden gegebenenfalls entfernt.

```

stInhalt = Trim(stInhalt)
IF stInhalt <> "" THEN
    IF NameTabellenFeld2 <> "" THEN

```

Wenn ein zweites Tabellenfeld existiert, muss der Inhalt des Kombinationsfeldes gesplittet werden. Um zu wissen, an welcher Stelle die Aufteilung vorgenommen werden soll, ist der Feldtrenner von Bedeutung, der der Funktion als Variable mitgegeben wird. Ein Leerzeichen aus dem Feldtrenner wird bei der Funktion «Split» nicht direkt erkannt. Deswegen wird das ASCII-Zeichen noch einmal in den entsprechenden Feldtrenner umgewandelt.

```
IF ASC(Feldtrenner) = 32 THEN
    Feldtrenner = " "
END IF
a_stTeile = Split(stInhalt, Feldtrenner, 2)
```

Der letzte Parameter weist darauf hin, dass maximal 2 Teile erzeugt werden.

Abhängig davon, welcher Eintrag mit dem Feld 1 und welcher mit dem Feld 2 zusammenhängt, wird jetzt der Inhalt des Kombinationsfeldes den einzelnen Variablen zugewiesen. «Position = 2» wird hier als Zeichen dafür genommen, dass an zweiter Position der Inhalt für das Feld 2 steht. Das ist auch dann der Fall, wenn beide Felder aus einer Tabelle stammen.

```
IF Position = 2 OR Position = 0 THEN
    stInhalt = Trim(a_stTeile(0))
    IF UBound(a_stTeile()) > 0 THEN
        stInhaltFeld2 = Trim(a_stTeile(1))
    ELSE
        stInhaltFeld2 = ""
    END IF
    stInhaltFeld2 = Trim(a_stTeile(1))
ELSE
    stInhaltFeld2 = Trim(a_stTeile(0))
    IF UBound(a_stTeile()) > 0 THEN
        stInhalt = Trim(a_stTeile(1))
    ELSE
        stInhalt = ""
    END IF
    stInhalt = Trim(a_stTeile(1))
END IF
END IF
```

Es kann passieren, dass bei zwei voneinander zu trennenden Inhalten die Größeneinstellung des Kombinationsfeldes (Textlänge) nicht zu einem der abzuspeichernden Tabellenfelder passt. Bei Kombinationsfeldern für nur ein Feld wird dies in der Regel durch Einstellungen im Formularkontrollfeld erledigt. Hier muss hingegen ein eventueller Fehler abgefangen werden. Es wird darauf hingewiesen, wie lang der Inhalt für das jeweilige Feld sein darf.

```
IF (LaengeFeld1 > 0 AND Len(stInhalt) > LaengeFeld1) OR
    (LaengeFeld2 > 0 AND Len(stInhaltFeld2) > LaengeFeld2) THEN
```

Wenn die Feldlänge des 1. oder 2. Teiles zu groß ist, wird erst einmal ein Standardtext in je einer Variablen abgespeichert. **CHR(13)** fügt hier einen Zeilenumbruch hinzu.

```
stmsgbox1 = "Das Feld " + NameTabellenFeld1 + " darf höchstens " +
    LaengeFeld1 + "Zeichen lang sein." + CHR(13)
stmsgbox2 = "Das Feld " + NameTabellenFeld2 + " darf höchstens " +
    LaengeFeld2 + "Zeichen lang sein." + CHR(13)
```

Sind beide Feldinhalte zu lang, so wird der Text mit beiden Feldinhalten ausgegeben.

```
IF (LaengeFeld1 > 0 AND Len(stInhalt) > LaengeFeld1) AND
    (LaengeFeld2 > 0 AND Len(stInhaltFeld2) > LaengeFeld2) THEN
    msgbox ("Der eingegebene Text ist zu lang." + CHR(13) +
        stmsgbox1 + stmsgbox2 + "Bitte den Text kürzen.",
        64, "Fehlerhafte Eingabe")
```

Die Anzeige erfolgt mit der Funktion 'msgbox()'. Sie erwartet zuerst einen Text, dann optional einen Zahlenwert (der zu einer entsprechenden Darstellungsform gehört) und schließlich optional einen Text als Überschrift über dem Fenster. Das Fenster hat hier also die Überschrift "Fehlerhafte Eingabe", die '64' fügt das Informationssymbol hinzu.

Im Folgenden werden alle auftretenden weiteren Fälle zu großer Textlänge abgearbeitet.

```

ELSEIF (LaengeFeld1 > 0 AND Len(stInhalt) > LaengeFeld1) THEN
    msgbox ("Der eingegebene Text ist zu lang." + CHR(13) +
        stmsgbox1 + "Bitte den Text kürzen.", 64, "Fehlerhafte Eingabe")
ELSE
    msgbox ("Der eingegebene Text ist zu lang." + CHR(13) +
        stmsgbox2 + "Bitte den Text kürzen.", 64, "Fehlerhafte Eingabe")
END IF
ELSE

```

Liegt kein zu langer Text vor, so kann die Funktion weiter durchlaufen. Ansonsten endet sie hier.

Jetzt werden die Inhaltseingaben so maskiert, dass eventuell vorhandene Hochkommata keine Fehlermeldung erzeugen.

```

stInhalt = String_to_SQL(stInhalt)
IF stInhaltFeld2 <> "" THEN
    stInhaltFeld2 = String_to_SQL(stInhaltFeld2)
END IF

```

Zuerst werden Variablen vorbelegt, die anschließend per Abfrage geändert werden können. Die Variablen **inID1** und **inID2** sollen den Inhalt der Primärschlüsselfelder der beiden Tabellen speichern. Da bei einer Abfrage, die kein Ergebnis wiedergibt, durch Basic einer Integer-Variablen 0 zugewiesen wird, dies aber für das Abfrageergebnis auch bedeuten könnte, dass der ermittelte Primärschlüssel eben den Wert 0 hat, wird die Variable auf jeweils -1 voreingestellt. Diesen Wert nimmt ein Autowert-Feld bei der HSQLDB nicht automatisch an.

Anschließend wird die Datenbankverbindung erzeugt, soweit sie nicht schon besteht.

```

inID1 = -1
inID2 = -1
oDatenquelle = ThisComponent.Parent.CurrentController
If NOT (oDatenquelle.isConnected()) Then
    oDatenquelle.connect()
End If
oVerbindung = oDatenquelle.ActiveConnection()
oSQL_Anweisung = oVerbindung.createStatement()
IF NameTabellenFeld2 <> "" AND NOT IsEmpty(stInhaltFeld2) AND
    NameTabelle2 <> "" THEN

```

Wenn ein zweites Tabellenfeld existiert, muss zuerst die zweite Abhängigkeit geklärt werden.

Zuerst wird überprüft, ob für den zweiten Wert in der Tabelle 2 bereits ein Eintrag existiert. Existiert dieser Eintrag nicht, so wird er eingefügt.

Beispiel: Die Tabelle 2 ist die Tabelle "Ort". In ihr werden also Orte abgespeichert. Ist z.B. ein Eintrag für den Ort 'Rheine' vorhanden, so wird der entsprechende Primärschlüsseleintrag ausgelesen. Ist der Eintrag 'Rheine' nicht vorhanden, wird er eingefügt und anschließend der beim Einfügen erzeugte Primärschlüsselwert festgestellt.

```

stSql = "SELECT ""ID"" FROM "" + NameTabelle2 + "" WHERE "" +
    NameTabellenFeld2 + ""=" + stInhaltFeld2 + ""
oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
WHILE oAbfrageergebnis.next
    inID2 = oAbfrageergebnis.getInt(1)
WEND
IF inID2 = -1 THEN
    stSql = "INSERT INTO "" + NameTabelle2 + "" ("" +
        NameTabellenFeld2 + "" ) VALUES ('" + stInhaltFeld2 + "'" ) "
    oSQL_Anweisung.executeUpdate(stSql)
    stSql = "CALL IDENTITY()"

```

Ist der Inhalt in der entsprechenden Tabelle nicht vorhanden, so wird er eingefügt. Der dabei entstehende Primärschlüsselwert wird anschließend ausgelesen. Ist der Inhalt bereits vorhanden, so wird der Primärschlüsselwert durch die vorangehende Abfrage ermittelt. Die Funktion geht hier von automatisch erzeugten Primärschlüsselfeldern (**IDENTITY**) aus.

```

oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
WHILE oAbfrageergebnis.next
    inID2 = oAbfrageergebnis.getInt(1)
WEND

```

END IF

Der Primärschlüssel aus dem zweiten Wert wird in der Variablen '**inID2**' zwischengespeichert. Jetzt wird überprüft, ob eventuell dieser Schlüsselwert bereits in der Tabelle 1 zusammen mit dem Eintrag aus dem ersten Feld vorhanden ist. Ist diese Kombination nicht vorhanden, so wird sie neu eingefügt.

Beispiel: Für den Ort 'Rheine' aus der Tabelle 2 können in der Tabelle 1 mehrere Postleitzahlen verfügbar sein. Ist die Kombination '48431' und 'Rheine' vorhanden, so wird nur der Primärschlüssel aus der Tabelle 1 ausgelesen, in der die Postleitzahlen und der Fremdschlüssel aus der Tabelle 2 gespeichert wurden.

```
stSql = "SELECT ""ID"" FROM "" + NameTabelle1 + "" WHERE "" +  
        NameTab12ID + ""=''' + inID2 + '' AND "" +  
        NameTabellenFeld1 + "" = ''' + stInhalt + ''"  
oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)  
WHILE oAbfrageergebnis.next  
    inID1 = oAbfrageergebnis.getInt(1)  
WEND
```

War der Inhalt der ersten Tabelle noch nicht vorhanden, so wird der Inhalt neu abgespeichert (**INSERT**).

Beispiel: Existiert bereits die Postleitzahl '48429' in Kombination mit dem Fremdschlüssel aus der Tabelle 2 "Ort", so wird auf jeden Fall ein neuer Datensatz erzeugt, wenn jetzt die Postleitzahl '48431' auftaucht. Der vorhergehenden Datensatz wird also nicht auf die neue Postleitzahl geändert. Schließlich sind durch die n:1-Verknüpfung der Tabellen mehrere Postleitzahlen für einen Ort ermöglicht worden.

```
IF inID1 = -1 THEN  
    stSql = "INSERT INTO "" + NameTabelle1 + "" ("" +  
            NameTabellenFeld1 + "", "" + NameTab12ID + """)  
            VALUES ('' + stInhalt + ', '' + inID2 + '' ) "  
    oSQL_Anweisung.executeUpdate(stSql)
```

Der Primärschlüssel der ersten Tabelle muss schließlich wieder ausgelesen werden, damit er in die dem Formular zugrundeliegende Tabelle übertragen werden kann.

```
stSql = "CALL IDENTITY()  
oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)  
WHILE oAbfrageergebnis.next  
    inID1 = oAbfrageergebnis.getInt(1)  
WEND  
END IF  
END IF
```

Für den Fall, dass beide in dem Kombinationsfeld zugrundeliegenden Felder in einer Tabelle gespeichert sind (z. B. Nachname, Vorname in der Tabelle Name) muss eine andere Abfrage erfolgen:

```
IF NameTabellenFeld2 <> "" AND NameTabelle2 = "" THEN  
    stSql = "SELECT ""ID"" FROM "" + NameTabelle1 + "" WHERE "" +  
            NameTabellenFeld1 + ""=''' + stInhalt + '' AND "" +  
            NameTabellenFeld2 + ""=''' + stInhaltFeld2 + ''"  
    oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)  
    WHILE oAbfrageergebnis.next  
        inID1 = oAbfrageergebnis.getInt(1)  
    WEND  
    IF inID1 = -1 THEN
```

Wenn eine zweite Tabelle nicht existiert:

```
stSql = "INSERT INTO "" + NameTabelle1 + "" ("" +  
        NameTabellenFeld1 + "", "" + NameTabellenFeld2 + """)  
        VALUES ('' + stInhalt + ', '' + stInhaltFeld2 + '' ) "  
oSQL_Anweisung.executeUpdate(stSql)
```

Anschließend wird das Primärschlüsselfeld wieder ausgelesen.

```
stSql = "CALL IDENTITY()
```

```

        oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
        WHILE oAbfrageergebnis.next
            inID1 = oAbfrageergebnis.getInt(1)
        WEND
    END IF
END IF
IF NameTabellenFeld2 = "" THEN

```

Jetzt wird der Fall geklärt, der der einfachste ist: Das 2. Tabellenfeld existiert nicht und der Eintrag ist noch nicht in der Tabelle vorhanden. In das Kombinationsfeld ist also ein einzelner neuer Wert eingetragen worden.

```

    stSql = "SELECT ""ID"" FROM "" + NameTabelle1 + "" WHERE "" +
        NameTabellenFeld1 + ""=''" + stInhalt + ""'"
    oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
    WHILE oAbfrageergebnis.next
        inID1 = oAbfrageergebnis.getInt(1)
    WEND
    IF inID1 = -1 THEN

```

Wenn ein zweites Tabellenfeld nicht existiert, wird der Inhalt neu eingefügt ...

```

        stSql = "INSERT INTO "" + NameTabelle1 + "" ("" +
            NameTabellenFeld1 + """) VALUES ('" + stInhalt + "') "
        oSQL_Anweisung.executeUpdate(stSql)

```

... und die entsprechende ID direkt wieder ausgelesen. (HSQLDB, FIREBIRD)

```

        stSql = "CALL IDENTITY()"
        oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
        WHILE oAbfrageergebnis.next
            inID1 = oAbfrageergebnis.getInt(1)
        WEND
    END IF
END IF

```

Der Wert des Primärschlüsselfeldes muss ermittelt werden, damit er in die Haupttabelle des Formulars übertragen werden kann.

Anschließend wird der aus all diesen Schleifen ermittelte Primärschlüsselwert in das Feld der Haupttabelle und die darunterliegende Datenbank übertragen. Mit **findColumn** wird das mit dem Formularfeld verbundene Tabellenfeld erreicht. Mit **updateLong** wird eine Integer-Zahl (siehe *Datentypen in StarBasic*) diesem Feld zugewiesen.

```

        oForm.updateLong(oForm.findColumn(oFeldList.Tag), inID1)
    END IF
ELSE

```

Ist kein Primärschlüsselwert einzutragen, weil auch kein Eintrag in dem Kombinationsfeld erfolgte oder dieser Eintrag gelöscht wurde, so ist auch der Inhalt des Feldes zu löschen. Mit **updateNULL()** wird das Feld mit dem datenbankspezifischen Ausdruck für ein leeres Feld, **NULL**, versehen.

```

        oForm.updateNULL(oForm.findColumn(oFeldList.Tag), NULL)
    END IF
NEXT inCom
END IF
END SUB

```

Kontrollfunktion für die Zeichenlänge der Kombinationsfelder

Die folgende Funktion soll die Zeichenlänge der jeweiligen Tabellenspalten ermitteln, damit zu lange Eingaben nicht einfach gekürzt werden. Der Typ **FUNCTION** wurde hier wegen der Rückgabewerte gewählt.

```

FUNCTION Spaltengroesse(Tabellenname AS STRING, Feldname AS STRING) AS INTEGER
    oDatenquelle = ThisComponent.Parent.CurrentController
    If NOT (oDatenquelle.isConnected()) Then
        oDatenquelle.connect()
    End If

```

```

oVerbindung = oDatenquelle.ActiveConnection()
oSQL_Anweisung = oVerbindung.createStatement()
stSql = "SELECT ""COLUMN_SIZE"" FROM ""INFORMATION_SCHEMA"". ""SYSTEM_COLUMNS""
        WHERE ""TABLE_NAME"" = '" + Tabellenname + "' AND ""COLUMN_NAME"" = '"
        + Feldname + "'"
oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
WHILE oAbfrageergebnis.next
    i = oAbfrageergebnis.getInt(1)
WEND
Spaltengroesse = i
END FUNCTION

```

Hinweis

Für FIREBIRD muss der SQL-Code angepasst werden:

```

stSql = "SELECT B.RDB$FIELD_LENGTH
        FROM RDB$RELATION_FIELDS AS A, RDB$FIELDS AS B
        WHERE A.RDB$FIELD_SOURCE = B.RDB$FIELD_NAME
              AND A.RDB$RELATION_NAME = '" + Tabellenname + "'
              AND A.RDB$FIELD_NAME = '" + Feldname + "'"

```

Datensatzaktion erzeugen

```
SUB Datensatzaktion_erzeugen(oEvent AS OBJECT)
```

Dieses Makro sollte an das folgende Ereignis des Listenfeldes gebunden werden: 'Bei Fokuserhalt'. Es ist notwendig, damit auf jeden Fall bei einer Änderung des Listenfeldinhaltes die Speicherung abläuft. Ohne dieses Makro wird keine Änderung in der Tabelle erzeugt, die für Base wahrnehmbar ist, da die Combobox mit dem Formular nicht verbunden ist.

Dieses Makro stellt direkt die Eigenschaft des Formulars um.

```

DIM oForm AS OBJECT
oForm = oEvent.Source.Model.Parent
oForm.Modified = TRUE
END SUB

```

Bei Formularen, die bereits ihren Inhalt auch für die Kombinationsfelder aus Abfragen erhalten, ist dieses Makro nicht notwendig. Änderungen in den Kombinationsfeldern werden direkt registriert.

Navigation von einem Formular zum anderen

Ein Formular soll über ein entsprechendes Ereignis geöffnet werden.

Im Formularkontrollfeld wird unter den Eigenschaften in der Zeile "Zusatzinformationen" (Tag) hier der Name des Formulars eintragen. Hier können auch weitere Informationen eingetragen werden, die über den Befehl **Split()** anschließend voneinander getrennt werden.

```

SUB Zu_Formular_von_Formular(oEvent AS OBJECT)
DIM stTag AS String
stTag = oEvent.Source.Model.Tag
aForm() = Split(stTag, ",")

```

Das Array wird gegründet und mit den Formularnamen gefüllt, in diesem Fall zuerst in dem zu öffnenden Formular und als zweites dem aktuellen Formular, dass nach dem Öffnen des anderen geschlossen werden soll. Existiert ein zweiter Eintrag nicht, so wird durch das Makro nur ein neues Formular geöffnet.

```

ThisDatabaseDocument.FormDocuments.getByName( Trim(aForm(0)) ).open
IF Ubound(aForm()) > 0 THEN
    ThisDatabaseDocument.FormDocuments.getByName( Trim(aForm(1)) ).close
END IF
END SUB

```

Soll stattdessen nur beim Schließen ein anderes Formular geöffnet werden, weil z.B. ein Hauptformular existiert und alle anderen Formulare von diesem aus über entsprechende Buttons angesteuert werden.

ert werden, so ist das folgende Makro einfach an das Formular unter **Extras → Anpassen → Ereignisse → Dokument wird geschlossen** anzubinden:

```
SUB Hauptformular_oeffnen
    ThisDatabaseDocument.FormDocuments.getByName( "Hauptformular" ).open
END SUB
```

Wenn die Formlardokumente innerhalb der *.odb-Datei in Verzeichnissen sortiert sind, so muss das Makro für den Formularwechsel etwas umfangreicher sein:

```
SUB Zu_Formular_von_Formular_mit_Ordner(oEvent AS OBJECT)
    REM Das zu öffnende Formular wird als erstes angegeben.
    REM Liegt ein Formular in einem Ordner, so ist die Beziehung über "/" zu
    REM definieren,
    REM so dass der Unterordner zu finden ist.
    DIM stTag AS STRING
    stTag = oEvent.Source.Model.Tag 'Tag wird unter den Zusatzinformationen eingegeben
    aForms() = Split(stTag, ",") 'Hier steht zuerst der Formularname für das neue
    'Formular, dann der für das alte Formular
    aForms1() = Split(aForms(0), "/")
    aForms2() = Split(aForms(1), "/")
    IF UBound(aForms1()) = 0 THEN
        ThisDatabaseDocument.FormDocuments.getByName( Trim(aForms1(0)) ).open
    ELSE
        ThisDatabaseDocument.FormDocuments.getByName( Trim(aForms1(0)) ).getByName(
            Trim(aForms1(1)) ).open
    END IF
    IF UBound(aForms2()) = 0 THEN
        ThisDatabaseDocument.FormDocuments.getByName( Trim(aForms2(0)) ).close
    ELSE
        ThisDatabaseDocument.FormDocuments.getByName( Trim(aForms2(0)) ).getByName(
            Trim(aForms2(1)) ).close
    END IF
END SUB
```

Formlardokumente, die in einem Verzeichnis liegen, werden in den Zusatzinformationen als Verzeichnis/Formular angegeben. Dies muss umgewandelt werden zu
...getByName("Verzeichnis").getByName("Formular").

Tabellen, Abfragen, Formulare und Berichte öffnen

Ähnlich wie im vorhergehenden Kapitel lassen sich von einem Formular aus auch Berichte öffnen. Berichte sind wie Formulare in die Base-Datei eingebundene separate Dokumente. Statt **FormDocuments** ist hier lediglich **ReportDocuments** einzutragen. Außerdem ist noch darauf zu achten, dass sowohl Formulare als auch Berichte in Unterverzeichnissen liegen können. Schwieriger ist es hingegen, auch auf Tabellen, Abfragen und Ansichten zuzugreifen, da diese nicht als separate Dokumente vorliegen.

```
SUB Navigation(oEvent AS OBJECT)
    DIM stTag AS STRING
    DIM inType AS INTEGER
    stTag = oEvent.Source.Model.Tag
    aOpen() = Split(stTag, ",")
    SELECT CASE Trim(aOpen(0))
        CASE "form", "report"
            REM Forms and Reports could be saved also in subfolders.
            aForms1() = Split(Trim(aOpen(1)), "/")
            IF Trim(aOpen(0)) = "form" THEN
                oDoc = ThisDatabaseDocument.FormDocuments
            ELSE
                oDoc = ThisDatabaseDocument.ReportDocuments
            END IF
            IF Ubound(aForms1()) > 0 THEN
                oDoc.getByName( Trim(aForms1(0)) ).getByName( Trim(aForms1(1)) ).open
            ELSE
```



```

        oDoc.getByName( Trim(aForms1(0)) ).open
    END IF
    IF Trim(aOpen(0)) = "form" AND Ubound(aOpen()) > 1 THEN
        REM The Form, which starts the Macro, could also be closed ...
        aForms2() = Split(Trim(aOpen(2)), "/")
        IF Ubound(aForms2()) > 0 THEN
            ThisDatabaseDocument.FormDocuments.
                getByName( Trim(aForms2(0)) ).getByName( Trim(aForms2(1)) ).close
        ELSE
            ThisDatabaseDocument.FormDocuments.
                getByName( Trim(aForms2(0)) ).close
        END IF
    END IF
    EXIT SUB
CASE "query"
    inType = 1
    Open_Table_Query_View(Trim(aOpen(1)), inType)
CASE "table"
    inType = 0
    Open_Table_Query_View(Trim(aOpen(1)), inType)
END SELECT
END SUB

```

Über die Prozedur Navigation wird das Makro gestartet. Von den Buttons wird aus den Zusatzinformationen (**Tag**) die Information ausgelesen, ob ein Formular (**form**), ein Bericht (**report**) usw. aufgerufen werden soll. Der Name des Formulars, Berichtes usw. wird in den Zusatzinformationen durch ein Komma von dieser Information getrennt.

Enthält der erste Teil des daraus ermittelten Arrays die Bezeichnung **form**, so wird anschließend das Formular geöffnet. Entsprechendes gilt für die Bezeichnung **report**, die den **SELECT CASE** für den Bericht ergibt.

Für Abfragen und Tabellen muss ein anderer Weg beschritten werden. Hier wird sowohl der Name der Abfrage bzw. Tabelle als auch eine Integer-Zahl an die folgende Prozedur **Open_Table_Query_View** weitergegeben.

```

SUB Open_Table_Query_View(stName AS STRING, inType AS INTEGER)
    DIM oController AS OBJECT
    DIM oConnection AS OBJECT
    oController = ThisDatabaseDocument.CurrentController
    IF NOT oController.isConnected THEN oController.connect
    oConnection = oController.ActiveConnection
    DIM URL AS NEW com.sun.star.util.URL
    DIM Args(5) AS NEW com.sun.star.beans.PropertyValue
    URL.Complete = ".component:DB/DataSourceBrowser"
    Dispatch = StarDesktop.queryDispatch(URL, "_Blank", 8)
    Args(0).Name = "ActiveConnection"
    Args(0).Value = oConnection
    Args(1).Name = "CommandType"
    Args(1).Value = inType '0=Table 1=SQL_Query 2=Command
    Args(2).Name = "Command"
    Args(2).Value = stName
    Args(3).Name = "ShowMenu"
    Args(3).Value = True
    Args(4).Name = "ShowTreeView"
    Args(4).Value = False
    Args(5).Name = "ShowTreeViewButton"
    Args(5).Value = False
    Dispatch.dispatch(URL, Args)
END SUB

```

Zuerst wird die Verbindung zur Datenbank hergestellt, sofern sie noch nicht existiert. Diese Verbindung muss mit einigen zusätzlichen Informationen, unter anderem der Art des zu öffnenden Elementes (Tabelle oder Abfrage) sowie dem Namen des Elementes, in einem Array weiter gegeben werden.

Die Tabelle bzw. Abfrage wird schließlich über den **queryDispatch** mit dem Kommando **dispatch** geöffnet.

Hierarchische Listenfelder

Einstellungen in einem Listenfeld sollen die Einstellungen in einem zweiten Listenfeld direkt beeinflussen. Auf einfachere Art und Weise wurde dies schon bei der Filterung von Datensätzen weiter oben beschrieben. Jetzt soll aber hinzu kommen, dass das erste Listenfeld den Inhalt des zweiten Listenfeldes beeinflusst, der wiederum den Inhalt des dritten Listenfeldes beeinflusst usw.

Jahrgang	Klasse	Name
1	a	Karl Müller
2	b	Evelyn Maier
3	c	Maria Gott
4	d	Eduard Abgefahren
5	e	Kurt Drechsler
6	f	Kunigunde Schimmel
7	g	
8		
9		
10		
11		
12		
13		

Abbildung 59: Beispielhafte Listenfelder für eine hierarchische Anordnung von Listenfeldern.

In diesem Beispiel enthält Listenfeld 1 alle Jahrgänge der Schule. Die Klassen der jeweiligen Jahrgänge sind durch Buchstaben kenntlich gemacht. Die Namen enthalten die Schülerinnen und Schüler der Klasse.

Unter normalen Umständen zeigt das Listenfeld für den Jahrgang alle 13 Jahrgänge an. Das Listenfeld für die Klasse alle Buchstaben und das Listenfeld für die Schüler und Schülerinnen alle Schüler und Schülerinnen der Schule.

Wird mit hierarchischen Listenfeldern gearbeitet, so wird nach Auswahl des Jahrganges das Listenfeld für die Klasse eingegrenzt. Es werden nur noch die Klassenbezeichnungen angezeigt, die es in dem Jahrgang tatsächlich gibt. So könnte eben bei steigender Schüler- und Schülerinnenzahl die Anzahl der Klassen im Jahrgang ebenfalls steigen. Das letzte Listenfeld, die Namen, ist jetzt bereits stark eingegrenzt. Statt alle vermutlich deutlich über 1000 Schüler und Schülerinnen anzuzeigen, listet das letzte Feld nur noch die ca. 30 Schüler und Schülerinnen der einen letztlich ausgewählten Klasse auf.

Zum Beginn steht nur die Auswahl des Jahrganges zur Verfügung. Ist ein Jahrgang ausgewählt, so steht die (bereits eingeschränkte) Auswahl der Klasse zur Verfügung. Erst zum Schluss wird schließlich das Listenfeld für die Namen freigegeben.

Wird das Listenfeld des Jahrganges geändert, so muss der Durchlauf wieder wie vorher starten. Wird nur das Listenfeld der Klasse geändert, so muss der Wert des Jahrganges für das letzte Listenfeld der Namen weiter gelten.

Um solch eine Funktion bereitzustellen, muss innerhalb eines Formulars eine Variable zwischengespeichert werden. Dies erfolgt in einem versteckten Kontrollfeld.

Der Makrostart wird an die Veränderung des Inhaltes eines Listenfeldes gekoppelt: **Eigenschaft Listenfeld → Ereignisse → Modifiziert**. In den Zusatzinformationen des Listenfeldes werden die notwendigen Variablen gespeichert.

Hier der beispielhafte Inhalt der Zusatzinformationen:

MainForm, Jahrgang, verstecktes Kontrollfeld, Listenfeld 2

Das Formular hat die Bezeichnung «MainForm». Das aktuelle Listenfeld ist als «Listenfeld 1» bezeichnet. Dieses Listenfeld stellt den Inhalt des Tabellenfeldes «Jahrgang» dar. Nach diesem Eintrag muss also das darauffolgende Listenfeld gefiltert werden. Das versteckte Kontrollfeld ist in diesem Fall auch gleich mit dem entsprechenden Namen gekennzeichnet. Und schließlich wird noch darauf hingewiesen, dass ein 2. Listenfeld, «Listenfeld 2», existiert, an das die Filterung weiter gegeben wird.

```
SUB Hierarchisches_Kontrollfeld(oEvent AS OBJECT)
    DIM oDoc AS OBJECT
    DIM oDrawpage AS OBJECT
    DIM oForm AS OBJECT
    DIM oFeldHidden AS OBJECT
    DIM oFeld AS OBJECT
    DIM oFeld1 AS OBJECT
    DIM stSql AS STRING
    DIM aInhalt()
    DIM stTag AS STRING
    oFeld = oEvent.Source.Model
    stTag = oFeld.Tag
    oForm = oFeld.Parent
    REM Tag wird unter den Zusatzinformationen eingegeben
    REM Hier steht:
    REM 0. Feldname des zu filterndes Feldes in der Tabelle,
    REM 1. Feldname des versteckten Konrollfeldes, das den Filterwert speichern soll,
    REM 2. eventuell weiteres Listfeld
    REM Der Tag wird von dem auslösenden Element ausgelesen. Die Variable wird an die
    REM Prozedur weitergegeben, die gegebenenfalls alle weiteren Listenfelder
    REM einstellt.
    aFilter() = Split(stTag, ",")
    stFilter = ""
```

Nachdem die Variablen deklariert wurden, wird der Inhalt des Tags in ein Array übertragen. So kann auf die einzelnen Elemente zugegriffen werden. Anschließend wird der Zugang zu den verschiedenen Feldern im Formular deklariert.

Das Listenfeld wird aus dem Aufruf heraus ermittelt. Aus dem Listenfeld wird der Wert ausgelesen. Nur wenn dieser Wert einen Inhalt hat, wird er mit dem Feldnamen des zu filternden Feldes, in unserem Beispiel «Jahrgang», zu einer SQL-Bedingung kombiniert. Ansonsten bleibt der Filter leer. Sind die Listenfelder zur Filterung eines Formulars gedacht, dann ist kein verstecktes Kontrollfeld vorhanden. Unter dieser Bedingung wird der Filterwert direkt im Formular gespeichert.

```
IF Trim(aFilter(1)) = "" THEN
    IF oFeld.getCurrentValue <> "" THEN
        stFilter = """"+Trim(aFilter(0))+""""=''+oFeld.getCurrentValue()+""""
```

Existiert bereits vorher ein Filter (weil es sich z.B. um das Listenfeld 2 handelt, das jetzt betätigt wurde), so wird der neue Inhalt an den vorherigen angehängt, der in dem versteckten Feld zwischengespeichert wurde.

```
IF oForm.Filter <> ""
```

Dies darf allerdings nur dann geschehen, wenn das gleiche Feld noch nicht gefiltert wurde. Schließlich ist z.B. bei einer Filterung nach dem «Jahrgang» kein Datensatz unter «Name» mehr zu erwarten, wenn zusätzlich eine weitere Filterung nach «Jahrgang» erfolgt. Eine Person kann immer nur in einem «Jahrgang» existieren. Es muss also ausgeschlossen werden, dass in der Filterung der Filtername bereits vorkommt.

```
AND InStr(oForm.Filter, """"+Trim(aFilter(0))+""""='') = 0 THEN
stFilter = oForm.Filter + " AND " + stFilter
```

Existiert bereits ein Filter und kommt das Feld, nach dem gefiltert werden soll, bereits im Filter vor, so muss die vorherige Filterung ab diesem Feldnamen gelöscht und die neue Filterung eingefügt werden.

```
ELSEIF oForm.Filter <> "" THEN
stFilter = Left(oForm.Filter,
InStr(oForm.Filter, """"+Trim(aFilter(0))+""""='')-1) + stFilter
END IF
END IF
```

Anschließend wird der Filter in das Formular eingetragen. Dieser Filter kann auch leer sein, wenn direkt das erste Listenfeld ohne Inhalt gewählt wurde.

```
oForm.Filter = stFilter
oForm.reload()
```

Die gleiche Prozedur wird durchlaufen, wenn nicht ein Formular direkt gefiltert werden soll. In dem Fall wird der Filterwert in einem versteckten Kontrollfeld zwischengespeichert.

```
ELSE
oFeldHidden = oForm.GetByName(Trim(aFilter(1)))
IF oFeld.GetCurrentValue <> "" THEN
stFilter = """"+Trim(aFilter(0))+""""=''+oFeld.GetCurrentValue()+""
IF oFeldHidden.HiddenValue <> ""
AND InStr(oFeldHidden.HiddenValue, """"+Trim(aFilter(0))+""""='') = 0 THEN
stFilter = oFeldHidden.HiddenValue + " AND " + stFilter
ELSEIF oFeldHidden.HiddenValue <> "" THEN
stFilter = Left(oFeldHidden.HiddenValue,
InStr(oFeldHidden.HiddenValue, """"+Trim(aFilter(0))+""""='')-1) +
stFilter
END IF
END IF
oFeldHidden.HiddenValue = stFilter
END IF
```

Ist in den Zusatzinformationen ein 4. Eintrag (Arraynummerierung beginnt bei 0!) vorhanden, so muss das folgende Listenfeld jetzt auf den entsprechenden Eintrag des aufrufenden Listenfeldes eingestellt werden.

```
IF UBound(aFilter()) > 1 THEN
oFeld1 = oForm.GetByName(Trim(aFilter(2)))
aFilter1() = Split(oFeld1.Tag, ",")
```

Die notwendigen Daten für die Filterung werden aus den Zusatzinformationen («Tag») des entsprechenden Listenfeldes ausgelesen. Leider ist es nicht möglich, lediglich den SQL-Code in dem Listenfeld neu zu schreiben und anschließend das Listenfeld einzulesen. Vielmehr müssen die entsprechenden Werte direkt in das Listenfeld geschrieben werden.

Bei der Erstellung des Codes wird davon ausgegangen, dass die Tabelle, auf der das Formular beruht, die gleiche ist, auf der auch die Listenfelder beruhen. Für eine Weitergabe von Fremdschlüsseln an die Tabelle ist so ein Listenfeld also erst einmal nicht gedacht.

```
IF oFeld.GetCurrentValue <> "" THEN
stSql = "SELECT DISTINCT """"+Trim(aFilter1(0))+"""" FROM """"+oForm.Command+
"""" WHERE "+stFilter+" ORDER BY """"+Trim(aFilter1(0))+""""
oDatenquelle = ThisComponent.Parent.CurrentController
If NOT (oDatenquelle.isConnected()) THEN
oDatenquelle.connect()
END IF
oVerbindung = oDatenquelle.ActiveConnection()
```

```
oSQL_Anweisung = oVerbindung.createStatement()
oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
```

Die Werte werden in ein Array eingelesen. Das Array wird anschließend direkt in das Listenfeld übertragen. Die entsprechenden Zähler für das Array werden durch die Schleife kontinuierlich erhöht.

```
inZaehler = 0
WHILE oAbfrageergebnis.next
    ReDim Preserve aInhalt(inZaehler)
    aInhalt(inZaehler) = oAbfrageergebnis.getString(1)
    inZaehler = inZaehler+1
WEND
ELSE
    aInhalt(0) = ""
END IF
oFeld1.StringItemList = aInhalt()
```

Der Inhalte des Listenfeldes wurde neu erstellt. Das Listenfeld muss neu eingelesen werden. Anschließend wird anhand der Zusatzinformationen des neu eingestellten Listenfeldes jedes eventuell weiter folgende Listenfeld entsprechend geleert, indem eine Schleife für alle folgenden Listenfelder gestartet wird, bis eben ein letztes Listenfeld keinen 4. Eintrag in den Zusatzinformationen enthält.

```
oFeld1.refresh()
WHILE UBound(aFilter1()) > 1
    DIM aLeer()
    oFeld2 = oForm.getByName(Trim(aFilter1(2)))
    DIM aFilter1()
    aFilter1() = Split(oFeld2.Tag, ",")
    oFeld2.StringItemList = aLeer()
    oFeld2.refresh()
WEND
END IF
END SUB
```

Die sichtbaren Inhalte des Listenfeldes werden in `oFeld1.StringItemList` gespeichert. Soll zusätzlich auch ein Wert gespeichert werden, der als Fremdschlüssel an die darunterliegende Tabelle weitergegeben wird, wie bei Listenfeldern in Formularen üblich, so ist dieser Wert in der Abfrage zusätzlich zu ermitteln und anschließend mit `oFeld1.ValueItemList` abzuspeichern.

Für so eine Erweiterung sind allerdings zusätzliche Variablen notwendig wie z.B. neben der Tabelle, in der die Werte des Formulars gespeichert werden, noch die Tabelle, aus der die Listenfeldinhalte gelesen werden.

Besondere Aufmerksamkeit ist dabei der Formulierung des Filters zu widmen.

```
stFilter = """"+Trim(aFilter(1))+""""=''+oFeld.getCurrentValue()+'''
```

funktioniert dann nur noch, wenn es sich bei der zugrundeliegenden LO-Version um eine Version ab LO 4.1 handelt, da hier als `CurrentValue()` der Wert wiedergegeben wird, der auch abgespeichert wird – nicht der Wert, der lediglich angezeigt wird. Damit das einwandfrei über verschiedene Versionen hinweg funktioniert, sollte unter **Eigenschaften: Listenfeld → Daten → Gebundenes Feld → '0'** angegeben sein.

Zeiteingaben mit Millisekunden

Um Zeiten im Millisekunden-Bereich zu speichern, ist in der Tabelle ein Timestamp-Feld erforderlich, das zudem per SQL separat darauf eingestellt wird (siehe [Zeitfelder in Tabellen](#)) (`HSQLDB`, `FIREBIRD` erlaubt auch Millisekunden für normale Zeiten). Ein solches Feld kann vom Formular aus mit einem formatierten Feld beschrieben werden, das auch das Format **MM:SS,00** anbietet. Allerdings scheitert der erste Schreibversuch daran, dass der Eingabe der entsprechende Datumszusatz fehlt. Dies kann mit dem folgenden Makro erreicht werden, das an die Eigenschaft «Vor der Datensatzaktion» des Formulars gebunden wird:

```
SUB Timestamp
```

```

DIM unoStmp AS NEW com.sun.star.util.DateTime
DIM oDoc AS OBJECT
DIM oDrawpage AS OBJECT
DIM oForm AS OBJECT
DIM oFeld AS OBJECT
DIM stZeit AS STRING
DIM ar()
DIM arMandS()
DIM loNano AS LONG
DIM inSecond AS INTEGER
DIM inMinute AS INTEGER
oDoc = thisComponent
oDrawpage = oDoc.Drawpage
oForm = oDrawpage.Forms.getByName("MainForm")
oFeld = oForm.getByName("Zeit")
stZeit = oFeld.Text

```

Die Variablen werden vorher deklariert. Nur wenn das Feld «Zeit» einen Inhalt hat, wird der weitere Code ausgeführt. Sonst tritt der Mechanismus des Formulars in Kraft, der das Feld auf **NULL** setzt.

```

IF stZeit <> "" THEN
    ar() = Split(stZeit, ",")
    loNano = CLng(ar(1) & "00000000")
    arMandS() = Split(ar(0), ":")
    inSecond = CInt(arMandS(1))
    inMinute = CInt(arMandS(0))

```

Die Einträge aus dem Feld «Zeit» werden in ihre Bestandteile zerlegt.

Zuerst werden die Hundertstelsekunden abgetrennt und mit so vielen Nullen rechts aufgefüllt, dass sich insgesamt eine neunstellige Zahl ergibt. Eine so hohe Zahl kann nur in einer Long-Variablen gespeichert werden.

Anschließend werden aus dem verbleibenden Rest durch eine Trennung am Trennzeichen «:» die Minuten von den Sekunden getrennt und in Integer-Zahlen umgewandelt.

```

WITH unoStmp
    .NanoSeconds = loNano
    .Seconds = inSecond
    .Minutes = inMinute
    .Hours = 0
    .Day = 30
    .Month = 12
    .Year = 1899
END WITH

```

Dem Zeitstempel wird nun das Standarddatum 30.12.1899 zugewiesen, das dem Standard-Startdatum von LibreOffice entspricht. Hier kann natürlich auch das aktuelle Datum mitgespeichert werden.

Hinweis

Aktuelles Datum ermitteln und speichern:

```

DIM Jetzt AS DATE
Jetzt = Now()
WITH unoStmp
    .NanoSeconds = loNano
    .Seconds = inSecond
    .Minutes = inMinute
    .Hours = Hour(Jetzt)
    .Day = Day(Jetzt)
    .Month = Month(Jetzt)
    .Year = Year(Jetzt)
END WITH

```

```

oFeld.BoundField.updateTimestamp(unoStmp)
END IF
END SUB

```

Anschließend wird der erzeugte Zeitstempel über **updateTimestamp** in das Feld übertragen und mit dem Formular abgespeichert.

In älteren Anleitungen wird hier statt **NanoSeconds** der Begriff **HundrethSeconds** verwendet. Dieser entspricht aber nicht der API von LibreOffice und erzeugt deshalb nur Fehlermeldungen.

Ein Ereignis – mehrere Implementationen

Bei Formularen kommt es vor, dass ein Makro, mit einem Ereignis verknüpft, gleich zweimal ausgeführt wird. Dies liegt daran, dass mehrere Prozesse gleichzeitig z.B. mit dem Abspeichern eines geänderten Datensatzes verbunden sind. Die unterschiedlichen Ursachen für so ein Ereignis lassen sich folgendermaßen ermitteln:

```
SUB Ereignisursache_ermitteln(oEvent AS OBJECT)
    DIM oForm AS OBJECT
    oForm = oEvent.Source
    MsgBox oForm.ImplementationName
END SUB
```

Beim Abspeichern eines geänderten Datensatzes ergeben sich so zwei Implementationsnamen: **org.openoffice.comp.svx.FormController** und **com.sun.star.comp.forms.ODatabaseForm**. Über diese Namen kann jetzt gesteuert werden, dass ein Makro letztlich nur einmal den ganzen Code durchläuft. Die doppelte Durchführung ist oft nur eine (kleine) Bremse im Programmablauf. Sie kann aber auch dazu führen, dass sich z.B. ein Cursor nicht nur einen, sondern gleich zwei Datensätze zurück bewegt. Die Implementationen lassen auch nur bestimmte Befehle zu, so dass eine Kenntnis des Namens der Implementation von Bedeutung sein kann.

Abspeichern mit Nachfrage

Bei komplexen Änderungen eines Datensatzes kann es sinnvoll erscheinen, vor der Ausführung noch einmal nachzufragen, ob die Änderungen denn durchgeführt werden sollen. Wird der Dialog mit **Nein** beantwortet, so wird der Speicherprozess abgebrochen, die Änderungen verworfen und der Cursor bleibt im aktuellen Datensatz stehen.

```
SUB Speichernachfrage(oEvent AS OBJECT)
    DIM oFormFeature AS OBJECT
    DIM oFormOperations AS OBJECT
    DIM inAntwort AS INTEGER
    oFormFeature = com.sun.star.form.runtime.FormFeature
    SELECT CASE oEvent.Source.ImplementationName
        CASE "org.openoffice.comp.svx.FormController"
            inAntwort = MsgBox ("Soll der Datensatz geändert werden?" ,
                4 , "Datensatzänderung")
            SELECT CASE inAntwort
                CASE 6 ' Ja gedrückt, kein weiterer Eingriff nötig
                CASE 7 ' Nein gedrückt, Speichern unterbrechen
                    oFormOperations = oEvent.Source.FormOperations
                    oFormOperations.execute(oFormFeature.UndoRecordChanges)
            CASE ELSE
            END SELECT
        CASE "com.sun.star.comp.forms.ODatabaseForm"
    END SELECT
END SUB
```

Es gibt zwei Auslösemomente mit unterschiedlichem Implementationsnamen. Diese beiden Implementationsnamen werden in **SELECT CASE** unterschieden. Nur wenn es sich um den **FormController** handelt, wird der Code ausgeführt. Nur im **FormController** findet sich auch die Variable **FormOperations**.

Außer **Ja** und **Nein** kann auch das Schließkreuz gedrückt werden. Dies gibt aber den gleichen Wert wie **Nein** zurück: '7'.

Wird lediglich mit den Tabulatortasten durch die Formulare navigiert, so kommt nur die Nachfrage der MessageBox. Bei der Navigation über die Navigationsleiste hingegen erscheint noch die Meldung, dass der Datensatz nicht verändert wurde.

Primärschlüssel aus Nummerierung und Jahreszahl

Bei der Erstellung von Rechnungen werden jährlich Bilanzen gezogen. Das führt manchmal zu dem Wunsch, die Rechnungstabellen einer Datenbank nach Jahren getrennt zu sichern und jedes Jahr mit einer neuen Tabelle zu beginnen.

Die folgende Makrolösung geht einen anderen Weg. Sie schreibt automatisch den Wert für das Feld «ID» in die Tabelle, berücksichtigt dabei aber das «Jahr», das in der Tabelle als zweiter Primärschlüssel existiert. So tauchen dann in der Tabelle als Primärschlüssel z.B. die folgenden Werte auf:²⁶

<i>Jahr</i>	<i>ID</i>
2014	1
2014	2
2014	3
2015	1
2015	2

Damit lässt sich eine auf das Jahr bezogene Übersicht auch in den Dokumenten besser erzeugen.

```
SUB Datum_aktuell_ID_einfuegen
  DIM oDatenquelle AS OBJECT
  DIM oVerbindung AS OBJECT
  DIM oSQL_Anweisung AS OBJECT
  DIM stSql AS STRING
  DIM oAbfrageergebnis AS OBJECT
  DIM oDoc AS OBJECT
  DIM oDrawpage AS OBJECT
  DIM oForm AS OBJECT
  DIM oFeld1 AS OBJECT
  DIM oFeld2 AS OBJECT
  DIM oFeld3 AS OBJECT
  DIM inIDneu AS INTEGER
  DIM inYear AS INTEGER
  DIM unoDate
  oDoc = thisComponent
  oDrawpage = oDoc.drawpage
  oForm = oDrawpage.forms.getByName("MainForm")
  oFeld1 = oForm.getByName("fmtJahr")
  oFeld2 = oForm.getByName("fmtID")
  oFeld3 = oForm.getByName("datDatum")
  IF IsEmpty(oFeld2.getCurrentValue()) THEN
    IF IsEmpty(oFeld3.getCurrentValue()) THEN
      unoDate = createUnoStruct("com.sun.star.util.Date")
      unoDate.Year = Year(Date)
      unoDate.Month = Month(Date)
      unoDate.Day = Day(Date)
      inYear = Year(Date)
    ELSE
      inYear = oFeld3.CurrentValue.Year
    END IF
    oDatenquelle = ThisComponent.Parent.CurrentController
    IF NOT (oDatenquelle.isConnected()) THEN
      oDatenquelle.connect()
    END IF
```

²⁶ Dem Handbuch liegt die Datenbank «Beispiel_Fortlaufende_Nummer_Jahr.odt» bei.


```

oVerbindung = oDatenquelle.ActiveConnection()
oSQL_Anweisung = oVerbindung.createStatement()
stSql = "SELECT MAX( "ID" )+1 FROM "Auftraege" WHERE "Jahr" = '"
      + inYear + "'"
oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
WHILE oAbfrageergebnis.next
    inIDneu = oAbfrageergebnis.getInt(1)
WEND
IF inIDneu = 0 THEN
    inIDneu = 1
END IF
oFeld1.BoundField.updateInt(inYear)
oFeld2.BoundField.updateInt(inIDneu)
IF IsEmpty(oFeld3.getCurrentValue()) THEN
    oFeld3.BoundField.updateDate(unoDate)
END IF
END IF
END SUB

```

Alle Variablen werden deklariert. Die Formularfelder in dem Hauptformular werden angesteuert. Der Rest des Codes läuft nur ab, wenn der Eintrag für das Feld «fmtID» noch leer ist. Dann wird, wenn nicht schon ein Datum eingegeben wurde, zuerst ein Datumsstruct erstellt, um das aktuelle Datum und das aktuelle Jahr in die entsprechenden Felder übertragen zu können. Anschließend wird der Kontakt zu der Datenbank aufgebaut, sofern noch kein Kontakt existiert. Es wird zu dem höchsten Eintrag des Feldes "ID", bezogen auf das Jahr des Datumsfeldes, der Wert '1' addiert. Bleibt die Abfrage leer, so existiert noch kein Eintrag in dem Feld "ID". Jetzt könnte genauso gut '0' direkt in das Feld «fmtID» eingetragen werden. Die Nummerierung für die Aufträge sollte aber mit '1' beginnen, so dass der Variablen «inIDneu» eine '1' zugewiesen wird.

Die ermittelten Werte für das Jahr, die ID und, sofern nicht bereits ein Datum eingetragen wurde, das aktuelle Datum, werden schließlich in das Formular übertragen.

Im Formular sind die Felder für die Primärschlüssel "ID" und "Jahr" schreibgeschützt. Die Zuweisung kann so nur durch das Makro erfolgen.

Datenbankaufgaben mit Makros erweitert

Verbindung mit Datenbanken erzeugen

```

oDatenquelle = ThisComponent.Parent.DataSource
IF NOT oDatenquelle.IsPasswordRequired THEN
    oVerbindung = oDatenquelle.GetConnection("", "")

```

Hier wäre es möglich, fest einen Benutzernamen und ein Passwort einzugeben, wenn eine Passworteingabe erforderlich wäre. In den Klammern steht dann ("Benutzername", "Passwort"). Statt einen Benutzernamen und ein Passwort in Reinschrift einzutragen, wird für diesen Fall der Dialog für den Passwortschutz aufgerufen:

```

ELSE
    oAuthentifizierung = createUnoService("com.sun.star.sdb.InteractionHandler")
    oVerbindung = oDatenquelle.ConnectWithCompletion(oAuthentifizierung)
END IF

```

Wird allerdings von einem Formular innerhalb der Base-Datei auf die Datenbank zugegriffen, so reicht bereits

```

oDatenquelle = ThisComponent.Parent.CurrentController
IF NOT (oDatenquelle.isConnected()) Then
    oDatenquelle.connect()
End IF
oVerbindung = oDatenquelle.ActiveConnection()

```

Die Datenbank ist hier bekannt, ein Nutzernamen und ein Passwort sind nicht erforderlich, da diese bereits in den Grundeinstellungen der HSQLDB für die interne Version ausgeschaltet sind.

Für Formulare außerhalb von Base wird die Verbindung über das erste Formular hergestellt:

```
oDatenquelle = Thiscomponent.Drawpage.Forms(0)
oVerbindung = oDatenquelle.activeConnection
```

Daten von einer Datenbank in eine andere kopieren

Die interne Datenbank ist erst einmal eine Ein-Benutzer-Datenbank. Die Daten werden innerhalb der *.odb-Datei abgespeichert. Ein Austausch von Daten zwischen verschiedenen Datenbankdateien ist eigentlich nicht vorgesehen, über Export und Import allerdings möglich.

Manchmal werden aber auch *.odb-Dateien so eingesetzt, dass ein möglichst automatischer Datenaustausch von einer Datenbankdatei zu einer anderen erfolgen soll. Die folgende Prozedur kann da hilfreich sein.²⁷

Nach der Deklaration der Variablen wird der Pfad der aktuellen Datenbankdatei von einem Button im Formular aus ausgelesen. Von dem Pfad wird der Dateiname abgetrennt. Die Zieldatei für die Daten befindet sich ebenfalls in dem Verzeichnis. Der Name dieser Datei wird jetzt an den Pfad angehängt, damit der Kontakt zur Zieldatenbankdatei erstellt werden kann.

Der Kontakt zur Ausgangsdatenbank wird im Verhältnis zum Formular ermittelt, in dem der Button liegt: **ThisComponent.Parent.CurrentController**. Der Kontakt zur externen Datenbank wird über den **DatabaseContext** und den Pfad zur Datenbank erstellt.

```
SUB Datenkopie
  DIM oDatabaseContext AS OBJECT
  DIM oDatenquelle AS OBJECT
  DIM oDatenquelleZiel AS OBJECT
  DIM oVerbindung AS OBJECT
  DIM oVerbindungZiel AS OBJECT
  DIM oDB AS OBJECT
  DIM oSQL_Anweisung AS OBJECT
  DIM oSQL_AnweisungZiel AS OBJECT
  DIM oAbfrageergebnis AS OBJECT
  DIM oAbfrageergebnisZiel AS OBJECT
  DIM stSql AS String
  DIM stSqlZiel AS String
  DIM inID AS INTEGER
  DIM inIDZiel AS INTEGER
  DIM stName AS STRING
  DIM stOrt AS STRING
  oDB = ThisComponent.Parent
  stDir = Left(oDB.Location, Len(oDB.Location)-Len(oDB.Title))
  stDir = ConvertToUrl(stDir & "ZielDB.odb")
  oDatenquelle = ThisComponent.Parent.CurrentController
  If NOT (oDatenquelle.isConnected()) THEN
    oDatenquelle.connect()
  END IF
  oVerbindung = oDatenquelle.ActiveConnection()
  oDatabaseContext = createUnoService("com.sun.star.sdb.DatabaseContext")
  oDatenquelleZiel = oDatabaseContext.getByName(stDir)
  oVerbindungZiel = oDatenquelleZiel.GetConnection("", "")
  oSQL_Anweisung = oVerbindung.createStatement()
  stSql = "SELECT * FROM ""Tabelle""
  oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
  WHILE oAbfrageergebnis.next
    inID = oAbfrageergebnis.getInt(1)
    stName = oAbfrageergebnis.getString(2)
    stOrt = oAbfrageergebnis.getString(3)
    oSQL_AnweisungZiel = oVerbindungZiel.createStatement()
    stSqlZiel = "SELECT ""ID"" FROM ""Tabelle"" WHERE ""ID"" = '"+inID+"'"
    oAbfrageergebnisZiel = oSQL_AnweisungZiel.executeQuery(stSqlZiel)
    inIDZiel = - 1
    WHILE oAbfrageergebnisZiel.next
```

²⁷ Das Beispiel "Datenkopie_Quelle_Ziel" ist als gepacktes Verzeichnis diesem Handbuch beigelegt.

```

        inIDZiel = oAbfrageergebnisZiel.getInt(1)
    WEND
    IF inIDZiel = - 1 THEN
        stSqlZiel = "INSERT INTO ""Tabelle"" (""ID"", ""Name"", ""Ort"") VALUES
            ('"+inID+"', '"+stName+"', '"+stOrt+"')"
        oSQL_AnweisungZiel.executeUpdate(stSqlZiel)
    END IF
WEND
END SUB

```

Die komplette Tabelle der Ausgangsdatenbank wird ausgelesen und Zeile für Zeile anschließend über den Kontakt zur Zieldatenbank in die Tabelle der Zieldatenbank eingefügt. Vor dem Einfügen wird allerdings getestet, ob der Wert für den Primärschlüssel bereits vorhanden ist. Ist der Schlüsselwert vorhanden, so wird der Datensatz nicht kopiert.

Hier könnte gegebenenfalls auch eingestellt werden, dass statt einer Kopie des Datensatzes ein Update des bereits existierenden Datensatzes erfolgen soll. Auf jeden Fall wird so sichergestellt, dass die Zieldatenbank die Datensätze mit den entsprechenden Primärschlüsseln der Quelldatenbank enthält.

Direkter Import von Daten aus Calc

Häufig passiert es, dass Calc statt einer Datenbank zur Eingabe von Daten in eine Tabelle genutzt wird. Solche Daten lassen sich dann über die Zwischenablage oder per Drag-And-Drop in eine Base-Tabelle einlesen. Auch der Export in eine bereits in Base eingebundene *.csv-Textdatei ist möglich.

Soll allerdings Calc auf Dauer zur Dateneingabe genutzt werden und die Daten regelmäßig aus Calc ausgelesen werden, so ist der Kopierschritt vielleicht zu umständlich. Hier setzt das folgende Makro an, das aus einem Formular heraus über einen Button gestartet wird.²⁸

Das Makro geht von folgenden Voraussetzungen aus:

1. Die Daten liegen auf dem ersten Tabellenblatt des Calc-Dokuments
2. Auf diesem Tabellenblatt liegen nur die Daten in einer Spalte, nicht zusätzliche Einträge.
3. Die erste Datenzeile enthält die Feldbenennungen, die genau den Feldbezeichnungen in der Base-Tabelle entsprechen.

Die Dateneingabe muss nicht links oben auf dem Tabellenblatt erfolgen. Auch müssen die Felder nicht die gleiche Reihenfolge wie in der Tabelle haben. Spalten, deren Spaltenüberschrift nicht Feldern der Tabelle entspricht, werden ignoriert.

```

Sub CalcDataImport(oEvent AS OBJECT)
    DIM oDatasource AS OBJECT
    DIM oConnection AS OBJECT
    DIM oSQL_Command AS OBJECT
    DIM oResult AS OBJECT
    DIM oDB AS OBJECT
    DIM oDoc AS OBJECT
    DIM oDocView AS OBJECT
    DIM oRange AS OBJECT
    DIM Arg()
    DIM aColumn()
    DIM aColumns()
    DIM aType()
    DIM stSql AS STRING
    DIM stRow AS STRING
    DIM stDir AS STRING
    DIM stColumns AS STRING
    DIM stStartCol AS STRING
    DIM stEndCol AS STRING
    DIM stStartRow AS STRING
    DIM stEndRow AS STRING

```

²⁸ Die Beispieldatenbank «Beispiel_Daten_Import.odt» liegt diesem Handbuch bei.

```

DIM stRange AS STRING
DIM inCounter AS INTEGER
DIM loColumns AS LONG
DIM loRows AS LONG
DIM loID AS LONG
oDatasource = ThisComponent.Parent.CurrentController
If NOT (oDatasource.isConnected()) THEN
    oDatasource.connect()
END IF
oConnection = oDatasource.ActiveConnection()
oSQL_Command = oConnection.createStatement()

```

Nach der Deklaration der Variablen werden Spaltennamen und Spaltentypen aus der vorgegebenen Tabelle der Datenbank ausgelesen. Der Primärschlüssel der Tabelle mit der Bezeichnung "ID" wird als Integer-Feld unabhängig von der Calc-Tabelle erstellt.

```

stSql = "SELECT COLUMN_NAME, TYPE_NAME FROM INFORMATION_SCHEMA.SYSTEM_COLUMNS
        WHERE TABLE_NAME = 'Tabelle' AND NOT COLUMN_NAME = 'ID'"
oResult = oSQL_Command.executeQuery(stSql)
inCounter = 0
stColumns = ""

```

Die Spaltennamen und Spaltentypen werden ausgelesen und in getrennten Arrays gespeichert.

Hinweis

Für FIREBIRD muss der SQL-Code angepasst werden:

```

stSql = "SELECT A.RDB$FIELD_NAME, C.RDB$TYPE_NAME
        FROM RDB$RELATION_FIELDS AS A, RDB$FIELDS AS B, RDB$TYPES AS C
        WHERE A.RDB$FIELD_SOURCE = B.RDB$FIELD_NAME
              AND B.RDB$FIELD_TYPE = C.RDB$TYPE
              AND C.RDB$FIELD_NAME = 'RDB$FIELD_TYPE'
              AND A.RDB$RELATION_NAME = 'Tabelle'
              AND NOT A.RDB$FIELD_NAME = 'ID'"

```

```

WHILE oResult.next
    ReDim Preserve aColumn(inCounter)
    ReDim Preserve aType(inCounter)
    aColumn(inCounter) = oResult.getString(1)
    aType(inCounter) = oResult.getString(2)
    inCounter = inCounter+1
WEND

```

Der zur Zeit höchste Eintrag für den Primärschlüsselwert wird ermittelt und um 1 erhöht. In diesem Beispiel wird also der Schlüsselwert nicht automatisch von der HSQLDB hoch geschrieben, sondern über das Makro verwaltet. Dies geschieht hier zu Testzwecken, da die Tabelle laufend unter unterschiedlichen Kriterien testweise eingelesen wurde. Entsprechend könnte natürlich auch der Tabellenindex heruntergesetzt werden, wie dies in dem Makro «Tabellenindex_runter» passiert.

```

stSql = "SELECT MAX('ID') FROM 'Tabelle'"
oResult = oSQL_Command.executeQuery(stSql)
WHILE oResult.next
    loID = oResult.getInt(1) + 1
WEND

```

Der Pfad zur Calc-Datei wird anhand der Lage der Base-Datei im Dateisystem ermittelt. Die Calc-Datei liegt hier im gleichen Verzeichnis wie die Base-Datei.

Anschließend wird die Calc-Datei geladen und gleich unsichtbar geschaltet, damit sie sich nicht in den Vordergrund schiebt.

```

oDB = ThisComponent.Parent
stDir = Left(oDB.Location, Len(oDB.Location)-Len(oDB.Title))
stDir = ConvertToUrl(stDir & "Daten_Calc.ods")
oDoc = StarDesktop.loadComponentFromURL(stDir, "_blank", 0, Arg() )
oDocView = oDoc.CurrentController.Frame.ContainerWindow
oDocView.Visible = False

```

Die Position des beschrifteten Bereiches des Tabellenblattes wird ermittelt. Dies geschieht über die **ColumnDescriptions** und **RowDescriptions**. Sie geben genau die Anzahl der beschrifteten

Spalten und Zeilen wieder. Außerdem kann darüber die Bezeichnung der Spalte und der Zeile wie z.B. «B2» ausgelesen werden.

```
loColumns = uBound(oDoc.Sheets(0).ColumnDescriptions) ' Anzahl der Spalten
stStartCol = split(oDoc.Sheets(0).ColumnDescriptions(0))(1)
stEndCol = split(oDoc.Sheets(0).ColumnDescriptions(loColumns))(1)
loRows = uBound(oDoc.Sheets(0).RowDescriptions) ' Anzahl der Zeilen
stStartRow = split(oDoc.Sheets(0).RowDescriptions(0))(1)
stEndRow = split(oDoc.Sheets(0).RowDescriptions(loRows))(1)
stRange = stStartCol & stStartRow & ":" & stEndCol & stEndRow
```

Der Bereich wird als String zusammengesetzt. Dies erfolgt in der gleichen Art und Weise wie bei der Benennung innerhalb von Calc, also z.B. «A1:C7». Die Daten aus so einem Bereich können mit der Funktion **GetDataArray()** ausgelesen werden.

```
oRange = oDoc.Sheets(0).getCellRangeByName(stRange)
aDat = oRange.getDataArray()
```

Die Spaltennamen stehen in der ersten Zeile. Sie können eine andere Reihenfolge haben, als dies innerhalb der Tabelle von Base vorgesehen wird. Deshalb werden diese Bezeichnungen für einen späteren Vergleich in einem separaten Array gespeichert. Sie werden in der folgenden Schleife nicht als Werte zum Einlesen in die Tabelle abgefragt. Deshalb beginnt die Schleife mit **i=1**.

```
aColumns = aDat(0)
FOR i = 1 TO uBound(aDat)
    aRow = aDat(i)
    stColumns = ""ID""
    stRow = loID
    FOR k = 0 TO loColumns
        FOR n = 0 TO uBound(aColumn)
```

Im folgenden werden die Spaltenbezeichnungen aus Calc mit denen der Base-Tabelle verglichen. Die Base-Tabelle enthält hier neben Textspalten auch eine Spalte für einen Währungsbetrag, die als **DECIMAL** definiert ist, sowie ein Datum.

Bei dem Währungsbetrag muss das Dezimalkomma gegebenenfalls zu einem Dezimalpunkt umgewandelt werden. Deshalb hier die Funktion **Cdbl**, gekoppelt mit der Funktion **Str**.

Bei dem Datumsfeld gibt Calc den Inhalt als ISO-Zahlencode aus. Dieser Code muss zuerst daraufhin überprüft werden, ob denn überhaupt ein Datum daraus gebildet werden kann. Ist dies möglich, so wird ein SQL-konformes Datum im Format YYYY-MM-DD zusammengestellt, das auch bei einstelligen Tageswerten und Monatswerten nicht versagt.

```
IF aColumns(k) = aColumn(n) THEN
    IF aType(n) = "DECIMAL" THEN
        IF aRow(k) <> "" THEN
            aRow(k) = Str(Cdbl(aRow(k)))
        END IF
    END IF
    IF aType(n) = "DATE" THEN
        IF isDate(CDate(aRow(k))) AND aRow(k) <> "" THEN
            aRow(k) = Year(CDate(aRow(k)))&"-"
                &Right("0"&Month(CDate(aRow(k))),2)
                &"-"&Right("0"&Day(CDate(aRow(k))),2)
        ELSE
            aRow(k) = ""
        END IF
    END IF
```

Ist der Zellinhalt leer oder für ein Dezimalfeld bzw. Datumsfeld gegebenenfalls nicht gültig, so wird an die Base-Tabelle **NULL** weitergegeben. Andernfalls wird der Inhalt in Hochkommata gesetzt. Anschließend werden die Inhalte über Kommata miteinander verbunden. Auch die Bezeichnung der Spalten erfolgt in der Reihenfolge, in der sie aus der Calc-Tabelle ausgelesen wurden.

```
IF aRow(k) = "" THEN
    aRow(k) = "NULL"
ELSE
    aRow(k) = "" & aRow(k) & ""
```

```

        END IF
        stRow = stRow & "," & aRow(k)
        stColumns = stColumns & "," & aColumns(k) & "'''"
    END IF
NEXT
NEXT
stSql = "INSERT INTO ""Tabelle"" (& stColumns &") VALUES (& stRow &)"
oSQL_Command.executeUpdate(stSql)

```

Der Primärschlüsselwert wird hier innerhalb des Makros um 1 heraufgesetzt.

```

        loID = loID + 1
    NEXT
oDoc.close(True) ' Schließen des Calc-Dokumentes
oEvent.Source.Model.parent.reload() ' Neuladen des Formulars
End Sub

```

Ist der gesamte Inhalt aus dem Calc-Tabellenblatt ausgelesen, so wird das unsichtbare Dokument geschlossen. Anschließend wird das Formular, in dem sich der auslösende Button befindet, neu eingelesen. Dabei wird das Formular über den Button mit **oEvent.Source.Model.parent** ermittelt.

Zugriff auf Abfragen

Abfragen lassen sich in der grafischen Benutzeroberfläche einfacher zusammenstellen als den gesamten Text in Makros zu übertragen, zumal dann auch noch innerhalb des Makros alle Felder- und Tabellenbezeichnungen in zweifach doppelte Anführungszeichen gesetzt werden müssen.

```

SUB Abfrageninhalt
    DIM oDatenDatei AS OBJECT
    DIM oAbfragen AS OBJECT
    DIM stQuery AS STRING
    oDatenDatei = ThisComponent.Parent.CurrentController.DataSource
    oAbfragen = oDatenDatei.getQueryDefinitions()
    stQuery = oAbfragen.getByName("Query").Command
    msgbox stQuery
END SUB

```

Aus einem Formular heraus wird auf den Inhalt der *.odb-Datei zugegriffen. Die Abfragen werden über **getQueryDefinitions()** ermittelt. Die SQL-Formulierung der Abfrage "Query" wird über den Zusatz **Command** bereit gestellt. **Command** kann schließlich dazu genutzt werden, eine entsprechende Abfrage auch innerhalb eines Makros weiter zu nutzen.

Allerdings muss bei der Nutzung des SQLCodes der Abfrage darauf geachtet werden, dass sich der Code nicht wiederum auf eine Abfrage bezieht. Das führt dann unweigerlich zu der Meldung, dass die (angebliche) Tabelle der Datenbank unbekannt ist. Einfacher ist es daher, aus Abfragen Ansichten zu erstellen und entsprechend auf die Ansichten in Makros zuzugreifen.

Datenbanksicherungen erstellen

Vor allem beim Erstellen von Datenbanken kann es hin und wieder vorkommen, dass die *.odb-Datei unvermittelt beendet wird. Vor allem beim Berichtsmodul ist ein häufigeres Abspeichern nach dem Editieren sinnvoll.

Ist die Datenbank erst einmal im Betrieb, so kann sie durch Betriebssystemabstürze beschädigt werden, wenn der Absturz gerade während des Schließens der Base-Datei erfolgt. Schließlich wird in diesem Moment der Inhalt der Datenbank in die Datei zurückgeschrieben.

Außerdem gibt es die üblichen Verdächtigen für plötzlich nicht mehr zu öffnende Dateien wie Festplattenfehler usw. Da kann es dann nicht schaden, eine Sicherheitskopie möglichst mit dem aktuellsten Datenstand parat zu haben. Der Datenbestand ändert sich allerdings nicht, während die *.odb-Datei geöffnet ist. Deshalb kann die Sicherung direkt mit dem Öffnen der Datei verbunden werden. Es werden einfach Kopien der Datei in das unter **Extras → Optionen → LibreOffice → Pfade**

angegebene Backup-Verzeichnis erstellt. Dabei beginnt das Makro nach einer voreingestellten Zahl an Kopien (**inMax**) damit, die jeweils älteste Variante zu überschreiben.

```
SUB Datenbankbackup(inMax AS INTEGER)
    DIM oPath AS OBJECT
    DIM oDoc AS OBJECT
    DIM sTitel AS STRING
    DIM sUrl_Ziel AS STRING
    DIM sUrl_Start AS STRING
    DIM i AS INTEGER
    DIM k AS INTEGER
    oDoc = ThisComponent
    sTitel = oDoc.Title
    sUrl_Start = oDoc.URL
    DO WHILE sUrl_Start = ""
        oDoc = oDoc.Parent
        sTitel = oDoc.Title
        sUrl_Start = oDoc.URL
    LOOP
```

Wird das Makro direkt beim Start der *.odb-Datei ausgeführt, so stimmen **sTitel** und **sUrl_Start**. Wird es hingegen von einem Formular aus ausgeführt, so muss erst einmal ermittelt werden, ob überhaupt eine URL verfügbar ist. Ist die URL leer, so wird eine Ebene höher (**oDoc.Parent**) nach einem Wert nachgesehen.

```
oPath = createUnoService("com.sun.star.util.PathSettings")
FOR i = 1 TO inMax + 1
    IF NOT FileExists(oPath.Backup & "/" & i & "_" & sTitel) THEN
        IF i > inMax THEN
            FOR k = inMax - 1 TO 1 STEP -1
                IF FileDateTime(oPath.Backup & "/" & k & "_" & sTitel) <=
                    FileDateTime(oPath.Backup & "/" & k+1 & "_" & sTitel) THEN
                    IF k = 1 THEN
                        i = k
                        EXIT FOR
                    END IF
                ELSE
                    i = k + 1
                    EXIT FOR
                END IF
            NEXT
        END IF
        EXIT FOR
    END IF
NEXT
sUrl_Ziel = oPath.Backup & "/" & i & "_" & sTitel
FileCopy(sUrl_Start,sUrl_Ziel)
END SUB
```

Werden vor der Ausführung der Prozedur «Datenbankbackup» während der Nutzung von Base die Daten aus dem Cache in die Datei zurückgeschrieben, so kann ein entsprechendes Backup auch z.B. nach einer bestimmten Nutzerzeit oder durch Betätigung eines Buttons sinnvoll sein. Das Zurückschreiben regelt die folgende Prozedur:

```
SUB Daten_aus_Cache_schreiben
    DIM oDaten AS OBJECT
    DIM oDataSource AS OBJECT
    oDaten = ThisDatabaseDocument.CurrentController
    IF NOT ( oDaten.isConnected() ) THEN oDaten.connect()
    oDataSource = oDaten.DataSource
    oDataSource.flush
END SUB
```

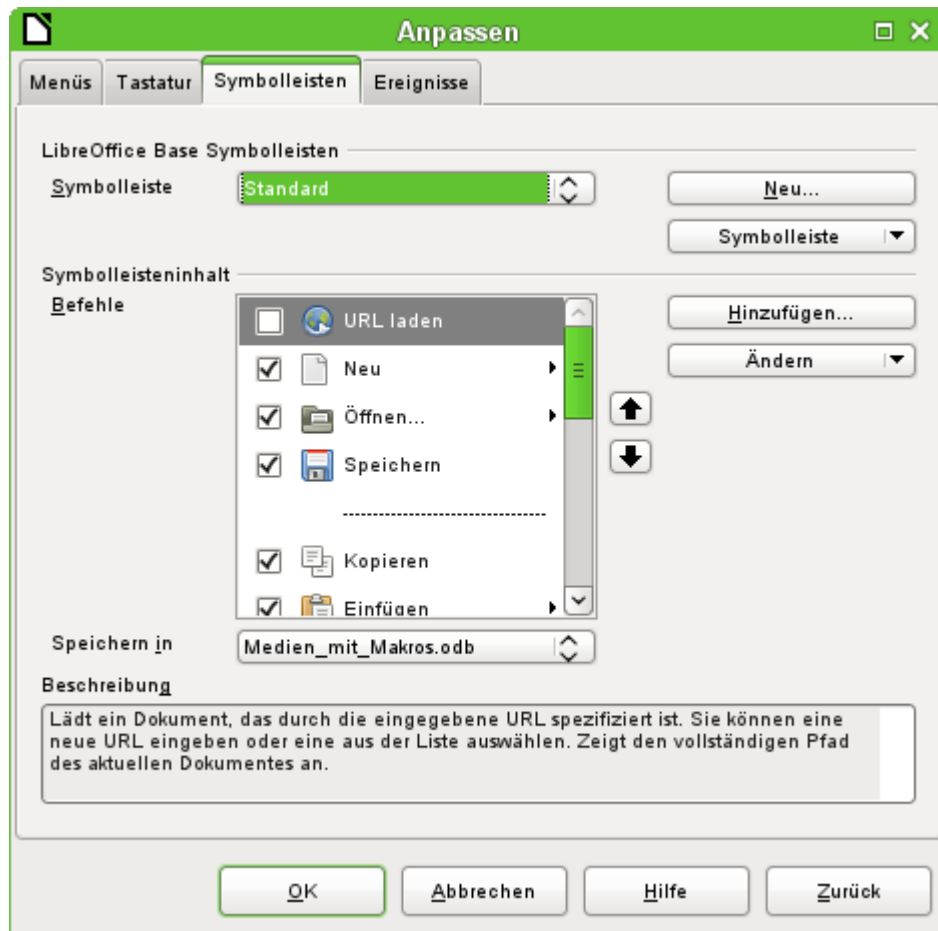
Soll alles zusammen aus einem Formular heraus über einen Button gestartet werden, so müssen beide Prozeduren über eine weitere Prozedur angesprochen werden:


```

SUB Backup_sofort
  Daten_aus_Cache_schreiben
  Datenbankbackup(10)
END SUB

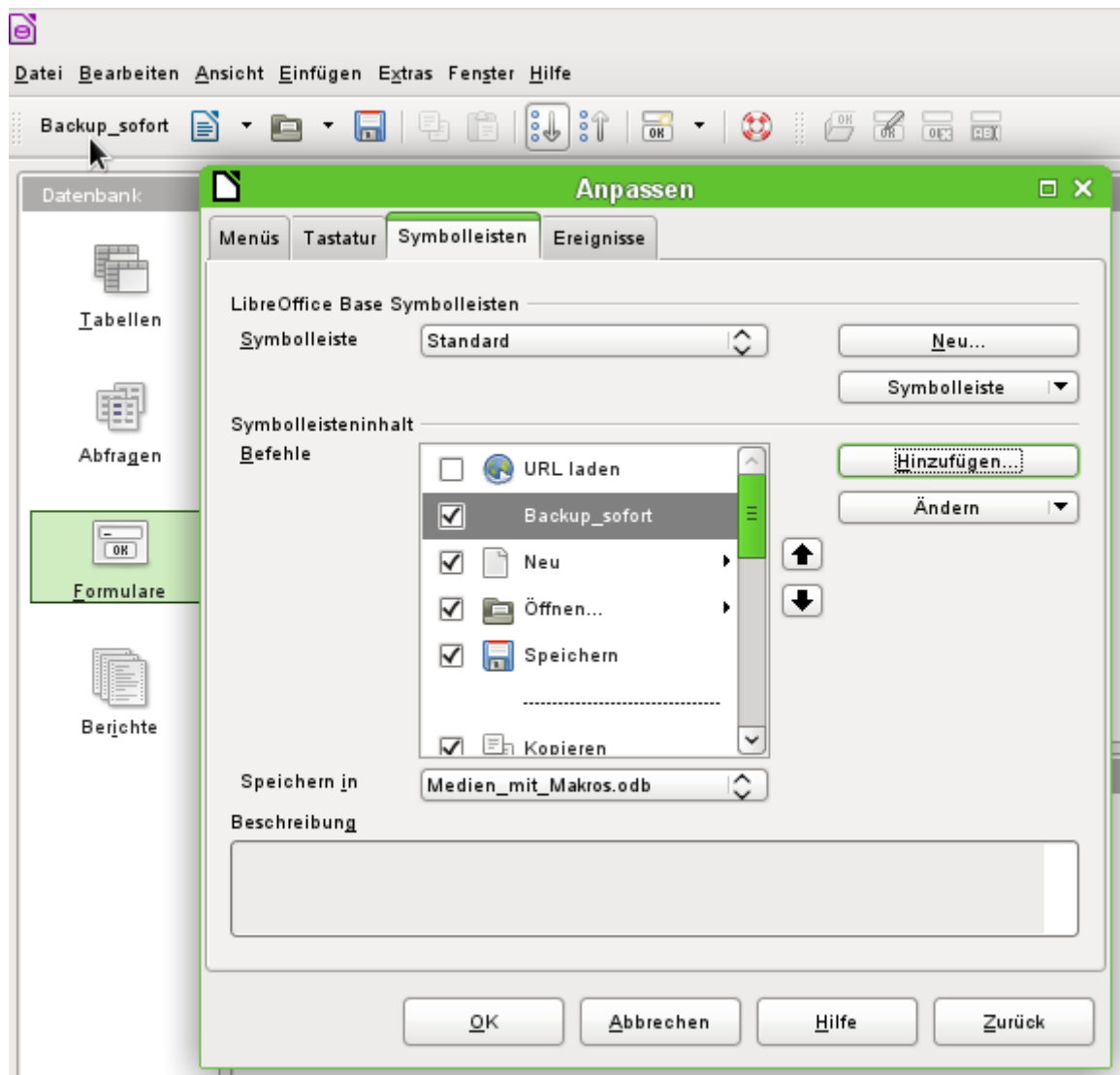
```

Gerade bei einem Sicherungsmakro ist es vielleicht sinnvoll, das Makro über die Symbolleiste der Datenbank erreichbar zu machen. Dies geschieht im Hauptfenster der Base-Datei unter **Extras → Anpassen → Symbolleisten**. Dort wird die für alle Bereiche zuständige Symbolleiste «Standard» aus-
gesucht.



Der Befehl soll anschließend in der Base-Datei gespeichert werden, in diesem Fall also in der Datei «Medien_mit_Makros.odt».

Über **Hinzufügen** erscheint das Fenster für das Hinzufügen von Befehlen. Ganz unten in dem Fenster ist auch die Möglichkeit gegeben, Makros aus der Datenbankdatei hinzuzufügen. Hier wurde die Prozedur «Backup_Sofort» ausgesucht.

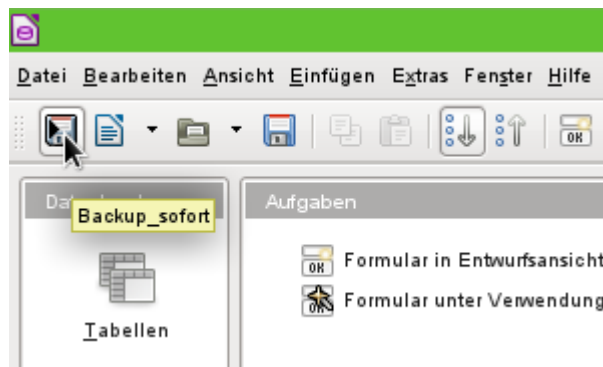


Der Befehl ist jetzt in der Symbolleiste an erster Stelle verfügbar. Um jetzt die Prozedur auszuführen genügt ein Auslösen des Buttons in der Symbolleiste.

Jetzt bietet es sich noch an, dem Befehl ein Symbol zuzuweisen. Über **Ändern** → **Symbol austauschen** wird der folgende Dialog geöffnet.



Hier wird jetzt ein passendes Symbol gesucht. Es kann auch ein eigenes Symbol erstellt und eingebunden werden.



Das Symbol erscheint anschließend statt der Benennung der Prozedur. Die Benennung wird als Tooltip angezeigt.

Datenbanken komprimieren

Eigentlich nur ein SQL-Befehl (**SHUTDOWN COMPACT**), der hin- und wieder, vor allem nach der Löschung von vielen Daten, durchgeführt werden sollte. Die Datenbank speichert neue Daten ab, hält aber gleichzeitig den Platz für die eventuell gelöschten Daten vor. Bei starker Änderung des Datenbestandes muss deshalb der Datenbestand wieder komprimiert werden.

Hinweis

Für die interne HSQLDB wird dieser Befehl seit der LibreOffice-Version 3.6.* automatisch beim Schließen der Datenbank durchgeführt. Diese Makro ist dort also nicht mehr nötig. Bei der **FIREBIRD**-Datenbank gibt es keinen entsprechenden Befehl.

Wird die Komprimierung durchgeführt, so kann anschließend nicht mehr auf die Tabellen zugegriffen werden. Die Datei muss neu geöffnet werden. Deshalb schließt das Makro auch das Formular, aus dem heraus es aufgerufen wird. Leider lässt sich das Dokument selbst nicht schließen, ohne dass beim Neuaufruf die Wiederherstellung anspringt. Deshalb ist diese Funktion auskommentiert.

```
SUB Datenbank_komprimieren
  DIM stMessage AS STRING
  oDatenquelle = ThisComponent.Parent.CurrentController ' Zugriffsmöglichkeit aus dem
  Formular heraus
  IF NOT (oDatenquelle.isConnected()) THEN
    oDatenquelle.connect()
  END IF
  oVerbindung = oDatenquelle.ActiveConnection()
  oSQL_Anweisung = oVerbindung.createStatement()
  stSql = "SHUTDOWN COMPACT" ' Die Datenbank wird komprimiert und geschlossen
  oSQL_Anweisung.executeQuery(stSql)
  stMessage = "Die Datenbank wurde komprimiert." + CHR(13)
  + "Das Formular wird jetzt geschlossen."
  stMessage = stMessage + CHR(13)
  + "Anschließend sollte die Datenbankdatei geschlossen werden."
  stMessage = stMessage + CHR(13) + "Auf die Datenbank kann erst nach dem erneuten
  Öffnen der Datenbankdatei zugegriffen werden."
  msgbox stMessage
  ThisDatabaseDocument.FormDocuments.getByName( "Wartung" ).close
  REM Das Schließen der Datenbankdatei führt beim Neustart zu einem
  Wiederherstellungsablauf.
  ' ThisDatabaseDocument.close(True)
END SUB
```

Tabellenindex heruntersetzen bei Autowert-Feldern

Werden viele Daten aus Tabellen gelöscht, so stören sich Nutzer häufig daran, dass die automatisch erstellten Primärschlüssel einfach weiter hochgezählt werden, statt direkt an den bisher höchsten Schlüsselwert anzuschließen. Die folgende Prozedur liest für eine Tabelle den bisherigen

Höchstwert des Feldes "ID" aus und stellt den nächsten Schlüsselstartwert um 1 höher als das Maximum ein.

Heißt das Primärschlüsselfeld nicht "ID", so müsste das Makro entsprechend angepasst werden.

```
SUB Tabellenindex_runter(stTabelle AS STRING)
    REM Mit dieser Prozedur wird das automatisch hochgeschriebene Primärschlüsselfeld
    REM mit der vorgegebenen Bezeichnung "ID" auf den niedrigst möglichen Wert
    REM eingestellt.
    DIM inAnzahl AS INTEGER
    DIM inSequence_Value AS INTEGER
    oDatenquelle = ThisComponent.Parent.CurrentController ' Zugriffsmöglichkeit aus dem
    Formular heraus
    IF NOT (oDatenquelle.isConnected()) THEN
        oDatenquelle.connect()
    END IF
    oVerbindung = oDatenquelle.ActiveConnection()
    oSQL_Anweisung = oVerbindung.createStatement()
    stSql = "SELECT MAX("ID") FROM ""+stTabelle+"" ' Der höchste in "ID"
    eingetragene Wert wird ermittelt
    oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql) ' Abfrage starten und den
    Rückgabewert in einer Variablen oAbfrageergebnis speichern
    WHILE oAbfrageergebnis.next
        inAnzahl = oAbfrageergebnis.getInt(1) ' Erstes Datenfeld wird ausgelesen
    WEND ' nächster Datensatz, in diesem Fall nicht mehr erforderlich, da nur ein
    Datensatz existiert
    IF inAnzahl = "" THEN ' Falls der höchste Wert gar kein Wert ist, also die
    Tabelle leer ist wird der höchste Wert als -1 angenommen
        inAnzahl = -1
    END IF
    inSequence_Value = inAnzahl+1 ' Der höchste Wert wird um 1 erhöht
    REM Ein neuer Befehl an die Datenbank wird vorbereitet. Die ID wird als neu
    REM startend ab inAnzahl+1 deklariert.
    REM Diese Anweisung hat keinen Rückgabewert, da ja kein Datensatz ausgelesen
    REM werden muss
    oSQL_Anweisung1 = oVerbindung.createStatement()
    oSQL_Anweisung1.executeQuery("ALTER TABLE "" + stTabelle + ""
    ALTER COLUMN ""ID"" RESTART WITH " + inSequence_Value + "")
END SUB
```

Drucken aus Base heraus

Der Standard, um aus Base heraus ein druckbares Dokument zu erzeugen, ist die Nutzung eines Berichtes. Daneben gibt es noch Möglichkeiten, Tabellen und Abfragen einfach nach Calc zu kopieren und dort zum Druck aufzubereiten. Auch der direkte Druck eines Formularinhaltes vom Bildschirm ist natürlich möglich.

Druck von Berichten aus einem internen Formular heraus

Um einen Bericht zu starten, muss normalerweise die Benutzeroberfläche von Base aufgesucht werden. Ein Mausklick auf den Berichtsnamen startet dann die Ausführung des Berichtes. Einfacher geht es natürlich, den Bericht direkt aus dem Formular heraus zu starten:

```
SUB Berichtsstart
    ThisDatabaseDocument.ReportDocuments.getByName("Bericht").open
END SUB
```

Sämtliche Berichte werden über **ReportDocuments** mit ihrem Namen angesprochen. Mit **open** werden sie geöffnet. Wird ein Bericht jetzt an eine Abfrage gebunden, die über das Formular gefiltert wird, so kann auf diese Art und Weise der zum aktuellen Datensatz anfallende Ausdruck erfolgen.

Start, Formatierung, direkter Druck und Schließen des Berichts

Noch schöner ist es, wenn der Bericht direkt an den Drucker geschickt wird. Die folgende Kombination an Prozeduren legt sogar noch ein paar kleine Features zu. Sie selektiert zuerst den aktiven Datensatz des Formulars, formatiert danach den Bericht um, indem die Felder für den Text auf automatische Höhe eingestellt werden, um anschließend den Bericht zu starten. Schließlich wird der Bericht auch noch gedruckt und ggf. noch als *pdf-Dokument abgespeichert. Und all das passiert nahezu vollständig im Hintergrund, da der Bericht direkt nach dem Öffnen auf unsichtbar geschaltet und nach dem Ausdruck wieder geschlossen wird. Anregungen für die verschiedenen Prozeduren stammen hier von Andrew Piontak, Thomas Krumbein und Lionel Elie Mamane.

```
SUB BerichtStart(oEvent AS OBJECT)
    DIM oForm AS OBJECT
    DIM stSql AS STRING
    DIM oDatenquelle AS OBJECT
    DIM oVerbindung AS OBJECT
    DIM oSQL_Anweisung AS OBJECT
    DIM oReport AS OBJECT
    DIM oReportView AS OBJECT
    oForm = oEvent.Source.model.parent
    stSql = "UPDATE ""Filter"" SET ""Integer"" = '" +
        oForm.getInt(oForm.findColumn("ID")) + "' WHERE ""ID"" = TRUE"
    oDatenquelle = ThisComponent.Parent.CurrentController
    If NOT (oDatenquelle.isConnected()) THEN
        oDatenquelle.connect()
    END IF
    oVerbindung = oDatenquelle.ActiveConnection()
    oSQL_Anweisung = oVerbindung.createStatement()
    oSQL_Anweisung.executeUpdate(stSql)
    oReport = ThisDatabaseDocument.ReportDocuments.getByName("Berichtsname").open
    oReportView = oReport.CurrentController.Frame.ContainerWindow
    oReportView.Visible = False
    BerichtZeilenhoeheAuto(oReport)
END SUB
```

Die Prozedur «BerichtStart» wird mit einem Button innerhalb eines Formulars verknüpft. Es kann dabei über den Button der Primärschlüssel des aktuellen Datensatzes des Formulars ausgelesen werden. Dies geschieht hier über das auslösende Ereignis, von dem heraus auf das Formular **oForm** geschlossen wird. Der Name des Schlüsselfeldes ist hier mit **"ID"** angegeben. Über **oForm.getInt(oForm.findColumn("ID"))** wird aus dem Feld der Schlüsselwert als Integer-Wert gelesen. Dieser Wert wird anschließend in einer Filtertabelle abgespeichert. Diese Filtertabelle steuert über eine Abfrage, dass nur der aktuelle Datensatz des Formulars für den Bericht verwendet wird.

Ohne Bezug auf das Formular könnte auch nur der Bericht aufgerufen werden. Dabei ist der aufgerufene Bericht gleich als Objekt ansprechbar (**oReport**). Anschließend wird das Fenster auf unsichtbar eingestellt. Dies geht leider nicht direkt mit dem Aufruf, so dass ganz kurz das Fenster erscheint, dann aber ggf. in Ruhe im Hintergrund mit dem entsprechenden Inhalt gefüllt wird.

Anschließend wird die Prozedur «BerichtZeilenhoeheAuto» gestartet. Dieser Prozedur wird der Hinweis auf den geöffneten Bericht mitgegeben.

Die Zeilenhöhe kann zur Zeit nicht beim Berichtsentswurf automatisch angepasst werden. Ist zu viel Text für ein Feld vorgesehen, so wird der Text abgeschnitten und darauf mit Hilfe eines roten Dreiecks hingewiesen. Solange dies nicht funktioniert, stellt die folgende Prozedur sicher, dass z.B. in allen Tabellen mit der Bezeichnung «Detail» die automatische Höhe eingeschaltet wird.

```
SUB BerichtZeilenhoeheAuto(oReport AS OBJECT)
    DIM oTables AS OBJECT
    DIM oTable AS OBJECT
    DIM inT AS INTEGER
    DIM inI AS INTEGER
    DIM oRows AS OBJECT
    DIM oRow AS OBJECT
    oTables = oReport.getTextTables()
```

```

FOR inT = 0 TO oTables.count() - 1
    oTable = oTables.getByIndex(inT)
    IF Left$(oTable.name, 6) = "Detail" THEN
        oRows = oTable.Rows
        FOR inI = 0 TO oRows.count - 1
            oRow = oRows.getByIndex(inI)
            oRow.IsAutoHeight = True
        NEXT inI
    ENDIF
NEXT inT
BerichtDruckenUndSchliessen(oReport)
END SUB

```

Bei dem Entwurf des Berichtes muss darauf geachtet werden, dass alle in einer Zeile des Bereiches «Detail» befindlichen Felder tatsächlich die gleiche Höhe haben. Sonst kann es zusammen mit der Automatik passieren, dass ein Feld plötzlich auf die doppelte Zeilenhöhe gesetzt wird.

Nachdem in allen Tabellen mit der Bezeichnung «Detail» die automatische Höhe eingestellt wurde, wird anschließend der Bericht über die Prozedur «BerichtDruckenUndSchliessen» weiter an den Drucker geschickt.

Das Array Props enthält die verschiedenen Werte, die mit dem Drucker bei einem Dokument verbunden sind. Für den Druckbefehl ist hier lediglich der Name des Standarddruckers wichtig. Das Berichtsdokument soll so lange geöffnet bleiben, bis der Druck tatsächlich abgeschlossen ist. Dies geschieht, indem dem Druckbefehl der Name und der Befehl «Warte, bis ich fertig bin» (**wait**) mitgegeben wird.

```

Sub BerichtDruckenUndSchliessen(oReport AS OBJECT)
    DIM Props
    DIM stDrucker AS STRING
    Props = oReport.getPrinter()
    stDrucker = Props(0).value
    DIM arg(1) AS NEW com.sun.star.beans.PropertyValue
    arg(0).name = "Name"
    arg(0).value = "<" & stDrucker & ">"
    arg(1).name = "wait"
    arg(1).value = True
    oReport.print(arg())
    oReport.close(true)
End Sub

```

Erst wenn der Druck komplett an den Drucker abgeschickt wurde, wird das Dokument geschlossen.

Zu Einstellungen des Druckers siehe die [Drucker und Druckeinstellungen](#) aus dem AOO-Wiki.

Soll statt des Drucks oder zusätzlich zu dem Druck auch eine *.pdf-Datei des Dokumentes als Sicherungskopie abgelegt werden, so wird darauf mit der Methode **storeToURL()** zugegriffen:

```

Sub BerichtAlsPDFspeichern(oReport AS OBJECT)
    DIM stUrl AS STRING
    DIM arg(0) AS NEW com.sun.star.beans.PropertyValue
    arg(0).name = "FilterName"
    arg(0).value = "writer_pdf_Export"
    stUrl = "file:///..."
    oReport.storeToURL(stUrl, arg())
End Sub

```

Bei der URL muss natürlich eine komplette URL-Adresse angegeben werden. Noch sinnvoller ist es, diese Adresse z.B. gekoppelt mit einem unverwechselbaren Merkmal des gedruckten Dokumentes zu versehen, wie z.B. der Rechnungsnummer. Sonst könnte es passieren, dass eine Sicherungsdatei beim nächsten Druck einfach überschrieben wird.

Druck von Berichten aus einem externen Formular heraus

Schwierig wird es, wenn mit externen Formularen gearbeitet wird. Die Berichte liegen dann in der *.odb-Datei und sind auch über den Datenquellenbrowser erst einmal nicht verfügbar.

```

SUB Berichtsstart(oEvent AS OBJECT)
    DIM oFeld AS OBJECT
    DIM oForm AS OBJECT
    DIM oDocument AS OBJECT
    DIM oDocView AS OBJECT
    DIM Arg()
    oFeld = oEvent.Source.Model
    oForm = oFeld.Parent
    sURL = oForm.DataSourceName
    oDocument = StarDesktop.loadComponentFromURL(sURL, "_blank", 0, Arg() )
    oDocView = oDocument.CurrentController.Frame.ContainerWindow
    oDocView.Visible = False
    oDocument.getCurrentController().connect
    Wait(100)
    oDocument.ReportDocuments.getByName("Bericht").open
    oDocument.close(True)
END SUB

```

Der Bericht wird von einem Button des externen Formulars gestartet. Über den Button wird das Formular ermittelt, in dem der Pfad zur *.odb-Datei verzeichnet ist: **oForm.DataSourceName**. Anschließend wird mit **loadComponentFromURL** die *.odb-Datei geöffnet. Die Datei soll nur im Hintergrund liegen. Deshalb wird gleich auf die Ansicht zugegriffen und die Oberfläche der *.odb-Datei auf **Visible = False** gestellt. Dies sollte auch direkt beim Aufruf über die Argumentenliste **Arg()** funktionieren, brachte aber bei Tests nicht den gewünschten Erfolg.

Wird jetzt direkt der Bericht des geöffneten Dokumentes aufgerufen, so ist die Datenbankverbindung noch nicht verfügbar. Der Bericht erscheint nur mit einem grauen Hintergrund und LibreOffice verzeichnet einen Absturz. Schon eine kleine Wartezeit von 100 Millisekunden (**Wait(100)**) löst dieses Problem. Hier müssen praktische Tests zeigen, wie kurz diese Zeit eingestellt werden kann. Anschließend wird der Bericht gestartet. Da es sich bei dem ausgeführten Bericht um eine separate Textdatei handelt, kann die geöffnete *.odb-Datei anschließend geschlossen werden. Mit **oDocument.close(True)** wird der Befehl an die *.odb-Datei weiter gegeben. Die Datei wird allerdings erst dann geschlossen, wenn sie nicht mehr aktiv z.B. Daten an die Berichtsdatei weiter geben muss.

Mit einem entsprechenden Zugriff können auch die Formulare innerhalb der *.odb-Datei gestartet werden. Hier sollte dann aber das Schließen des Dokumentes unterbleiben.

Deutlich schneller als mit dem Report-Builder und trotzdem gut gestaltet geht der Druck aber über Makros mit Hilfe von Serienbrieffunktionen oder Textfeldern.

Serienbriefdruck aus Base heraus

Manchmal reicht einfach ein Bericht nicht aus, um sauber Briefe an die Adressaten zu erstellen. Schon die Textfelder in einem Bericht sind hier in der Nutzung doch sehr eingeschränkt. Hierzu wird dann ein Serienbrief im Writer erstellt. Es muss aber nicht sein, dass erst der Writer geöffnet wird, dort dann über den Serienbriefdruck alle Auswahlmöglichkeiten und Eingaben gemacht werden und schließlich der Druck erfolgt. Dies alles geht auch per Makro direkt aus Base heraus.

```

SUB Serienbriefdruck
    DIM oMailMerge AS OBJECT
    DIM aProps()
    oMailMerge = createunoservice("com.sun.star.text.MailMerge")

```

Als Name der Datenquelle wird der Name angegeben, unter dem die Datenbank in LO angemeldet ist. Dieser Name muss nicht identisch mit dem Dateinamen sein. Der Anmeldenamen in diesem Beispiel lautet "Adressen"

```

    oMailMerge.DataSourceName = "Adressen"

```

Die Pfadbeschreibung mit der Serienbriefdatei erfolgt in der Art der jeweiligen Betriebssystemumgebung, hier ab dem Wurzelpfad eines Linux-Systems.

```

    oMailMerge.DocumentURL = ConvertToUrl("home/user/Dokumente/Serienbrief.odt")

```


Der Typ des Kommandos wird festgelegt. '0' steht für eine Tabelle, '1' für eine Abfrage und '2' für ein direktes SQL-Kommando.

```
oMailMerge.CommandType = 1
```

Hier wurde eine Abfrage gewählt, die den Namen "Serienbriefabfrage" trägt.

```
oMailMerge.Command = "Serienbriefabfrage"
```

Über den Filter wird festgelegt, für welche Datensätze aus der Serienbriefabfrage ein Druck erfolgen soll. Dieser Filter könnte z.B. über ein Formularfeld aus Base heraus an das Makro weitergegeben werden. Mit dem Primärschlüssel eines Datensatzes könnte so der Ausdruck eines einzelnen Dokumentes erfolgen.

In diesem Beispiel wird aus der "Serienbriefabfrage" das Feld "Geschlecht" aufgesucht und dort nach Datensätzen gesucht, die in diesem Feld mit einem 'm' versehen sind.

```
oMailMerge.Filter = ""'"Geschlecht"'='m'""
```

Es gibt die Ausgabetypen Drucker (1), Datei (2) und Mail (3). Hier wurde zu Testzwecken die Ausgabe in eine Datei gewählt. Diese Datei wird in dem angegebenen Pfad abgespeichert. Für jeden Serienbriefdatensatz wird ein Druck erzeugt. Damit dieser Druck unterscheidbar ist, wird das Feld Nachname in den Dateinamen aufgenommen.

```
oMailMerge.OutputType = 2
oMailMerge.OutputUrl = ConvertToUrl("home/user/Dokumente")
oMailMerge.FileNameFromColumn = True
oMailMerge.FileNamePrefix = "Nachname"
oMailMerge.execute(aProps())
```

```
END SUB
```

Wird allein der Filter über ein Formular bestückt, so kann auf diese Art und Weise, also ohne die Öffnung des Writer-Dokumentes, ein Serienbriefdruck erfolgen.

Drucken über Textfelder

Über **Einfügen → Feldbefehl → Funktionen → Platzhalter** wird im Writer eine Vorlage für das zukünftig zu druckende Dokument erstellt. Die Platzhalter sollten dabei sinnvollerweise mit dem Namen versehen werden, den die Felder auch in der Datenbank bzw. der Tabelle/Abfrage für das Formular haben, aus dem heraus das Makro aufgerufen wird.

Für einfache Zwecke wird «Text» als Typ für den Platzhalter gewählt.

In dem Makro wird der Pfad zur Vorlage hinterlegt. Es wird ein neues Dokument «Unbenannt1.odt» erstellt. Vom Makro werden die Platzhalter über die Abfrage des Inhaltes des aktuellen Datensatzes des Formulars befüllt. Das offene Dokument kann nun noch nach Belieben verändert werden.

In der Beispieldatenbank «Beispiel_Datenbank_Serienbrief_direkt.odt» wird gezeigt, wie mit Hilfe von Textfeldern und einem Zugriff auf eine in der Vorlage bereits vorgesehene Tabelle eine komplette Rechnung erstellt werden kann. Im Gegensatz zum Report-Builder sind bei dieser Form der Rechnungserstellung die entsprechenden Felder für den Tabelleninhalt nicht in der Höhe begrenzt. Deshalb wird immer aller Text angezeigt.

Hier Teile des Codes, der im Wesentlichen diesem Beitrag von DPunch zu verdanken ist:

<http://de.openoffice.info/viewtopic.php?f=8&t=45868#p194799>

```
SUB Textfelder_Fuellen
oForm = thisComponent.Drawpage.Forms.MainForm
IF oForm.RowCount = 0 THEN
    msgbox "Kein Datensatz zum Drucken vorhanden"
    EXIT SUB
END IF
```

Das Hauptformular wird angesteuert. Hier könnte auch die Lage des auslösenden Buttons das Formular selbst ermitteln. Anschließend wird geklärt, ob in dem Formular überhaupt Daten liegen, die einen Druck ermöglichen.

```
oColumns = oForm.Columns
```

```
oDB = ThisComponent.Parent
```

Der Zugriff auf die URL ist nicht vom Formular aus direkt möglich. Es muss auf den darüber liegenden Frame der Datenbank Bezug genommen werden.

```
stDir = Left(oDB.Location, Len(oDB.Location) - Len(oDB.Title))
```

Der Titel der Datenbank wird von der URL abgetrennt.

```
stDir = stDir & "Beispiel_Textfelder.ott"
```

Die Vorlage wird aufgesucht und geöffnet

```
DIM args(0) AS NEW com.sun.star.beans.PropertyValue
args(0).Name = "AsTemplate"
args(0).Value = True
oNewDoc = StarDesktop.loadComponentFromURL(stDir, "_blank", 0, args)
```

Die Textfelder werden eingelesen

```
oTextfields = oNewDoc.Textfields.createEnumeration
DO WHILE oTextfields.hasMoreElements
    oTextfield = oTextfields.nextElement
    IF oTextfield.supportsService("com.sun.star.text.TextField.JumpEdit") THEN
        stColumnname = oTextfield.Placeholder
```

Placeholder ist die Benennung für das Textfeld

```
IF oColumns.hasByName(stColumnname) THEN
```

Wenn der Name des Textfeldes gleich dem Spaltennamen der Daten ist, die dem Formular zugrunde liegen, wird der Inhalt aus der Datenbank auf das Feld in dem Textdokument übertragen.

```
        inIndex = oForm.findColumn(stColumnname)
        oTextfield.Anchor.String = oForm.getString(inIndex)
    END IF
END IF
LOOP
END SUB
```

Aufruf von Anwendungen zum Öffnen von Dateien

Mit dieser Prozedur kann durch einen Mausklick in ein Textfeld ein Programm aufgerufen werden, das im eigenen Betriebssystem mit der Dateinamensendung verbunden ist. Damit werden auch Links ins Internet oder sogar das Öffnen des Mailprogramms mit einer bestimmten Mailadresse möglich, die gerade in der Datenbank gespeichert wurde.

Siehe zu diesem Abschnitt auch die **Beispieldatenbank** «Beispiel_Mailstart_Dateiaufruf.odt»

```
SUB Link_oeffnen
    DIM oDoc AS OBJECT
    DIM oDrawpage AS OBJECT
    DIM oForm AS OBJECT
    DIM oFeld AS OBJECT
    DIM oShell AS OBJECT
    DIM stFeld AS STRING
    oDoc = thisComponent
    oDrawpage = oDoc.Drawpage
    oForm = oDrawpage.Forms.getByName("Formular")
    oFeld = oForm.getByName("Adresse")
```

Aus dem benannten Feld wird der Inhalt ausgelesen. Dies kann eine Webadresse, beginnend mit 'http://', eine Mailadresse mit einem '@' oder ein einfaches Dokument sein, das durch eine entsprechende Pfadangabe aufgesucht werden soll (z.B. externe Bilder, *.pdf-Dateien, Tondokumente ...).

```
    stFeld = oFeld.Text
    IF stFeld = "" THEN
        EXIT SUB
    END IF
```


Ist das Feld leer, so soll das Makro sofort enden. Bei der Eingabe passiert es ja sehr oft, dass Felder mit der Maus aufgesucht werden. Ein Mausklick in das Feld, um dort zum ersten Mal schreiben zu können, soll aber noch nicht das Makro ausführen.

Jetzt wird gesucht, ob in dem Feld ein '@' enthalten ist. Dies deutet auf eine E-Mail-Adresse hin. Es soll also das Mailprogramm gestartet werden und eine Mail an diese Mailadresse gesandt werden.

```
IF InStr(stFeld,"@") THEN
    stFeld = "mailto:"+stFeld
```

Ist kein '@' vorhanden, so wird der Begriff so konvertiert, dass die Datei im Dateisystem gefunden werden kann. Steht ein 'http://' am Anfang, so wird bei dieser Funktion nicht im Dateisystem, sondern über den Webbrowser direkt im Internet nachgesehen. Ansonsten beginnt der erstellte Pfad anschließend mit dem Begriff 'file:///'

```
ELSE
    stFeld = convertToUrl(stFeld)
END IF
```

Jetzt wird das Programm aufgesucht, das in dem eigenen Betriebssystem mit der entsprechenden Dateiendung verbunden ist. Bei dem Stichwort 'mailto:' ist dies das Mailprogramm, bei 'http://' der Webbrowser und bei allen anderen ist die Entscheidung des Systems mit den Endungen der Datei verbunden.

```
oShell = createUnoService("com.sun.star.system.SystemShellExecute")
oShell.execute(stFeld,,0)
END SUB
```

Aufruf eines Mailprogramms mit Inhaltvorgaben

Eine Erweiterung des vorhergehenden Beispiels zum Programmaufruf stellt dieser Aufruf eines Mailprogramms mit Vorgaben in der Betreffzeile und inhaltlichen Vorgaben dar.

Siehe auch zu diesem Abschnitt die **Beispieldatenbank** «Mailstart_Dateiaufruf.odt»

Der Mailaufruf erfolgt mit '**mailto:Empfänger?subject= &body= &cc= &bcc=**'. Die letzten beiden Eingaben sind im Formular allerdings nicht aufgeführt. Anhänge kommen in der Definition von '**mailto**' nicht vor. Manchmal funktioniert allerdings '**attachment=**'.

```
SUB Mail_Aufruf
    DIM oDoc AS OBJECT
    DIM oDrawpage AS OBJECT
    DIM oForm AS OBJECT
    DIM oFeld1 AS OBJECT
    DIM oFeld2 AS OBJECT
    DIM oFeld3 AS OBJECT
    DIM oFeld4 AS OBJECT
    DIM oShell AS OBJECT
    DIM stFeld1 AS STRING
    DIM stFeld2 AS STRING
    DIM stFeld3 AS STRING
    DIM stFeld4 AS STRING
    oDoc = thisComponent
    oDrawpage = oDoc.Drawpage
    oForm = oDrawpage.Forms.getByName("Formular")
    oFeld1 = oForm.getByName("E-Mail-Adresse")
    oFeld2 = oForm.getByName("E-Mail-Betreff")
    oFeld3 = oForm.getByName("E-Mail-Inhalt")
    stFeld1 = oFeld1.Text
    IF stFeld1 = "" THEN
        msgbox "Keine Mailadresse vorhanden." & CHR(13) &
            "Das Mailprogramm wird nicht aufgerufen" , 48, "Mail senden"
    END IF
END SUB
```

END IF

Die Konvertierung zu URL ist notwendig, damit Sonderzeichen und Zeilenumbrüche den Aufruf nicht stören. Dabei wird allerdings auch der Begriff '**file:///**' vorangestellt. Diese 8 Zeichen zu Beginn werden nicht mit übernommen.

```
stFeld2 = Mid(ConvertToUrl(oFeld2.Text), 9)
stFeld3 = Mid(ConvertToUrl(oFeld3.Text), 9)
```

Im Gegensatz zum einfachen Programmaufruf werden hier Details des Mailaufrufes über den Aufruf des Mailprogramms mitgegeben.

```
oShell = createUnoService("com.sun.star.system.SystemShellExecute")
oShell.execute("mailto:" + stFeld1 + "?subject=" + stFeld2 + "&body=" + stFeld3,,0)
END SUB
```

Hinweis

Das Versenden von Mails mit Hilfe des Mailprogramms kann auch mit folgendem Code erfolgen. Allerdings kann der eigentliche Inhalt der Mail hier nicht mit eingefügt werden.

```
DIM attaches(0)
oMailer = createUnoService("com.sun.star.system.SimpleSystemMail")
oMailProgramm = oMailer.querySimpleMailClient()
oNeueNachricht = oMailProgramm.createSimpleMailMessage()
oNeueNachricht.setRecipient(stFeld1)
oNeueNachricht.setSubject(stFeld2)
attaches(0) = "file:///..."
oNeueNachricht.setAttachement(attaches())
oMailprogramm.sendSimpleMailMessage(oNeuenachricht, 0 )
```

Zu den möglichen Parametern siehe: http://api.libreoffice.org/docs/idl/ref/interface-com_1_sun_1_star_1_system_1_XSimpleMailMessage.html

Mauszeiger beim Überfahren eines Links ändern

Im Internet üblich, bei Base nachgebaut: Der Mauszeiger fährt über ein Textfeld und verändert seine Form zu einer zeigenden Hand. Der enthaltene Text kann jetzt noch in den Eigenschaften des Feldes zu der Farbe Blau und unterstrichen geändert werden – schon ist der Eindruck eines Links aus dem Internet perfekt. Jeder Nutzer erwartet nach einem Klick, dass sich ein externes Programm öffnet.

Siehe auch zu diesem Abschnitt die **Beispieldatenbank** «Mailstart_Dateiaufruf.odt»

Diese kurze Prozedur sollte mit dem Ereignis '**Maus innerhalb**' des Textfeldes verbunden werden.

```
SUB Mauszeiger(Event AS OBJECT)
REM Siehe auch Standardbibliotheken: Tools → ModuleControls → SwitchMousePointer
DIM oPointer AS OBJECT
oPointer = createUnoService("com.sun.star.awt.Pointer")
oPointer.setType(27) 'Typen in com.sun.star.awt.SystemPointer
Event.Source.Peer.SetPointer(oPointer)
END SUB
```

Formulare ohne Symbolleisten präsentieren

Neunutzer von Base sind häufig irritiert, dass z.B. eine Menüleiste existiert, diese aber im Formular so gar nicht verfügbar ist. Diese Menüleisten können auf verschiedene Arten ausgeblendet werden. Am erfolgreichsten unter allen LO-Versionen sind die beiden im Folgenden vorgestellten Vorgehensweisen.

Fenstergrößen und Symbolleisten werden in der Regel über ein Makro beeinflusst, das in einem Formlardokument unter **Extras → Anpassen → Ereignisse → Dokument öffnen** gestartet wird. Gemeint ist hier das Dokument, nicht ein einzelnes Haupt- oder Unterformular.

Formulare ohne Symbolleisten in einem Fenster

Ein Fenster lässt sich in der Größe variieren. Über den entsprechenden Button lässt es sich auch schließen. Diese Aufgaben übernimmt der Window-Manager des jeweiligen Betriebssystems. Lage und Größe des Fensters auf dem Bildschirm kann beim Start über ein Makro mitgegeben werden.

```
SUB Symbolleisten_Ausblenden
    DIM oFrame AS OBJECT
    DIM oWin AS OBJECT
    DIM oLayoutMng AS OBJECT
    DIM aElemente()
    oFrame = StarDesktop.getCurrentFrame()
```

Der Titel für das Formular wird in der Titelleiste des Fensters angezeigt.

```
oFrame.setTitle "Mein Formular"
oWin = oFrame.getContainerWindow()
```

Das Fenster wird auf die maximale Größe eingestellt. Dies entspricht nicht dem Vollbildmodus, da z.B. eine Kontrollleiste noch sichtbar ist und das Fenster eine Titelleiste hat, über die die Größe des Fensters geändert und das Fenster geschlossen werden kann.

```
oWin.IsMaximized = true
```

Es besteht auch die Möglichkeit, das Fenster in einer ganz bestimmten Größe und mit einer festen Position darzustellen. Dies würde mit '**oWin.setSize(0,0,600,400,15)**' geschehen. Hier wird das Fenster an der linken oberen Ecke des Bildschirms mit einer Breite von 600 Punkten und einer Höhe von 400 Punkten dargestellt. Die letzte Ziffer weist darauf hin, dass alle Punkte angegeben wurden. Sie wird als '**Flag**' bezeichnet. Das '**Flag**' wird aus den folgenden Werten über eine Summierung berechnet: x=1, y=2, Breite=4, Höhe=8. Da x, y, Breite und Höhe angegeben sind, hat das '**Flag**' die Größe $1 + 2 + 4 + 8 = 15$.

```
oLayoutMng = oFrame.LayoutManager
aElemente = oLayoutMng.getElements()
FOR i = LBound(aElemente) TO UBound(aElemente)
    IF aElemente(i).ResourceURL =
        "private:resource/toolbar/formsnavigationbar" THEN
    ELSE
        oLayoutMng.hideElement(aElemente(i).ResourceURL)
    END IF
NEXT
END SUB
```

Wenn es sich um die Navigationsleiste handelt, soll nichts geschehen. Das Formular soll schließlich bedienbar bleiben, wenn nicht das Kontrollfeld für die Navigationsleiste eingebaut und die Navigationsleiste sowieso ausgeblendet wurde. Nur wenn es sich nicht um die **Navigationsleiste** handelt, soll die entsprechende Leiste verborgen werden. Deswegen erfolgt zu dieser Bedingung **keine Aktion**.

Werden die Symbolleisten nicht wieder direkt beim Beenden des Formulars eingeblendet, so bleiben sie weiterhin verborgen. Sie können natürlich über **Ansicht → Symbolleisten** wieder aufgerufen werden. Etwas irritierend ist es jedoch, wenn gerade die Standardleiste (**Ansicht → Symbolleisten → Standardleiste**) oder die Statusleiste (**Ansicht → Statusleiste**) fehlt.

Mit dieser Prozedur werden die Symbolleisten aus dem Versteck ('**hideElement**') wieder hervorgeholt ('**showElement**'). Der Kommentar enthält die Leisten, die oben als sonst fehlende Leisten am ehesten auffallen.

```
SUB Symbolleisten_Einblenden
    DIM oFrame AS OBJECT
    DIM oLayoutMng AS OBJECT
    DIM aElemente()
    oFrame = StarDesktop.getCurrentFrame()
    oLayoutMng = oFrame.LayoutManager
    aElemente = oLayoutMng.getElements()
    FOR i = LBound(aElemente) TO UBound(aElemente)
        oLayoutMng.showElement(aElemente(i).ResourceURL)
```

```

NEXT
' eventuell fehlende wichtige Elemente:
' "private:resource/toolbar/standardbar"
' "private:resource/statusbar/statusbar"
END SUB

```

Die Makros werden an die Eigenschaften des Formularfensters gebunden: **Extras → Anpassen → Ereignisse → Dokument öffnen → Symbolleisten_Ausblenden** bzw. ... **Dokument geschlossen → Symbolleisten_Einblenden**

Leider tauchen häufig Symbolleisten trotzdem nicht wieder auf. In hartnäckigen Fällen kann es daher helfen, nicht die Elemente auszulesen, die der Layoutmanager bereits kennt, sondern definitiv bestimmte Symbolleisten erst zu erstellen und danach schließlich zu zeigen:

```

Sub Symbolleisten_Einblenden
    DIM oFrame AS OBJECT
    DIM oLayoutMng AS OBJECT
    DIM i AS INTEGER
    DIM aElemente(5) AS STRING
    oFrame = StarDesktop.getCurrentFrame()
    oLayoutMng = oFrame.LayoutManager
    aElemente(0) = "private:resource/menubar/menubar"
    aElemente(1) = "private:resource/statusbar/statusbar"
    aElemente(2) = "private:resource/toolbar/formsnavigationbar"
    aElemente(3) = "private:resource/toolbar/standardbar"
    aElemente(4) = "private:resource/toolbar/formdesign"
    aElemente(5) = "private:resource/toolbar/formcontrols"
    FOR EACH i IN aElemente()
        IF NOT(oLayoutMng.requestElement(i)) THEN
            oLayoutMng.createElement(i)
        END IF
        oLayoutMng.showElement(i)
    NEXT i
END SUB

```

Die darzustellenden Symbolleisten werden explizit benannt. Ist eine der entsprechenden Symbolleisten nicht für den Layoutmanager vorhanden, so wird sie zuerst über **createElement** erstellt und danach über **showElement** gezeigt.

Formulare im Vollbildmodus

Beim Vollbildmodus wird der gesamte Bildschirm vom Formular bedeckt. Hier steht keine Kontrollleiste o.ä. mehr zur Verfügung, die gegebenenfalls anzeigt, ob noch irgendwelche anderen Programme laufen.

```

FUNCTION Fullscreen(boSwitch AS BOOLEAN)
    DIM oDispatcher AS OBJECT
    DIM Props(0) AS NEW com.sun.star.beans.PropertyValue
    oDispatcher = createUnoService("com.sun.star.frame.DispatchHelper")
    Props(0).Name = "FullScreen"
    Props(0).Value = boSwitch
    oDispatcher.executeDispatch(ThisComponent.CurrentController.Frame,
        ".uno:FullScreen", "", 0, Props())
END FUNCTION

```

Diese Funktion wird durch die folgenden Prozeduren eingeschaltet. In den Prozeduren läuft gleichzeitig die vorhergehende Prozedur zum Ausblenden der Symbolleisten ab – sonst erscheint die Symbolleiste, mit der der Vollbildmodus wieder ausgeschaltet werden kann. Auch dies ist eine Symbolleiste, wenn auch nur mit einem Symbol.

```

SUB Vollbild_ein
    Fullscreen(true)
    Symbolleisten_Ausblenden
END SUB

```

Aus dem Vollbild-Modus geht es wieder heraus über die **'ESC'**-Taste. Wenn stattdessen ein Button mit einem entsprechenden Befehl belegt werden soll, so reichen auch die folgenden Zeilen:

```

SUB Vollbild_aus
  Fullscreen(false)
  Symbolleisten_Einblenden
END SUB

```

Formular direkt beim Öffnen der Datenbankdatei starten

Wenn jetzt schon die Symbolleisten weg sind oder gar das Formular im Vollbildmodus erscheint, dann müsste nur noch die Datenbankdatei beim Öffnen direkt in dieses Formular hinein starten. Der einfache Befehl zum Öffnen von Formularen reicht dabei leider nicht aus, da die Datenbankverbindung beim Öffnen des Base-Dokumentes noch nicht besteht.

Das folgende Makro wird über **Extras → Anpassen → Ereignisse → Dokument öffnen** gestartet. Dabei ist **Speichern in → Datenbankdatei.odt** zu wählen.

```

SUB Formular_Direktstart
  DIM oDatenquelle AS OBJECT
  oDatenquelle = ThisDatabaseDocument.CurrentController
  If NOT (oDatenquelle.isConnected()) THEN
    oDatenquelle.connect()
  END IF
  ThisDatabaseDocument.FormDocuments.getByName("Formularname").open
END SUB

```

Zuerst muss der Kontakt mit der Datenquelle hergestellt werden. Der Controller hängt ebenso mit **ThisDatabaseDocument** zusammen wie das Formular. Anschließend kann das Formular gestartet werden und liest auch die Datenbankinhalte aus.

MySQL-Datenbank mit Makros ansprechen

Sämtliche bisher vorgestellten Makros wurden mit der internen HSQLDB verbunden. Bei der Arbeit mit externen Datenbanken sind ein paar Änderungen und Erweiterungen notwendig.

MySQL-Code in Makros

Wird die interne Datenbank angesprochen, so werden die Tabellen und Felder mit doppelten Anführungszeichen gegenüber dem SQL-Code abgesetzt:

```
SELECT "Feld" FROM "Tabelle"
```

Da in Makros der SQL-Befehl Text darstellt, müssen die doppelten Anführungszeichen zusätzlich maskiert werden:

```
stSQL = "SELECT ""Feld"" FROM ""Tabelle"""
```

MySQL-Abfragen hingegen werden anders maskiert:

```
SELECT `Feld` FROM `Datenbank`.`Tabelle`
```

Durch diese andere Form der Maskierung wird daraus im Makro-Code:

```
stSql = "SELECT `Feld` FROM `Datenbank`.`Tabelle`"
```

Temporäre Tabelle als individueller Zwischenspeicher

In den vorhergehenden Kapiteln wurde häufiger eine einzeilige Tabelle zum Suchen oder Filtern von Tabellen genutzt. In einem Mehrbenutzersystem kann darauf nicht zurückgegriffen werden, da sonst andere Nutzer von dem Filterwert eines anderen Nutzers abhängig würden. Temporäre Tabellen sind in MySQL nur für den Nutzer der gerade aktiven Verbindung zugänglich, so dass für die Such- und Filterfunktionen auf diese Tabellenform zugegriffen werden kann.

Diese Tabellen können natürlich nicht vorher erstellt worden sein. Sie müssen beim Öffnen der Base-Datei erstellt werden. Deshalb ist das folgende Makro mit dem Öffnen der *.odt-Datei zu verbinden:

```

SUB CreateTempTable
  oDatenquelle = thisDatabaseDocument.CurrentController
  IF NOT (oDatenquelle.isConnected()) THEN oDatenquelle.connect()
  oVerbindung = oDatenquelle.ActiveConnection()
  oSQL_Anweisung = oVerbindung.createStatement()
  stSql = "CREATE TEMPORARY TABLE IF NOT EXISTS `Suchtmp` (`ID` INT PRIMARY KEY,
    `Name` VARCHAR(50))"
  oSQL_Anweisung.executeUpdate(stSql)
END SUB

```

Zum Start der *.odb-Datei besteht noch keine Verbindung zur externen MySQL-Datenbank. Die Verbindung muss erst einmal hergestellt werden. Dann wird eine temporäre Tabelle mit entsprechend notwendigen Feldern erstellt.

Leider zeigt Base die temporären Tabellen nicht im Tabellencontainer an. Es kann über Abfragen auf diese Tabellen zugegriffen werden. Der Zugriff ist allerdings nur lesend möglich, so dass neue Inhalte für diese Tabellen nur über die direkte SQL-Eingabe oder über Makros erfolgen kann. Für einen einfachen Filterzugriff bietet sich deshalb an, statt einer temporären Tabelle eine feste Tabelle zu nutzen, in der die Filterinhalte zusammen mit der Verbindungsnummer (**CONNECTION_ID**) gespeichert werden.

Filterung über die Verbindungsnummer

Hier wird die Filtertabelle bereits vorher über die GUI erstellt. Die Tabelle wird beim Öffnen der Datenbankdatei allerdings direkt mit entsprechendem Inhalt versorgt:

```

stSql = "REPLACE INTO `Filter` (`Connection_ID`, `Name`)
VALUES(CONNECTION_ID(), NULL)"

```

Die Tabelle ist jetzt auch in Formularen beschreibbar und kann entsprechend einfacher genutzt werden. Für andere Nutzer ist jetzt allerdings sichtbar, nach welchen Begriffen der einzelnen Nutzer gerade sucht. Prinzipiell lässt sich aber der entsprechende auf den einzelnen Nutzer festgelegte Datensatz immer über **CONNECTION_ID()** ermitteln.

Wird die Datenbankdatei wieder geschlossen, so kann auch die Filter-Tabelle entsprechend bereinigt werden:

```

SUB DeleteFilter
  oDatasource = thisDatabaseDocument.CurrentController
  IF NOT (oDatasource.isConnected()) THEN oDatasource.connect()
  oConnection = oDatasource.ActiveConnection()
  oSQL_Command = oConnection.createStatement()
  stSql = "DELETE FROM `Filter` WHERE `Connection_ID` = CONNECTION_ID()"
  oSQL_Command.executeUpdate(stSql)
END SUB

```

Gespeicherte Prozeduren

In MySQL/MariaDB können Prozeduren gespeichert werden. Sollen diese Prozeduren zu bestimmten Zeiten ablaufen, so können sie über **Extras → SQL** mit dem Befehl **CALL `Prozedurname`()**; aufgerufen werden. Erstellen solche Prozeduren von sich aus eine Ergebnismenge in einer temporären Tabelle, so lässt sich diese temporäre Tabelle als nicht bearbeitbare Informationsquelle nutzen.

Hinweis

Auch hier gilt wie im Kapitel «Datenbank erstellen» der Hinweis, dass die direkte Verbindung mit MySQL/MariaDB hier einen Bug hat, der den erneuten Aufruf einer Prozedur verhindert und damit die Arbeit mit Prozeduren unmöglich macht. Die JDBC-Verbindung zur MySQL- bzw. MariaDB ist davon nicht betroffen. Vor der Nutzung von gespeicherten Prozeduren sollte also wenigstens unter **Extras → SQL** zweimal hintereinander die Prozedur mit **CALL ...** testweise abgerufen werden.

Automatischer Aufruf einer Prozedur

Die folgende Prozedur **AlleNamen()** könnte beim Laden eines Formulars ausgelöst werden. Sie muss ablaufen, bevor das Formular selbst Inhalt laden will. Kann das nicht erfolgen, so muss zusätzlich auf das auslösende Formular über das Ereignis Bezug genommen werden und das Formular nach der Ausführung der Prozedur erneut geladen werden.

```
SUB ProcExecute
  oDatasource = thisDatabaseDocument.CurrentController
  IF NOT (oDatasource.isConnected()) THEN oDatasource.connect()
  oConnection = oDatasource.ActiveConnection()
  oSQL_Command = oConnection.createStatement()
  oSQL_Command.executeUpdate("CALL `AlleNamen`();")
END SUB
```

Die Prozedur ersetzt lediglich den Umweg, das Kommando **CALL `AlleNamen`()**; über Extras → SQL eingeben zu müssen. Die Prozedur wird ohne Rückgabewert genutzt. Der Rückgabewert muss per SQL in der Prozedur selbst definiert sein.

Übertragung der Ausgabe einer Prozedur in eine temporäre Tabelle

Dieses Makro geht davon aus, dass die gespeicherte Prozedur von MySQL/MariaDB einen Rückgabewert hat, der aber leider nicht über eine Abfrage, sondern nur direkt über SQL auf der Konsole direkt ausgegeben wird.

```
SUB ProcContentShow
  oDatasource = thisDatabaseDocument.CurrentController
  IF NOT (oDatasource.isConnected()) THEN oDatasource.connect()
  oConnection = oDatasource.ActiveConnection()
  oSQL_Command = oConnection.createStatement()
  oResult = oSQL_Command.executeQuery("CALL `AlleNamen`();")
  stFields = ""
  FOR i = 1 TO oResult.Columns.Count
    stFields = stFields + "`" + oResult.Columns.ElementNames(i-1) + "` TINYTEXT,"
  NEXT
  stFields = Left(stFields, Len(stFields)-1)
  stProcedure = "("
  WHILE oResult.next
    FOR i = 1 TO oResult.Columns.Count
      stProcedure = stProcedure + "'" + oResult.getString(i) + "',"
    NEXT
    stProcedure = Left(stProcedure, Len(stProcedure)-1)
    stProcedure = stProcedure + "),("
  WEND
  stProcedure = Left(stProcedure, Len(stProcedure)-2)
  oSQL_Command.executeUpdate("DROP TEMPORARY TABLE IF EXISTS `TempNamen`")
  oSQL_Command.executeUpdate("CREATE TEMPORARY TABLE `TempNamen` (" + stFields + ")")
  oSQL_Command.executeUpdate("INSERT INTO `TempNamen` VALUES " + stProcedure + ")")
END SUB
```

Zuerst wird die Prozedur ausgeführt. Ein eventueller Rückgabewert wird in **oResult** gespeichert. Aus diesem Rückgabewert lassen sich die Spaltennamen (**oResult.Columns.ElementNames()**) und der Inhalt (**oResult.getString()**) auslesen. Die Feldtypen sind leider nicht zu ermitteln, so dass der Inhalt jeder Spalte einfach als Text interpretiert wird. Dieser Text wird als **TINYTEXT** mit einer Maximallänge von 255 Zeichen anschließend in einer temporären Tabelle gespeichert. Diese Tabelle kann dann zum Recherchieren genutzt werden.

Dialoge

Statt Formularen können für Base auch Dialoge zur Eingabe von Daten, zum Bearbeiten von Daten oder auch zur Wartung der Datenbank genutzt werden. Dialoge lassen sich auf das jeweilige Anwendungsgebiet direkt zuschneiden, sind aber natürlich nicht so komfortabel vordefiniert

wie Formulare. Hier eine kurze Einführung, die mit einem recht komplexen Beispiel zur Datenbankwartung endet.

Dialoge starten und beenden

Zuerst muss für den Dialog²⁹ ein entsprechender Ordner erstellt werden. Dies geschieht über **Extras → Makros → Dialoge verwalten → Datenbankdatei → Standard → Neu**. Der Dialog erscheint mit einer grauen Fläche und einer Titelleiste sowie einem Schließkreuz. Bereits dieser leere Dialog könnte jetzt aufgerufen und über das Schließkreuz wieder geschlossen werden.

Wird der Dialog angeklickt, so gibt es bei den allgemeinen Eigenschaften die Möglichkeit, die Größe und Position einzustellen. Außerdem kann dort der Inhalt des Titels «Dialoge starten» eingegeben werden.



In der am unteren Fensterrand befindlichen Symbolleiste befinden sich die verschiedensten Formular-Steuerelemente. Aus diesen Steuerelementen sind für den abgebildeten Dialog zwei Schaltflächen ausgesucht worden, von denen aus andere Dialoge gestartet werden sollen. Die Bearbeitung des Inhaltes und der Verknüpfung zu Makros ist gleich den Schaltflächen im Formular.

Die Lage der Deklaration der Variablen für den Dialog ist besonders zu beachten. Der Dialog wird als globale Variable gesetzt, damit auf ihn von unterschiedlichen Prozeduren aus zugegriffen werden kann. In diesem Falle ist der Dialog mit der Variablen `oDialog0` versehen, weil es noch weitere Dialoge gibt, die einfach entsprechend durchnummeriert wurden.

```
DIM oDialog0 AS OBJECT
```

Zuerst wird die Bibliothek für den Dialog geladen. Sie liegt in dem Verzeichnis «Standard», sofern bei der Erstellung des Dialogs keine andere Bezeichnung gewählt wurde. Der Dialog selbst ist über den Reiter mit der Bezeichnung «Dialog0» in dieser Bibliothek erreichbar. Mit **Execute()** wird der Dialog aufgerufen.

```
SUB Dialog0Start
    DialogLibraries.LoadLibrary("Standard")
    oDialog0 = createUnoDialog(DialogLibraries.Standard.Dialog0)
    oDialog0.Execute()
END SUB
```

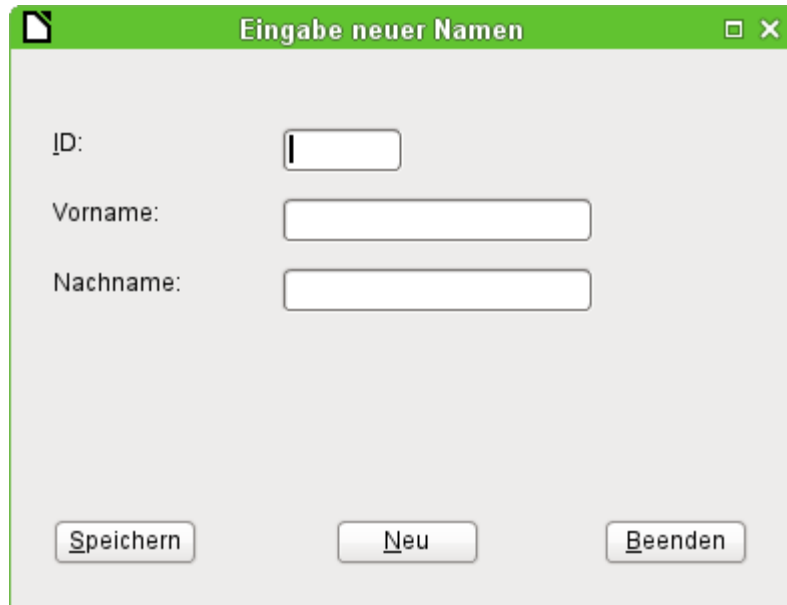
Prinzipiell kann ein Dialog durch Betätigung des Schließkreuzes geschlossen werden. Soll dafür aber ein entsprechender Button vorgesehen werden, so reicht hier einfach der Befehl **EndExecute()** innerhalb einer Prozedur.

```
SUB Dialog0Ende
    oDialog0.EndExecute()
END SUB
```

Mit diesem Rahmen können beliebig gestaltete Dialoge gestartet und wieder geschlossen werden.

²⁹ Die Beispieldatenbank «Beispiel_Dialoge.odt» zu den folgenden Kapiteln ist den Beispieldatenbanken für dieses Handbuch beigelegt.

Einfacher Dialog zur Eingabe neuer Datensätze



The dialog box has a title bar with a green background and the text 'Eingabe neuer Namen'. Inside, there are three labels on the left: 'ID:', 'Vorname:', and 'Nachname:'. To the right of each label is a text input field. The 'ID' field is a small box, while 'Vorname' and 'Nachname' are larger. At the bottom of the dialog, there are three buttons: 'Speichern', 'Neu', and 'Beenden'. The buttons are rectangular with a light gray background and a thin border.

Dieser Dialog stellt eine Vorstufe für den nächstfolgenden Dialog zur Bearbeitung von Datensätzen dar. Erst einmal sollen nur grundlegende Vorgehensweisen im Umgang mit der Tabelle einer Datenbank geklärt werden. Hier ist dies das Speichern von Datensätzen mit neuem Primärschlüssel bzw. das komplett neue Eingeben von Datensätzen. Wie weit so ein kleiner Dialog ausreichend für eine bestimmte Datenbankaufgabe ist, hängt natürlich von den Bedürfnissen des Nutzer ab.

Mit

```
DIM oDialog1 AS OBJECT
```

wird wieder direkt auf der untersten Ebene des Moduls vor allen Prozeduren die globale Variable für den Dialog erstellt.

Der Dialog wird genauso gestartet und beendet wie der vorhergehende Dialog. Lediglich die Bezeichnung ändert sich von Dialog0 auf Dialog1. Die Prozedur zum Beenden des Dialogs ist mit dem Button **Beenden** verbunden.

Über den Button **Neu** werden alle Kontrollfelder des Dialogs durch die Prozedur «DatenfelderLeeren» von vorhergehenden Eingaben befreit:

```
SUB DatenfelderLeeren
    oDialog1.getControl("NumericField1").Text = ""
    oDialog1.getControl("TextField1").Text = ""
    oDialog1.getControl("TextField2").Text = ""
END SUB
```

Jedes Feld, das in einen Dialog eingefügt wird, ist über einen eigenen Namen ansprechbar. Im Gegensatz zu Feldern eines Formulars wird hier durch die Benutzeroberfläche darauf geachtet, dass keine Namen doppelt vergeben werden.

Über **getControl** wird zusammen mit dem Namen auf ein Kontrollfeld zugegriffen. Auch ein numerisches Feld hat hier die Eigenschaft **Text** zur Verfügung. Nur so lässt sich schließlich ein numerisches Feld leeren. Einen leeren Text gibt es, eine leere Nummer hingegen nicht – stattdessen müsste 0 in das Feld für den Primärschlüssel geschrieben werden.

Der Button **Speichern** löst schließlich die Prozedur «Daten1Speichern» aus:

```
SUB Daten1Speichern
    DIM oDatenquelle AS OBJECT
    DIM oVerbindung AS OBJECT
    DIM oSQL_Anweisung AS OBJECT
    DIM loID AS LONG
```

```

DIM stVorname AS STRING
DIM stNachname AS STRING
loID = oDialog1.getControl("NumericField1").Value
stVorname = oDialog1.getControl("TextField1").Text
stNachname = oDialog1.getControl("TextField2").Text
IF loID > 0 AND stNachname <> "" THEN
    oDatenquelle = thisDatabaseDocument.CurrentController
    If NOT (oDatenquelle.isConnected()) THEN
        oDatenquelle.connect()
    END IF
    oVerbindung = oDatenquelle.ActiveConnection()
    oSQL_Anweisung = oVerbindung.createStatement()
    stSql = "SELECT ""ID"" FROM ""Name"" WHERE ""ID"" = '"+loID+"'"
    oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
    WHILE oAbfrageergebnis.next
        MsgBox ("Der Wert für das Feld 'ID' existiert schon",16,
            "Doppelte Dateneingabe")
    EXIT SUB
WEND
stSql = "INSERT INTO ""Name"" (""ID"", ""Vorname"", ""Nachname"")
VALUES ('"+loID+"', '"+stVorname+"', '"+stNachname+"')"
oSQL_Anweisung.executeUpdate(stSql)
DatenfelderLeeren
END IF
END SUB

```

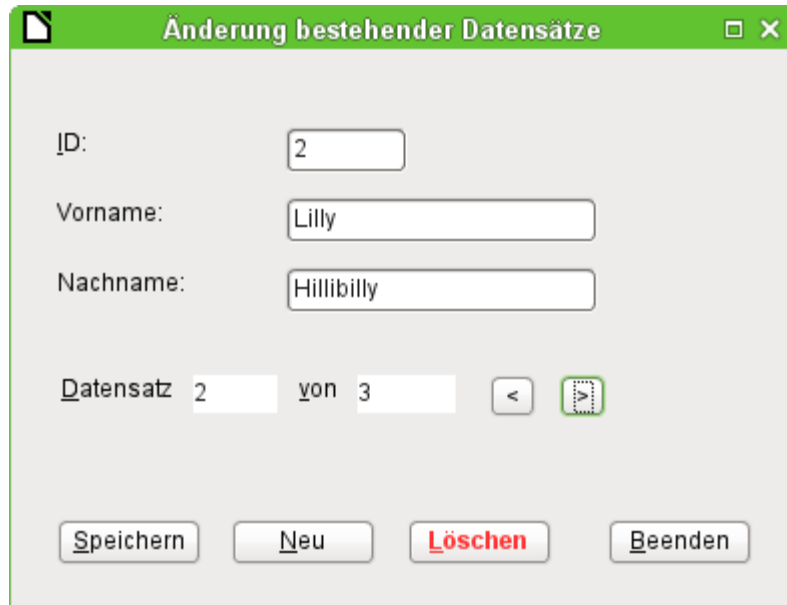
Wie in der Prozedur «DatenfelderLeeren» wird auf die Eingabefelder zugegriffen. Dieses Mal erfolgt der Zugriff allerdings lesend, nicht schreibend. Nur wenn das Feld «ID» eine Eingabe größer als 0 enthält und in dem Feld für den Nachnamen auch Text steht, soll der Datensatz weitergegeben werden. Die Null muss alleine schon deshalb ausgeschlossen werden, weil eine Zahlenvariable für Zahlen ohne Nachkommastellen grundsätzlich mit dem Wert 0 initialisiert wird. Auch bei einem leeren Feld würde also schließlich 0 zur Speicherung weitergegeben.

Sind die beiden Felder entsprechend mit Inhalt versehen, so wird eine Verbindung zur Datenbank aufgenommen. Da sich die Kontrollfelder nicht in einem Formular befinden, muss die Datenbankverbindung über **thisDatabaseDocument.CurrentController** hergestellt werden.

Zuerst wird eine Abfrage an die Datenbank geschickt, ob vielleicht ein Datensatz mit dem vorgegebenen Primärschlüssel schon existiert. Hat die Abfrage Erfolg, so wird eine Meldung über eine Messagebox ausgegeben, die mit einem Stopp-Symbol versehen ist (Code: **16**) und die Überschrift «Doppelte Dateneingabe» trägt. Danach wird durch **Exit SUB** die Prozedur beendet.

Hat die Abfrage keinen Datensatz gefunden, der den gleichen Primärschlüssel hat, so wird der neue Datensatz über den Insert-Befehl in die Datenbank eingefügt. Anschließend wird über die Prozedur «DatenfelderLeeren» wieder ein leeres Formular präsentiert.

Dialog zum Bearbeiten von Daten in einer Tabelle



Dieser Dialog stellt schon deutlich mehr Möglichkeiten zur Verfügung als der vorhergehende Dialog. Hier lassen sich alle Datensätze anzeigen, durch Datensätze navigieren, Datensätze neu erstellen, ändern oder auch löschen. Natürlich ist der Code entsprechend umfangreicher.

Die Button **Beenden** ist mit der entsprechend auf den Dialog2 abgewandelten Prozedur des vorhergehenden Dialogs zur Eingabe neuer Datensätze verbunden. Hier werden nur die weiteren Buttons mit ihren entsprechenden Funktionen beschrieben.

Die Dateneingabe ist im Dialog so beschränkt, dass im Feld «ID» der Mindestwert auf '1' eingestellt wurde. Diese Einschränkung hat mit dem Umgang mit Variablen in Basic zu tun: Zahlenvariablen werden bei der Definition bereits mit '0' als Grundwert vorbelegt. Werden Zahlenwerte von leeren Feldern und Feldern mit '0' ausgelesen, so ist für Basic der anschließende Inhalt der Variablen gleich. Es müsste bei der Nutzung von '0' im Feld «ID» also zur Unterscheidung erst Text ausgelesen und vielleicht später in eine Zahl umgewandelt werden.

Der Dialog wird unter den gleichen Voraussetzungen geladen wie vorher auch. Hier wird allerdings die Ladeprozedur davon abhängig gemacht, ob die Variable, die der Prozedur «DatenLaden» mitgegeben wird, 0 ist.

```
SUB DatenLaden(loID AS LONG)
    DIM oDatenquelle AS OBJECT
    DIM oVerbindung AS OBJECT
    DIM oSQL_Anweisung AS OBJECT
    DIM stVorname AS STRING
    DIM stNachname AS STRING
    DIM loRow AS LONG
    DIM loRowMax AS LONG
    DIM inStart AS INTEGER
    oDatenquelle = thisDatabaseDocument.CurrentController
    If NOT (oDatenquelle.isConnected()) THEN
        oDatenquelle.connect()
    END IF
    oVerbindung = oDatenquelle.ActiveConnection()
    oSQL_Anweisung = oVerbindung.createStatement()
    IF loID < 1 THEN
        stSql = "SELECT MIN(""ID"") FROM ""Name""
        oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
        WHILE oAbfrageergebnis.next
            loID = oAbfrageergebnis.getInt(1)
        WEND
        inStart = 1
    END IF
```

END IF

Die Variablen werden deklariert. Die Datenbankverbindung wird, wie weiter oben erklärt, für den Dialog hergestellt. Zum Start ist **loID** 0. Für diesen Fall wird per SQL der niedrigste Wert für den Primärschlüssel ermittelt. Der entsprechende Datensatz soll in dem Dialog später angezeigt werden. Gleichzeitig wird die Variable **inStart** auf 1 gestellt, damit der Dialog später gestartet wird. Enthält die Tabelle keine Daten, so bleibt **loID** 0. Entsprechend muss auch nicht nach dem Inhalt und der Anzahl irgendwelcher Datensätze im Folgenden gesucht werden.

Nur wenn **loID** größer als 0 ist, wird zuerst mit einer Abfrage überprüft, welche Daten in dem Datensatz enthalten sind. Anschließend werden in einer zweiten Abfrage alle Datensätze für die Datensatzanzeige gezählt. Mit der dritten Abfrage wird die Position des aktuellen Datensatzes ermittelt, indem alle Datensätze, die einen kleineren oder eben den aktuellen Primärschlüssel haben, zusammengezählt werden.

```
IF loID > 0 THEN
    stSql = "SELECT * FROM ""Name"" WHERE ""ID"" = '"+loID+"'"
    oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
    WHILE oAbfrageergebnis.next
        loID = oAbfrageergebnis.getInt(1)
        stVorname = oAbfrageergebnis.getString(2)
        stNachname = oAbfrageergebnis.getString(3)
    WEND
    stSql = "SELECT COUNT(""ID"") FROM ""Name""""
    oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
    WHILE oAbfrageergebnis.next
        loRowMax = oAbfrageergebnis.getInt(1)
    WEND
    stSql = "SELECT COUNT(""ID"") FROM ""Name"" WHERE ""ID"" <= '"+loID+"'"
    oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
    WHILE oAbfrageergebnis.next
        loRow = oAbfrageergebnis.getInt(1)
    WEND
    oDialog2.getControl("NumericField1").Value = loID
    oDialog2.getControl("TextField1").Text = stVorname
    oDialog2.getControl("TextField2").Text = stNachname
END IF
oDialog2.getControl("NumericField2").Value = loRow
oDialog2.getControl("NumericField3").Value = loRowMax
IF loRow = 1 THEN
    ' Vorheriger Datensatz
    oDialog2.getControl("CommandButton4").Model.enabled = False
ELSE
    oDialog2.getControl("CommandButton4").Model.enabled = True
END IF
IF loRow <= loRowMax THEN
    ' Nächster Datensatz | Neuer Datensatz | Löschen
    oDialog2.getControl("CommandButton5").Model.enabled = True
    oDialog2.getControl("CommandButton2").Model.enabled = True
    oDialog2.getControl("CommandButton6").Model.enabled = True
ELSE
    oDialog2.getControl("CommandButton5").Model.enabled = False
    oDialog2.getControl("CommandButton2").Model.enabled = False
    oDialog2.getControl("CommandButton6").Model.enabled = False
END IF
IF inStart = 1 THEN
    oDialog2.Execute()
END IF
END SUB
```

Die ermittelten Werte für die Formularfelder werden übertragen. Die Einträge für die Nummer des aktuellen Datensatzes sowie die Anzahl aller Datensätze werden auf jeden Fall mit einer Zahl versorgt. Ist kein Datensatz vorhanden, so wird hier über den Default-Wert für eine numerische Variable 0 eingefügt.

Die Buttons zum Navigieren  («CommandButton5») und  («CommandButton4») sind nur verfügbar, wenn es möglich ist, einen entsprechenden Datensatz über die Navigation zu erreichen. Ansonsten werden sie vorübergehend mit **enabled = False** deaktiviert. Gleiches gilt für die Buttons **Neu** und **Löschen**. Sie sollen dann nicht verfügbar sein, wenn die Zahl der angezeigten Zeilen höher ist als die maximal ermittelte Zeilenzahl. Dies ist für die Eingabe neuer Datensätze die Standardeinstellung dieses Dialogs.

Der Dialog soll möglichst nur dann gestartet werden, wenn er direkt aus einer Startdatei über **DatenLaden(0)** erstellt werden soll. Deshalb wurde die gesonderte Variable **inStart** mit dem Wert 1 zu Beginn der Prozedur versehen..

Über den Button  soll zu dem vorhergehenden Datensatz navigiert werden können. Der Button ist nur dann aktiv ist, wenn nicht bereits der erste Datensatz angezeigt wird. Zum Navigieren wird von dem aktuellen Datensatz der Wert für den Primärschlüssel aus dem Feld «NumericField1» ausgelesen.

Hier gilt es zwei Fälle zu unterscheiden:

1. Es wurde vorher vorwärts zu einer Neueingabe navigiert, so dass das entsprechende Feld keinen Wert enthält. **loID** gibt dann den Standardwert wieder, der durch die Definition als Zahlvariable vorgegeben ist: 0.
2. Ansonsten enthält loID einen Wert, der größer als 0 ist. Entsprechend kann über eine Abfrage die nächstkleinere «ID» ermittelt werden.

```
SUB vorherigerDatensatz
DIM loID AS LONG
DIM loIDneu AS LONG
loID = oDialog2.getControl("NumericField1").Value
oDatenquelle = thisDatabaseDocument.CurrentController
If NOT (oDatenquelle.isConnected()) THEN
    oDatenquelle.connect()
END IF
oVerbindung = oDatenquelle.ActiveConnection()
oSQL_Anweisung = oVerbindung.createStatement()
IF loID < 1 THEN
    stSql = "SELECT MAX(""ID"") FROM ""Name""
ELSE
    stSql = "SELECT MAX(""ID"") FROM ""Name"" WHERE ""ID"" < '"+loID+"'"
END IF
oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
WHILE oAbfrageergebnis.next
    loIDneu = oAbfrageergebnis.getInt(1)
WEND
IF loIDneu > 0 THEN
    DatenLaden(loIDneu)
END IF
END SUB
```

Bei einem leeren «ID»-Feld soll auf den Datensatz mit dem höchsten Wert in der Primärschlüsselnummer gewechselt werden. Können hingegen aus dem «ID»-Feld Daten entnommen werden, so wird der entsprechend nachrangige Wert für die "ID" ermittelt.

Das Ergebnis dieser Abfrage dient dazu, die Prozedur «DatenLaden» mit dem entsprechenden Schlüsselwert erneut durchlaufen zu lassen.

Über den Button  wird zum nächsten Datensatz navigiert. Diese Navigationsmöglichkeit steht nur zur Verfügung, wenn nicht bereits der Dialog für die Eingabe eines neuen Datensatzes geleert wurde. Dies ist natürlich auch beim Start und leerer Tabelle der Fall.

Zwangsläufig ist in dem Feld «NumericField1» ein Wert vorhanden. Von diesem Wert ausgehend kann also per SQL nachgesehen werden, welcher Primärschlüsselwert der nächsthöhere in der Tabelle ist. Bleibt die Abfrage leer, weil es keinen entsprechenden Datensatz gibt, so ist der Wert für **loIDneu = 0**. Ansonsten kann über die Prozedur «DatenLaden» der Inhalt des nächsten Datensatzes geladen werden.

```

SUB naechsterDatensatz
    DIM loID AS LONG
    DIM loIDneu AS LONG
    loID = oDialog2.getControl("NumericField1").Value
    oDatenquelle = thisDatabaseDocument.CurrentController
    If NOT (oDatenquelle.isConnected()) THEN
        oDatenquelle.connect()
    END IF
    oVerbindung = oDatenquelle.ActiveConnection()
    oSQL_Anweisung = oVerbindung.createStatement()
    stSql = "SELECT MIN(""ID"") FROM ""Name"" WHERE ""ID"" > '"+loID+"'"
    oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
    WHILE oAbfrageergebnis.next
        loIDneu = oAbfrageergebnis.getInt(1)
    WEND
    IF loIDneu > 0 THEN
        DatenLaden(loIDneu)
    ELSE
        Datenfelder2Leeren
    END IF
END SUB

```

Existiert beim Navigieren zum nächsten Datensatz kein weiterer Datensatz, so löst die Navigation die folgende Prozedur «Datenfelder2Leeren» aus, die zur Eingabe neuer Daten dient.

Mit der Prozedur «Datenfelder2Leeren» werden nicht nur die Datenfelder selbst geleert. Die Position des aktuellen Datensatzes wird um einen Datensatz höher als die maximale Datensatzzahl eingestellt. Das soll verdeutlichen, dass der aktuell bearbeitete Datensatz noch nicht in der Datenbank enthalten ist.

Sobald «Datenfelder2Leeren» ausgelöst wird, wird außerdem die Möglichkeit des Sprungs zum vorhergehenden Datensatz aktiviert. Sprünge zu einem nachfolgenden Datensatz, das erneute Aufrufen der Prozedur über **Neu** oder das **Löschen** sind deaktiviert.

```

SUB Datenfelder2Leeren
    loRowMax = oDialog2.getControl("NumericField3").Value
    oDialog2.getControl("NumericField1").Text = ""
    oDialog2.getControl("TextField1").Text = ""
    oDialog2.getControl("TextField2").Text = ""
    oDialog2.getControl("NumericField2").Value = loRowMax + 1
    oDialog2.getControl("CommandButton4").Model.enabled = True ' Vorheriger Datensatz
    oDialog2.getControl("CommandButton5").Model.enabled = False ' Nächster Datensatz
    oDialog2.getControl("CommandButton2").Model.enabled = False ' Neuer Datensatz
    oDialog2.getControl("CommandButton6").Model.enabled = False ' Löschen
END SUB

```

Das Speichern der Daten soll nur möglich sein, wenn in den Feldern für «ID» und «Nachname» ein Eintrag erfolgt ist. Ist diese Bedingung erfüllt, so wird überprüft, ob der Datensatz ein neuer Datensatz ist. Das funktioniert über den Datensatzanzeiger, der bei neuen Datensätzen so eingestellt wurde, dass er für den aktuellen Datensatz einen um 1 höheren Wert als den maximalen Wert an Datensätzen ausgibt.

Im Falle eines neuen Datensatzes gibt es weiteren Überprüfungsbedarf, damit eine Speicherung einwandfrei funktionieren kann. Kommt die Ziffer für den Primärschlüssel bereits einmal vor, so erfolgt eine Warnung. Wird die entsprechende Frage mit **Ja** bestätigt, so wird der alte Datensatz mit der gleichen Schlüsselnummer überschrieben. Ansonsten erfolgt keine Speicherung. Solange noch gar kein Datensatz in der Datenbank enthalten ist (**loRowMax = 0**) braucht diese Überprüfung nicht zu erfolgen. In dem Falle kann der Datensatz direkt als neuer Datensatz abgespeichert werden. Bei einem neuen Datensatz wird schließlich noch die Zahl der Datensätze um 1 erhöht und die Eingabe für den nächsten Datensatz frei gemacht.

Bei bestehenden Datensätzen wird einfach der alte Datensatz durch ein Update mit dem neuen Datensatz überschrieben.

```

SUB Daten2Speichern(oEvent AS OBJECT)
    DIM oDatenquelle AS OBJECT

```

```

DIM oVerbindung AS OBJECT
DIM oSQL_Anweisung AS OBJECT
DIM oDlg AS OBJECT
DIM loID AS LONG
DIM stVorname AS STRING
DIM stNachname AS STRING
DIM inMsg AS INTEGER
DIM loRow AS LONG
DIM loRowMax AS LONG
DIM stSql AS STRING
oDlg = oEvent.Source.getContext()
loID = oDlg.getControl("NumericField1").Value
stVorname = oDlg.getControl("TextField1").Text
stNachname = oDlg.getControl("TextField2").Text
IF loID > 0 AND stNachname <> "" THEN
    oDatenquelle = thisDatabaseDocument.CurrentController
    IF NOT (oDatenquelle.isConnected()) THEN
        oDatenquelle.connect()
    END IF
    oVerbindung = oDatenquelle.ActiveConnection()
    oSQL_Anweisung = oVerbindung.createStatement()
    loRow = oDlg.getControl("NumericField2").Value
    loRowMax = oDlg.getControl("NumericField3").Value
    IF loRowMax < loRow THEN
        IF loRowMax > 0 THEN
            stSql = "SELECT ""ID"" FROM ""Name"" WHERE ""ID"" = '"+loID+"'"
            oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
            WHILE oAbfrageergebnis.next
                inMsg = MsgBox ("Der Wert für das Feld 'ID' existiert schon." &
                    CHR(13) & "Soll der Datensatz überschrieben werden?", 20,
                    "Doppelte Dateneingabe")
                IF inMsg = 6 THEN
                    stSql = "UPDATE ""Name"" SET ""Vorname""='"+stVorname+"',
                        ""Nachname""='"+stNachname+"' WHERE ""ID"" = '"+loID+"'"
                    oSQL_Anweisung.executeUpdate(stSql)
                    DatenLaden(loID) ' Beim Update wurde ein bestehender Datensatz
                        überschrieben. Neueinlesen zur Korrektur der Datensatzzahlen
                END IF
            EXIT SUB
        WEND
    END IF
    stSql = "INSERT INTO ""Name"" (""ID"", ""Vorname"", ""Nachname"") VALUES
        ('"+loID+"', '"+stVorname+"', '"+stNachname+"')"
    oSQL_Anweisung.executeUpdate(stSql)
    oDlg.getControl("NumericField3").Value = loRowMax + 1
    ' Nach dem Insert existiert ein Datensatz mehr
    Datenfelder2Leeren
    ' Nach einem Insert wird grundsätzlich zum nächsten Insert geschaltet
ELSE
    stSql = "UPDATE ""Name"" SET ""Vorname""='"+stVorname+"',
        ""Nachname""='"+stNachname+"' WHERE ""ID"" = '"+loID+"'"
    oSQL_Anweisung.executeUpdate(stSql)
END IF
END IF
END SUB

```

Die Löschprozedur ist mit einer Nachfrage versehen, die versehentliches Löschen verhindern soll. Dadurch, dass der Button deaktiviert wird, wenn die Eingabefelder leer sind, dürfte es nicht vorkommen, dass das Feld «NumericField1» leer ist. Deshalb könnte die Überprüfung der Bedingung **IF loID > 0** auch entfallen.

Beim Löschen wird die Zahl der Datensätze um einen Datensatz herabgesetzt. Dies muss entsprechend mit **loRowMax - 1** korrigiert werden. Anschließend wird der dem aktuellen Datensatz folgende Datensatz angezeigt.

```

SUB DatenLoeschen(oEvent AS OBJECT)
    DIM oDatenquelle AS OBJECT
    DIM oVerbindung AS OBJECT

```



```

DIM oSQL_Anweisung AS OBJECT
DIM oDlg AS OBJECT
DIM loID AS LONG
oDlg = oEvent.Source.getContext()
loID = oDlg.getControl("NumericField1").Value
IF loID > 0 THEN
    inMsg = MsgBox ("Soll der Datensatz wirklich gelöscht werden?", 20,
        "Löschen eines Datensatzes")
    IF inMsg = 6 THEN
        oDatenquelle = thisDatabaseDocument.CurrentController
        If NOT (oDatenquelle.isConnected()) THEN
            oDatenquelle.connect()
        END IF
        oVerbindung = oDatenquelle.ActiveConnection()
        oSQL_Anweisung = oVerbindung.createStatement()
        stSql = "DELETE FROM ""Name"" WHERE ""ID"" = '"+loID+"'"
        oSQL_Anweisung.executeUpdate(stSql)
        loRowMax = oDlg.getControl("NumericField3").Value
        oDlg.getControl("NumericField3").Value = loRowMax - 1
        naechsterDatensatz
    END IF
ELSE
    MsgBox ("Kein Datensatz gelöscht." & CHR(13) &
        "Es fehlt eine Datensatzauswahl.", 64, "Löschung nicht möglich")
END IF
END SUB

```

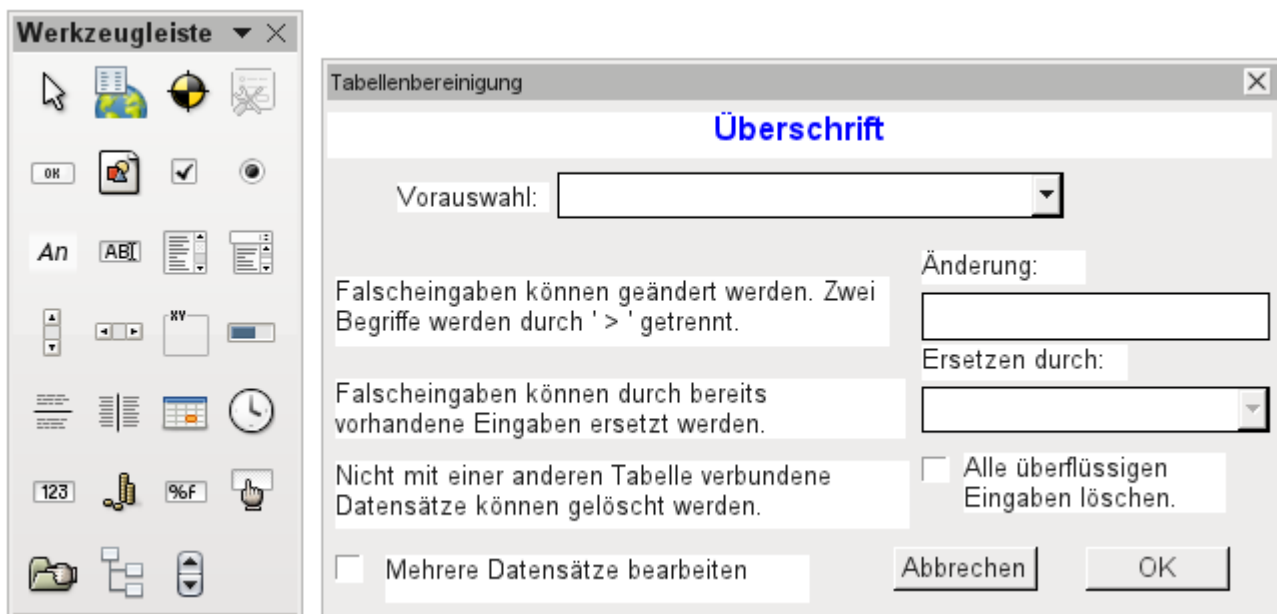
Bereits dieser kleine Dialog zur Bearbeitung von Daten zeigt, dass der Aufwand im Makrocode schon erheblich ist, um die Grundlagen einer Datenbearbeitung zu gewährleisten. Der Zugriff über ein Formular ist hier erheblich einfacher. Der Dialog kann dagegen recht flexibel an die Bedürfnisse des Programms angepasst werden. Nur ist das eben nicht für die Erstellung einer Datenbankbedienung im Schnellverfahren gedacht.

Fehleinträge von Tabellen mit Hilfe eines Dialogs bereinigen

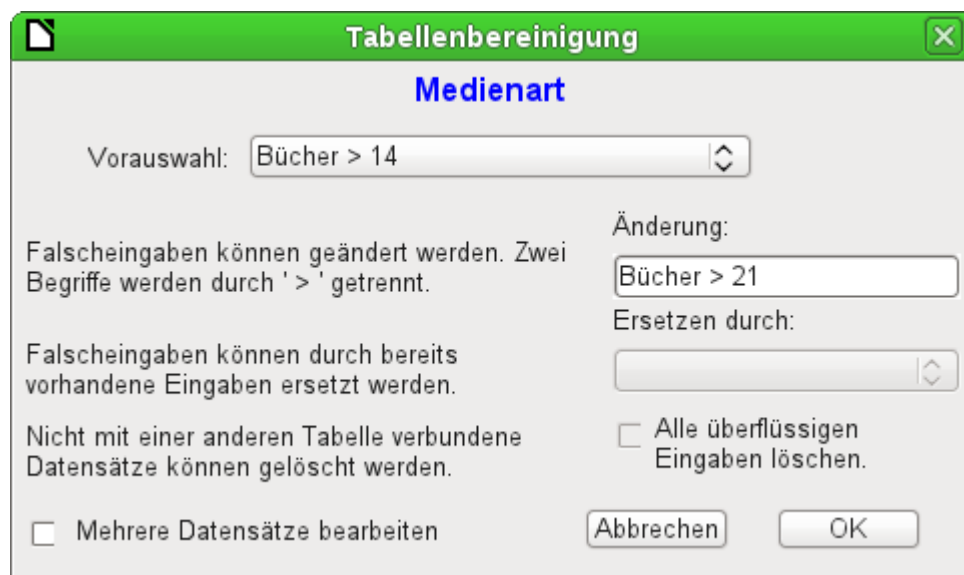
Fehleingaben in Feldern fallen häufig erst später auf. Manchmal müssen auch gleich mehrere Datensätze mit der gleichen Eingabe auf einmal geändert werden. Dies ist in der normalen Tabellenansicht umständlich, je mehr Änderungen vorgenommen werden müssen, da für jeden Datensatz einzeln eine Eingabe erforderlich ist.

Formulare könnten hier mit Makros greifen. Wird aber für viele Tabellen ein identisch aufgebautes Formular benötigt, so bietet sich an, dies mit Dialogen zu erledigen. Ein Dialog wird zu Beginn mit den notwendigen Daten zu der jeweiligen Tabelle versehen und kann so statt mehrerer Formulare genutzt werden.

Diese Dialoge müssen für FIREBIRD wegen der unterschiedlichen Systemtabellen entsprechend angepasst werden. Zu den Systemtabellen siehe den Anhang dieses Handbuches.



Dialoge werden neben den Modulen für Makros abgespeichert. Ihre Erstellung erfolgt ähnlich der eines Formulars. Hier stehen auch weitgehend ähnliche Kontrollfelder zur Verfügung. Lediglich das Tabellenkontrollfeld aus dem Formular fehlt als besondere Eingabemöglichkeit.



Wird ein Dialog ausgeführt, so erscheinen die Kontrollfelder entsprechend der Einstellung der grafischen Benutzeroberfläche.

Der oben abgebildete Dialog der Beispieldatenbank soll dazu dienen, die Tabellen zu bearbeiten, die nicht direkt in einem der Formulare als Grundlage vorhanden sind. So ist z.B. die Medienart über ein Listefeld zugänglich, in der Makro-Version bereits durch ein Kombinationsfeld. In der Makro-Version können die Inhalte der Felder zwar durch neue Inhalte ergänzt werden, eine Änderung alter Inhalte ist aber nicht möglich. In der Version ohne Makros erfolgt die Änderung über ein separates Tabellenkontrollfeld.

Während die Änderung noch ohne Makros recht einfach in den Griff zu bekommen ist, so ist es doch recht umständlich, die Medienart vieler Medien auf eine andere Medienart zu ändern. Angenommen, es gäbe die Medienarten 'Buch, gebunden', 'Buch, kartoniert', 'Taschenbuch' und 'Ringbuch'. Jetzt stellt sich nach längerem Betrieb der Datenbank heraus, dass muntere Zeitgenossen noch weitere ähnliche Medienarten für gedruckte Werke vorgesehen haben. Nur ist uns die Differenzierung viel zu weitgehend. Es soll also reduziert werden, am liebsten auf nur einen Begriff.

Ohne Makro müssten jetzt die Datensätze in der Tabelle Medien (mit Hilfe von Filtern) aufgesucht werden und einzeln geändert werden. Mit Kenntnis von SQL geht dies über die SQL-Eingabe schon wesentlich besser. Mit einer Eingabe werden alle Datensätze der Tabelle Medien geändert. Mit einer zweiten SQL-Anweisung wird dann die jetzt überflüssige Medienart gelöscht, die keine Verbindung mehr zur Tabelle "Medien" hat. Genau dieses Verfahren wird mit diesem Dialog über «Ersetzen durch:» angewandt – nur dass eben die SQL-Anweisung erst über das Makro an die Tabelle "Medienart" angepasst wird, da das Makro auch andere Tabellen bearbeiten können soll.

Manchmal schleichen sich auch Eingaben in eine Tabelle ein, die im Nachhinein in den Formularen geändert wurden, also eigentlich gar nicht mehr benötigt werden. Da kann es nicht schaden, solche verwaisten Datensätze einfach zu löschen. Nur sind die über die grafische Oberfläche recht schwer ausfindig zu machen. Hier hilft wieder eine entsprechende SQL-Abfrage, die mit einer Löschanweisung gekoppelt ist. Diese Anweisung ist im Dialog je nach betroffener Tabelle unter «Alle überflüssigen Eingaben löschen» hinterlegt.

Sollen mit dem Dialog mehrere Änderungen durchgeführt werden, so ist dies über das Markierfeld «Mehrere Datensätze bearbeiten» anzugeben. Dann endet der Dialog nicht mit der Betätigung des Buttons «OK».

Der Makrocode für diesen Dialog ist aus der Beispieldatenbank ersichtlich. Im Folgenden werden nur Ausschnitte daraus erläutert.

```
SUB Tabellenbereinigung(oEvent AS OBJECT)
```

Das Makro soll über Einträge im Bereich «Zusatzinformationen» des jeweiligen Buttons gestartet werden.

```
0: Formular, 1: Unterformular, 2: UnterUnterformular, 3: Kombinationsfeld oder  
Tabellenkontrollfeld, 4: Fremdschlüsselfeld im Formular, bei Tabellenkontrollfeld  
leer, 5: Tabellename Nebentabelle, 6: Tabellenfeld1 Nebentabelle, 7: Tabellenfeld2  
Nebentabelle, ggf. 8: Tabellename Nebentabelle für Tabellenfeld 2
```

Die Einträge in diesem Bereich werden zu Beginn des Makros als Kommentar aufgelistet. Die damit verbundenen Ziffern geben die Ziffern wieder, unter denen der jeweilige Eintrag aus dem Array ausgelesen wird. Das Makro kann Listenfelder verarbeiten, die zwei Einträge, getrennt durch «>», enthalten. Diese beiden Einträge können auch aus unterschiedlichen Tabellen stammen und über eine Abfrage zusammengeführt sein, wie z.B. bei der Tabelle "Postleitzahl", die für die Orte lediglich das Fremdschlüsselfeld "Ort_ID" enthält, zur Darstellung des Ortes also die Tabelle "Ort" benötigt.

```
DIM aFremdTabellen(0, 0 to 1)  
DIM aFremdTabellen2(0, 0 to 1)
```

Unter den zu Beginn definierten Variablen fallen zwei Arrays auf. Während normale Arrays auch durch den Befehl '**Split()**' während der Laufzeit der Prozedur erstellt werden können, müssen zweidimensionale Arrays vorher definiert werden. Zweidimensionale Arrays werden z.B. benötigt, um aus einer Abfrage mehrere Datensätze zu speichern, bei denen die Abfrage selbst über mehr als ein Feld geht. Die beiden obigen Arrays müssen Abfragen auswerten, die sich jeweils auf zwei Tabellenfelder beziehen. Deshalb werden sie in der zweiten Dimension mit '0 to 1' auf zwei unterschiedliche Inhalte festgelegt.

```
stTag = oEvent.Source.Model.Tag  
aTabelle() = Split(stTag, ", ")  
FOR i = LBound(aTabelle()) TO UBound(aTabelle())  
    aTabelle(i) = trim(aTabelle(i))  
NEXT
```

Die mitgegebenen Variablen werden ausgelesen. Die Reihenfolge steht im obigen Kommentar. Es gibt maximal 9 Einträge, wobei geklärt werden muss, ob ein 8. Eintrag für das Tabellenfeld2 und ein 9. Eintrag für eine zweite Tabelle existieren.

Wenn Werte aus einer Tabelle entfernt werden, so muss zuerst einmal berücksichtigt werden, ob sie nicht noch als Fremdschlüssel in anderen Tabellen existieren. In einfachen Tabellenkonstruktionen gibt es von einer Tabelle aus lediglich eine Fremdschlüsselverbindung zu einer anderen

Tabelle. In der vorliegenden Beispieldatenbank aber wird z.B. die Tabelle "Ort" genutzt, um die Erscheinungsorte der Medien und die Orte für die Adressen zu speichern. Es wird also zweimal der Primärschlüssel der Tabelle "Ort" in unterschiedlichen Tabellen eingetragen. Diese Tabellen und Fremdschlüsselbezeichnungen könnten natürlich auch über die «Zusatzinformationen» eingegeben werden. Schöner wäre es aber, wenn sie universell für alle Fälle ermittelt werden. Dies geschieht durch die folgende Abfrage.

```
stSql = "SELECT ""FKTABLE_NAME"", ""FKCOLUMN_NAME"" FROM
""INFORMATION_SCHEMA"". ""SYSTEM_CROSSREFERENCE"" WHERE ""PKTABLE_NAME"" = '"
+ aTabelle(5) + "'"
```

In der Datenbank sind im Bereich "INFORMATION_SCHEMA" alle Informationen zu den Tabellen der Datenbank abgespeichert, so auch die Informationen zu den Fremdschlüsseln. Die entsprechende Tabelle, die diese Informationen enthält, ist über "INFORMATION_SCHEMA"."SYSTEM_CROSSREFERENCE" erreichbar. Mit "PKTABLE_NAME" wird die Tabelle erreicht, die ihren Primärschlüssel ("Primary Key") in die Beziehung mit einbringt. Mit "FKTABLE_NAME" wird die Tabelle erreicht, die diesen Primärschlüssel als Fremdschlüssel ("Foreign Key") nutzt. Über "FKCOLUMN_NAME" wird schließlich die Bezeichnung des Fremdschlüsselfeldes ermittelt.

Die Tabelle, die einen Primärschlüssel als Fremdschlüssel zur Verfügung stellt, befindet sich in dem vorher erstellten Array an der 6. Position. Da die Zählung mit 0 beginnt, wird der Wert aus dem Array mit **aTabelle(5)** ermittelt.

```
inZaehler = 0
stFremdIDTab1Tab2 = "ID"
stFremdIDTab2Tab1 = "ID"
stNebentabelle = aTabelle(5)
```

Bevor die Auslesung des Arrays gestartet wird, müssen einige Standardwerte gesetzt werden. Dies sind der Zähler für das Array, in das die Werte der Nebentabelle geschrieben werden, der Standardprimärschlüssel, wenn nicht der Fremdschlüssel für eine zweite Tabelle benötigt wird und die Standardnebentabelle, die sich auf die Haupttabelle bezieht, bei Postleitzahl und Ort z.B. die Tabelle für die Postleitzahl.

Bei der Verknüpfung von zwei Feldern zur Anzeige in den Listenfeldern kann es ja, wie oben erwähnt, zu einer Verknüpfung über zwei Tabellen kommen. Für die Darstellung von Postleitzahl und Ort lautet hier die Abfrage

```
SELECT "Postleitzahl"."Postleitzahl" || ' > ' || "Ort"."Ort"
FROM "Postleitzahl", "Ort"
WHERE "Postleitzahl"."Ort_ID" = "Ort"."ID"
```

Die Tabelle, die sich auf das erste Feld bezieht (Postleitzahl), ist mit der zweiten Tabelle über einen Fremdschlüssel verbunden. Lediglich die Information der beiden Tabellen und der Felder "Postleitzahl" und "Ort" wurde dem Makro mitgegeben. Die Primärschlüssel sind standardmäßig in dem Beispiel mit der Bezeichnung "ID" versehen. Der Fremdschlüssel von "Ort" in "Postleitzahl" muss also über das Makro ermittelt werden.

Genauso muss über das Makro jede andere Tabelle ermittelt werden, mit der die Inhalte des Listenfeldes über Fremdschlüssel in Verbindung stehen.

```
oAbfrageergebnis = oSQL_Anweisung.executeQuery(stSql)
WHILE oAbfrageergebnis.next
  ReDim Preserve aFremdTabellen(inZaehler,0 to 1)
```

Das Array muss jedes Mal neu dimensioniert werden. Damit die alten Inhalte erhalten bleiben, erfolgt über (Preserve) eine Sicherung des vorherigen Inhaltes.

```
aFremdTabellen(inZaehler,0) = oAbfrageergebnis.getString(1)
```

Auslesen des ersten Feldes mit dem Namen der Tabelle, die den Fremdschlüssel enthält. Ergebnis für die Tabelle "Postleitzahl" ist hier die Tabelle "Adresse".

```
aFremdTabellen(inZaehler,1) = oAbfrageergebnis.getString(2)
```

Auslesen des zweiten Feldes mit der Bezeichnung des Fremdschlüsselfeldes. Ergebnis für die Tabelle "Postleitzahl" ist hier das Feld "Postleitzahl_ID" in der Tabelle "Adresse".

Für den Fall, dass dem Aufruf der Prozedur auch der Name einer zweiten Tabelle mitgegeben wurde, erfolgt die folgende Schleife. Nur wenn der Name der zweiten Tabelle als Fremdschlüssel-tabelle für die erste Tabelle auftaucht, erfolgt hier eine Änderung der Standardeinträge. In unserem Fall kommt dies nicht vor, da die Tabelle "Ort" keinen Fremdschlüssel der Tabelle "Postleitzahl" enthält. Der Standardeintrag für die Nebentabelle bleibt also bei "Postleitzahl"; schließlich ist die Kombination von Postleitzahl und Ort eine Grundlage für die Adressentabelle, die einen Fremdschlüssel zu der Tabelle "Postleitzahl" enthält.

```
IF UBound(aTabelle()) = 8 THEN
  IF aTabelle(8) = aFremdTabellen(inZaehler,0) THEN
    stFremdIDTab2Tab1 = aFremdTabellen(inZaehler,1)
    stNebentabelle = aTabelle(8)
  END IF
END IF
inZaehler = inZaehler + 1
```

Da eventuell noch weitere Werte auszulesen sind, erfolgt eine Erweiterung des Zählers zur Neudimensionierung des Arrays. Anschließend wird die Schleife beendet.

```
WEND
```

Existiert im Aufruf der Prozedur ein zweiter Tabellennamen, so wird die gleiche Abfrage jetzt mit dem zweiten Tabellennamen gestartet:

```
IF UBound(aTabelle()) = 8 THEN
```

Der Ablauf ist identisch. Nur wird in der Schleife jetzt gesucht, ob vielleicht der erste Tabellennamen als Fremdschlüssel-Tabellennamen auftaucht. Das ist hier der Fall: Die Tabelle "Postleitzahl" enthält den Fremdschlüssel "Ort_ID" aus der Tabelle "Ort". Dieser Fremdschlüssel wird also jetzt der Variablen «stFremdIDTab1Tab2» zugewiesen, so dass die Beziehung der Tabellen untereinander definiert werden kann.

```
IF aTabelle(5) = aFremdTabellen2(inZaehler,0) THEN
  stFremdIDTab1Tab2 = aFremdTabellen2(inZaehler,1)
END IF
```

Nach einigen weiteren Einstellungen zur korrekten Rückkehr nach Aufruf des Dialogs in die entsprechenden Formulare (Ermittlung der Zeilennummer des Formulars, damit nach einem Neueinlesen auf die Zeilennummer wieder gesprungen werden kann) startet die Schleife, die den Dialog gegebenenfalls wieder neu erstellt, wenn die erste Aktion erfolgt ist, der Dialog aber für weitere Aktionen offen gehalten werden soll. Die Einstellung zur Wiederholung erfolgt über das entsprechende Markierfeld.

```
DO
```

Bevor der Dialog gestartet wird, wird erst einmal der Inhalt der Listenfelder ermittelt. Dabei muss berücksichtigt werden, ob die Listenfelder zwei Tabellenfelder darstellen und eventuell sogar einen Bezug zu zwei Tabellen haben.

```
IF UBound(aTabelle()) = 6 THEN
```

Das Listenfeld bezieht sich nur auf eine Tabelle und ein Feld, da das Array bei dem Tabellenfeld1 der Nebentabelle endet.

```
stSql = "SELECT "" + aTabelle(6) + "" FROM "" + aTabelle(5)
+ "" ORDER BY "" + aTabelle(6) + ""
ELSEIF UBound(aTabelle()) = 7 THEN
```

Das Listenfeld bezieht sich auf zwei Tabellenfelder, aber nur auf eine Tabelle, da das Array bei dem Tabellenfeld2 der Nebentabelle endet.

```
stSql = "SELECT "" + aTabelle(6) + ""||' > '|"" + aTabelle(7)
+ "" FROM "" + aTabelle(5) + "" ORDER BY "" + aTabelle(6) + ""
ELSE
```

Das Listenfeld hat zwei Tabellenfelder und zwei Tabellen als Grundlage. Diese Abfrage trifft also auf das Beispiel mit der Postleitzahl und den Ort zu.

```

stSql = "SELECT "" + aTabelle(5) + ""."" + aTabelle(6) + ""||' > '||""
+ aTabelle(8) + ""."" + aTabelle(7) + "" FROM "" + aTabelle(5)
+ """, "" + aTabelle(8) + "" WHERE "" + aTabelle(8) + "".""
+ stFremdIDTab2Tab1 + "" = "" + aTabelle(5) + "".""
+ stFremdIDTab1Tab2 + "" ORDER BY "" + aTabelle(6) + ""
END IF

```

Hier erfolgt die erste Auswertung zur Ermittlung von Fremdschlüsseln. Die Variablen «stFremdIDTab2Tab1» und «stFremdIDTab1Tab2» starten mit dem Wert "ID". Für «stFremdIDTab1Tab2» wurde in der Auswertung der vorhergehenden Abfrage ein anderer Wert ermittelt, nämlich der Wert "Ort_ID". Damit ergibt die vorherige Abfragekonstruktion genau den Inhalt, der weiter oben bereits für Postleitzahl und Ort formuliert wurde – lediglich erweitert um die Sortierung.

Jetzt muss der Kontakt zu den Listefeldern erstellt werden, damit diese mit dem Inhalt der Abfragen bestückt werden. Diese Listfelder existieren noch nicht, da noch gar kein Dialog existiert. Dieser Dialog wird mit den folgenden Zeilen erst einmal im Speicher erstellt, bevor er tatsächlich auf dem Bildschirm ausgeführt wird.

```

DialogLibraries.LoadLibrary("Standard")
oDlg = CreateUnoDialog(DialogLibraries.Standard.Dialog_Tabellenbereinigung)

```

Anschließend werden Einstellungen für die Felder, die der Dialog enthält, ausgeführt. Hier als Beispiel das Auswahllistenfeld, das mit dem Ergebnis der obigen Abfrage bestückt wird:

```

oCtlList1 = oDlg.GetControl("ListBox1")
oCtlList1.addItem(aInhalt(),0)

```

Der Zugriff auf die Felder des Dialogs erfolgt über **GetControl** sowie die entsprechende Bezeichnung. Bei Dialogen ist es nicht möglich, für zwei Felder die gleichen Bezeichnungen zu verwenden, da sonst eine Auswertung des Dialoges problematisch wäre.

Das Listefeld wird mit den Inhalten aus der Abfrage, die in dem Array «aInhalt()» gespeichert wurden, ausgestattet. Das Listefeld enthält nur die darzustellenden Inhalte als ein Feld, wird also nur in der Position '0' bestückt.

Nachdem alle Felder mit den gewünschten Inhalten versorgt wurden, wird der Dialog gestartet.

```

Select Case oDlg.Execute()
Case 1 'Case 1 bedeutet die Betätigung des Buttons "OK"
Case 0 'Wenn Button "Abbrechen"
    inWiederholung = 0
End Select
LOOP WHILE inWiederholung = 1

```

Der Dialog wird so lange durchgeführt, wie der Wert für «inWiederholung» auf 1 steht. Diese Setzung erfolgt mit dem entsprechenden Markierfeld.

Hier der Inhalt nach Betätigung des Buttons «OK» im Kurzüberblick:

```

Case 1
    stInhalt1 = oCtlList1.getSelecteditem() 'Wert aus Listbox1 auslesen ...
    REM ... und den dazugehoerigen ID-Wert bestimmen.

```

Der ID-Wert des ersten Listefeldes wird in der Variablen «inLB1» gespeichert.

```

stText = oCtlText.Text ' Den Wert des Feldes auslesen.

```

Ist das Textfeld nicht leer, so wird nur der Eintrag im Textfeld erledigt. Weder das Listefeld für eine andere Zuweisung noch das Markierfeld für eine Löschung aller Daten ohne Bezug werden berücksichtigt. Dies wird auch dadurch verdeutlicht, dass bei Texteingabe die anderen Felder inaktiv geschaltet werden.

```

IF stText <> "" THEN

```

Ist das Textfeld nicht leer, dann wird der neue Wert anstelle des alten Wertes mit Hilfe des vorher ausgelesenen ID-Feldes in die Tabelle geschrieben. Dabei werden wieder zwei Einträge ermöglicht, wie dies auch in dem Listefeld geschieht. Das Trennzeichen ist «>». Bei Zwei Einträgen in verschiedenen Tabellen müssen auch entsprechend zwei UPDATE-Kommandos gestartet werden, die hier gleichzeitig erstellt und, durch ein Semikolon getrennt, weitergeleitet werden.

```
ELSEIF oCtlList2.getSelectedItemAt() <> "" THEN
```

Wenn das Textfeld leer ist und das Listenfeld 2 einen Wert aufweist, muss der Wert des Listenfeldes 1 durch den Wert des Listenfeldes 2 ersetzt werden. Das bedeutet, dass alle Datensätze der Tabellen, in denen die Datensätze der Listenfelder Fremdschlüssel sind, überprüft und gegebenenfalls mit einem geänderten Fremdschlüssel beschrieben werden müssen.

```
stInhalt2 = oCtlList2.getSelectedItemAt()
REM Den Wert der Listbox auslesen.
REM ID für den Wert das Listenfeld ermitteln.
```

Der ID-Wert des zweiten Listenfeldes wird in der Variablen «inLB2» gespeichert. Auch dieses erfolgt wieder unterschiedlich, je nachdem, ob ein oder zwei Felder in dem Listenfeld enthalten sind sowie eine oder zwei Tabellen Ursprungstabellen des Listenfeldinhaltes sind.

Der Ersetzungsprozess erfolgt danach, welche Tabelle als die Tabelle definiert wurde, die für die Haupttabelle den Fremdschlüssel darstellt. Für das oben erwähnte Beispiel ist dies die Tabelle "Postleitzahl", da die "Postleitzahl_ID" der Fremdschlüssel ist, der durch Listenfeld 1 und Listenfeld 2 wiedergegeben wird.

```
IF stNebentabelle = aTabelle(5) THEN
  FOR i = LBound(aFremdTabelle()) TO UBound(aFremdTabelle())
```

Ersetzen des alten ID-Wertes durch den neuen ID-Wert. Problematisch ist dies bei n:m-Beziehungen, da dann der gleiche Wert doppelt zugeordnet werden kann. Dies kann erwünscht sein, muss aber vermieden werden, wenn der Fremdschlüssel hier Teil des Primärschlüssels ist. So darf in der Tabelle "rel_Medien_Verfasser" ein Medium nicht zweimal den gleichen Verfasser haben, da der Primärschlüssel aus der "Medien_ID" und der "Verfasser_ID" gebildet wird. In der Abfrage werden alle Schlüsselfelder untersucht, die zusammen die Eigenschaft UNIQUE haben oder als Fremdschlüssel mit der Eigenschaft 'UNIQUE' über einen Index definiert wurden.

Sollte also der Fremdschlüssel die Eigenschaft 'UNIQUE' haben und bereits mit der gewünschten zukünftigen «inLB2» dort vertreten sein, so kann der Schlüssel nicht ersetzt werden.

```
stSql = "SELECT ""COLUMN_NAME"" FROM ""INFORMATION_SCHEMA"". ""SYSTEM_INDEXINFO""
WHERE ""TABLE_NAME"" = ' ' + aFremdTabelle(i,0) + ' ' AND ""NON_UNIQUE"" = False
AND ""INDEX_NAME"" = (SELECT ""INDEX_NAME"" FROM
""INFORMATION_SCHEMA"". ""SYSTEM_INDEXINFO"" WHERE ""TABLE_NAME"" = ' '
+ aFremdTabelle(i,0) + ' ' AND ""COLUMN_NAME"" = ' ' + aFremdTabelle(i,1) + ' ')"
```

Mit **"NON_UNIQUE" = False** werden die Spaltennamen angegeben, die 'UNIQUE' sind. Allerdings werden nicht alle Spaltennamen benötigt, sondern nur die, die gemeinsam mit dem Fremdschlüsselfeld einen Index bilden. Dies ermittelt der 'Subselect' mit dem gleichen Tabellennamen (der den Fremdschlüssel enthält) und dem Namen des Fremdschlüsselfeldes.

Wenn jetzt der Fremdschlüssel in der Ergebnismenge vorhanden ist, dann darf der Schlüsselwert nur dann ersetzt werden, wenn gleichzeitig andere Felder dazu benutzt werden, den entsprechenden Index als 'UNIQUE' zu definieren. Hierzu muss beim Ersetzen darauf geachtet werden, dass die Einzigartigkeit der Indexkombination nicht verletzt wird.

```
IF aFremdTabelle(i,1) = stFeldbezeichnung THEN
  inUnique = 1
ELSE
  ReDim Preserve aSpalten(inZaehler)
  aSpalten(inZaehler) = oAbfrageergebnis.getString(1)
  inZaehler = inZaehler + 1
END IF
```

Alle Spaltennamen, die neben dem bereits bekannten Spaltennamen des Fremdschlüsselfeldes als Index mit der Eigenschaft UNIQUE auftauchen, werden in einem Array abgespeichert. Da der Spaltenname des Fremdschlüsselfeldes auch zu der Gruppe gehört, wird durch ihn gekennzeichnet, dass die Einzigartigkeit bei der Datenänderung zu berücksichtigen ist.

```
IF inUnique = 1 THEN
  stSql = "UPDATE "" + aFremdTabelle(i,0) + "" AS ""a"" SET ""
  + aFremdTabelle(i,1) + ""=' ' + inLB2 + ' ' WHERE "" + aFremdTabelle(i,1)
```



```

+ """"=' + inLB1 + '' AND ( SELECT COUNT(*) FROM """" + aFremdTabelle(i,0)
+ """" WHERE """" + aFremdTabelle(i,1) + """"=' + inLB2 + '' )"
IF inZaehler > 0 THEN
    stFeldgruppe = Join(aSpalten(), """"|| ||""")

```

Gibt es mehrere Felder, die neben dem Fremdschlüsselfeld gemeinsam einen 'UNIQUE'-Index bilden, so werden die hier für eine SQL-Gruppierung zusammengeführt. Ansonsten erscheint als «stFeldgruppe» nur «aSpalten(0)».

```

stFeldbezeichnung = ""
FOR ink = LBound(aSpalten()) TO UBound(aSpalten())
    stFeldbezeichnung = stFeldbezeichnung + " AND """" + aSpalten(ink)
    + """" = ""a""."" + aSpalten(ink) + """" "

```

Die SQL-Teilstücke werden für eine korrelierte Unterabfrage zusammengefügt.

```

NEXT
stSql = Left(stSql, Len(stSql) - 1)

```

Die vorher erstellte Abfrage endet mit einer Klammer. Jetzt sollen noch Inhalte zu der Unterabfrage hinzugefügt werden. Also muss die Schließung wieder aufgehoben werden. Anschließend wird die Abfrage durch die zusätzlich ermittelten Bedingungen ergänzt.

```

stSql = stSql + stFeldbezeichnung + "GROUP BY ("""" + stFeldgruppe + """" ) < 1"
END IF

```

Wenn die Feldbezeichnung des Fremdschlüssels nichts mit dem Primärschlüssel oder einem 'UNIQUE'-Index zu tun hat, dann kann ohne weiteres auch ein Inhalt doppelt erscheinen

```

ELSE
    stSql = "UPDATE """" + aFremdTabelle(i,0) + """" SET """" + aFremdTabelle(i,1)
    + """"=' + inLB2 + '' WHERE """" + aFremdTabelle(i,1) + """"=' + inLB1 + ''"
END IF
oSQL_Anweisung.executeQuery(stSql)
NEXT

```

Das Update wird so lange durchgeführt, wie unterschiedliche Verbindungen zu anderen Tabellen vorkommen, d. h. die aktuelle Tabelle einen Fremdschlüssel in anderen Tabellen liegen hat. Dies ist z. B. bei der Tabelle "Ort" zweimal der Fall: in der Tabelle "Medien" und in der Tabelle "Postleitzahl".

Anschließend kann der alte Wert aus dem Listenfeld 1 gelöscht werden, weil er keine Verbindung mehr zu anderen Tabellen hat.

```

stSql = "DELETE FROM """" + aTabelle(5) + """" WHERE ""ID""=' + inLB1 + ''"
oSQL_Anweisung.executeQuery(stSql)

```

Das gleiche Verfahren muss jetzt auch für eine eventuelle zweite Tabelle durchgeführt werden, aus der die Listenfelder gespeist werden. In unserem Beispiel ist die erste Tabelle die Tabelle "Postleitzahl", die zweite Tabelle die Tabelle "Ort".

Wenn das Textfeld leer ist und das Listenfeld 2 ebenfalls nichts enthält, wird nachgesehen, ob eventuell das Markierfeld darauf hindeutet, dass alle überflüssigen Einträge zu löschen sind. Dies ist für die Einträge der Fall, die nicht mit anderen Tabellen über einen Fremdschlüssel verbunden sind.

```

ELSEIF oCtlCheck1.State = 1 THEN
    stBedingung = ""
    IF stNebentabelle = aTabelle(5) THEN
        FOR i = LBound(aFremdTabelle()) TO UBound(aFremdTabelle())
            stBedingung = stBedingung + ""ID"" NOT IN (SELECT """"
            + aFremdTabelle(i,1) + """" FROM """" + aFremdTabelle(i,0) + """" ) AND "
        NEXT
    ELSE
        FOR i = LBound(aFremdTabelle2()) TO UBound(aFremdTabelle2())
            stBedingung = stBedingung + ""ID"" NOT IN (SELECT """" +
            aFremdTabelle2(i,1) + """" FROM """" + aFremdTabelle2(i,0) + """" ) AND "
        NEXT
    END IF

```

Das letzte «AND» muss abgeschnitten werden, da sonst die Löschanweisung mit einem «AND» enden würde.

```
stBedingung = Left(stBedingung, Len(stBedingung) - 4) '
stSql = "DELETE FROM "" + stNebentabelle + "" WHERE " + stBedingung + ""
oSQL_Anweisung.executeQuery(stSql)
```

Da nun schon einmal die Tabelle bereinigt wurde, kann auch gleich der Tabellenindex überprüft und gegebenenfalls nach unten korrigiert werden. Siehe hierzu die in dem vorhergehenden Kapitel *Tabellenindex heruntersetzen bei Autowert-Feldern* erwähnte Prozedur.

```
Tabellenindex_runter(stNebentabelle)
```

Anschließend wird noch gegebenenfalls das Listenfeld des Formulars, aus dem der Tabellenbereinigungsdialog aufgerufen wurde, auf den neuesten Stand gebracht. Unter Umständen ist das gesamte Formular neu einzulesen. Hierzu wurde zu Beginn der Prozedur der aktuelle Datensatz ermittelt, so dass nach einem Auffrischen des Formulars der aktuelle Datensatz auch wieder eingestellt werden kann.

```
oDlg.EndExecute() 'Dialog beenden ...
oDlg.Dispose() '... und aus dem Speicher entfernen
END SUB
```

Dialoge werden mit **EndExecute()** beendet und mit **Dispose()** komplett aus dem Speicher entfernt.

Makrozugriff mit Access2Base

In LibreOffice ist seit der Version 4.2 die Erweiterung Access2Base integriert. Der Zugriff auf diese Bibliothek erfolgt über

```
Sub DBOpen(Optional oEvent As Object)
    If GlobalScope.BasicLibraries.HasByName("Access2Base") Then
        GlobalScope.BasicLibraries.LoadLibrary("Access2Base")
    End If
    Call Application.OpenConnection(ThisDatabaseDocument)
End Sub
```

Eine englischsprachige Beschreibung mit Beispielen ist auf der Seite <http://www.access2base.com/access2base.html> zu finden.

Die Bibliothek stellt nicht zusätzliche Funktionen zur Verfügung, sondern versucht, dem Anwender den Zugriff auf die Möglichkeiten der LibreOffice-API zu vereinfachen. Eine kurze Beschreibung ist auch in der Hilfe zu LO zu finden.

Wartung

Allgemeines zur Wartung von Datenbanken

Werden in einer Datenbank die Datenbestände häufig geändert, vor allem auch gelöscht, so macht sich das an zwei Stellen besonders bemerkbar. Zum einen wird die Datenbank immer größer, obwohl sie ja eigentlich gar nicht mehr Daten enthält. Zum anderen wird der automatisch erstellte Primärschlüssel einfach weiter hoch geschrieben ohne Rücksicht auf die tatsächlich erforderliche nächste Schlüsselzahl.

Datenbank komprimieren

Die **HSQLDB** hat die Eigenart, auch für bereits gelöschte Daten weiterhin Speicherplatz bereitzustellen. Datenbanken, die zum Test einmal mit Daten, vor allem auch Bildern gefüllt waren, weisen auch dann noch die gleiche Größe auf, wenn all diese Daten gelöscht wurden.

Um den Speicherplatz wieder frei zu geben, müssen die Datenbankdateien neu geschrieben werden (Tabellen, Beschreibungen zu diesen Tabellen usw.).

Direkt auf der Oberfläche von Base kann über **Extras → SQL** ein einfaches Kommando direkt eingegeben werden, dass bei Serverdatenbanken nur dem Systemadministrator vorbehalten ist:

```
SHUTDOWN COMPACT
```

Die Datenbank wird heruntergefahren und dabei von allen Altlasten befreit. Anschließend muss allerdings Base neu gestartet werden, wenn wieder auf die Datenbank zugegriffen werden soll.

Seit der Version LO 3.6 wird die Komprimierung standardmäßig spätestens beim Schließen der Datenbank durchgeführt.

Autowerte neu einstellen

Eine Datenbank wird erstellt, alle möglichen Funktionen mit Beispieldaten ausprobiert, Korrekturen vorgenommen, bis alles klappt. Dann ist mittlerweile der Wert für manch einen Primärschlüssel über 100 gestiegen. Pech nur, wenn der Primärschlüssel, wie meistens üblich, als Autowert-Schlüssel angelegt wurde. Werden die Tabellen zum Normalbetrieb oder zur Weitergabe der Datenbank an andere Personen geleert, so zählt anschließend der Primärschlüssel trotzdem weiter und startet nicht neu ab 0.

Mit dem folgenden SQL-Kommando, wieder eingegeben über **Extras → SQL**, lässt sich der Startwert neu festlegen:

```
ALTER TABLE "Tabellenname" ALTER COLUMN "ID" RESTART WITH NeuerWert
```

Hier wird davon ausgegangen, dass das Primärschlüsselfeld die Bezeichnung "ID" hat und außerdem als Autowert definiert wird. Der neue Wert sollte der sein, der als nächster automatisch eingefügt wird. Existieren z.B. noch Einträge bis zum Wert 4, so ist als neuer Wert 5 (ohne einfache Anführungsstriche) anzugeben.

Datenbankeigenschaften abfragen

In einem gesonderten Bereich der **HSQLDB** sind alle Informationen zu den Tabellen der Datenbank in Tabellenform abgelagert. Dieser besondere Bereich ist über den Zusatz "INFORMATION_SCHEMA" zu erreichen.

Mit der folgenden Abfrage können Feldnamen, Feldtypen, Spaltengrößen und Standardwerte ermittelt werden, hier am Beispiel der Tabelle mit dem Namen 'Suchtabelle'.

```

SELECT "COLUMN_NAME",
       "TYPE_NAME",
       "COLUMN_SIZE",
       "COLUMN_DEF" AS "Default Value"
FROM "INFORMATION_SCHEMA"."SYSTEM_COLUMNS"
WHERE "TABLE_NAME" = 'Suchtabelle'
ORDER BY "ORDINAL_POSITION"

```

Im Anhang sind alle Spezialtabellen der HSQLDB aufgeführt. Informationen über den Inhalt der jeweiligen Tabelle sind am einfachsten über direkte Abfragen zu erreichen:

```

SELECT * FROM "INFORMATION_SCHEMA"."SYSTEM_PRIMARYKEYS"

```

Über das Sternchen werden alle verfügbaren Spalten der Tabelle angezeigt. Die obige Tabelle gibt dabei in der Hauptsache Auskunft über die Primärschlüssel der verschiedenen Tabellen.

Diese Informationen lassen sich besonders mittels Makros gut nutzen. So können statt detaillierter Informationen zu einer gerade erstellten Tabelle oder Datenbank gleich Prozeduren geschrieben werden, die diese Informationen direkt aus der Datenbank holen und somit universeller nutzbar sind. Die Beispieldatenbank zeigt dies unter anderem an der Ermittlung von Fremdschlüsseln bei einem entsprechenden Wartungsmodul.

Die **FIREBIRD**-Datenbank hat entsprechende Informationen, nur leider etwas mehr in ihren Systemdatenbanken verstreut:

```

SELECT "a".RDB$RELATION_NAME AS "Tables",
       "a".RDB$FIELD_NAME AS "Fields",
       "c".RDB$TYPE_NAME AS "Types",
       "a".RDB$FIELD_POSITION AS "Fieldposition",
       "a".RDB$NULL_FLAG AS "Nullflag",
       "b".RDB$FIELD_LENGTH AS "Fieldlength"
FROM RDB$RELATION_FIELDS AS "a", RDB$FIELDS AS "b", RDB$TYPES AS "c"
WHERE "a".RDB$FIELD_SOURCE = "b".RDB$FIELD_NAME
      AND "b".RDB$FIELD_TYPE = "c".RDB$TYPE
      AND "c".RDB$FIELD_NAME = 'RDB$FIELD_TYPE'
      AND "a".RDB$RELATION_NAME = 'Suchtabelle'
ORDER BY "Tables", "Fieldposition"

```

Weitere Informationen hierzu auch im Anhang.

Daten exportieren

Neben der Möglichkeit, Daten durch Öffnen der gepackten *.odb-Datei zu exportieren, existiert auch eine wesentlich einfachere Möglichkeit. Direkt auf der Oberfläche von Base kann über **Extras** → **SQL** ein einfaches Kommando in die **HSQLDB** direkt eingegeben werden, das bei Serverdatenbanken nur dem Systemadministrator vorbehalten ist:

```

SCRIPT 'Datenbank'

```

Dies erzeugt einen kompletten SQL-Auszug der Datenbank mit allen Definitionen der Tabellen, Beziehungen der Tabellen untereinander und Daten. Davon sind allerdings die Abfragen nicht berührt, da diese in der grafischen Benutzeroberfläche erstellt und nicht in der eigentlichen internen Datenbank gespeichert sind. Sehr wohl sind aber alle Ansichten (Views) enthalten.

Die Datei wird standardmäßig als normale Textdatei erzeugt. Sie kann allerdings vor allem für große Formate in binärem oder gepacktem Format ausgegeben werden. Nur ist dann der Transport zurück in LO etwas komplizierter.

Das Format der exportierten Datei lässt sich über

```

SET SCRIPTFORMAT {TEXT | BINARY | COMPRESSED};

```

ändern.

Die entsprechende Datei kann über **Extras → SQL** eingelesen werden und erzeugt so eine Datenbank mit den entsprechenden Daten. Dabei müssen für eine interne Datenbank allerdings die folgenden Zeilen entfernt werden:

```
CREATE SCHEMA PUBLIC AUTHORIZATION DBA
```

Das Schema existiert schon. Es wird also ein ungültiger Schema-Name bemängelt.

```
CREATE USER SA PASSWORD ""  
GRANT DBA TO SA  
SET WRITE_DELAY 60  
SET SCHEMA PUBLIC
```

Diese Eingaben haben etwas mit dem Benutzerprofil und anderen Standardeinstellungen zu tun. Die Einstellungen werden bei den internen Datenbanken von LO standardmäßig schon gesetzt. Diese Einträge befinden sich direkt vor den Inhalten, die in die Tabellen über "INSERT" eingefügt werden.

Tabellen auf unnötige Einträge überprüfen

Eine Datenbank besteht aus einer oder mehreren Haupttabellen, die Fremdschlüssel aus anderen Tabellen aufnehmen. In der Beispieldatenbank sind das besonders die Tabellen "Medien" und "Adresse". In der Tabelle "Adresse" wird der Primärschlüssel der Postleitzahl als Fremdschlüssel geführt. Zieht eine Person um, so wird die Adresse geändert. Dabei kann es vorkommen, dass zu der Postleitzahl anschließend überhaupt kein Fremdschlüssel "Postleitzahl_ID" mehr existiert. Die Postleitzahl könnte also gelöscht werden. Nur fällt im Normalbetrieb nicht auf, dass sie gar nicht mehr benötigt wird. Um so etwas zu unterbinden, gibt es verschiedene Methoden.

Einträge durch Beziehungsdefinition kontrollieren

Über die Definition von Beziehungen kann die Integrität der Daten abgesichert werden. Das heißt, dass verhindert wird, dass die Löschung oder Änderung von Schlüsseln zu Fehlermeldungen der Datenbank führt.

Das folgende Fenster steht in **Extras → Beziehungen** nach einem Rechtsklick auf die Verbindungslinie zwischen zwei Tabellen zur Verfügung.

Relationen

beteiligte Tabellen: Adresse, Strasse

beteiligte Felder:

Adresse	Strasse
Strasse_ID	ID

Update Optionen:

- ☐ Keine Aktion
- ☒ Kask. Update
- ☐ Null setzen
- ☐ Default setzen

Löschoptionen:

- ☐ Keine Aktion
- ☐ Kask. Löschen
- ☒ Null setzen
- ☐ Default setzen

OK Abbrechen Hilfe

Hier sind die Tabelle "Adresse" und "Strasse" betroffen. **Alle aufgeführten Aktionen gelten für die Tabelle "Adresse"**, die den Fremdschlüssel "Strasse_ID" enthält. Update-Optionen beziehen sich auf ein Update des Feldes "ID" aus der Tabelle "Strasse". Wenn also in dem Feld "Strasse"."ID" die Schlüsselnummer geändert wird, so bedeutet «Keine Aktion», dass sich die Datenbank gegen diese Änderung wehrt, wenn "Strasse"."ID" mit der entsprechenden Schlüsselnummer als Fremdschlüssel in der Tabelle "Adresse" verwandt wird.

Mit «Kask. Update» wird die Schlüsselnummer einfach mitgeführt. Wenn die Straße 'Burgring' in der Tabelle "Strasse" die "ID" '3' hat und damit auch in "Adresse"."Strasse_ID" verzeichnet ist, kann die "ID" gefahrlos auf z.B. '67' geändert werden – die "Adresse"."Strasse_ID" wird dabei automatisch auch auf '67' geändert.

Wird «Null setzen» gewählt, so erzeugt die Änderung der "ID" in "Adresse"."Strasse_ID" ein leeres Feld.

Entsprechend werden die Löschoptionen gehandhabt.

Für beide Optionen steht über die GUI die Möglichkeit «Default setzen» zur Zeit nicht zur Verfügung, da die GUI-Standard Einstellungen sich von denen der Datenbank unterscheiden. Siehe hierzu den Abschnitt «[Default setzen](#)» aus dem Kapitel Tabellen.

Die Beziehungsdefinition hilft also nur, die Beziehung selbst sauber zu halten, nicht aber unnötige Datensätze in der Tabelle zu entfernen, die ihren Primärschlüssel als Fremdschlüssel in der Beziehung zur Verfügung stellt. Es können also beliebig viele Straßen ohne Adresse existieren.

Einträge durch Formular und Unterformular bearbeiten

Vom Prinzip her kann der gesamte Zusammenhang zwischen Tabellen innerhalb von Formularen dargestellt werden. Am einfachsten ist dies natürlich, wenn eine Tabelle nur zu einer anderen Tabelle in Beziehung steht. So gibt z.B. in dem folgenden Beispiel der Verfasser seinen Primärschlüssel als Fremdschlüssel an die Tabelle "rel_Medien_Verfasser" weiter; "rel_Medien_Verfasser"

enthält außerdem einen Fremdschlüssel von "Medien", so dass die folgende Anordnung eine n:m-Beziehung mit drei Formularen aufzeigt. Jede wird durch eine Tabelle präsentiert.

Die erste Abbildung zeigt, dass der Titel 'I hear you knocking' dem Verfasser 'Dave Edmunds' zugeordnet wurde. 'Dave Edmunds' darf also nicht gelöscht werden – sonst fehlt eine Information zu dem Medium 'I hear you knocking'. Es kann aber durch das Listenfeld statt 'Dave Edmunds' ein anderer Datensatz ausgewählt werden.

Nachname	Vorname
Edmunds	Dave
Götze	Dr. Lutz
Hawking	Steven W.
Heller	Dr. Klaus
Hermann	Ursula

Verfasser	Verf_Sort
Edmunds, Dave	1

Titel
I hear you knocking

Datensatz 1 von 10

In dem Formular ist ein Filter eingebaut, so dass bei Betätigung des Filters zu erkennen ist, welche Kategorien denn überhaupt nicht in der Tabelle Medien benötigt werden. In dem gerade abgebildeten Fall sind fast alle Beispielverfasser in Benutzung. Es kann also nur der Datensatz 'Erich Kästner' gelöscht werden, ohne dass dies eine Konsequenz für einen anderen Datensatz in "Medien" hätte.

Nachname	Vorname
Kästner	Erich

Verfasser	Verf_Sort

Titel

Datensatz 1 von 1

Filter anwenden

Der Filter ist in diesem Fall fest vorgegeben. Er befindet sich in den Formular-Eigenschaften. Ein solcher Filter tritt direkt beim Start des Formulars automatisch in Kraft. Er kann ausgeschaltet und angewandt werden. Wurde er aber einmal gelöscht, so ist er nur dann wieder erreichbar, wenn das Formular komplett neu gestartet wurde. Das bedeutet, dass nicht nur die Daten zu aktualisieren sind, sondern das ganze Formulardokument erst geschlossen und dann wieder geöffnet werden muss.

Formular-Eigenschaften	
Allgemein Daten Ereignisse	
Art des Inhaltes.....	Tabelle
Inhalt.....	Verfasser
SQL-Befehl analysieren...	Ja
Filter.....	"ID" NOT IN (SELECT "Verfasser_ID" FROM "rel_Medien_Verfasser")
Sortierung.....	"Nachname" ASC, "Vorname" ASC
Daten hinzufügen.....	Ja
Daten ändern.....	Ja
Daten löschen.....	Ja
Nur Daten hinzufügen.....	Nein
Navigationsleiste.....	Nein
Zyklus.....	Standard

Verwaiste Einträge durch Abfrage ermitteln

Der obige Filter ist Teil einer Abfrage, nach der die verwaisten Einträge ermittelt werden können.

```
SELECT "Nachname", "Vorname" FROM "Verfasser" WHERE "ID" NOT IN
(SELECT "Verfasser_ID" FROM "rel_Medien_Verfasser")
```

Bezieht sich eine Tabelle mit Fremdschlüsseln auf mehrere andere Tabellen, so ist die Abfrage entsprechend zu erweitern. Dies trifft z.B. auf die Tabelle "Ort" zu, die sowohl in der Tabelle "Medien" als auch in der Tabelle "Postleitzahl" Fremdschlüssel liegen hat. Daten aus der Tabelle "Ort", die gelöscht werden, sollten also möglichst in keiner dieser Tabellen noch benötigt werden. Dies zeigt die folgende Abfrage auf:

```
SELECT "Ort" FROM "Ort" WHERE "ID" NOT IN (SELECT "Ort_ID" FROM
"Medien") AND "ID" NOT IN (SELECT "Ort_ID" FROM "Postleitzahl")
```

Verwaiste bzw. nicht benötigte Einträge können so durch die Markierung sämtlicher Einträge bei gesetztem Filter und Betätigung der rechten Maustaste über das Kontextmenü des Datensatzanzeigers gelöscht werden.

Datenbankgeschwindigkeit

Einfluss von Abfragen

Gerade Abfragen, die im vorherigen Abschnitt zur Filterung der Daten verwandt wurden, sollten allerdings im Hinblick auf die Geschwindigkeit einer Datenbank möglichst unterbleiben. Hier handelt es sich um Unterabfragen, die bei größeren Datenbanken eine entsprechend große Datenmenge für jeden einzelnen anzuzeigenden Datensatz zum Vergleich bereitstellen. Nur Vergleiche mit der Beziehung «IN» ermöglichen es überhaupt, einen Wert mit einer Menge von Werten zu vergleichen. In sofern kann die folgende Unterabfrage

```
... WHERE "ID" NOT IN (SELECT "Verfasser_ID" FROM "rel_Medien_Verfasser")
```

eine große Menge an vorhandenen Fremdschlüsseln der Tabelle "rel_Medien_Verfasser" ergeben, die erst mit dem Primärschlüssel der Tabelle "Verfasser" für jeden Datensatz der Tabelle "Verfasser"

ser" verglichen werden muss. So eine Abfrage sollte also nicht für den täglichen Gebrauch gedacht sein, sondern nur vorübergehend wie hier zum Beispiel der Wartung von Tabellen. Suchfunktionen sind anders aufzubauen, damit die Suche von Daten nicht endlos dauert und die Arbeit mit der Datenbank im Alltagsbetrieb verleidet.

Einfluss von Listenfeldern und Kombinationsfeldern

Je mehr Listenfelder in einem Formular eingebaut sind und je größer der Inhalt ist, den sie bereitstellen müssen, desto länger braucht das Formular zum Aufbau.

Je besser das Programm Base zuerst einmal die grafische Oberfläche bereitstellt und erst einmal Listenfelder nur teilweise einliest, desto weniger fällt eine entsprechende Last auf.

Listenfelder werden durch Abfragen erstellt, und diese Abfragen finden beim Formularstart für jedes Listenfeld statt.

Gleiche Abfragestrukturen für z.B. mehrere Listenfelder können besser in eine gemeinsame Tabellenansicht (*View*) ausgelagert werden, statt mehrmals hintereinander mit gleicher Syntax über in den Listenfeldern abgespeicherte SQL-Kommandos entsprechende Felder zu erstellen. Ansichten sind vor allem bei externen Datenbanksystemen vorzuziehen, da hier der Server deutlich schneller läuft als eine Abfrage, die erst von der GUI zusammengestellt und an den Server neu gestellt wird. Ansichten (*Views*) hält der Server als fertige Abfragen schließlich vorrätig.

Einfluss des verwendeten Datenbanksystems

Die interne HSQLDB ist auf eine gut funktionierende Zusammenarbeit von Base mit Java ausgelegt. Leider hat Base seit der LO-Version 3.5 mit Geschwindigkeitsproblemen gerade in dieser Kombination zu kämpfen. Dies macht sich vor allem bei großen Tabellen mit mehreren tausend Datensätzen bemerkbar. Diese Geschwindigkeitsprobleme hatten unterschiedliche Ursachen und sind erst mit der LO-Version 4.1 verschwunden.

Externe Datenbanken laufen hier deutlich schneller. Von der Geschwindigkeit sind die direkten Verbindungen zu MySQL oder PostgreSQL sowie die Verbindungen über ODBC nahezu gleichwertig. JDBC ist ebenfalls auf das Zusammenspiel mit Java angewiesen, funktioniert aber deutlich schneller als eine interne Verbindung zur HSQLDB.

Anhang

Barcode

Um die Barcode-Druckfunktion nutzen zu können, muss der Font «ean13.ttf» installiert sein. Dieser Font ist frei verfügbar.

EAN13-Barcodes können mittels «ean13.ttf» folgendermaßen erstellt werden:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Zahl	Großbuchstaben, A=0 B=1 usw.						*	Kleinbuchstaben, a=0 b=1 usw.						+

Siehe hierzu die Abfrage "Barcode_EAN13_ttf_Bericht" der Beispieldatenbank
«Medien_ohne_Makros»

Datentypen des Tabelleneditors

Ganzzahlen

Typ	Zusatz	HSQldb	Umfang	Speicherbedarf
Tiny Integer	TINYINT	TINYINT	$2^8 = 256$ - 128 bis + 127	1 Byte
Small Integer	SMALLINT	SMALLINT	$2^{16} = 65536$ - 32768 bis + 32767	2 Byte
Integer	INTEGER	INTEGER INT	$2^{32} = 4294967296$ - 2147483648 bis + 2147483647	4 Byte
BigInt	BIGINT	BIGINT	2^{64}	8 Byte

Fließkommazahlen

Typ	Zusatz	HSQldb	Umfang	Speicherbedarf
Dezimal	DECIMAL	DECIMAL	Unbegrenzt, durch GUI auf 50 Stellen, einstellbar, feste Nachkommastellen, exakte Genauigkeit	variabel
Zahl	NUMERIC	NUMERIC	Unbegrenzt, durch GUI auf 50 Stellen, einstellbar, feste Nachkommastellen, exakte Genauigkeit	variabel
Float	FLOAT	(Double wird stattdessen genutzt)		
Real	REAL	REAL		
Double	DOUBLE	DOUBLE [PRECISION] FLOAT	Einstellbar, nicht exakt, 15 Dezimalstellen maximal	8 Byte

Text				
Typ	Zusatz	HSQLDB	Umfang	Speicherbedarf
Text	VARCHAR	VARCHAR	einstellbar	variabel
Text	VARCHAR_IG NORECASE	VARCHAR_IG NORECASE	Einstellbar, Auswirkung auf Sortierung, ignoriert Unterschiede zwischen Groß- und Kleinschreibung	variabel
Text (fix)	CHAR	CHAR CHARACTER	Einstellbar, Rest zum tatsächlichen Text wird mit Leerzeichen aufgefüllt	fest
Memo	LONGVARCHAR	LONGVARCHAR	Nur dieses Format stellt auch Absätze sowie Zeilenumbrüche dar.	variabel

Zeit				
Typ	Zusatz	HSQLDB	Umfang	Speicherbedarf
Datum	DATE	DATE		4 Byte
Zeit	TIME	TIME		4 Byte
Datum/Zeit	TIMESTAMP	TIMESTAMP DATETIME	Einstellbar (0, 6 – 6 bedeutet mit Millisekunden)	8 Byte

Sonstige				
Typ	Zusatz	HSQLDB	Umfang	Speicherbedarf
Ja/Nein	BOOLEAN	BOOLEAN BIT		
Binärfeld (fix)	BINARY	BINARY	Wie Integer	fest
Binärfeld	VARBINARY	VARBINARY	Wie Integer	variabel
Bild	LONGVARBINARY	LONGVARBINARY	Wie Integer	variabel, für größere Bilder gedacht
OTHER	OTHER	OTHER OBJECT		

In den Tabellendefinitionen und bei der Änderung von Datentypen in Abfragen mit den Funktionen «**CONVERT**» oder «**CAST**» werden bei einigen Datentypen Informationen zur Anzahl an Zeichen (a), zur Genauigkeit (g, entspricht der Gesamtzahl an Ziffern) und zu den Dezimalstellen (d) erwartet: CHAR(a), VARCHAR(a), DOUBLE(g), NUMERIC(g,d), DECIMAL(g,d) und TIMESTAMP(g).

TIMESTAMP(g) kann nur die Werte '0' oder '6' annehmen. '0' bedeutet, dass keine Sekunden im Nachkommabereich (Zehntel, Hundertstel ...) gespeichert werden. Die Genauigkeit des Timestamps kann *nur direkt über den SQL-Befehl* eingegeben werden. Sollen also Zeitangaben im Sportbereich eingetragen werden, so ist TIMESTAMP(6) über **Extras → SQL** voreinzustellen.

Datentypen in StarBasic

Zahlen

Typ	Entspricht in HSQLDB	Startwert	Anmerkung	Speicherbedarf
Integer	SMALLINT	0	$2^{16} = - 32768$ bis $+ 32767$	2 Byte
Long	INTEGER	0	$2^{32} = - 2147483648$ bis $+ 2147483647$	4 Byte
Single		0.0	Dezimaltrenner: «.»	4 Byte
Double	DOUBLE	0.0	Dezimaltrenner: «.»	8 Byte
Currency	Ähnlich DECIMAL, NUMERIC	0.0000	Währung, 4 Dezimalstellen fest	8 Byte

Sonstige

Typ	Entspricht in HSQLDB	Startwert	Anmerkung	Speicherbedarf
Boolean	BOOLEAN	False	1 = „ja“, alles andere: „nein“	1 Byte
Date	TIMESTAMP	00:00:00	Datum und Zeit	8 Byte
String	VARCHAR	Leerer String	bis 65536 Zeichen	variabel
Object	OTHER	Null		variabel
Variant		Leer	Kann jeden (anderen) Datentyp annehmen	variabel

Vor allem bei Zahlenwerten besteht große Verwechslungsgefahr. In der Datenbank steht z.B. häufig im Primärschlüssel der Datentyp «INTEGER». Wird jetzt per Makro ausgelesen, so muss dort die aufnehmende Variable den Typ «Long» haben, da diese vom Umfang her mit «INTEGER» aus der Datenbank übereinstimmt. Der entsprechende Auslesebefehl heißt dann auch «getLong».

Eingebaute Funktionen und abgespeicherte Prozeduren

In der eingebauten HSQLDB bzw. in Firebird sind die folgenden Funktionen verfügbar. Ein paar Funktionen können leider nur dann genutzt werden, wenn in der Abfrage «SQL-Kommando direkt ausführen» gewählt wurde. Dies verhindert dann gleichzeitig, dass die Abfragen editierbar bleiben.

Funktionen, die mit der grafischen Benutzeroberfläche zusammenarbeiten, sind gekennzeichnet mit **GUI**. Funktionen, die nur über «SQL-Kommando direkt ausführen» ansprechbar sind, sind gekennzeichnet mit **GUI**.

Bei den Funktionen gibt es solche, die mit der HSQLDB laufen, nicht aber mit der eingebauten Version von Firebird - und umgekehrt. Ist der Datenbankname grün dargestellt, so ist die Funktion verfügbar und gibt ordnungsgemäße Werte aus. Ist der Datenbankname rot und durchgestrichen dargestellt, so ist die Funktion nicht verfügbar oder aber, was deutlich bedenklicher ist, verfügbar, nur eben mit falschen Ergebnissen. Das ist dann mit dem Zusatz **(Bug)** gekennzeichnet.

Numerisch

Da hier mit Fließkommazahlen gerechnet wird, empfiehlt es sich, gegebenenfalls auf die Einstellung der Felder bei einer Abfrage zu achten. Meist ist hier die Anzeige der Nachkommazahlen begrenzt, so dass eventuell überraschende Ergebnisse zustande kommen. Dann wird z.B. in Spalte1 0,00, in Spalte2 1000 angezeigt. In Spalte3 soll dann Spalte1 * Spalte2 stehen – dort steht plötzlich 1.

ABS(d) HSQLDB FIREBIRD	Gibt des absoluten Wert einer Zahl wieder, entfernt also ggf. das Minus-Vorzeichen. GUI
ACOS(d) HSQLDB FIREBIRD	Gibt den Arcuscosinus wieder. Bereich: [-1, 1]. GUI
ASIN(d) HSQLDB FIREBIRD	Gibt den Arcussinus wieder. Bereich: [-1, 1]. GUI
ATAN(d) HSQLDB FIREBIRD	Gibt den Arcustangens wieder. GUI
ATAN2(a,b) HSQLDB FIREBIRD	Gibt den Arcustangens über Koordinaten wieder. 'a' ist der Wert der x-Achse, 'b' der Wert der y-Achse GUI
BITAND(a,b) HSQLDB BIN_AND(a,b) FIREBIRD	Sowohl die binäre Schreibweise von 'a' als auch die binäre Schreibweise von 'b' müssen an der gleichen Stelle eine '1' stehen haben, damit die '1' in das Ergebnis übernommen wird. BITAND(3, 5) ergibt 1; 0011 AND 0101 = 0001 GUI
BITOR(a,b) HSQLDB BIN_OR(a,b) FIREBIRD	Entweder die binäre Schreibweise von 'a' oder die binäre Schreibweise von 'b' müssen an der gleichen Stelle eine '1' stehen haben, damit die '1' in das Ergebnis übernommen wird. BITOR(3, 5) ergibt 7; 0011 OR 0101 = 0111 GUI
BIN_SHL(a,b) HSQLDB FIREBIRD	$a \cdot 2^b$
BIN_SHR(a,b) HSQLDB FIREBIRD	$a / 2^b$
BITXOR(a,b) HSQLDB BIN_XOR(a,b) FIREBIRD	Entweder die binäre Schreibweise von 'a' oder die binäre Schreibweise von 'b', nicht aber beide, müssen an der gleichen Stelle eine '1' stehen haben, damit die '1' in das Ergebnis übernommen wird. BITXOR(3, 5) ergibt 6; 0011 XOR 0101 = 0110 GUI
CEIL(d) HSQLDB FIREBIRD CEILING(d) HSQLDB FIREBIRD	Gibt die kleinste Ganzzahl an, die nicht kleiner als d ist. GUI
COS(d) HSQLDB FIREBIRD	Gibt den Cosinus wieder. GUI
COSH(d) HSQLDB FIREBIRD	Gibt den Cosinus-Hyperbolicus wieder. GUI
COT(d) HSQLDB FIREBIRD	Gibt den Cotangens wieder. GUI

DEGREES(d) HSQLDB FIREBIRD	Gibt zu Bogenmaßen die Gradzahl wieder. GUI
EXP(d) HSQLDB FIREBIRD	Gibt e^d (e : (2.718...)) wieder. GUI
FLOOR(d) HSQLDB FIREBIRD	Gibt die größte Ganzzahl an, die nicht größer als d ist. GUI
LOG(d) HSQLDB FIREBIRD LN(d) HSQLDB FIREBIRD	Gibt den natürlichen Logarithmus zur Basis 'e' wieder. GUI
LOG(x,y) HSQLDB FIREBIRD	Gibt den Logarithmus zur Basis x wieder. LOG(2, 8) ergibt 3, weil $2^3 = 8$ GUI
LOG10(d) HSQLDB FIREBIRD	Gibt den Logarithmus zur Basis 10 wieder. GUI
MOD(a,b) HSQLDB FIREBIRD	Gibt den Rest als Ganzzahl wieder, der bei der Division von 2 Ganzzahlen entsteht. MOD(11, 3) ergibt 2, weil $3*3+2=11$ GUI
PI() HSQLDB FIREBIRD	Gibt π (3.1415...) wieder. GUI
POWER(a,b) HSQLDB FIREBIRD	ab , POWER(2, 3) = 8, weil $2^3 = 8$ GUI
RADIANS(d) HSQLDB FIREBIRD	Gibt zu den Gradzahlen das Bogenmaß wieder. GUI
RAND() HSQLDB FIREBIRD	Gibt eine Zufallszahl x größer oder gleich 0.0 und kleiner als 1.0 wieder. GUI
ROUND(a,b) HSQLDB FIREBIRD	Rundet a auf b Stellen nach dem Dezimalzeichen. GUI
ROUNDMAGIC(d) HSQLDB FIREBIRD	Löst Rundungsprobleme, die durch Fließkommazahlen entstehen. 3.11-3.1-0.01 ist eventuell nicht genau 0, wird aber als 0 in der GUI angezeigt. ROUNDMAGIC macht daraus einen tatsächlichen 0-Wert. GUI
SIGN(d) HSQLDB FIREBIRD	Gibt -1 wieder, wenn 'd' kleiner als 0 ist, 0 wenn 'd'==0 und 1 wenn 'd' größer als 0 ist. GUI
SIN(A) HSQLDB FIREBIRD	Gibt den Sinus eines Bogenmaßes wieder. GUI
SINH(d) HSQLDB FIREBIRD	Gibt den Sinus-Hyperbolicus wieder. GUI
SQRT(d) HSQLDB FIREBIRD	Gibt die Quadratwurzel wieder. GUI
TAN(A) HSQLDB FIREBIRD	Gibt den Tangens eines Bogenmaßes wieder. GUI

TANH(d) HSQLDB FIREBIRD	Gibt den Tangens-Hyperbolicus wieder. GUI
TRUNCATE(a,b) HSQLDB FIREBIRD TRUNC(a,b) HSQLDB FIREBIRD	Schneidet 'a' auf 'b' Zeichen nach dem Dezimalpunkt ab. TRUNCATE(2.37456,2) = 2.37 GUI
Text	
ASCII(s) HSQLDB ASCII_VAL(s) FIREBIRD	Gibt den ASCII-Code des ersten Buchstaben des Strings wieder. GUI
BIT_LENGTH(s) HSQLDB FIREBIRD	Gibt die Länge des Textes s in Bits wieder. HSQLDB: Jedes Zeichen wird als 16bit ausgegeben. Firebird: Normale Zeichen 8bit, Sonderzeichen 16bit. GUI
CHAR(c) HSQLDB ASCII_CHAR(c) FIREBIRD	Gibt den Buchstaben wieder, der zu dem ASCII-Code c gehört. Dabei geht es nicht nur um Buchstaben, sondern auch um Steuerzeichen. CHAR(13) erzeugt in einer Abfrage einen Zeilenumbruch, der in mehrzeiligen Feldern eines Formulars oder in Berichten sichtbar wird. GUI
CHAR_LENGTH(s) CHARACTER_LENGTH() HSQLDB FIREBIRD	Gibt die Länge des Textes in Buchstaben wieder. GUI
CHAR_TO_UUID (ascii_uuid) HSQLDB FIREBIRD (Bug)	Ein String mit 36 Zeichen und einem '-' an Position 9, 14, 19 und 24 und ansonsten gültigen Hexadezimalzeichen wird in einen String ohne '-' und in Großschreibung geändert. CHAR_TO_UUID('A0bF4E45-3029-2a44-D493-4998c9b439A3') soll ergeben: A0BF4E4530292A44D4934998C9B439A3 GUI
CONCAT(str1,str2) HSQLDB FIREBIRD	Verbindet str1 + str2 GUI
'str1' 'str2' 'str3' HSQLDB FIREBIRD	Verbindet str1 + str2 + str3, einfachere Alternative zu CONCAT GUI
'str1'+'str2'+'str3' HSQLDB FIREBIRD	Verbindet str1 + str2 + str3, einfachere Alternative zu CONCAT GUI
DIFFERENCE(s1,s2) HSQLDB FIREBIRD	Gibt den ?Klang?unterschied zwischen s1 und s2 wieder. Hier wird lediglich eine Ganzzahl ausgegeben. 0 bedeutet dabei gleichen Klang. So erscheint 'for' und 'four' mit 0 gleich, 'Kürzen' und 'Würzen' wird auf 1 gesetzt, 'Mund' und 'Mond' wieder auf 0 GUI
GEN_UUID HSQLDB FIREBIRD (Bug)	Soll einen einzigartigen String mit 16 Byte Länge erstellen. GUI
HASH(s) HSQLDB FIREBIRD	Erstellt einen Hash (Streuwert) für einen eingegebenen Text. Das Ergebnis ist eine BIGINT-Zahl. GUI

HEXTORAW(s) HSQLDB FIREBIRD	Übersetzt Hexadezimalcode in andere Zeichen. Der Hexadezimalcode muss dabei aus einer durch 4 teilbaren Anzahl an Zeichen bestehen. HEXTORAW('0041') = 'A' (siehe auch den Hexadezimalcode über Einfügen → Sonderzeichen im Writer) GUI
INSERT(s1,start,len,s2) HSQLDB FIREBIRD OVERLAY(s1 PLACING s2 FROM start [FOR len]) HSQLDB FIREBIRD	Gibt einen Text wieder, bei dem Teile ersetzt werden. Beginnend mit «start» wird über eine Länge «len» aus dem Text s1 Text ausgeschnitten und durch den Text s2 ersetzt. INSERT('Bundesbahn', 3, 4, 'mme1') macht aus 'Bundesbahn' 'Bummelbahn', wobei die Länge des eingefügten Textes auch ohne weiteres größer als die des ausgeschnittenen Textes sein darf. So ergibt INSERT('Bundesbahn', 3, 5, 's und B') oder OVERLAY('Bundesbahn' PLACING 's und B' FROM 3 FOR 5) 'Bus und Bahn'. GUI (HSQLDB) GUI(FIREBIRD)
LCASE(s) HSQLDB FIREBIRD	Wandelt den String in Kleinbuchstaben um. GUI
LEFT(s,count) HSQLDB FIREBIRD	Gibt die mit count angegebene Zeichenanzahl vom Beginn des Textes s wieder. GUI
LENGTH(s) HSQLDB FIREBIRD	Gibt die Länge eines Textes in Anzahl der Buchstaben wieder. GUI
LIST ([ALL DISTINCT] expression [, separator]) HSQLDB FIREBIRD	Fasst alle Einträge zu einem Feldinhalt zusammen, die in einem Feld (in Abhängigkeit von der Gruppierung anderer Felder) stehen. SELECT "Nachname", CAST(LIST("Vorname", ', ') AS VARCHAR (32000)) AS "Vornamen" FROM "Tabelle" GROUP BY "Nachname" Die Umformung in den VARCHAR-Datentyp ist bei Einführung von LO 5.3 noch notwendig. GUI
LOCATE(search,s[,start]) HSQLDB FIREBIRD	Gibt den ersten Treffer für den Begriff aus search in dem Text s wieder. Der Treffer wird numerisch angegeben: (1=left, 0=not found) Die Angabe eines Startes innerhalb des Textes ist optional. GUI
LOWER(s) HSQLDB FIREBIRD	Wie LCASE(s) GUI
LPAD(s,len[,pad]) HSQLDB FIREBIRD	Der String s wird mit den in pad angegebenen Zeichen bis zur Länge len von links an aufgefüllt. Ist pad nicht angegeben, so wird ein Leerzeichen genutzt. Ist s länger als len, so wird s auf len von rechts aus gekürzt. GUI
LTRIM(s) HSQLDB FIREBIRD	Entfernt führende Leerzeichen und nicht druckbare Zeichen von einem Text. GUI

OCTET_LENGTH(s)	Gibt die Länge eines Textes in Bytes an. Dies entspricht dem doppelten Wert der Zeichenanzahl. HSQLDB Dies entspricht dem UTF8-Wert der Zeichenanzahl. Sonderzeichen haben eine Länge von 2. FIREBIRD GUI
POSITION(s1 IN s2) HSQLDB FIREBIRD POSITION(s1, s2[, start]) HSQLDB FIREBIRD	Wenn der erste Text in dem zweiten enthalten ist wird die Position des ersten Textes wiedergeben, ansonsten 0. Bei der 2. Version kann auch der Startpunkt der Suche festgelegt werden. Dies könnte statt einer Suchmöglichkeit mit «LIKE» besonders bei umfangreichen Texten genutzt werden. GUI
RAWTOHEX(s) HSQLDB FIREBIRD	Verwandelt in die Hexadezimalschreibweise, Umkehr von HEXTO-RAW() GUI
REPEAT(s,count) HSQLDB FIREBIRD	Wiederholt den Text s count Mal GUI
REPLACE(s1,replace,s2) HSQLDB FIREBIRD	Ersetzt alle vorkommenden Textstücke mit dem Inhalt replace im Text s1 durch den Text s2 GUI
REVERSE(s) HSQLDB FIREBIRD	Schreibt den String verkehrt herum. GUI
RIGHT(s,count) HSQLDB FIREBIRD	Umgekehrt zu LEFT; gibt die mit count angegebene Zeichenzahl vom Textende aus wieder. GUI
RPAD(s,len[,pad]) HSQLDB FIREBIRD	Der String s wird mit den in pad angegebenen Zeichen bis zur Länge len von rechts an aufgefüllt. Ist pad nicht angegeben, so wird ein Leerzeichen genutzt. Ist s länger als len, so wird s auf len von rechts aus gekürzt. GUI
RTRIM(s) HSQLDB FIREBIRD	Entfernt alle Leerzeichen und nicht druckbaren Zeichen am Textende. GUI
SOUNDEX(s) HSQLDB FIREBIRD	Gibt einen Code von 4 Zeichen wieder, die dem Klang von s entsprechen sollen – passt zu der Funktion DIFFERENCE() GUI
SPACE(count) HSQLDB FIREBIRD	Gibt die in count angegebene Zahl an Leertasten wieder. GUI
SUBSTR(s,start[,len]) HSQLDB FIREBIRD	Kürzel für SUBSTRING GUI
SUBSTRING(s,start[,len]) HSQLDB FIREBIRD SUBSTRING(s FROM start [FOR len]) HSQLDB FIREBIRD	Gibt den Text s ab der Startposition wieder. (1=links) . Wird die Länge ausgelassen, so wird der gesamte Text wiedergegeben. Auch bei einer größeren Länge als der Textlänge wird der restliche Text wiedergegeben. GUI Liefert den Teil eines Textes ab der in FROM angegebenen Startposition, optional in der in FOR angegebenen Länge. Steht im Feld "Name" z.B. 'Roberta', so ergibt SUBSTRING("Name" FROM 3 FOR 3) den Teilstring 'bert'. GUI

TRIM([LEADING TRAILING BOTH]) FROM s) HSQLDB FIREBIRD	Nicht druckbare Sonderzeichen und Leerzeichen werden entfernt. GUI
UCASE(s) HSQLDB FIREBIRD	Wandelt den String in Großbuchstaben um. GUI
UPPER(s) HSQLDB FIREBIRD	Wie UCASE(s) GUI
UUID_TO_CHAR HSQLDB FIREBIRD	Macht aus einer 16-Byte UUID eine 36 Zeichen lange UUID-Kette mit Bindestrichen. Sonderzeichen haben eine Länge von 2 Byte, so dass dafür die Zeichenzahl gekürzt werden müsste. UUID_TO_CHAR('LibreOffice Base') ergibt 4C696272-654F-6666-6963-652042617365 GUI
Datum/Zeit	
CURDATE() HSQLDB FIREBIRD	Gibt das aktuelle Datum wieder. GUI
CURRENT_DATE HSQLDB FIREBIRD	Synonym für CURDATE(), SQL-Standard GUI
CURTIME() HSQLDB FIREBIRD	Gibt die aktuelle Zeit wieder. GUI
CURRENT_TIME HSQLDB FIREBIRD	Synonym für CURTIME(), SQL-Standard GUI
CURRENT_TIMESTAMP HSQLDB FIREBIRD	Synonym für NOW(), SQL-Standard GUI
DATEADD(string, n, datetime) DATEADD(n string TO datetime) HSQLDB FIREBIRD	Addiert zu einer Datums bzw. Datumszeitangabe eine entsprechende Zeit, die über die Ganzzahl n und den string festgelegt wird. Der Eintrag in string entscheidet darüber, in welcher Einheit der Unterschied wiedergegeben wird: millisecond, second, minute, hour, day, week, month, year. Die string-Eingaben dürfen nicht mit einfachen Anführungszeichen maskiert sein. GUI Alternativ ist für Tagesangaben in FIREBIRD möglich: "Datum" + "Zeit" Timestamp GUI "Datum" + 1 Datum, Integerzahl wird als Tag gewertet GUI "Datum" - 1 Datum, Integerzahl wird als Tag gewertet GUI "Zeit" + 1 Zeit, Integerzahl wird als Sekunde gewertet GUI "Zeit" - 1 Zeit, Integerzahl wird als Sekunde gewertet GUI "Zeitstempel" + 2.75 Zeitstempel, Ganzzahl wird als Datum, Nachkommastellen als Bruchteil des Tages gewertet, hier also 2 Tage und 18 Stunden GUI "Zeitstempel" - 2.75 Zeitstempel GUI Nur als Differenz, nicht als Summe funktionieren: "Datum1" - "Datum2" Differenz in Tagen GUI "Zeit1" - "Zeit2" Differenz in Sekunden GUI "Zeitstempel1" - "Zeitstempel2" Differenz in Tagen und Bruchteilen von Tagen GUI

DATEDIFF(string, datetime1, datetime2) HSQldb FIREBIRD DATEDIFF(string FROM datetime1 TO datetime2) HSQldb FIREBIRD	Datumsunterschied zwischen zwei Datums- bzw. Datumszeitangaben. HSQldb: Der Eintrag in string entscheidet darüber, in welcher Einheit der Unterschied wiedergegeben wird: 'ms'='millisecond', 'ss'='second', 'mi'='minute', 'hh'='hour', 'dd'='day', 'mm'='month', 'yy' = 'year'. Sowohl die Langfassung als auch die Kurzfassung ist für den einzusetzenden string möglich. GUI Firebird: Der Eintrag in string entscheidet darüber, in welcher Einheit der Unterschied wiedergegeben wird: millisecond, second, minute, hour, day, week, month, year. Die string-Eingaben dürfen nicht mit einfachen Anführungszeichen maskiert sein. GUI
DAY(date) HSQldb FIREBIRD	Gibt den Tag im Monat wieder. (1-31) GUI
DAYNAME(date) HSQldb FIREBIRD	Gibt den englischen Namen des Tages wieder. GUI
DAYOFMONTH(date) HSQldb FIREBIRD	Gibt den Tag im Monat wieder. (1-31), Synonym für DAY() GUI
DAYOFWEEK(date) HSQldb FIREBIRD	Gibt den Wochentag als Zahl wieder. (1 bedeutet Sonntag) GUI
DAYOFYEAR(date) HSQldb FIREBIRD	Gibt den Tag im Jahr wieder. (1-366) GUI
HOURL(time) HSQldb FIREBIRD	Gibt die Stunde wieder. (0-23) GUI
MINUTE(time) HSQldb FIREBIRD	Gibt die Minute wieder. (0-59) GUI
MONTH(date) HSQldb FIREBIRD	Gibt den Monat wieder. (1-12) GUI
MONTHNAME(date) HSQldb FIREBIRD	Gibt den englischen Namen des Monats wieder. GUI
NOW() HSQldb FIREBIRD DATE 'NOW', TIME 'NOW', TIMESTAMP 'NOW' HSQldb FIREBIRD	Gibt das aktuelle Datum und die aktuelle Zeit zusammen als Zeitstempel wieder. Stattdessen kann auch CURRENT_TIMESTAMP genutzt werden. GUI Kurzform TIMESTAMP 'NOW' usw.: GUI Langform CAST('NOW' AS TIMESTAMP) usw.: GUI
QUARTER(date) HSQldb FIREBIRD	Gibt das Quartal im Jahr wieder. (1-4) GUI
SECOND(time) HSQldb FIREBIRD	Gibt die Sekunden einer Zeitangabe wieder. (0-59) GUI
WEEK(date) HSQldb FIREBIRD	Gibt die Woche des Jahres wieder. (1-53) GUI
YEAR(date) HSQldb FIREBIRD	Gibt das Jahr aus einer Datumseingabe wieder. GUI

DATE 'TODAY' HSQLDB FIREBIRD	Gibt das aktuelle Datum an, Langform: CAST('TODAY' AS DATE) GUI (Langform) GUI (Kurzform)
DATE 'YESTERDAY' HSQLDB FIREBIRD	Gibt das Datum von Gestern an, Langform: CAST('YESTERDAY' AS DATE) GUI (Langform) GUI (Kurzform)
DATE 'TOMORROW' HSQLDB FIREBIRD	Gibt das Datum von Morgen an, Langform: CAST('TOMORROW' AS DATE) GUI (Langform) GUI (Kurzform)
TO_CHAR(datetime, string) HSQLDB FIREBIRD	Setzt eine Datums- bzw. Timestampeingabe in einen entsprechenden String um. Folgende Zeichen sind definiert: 'Y' bis 'YYYY' (Jahr, 1 bis 4 Stellen) 'IY' bis 'IYYY' (Jahr nach ISO, 2 bis 4 Stellen) 'MM' (Monat, zweistellig) 'MON' (Monatsname, Kurzfassung, berücksichtigt GUI-Sprache) 'MONTH' (Monatsname, Langfassung) 'w' (Woche des Jahres) 'W' (Woche des Monats) 'IW' (Woche des Jahres nach ISO-Standard) 'd' (Tag der Woche) 'D' (Tagesname, Kurzfassung, berücksichtigt GUI-Sprache) 'DD' oder 'dd' (Tag des Monats, zweistellig) 'H' (Stunde von 0-23) 'HH' (Stunde von 0-11) 'm' (Minute) 's' (Sekunde) 'a' (AM oder PM für Stunde von 0-11) viele andere Zeichen werden direkt übernommen: TO_CHAR(CURRENT_DATE, 'D, DD.MON.YYYY') = Sa, 05.Nov.2016 Die Funktion läuft nur mit bereits eingegebenen Datumswerten, die aus der Datenbank gelesen werden. GUI
EXTRACT(string FROM datetime) HSQLDB FIREBIRD	Kann viele der Datums- und Zeitfunktionen ersetzen. Gibt das Jahr, den Monat, den Tag usw. von einem Datums- bzw. Datumszeitwert wieder. EXTRACT(DAY FROM "Datum") gibt den Tag im Monat wieder. HSQLDB: YEAR, MONTH, DAY, HOUR, MINUTE, SECOND Firebird zusätzlich: WEEK (GUI) FIREBIRD (Bug): WEEKDAY, YEARDAY, MILLISECOND. GUI
Datenbankverbindung	
DATABASE() HSQLDB FIREBIRD	Gibt den kompletten Pfad und Namen der Datenbank, die zu dieser Verbindung gehört, wieder. GUI
USER() HSQLDB FIREBIRD USER HSQLDB FIREBIRD	Gibt den Benutzernamen dieser Verbindung wieder. Der Nutzernamen ist dann von Bedeutung, wenn die Datenbank in eine externe Datenbank umgewandelt werden soll. Standard in HSQLDB: SA Standard in Firebird: SYSDBA GUI (HSQLDB) GUI (FIREBIRD)

CURRENT_USER HSQLDB FIREBIRD	SQL Standardfunktion, Synonym für USER(). Zu beachten ist, dass hier keine Klammern zu setzen sind. GUI
CURRENT_CONNECTION HSQLDB FIREBIRD	Gibt einen INTEGER-Wert für die aktuelle Datenbankverbindung wieder. Funktioniert nicht bei der Abfrage von Tabellen, sondern nur bei direkter Abfrage an die Datenbank selbst: SELECT CURRENT_CONNECTION FROM RDB\$DATABASE Mit RDB\$GET_CONTEXT() ist die Abfrage auch innerhalb einer Abfrage möglich, die sich nur auf eine Tabelle bezieht. GUI
CURRENT_ROLE HSQLDB FIREBIRD	Gibt in der Regel NONE für die aktuelle Datenbankverbindung wieder. Funktioniert nicht bei der Abfrage von Tabellen, sondern nur bei direkter Abfrage an die Datenbank selbst. Mit RDB\$GET_CONTEXT() ist die Abfrage auch innerhalb einer Abfrage möglich, die sich nur auf eine Tabelle bezieht. GUI
CURRENT_TRANSACTION HSQLDB FIREBIRD	Gibt die Transaktionsnummer für die aktuelle Datenbankverbindung wieder. Funktioniert nicht bei der Abfrage von Tabellen, sondern nur bei direkter Abfrage an die Datenbank selbst. Mit RDB\$GET_CONTEXT() ist die Abfrage auch innerhalb einer Abfrage möglich, die sich nur auf eine Tabelle bezieht. GUI
IDENTITY() HSQLDB FIREBIRD	Gibt den letzten Wert für ein Autowertfeld wieder, das in der aktuellen Verbindung erzeugt wurde. Dies wird bei der Makroprogrammierung genutzt, um aus einem erstellten Primärschlüssel für eine Tabelle einen Fremdschlüssel für eine andere Tabelle zu erstellen. GUI
RDB\$GET_CONTEXT('name', 'varname') HSQLDB FIREBIRD	Als 'name' können angegeben werden: SYSTEM, USER_SESSION, USER_TRANSACTION. USER_SESSION und USER_TRANSACTION sind zum Start leer und können mit beliebigen Variablen belegt werden. SYSTEM hat folgende vorbelegten 'varname': DB_NAME Pfad zu der ausgepackten Firebird-Datenbank NETWORK_PROTOCOL 'TCPv4', 'WNET', 'XNET' oder NULL CLIENT_ADDRESS abhängig von PROTOCOL, intern NULL CURRENT_USER wie CURRENT_USER direkt CURRENT_ROLE wie CURRENT_ROLE direkt SESSION_ID wie CURRENT_CONNECTION direkt TRANSACTION_ID wie CURRENT_TRANSACTION direkt ISOLATION_LEVEL 'READ COMMITTED', 'SNAPSHOT' oder 'CONSISTENCY'. ENGINE_VERSION Firebird-Server, LO 5.3: '3.0.0' GUI

RDB\$SET_CONTEXT ('name', 'varname', wert NULL) HSQLDB FIREBIRD	Die ist das Gegenstück zu RDB\$GET_CONTEXT(). Als 'name' können aber nur USER_SESSION und USER_TRANSACTION angegeben werden. SYSTEM kann nicht beschrieben werden. RDB\$SET_CONTEXT('USER_SESSION', 'Autor', 'Robert') schreibt die Variable über eine Abfrage nach USER_SESSION. In der aktuellen Zeile der Abfrage ist der Wert der Variablen noch NULL oder eben der vorhergehende Wert. mit RDB\$GET_CONTEXT('USER_SESSION', 'Autor') ist in den folgenden Zeilen der Abfrage der Wert 'Robert' verfügbar. GUI
System	
IFNULL(exp,value) HSQLDB FIREBIRD	Wenn exp NULL ist, wird value zurückgegeben, sonst exp. Stattdessen kann als Erweiterung auch COALESCE() genutzt werden. Exp und value müssen den gleichen Datentyp haben. IFNULL ist eine wichtige Funktion, wenn Felder durch Rechnung oder CONCAT miteinander verbunden werden. Der Inhalt des Ergebnisses wäre NULL, wenn auch nur ein Wert NULL ist. "Nachname" ', ' "Vorname" würde für Personen, bei denen z.B. der Eintrag für "Vorname" fehlt, ein leeres Feld, also NULL ergeben. "Nachname" IFNULL(', ' "Vorname", ' ') würde stattdessen auch nur "Nachname" ausgeben. GUI
CASEWHEN(exp,v1,v2) HSQLDB FIREBIRD IIF(exp,v1,v2) HSQLDB FIREBIRD	Wenn exp wahr ist wird v1 zurückgegeben, sonst v2. Stattdessen kann auch CASE WHEN genutzt werden. CASEWHEN("a">10, 'Ziel erreicht', 'noch üben') gibt 'Ziel erreicht' aus, wenn der Inhalt des Feldes "a" größer als 10 ist. GUI
CONVERT(term,type) HSQLDB FIREBIRD CAST(term AS type) HSQLDB FIREBIRD	Wandelt term in einen anderen Datentyp um. CONVERT("a", DECIMAL(5,2)) macht aus dem Feld "a" ein Feld mit 5 Ziffern, davon 2 Nachkommastellen. Ist die Zahl zu groß, so wird ein Fehler ausgegeben. GUI
COALESCE(expr1,expr2,expr3,...) HSQLDB FIREBIRD	Wenn expr1 nicht NULL ist, wird expr1 wiedergegeben, ansonsten wird expr2 überprüft, danach dann expr3 usw. Sämtliche Ausdrücke müssen zumindest einen ähnlichen Datentyp haben. So geht die alternative Darstellung von Ganzzahlen und Fließkommazahlen, aber nicht auch noch des eines Datums- oder Zeitwertes. COALESCE("Spitzname", "Vorname", 'Herrn/Frau') wählt den Spitznamen, wenn dieser vorhanden ist, ansonsten den Vornamen und wenn dieser auch nicht bekannt ist, dann allgemein 'Herrn/Frau'. GUI
NULLIF(v1,v2) HSQLDB FIREBIRD	Wenn v1 gleich v2 ist, wird NULL wiedergegeben, ansonsten v1. Die Daten müssen vom Typ her vergleichbar sein. GUI

CASE v1 WHEN v2 THEN v3 [ELSE v4] END HSQLDB FIREBIRD DECODE(v1, v2, v3, v4) HSQLDB FIREBIRD	Wenn v1 gleich v2 ist, wird v3 wiedergegeben. Sonst wird v4 wieder- gegeben oder NULL, wenn kein ELSE formuliert ist. GUI (HSQLDB) GUI (FIREBIRD)
CASE WHEN expr1 THEN v1[WHEN expr2 THEN v2] [ELSE v4] END HSQLDB FIREBIRD	Wenn expr1 wahr ist wird v1 zurückgegeben. [Optional können wei- tere Fälle angegeben werden] Sonst wird v4 wiedergegeben oder NULL, wenn kein ELSE formuliert ist. CASE WHEN DAYOFWEEK("Datum")=1 THEN 'Sonntag' WHEN DAYOFWEEK("Datum")=2 THEN 'Montag' ... END könnte per SQL den Tagesnamen ausgeben, der sonst in der Funk- tion nur in Englisch verfügbar ist. GUI
MAXVALUE(expr [, expr ...]) HSQLDB FIREBIRD	Sucht den höchsten Wert im Vergleich zu verschiedenen Ausdrücken heraus. Ergibt NULL, wenn ein Wert leer ist. Im Gegensatz zur Aggre- gatfunktion MAX() können hier die Werte verschiedener Felder mit- einander verglichen werden. GUI
MINVALUE(expr [, expr ...]) HSQLDB FIREBIRD	Sucht den niedrigsten Wert im Vergleich zu verschiedenen Ausdrü- cken heraus. Ergibt NULL, wenn ein Wert leer ist. Im Gegensatz zur Aggregatfunktion MIN() können hier die Werte verschiedener Felder miteinander verglichen werden. GUI

Steuerzeichen zur Nutzung in Abfragen

In Abfragen lassen sich Felder miteinander verknüpfen. Aus zwei Feldern in

```
SELECT "Vorname", "Nachname" FROM "Tabelle"
```

wird durch

```
SELECT "Vorname" || ' ' || "Nachname" FROM "Tabelle"
```

ein Feld. Hier wird noch ein Leerzeichen mit eingefügt. Natürlich lassen sich hier alle möglichen beliebigen Zeichen einfügen. Solange diese in ' ' stehen werden sie als Text interpretiert. Manchmal ist es aber auch sinnvoll, Zeilenumbrüche z.B. für einen Bericht einzufügen. Deshalb hier eine kleine Liste von Steuerzeichen, die entsprechend durch einen Blick auf <http://de.wikipedia.org/wiki/Steuerzeichen> erweitert werden kann.

CHAR(9)	Horizontaler Tabulator	
CHAR(10)	Zeilenvorschub	Erzeugt in Serienbriefen und im Report-Builder in einem Feld einen Zeilenumbruch (Linux, Unix, Mac) Um den Zeilenumbruch auch in Abfragen anzuzeigen, muss das Feld in ein LONGVARCHAR-Feld umgewandelt werden.
CHAR(13)	Wagenrücklauf	Zeilenumbruch zusammen mit dem Zeilenvorschub in Windows. CHAR(13) CHAR(10) ist auch für Linux, Mac usw. möglich, daher die universellere Variante.

Einige uno-Befehle zur Nutzung mit einer Schaltfläche

Einer Schaltfläche können verschiedene uno-Befehle direkt zugeordnet werden. Dafür muss unter **Eigenschaften: Schaltfläche → Aktion → Dokument/Webseite öffnen** gewählt werden sowie z. B. als **URL → .uno:RecSearch** zum Öffnen der Suchfunktion eingetragen werden. Manchmal muss zusätzlich beachtet werden, dass **Fokussieren bei Klick → Nein** angewählt wird, wenn bestimmte Aktionen direkt auf ein Formularfeld zugreifen sollen, das dafür den Focus braucht, wie z. B. **.uno:Paste**, das den Inhalt aus der Zwischenablage einfügen kann.

Die folgende Liste gibt nur wenige Befehle wieder. Sämtliche Befehle aus der Navigationsleiste sind ja bereits in der Schaltfläche so verfügbar, könnten aber auch über die uno-Befehle erstellt werden. Viele Befehle lassen sich auch über den Makrorecorder ermitteln, der häufig diese uno-Befehle über einen Dispatcher aufruft.

Uno-Befehl	Anwendung für ...
.uno:RecSearch	Öffnen der Suchfunktion im Formular
.uno:Paste	Zwischenablage einfügen, nur mit Fokussieren bei Klick → Nein
.uno:Copy	Kopiert den markierten Inhalt in die Zwischenablage, nur mit Fokussieren bei Klick → Nein
.uno:Print	Druckdialog für das Formular öffnen
.uno:PrintDefault	Drucken mit dem Defaultprinter ohne Dialog
.uno:NextRecord	Nächster Datensatz
.uno:PrevRecord	Vorhergehender Datensatz
.uno:FirstRecord	Erster Datensatz
.uno:LastRecord	Letzter Datensatz
.uno:NewRecord	Neuer Datensatz
.uno:RecSave	Datensatz speichern
.uno>DeleteRecord	Lösche den Datensatz
.uno:RecUndo	Nehme die letzten Eingaben zurück.
.uno:OpenURL	Öffne die angezeigte URL
.uno:Refresh	Angezeigte Daten aktualisieren
.uno:ViewFormAsGrid	Formular: Datenquelle als Tabelle aufrufen

Informationstabellen der HSQLDB

Innerhalb von Datenbanken wird in dem Bereich *"INFORMATION_SCHEMA"* die Information über alle Tabelleneigenschaften sowie ihre Verbindung untereinander abgelegt. Diese Informationen ermöglichen in Base bei der Erstellung von Makros, Prozeduren mit weniger Parametern zu starten. Eine Anwendung findet sich in der Beispieldatenbank unter anderem im Modul *«Wartung»* in der Prozedur *«Tabellenbereinigung»* für die Ansteuerung des Dialoges.

In einer Abfrage können die einzelnen Informationen sowie sämtliche dazugehörigen Felder auf die folgende Art ermittelt werden.

```
SELECT * FROM "INFORMATION_SCHEMA"."SYSTEM_ALIASES"
```


Im Gegensatz zu einer normalen Tabelle ist es hier notwendig, dem jeweiligen folgenden Begriff "INFORMATION_SCHEMA" voranzustellen.

```
SYSTEM_ALIASES
SYSTEM_ALLTYPEINFO
SYSTEM_BESTROWIDENTIFIER
SYSTEM_CACHEINFO
SYSTEM_CATALOGS
SYSTEM_CHECK_COLUMN_USAGE
SYSTEM_CHECK_CONSTRAINTS
SYSTEM_CHECK_ROUTINE_USAGE
SYSTEM_CHECK_TABLE_USAGE
SYSTEM_CLASSPRIVILEGES
SYSTEM_COLUMNPRIVILEGES
SYSTEM_COLUMNS
SYSTEM_CROSSREFERENCE
SYSTEM_INDEXINFO
SYSTEM_PRIMARYKEYS
SYSTEM_PROCEDURECOLUMNS
SYSTEM_PROCEDURES
SYSTEM_PROPERTIES
SYSTEM_SCHEMAS
SYSTEM_SEQUENCES
SYSTEM_SESSIONINFO
SYSTEM_SESSIONS
SYSTEM_SUPERTABLES
SYSTEM_SUPERTYPES
SYSTEM_TABLEPRIVILEGES
SYSTEM_TABLES
SYSTEM_TABLETYPES
SYSTEM_TABLE_CONSTRAINTS
SYSTEM_TEXTTABLES
SYSTEM_TRIGGERCOLUMNS
SYSTEM_TRIGGERS
SYSTEM_TYPEINFO
SYSTEM_UDTATTRIBUTES
SYSTEM_UDTS
SYSTEM_USAGE_PRIVILEGES
SYSTEM_USERS
SYSTEM_VERSIONCOLUMNS
SYSTEM_VIEWS
SYSTEM_VIEW_COLUMN_USAGE
SYSTEM_VIEW_ROUTINE_USAGE
SYSTEM_VIEW_TABLE_USAGE
```

Die folgende Abfrage gibt z.B. eine komplette Übersicht über alle in der Datenbank genutzten Tabellen mit Feldtypen, Primärschlüsseln und Fremdschlüsseln:

```
SELECT
  "A"."TABLE_NAME",
  "A"."COLUMN_NAME",
  "A"."TYPE_NAME",
  "A"."NULLABLE",
  "B"."KEY_SEQ" AS "PRIMARYKEY",
  "C"."PKTABLE_NAME" || '.' || "C"."PKCOLUMN_NAME" AS "FOREIGNKEY FOR"
FROM "INFORMATION_SCHEMA"."SYSTEM_COLUMNS" AS "A"
  LEFT JOIN "INFORMATION_SCHEMA"."SYSTEM_PRIMARYKEYS" AS "B"
```

```

ON ( "B"."TABLE_NAME" = "A"."TABLE_NAME" AND "B"."COLUMN_NAME" =
"A"."COLUMN_NAME" )
LEFT JOIN "INFORMATION_SCHEMA"."SYSTEM_CROSSREFERENCE" AS "C"
ON ( "C"."FKTABLE_NAME" = "A"."TABLE_NAME" AND "C"."FKCOLUMN_NAME"
= "A"."COLUMN_NAME" )
WHERE "A"."TABLE_SCHEM" = 'PUBLIC'

```

Informationstabellen der Firebird-Datenbank

Die Firebird-Informationstabellen sind deutlich mehr gesplittet als die der HSQLDB. So stehen z.B. Tabellennamen, Feldnamen und Feldtypen nicht zusammen in einer Tabelle, sondern müssen über mehrere Tabellen hinweg zusammen gebracht werden:

```

SELECT
"a".RDB$RELATION_NAME AS "Tables",
"a".RDB$FIELD_NAME AS "Fields",
"c".RDB$TYPE_NAME AS "Types",
"a".RDB$Field_POSITION AS "Fieldposition",
"a".RDB$NULL_FLAG AS "Nullflag"
FROM RDB$RELATION_FIELDS AS "a", RDB$FIELDS AS "b", RDB$TYPES AS "c"
WHERE "a".RDB$FIELD_SOURCE = "b".RDB$FIELD_NAME
AND "b".RDB$FIELD_TYPE = "c".RDB$TYPE
AND "c".RDB$FIELD_NAME = 'RDB$FIELD_TYPE'
AND "a".RDB$SYSTEM_FLAG = 0
ORDER BY "Tables", "Fieldposition"

```

Hiermit wird eine Übersicht über alle selbst erstellten Tabellen mit den Feldnamen, Feldtypen, Feldpositionen innerhalb der Tabelle und der Information, ob sie leer sein dürfen, ermöglicht. Um den Feldtypen zuordnen zu können muss also über die Tabelle "RDB\$FIELD_TYPE" zur Tabelle "RDB\$TYPE" verbunden werden.

Die Firebird-Systemtabellen starten alle mit den Anfangsbuchstaben RDB\$:

```

RDB$BACKUP_HISTORY
RDB$CHARACTER_SETS
RDB$CHECK_CONSTRAINTS
RDB$COLLATIONS
RDB$DATABASE
RDB$DEPENDENCIES
RDB$EXCEPTIONS
RDB$FIELDS
RDB$FIELD_DIMENSIONS
RDB$FILES
RDB$FILTERS
RDB$FORMATS
RDB$FUNCTIONS
RDB$FUNCTION_ARGUMENTS
RDB$GENERATORS
RDB$INDICES
RDB$INDEX_SEGMENTS
RDB$LOG_FILES
RDB$PAGES
RDB$PROCEDURES
RDB$PROCEDURE_PARAMETERS
RDB$REF_CONSTRAINTS
RDB$RELATIONS
RDB$RELATION_CONSTRAINTS

```

RDB\$RELATION_FIELDS
RDB\$ROLES
RDB\$SECURITY_CLASSES
RDB\$TRANSACTIONS
RDB\$TRIGGERS
RDB\$TRIGGER_MESSAGES
RDB\$TYPES
RDB\$USER_PRIVILEGES
RDB\$VIEW_RELATIONS

Datenbankreparatur für *.odb-Dateien

Regelmäßige Datensicherung sollte eigentlich Grundlage für den Umgang mit dem PC sein. Sicherheitskopien sind so der einfachste Weg, auf einen halbwegs aktuellen Datenstand zurückgreifen zu können. Doch in der Praxis mangelt es eben häufig an dieser Stelle.

Formulare, Abfragen und Berichte können, sofern eine Vorversion der Datenbank gesichert wurde, über die Zwischenablage in eine neue Datenbank kopiert werden. Lässt sich allerdings, aus welchen Gründen auch immer, eine aktuelle Datenbankdatei nicht mehr öffnen, so ist das Hauptproblem: Wie komme ich (hoffentlich) an die Daten.

Bei plötzlichen Abstürzen des PC kann es passieren, dass geöffnete Datenbanken von LO (interne Datenbank HSQLDB) nicht mehr zu öffnen sind. Stattdessen wird beim Versuch, die Datenbank zu öffnen, nach einem entsprechenden Filter für das Format gefragt.

Das Ganze liegt daran, dass Teile der Daten der geöffneten Datenbank im Arbeitsspeicher liegen und lediglich temporär zwischengespeichert werden. Oft wird erst beim Schließen der Datei die gesamte Datenbank in die Datei zurückgeschrieben und gepackt. Lediglich Eingaben, die direkt in die Tabellen erfolgen, werden auch direkt in die Datenbankdatei gesichert. Seit LO 5.1 kann außerdem die Datenbank noch über den Button **Speichern** gesichert werden. Dies war vorher nicht der Fall, da der Button nicht den Zugang zu einem Untermenü anbot und deshalb inaktiv war.

Wiederherstellung der Datenbank-Archivdatei

Um eventuell doch noch an die Daten zu kommen, kann das folgende Verfahren hilfreich sein:

1. Fertigen sie eine Kopie ihrer Datenbank für die weiteren Schritte an.
2. Versuchen Sie die Kopie mit einem Packprogramm zu öffnen. Es handelt sich bei der *.odb-Datei um ein gepacktes Format, ein Zip-Archiv. Lässt sich die Datei so nicht direkt öffnen, so funktioniert das Ganze vielleicht auch über die Umbenennung der Endung von *.odb zu *.zip.
Funktioniert das Öffnen nicht, so ist vermutlich von der Datenbank nichts mehr zu retten.
3. Folgende Verzeichnisse sehen Sie nach dem Öffnen einer Datenbankdatei im Packprogramm auf jeden Fall:



Abbildung 60: Aufbau einer Datenbankdatei ohne Tabellen, Abfragen, Formulare und Berichte

4. Die Datenbankdatei muss ausgepackt werden. Die entscheidenden Informationen für die Daten liegen im Unterverzeichnis «database» in den Dateien «data» und «script».
5. Gegebenenfalls empfiehlt es sich, die Datei «script» einmal anzuschauen und auf Ungeheimheiten zu überprüfen. Dieser Schritt kann aber auch erst einmal zum Testen übersprungen werden. Die «script»-Datei enthält vor allem die Beschreibung der Tabellenstruktur.
6. Gründen sie eine neue, leere Datenbankdatei und öffnen diese Datenbankdatei mit dem Packprogramm.
7. Ersetzen sie die Dateien «data» und «script» aus der neuen Datenbankdatei durch die unter «4.» entpackten Dateien.
8. Das Packprogramm muss nun geschlossen werden. War es, je nach Betriebssystem, notwendig, die Dateien vor dem Öffnen durch das Packprogramm nach *.zip umzubenennen, so ist das jetzt wieder nach *.odb zu wandeln.
9. Öffnen sie die Datenbankdatei jetzt mit LO. Sie können hoffentlich wieder auf ihre Tabellen zugreifen.
10. Wie weit sich jetzt auch Abfragen, Formulare und Berichte auf ähnliche Weise wiederherstellen lassen, bleibt dem weiteren Testen überlassen.

Siehe hierzu auch: <http://user.services.LO oder Ooo.org/en/forum/viewtopic.php?f=83&t=17125>

Weitere Informationen zur Datenbank-Archivdatei

Eine Datenbank-Archivdatei enthält im tatsächlichen Gebrauch neben dem grundlegenden Verzeichnis für die Datenbank und dem für das OpenDocument-Format vorgeschriebenen Verzeichnis «META-INF» noch weitere Verzeichnisse, um Formulare und Berichte abzuspeichern. Eine Beschreibung zum grundsätzlichen Aufbau des OpenDocument-Formates ist u.a. unter <http://de.wikipedia.org/wiki/OpenDocument> zu finden.

Die folgende Übersicht zeigt eine Datenbank, die Tabellen ein Formular und einen Bericht enthält. Nicht offen sichtbar ist hier, dass auch eine Abfrage zur Datenbank gehört. Solche Abfragen werden nicht in separaten Verzeichnissen gespeichert, sondern sind in der Datei «content.xml» enthalten. Die dafür notwendigen Informationen beschränken sich schließlich auf eine einfache SQL-Formulierung.

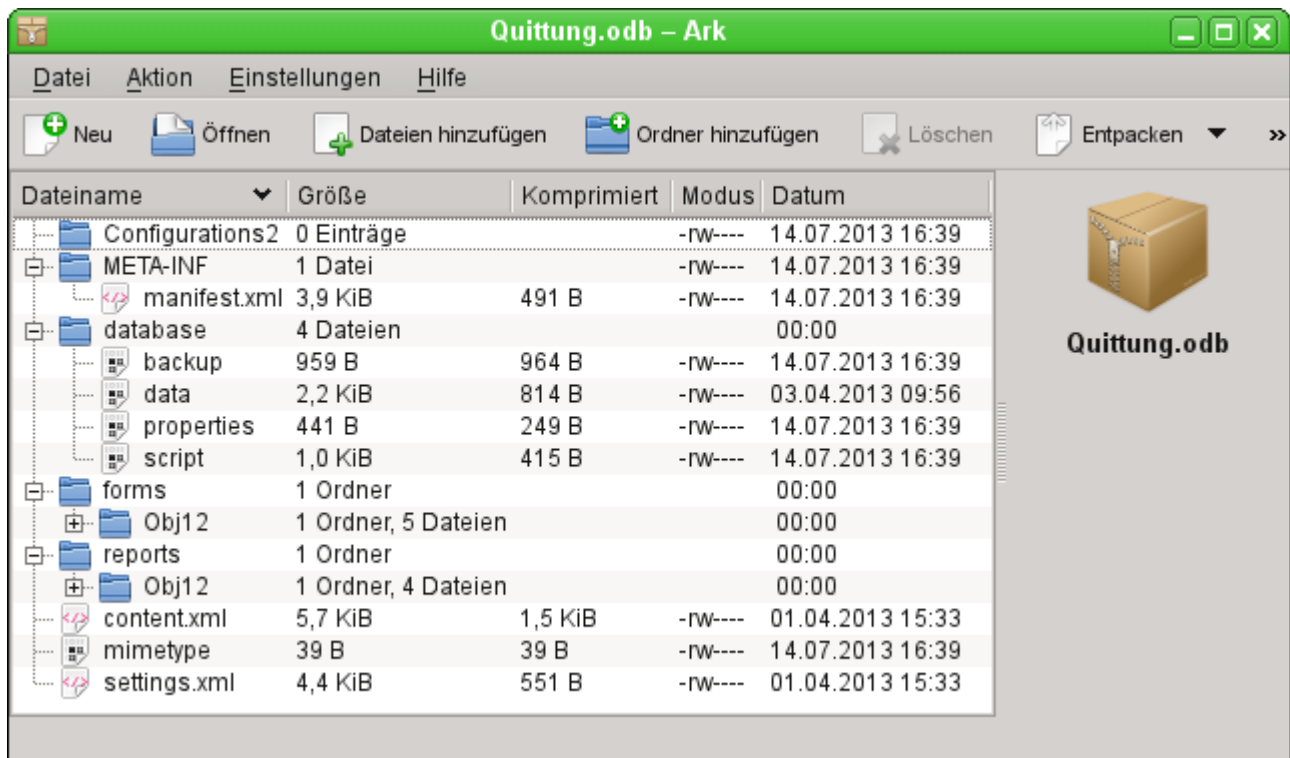


Abbildung 61: Datenbankdatei, die neben der Datenbank auch Informationen zu einem Formular und einem Bericht abgespeichert hat.

Hier einige der Dateien aus der Datenbank-Archivdatei im Überblick:

mimetype

application/vnd.oasis.opendocument.base

Diese kleine Textdatei enthält lediglich den Hinweis, dass es sich bei der Archivdatei um eine Datenbankdatei im OpenDocument-Format handelt.

content.xml einer Datenbank ohne Inhalt

```
<?xml version="1.0" encoding="UTF-8"?>
<office:document-content
  xmlns:office="urn:oasis:names:tc:opendocument:xmlns:office:1.0"
  xmlns:style="urn:oasis:names:tc:opendocument:xmlns:style:1.0"
  xmlns:text="urn:oasis:names:tc:opendocument:xmlns:text:1.0"
  xmlns:table="urn:oasis:names:tc:opendocument:xmlns:table:1.0"
  xmlns:draw="urn:oasis:names:tc:opendocument:xmlns:drawing:1.0"
  xmlns:fo="urn:oasis:names:tc:opendocument:xmlns:xsl-fo-compatible:1.0"
  xmlns:xlink="http://www.w3.org/1999/xlink"
  xmlns:dc="http://purl.org/dc/elements/1.1/"
  xmlns:meta="urn:oasis:names:tc:opendocument:xmlns:meta:1.0"
  xmlns:number="urn:oasis:names:tc:opendocument:xmlns:datatypes:1.0"
  xmlns:svg="urn:oasis:names:tc:opendocument:xmlns:svg-compatible:1.0"
  xmlns:chart="urn:oasis:names:tc:opendocument:xmlns:chart:1.0"
  xmlns:dr3d="urn:oasis:names:tc:opendocument:xmlns:dr3d:1.0"
  xmlns:math="http://www.w3.org/1998/Math/MathML"
  xmlns:form="urn:oasis:names:tc:opendocument:xmlns:form:1.0"
  xmlns:script="urn:oasis:names:tc:opendocument:xmlns:script:1.0"
  xmlns:ooo="http://openoffice.org/2004/office"
  xmlns:ooow="http://openoffice.org/2004/writer"
  xmlns:oooc="http://openoffice.org/2004/calc"
  xmlns:dom="http://www.w3.org/2001/xml-events"
  xmlns:db="urn:oasis:names:tc:opendocument:xmlns:database:1.0"
  xmlns:xforms="http://www.w3.org/2002/xforms"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:rpt="http://openoffice.org/2005/report">
```

```

xmlns:of="urn:oasis:names:tc:opendocument:xmlns:of:1.2"
xmlns:xhtml="http://www.w3.org/1999/xhtml"
xmlns:grddl="http://www.w3.org/2003/g/data-view#"
xmlns:tableooo="http://openoffice.org/2009/table"
xmlns:drawooo="http://openoffice.org/2010/draw"
xmlns:calcext="urn:org:documentfoundation:names:experimental:calc:xmlns:calcext:1.0"
xmlns:field="urn:openoffice:names:experimental:ooo-ms-interop:xmlns:field:1.0"
xmlns:formx="urn:openoffice:names:experimental:ooxml-odf-interop:xmlns:form:1.0"
xmlns:css3t="http://www.w3.org/TR/css3-text/"
office:version="1.2">
  <office:scripts/>
  <office:font-face-decls/>
  <office:automatic-styles/>
  <office:body>
    <office:database>
      <db:data-source>
        <db:connection-data>
          <db:connection-resource xlink:href="sdbc:embedded:hsqldb"/>
          <db:login db:is-password-required="false"/>
        </db:connection-data>
        <db:driver-settings>
          db:system-driver-settings=""
          db:base-dn=""
          db:parameter-name-substitution="false"/>
        <db:application-connection-settings>
          db:is-table-name-length-limited="false"
          db:append-table-alias-name="false"
          db:max-row-count="100">
            <db:table-filter>
              <db:table-include-filter>
                <db:table-filter-pattern>%</db:table-filter-pattern>
              </db:table-include-filter>
            </db:table-filter>
          </db:application-connection-settings>
        </db:data-source>
      </office:database>
    </office:body>
  </office:document-content>

```

Zu Beginn wird die xml-Version und der verwendete Zeichensatz geklärt. Der gesamte nachfolgende Inhalt wird ohne Absatz direkt in einer Zeile ausgegeben. In der oben erstellten Übersicht wird hoffentlich der Inhalt etwas klarer. Zusammengehörige Elemente werden in «*Tags*» gefasst.

Mit den Anfangsdefinitionen werden durch «xmlns» (XML-Namespace) die Namensräume umschrieben, auf die innerhalb dieser Datei zugegriffen werden kann. Anschließend werden etwas konkretere Angaben zum Inhalt gemacht. Hier ist dann z.B. ersichtlich, dass es sich um eine interne HSQLDB-Datenbank handelt und die Angabe eines Passwortes nicht erforderlich ist.

content.xml einer Datenbank mit Inhalt

Der folgende Inhalt ist nur ein Auszug der content.xml-Datei und soll nur die Struktur klären.

```

<office:scripts/>
<office:font-face-decls>
  <style:font-face style:name="F" svg:font-family=""/>
</office:font-face-decls>
<office:automatic-styles>
  <style:style>
    style:name="co1"
    style:family="table-column"
    style:data-style-name="N0"/>
  <style:style>
    style:name="co2"
    style:family="table-column"
    style:data-style-name="N107"/>
  <style:style style:name="ce1" style:family="table-cell">
    <style:paragraph-properties fo:text-align="start"/>
  </style:style>

```

```

<number:number-style style:name="N0" number:language="de" number:country="DE">
  <number:number number:min-integer-digits="1"/>
</number:number-style>
<number:currency-style
  style:name="N107P0"
  style:volatile="true"
  number:language="de"
  number:country="DE">
  <number:number
    number:decimal-places="2"
    number:min-integer-digits="1"
    number:grouping="true"/>
  <number:text> </number:text>
  <number:currency-symbol
    number:language="de"
    number:country="DE">€
  </number:currency-symbol>
</number:currency-style>

```

Hier wird ein Feld als Währungsfeld festgelegt. Die Anzahl der Dezimalstellen werden genannt, der Abstand zwischen Zahlen und Währungssymbol sowie das Währungssymbol selbst.

```

<number:currency-style
  style:name="N107"
  number:language="de"
  number:country="DE">
  <style:text-properties fo:color="#ff0000"/>
  <number:text>-</number:text>
  <number:number
    number:decimal-places="2"
    number:min-integer-digits="1"
    number:grouping="true"/>
  <number:text> </number:text>
  <number:currency-symbol
    number:language="de"
    number:country="DE">€
  </number:currency-symbol>
  <style:map style:condition="value()>=0" style:apply-style-name="N107P0"/>
</number:currency-style>

```

Im zweiten Abschnitt erfolgt die Festlegung, dass bis zu einem bestimmten Wert die Währung in der Farbe Rot («#ff0000») erscheinen soll.

```

</office:automatic-styles>
<office:body>
  <office:database>
    <db:data-source>

```

Dieser Eintrag entspricht mit allen Unterpunkten dem aus der oben beschriebenen content.xml einer Datenbank-Archivdatei ohne Inhalt.

```

    </db:data-source>
    <db:forms>
      <db:component
        db:name="Quittung"
        xlink:href="forms/Obj12"
        db:as-template="false"/>
    </db:forms>

```

Die Datenbank-Archivdatei enthält einen Unterordner, in dem die Details zu einem Formular abgespeichert sind. Das Formular ist auf der Benutzeroberfläche mit dem Namen «Quittung» verzeichnet.

```

    <db:reports>
      <db:component
        db:name="Quittung"
        xlink:href="reports/Obj12"
        db:as-template="false"/>
    </db:reports>

```


Die Datenbank-Archivdatei enthält einen Unterordner, in dem die Details zu einem Bericht abgespeichert sind. Der Bericht ist auf der Benutzeroberfläche ebenfalls mit dem Namen «Quittung» verzeichnet.

```
<db:queries>
  <db:query
    db:name="Verkauf_berechnet"
    db:command="SELECT &quot;a&quot;.*, ( SELECT &quot;Preis&quot; *
      &quot;a&quot;.&quot;Anzahl&quot; FROM &quot;Ware&quot; WHERE
      &quot;ID&quot; = &quot;a&quot;.&quot;Ware_ID&quot; ) AS
      &quot;Anzahl*Preis&quot; FROM &quot;Verkauf&quot; AS &quot;a&quot;"/>
  </db:queries>
```

Sämtliche Abfragen werden direkt in der content.xml gespeichert. «"» steht dabei für ein doppeltes Anführungszeichen oben «"». Die oben stehende Abfrage ist in diesem Beispiel eigentlich recht umfangreich und besteht aus vielen korrelierenden Unterabfragen. Sie ist hier nur verkürzt wiedergegeben.

```
<db:table-representations>
  <db:table-representation db:name="Quittung"/>
  <db:table-representation db:name="Verkauf"/>
  <db:table-representation db:name="Ware">
    <db:columns>
      <db:column
        db:name="ID"
        db:style-name="co1"
        db:default-cell-style-name="ce1"/>
      <db:column
        db:name="MWSt"
        db:style-name="co1"
        db:default-cell-style-name="ce1"/>
      <db:column
        db:name="Preis"
        db:help-message="Angabe als Nettopreis"
        db:style-name="co2"
        db:default-cell-style-name="ce1"/>
      <db:column
        db:name="Ware"
        db:style-name="co1"
        db:default-cell-style-name="ce1"/>
    </db:columns>
  </db:table-representation>
</db:table-representations>
```

Wie sollen verschiedene Tabellen von der Ansicht her erscheinen. An dieser Stelle wird das Erscheinungsbild bestimmter Spalten gespeichert; in diesem Beispiel wurden Einstellungen der Tabelle "Ware" mit ihren einzelnen Feldern "ID", "MWSt" usw. abgespeichert. Hier wurde zum einen beim Preis eine Zusatzinformation angegeben. Zum anderen ist eine Formatierung der Angabe vorgenommen worden. Der **style-name** 'co2' entspricht beim Preis dem **style-name**, der zu Beginn der content.xml definiert wurde und mit dem **data-style-name** 'N107' verbunden ist. Der **data-style-name** 'N107' wiederum verweist auf die Formatierung des Feldes als Währungsfeld.

```
</office:database>
</office:body>
```

Grundsätzlich ist in der content.xml der Inhalt der Abfragen und Informationen zum Erscheinungsbild der Tabellen direkt gespeichert. Außerdem ist eine Definition der Verbindung zur Datenbank enthalten. Schließlich kommen noch Verweise auf Formulare und Berichte hinzu.

settings.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<office:document-settings
  xmlns:office="urn:oasis:names:tc:opendocument:xmlns:office:1.0"
  xmlns:table="urn:oasis:names:tc:opendocument:xmlns:table:1.0"
  xmlns:xlink="http://www.w3.org/1999/xlink"
  xmlns:number="urn:oasis:names:tc:opendocument:xmlns:datastyle:1.0">
```



```

xmlns:svg="http://www.w3.org/2000/svg"
xmlns:config="urn:oasis:names:tc:opendocument:xmlns:config:1.0"
xmlns:ooo="http://openoffice.org/2004/office"
xmlns:db="urn:oasis:names:tc:opendocument:xmlns:database:1.0"
office:version="1.2"/>

```

Bei einer Datenbank ohne weiteren Inhalt stehen hier nur die Grunddefinitionen. Mit Inhalt werden hier aber die unterschiedlichsten Einstellungen abgespeichert. Nach dem Start mit der obigen Definition sind folgende Einstellungen in der Beispieldatei abgespeichert:

```

<office:settings>
  <config:config-item-set config:name="ooo:view-settings">
    <config:config-item-set config:name="Queries">
      <config:config-item-set config:name="Verkauf_berechnet">
        <config:config-item-set config:name="Tables">
          <config:config-item-set config:name="Table1">
            <config:config-item config:name="WindowName"
              config:type="string">Verkauf</config:config-item>
            <config:config-item config:name="WindowLeft"
              config:type="int">153</config:config-item>
            <config:config-item config:name="ShowAll"
              config:type="boolean">true</config:config-item>
            <config:config-item config:name="WindowTop"
              config:type="int">17</config:config-item>
            <config:config-item config:name="WindowWidth"
              config:type="int">120</config:config-item>
            <config:config-item config:name="WindowHeight"
              config:type="int">120</config:config-item>
            <config:config-item config:name="ComposedName"
              config:type="string">Verkauf</config:config-item>
            <config:config-item config:name="TableName"
              config:type="string">Verkauf</config:config-item>
          </config:config-item-set>
        </config:config-item-set>
      <config:config-item config:name="SplitterPosition"
        config:type="int">105</config:config-item>
      <config:config-item config:name="VisibleRows"
        config:type="int">1024</config:config-item>
    </config:config-item-set>
  </config:config-item-set>
</office:settings>
<config:config-item-set config:name="ooo:configuration-settings">
  <config:config-item-set config:name="layout-settings">
    <config:config-item-set config:name="Tables">
      <config:config-item-set config:name="Table1">
        <config:config-item config:name="WindowName"
          config:type="string">Verkauf</config:config-item>
        <config:config-item config:name="WindowLeft"
          config:type="int">186</config:config-item>
        <config:config-item config:name="ShowAll"
          config:type="boolean">false</config:config-item>
        <config:config-item config:name="WindowTop"
          config:type="int">17</config:config-item>
        <config:config-item config:name="WindowWidth"
          config:type="int">120</config:config-item>
        <config:config-item config:name="WindowHeight"
          config:type="int">120</config:config-item>
        <config:config-item config:name="ComposedName"
          config:type="string">Verkauf</config:config-item>
        <config:config-item config:name="TableName"
          config:type="string">Verkauf</config:config-item>
      </config:config-item-set>
      <config:config-item-set config:name="Table2">
        ... (identische config:type-Punkte wie "Table1"
        <config:config-item config:name="TableName"
          config:type="string">Ware</config:config-item>
      </config:config-item-set>
      <config:config-item-set config:name="Table3">
        ... (identische config:type-Punkte wie "Table1"

```

```

        <config:config-item config:name="TableName"
            config:type="string">Quittung</config:config-item>
    </config:config-item-set>
</config:config-item-set>
</config:config-item-set>
</config:config-item-set>
</office:settings>

```

Die gesamte Übersicht bezieht sich auf verschiedene Ansichten der Fenster für die (eine) Abfrage "Verkauf berechnet" und für die Tabellen "Verkauf", "Ware" und "Quittung". Die letzten beiden wurden hier nur verkürzt wiedergegeben. Würden diese Einstellungen bei einer defekten *.odb-Datei fehlen, so wäre das also nicht weiter von Bedeutung. Sie würden wieder erstellt, wenn die entsprechenden Fenster das nächste Mal geöffnet werden.

META-INF/manifest.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<manifest:manifest
  xmlns:manifest="urn:oasis:names:tc:opendocument:xmlns:manifest:1.0">
  <manifest:file-entry
    manifest:full-path="/"
    manifest:media-type="application/vnd.oasis.opendocument.base"/>
  <manifest:file-entry
    manifest:full-path="database/script"
    manifest:media-type=""/>
  <manifest:file-entry
    manifest:full-path="database/properties"
    manifest:media-type=""/>
  <manifest:file-entry
    manifest:full-path="settings.xml"
    manifest:media-type="text/xml"/>
  <manifest:file-entry
    manifest:full-path="content.xml"
    manifest:media-type="text/xml"/>
</manifest:manifest>

```

Bei dieser Datei im Unterverzeichnis META-INF handelt es sich um ein Inhaltsverzeichnis der gesamten Datenbank-Archivdatei. Da es sich bei der oben gezeigten Datei um die Datei handelt, die in der leeren Datenbank (Abbildung 60) enthalten ist, gibt es hier nur 5 Datei-Einträge («file-entry»). Bei der mit Formular und Bericht versehenen Datenbank-Archivdatei sind die Einträge in der META-INF entsprechend umfangreicher.

database/properties

```

#HSQL Database Engine 1.8.0.10
#Sun Jul 14 18:02:08 CEST 2013
hsqldb.script_format=0
runtime.gc_interval=0
sql.enforce_strict_size=true
hsqldb.cache_size_scale=8
readonly=false
hsqldb.nio_data_file=false
hsqldb.cache_scale=13
version=1.8.0
hsqldb.default_table_type=cached
hsqldb.cache_file_scale=1
hsqldb.lock_file=true
hsqldb.log_size=10
modified=no
hsqldb.cache_version=1.7.0
hsqldb.original_version=1.8.0
hsqldb.compatible_version=1.8.0

```

Die properties-Datei enthält die Grundeinstellungen für die interne HSQL Datenbank. Siehe dazu auch das folgende Kapitel.

database/script

```

SET DATABASE COLLATION "German"

```

```
CREATE SCHEMA PUBLIC AUTHORIZATION DBA
CREATE USER SA PASSWORD ""
GRANT DBA TO SA
SET WRITE_DELAY 60
```

In der script-Datei finden sich die Standardeinstellungen für die Verbindung zur Datenbank, zur benutzten Sprache usw. Hier erscheint auch der später erwähnte Benutzer «SA».

In einer mit Inhalt gefüllten Datenbank werden in dieser Datei die Grundlagen für die Tabellendefinitionen gespeichert:

```
SET DATABASE COLLATION "German"
CREATE SCHEMA PUBLIC AUTHORIZATION DBA
```

Die Tabellen werden definiert, bevor der Datenbanknutzer definiert wird. Zuerst werden die Tabellen mit ihren Feldern im Cache erstellt.

```
CREATE CACHED TABLE "Ware"
("ID" INTEGER GENERATED BY DEFAULT AS IDENTITY(START WITH 0) NOT NULL
PRIMARY KEY,"Ware" VARCHAR(50),"Preis" DECIMAL(8,2),"MWSt" TINYINT)
CREATE CACHED TABLE "Verkauf"
("ID" INTEGER GENERATED BY DEFAULT AS IDENTITY(START WITH 0) NOT NULL
PRIMARY KEY,"Anzahl" TINYINT,"Ware_ID" INTEGER,"Quittung_ID" INTEGER,
CONSTRAINT SYS_FK_59 FOREIGN KEY("Ware_ID") REFERENCES "Ware"("ID"))
CREATE CACHED TABLE "Quittung"
("ID" INTEGER GENERATED BY DEFAULT AS IDENTITY(START WITH 0) NOT NULL
PRIMARY KEY,"Datum" DATE)
```

Anschließend werden noch Änderungen an den Tabellen vorgenommen, damit die Beziehungen («REFERENCES») stimmig sind

```
ALTER TABLE "Verkauf" ADD CONSTRAINT SYS_FK_76 FOREIGN KEY("Quittung_ID")
REFERENCES "Quittung"("ID")
SET TABLE "Ware" INDEX'608 20'
SET TABLE "Verkauf" INDEX'1872 1656 1872 12'
SET TABLE "Quittung" INDEX'2232 1'
```

Nach der Einstellung der Position des Indexes in der data-Datei (erscheint nur hier in der script-Datei, wird nie direkt in SQL eingegeben!) werden die automatisch hoch schreibenden Felder der Tabellen («AutoWert») so eingestellt, dass sie die nächsten Werte bei Neueingaben erstellen. So ist z.B. der letzte eingetragene Wert im Feld "ID" der Tabelle "Ware" die Nummer 19. Das automatische Hochschreiben beginnt also mit der Nummer 20.

```
ALTER TABLE "Ware" ALTER COLUMN "ID" RESTART WITH 20
ALTER TABLE "Verkauf" ALTER COLUMN "ID" RESTART WITH 12
ALTER TABLE "Quittung" ALTER COLUMN "ID" RESTART WITH 1
CREATE USER SA PASSWORD ""
GRANT DBA TO SA
SET WRITE_DELAY 60
```

Behebung von Versionsproblemen

Wenn, wie auf den folgenden Seiten beschrieben, die externe HSQLDB verwendet wird, kann eventuell ein weiteres Problem mit den *.odb-Dateien in Verbindung mit manchen LO-Versionen auftauchen. Wird eine externe HSQLDB genutzt, so ist der sicherste Weg der über das hsqldb.jar-Archiv, das mit LO mitgeliefert wird. Wird ein anderes Archiv verwendet, so kann das dazu führen, dass die internen Datenbanken plötzlich nicht mehr zugänglich sind. Dies liegt daran, dass LO Schwierigkeiten hat, zwischen interner und externer HSQLDB zu unterscheiden und Meldungen von einem Versionskonflikt produziert.

Es muss als externe Datenbank die mitgelieferte hsqldb.jar-Datei genutzt werden. Außerdem muss aus der *.odb-Datei das database-Verzeichnis extrahiert werden. Die Datei properties hat hier einen Eintrag, der in LO 3.3 zu dieser Fehlermeldung führt:

```
version=1.8.1
```

steht in Zeile 11.

Diese Zeile ist zu ändern auf

```
version=1.8.0
```

Danach ist das database-Verzeichnis wieder in das *.odb-Päckchen einzulesen und die interne Datenbank lässt sich auch wieder unter LO öffnen.

Weitere Tipps

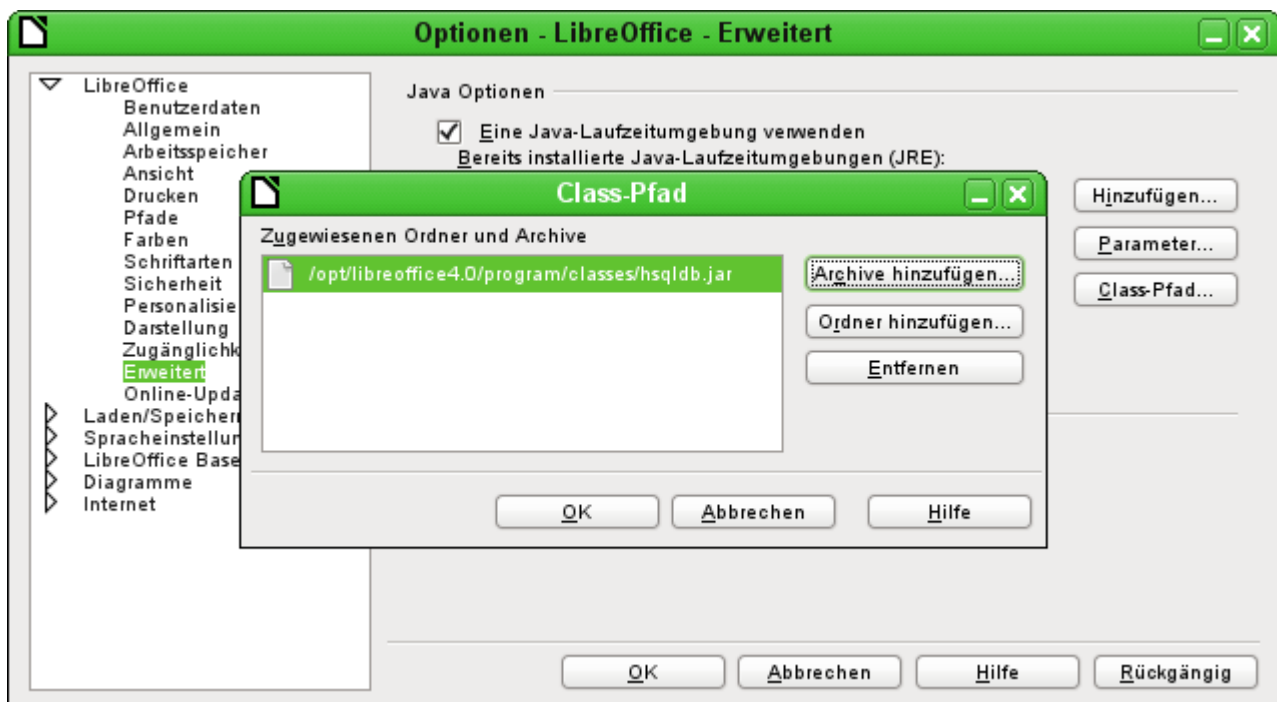
Wenn aus irgendwelchen Gründen wohl die Datenbankdatei geöffnet wird, aber kein Zugang mehr zu den Tabellen existiert, kann direkt über **Extras → SQL** der Befehl **SHUTDOWN SCRIPT** eingegeben werden. Anschließend wird die Datenbank geschlossen und neu gestartet. Das Ganze funktioniert aber nicht, wenn bereits ein «Error im Script-file» gemeldet wird.

Die Daten der Datenbank liegen in der *.odb-Datei im Unterverzeichnis «database». Hier gibt es eine Datei «data» und eine Datei «backup». Ist die Datei «data» defekt, so kann sie über die Datei «backup» wiederhergestellt werden. Hierzu muss die im Verzeichnis «database» liegende Datei «properties» bearbeitet werden. Hier gibt es eine Zeile «modified=no». Diese muss umgeschrieben werden zu «modified=yes». Das zeigt dem System an, dass die Datenbank nicht korrekt beendet wurde. Jetzt wird aus der komprimierten «backup»-Datei beim Neustart eine neue «data»-Datei erstellt.

Datenbankverbindung zu einer externen HSQLDB

Die interne HSQLDB unterscheidet sich erst einmal nicht von der externen Variante. Wenn, wie im Folgenden beschrieben, erst einmal nur der Zugriff auf die Datenbank nach außerhalb gelegt werden soll, dann ist keine Serverfunktion erforderlich. Hier reicht schlicht das Archiv, was in LO mitgeliefert wurde. Im LO-Pfad liegt unter /program/classes/hsqldb.jar. Die Verwendung dieses Archivs ist die sicherste Variante, da dann keine Versionsprobleme auftauchen.

Die externe HSQLDB steht unter <http://hsqldb.org/> zum Download frei zur Verfügung. Diese anderen Versionen sollten allerdings nur genutzt werden, wenn Klarheit darüber besteht, wie Versionsprobleme zwischen interner und externer Variante behoben werden können.



Der Datenbanktreiber muss, sofern er nicht in dem Pfad der Java-Runtime liegt, als ClassPath unter Extras – Optionen – Java hinzugefügt werden.

Die Verbindung zu der externen Datenbank erfolgt über JDBC. Die Datenbankdateien sollen in einem bestimmten Verzeichnis abgelegt werden. Dieses Verzeichnis kann beliebig gewählt werden. Es liegt in dem folgenden Beispiel im home-Ordner. Nicht angegeben ist hier der weitere Verzeichnisverlauf sowie der Name der Datenbank.

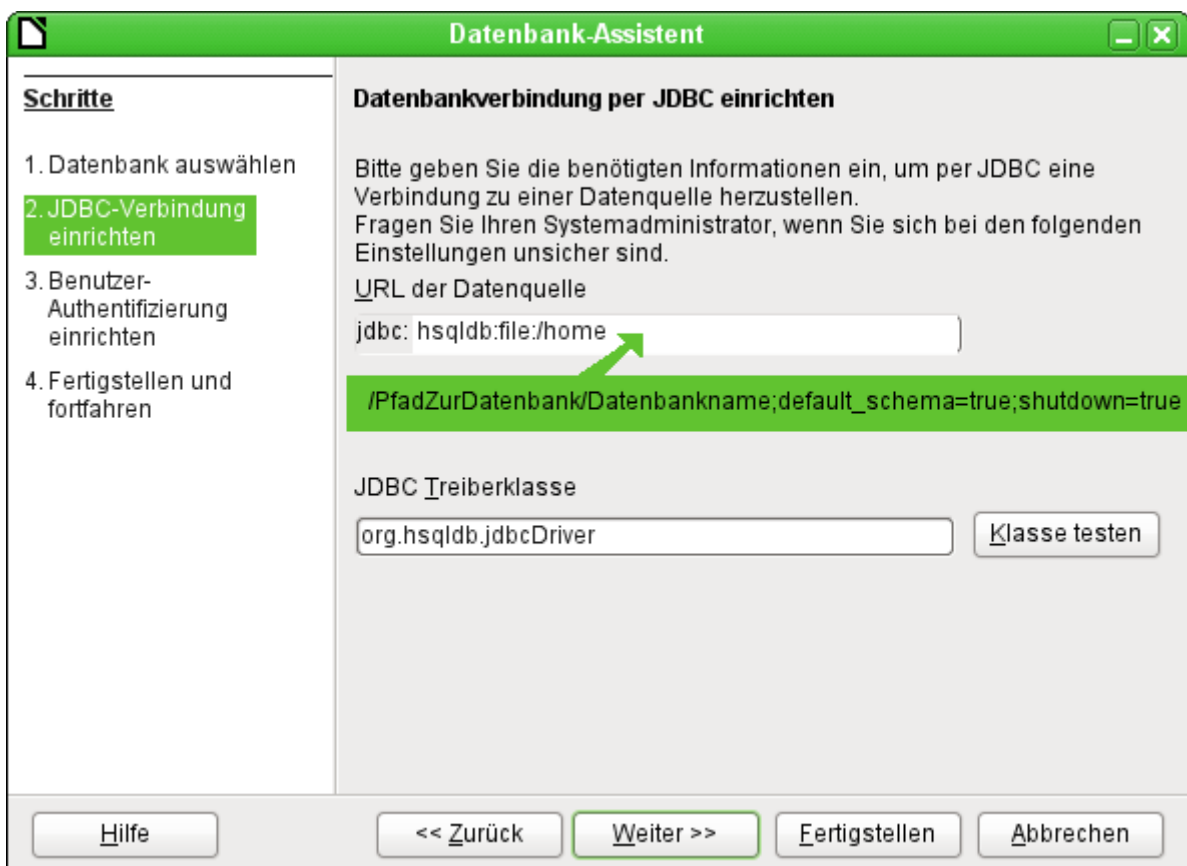
Wichtig, damit auch Daten in die Datenbank über die GUI geschrieben werden können, muss: ergänzend neben dem Datenbanknamen «**;default_schema=true**» stehen. Dies kann noch durch ein «**;shutdown=true**» ergänzt werden, damit die DB nach dem Schließen von LO heruntergefahren wird.

Also:

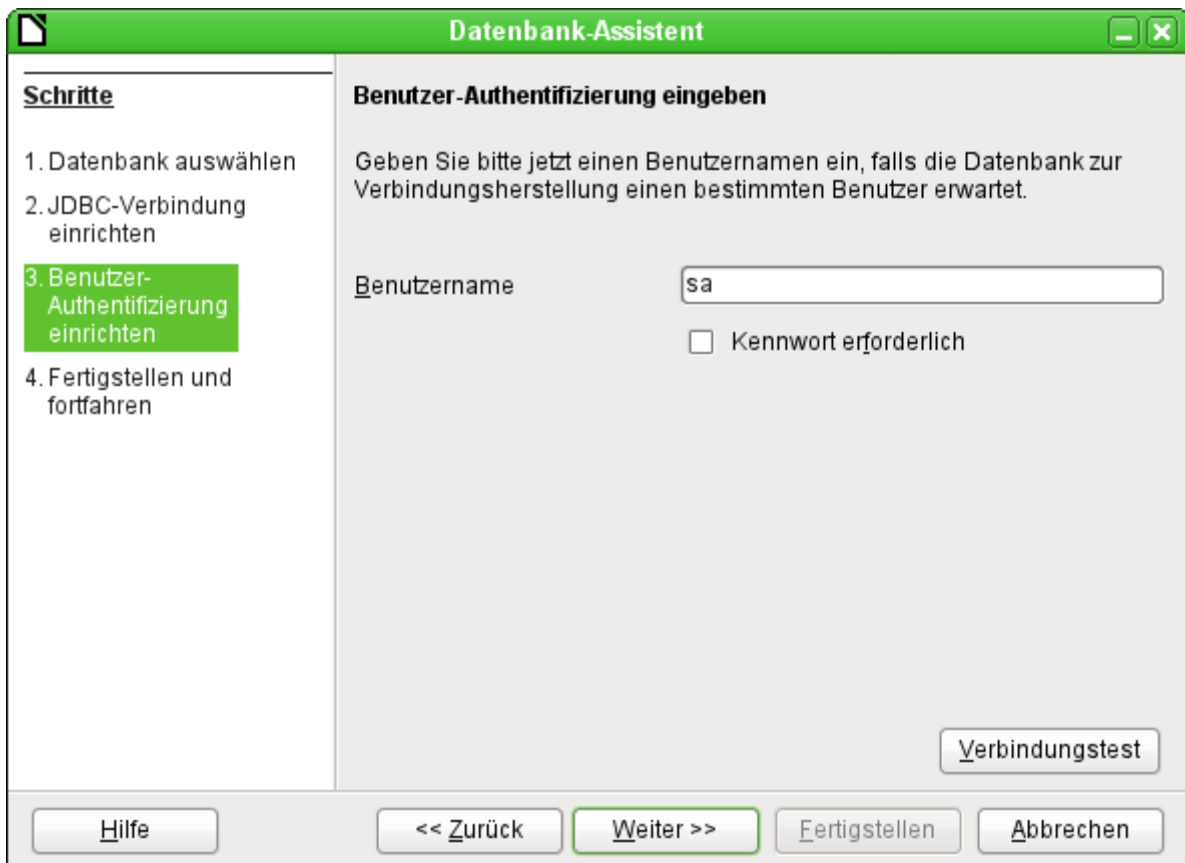
```
jdbc:hsqldb:file:/home/PfadZurDatenbank/Datenbankname;default_schema=true;  
shutdown=true
```

In dem Ordner befinden sich die Dateien

- Datenbankname.backup
- Datenbankname.data
- Datenbankname.properties
- Datenbankname.script
- Datenbankname.log



Weiter geht es mit der Angabe des Standardnutzers, sofern nichts an der HSQLDB-Konfiguration geändert wurde:



Damit ist die Verbindung erstellt und es kann auf die Datenbank zugegriffen werden.

Vorsicht



Wird eine externe Datenbank mit einer Version HSQLDB 2.* bearbeitet, so kann sie anschließend nicht mehr in eine interne Datenbank unter LibreOffice umgewandelt werden. Dies liegt an den zusätzlichen Funktionen, die in der Version 1.8.* noch nicht vorhanden sind. Dadurch endet der Aufruf mit der Version 1.8.* bereits beim Einlesen der Script-Datei der Datenbank.

Ebenso wenig kann eine externe Datenbank, die einmal mit einer Version der 2er-Reihe bearbeitet wurde, anschließend wieder mit der externen Version 1.8.* bearbeitet werden, die kompatibel zu LibreOffice ist.

Auch kann bei der Nutzung einer anderen Version in der externen Datenbank die Version der internen Datenbank überschrieben werden. Der Aufruf als interne Datenbank ist dann unmöglich.

Parallelinstallation von interner und externer HSQLDB

Die Einbindung der externen Datei hsqldb.jar in den Class-Pfad kann bei unterschiedlichen Versionen dazu führen, dass interne Datenbanken anschließend nicht mehr zu öffnen sind. Base kommt mit den gleichlautenden Treibern nicht zurecht und will die externe Variante auch für die internen Datenbanken nutzen. Das geht beim ersten Öffnen noch gut. Beim zweiten Öffnen kommt dann aber die Meldung, dass die Datenbank nicht mehr zu öffnen ist, da sie mit einer neueren Version der HSQLDB geschrieben wurde.

Dem kann abgeholfen werden, indem für die externen Datenbanken die Datei hsqldb.jar nicht über den Class-Pfad in LO fest eingegeben wird, sondern stattdessen der Class-Pfad für die jeweilige Datenbankdatei über ein Makro geschrieben wird, wie auch [hier](http://forum.openoffice.org/en/forum/viewtopic.php?f=40&t=61155) (<http://forum.openoffice.org/en/forum/viewtopic.php?f=40&t=61155>) zu lesen ist.


```

SUB Start
  Const cPath = "/home/robby/public_html/hsqldb_test/hsqldb.jar"
  DIM oDataSource AS OBJECT
  DIM oSettings AS OBJECT
  DIM sURL AS STRING
  sURL = ConvertToURL(cPath)
  oDataSource = ThisComponent.DataSource
  oSettings = oDataSource.Settings
  oSettings.JavaDriverClassPath = sURL
END SUB

```

Hier wird die Datei «hsqldb.jar» unter Linux in dem o.g. Pfad abgelegt. Anschließend wird dieser Pfad der momentan geöffneten Datenbank als Treiberdatei zugewiesen.

Dieses Makro wird nach dem Öffnen der *.odb-Datei einmal aufgerufen. Es schreibt dann in die in der *.odb-Datei befindlichen Datei content.xml die entsprechende Verbindung zur Java-Klasse:

```

<db:data-source-settings>
  <db:data-source-setting
    db:data-source-setting-is-list="false"
    db:data-source-setting-name="JavaDriverClass"
    db:data-source-setting-type="string">
  <db:data-source-setting-value>
    org.hsqldb.jdbcDriver
  </db:data-source-setting-value>
</db:data-source-setting>
  <db:data-source-setting
    db:data-source-setting-is-list="false"
    db:data-source-setting-name="JavaDriverClassPath"
    db:data-source-setting-type="string">
  <db:data-source-setting-value>
    file:///home/robby/public_html/hsqldb_test/hsqldb.jar
  </db:data-source-setting-value>
</db:data-source-setting>
</db:data-source-settings>

```

Letztlich könnte also auch ohne das Makro ein entsprechender Pfad direkt in die content.xml der *.odb-Datei eingetragen werden. Nur ist dieser Weg für den Normalnutzer sicher nicht so komfortabel zu handhaben.

Änderung der Datenbankverbindung zur externen HSQLDB

Die interne HSQL-Datenbank hat den Nachteil, dass die Abspeicherung der Daten innerhalb eines gepackten Archivs erfolgt. Erst mit dem Packen werden alle Daten festgeschrieben. Dies kann leichter zu Datenverlust führen als es bei der Arbeit mit einer externen Datenbank der Fall ist. Im folgenden werden die Schritte gezeigt, die notwendig sind, um den Umstieg einer bestehenden Datenbank vom *.odb-Päckchen zur externen Version in HSQL zu erreichen.

Aus einer Kopie der bestehenden Datenbank wird das Verzeichnis «database» extrahiert. Der Inhalt wird in das oben beschriebene frei wählbare Verzeichnis kopiert. Dabei sind die enthaltenen Dateien um den Datenbanknamen zu ergänzen:

```

Datenbankname.backup
Datenbankname.data
Datenbankname.properties
Datenbankname.script
Datenbankname.log

```

Jetzt muss noch die «content.xml» aus dem *.odb-Päckchen extrahiert werden. Hier sind mit einem einfachen Texteditor die folgenden Inhalte zu suchen, die leider in einer durchlaufenden Zeile stehen, es sei denn, man verwendet für diese Arbeit einen speziellen sog. XML-Editor:

```

<db:connection-data>
  <db:connection-resource xlink:href="sdbc:embedded:hsqldb"/>

```

```

    <db:login db:is-password-required="false"/>
  </db:connection-data>
  <db:driver-settings
    db:system-driver-settings=""
    db:base-dn=""
    db:parameter-name-substitution="false"/>

```

Diese Zeilen sind mit der Verbindung zur externen Datenbank zu ersetzen, hier der Verbindung zu einer Datenbank mit dem Namen "medien", die jetzt im Verzeichnis «hsqldb_data» liegt.

```

<db:connection-data>
  <db:connection-resource
xlink:href="jdbc:hsqldb:file:/home/robby/Dokumente/Datenbanken/hsqldb_data/medien;default_schema=true;shutdown=true"/>
    <db:login db:user-name="sa" db:is-password-required="false"/>
  </db:connection-data>
  <db:driver-settings
    db:java-driver-class="org.hsqldb.jdbcDriver"/>

```

Falls, wie oben geschrieben, die Grundkonfiguration der HSQLDB nicht angetastet wurde, stimmt auch der Nutzernamen und die nicht erforderliche Passwordeinstellung.

Nach Änderung des Codes muss die content.xml wieder in das *.odb-Päckchen eingepackt werden. Das Verzeichnis «database» ist in dem Päckchen jetzt überflüssig. Die Daten werden in Zukunft durch die externe Datenbank zur Verfügung gestellt.

Änderung der Datenbankverbindung für einen Mehrbenutzerbetrieb

Für die Mehrbenutzerfunktion muss die HSQLDB über einen Server zur Verfügung gestellt werden. Wie die Installation des Servers erfolgt, ist je nach Betriebssystem unterschiedlich. Für OpenSuSE war nur ein entsprechendes Paket herunter zu laden und der Server zentral über YAST zu starten (Runlevel-Einstellungen). Nutzer anderer Betriebssysteme und Linux-Varianten finden sicher geeignete Hinweise im Netz.

Im Heimatverzeichnis des Servers, unter SuSE /var/lib/hsqldb, befinden sich unter anderem ein Verzeichnis «data», in dem die Datenbank abzulegen ist, und eine Datei «server.properties», die den Zugang zu den (eventuell also auch mehreren) Datenbanken in diesem Verzeichnis regelt.

Die folgenden Zeilen geben den kompletten Inhalt dieser Datei auf dem Rechner wieder. Es wird darin der Zugang zu 2 Datenbanken geregelt, nämlich der ursprünglichen Standard-Datenbank (die als neue Datenbank genutzt werden kann) als auch der Datenbank, die aus der *.odb-Datei extrahiert wurde.

```

# Hsqldb Server cfg file.
# See the Advanced Topics chapter of the Hsqldb User Guide.

server.database.0    file:data/db0
server.dbname.0      firstdb
server.urlid.0       db0-url

server.database.1    file:data/medien
server.dbname.1      medien
server.urlid.1       medien-url

server.silent        true
server.trace         false

server.port          9001
server.no_system_exit true

```

Die Datenbank 0 wird mit dem Namen "firstdb" angesprochen, obwohl die einzelnen Dateien in dem Verzeichnis data mit "db0" beginnen. Die neue Datenbank wird als "Datenbank 1" hinzugefügt. Hier sind Datenbankname und Dateibeginn identisch.

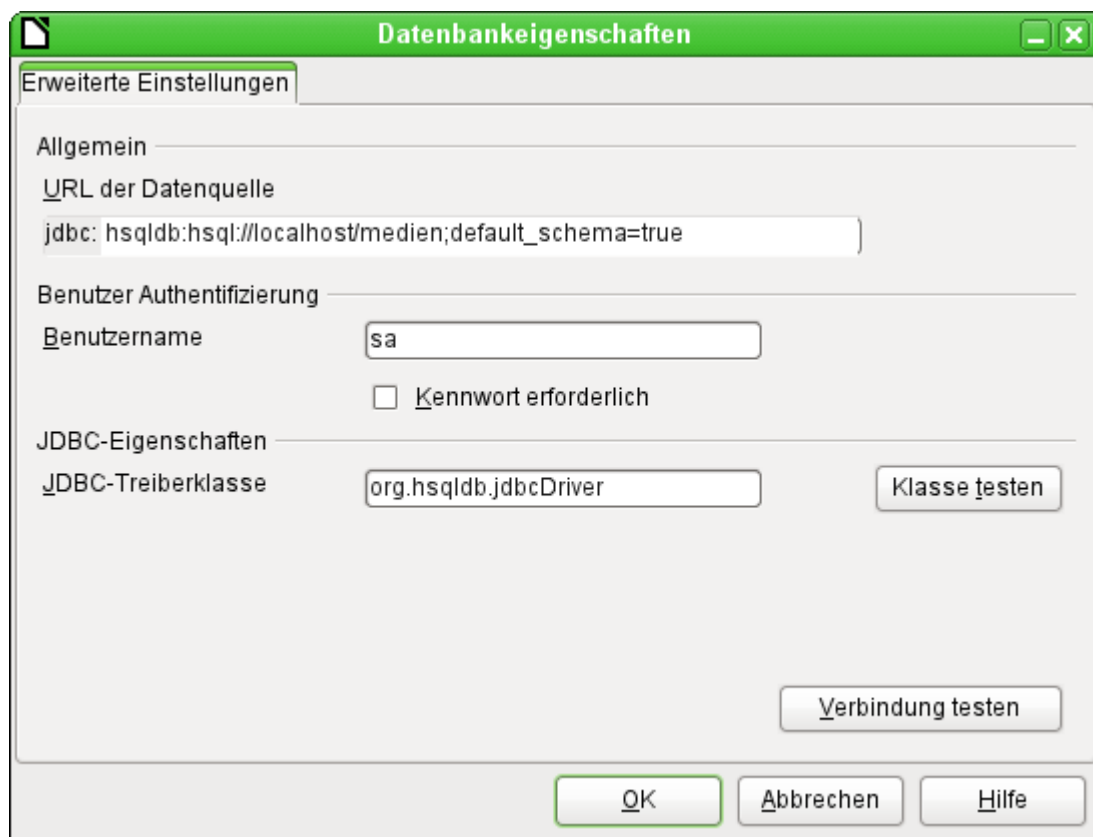
Die beiden Datenbanken werden mit folgenden Zugängen angesprochen:

```
jdbc:hsqldb:hsql://localhost/firstdb;default_schema=true
username sa
password
jdbc:hsqldb:hsql://localhost/medien;default_schema=true
username sa
password
```

Die URL wurde hier bereits jeweils um den für den Schreibzugang über die grafische Benutzeroberfläche von LO erforderlichen Zusatz «**;default_schema=true**» ergänzt.

Wenn tatsächlich im Serverbetrieb gearbeitet werden soll, ist natürlich aus Sicherheitsgründen zu überlegen, ob die Datenbank nicht mit einem Passwort geschützt werden soll.

Nun erfolgt die Serververbindung über LO. Im Hauptfenster von Base wird **Bearbeiten → Datenbank → Eigenschaften** aufgesucht.



Mit diesen Zugangsdaten wird auf den Server des eigenen Rechners zugegriffen. Im Netzwerk mit anderen Rechnern müsste dann entweder über Rechnernamen oder die IP-Adresse auf den Server, der ja auf dem aktuellen Rechner läuft, zugegriffen werden.

Beispiel: Der Rechner hat die IP 192.168.0.20 und ist im Netz bekannt mit dem Namen lin_serv. Jetzt ist an anderen Rechnern für die Verbindung zur Datenbank einzugeben:

```
jdbc:hsqldb:hsql://192.168.0.20/medien;default_schema=true
```

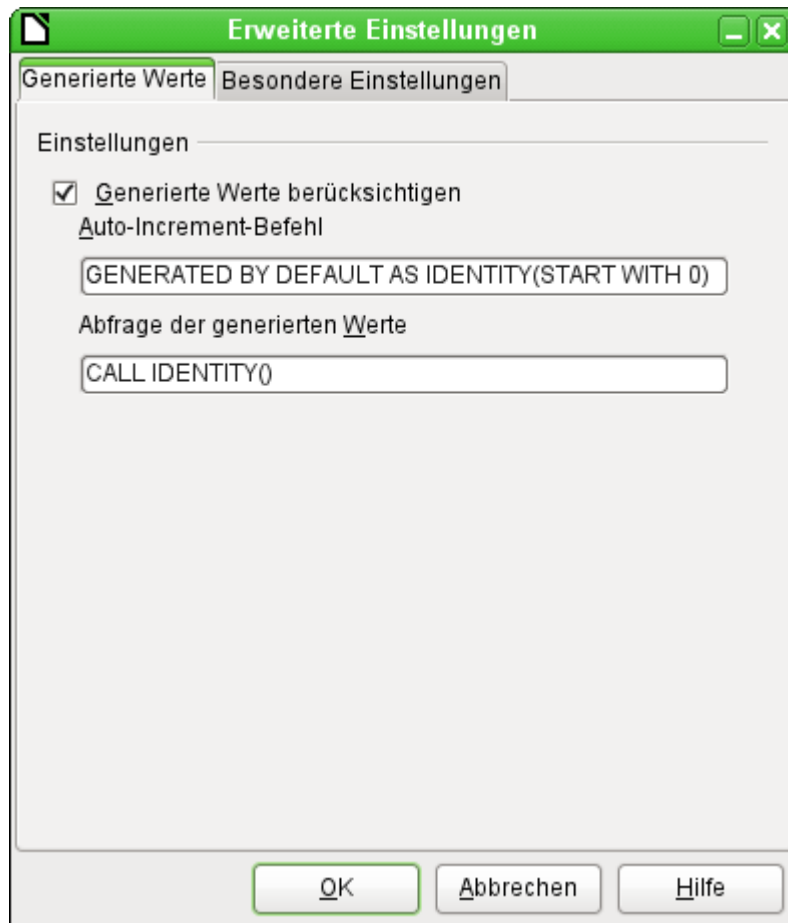
bzw.:

```
jdbc:hsqldb:hsql://lin_serv/medien;default_schema=true
```

Die Datenbank ist nun angebunden und kann beschrieben werden. Hier kann allerdings ein zusätzliches Problem auftauchen. Die vorher automatisch generierten Werte werden plötzlich nicht mehr hochgeschrieben. Hier fehlt es noch an einer zusätzlichen Einstellung.

Autoinkrementwerte mit der externen HSQLDB

Für die Nutzung der Auto-Werte müssen je nach Version von Base bei der Tabellenerstellung verschiedene Wege beschritten werden. Allen gleich ist erst einmal der folgende Eintrag unter **Bearbeiten → Datenbank → Erweiterte Einstellungen** erforderlich:



Mit dem Zusatz **GENERATED BY DEFAULT AS IDENTITY(START WITH 0)** soll die Funktion des automatisch hochzählenden Wertes für den Primärschlüssel erstellt werden. Die GUI von LO übernimmt zwar diesen Befehl (auch in der aktuellen LO-Version), schreibt davor aber leider die Anweisung **NOT NULL**, so dass die Reihenfolge der Befehlsfolge für die HSQLDB nicht lesbar ist. Hierbei ist zu berücksichtigen, dass die HSQLDB mit dem obigen Befehl ja bereits mitgeteilt bekommt, dass das entsprechende Feld den Primärschlüssel enthält.

Hinweis

In LO ist deshalb die Eingabe des Autowertes in der GUI nicht möglich. Nutzer dieser Versionen erstellen zuerst eine Tabelle mit einem Primärschlüsselfeld ohne Autowert und geben dann direkt über **Extras → SQL** ein:

```
ALTER TABLE "Tabellenname" ALTER COLUMN "ID" INT GENERATED BY  
DEFAULT AS IDENTITY(START WITH 0)
```

... wobei davon ausgegangen wird, dass das Primärschlüsselfeld den Namen "ID" hat.

Mit dem Auslesen des letzten Wertes und dem Hochlesen zum nächsten Wert hingegen klappt es in allen Versionen von LO über den Befehl **CALL IDENTITY()**. Dies trifft dann z.B. auf die Lösung zu, die Datenbank zuerst einmal als «*.odb-Päckchen» zu erstellen, gründlich zu testen und danach dann die Datenbanktabellen einfach auszulagern.

Sämtliche Abfragen, Formulare und Berichte lassen sich so weiter nutzen, da die Datenbank für die «*.odb-Datei» weiter auf die gleiche Weise angesprochen wird und eventuell spezifische SQL-Befehle mit der externen HSQLDB weiter gelesen werden können.

Umgang mit der internen Firebird-Datenbank

Die interne Firebird-Datenbank ist bisher nur als experimentelle Funktion verfügbar. Um solch eine Datenbank zu erstellen oder anschließend bearbeiten zu können, muss **Extras → Optionen → LibreOffice → Erweitert → Optionale (instabile) Einstellungen → Experimentelle Funktionen aktivieren** eingeschaltet sein. Schon dieser Weg zeigt auf, dass solch eine Datenbank nicht für den täglichen Gebrauch geeignet ist.

Über den folgenden Link können die wesentlichsten Bugs der internen Firebird-Datenbank zusammen mit LibreOffice eingesehen werden: [Gemeldete Bugs für Firebird in Zusammenhang mit Base](https://bugs.documentfoundation.org/buglist.cgi?bug_status=UNCONFIRMED&bug_status=NEW&bug_status=ASSIGNED&bug_status=REOPENED&bug_status=NEEDINFO&component=Base&known_name=Firebird_open&list_id=522642&product=LibreOffice&query_based_on=Firebird_open&query_format=advanced&short_desc=Firebird&short_desc_type=allwordssubstr) ([https://bugs.documentfoundation.org/buglist.cgi?](https://bugs.documentfoundation.org/buglist.cgi?bug_status=UNCONFIRMED&bug_status=NEW&bug_status=ASSIGNED&bug_status=REOPENED&bug_status=NEEDINFO&component=Base&known_name=Firebird_open&list_id=522642&product=LibreOffice&query_based_on=Firebird_open&query_format=advanced&short_desc=Firebird&short_desc_type=allwordssubstr)

[bug_status=UNCONFIRMED&bug_status=NEW&bug_status=ASSIGNED&bug_status=REOPENED&bug_status=NEEDINFO&component=Base&known_name=Firebird_open&list_id=522642&product=LibreOffice&query_based_on=Firebird_open&query_format=advanced&short_desc=Firebird&short_desc_type=allwordssubstr](https://bugs.documentfoundation.org/buglist.cgi?bug_status=UNCONFIRMED&bug_status=NEW&bug_status=ASSIGNED&bug_status=REOPENED&bug_status=NEEDINFO&component=Base&known_name=Firebird_open&list_id=522642&product=LibreOffice&query_based_on=Firebird_open&query_format=advanced&short_desc=Firebird&short_desc_type=allwordssubstr)) . Bitte nicht von der vermeintlich letzten Änderung am 3.01.2015 irritieren lassen. Zu dem Zeitpunkt hat sich nur ein QA-Mitglied ins CC gesetzt. Tatsächlich sind die Bugs leider meist so alt wie das Erscheinen der ersten internen Firebird-Version ...

Folgende Besonderheiten fallen dem Nutzer gegenüber der HSQLDB direkt auf:

1. Werden neue Daten eingegeben, so werden diese nicht automatisch in der Datenbankdatei abgespeichert. Der Speicherbutton wird bei jeder Dateneingabe erneut aktiviert. Bei der eingebauten HSQLDB ist die Abspeicherung nach Dateneingaben nicht notwendig.
2. Das Erstellen von Beziehungen ist nicht möglich, obwohl die externe Firebird-Datenbank Beziehungen unterstützt.

Funktionserweiterungen und -änderungen in Base im Laufe der LO-Versionen

LO 3.6

Der Befehl **SHUTDOWN COMPACT** wird automatisch beim Schließen der Datenbankdatei ausgeführt.

LO 4.1

Im **Abfrageeditor** wurde die Einstellung des **Limits** für die auszugebenden Daten möglich. Diese Einstellung arbeitet so auch mit der GUI und berührt nicht die Editierbarkeit von Abfragen.

Bei **Listenfeldern** ist die Angabe des **gebundenen Feldes** auch als 0 (gleiches Feld wie angezeigt) und als -1 (Position des ausgewählten Feldes) möglich.

Wird mit Makros auf ein **Listenfeld** zugegriffen, so ergibt seit dieser Version der **currentValue** den Wert, der an die Datenbank weitergegeben wird, und nicht unbedingt den Wert, den das Feld in dem Formular anzeigt.

Der **Report-Designer** ist in LO als Erweiterung integriert, so dass er nicht mehr als Erweiterung in den Erweiterungsdialogen erscheint.

LO 4.1.2

Der Zugriff auf **Datumswerte** mit Makros in Formularen wurde geändert. Der Datumswert wird jetzt im Datumsfeld als eine Kombination von Tag, Monat und Jahr wiedergegeben und nicht als ISO-Zahlenwert: oFeld.CurrentValue.Year ist so z.B. die Jahresangabe.

LO 4.2

Access2Base ist standardmäßig als Erweiterung in LO integriert.

Firebird ist als experimentelle Datenbank in LO eingebaut. Die HSQLDB bleibt aber weiterhin die Standarddatenbank.

LO 4.3.1

Aliaszuweisungen in der grafischen Benutzeroberfläche von LO erfolgen ohne den Zusatz **AS**. Die Funktionsweise der internen HSQLDB ist davon nicht berührt. Oracle konnte mit dieser Form der Aliaszuweisung nicht umgehen.

LO 4.4

Parameter in Abfragen: Ein leerer Parameter wurde bisher als leerer Text in Abfragen weiter gegeben, innerhalb von Formularen aber als **NULL** angesehen. Dies wurde geändert. Ein leerer Parameter ist jetzt auch in Abfragen **NULL** und kann also z.B. mit **IFNULL** abgefragt werden.

LO 5.0

Grafisches Kontrollfeld: Das grafische Kontrollfeld konnte bis zu diesem Zeitpunkt nur Bildformate einlesen und darstellen. Mit der Version 5.0 ist das Einlesen bzw. Verknüpfen sämtlicher Dateien möglich. Bei *.pdf-Dateien wird in dem Kontrollfeld die 1. Seite dargestellt. Bei unbekannten Formaten bleibt die Anzeige im Kontrollfeld einfach leer.

LO 5.3

Interne Firebird Datenbank: Die interne Datenbank wechselt von der Version 2.5 auf die Version 3.0.0. Die Version 3.0.0 hat keine Abwärtskompatibilität, so dass vorher erstellte interne Firebird-Datenbanken mit der Version LO 5.3 nicht geöffnet werden können.

Die neue Firebird-Version hat ein besonderes Archiv-Format. Ältere Firebird-Datenbanken können nur über das Entpacken der *.odb-Datei und ein Umwandeln mit einer externen Firebird-Datenbank in das Archivformat überführt werden. Ansonsten bietet es sich an, Tabellen älterer Datenbanken ggf. in eine Calc-Datei zu überführen und von dort in die neue Version zu importieren.

Hinweis

Die Umwandlung von der alten Firebird Datenbank zur Version 3.0 kann lt. Firebird-Homepage mit Hilfe einer Firebird 2.5 und einer Firebird 3.0 - Serverinstallation erfolgen (http://www.firebirdsql.org/file/documentation/release_notes/html/en/3_0/rnfb30-compat-upgrade-secd.db.html)

Absolut speichern

In einigen Dialogen (z. B. **Bearbeiten** → **AutoText**) können Sie wählen, ob eine Datei relativ oder absolut gespeichert werden soll.

Wenn Sie absolut speichern wählen, werden alle Referenzen auf andere Dateien ebenfalls als absolut definiert, wobei sich diese Definition am jeweiligen Laufwerk, Volumen oder Quellverzeichnis orientiert. Der Vorteil dieser Methode besteht darin, dass das Dokument mit den Referenzen in ein anderes Verzeichnis oder einen anderen Ordner verschoben werden kann und die Referenzen weiterhin gültig bleiben.

Vergleichen Sie auch mit *Relativ speichern*.

Andocken

Einige Fenster in LibreOffice, zum Beispiel das Fenster *Formatvorlagen* und der *Navigator*, sind andockbare Fenster. Sie können diese Fenster verschieben, vergrößern oder sie an einer Kante des Arbeitsbereichs andocken. An jeder Kante können Sie auch mehrere Fenster über oder nebeneinander andocken. Sie können dann durch das Verschieben der Umrandungslinien die relativen Proportionen der Fenster verändern.

Lesen Sie den Abschnitt „Symbolleisten andocken und frei schweben lassen“ im Kapitel 01 „LibreOffice Einführung“ des Handbuchs *Erste Schritte* für eine Beschreibung des Vorgehens.

Andocken (AutoHide)

An jedem Fensterrand, an dem ein anderes Fenster angedockt ist, befindet sich eine Schaltfläche, die zum Einblenden oder Ausblenden des Fensters dient.

Wenn Sie zum Anzeigen des Fensters auf die Schaltfläche am Fensterrand klicken, bleibt das Fenster so lange eingeblendet, bis Sie es mit derselben Schaltfläche wieder ausblenden.

Wenn Sie das Fenster durch Klicken auf den Fensterrand einblenden, aktivieren Sie die Funktion *AutoHide*. *AutoHide* ermöglicht es, ein eigentlich ausgeblendetes Fenster durch Klicken auf dessen Rand kurzzeitig einzublenden. Sobald Sie mit der Maus in den Arbeitsbereich oder ein anderes Fenster klicken, wird das Fenster wieder verborgen.

ANSI

Das "American National Standards Institute" ist ein US-Normungsgremium, vergleichbar mit dem deutschen *DIN*.

ANSI-Zeichensatz

Der ANSI-Zeichensatz stimmt zwar weitgehend, aber nicht vollständig mit dem in den westeuropäischen Windows-Betriebssystemen von Microsoft häufig verwendeten *Windows-Zeichensatz* 1252 überein. Obwohl die gebräuchliche Bezeichnung etwas anderes suggeriert, ist der ANSI-Zeichensatz keine Norm der *ANSI*, sondern wurde durch Microsoft geprägt.

API

Eine API (Application Programming Interface) ist eine Schnittstelle, die es anderen Programmen ermöglicht, sich in das Programm bzw. die Bibliothek zu integrieren. In LibreOffice wird diese Schnittstelle als *Extension* (Erweiterung) bezeichnet.

ASCII

Abkürzung für „American Standard Code for Information Interchange“ (zu deutsch: amerikanischer Standardcode zum Informationsaustausch). ASCII ist ein Zeichensatz für die Zeichendarstellung bei Personal Computern (Zeichenkodierung) und entspricht der US-Variante von *ISO 646*. Er besteht aus 128 Zeichen mit Buchstaben, Ziffern, Satzzeichen sowie Sonderzeichen und dient als Grundlage für spätere auf mehr Bits basierende Kodierungen für Zeichensätze.

Dieser Zeichensatz wurde von *MS-DOS* benutzt und ist nicht kompatibel zum *ANSI-Zeichensatz*, *Unicode-Zeichensatz* und *Windows-Zeichensatz*.

Der erweiterte ASCII-Zeichensatz (8 bit) enthält 256 Zeichen. Jedem Zeichen ist eine eindeutige Nummer zugewiesen, die man auch als ASCII-Code bezeichnet.

In *HTML*-Seiten sollten nur die Zeichen des 7-Bit-ASCII-Zeichensatzes vorkommen. Andere Zeichen, wie etwa die deutschen Umlaute, werden durch Umschreibungen gekennzeichnet. So wird das kleine „ü“ etwa zu „ü“. Sie können in LibreOffice Zeichen im erweiterten ASCII-Code eingeben; der Exportfilter sorgt für die erforderliche Umwandlung.

Austauschformat

Ein Dateiformat, welches dem Austausch von Daten dient.

Das Austauschformat können Grafiken oder allgemeine Daten sein, z. B. von einer Tabellenkalkulation. Austauschformate dienen auch zum Import oder Export von einer Anwendung in eine Andere.

Barcode

Als Barcode (oder Strichcode bzw. Streifencode) wird eine lesbare Schrift, die aus verschiedenen breiten, parallelen Linien besteht, bezeichnet. Diese Linien werden von einem Scanner gelesen und interpretiert, so dass diese elektronisch weiter verarbeitet werden können. Gängige Vertreter sind hier Etiketten (EAN-Code) oder ISBN-Nummern bei Büchern. Einen „Nachfolger“ stellt der QR-Code dar.

Bézierobjekt

Bézierkurven sind nach einem vom französischen Mathematiker Pierre Bézier entwickelten Verfahren mathematisch dargestellte Kurven, wie sie in zweidimensionalen Grafikanwendungen zum Einsatz kommen. Eine solche Kurve definiert sich durch vier Punkte: den Anfangspunkt, den Endpunkt und zwei separate Zwischenpunkte. Ein Bézier-Objekt lässt sich durch Verschieben dieser Punkte mit der Maus verformen.

Bildkompression

Bildkompression beruht wie jede Art der Datenkompression darauf, den ursprünglichen Datensatz entweder über eine *Verlustfreie Kompression* in einer vollständig rekonstruierbaren Form zu speichern oder Daten zu entfernen, deren Verlust kaum wahrnehmbar ist (*Verlustbehaftete Kompression*). Es gibt sehr viele Grafikformate, von denen aber viele veraltet sind und viele keine Kompression unterstützen, da sie *Austauschformate* für Grafikprogramme sind.

Complex Text Layout (CTL)

Sprachen mit komplexen Skripten können unter anderem folgende Eigenschaften aufweisen:

- Es werden Zeichen verwendet, die aus mehreren Teilen zusammengesetzt sind.
- Die Schreibrichtung des Texts läuft von rechts nach links.

LibreOffice unterstützt derzeit Hindi, Thai, Hebräisch und Arabisch als CTL-Sprachen. Aktivieren Sie die CTL-Unterstützung unter **Extras → Optionen... → Spracheinstellungen → Sprachen**, um diese zu verwenden.

Dateiendung

Die Dateiendung gibt normalerweise das Format der Datei an und stellt die Buchstabenkombination hinter dem letzten Punkt des Dateinamens dar.

Datenbank

In einer Datenbank, meist Datenbanksystem (DBS) genannt, werden Daten gespeichert, verwaltet und miteinander in Beziehung gebracht. Zusätzlich können Informationen gesucht sowie gefiltert dargestellt werden. Stellt das System weitere Möglichkeiten, wie beispielsweise die

Verwaltung von Anwendern, Transaktionskontrolle usw. zur Verfügung, spricht man von einem Datenbankmanagementsystem (DBMS).

dBASE

dBASE war eines der ersten dateibasierten Datenbankverwaltungssysteme für PCs. Die strukturierten Daten wurden in „DataBaseFiles“ (daher die Dateierendung DBF) gespeichert. dBASE-Dateien und -Programme sind heute technisch veraltet, wobei das Dateiformat hin- und wieder als Austauschformat verwendet wird.

DDE

DDE steht für "Dynamic Data Exchange", also dem dynamischen Datenaustausch. Dies ist ein Vorgänger von *OLE*, dem "Object Linking and Embedding". Bei DDE wird ein *Objekt* in Form einer *Verknüpfung* zur Datei eingebunden, aber im Gegensatz zu OLE nicht selbst eingebettet.

DDE-Verknüpfungen lassen sich wie folgt erzeugen: Wählen Sie in einem LibreOffice-Calc Tabellendokument Zellen aus, kopieren Sie diese in die *Zwischenablage*, wechseln Sie in ein anderes Tabellendokument, und wählen Sie **Bearbeiten → Inhalte einfügen**. Wählen Sie die Option "Verknüpfen" aus, um den Inhalt als DDE-Verknüpfung einzufügen. Bei Aktivierung der Verknüpfung wird der eingefügte Zellbereich aus der Originaldatei eingelesen.

Debian

Debian ist eine Linux-Distribution, die es seit 1996 gibt und den Linux-Kernel sowie alle für den Betrieb des Systems erforderliche Programme enthält. Seit der Version 6.0 enthält Debian nur noch freie Software.

DIN

Das "Deutsche Institut für Normung e. V." ist die bedeutendste nationale Normungsorganisation in der Bundesrepublik Deutschland.

Der heutige Name wurde 1975 im Zusammenhang mit dem zwischen der Organisation und der Bundesrepublik Deutschland abgeschlossenen Normenvertrag gewählt. Die unter der Leitung von Arbeitsausschüssen dieser Normungsorganisation erarbeiteten Standards werden als "DIN-Normen" bezeichnet. Das DIN arbeitet in den internationalen und europäischen Normengremien mit, um die deutschen Interessen zu vertreten und den internationalen freien Warenverkehr zu fördern. Es organisiert die Eingliederung internationaler Normen in das deutsche Normenwerk.

Direkte Formatierung

Wenn Sie Ihre Dokumente ohne Hilfe von Formatvorlagen formatieren, spricht man von direkter oder "harter" *Formatierung*. Darunter versteht man die Veränderung von Text oder von einem anderen *Objekt*, wie ein Rahmen oder eine Tabelle, über die Zuweisung verschiedener Attribute. Die Formatierung gilt nur für den ausgewählten Bereich und alle Änderungen müssen einzeln bearbeitet werden.

Direkte Formatierungen können Sie aus Ihrem Dokument entfernen, indem Sie mit den Text markieren und im Menü **Format → Direkte Formatierung löschen** wählen.

Dem gegenüber steht die zu bevorzugende *Indirekte Formatierung*.

Drag&Drop

Sehen Sie unter *Ziehen&Ablegen* nach.

Drehfeld

Ein Drehfeld ist eine Eigenschaft eines Zahlen-, Währungs-, Datums- oder Zeitfelds bei Formular-Steuerelementen. Wenn die Eigenschaft *Drehfeld* aktiviert ist, zeigt das Feld zwei Symbole mit Pfeilen an. Diese zeigen entweder senkrecht oder waagerecht in entgegengesetzte Richtungen.

In der Basic IDE wird ein numerisches Feld mit zwei Pfeilsymbolen als Drehfeld bezeichnet. Sie können entweder einen numerischen Wert direkt in das Drehfeld eingeben oder mit dem Auf- und Abwärtspfeil auswählen. Mit den Tasten *Nach oben* und *Nach unten* Ihrer Tastatur lässt sich der Wert im Drehfeld ebenfalls vergrößern oder verkleinern. Mit den Tasten *Bild nach oben* und *Bild nach unten* können Sie den Höchst- und den Mindestwert für das Drehfeld erreichen.

Handelt es sich um ein Drehfeld für numerische Werte, dann können Sie auch eine Maßeinheit angeben, also z. B. „1 cm“, „5 mm“, „12 pt“ oder „2“.

DTP-Programm

Mittels eines DTP-Programms (**Desktop-Publishing-Programm**) werden hochwertige Dokumente, die aus Texten und Bildern bestehen, erstellt. Diese Dokumente können zum Beispiel für den Druck von Broschüren, Magazinen, Büchern oder Katalogen verwendet werden.

Extension

Eine Extension (deutsch: Erweiterung) ist ein Programm, welches sich in LibreOffice integriert, um dessen Funktionen zu verbessern oder zu ersetzen. Die Extensions steuern LibreOffice über eine *API* an.

Firebird

Firebird ist ein freies Datenbankverwaltungsprogramm, welches auf Interbase basiert. Seit der LibreOffice-Version 4.2 ist Firebird in LibreOffice-Base integriert, vorerst aber nur als experimentelle Funktion neben der HSQLDB zugänglich.

Formatierung

Unter Formatieren versteht man in diesem Zusammenhang das optische Gestalten von Texten mit einem Textverarbeitungs- oder *DTP-Programm*. Dazu gehören das Festlegen des Papierformats, der Seitenränder, der Schriftarten, der Schrifteffekte sowie der Einzüge und Abstände.

Sie können Text durch *Direkte Formatierung* bei der Eingabe oder mithilfe von Formatvorlagen in LibreOffice formatieren (*Indirekte Formatierung*).

Gallery

Die LibreOffice Gallery ist ein Fenster, in dem Sie Bilder und Klänge in thematischen Ordnern verwalten und zur Nutzung in LibreOffice bereit stellen können.

GIF

Das "**Graphics Interchange Format**" (deutsch: *Grafik-Austauschformat*) ist ein Grafikformat, das eine gute *Verlustfreie Kompression* für Bilder mit geringer Farbtiefe besitzt (bis zu 256 verschiedene Farben pro Einzelbild).

Darüber hinaus können mehrere Einzelbilder in einer Datei abgespeichert werden, die von geeigneten Betrachtungsprogrammen wie Webbrowsern als Animationen interpretiert werden.

GPL

Die GNU **General Public License** (GPL) ist eine sehr verbreitete Software-Lizenz, welche den Anwendern die Freiheiten garantiert, die Software zu nutzen, studieren, verbreiten (kopieren) und ändern zu dürfen. Software unter GPL wird als „Freie Software“ bezeichnet. Die aktuelle Version hat die Version 3.0 und ist unter <http://www.gnu.org/licenses/gpl-3.0.html> zu finden.

HSQLDB

HSQLDB (**H**yper **S**tructured **Q**uery **L**anguage **D**atabase) ist ein freies, vollständig in Java programmiertes relationales Datenbankverwaltungssystem (RDBMS). Schon zu Zeiten von OpenOffice ist HSQLDB in Base integriert. Der Vorteil von HSQLDB ist, dass es sehr leicht in andere Produkte integriert werden kann (die entsprechende Bibliothek hat nur etwa eine Größe von einem Megabyte).

HTML

Die "Hypertext Markup Language" ist eine textbasierte Auszeichnungssprache zur Strukturierung von Inhalten wie Texten, Bildern und Hyperlinks in Dokumenten.

HTML-Dokumente sind die Grundlage des World Wide Web und werden von einem Webbrowser dargestellt.

Hurenkinderregelung

Schusterjungen und Hurenkinder sind historische Begriffe aus der Typographie, die seit langem benutzt werden.

Ein Hurenkind ist die letzte Zeile eines Absatzes, die alleine am oberen Rand der nächsten Seite steht. Mit einem Textdokument von LibreOffice können Sie diese unschöne Erscheinung automatisch für die gewünschte Absatzvorlage vermeiden. Dabei können Sie sogar wählen, wie viele Zeilen mindestens immer zusammen auf einer Seite gehalten werden sollen.

Dem gegenüber steht die *Schusterjungenregelung*.

IEC

Die Internationale Elektrotechnische Kommission (International **E**lectrotechnical **C**ommission) ist eine internationale Normungsorganisation mit Sitz in Genf für Normen im Bereich der Elektrotechnik und Elektronik. Einige Normen werden gemeinsam mit der *ISO* entwickelt.

IME

IME ist eine Abkürzung für "Input **M**ethod **E**ditor", also ein Eingabemethoden-Editor. Dies ist ein Programm, das die Eingabe von komplexen Zeichen aus nichtwestlichen Zeichensätzen über eine Standardtastatur ermöglicht.

Indirekte Formatierung

Bei der indirekten oder "weichen" *Formatierung* nehmen Sie Formatierungen nicht direkt am Text vor, sondern durch Zuweisen von Formatvorlagen. Der große Vorteil besteht darin, dass Sie mit der Änderung einer Formatvorlage jedes *Objekt* (Absätze, Rahmen, usw.) ändern, dem Sie diese Formatvorlage zugewiesen haben.

Dem gegenüber steht die *Direkte Formatierung*.

ISO

Die Internationale Organisation für Normung (International **O**rganization for **S**tandardization) ist die internationale Vereinigung von Normungsorganisationen und erarbeitet internationale Normen in allen Bereichen mit Ausnahme der Elektrik, der Elektronik und der Telekommunikation, für welche die *IEC* zuständig ist.

Java

Java ist eine Programmiersprache, die von der Firma Oracle weiterentwickelt (ursprünglich von der Firma Sun Microsystems entwickelt) und bereitgestellt wird. Für die Ausführung von Java-Programmen in einer Anwendung ist die *JRE* (Java Runtime Environment) notwendig.

JDBC

Sie können die *API* der **J**ava **D**ata**B**ase **C**onnectivity (JDBC) zum Herstellen einer Verbindung mit einer *Datenbank* von LibreOffice verwenden. JDBC-Treiber sind in der Programmiersprache *Java* geschrieben. Dabei unterscheidet man vier Typen:

1. JDBC-ODBC bridge plus ODBC driver

Die Verbindung zur Datenbank wird über ODBC hergestellt. Dabei stellt der JDBC-Treiber nur eine Kapselung der Aufrufe dar. Zusätzlich muss der ODBC-Treiber installiert werden.

2. Native-API partly-Java driver
Die Verbindung wird über einen Datenbanktreiber hergestellt. Der JDBC-Treiber ruft die entsprechenden Datenbank-Client-Funktionen auf. Der DBMS-Client muss dabei installiert sein.
3. JDBC-Net pure Java driver
Hier werden durch den JDBC-Treiber die Aufrufe in das entsprechende Protokoll übersetzt und dann von einem entsprechenden Server (middleware) in das DBMS-Protokoll übersetzt und an das DBMS geschickt.
4. Native-protocol pure Java driver
Hier werden durch den JDBC-Treiber die Aufrufe direkt in das Netzwerkprotokoll des DBMS übersetzt. Hierzu muss aber das entsprechende proprietäre Protokoll des DBMS unterstützt werden. Dieser Typ ist dann auch komplett plattformunabhängig.

JPEG

JPEG wurde von der Joint Photographic Experts Group im Jahr 1992 in der ISO Norm 10918-1 vorgestellt und ist heute eines der am meisten genutzten Bildformate.

JPG

Sehen Sie unter *JPEG* nach.

JRE

JRE (kurz für **J**ava **R**untime **E**nvironment) ist die Laufzeit- bzw. Softwareumgebung, mit der Java-Anwendungen ausgeführt werden können. Java und JRE werden von der Firma Oracle entwickelt und bereit gestellt.

LibreOffice ist ohne JRE lauffähig, JRE ist jedoch für die LibreOffice-Komponente Base wichtig.

Kerning

Kerning ist die englische Bezeichnung für eine Unterschneidung oder Spationierung. Darunter versteht man das Verringern oder Vergrößern des Abstandes zwischen Buchstabenpaaren zum optischen Ausgleich des Schriftbildes, z. B. bei „W“ und „a“.

In Kerning-Tabellen ist vermerkt, welche Buchstabenpaare mehr oder weniger Abstand benötigen. Diese Tabellen sind in der Regel Bestandteil der jeweiligen Schrift.

Kontextmenü

Kontextmenüs werden durch Rechtsklick auf ein Objekt oder in eine Markierung aufgerufen. Sie zeigen kontextsensitive Menüeinträge zu dem jeweiligen Objekt bzw. der Markierung in einem Aufklappmenü an. Nahezu überall in LibreOffice sind Kontextmenüs vorhanden.

LGPL

Die GNU Lesser **G**eneral **P**ublic **L**icense (LGPL) ist von der GPL abgeleitet, erlaubt aber im Gegensatz zur GPL, Bibliotheken etc. in eigene (proprietäre) Software einzubinden.

Link

Ein Link (oder Hyperlink) verknüpft den Inhalt mit einem anderen Dokument oder einem Abschnitt im gleichen Dokument. Links können sich hinter Texten, Symbolen oder Bildern verbergen.

Makro

Ein Makro ist eine Abfolge von Befehlsschritten, die in einer Programmiersprache wie LibreOffice Basic, JavaScript, BeanShell oder Python definiert werden. Ein Makro kann über den Makrorekorder oder über eigene Textzeilenprogrammierung erstellt und über die Menü- oder *Symbolleiste* abgerufen werden.

Markieren

Durch Markieren legen Sie fest, welche Teile eines Textes, einer Tabelle oder eines Bildes Sie bearbeiten wollen. Hierzu führen Sie mit gedrückter Maustaste den Mauszeiger über den gewünschten Bereich.

MariaDB

MariaDB ist ein relationales Datenbankverwaltungssystem, welches als Fork (Abspaltung) von MySQL entstanden ist. MariaDB steht unter GPL Lizenz, die Client-Bibliotheken unter LGPL.

MS-DOS

MS-DOS, kurz für Microsoft Disk Operating System, war Microsofts erstes Betriebssystem für PCs.

Es wurde ursprünglich für den Intel-Prozessor 8086/8088 entwickelt und war in den späten 1980er und frühen 1990er Jahren das dominierende Betriebssystem für Einzelplatzrechner. Die früheren Windows-Versionen 1.0 bis 3.11, 95 (4.0), 98 (4.1) und ME (4.9) waren von DOS abhängig. Windows NT und die darauf basierenden Microsoft-Betriebssysteme (Windows 2000, XP, Vista und 7) bauen nicht mehr auf MS-DOS auf und können DOS-Software nicht mehr oder nur noch eingeschränkt ausführen.

MySQL

MySQL ist ein sehr verbreitetes relationales Datenbankverwaltungssystem und steht für viele Betriebssysteme zur Verfügung (Unix, Linux, Windows, Mac OS, ...). Es steht unter einem dualen Lizenzsystem, einmal proprietär und einmal unter der GPL. Ursprünglich wurde MySQL von der schwedischen Firma MySQL AB entwickelt und dann von Sun Microsystems aufgekauft, das seinerseits von Oracle gekauft worden ist.

OASIS

Die "**O**rganization for the **A**dvancement of **S**tructured **I**nformation **S**tandards" ist eine internationale, nicht-gewinnorientierte Organisation, die sich mit der Weiterentwicklung von E-Business- und Webservice-Standards beschäftigt. Bekannte Standards der OASIS sind u. a. OpenDocument (*ODF*) und DocBook.

OASIS Open Document Format for Office Applications

Sehen Sie unter *ODF* nach.

Objekt

Ein Objekt ist ein Element auf dem Bildschirm, das Informationen enthält. Dabei kann es sich z. B. um Anwendungsdaten handeln, wie Texte oder Grafiken.

Objekte sind selbstständig und beeinflussen sich nicht gegenseitig. Jedem Objekt, das Daten enthält, werden bestimmte Befehle zugeordnet. Ein Grafikobjekt wird so mit Befehlen zur Bildbearbeitung versehen, eine Tabelle mit Befehlen zur Berechnung usw.

ODBC

Open **D**ata**B**ase **C**onnectivity (ODBC) ist ein Protokoll für den Zugriff auf *Datenbank*-Systeme durch Anwendungen. Dabei kommt die Abfragesprache *SQL* zum Einsatz. In LibreOffice können Sie von Fall zu Fall entscheiden, ob die Abfragen durch Eingabe von *SQL*-Befehlen oder anhand der interaktiven Hilfe erfolgen sollen. Bei letzterer Methode definieren Sie die Abfrage per Mausklick und LibreOffice übersetzt sie dann automatisch in *SQL*.

Die nötigen 32-Bit ODBC Funktionen installieren Sie mit Hilfe eines Setup-Programms des Datenbankherstellers in Ihr Betriebssystem. Die Eigenschaften bearbeiten Sie dann in der Systemsteuerung.

ODF

Das von OASIS entwickelte ODF ("Open Document Format for Office Applications" kurz: OpenDocument / ODF; deutsch: **O**ffenes **D**okument**F**ormat für Büroanwendungen) ist ein international genormter quelloffener Standard für Dateiformate von Bürodokumenten wie Texte, Tabellendokumente, Präsentationen, Zeichnungen, Bilder und Diagramme. Es wurde ursprünglich von Sun entwickelt, durch die Organisation OASIS als Standard spezifiziert und 2006 als internationale Norm ISO/IEC 26300 veröffentlicht.

OLE

OLE steht für "**O**bject **L**inking and **E**mbodding", also etwa Verknüpfung und Einbettung von Objekten. Ein OLE-Objekt kann wahlweise als *Verknüpfung* in ein Dokument eingefügt oder selbst darin eingebettet werden. Beim Einbetten wird eine Kopie des Objekts zusammen mit Angaben zum Quellprogramm in das Zieldokument eingefügt. Wenn Sie das Objekt bearbeiten möchten, müssen Sie nur darauf doppelklicken und aktivieren so das Quellprogramm.

OpenDocument

Sehen Sie unter *ODF* nach.

OpenGL

OpenGL ist eine ursprünglich von SGI (**S**ilicon **G**raphics **I**nc) entwickelte 3D-Grafiksprache. Zwei Varianten dieser Sprache sind weit verbreitet: das auf die Verwendung unter Windows NT ausgerichtete Microsoft OpenGL und Cosmo OpenGL von SGI. Cosmo OpenGL ist eine für alle Plattformen und Computertypen geeignete, unabhängige Grafiksprache, die sogar auf Systemen ohne spezielle 3D-Grafikhardware eingesetzt werden kann.

PDF

Das **P**ortable **D**ocument **F**ormat (PDF; deutsch: (trans)portables Dokumentenformat) ist ein plattformunabhängiges Dateiformat für Dokumente, das vom Unternehmen Adobe Systems entwickelt und 1993 veröffentlicht wurde.

Ziel war es, ein Dateiformat für elektronische Dokumente zu schaffen, das diese unabhängig vom ursprünglichen Anwendungsprogramm, vom Betriebssystem oder von der Hardwareplattform originalgetreu weitergeben kann. Ein Leser einer PDF-Datei soll das Dokument immer in der Form betrachten und ausdrucken können, die der Autor festgelegt hat. Die typischen Konvertierungsprobleme (wie zum Beispiel veränderter Seitenumbruch oder falsche Schriftarten) beim Austausch eines Dokuments zwischen verschiedenen Anwendungsprogrammen entfallen.

Neben Text, Bildern und Grafik kann eine PDF-Datei auch Hilfen enthalten, die die Navigation innerhalb des Dokumentes erleichtern. Dazu gehören zum Beispiel anklickbare Inhaltsverzeichnisse und miniaturisierte Seitenvorschauen.

PDF ist mittlerweile weit verbreitet und wird z. B. von vielen elektronischen Zeitschriften (E-Journals) genutzt. Mittlerweile gibt es auf dem Markt zahlreiche Softwareprodukte, die Dateien als PDF erzeugen können, wenn sie auch nicht immer den vollen Funktionsumfang von Adobe Acrobat bieten.

PDF-Dateien sind nicht auf PCs beschränkt, sondern kommen auch z. B. in Druckmaschinen zum Einsatz. Des Weiteren gibt es spezielle Erweiterungen zur Langzeitarchivierung.

Pixel

Das Bild eines digitalen Fotos und eines Monitors setzt sich aus Punkten (Pixeln) zusammen. Die Anzahl der Bildpunkte wird in einem Zahlenpaar angegeben.

Pixelgrafik

Eine Pixelgrafik, auch Rastergrafik genannt, ist eine Form der Beschreibung eines Bildes in Form von computerlesbaren Daten. Rastergrafiken bestehen aus einer rasterförmigen Anord-

nung von Bildpunkten (*Pixel* genannt), denen jeweils eine Farbe zugeordnet ist. Die Hauptmerkmale einer Rastergrafik sind daher die Bildgröße (Breite und Höhe gemessen in Pixeln) sowie die Farbtiefe.

Die Erzeugung und Bearbeitung von Rastergrafiken fällt in den Bereich der Computergrafik und Bildbearbeitung. Eine andere Art der Beschreibung von Bildern ist die *Vektorgrafik*.

PostgreSQL

PostgreSQL ist ein freies, objektrelationales Datenbankverwaltungssystem (ORDBMS) und ist weitestgehend mit dem SQL-Standard SQL:2008 konform. Zu den relationalen Funktionen kommt beispielsweise die Möglichkeit hinzu, selbstdefinierte Datentypen und Operationen zu verwenden.

Primärschlüssel

Ein Primärschlüssel (PK – primary key) dient zur eindeutigen Kennzeichnung eines Datenbankfeldes. Meist wird für den Primärschlüssel automatisch auch ein Index erstellt. Diese eindeutige Identifikation von Datenbankfeldern wird bei einer relationalen *Datenbank* verwendet, bei denen von einer Tabelle auf die Daten einer anderen Tabelle zugegriffen werden kann. Wird von einer anderen Tabelle auf einen Primärschlüssel verwiesen, so bezeichnet man ihn als Fremdschlüssel (FK – foreign key). Durch die beiden Schlüssel wird die Referentielle Integrität sichergestellt, was vereinfacht bedeutet, dass der Datensatz des Fremdschlüssels nur auf existierende Datensätze des Primärschlüssels verweisen darf. Dies wird vom DBMS-System sichergestellt.

In LibreOffice definieren Sie Primärschlüssel in der Entwurfsansicht einer Tabelle, indem Sie im *Kontextmenü* eines Zeilenkopfes für das ausgewählte Feld den entsprechenden Befehl wählen.

PNG

"Portable Network Graphics" (deutsch: portable Netzwerkgrafik) ist ein *Rastergrafik*-Format, das eine *Verlustfreie Kompression* benutzt. Es wurde als freier Ersatz für das ältere, bis zum Jahr 2004 mit Patentforderungen belastete Format *GIF* entworfen und ist weniger komplex als *TIFF*. Die Dateien werden mit einem wählbaren Faktor und, im Gegensatz zu *JPG*, stets verlustfrei komprimiert. PNG unterstützt neben unterschiedlichen Farbtiefen auch Transparenz per Alphakanal.

QR-Code

Der QR-Code (**Q**uick **R**esponse **C**ode) ist ein zweidimensionaler Code, der ursprünglich für die Markierung von Baugruppen und Komponenten entwickelt worden ist. Der Code enthält redundante Daten, so dass in einem gewissen Umfang eine Fehlerkorrektur möglich ist, wenn beispielsweise durch Verschmutzung nicht alles erkennbar ist (Level H erlaubt, dass 30% wiederhergestellt werden können). Heute wird der QR-Code nicht nur als Produktetikette verwendet, sondern beispielsweise für URLs (Verweise auf Webseiten), als Visitenkarte (VCARD) usw.

Rastergrafik

Sehen Sie unter *Pixelgrafik* nach.

Registerhaltigkeit

Registerhaltigkeit ist ein Begriff aus der Typographie. Darunter versteht man den deckungsgleichen Abdruck der Zeilen eines Satzspiegels auf der Vorder- und Rückseite von Büchern, Zeitschriften und Zeitungen. Mit der Funktion Registerhaltigkeit können diese Seiten einfacher gelesen werden, indem verhindert wird, dass graue Schatten durch die Textzeilen hindurch scheinen. Von Zeilenregisterhaltigkeit spricht man auch dann, wenn sich bei Textspalten alle nebeneinander liegenden Zeilen auf gleicher Höhe befinden.

Wenn Sie einen Absatz, eine Absatzvorlage oder eine Seitenvorlage als registerhaltig definieren, werden die Grundlinien der betroffenen Zeichen an einem vertikalen Seitenraster ausge-

richtet. Dabei spielt die Schriftgröße oder das Vorhandensein einer Grafik keine Rolle. Sie können die Einstellung für dieses Raster auch als eine Eigenschaft der Seitenvorlage festlegen.

Relationale Datenbank

Eine relationale *Datenbank* ist ein Datenbanksystem, in dem Daten in Form miteinander verbundener Tabellen verwaltet werden. Die Daten können in verschiedener Weise abgefragt werden, ohne dass die zugrunde liegenden Tabellen reorganisiert werden müssen.

Ein relationales Datenbankverwaltungssystem (RDBMS) ist ein Programm, mit dem Sie relationale Datenbanken erstellen, aktualisieren und verwalten können. Ein RDBMS akzeptiert SQL-Anweisungen, die entweder vom Benutzer eingegeben werden oder in einer Anwendung enthalten sind, und erzeugt, aktualisiert oder ermöglicht den Zugriff auf Datenbanken.

Als typisches Beispiel lässt sich eine Datenbank mit Kunden-, Verkaufs- und Rechnungstabellen heranziehen. In der Rechnungstabelle sind nicht die eigentlichen Kunden- oder Verkaufsdaten, sondern Referenzen, in Form von relationalen Verknüpfungen, oder Relationen auf die Tabellenfelder mit den entsprechenden Kunden- und Verkaufsdaten (z. B. das Kundennummernfeld aus der Kundentabelle) enthalten.

Relativ speichern

In einigen Dialogen (z. B. **Bearbeiten** → **AutoText**) können Sie wählen, ob eine Datei relativ oder absolut gespeichert werden soll.

Wenn Sie sich für das relative Speichern entscheiden, werden Referenzen auf eine eingebettete Grafik oder ein anderes *Objekt* im Dokument relativ zur Position im Dateisystem gespeichert. In diesem Fall spielt es keine Rolle, wo die referenzierte Verzeichnisstruktur eingetragen ist. Solange die Referenz auf derselben Festplatte bzw. demselben Volume bleibt, werden die Dateien unabhängig vom Speicherort immer aufgefunden. Dies ist für solche Dokumente von besonderer Bedeutung, die auch auf Computern mit einer möglicherweise ganz anderen Verzeichnisstruktur oder anderen Laufwerks- oder Volume-Namen verwendet werden sollen. Auch für das Anlegen von Verzeichnisstrukturen auf einem Internetserver empfiehlt es sich, Dokumente relativ zu speichern.

Vergleichen Sie auch mit *Absolut speichern*.

RTF

RTF (**Rich Text Format**) ist ein für den Austausch von Textdateien entwickeltes Dateiformat. Es zeichnet sich dadurch aus, dass Formatierungsinformationen in direkt lesbare Textdaten konvertiert werden. Leider entstehen dabei im Vergleich zu anderen Formaten recht große Dateien.

Schusterjungenregelung

Schusterjungen und Hurenkinder sind historische Begriffe aus der Typographie, die seit langem benutzt werden.

Ein Schusterjunge ist die erste Zeile eines Absatzes, die alleine am unteren Rand der Vorseite steht. Mit einem Textdokument von LibreOffice können Sie diese unschöne Erscheinung automatisch für die gewünschte Absatzvorlage vermeiden. Dabei können Sie sogar wählen, wie viele Zeilen mindestens immer zusammen auf einer Seite gehalten werden sollen.

Dem gegenüber steht die *Hurenkinderregelung*.

SQL

SQL (**Structured Query Language**) ist eine Sprache zur Definition von Datenstrukturen und zur Spezifikation von Abfragen einer relationalen *Datenbank*. In LibreOffice haben Sie die Möglichkeit, Abfragen entweder in SQL oder mithilfe der Maus zu definieren.

SQL (damals SEQUEL) wurde im Jahre 1968 im Rahmen einer studentischen Arbeit in Berkley entwickelt und 1986 als ANSI-Standard (SQL1) verabschiedet. Die letzte Änderung war 2011,

daher wird diese Version als SQL:2011 bezeichnet.
SQL teilt sich in folgende Unterbereiche auf:

- DDL: **Data Definition Language**
z. B. CREATE TABLE
- DCL: **Data Control Language**
GRANT, REVOKE
- DML: **Data Manipulation Language**
z. B. INSERT, UPDATE, DELETE
- DQL: **Data Query Language**
SELECT
- TCL: **Transaction Control Language**
COMMIT, ROLLBACK

SQL-Datenbank

Eine *SQL*-Datenbank ist ein *Datenbank*-System, das eine *SQL*-Schnittstelle bietet. *SQL*-Datenbanken werden oft in Client/Server-Netzwerken eingesetzt, in denen verschiedene Clients auf einen zentralen Server (z. B. einen *SQL*-Server) zugreifen, daher bezeichnet man sie auch als *SQL*-Server-Datenbanken oder kurz *SQL*-Server.

In LibreOffice können Sie externe *SQL*-Datenbanken einbinden. Diese können sich sowohl auf einer Festplatte des Rechners, auf dem LibreOffice läuft, als auch im Netzwerk befinden. Der Zugriff erfolgt entweder über *ODBC*, *JDBC* oder über einen in LibreOffice integrierten systemeigenen Treiber.

SQL-Server

Sehen Sie unter *SQL-Datenbank* nach.

SWF

Das Kürzel „SWF“ steht für **Shockwave Flash**. Unter dem Namen Shockwave vermarktete der damalige Hersteller Macromedia nicht nur Flash, sondern auch eine um 3D-Funktionen, eine objektorientierte Sprache und andere Features erweiterte Variante, die mit Adobe Director produziert werden kann. Während das Shockwave-Format von Anfang an für eine rechenintensive Nutzung konzipiert war, sollte mit dem Webbrowserplugin Flash ein Präsentationsformat geschaffen werden, welches der Universalität des Internets in Bezug auf Hardwareausstattung und Bandbreite entspricht.

Symbolleiste

Ein kleines Fenster mit mehreren Symbolen (Icons), mit denen gängige Aufgaben erledigt werden. Die Symbolleisten können in LibreOffice an beliebigen Stellen positioniert werden.

TIF

Sehen Sie unter *TIFF* nach.

TIFF

Das "**Tagged Image File Format**" (deutsch: Markiertes Grafikdatei Format) ist ein Dateiformat zur Speicherung von Bilddaten. Das TIFF-Format wurde ursprünglich von Aldus (1994 von Adobe übernommen) und Microsoft für die Farbseparation bei einer gescannten *Pixelgrafik* entwickelt.

Größter Nachteil von TIFF ist seine Komplexität. Die Vielfalt möglicher gültiger TIFF-Dateien kann nur schwer von einzelnen Programmen unterstützt werden. In der Spezifikation des Dateiformates ist deswegen eine Untermenge gültiger TIFF-Dateien definiert, die jedes TIFF-fähige Programm verarbeiten können sollte, genannt *Baseline TIFF*.

Unicode-Zeichensatz

Unicode ist ein internationaler Standard, in dem langfristig für jedes sinntragende Schriftzeichen oder Textelement aller bekannten Schriftkulturen und Zeichensysteme ein digitaler Code festgelegt wird. Ziel ist es, die Verwendung unterschiedlicher und inkompatibler Kodierungen in verschiedenen Ländern oder Kulturkreisen zu beseitigen. Unicode wird ständig um Zeichen weiterer Schriftsysteme ergänzt. ISO 10646 ist die von ISO verwendete, praktisch bedeutungsgleiche Bezeichnung des Unicode-Zeichensatzes; er wird dort als "**Universal Character Set**" (UCS) bezeichnet.

Es gibt drei verschiedene Hauptausführungen der Unicode-Zeichensätze, *UTF-8*, *UTF-16* und *UTF-32* und folgende Anpassungen bzgl. der Byte-Reihenfolge *Big-Endian* (BG) und *Little-Endian* (LE): UTF-16BE, UTF-16LE, UTF-32BE, UTF-32LE. Bei Big-Endian wird das höchstwertige Byte zuerst gespeichert, bei Little-Endian zuletzt. Die Byte-Reihenfolge wird meist über die **Byte Order Mark** (BOM) umgesetzt. Dabei wird am Anfang eine bestimmte Bytefolge hinzugefügt, die die Bytereihenfolge festlegt.

- UTF-8: 00 00 FE FF
- UTF-16LE: FF FE
- UTF-16BE: FE FF
- UTF-32LE: FF FE 00 00
- UTF-32BE: 00 00 FE FF

UTF-8

UTF-8 (Abk. für 8-bit UCS Transformation Format) ist die am weitesten verbreitete Kodierung vom *Unicode-Zeichensatz*.

Bei der UTF-8 Kodierung wird jedem Unicode-Zeichen eine speziell kodierte Bytekette variabler Länge zugeordnet. UTF-8 unterstützt bis zu vier Byte, mit denen sich zur Zeit ca. 113.000 Zeichen abbilden lassen. Dabei sind die ersten 128-Zeichen wie ASCII-7bit kodiert. UTF-8 hat eine zentrale Bedeutung als globale Zeichenkodierung im Internet. Die „Internet Engineering Task Force“ verlangt von allen neuen Internetkommunikationsprotokollen, dass die Zeichenkodierung deklariert wird und dass UTF-8 eine der unterstützten Kodierungen ist. Das Internet Mail Consortium (IMC) empfiehlt, dass alle E-Mail-Programme UTF-8 darstellen und senden können. Auch bei dem in Webbrowsern verwendeten *HTML* setzt sich UTF-8 zur Darstellung sprachspezifischer Zeichen zunehmend durch und ersetzt die vorher benutzten HTML-Entities (Entity ist der Name für ein bestimmtes Sonderzeichen. Entities werden durch ein „&“-Zeichen am Anfang und ein Semikolon am Ende gekennzeichnet). UTF-8 ist die Standard-Zeichenkodierung von MacOS X und einigen Linux-Distributionen. Windows kann zwar mit UTF-8 umgehen, benutzt aber intern einen anderen Zeichensatz (*Windows-Zeichensatz*). Speichern Sie ein Dokument in einem *ODF*-Format, ist es immer UTF-8 kodiert.

UTF-16

UTF-16 ist eine Kodierung vom *Unicode-Zeichensatz*, die für die häufig gebrauchten Zeichen optimiert ist. Es ist das älteste der Unicode-Kodierungsformate und wird standardmäßig in Java intern verwendet, daher hat ein char in Java 16 Bit.

UTF-32

UTF-32 ist eine Kodierung vom *Unicode-Zeichensatz*, bei der jedes Zeichen mit vier Byte (32 Bit) kodiert wird. Sie kann deshalb als die einfachste Kodierung bezeichnet werden, da alle anderen UTF-Kodierungen variable Bytelängen benutzen. Der entscheidende Nachteil von UTF-32 ist der hohe Speicherbedarf. Bei Texten, die überwiegend aus lateinischen Buchstaben bestehen, wird verglichen mit dem verbreiteten UTF-8-Zeichensatz etwa der vierfache Speicherplatz belegt, bei asiatischen Zeichen beispielsweise bietet UTF-32 einen Vorteil.

Vektorgrafik

Eine Vektorgrafik ist eine Computergrafik, die aus grafischen Primitiven wie Linien, Kreisen, Polygonen oder allgemeinen Kurven (Splines) zusammengesetzt ist. Meist sind mit Vektorgrafiken Darstellungen gemeint, deren Primitiven sich zweidimensional in der Ebene beschreiben lassen. Eine Bildbeschreibung, die sich auf dreidimensionale Primitiven stützt, wird eher 3D-Modell oder Szene genannt.

Um beispielsweise das Bild eines Kreises zu speichern, benötigt eine Vektorgrafik mindestens zwei Werte: die Lage des Kreismittelpunkts und den Kreisdurchmesser. Neben der Form und Position der Primitiven werden eventuell auch die Farbe, Strichstärke, diverse Füllmuster und weitere, das Aussehen bestimmende Daten, angegeben. Damit kann sie, im Gegensatz zu einer *Pixelgrafik* verlustfrei vergrößert oder verkleinert werden.

Verknüpfung

Verknüpfungen dienen zum schnellen Aufrufen von Dateien auf der Arbeitsoberfläche. Mit einem einfachen oder einem Doppelklick (je nach Einstellung des Betriebssystems) öffnet sich das entsprechende Programm.

Verlustbehaftete Kompression

Bei der verlustbehafteten Kompression wird versucht, den Informationsverlust unmerklich oder wenigstens ästhetisch erträglich zu halten. Diese Methoden nutzen aus, dass kleine Farbänderungen für das Auge nicht sichtbar sind. Ähnlich wie bei der verlustbehafteten Audiokomprimierung basiert die Bildkomprimierung auf einem Modell der menschlichen Wahrnehmung. Der Komprimierungsalgorithmus soll bevorzugt die Bildinformationen entfernen, die über die Aufnahmefähigkeit der menschlichen Bildwahrnehmung hinausgehen. Das Wahrnehmungsmodell ist jedoch, im Gegensatz zur Audiokompression, nicht explizit formuliert und in die Algorithmen eingearbeitet, sondern mehr intuitiv.

Technisch bedingt ist eine Wiederherstellung des Originalzustandes nicht mehr möglich.

Im Gegensatz dazu steht die *Verlustfreie Kompression*.

Verlustfreie Kompression

Bei der verlustfreien Kompression geht keine Information verloren. Die Daten werden nur anders als vorher organisiert, indem bestimmte Redundanzen erkannt und zusammengefasst werden. Zum Beispiel können sich wiederholende Bitfolgen einmal in einem Wörterbuch abgelegt und dann nur noch durch ihre Nummer repräsentiert werden. Es können beliebige allgemeine Komprimierungsverfahren verwendet werden, die sich auch auf andere Arten von Daten wie Text anwenden lassen.

Dadurch, dass keine Information verloren geht, ist die Wiederherstellung des Originalzustandes jederzeit wieder möglich.

Im Gegensatz dazu steht die *Verlustbehaftete Kompression*.

Windows-Zeichensatz

„Windows-1252 Westeuropäisch (Western European)“ ist eine 8-Bit-Zeichenkodierung des Microsoft-Betriebssystems Windows, die die meisten westeuropäischen Sprachen unterstützt. Sie baut auf ISO-8859-1 und ISO-8859-15 auf.

Manche Applikationen vermischen die Definition von ISO-8859-1 und Windows-1252. Diese Codierungen unterscheiden sich jedoch nur in den nichtdruckbaren Steuerzeichen. Da diese beispielsweise in *HTML* keine Bedeutung haben, werden oft die druckbaren Zeichen aus Windows-1252 verwendet. Aus diesem Grund schreibt der neue HTML5-Standard vor, dass als ISO-8859-1 markierte Texte als Windows-1252 zu interpretieren sind.

XHTML

XHTML (Extensible **H**ypertext **M**arkup **L**anguage) ist eine erweiterte Version des *HTML*.

XML

Die **Extensible Markup Language** (engl. für „erweiterbare Auszeichnungssprache“), abgekürzt XML, ist eine Auszeichnungssprache zur Darstellung hierarchisch strukturierter Daten in Form von Textdaten. XML wird u. a. für den plattform- und implementationsunabhängigen Austausch von Daten zwischen Computersystemen eingesetzt, insbesondere über das Internet. Ein XML-Dokument besteht aus Textzeichen (meist *UTF-8* Kodierung, aber jede andere Kodierung ist auch möglich), und ist damit menschenlesbar – Binärdaten enthält es per Definition nicht. Alle wesentlichen Dokumentteile eines *ODF*-Dokuments sind intern in XML verfasst.

Zahlensystem

Ein Zahlensystem ist durch die Anzahl der zur Darstellung von Zahlen verfügbaren Zeichen gekennzeichnet. So basiert beispielsweise das Dezimalsystem auf zehn Ziffern (0 bis 9 – Basis 10), das Binär- oder Dualzahlensystem auf zwei Ziffern (0 und 1 – Basis 2) und das Hexadezimalsystem auf 16 Zeichen (0 bis 9 und A bis F – Basis 16).

Ziehen&Ablegen

Ziehen&Ablegen (auch unter Drag&Drop bekannt) bezeichnet eine bestimmte Mausbewegung, um ein *Objekt* zu kopieren oder zu verschieben. Dabei wird ein Objekt markiert, bei festgehaltener linker Maustaste von einem Platz zu einem anderen gezogen und dort mit Loslassen der Maustaste abgelegt. Dies kann innerhalb eines Dokumentes aber auch von einem Dokument zu einem anderen erfolgen.

Zwischenablage

Die Zwischenablage ist ein virtueller Speicher zum Austausch von Daten zwischen mehreren Programmen. Mit Herunterfahren des PCs werden die Daten der Zwischenablage gelöscht.

Stichwortverzeichnis

Abfragen.....	23, 233	über Zwischenablage einfügen.....	363
Beziehungsdefinition.....	244	Zusammenfassen.....	406
Bezugstabellen.....	279	Datenbank.....	17, 60, 562
Eingabe.....	233	Anbindung.....	335
Eingabemöglichkeiten.....	271	anlegen.....	60
Ergebnis.....	237	anmelden.....	69
Erweiterungen mit SQL.....	250	Assistent.....	65
Funktionen.....	241	Aufgaben.....	372
in Formularen verwenden.....	270	aus Calc importieren.....	362
Korrelierte Unterabfrage.....	278	Autowerte neu einstellen.....	518
Parameter.....	276	Benutzername.....	68
per GUI.....	233	Calc.....	357
SQL.....	238	Database management system.....	12
Tabellenansichten.....	283	Datenbankmanagementsystem.....	563
Unterabfragen.....	277	Datenbanksystem.....	562
Zeilenumbruch.....	405	dBASE.....	76, 563
Zusammenfassung.....	282	DBMS.....	12, 17, 563
Addon.....	315	DBS.....	562
Alter ermitteln.....	396	Direkte Verbindung.....	67
andocken.....	561	Eigenschaften.....	518
ANSI.....	561	Einträge bearbeiten.....	521
API.....	561	extern.....	62
Austauschformat.....	562	externe HSQLDB einbinden.....	552
AutoHide.....	561	Firebird.....	564
Barcode.....	526, 562	Geschwindigkeit.....	523
Benutzeroberfläche.....	101	HSQLDB.....	63, 564
Berichte.....	23	Informationstabellen.....	540
Beziehungen.....	21, 95	intern.....	60
Eins-zu-Eins (1:1).....	96	JDBC.....	63, 72
Eins-zu-Viele (1:n).....	95	Kennwort.....	68
Viele-zu-Viele (n:m).....	96	Komprimieren.....	518
Bézierkurven.....	562	Konvertieren.....	363
Bildkompression.....	562	Managementsystem.....	17
Codeschnipsel.....	395	MySQL.....	60, 567
Comboboxen.....	458	Name.....	67
Complex Text Layout.....	562	neu erstellen.....	66
Container.....	18	odb.....	61
CTL.....	562	ODBC.....	63, 71
Daten.....		ORDBMS.....	569
aus Calc exportieren.....	362	PostgreSQL.....	569
Einfügen als Felder.....	339, 341	RDBMS.....	564, 570
Felder.....	341	relational.....	570
Gruppieren.....	406	Reparatur.....	543
in Calc einfügen.....	358	übers Netz ansprechen.....	67
Text.....	337	unnötige Einträge.....	520
		Verbindung herstellen.....	62
		Versionsprobleme.....	551
		Verwaiste Einträge ermitteln.....	523
		Wartung.....	518
		Wiederherstellung.....	543
		Datenfilterung.....	372
		Abfrage.....	372

Suche.....	374	Gruppierungsrahmen.....	179
Datenquelle.....	335	Handbuch.....	12
Browser.....	335	Hilfesystem.....	12
Calc.....	335	HSQLDB.....	
Writer.....	335	Eingaben.....	115
DDE.....	563	externe Verbindung.....	552
Defaultwert.....	101	Mehrbenutzerbetrieb.....	556
DIN.....	561, 563	Version aktuell.....	115
Drag&Drop.....	574	Version in Base.....	115
Drehfeld.....	563	HTML.....	565
DTP.....	564	Hurenkinder.....	565, 570
Dynamic Data Exchange.....	563	Hyperlink.....	566
Erweiterung.....	564	IEC.....	565
Etikettendruck.....	335	IMC.....	572
Extension.....	564	IME.....	565
FAQ.....	13, 15	Index.....	109
Feld.....	22	ISO.....	565
foreign key.....	569	Java.....	565
Formatierung.....		Archiv einbinden.....	64
direkt.....	563	Class-Path.....	64
hart.....	563	Java Runtime Environment.....	566
indirekt.....	565	JRE.....	566
Formular.....	20	Verzeichnispfad.....	64
Ansicht.....	215	JDBC.....	64, 72, 565
Daten.....	151	JPEG.....	566
Eigenschaften.....	150	JPG.....	566
Entwurf.....	147	Kerning.....	566
Ereignisse.....	152	Kombinationsfelder.....	457
erstellen.....	145	Kompression.....	
Fehlermeldungen.....	222	verlustbehaftet.....	573
Feld.....	148	verlustfrei.....	573
Hauptformular.....	201	Kontextmenü.....	566
Kontrollfelder.....	153	Kontostand nach Kategorie ermitteln.....	401
Name.....	150	LGPL.....	566
Navigator.....	147, 218	Link.....	566
Unterformular.....	201	Listenfeld.....	22
Forum.....	13	localhost.....	67
Fremdschlüssel.....	22, 98, 569	Macintosh Tastenbelegung.....	13
Funktionen.....	528	Mailinglisten.....	13
Datenbankverbindung.....	536	Makros.....	566
Datum/Zeit.....	534	allgemein.....	409
Numerisch.....	529	Basic.....	409
System.....	538	benutzen.....	413
Text.....	531	Bestandteile.....	416
Gallery.....	564	Datenbankaufgaben erweitern.....	477
GIF.....	564		
GPL.....	564, 566f.		

Datenbanken komprimieren.....	486	Rastergrafik.....	568
Datenbanksicherungen.....	482	Referentielle Integrität.....	22
Dialoge.....	499	Registerhaltigkeit.....	569
DIM.....	417	Relation.....	21, 245
Editor.....	409	Report-Designer.....	
Filtern von Datensätzen.....	439	aufrufen.....	287
Fremdschlüssel übertragen.....	460	Bedingte Anzeige.....	314
Kontrollfunktionen.....	466	Bedingte Formatierung.....	315
Mailprogramm.....	493	Benutzerdefinierte Funktionen.....	312
Makrosicherheit.....	410	Dateneigenschaften von Feldern.....	304
Navigation Formular.....	467	Diagramme einbinden.....	299
Prozedur.....	409	Eigenschaften von Feldern.....	295
Rahmen.....	417	Formeleingabe.....	314
Suchen von Datensätzen.....	449	Formeleingaben.....	305
Variablen definieren.....	417	Funktionen.....	305, 307
verwalten.....	409	RESTRICT.....	121
Zugriff auf Elemente.....	419	RTF.....	570
Zugriff auf Formular.....	418	Schaltfläche.....	182
zuweisen.....	413	Schlüsselfeld.....	21
Markieren.....	567	Schusterjungen.....	565, 570
Mehrfachselektion.....	188	SEQUEL.....	570
MS-DOS.....	567	Serienbrief.....	25, 61
MySQL.....		Seriendruck.....	341
JDBC.....	64	Assistent.....	341
ODBC.....	65	Etiketten.....	349
ODBC-Verbindung.....	65	Externe Formulare.....	356
Navigationsleiste.....	184	Feldbefehl.....	354
NULL.....	110, 124	Felder.....	353
OASIS.....	567f.	Serienbrief.....	342
Objekt.....	567	Serienbriefassistent.....	342
ODBC.....	65, 567	SGL.....	568
ODF.....	568	Spationierung.....	566
OLE.....	568	Speicherfresser.....	94
OpenGL.....	568	speichern.....	
PDF.....	568	absolut.....	561
Pixel.....	568	relativ.....	570
Pixelgrafik.....	568	SQL.....	100, 570
PK.....	569	Alias.....	266
PNG.....	569	ALTER COLUMN.....	518
Primärschlüssel.....	21, 94, 98, 569	ALTER TABLE.....	119, 518
primary key.....	569	Bedingungen.....	132
Programmiersprachen.....		CASCADE.....	121
Basic.....	566	CASEWHEN.....	396
BeanShell.....	566	Char.....	405
JavaScript.....	566	COUNT.....	402
Python.....	566	CREATE TABLE.....	116
QR-Code.....	569	CURDATE.....	396
		DATEDIFF.....	396
		Datenbank.....	571

DAYOFYEAR.....	396	Eltern-Tabelle.....	123
DCL.....	571	filtern.....	131
DDL.....	571	Index.....	109
DELETE.....	134	NULL.....	110
direkte Eingabe.....	114	Quellen.....	78
DISTINCT.....	251, 373	sortieren.....	129
DML.....	571	Sortierung.....	109
DQL.....	571	Suchfunktion.....	129
DROP TABLE.....	121	Timestamp.....	106
Fehlercode.....	243	Unterschied zu Tabellenkalkulation.....	94
GENERATED BY DEFAULT.....	558	verknüpfen.....	122
GROUP BY.....	242, 251	Wiederholungen.....	94
Group_Concat.....	406	Zeitfelder.....	106
HAVING.....	251	Tabellenkalkulationen.....	17
IF EXISTS.....	121	Tabellenkontrollfeld.....	196
INSERT INTO.....	133	Thunderbird Adressbuch.....	79
LIKE.....	375	TIFF.....	571
LIMIT.....	251	UCS.....	572
Listenfelder.....	267	Unicode.....	572
LOWER.....	375	Unterschneidung.....	566
ORDER BY.....	251, 373	URL.....	569
RESTART WITH.....	518	UTF.....	
RESTRICT.....	121	BOM.....	572
SELECT.....	238, 251, 373	UTF-16.....	572
Server.....	571	UTF-16BE.....	572
SQL:2011.....	571	UTF-16LE.....	572
Sub-Select.....	373	UTF-32.....	572
SUM.....	402	UTF-32BE.....	572
Tabelle erstellen.....	116	UTF-32LE.....	572
Tabelle löschen.....	121	UTF-8.....	572
Tabellenänderung.....	119	VCARD.....	569
TCL.....	571	Vektorgrafik.....	573
UNION.....	251	Verknüpfung.....	573
UPDATE.....	133	View.....	283, 336
WHERE.....	251	XHTML.....	573
Streifencode.....	562	XML.....	574
Strichcode.....	562	Zahlensystem.....	574
Support.....	13	Zeichensatz.....	
SWF.....	571	ANSI.....	561
Symbolleiste.....	571	ASCII.....	561
Tabellen.....	21, 94	ISO-8859-1.....	573
allgemein.....	94	ISO-8859-15.....	573
AutoWert.....	106, 126	UTF.....	572
Bezeichnung.....	100	Windows-1252.....	573
Beziehung.....	94	Zeilennummerierung.....	402
Datentypen.....	94	Zwischenablage.....	574
Defaultwert.....	101		
Dokument.....	78		
Editor.....			
Datentypen.....	526		
Eingabe von Daten.....	126		

Base 5.3-Handbuch

Verwalten Sie Ihre Daten

**Zu diesem Buch:**

Dieses Buch bietet einen Einblick in das Datenbankmodul Base der Office Suite LibreOffice. Egal, ob Sie noch nie mit Datenbanken gearbeitet haben oder Sie ein anderes Datenbankprogramm aus einer OfficeSuite gewohnt sind.

Dieses Buch führt Sie in die gebräuchlichsten Funktionen von LibreOffice-Base ein:

- Datenbank erstellen
- Ein- und Ausgabe der Datenbank: Tabellen, Formulare, Abfragen, Berichte
- Aufgaben einer Datenbank
- Makros
- Pflege und Wartung von Datenbanken und einiges mehr

Zu den Autoren:

Dieses Buch entstand durch Freiwillige der LibreOffice-Gemeinschaft.

Eine PDF-Version dieses Buches und aller Updates zu diesem Buch kann frei unter <http://de.libreoffice.org/get-help/documentation> heruntergeladen werden.

Über LibreOffice:

LibreOffice ist eine leistungsfähige Office-Suite, für verbreitete Betriebssysteme wie Windows, GNU/Linux 32-/64-Bit und Apple Mac OS X geeignet. Es bietet sechs Anwendungen für die Erstellung von Dokumenten und zur Datenverarbeitung: Writer, Calc, Impress, Draw, Base und Math.

LibreOffice entsteht aus der kreativen Zusammenarbeit von Entwicklern und der Gemeinschaft der Stiftung "The Document Foundation". Die Stiftung hat ihren Sitz in Deutschland.

LibreOffice kann unter der Adresse <http://de.libreoffice.org/download> kostenlos heruntergeladen werden.